# MU Study

Complete Tutorial Series — Easiest to Hardest

Thirty tutorials that teach every layer of the Flutter + Supabase school platform: product, database, auth, Riverpod, go_router, homework, marks, attendance, chat, notifications, themes, localization, testing, and release. Ordered from concepts that need no code to complete features that stitch the whole system together.

| | |
|---|---|
| **Product** | School management + study platform (Math & English) |
| **Client** | Flutter · Dart ^3.11.5 · Riverpod · go_router |
| **Backend** | Supabase · PostgreSQL · Auth · Storage · Realtime |
| **Roles** | Student · Teacher (math/english) · Admin |
| **Locales** | English · Russian · Uzbek (not Arabic) |
| **Login** | Username + password (synthetic @mustudy.internal email) |
| **Registration** | None — admins create every account |
| **Secrets** | Placeholders only. Never paste a real anon key into notes you share. |

*Generated from the inspected MU Study repository and the live PostgreSQL schema. Follow tutorials in order. Type the code. Verify every checkpoint independently.*

# How to use this book

This is a reconstruction textbook, not a slide deck. Each tutorial has a Learn block, real MU Study files, a practical TASK, an independent verification CHECKPOINT, common mistakes, and video links. Later tutorials assume earlier checkpoints passed.

## Learning contract

1. Work sequentially from Tutorial 1. Skipping fundamentals is the fastest way to get stuck on RLS.
2. Type SQL and Dart yourself. Copy-paste only when a lesson says the file is a reference dump.
3. Never commit .env. Never put the service_role key in Flutter. Never invent auth emails in the client.
4. After each TASK, prove the CHECKPOINT without looking at the answer first.
5. When something fails, use Appendix D (Troubleshooting) before changing random code.

## What “complete” means for MU Study

A complete recreation includes: three role shells (student, teacher, admin); username login via get_auth_email_by_username; PostgreSQL enums, tables, views, SECURITY DEFINER RPCs, RLS, Storage buckets, Realtime on chat and notifications; EN/RU/UZ localization; four visual styles; homework packs with tasks; attendance sessions; weighted marks; group chat with replies and reactions; admin mass tools and printed credential slips.

**WARNING — This book teaches the real app, not a simplified fantasy**
Some generic Flutter+Supabase tutorials use public email signup, Arabic localization, or Firebase Cloud Messaging. MU Study does none of those. Login is username-based. Languages are English, Russian, and Uzbek. Notifications are in-app rows plus Realtime, not FCM. Tutorial 20 explains that honestly and shows the real notification path.

**Independent verification**
A checkpoint is independent when you can prove it without the author’s machine: a SQL query result, a flutter analyze exit code, a screenshot of a named screen, or a second account that cannot see another student’s marks. “It looked fine” is not verification.

# Table of contents


| Section | What you gain |
|---|---|
| How to use this book | Start here |
| Tier 1 — Fundamentals | Tutorials 1–4 · no coding required at first |
| 1. Understanding MU Study | Product, roles, stack, architecture |
| 2. Database concepts | Tables, keys, types, 20-table map |
| 3. Supabase project setup | Account, keys, Table Editor, SQL Editor |
| 4. Environment variables | .env, dart-define, Env.sanitize |
| Tier 2 — Flutter basics | Tutorials 5–9 |
| 5. Flutter project structure | lib/, pubspec, platforms |
| 6. Dart basics | Types, functions, classes, null safety |
| 7. Widgets & layouts | Material, Scaffold, forms |
| 8. Stateful vs Stateless | setState and when not to use it |
| 9. Navigation with GoRouter | Redirects, shells, path params |
| Tier 3 — State management | Tutorials 10–11 |
| 10. Riverpod basics | Provider, Future, Stream, Notifier |
| 11. Models & data classes | fromJson, enums, why not Freezed here |
| Tier 4 — Database & backend | Tutorials 12–15 |
| 12. SQL basics | SELECT through JOIN and aggregates |
| 13. Supabase authentication | Username → email RPC → JWT |
| 14. Database queries from Flutter | PostgREST, RPC, repositories |
| 15. Row Level Security | Policies, helpers, role matrix |
| Tier 5 — Advanced platform | Tutorials 16–20 |
| 16. File upload & Storage | Buckets, paths, signed URLs |
| 17. Realtime | Chat and notification channels |
| 18. Localization | ARB EN/RU/UZ |
| 19. Theming & dark mode | Four styles × light/dark |
| 20. Notifications (real system) | Table + Realtime; FCM is future work |
| Tier 6 — Complete features | Tutorials 21–25 |
| 21. Authentication feature | Splash → login → language → shell |
| 22. Assignments feature | Packs, tasks, submit, grade |
| 23. Chat feature | Rooms, media, reactions, unread |
| 24. Attendance feature | Sessions, statuses, reports |
| 25. Announcements feature | Group posts, attachments, notify |
| Tier 7 — Going deep | Tutorials 26–30 |
| 26. Custom widgets | Shell, cards, bubbles, empty states |
| 27. Testing | Unit, widget, integration habits |
| 28. Performance | Cache, paging, image, query shape |
| 29. Error handling | AsyncValue, offline, user copy |

| Section | What you gain |
|---|---|
| 30. Deployment | APK, web, iOS, store hygiene |
| Source atlas | Every Dart file with purpose + full listing |
| Annotated source | Every interesting line explained |
| SQL atlas + annotated SQL | Functions and statement-by-statement |
| Screen / provider / component manuals | QA for every UI module |
| 30-day journal + oral exam | Calendar and 50 spoken answers |
| Every-line Dart commentary | Non-trivial lines with a one-sentence why |
| ARB key catalog | EN / RU / UZ side by side |
| RLS in English | Every policy said as a sentence |
| Appendix A — Complete backend SQL | Run top to bottom on a new project |
| Appendix B — Dependencies | Every pubspec package explained |
| Appendix C — Video index | All links by topic |
| Appendix D — Troubleshooting | Errors you will actually hit |
| Appendix E — Architecture diagrams | Data, widgets, providers, schema |
| Appendix F — Code style guide | Names, files, comments |
| Appendix G — Column catalog | Every table, every column |
| Appendix H — ARB catalogs | EN / RU / UZ |
| Appendix I — First admin bootstrap | Create @admin safely |
| Appendix J — Config files | Makefile, l10n, manifests |
| Appendix K — Glossary & quizzes | Terms + extra practice |
| Appendix L — Foundation essays | HTTP, JSON, JWT, Git, Postgres types |

# Tier 1 — Fundamentals

No Flutter coding yet. You learn the product, the database vocabulary, the Supabase dashboard, and how secrets stay out of Git. If you cannot explain who creates accounts and why students must not see each other’s marks, do not start Tutorial 5.

# Tutorial 1 — Understanding the MU Study project

| Field | Value |
|---|---|
| Tutorial | 1 — Understanding MU Study |
| Difficulty | ★ easiest |
| Time | 20–30 minutes |
| Prerequisites | None |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| What is Flutter? — Flutter Official | https://www.youtube.com/watch?v=fDkQWZDu6UU |
| What is Supabase? — Supabase Channel | https://www.youtube.com/watch?v=WiwfiP2jtjY |

## What MU Study is

MU Study is a school management and study platform built for Math and English teachers and their students. It is not a public social network. There is no self-registration button. An administrator creates every username and password. Parents are not a first-class role in the current schema — a parent who needs access would be given a student or admin account by policy, not a parent table.

The client is a Flutter app that runs on Android, iOS, and Web. Phones use a bottom navigation bar. Wider layouts use a navigation rail. A permanent desktop-only sidebar is intentionally omitted (see README). The look is Material 3 with a chocolate/sand default plus three additional visual styles.

## Who uses it

| Role | Database | What they do in the app |
|---|---|---|
| Student | profiles + students + group_students | Homework, marks, attendance view, chat, focus timer, notifications |
| Teacher | profiles + teachers (sphere) | Own groups, assignments, gradebook, take attendance, announcements, chat |
| Admin | profiles.role = admin | Users, groups, academic years, school announcements, credential slips, mass generator |

A teacher's sphere is required: **math** or **english** (Postgres enum sphere_type). Admins do not auto-create groups when they add a teacher. They type exact group titles later.

## What you can do (feature map)

- Sign in with username + password (behind the scenes: synthetic email + Supabase Auth).
- Pick English, Russian, or Uzbek once; the choice is stored on the profile.
- Students: dashboard announcements, assignments (static vs homework pack), task submit (text/file/voice), marks, attendance calendar, group chat, focus mode, inbox.
- Teachers: create assignments and tasks, review submissions, gradebook, attendance grid, analytics charts, group announcements, chat.

• Admins: create/disable users, create groups and memberships, academic years/terms, print login slips, optional mass data generator.

## Technology stack at a glance

| Layer | Choice in this repo | Why |
|---|---|---|
| UI | Flutter 3 / Dart ^3.11.5 | One codebase for Android, iOS, Web |
| State | flutter_riverpod | Typed providers, no BuildContext lookup |
| Routes | go_router | Path URLs + redirect on auth/language |
| Backend | Supabase | Auth + Postgres + Storage + Realtime hosted together |
| Database | PostgreSQL 17 | RLS, enums, triggers, views |
| Files | Supabase Storage | Avatars public; work and chat private |
| Live updates | Supabase Realtime | Chat + notifications without polling |
| i18n | gen-l10n ARB | en / ru / uz |

## Architecture overview

**Mental model**

```
Phone / browser
  Flutter widgets (screens + components)
    Riverpod providers (auth, lists, streams)
      supabase_flutter client
        HTTPS → Supabase project
          Auth  |  PostgREST (tables/RPC)  |  Storage  |  Realtime
            PostgreSQL with RLS + triggers
```

Security is not “hide the admin button in Flutter.” A student who crafts an HTTP request still hits RLS. If a student can SELECT another student’s marks in the SQL Editor while logged in as that student JWT, the product is broken — even if the UI never showed a button.

## Screens you will tour

On a working instance you will see Splash → Login → Language (first time) → a role home. Student home shows announcements. Teacher home shows group shortcuts. Admin home shows platform counts. Chat is a shared pair of screens under /student/chat, /teacher/chat, /admin/chat.

**TASK — 1.1 Screenshot tour**
1) Open a running MU Study build (this repo: make run, or the web target).
2) Log in as each role you have (student, teacher, admin).
3) Capture: login, language (if shown), dashboard, assignments, marks, chat list, profile.
4) On each screenshot, write which table the visible titles/dates come from (announcements, assignments, marks, chat_room_summaries, profiles).
5) Store the images in a private notes folder — not in git if they show real names.

**CHECKPOINT — Independent verification**
Hand your screenshots to a classmate (or your future self tomorrow). They should be able to say: three roles, no public Register button, chat is per group, marks exist as a separate screen from assignments. If they think anyone can sign up, you labeled the product wrong — re-read this tutorial.

### Common mistakes

**WARNING — Treating MU Study like Instagram**
There is no public feed of all students. Data is scoped by group membership and RLS.

**WARNING — Assuming email login**
The field says Username. Email is an implementation detail of Auth.

**WARNING — Assuming Arabic or FCM**
This product ships EN/RU/UZ and in-app notifications.

## What you just learned

- MU Study is a closed school platform with admin-created accounts.
- Three roles sit on one Flutter client with different shells.
- Supabase is the backend; Flutter never talks to Postgres with a raw password.

## Self-check quiz (cover the answers)

**Q1.** Who creates student accounts?

**Answer 1**
An admin, via Admin → Users / admin_create_user. Not the student.

**Q2.** Name the three locales.

**Answer 2**
en, ru, uz.

**Q3.** What is a teacher sphere?

**Answer 3**
math or english — required when creating a teacher.

# Tutorial 2 — Database concepts for MU Study

| Field | Value |
|---|---|
| Tutorial | 2 — Database concepts |
| Difficulty | ★ easiest |
| Time | 30–40 minutes |
| Prerequisites | Tutorial 1 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Database Basics — Crash Course | https://www.youtube.com/watch?v=oCgc3fumTW0 |
| SQL for Beginners — freeCodeCamp | https://www.youtube.com/watch?v=HXV3zeQKqGY |
| PostgreSQL Tutorial — Amigoscode | https://www.youtube.com/watch?v=qw--VYLpxG4 |

## What a database is

A database is organized storage with rules. A spreadsheet can hold names, but it will not stop two students from sharing the same impossible grade or stop a deleted group from leaving orphan attendance rows. PostgreSQL enforces primary keys, foreign keys, checks, and — in this project — row-level security.

## Tables, rows, columns — using students and groups

A **table** is a named collection of rows with the same columns. A **row** is one student, one group, one mark. A **column** is one field (id, title, sphere). MU Study's students table does not store the display name. The name lives on profiles. students.profile_id points at profiles.id. That is a foreign key.

**Conceptual (IDs are UUIDs in the real app)**

```
profiles
id (PK=auth.users.id) | username | full_name | role
aaa...                | ali      | Ali Karimov| student
bbb...                | sara     | Sara Lee   | student
ccc...                | ms_ng    | N. Green   | teacher

students                teachers
id (PK) | profile_id(FK)   id (PK) | profile_id(FK) | sphere
s1      | aaa...           t1      | ccc...         | math

groups
id (PK) | teacher_id (FK→teachers.id) | title
g1      | t1                          | Algebra A

group_students
id | group_id (FK) | student_id (FK)
1  | g1            | s1
```

Notice students.id is not the same as profiles.id. Chat sender_id is a profile id (auth.uid()). Attendance.student_id is students.id. Mixing those UUIDs is the #1 join bug in this codebase. Always ask: "is this a profile id or a student row id?"

## Primary keys and foreign keys

A primary key uniquely identifies a row. MU Study uses UUID keys generated by gen_random_uuid(), except profiles.id which is the Auth user id (no default — you insert the same UUID you put into auth.users). A foreign key says “this column must point at a real row.” ON DELETE CASCADE means deleting a group deletes its assignments, memberships, and chat room. That is intentional: a group is a container.

## Data types you will see

| Type | Example column | Meaning |
|---|---|---|
| uuid | assignments.id | 128-bit identifier |
| text | groups.title | Unlimited string |
| timestamptz | assignments.due_date | Instant with time zone |
| date | attendance.date | Calendar day |
| boolean | profiles.is_disabled | true/false |
| numeric | marks.percentage | Decimal 0–100 |
| jsonb | announcements.image_urls | JSON array of URLs |
| user_role | profiles.role | enum student\|teacher\|admin |
| sphere_type | teachers.sphere | enum math\|english |

## Schema overview — why ~20 tables

| Table | Purpose |
|---|---|
| profiles | Login identity, role, prefs, avatar |
| teachers / students | Role-specific rows (sphere; student memberships) |
| groups / group_students | Class sections and roster |
| assignments / tasks / submissions / assignment_templates | Work and answers |
| lesson_sessions / attendance | Roll call per day |
| marks | Gradebook (weighted percentage) |
| announcements / notifications | Posts and per-user inbox |
| academic_years / academic_terms | School calendar (semesters 1–4) |
| chat_rooms / chat_messages / message_reads / message_reactions | Group chat |

Views are stored queries, not tables: chat_room_summaries, student_mark_averages, student_group_mark_averages. The Flutter chat list reads the summaries view, then merges unread counts from the chat_unread_counts RPC.

**TASK — 2.1 Data modeling**
On paper, draw: auth.users 1—1 profiles 1—0..1 teachers|students. teachers 1—* groups *—* students via group_students. groups 1—1 chat_rooms. groups 1—* assignments 1—* tasks 1—* submissions.
Label every FK column name exactly (profile_id, teacher_id, student_id, group_id).
Circle the two different IDs for a student (profiles.id vs students.id).

**CHECKPOINT — Independent verification**
Without this book, list eight table names and one FK each. If you wrote students.group_id, you invented a column — MU Study uses group_students instead. Correct your diagram.

## Common mistakes

**WARNING — Putting group_id on students**
A student can be in many groups. Use group_students.

**WARNING — Using integer IDs in examples as if they were real**
Production IDs are UUIDs.

**WARNING — Forgetting CASCADE**
Deleting a teacher deletes their groups if FK is CASCADE — know the blast radius.

## What you just learned

- Tables/rows/columns, PK/FK, enums, views.
- Student identity is two UUIDs; do not mix them.
- The 20-table map matches the live schema, not a blog example.

## Self-check quiz (cover the answers)

**Q1.** What table links a student to a group?

**Answer 1**
group_students.

**Q2.** Where is the teacher's subject stored?

**Answer 2**
teachers.sphere (sphere_type).

**Q3.** Is focus mode a table?

**Answer 3**
No. SharedPreferences only.

# Tutorial 3 — Supabase project setup

| Field | Value |
|---|---|
| Tutorial | 3 — Supabase setup |
| Difficulty | ★★ very easy |
| Time | 15–25 minutes |
| Prerequisites | Tutorial 1; a browser; an email |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Setup & Tutorial — Official | https://www.youtube.com/watch?v=I6nnp6kS3F8 |
| Supabase Full Course — Codementor | https://www.youtube.com/watch?v=rypsG5tqz8Q |

## Create the project

1. Go to supabase.com and sign in.
2. New project. Name it something like mu_study_dev. Pick the closest region. Set a database password and store it in a private notes file — not in the Flutter repo.
3. Wait until the project is healthy (a few minutes).
4. Settings → Data API (or API): copy Project URL https://YOURREF.supabase.co.
5. Copy the anon / publishable key (starts with eyJ or sb_publishable_). Never copy service_role into the app.
6. Open Table Editor (empty until you run SQL). Open SQL Editor for Tutorial 12+.

> **WARNING — Keys**
> anon/publishable = client. service_role = god mode. If service_role leaks, rotate it immediately. The Flutter Env class even rejects keys that start with sb_secret_.

## What you will run later

This tutorial does not apply the full schema yet (that is safer after you understand SQL). When you are ready, paste docs/tutorial/COMPLETE_BACKEND.sql in the SQL Editor top to bottom, then first_admin_bootstrap.sql with YOUR_STRONG_PASSWORD.

**SQL Editor smoke test (now)**

```
SELECT current_timestamp AS now, current_database() AS db;
```

> **TASK — 3.1 Supabase exploration**
> Create the project. Screenshot: Settings page showing Project URL (blur the key if you share the image). Run SELECT current_timestamp; in SQL Editor and screenshot the result. Write down region and project ref in private notes.

**CHECKPOINT — Independent verification**
A friend should be able to open your (blurred) URL screenshot and see a supabase.co host. Your SQL screenshot must show a timestamp, not an error. If Table Editor already shows 20 tables, you ran the schema early — that is fine, but still keep the smoke query.

## Common mistakes

**WARNING — Putting service_role in .env used by Flutter**
Env.sanitize should empty it; if you bypass that, any user of the APK is a DBA.

**WARNING — Using the hosted demo project as production**
nmmdvdprczukaskxxdrs in Env is a development default. Create your own for learning.

## What you just learned

- Project URL + anon key are the only client secrets.
- SQL Editor is your source of truth for schema work.

## Self-check quiz (cover the answers)

**Q1.** Which key goes in Flutter?

**Answer 1**
anon / publishable, never service_role.

**Q2.** What does SELECT current_timestamp prove?

**Answer 2**
The database is reachable and you can run SQL.

# Tutorial 4 — Environment variables and configuration

| Field | Value |
|---|---|
| Tutorial | 4 — Environment variables |
| Difficulty | ★★ very easy |
| Time | 20 minutes |
| Prerequisites | Tutorial 3 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Environment Variables — Fireship | https://www.youtube.com/watch?v=HXZpY4sUmqQ |
| flutter_dotenv package walkthrough | https://pub.dev/packages/flutter_dotenv |

## Why .env exists

Source code is copied, forked, and pushed. Secrets must not ride along. MU Study loads SUPABASE_URL and SUPABASE_ANON_KEY (or SUPABASE_PUBLISHABLE_KEY) from a .env file via flutter_dotenv, or from --dart-define-from-file=.env at run time.

**.env (local only — already in .gitignore)**

```
SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
SUPABASE_ANON_KEY=eyJ_your_anon_or_publishable_key
```

## How Flutter loads it in this repo

lib/core/constants/env.dart is the whole story. Env.load() first checks String.fromEnvironment (dart-define). If SUPABASE_ANON_KEY is empty, it tries dotenv.load(fileName: '.env'). Then _sanitizeKey() rejects placeholders, your_ prefixes, paste_your, sb_secret_, and keys that do not start with eyJ or sb_publishable_. If the key is rejected, Env.isConfigured is false and main.dart shows SetupRequiredScreen instead of initializing Supabase.

pubspec.yaml lists .env under flutter assets so dotenv can load it on devices. That means a release APK can contain the anon key — which is expected for a public client key — but it must never contain service_role. Prefer CI dart-defines for production builds so you are not editing a committed asset.

**Run (README)**

```
flutter pub get
flutter run --dart-define-from-file=.env
```

> **TASK — 4.1 Create .env**
> 1) In the project root, create .env with YOUR URL and anon key (placeholders shown above).
> 2) Confirm .gitignore contains .env (it does in this repo).
> 3) Run flutter run --dart-define-from-file=.env (or make run if your Makefile injects env).
> 4) You must not stay on SetupRequiredScreen if the key is valid.

**CHECKPOINT — Independent verification**
Run: git check-ignore -v .env — it must report .gitignore. Run the app with an empty key: SetupRequiredScreen appears. Restore the key: splash/login appears. That A/B test proves Env, not luck.

## Common mistakes

**WARNING — Committing .env**
Rotate the anon key if it hit a remote.

**WARNING — Whitespace or quotes around the key**
trim() helps, but quotes can still break JWT parse.

**WARNING — Using the service role because “login failed”**
Login failures are almost never fixed by god keys. Fix RPC grants and passwords.

## What you just learned

- Env.isConfigured gates the entire app.
- .env is gitignored; dart-define is the clean run path.

## Self-check quiz (cover the answers)

**Q1.** What screen appears when the key is sanitized to empty?

**Answer 1**
SetupRequiredScreen.

**Q2.** Name two accepted key prefixes.

**Answer 2**
eyJ and sb_publishable_.

# Tier 2 — Flutter basics

You create the project shape, learn Dart, build widgets, and wire go_router the way MU Study does: splash, login, language gate, then a role shell.

# Tutorial 5 — Flutter project structure

| Field | Value |
|---|---|
| Tutorial | 5 — Project structure |
| Difficulty | ★★ very easy |
| Time | 25 minutes |
| Prerequisites | Flutter SDK; Tutorial 1 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Project Structure — The Net Ninja | https://www.youtube.com/watch?v=u3R2cCP_ynI |
| Flutter for Beginners — Udacity | https://www.youtube.com/watch?v=7nAMS3cAr3c |

## What flutter create gives you

flutter create --org com.example --project-name mu_study . produces android/, ios/, web/, lib/main.dart, and pubspec.yaml. MU Study's applicationId is com.example.mu_study and the iOS bundle is com.example.muStudy. You will not write Kotlin/Swift for features; you will add microphone/photo permissions for chat voice and images.

**Actual lib/ layout**

```
lib/
  main.dart
  core/          constants, theme, router, services, utils
  models/        enums + domain classes
  providers/     Riverpod (auth, homework, chat, …)
  screens/       auth, student, teacher, admin, chat, profile
  components/    shell, cards, attachments, charts
  l10n/          ARB + generated localizations
  supabase/      supabase_client.dart one-liner
```

pubspec.yaml is the manifest: SDK constraint, dependencies, assets (images, lottie, .env), and generate: true for l10n. After editing it, flutter pub get. After editing ARB files, flutter gen-l10n (or the next flutter run).

**TASK — 5.1 Walk the tree**
From the repo root: list lib/ directories. Open pubspec.yaml and count dependencies. Open android/app/src/main/AndroidManifest.xml and find INTERNET and RECORD_AUDIO. Open ios/Runner/Info.plist and find microphone / photo usage strings. Run flutter pub get and confirm no errors.

**CHECKPOINT — Independent verification**
Write a 10-line tree of lib/ from memory. If you invented lib/services/ as a top-level folder, look again — services live under lib/core/services/ and screens call providers.

### Common mistakes

> **WARNING — Editing files under .dart_tool or build/**
> Those are generated. Your code is lib/.

> **WARNING — Forgetting generate: true**
> AppLocalizations will not exist.

## What you just learned

- lib/ is the product. pubspec is the bill of materials.

## Self-check quiz (cover the answers)

**Q1.** Where is go_router configured?

> **Answer 1**
> lib/core/router/app_router.dart

**Q2.** Where is the Supabase client accessor?

> **Answer 2**
> lib/supabase/supabase_client.dart

# Tutorial 6 — Dart basics

| Field | Value |
|---|---|
| Tutorial | 6 — Dart basics |
| Difficulty | ★★ very easy |
| Time | 45 minutes |
| Prerequisites | Tutorial 5 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Dart for Beginners — Fireship | https://www.youtube.com/watch?v=Ej_Pcr4uC2Q |
| Dart Null Safety — Dart Official | https://www.youtube.com/watch?v=IqD_wjsDMKU |

## Types you will use daily

**dart**

```
final String username = 'ali';
const int pageSize = 20;
bool isDisabled = false;
String? avatarUrl; // null until uploaded
final roles = <String>['student', 'teacher', 'admin'];
final row = <String, dynamic>{'full_name': 'Ali', 'role': 'student'};
```

final is assign-once. const is compile-time. var exists but this codebase prefers explicit types on public APIs. dynamic appears in JSON maps from Supabase — you immediately cast into models.

## Functions and named parameters

**Like AuthRepository.changePassword**

```
Future<void> changePassword({
  required String username,
  required String currentPassword,
  required String newPassword,
}) async {
  // …
}
```

## Classes — Profile is the teaching example

lib/models/profile.dart is a plain class: constructor, fields, fromJson, toJson. JSON keys are snake_case. There is no Freezed code generation in the models folder even though freezed is in pubspec (available, not used for Profile). Learn both styles; ship what the repo actually compiles.

**Null safety in login**

```
final email = await resolveAuthEmail(username);
if (email == null) {
  throw const AuthException('Incorrect username or password');
}
await supabase.auth.signInWithPassword(email: email, password: password);
```

**TASK — 6.1 Dart exercises (file: test or dartpad)**
1) Class User { name, email, role } with a role string.
2) Function studentsOnly(List) → only role == 'student'.
3) Function printUsers(Map) prints username → role.
4) Give email a String? if it can be missing; do not use ! unless you just checked null.

**CHECKPOINT — Independent verification**
Write a tiny test or main() that fails if studentsOnly keeps a teacher. Run it. If you only "looked at the code," you did not verify.

## Common mistakes

**WARNING — Using user.role! everywhere**
If role is non-nullable, do not add ?. If it is nullable, handle both cases.

**WARNING — Parsing JSON without fromJson**
You will typo fullName vs full_name.

## What you just learned

- final/const, named args, null-aware login path, model classes.

## Self-check quiz (cover the answers)

**Q1.** Why is email String? from the RPC?

**Answer 1**
Disabled or unknown usernames return null.

**Q2.** What does Profile.fromJson read for the name?

**Answer 2**
json['full_name'].

# Tutorial 7 — Flutter widgets and layouts

| Field | Value |
|---|---|
| Tutorial | 7 — Widgets & layouts |
| Difficulty | ★★★ easy |
| Time | 45 minutes |
| Prerequisites | Tutorial 6 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Widgets — The Net Ninja | https://www.youtube.com/watch?v=u_rbIqnRDms |
| Flutter Layout Tutorial — Flutter Official | https://www.youtube.com/watch?v=_58-kTsPq9c |
| Material Design in Flutter — Google | https://www.youtube.com/watch?v=DL0Ix1lnK4k |

Everything on screen is a widget. LoginScreen is a ConsumerStatefulWidget: it owns TextEditingControllers and a loading flag. The visual tree is Scaffold → decorations → fields → PrimaryButton. Read lib/screens/auth/login_screen.dart after you can build a boring Column yourself.

**Minimum login-shaped UI (practice, not the production file)**

```
Scaffold(
  appBar: AppBar(title: const Text('MU Study')),
  body: Padding(
    padding: const EdgeInsets.all(24),
    child: Column(
      children: [
        TextField(decoration: InputDecoration(labelText: 'Username')),
        TextField(obscureText: true, decoration: InputDecoration(labelText: 'Password')),
        ElevatedButton(onPressed: () {}, child: Text('Sign in')),
      ],
    ),
  ),
);
```

Production login uses localized labels (context.l10n.loginUsername), AppLogo, paper décor for Nature Paper style, and maps errors to loginIncorrect / loginConfigError. There is no “Forgot password?” for students — admins reset passwords (admin_reset_password). Do not add a fake forgot-password that emails mustudy.internal addresses.

**TASK — 7.1 Build a practice login UI**
In a scratch file or a temporary route, build username + password + sign-in button with spacing. Then open the real LoginScreen and list five widgets it uses that yours does not (logo, l10n, PrimaryButton, error text, loading).

**CHECKPOINT — Independent verification**
flutter run shows your practice UI. A screenshot with empty AppBar title means you skipped MaterialApp or localization — wrap with MaterialApp for the exercise.

## Common mistakes

**WARNING — Unbounded ListView inside Column**
Use Expanded or ListView as the body.

**WARNING — Hard-coded English on production screens**
Use ARB keys; Tutorial 18.

## What you just learned

- Scaffold, fields, buttons, why production login is richer.

## Self-check quiz (cover the answers)

**Q1.** Who resets a forgotten password today?

**Answer 1**
An admin, not a public email flow.

# Tutorial 8 — Stateful vs Stateless widgets

| Field | Value |
|---|---|
| Tutorial | 8 — Stateful vs Stateless |
| Difficulty | ★★★ easy |
| Time | 30 minutes |
| Prerequisites | Tutorial 7 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Stateful vs Stateless — Flutter Official | https://www.youtube.com/watch?v=wE7khGHVkYY |
| Flutter State Management — Fireship | https://www.youtube.com/watch?v=lTnXr8XO0Ys |

StatelessWidget is a function of its constructor arguments. Greeting(name: 'Ali') never changes unless the parent passes a new name. StatefulWidget owns a State object. LoginScreen's _loading and _error live there. setState schedules a rebuild.

MU Study uses State for controllers and ephemeral UI (forms, sheets). Shared data (profile, assignments, chat) lives in Riverpod so two screens stay in sync. If you put the assignment list in a State object on one screen, the dashboard will not see updates. That is why Tutorial 10 exists.

**Ephemeral vs app state**

```
// Ephemeral — State is correct
bool _obscure = true;

// App state — Riverpod is correct
final currentProfileProvider = FutureProvider<Profile?>(...);
```

> **TASK — 8.1 Mini todo widget**
> StatefulWidget with List todos, TextField, add button, ListView. Then write two sentences: why this todo list should NOT be how MU Study stores assignments (no RLS, no teacher, no persistence).

> **CHECKPOINT — Independent verification**
> Adding a todo updates the list without a full app restart. Hot reload may reset State — that is expected. If add does nothing, you forgot setState.

## Common mistakes

> **WARNING — Calling setState after dispose**
> Guard with if (!mounted) return; after awaits — LoginScreen does this.

## What you just learned

- State is for local UI. Riverpod is for shared/async data.

## Self-check quiz (cover the answers)

**Q1.** Is currentProfileProvider a State field?

> **Answer 1**
> No. It is a FutureProvider.

# Tutorial 9 — Navigation with GoRouter

| Field | Value |
|---|---|
| Tutorial | 9 — GoRouter |
| Difficulty | ★★★ easy |
| Time | 40 minutes |
| Prerequisites | Tutorial 8 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| GoRouter — Flutter Official | https://www.youtube.com/watch?v=Y9DLpnBckso |
| Navigation in Flutter — Fireship | https://www.youtube.com/watch?v=xYPCrZ5iHzQ |

lib/core/router/app_router.dart defines routerProvider. GoRouter starts at /splash. _RouterNotifier listens to authStateProvider and currentProfileProvider and calls notifyListeners so redirect re-runs. That is how login “just works” without a manual context.go in every success path (login still goes to /boot to settle).

## Redirect rules (actual)

1. If location is /splash, do not redirect (splash decides).
2. If no session → /login (unless already there).
3. If profile is still loading (first load) → /boot.
4. If profile is null → /login.
5. If profile.isDisabled → /account-disabled.
6. If preferred_language is empty → /language.
7. If on /login, /boot, or /language after all gates pass → role home (/student/dashboard, /teacher/dashboard, /admin/dashboard).
8. Otherwise null (allow the navigation).

Each role is a ShellRoute with destinations. Student includes Focus. Teacher includes gradebook and attendance. Admin includes users and groups. Chat is parameterized: ChatListScreen(basePath: '/teacher/chat') so one widget serves three shells.

**Path parameters (homework)**

```
/student/tasks/:taskId  → StudentTaskSubmitScreen
/teacher/assignments/:id → TeacherHomeworkPackScreen
/*/chat/:roomId         → ChatRoomScreen
```

> **TASK — 9.1 Trace one navigation**
> On paper, start at /splash with no session. Write every location until student home for a user who has never chosen a language. Then repeat for a disabled teacher. Open app_router.dart and confirm your trace.

**CHECKPOINT — Independent verification**
Log in as a new user (no preferred_language). You must hit LanguageSelectionScreen before any dashboard. Log in as a disabled profile (set is_disabled true in SQL). You must see account disabled copy, not a dashboard.

## Common mistakes

**WARNING — Redirecting /splash to /login immediately**
You skip the brand splash and may race auth restore.

**WARNING — Checking role only in the widget**
Deep links would still open teacher URLs. Redirect is the gate.

## What you just learned

- refreshListenable + redirect is the session backbone.

## Self-check quiz (cover the answers)

**Q1.** What path is the language gate?

**Answer 1**
/language

**Q2.** Why /boot exists?

**Answer 2**
Hold UI while the first profile Future loads without tearing down a shell.

# Tier 3 — State management

Riverpod is how MU Study holds session, lists, and streams. Models are how JSON from PostgREST becomes typed Dart. Master both before writing repositories.

# Tutorial 10 — Riverpod basics

| Field | Value |
|---|---|
| Tutorial | 10 — Riverpod |
| Difficulty | ★★★ easy |
| Time | 50 minutes |
| Prerequisites | Tutorial 9 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Riverpod Tutorial — Fireship | https://www.youtube.com/watch?v=9EDFCZ0SqEA |
| Riverpod for State Management — Codepur | https://www.youtube.com/watch?v=GxXSiSkJUvY |
| Riverpod Documentation | https://riverpod.dev/ |
| Riverpod Complete Course — Tomic Riedel | https://www.youtube.com/watch?v=W_nqHaYVHjE |

main.dart wraps the app in ProviderScope. Widgets that need data extend ConsumerWidget or ConsumerStatefulWidget and call ref.watch / ref.read. watch rebuilds when the value changes. read is for clicks (login, increment).

## Provider types in this repo

| Type | Example | Use |
|---|---|---|
| Provider | authRepositoryProvider | Construct a service once |
| StreamProvider | authStateProvider | Supabase onAuthStateChange |
| FutureProvider | currentProfileProvider | One-shot fetch with loading/error |
| FutureProvider.family | announcementsProvider | Parameterized by groupId |
| StreamProvider (inbox) | notificationsProvider | Yield list, then re-yield on Realtime |
| Notifier / StateNotifier | chat messages, paging lists | Mutable collections + methods |
| State-like controllers | themeModeProvider, localeProvider | User preferences |

**Watching in a widget**

```
final profile = ref.watch(currentProfileProvider);
return profile.when(
  loading: () => const CircularProgressIndicator(),
  error: (e, _) => Text('Error: $e'),
  data: (p) => Text(p?.fullName ?? ''),
);
```

ref.invalidate(currentProfileProvider) after login forces a refetch. ref.refresh is similar and returns the new future. Use invalidate when you already navigated away and only need the next watch to reload.

**TASK — 10.1 / 10.2 Practice providers**
10.1 Build a counter with StateProvider and two buttons in a scratch app.
10.2 Build a FutureProvider that returns a fake Profile after a 1s delay. Show loading, error (throw once), and data. Add a retry that invalidates the provider.
Then open auth_provider.dart and match each real provider to a type in the table.

**CHECKPOINT — Independent verification**
Your counter updates without setState in the widget (the provider owns the int). Your FutureProvider demo shows all three AsyncValue branches. If you only used setState, redo 10.1.

## Common mistakes

**WARNING — watch inside onPressed**
Use read for events. watch belongs in build.

**WARNING — Forgetting ProviderScope**
Providers throw. main.dart already adds it.

## What you just learned

- watch vs read, AsyncValue.when, invalidate after login.

## Self-check quiz (cover the answers)

**Q1.** Which provider drives router refresh?

**Answer 1**
authStateProvider (+ currentProfileProvider).

# Tutorial 11 — Models and data classes

| Field | Value |
|---|---|
| Tutorial | 11 — Models |
| Difficulty | ★★★ easy |
| Time | 35 minutes |
| Prerequisites | Tutorial 6 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Freezed Package — Codepur | https://www.youtube.com/watch?v=PkJCU-Qm3QM |
| Dart Models & Serialization — Fireship | https://www.youtube.com/watch?v=xXCG15nVP_8 |

Models are the contract with PostgREST. lib/models/enums.dart maps Dart enums to database strings. homework_pack is the important mismatch: Dart uses homeworkPack, Postgres uses homework_pack. assignmentTypeFromString / assignmentTypeToDb exist so you never send 'homeworkPack' to SQL.

Freezed is in pubspec for future use and for people following generic tutorials. The shipped models are hand-written. That is easier to read in a textbook and avoids a build_runner step for every field change. If you introduce Freezed, you must run dart run build_runner build and commit the generated files your team agrees on.

**Assignment.fromJson (real mapping)**

```
type: assignmentTypeFromString(json['type'] as String),
dueDate: json['due_date'] != null
    ? DateTime.parse(json['due_date'] as String)
    : null,
```

**TASK — 11.1 Create / inspect MU Study models**
Open profile.dart, assignment.dart, attendance.dart, mark.dart, chat.dart, notification.dart. For each class, write the table name and the JSON keys. Write a Dart snippet that parses a fake assignments row map and prints title + type. Assert type == AssignmentType.homeworkPack when type is homework_pack.

**CHECKPOINT — Independent verification**
Your snippet fails if you pass type: 'homeworkPack' (wrong) and succeeds for 'homework_pack'. That single test prevents a silent insert bug.

## Common mistakes

**WARNING — Sending enum.toString()**
Gives AssignmentType.homeworkPack — invalid for Postgres.

**WARNING — Assuming marks live on submissions only**
marks is its own table; submission_id is optional/unique.

## What you just learned

- snake_case JSON, enum DB helpers, hand-written models.

## Self-check quiz (cover the answers)

**Q1.** What helper writes assignment type to the database?

> **Answer 1**
> assignmentTypeToDb.

# Tier 4 — Database and backend

SQL, Auth, Flutter queries, and RLS. This tier is the difference between a demo and a school product.

# Tutorial 12 — SQL basics

| Field | Value |
|---|---|
| Tutorial | 12 — SQL basics |
| Difficulty | ★★★ easy |
| Time | 50 minutes |
| Prerequisites | Tutorials 2 and 3 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| SQL for Beginners — freeCodeCamp | https://www.youtube.com/watch?v=HXV3zeQKqGY |
| PostgreSQL Tutorial — Amigoscode | https://www.youtube.com/watch?v=qw--VYLpxG4 |
| SQL for Beginners — Programming with Mosh | https://www.youtube.com/watch?v=7S_tz1z_5bA |

Open SQL Editor on your project. After COMPLETE_BACKEND.sql is applied, these queries match real names. Until then, run them mentally or on the development project.

**Read / filter / join / aggregate**

```
-- All group titles for a teacher row
SELECT id, title FROM groups WHERE teacher_id = '<teacher-uuid>';

-- Students in a group with names
SELECT p.username, p.full_name
FROM group_students gs
JOIN students s ON s.id = gs.student_id
JOIN profiles p ON p.id = s.profile_id
WHERE gs.group_id = '<group-uuid>'
ORDER BY p.full_name;

-- Assignments due after now
SELECT title, due_date FROM assignments
WHERE due_date > now()
ORDER BY due_date;

-- Submissions per assignment (via tasks)
SELECT a.title, COUNT(sub.id) AS submitted
FROM assignments a
JOIN tasks t ON t.assignment_id = a.id
LEFT JOIN submissions sub ON sub.task_id = t.id AND sub.status <> 'pending'
GROUP BY a.id, a.title;

-- Weighted average (same idea as student_mark_averages view)
SELECT student_id,
  ROUND(SUM(percentage * weight) / NULLIF(SUM(weight), 0), 2) AS avg
FROM marks GROUP BY student_id;
```

> **WARNING — Writes**
> INSERT/UPDATE/DELETE in the SQL Editor run as the postgres role and bypass RLS. That is fine for seeding. It is not how the app works. Never “test security” as postgres.

**TASK — 12.1 Write SQL**
1) Assignments due after today (query above).
2) Marks for one student_id with group title (JOIN groups).
3) COUNT submissions per assignment.
Run them. Save the result screenshots privately.

**CHECKPOINT — Independent verification**
Each query has a WHERE or GROUP BY you can explain. If COUNT(*) from marks as postgres returns rows a student JWT cannot see, that is expected — you are DBA here.

## Common mistakes

**WARNING — SELECT * FROM students expecting names**
Join profiles.

**WARNING — Filtering assignments with due_date > CURRENT_DATE and forgetting time**
due_date is timestamptz.

## What you just learned

- SELECT/JOIN/GROUP BY on real MU Study names.

## Self-check quiz (cover the answers)

**Q1.** Which view already averages marks?

**Answer 1**
student_mark_averages and student_group_mark_averages.

# Tutorial 13 — Supabase authentication (the real login)

| Field | Value |
|---|---|
| Tutorial | 13 — Authentication |
| Difficulty | ★★★ easy |
| Time | 45 minutes |
| Prerequisites | Tutorials 3 and 12 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Auth — Official | https://www.youtube.com/watch?v=6ow_jW7iS24 |
| Flutter Authentication — Fireship | https://www.youtube.com/watch?v=yH3NQFr3k0s |
| JWT explained — Fireship | https://www.youtube.com/watch?v=7Q17ubqLfaM |

## Flow

**Username login**

```
User types username + password
  → AuthRepository.signIn
  → rpc get_auth_email_by_username(p_username)
  → NULL if missing or is_disabled
  → signInWithPassword(email, password)
  → JWT stored by supabase_flutter
  → authStateProvider emits signedIn
  → currentProfileProvider loads profiles row
  → router redirect to language or role home
```

**get_auth_email_by_username (exact idea)**

```
SELECT auth_email FROM public.profiles
WHERE lower(username) = lower(trim(p_username))
  AND is_disabled = FALSE
LIMIT 1;
```

admin_create_user builds email as lower(trim(replace(username,'@',''))) || '@mustudy.internal'. Username @admin becomes admin@mustudy.internal. The first admin cannot call that RPC (nobody is admin yet). Use first_admin_bootstrap.sql once.

Do not create users in Authentication → Add user and forget profiles. The app will sign in then bounce to login because currentProfileProvider is null. Always create the pair: auth.users + identities + profiles + teachers/students.

**TASK — 13.1 / 13.2 First users**
13.1 After schema: run first_admin_bootstrap.sql with a password you invented. Verify: SELECT username, auth_email, role FROM profiles WHERE role = 'admin';
13.2 Run the Flutter app, log in as @admin. You should reach language then admin dashboard. Then create a teacher (math) and a student from Admin → Users. Log in as each.

**CHECKPOINT — Independent verification**
Wrong password → localized loginIncorrect, not a stack trace in release. Disabled user → RPC returns null → same incorrect message. SQL: SELECT auth.uid(); in a user session is not available in the dashboard the same way — instead confirm auth.users.email matches profiles.auth_email.

## Common mistakes

**WARNING — Inventing email in Dart**
Breaks if the scheme changes; leaks disabled usernames if you only skip is_disabled in the UI.

**WARNING — Unconfirmed email**
Bootstrap sets email_confirmed_at. Dashboard-created users may need confirm email disabled in Auth settings.

## What you just learned

- RPC → Auth → profile → router. First admin is a SQL bootstrap.

## Self-check quiz (cover the answers)

**Q1.** What email does username 'ms.green' become?

**Answer 1**
ms.green@mustudy.internal (dots stay; @ is stripped if present).

# Tutorial 14 — Supabase queries from Flutter

| Field | Value |
|---|---|
| Tutorial | 14 — Flutter queries |
| Difficulty | ★★★ easy |
| Time | 45 minutes |
| Prerequisites | Tutorials 12–13 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Database Queries — Official | https://www.youtube.com/watch?v=zVlc_O_oJgA |
| Flutter + Supabase — Fireship | https://www.youtube.com/watch?v=eKvj2P3sJVU |

lib/supabase/supabase_client.dart exports supabase. Providers call supabase.from('table').select() with filters. RLS silently filters rows — an “empty list” is often a policy or membership problem, not an empty school.

**Patterns used in MU Study**

```
// Profile
await supabase.from('profiles').select().eq('id', uid).maybeSingle();

// Nested join (group members)
from('group_students').select('..., students(profiles(...))');

// RPC
await supabase.rpc('get_auth_email_by_username', params: {'p_username': name});
await supabase.rpc('ensure_lesson_session', params: {...});

// Insert assignment
await supabase.from('assignments').insert({...});
```

Ordering and limits: assignments and notifications use .order('created_at') and .limit(50) or paging helpers (PageResult). Do not download an entire marks table to the phone when a group filter exists.

**TASK — 14.1 Read the real repositories**
Open auth_provider.dart (profile fetch), group_providers.dart, assignment_providers.dart, marks_providers.dart. Write a table: method → .from/.rpc → filters. You do not need a separate students_repository.dart — this app folds access into providers. If you create one for practice, keep it out of the production path until it matches RLS.

**CHECKPOINT — Independent verification**
As a student JWT (use the app, not the SQL Editor as postgres), open marks. You should see only your rows. As that student’s classmate, you should not see the first student’s marks. That is Tutorial 15, but you can already observe it.

## Common mistakes

> **WARNING — Using .single() when 0 rows possible**
> Use maybeSingle() like ProfileRepository.

> **WARNING — .select() without filter on a huge table**
> Page and filter by group_id.

## What you just learned

- from/select/eq/rpc, empty list vs error, provider-as-repository.

## Self-check quiz (cover the answers)

**Q1.** Where is supabase defined?

> **Answer 1**
> Supabase.instance.client via supabase_client.dart.

# Tutorial 15 — Row Level Security and permissions

| Field | Value |
|---|---|
| Tutorial | 15 — RLS |
| Difficulty | ★★★★ medium |
| Time | 60 minutes |
| Prerequisites | Tutorials 12–14 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase RLS — Official | https://www.youtube.com/watch?v=Ow_Utn9OIMQ |
| PostgreSQL Row-Level Security — Amigoscode | https://www.youtube.com/watch?v=2IXAHGQJ0as |

ALTER TABLE … ENABLE ROW LEVEL SECURITY is not optional. Without policies, authenticated users see zero rows (default deny). Policies are named SQL predicates. MU Study factors repeated checks into functions: is_admin(), is_teacher_of_group(id), is_student_in_group(id), can_access_chat_room(id), current_profile_role().

**Mental policies (see export/rls_policies.sql for exact SQL)**

```
marks SELECT student: student_id = my students.id
marks SELECT teacher: is_teacher_of_group(group_id)
marks SELECT admin: is_admin()
marks INSERT/UPDATE: teacher of group or admin
notifications SELECT: user_id = auth.uid()
chat_messages *: can_access_chat_room(room_id)
```

UPDATE needs a SELECT policy on the same rows. If teachers can UPDATE marks but cannot SELECT them, updates affect 0 rows with no drama. When debugging “save does nothing,” check SELECT policies first.

**WARNING — Never authorize from user_metadata**
Supabase user_metadata is user-editable. Role lives in profiles.role. Helpers read that table (or related rows), not JWT user_metadata.

**TASK — 15.1 Prove RLS with two students**
Create student A and B in the same group. As teacher, insert two different marks. Log in as A — only A’s marks. Log in as B — only B’s. Log in as teacher — both. Optional: as A, try supabase.from('marks').select() in a debug print; count rows.

**CHECKPOINT — Independent verification**
A screenshot of two student accounts showing different mark lists, plus one SQL query as postgres showing both rows, is the proof. If both students see both rows, RLS is missing or you queried as service_role.

## Common mistakes

**WARNING — Testing RLS in SQL Editor as postgres**
You will always see everything. Use the app JWT.

**WARNING — Policy using submissions.group_id on marks**
marks has its own group_id. Copy-paste from blogs will not compile.

## What you just learned

- Default deny, helper functions, UPDATE needs SELECT.

## Self-check quiz (cover the answers)

**Q1.** Who can SELECT all profiles?

**Answer 1**
Admins; others see self, groupmates, chat senders — see policies.

# Tier 5 — Advanced platform features

Storage, Realtime, localization, theming, and the real notification system (not FCM).

# Tutorial 16 — File upload and Supabase Storage

| Field | Value |
|---|---|
| Tutorial | 16 — Storage |
| Difficulty | ★★★★ medium |
| Time | 50 minutes |
| Prerequisites | Tutorial 14 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Storage — Official | https://www.youtube.com/watch?v=s7EwCRwxo5s |
| File Upload in Flutter — Fireship | https://www.youtube.com/watch?v=KLY_2vtuWsQ |

Buckets in this product: avatars (public), submissions, assignments, announcements, chat-images, chat-videos, chat-voices, chat-files (private). Public avatars use getPublicUrl. Private objects use createSignedUrl after upload.

**Path conventions**

```
avatars/{profileId}/avatar.jpg
submissions/{studentId}/{taskId}/{filename}
chat-images/{roomId}/{profileId}/{timestamp}_{filename}
```

Storage policies parse folder segments. Chat uploads require folder[1]=room the user can access and folder[2]=auth.uid(). A mismatch is 403, not a mysterious empty URL.

> **TASK — 16.1 Submit a file as a student**
> Create a homework_pack with one task. As student, open StudentTaskSubmitScreen, pick a small PDF or image, submit. In Storage, open submissions and find the object. In Table Editor, confirm submissions.file_url is a signed or stored URL. As another student, you must not download that object.

> **CHECKPOINT — Independent verification**
> Teacher can open the file from the review UI. Second student cannot. If both can, the bucket is public or the policy is USING(true).

## Common mistakes

> **WARNING — Uploading to avatars for homework**
> Wrong bucket; URLs become world-readable.

> **WARNING — Granting only INSERT on storage**
> Upsert/replace needs INSERT+SELECT+UPDATE.

## What you just learned

- Bucket privacy, path folders, signed URLs.

## Self-check quiz (cover the answers)

**Q1.** Which bucket is public?

> **Answer 1**
> avatars.

# Tutorial 17 — Realtime live updates

| Field | Value |
|---|---|
| Tutorial | 17 — Realtime |
| Difficulty | ★★★★ medium |
| Time | 50 minutes |
| Prerequisites | Tutorial 14 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Real-time — Official | https://www.youtube.com/watch?v=lsKDFTL1YbQ |
| Real-time Chat Flutter — Fireship | https://www.youtube.com/watch?v=F7OX-pxjvPQ |

supabase_flutter v2 uses channel.onPostgresChanges, not the old .from().on() API you will still see in outdated blogs. notificationsProvider subscribes to public.notifications filtered by user_id. Chat providers subscribe to chat_messages, message_reads, and message_reactions. Tables must be in the supabase_realtime publication. message_reactions uses REPLICA IDENTITY FULL so updates/deletes emit payloads.

**notificationsProvider (shape)**

```
channel('notifications-${profile.id}')
  .onPostgresChanges(
    event: PostgresChangeEvent.all,
    schema: 'public',
    table: 'notifications',
    filter: PostgresChangeFilter(
      type: PostgresChangeFilterType.eq,
      column: 'user_id',
      value: profile.id,
    ),
    callback: (_) => changes.add(null),
  )
  .subscribe();
```

The stream yields an initial load, then re-fetches on each change. That is simpler than applying diffs and matches a 50-row inbox. Chat may append incrementally — read chat_providers.dart for the exact strategy.

> **TASK — 17.1 Two devices, one room**
> Open the same group chat on two accounts (teacher + student). Send a text. The other device must show it without pulling to refresh. Insert a notification row for user A in SQL; A's inbox badge must update.

> **CHECKPOINT — Independent verification**
> If only refresh-after-navigate works, Realtime is not subscribed or the table is not in the publication. Check Database → Publications → supabase_realtime.

## Common mistakes

> **WARNING — Copying RealtimeListenTypes from old tutorials**
> Use onPostgresChanges.

> **WARNING — Forgetting ref.onDispose(removeChannel)**
> Leaked channels; missed events after logout.

## What you just learned

- Channels, filters, publication, dispose.

## Self-check quiz (cover the answers)

**Q1.** Why REPLICA IDENTITY FULL on reactions?

> **Answer 1**
> Realtime needs old row keys on UPDATE/DELETE.

# Tutorial 18 — Localization (English, Russian, Uzbek)

| Field | Value |
|---|---|
| Tutorial | 18 — Localization |
| Difficulty | ★★★★ medium |
| Time | 45 minutes |
| Prerequisites | Tutorials 7 and 10 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Localization — Official | https://www.youtube.com/watch?v=Zw3-fHMjVLw |
| i18n in Flutter — Fireship | https://www.youtube.com/watch?v=QDmKQkeSLfI |

l10n.yaml points at lib/l10n, template app_en.arb, class AppLocalizations, nullable-getter: false. That last flag means AppLocalizations.of(context) is non-null if delegates are installed — the code uses context.l10n via l10n_x.dart.

Languages are English, Russian, and Uzbek — not Arabic. First-run LanguageSelectionScreen writes profiles.preferred_language and SharedPreferences. The router blocks dashboards until that column is set. Profile settings can change it later via localeProvider.

**lib/l10n/app_en.arb (excerpt)**

```
 "loginSignIn": "Sign in",
 "navFocus": "Focus",
 "chatTitle": "Chat",
```

> **TASK — 18.1 Switch languages**
> Log in, pick Russian, confirm dashboard strings change. Repeat Uzbek. Restart the app — language persists. Add a harmless ARB key to all three files, run flutter gen-l10n, display it on the profile screen, then remove it if you do not want to keep it.

> **CHECKPOINT — Independent verification**
> SQL: preferred_language is ru or uz. flutter gen-l10n exits 0. A missing key in app_uz.arb fails generation — that failure is the test that you must edit all three.

## Common mistakes

> **WARNING — Hard-coding Arabic from a generic prompt**
> Wrong product.

> **WARNING — Only editing app_en.arb**
> Other locales fall back or fail CI.

## What you just learned

- ARB trio, language gate, context.l10n.

## Self-check quiz (cover the answers)

**Q1.** Which file is the template?

> **Answer 1**
> app_en.arb

# Tutorial 19 — Theming and dark mode

| Field | Value |
|---|---|
| Tutorial | 19 — Theming |
| Difficulty | ★★★★ medium |
| Time | 40 minutes |
| Prerequisites | Tutorials 7 and 10 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Theming — Official | https://www.youtube.com/watch?v=oP3asNgo_Tw |
| Dark Mode in Flutter — Fireship | https://www.youtube.com/watch?v=h3bkKqFtaO4 |

AppStyle has four values: modern, naturePaper, softNeumorphism, monochromeWorkspace. themeMode is light, dark, or system. Both persist locally and on profiles.preferred_style / preferred_theme. main.dart overrides the controllers so a profile load applies the saved look. themeAnimationDuration is zero.

Nature Paper and Soft Neumorphism add backdrop widgets in MaterialApp.builder alongside ResponsiveBreakpoints. Tokens live in app_style_tokens.dart and per-style color files. Read AppTheme.light(style) / dark(style).

**TASK — 19.1 Toggle every combination**
On AppProfileScreen, walk 4 styles × 2 brightness modes (ignore system or set it). Confirm Nature Paper shows paper décor and Soft Neumorphism shows its backdrop. Restart — prefs survive. Change on device A, confirm profiles row updated, then login on device B (optional) to see server-followed style.

**CHECKPOINT — Independent verification**
preferred_style in SQL equals the enum name (naturePaper camelCase). If you stored nature_paper, tryParse fails and the UI falls back.

## Common mistakes

**WARNING — Animating theme between incompatible palettes**
The repo disables animation on purpose.

## What you just learned

- Four styles, themeMode, profile persistence.

## Self-check quiz (cover the answers)

**Q1.** Where are backdrops wrapped?

**Answer 1**
MuStudyApp builder → NaturePaperBackdrop → SoftNeumorphismBackdrop.

# Tutorial 20 — Notifications (what MU Study actually ships)

| Field | Value |
|---|---|
| Tutorial | 20 — Notifications |
| Difficulty | ★★★★★ hard conceptually |
| Time | 60 minutes |
| Prerequisites | Tutorials 13–14, 17 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

### Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Real-time — Official | https://www.youtube.com/watch?v=lsKDFTL1YbQ |
| Firebase Cloud Messaging — Flutter | https://www.youtube.com/watch?v=aIRVqk9oE94 |
| Push Notifications in Flutter — Fireship | https://www.youtube.com/watch?v=7AtVo2H3g1s |

> **WARNING — Firebase Cloud Messaging is not in this repository**
> pubspec.yaml has no firebase_messaging. There is no FCM token column. Tutorial prompts that ask you to “send a test push from the Supabase dashboard” describe a different product. This tutorial teaches the real inbox, then sketches how FCM could be added later without pretending it already works.

## The real pipeline

**In-app notifications**

```
Teacher creates assignment
  → trg_assignment_notify / notify_user()
  → INSERT notifications (user_id, title, body, kind, action_path, ref_id)
  → Realtime event
  → notificationsProvider reloads
  → unreadNotificationsCountProvider updates shell badge
  → Student taps row → optional action_path via go_router
```

Similar triggers exist for announcements, submissions, marks, and chat messages (fn_trg_*.sql, fn_notify_user.sql, fn_notify_chat_message.sql). kind distinguishes system vs chat vs homework so the UI can icon and filter.

## If you add FCM later (design only)

You would store a device token per profile, send from an Edge Function on notification INSERT, and handle onMessageOpenedApp to context.go(action_path). That is a new project with APNs/FCM keys. Do not block reconstruction of MU Study on store push certificates.

> **TASK — 20.1 Prove in-app notifications**
> As teacher, create an assignment for a group that has a student. As student, watch the inbox and badge update without refresh. Mark as read; unread count drops. Read fn_trg_assignment_notify.sql and write the INSERT column list in your notes.

**CHECKPOINT — Independent verification**
Table Editor shows a notifications row with the student profile id. The student's phone showed it live. No Firebase console was involved. That is a pass.

## Common mistakes

**WARNING — Installing firebase_messaging "because the prompt said so"**
Out of scope unless you explicitly start a push project.

## What you just learned

- Inbox table + triggers + Realtime. FCM is optional future work.

## Self-check quiz (cover the answers)

**Q1.** Who inserts homework notifications?

**Answer 1**
A trigger after assignment insert, via notify helpers.

# Tier 6 — Complete features

You assemble end-to-end product flows. Each tutorial names the real screens, providers, tables, and RPCs.

# Tutorial 21 — Authentication feature (complete)

| Field | Value |
|---|---|
| Tutorial | 21 — Complete auth |
| Difficulty | ★★★★ medium |
| Time | 90 minutes |
| Prerequisites | Tutorials 10–15 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Supabase Auth — Official | https://www.youtube.com/watch?v=6ow_jW7iS24 |
| Flutter Authentication — Fireship | https://www.youtube.com/watch?v=yH3NQFr3k0s |

Assemble SplashScreen, SetupRequiredScreen, LoginScreen, LanguageSelectionScreen, disabled account, AuthRepository, ProfileRepository, router redirect, cache fallback. Session persistence is supabase_flutter's job (local storage). Profile JSON is also cached in CacheService for offline display.

**Files to implement in order**

```
lib/core/constants/env.dart
lib/supabase/supabase_client.dart
lib/models/enums.dart
lib/models/profile.dart
lib/providers/auth/auth_provider.dart
lib/core/router/app_router.dart
lib/screens/auth/*.dart
lib/main.dart
```

> **TASK — 21.1 Full loop**
> Cold start → splash → login → language → admin home → logout → login as student → student home. Kill the process, relaunch: session restores, language not asked again. Change password as a user who has current password (profile settings). Disable a user; they cannot resolve email.

> **CHECKPOINT — Independent verification**
> Three accounts, three homes. SharedPreferences still has supabase session after relaunch (or you would see login). SQL is_disabled true blocks RPC.

## Common mistakes

> **WARNING — Skipping /language**
> Users with null preferred_language must not see role chrome.

## What you just learned

- The production auth loop, not a toy email form.

## Self-check quiz (cover the answers)

**Q1.** What does signOut clear besides Auth?

> **Answer 1**
> App caches via clearAllAppCaches.

# Tutorial 22 — Assignments feature (complete)

| Field | Value |
|---|---|
| Tutorial | 22 — Assignments |
| Difficulty | ★★★★★ hard |
| Time | 120 minutes |
| Prerequisites | Tutorials 10–16 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

Teacher: TeacherAssignmentsScreen creates static or homework_pack rows for the active group. Packs open TeacherHomeworkPackScreen to add TaskItems (sort_order). Student: StudentAssignmentsScreen lists work. Packs open tasks; StudentTaskSubmitScreen upserts submissions (text, file_url, voice_url). TeacherSubmissionsScreen reviews. Marks may sync via fn_sync_submission_mark. trg_assignment_notify fans out inbox rows.

| Actor | Screen | Table/RPC |
|---|---|---|
| Teacher | TeacherAssignmentsScreen | assignments insert |
| Teacher | TeacherHomeworkPackScreen | tasks |
| Student | StudentTaskSubmitScreen | submissions + storage |
| Teacher | TeacherSubmissionsScreen | submissions update |
| Either | marks screens | marks / views |

**TASK — 22.1 End-to-end pack**
Teacher creates homework_pack “Fractions” with two tasks (one with a link). Student submits text on task 1 and a file on task 2. Teacher scores task 1. Student marks screen updates. Notification exists for the assignment create.

**CHECKPOINT — Independent verification**
UNIQUE (task_id, student_id) prevents a second submission row — upsert or error. SQL shows two tasks, two submissions, one mark. Classmate cannot open the file.

## Common mistakes

**WARNING — Treating static assignments as submit-able**
Product: static is display-only.

## What you just learned

- Pack vs static, task order, submission uniqueness, notify trigger.

## Self-check quiz (cover the answers)

**Q1.** Which enum value is stored for a pack?

**Answer 1**
homework_pack

# Tutorial 23 — Chat feature (complete)

| Field | Value |
|---|---|
| Tutorial | 23 — Chat |
| Difficulty | ★★★★★ hard |
| Time | 120 minutes |
| Prerequisites | Tutorials 10–17 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

Chat is group-scoped, not a free DM roster. ensure_chat_room_for_group creates chat_rooms when a group is inserted. ChatListScreen reads chat_room_summaries + chat_unread_counts. ChatRoomScreen sends text/voice/image/video/file, replies (same room check), reactions (toggle_message_reaction), and marks reads. Soft delete sets is_deleted. Content check constraint: text needs text_content; media types need media_url.

There is no separate 1-1 student–teacher table. A “direct” feel happens because a small group still shares one room. Typing indicators are not a first-class table in the recovered schema — if you add them, use a Realtime broadcast channel, not a row per keystroke.

**TASK — 23.1 Group chat fidelity**
Two members send text. One sends an image (chat-images path). One replies. Both toggle a reaction. Unread badge on the other user drops after they open the room. Delete a message — others see deleted state, not the old text.

**CHECKPOINT — Independent verification**
A third user not in the group cannot SELECT chat_messages for that room_id (RLS). Storage 403 if they guess the path. member_count = students + 1.

## Common mistakes

**WARNING — Creating chat_rooms in Flutter instead of the trigger**
You will get duplicates or miss rooms.

## What you just learned

- One room per group, media paths, unread RPC, reactions RPC.

## Self-check quiz (cover the answers)

**Q1.** What view feeds the chat list?

**Answer 1**
chat_room_summaries

# Tutorial 24 — Attendance feature (complete)

| Field | Value |
|---|---|
| Tutorial | 24 — Attendance |
| Difficulty | ★★★★ medium |
| Time | 90 minutes |
| Prerequisites | Tutorials 10–15 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

Teachers open TeacherAttendanceScreen, pick a group and date. ensure_lesson_session(group, date) returns/creates lesson_sessions. Attendance rows upsert status: present, absent, late, excused. Unique (group, student, date) and unique (session, student). Students see a calendar (AttendanceCalendarGrid). Analytics charts aggregate percentages. There is no parent-push integration; a parent-facing notification would be a new notifications.kind plus a product decision.

**TASK — 24.1 Roll call**
Mark a class mixed present/absent/late. Change one student to excused. SQL: one lesson_sessions row, N attendance rows. Student calendar matches. Teacher analytics shows a sane percentage.

**CHECKPOINT — Independent verification**
Re-opening the same date does not create a second session (UNIQUE group_id, session_date). A teacher of another group cannot write this group's attendance (RLS).

## Common mistakes

**WARNING — Inserting attendance without a session**
session_id is NOT NULL.

## What you just learned

- Session + status enum + student calendar.

## Self-check quiz (cover the answers)

**Q1.** Which RPC starts the day?

**Answer 1**
ensure_lesson_session

# Tutorial 25 — Announcements feature (complete)

| Field | Value |
|---|---|
| Tutorial | 25 — Announcements |
| Difficulty | ★★★ easy |
| Time | 60 minutes |
| Prerequisites | Tutorials 10–16 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

announcements belong to a group. teacher_id is nullable so an admin school-wide post can omit a teacher. image_urls and file_urls are jsonb arrays. TeacherAnnouncementsScreen and AdminAnnouncementsScreen insert rows and upload attachments to the announcements bucket. trg_announcement_notify inserts inbox rows. Student dashboard ExpandableAnnouncementCard reads announcementsProvider.

There is no announcements_reads table. “Read” is approximated by opening the card or by the related notification.is_read. If you need true per-student read receipts for announcements, that is a new table — do not overload message_reads.

**TASK — 25.1 Post and receive**
Teacher posts a title + description + one image. Students in that group see it on the dashboard and get a notification. Students in another group do not. Admin can post to a selected group from the admin announcements screen.

**CHECKPOINT — Independent verification**
SQL: announcements.group_id matches. notifications.kind/action_path point at the announcement. Storage object is not world-listable. Out-of-group student SELECT returns zero rows for that id.

## Common mistakes

**WARNING — Putting school-wide news in a random student’s group**
Admins still pick a group_id; plan which group is “all school” or post per group.

## What you just learned

- Group-scoped posts, jsonb URLs, notify trigger.

## Self-check quiz (cover the answers)

**Q1.** Can teacher_id be null?

**Answer 1**
Yes — admin posts.

# Tier 7 — Going deep

Reusable widgets, tests, performance, errors, and shipping a release artifact.

# Tutorial 26 — Custom widgets and components

| Field | Value |
|---|---|
| Tutorial | 26 — Custom widgets |
| Difficulty | ★★★★ medium |
| Time | 60 minutes |
| Prerequisites | Tutorial 7 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

lib/components/ is the widget library: AppShell, ShellNavBar, PremiumCard, EmptyState, PrimaryButton, GroupSelector, AttachmentPreview, VoiceNoteField, MarkAverageCard, AttendanceCalendarGrid, chat bubbles, PDF viewer sheet, analytics charts. Screens should compose these instead of restyling ListTile fifty times.

Good components take data + callbacks (Assignment-like cards take onTap). They do not call supabase themselves. If a card fetches, you cannot reuse it in a test without a fake backend.

**TASK — 26.1 Inventory and reuse**
List ten components and the screens that use them. Build a practice StudentProfileCard in a scratch file that only takes Profile. Then compare to ProfileHeroCard. Note what you missed (style tokens, l10n, avatar cache).

**CHECKPOINT — Independent verification**
Your practice card has no supabase import. flutter analyze on the scratch file is clean. You can point to AppShell as the reason student/teacher/admin feel like one product.

## Common mistakes

**WARNING — Copy-pasting MessageBubble into three role folders**
Chat is already shared.

## What you just learned

- Composition, props vs fetch, shared shell.

## Self-check quiz (cover the answers)

**Q1.** Where does bottom nav live?

**Answer 1**
lib/components/layouts/ — AppShell + ShellNavBar.

# Tutorial 27 — Testing

| Field | Value |
|---|---|
| Tutorial | 27 — Testing |
| Difficulty | ★★★★★ hard |
| Time | 90 minutes |
| Prerequisites | Tutorials 6 and 10 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Testing — Official | https://www.youtube.com/watch?v=X8uCMbpnb5s |

This repo already has test/widget_test.dart (SetupRequiredScreen), test/emoji_message_test.dart, and test/shell_layout_test.dart. They run without Supabase. That is the correct default: unit/widget tests mock or avoid the network. Integration tests that log in to a live project belong in a separate CI job with secrets.

**test/widget_test.dart (real)**

```
testWidgets('Setup screen shows configuration hint', (tester) async {
  await tester.pumpWidget(const SetupRequiredScreen());
  expect(find.textContaining('Supabase is not configured'), findsOneWidget);
});
```

For models, test assignmentTypeFromString('homework_pack') and fromJson with a fixture map. For AuthRepository, inject a fake supabase is hard — prefer testing pure functions (email normalization, error mapping) and a thin mock around rpc. Do not hit production with widget tests.

**TASK — 27.1 Write tests that belong in this repo**
1) Unit: assignmentTypeToDb / fromString round-trip.
2) Unit: Profile.fromJson missing full_name becomes ''.
3) Widget: SetupRequiredScreen (already exists — run it).
4) Widget: a dummy login form you built in Tutorial 7 (optional).
5) Manual integration checklist: login → dashboard → create assignment (teacher).
Run: flutter test

**CHECKPOINT — Independent verification**
flutter test exits 0. A new unit test fails when you break homework_pack mapping. CI should run flutter test on every PR.

## Common mistakes

**WARNING — Widget test that calls real signInWithPassword**
Flaky, slow, pollutes Auth users.

## What you just learned

- Test without network first. Manual checklist for RLS.

## Self-check quiz (cover the answers)

**Q1.** Why is SetupRequiredScreen easy to test?

> **Answer 1**
> No Supabase init; it is a static widget.

# Tutorial 28 — Performance and optimization

| Field | Value |
|---|---|
| Tutorial | 28 — Performance |
| Difficulty | ★★★★★ hard |
| Time | 60 minutes |
| Prerequisites | Tutorials 10 and 14 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Performance — Official | https://www.youtube.com/watch?v=pk05Vmtfl5g |

Riverpod already caches FutureProvider results until invalidate. Lists page (PageResult, LoadMoreButton). Images use cached_network_image and chat_media_cache_service. Selects should name columns when payloads are large (chat list uses a view instead of N+1 queries). Focus and theme reads hit SharedPreferences, not Postgres.

**Habits**

```
- Filter by group_id / user_id in the query, not in Dart after download.
- limit(50) on notifications (already).
- Do not watch a high-churn stream in AppShell if only a badge integer is needed
  (unreadNotificationsCountProvider derives from the list).
- Chat video thumbnails are computed once (chat_video_thumbnail_service).
```

> **TASK — 28.1 Measure one list**
> Open teacher gradebook for a group with many marks. Note the select and whether it pages. If you add a student directory, implement page size 20 and a LoadMoreButton. Confirm DevTools rebuild counts do not rebuild the entire shell on each mark edit.

> **CHECKPOINT — Independent verification**
> Network tab (or Supabase logs) shows bounded payloads. Opening chat does not re-download all academic years. That is a pass.

## Common mistakes

> **WARNING — select() * on chat_messages without room filter**
> You will ship the whole school.

## What you just learned

- Cache, page, filter in SQL, isolate rebuilds.

## Self-check quiz (cover the answers)

**Q1.** Where is offline profile stored?

**Answer 1**
CacheService / SharedPreferences via app_cache.

# Tutorial 29 — Error handling and user feedback

| Field | Value |
|---|---|
| Tutorial | 29 — Errors & feedback |
| Difficulty | ★★★★ medium |
| Time | 50 minutes |
| Prerequisites | Tutorials 10 and 14 |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

LoginScreen maps AuthException → loginIncorrect, PostgrestException with jwt/key → loginConfigError, other → loginFailed (debug shows the raw error). ProfileRepository catches fetch errors and falls back to cachedProfile. Lists should use AsyncValue.when with a retry that invalidates the provider. Skeletonizer/shimmer packages are in pubspec for loading chrome.

**Pattern**

```
asyncValue.when(
  loading: () => const Skeletonizer(child: PlaceholderList()),
  error: (e, _) => EmptyState(
    title: context.l10n.genericError,
    action: () => ref.invalidate(assignmentsProvider),
  ),
  data: (items) => items.isEmpty ? EmptyState(...) : ListView(...),
);
```

**TASK — 29.1 Break the network on purpose**
Airplane mode on a logged-in student. Open marks — cached or friendly error, not a red screen of death. Turn network on, tap retry. Wrong login password — no exception dump in release. Invalid anon key — SetupRequired or loginConfigError, not a hang.

**CHECKPOINT — Independent verification**
A non-developer testers' notes: they always saw a sentence in their language and a next action (retry, contact admin). If they saw PostgrestException, you leaked internals.

## Common mistakes

**WARNING — catch (_) {}**
You hide RLS bugs. Log in debug, show l10n in release.

## What you just learned

- Typed errors, cache fallback, retry, empty vs error.

## Self-check quiz (cover the answers)

**Q1.** What does LoginScreen do after a successful sign-in?

**Answer 1**
invalidate currentProfileProvider and context.go('/boot').

# Tutorial 30 — Deployment to app stores and web

| Field | Value |
|---|---|
| Tutorial | 30 — Deployment |
| Difficulty | ★★★★★ hard |
| Time | 120 minutes |
| Prerequisites | All previous tutorials |
| Verify yourself | Every TASK has an independent CHECKPOINT. Do not skip it. |

## Video resources

Watch at least one video before the task. Videos teach the general technology; this book teaches how MU Study uses that technology. Pause the video and map each concept onto a file in this repository.

| Resource | URL |
|---|---|
| Flutter Release Builds — Official | https://www.youtube.com/watch?v=Yx1kSisJo4k |

Makefile apk target: flutter build apk --release then copies to mu_study.apk. Web: flutter build web (and make ch for Chrome debug). iOS: flutter build ipa requires Apple developer membership, signing, and bundle com.example.muStudy — change the bundle before a real store listing.

Store listing is an organizational process (screenshots, privacy policy, age ratings, data-safety forms). This book cannot create your Play Console or App Store Connect accounts. It can demand hygiene: no service_role, production Supabase project, Auth site URL allow-list, Syncfusion license if required, real passwords instead of password123, microphone/photo purpose strings that match what you actually record.

**Release commands**

```
flutter test
flutter build apk --release --dart-define-from-file=.env.production
flutter build web --release --dart-define-from-file=.env.production
flutter build ipa --release --dart-define-from-file=.env.production
```

**TASK — 30.1 Produce artifacts (local)**
1) flutter test passes.
2) flutter build apk --release succeeds; install on a phone and log in against YOUR project.
3) flutter build web succeeds; serve build/web and log in.
4) Write a one-page launch checklist: rotate demo passwords, confirm RLS with two students, confirm Storage 403, confirm Auth redirect URLs.
Uploading to stores is optional and account-specific — do not paste store keys here.

**CHECKPOINT — Independent verification**
The APK/web build logs in without SetupRequiredScreen. strings.xml / Info.plist usage descriptions exist. .env.production is not in git. A second student still cannot see the first student's marks on the release build — RLS does not care about Debug.

## Common mistakes

**WARNING — Shipping the development project URL to the world with password123**
Read the README warning again.

**WARNING — Debug signing as a Play Store AAB**
Play requires a real upload keystore.

## What you just learned

- Release builds, secrets, store process vs engineering artifacts.

## Self-check quiz (cover the answers)

**Q1.** What Makefile target copies the APK?

**Answer 1**
make apk

**Q2.** Why dart-define-from-file on release?

**Answer 2**
Inject URL/key without relying on a bundled .env you might forget to strip.

**You finished the 30 tutorials**
The source atlas and SQL atlas after this page are the reconstruction encyclopedia. Use them when a lesson said “open file X” and you want the entire listing. Score yourself with Appendix K quizzes and the feature checkpoints — not with page count.

# Source atlas — every application Dart file

This atlas is the reconstruction reference. For each file: why it exists, how to read it, the complete listing, and five questions you can answer without running the app. Generated localization files are omitted (run flutter gen-l10n instead).

File count in this atlas: **155**.

## lib/components/analytics/teacher_analytics_charts.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 377. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';
- import 'package:syncfusion_flutter_charts/charts.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/analytics/teacher_analytics_charts.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:syncfusion_flutter_charts/charts.dart';

class AnalyticsChartPoint {
  const AnalyticsChartPoint(this.label, this.value, this.color);
  final String label;
  final double value;
  final Color color;
}

class MarkDistributionDoughnut extends StatelessWidget {
  const MarkDistributionDoughnut({super.key, required this.points});

  final List<AnalyticsChartPoint> points;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final scheme = Theme.of(context).colorScheme;
    final total = points.fold<double>(0, (sum, p) => sum + p.value).round();
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final labelsInside =
        AppStyleTokens.maybeOf(context)?.prefersChartLabelsInside ?? false;

    return Column(
      children: [
        SizedBox(
          height: 280,
          child: SfCircularChart(
            margin: EdgeInsets.zero,
            annotations: <CircularChartAnnotation>[
              CircularChartAnnotation(
                widget: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
```

```
                    l10n.analyticsTotalMarks(total),
                    textAlign: TextAlign.center,
                    style: TextStyle(
                      color: scheme.onSurface,
                      fontWeight: FontWeight.w700,
                      fontSize: 14,
                      height: 1.25,
                    ),
                  ),
                ],
              ),
            ),
          ],
          series: <CircularSeries<AnalyticsChartPoint, String>>[
            DoughnutSeries<AnalyticsChartPoint, String>(
              dataSource: points,
              xValueMapper: (d, _) => d.label,
              yValueMapper: (d, _) => d.value,
              pointColorMapper: (d, _) => d.color,
              innerRadius: '62%',
              radius: '78%',
              strokeWidth: isDark ? 2 : 1,
              strokeColor: isDark ? const Color(0xFF1A1A1A) : scheme.surface,
              animationDuration: 1100,
              animationDelay: 80,
              dataLabelMapper: (d, _) {
                final pct = total == 0
                    ? 0
                    : ((d.value / total) * 100).round();
                return '${d.value.round()}\n$pct%';
              },
              dataLabelSettings: DataLabelSettings(
                isVisible: true,
                labelPosition: labelsInside
                    ? ChartDataLabelPosition.inside
                    : ChartDataLabelPosition.outside,
                connectorLineSettings: ConnectorLineSettings(
                  type: ConnectorType.curve,
                  length: '12%',
                  width: labelsInside ? 0 : 1.2,
                  color: scheme.onSurface.withValues(alpha: 0.35),
                ),
                textStyle: TextStyle(
                  color: labelsInside ? Colors.white : scheme.onSurface,
                  fontSize: 11,
                  fontWeight: labelsInside
                      ? FontWeight.w700
                      : FontWeight.w600,
                  height: 1.2,
                ),
              ),
            ),
          ],
        ),
      ),
      const SizedBox(height: 8),
      Wrap(
        spacing: 8,
        runSpacing: 8,
        alignment: WrapAlignment.center,
        children: [
          for (final p in points)
            _LegendPill(
              color: p.color,
              label: '${p.label}: ${p.value.round()}',
            ),
        ],
      ),
    ],
  );
 }
}
```

```
class AttendanceTrendBars extends StatelessWidget {
  const AttendanceTrendBars({super.key, required this.trend});

  final AttendanceWeekTrend trend;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final tokens = AppStyleTokens.maybeOf(context);
    final isMono = tokens?.isMonochromeWorkspace ?? false;
    final barRadius = isMono
        ? const BorderRadius.vertical(top: Radius.circular(2))
        : const BorderRadius.vertical(top: Radius.circular(6));
    final labelOnBar = isMono && !isDark
        ? AppColors.ink(context)
        : Colors.white;
    final categories = <_WeekCategory>[
      _WeekCategory(
        l10n.analyticsThisWeek,
        trend.thisWeek.present.toDouble(),
        trend.thisWeek.absent.toDouble(),
        trend.thisWeek.late.toDouble(),
      ),
      _WeekCategory(
        l10n.analyticsLastWeek,
        trend.lastWeek.present.toDouble(),
        trend.lastWeek.absent.toDouble(),
        trend.lastWeek.late.toDouble(),
      ),
    ];
    final maxY = [
      trend.thisWeek.max,
      trend.lastWeek.max,
      1,
    ].reduce((a, b) => a > b ? a : b);

    return Column(
      children: [
        SizedBox(
          height: 260,
          child: SfCartesianChart(
            margin: const EdgeInsets.only(top: 8, right: 8),
            plotAreaBorderWidth: 0,
            primaryXAxis: CategoryAxis(
              majorGridLines: const MajorGridLines(width: 0),
              axisLine: const AxisLine(width: 0),
              majorTickLines: const MajorTickLines(size: 0),
              labelStyle: TextStyle(
                color: scheme.onSurface.withValues(alpha: 0.75),
                fontWeight: FontWeight.w600,
                fontSize: 12,
              ),
            ),
            primaryYAxis: NumericAxis(
              minimum: 0,
              maximum: (maxY + 1).toDouble(),
              interval: maxY <= 5 ? 1 : null,
              majorGridLines: MajorGridLines(
                width: 1,
                color: scheme.onSurface.withValues(alpha: isDark ? 0.12 : 0.08),
              ),
              axisLine: const AxisLine(width: 0),
              majorTickLines: const MajorTickLines(size: 0),
              labelStyle: TextStyle(
                color: scheme.onSurface.withValues(alpha: 0.45),
                fontSize: 11,
              ),
            ),
            legend: Legend(
              isVisible: true,
              position: LegendPosition.bottom,
```

```
      overflowMode: LegendItemOverflowMode.wrap,
      iconHeight: 10,
      iconWidth: 10,
      padding: 0,
      itemPadding: 12,
      textStyle: TextStyle(
        color: scheme.onSurface.withValues(alpha: 0.8),
        fontSize: 12,
      ),
    ),
    series: <CartesianSeries<_WeekCategory, String>>[
      ColumnSeries<_WeekCategory, String>(
        name: l10n.attendancePresentWord,
        dataSource: categories,
        xValueMapper: (d, _) => d.label,
        yValueMapper: (d, _) => d.present,
        dataLabelMapper: (d, _) =>
            d.present == 0 ? '' : '${d.present.round()}',
        color: AppColors.success(context),
        gradient: isMono ? null : tokens?.chartPositive,
        borderRadius: barRadius,
        width: isMono ? 0.55 : 0.7,
        spacing: 0.18,
        animationDuration: 1000,
        animationDelay: 60,
        dataLabelSettings: DataLabelSettings(
          isVisible: true,
          labelAlignment: ChartDataLabelAlignment.middle,
          textStyle: TextStyle(
            color: isMono && !isDark ? Colors.white : labelOnBar,
            fontSize: 11,
            fontWeight: FontWeight.w700,
          ),
        ),
      ),
      ColumnSeries<_WeekCategory, String>(
        name: l10n.attendanceAbsentWord,
        dataSource: categories,
        xValueMapper: (d, _) => d.label,
        yValueMapper: (d, _) => d.absent,
        dataLabelMapper: (d, _) =>
            d.absent == 0 ? '' : '${d.absent.round()}',
        color: AppColors.danger(context),
        gradient: isMono ? null : tokens?.chartDanger,
        borderRadius: barRadius,
        width: isMono ? 0.55 : 0.7,
        spacing: 0.18,
        animationDuration: 1100,
        animationDelay: 160,
        dataLabelSettings: DataLabelSettings(
          isVisible: true,
          labelAlignment: ChartDataLabelAlignment.middle,
          textStyle: TextStyle(
            color: isMono && !isDark ? Colors.white : labelOnBar,
            fontSize: 11,
            fontWeight: FontWeight.w700,
          ),
        ),
      ),
      ColumnSeries<_WeekCategory, String>(
        name: l10n.attendanceLateWord,
        dataSource: categories,
        xValueMapper: (d, _) => d.label,
        yValueMapper: (d, _) => d.late,
        dataLabelMapper: (d, _) =>
            d.late == 0 ? '' : '${d.late.round()}',
        color: AppColors.warning(context),
        gradient: isMono ? null : tokens?.chartWarning,
        borderRadius: barRadius,
        width: isMono ? 0.55 : 0.7,
        spacing: 0.18,
        animationDuration: 1200,
        animationDelay: 260,
```

```
                dataLabelSettings: DataLabelSettings(
                  isVisible: true,
                  labelAlignment: ChartDataLabelAlignment.middle,
                  textStyle: TextStyle(
                    color: isMono && !isDark ? Colors.white : labelOnBar,
                    fontSize: 11,
                    fontWeight: FontWeight.w700,
                  ),
                ),
              ),
            ],
          ),
        ),
      ],
    );
  }
}

List<AnalyticsChartPoint> markBandPoints(
  BuildContext context,
  AppLocalizations l10n,
  List<double> averages,
) {
  var excellent = 0, good = 0, needsWork = 0, atRisk = 0;
  for (final value in averages) {
    if (value >= 85) {
      excellent++;
    } else if (value >= 70) {
      good++;
    } else if (value >= 50) {
      needsWork++;
    } else {
      atRisk++;
    }
  }
  return [
    AnalyticsChartPoint(
      l10n.chartBandExcellent,
      excellent.toDouble(),
      AppColors.success(context),
    ),
    AnalyticsChartPoint(
      l10n.chartBandGood,
      good.toDouble(),
      AppColors.goldAccent(context),
    ),
    AnalyticsChartPoint(
      l10n.chartBandNeedsWork,
      needsWork.toDouble(),
      AppColors.warning(context),
    ),
    AnalyticsChartPoint(
      l10n.chartBandAtRisk,
      atRisk.toDouble(),
      AppColors.danger(context),
    ),
  ].where((p) => p.value > 0).toList();
}

class _WeekCategory {
  const _WeekCategory(this.label, this.present, this.absent, this.late);
  final String label;
  final double present;
  final double absent;
  final double late;
}

class _LegendPill extends StatelessWidget {
  const _LegendPill({required this.color, required this.label});

  final Color color;
  final String label;
```

```
  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
      decoration: BoxDecoration(
        color: isDark
            ? Colors.white.withValues(alpha: 0.06)
            : scheme.onSurface.withValues(alpha: 0.05),
        borderRadius: BorderRadius.circular(isMono ? 4 : 20),
        border: isMono
            ? Border.all(color: scheme.onSurface.withValues(alpha: 0.08))
            : null,
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          Container(
            width: 8,
            height: 8,
            decoration: BoxDecoration(
              color: color,
              shape: BoxShape.circle,
              boxShadow: [
                BoxShadow(color: color.withValues(alpha: 0.55), blurRadius: 6),
              ],
            ),
          ),
          const SizedBox(width: 6),
          Text(
            label,
            style: TextStyle(
              color: scheme.onSurface.withValues(alpha: 0.85),
              fontSize: 12,
              fontWeight: FontWeight.w600,
            ),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/analytics/teacher_analytics_charts.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/attachments/attachment_preview.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 515. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:typed_data';
- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/components/dialogs/pdf_viewer_sheet.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:url_launcher/url_launcher.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/attachments/attachment_preview.dart**

```
import 'dart:typed_data';

import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/components/dialogs/pdf_viewer_sheet.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:url_launcher/url_launcher.dart';

const _imageExtensions = {'jpg', 'jpeg', 'png', 'gif', 'webp', 'heic', 'heif'};

bool isImageAttachment(String? nameOrUrl) {
  final ext = attachmentExtension(nameOrUrl);
  return ext != null && _imageExtensions.contains(ext);
}

bool isPdfAttachment(String? nameOrUrl) {
  return attachmentExtension(nameOrUrl) == 'pdf';
}

String? attachmentExtension(String? nameOrUrl) {
  if (nameOrUrl == null || nameOrUrl.isEmpty) return null;
  final clean = Uri.tryParse(nameOrUrl)?.path ?? nameOrUrl;
  final base = clean.split('/').last.split('?').first;
  final dot = base.lastIndexOf('.');
  if (dot < 0 || dot == base.length - 1) return null;
  return base.substring(dot + 1).toLowerCase();
}

String attachmentFileName(String? nameOrUrl, {String fallback = 'Attachment'}) {
  if (nameOrUrl == null || nameOrUrl.isEmpty) return fallback;
  final clean = Uri.tryParse(nameOrUrl)?.path ?? nameOrUrl;
  final base = clean.split('/').last.split('?').first;
  return base.isEmpty ? fallback : Uri.decodeComponent(base);
}

String? storagePathFromSignedUrl(String url, {String? bucket}) {
  final uri = Uri.tryParse(url);
  if (uri == null) return null;
```

```
  final segments = uri.pathSegments;
  final signIdx = segments.indexOf('sign');
  if (signIdx >= 0 && signIdx + 2 < segments.length) {
    final bucketName = segments[signIdx + 1];
    if (bucket != null && bucketName != bucket) return null;
    return segments.sublist(signIdx + 2).map(Uri.decodeComponent).join('/');
  }
  return null;
}

Future<void> openAttachment(
  BuildContext context, {
  required String url,
  String? fileName,
  Uint8List? bytes,
}) async {
  final name = fileName ?? attachmentFileName(url);

  if (isImageAttachment(name)) {
    await showDialog<void>(
      context: context,
      useRootNavigator: true,
      barrierColor: Colors.black.withValues(alpha: 0.92),
      builder: (ctx) => _ImageLightbox(url: url, bytes: bytes, title: name),
    );
    return;
  }

  if (isPdfAttachment(name)) {
    await openPdfViewer(context, url, title: name);
    return;
  }

  final uri = Uri.tryParse(url);
  if (uri != null && await canLaunchUrl(uri)) {
    await launchUrl(uri, mode: LaunchMode.externalApplication);
  } else if (context.mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(context.l10n.attachmentCouldNotOpen)),
    );
  }
}

class AttachmentPreview extends StatelessWidget {
  const AttachmentPreview({
    super.key,
    required this.url,
    this.fileName,
    this.bytes,
    this.onRemove,
    this.enabled = true,
  });

  final String url;
  final String? fileName;
  final Uint8List? bytes;
  final VoidCallback? onRemove;
  final bool enabled;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final name =
        fileName ?? attachmentFileName(url, fallback: l10n.taskAttachment);
    final isImage = isImageAttachment(name);

    return isImage
        ? _ImagePreviewCard(
            url: url,
            bytes: bytes,
            fileName: name,
            subtitle: l10n.attachmentImage,
            tapPreview: l10n.attachmentTapPreview,
```

```
            onOpen: enabled
                ? () => openAttachment(
                    context,
                    url: url,
                    fileName: name,
                    bytes: bytes,
                  )
                : null,
            onRemove: enabled ? onRemove : null,
          )
        : _FilePreviewCard(
            fileName: name,
            kindLabel: isPdfAttachment(name)
                ? l10n.attachmentPdf
                : l10n.attachmentFile,
            tapOpen: l10n.attachmentTapOpen,
            onOpen: enabled
                ? () => openAttachment(context, url: url, fileName: name)
                : null,
            onRemove: enabled ? onRemove : null,
          );
  }
}

class _ImagePreviewCard extends StatelessWidget {
  const _ImagePreviewCard({
    required this.url,
    required this.fileName,
    required this.subtitle,
    required this.tapPreview,
    this.bytes,
    this.onOpen,
    this.onRemove,
  });

  final String url;
  final String fileName;
  final String subtitle;
  final String tapPreview;
  final Uint8List? bytes;
  final VoidCallback? onOpen;
  final VoidCallback? onRemove;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onOpen,
        borderRadius: BorderRadius.circular(16),
        child: Ink(
          decoration: BoxDecoration(
            color: AppColors.controlSurface(context),
            borderRadius: BorderRadius.circular(16),
            border: Border.all(color: AppColors.inkAlpha(context, 0.06)),
            boxShadow: [
              BoxShadow(
                color: AppColors.inkAlpha(context, 0.06),
                blurRadius: 16,
                offset: const Offset(0, 6),
              ),
            ],
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              ClipRRect(
                borderRadius: const BorderRadius.vertical(
                  top: Radius.circular(16),
                ),
                child: AspectRatio(
                  aspectRatio: 16 / 10,
                  child: Stack(
```

```
fit: StackFit.expand,
children: [
  ColoredBox(
    color: AppColors.softWash(context, lightAlpha: 0.5),
    child: bytes != null
        ? Image.memory(bytes!, fit: BoxFit.cover)
        : CachedNetworkImage(
            imageUrl: url,
            fit: BoxFit.cover,
            placeholder: (_, _) => Center(
              child: SizedBox(
                width: 24,
                height: 24,
                child: CircularProgressIndicator(
                  strokeWidth: 2,
                ),
              ),
            ),
            errorWidget: (_, _, _) => Center(
              child: Icon(
                Icons.broken_image_outlined,
                color: AppColors.accent(context),
                size: 32,
              ),
            ),
          ),
  ),
  Positioned(
    left: 0,
    right: 0,
    bottom: 0,
    child: DecoratedBox(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [
            Colors.transparent,
            AppColors.inkAlpha(context, 0.45),
          ],
        ),
      ),
      child: const SizedBox(height: 48),
    ),
  ),
  Positioned(
    left: 12,
    bottom: 10,
    child: Row(
      children: [
        Icon(
          Icons.zoom_in_rounded,
          size: 16,
          color: Colors.white.withValues(alpha: 0.95),
        ),
        const SizedBox(width: 6),
        Text(
          tapPreview,
          style: TextStyle(
            color: Colors.white.withValues(alpha: 0.95),
            fontSize: 12,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  ),
  if (onRemove != null)
    Positioned(
      top: 8,
      right: 8,
      child: _RemoveButton(onPressed: onRemove!),
    ),
```

```
                    ],
                  ),
                ),
              ),
              Padding(
                padding: const EdgeInsets.fromLTRB(14, 12, 14, 14),
                child: Row(
                  children: [
                    Container(
                      width: 36,
                      height: 36,
                      decoration: BoxDecoration(
                        color: AppColors.mutedGold.withValues(alpha: 0.15),
                        borderRadius: BorderRadius.circular(10),
                      ),
                      child: const Icon(
                        Icons.image_outlined,
                        size: 18,
                        color: AppColors.mutedGold,
                      ),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            fileName,
                            maxLines: 1,
                            overflow: TextOverflow.ellipsis,
                            style: const TextStyle(
                              fontWeight: FontWeight.w700,
                              fontSize: 14,
                            ),
                          ),
                          const SizedBox(height: 2),
                          Text(
                            subtitle,
                            style: TextStyle(
                              fontSize: 12,
                              color: AppColors.inkAlpha(context, 0.45),
                            ),
                          ),
                        ],
                      ),
                    ),
                    Icon(
                      Icons.chevron_right_rounded,
                      color: AppColors.inkAlpha(context, 0.3),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _FilePreviewCard extends StatelessWidget {
  const _FilePreviewCard({
    required this.fileName,
    required this.kindLabel,
    required this.tapOpen,
    this.onOpen,
    this.onRemove,
  });

  final String fileName;
  final String kindLabel;
  final String tapOpen;
```

```
final VoidCallback? onOpen;
final VoidCallback? onRemove;

@override
Widget build(BuildContext context) {
  final isPdf = isPdfAttachment(fileName);
  final accent = isPdf ? AppColors.coffeeBrown : AppColors.softOlive;
  final icon = isPdf
      ? Icons.picture_as_pdf_rounded
      : Icons.insert_drive_file_outlined;

  return Material(
    color: Colors.transparent,
    child: InkWell(
      onTap: onOpen,
      borderRadius: BorderRadius.circular(16),
      child: Ink(
        padding: const EdgeInsets.all(14),
        decoration: BoxDecoration(
          color: AppColors.controlSurface(context),
          borderRadius: BorderRadius.circular(16),
          border: Border.all(color: AppColors.inkAlpha(context, 0.06)),
          boxShadow: [
            BoxShadow(
              color: AppColors.inkAlpha(context, 0.06),
              blurRadius: 16,
              offset: const Offset(0, 6),
            ),
          ],
        ),
        child: Row(
          children: [
            Container(
              width: 52,
              height: 52,
              decoration: BoxDecoration(
                color: accent.withValues(alpha: 0.12),
                borderRadius: BorderRadius.circular(14),
              ),
              child: Icon(icon, color: accent, size: 26),
            ),
            const SizedBox(width: 14),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    fileName,
                    maxLines: 2,
                    overflow: TextOverflow.ellipsis,
                    style: const TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 14,
                      height: 1.25,
                    ),
                  ),
                  const SizedBox(height: 4),
                  Text(
                    kindLabel,
                    style: TextStyle(
                      fontSize: 12,
                      color: AppColors.inkAlpha(context, 0.45),
                    ),
                  ),
                  const SizedBox(height: 2),
                  Text(
                    tapOpen,
                    style: TextStyle(
                      fontSize: 11,
                      fontWeight: FontWeight.w600,
                      color: accent.withValues(alpha: 0.9),
                    ),
                  ),
```

```
                  ],
                ),
              ),
              if (onRemove != null) ...[
                const SizedBox(width: 4),
```

**lib/components/attachments/attachment_preview.dart — continued (2/2)**

```
                _RemoveButton(onPressed: onRemove!),
              ] else
                Icon(
                  Icons.chevron_right_rounded,
                  color: AppColors.inkAlpha(context, 0.3),
                ),
            ],
          ),
        ),
      ),
    );
  }
}

class _RemoveButton extends StatelessWidget {
  const _RemoveButton({required this.onPressed});

  final VoidCallback onPressed;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.white.withValues(alpha: 0.95),
      shape: const CircleBorder(),
      elevation: 1,
      shadowColor: AppColors.inkAlpha(context, 0.2),
      child: InkWell(
        customBorder: const CircleBorder(),
        onTap: onPressed,
        child: const Padding(
          padding: EdgeInsets.all(8),
          child: Icon(
            Icons.delete_outline_rounded,
            size: 18,
            color: AppColors.errorRed,
          ),
        ),
      ),
    );
  }
}

class _ImageLightbox extends StatelessWidget {
  const _ImageLightbox({required this.url, required this.title, this.bytes});

  final String url;
  final String title;
  final Uint8List? bytes;

  @override
  Widget build(BuildContext context) {
    return Dialog.fullscreen(
      backgroundColor: Colors.black,
      child: SafeArea(
        child: Stack(
          children: [
            Center(
              child: InteractiveViewer(
                minScale: 0.8,
                maxScale: 4,
                child: bytes != null
                    ? Image.memory(bytes!)
                    : CachedNetworkImage(
                        imageUrl: url,
                        fit: BoxFit.contain,
```

```
                        placeholder: (_, _) => const CircularProgressIndicator(
                          color: Colors.white,
                        ),
                        errorWidget: (_, _, _) => const Icon(
                          Icons.broken_image_outlined,
                          color: Colors.white70,
                          size: 48,
                        ),
                      ),
                    ),
                  ),
                ),
                Positioned(
                  top: 4,
                  left: 8,
                  right: 8,
                  child: Row(
                    children: [
                      Expanded(
                        child: Text(
                          title,
                          maxLines: 1,
                          overflow: TextOverflow.ellipsis,
                          style: const TextStyle(
                            color: Colors.white,
                            fontWeight: FontWeight.w600,
                          ),
                        ),
                      ),
                      IconButton(
                        onPressed: () => Navigator.of(context).pop(),
                        icon: const Icon(Icons.close_rounded, color: Colors.white),
                      ),
                    ],
                  ),
                ),
              ],
            ),
          ),
        ),
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/attachments/attachment_preview.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/attachments/voice_note_field.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 615. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:cross_file/cross_file.dart';
- import 'package:flutter/foundation.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/utils/voice_playback.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:path/path.dart' as p;
- import 'package:path_provider/path_provider.dart';
- import 'package:record/record.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/attachments/voice_note_field.dart**

```
import 'dart:async';

import 'package:cross_file/cross_file.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/utils/voice_playback.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:record/record.dart';

class VoiceNoteField extends StatefulWidget {
  const VoiceNoteField({
    super.key,
    this.voiceUrl,
    this.enabled = true,
    required this.onUpload,
    this.onRemove,
  });

  final String? voiceUrl;
  final bool enabled;
  final Future<void> Function(Uint8List bytes, String fileName) onUpload;
  final VoidCallback? onRemove;

  @override
  State<VoiceNoteField> createState() => _VoiceNoteFieldState();
}

class _VoiceNoteFieldState extends State<VoiceNoteField> {
  final AudioRecorder _recorder = AudioRecorder();
  final _voice = VoicePlaybackController.instance;
```

```
bool _recording = false;
bool _busy = false;
String? _localPath;
Duration _elapsed = Duration.zero;
Timer? _tick;

bool get _hasNote =>
    (widget.voiceUrl != null && widget.voiceUrl!.isNotEmpty) ||
    _localPath != null;

String? get _playKey => _localPath ?? widget.voiceUrl;

@override
void initState() {
  super.initState();
  _voice.addListener(_onVoiceChanged);
}

@override
void dispose() {
  _tick?.cancel();
  _voice.removeListener(_onVoiceChanged);
  _recorder.dispose();
  super.dispose();
}

void _onVoiceChanged() {
  if (mounted) setState(() {});
}

String _format(Duration d) {
  final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');
  final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');
  return '$m:$s';
}

Future<String> _recordingPath() async {
  final stamp = DateTime.now().millisecondsSinceEpoch;
  final ext = kIsWeb ? 'wav' : 'm4a';
  final name = 'voice_$stamp.$ext';
  if (kIsWeb) return name;
  final dir = await getTemporaryDirectory();
  return p.join(dir.path, name);
}

Future<void> _startRecording() async {
  if (!widget.enabled || _busy || _recording) return;

  final allowed = await _recorder.hasPermission();
  if (!allowed) {
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(context.l10n.voiceMicPermission)));
    return;
  }

  try {
    await _voice.stop();
    final path = await _recordingPath();
    await _recorder.start(
      RecordConfig(
        encoder: kIsWeb ? AudioEncoder.wav : AudioEncoder.aacLc,
        bitRate: 128000,
        sampleRate: 44100,
      ),
      path: path,
    );
    if (!mounted) return;
    setState(() {
      _recording = true;
      _elapsed = Duration.zero;
      _localPath = null;
```

```
      });
      _tick?.cancel();
      _tick = Timer.periodic(const Duration(seconds: 1), (_) {
        if (!mounted) return;
        setState(() => _elapsed += const Duration(seconds: 1));
      });
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.voiceCouldNotStart('$e'))),
      );
    }
  }

  Future<void> _stopRecording() async {
    if (!_recording) return;
    _tick?.cancel();
    setState(() {
      _recording = false;
      _busy = true;
    });

    try {
      final path = await _recorder.stop();
      if (path == null || path.isEmpty) {
        throw Exception('No audio was captured.');
      }
      final bytes = await XFile(path).readAsBytes();
      if (bytes.isEmpty) throw Exception('Recording was empty.');

      final fileName = kIsWeb
          ? 'voice_${DateTime.now().millisecondsSinceEpoch}.wav'
          : p.basename(path);

      setState(() => _localPath = path);
      await widget.onUpload(bytes, fileName);
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.voiceCouldNotSave('$e'))),
        );
      }
      setState(() => _localPath = null);
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  Future<void> _cancelRecording() async {
    _tick?.cancel();
    try {
      await _recorder.cancel();
    } catch (_) {}
    if (!mounted) return;
    setState(() {
      _recording = false;
      _elapsed = Duration.zero;
    });
  }

  Future<void> _togglePlay() async {
    if (_busy || _recording || !_hasNote) return;
    final key = _playKey;
    if (key == null) return;

    try {
      if (_localPath != null) {
        await _voice.toggleLocal(_localPath!);
      } else if (widget.voiceUrl != null) {
        await _voice.toggleRemote(widget.voiceUrl!);
      }
    } catch (e) {
      if (!mounted) return;
```

```
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),
    );
  }
}

Future<void> _remove() async {
  await _voice.stop();
  setState(() {
    _localPath = null;
  });
  widget.onRemove?.call();
}

@override
Widget build(BuildContext context) {
  if (_recording) return _buildRecordingCard();
  if (_hasNote) return _buildPlaybackCard();
  return _buildIdleCard();
}

Widget _buildIdleCard() {
  return _VoiceShell(
    child: Row(
      children: [
        _RoundAction(
          color: AppColors.accent(context),
          icon: Icons.mic_rounded,
          onPressed: widget.enabled && !_busy ? _startRecording : null,
        ),
        const SizedBox(width: 14),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                context.l10n.taskVoice,
                style: TextStyle(
                  fontWeight: FontWeight.w700,
                  fontSize: 14,
                  color: AppColors.ink(context),
                ),
              ),
              const SizedBox(height: 2),
              Text(
                context.l10n.voiceTapToRecord,
                style: TextStyle(
                  fontSize: 12,
                  color: AppColors.inkAlpha(context, 0.55),
                ),
              ),
            ],
          ),
        ),
        if (_busy)
          const SizedBox(
            width: 20,
            height: 20,
            child: CircularProgressIndicator(strokeWidth: 2),
          ),
      ],
    ),
  );
}

Widget _buildRecordingCard() {
  return _VoiceShell(
    accent: AppColors.errorRed.withValues(alpha: 0.12),
    child: Row(
      children: [
        _PulsingDot(active: _recording),
        const SizedBox(width: 12),
        Expanded(
```

```
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  context.l10n.voiceRecording,
                  style: const TextStyle(
                    fontWeight: FontWeight.w800,
                    fontSize: 14,
                    color: AppColors.errorRed,
                  ),
                ),
                const SizedBox(height: 2),
                Text(
                  _format(_elapsed),
                  style: TextStyle(
                    fontSize: 13,
                    fontFeatures: const [FontFeature.tabularFigures()],
                    color: AppColors.inkAlpha(context, 0.7),
                  ),
                ),
              ],
            ),
          ),
          IconButton(
            tooltip: context.l10n.commonCancel,
            onPressed: _cancelRecording,
            icon: const Icon(Icons.close_rounded),
            color: AppColors.inkAlpha(context, 0.5),
          ),
          const SizedBox(width: 4),
          _RoundAction(
            color: AppColors.errorRed,
            icon: Icons.stop_rounded,
            onPressed: _stopRecording,
          ),
        ],
      ),
    );
  }

  Widget _buildPlaybackCard() {
    final key = _playKey ?? '';
    final playing = _voice.isPlaying(key);
    final loading = _voice.isLoading(key);
    final position = playing || loading ? _voice.position : Duration.zero;
    final duration = playing || loading ? _voice.duration : Duration.zero;
    final progress = duration.inMilliseconds == 0
        ? 0.0
        : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);

    return _VoiceShell(
      child: Column(
        children: [
          Row(
            children: [
              _RoundAction(
                color: AppColors.mutedGold,
                icon: loading
                    ? Icons.hourglass_top_rounded
                    : playing
                    ? Icons.stop_rounded
                    : Icons.play_arrow_rounded,
                onPressed: widget.enabled && !_busy ? _togglePlay : null,
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      context.l10n.voiceNote,
                      style: TextStyle(
                        fontWeight: FontWeight.w700,
```

```
                        fontSize: 14,
                        color: AppColors.ink(context),
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      duration == Duration.zero
                          ? context.l10n.voiceTapPlay
                          : '${_format(position)} / ${_format(duration)}',
                      style: TextStyle(
                        fontSize: 12,
                        fontFeatures: const [FontFeature.tabularFigures()],
                        color: AppColors.inkAlpha(context, 0.55),
                      ),
                    ),
                  ],
                ),
              ),
              if (_busy)
                const Padding(
                  padding: EdgeInsets.only(right: 8),
                  child: SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  ),
                ),
              if (widget.onRemove != null && widget.enabled)
                IconButton(
                  tooltip: context.l10n.commonDelete,
                  onPressed: _busy ? null : _remove,
                  icon: const Icon(Icons.delete_outline_rounded),
                  color: AppColors.errorRed,
                ),
            ],
          ),
          const SizedBox(height: 10),
          ClipRRect(
            borderRadius: BorderRadius.circular(4),
            child: LinearProgressIndicator(
              value: playing || position > Duration.zero ? progress : 0,
              minHeight: 4,
              backgroundColor: AppColors.softSurface(context),
              color: AppColors.mutedGold,
            ),
          ),
          if (widget.enabled && widget.onRemove != null) ...[
            const SizedBox(height: 8),
            Align(
              alignment: Alignment.centerLeft,
              child: TextButton.icon(
                onPressed: _busy || _recording ? null : _startRecording,
                icon: const Icon(Icons.mic_none_rounded, size: 18),
                label: Text(context.l10n.voiceRerecord),
              ),
            ),
          ],
        ],
      ),
    );
  }
}

class VoiceNotePlayer extends StatefulWidget {
  const VoiceNotePlayer({super.key, required this.url, this.title});

  final String url;
  final String? title;

  @override
  State<VoiceNotePlayer> createState() => _VoiceNotePlayerState();
}
```

```
class _VoiceNotePlayerState extends State<VoiceNotePlayer> {
  final _voice = VoicePlaybackController.instance;

  @override
  void initState() {
    super.initState();
    _voice.addListener(_onVoiceChanged);
  }

  @override
  void dispose() {
    _voice.removeListener(_onVoiceChanged);
    super.dispose();
  }

  void _onVoiceChanged() {
    if (mounted) setState(() {});
  }

  String _format(Duration d) {
    final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');
    final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');
    return '$m:$s';
  }

  Future<void> _toggle() async {
    try {
      await _voice.toggleRemote(widget.url);
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final playing = _voice.isPlaying(widget.url);
    final loading = _voice.isLoading(widget.url);
    final position = playing || loading ? _voice.position : Duration.zero;
    final duration = playing || loading ? _voice.duration : Duration.zero;
    final progress = duration.inMilliseconds == 0
        ? 0.0
        : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);

    return _VoiceShell(
      child: Column(
        children: [
          Row(
            children: [
              _RoundAction(
                color: AppColors.mutedGold,
                icon: loading
                    ? Icons.hourglass_top_rounded
                    : playing
                    ? Icons.stop_rounded
                    : Icons.play_arrow_rounded,
                onPressed: _toggle,
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      widget.title ?? l10n.voiceNote,
                      style: TextStyle(
                        fontWeight: FontWeight.w700,
                        fontSize: 14,
```

**lib/components/attachments/voice_note_field.dart — continued (2/2)**

```
                        color: AppColors.ink(context),
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      duration == Duration.zero
                          ? l10n.voiceStudentAnswer
                          : '${_format(position)} / ${_format(duration)}',
                      style: TextStyle(
                        fontSize: 12,
                        color: AppColors.inkAlpha(context, 0.55),
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
          const SizedBox(height: 10),
          ClipRRect(
            borderRadius: BorderRadius.circular(4),
            child: LinearProgressIndicator(
              value: playing || position > Duration.zero ? progress : 0,
              minHeight: 4,
              backgroundColor: AppColors.softSurface(context),
              color: AppColors.mutedGold,
            ),
          ),
        ],
      ),
    );
  }
}

class _VoiceShell extends StatelessWidget {
  const _VoiceShell({required this.child, this.accent});

  final Widget child;
  final Color? accent;

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final bg =
        accent ??
        (isDark
            ? AppColors.darkSurfaceElevated
            : AppColors.sand.withValues(alpha: 0.85));

    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(14),
      decoration: BoxDecoration(
        color: bg,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(
          color: isDark
              ? AppColors.darkBorder.withValues(alpha: 0.55)
              : AppColors.inkAlpha(context, 0.08),
        ),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: isDark ? 0.28 : 0.06),
            blurRadius: 16,
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: DefaultTextStyle.merge(
        style: TextStyle(color: AppColors.ink(context)),
        child: child,
      ),
    );
```

```
  }
}

class _RoundAction extends StatelessWidget {
  const _RoundAction({
    required this.color,
    required this.icon,
    required this.onPressed,
  });

  final Color color;
  final IconData icon;
  final VoidCallback? onPressed;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: color.withValues(alpha: onPressed == null ? 0.35 : 1),
      shape: const CircleBorder(),
      child: InkWell(
        customBorder: const CircleBorder(),
        onTap: onPressed,
        child: SizedBox(
          width: 48,
          height: 48,
          child: Icon(icon, color: Colors.white, size: 26),
        ),
      ),
    );
  }
}

class _PulsingDot extends StatefulWidget {
  const _PulsingDot({required this.active});

  final bool active;

  @override
  State<_PulsingDot> createState() => _PulsingDotState();
}

class _PulsingDotState extends State<_PulsingDot>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 900),
  )..repeat(reverse: true);

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return ScaleTransition(
      scale: Tween(
        begin: 0.85,
        end: 1.15,
      ).animate(CurvedAnimation(parent: _controller, curve: Curves.easeInOut)),
      child: Container(
        width: 14,
        height: 14,
        decoration: const BoxDecoration(
          color: AppColors.errorRed,
          shape: BoxShape.circle,
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/attachments/voice_note_field.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/attendance/attendance_calendar_grid.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 409. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/attendance/attendance_calendar_grid.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';

Color attendanceStatusColor(AttendanceStatus? status, {BuildContext? context}) {
  return switch (status) {
    AttendanceStatus.present => AppColors.successGreen,
    AttendanceStatus.late => AppColors.warmAmber,
    AttendanceStatus.absent => AppColors.errorRed,
    AttendanceStatus.excused => const Color(0xFF5B8DB8),
    null =>
      context != null
          ? AppColors.inkAlpha(context, 0.08)
          : const Color(0xFF808080).withValues(alpha: 0.2),
  };
}

List<List<DateTime?>> buildMonthWeekRows(DateTime month) {
  final first = DateTime(month.year, month.month, 1);
  final daysInMonth = DateTime(month.year, month.month + 1, 0).day;
  final startOffset = first.weekday - 1;
  final cells = <DateTime?>[];
  for (var i = 0; i < startOffset; i++) {
    cells.add(null);
  }
  for (var d = 1; d <= daysInMonth; d++) {
    cells.add(DateTime(month.year, month.month, d));
  }
  while (cells.length % 7 != 0) {
    cells.add(null);
  }

  final weeks = <List<DateTime?>>[];
  for (var i = 0; i < cells.length; i += 7) {
    weeks.add(cells.sublist(i, i + 5));
  }
  while (weeks.isNotEmpty && weeks.last.every((d) => d == null)) {
```

```
    weeks.removeLast();
  }
  return weeks;
}

class AttendanceCalendarGrid extends StatelessWidget {
  const AttendanceCalendarGrid({
    super.key,
    required this.month,
    required this.statusByDate,
    this.onDayTap,
    this.editable = false,
  });

  final DateTime month;

  final Map<DateTime, AttendanceStatus> statusByDate;
  final ValueChanged<DateTime>? onDayTap;
  final bool editable;

  DateTime _dayKey(DateTime d) => DateTime(d.year, d.month, d.day);

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final weeks = buildMonthWeekRows(month);
    final labels = [
      l10n.attendanceWeekdayMon,
      l10n.attendanceWeekdayTue,
      l10n.attendanceWeekdayWed,
      l10n.attendanceWeekdayThu,
      l10n.attendanceWeekdayFri,
    ];

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Row(
          children: [
            for (final label in labels)
              Expanded(
                child: Center(
                  child: Text(
                    label,
                    style: Theme.of(context).textTheme.labelSmall?.copyWith(
                      fontWeight: FontWeight.w700,
                      color: AppColors.inkAlpha(context, 0.45),
                    ),
                  ),
                ),
              ),
          ],
        ),
        const SizedBox(height: 8),
        for (final week in weeks) ...[
          Row(
            children: [
              for (final day in week)
                Expanded(
                  child: Padding(
                    padding: const EdgeInsets.all(3),
                    child: _DayCell(
                      day: day,
                      status: day == null ? null : statusByDate[_dayKey(day)],
                      editable:
                          editable &&
                          day != null &&
                          isAttendanceDateAllowed(day),
                      isFuture: day != null && !isAttendanceDateAllowed(day),
                      onTap:
                          day == null ||
                              onDayTap == null ||
                              !isAttendanceDateAllowed(day)
```

```
                          ? null
                          : () => onDayTap!(day),
                    ),
                  ),
                ),
              ],
            ),
          ],
        ],
      );
    }
  }

  class _DayCell extends StatelessWidget {
    const _DayCell({
      required this.day,
      required this.status,
      required this.editable,
      this.isFuture = false,
      this.onTap,
    });

    final DateTime? day;
    final AttendanceStatus? status;
    final bool editable;
    final bool isFuture;
    final VoidCallback? onTap;

    @override
    Widget build(BuildContext context) {
      if (day == null) {
        return AspectRatio(
          aspectRatio: 1,
          child: DecoratedBox(
            decoration: BoxDecoration(
              color: AppColors.inkAlpha(context, 0.03),
              borderRadius: BorderRadius.circular(10),
            ),
          ),
        );
      }

      final color = attendanceStatusColor(status, context: context);
      final hasStatus = status != null;
      final today = DateTime.now();
      final isToday =
          day!.year == today.year &&
          day!.month == today.month &&
          day!.day == today.day;

      final bg = isFuture
          ? AppColors.inkAlpha(context, 0.04)
          : hasStatus
          ? color.withValues(alpha: 0.92)
          : AppColors.inkAlpha(context, 0.06);

      return AspectRatio(
        aspectRatio: 1,
        child: Material(
          color: bg,
          borderRadius: BorderRadius.circular(10),
          child: InkWell(
            onTap: editable ? onTap : null,
            borderRadius: BorderRadius.circular(10),
            child: DecoratedBox(
              decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(10),
                border: isToday
                    ? Border.all(color: AppColors.accent(context), width: 1.5)
                    : null,
              ),
              child: Center(
                child: Text(
```

```
                '${day!.day}',
                style: TextStyle(
                  fontWeight: FontWeight.w700,
                  fontSize: 12,
                  color: isFuture
                      ? AppColors.inkAlpha(context, 0.22)
                      : hasStatus
                      ? Colors.white
                      : AppColors.inkAlpha(context, 0.45),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

class AttendanceMonthHeader extends StatelessWidget {
  const AttendanceMonthHeader({
    super.key,
    required this.month,
    required this.onPrev,
    required this.onNext,
  });

  final DateTime month;
  final VoidCallback onPrev;
  final VoidCallback onNext;

  @override
  Widget build(BuildContext context) {
    final label = DateFormat.yMMMM(
      Localizations.localeOf(context).toString(),
    ).format(month);

    return Row(
      children: [
        IconButton(
          onPressed: onPrev,
          icon: const Icon(Icons.chevron_left_rounded),
        ),
        Expanded(
          child: Text(
            label,
            textAlign: TextAlign.center,
            style: Theme.of(
              context,
            ).textTheme.titleMedium?.copyWith(fontWeight: FontWeight.w800),
          ),
        ),
        IconButton(
          onPressed: onNext,
          icon: const Icon(Icons.chevron_right_rounded),
        ),
      ],
    );
  }
}

class AttendanceStatsBar extends StatelessWidget {
  const AttendanceStatsBar({super.key, required this.stats});

  final AttendanceStats stats;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return Row(
      children: [
        Expanded(
          child: _StatChip(
```

```
              label: l10n.attendancePercentLabel,
              value: '${stats.presentPercent.toStringAsFixed(0)}%',
              color: AppColors.accent(context),
            ),
          ),
          const SizedBox(width: 8),
          Expanded(
            child: _StatChip(
              label: l10n.attendancePresentWord,
              value: '${stats.present}',
              color: AppColors.successGreen,
            ),
          ),
          const SizedBox(width: 8),
          Expanded(
            child: _StatChip(
              label: l10n.attendanceAbsentWord,
              value: '${stats.absent}',
              color: AppColors.errorRed,
            ),
          ),
          const SizedBox(width: 8),
          Expanded(
            child: _StatChip(
              label: l10n.attendanceLateWord,
              value: '${stats.late}',
              color: AppColors.warmAmber,
            ),
          ),
        ],
      );
    }
  }

  class _StatChip extends StatelessWidget {
    const _StatChip({
      required this.label,
      required this.value,
      required this.color,
    });

    final String label;
    final String value;
    final Color color;

    @override
    Widget build(BuildContext context) {
      return Container(
        padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 10),
        decoration: BoxDecoration(
          color: color.withValues(alpha: 0.1),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Column(
          children: [
            Text(
              value,
              style: TextStyle(
                fontWeight: FontWeight.w800,
                fontSize: 16,
                color: color,
              ),
            ),
            const SizedBox(height: 2),
            Text(
              label,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: TextStyle(
                fontSize: 10,
                fontWeight: FontWeight.w600,
                color: AppColors.inkAlpha(context, 0.55),
              ),
```

```
          ),
        ],
      ),
    );
  }
}

class AttendanceLegendRow extends StatelessWidget {
  const AttendanceLegendRow({super.key});

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return Wrap(
      spacing: 12,
      runSpacing: 8,
      children: [
        _Legend(
          color: AppColors.successGreen,
          label: l10n.attendancePresentWord,
        ),
        _Legend(color: AppColors.warmAmber, label: l10n.attendanceLateWord),
        _Legend(color: AppColors.errorRed, label: l10n.attendanceAbsentWord),
        _Legend(
          color: const Color(0xFF5B8DB8),
          label: l10n.attendanceExcusedWord,
        ),
        _Legend(
          color: AppColors.inkAlpha(context, 0.12),
          label: l10n.attendanceNoLesson,
        ),
      ],
    );
  }
}

class _Legend extends StatelessWidget {
  const _Legend({required this.color, required this.label});
  final Color color;
  final String label;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        Container(
          width: 12,
          height: 12,
          decoration: BoxDecoration(
            color: color,
            borderRadius: BorderRadius.circular(3),
          ),
        ),
        const SizedBox(width: 6),
        Text(
          label,
          style: TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.w600,
            color: AppColors.inkAlpha(context, 0.6),
          ),
        ),
      ],
    );
  }
}

AttendanceStatus nextAttendanceStatus(AttendanceStatus? current) {
  return switch (current) {
    null => AttendanceStatus.present,
    AttendanceStatus.present => AttendanceStatus.late,
    AttendanceStatus.late => AttendanceStatus.absent,
```

```
      AttendanceStatus.absent => AttendanceStatus.excused,
      AttendanceStatus.excused => AttendanceStatus.present,
    };
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/attendance/attendance_calendar_grid.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

# lib/components/backgrounds/nature_paper_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 505. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math' as math;
- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/backgrounds/nature_paper_backdrop.dart**

```
import 'dart:math' as math;

import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

const _kAssetDir = 'assets/images/nature-paper';

class NaturePaperBackdrop extends StatefulWidget {
  const NaturePaperBackdrop({super.key, required this.child});

  final Widget child;

  @override
  State<NaturePaperBackdrop> createState() => _NaturePaperBackdropState();
}

class _NaturePaperBackdropState extends State<NaturePaperBackdrop>
    with SingleTickerProviderStateMixin {
  late final AnimationController _transition;
  bool? _wasNightScene;

  @override
  void initState() {
    super.initState();
    _transition = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1700),
    );
  }

  @override
  void dispose() {
    _transition.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    final isNature = tokens?.isNaturePaper ?? false;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    final bool? nightScene = isNature ? isDark : null;
```

```
    if (nightScene != null &&
        _wasNightScene != null &&
        nightScene != _wasNightScene) {
      _transition.forward(from: 0);
    }
    _wasNightScene = nightScene;

    return Stack(
      children: [
        Positioned.fill(
          child: RepaintBoundary(
            child: ExcludeSemantics(
              child: isNature
                  ? AnimatedSwitcher(
                      duration: const Duration(milliseconds: 650),
                      switchInCurve: Curves.easeOut,
                      switchOutCurve: Curves.easeIn,
                      transitionBuilder: _fadeWithoutChildKey,
                      child: isDark
                          ? const _NightScene(key: ValueKey('nature-night'))
                          : const _DayScene(key: ValueKey('nature-day')),
                    )
                  : const SizedBox.shrink(),
            ),
          ),
        ),
        widget.child,
        Positioned.fill(
          child: IgnorePointer(
            child: _TransitionLeaves(animation: _transition),
          ),
        ),
      ],
    );
  }
}

Widget _fadeWithoutChildKey(Widget child, Animation<double> animation) {
  return FadeTransition(opacity: animation, child: child);
}

class _DayScene extends StatefulWidget {
  const _DayScene({super.key});

  @override
  State<_DayScene> createState() => _DaySceneState();
}

class _DaySceneState extends State<_DayScene>
    with SingleTickerProviderStateMixin {
  late final AnimationController _drift;

  static const _clouds = [
    _CloudSpec('white-cloud.png', y: 0.07, size: 0.30, speed: 1.7, phase: 0.1),
    _CloudSpec('flat-cloud.png', y: 0.16, size: 0.22, speed: 1.15, phase: 0.55),
    _CloudSpec('gray-cloud.png', y: 0.26, size: 0.17, speed: 1.4, phase: 0.85),
  ];

  static const _leaves = [
    _LeafSpec('beige-leaf.png', x: 0.12, size: 26, speed: 0.9, phase: 0.0),
    _LeafSpec('tan-seed.png', x: 0.32, size: 18, speed: 1.3, phase: 0.35),
    _LeafSpec('white-daisy.png', x: 0.55, size: 22, speed: 0.7, phase: 0.6),
    _LeafSpec('beige-leaf.png', x: 0.74, size: 20, speed: 1.1, phase: 0.2),
    _LeafSpec('salmon-flower.png', x: 0.88, size: 24, speed: 0.8, phase: 0.8),
  ];

  @override
  void initState() {
    super.initState();
    _drift = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 36),
    )..repeat();
```

```
  }

  @override
  void dispose() {
    _drift.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final w = constraints.maxWidth;
        final h = constraints.maxHeight;
        return Stack(
          fit: StackFit.expand,
          children: [
            const DecoratedBox(
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  begin: Alignment.topCenter,
                  end: Alignment.bottomCenter,
                  stops: [0.0, 0.52, 1.0],
                  colors: [
                    NaturePaperColors.cream,
                    Color(0xFFEFE4CD),
                    NaturePaperColors.paleBlue,
                  ],
                ),
              ),
            ),
            Positioned(
              top: -h * 0.12,
              left: -w * 0.15,
              child: Container(
                width: w * 0.9,
                height: w * 0.9,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      NaturePaperColors.sunGlow.withValues(alpha: 0.30),
                      NaturePaperColors.sunGlow.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
            const CustomPaint(painter: _PaperGrainPainter(isDark: false)),
            Positioned(
              left: 0,
              right: 0,
              bottom: 0,
              child: Image.asset(
                '$_kAssetDir/landscape-hills-footer.png',
                width: w,
                fit: BoxFit.fitWidth,
                opacity: const AlwaysStoppedAnimation(0.4),
                filterQuality: FilterQuality.medium,
              ),
            ),
            AnimatedBuilder(
              animation: _drift,
              builder: (context, _) {
                final t = _drift.value;
                return Stack(
                  children: [
                    for (final c in _clouds)
                      Positioned(
                        left: (((t * c.speed + c.phase) % 1.2) - 0.15) * w,
                        top: c.y * h,
                        child: Image.asset(
                          '$_kAssetDir/${c.asset}',
```

```
                        width: (w * c.size).clamp(70.0, 220.0),
                        opacity: const AlwaysStoppedAnimation(0.55),
                        filterQuality: FilterQuality.medium,
                      ),
                    ),
                  for (final leaf in _leaves)
                    Positioned(
                      left:
                          leaf.x * w +
                          math.sin(t * math.pi * 8 + leaf.phase * 7) * 18,
                      top:
                          (((t * leaf.speed * 2 + leaf.phase) % 1.1) - 0.05) *
                          h,
                      child: Transform.rotate(
                        angle:
                            math.sin(t * math.pi * 6 + leaf.phase * 9) * 0.5,
                        child: Image.asset(
                          '$_kAssetDir/${leaf.asset}',
                          width: leaf.size,
                          opacity: const AlwaysStoppedAnimation(0.45),
                          filterQuality: FilterQuality.medium,
                        ),
                      ),
                    ),
                ],
              );
            },
          ),
        ],
      );
    },
  );
  }
}

class _NightScene extends StatefulWidget {
  const _NightScene({super.key});

  @override
  State<_NightScene> createState() => _NightSceneState();
}

class _NightSceneState extends State<_NightScene>
    with SingleTickerProviderStateMixin {
  late final AnimationController _drift;

  @override
  void initState() {
    super.initState();
    _drift = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 40),
    )..repeat();
  }

  @override
  void dispose() {
    _drift.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final w = constraints.maxWidth;
        final h = constraints.maxHeight;
        return Stack(
          fit: StackFit.expand,
          children: [
            const DecoratedBox(
              decoration: BoxDecoration(
                gradient: LinearGradient(
```

```
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
                stops: [0.0, 1.0],
                colors: [
                  NaturePaperColors.midnight,
                  NaturePaperColors.forestNight,
                ],
              ),
            ),
          ),
          Positioned(
            top: -h * 0.10,
            right: -w * 0.20,
            child: Container(
              width: w * 0.85,
              height: w * 0.85,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: RadialGradient(
                  colors: [
                    NaturePaperColors.moonGlow.withValues(alpha: 0.16),
                    NaturePaperColors.moonGlow.withValues(alpha: 0.0),
                  ],
                ),
              ),
            ),
          ),
          const CustomPaint(painter: _PaperGrainPainter(isDark: true)),
          AnimatedBuilder(
            animation: _drift,
            builder: (context, _) =>
                CustomPaint(painter: _NightSkyPainter(t: _drift.value)),
          ),
          Positioned(
            left: 0,
            right: 0,
            bottom: h * 0.03,
            child: ColorFiltered(
              colorFilter: const ColorFilter.mode(
                Color(0xFF1C2822),
                BlendMode.srcIn,
              ),
              child: Image.asset(
                '$_kAssetDir/landscape-hills-footer.png',
                width: w,
                fit: BoxFit.fitWidth,
                opacity: const AlwaysStoppedAnimation(0.9),
                filterQuality: FilterQuality.medium,
              ),
            ),
          ),
          Positioned(
            left: 0,
            right: 0,
            bottom: 0,
            child: ColorFiltered(
              colorFilter: const ColorFilter.mode(
                Color(0xFF121B16),
                BlendMode.srcIn,
              ),
              child: Image.asset(
                '$_kAssetDir/landscape-hills-footer.png',
                width: w,
                fit: BoxFit.fitWidth,
                filterQuality: FilterQuality.medium,
              ),
            ),
          ),
        ],
      );
    },
  );
}
```

```
}

class _NightSkyPainter extends CustomPainter {
  const _NightSkyPainter({required this.t});

  final double t;

  static const _starCount = 70;
  static const _fireflyCount = 9;

  @override
  void paint(Canvas canvas, Size size) {
    final rng = math.Random(7);
    final starPaint = Paint();
    for (var i = 0; i < _starCount; i++) {
      final x = rng.nextDouble() * size.width;
      final y = rng.nextDouble() * size.height * 0.55;
      final r = 0.5 + rng.nextDouble() * 0.9;
      final baseAlpha = 0.10 + rng.nextDouble() * 0.25;
      final twinkle = i % 3 == 0
          ? 0.5 + 0.5 * math.sin(t * math.pi * 10 + i * 1.7)
          : 1.0;
      starPaint.color = NaturePaperColors.moonBeige.withValues(
        alpha: baseAlpha * twinkle,
      );
      canvas.drawCircle(Offset(x, y), r, starPaint);
    }

    final glowPaint = Paint()
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 5);
    final corePaint = Paint();
    final fly = math.Random(23);
    for (var i = 0; i < _fireflyCount; i++) {
      final baseX = fly.nextDouble();
      final baseY = 0.35 + fly.nextDouble() * 0.55;
      final phase = fly.nextDouble() * math.pi * 2;
      final x =
          (baseX +
              0.05 * math.sin(t * math.pi * 4 + phase) +
              0.02 * math.sin(t * math.pi * 14 + phase * 3)) *
          size.width;
      final y =
          (baseY + 0.03 * math.cos(t * math.pi * 6 + phase * 2)) * size.height;
      final pulse = (0.5 + 0.5 * math.sin(t * math.pi * 8 + phase * 5)).clamp(
        0.0,
        1.0,
      );
      if (pulse < 0.15) continue;
      glowPaint.color = NaturePaperColors.amber.withValues(alpha: 0.35 * pulse);
      corePaint.color = const Color(0xFFF4D9A0).withValues(alpha: 0.85 * pulse);
      canvas.drawCircle(Offset(x, y), 5, glowPaint);
      canvas.drawCircle(Offset(x, y), 1.6, corePaint);
    }
  }

  @override
  bool shouldRepaint(_NightSkyPainter oldDelegate) => oldDelegate.t != t;
}

class _PaperGrainPainter extends CustomPainter {
  const _PaperGrainPainter({required this.isDark});

  final bool isDark;

  @override
  void paint(Canvas canvas, Size size) {
    final rng = math.Random(42);
    final paint = Paint()
      ..color = isDark
          ? NaturePaperColors.moonBeige.withValues(alpha: 0.025)
          : NaturePaperColors.warmBrown.withValues(alpha: 0.035);
    final count = (size.width * size.height / 3200).round().clamp(120, 700);
    for (var i = 0; i < count; i++) {
```

```
      canvas.drawCircle(
        Offset(rng.nextDouble() * size.width, rng.nextDouble() * size.height),
        0.6 + rng.nextDouble() * 0.7,
        paint,
      );
    }
  }

  @override
  bool shouldRepaint(_PaperGrainPainter oldDelegate) =>
      oldDelegate.isDark != isDark;
}

class _TransitionLeaves extends StatelessWidget {
  const _TransitionLeaves({required this.animation});

  final Animation<double> animation;

  static const _sprites = [
    _LeafSpec('beige-leaf.png', x: 0.08, size: 30, speed: 1.0, phase: 0.0),
    _LeafSpec('white-daisy.png', x: 0.25, size: 24, speed: 1.3, phase: 0.3),
    _LeafSpec('tan-seed.png', x: 0.42, size: 20, speed: 0.9, phase: 0.6),
    _LeafSpec('beige-leaf.png', x: 0.58, size: 26, speed: 1.2, phase: 0.15),
    _LeafSpec('salmon-flower.png', x: 0.72, size: 24, speed: 1.0, phase: 0.45),
    _LeafSpec('beige-leaf.png', x: 0.88, size: 22, speed: 1.4, phase: 0.75),
  ];

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
```

**lib/components/backgrounds/nature_paper_backdrop.dart — continued (2/2)**

```
      animation: animation,
      builder: (context, _) {
        final t = animation.value;
        if (t == 0 || animation.isCompleted) return const SizedBox.shrink();
        final size = MediaQuery.sizeOf(context);
        final opacity = math.sin(t * math.pi).clamp(0.0, 1.0) * 0.85;
        return Stack(
          children: [
            for (final leaf in _sprites)
              Positioned(
                left:
                    leaf.x * size.width +
                    math.sin(t * math.pi * 3 + leaf.phase * 8) * 26,
                top:
                    (t * leaf.speed * 1.1 - 0.08 + leaf.phase * 0.1) *
                    size.height,
                child: Opacity(
                  opacity: opacity,
                  child: Transform.rotate(
                    angle: t * math.pi * 2 * leaf.speed + leaf.phase * 5,
                    child: Image.asset(
                      '$_kAssetDir/${leaf.asset}',
                      width: leaf.size,
                      filterQuality: FilterQuality.medium,
                    ),
                  ),
                ),
              ),
          ],
        );
      },
    );
  }
}

class _CloudSpec {
  const _CloudSpec(
    this.asset, {
    required this.y,
    required this.size,
```

```
    required this.speed,
    required this.phase,
  });

  final String asset;
  final double y;
  final double size;
  final double speed;
  final double phase;
}

class _LeafSpec {
  const _LeafSpec(
    this.asset, {
    required this.x,
    required this.size,
    required this.speed,
    required this.phase,
  });

  final String asset;
  final double x;
  final double size;
  final double speed;
  final double phase;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/backgrounds/nature_paper_backdrop.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/backgrounds/soft_neumorphism_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 170. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/backgrounds/soft_neumorphism_backdrop.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

class SoftNeumorphismBackdrop extends StatelessWidget {
  const SoftNeumorphismBackdrop({super.key, required this.child});

  final Widget child;

  @override
  Widget build(BuildContext context) {
    final isNeu = AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    return Stack(
      children: [
        Positioned.fill(
          child: RepaintBoundary(
            child: ExcludeSemantics(
              child: isNeu
                  ? AnimatedSwitcher(
                      duration: const Duration(milliseconds: 550),
                      switchInCurve: Curves.easeOut,
                      switchOutCurve: Curves.easeIn,
                      transitionBuilder: _fadeWithoutChildKey,
                      child: isDark
                          ? const _NightAmbient(key: ValueKey('neu-night'))
                          : const _MorningAmbient(key: ValueKey('neu-day')),
                    )
                  : const SizedBox.shrink(),
            ),
          ),
        ),
        child,
      ],
    );
  }
}

Widget _fadeWithoutChildKey(Widget child, Animation<double> animation) {
  return FadeTransition(opacity: animation, child: child);
}

class _MorningAmbient extends StatelessWidget {
  const _MorningAmbient({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [
            SoftNeumorphismColors.morningBg,
            SoftNeumorphismColors.morningBgCool,
            Color(0xFFECE8E0),
          ],
          stops: [0.0, 0.55, 1.0],
        ),
      ),
      child: Stack(
        children: [
          Positioned(
            top: -80,
            right: -60,
            child: IgnorePointer(
              child: Container(
                width: 280,
                height: 280,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.sage.withValues(alpha: 0.10),
                      SoftNeumorphismColors.sage.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
          Positioned(
            bottom: -40,
            left: -40,
            child: IgnorePointer(
              child: Container(
                width: 240,
                height: 240,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.amber.withValues(alpha: 0.08),
                      SoftNeumorphismColors.amber.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _NightAmbient extends StatelessWidget {
  const _NightAmbient({super.key});

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [
            SoftNeumorphismColors.nightBgCool,
```

```
            SoftNeumorphismColors.nightBg,
            Color(0xFF1C2220),
          ],
          stops: [0.0, 0.5, 1.0],
        ),
      ),
      child: Stack(
        children: [
          Positioned(
            top: -100,
            left: -40,
            child: IgnorePointer(
              child: Container(
                width: 300,
                height: 300,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.emerald.withValues(alpha: 0.09),
                      SoftNeumorphismColors.emerald.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
          Positioned(
            bottom: -60,
            right: -50,
            child: IgnorePointer(
              child: Container(
                width: 260,
                height: 260,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.amber.withValues(alpha: 0.06),
                      SoftNeumorphismColors.amber.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/backgrounds/soft_neumorphism_backdrop.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/components/brand/app_logo.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 34. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/brand/app_logo.dart**

```
import 'package:flutter/material.dart';

class AppLogo {
  static const asset = 'assets/images/logo/logo.jpeg';
}

class AppLogoImage extends StatelessWidget {
  const AppLogoImage({
    super.key,
    this.size = 40,
    this.borderRadius,
    this.fit = BoxFit.cover,
  });

  final double size;
  final BorderRadius? borderRadius;
  final BoxFit fit;

  @override
  Widget build(BuildContext context) {
    final radius = borderRadius ?? BorderRadius.circular(size * 0.22);
    return ClipRRect(
      borderRadius: radius,
      child: Image.asset(
        AppLogo.asset,
        width: size,
        height: size,
        fit: fit,
        filterQuality: FilterQuality.medium,
        semanticLabel: 'MU Study',
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/brand/app_logo.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/buttons/primary_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 83. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/buttons/primary_button.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

class PrimaryButton extends StatelessWidget {
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.loading = false,
    this.icon,
    this.backgroundColor,
    this.foregroundColor,
  });

  final String label;
  final VoidCallback? onPressed;
  final bool loading;
  final IconData? icon;
  final Color? backgroundColor;
  final Color? foregroundColor;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final nature = isNaturePaper(context);
    final bg =
        backgroundColor ??
        (nature ? AppColors.accent(context) : AppColors.coffeeBrown);
    final fg =
        foregroundColor ??
        (nature ? theme.colorScheme.onPrimary : Colors.white);

    final ButtonStyle style;
    if (nature && backgroundColor == null && foregroundColor == null) {
      style = (theme.elevatedButtonTheme.style ?? const ButtonStyle()).merge(
        ElevatedButton.styleFrom(minimumSize: const Size.fromHeight(56)),
      );
    } else {
      style = ElevatedButton.styleFrom(
        backgroundColor: bg,
        foregroundColor: fg,
        minimumSize: const Size.fromHeight(56),
```

```
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(nature ? 18 : 14),
        ),
        elevation: nature ? 3 : 0,
        shadowColor: nature
            ? NaturePaperColors.warmBrown.withValues(alpha: 0.45)
            : null,
      );
    }

    return ElevatedButton(
      onPressed: loading ? null : onPressed,
      style: style,
      child: loading
          ? SizedBox(
              height: 24,
              width: 24,
              child: CircularProgressIndicator(strokeWidth: 2.5, color: fg),
            )
          : Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                if (icon != null) ...[
                  PaperGlyph(icon: icon!, size: 20, color: fg),
                  const SizedBox(width: 10),
                ],
                Text(
                  label,
                  style: theme.textTheme.labelLarge?.copyWith(
                    fontWeight: FontWeight.w700,
                    color: fg,
                    fontSize: 16,
                  ),
                ),
              ],
            ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/buttons/primary_button.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/cards/premium_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 245. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_animate/flutter_animate.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/cards/premium_card.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_animate/flutter_animate.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';

class PremiumCard extends StatefulWidget {
  const PremiumCard({
    super.key,
    required this.child,
    this.onTap,
    this.padding = const EdgeInsets.all(20),
    this.color,
    this.borderColor,
    this.borderRadius,
    this.elevation,
  });

  final Widget child;
  final VoidCallback? onTap;
  final EdgeInsets padding;
  final Color? color;
  final Color? borderColor;

  final double? borderRadius;

  final NeuElevation? elevation;

  @override
  State<PremiumCard> createState() => _PremiumCardState();
}

class _PremiumCardState extends State<PremiumCard>
    with SingleTickerProviderStateMixin {
  late final AnimationController _hoverController;
  late final Animation<double> _scaleAnimation;
  late final Animation<double> _elevationAnimation;
  bool _pressed = false;

  @override
```

```
void initState() {
  super.initState();
  _hoverController = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 200),
  );
  _scaleAnimation = Tween<double>(begin: 1.0, end: 1.01).animate(
    CurvedAnimation(parent: _hoverController, curve: Curves.easeOutCubic),
  );
  _elevationAnimation = Tween<double>(begin: 4.0, end: 12.0).animate(
    CurvedAnimation(parent: _hoverController, curve: Curves.easeOutCubic),
  );
}

@override
void dispose() {
  _hoverController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final scheme = Theme.of(context).colorScheme;
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final tokens = AppStyleTokens.maybeOf(context);
  final isNature = tokens?.isNaturePaper ?? false;
  final isNeu = tokens?.isSoftNeumorphism ?? false;
  final isMono = tokens?.isMonochromeWorkspace ?? false;
  final radius = widget.borderRadius ?? tokens?.cardRadius ?? 16.0;

  return AnimatedBuilder(
        animation: _hoverController,
        builder: (context, child) {
          final hover = _hoverController.value;
          final cardColor =
              widget.color ??
              tokens?.cardSurface ??
              (isDark ? AppColors.darkSurfaceElevated : scheme.surface);
          final borderColor =
              widget.borderColor ??
              tokens?.cardBorder ??
              (isDark
                  ? AppColors.darkBorder
                  : AppColors.inkAlpha(context, 0.06));
          final shadowColor = isDark
              ? Colors.black.withValues(alpha: 0.55 + hover * 0.25)
              : scheme.onSurface.withValues(alpha: 0.02 + hover * 0.03);

          final List<BoxShadow> shadows;
          final bool showBorder;
          Color fillColor = cardColor;

          if (isNeu) {
            final baseElev = widget.elevation ?? NeuElevation.raised;
            final elev = _pressed
                ? NeuElevation.pressed
                : (hover > 0.5 ? NeuElevation.floating : baseElev);
            fillColor =
                widget.color ?? Neumorphic.surface(context, elevation: elev);
            shadows = Neumorphic.shadows(
              context,
              elevation: elev,
              intensity: 1.15 + hover * 0.2,
            );
            showBorder = false;
          } else if (isMono) {
            shadows = const [];
            showBorder = true;
          } else if (isNature && !isDark) {
            fillColor = widget.color ?? NaturePaperColors.dayCard;
            shadows = [
              BoxShadow(
                color: NaturePaperColors.warmBrown.withValues(
```

```
        alpha: 0.12 + hover * 0.06,
      ),
      blurRadius: 16 + hover * 8,
      offset: Offset(0, 6 + hover * 2),
    ),
    const BoxShadow(
      color: NaturePaperColors.dayCardEdge,
      offset: Offset(0, 2),
    ),
  ];
  showBorder = true;
} else if (isNature && isDark) {
  fillColor = widget.color ?? NaturePaperColors.nightCardHigh;
  shadows = [
    BoxShadow(
      color: Colors.black.withValues(alpha: 0.45 + hover * 0.12),
      blurRadius: 18,
      offset: const Offset(0, 8),
    ),
    BoxShadow(
      color: NaturePaperColors.moonGlow.withValues(
        alpha: 0.05 + hover * 0.08,
      ),
      blurRadius: 20 + hover * 10,
    ),
  ];
  showBorder = true;
} else {
  shadows = [
    BoxShadow(
      color: shadowColor,
      blurRadius: _elevationAnimation.value + (isDark ? 6 : 0),
      offset: Offset(0, _elevationAnimation.value / 2),
    ),
  ];
  showBorder = true;
}

final Color hoverBorder;
if (isMono) {
  hoverBorder = hover > 0
      ? AppColors.inkAlpha(context, isDark ? 0.45 : 0.28)
      : borderColor;
} else if (isNature) {
  hoverBorder = isDark
      ? NaturePaperColors.moonGlow.withValues(
          alpha: 0.30 + hover * 0.18,
        )
      : NaturePaperColors.warmBrown.withValues(
          alpha: 0.30 + hover * 0.10,
        );
} else if (isDark && hover > 0 && !isNeu) {
  hoverBorder = AppColors.mutedGold.withValues(
    alpha: 0.35 + hover * 0.25,
  );
} else {
  hoverBorder = borderColor;
}

final scale = isMono
    ? 1.0
    : (isNeu
          ? (_pressed ? 0.985 : (1.0 + hover * 0.012))
          : _scaleAnimation.value);

return Transform.scale(
  scale: scale,
  child: AnimatedContainer(
    duration: const Duration(milliseconds: 160),
    curve: Curves.easeOutCubic,
    margin: const EdgeInsets.symmetric(vertical: 8),
    decoration: BoxDecoration(
      color: fillColor,
```

```
                  borderRadius: BorderRadius.circular(radius),
                  border: showBorder
                      ? Border.all(
                          color: hoverBorder,
                          width: isNature ? 1.4 : 1.0,
                        )
                      : null,
                  boxShadow: shadows,
                ),
                child: Material(
                  color: Colors.transparent,
                  child: InkWell(
                    onTap: widget.onTap,
                    onTapDown: widget.onTap == null
                        ? null
                        : (_) => setState(() => _pressed = true),
                    onTapUp: widget.onTap == null
                        ? null
                        : (_) => setState(() => _pressed = false),
                    onTapCancel: widget.onTap == null
                        ? null
                        : () => setState(() => _pressed = false),
                    onHover: (isHovered) {
                      if (widget.onTap != null) {
                        if (isHovered) {
                          _hoverController.forward();
                        } else {
                          _hoverController.reverse();
                        }
                      }
                    },
                    borderRadius: BorderRadius.circular(radius),
                    highlightColor: AppColors.accent(
                      context,
                    ).withValues(alpha: isDark ? 0.12 : 0.05),
                    splashColor: AppColors.accent(
                      context,
                    ).withValues(alpha: isDark ? 0.16 : 0.05),
                    child: Padding(
                      padding: widget.padding,
                      child: widget.child,
                    ),
                  ),
                ),
              ),
            );
          },
        )
        .animate()
        .fadeIn(duration: 400.ms, curve: Curves.easeOutCubic)
        .slideY(
          begin: 0.02,
          end: 0,
          duration: 400.ms,
          curve: Curves.easeOutCubic,
        );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/cards/premium_card.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/chat/animated_emoji_message.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 164. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:lottie/lottie.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/chat/animated_emoji_message.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:lottie/lottie.dart';

class AnimatedEmojiMessage extends StatefulWidget {
  const AnimatedEmojiMessage({
    super.key,
    required this.assetPath,
    required this.emoji,
    this.size = 144,
  });

  final String assetPath;

  final String emoji;

  final double size;

  @override
  State<AnimatedEmojiMessage> createState() => _AnimatedEmojiMessageState();
}

class _AnimatedEmojiMessageState extends State<AnimatedEmojiMessage>
    with TickerProviderStateMixin {
  LottieComposition? _composition;
  bool _failed = false;
  late final AnimationController _lottieController;
  late final AnimationController _bounceController;
  late final Animation<double> _bounceScale;

  @override
  void initState() {
    super.initState();
    _lottieController = AnimationController(vsync: this);
    _lottieController.addStatusListener(_onLottieStatus);
    _bounceController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 280),
    );
    _bounceScale = TweenSequence<double>([
      TweenSequenceItem(
        tween: Tween(
          begin: 1.0,
          end: 1.14,
        ).chain(CurveTween(curve: Curves.easeOut)),
```

```
      weight: 40,
    ),
    TweenSequenceItem(
      tween: Tween(
        begin: 1.14,
        end: 1.0,
      ).chain(CurveTween(curve: Curves.easeInOut)),
      weight: 60,
    ),
  ]).animate(_bounceController);
  _load();
}

void _onLottieStatus(AnimationStatus status) {
  if (status != AnimationStatus.completed) return;
  _holdLastVisibleFrame();
}

void _holdLastVisibleFrame() {
  final composition = _composition;
  if (composition == null || !mounted) return;
  final span = composition.endFrame - composition.startFrame;
  if (span <= 0) return;
  const epsilonFrames = 0.5;
  final hold = ((span - epsilonFrames) / span).clamp(0.0, 0.999);
  if (_lottieController.value != hold) {
    _lottieController.value = hold;
  }
}

@override
void didUpdateWidget(covariant AnimatedEmojiMessage oldWidget) {
  super.didUpdateWidget(oldWidget);
  if (oldWidget.assetPath != widget.assetPath) {
    _composition = null;
    _failed = false;
    _lottieController
      ..stop()
      ..reset();
    _load();
  }
}

Future<void> _load() async {
  try {
    final composition = await AssetLottie(widget.assetPath).load();
    if (!mounted) return;
    setState(() {
      _composition = composition;
      _failed = false;
      _lottieController.duration = composition.duration;
      _lottieController.value = 0;
    });
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted || _composition == null) return;
      _lottieController.forward(from: 0);
    });
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _failed = true;
      _composition = null;
    });
  }
}

void _replay() {
  if (_composition == null) return;
  HapticFeedback.selectionClick();
  _bounceController.forward(from: 0);
  _lottieController.forward(from: 0);
}
```

```
  @override
  void dispose() {
    _lottieController.removeStatusListener(_onLottieStatus);
    _lottieController.dispose();
    _bounceController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTap: _composition != null ? _replay : null,
      child: SizedBox(
        width: widget.size,
        height: widget.size,
        child: RepaintBoundary(
          child: ScaleTransition(scale: _bounceScale, child: _buildBody()),
        ),
      ),
    );
  }

  Widget _buildBody() {
    if (_failed) {
      return Center(
        child: Text(
          widget.emoji,
          style: TextStyle(fontSize: widget.size * 0.72, height: 1),
        ),
      );
    }
    final composition = _composition;
    if (composition == null) {
      return const SizedBox.expand();
    }
    return Lottie(
      composition: composition,
      controller: _lottieController,
      width: widget.size,
      height: widget.size,
      fit: BoxFit.contain,
      repeat: false,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/chat/animated_emoji_message.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/chat/chat_lottie_emoji.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 165. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:lottie/lottie.dart';
- import 'package:mu_study/core/chat/animated_emoji_registry.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/chat/chat_lottie_emoji.dart**

```
import 'package:flutter/material.dart';
import 'package:lottie/lottie.dart';
import 'package:mu_study/core/chat/animated_emoji_registry.dart';

class ChatLottieEmoji extends StatefulWidget {
  const ChatLottieEmoji({
    super.key,
    required this.emoji,
    this.size = 22,
    this.playOnTap = true,
    this.autoplay = true,
    this.repeat = false,
  });

  final String emoji;
  final double size;
  final bool playOnTap;
  final bool autoplay;

  final bool repeat;

  @override
  State<ChatLottieEmoji> createState() => _ChatLottieEmojiState();
}

class _ChatLottieEmojiState extends State<ChatLottieEmoji>
    with SingleTickerProviderStateMixin {
  LottieComposition? _composition;
  bool _failed = false;
  late final AnimationController _controller;
  String? _assetPath;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this);
    _controller.addStatusListener(_onStatus);
    _assetPath = animatedEmojiAssetPath(widget.emoji);
    _load();
  }

  @override
  void didUpdateWidget(covariant ChatLottieEmoji oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.emoji != widget.emoji) {
```

```
    _composition = null;
    _failed = false;
    _controller
      ..stop()
      ..reset();
    _assetPath = animatedEmojiAssetPath(widget.emoji);
    _load();
  }
}

void _onStatus(AnimationStatus status) {
  if (widget.repeat) return;
  if (status != AnimationStatus.completed) return;
  _holdLastVisibleFrame();
}

void _holdLastVisibleFrame() {
  final composition = _composition;
  if (composition == null || !mounted) return;
  final span = composition.endFrame - composition.startFrame;
  if (span <= 0) return;
  const epsilonFrames = 0.5;
  final hold = ((span - epsilonFrames) / span).clamp(0.0, 0.999);
  if (_controller.value != hold) _controller.value = hold;
}

Future<void> _load() async {
  final path = _assetPath;
  if (path == null) {
    if (mounted) setState(() => _failed = true);
    return;
  }
  try {
    final composition = await AssetLottie(path).load();
    if (!mounted) return;
    setState(() {
      _composition = composition;
      _failed = false;
      _controller.duration = composition.duration;
      _controller.value = 0;
    });
    if (widget.autoplay) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!mounted || _composition == null) return;
        _startPlayback();
      });
    } else if (!widget.repeat) {
      _holdLastVisibleFrame();
    }
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _failed = true;
      _composition = null;
    });
  }
}

void _startPlayback() {
  if (_composition == null) return;
  if (widget.repeat) {
    _controller.repeat(min: 0, max: _loopMax);
  } else {
    _controller.forward(from: 0);
  }
}

double get _loopMax {
  final composition = _composition;
  if (composition == null) return 0.999;
  final span = composition.endFrame - composition.startFrame;
  if (span <= 0) return 0.999;
  return ((span - 0.5) / span).clamp(0.0, 0.999);
```

```
  }

  void _replay() => _startPlayback();

  @override
  void dispose() {
    _controller.removeStatusListener(_onStatus);
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final child = SizedBox(
      width: widget.size,
      height: widget.size,
      child: _buildBody(),
    );
    if (!widget.playOnTap || _composition == null) return child;
    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTap: _replay,
      child: child,
    );
  }

  Widget _buildBody() {
    if (_failed || _assetPath == null) {
      return Center(
        child: Text(
          widget.emoji,
          style: TextStyle(fontSize: widget.size * 0.85, height: 1),
        ),
      );
    }
    final composition = _composition;
    if (composition == null) return const SizedBox.expand();
    return Lottie(
      composition: composition,
      controller: _controller,
      width: widget.size,
      height: widget.size,
      fit: BoxFit.contain,
      repeat: false,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/chat/chat_lottie_emoji.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/chat/chat_media_gallery.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 714. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'dart:io';
- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/foundation.dart';
- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:mu_study/core/services/chat_media_cache_service.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:video_player/video_player.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/chat/chat_media_gallery.dart**

```
import 'dart:async';
import 'dart:io';

import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:video_player/video_player.dart';

const _speedOptions = <double>[0.5, 0.75, 1.0, 1.25, 1.5, 2.0];
const _dismissDistance = 120.0;

enum ChatMediaKind { image, video }

class ChatMediaItem {
  const ChatMediaItem({
    required this.id,
    required this.url,
    required this.kind,
  });

  final String id;
  final String url;
  final ChatMediaKind kind;
}

List<ChatMediaItem> chatMediaItemsFromMessages(Iterable<ChatMessage> messages) {
  final items = <ChatMediaItem>[];
  for (final m in messages) {
    if (m.isDeleted) continue;
    final url = m.mediaUrl;
```

```
    if (url == null || url.isEmpty) continue;
    final kind = switch (m.messageType) {
      ChatMessageType.image => ChatMediaKind.image,
      ChatMessageType.video => ChatMediaKind.video,
      _ => null,
    };
    if (kind == null) continue;
    items.add(ChatMediaItem(id: m.id, url: url, kind: kind));
  }
  return items;
}

Future<void> showChatMediaGallery(
  BuildContext context, {
  required List<ChatMediaItem> items,
  required int initialIndex,
}) {
  assert(items.isNotEmpty);
  final index = initialIndex.clamp(0, items.length - 1);
  return showGeneralDialog<void>(
    context: context,
    barrierDismissible: true,
    barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,
    barrierColor: Colors.black,
    transitionDuration: const Duration(milliseconds: 220),
    pageBuilder: (context, animation, secondaryAnimation) {
      return ChatMediaGalleryPage(items: items, initialIndex: index);
    },
    transitionBuilder: (context, animation, secondaryAnimation, child) {
      return FadeTransition(
        opacity: animation,
        child: ScaleTransition(
          scale: Tween<double>(begin: 0.96, end: 1).animate(
            CurvedAnimation(parent: animation, curve: Curves.easeOutCubic),
          ),
          child: child,
        ),
      );
    },
  );
}

Future<void> showChatVideoPlayer(BuildContext context, {required String url}) {
  return showChatMediaGallery(
    context,
    items: [ChatMediaItem(id: url, url: url, kind: ChatMediaKind.video)],
    initialIndex: 0,
  );
}

class ChatMediaGalleryPage extends StatefulWidget {
  const ChatMediaGalleryPage({
    super.key,
    required this.items,
    required this.initialIndex,
  });

  final List<ChatMediaItem> items;
  final int initialIndex;

  @override
  State<ChatMediaGalleryPage> createState() => _ChatMediaGalleryPageState();
}

class _ChatMediaGalleryPageState extends State<ChatMediaGalleryPage> {
  late final PageController _pageController;
  late int _index;
  double _dragY = 0;
  bool _dragging = false;

  @override
  void initState() {
    super.initState();
```

```
    _index = widget.initialIndex;
    _pageController = PageController(initialPage: _index);
    SystemChrome.setEnabledSystemUIMode(SystemUiMode.immersiveSticky);
  }

  @override
  void dispose() {
    _pageController.dispose();
    SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
    super.dispose();
  }

  void _onVerticalDragUpdate(DragUpdateDetails details) {
    final dy = details.delta.dy;
    setState(() {
      _dragging = true;
      _dragY = (_dragY + dy).clamp(0.0, 400.0);
    });
  }

  void _onVerticalDragEnd(DragEndDetails details) {
    final fling = details.primaryVelocity ?? 0;
    final shouldClose = _dragY > _dismissDistance || fling > 900;
    if (shouldClose) {
      Navigator.of(context).maybePop();
      return;
    }
    setState(() {
      _dragging = false;
      _dragY = 0;
    });
  }

  void _onVerticalDragCancel() {
    setState(() {
      _dragging = false;
      _dragY = 0;
    });
  }

  @override
  Widget build(BuildContext context) {
    final progress = (_dragY / _dismissDistance).clamp(0.0, 1.0);
    final bgOpacity = (1.0 - progress * 0.55).clamp(0.35, 1.0);
    final scale = (1.0 - progress * 0.08).clamp(0.9, 1.0);

    return Material(
      color: Colors.black.withValues(alpha: bgOpacity),
      child: GestureDetector(
        onVerticalDragUpdate: _onVerticalDragUpdate,
        onVerticalDragEnd: _onVerticalDragEnd,
        onVerticalDragCancel: _onVerticalDragCancel,
        child: AnimatedContainer(
          duration: _dragging
              ? Duration.zero
              : const Duration(milliseconds: 180),
          curve: Curves.easeOutCubic,
          transform: Matrix4.identity()
            ..translateByDouble(0, _dragging ? _dragY : 0, 0, 1)
            ..scaleByDouble(scale, scale, 1, 1),
          transformAlignment: Alignment.center,
          child: SafeArea(
            child: Stack(
              fit: StackFit.expand,
              children: [
                PageView.builder(
                  controller: _pageController,
                  itemCount: widget.items.length,
                  onPageChanged: (i) => setState(() => _index = i),
                  itemBuilder: (context, i) {
                    final item = widget.items[i];
                    final active = i == _index && _dragY < 24;
                    return switch (item.kind) {
```

```
                      ChatMediaKind.image => _GalleryImagePage(url: item.url),
                      ChatMediaKind.video => _GalleryVideoPage(
                        url: item.url,
                        isActive: active,
                      ),
                    };
                },
              ),
              Positioned(
                top: 4,
                left: 4,
                child: IconButton(
                  onPressed: () => Navigator.of(context).maybePop(),
                  icon: const Icon(Icons.close_rounded, color: Colors.white),
                ),
              ),
              if (widget.items.length > 1)
                Positioned(
                  top: 14,
                  right: 16,
                  child: Text(
                    '${_index + 1} / ${widget.items.length}',
                    style: const TextStyle(
                      color: Colors.white70,
                      fontWeight: FontWeight.w600,
                      fontSize: 13,
                    ),
                  ),
                ),
            ],
          ),
        ),
      ),
    );
  }
}

class _GalleryImagePage extends StatefulWidget {
  const _GalleryImagePage({required this.url});
  final String url;

  @override
  State<_GalleryImagePage> createState() => _GalleryImagePageState();
}

class _GalleryImagePageState extends State<_GalleryImagePage>
    with AutomaticKeepAliveClientMixin {
  File? _file;
  bool _loading = true;

  @override
  bool get wantKeepAlive => true;

  @override
  void initState() {
    super.initState();
    _resolve();
  }

  Future<void> _resolve() async {
    if (!kIsWeb && ChatMediaCacheService.instance.isSupported) {
      final file = await ChatMediaCacheService.instance.getOrFetch(
        widget.url,
        preferredExtension: ChatMediaCacheService.extensionForUrl(
          widget.url,
          fallback: 'jpg',
        ),
      );
      if (!mounted) return;
      setState(() {
        _file = file;
        _loading = false;
```

```
      });
      return;
    }
    if (mounted) setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    super.build(context);
    if (_loading) {
      return const Center(
        child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
      );
    }
    final child = _file != null
        ? Image.file(_file!, fit: BoxFit.contain)
        : CachedNetworkImage(
            imageUrl: widget.url,
            fit: BoxFit.contain,
            placeholder: (_, _) => const Center(
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: Colors.white,
              ),
            ),
            errorWidget: (_, _, _) => const Icon(
              Icons.broken_image_outlined,
              color: Colors.white54,
              size: 48,
            ),
          );
    return InteractiveViewer(
      minScale: 1,
      maxScale: 4,
      child: Center(child: child),
    );
  }
}

class _GalleryVideoPage extends StatefulWidget {
  const _GalleryVideoPage({required this.url, required this.isActive});

  final String url;
  final bool isActive;

  @override
  State<_GalleryVideoPage> createState() => _GalleryVideoPageState();
}

class _GalleryVideoPageState extends State<_GalleryVideoPage>
    with AutomaticKeepAliveClientMixin {
  VideoPlayerController? _controller;
  bool _ready = false;
  bool _failed = false;
  bool _controlsVisible = true;
  double _speed = 1.0;
  Timer? _hideControls;

  @override
  bool get wantKeepAlive => true;

  @override
  void initState() {
    super.initState();
    _open();
  }

  @override
  void didUpdateWidget(covariant _GalleryVideoPage oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.isActive != widget.isActive) {
      _syncPlayback();
    }
```

```
}

Future<void> _open() async {
  VideoPlayerController controller;
  if (!kIsWeb && ChatMediaCacheService.instance.isSupported) {
    final file = await ChatMediaCacheService.instance.getOrFetch(
      widget.url,
      preferredExtension: ChatMediaCacheService.extensionForUrl(
        widget.url,
        fallback: 'mp4',
      ),
    );
    if (!mounted) return;
    controller = file != null
        ? VideoPlayerController.file(file)
        : VideoPlayerController.networkUrl(Uri.parse(widget.url));
  } else {
    controller = VideoPlayerController.networkUrl(Uri.parse(widget.url));
  }

  _controller = controller;
  controller.addListener(_onTick);
  try {
    await controller.initialize();
    await controller.setPlaybackSpeed(_speed);
    if (!mounted) return;
    setState(() => _ready = true);
    await _syncPlayback();
  } catch (_) {
    if (mounted) setState(() => _failed = true);
  }
}

Future<void> _syncPlayback() async {
  final c = _controller;
  if (c == null || !c.value.isInitialized) return;
  if (widget.isActive) {
    await c.play();
    _scheduleHideControls();
  } else {
    await c.pause();
    _hideControls?.cancel();
    if (mounted) setState(() => _controlsVisible = true);
  }
}

void _onTick() {
  if (mounted) setState(() {});
}

void _scheduleHideControls() {
  _hideControls?.cancel();
  if (!(_controller?.value.isPlaying ?? false)) return;
  _hideControls = Timer(const Duration(seconds: 3), () {
    if (mounted && (_controller?.value.isPlaying ?? false)) {
      setState(() => _controlsVisible = false);
    }
  });
}

void _toggleControls() {
  setState(() => _controlsVisible = !_controlsVisible);
  if (_controlsVisible) _scheduleHideControls();
}

Future<void> _togglePlay() async {
  final c = _controller;
  if (c == null || !c.value.isInitialized) return;
  if (c.value.isPlaying) {
    await c.pause();
    _hideControls?.cancel();
    setState(() => _controlsVisible = true);
  } else {
```

```
    await c.play();
    _scheduleHideControls();
  }
}

Future<void> _seekBy(Duration delta) async {
  final c = _controller;
  if (c == null || !c.value.isInitialized) return;
  final total = c.value.duration;
  var next = c.value.position + delta;
  if (next < Duration.zero) next = Duration.zero;
  if (next > total) next = total;
  await c.seekTo(next);
  _scheduleHideControls();
}

Future<void> _setSpeed(double speed) async {
  final c = _controller;
  if (c == null) return;
  setState(() => _speed = speed);
  await c.setPlaybackSpeed(speed);
  _scheduleHideControls();
}

Future<void> _showSpeedSheet() async {
  _hideControls?.cancel();
  setState(() => _controlsVisible = true);
  final selected = await showModalBottomSheet<double>(
    context: context,
    backgroundColor: const Color(0xFF1C1C1E),
    builder: (ctx) {
      final l10n = context.l10n;
      return SafeArea(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 16, 20, 8),
              child: Align(
                alignment: Alignment.centerLeft,
                child: Text(
                  l10n.chatVideoPlaybackSpeed,
                  style: const TextStyle(
                    color: Colors.white,
                    fontWeight: FontWeight.w700,
                    fontSize: 16,
                  ),
                ),
              ),
            ),
            for (final speed in _speedOptions)
              ListTile(
                selected: speed == _speed,
                selectedColor: Colors.white,
                selectedTileColor: Colors.white12,
                title: Text(
                  speed == 1.0
                      ? l10n.chatVideoSpeedNormal
                      : l10n.chatVideoSpeedValue(speed),
                  style: const TextStyle(color: Colors.white),
                ),
                trailing: speed == _speed
                    ? const Icon(Icons.check, color: Colors.white)
                    : null,
                onTap: () => Navigator.pop(ctx, speed),
              ),
            const SizedBox(height: 8),
          ],
        ),
      );
    },
  );
  if (selected != null) await _setSpeed(selected);
```

```
    _scheduleHideControls();
  }

  String _fmt(Duration d) {
    final total = d.inSeconds;
    final m = (total ~/ 60).toString().padLeft(2, '0');
    final s = (total % 60).toString().padLeft(2, '0');
```

**lib/components/chat/chat_media_gallery.dart — continued (2/2)**

```
    return '$m:$s';
  }

  @override
  void dispose() {
    _hideControls?.cancel();
    _controller?.removeListener(_onTick);
    _controller?.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    super.build(context);
    final l10n = context.l10n;
    final c = _controller;

    return Stack(
      fit: StackFit.expand,
      children: [
        if (_failed)
          Center(
            child: Text(
              l10n.chatVideoCouldNotPlay,
              style: const TextStyle(color: Colors.white70),
            ),
          )
        else if (!_ready || c == null)
          const Center(
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: Colors.white,
            ),
          )
        else
          GestureDetector(
            behavior: HitTestBehavior.opaque,
            onTap: _toggleControls,
            child: Center(
              child: AspectRatio(
                aspectRatio: c.value.aspectRatio == 0
                    ? 16 / 9
                    : c.value.aspectRatio,
                child: VideoPlayer(c),
              ),
            ),
          ),
        if (_ready && c != null)
          Positioned(
            left: 0,
            right: 0,
            bottom: 0,
            child: AnimatedOpacity(
              opacity: _controlsVisible ? 1 : 0,
              duration: const Duration(milliseconds: 180),
              child: IgnorePointer(
                ignoring: !_controlsVisible,
                child: _ControlsBar(
                  controller: c,
                  onSeekBack: () => _seekBy(const Duration(seconds: -10)),
                  onSeekForward: () => _seekBy(const Duration(seconds: 10)),
                  onTogglePlay: _togglePlay,
                  onSpeedTap: _showSpeedSheet,
```

```
                  onSeek: (value) async {
                    final total = c.value.duration.inMilliseconds;
                    await c.seekTo(
                      Duration(milliseconds: (value * total).round()),
                    );
                    _scheduleHideControls();
                  },
                  format: _fmt,
                  speedLabel: _speed == 1.0
                      ? l10n.chatVideoSpeedNormal
                      : l10n.chatVideoSpeedValue(_speed),
                  rewindLabel: l10n.chatVideoRewind10,
                  forwardLabel: l10n.chatVideoForward10,
                ),
              ),
            ),
          ),
        ),
      ],
    );
  }
}

class _ControlsBar extends StatelessWidget {
  const _ControlsBar({
    required this.controller,
    required this.onSeekBack,
    required this.onSeekForward,
    required this.onTogglePlay,
    required this.onSpeedTap,
    required this.onSeek,
    required this.format,
    required this.speedLabel,
    required this.rewindLabel,
    required this.forwardLabel,
  });

  final VideoPlayerController controller;
  final VoidCallback onSeekBack;
  final VoidCallback onSeekForward;
  final VoidCallback onTogglePlay;
  final VoidCallback onSpeedTap;
  final ValueChanged<double> onSeek;
  final String Function(Duration) format;
  final String speedLabel;
  final String rewindLabel;
  final String forwardLabel;

  @override
  Widget build(BuildContext context) {
    final value = controller.value;
    final duration = value.duration;
    final position = value.position;
    final progress = duration.inMilliseconds == 0
        ? 0.0
        : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);

    return Container(
      padding: const EdgeInsets.fromLTRB(12, 28, 12, 16),
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Colors.transparent, Colors.black87],
        ),
      ),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Row(
            children: [
              Text(
                format(position),
                style: const TextStyle(color: Colors.white70, fontSize: 12),
```

```
              ),
              Expanded(
                child: SliderTheme(
                  data: SliderTheme.of(context).copyWith(
                    trackHeight: 2.5,
                    thumbShape: const RoundSliderThumbShape(
                      enabledThumbRadius: 7,
                    ),
                    overlayShape: const RoundSliderOverlayShape(
                      overlayRadius: 14,
                    ),
                    activeTrackColor: Colors.white,
                    inactiveTrackColor: Colors.white24,
                    thumbColor: Colors.white,
                  ),
                  child: Slider(value: progress, onChanged: onSeek),
                ),
              ),
              Text(
                format(duration),
                style: const TextStyle(color: Colors.white70, fontSize: 12),
              ),
            ],
          ),
          const SizedBox(height: 4),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              _ControlButton(
                onPressed: onSeekBack,
                icon: Icons.replay_10_rounded,
                label: rewindLabel,
              ),
              IconButton(
                onPressed: onTogglePlay,
                iconSize: 56,
                icon: Icon(
                  value.isPlaying
                      ? Icons.pause_circle_filled_rounded
                      : Icons.play_circle_filled_rounded,
                  color: Colors.white,
                ),
              ),
              _ControlButton(
                onPressed: onSeekForward,
                icon: Icons.forward_10_rounded,
                label: forwardLabel,
              ),
              TextButton(
                onPressed: onSpeedTap,
                style: TextButton.styleFrom(
                  foregroundColor: Colors.white,
                  padding: const EdgeInsets.symmetric(horizontal: 10),
                ),
                child: Text(
                  speedLabel,
                  style: const TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 14,
                  ),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

class _ControlButton extends StatelessWidget {
  const _ControlButton({
    required this.onPressed,
```

```
    required this.icon,
    required this.label,
  });

  final VoidCallback onPressed;
  final IconData icon;
  final String label;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        IconButton(
          onPressed: onPressed,
          icon: Icon(icon, color: Colors.white, size: 32),
          tooltip: label,
        ),
        Text(
          label,
          style: const TextStyle(color: Colors.white70, fontSize: 11),
        ),
      ],
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/chat/chat_media_gallery.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/decor/paper_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 510. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/decor/paper_decor.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

class PaperAssets {
  static const _dir = 'assets/images/nature-paper';

  static const avatarSilhouette = '$_dir/avatar-silhouette.png';
  static const beigeLeaf = '$_dir/beige-leaf.png';
  static const bookshelfSingle = '$_dir/bookshelf-single-shelf.png';
  static const bookshelfDouble = '$_dir/bookshelf-two-shelves.png';
  static const brainVerified = '$_dir/brain-verified.png';
  static const browserDashboard = '$_dir/browser-dashboard.png';
  static const calendar = '$_dir/calendar.png';
  static const calendarWithClock = '$_dir/calendar-with-clock.png';
  static const cardboardStar = '$_dir/cardboard-star.png';
  static const chatBubbles = '$_dir/chat-bubbles.png';
  static const clipboardChecklist = '$_dir/clipboard-checklist.png';
  static const clipboardProfile = '$_dir/clipboard-profile.png';
  static const clipboardWithPencil = '$_dir/clipboard-with-pencil.png';
  static const closeButton = '$_dir/close-button.png';
  static const creamBird = '$_dir/cream-bird.png';
  static const creamPaperGreenTape = '$_dir/cream-paper-green-tape.png';
  static const divisionButton = '$_dir/division-button.png';
  static const documentLayout = '$_dir/document-layout.png';
  static const downloadInbox = '$_dir/download-inbox.png';
  static const dustyRoseFlower = '$_dir/dusty-rose-flower.png';
  static const envelopeSuccess = '$_dir/envelope-success.png';
  static const flatCloud = '$_dir/flat-cloud.png';
  static const grayCloud = '$_dir/gray-cloud.png';
  static const groupChat = '$_dir/group-chat.png';
  static const hangingGarland = '$_dir/hanging-garland.png';
  static const homeHouse = '$_dir/home-house.png';
  static const kraftPaperStrip = '$_dir/kraft-paper-strip.png';
  static const landscapeHills = '$_dir/landscape-hills-footer.png';
  static const linedPaperPencil = '$_dir/lined-paper-pencil.png';
  static const magnifyingGlass = '$_dir/magnifying-glass.png';
  static const manilaFolder = '$_dir/manila-folder.png';
  static const marketStall = '$_dir/market-stall.png';
  static const minusButton = '$_dir/minus-button.png';
  static const notificationBell = '$_dir/notification-bell-alert.png';
  static const openBook = '$_dir/open-book.png';
  static const paperFunnel = '$_dir/paper-funnel.png';
```

```
  static const puzzlePieces = '$_dir/puzzle-pieces.png';
  static const roundedTanStar = '$_dir/rounded-tan-star.png';
  static const sageScallopedFrame = '$_dir/sage-scalloped-frame.png';
  static const salmonFlower = '$_dir/salmon-flower.png';
  static const settingsGear = '$_dir/settings-gear.png';
  static const singleTree = '$_dir/single-tree.png';
  static const spiralNotebook = '$_dir/spiral-notebook.png';
  static const stickyNotesPair = '$_dir/sticky-notes-pair.png';
  static const stripedWashiTape = '$_dir/striped-washi-tape.png';
  static const tanNoteGreenTab = '$_dir/tan-note-green-tab.png';
  static const tanSeed = '$_dir/tan-seed.png';
  static const teacherChalkboard = '$_dir/teacher-chalkboard.png';
  static const tealBird = '$_dir/teal-bird.png';
  static const terracottaScallopedFrame =
      '$_dir/terracotta-scalloped-frame.png';
  static const texturedTanStar = '$_dir/textured-tan-star.png';
  static const tornPaperScrap = '$_dir/torn-paper-scrap.png';
  static const twoTrees = '$_dir/two-trees.png';
  static const wallClock = '$_dir/wall-clock.png';
  static const whiteCloud = '$_dir/white-cloud.png';
  static const whiteDaisy = '$_dir/white-daisy.png';
  static const yellowPencil = '$_dir/yellow-pencil.png';
}

bool isNaturePaper(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.isNaturePaper ?? false;

bool isSoftNeumorphism(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;

bool isMonochromeWorkspace(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;

String? paperAssetForIcon(IconData? icon) {
  if (icon == null) return null;
  if (icon == Icons.inbox_outlined || icon == Icons.inbox) {
    return PaperAssets.downloadInbox;
  }
  if (icon == Icons.grade_outlined ||
      icon == Icons.grade ||
      icon == Icons.star_outline ||
      icon == Icons.star ||
      icon == Icons.star_rounded) {
    return PaperAssets.cardboardStar;
  }
  if (icon == Icons.assignment_outlined ||
      icon == Icons.assignment ||
      icon == Icons.task_outlined ||
      icon == Icons.assignment_turned_in_outlined) {
    return PaperAssets.spiralNotebook;
  }
  if (icon == Icons.task_alt_rounded ||
      icon == Icons.edit_outlined ||
      icon == Icons.edit ||
      icon == Icons.add ||
      icon == Icons.add_rounded) {
    return PaperAssets.yellowPencil;
  }
  if (icon == Icons.inventory_2_rounded || icon == Icons.inventory_2_outlined) {
    return PaperAssets.manilaFolder;
  }
  if (icon == Icons.chat_bubble_outline_rounded ||
      icon == Icons.chat_outlined ||
      icon == Icons.chat ||
      icon == Icons.forum_outlined ||
      icon == Icons.chat_bubble_outline) {
    return PaperAssets.chatBubbles;
  }
  if (icon == Icons.campaign_rounded || icon == Icons.campaign_outlined) {
    return PaperAssets.envelopeSuccess;
  }
  if (icon == Icons.event_available_outlined ||
      icon == Icons.event_available_rounded ||
```

```
    icon == Icons.event_outlined) {
  return PaperAssets.clipboardChecklist;
}
if (icon == Icons.calendar_today_outlined ||
    icon == Icons.calendar_month_outlined ||
    icon == Icons.calendar_today) {
  return PaperAssets.calendar;
}
if (icon == Icons.people_outline ||
    icon == Icons.groups_outlined ||
    icon == Icons.groups ||
    icon == Icons.people_alt_outlined) {
  return PaperAssets.groupChat;
}
if (icon == Icons.notifications_outlined ||
    icon == Icons.notifications_none_rounded ||
    icon == Icons.notifications_active_outlined ||
    icon == Icons.notifications) {
  return PaperAssets.notificationBell;
}
if (icon == Icons.search || icon == Icons.search_rounded) {
  return PaperAssets.magnifyingGlass;
}
if (icon == Icons.menu_book ||
    icon == Icons.subject_outlined ||
    icon == Icons.language_rounded ||
    icon == Icons.language) {
  return PaperAssets.openBook;
}
if (icon == Icons.calculate || icon == Icons.calculate_outlined) {
  return PaperAssets.divisionButton;
}
if (icon == Icons.badge_outlined ||
    icon == Icons.person_outline ||
    icon == Icons.person ||
    icon == Icons.person_rounded ||
    icon == Icons.account_circle_outlined) {
  return PaperAssets.clipboardProfile;
}
if (icon == Icons.settings_outlined ||
    icon == Icons.settings ||
    icon == Icons.tune_rounded) {
  return PaperAssets.settingsGear;
}
if (icon == Icons.home_outlined || icon == Icons.home) {
  return PaperAssets.homeHouse;
}
if (icon == Icons.dashboard_outlined ||
    icon == Icons.dashboard ||
    icon == Icons.insights_outlined ||
    icon == Icons.analytics_outlined ||
    icon == Icons.analytics_rounded) {
  return PaperAssets.browserDashboard;
}
if (icon == Icons.warning_amber_rounded ||
    icon == Icons.cleaning_services_outlined ||
    icon == Icons.error_outline) {
  return PaperAssets.tornPaperScrap;
}
if (icon == Icons.schedule ||
    icon == Icons.access_time_rounded ||
    icon == Icons.timer_outlined) {
  return PaperAssets.wallClock;
}
if (icon == Icons.extension_outlined) return PaperAssets.puzzlePieces;
if (icon == Icons.school || icon == Icons.school_outlined) {
  return PaperAssets.teacherChalkboard;
}
if (icon == Icons.self_improvement_outlined ||
    icon == Icons.self_improvement) {
  return PaperAssets.openBook;
}
if (icon == Icons.folder_outlined || icon == Icons.folder) {
```

```
    return PaperAssets.manilaFolder;
  }
  if (icon == Icons.psychology_outlined || icon == Icons.psychology) {
    return PaperAssets.brainVerified;
  }
  return null;
}

double paperControlRadius(BuildContext context) {
  if (isNaturePaper(context)) return 18;
  if (isMonochromeWorkspace(context)) return 6;
  return 14;
}

BoxDecoration paperControlDecoration(
  BuildContext context, {
  Color? color,
  double? radius,
}) {
  final r = radius ?? paperControlRadius(context);
  final nature = isNaturePaper(context);
  return BoxDecoration(
    color: color ?? AppColors.controlSurface(context),
    borderRadius: BorderRadius.circular(r),
    border: Border.all(
      color: nature
          ? AppColors.inkAlpha(context, 0.12)
          : AppColors.inkAlpha(context, 0.08),
    ),
    boxShadow: nature && Theme.of(context).brightness == Brightness.light
        ? [
            BoxShadow(
              color: const Color(0xFF8A6A4A).withValues(alpha: 0.08),
              blurRadius: 10,
              offset: const Offset(0, 3),
            ),
          ]
        : null,
  );
}

class PaperDecor extends StatelessWidget {
  const PaperDecor(
    this.asset, {
    super.key,
    this.size,
    this.height,
    this.angle = 0,
    this.opacity = 1,
    this.fit = BoxFit.contain,
  });

  final String asset;
  final double? size;
  final double? height;
  final double angle;
  final double opacity;
  final BoxFit fit;

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.shrink();
    Widget img = Image.asset(
      asset,
      width: size,
      height: height ?? size,
      fit: fit,
      opacity: AlwaysStoppedAnimation(opacity),
      filterQuality: FilterQuality.medium,
    );
    if (angle != 0) img = Transform.rotate(angle: angle, child: img);
    return ExcludeSemantics(child: img);
  }
```

```
}

class PaperGlyph extends StatelessWidget {
  const PaperGlyph({
    super.key,
    required this.icon,
    this.asset,
    this.size = 22,
    this.color,
    this.opacity = 1,
    this.retintInDark = true,
  });

  final IconData icon;

  final String? asset;
  final double size;
  final Color? color;
  final double opacity;

  /// When false, keep original pack colors (e.g. shell bottom nav).
  final bool retintInDark;

  @override
  Widget build(BuildContext context) {
    final resolved = asset ?? paperAssetForIcon(icon);
    if (!isNaturePaper(context) || resolved == null) {
      return Icon(
        icon,
        size: size,
        color: opacity == 1 ? color : color?.withValues(alpha: opacity),
      );
    }

    Widget img = Image.asset(
      resolved,
      width: size,
      height: size,
      fit: BoxFit.contain,
      filterQuality: FilterQuality.medium,
    );

    // Pack sprites are painted for cream paper; retint on night cards so
    // dark brown assets (stars, notebooks) stay readable.
    final isDark = Theme.of(context).brightness == Brightness.dark;
    if (isDark && retintInDark) {
      final provided = color;
      final tint =
          (provided != null && provided.computeLuminance() > 0.38)
          ? provided
          : NaturePaperColors.moonBeige;
      img = ColorFiltered(
        colorFilter: ColorFilter.mode(tint, BlendMode.srcIn),
        child: img,
      );
    }

    if (opacity < 1) {
      img = Opacity(opacity: opacity, child: img);
    }

    return ExcludeSemantics(child: img);
  }
}

class PaperEmptyIllustration extends StatelessWidget {
  const PaperEmptyIllustration({super.key, required this.asset});

  final String asset;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
```

```
      width: 128,
      height: 104,
      child: Stack(
        alignment: Alignment.center,
        children: [
          Positioned(
            left: 0,
            top: 10,
            child: Transform.rotate(
              angle: -0.5,
              child: Image.asset(
                PaperAssets.beigeLeaf,
                width: 26,
                opacity: const AlwaysStoppedAnimation(0.8),
              ),
            ),
          ),
          Positioned(
            right: 4,
            top: 0,
            child: Transform.rotate(
              angle: 0.35,
              child: Image.asset(
                PaperAssets.roundedTanStar,
                width: 20,
                opacity: const AlwaysStoppedAnimation(0.85),
              ),
            ),
          ),
          Positioned(
            right: 0,
            bottom: 2,
            child: Image.asset(
              PaperAssets.whiteDaisy,
              width: 22,
              opacity: const AlwaysStoppedAnimation(0.9),
            ),
          ),
          Positioned(
            left: 8,
            bottom: 0,
            child: Image.asset(
              PaperAssets.tanSeed,
              width: 14,
              opacity: const AlwaysStoppedAnimation(0.7),
            ),
          ),
          Image.asset(
            asset,
            width: 76,
            height: 76,
            fit: BoxFit.contain,
            filterQuality: FilterQuality.medium,
          ),
        ],
      ),
    );
  }
}

class PaperTaped extends StatelessWidget {
  const PaperTaped({
    super.key,
    required this.child,
    this.angle = -0.07,
    this.tapeWidth = 88,
    this.alignment = 0,
  });

  final Widget child;
  final double angle;
  final double tapeWidth;
```

```
  final double alignment;

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return child;
    return Stack(
      clipBehavior: Clip.none,
      children: [
        child,
        Positioned(
          top: 0,
          left: 0,
          right: 0,
          child: Align(
            alignment: Alignment(alignment, 0),
            child: ExcludeSemantics(
              child: Transform.rotate(
                angle: angle,
                child: Image.asset(
                  PaperAssets.stripedWashiTape,
                  width: tapeWidth,
                  opacity: const AlwaysStoppedAnimation(0.92),
```

**lib/components/decor/paper_decor.dart — continued (2/2)**

```
                  filterQuality: FilterQuality.medium,
                ),
              ),
            ),
          ),
        ),
      ],
    );
  }
}

class PaperDivider extends StatelessWidget {
  const PaperDivider({super.key, this.width = 180});

  final double width;

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.shrink();
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 10),
      child: Center(
        child: ExcludeSemantics(
          child: Image.asset(
            PaperAssets.kraftPaperStrip,
            width: width,
            height: 14,
            fit: BoxFit.fill,
            opacity: const AlwaysStoppedAnimation(0.75),
          ),
        ),
      ),
    );
  }
}

class PaperScallopedAvatar extends StatelessWidget {
  const PaperScallopedAvatar({
    super.key,
    required this.child,
    required this.avatarSize,
  });

  final Widget child;

  final double avatarSize;

  @override
```

```
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return child;
    final frameSize = avatarSize * 1.30;
    return SizedBox(
      width: frameSize,
      height: frameSize,
      child: Stack(
        alignment: Alignment.center,
        children: [
          ExcludeSemantics(
            child: Image.asset(
              PaperAssets.sageScallopedFrame,
              width: frameSize,
              height: frameSize,
              fit: BoxFit.contain,
              filterQuality: FilterQuality.medium,
            ),
          ),
          child,
        ],
      ),
    );
  }
}

String? paperStarForPercent(double? percent) {
  if (percent == null) return null;
  if (percent >= 85) return PaperAssets.cardboardStar;
  if (percent >= 70) return PaperAssets.texturedTanStar;
  return null;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/decor/paper_decor.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/decor/sidebar_grove.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 352. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math' as math;
- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/decor/sidebar_grove.dart**

```
import 'dart:math' as math;

import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';

class _CloudSpec {
  const _CloudSpec({
    required this.asset,
    required this.y,
    required this.height,
    required this.speed,
    required this.phase,
    required this.opacity,
  });

  final String asset;
  final double y;
  final double height;
  final double speed;
  final double phase;
  final double opacity;
}

class SidebarSkyClouds extends StatefulWidget {
  const SidebarSkyClouds({super.key});

  @override
  State<SidebarSkyClouds> createState() => _SidebarSkyCloudsState();
}

class _SidebarSkyCloudsState extends State<SidebarSkyClouds>
    with SingleTickerProviderStateMixin {
  static const _clouds = [
    _CloudSpec(
      asset: PaperAssets.whiteCloud,
      y: 0.12,
      height: 28,
      speed: 0.55,
      phase: 0.05,
      opacity: 0.55,
    ),
    _CloudSpec(
      asset: PaperAssets.flatCloud,
      y: 0.38,
      height: 22,
```

```
    speed: 0.35,
    phase: 0.4,
    opacity: 0.4,
  ),
  _CloudSpec(
    asset: PaperAssets.grayCloud,
    y: 0.62,
    height: 18,
    speed: 0.7,
    phase: 0.72,
    opacity: 0.28,
  ),
  _CloudSpec(
    asset: PaperAssets.whiteCloud,
    y: 0.82,
    height: 16,
    speed: 0.45,
    phase: 0.2,
    opacity: 0.35,
  ),
];

late final AnimationController _controller = AnimationController(
  vsync: this,
  duration: const Duration(seconds: 22),
)..repeat();

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  if (!isNaturePaper(context)) return const SizedBox.expand();
  if (Theme.of(context).brightness == Brightness.dark) {
    return const SizedBox.expand();
  }

  return IgnorePointer(
    child: RepaintBoundary(
      child: ClipRect(
        child: LayoutBuilder(
          builder: (context, constraints) {
            final w = constraints.maxWidth;
            final h = constraints.maxHeight;
            if (h < 24) return const SizedBox.expand();
            return AnimatedBuilder(
              animation: _controller,
              builder: (context, _) {
                final t = _controller.value;
                return Stack(
                  clipBehavior: Clip.hardEdge,
                  children: [
                    for (final c in _clouds)
                      Positioned(
                        left:
                            w - (((t * c.speed + c.phase) % 1.0) * (w + 100)),
                        top: c.y * h,
                        child: Image.asset(
                          c.asset,
                          height: c.height,
                          opacity: AlwaysStoppedAnimation(c.opacity),
                          filterQuality: FilterQuality.medium,
                        ),
                      ),
                  ],
                );
              },
            );
          },
        ),
```

```
        ),
      ),
    );
  }
}

class SidebarGrove extends StatefulWidget {
  const SidebarGrove({super.key});

  @override
  State<SidebarGrove> createState() => _SidebarGroveState();
}

class _SidebarGroveState extends State<SidebarGrove>
    with SingleTickerProviderStateMixin {
  static const _clouds = [
    _CloudSpec(
      asset: PaperAssets.whiteCloud,
      y: 4,
      height: 24,
      speed: 3.0,
      phase: 0.0,
      opacity: 0.7,
    ),
    _CloudSpec(
      asset: PaperAssets.flatCloud,
      y: 22,
      height: 18,
      speed: 1.8,
      phase: 0.45,
      opacity: 0.45,
    ),
    _CloudSpec(
      asset: PaperAssets.grayCloud,
      y: 10,
      height: 14,
      speed: 2.4,
      phase: 0.78,
      opacity: 0.32,
    ),
  ];

  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 14),
  )..repeat();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Widget _swaying({
    required double t,
    required double phase,
    required Widget child,
  }) {
    final angle = math.sin(t * 2 * math.pi + phase) * 0.022;
    return Transform.rotate(
      angle: angle,
      alignment: Alignment.bottomCenter,
      child: child,
    );
  }

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.shrink();
    final isDark = Theme.of(context).brightness == Brightness.dark;

    return RepaintBoundary(
      child: ClipRect(
```

```
child: SizedBox(
  height: 148,
  width: double.infinity,
  child: LayoutBuilder(
    builder: (context, constraints) {
      final width = constraints.maxWidth;
      return AnimatedBuilder(
        animation: _controller,
        builder: (context, _) {
          final t = _controller.value;
          final travel = width + 90;
          return Stack(
            clipBehavior: Clip.hardEdge,
            children: [
              if (!isDark)
                for (final c in _clouds)
                  Positioned(
                    left:
                        width -
                        (((t * c.speed + c.phase) % 1.0) * travel),
                    top: c.y,
                    child: Image.asset(
                      c.asset,
                      height: c.height,
                      opacity: AlwaysStoppedAnimation(c.opacity),
                      filterQuality: FilterQuality.medium,
                    ),
                  ),
              Positioned(
                left: 0,
                right: 0,
                bottom: 0,
                child: Image.asset(
                  PaperAssets.landscapeHills,
                  fit: BoxFit.fitWidth,
                  opacity: AlwaysStoppedAnimation(isDark ? 0.5 : 0.85),
                ),
              ),
              Positioned(
                left: 22,
                bottom: 26,
                child: _swaying(
                  t: t,
                  phase: 1.9,
                  child: Image.asset(
                    PaperAssets.singleTree,
                    height: 46,
                    opacity: AlwaysStoppedAnimation(isDark ? 0.85 : 1),
                  ),
                ),
              ),
              Positioned(
                right: 22,
                bottom: 12,
                child: _swaying(
                  t: t,
                  phase: 0,
                  child: Image.asset(
                    PaperAssets.twoTrees,
                    height: 78,
                    opacity: AlwaysStoppedAnimation(isDark ? 0.9 : 1),
                  ),
                ),
              ),
              Positioned(
                left: 78,
                bottom: 16,
                child: Image.asset(
                  PaperAssets.whiteDaisy,
                  height: 15,
                  opacity: AlwaysStoppedAnimation(isDark ? 0.55 : 0.9),
                ),
              ),
```

```
                      Positioned(
                        right: 96,
                        bottom: 8,
                        child: Image.asset(
                          PaperAssets.salmonFlower,
                          height: 17,
                          opacity: AlwaysStoppedAnimation(isDark ? 0.55 : 0.9),
                        ),
                      ),
                      Positioned(
                        left: 46,
                        bottom: 6,
                        child: Image.asset(
                          PaperAssets.tanSeed,
                          height: 10,
                          opacity: AlwaysStoppedAnimation(isDark ? 0.45 : 0.7),
                        ),
                      ),
                      if (!isDark)
                        Positioned(
                          left: 96,
                          bottom: 64,
                          child: _swaying(
                            t: t,
                            phase: 3.4,
                            child: Image.asset(
                              PaperAssets.tealBird,
                              height: 18,
                              opacity: const AlwaysStoppedAnimation(0.9),
                            ),
                          ),
                        ),
                      if (isDark)
                        Positioned.fill(
                          child: CustomPaint(painter: _FirefliesPainter(t)),
                        ),
                    ],
                  );
                },
              );
            },
          ),
        ),
      ),
    );
  }
}

class _FirefliesPainter extends CustomPainter {
  _FirefliesPainter(this.t);

  final double t;

  static const _anchors = [
    Offset(0.16, 0.42),
    Offset(0.34, 0.62),
    Offset(0.52, 0.30),
    Offset(0.68, 0.55),
    Offset(0.82, 0.28),
    Offset(0.90, 0.60),
  ];

  @override
  void paint(Canvas canvas, Size size) {
    final glow = Paint()
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4);
    final core = Paint();

    for (var i = 0; i < _anchors.length; i++) {
      final phase = i * 1.7;
      final angle = t * 2 * math.pi;
      final dx = math.sin(angle + phase) * 7;
      final dy = math.sin((angle + phase) * 2) * 4;
```

```
        final center = Offset(
          _anchors[i].dx * size.width + dx,
          _anchors[i].dy * size.height + dy,
        );
        final pulse = 0.25 + 0.75 * (0.5 + 0.5 * math.sin(angle * 3 + phase * 2));
        glow.color = const Color(0xFFE8C57F).withValues(alpha: 0.45 * pulse);
        core.color = const Color(0xFFF6E3B4).withValues(alpha: 0.9 * pulse);
        canvas.drawCircle(center, 3.4, glow);
        canvas.drawCircle(center, 1.1, core);
      }
    }

    @override
    bool shouldRepaint(_FirefliesPainter oldDelegate) => oldDelegate.t != t;
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/decor/sidebar_grove.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/components/dialogs/pdf_viewer_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 37. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:syncfusion_flutter_pdfviewer/pdfviewer.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/dialogs/pdf_viewer_sheet.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:syncfusion_flutter_pdfviewer/pdfviewer.dart';

Future<void> openPdfViewer(BuildContext context, String url, {String? title}) {
  return showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    useSafeArea: true,
    // The shell's nav bar floats above its own body, so a sheet opened on the
    // shell navigator would come up underneath it.
    useRootNavigator: true,
    builder: (ctx) => DraggableScrollableSheet(
      expand: false,
      initialChildSize: 0.92,
      maxChildSize: 0.98,
      minChildSize: 0.5,
      builder: (_, controller) => Column(
        children: [
          ListTile(
            title: Text(title ?? ctx.l10n.pdfDefaultTitle),
            trailing: IconButton(
              icon: const Icon(Icons.close),
              onPressed: () => Navigator.pop(ctx),
            ),
          ),
          Expanded(
            child: SfPdfViewer.network(
              url,
              scrollDirection: PdfScrollDirection.vertical,
            ),
          ),
        ],
      ),
    ),
  );
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/dialogs/pdf_viewer_sheet.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?

4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/homework/due_date_label.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 111. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/core/homework/assignment_due.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/homework/due_date_label.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/models/models.dart';

class DueDateLabel extends StatelessWidget {
  const DueDateLabel({
    super.key,
    required this.assignment,
    this.compact = false,
    this.onDark = false,
  });

  final Assignment assignment;
  final bool compact;
  final bool onDark;

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context);
    final status = assignmentDueStatus(assignment);
    final due = assignment.dueDate;
    final muted = onDark
        ? Colors.white.withValues(alpha: 0.85)
        : AppColors.inkAlpha(context, 0.55);

    if (status == AssignmentDueStatus.none || due == null) {
      return Text(
        l10n.commonNoDueDate,
        style: TextStyle(color: muted, fontSize: compact ? 12 : 13),
      );
    }

    final (label, color) = _labelAndColor(context, l10n, status, due, onDark);

    if (compact) {
      return Text(
        label,
```

```
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
        style: TextStyle(
          color: color,
          fontSize: 12,
          fontWeight: FontWeight.w600,
        ),
      );
    }

    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
      decoration: BoxDecoration(
        color: color.withValues(alpha: onDark ? 0.22 : 0.12),
        borderRadius: BorderRadius.circular(20),
      ),
      child: Text(
        label,
        style: TextStyle(
          color: onDark ? Colors.white : color,
          fontSize: 12,
          fontWeight: FontWeight.w700,
        ),
      ),
    );
  }

  static (String, Color) _labelAndColor(
    BuildContext context,
    AppLocalizations l10n,
    AssignmentDueStatus status,
    DateTime due,
    bool onDark,
  ) {
    final dateStr = DateFormat.MMMd().format(due);
    final isDark = Theme.of(context).brightness == Brightness.dark;
    switch (status) {
      case AssignmentDueStatus.overdue:
        return (
          l10n.homeworkOverdue(dateStr),
          onDark
              ? const Color(0xFFFFCDD2)
              : (isDark ? const Color(0xFFE57373) : AppColors.errorRed),
        );
      case AssignmentDueStatus.dueToday:
        return (
          l10n.homeworkDueToday,
          onDark
              ? const Color(0xFFFFE0B2)
              : (isDark ? const Color(0xFFFFB74D) : AppColors.warmAmber),
        );
      case AssignmentDueStatus.dueSoon:
        final today = assignmentCalendarDay(DateTime.now());
        final days = assignmentCalendarDay(due).difference(today).inDays;
        final label = days == 1
            ? l10n.homeworkDueTomorrow
            : l10n.homeworkDueInDays(days);
        return (label, onDark ? const Color(0xFFFFE0B2) : AppColors.mutedGold);
      case AssignmentDueStatus.upcoming:
        return (
          l10n.commonDueDate(dateStr),
          onDark ? Colors.white : AppColors.inkAlpha(context, 0.7),
        );
      case AssignmentDueStatus.none:
        return (
          l10n.commonNoDueDate,
          onDark ? Colors.white70 : AppColors.inkAlpha(context, 0.55),
        );
    }
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/homework/due_date_label.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/inputs/group_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 67. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/inputs/group_selector.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/providers/groups/group_providers.dart';

class GroupSelector extends ConsumerWidget {
  const GroupSelector({super.key, this.padding = const EdgeInsets.all(20)});

  final EdgeInsets padding;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groups = ref.watch(myGroupsProvider);
    final activeId = ref.watch(activeGroupIdProvider);
    final list = groups.valueOrNull ?? const [];

    if (list.length < 2) return const SizedBox.shrink();

    final radius = paperControlRadius(context);

    return Padding(
      padding: padding,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
        decoration: paperControlDecoration(context),
        child: Row(
          children: [
            PaperGlyph(
              icon: Icons.groups_outlined,
              size: 18,
              color: AppColors.accent(context),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: DropdownButtonHideUnderline(
                child: DropdownButton<String>(
                  value: activeId,
                  isExpanded: true,
                  borderRadius: BorderRadius.circular(radius),
                  dropdownColor: AppColors.controlSurface(context),
                  style: TextStyle(
```

```
                  fontWeight: FontWeight.w700,
                  fontSize: 15,
                  color: AppColors.ink(context),
                ),
                items: list
                    .map(
                      (g) => DropdownMenuItem(
                        value: g.id,
                        child: Text(g.title, overflow: TextOverflow.ellipsis),
                      ),
                    )
                    .toList(),
                onChanged: (value) {
                  if (value == null) return;
                  ref.read(selectedGroupIdProvider.notifier).state = value;
                },
              ),
            ),
          ),
        ),
      ],
    ),
  ),
);
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/inputs/group_selector.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/inputs/load_more_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 43. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/inputs/load_more_button.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class LoadMoreButton extends StatelessWidget {
  const LoadMoreButton({
    super.key,
    required this.hasMore,
    required this.loading,
    required this.onPressed,
  });

  final bool hasMore;
  final bool loading;
  final VoidCallback onPressed;

  @override
  Widget build(BuildContext context) {
    if (!hasMore) return const SizedBox.shrink();

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 12),
      child: Center(
        child: loading
            ? SizedBox(
                width: 24,
                height: 24,
                child: CircularProgressIndicator(strokeWidth: 2),
              )
            : TextButton(
                onPressed: onPressed,
                child: Text(
                  context.l10n.commonLoadMore,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    color: AppColors.accent(context),
                  ),
                ),
              ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/inputs/load_more_button.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/layouts/app_shell.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 417. Import count: 16. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/brand/app_logo.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/decor/sidebar_grove.dart';
- import 'package:mu_study/components/layouts/shell_nav_bar.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/utils/responsive.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/l10n/nav_labels.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/chat/chat_providers.dart';
- import 'package:mu_study/providers/notifications/notification_providers.dart';
- import 'package:mu_study/providers/focus/focus_session_provider.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/layouts/app_shell.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/brand/app_logo.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/decor/sidebar_grove.dart';
import 'package:mu_study/components/layouts/shell_nav_bar.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/utils/responsive.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/l10n/nav_labels.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/providers/focus/focus_session_provider.dart';

export 'package:mu_study/components/layouts/shell_nav_bar.dart';

class AppShell extends ConsumerStatefulWidget {
  const AppShell({
    super.key,
    required this.role,
    required this.child,
```

```
      required this.destinations,
    });

    final UserRole role;
    final Widget child;
    final List<ShellDestination> destinations;

    @override
    ConsumerState<AppShell> createState() => _AppShellState();
  }

  class _AppShellState extends ConsumerState<AppShell> {
    bool _isSidebarExpanded = true;

    int _indexFromLocation(String location) {
      for (var i = 0; i < widget.destinations.length; i++) {
        if (location.startsWith(widget.destinations[i].path)) return i;
      }
      return 0;
    }

    void _go(int index) {
      context.go(widget.destinations[index].path);
    }

    @override
    Widget build(BuildContext context) {
      final location = GoRouterState.of(context).uri.toString();
      final index = _indexFromLocation(location);
      final unreadNotifications = ref.watch(unreadNotificationsCountProvider);
      final unreadChat = ref.watch(focusChatMutedProvider)
          ? 0
          : ref.watch(unreadChatCountProvider);
      final isDesktop = MediaQuery.sizeOf(context).width >= Breakpoints.desktop;
      final isTablet = context.isTablet;
      final l10n = context.l10n;
      String labelOf(ShellDestination d) => navLabel(l10n, d.labelKey);

      int badgeCount(ShellDestination d) => switch (d.badge) {
        ShellBadgeKind.notifications => unreadNotifications,
        ShellBadgeKind.chat => unreadChat,
        ShellBadgeKind.none => 0,
      };

      Widget icon(
        ShellDestination d, {
        bool isSelected = false,
        bool usePaperAssets = true,
      }) {
        final isMono =
            AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
        final tone = isSelected
            ? AppColors.accent(context)
            : AppColors.inkAlpha(context, 0.5);
        final Widget base = usePaperAssets
            ? PaperGlyph(
                icon: d.icon,
                size: 24,
                color: tone,
                opacity: isSelected || !isNaturePaper(context) ? 1 : 0.72,
              )
            : Icon(d.icon, size: 24, color: tone);
        final count = badgeCount(d);
        if (count > 0) {
          return Badge(
            label: Text(
              count > 99 ? '99+' : '$count',
              style: TextStyle(
                fontSize: 10,
                color: isMono
                    ? (Theme.of(context).brightness == Brightness.dark
                          ? Colors.black
                          : Colors.white)
```

```
                : null,
          ),
        ),
        backgroundColor: isMono
            ? AppColors.ink(context)
            : AppColors.warmAmber,
        child: base,
      );
    }
    return base;
  }

  if (isDesktop) {
    return Scaffold(
      body: Row(
        children: [
          AnimatedContainer(
            duration: const Duration(milliseconds: 300),
            curve: Curves.easeInOut,
            width: _isSidebarExpanded ? 260 : 80,
            decoration: BoxDecoration(
              color: Theme.of(context).colorScheme.surface,
              border: Border(
                right: BorderSide(
                  color: Theme.of(
                    context,
                  ).colorScheme.onSurface.withValues(alpha: 0.05),
                ),
              ),
            ),
            child: ClipRect(
              child: Column(
                children: [
                  const SizedBox(height: 24),
                  Padding(
                    padding: const EdgeInsets.symmetric(horizontal: 16),
                    child: Row(
                      mainAxisAlignment: _isSidebarExpanded
                          ? MainAxisAlignment.spaceBetween
                          : MainAxisAlignment.center,
                      children: [
                        if (_isSidebarExpanded)
                          Expanded(
                            child: ClipRect(
                              child: Row(
                                children: [
                                  const AppLogoImage(size: 28),
                                  const SizedBox(width: 12),
                                  Text(
                                    l10n.shellMuStudy,
                                    maxLines: 1,
                                    overflow: TextOverflow.clip,
                                    softWrap: false,
                                    style: Theme.of(context)
                                        .textTheme
                                        .titleMedium
                                        ?.copyWith(
                                          fontWeight: FontWeight.w800,
                                          letterSpacing: -0.5,
                                        ),
                                  ),
                                ],
                              ),
                            ),
                          ),
                        IconButton(
                          onPressed: () => setState(
                            () => _isSidebarExpanded = !_isSidebarExpanded,
                          ),
                          icon: Icon(
                            _isSidebarExpanded ? Icons.menu_open : Icons.menu,
                            color: AppColors.inkAlpha(context, 0.5),
                          ),
```

```
      ),
    ],
  ),
),
if (_isSidebarExpanded)
  const Padding(
    padding: EdgeInsets.only(top: 12),
    child: SizedBox(
      width: double.infinity,
      child: PaperDecor(
        PaperAssets.hangingGarland,
        height: 52,
        fit: BoxFit.fitWidth,
        opacity: 0.95,
      ),
    ),
  ),
const SizedBox(height: 20),
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 12),
  child: Column(
    mainAxisSize: MainAxisSize.min,
    children: [
      for (var i = 0; i < widget.destinations.length; i++)
        Builder(
          builder: (context) {
            final d = widget.destinations[i];
            final isSelected = index == i;
            return Padding(
              padding: const EdgeInsets.only(bottom: 4),
              child: InkWell(
                onTap: () => _go(i),
                borderRadius: BorderRadius.circular(8),
                child: AnimatedContainer(
                  duration: const Duration(
                    milliseconds: 200,
                  ),
                  padding: const EdgeInsets.symmetric(
                    horizontal: 12,
                    vertical: 10,
                  ),
                  decoration: BoxDecoration(
                    color: isSelected
                        ? AppColors.accent(
                            context,
                          ).withValues(alpha: 0.12)
                        : Colors.transparent,
                    borderRadius: BorderRadius.circular(
                      isNaturePaper(context) ? 16 : 8,
                    ),
                  ),
                  child: ClipRect(
                    child: Row(
                      mainAxisAlignment: _isSidebarExpanded
                          ? MainAxisAlignment.start
                          : MainAxisAlignment.center,
                      children: [
                        icon(d, isSelected: isSelected),
                        if (_isSidebarExpanded) ...[
                          const SizedBox(width: 12),
                          Expanded(
                            child: Text(
                              labelOf(d),
                              maxLines: 1,
                              overflow: TextOverflow.clip,
                              softWrap: false,
                              style: Theme.of(context)
                                  .textTheme
                                  .bodyMedium
                                  ?.copyWith(
                                    fontWeight: isSelected
                                        ? FontWeight.w700
                                        : FontWeight.w500,
```

```
                                        color: isSelected
                                            ? AppColors.accent(
                                                context,
                                              )
                                            : AppColors.inkAlpha(
                                                context,
                                                0.7,
                                              ),
                                      ),
                                    ),
                                  ),
                                ),
                              ],
                            ],
                          ),
                        ),
                      ),
                    ),
                  ),
                );
              },
            ),
          ],
        ),
      ),
      if (_isSidebarExpanded)
        const Expanded(child: SidebarSkyClouds())
      else
        const Spacer(),
      if (_isSidebarExpanded) const SidebarGrove(),
      if (_isSidebarExpanded)
        Padding(
          padding: const EdgeInsets.all(16),
          child: Container(
            padding: const EdgeInsets.all(12),
            decoration: paperControlDecoration(
              context,
              color: AppColors.softWash(context, lightAlpha: 0.2),
              radius: isNaturePaper(context) ? 16 : 12,
            ),
            child: Row(
              children: [
                if (isNaturePaper(context))
                  const PaperGlyph(
                    icon: Icons.person_outline,
                    size: 28,
                  )
                else
                  CircleAvatar(
                    radius: 16,
                    backgroundColor: AppColors.accent(context),
                    child: Icon(
                      Icons.person,
                      size: 16,
                      color:
                          Theme.of(context).brightness ==
                              Brightness.dark
                          ? Colors.black
                          : Colors.white,
                    ),
                  ),
                const SizedBox(width: 12),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        switch (widget.role) {
                          UserRole.student => l10n.commonStudent,
```

**lib/components/layouts/app_shell.dart — continued (2/2)**

```
                          UserRole.teacher => l10n.commonTeacher,
                          UserRole.admin => l10n.commonAdmin,
                        }.toUpperCase(),
```

```
                                  style: TextStyle(
                                    fontSize: 10,
                                    fontWeight: FontWeight.w800,
                                    color: AppColors.accent(context),
                                  ),
                                ),
                                Text(
                                  l10n.commonWorkspace,
                                  style: const TextStyle(
                                    fontSize: 12,
                                    fontWeight: FontWeight.w500,
                                  ),
                                ),
                              ],
                            ),
                          ),
                        ),
                      ),
                    ),
                  ),
                  const SizedBox(height: 16),
                ],
              ),
            ),
          ),
          Expanded(
            child: Center(
              child: ConstrainedBox(
                constraints: BoxConstraints(
                  maxWidth: context.contentMaxWidth,
                ),
                child: widget.child,
              ),
            ),
          ),
        ],
      ),
    );
  }

  if (isTablet) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            right: false,
            child: NavigationRail(
              selectedIndex: index,
              onDestinationSelected: _go,
              labelType: NavigationRailLabelType.none,
              backgroundColor: Theme.of(context).colorScheme.surface,
              indicatorColor: AppColors.accent(
                context,
              ).withValues(alpha: 0.15),
              leading: const Padding(
                padding: EdgeInsets.symmetric(vertical: 24),
                child: AppLogoImage(size: 28),
              ),
              destinations: widget.destinations
                  .map(
                    (d) => NavigationRailDestination(
                      icon: icon(d),
                      selectedIcon: icon(d, isSelected: true),
                      label: Text(labelOf(d)),
                    ),
                  )
                  .toList(),
            ),
          ),
          const VerticalDivider(width: 1, thickness: 1),
          Expanded(
            child: Center(
              child: ConstrainedBox(
```

```
                  constraints: BoxConstraints(
                    maxWidth: context.contentMaxWidth,
                  ),
                  child: widget.child,
                ),
              ),
            ),
          ],
        ),
      );
    }

    return Scaffold(
      extendBody: true,
      resizeToAvoidBottomInset: false,
      body: Center(
        child: ConstrainedBox(
          constraints: BoxConstraints(maxWidth: context.contentMaxWidth),
          child: widget.child,
        ),
      ),
      bottomNavigationBar: ShellNavBar(
        destinations: widget.destinations,
        selectedIndex: index,
        onDestinationSelected: _go,
        iconBuilder: (d, isSelected) =>
            icon(d, isSelected: isSelected, usePaperAssets: false),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/layouts/app_shell.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/layouts/empty_state.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 121. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/layouts/empty_state.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class EmptyState extends StatelessWidget {
  const EmptyState({
    super.key,
    required this.title,
    this.subtitle,
    this.icon = Icons.inbox_outlined,
  });

  final String title;
  final String? subtitle;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    final paperAsset = isNaturePaper(context)
        ? (paperAssetForIcon(icon) ?? PaperAssets.tornPaperScrap)
        : null;
    final isNeu = isSoftNeumorphism(context);
    final isMono = isMonochromeWorkspace(context);

    Widget leading;
    if (paperAsset != null) {
      leading = PaperEmptyIllustration(asset: paperAsset);
    } else if (isNeu) {
      leading = NeuIconWell(
        icon: icon,
        color: AppColors.accent(context),
        size: 36,
        padding: 26,
        elevation: NeuElevation.raised,
      );
    } else if (isMono) {
      leading = Container(
        width: 72,
        height: 72,
        decoration: BoxDecoration(
```

```
          borderRadius: BorderRadius.circular(6),
          border: Border.all(color: AppColors.inkAlpha(context, 0.12)),
        ),
        child: Icon(icon, size: 28, color: AppColors.inkAlpha(context, 0.45)),
      );
    } else {
      leading = Icon(icon, size: 56, color: AppColors.inkAlpha(context, 0.25));
    }

    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            leading,
            const SizedBox(height: 16),
            Text(
              title,
              textAlign: TextAlign.center,
              style: Theme.of(context).textTheme.titleMedium?.copyWith(
                fontWeight: FontWeight.w600,
                color: AppColors.ink(context),
              ),
            ),
            if (subtitle != null) ...[
              const SizedBox(height: 8),
              Text(
                subtitle!,
                textAlign: TextAlign.center,
                style: Theme.of(context).textTheme.bodyMedium?.copyWith(
                  color: AppColors.inkAlpha(context, 0.55),
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

class ErrorState extends StatelessWidget {
  const ErrorState({super.key, required this.message, this.onRetry});

  final String message;
  final VoidCallback? onRetry;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            if (isNaturePaper(context))
              const PaperEmptyIllustration(asset: PaperAssets.tornPaperScrap)
            else
              Icon(
                Icons.error_outline,
                color: AppColors.danger(context),
                size: 48,
              ),
            const SizedBox(height: 12),
            Text(message, textAlign: TextAlign.center),
            if (onRetry != null) ...[
              const SizedBox(height: 16),
              TextButton(
                onPressed: onRetry,
                child: Text(context.l10n.commonTryAgain),
              ),
            ],
```

```
            ],
          ),
        ),
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/layouts/empty_state.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/layouts/premium_section_header.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 118. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/layouts/premium_section_header.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';

class PremiumSectionHeader extends StatelessWidget {
  const PremiumSectionHeader({
    super.key,
    required this.title,
    this.subtitle,
    this.trailing,
    this.horizontalPadding = 0,
    this.decorAsset,
  });

  final String title;
  final String? subtitle;
  final Widget? trailing;
  final double horizontalPadding;

  final String? decorAsset;

  @override
  Widget build(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    final headerGlow = tokens?.headerGlow ?? Colors.transparent;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    Widget content = Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        if (subtitle != null)
          Padding(
            padding: const EdgeInsets.only(bottom: 4),
            child: Text(
              subtitle!.toUpperCase(),
              style: Theme.of(context).textTheme.labelSmall?.copyWith(
                letterSpacing: tokens?.isMonochromeWorkspace == true
                    ? 1.4
                    : 1.2,
                fontWeight: FontWeight.w700,
                color: tokens?.isMonochromeWorkspace == true
                    ? AppColors.inkAlpha(context, 0.45)
                    : AppColors.accent(context).withValues(alpha: 0.92),
```

```
              ),
            ),
          ),
        Row(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            Flexible(
              child: Text(
                title,
                style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                  fontWeight: FontWeight.w800,
                  letterSpacing: tokens?.isMonochromeWorkspace == true
                      ? -0.7
                      : -0.5,
                  fontSize: tokens?.isMonochromeWorkspace == true ? 26 : null,
                ),
              ),
            ),
            const SizedBox(width: 8),
            PaperDecor(
              decorAsset ?? PaperAssets.beigeLeaf,
              size: decorAsset == null ? 16 : 22,
              angle: decorAsset == null ? 0.5 : 0.1,
              opacity: 0.9,
            ),
          ],
        ),
      ],
    );

    if (headerGlow != Colors.transparent) {
      content = Stack(
        clipBehavior: Clip.none,
        children: [
          Positioned(
            left: -28,
            top: -20,
            child: IgnorePointer(
              child: Container(
                width: 190,
                height: 84,
                decoration: BoxDecoration(
                  borderRadius: BorderRadius.circular(48),
                  gradient: RadialGradient(
                    colors: [
                      headerGlow.withValues(alpha: isDark ? 0.10 : 0.26),
                      headerGlow.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
          content,
        ],
      );
    }

    return Padding(
      padding: EdgeInsets.symmetric(
        horizontal: horizontalPadding,
        vertical: 12,
      ),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.end,
        children: [
          Expanded(child: content),
          ?trailing,
        ],
      ),
    );
  }
```

```
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/layouts/premium_section_header.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/layouts/shell_nav_bar.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 405. Import count: 9. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math' as math;
- import 'dart:ui';
- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/l10n/nav_labels.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/layouts/shell_nav_bar.dart**

```
import 'dart:math' as math;
import 'dart:ui';

import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/l10n/nav_labels.dart';

/// Height of the floating navigation pill, excluding the safe area beneath it.
const double kShellNavBarHeight = 64;

/// Inset of the pill from the left and right edges of the screen.
const double kShellNavBarSideMargin = 16;

/// Gap between the pill and the bottom of the screen on devices without a home
/// indicator or gesture bar. Devices that have one use that inset instead, so
/// the pill never sits inside the system gesture area.
const double kShellNavBarMinBottomGap = 16;

/// Gap between a floating action button and the top of the pill.
const double kShellFabGap = 16;

/// Height of a standard [FloatingActionButton]; extended ones are shorter.
const double kShellFabHeight = 56;

/// Duration of the nav bar's selection animations.
const Duration kShellNavBarAnimationDuration = Duration(milliseconds: 240);

enum ShellBadgeKind { none, notifications, chat }

class ShellDestination {
  const ShellDestination({
    required this.labelKey,
```

```
    required this.icon,
    required this.path,
    this.badge = ShellBadgeKind.none,
  });

  /// Key into [AppLocalizations] nav_* getters (e.g. `navDashboard`).
  final String labelKey;
  final IconData icon;
  final String path;
  final ShellBadgeKind badge;
}

/// Space the shell's navigation bar occupies at the bottom of the screen,
/// including the safe area beneath it.
///
/// Screens inside the shell have their own `Scaffold`, which knows nothing
/// about the shell's floating pill and would otherwise place floating action
/// buttons and trailing content underneath it.
///
/// The shell renders the pill with `Scaffold.extendBody`, which folds its
/// height into the bottom padding of the body and zeroes the body's bottom
/// `viewPadding` — so the padding alone is the whole story. Outside the shell,
/// on tablet, on desktop and on routes pushed above it, this is the plain
/// system safe area, which is what those layouts need anyway.
double shellBottomInset(BuildContext context) =>
    MediaQuery.paddingOf(context).bottom;

/// Bottom padding for scrollable content inside the shell so the last item
/// clears the navigation bar and, when [hasFab] is set, the floating action
/// button above it.
double shellContentBottomPadding(
  BuildContext context, {
  bool hasFab = false,
  double extra = 20,
}) {
  final double base = shellBottomInset(context) + extra;
  return hasFab ? base + kShellFabHeight + kShellFabGap : base;
}

/// Positions a [Scaffold.floatingActionButton] above the shell's navigation
/// bar, the safe area and the keyboard.
FloatingActionButtonLocation shellFabLocation(BuildContext context) =>
    ShellFabLocation(bottomInset: shellBottomInset(context));

/// End-aligned floating action button location that floats a fixed distance
/// above the shell's navigation bar rather than above the scaffold's own
/// bottom edge.
class ShellFabLocation extends StandardFabLocation with FabEndOffsetX {
  const ShellFabLocation({required this.bottomInset});

  /// Result of [shellBottomInset] for the screen this button belongs to.
  final double bottomInset;

  @override
  double getOffsetY(ScaffoldPrelayoutGeometry geometry, double adjustment) {
    final double fabHeight = geometry.floatingActionButtonSize.height;
    final double keyboard = geometry.minInsets.bottom;

    // The keyboard covers the nav bar rather than moving it, so the reserved
    // space shrinks by however much of it the keyboard hides. The button holds
    // still until the keyboard reaches it, then rides up with it.
    final double margin = math.max(
      kFloatingActionButtonMargin,
      bottomInset + kShellFabGap - keyboard,
    );

    double y = geometry.contentBottom - fabHeight - margin;
    final double snackBarHeight = geometry.snackBarSize.height;
    if (snackBarHeight > 0) {
      y = math.min(
        y,
        geometry.contentBottom -
            snackBarHeight -
```

```
            fabHeight -
            kFloatingActionButtonMargin,
      );
    }
    return y + adjustment;
  }

  @override
  bool operator ==(Object other) =>
      other is ShellFabLocation && other.bottomInset == bottomInset;

  @override
  int get hashCode => bottomInset.hashCode;

  @override
  String toString() => 'ShellFabLocation(bottomInset: $bottomInset)';
}

/// Frosted navigation pill used as the shell's `bottomNavigationBar` on mobile.
///
/// The pill is [kShellNavBarHeight] tall and sits on top of the larger of the
/// bottom safe area and [kShellNavBarMinBottomGap], so it clears home
/// indicators and gesture bars on any device. That total is what screens read
/// back through [shellBottomInset].
class ShellNavBar extends StatelessWidget {
  const ShellNavBar({
    super.key,
    required this.destinations,
    required this.selectedIndex,
    required this.onDestinationSelected,
    required this.iconBuilder,
  });

  /// Identifies the pill itself, so tests can measure its bounds.
  static const Key pillKey = Key('shellNavBarPill');

  final List<ShellDestination> destinations;
  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;
  final Widget Function(ShellDestination destination, bool isSelected)
  iconBuilder;

  @override
  Widget build(BuildContext context) {
    // viewPadding rather than padding: the bar sits below the keyboard, so it
    // must not resize while the keyboard is up.
    final EdgeInsets viewPadding = MediaQuery.viewPaddingOf(context);

    return Padding(
      padding: EdgeInsets.only(
        left: kShellNavBarSideMargin + viewPadding.left,
        right: kShellNavBarSideMargin + viewPadding.right,
        bottom: math.max(viewPadding.bottom, kShellNavBarMinBottomGap),
      ),
      child: _buildPill(context),
    );
  }

  Widget _buildPill(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final tokens = AppStyleTokens.maybeOf(context);
    final isNature = tokens?.isNaturePaper ?? false;
    final isNeu = tokens?.isSoftNeumorphism ?? false;
    final isMono = tokens?.isMonochromeWorkspace ?? false;

    // Monochrome: compact rectangular dock. Others keep the stadium pill.
    final BorderRadius radius = BorderRadius.all(
      Radius.circular(isMono ? 12 : kShellNavBarHeight / 2),
    );

    final Color fill =
        tokens?.navSurface ??
        (isDark
```

```
        ? AppColors.darkSurfaceHighest.withValues(alpha: 0.96)
        : Theme.of(context).colorScheme.surface.withValues(alpha: 0.88));
final Color borderColor;
if (isNeu) {
  borderColor = Colors.transparent;
} else if (isMono) {
  borderColor = isDark
      ? AppColors.inkAlpha(context, 0.18)
      : AppColors.inkAlpha(context, 0.10);
} else if (isNature) {
  borderColor = isDark
      ? NaturePaperColors.moonGlow.withValues(alpha: 0.22)
      : NaturePaperColors.warmBrown.withValues(alpha: 0.18);
} else {
  borderColor = isDark ? AppColors.darkBorder : Colors.transparent;
}

final List<BoxShadow> shadows;
if (isNeu) {
  // Floating extruded dock — dual soft shadows, no glass blur.
  shadows = Neumorphic.shadows(context, elevation: NeuElevation.floating);
} else if (isMono) {
  // Editorial: no soft shadow — border alone defines the dock.
  shadows = const [];
} else {
  shadows = [
    BoxShadow(
      color: isDark
          ? Colors.black.withValues(alpha: 0.55)
          : (isNature
                ? NaturePaperColors.warmBrown.withValues(alpha: 0.28)
                : AppColors.inkAlpha(context, 0.12)),
      blurRadius: 24,
      offset: const Offset(0, 8),
    ),
    if (isNature && isDark)
      BoxShadow(
        color: NaturePaperColors.moonGlow.withValues(alpha: 0.08),
        blurRadius: 28,
      ),
  ];
}

final navRow = Material(
  color: fill,
  shape: RoundedRectangleBorder(
    borderRadius: radius,
    side: BorderSide(color: borderColor),
  ),
  clipBehavior: Clip.antiAlias,
  child: Row(
    children: [
      for (var i = 0; i < destinations.length; i++)
        Expanded(
          child: _ShellNavItem(
            destination: destinations[i],
            isSelected: i == selectedIndex,
            icon: iconBuilder(destinations[i], i == selectedIndex),
            onTap: () => onDestinationSelected(i),
            pressedActive: isNeu,
            editorialActive: isMono,
          ),
        ),
    ],
  ),
);

return Container(
  key: pillKey,
  height: kShellNavBarHeight,
  decoration: BoxDecoration(borderRadius: radius, boxShadow: shadows),
  child: ClipRRect(
    borderRadius: radius,
```

```
        // Soft Neu + Monochrome avoid glassmorphism; Modern/Nature frost.
        child: (isNeu || isMono)
            ? navRow
            : BackdropFilter(
                filter: ImageFilter.blur(sigmaX: 18, sigmaY: 18),
                child: navRow,
              ),
      ),
    );
  }
}

class _ShellNavItem extends StatelessWidget {
  const _ShellNavItem({
    required this.destination,
    required this.isSelected,
    required this.icon,
    required this.onTap,
    this.pressedActive = false,
    this.editorialActive = false,
  });

  final ShellDestination destination;
  final bool isSelected;
  final Widget icon;
  final VoidCallback onTap;

  /// Neumorphism: active tab looks physically pressed into the dock.
  final bool pressedActive;

  /// Monochrome Workspace: filled ink well + thin underline indicator.
  final bool editorialActive;

  @override
  Widget build(BuildContext context) {
    final glow = AppStyleTokens.maybeOf(context)?.navGlow ?? Colors.transparent;
    final ink = AppColors.ink(context);
    final isDark = Theme.of(context).brightness == Brightness.dark;

    return Semantics(
      button: true,
      selected: isSelected,
      label: navLabel(context.l10n, destination.labelKey),
      child: InkWell(
        onTap: onTap,
        customBorder: editorialActive
            ? RoundedRectangleBorder(borderRadius: BorderRadius.circular(10))
            : const StadiumBorder(),
        child: SizedBox(
          height: kShellNavBarHeight,
          child: Stack(
            alignment: Alignment.center,
            children: [
              // Soft Neu: pressed well for the active destination.
              if (pressedActive)
                AnimatedOpacity(
                  duration: kShellNavBarAnimationDuration,
                  curve: Curves.easeOutCubic,
                  opacity: isSelected ? 1 : 0,
                  child: Container(
                    width: 48,
                    height: 40,
                    decoration: BoxDecoration(
                      color: Neumorphic.surface(
                        context,
                        elevation: NeuElevation.pressed,
                      ),
                      borderRadius: BorderRadius.circular(14),
                      boxShadow: Neumorphic.shadows(
                        context,
                        elevation: NeuElevation.pressed,
                      ),
                    ),
```

```
                ),
              ),
            // Monochrome: solid ink fill behind the active icon.
            if (editorialActive)
              AnimatedOpacity(
                duration: kShellNavBarAnimationDuration,
                curve: Curves.easeOutCubic,
                opacity: isSelected ? 1 : 0,
                child: Container(
                  width: 40,
                  height: 36,
                  decoration: BoxDecoration(
                    color: ink,
                    borderRadius: BorderRadius.circular(8),
                  ),
                ),
              ),
            if (glow != Colors.transparent)
              AnimatedOpacity(
                duration: kShellNavBarAnimationDuration,
                curve: Curves.easeOutCubic,
                opacity: isSelected ? 1 : 0,
                child: Container(
                  width: 44,
                  height: 44,
                  decoration: BoxDecoration(
                    shape: BoxShape.circle,
                    gradient: RadialGradient(
                      colors: [
                        glow.withValues(alpha: pressedActive ? 0.28 : 0.40),
                        glow.withValues(alpha: 0.0),
                      ],
                    ),
                  ),
                ),
              ),
            // The icon is centred on the pill and the indicator floats
            // underneath it, so selecting a tab never shifts the icon.
            AnimatedScale(
              duration: kShellNavBarAnimationDuration,
              curve: Curves.easeOutCubic,
              scale: isSelected
                  ? (pressedActive || editorialActive ? 1.0 : 1.08)
                  : 1,
              child: editorialActive && isSelected
                  ? ColorFiltered(
                      colorFilter: ColorFilter.mode(
                        isDark ? Colors.black : Colors.white,
                        BlendMode.srcIn,
                      ),
                      child: icon,
                    )
                  : icon,
            ),
            if (!pressedActive)
              Positioned(
                bottom: editorialActive ? 8 : 12,
                child: AnimatedContainer(
                  duration: kShellNavBarAnimationDuration,
```

**lib/components/layouts/shell_nav_bar.dart — continued (2/2)**

```
                  curve: Curves.easeOutCubic,
                  height: editorialActive ? 2 : 4,
                  width: isSelected ? (editorialActive ? 12 : 16) : 0,
                  decoration: BoxDecoration(
                    color: editorialActive
                        ? ink.withValues(alpha: 0.35)
                        : AppColors.accent(context),
                    borderRadius: BorderRadius.circular(1),
                  ),
                ),
              ),
```

```
              ],
            ),
          ),
        ),
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/layouts/shell_nav_bar.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/marks/mark_average_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 121. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/marks/mark_average_card.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class MarkAverageCard extends StatelessWidget {
  const MarkAverageCard({
    super.key,
    required this.average,
    required this.markCount,
    this.onTap,
  });

  final double? average;
  final int markCount;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final value = average;
    final color = value == null
        ? AppColors.accent(context)
        : percentColor(value, context: context);
    final starAsset = paperStarForPercent(value);

    return PremiumCard(
      onTap: onTap,
      padding: const EdgeInsets.all(24),
      child: Row(
        children: [
          SizedBox(
            width: 76,
            height: 76,
            child: TweenAnimationBuilder<double>(
              tween: Tween<double>(begin: 0, end: (value ?? 0) / 100),
              duration: const Duration(milliseconds: 900),
              curve: Curves.easeOutCubic,
              builder: (context, progress, _) {
```

```
        const stroke = 8.0;
        const labelInset = stroke * 2 + 10;
        return Stack(
          alignment: Alignment.center,
          children: [
            Positioned.fill(
              child: CircularProgressIndicator(
                value: progress,
                backgroundColor: AppColors.softSurface(context),
                valueColor: AlwaysStoppedAnimation(color),
                strokeWidth: stroke,
                strokeCap: StrokeCap.round,
              ),
            ),
            Padding(
              padding: const EdgeInsets.all(labelInset / 2),
              child: FittedBox(
                fit: BoxFit.scaleDown,
                child: Text(
                  formatPercent(value),
                  maxLines: 1,
                  textAlign: TextAlign.center,
                  style: const TextStyle(
                    fontWeight: FontWeight.w800,
                    fontSize: 16,
                    height: 1,
                    letterSpacing: -0.4,
                  ),
                ),
              ),
            ),
          ],
        );
      },
    ),
  ),
  const SizedBox(width: 20),
  Expanded(
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          context.l10n.marksOverallAverage,
          style: const TextStyle(
            fontWeight: FontWeight.w800,
            fontSize: 16,
          ),
        ),
        const SizedBox(height: 4),
        Text(
          value == null
              ? context.l10n.marksNoneYet
              : context.l10n.marksBasedOn(
                  percentLabel(value, context.l10n),
                  markCount,
                ),
          style: TextStyle(
            color: AppColors.inkAlpha(context, 0.6),
            fontSize: 13,
            height: 1.4,
          ),
        ),
      ),
    ],
  ),
  ),
  if (starAsset != null)
    Padding(
      padding: const EdgeInsets.only(left: 8),
      child: PaperDecor(starAsset, size: 42, angle: 0.14),
    ),
  if (onTap != null)
    Icon(
      Icons.chevron_right_rounded,
```

```
                size: 20,
                color: AppColors.inkAlpha(context, 0.3),
              ),
            ],
          ),
        );
    }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/marks/mark_average_card.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/components/marks/percent_badge.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 95. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/app_localizations.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/marks/percent_badge.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/app_localizations.dart';

Color percentColor(double percent, {BuildContext? context}) {
  final mono =
      context != null &&
      (AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false);
  final isDark =
      context != null && Theme.of(context).brightness == Brightness.dark;

  if (mono) {
    if (isDark) {
      if (percent >= 85) return const Color(0xFFF5F5F5);
      if (percent >= 70) return const Color(0xFFC8C8C8);
      if (percent >= 50) return const Color(0xFF9A9A9A);
      return const Color(0xFF707070);
    }
    if (percent >= 85) return const Color(0xFF111111);
    if (percent >= 70) return const Color(0xFF3A3A3A);
    if (percent >= 50) return const Color(0xFF6B6B6B);
    return const Color(0xFF8E8E8E);
  }

  if (percent >= 85) return AppColors.successGreen;
  if (percent >= 70) return AppColors.mutedGold;
  if (percent >= 50) return AppColors.warmAmber;
  return AppColors.errorRed;
}

String percentLabel(double percent, [AppLocalizations? l10n]) {
  if (percent >= 85) return l10n?.percentExcellent ?? 'Excellent';
  if (percent >= 70) return l10n?.percentGood ?? 'Good';
  if (percent >= 50) return l10n?.percentNeedsWork ?? 'Needs work';
  return l10n?.percentAtRisk ?? 'At risk';
}

String formatPercent(double? percent) {
  if (percent == null) return '—';
  final rounded = percent.roundToDouble();
  return percent == rounded
      ? '${rounded.toInt()}%'
      : '${percent.toStringAsFixed(1)}%';
```

```
}

class PercentBadge extends StatelessWidget {
  const PercentBadge({super.key, required this.percent, this.compact = false});

  final double? percent;
  final bool compact;

  @override
  Widget build(BuildContext context) {
    final value = percent;
    final color = value == null
        ? AppColors.inkAlpha(context, 0.3)
        : percentColor(value, context: context);
    final tokens = AppStyleTokens.maybeOf(context);
    final isNature = tokens?.isNaturePaper ?? false;
    final radius = tokens?.isMonochromeWorkspace == true
        ? (compact ? 4.0 : 6.0)
        : isNature
        ? (compact ? 12.0 : 16.0)
        : (compact ? 8.0 : 12.0);

    return Container(
      padding: EdgeInsets.symmetric(
        horizontal: compact ? 10 : 14,
        vertical: compact ? 5 : 8,
      ),
      decoration: BoxDecoration(
        color: color.withValues(alpha: isNature ? 0.14 : 0.12),
        borderRadius: BorderRadius.circular(radius),
        border: tokens?.isMonochromeWorkspace == true
            ? Border.all(color: color.withValues(alpha: 0.28))
            : isNature
            ? Border.all(color: color.withValues(alpha: 0.22))
            : null,
      ),
      child: Text(
        formatPercent(value),
        style: TextStyle(
          color: color,
          fontWeight: FontWeight.w800,
          fontSize: compact ? 13 : 16,
          letterSpacing: -0.3,
          fontFeatures: tokens?.isMonochromeWorkspace == true
              ? const [FontFeature.tabularFigures()]
              : null,
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/marks/percent_badge.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/marks/term_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 95. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/components/marks/term_selector.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class TermSelector extends ConsumerWidget {
  const TermSelector({super.key, this.padding = EdgeInsets.zero});

  final EdgeInsets padding;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final current = ref.watch(selectedMarkTermProvider);
    final optionsAsync = ref.watch(academicTermOptionsProvider);
    final radius = paperControlRadius(context);

    return optionsAsync.when(
      loading: () => Padding(
        padding: padding,
        child: const SizedBox(
          height: 48,
          child: Center(child: CircularProgressIndicator(strokeWidth: 2)),
        ),
      ),
      error: (e, _) => Padding(
        padding: padding,
        child: Text(
          e.toString(),
          style: TextStyle(color: AppColors.errorRed, fontSize: 12),
        ),
      ),
      data: (options) {
        final selected = options.cast<AcademicTerm?>().firstWhere(
          (t) => t!.id == current.id,
          orElse: () => current,
        )!;

        return Padding(
```

```
          padding: padding,
          child: Container(
            padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 2),
            decoration: paperControlDecoration(context),
            child: Row(
              children: [
                PaperGlyph(
                  icon: Icons.calendar_month_outlined,
                  size: 18,
                  color: AppColors.accent(context),
                ),
                const SizedBox(width: 10),
                Expanded(
                  child: DropdownButtonHideUnderline(
                    child: DropdownButton<String>(
                      value: options.any((t) => t.id == selected.id)
                          ? selected.id
                          : options.first.id,
                      isExpanded: true,
                      borderRadius: BorderRadius.circular(radius),
                      dropdownColor: AppColors.controlSurface(context),
                      style: TextStyle(
                        fontWeight: FontWeight.w700,
                        fontSize: 14,
                        color: AppColors.ink(context),
                      ),
                      items: options
                          .map(
                            (t) => DropdownMenuItem(
                              value: t.id,
                              child: Text(
                                t.label(context.l10n),
                                overflow: TextOverflow.ellipsis,
                              ),
                            ),
                          )
                          .toList(),
                      onChanged: (id) {
                        if (id == null) return;
                        final next = options.firstWhere((t) => t.id == id);
                        ref
                            .read(selectedMarkTermProvider.notifier)
                            .select(next);
                      },
                    ),
                  ),
                ),
              ],
            ),
          ),
        );
      },
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/marks/term_selector.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/components/settings/language_settings_tile.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 141. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/locale/app_locale.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/settings/locale_provider.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/components/settings/language_settings_tile.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/locale/app_locale.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/settings/locale_provider.dart';

class LanguageSettingsTile extends ConsumerWidget {
  const LanguageSettingsTile({super.key, this.embedded = false});

  final bool embedded;

  String _nativeName(AppLanguage lang) => switch (lang) {
    AppLanguage.english => 'English',
    AppLanguage.russian => 'Русский',
    AppLanguage.uzbek => 'Oʻzbekcha',
  };

  String _codeLabel(AppLanguage lang) => switch (lang) {
    AppLanguage.english => 'EN',
    AppLanguage.russian => 'RU',
    AppLanguage.uzbek => 'UZ',
  };

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final l10n = context.l10n;
    final current = AppLanguage.fromLocale(ref.watch(localeProvider));
    final onSurface = Theme.of(context).colorScheme.onSurface;

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        if (!embedded) ...[
          Text(
            l10n.languageSection,
            style: Theme.of(
              context,
```

```
      ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
    ),
    const SizedBox(height: 4),
    Text(
      l10n.languageHint,
      style: TextStyle(
        fontSize: 12,
        color: AppColors.inkAlpha(context, 0.5),
      ),
    ),
    const SizedBox(height: 12),
  ] else ...[
    Text(
      l10n.languageSection,
      style: Theme.of(
        context,
      ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
    ),
    const SizedBox(height: 10),
  ],
  Container(
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
    decoration: paperControlDecoration(
      context,
      color: embedded
          ? AppColors.inkAlpha(context, 0.05)
          : AppColors.controlSurface(context),
    ),
    child: Row(
      children: [
        PaperGlyph(
          icon: Icons.language_rounded,
          size: 18,
          color: AppColors.accent(context),
        ),
        const SizedBox(width: 12),
        Expanded(
          child: DropdownButtonHideUnderline(
            child: DropdownButton<AppLanguage>(
              value: current,
              isExpanded: true,
              borderRadius: BorderRadius.circular(
                paperControlRadius(context),
              ),
              dropdownColor: AppColors.controlSurface(context),
              icon: Icon(
                Icons.keyboard_arrow_down_rounded,
                color: AppColors.inkAlpha(context, 0.45),
              ),
              style: TextStyle(
                fontWeight: FontWeight.w700,
                fontSize: 15,
                color: onSurface,
              ),
              items: AppLanguage.values
                  .map(
                    (lang) => DropdownMenuItem(
                      value: lang,
                      child: Row(
                        children: [
                          SizedBox(
                            width: 28,
                            child: Text(
                              _codeLabel(lang),
                              style: TextStyle(
                                fontWeight: FontWeight.w800,
                                fontSize: 12,
                                color: lang == current
                                    ? AppColors.accent(context)
                                    : AppColors.inkAlpha(context, 0.45),
                              ),
                            ),
                          ),
```

```
                              const SizedBox(width: 8),
                              Expanded(
                                child: Text(
                                  _nativeName(lang),
                                  overflow: TextOverflow.ellipsis,
                                ),
                              ),
                            ],
                          ),
                        ),
                      )
                      .toList(),
                  onChanged: (lang) {
                    if (lang == null || lang == current) return;
                    // rebuild (avoids semantics.parentDataDirty).
                    WidgetsBinding.instance.addPostFrameCallback((_) {
                      ref.read(localeProvider.notifier).setLanguage(lang);
                    });
                  },
                ),
              ),
            ),
          ],
        ),
      ),
    ],
  );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/components/settings/language_settings_tile.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/core/admin/credential_slip.dart

Credential slip PDF generation for printed login cards. This file (credential slip) is one concrete piece of that layer.

Line count: 33. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/admin/credential_slip.dart**

```
class CredentialSlip {
  const CredentialSlip({
    this.profileId,
    required this.username,
    required this.fullName,
    required this.password,
  });

  final String? profileId;
  final String username;
  final String fullName;
  final String password;

  CredentialSlip copyWith({String? password}) {
    return CredentialSlip(
      profileId: profileId,
      username: username,
      fullName: fullName,
      password: password ?? this.password,
    );
  }
}

List<CredentialSlip> uniqueCredentialSlips(Iterable<CredentialSlip> slips) {
  final seen = <String>{};
  final unique = <CredentialSlip>[];
  for (final slip in slips) {
    final key = (slip.profileId ?? slip.username).trim().toLowerCase();
    if (key.isEmpty || !seen.add(key)) continue;
    unique.add(slip);
  }
  return unique;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/admin/credential_slip.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/admin/credential_slips_pdf.dart

Credential slip PDF generation for printed login cards. This file (credential slips pdf) is one concrete piece of that layer.

Line count: 188. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:typed_data';
- import 'dart:ui';
- import 'package:mu_study/core/admin/credential_slip.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:syncfusion_flutter_pdf/pdf.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/admin/credential_slips_pdf.dart**

```
import 'dart:typed_data';
import 'dart:ui';

import 'package:mu_study/core/admin/credential_slip.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:syncfusion_flutter_pdf/pdf.dart';

const _slipsPerPage = 3;

/// A4 page with up to 3 unique cut-apart login cards. Never repeats a person
/// to fill empty slots.
Uint8List buildCredentialSlipsPdf(
  List<CredentialSlip> slips,
  AppLocalizations l10n,
) {
  final unique = uniqueCredentialSlips(slips);
  if (unique.isEmpty) {
    throw ArgumentError(l10n.adminCredentialNeedUsers);
  }

  final document = PdfDocument();
  document.pageSettings.size = PdfPageSize.a4;
  document.pageSettings.margins.all = 28;

  final muFont = PdfStandardFont(
    PdfFontFamily.helvetica,
    18,
    style: PdfFontStyle.bold,
  );
  final studyFont = PdfStandardFont(
    PdfFontFamily.helvetica,
    18,
    style: PdfFontStyle.bold,
  );
  final disclaimerFont = PdfStandardFont(
    PdfFontFamily.helvetica,
    8,
    style: PdfFontStyle.italic,
  );
  final labelFont = PdfStandardFont(
    PdfFontFamily.helvetica,
    11,
```

```
      style: PdfFontStyle.bold,
    );
    final valueFont = PdfStandardFont(PdfFontFamily.helvetica, 11);

    final muInk = PdfColor(45, 36, 22);
    final gold = PdfColor(197, 160, 89);
    final valueInk = PdfColor(20, 20, 20);
    final muted = PdfColor(120, 120, 120);
    final border = PdfColor(210, 210, 210);
    final fill = PdfColor(255, 255, 255);
    final linePen = PdfPen(PdfColor(230, 230, 230), width: 0.8);

    for (var i = 0; i < unique.length; i += _slipsPerPage) {
      final page = document.pages.add();
      final size = page.getClientSize();
      final g = page.graphics;
      const gap = 18.0;
      final slipHeight = (size.height - gap * (_slipsPerPage - 1)) / _slipsPerPage;
      final batch = unique.skip(i).take(_slipsPerPage).toList();

      for (var s = 0; s < batch.length; s++) {
        final top = s * (slipHeight + gap);
        final bounds = Rect.fromLTWH(0, top, size.width, slipHeight);
        _drawSlip(
          g,
          bounds: bounds,
          slip: batch[s],
          l10n: l10n,
          muFont: muFont,
          studyFont: studyFont,
          disclaimerFont: disclaimerFont,
          labelFont: labelFont,
          valueFont: valueFont,
          muInk: muInk,
          gold: gold,
          valueInk: valueInk,
          muted: muted,
          border: border,
          fill: fill,
          linePen: linePen,
        );
      }
    }

    final bytes = Uint8List.fromList(document.saveSync());
    document.dispose();
    return bytes;
  }

  void _drawSlip(
    PdfGraphics g, {
    required Rect bounds,
    required CredentialSlip slip,
    required AppLocalizations l10n,
    required PdfFont muFont,
    required PdfFont studyFont,
    required PdfFont disclaimerFont,
    required PdfFont labelFont,
    required PdfFont valueFont,
    required PdfColor muInk,
    required PdfColor gold,
    required PdfColor valueInk,
    required PdfColor muted,
    required PdfColor border,
    required PdfColor fill,
    required PdfPen linePen,
  }) {
    const radius = 10.0;
    final card = _roundedRect(bounds, radius);
    g.drawPath(card, brush: PdfSolidBrush(fill), pen: PdfPen(border, width: 1));

    const barWidth = 7.0;
    g.drawRectangle(
```

```
      brush: PdfSolidBrush(gold),
      bounds: Rect.fromLTWH(
        bounds.left,
        bounds.top + radius,
        barWidth,
        bounds.height - radius * 2,
      ),
    );

    final left = bounds.left + 28;
    final width = bounds.width - 48;
    var y = bounds.top + 22;

    g.drawString(
      'MU',
      muFont,
      brush: PdfSolidBrush(muInk),
      bounds: Rect.fromLTWH(left, y, 40, 24),
    );
    g.drawString(
      'Study',
      studyFont,
      brush: PdfSolidBrush(gold),
      bounds: Rect.fromLTWH(left + 34, y, 80, 24),
    );
    y += 28;

    g.drawLine(
      linePen,
      Offset(left, y),
      Offset(bounds.right - 20, y),
    );
    y += 10;

    g.drawString(
      l10n.adminCredentialConfidential,
      disclaimerFont,
      brush: PdfSolidBrush(muted),
      bounds: Rect.fromLTWH(left, y, width, 28),
    );
    y += 36;

    void row(String label, String value) {
      g.drawString(
        '$label:',
        labelFont,
        brush: PdfSolidBrush(muInk),
        bounds: Rect.fromLTWH(left, y, 100, 16),
      );
      g.drawString(
        value,
        valueFont,
        brush: PdfSolidBrush(valueInk),
        bounds: Rect.fromLTWH(left + 108, y, width - 108, 16),
      );
      y += 22;
    }

    row(l10n.loginUsername, slip.username);
    row(l10n.adminFullName, slip.fullName);
    row(l10n.adminPassword, slip.password);
  }

  PdfPath _roundedRect(Rect r, double radius) {
    final path = PdfPath();
    final d = radius * 2;
    path.addArc(Rect.fromLTWH(r.left, r.top, d, d), 180, 90);
    path.addArc(Rect.fromLTWH(r.right - d, r.top, d, d), 270, 90);
    path.addArc(Rect.fromLTWH(r.right - d, r.bottom - d, d, d), 0, 90);
    path.addArc(Rect.fromLTWH(r.left, r.bottom - d, d, d), 90, 90);
    path.closeFigure();
    return path;
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/admin/credential_slips_pdf.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/admin/credential_slips_source.dart

Credential slip PDF generation for printed login cards. This file (credential slips source) is one concrete piece of that layer.

Line count: 192. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math';
- import 'package:mu_study/core/admin/credential_slip.dart';
- import 'package:mu_study/models/enums.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:shared_preferences/shared_preferences.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/admin/credential_slips_source.dart**

```
import 'dart:math';

import 'package:mu_study/core/admin/credential_slip.dart';
import 'package:mu_study/models/enums.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const _passwordStorePrefix = 'login_slip_pw_';

Future<void> saveLoginPassword({
  required String userId,
  required String password,
}) async {
  if (userId.isEmpty || password.isEmpty) return;
  await _writeStoredPassword(userId, password);
  try {
    await supabase.rpc(
      'admin_save_login_password',
      params: {'p_user_id': userId, 'p_password': password},
    );
  } catch (_) {
    // Optional RPC; local store + auth reset are enough to print.
  }
}

Future<void> saveLoginPasswordsByUsername(
  Iterable<Map<String, dynamic>> rows,
) async {
  for (final row in rows) {
    final username = (row['username'] as String?)?.trim() ?? '';
    final password = (row['password'] as String?) ?? '';
    if (username.isEmpty || password.isEmpty) continue;
    try {
      final profile = await supabase
          .from('profiles')
          .select('id')
          .eq('username', username)
          .maybeSingle();
      final id = profile?['id'] as String?;
```

```
      if (id == null) continue;
      await saveLoginPassword(userId: id, password: password);
    } catch (_) {}
  }
}

Future<List<CredentialSlip>> fetchCredentialSlips({
  required UserRole role,
  String? groupId,
}) async {
  var slips = await _fetchFromRpc(role: role, groupId: groupId);
  slips ??= await fetchCredentialSlipPreviews(role: role, groupId: groupId);
  return uniqueCredentialSlips(await _ensurePrintablePasswords(slips));
}

Future<List<CredentialSlip>?> _fetchFromRpc({
  required UserRole role,
  String? groupId,
}) async {
  try {
    final rows = await supabase.rpc(
      'admin_list_login_slips',
      params: {
        'p_role': role.name,
        'p_group_id': groupId,
      },
    );
    if (rows is! List) return null;
    return uniqueCredentialSlips(
      rows.map((raw) {
        final row = Map<String, dynamic>.from(raw as Map);
        return CredentialSlip(
          profileId: row['profile_id'] as String?,
          username: (row['username'] as String? ?? '').trim(),
          fullName: (row['full_name'] as String? ?? '').trim(),
          password: (row['password'] as String? ?? '').trim(),
        );
      }),
    );
  } on PostgrestException {
    return null;
  }
}

Future<List<CredentialSlip>> fetchCredentialSlipPreviews({
  required UserRole role,
  String? groupId,
}) async {
  final slips = <CredentialSlip>[];

  if (role == UserRole.student && groupId != null) {
    final rows = await supabase
        .from('group_students')
        .select(
          'students(profile_id, profiles(id, username, full_name, role, is_disabled))',
        )
        .eq('group_id', groupId);
    for (final raw in rows) {
      final student = raw['students'] as Map?;
      final profile = student?['profiles'] as Map?;
      final slip = _slipFromProfile(profile);
      if (slip != null) slips.add(slip);
    }
    return uniqueCredentialSlips(slips);
  }

  final rows = await supabase
      .from('profiles')
      .select('id, username, full_name, role, is_disabled')
      .eq('role', role.name)
      .eq('is_disabled', false)
      .order('full_name');
  for (final raw in rows) {
```

```
    final slip = _slipFromProfile(raw);
    if (slip != null) slips.add(slip);
  }
  return uniqueCredentialSlips(slips);
}

CredentialSlip? _slipFromProfile(Map<dynamic, dynamic>? json) {
  if (json == null) return null;
  if (json['is_disabled'] == true) return null;
  final username = (json['username'] as String? ?? '').trim();
  if (username.isEmpty) return null;
  return CredentialSlip(
    profileId: json['id'] as String?,
    username: username,
    fullName: (json['full_name'] as String? ?? '').trim(),
    password: '',
  );
}

Future<List<CredentialSlip>> _ensurePrintablePasswords(
  List<CredentialSlip> slips,
) async {
  final used = <String>{
    for (final slip in slips)
      if (slip.password.isNotEmpty) slip.password,
  };
  final out = <CredentialSlip>[];

  for (final slip in slips) {
    if (slip.password.isNotEmpty) {
      final id = slip.profileId;
      if (id != null) await _writeStoredPassword(id, slip.password);
      out.add(slip);
      continue;
    }

    final id = slip.profileId;
    if (id == null || id.isEmpty) continue;

    final stored = await _readStoredPassword(id);
    if (stored != null && stored.isNotEmpty) {
      used.add(stored);
      out.add(slip.copyWith(password: stored));
      continue;
    }

    var password = _newPassword();
    while (used.contains(password)) {
      password = _newPassword();
    }
    used.add(password);

    await supabase.rpc(
      'admin_reset_password',
      params: {'p_user_id': id, 'p_password': password},
    );
    await saveLoginPassword(userId: id, password: password);
    out.add(slip.copyWith(password: password));
  }

  return out;
}

Future<String?> _readStoredPassword(String userId) async {
  final prefs = await SharedPreferences.getInstance();
  return prefs.getString('$_passwordStorePrefix$userId');
}

Future<void> _writeStoredPassword(String userId, String password) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('$_passwordStorePrefix$userId', password);
}
```

```
String _newPassword() {
  const alphabet =
      'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
  final rng = Random.secure();
  return List.generate(12, (_) => alphabet[rng.nextInt(alphabet.length)]).join();
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/admin/credential_slips_source.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/chat/animated_emoji_registry.dart

Emoji helpers and animated emoji registry. This file (animated emoji registry) is one concrete piece of that layer.

Line count: 130. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/chat/animated_emoji_registry.dart**

```
library;

const String kAnimatedEmojiAssetPrefix = 'assets/lottie/';

const Map<String, String> kAnimatedEmojiAssets = {
  '❤■': 'Heart.json',
  '❤': 'Heart.json',
  '■': 'Heart eyes.json',
  '■': 'Emoji 30 - Lovely Kiss.json', // face + hearts (kiss), not ■ mark
  '■': 'Emoji 30 - Lovely Kiss.json',

  '■': 'LMAO-2 side tears.json',
  '■': 'LMAO.json',
  '■': 'smiley emoji.json', // composition: Grinning Face
  '■': 'Cool emoji.json',
  '■': 'Winking Emoji.json',
  '■': 'Zany tongue.json',
  '■': 'Calm.json',
  '■': 'Yummy.json',

  '■': 'Crying emoji.json',
  '■': 'Sad emoji to Crying emoji.json',
  '■': 'Pensive.json',
  '■': 'Sad Emoji.json', // composition is pensive-like; closest sad we have
  '■': 'Holding back tears.json',

  '■': 'Angry Face.json',
  '■': 'Angry.json', // composition: emoji_pouting
  '■': 'Mad emoji.json', // composition: steam from nose
  '■': 'Angry Face with Horns.json',
  '■': 'Angry Face with Symbols on Mouth Emoji.json',
  '■': 'Confounded Face.json', // filename; layers read as confused mouth
  '■': 'Confounded Face.json',
  '■': 'Puke Emoji.json',
  '■': 'Sick face with thermometer.json',
  '■': 'Sick face with thermometer.json',
  '■': 'injury head face emoji.json',
  '■': 'Dizzy Face.json',
  '■': 'Cold Face.json',
  '■': 'High Temperature.json',
  '■': 'Sleepy.json',
  '■': 'Yawn Emoji.json',
  '■': 'Drooling Face.json',
  '■': 'Zipper mouth Emoji.json',
  '■': 'Rolling Eyes Emoji.json',
  '■': 'Neutral face.json',
  '■': 'Thinking.json',
  '■': 'Anxious Face with Sweat.json',
  '■': 'Astonished Face.json',
  '■': 'Astonished Face.json',
```

```
  '■': 'Party.json',
  '■': 'Confetti emoji.json',
  '■': 'Confetti emoji.json',
  '■': 'Sparkles Emoji.json',
  '■': 'Star Strike Emoji.json',
  '■': 'Sparkles Emoji.json', // sparkles, not a star-struck face

  '■': 'Thumb.json',
  '■': 'Clapping Hands.json',
  '■': 'Muscle.json',
  '■': 'Please.json', // U+1F64F folded hands
  '■': 'Crossed Fingers.json',
  '■■': 'Index finger.json',
  '■': 'Backhand Index Pointing Up.json',
  '■': 'Backhand Index Pointing Down.json',
  '■': 'Backhand Index Pointing Right.json',
  '■': 'Call Me Hand.json',
  '■': 'Hand.json',
  '■': 'Hand.json',
  '■': 'Middle finger.json',

  '■': 'Fire.json',
  '■': 'check-mark.json',
  '✔■': 'check-mark.json',
  '✔': 'check-mark.json',
  '■■': 'check-mark.json',
  '■': 'check-mark.json',
  '■': 'X.json',
  '■': 'Eyes.json',
  '■': 'Rocket.json',
  '■': 'Money.json',
  '■': 'FlyingMoney.json',
  '■': 'Unicorn Emoji.json',
  '■': 'Alien Emoji.json',
  '■': 'Animated Clown Face.json',
  '■': 'Cowboy Hat Face.json',
  '■': 'Halo.json',
  '■': 'Taxi.json',
  '■': 'Laptop.json',
  '■': 'Hourglass.json',
  '■': 'Hourglass.json',
  '■': '100% Emoji.json',
  '■': 'Bomb Emoji.json',
  '■': 'emoji head explosion.json',
  '■': 'emoji head explosion.json', // exploding head
  '■': 'Medal Animations.json',
  '■': 'Medal Animations.json',
  '■■': 'High Temperature.json',
  '■': 'Emoji Walking.json',
  '■■■■': 'Emoji Walking.json',
  '■■■■': 'Emoji Walking.json',
};

String? animatedEmojiAssetPath(String emoji) {
  final direct = kAnimatedEmojiAssets[emoji];
  if (direct != null) return '$kAnimatedEmojiAssetPrefix$direct';

  final stripped = emoji.replaceAll('\uFE0F', '').replaceAll('\uFE0E', '');
  if (stripped != emoji) {
    final alt = kAnimatedEmojiAssets[stripped];
    if (alt != null) return '$kAnimatedEmojiAssetPrefix$alt';
  }

  final withVs16 = '$stripped\uFE0F';
  final vs = kAnimatedEmojiAssets[withVs16];
  if (vs != null) return '$kAnimatedEmojiAssetPrefix$vs';

  return null;
}

bool isAnimatedEmojiSupported(String emoji) =>
    animatedEmojiAssetPath(emoji) != null;
```

```
Set<String> get registeredAnimatedEmojiFilenames =>
    kAnimatedEmojiAssets.values.toSet();

const Set<String> kAnimatedEmojiRedundantFilenames = {
  'Starstruck.json', // same star-struck face as Star Strike Emoji.json → ■
};
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/chat/animated_emoji_registry.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/chat/emoji_message.dart

Emoji helpers and animated emoji registry. This file (emoji message) is one concrete piece of that layer.

Line count: 51. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:characters/characters.dart';
- import 'package:mu_study/core/chat/animated_emoji_registry.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/chat/emoji_message.dart**

```
import 'package:characters/characters.dart';
import 'package:mu_study/core/chat/animated_emoji_registry.dart';

String? singleAnimatedEmojiAsset(String? text) {
  if (text == null || text.isEmpty) return null;

  final trimmed = text.trim();
  if (trimmed.isEmpty) return null;

  if (_containsWhitespace(trimmed)) return null;

  final graphemes = trimmed.characters;
  if (graphemes.length != 1) return null;

  final emoji = graphemes.first;
  return animatedEmojiAssetPath(emoji) ??
      animatedEmojiAssetPath(stripEmojiVariationSelectors(emoji));
}

bool isSingleAnimatedEmojiMessage(String? text) =>
    singleAnimatedEmojiAsset(text) != null;

String? singleAnimatedEmojiGrapheme(String? text) {
  if (!isSingleAnimatedEmojiMessage(text)) return null;
  return text!.trim().characters.first;
}

String stripEmojiVariationSelectors(String emoji) {
  final buffer = StringBuffer();
  for (final r in emoji.runes) {
    if (r == 0xFE0F || r == 0xFE0E) continue;
    buffer.writeCharCode(r);
  }
  return buffer.toString();
}

bool _containsWhitespace(String text) {
  for (final unit in text.runes) {
    if (unit == 0x20 ||
        unit == 0x09 ||
        unit == 0x0A ||
        unit == 0x0D ||
        unit == 0x200B ||
        unit == 0x00A0 ||
        unit == 0xFEFF ||
        unit == 0x2060) {
      return true;
    }
```

```
    }
    return false;
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/chat/emoji_message.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/constants/env.dart

Compile-time and runtime configuration (Env). This file (env) is one concrete piece of that layer.

Line count: 51. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter_dotenv/flutter_dotenv.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/constants/env.dart**

```
import 'package:flutter_dotenv/flutter_dotenv.dart';

class Env {
  static const _defaultUrl = 'https://nmmdvdprczukaskxxdrs.supabase.co';
  static const _placeholderKey = 'your_anon_key_here';

  static String supabaseUrl = _defaultUrl;
  static String supabaseAnonKey = '';

  static bool get isConfigured => supabaseAnonKey.isNotEmpty;

  static void _sanitizeKey() {
    final key = supabaseAnonKey.trim();
    if (key.isEmpty ||
        key == _placeholderKey ||
        key.startsWith('your_') ||
        key.contains('paste_your') ||
        key.startsWith('sb_secret_')) {
      supabaseAnonKey = '';
      return;
    }
    if (!key.startsWith('eyJ') && !key.startsWith('sb_publishable_')) {
      supabaseAnonKey = '';
    }
  }

  static Future<void> load() async {
    const defineUrl = String.fromEnvironment('SUPABASE_URL');
    const defineKey = String.fromEnvironment('SUPABASE_ANON_KEY');

    if (defineKey.isNotEmpty) {
      supabaseAnonKey = defineKey;
      if (defineUrl.isNotEmpty) supabaseUrl = defineUrl;
      _sanitizeKey();
      return;
    }

    try {
      await dotenv.load(fileName: '.env');
      supabaseAnonKey =
          (dotenv.env['SUPABASE_PUBLISHABLE_KEY'] ??
                  dotenv.env['SUPABASE_ANON_KEY'])
              ?.trim() ??
          '';
      final url = dotenv.env['SUPABASE_URL']?.trim();
      if (url != null && url.isNotEmpty) supabaseUrl = url;
    } catch (_) {}

    _sanitizeKey();
  }
```

```
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/constants/env.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

# lib/core/focus/focus_time_up_cue.dart

Focus-mode audio/visual cues (local, not a database table). This file (focus time up cue) is one concrete piece of that layer.

Line count: 10. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/services.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/focus/focus_time_up_cue.dart**

```
import 'package:flutter/services.dart';

Future<void> playFocusTimeUpCue() async {
  try {
    await HapticFeedback.heavyImpact();
    await Future<void>.delayed(const Duration(milliseconds: 140));
    await HapticFeedback.mediumImpact();
    await SystemSound.play(SystemSoundType.alert);
  } catch (_) {}
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/focus/focus_time_up_cue.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/homework/assignment_due.dart

Due-date helpers and assignment template JSON. This file (assignment due) is one concrete piece of that layer.

Line count: 85. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/homework/assignment_due.dart**

```
import 'package:mu_study/models/models.dart';

enum AssignmentDueStatus { none, upcoming, dueSoon, dueToday, overdue }

DateTime assignmentCalendarDay(DateTime date) =>
    DateTime(date.year, date.month, date.day);

DateTime? assignmentDueDay(Assignment a) {
  final d = a.dueDate;
  if (d == null) return null;
  return assignmentCalendarDay(d);
}

DateTime startOfWeek(DateTime day) {
  final d = assignmentCalendarDay(day);
  return d.subtract(Duration(days: d.weekday - DateTime.monday));
}

DateTime endOfWeek(DateTime day) =>
    startOfWeek(day).add(const Duration(days: 6));

AssignmentDueStatus assignmentDueStatus(Assignment a, [DateTime? now]) {
  final due = assignmentDueDay(a);
  if (due == null) return AssignmentDueStatus.none;
  final today = assignmentCalendarDay(now ?? DateTime.now());
  if (due.isBefore(today)) return AssignmentDueStatus.overdue;
  if (due == today) return AssignmentDueStatus.dueToday;
  final days = due.difference(today).inDays;
  if (days <= 2) return AssignmentDueStatus.dueSoon;
  return AssignmentDueStatus.upcoming;
}

bool assignmentIsDueThisWeek(Assignment a, [DateTime? now]) {
  final due = assignmentDueDay(a);
  if (due == null) return false;
  final n = now ?? DateTime.now();
  final start = startOfWeek(n);
  final end = endOfWeek(n);
  return !due.isBefore(start) && !due.isAfter(end);
}

bool assignmentIsOverdue(Assignment a, [DateTime? now]) =>
    assignmentDueStatus(a, now) == AssignmentDueStatus.overdue;

List<Assignment> sortAssignmentsByDue(
  Iterable<Assignment> items, [
  DateTime? now,
]) {
```

```
  final list = items.toList();
  final n = now ?? DateTime.now();
  list.sort((a, b) {
    final da = assignmentDueDay(a);
    final db = assignmentDueDay(b);
    if (da == null && db == null) {
      return (b.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0)).compareTo(
        a.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0),
      );
    }
    if (da == null) return 1;
    if (db == null) return -1;
    final today = assignmentCalendarDay(n);
    final aOver = da.isBefore(today);
    final bOver = db.isBefore(today);
    if (aOver != bOver) return aOver ? -1 : 1;
    return da.compareTo(db);
  });
  return list;
}

List<Assignment> assignmentsDueThisWeekFocus(
  Iterable<Assignment> items, [
  DateTime? now,
]) {
  final n = now ?? DateTime.now();
  final filtered = items.where((a) {
    if (assignmentIsOverdue(a, n)) return true;
    return assignmentIsDueThisWeek(a, n);
  });
  return sortAssignmentsByDue(filtered, n);
}

List<Assignment> assignmentsForFocusSession(
  Iterable<Assignment> items, [
  DateTime? now,
]) => sortAssignmentsByDue(items, now);
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/homework/assignment_due.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/homework/assignment_templates.dart

Due-date helpers and assignment template JSON. This file (assignment templates) is one concrete piece of that layer.

Line count: 297. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/homework/assignment_templates.dart**

```
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class TemplateTaskDraft {
  const TemplateTaskDraft({
    required this.title,
    this.description = '',
    this.attachmentUrl,
    this.linkUrl,
    this.sortOrder = 0,
  });

  final String title;
  final String description;
  final String? attachmentUrl;
  final String? linkUrl;
  final int sortOrder;

  factory TemplateTaskDraft.fromJson(Map<String, dynamic> json) {
    return TemplateTaskDraft(
      title: json['title'] as String? ?? '',
      description: json['description'] as String? ?? '',
      attachmentUrl: json['attachment_url'] as String?,
      linkUrl: json['link_url'] as String?,
      sortOrder: json['sort_order'] as int? ?? 0,
    );
  }

  factory TemplateTaskDraft.fromTask(TaskItem task) {
    return TemplateTaskDraft(
      title: task.title,
      description: task.description,
      attachmentUrl: task.attachmentUrl,
      linkUrl: task.linkUrl,
      sortOrder: task.sortOrder,
    );
  }

  Map<String, dynamic> toJson() => {
    'title': title,
    'description': description,
    'attachment_url': attachmentUrl,
    'link_url': linkUrl,
    'sort_order': sortOrder,
  };
}
```

```
class AssignmentTemplate {
  const AssignmentTemplate({
    required this.id,
    required this.teacherId,
    required this.type,
    required this.title,
    this.description = '',
    this.tasks = const [],
    this.createdAt,
    this.updatedAt,
  });

  final String id;
  final String teacherId;
  final AssignmentType type;
  final String title;
  final String description;
  final List<TemplateTaskDraft> tasks;
  final DateTime? createdAt;
  final DateTime? updatedAt;

  factory AssignmentTemplate.fromJson(Map<String, dynamic> json) {
    final rawTasks = json['tasks'];
    final tasks = <TemplateTaskDraft>[];
    if (rawTasks is List) {
      for (final item in rawTasks) {
        if (item is Map) {
          tasks.add(
            TemplateTaskDraft.fromJson(Map<String, dynamic>.from(item)),
          );
        }
      }
    }
    return AssignmentTemplate(
      id: json['id'] as String,
      teacherId: json['teacher_id'] as String,
      type: assignmentTypeFromString(json['type'] as String? ?? 'static'),
      title: json['title'] as String? ?? '',
      description: json['description'] as String? ?? '',
      tasks: tasks,
      createdAt: json['created_at'] != null
          ? DateTime.tryParse(json['created_at'] as String)
          : null,
      updatedAt: json['updated_at'] != null
          ? DateTime.tryParse(json['updated_at'] as String)
          : null,
    );
  }
}

Future<List<AssignmentTemplate>> fetchAssignmentTemplates(
  String teacherId,
) async {
  final rows = await supabase
      .from('assignment_templates')
      .select()
      .eq('teacher_id', teacherId)
      .order('updated_at', ascending: false);
  return rows
      .map((e) => AssignmentTemplate.fromJson(Map<String, dynamic>.from(e)))
      .toList();
}

Future<AssignmentTemplate> saveAssignmentAsTemplate({
  required String teacherId,
  required Assignment assignment,
}) async {
  final taskRows = await supabase
      .from('tasks')
      .select()
      .eq('assignment_id', assignment.id)
      .order('sort_order', ascending: true);
  var drafts = taskRows
```

```
      .map(
        (e) => TemplateTaskDraft.fromTask(
          TaskItem.fromJson(Map<String, dynamic>.from(e)),
        ),
      )
      .toList();
  if (assignment.type == AssignmentType.staticAssignment) {
    if (drafts.isEmpty) {
      drafts = [
        TemplateTaskDraft(
          title: assignment.title,
          description: assignment.description,
        ),
      ];
    } else {
      final first = drafts.first;
      drafts = [
        TemplateTaskDraft(
          title: first.title,
          description: first.description,
          attachmentUrl: first.attachmentUrl,
          linkUrl: first.linkUrl,
        ),
      ];
    }
  }

  final row = await supabase
      .from('assignment_templates')
      .insert({
        'teacher_id': teacherId,
        'type': assignmentTypeToDb(assignment.type),
        'title': assignment.title,
        'description': assignment.description,
        'tasks': drafts.map((d) => d.toJson()).toList(),
        'updated_at': DateTime.now().toIso8601String(),
      })
      .select()
      .single();

  return AssignmentTemplate.fromJson(Map<String, dynamic>.from(row));
}

Future<void> deleteAssignmentTemplate(String templateId) async {
  await supabase.from('assignment_templates').delete().eq('id', templateId);
}

Future<Assignment> createAssignmentFromTemplate({
  required AssignmentTemplate template,
  required String teacherId,
  required String groupId,
  DateTime? dueDate,
  String? titleOverride,
  String? descriptionOverride,
}) async {
  final inserted = await supabase
      .from('assignments')
      .insert({
        'teacher_id': teacherId,
        'group_id': groupId,
        'type': assignmentTypeToDb(template.type),
        'title': titleOverride?.trim().isNotEmpty == true
            ? titleOverride!.trim()
            : template.title,
        'description': descriptionOverride ?? template.description,
        'due_date': dueDate?.toIso8601String(),
      })
      .select()
      .single();

  final assignment = Assignment.fromJson(Map<String, dynamic>.from(inserted));

  final title = assignment.title;
```

```
    final description = assignment.description;
    List<TemplateTaskDraft> tasksToInsert;
    if (template.type == AssignmentType.staticAssignment) {
      final first = template.tasks.isNotEmpty ? template.tasks.first : null;
      tasksToInsert = [
        TemplateTaskDraft(
          title: first?.title.isNotEmpty == true ? first!.title : title,
          description: first?.description ?? description,
          attachmentUrl: first?.attachmentUrl,
          linkUrl: first?.linkUrl,
        ),
      ];
    } else {
      tasksToInsert = template.tasks;
    }

    if (tasksToInsert.isNotEmpty) {
      await supabase.from('tasks').insert([
        for (var i = 0; i < tasksToInsert.length; i++)
          {
            'assignment_id': assignment.id,
            'title': tasksToInsert[i].title,
            'description': tasksToInsert[i].description,
            'attachment_url': tasksToInsert[i].attachmentUrl,
            'link_url': tasksToInsert[i].linkUrl,
            'sort_order': i,
          },
      ]);
    }

    return assignment;
  }

  Future<Assignment> duplicateAssignment({
    required Assignment source,
    required String teacherId,
    required String groupId,
    DateTime? dueDate,
    String? titleOverride,
  }) async {
    final taskRows = await supabase
        .from('tasks')
        .select()
        .eq('assignment_id', source.id)
        .order('sort_order', ascending: true);

    final title = titleOverride?.trim().isNotEmpty == true
        ? titleOverride!.trim()
        : '${source.title} (copy)';

    final inserted = await supabase
        .from('assignments')
        .insert({
          'teacher_id': teacherId,
          'group_id': groupId,
          'type': assignmentTypeToDb(source.type),
          'title': title,
          'description': source.description,
          'due_date': dueDate?.toIso8601String(),
        })
        .select()
        .single();

    final assignment = Assignment.fromJson(Map<String, dynamic>.from(inserted));

    if (source.type == AssignmentType.staticAssignment) {
      if (taskRows.isEmpty) {
        await supabase.from('tasks').insert({
          'assignment_id': assignment.id,
          'title': title,
          'description': source.description,
          'sort_order': 0,
        });
```

```
    } else {
      final t = TaskItem.fromJson(Map<String, dynamic>.from(taskRows.first));
      await supabase.from('tasks').insert({
        'assignment_id': assignment.id,
        'title': t.title,
        'description': t.description,
        'attachment_url': t.attachmentUrl,
        'link_url': t.linkUrl,
        'sort_order': 0,
      });
    }
  } else if (taskRows.isNotEmpty) {
    final tasks = [
      for (final row in taskRows)
        TaskItem.fromJson(Map<String, dynamic>.from(row)),
    ];
    await supabase.from('tasks').insert([
      for (var i = 0; i < tasks.length; i++)
        {
          'assignment_id': assignment.id,
          'title': tasks[i].title,
          'description': tasks[i].description,
          'attachment_url': tasks[i].attachmentUrl,
          'link_url': tasks[i].linkUrl,
          'sort_order': i,
        },
    ]);
  }

  return assignment;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/homework/assignment_templates.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/homework/feedback_reuse.dart

Due-date helpers and assignment template JSON. This file (feedback reuse) is one concrete piece of that layer.

Line count: 131. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:convert';
- import 'package:shared_preferences/shared_preferences.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/homework/feedback_reuse.dart**

```
import 'dart:convert';

import 'package:shared_preferences/shared_preferences.dart';
import 'package:mu_study/supabase/supabase_client.dart';

const _snippetsPrefsKey = 'teacher_feedback_snippets';
const _maxSnippets = 20;

Future<List<String>> loadFeedbackSnippets({required String teacherId}) async {
  final prefs = await SharedPreferences.getInstance();
  final local = _readLocal(prefs, teacherId);

  final remote = await _fetchRecentFeedbackFromDb(teacherId);
  return _mergeUnique(local, remote);
}

Future<void> rememberFeedbackSnippet({
  required String teacherId,
  required String feedback,
}) async {
  final text = feedback.trim();
  if (text.isEmpty) return;

  final prefs = await SharedPreferences.getInstance();
  final current = _readLocal(prefs, teacherId);
  final next = _mergeUnique([text], current).take(_maxSnippets).toList();

  final all = _readAll(prefs);
  all[teacherId] = next;
  await prefs.setString(_snippetsPrefsKey, jsonEncode(all));
}

Future<int> applyFeedbackToOtherPendingForSameTask({
  required String submissionId,
  required String feedback,
}) async {
  final text = feedback.trim();
  if (text.isEmpty) return 0;

  final row = await supabase
      .from('submissions')
      .select('task_id')
      .eq('id', submissionId)
      .maybeSingle();
  final taskId = row?['task_id'] as String?;
```

```
    if (taskId == null) return 0;

    final siblings = await supabase
        .from('submissions')
        .select('id')
        .eq('task_id', taskId)
        .neq('id', submissionId)
        .inFilter('status', ['pending', 'submitted']);

    if (siblings.isEmpty) return 0;

    final ids = siblings.map((e) => e['id'] as String).toList();
    await supabase
        .from('submissions')
        .update({'feedback': text})
        .inFilter('id', ids);
    return ids.length;
  }

  Future<List<String>> _fetchRecentFeedbackFromDb(String teacherId) async {
    final assignments = await supabase
        .from('assignments')
        .select('id')
        .eq('teacher_id', teacherId);
    final assignmentIds = assignments.map((e) => e['id'] as String).toList();
    if (assignmentIds.isEmpty) return const [];

    final tasks = await supabase
        .from('tasks')
        .select('id')
        .inFilter('assignment_id', assignmentIds);
    final taskIds = tasks.map((e) => e['id'] as String).toList();
    if (taskIds.isEmpty) return const [];

    final rows = await supabase
        .from('submissions')
        .select('feedback')
        .inFilter('task_id', taskIds)
        .order('submitted_at', ascending: false)
        .limit(40);

    final out = <String>[];
    for (final row in rows) {
      final f = (row['feedback'] as String?)?.trim();
      if (f == null || f.isEmpty) continue;
      if (!out.contains(f)) out.add(f);
      if (out.length >= _maxSnippets) break;
    }
    return out;
  }

  List<String> _readLocal(SharedPreferences prefs, String teacherId) {
    final all = _readAll(prefs);
    final list = all[teacherId];
    if (list is! List) return const [];
    return list
        .whereType<String>()
        .map((e) => e.trim())
        .where((e) => e.isNotEmpty)
        .toList();
  }

  Map<String, dynamic> _readAll(SharedPreferences prefs) {
    final raw = prefs.getString(_snippetsPrefsKey);
    if (raw == null || raw.isEmpty) return {};
    try {
      final decoded = jsonDecode(raw);
      if (decoded is Map<String, dynamic>) return decoded;
      if (decoded is Map) {
        return decoded.map((k, v) => MapEntry(k.toString(), v));
      }
    } catch (_) {}
    return {};
```

```
}

List<String> _mergeUnique(List<String> primary, List<String> secondary) {
  final seen = <String>{};
  final out = <String>[];
  for (final item in [...primary, ...secondary]) {
    final t = item.trim();
    if (t.isEmpty || seen.contains(t)) continue;
    seen.add(t);
    out.add(t);
  }
  return out;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/homework/feedback_reuse.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/core/locale/app_locale.dart

Supported locales EN/RU/UZ. This file (app locale) is one concrete piece of that layer.

Line count: 24. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/locale/app_locale.dart**

```
import 'package:flutter/material.dart';

enum AppLanguage {
  english('en'),
  russian('ru'),
  uzbek('uz');

  const AppLanguage(this.code);
  final String code;

  Locale get locale => Locale(code);

  static AppLanguage? tryParse(String? code) {
    if (code == null || code.isEmpty) return null;
    for (final lang in AppLanguage.values) {
      if (lang.code == code) return lang;
    }
    return null;
  }

  static AppLanguage fromLocale(Locale locale) {
    return tryParse(locale.languageCode) ?? AppLanguage.english;
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/locale/app_locale.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/marks/academic_term.dart

Academic term math and CSV export. This file (academic term) is one concrete piece of that layer.

Line count: 278. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/marks/academic_term.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AcademicTerm {
  const AcademicTerm({
    required this.id,
    required this.academicYearStart,
    required this.semester,
    this.startInclusive,
    this.endInclusive,
    this.yearIsCurrent = false,
    this.dbTermId,
    this.dbYearId,
  });

  final String id;

  final int? academicYearStart;

  final int? semester;

  final DateTime? startInclusive;
  final DateTime? endInclusive;

  final bool yearIsCurrent;

  final String? dbTermId;
  final String? dbYearId;

  bool get isAll => id == 'all';

  static const AcademicTerm all = AcademicTerm(
    id: 'all',
    academicYearStart: null,
    semester: null,
  );

  static (DateTime, DateTime) bounds(int academicYearStart, int semester) {
    assert(semester >= 1 && semester <= 4);
    return switch (semester) {
      1 => (
        DateTime(academicYearStart, 8, 1),
        DateTime(academicYearStart, 11, 2),
      ),
      2 => (
        DateTime(academicYearStart, 11, 11),
```

```
      DateTime(academicYearStart, 12, 28),
    ),
    3 => (
      DateTime(academicYearStart + 1, 1, 11),
      DateTime(academicYearStart + 1, 3, 20),
    ),
    _ => (
      DateTime(academicYearStart + 1, 3, 28),
      DateTime(academicYearStart + 1, 6, 19),
    ),
  };
}

static AcademicTerm semesterOf(
  int academicYearStart,
  int semester, {
  bool yearIsCurrent = false,
  String? dbTermId,
  String? dbYearId,
  DateTime? startInclusive,
  DateTime? endInclusive,
}) {
  assert(semester >= 1 && semester <= 4);
  final (start, end) = bounds(academicYearStart, semester);
  return AcademicTerm(
    id: '$academicYearStart-$semester',
    academicYearStart: academicYearStart,
    semester: semester,
    startInclusive: startInclusive ?? start,
    endInclusive: endInclusive ?? end,
    yearIsCurrent: yearIsCurrent,
    dbTermId: dbTermId,
    dbYearId: dbYearId,
  );
}

static int academicYearFor(DateTime now) =>
    now.month >= 8 ? now.year : now.year - 1;

static int semesterFor(DateTime now) {
  final ay = academicYearFor(now);
  final day = DateTime(now.year, now.month, now.day);
  for (var s = 1; s <= 4; s++) {
    final (start, end) = bounds(ay, s);
    if (!day.isBefore(start) && !day.isAfter(end)) return s;
  }
  for (var s = 1; s <= 4; s++) {
    final (start, _) = bounds(ay, s);
    if (day.isBefore(start)) return s;
  }
  return 4;
}

static AcademicTerm currentFrom(List<AcademicTerm> options, [DateTime? now]) {
  final n = now ?? DateTime.now();
  final day = DateTime(n.year, n.month, n.day);
  final terms = options.where((t) => !t.isAll).toList();
  if (terms.isEmpty) return all;

  final currentYearTerms = terms.where((t) => t.yearIsCurrent).toList();
  final pool = currentYearTerms.isNotEmpty ? currentYearTerms : terms;

  final matching = pool.where((t) {
    final start = t.startInclusive;
    final end = t.endInclusive;
    if (start == null || end == null) return false;
    return !day.isBefore(start) && !day.isAfter(end);
  }).toList();
  if (matching.isNotEmpty) return matching.first;

  pool.sort((a, b) => (a.semester ?? 0).compareTo(b.semester ?? 0));
  final first = pool.first;
  final last = pool.last;
```

```
    if (first.startInclusive != null && day.isBefore(first.startInclusive!)) {
      return all;
    }

    if (last.endInclusive != null && day.isAfter(last.endInclusive!)) {
      return all;
    }

    for (final t in pool) {
      final start = t.startInclusive;
      if (start != null && day.isBefore(start)) return t;
    }
    return last;
  }

  String label(AppLocalizations l10n) {
    if (isAll) return l10n.marksTermAll;
    final start = academicYearStart!;
    final endShort = (start + 1) % 100;
    final yearRange = '$start–${endShort.toString().padLeft(2, '0')}';
    return l10n.marksTermSemester(yearRange, semester!);
  }

  String? get startKey => startInclusive?.toIso8601String().split('T').first;

  String? get endKey => endInclusive?.toIso8601String().split('T').first;
}

class AcademicYearRecord {
  const AcademicYearRecord({
    required this.id,
    required this.startYear,
    required this.label,
    required this.isCurrent,
    required this.terms,
  });

  final String id;
  final int startYear;
  final String label;
  final bool isCurrent;
  final List<AcademicTerm> terms;

  factory AcademicYearRecord.fromJson(
    Map<String, dynamic> json,
    List<Map<String, dynamic>> termRows,
  ) {
    final yearId = json['id'] as String;
    final startYear = json['start_year'] as int;
    final isCurrent = json['is_current'] as bool? ?? false;
    final terms = termRows.map((t) {
      final semester = t['semester'] as int;
      final start = DateTime.parse(t['starts_on'] as String);
      final end = DateTime.parse(t['ends_on'] as String);
      return AcademicTerm.semesterOf(
        startYear,
        semester,
        yearIsCurrent: isCurrent,
        dbTermId: t['id'] as String?,
        dbYearId: yearId,
        startInclusive: DateTime(start.year, start.month, start.day),
        endInclusive: DateTime(end.year, end.month, end.day),
      );
    }).toList()..sort((a, b) => (a.semester ?? 0).compareTo(b.semester ?? 0));

    return AcademicYearRecord(
      id: yearId,
      startYear: startYear,
      label:
          json['label'] as String? ??
          '$startYear–${((startYear + 1) % 100).toString().padLeft(2, '0')}',
      isCurrent: isCurrent,
      terms: terms,
```

```
    );
  }
}

Future<List<AcademicYearRecord>> fetchAcademicYears() async {
  final years = await supabase
      .from('academic_years')
      .select('id, start_year, label, is_current, created_at')
      .order('start_year', ascending: false);

  final yearList = (years as List).cast<Map<String, dynamic>>();
  if (yearList.isEmpty) return const [];

  final terms = await supabase
      .from('academic_terms')
      .select('id, academic_year_id, semester, starts_on, ends_on')
      .order('semester');

  final termList = (terms as List).cast<Map<String, dynamic>>();
  final byYear = <String, List<Map<String, dynamic>>>{};
  for (final t in termList) {
    final yid = t['academic_year_id'] as String;
    byYear.putIfAbsent(yid, () => []).add(t);
  }

  return [
    for (final y in yearList)
      AcademicYearRecord.fromJson(y, byYear[y['id'] as String] ?? const []),
  ];
}

final academicTermOptionsProvider = FutureProvider<List<AcademicTerm>>((
  ref,
) async {
  final years = await fetchAcademicYears();
  return [AcademicTerm.all, for (final y in years) ...y.terms];
});

final academicYearsProvider = FutureProvider<List<AcademicYearRecord>>((
  ref,
) async {
  return fetchAcademicYears();
});

class SelectedMarkTermNotifier extends Notifier<AcademicTerm> {
  bool _initializedFromDb = false;

  @override
  AcademicTerm build() {
    ref.listen(academicTermOptionsProvider, (prev, next) {
      next.whenData((options) {
        if (options.length <= 1) return;
        final current = state;
        AcademicTerm? match;
        for (final t in options) {
          if (t.id == current.id) {
            match = t;
            break;
          }
        }
        if (!_initializedFromDb || match == null) {
          _initializedFromDb = true;
          state = AcademicTerm.currentFrom(options);
          return;
        }
        if (match.startKey != current.startKey ||
            match.endKey != current.endKey ||
            match.yearIsCurrent != current.yearIsCurrent) {
          state = match;
        }
      });
    });
    return AcademicTerm.all;
```

```
  }

  void select(AcademicTerm term) {
    state = term;
  }
}

final selectedMarkTermProvider =
    NotifierProvider<SelectedMarkTermNotifier, AcademicTerm>(
      SelectedMarkTermNotifier.new,
    );
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/marks/academic_term.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/marks/marks_csv_export.dart

Academic term math and CSV export. This file (marks csv export) is one concrete piece of that layer.

Line count: 87. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:convert';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:share_plus/share_plus.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/marks/marks_csv_export.dart**

```
import 'dart:convert';

import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:share_plus/share_plus.dart';

String _csvCell(Object? value) {
  final s = '${value ?? ''}';
  if (s.contains(',') ||
      s.contains('"') ||
      s.contains('\n') ||
      s.contains('\r')) {
    return '"${s.replaceAll('"', '""')}"';
  }
  return s;
}

Future<void> exportGroupMarksCsv({
  required String groupId,
  required String groupTitle,
  required AcademicTerm term,
}) async {
  var query = supabase
      .from('marks')
      .select('*, students(profiles(full_name, username))')
      .eq('group_id', groupId);

  final start = term.startKey;
  final end = term.endKey;
  if (start != null) query = query.gte('marked_on', start);
  if (end != null) query = query.lte('marked_on', end);

  final rows = await query
      .order('marked_on', ascending: false)
      .order('created_at', ascending: false);

  final buffer = StringBuffer()
    ..writeln(
      [
        'Student',
```

```
        'Username',
        'Title',
        'Category',
        'Percentage',
        'Weight',
        'Date',
        'Comment',
      ].map(_csvCell).join(','),
    );

  for (final row in rows) {
    final mark = Mark.fromJson(Map<String, dynamic>.from(row));
    final student = row['students'] as Map?;
    final profile = student?['profiles'] as Map?;
    buffer.writeln(
      [
        profile?['full_name'] ?? mark.studentName ?? '',
        profile?['username'] ?? '',
        mark.title,
        mark.category.name,
        mark.percentage,
        mark.weight,
        dateKey(mark.markedOn),
        mark.comment,
      ].map(_csvCell).join(','),
    );
  }

  final safeTitle = groupTitle
      .toLowerCase()
      .replaceAll(RegExp(r'[^a-z0-9]+'), '_')
      .replaceAll(RegExp(r'^_|_$'), '');
  final fileName =
      'marks_${safeTitle.isEmpty ? groupId : safeTitle}_${term.id}.csv';
  final bytes = utf8.encode(buffer.toString());

  await SharePlus.instance.share(
    ShareParams(
      files: [XFile.fromData(bytes, mimeType: 'text/csv', name: fileName)],
      fileNameOverrides: [fileName],
      subject: 'Marks — $groupTitle',
      downloadFallbackEnabled: true,
    ),
  );
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/marks/marks_csv_export.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/core/paging/page_result.dart

Generic page-result type for list pagination. This file (page result) is one concrete piece of that layer.

Line count: 36. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/paging/page_result.dart**

```
class PageResult<T> {
  const PageResult({
    required this.items,
    required this.hasMore,
    this.loadingMore = false,
  });

  final List<T> items;
  final bool hasMore;
  final bool loadingMore;

  static const int pageSize = 40;

  static int fetchSize(int pageSize) => pageSize + 1;

  static PageResult<T> fromFetch<T>(
    List<T> fetched, {
    int pageSize = PageResult.pageSize,
  }) {
    final hasMore = fetched.length > pageSize;
    final items = hasMore ? fetched.sublist(0, pageSize) : fetched;
    return PageResult(items: items, hasMore: hasMore);
  }

  PageResult<T> copyWith({List<T>? items, bool? hasMore, bool? loadingMore}) {
    return PageResult(
      items: items ?? this.items,
      hasMore: hasMore ?? this.hasMore,
      loadingMore: loadingMore ?? this.loadingMore,
    );
  }

  PageResult<T> append(List<T> next, {required bool hasMore}) {
    return PageResult(items: [...items, ...next], hasMore: hasMore);
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/paging/page_result.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/progress/progress_report.dart

Student progress report model + PDF + share. This file (progress report) is one concrete piece of that layer.

Line count: 160. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:intl/intl.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/progress/progress_report.dart**

```
import 'package:intl/intl.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class ProgressReport {
  const ProgressReport({
    required this.studentName,
    required this.groupTitle,
    required this.termLabel,
    required this.term,
    required this.average,
    required this.markCount,
    required this.attendance,
    required this.categoryAverages,
    required this.recentMarks,
    this.generatedAt,
  });

  final String studentName;
  final String groupTitle;
  final String termLabel;
  final AcademicTerm term;
  final double? average;
  final int markCount;
  final AttendanceStats attendance;
  final List<({MarkCategory category, double average})> categoryAverages;
  final List<Mark> recentMarks;
  final DateTime? generatedAt;

  double get attendanceRate => attendance.presentPercent;
}

Future<ProgressReport> fetchProgressReport({
  required String studentId,
  required String groupId,
  required String studentName,
  required String groupTitle,
  required AcademicTerm term,
  required AppLocalizations l10n,
```

```
}) async {
  var marksQuery = supabase
      .from('marks')
      .select()
      .eq('student_id', studentId)
      .eq('group_id', groupId);
  final start = term.startKey;
  final end = term.endKey;
  if (start != null) marksQuery = marksQuery.gte('marked_on', start);
  if (end != null) marksQuery = marksQuery.lte('marked_on', end);

  final markRows = await marksQuery
      .order('marked_on', ascending: false)
      .order('created_at', ascending: false);
  final marks = markRows
      .map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))
      .toList();

  double? average;
  if (marks.isNotEmpty) {
    var total = 0.0;
    var weight = 0.0;
    for (final m in marks) {
      total += m.percentage * m.weight;
      weight += m.weight;
    }
    if (weight > 0) average = total / weight;
  }

  final byCategory = <MarkCategory, List<Mark>>{};
  for (final m in marks) {
    byCategory.putIfAbsent(m.category, () => []).add(m);
  }
  final categoryAverages = byCategory.entries.map((e) {
    var total = 0.0;
    var weight = 0.0;
    for (final m in e.value) {
      total += m.percentage * m.weight;
      weight += m.weight;
    }
    return (category: e.key, average: weight == 0 ? 0.0 : total / weight);
  }).toList()..sort((a, b) => b.average.compareTo(a.average));

  final now = DateTime.now();
  final from =
      term.startInclusive ??
      DateTime(
        now.year,
        now.month,
        now.day,
      ).subtract(const Duration(days: 120));
  final to = term.endInclusive ?? DateTime(now.year, now.month, now.day);

  final attendanceRows = await supabase
      .from('attendance')
      .select()
      .eq('group_id', groupId)
      .eq('student_id', studentId)
      .gte('date', dateKey(from))
      .lte('date', dateKey(to))
      .order('date', ascending: true);

  final attendance = AttendanceStats.fromRecords(
    attendanceRows.map(
      (e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)),
    ),
  );

  return ProgressReport(
    studentName: studentName,
    groupTitle: groupTitle,
    termLabel: term.label(l10n),
    term: term,
```

```
    average: average,
    markCount: marks.length,
    attendance: attendance,
    categoryAverages: categoryAverages,
    recentMarks: marks.take(12).toList(),
    generatedAt: DateTime.now(),
  );
}

String progressReportShareText(ProgressReport report, AppLocalizations l10n) {
  final avg = report.average;
  final avgLine = avg == null
      ? l10n.marksNoneYet
      : '${avg.toStringAsFixed(avg % 1 == 0 ? 0 : 1)}%';
  final att = report.attendance;
  final buf = StringBuffer()
    ..writeln(l10n.progressReportTitle)
    ..writeln('${report.studentName} · ${report.groupTitle}')
    ..writeln(report.termLabel)
    ..writeln()
    ..writeln('${l10n.marksOverallAverage}: $avgLine')
    ..writeln('${l10n.progressReportMarksCounted}: ${report.markCount}')
    ..writeln(
      '${l10n.progressReportAttendanceRate}: '
      '${att.total == 0 ? '—' : '${report.attendanceRate.toStringAsFixed(0)}%'}',
    );
  if (att.total > 0) {
    buf.writeln(
      '${l10n.attendancePresentWord}: ${att.present} · '
      '${l10n.attendanceAbsentWord}: ${att.absent} · '
      '${l10n.attendanceLateWord}: ${att.late} · '
      '${l10n.attendanceExcusedWord}: ${att.excused}',
    );
  }
  buf
    ..writeln()
    ..writeln(
      l10n.progressReportGeneratedOn(
        DateFormat.yMMMd().add_jm().format(
          report.generatedAt ?? DateTime.now(),
        ),
      ),
    );
  return buf.toString();
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/progress/progress_report.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/progress/progress_report_pdf.dart

Student progress report model + PDF + share. This file (progress report pdf) is one concrete piece of that layer.

Line count: 135. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:typed_data';
- import 'package:flutter/material.dart' show Rect, Offset;
- import 'package:intl/intl.dart';
- import 'package:mu_study/core/progress/progress_report.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:syncfusion_flutter_pdf/pdf.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/progress/progress_report_pdf.dart**

```
import 'dart:typed_data';

import 'package:flutter/material.dart' show Rect, Offset;
import 'package:intl/intl.dart';
import 'package:mu_study/core/progress/progress_report.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:syncfusion_flutter_pdf/pdf.dart';

Uint8List buildProgressReportPdf(ProgressReport report, AppLocalizations l10n) {
  final document = PdfDocument();
  final page = document.pages.add();
  final size = page.getClientSize();
  final g = page.graphics;

  final titleFont = PdfStandardFont(
    PdfFontFamily.helvetica,
    18,
    style: PdfFontStyle.bold,
  );
  final headingFont = PdfStandardFont(
    PdfFontFamily.helvetica,
    12,
    style: PdfFontStyle.bold,
  );
  final bodyFont = PdfStandardFont(PdfFontFamily.helvetica, 11);
  final smallFont = PdfStandardFont(PdfFontFamily.helvetica, 9);

  final ink = PdfColor(44, 36, 22);
  final muted = PdfColor(110, 100, 88);
  final accent = PdfColor(107, 79, 53);
  final line = PdfPen(PdfColor(220, 210, 195), width: 0.8);

  var y = 28.0;

  void drawText(
    String text,
    PdfFont font, {
    PdfColor? color,
```

```
    double? x,
    double? maxWidth,
  }) {
    g.drawString(
      text,
      font,
      brush: PdfSolidBrush(color ?? ink),
      bounds: Rect.fromLTWH(x ?? 28, y, maxWidth ?? (size.width - 56), 40),
    );
  }

  drawText('MU Study', titleFont, color: accent);
  y += 26;
  drawText(l10n.progressReportTitle, headingFont);
  y += 22;
  drawText('${report.studentName} · ${report.groupTitle}', bodyFont);
  y += 16;
  drawText(report.termLabel, bodyFont, color: muted);
  y += 14;
  g.drawLine(line, Offset(28, y), Offset(size.width - 28, y));
  y += 18;

  drawText(l10n.marksOverallAverage, headingFont);
  y += 18;
  final avg = report.average;
  final avgText = avg == null
      ? l10n.marksNoneYet
      : '${avg.toStringAsFixed(avg % 1 == 0 ? 0 : 1)}%  (${l10n.progressReportMarksCounted}: ${
report.markCount})';
  drawText(avgText, bodyFont);
  y += 22;

  if (report.categoryAverages.isNotEmpty) {
    drawText(l10n.marksByCategory, headingFont);
    y += 16;
    for (final row in report.categoryAverages) {
      drawText(
        '${row.category.l10nLabel(l10n)}  —  ${row.average.toStringAsFixed(1)}%',
        bodyFont,
      );
      y += 14;
      if (y > size.height - 120) break;
    }
    y += 8;
  }

  drawText(l10n.progressReportAttendance, headingFont);
  y += 16;
  final att = report.attendance;
  if (att.total == 0) {
    drawText(l10n.progressReportNoAttendance, bodyFont, color: muted);
    y += 16;
  } else {
    drawText(
      '${l10n.progressReportAttendanceRate}: ${report.attendanceRate.toStringAsFixed(0)}%  (${
att.total} ${l10n.progressReportSessions})',
      bodyFont,
    );
    y += 14;
    drawText(
      '${l10n.attendancePresentWord}: ${att.present}   '
      '${l10n.attendanceAbsentWord}: ${att.absent}   '
      '${l10n.attendanceLateWord}: ${att.late}   '
      '${l10n.attendanceExcusedWord}: ${att.excused}',
      bodyFont,
    );
    y += 20;
  }

  if (report.recentMarks.isNotEmpty && y < size.height - 100) {
    drawText(l10n.progressReportRecentMarks, headingFont);
    y += 16;
    for (final m in report.recentMarks) {
```

```
        if (y > size.height - 60) break;
        final date = DateFormat.yMMMd().format(m.markedOn);
        drawText(
          '$date  ·  ${m.title}  ·  ${m.percentage.toStringAsFixed(0)}%',
          smallFont,
          color: muted,
        );
        y += 12;
      }
    }

    final generated = DateFormat.yMMMd().add_jm().format(
      report.generatedAt ?? DateTime.now(),
    );
    g.drawString(
      l10n.progressReportGeneratedOn(generated),
      smallFont,
      brush: PdfSolidBrush(muted),
      bounds: Rect.fromLTWH(28, size.height - 36, size.width - 56, 20),
    );

    final bytes = Uint8List.fromList(document.saveSync());
    document.dispose();
    return bytes;
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/progress/progress_report_pdf.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/progress/progress_report_share.dart

Student progress report model + PDF + share. This file (progress report share) is one concrete piece of that layer.

Line count: 45. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/progress/progress_report.dart';
- import 'package:mu_study/core/progress/progress_report_pdf.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:share_plus/share_plus.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/progress/progress_report_share.dart**

```
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/progress/progress_report.dart';
import 'package:mu_study/core/progress/progress_report_pdf.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:share_plus/share_plus.dart';

Future<void> shareProgressReport({
  required String studentId,
  required String groupId,
  required String studentName,
  required String groupTitle,
  required AcademicTerm term,
  required AppLocalizations l10n,
}) async {
  final report = await fetchProgressReport(
    studentId: studentId,
    groupId: groupId,
    studentName: studentName,
    groupTitle: groupTitle,
    term: term,
    l10n: l10n,
  );

  final pdfBytes = buildProgressReportPdf(report, l10n);
  final text = progressReportShareText(report, l10n);

  final safeStudent = studentName
      .toLowerCase()
      .replaceAll(RegExp(r'[^a-z0-9]+'), '_')
      .replaceAll(RegExp(r'^_|_$'), '');
  final fileName =
      'progress_${safeStudent.isEmpty ? studentId : safeStudent}_${term.id}.pdf';

  await SharePlus.instance.share(
    ShareParams(
      text: text,
      subject: '${l10n.progressReportTitle} — $studentName',
      files: [
        XFile.fromData(pdfBytes, mimeType: 'application/pdf', name: fileName),
      ],
      fileNameOverrides: [fileName],
      downloadFallbackEnabled: true,
```

```
      ),
    );
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/progress/progress_report_share.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/router/app_router.dart

go_router route table and auth/language/disabled redirects. This file (app router) is one concrete piece of that layer.

Line count: 443. Import count: 41. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/screens/admin/academic_years/admin_academic_years_screen.dart';
- import 'package:mu_study/screens/admin/analytics/admin_analytics_screen.dart';
- import 'package:mu_study/screens/admin/announcements/admin_announcements_screen.dart';
- import 'package:mu_study/screens/admin/dashboard/admin_dashboard_screen.dart';
- import 'package:mu_study/screens/admin/groups/admin_groups_screen.dart';
- import 'package:mu_study/screens/admin/credentials/admin_credential_print_screen.dart';
- import 'package:mu_study/screens/admin/data_generator/admin_mass_data_generator_screen.dart';
- import 'package:mu_study/screens/admin/notifications/admin_notifications_screen.dart';
- import 'package:mu_study/screens/admin/profile/admin_profile_screen.dart';
- import 'package:mu_study/screens/admin/users/admin_users_screen.dart';
- import 'package:mu_study/screens/auth/language_selection_screen.dart';
- … and 23 more imports

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/router/app_router.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/screens/admin/academic_years/admin_academic_years_screen.dart';
import 'package:mu_study/screens/admin/analytics/admin_analytics_screen.dart';
import 'package:mu_study/screens/admin/announcements/admin_announcements_screen.dart';
import 'package:mu_study/screens/admin/dashboard/admin_dashboard_screen.dart';
import 'package:mu_study/screens/admin/groups/admin_groups_screen.dart';
import 'package:mu_study/screens/admin/credentials/admin_credential_print_screen.dart';
import 'package:mu_study/screens/admin/data_generator/admin_mass_data_generator_screen.dart';
import 'package:mu_study/screens/admin/notifications/admin_notifications_screen.dart';
import 'package:mu_study/screens/admin/profile/admin_profile_screen.dart';
import 'package:mu_study/screens/admin/users/admin_users_screen.dart';
import 'package:mu_study/screens/auth/language_selection_screen.dart';
import 'package:mu_study/screens/auth/login_screen.dart';
import 'package:mu_study/screens/auth/splash_screen.dart';
```

```
import 'package:mu_study/screens/chat/list/chat_list_screen.dart';
import 'package:mu_study/screens/chat/room/chat_room_screen.dart';
import 'package:mu_study/screens/student/attendance/student_attendance_screen.dart';
import 'package:mu_study/screens/student/notifications/student_notifications_screen.dart';
import 'package:mu_study/screens/student/profile/student_profile_screen.dart';
import 'package:mu_study/screens/student/homework/student_assignments_screen.dart';
import 'package:mu_study/screens/student/home/student_dashboard_screen.dart';
import 'package:mu_study/screens/student/focus/student_focus_screen.dart';
import 'package:mu_study/screens/student/marks/student_marks_screen.dart';
import 'package:mu_study/screens/student/homework/student_task_submit_screen.dart';
import 'package:mu_study/screens/teacher/analytics/teacher_analytics_screen.dart';
import 'package:mu_study/screens/teacher/announcements/teacher_announcements_screen.dart';
import 'package:mu_study/screens/teacher/assignments/teacher_assignments_screen.dart';
import 'package:mu_study/screens/teacher/attendance/teacher_attendance_screen.dart';
import 'package:mu_study/screens/teacher/dashboard/teacher_dashboard_screen.dart';
import 'package:mu_study/screens/teacher/notifications/teacher_notifications_screen.dart';
import 'package:mu_study/screens/teacher/profile/teacher_profile_screen.dart';
import 'package:mu_study/screens/teacher/homework/teacher_homework_pack_screen.dart';
import 'package:mu_study/screens/teacher/marks/teacher_marks_screen.dart';
import 'package:mu_study/screens/teacher/homework/teacher_submissions_screen.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

final routerProvider = Provider<GoRouter>((ref) {
  final notifier = _RouterNotifier(ref);

  return GoRouter(
    initialLocation: '/splash',
    refreshListenable: notifier,
    redirect: notifier.redirect,
    routes: [
      GoRoute(path: '/splash', builder: (_, _) => const SplashScreen()),
      GoRoute(path: '/login', builder: (_, _) => const LoginScreen()),
      GoRoute(
        path: '/language',
        builder: (_, _) => const LanguageSelectionScreen(),
      ),
      GoRoute(
        path: '/boot',
        builder: (_, _) =>
            const Scaffold(body: Center(child: CircularProgressIndicator())),
      ),
      GoRoute(
        path: '/account-disabled',
        builder: (context, _) =>
            Scaffold(body: Center(child: Text(context.l10n.accountDisabled))),
      ),
      _studentShell(),
      _teacherShell(),
      _adminShell(),
    ],
  );
});

class _RouterNotifier extends ChangeNotifier {
  _RouterNotifier(this._ref) {
    _ref.listen(authStateProvider, (_, _) => notifyListeners());
    _ref.listen(currentProfileProvider, (_, _) => notifyListeners());
  }

  final Ref _ref;

  String? redirect(BuildContext context, GoRouterState state) {
    final loc = state.matchedLocation;
    if (loc == '/splash') return null;

    final session = Supabase.instance.client.auth.currentSession;
    if (session == null) {
      return loc == '/login' ? null : '/login';
    }

    final profileAsync = _ref.read(currentProfileProvider);

    // Only block on the *first* profile load. A refresh (e.g. after changing
```

```
    // language) still has the previous value — bouncing through /boot tears
    // down the shell mid-gesture and floods semantics.parentDataDirty asserts.
    if (!profileAsync.hasValue &&
        (profileAsync.isLoading || profileAsync.isRefreshing)) {
      if (loc == '/boot' || loc == '/language') return null;
      return '/boot';
    }

    final profile = profileAsync.valueOrNull;
    if (profile == null) {
      return loc == '/login' ? null : '/login';
    }
    if (profile.isDisabled) {
      return loc == '/account-disabled' ? null : '/account-disabled';
    }

    final home = switch (profile.role) {
      UserRole.student => '/student/dashboard',
      UserRole.teacher => '/teacher/dashboard',
      UserRole.admin => '/admin/dashboard',
    };

    if (!profile.hasChosenLanguage) {
      return loc == '/language' ? null : '/language';
    }

    if (loc == '/login' || loc == '/boot' || loc == '/language') {
      return home;
    }

    return null;
  }
}

ShellRoute _studentShell() {
  const destinations = [
    ShellDestination(
      labelKey: 'navDashboard',
      icon: Icons.dashboard_outlined,
      path: '/student/dashboard',
    ),
    ShellDestination(
      labelKey: 'navAssignments',
      icon: Icons.assignment_outlined,
      path: '/student/assignments',
    ),
    ShellDestination(
      labelKey: 'navFocus',
      icon: Icons.self_improvement_outlined,
      path: '/student/focus',
    ),
    ShellDestination(
      labelKey: 'navMarks',
      icon: Icons.grade_outlined,
      path: '/student/marks',
    ),
    ShellDestination(
      labelKey: 'navChat',
      icon: Icons.chat_bubble_outline_rounded,
      path: '/student/chat',
      badge: ShellBadgeKind.chat,
    ),
    ShellDestination(
      labelKey: 'navAttendance',
      icon: Icons.event_available_outlined,
      path: '/student/attendance',
    ),
    ShellDestination(
      labelKey: 'navProfile',
      icon: Icons.person_outline,
      path: '/student/profile',
    ),
  ];
```

```
  return ShellRoute(
    builder: (context, state, child) => AppShell(
      role: UserRole.student,
      destinations: destinations,
      child: child,
    ),
    routes: [
      GoRoute(
        path: '/student/dashboard',
        pageBuilder: (_, s) => _fade(const StudentDashboardScreen(), s),
      ),
      GoRoute(
        path: '/student/assignments',
        pageBuilder: (_, s) => _fade(const StudentAssignmentsScreen(), s),
        routes: [
          GoRoute(
            path: ':id',
            builder: (_, s) => StudentAssignmentDetailScreen(
              assignmentId: s.pathParameters['id']!,
            ),
          ),
        ],
      ),
      GoRoute(
        path: '/student/tasks/:taskId',
        builder: (_, s) =>
            StudentTaskSubmitScreen(taskId: s.pathParameters['taskId']!),
      ),
      GoRoute(
        path: '/student/focus',
        pageBuilder: (_, s) => _fade(const StudentFocusScreen(), s),
      ),
      GoRoute(
        path: '/student/marks',
        pageBuilder: (_, s) => _fade(const StudentMarksScreen(), s),
      ),
      GoRoute(
        path: '/student/chat',
        pageBuilder: (_, s) =>
            _fade(const ChatListScreen(basePath: '/student/chat'), s),
        routes: [
          GoRoute(
            path: ':roomId',
            builder: (_, s) =>
                ChatRoomScreen(roomId: s.pathParameters['roomId']!),
          ),
        ],
      ),
      GoRoute(
        path: '/student/attendance',
        pageBuilder: (_, s) => _fade(const StudentAttendanceScreen(), s),
      ),
      GoRoute(
        path: '/student/notifications',
        pageBuilder: (_, s) => _fade(const StudentNotificationsScreen(), s),
      ),
      GoRoute(
        path: '/student/profile',
        pageBuilder: (_, s) => _fade(const StudentProfileScreen(), s),
      ),
    ],
  );
}

ShellRoute _teacherShell() {
  const destinations = [
    ShellDestination(
      labelKey: 'navDashboard',
      icon: Icons.dashboard_outlined,
      path: '/teacher/dashboard',
    ),
    ShellDestination(
      labelKey: 'navAssignments',
```

```
      icon: Icons.assignment_outlined,
      path: '/teacher/assignments',
    ),
    ShellDestination(
      labelKey: 'navChat',
      icon: Icons.chat_bubble_outline_rounded,
      path: '/teacher/chat',
      badge: ShellBadgeKind.chat,
    ),
    ShellDestination(
      labelKey: 'navAttendance',
      icon: Icons.event_available_outlined,
      path: '/teacher/attendance',
    ),
    ShellDestination(
      labelKey: 'navGradebook',
      icon: Icons.grade_outlined,
      path: '/teacher/marks',
    ),
    ShellDestination(
      labelKey: 'navProfile',
      icon: Icons.person_outline,
      path: '/teacher/profile',
    ),
  ];

  return ShellRoute(
    builder: (context, state, child) => AppShell(
      role: UserRole.teacher,
      destinations: destinations,
      child: child,
    ),
    routes: [
      GoRoute(
        path: '/teacher/dashboard',
        pageBuilder: (_, s) => _fade(const TeacherDashboardScreen(), s),
      ),
      GoRoute(
        path: '/teacher/submissions',
        builder: (_, _) => const TeacherSubmissionsScreen(),
      ),
      GoRoute(
        path: '/teacher/assignments',
        pageBuilder: (_, s) => _fade(const TeacherAssignmentsScreen(), s),
        routes: [
          GoRoute(
            path: ':id',
            builder: (_, s) => TeacherHomeworkPackScreen(
              assignmentId: s.pathParameters['id']!,
            ),
          ),
        ],
      ),
      GoRoute(
        path: '/teacher/chat',
        pageBuilder: (_, s) =>
            _fade(const ChatListScreen(basePath: '/teacher/chat'), s),
        routes: [
          GoRoute(
            path: ':roomId',
            builder: (_, s) =>
                ChatRoomScreen(roomId: s.pathParameters['roomId']!),
          ),
        ],
      ),
      GoRoute(
        path: '/teacher/attendance',
        pageBuilder: (_, s) => _fade(const TeacherAttendanceScreen(), s),
      ),
      GoRoute(
        path: '/teacher/marks',
        pageBuilder: (_, s) => _fade(const TeacherMarksScreen(), s),
      ),
```

```
      GoRoute(
        path: '/teacher/analytics',
        builder: (_, _) => const TeacherAnalyticsScreen(),
      ),
      GoRoute(
        path: '/teacher/announcements',
        pageBuilder: (_, s) => _fade(const TeacherAnnouncementsScreen(), s),
      ),
      GoRoute(
        path: '/teacher/notifications',
        pageBuilder: (_, s) => _fade(const TeacherNotificationsScreen(), s),
      ),
      GoRoute(
        path: '/teacher/profile',
        pageBuilder: (_, s) => _fade(const TeacherProfileScreen(), s),
      ),
    ],
  );
}

ShellRoute _adminShell() {
  const destinations = [
    ShellDestination(
      labelKey: 'navDashboard',
      icon: Icons.dashboard_outlined,
      path: '/admin/dashboard',
    ),
    ShellDestination(
      labelKey: 'navUsers',
      icon: Icons.people_outline,
      path: '/admin/users',
    ),
    ShellDestination(
      labelKey: 'navGroups',
      icon: Icons.groups_outlined,
      path: '/admin/groups',
    ),
    ShellDestination(
      labelKey: 'navChat',
      icon: Icons.chat_bubble_outline_rounded,
      path: '/admin/chat',
      badge: ShellBadgeKind.chat,
    ),
    ShellDestination(
      labelKey: 'navAnalytics',
      icon: Icons.insights_outlined,
      path: '/admin/analytics',
    ),
    ShellDestination(
      labelKey: 'navProfile',
      icon: Icons.person_outline,
      path: '/admin/profile',
    ),
  ];

  return ShellRoute(
    builder: (context, state, child) => AppShell(
      role: UserRole.admin,
      destinations: destinations,
      child: child,
    ),
    routes: [
      GoRoute(
        path: '/admin/dashboard',
        pageBuilder: (_, s) => _fade(const AdminDashboardScreen(), s),
      ),
      GoRoute(
        path: '/admin/users',
        pageBuilder: (_, s) => _fade(const AdminUsersScreen(), s),
      ),
      GoRoute(
        path: '/admin/groups',
        pageBuilder: (_, s) => _fade(const AdminGroupsScreen(), s),
```

```
      ),
      GoRoute(
        path: '/admin/chat',
        pageBuilder: (_, s) =>
            _fade(const ChatListScreen(basePath: '/admin/chat'), s),
        routes: [
          GoRoute(
            path: ':roomId',
            builder: (_, s) =>
                ChatRoomScreen(roomId: s.pathParameters['roomId']!),
          ),
        ],
      ),
      GoRoute(
        path: '/admin/analytics',
        pageBuilder: (_, s) => _fade(const AdminAnalyticsScreen(), s),
      ),
      GoRoute(
        path: '/admin/academic-years',
        pageBuilder: (_, s) => _fade(const AdminAcademicYearsScreen(), s),
      ),
      GoRoute(
        path: '/admin/announcements',
        pageBuilder: (_, s) => _fade(const AdminAnnouncementsScreen(), s),
      ),
      GoRoute(
        path: '/admin/notifications',
        pageBuilder: (_, s) => _fade(const AdminNotificationsScreen(), s),
      ),
      GoRoute(
        path: '/admin/profile',
        pageBuilder: (_, s) => _fade(const AdminProfileScreen(), s),
      ),
      GoRoute(
        path: '/admin/mass-generator',
        pageBuilder: (_, s) => _fade(
          const AdminMassDataGeneratorScreen(),
          s,
        ),
      ),
      GoRoute(
        path: '/admin/credential-print',
        pageBuilder: (_, s) =>
            _fade(const AdminCredentialPrintScreen(), s),
      ),
```

**lib/core/router/app_router.dart — continued (2/2)**

```
    ],
  );
}

CustomTransitionPage<void> _fade(Widget child, GoRouterState state) {
  return CustomTransitionPage(
    key: state.pageKey,
    child: child,
    transitionsBuilder: (_, animation, _, child) =>
        FadeTransition(opacity: animation, child: child),
  );
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/router/app_router.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/services/app_cache.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (app cache) is one concrete piece of that layer.

Line count: 55. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter_cache_manager/flutter_cache_manager.dart';
- import 'package:mu_study/core/services/cache_service.dart';
- import 'package:mu_study/core/services/chat_media_cache_service.dart';
- import 'package:mu_study/core/services/chat_video_thumbnail_service.dart';
- import 'package:mu_study/core/services/voice_cache_service.dart';
- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/services/app_cache.dart**

```
import 'package:flutter_cache_manager/flutter_cache_manager.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/core/services/chat_video_thumbnail_service.dart';
import 'package:mu_study/core/services/voice_cache_service.dart';
import 'package:mu_study/models/models.dart';

Future<void> clearChatAndMediaCaches(CacheService? prefsCache) async {
  await prefsCache?.clearChatMessageCaches();
  await VoiceCacheService.instance.clear();
  await ChatMediaCacheService.instance.clear();
  await ChatVideoThumbnailService.instance.clear();
  try {
    await DefaultCacheManager().emptyCache();
  } catch (_) {}
}

Future<void> clearAllAppCaches(CacheService? prefsCache) async {
  await prefsCache?.clearAll();
  await VoiceCacheService.instance.clear();
  await ChatMediaCacheService.instance.clear();
  await ChatVideoThumbnailService.instance.clear();
  try {
    await DefaultCacheManager().emptyCache();
  } catch (_) {}
}

void prefetchChatMedia(Iterable<ChatMessage> messages) {
  for (final m in messages) {
    final url = m.mediaUrl;
    if (url == null || url.isEmpty) continue;
    if (m.messageType == ChatMessageType.voice) {
      continue;
    }
    final ext = switch (m.messageType) {
      ChatMessageType.image => ChatMediaCacheService.extensionForUrl(
        url,
        fallback: 'jpg',
      ),
      ChatMessageType.video => ChatMediaCacheService.extensionForUrl(
```

```
          url,
          fallback: 'mp4',
        ),
        ChatMessageType.file => ChatMediaCacheService.extensionForUrl(
          url,
          fallback: (m.textContent ?? 'bin').split('.').last,
        ),
        _ => 'bin',
      };
      ChatMediaCacheService.instance.prefetch(url, preferredExtension: ext);
      if (m.messageType == ChatMessageType.video) {
        ChatVideoThumbnailService.instance.prefetch(url);
      }
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/services/app_cache.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/services/cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (cache service) is one concrete piece of that layer.

Line count: 98. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:convert';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:shared_preferences/shared_preferences.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/services/cache_service.dart**

```
import 'dart:convert';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:shared_preferences/shared_preferences.dart';

class CacheService {
  CacheService(this._prefs);

  final SharedPreferences _prefs;

  static const _profileKey = 'cached_profile';
  static const _notificationsKey = 'cached_notifications';
  static const _assignmentsKey = 'cached_assignments';
  static const _dashboardKey = 'cached_dashboard';
  static const _chatRoomIdsKey = 'cached_chat_room_ids';

  static String _chatMessagesKey(String roomId) =>
      'cached_chat_messages_$roomId';

  Future<void> saveJson(String key, Map<String, dynamic> data) async {
    await _prefs.setString(key, jsonEncode(data));
  }

  Map<String, dynamic>? readJson(String key) {
    final raw = _prefs.getString(key);
    if (raw == null) return null;
    return jsonDecode(raw) as Map<String, dynamic>;
  }

  Future<void> saveList(String key, List<Map<String, dynamic>> items) async {
    await _prefs.setString(key, jsonEncode(items));
  }

  List<Map<String, dynamic>>? readList(String key) {
    final raw = _prefs.getString(key);
    if (raw == null) return null;
    return (jsonDecode(raw) as List<dynamic>)
        .map((e) => Map<String, dynamic>.from(e as Map))
        .toList();
  }

  Future<void> cacheProfile(Map<String, dynamic> profile) =>
      saveJson(_profileKey, profile);

  Map<String, dynamic>? get cachedProfile => readJson(_profileKey);
```

```
  Future<void> cacheNotifications(List<Map<String, dynamic>> items) =>
      saveList(_notificationsKey, items);

  List<Map<String, dynamic>>? get cachedNotifications =>
      readList(_notificationsKey);

  Future<void> cacheAssignments(List<Map<String, dynamic>> items) =>
      saveList(_assignmentsKey, items);

  List<Map<String, dynamic>>? get cachedAssignments =>
      readList(_assignmentsKey);

  Future<void> cacheDashboard(Map<String, dynamic> data) =>
      saveJson(_dashboardKey, data);

  Map<String, dynamic>? get cachedDashboard => readJson(_dashboardKey);

  Future<void> cacheChatMessages(
    String roomId,
    List<Map<String, dynamic>> rows,
  ) async {
    await saveList(_chatMessagesKey(roomId), rows);
    final ids = _prefs.getStringList(_chatRoomIdsKey) ?? <String>[];
    if (!ids.contains(roomId)) {
      await _prefs.setStringList(_chatRoomIdsKey, [...ids, roomId]);
    }
  }

  List<Map<String, dynamic>>? cachedChatMessages(String roomId) =>
      readList(_chatMessagesKey(roomId));

  Future<void> clearChatMessageCaches() async {
    final ids = _prefs.getStringList(_chatRoomIdsKey) ?? <String>[];
    for (final id in ids) {
      await _prefs.remove(_chatMessagesKey(id));
    }
    await _prefs.remove(_chatRoomIdsKey);
  }

  Future<void> clearAll() async {
    await _prefs.remove(_profileKey);
    await _prefs.remove(_notificationsKey);
    await _prefs.remove(_assignmentsKey);
    await _prefs.remove(_dashboardKey);
    await clearChatMessageCaches();
  }
}

final cacheServiceProvider = FutureProvider<CacheService>((ref) async {
  final prefs = await SharedPreferences.getInstance();
  return CacheService(prefs);
});
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/services/cache_service.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/services/chat_media_cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (chat media cache service) is one concrete piece of that layer.

Line count: 228. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:convert';
- import 'dart:io';
- import 'package:crypto/crypto.dart';
- import 'package:flutter/foundation.dart';
- import 'package:http/http.dart' as http;
- import 'package:path/path.dart' as p;
- import 'package:path_provider/path_provider.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/services/chat_media_cache_service.dart**

```
import 'dart:convert';
import 'dart:io';

import 'package:crypto/crypto.dart';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

class ChatMediaCacheService {
  ChatMediaCacheService._();

  static final ChatMediaCacheService instance = ChatMediaCacheService._();

  static const _folder = 'chat_media_cache';
  static const _indexFile = 'index.json';
  static const maxEntries = 200;
  static const maxTotalBytes = 400 * 1024 * 1024; // 400 MB

  Directory? _dir;
  Map<String, _MediaCacheEntry>? _index;
  Future<void>? _initFuture;
  final _inflight = <String, Future<File?>>{};

  bool get isSupported => !kIsWeb;

  Future<void> _ensureReady() async {
    if (!isSupported) return;
    _initFuture ??= _init();
    await _initFuture;
  }

  Future<void> _init() async {
    final root = await getApplicationCacheDirectory();
    _dir = Directory(p.join(root.path, _folder));
    if (!await _dir!.exists()) {
      await _dir!.create(recursive: true);
    }
    await _loadIndex();
```

```
}

static String keyForUrl(String url) {
  final uri = Uri.tryParse(url);
  if (uri == null) return url;
  final path = uri.path;
  final signIdx = path.indexOf('/object/sign/');
  if (signIdx >= 0) {
    return path.substring(signIdx + '/object/sign/'.length);
  }
  final publicIdx = path.indexOf('/object/public/');
  if (publicIdx >= 0) {
    return path.substring(publicIdx + '/object/public/'.length);
  }
  return path.isEmpty ? url : path;
}

static String _hashKey(String key) =>
    sha1.convert(utf8.encode(key)).toString();

static String extensionForUrl(String url, {String fallback = 'bin'}) {
  final key = keyForUrl(url);
  final name = key.split('/').last;
  final dot = name.lastIndexOf('.');
  if (dot <= 0 || dot == name.length - 1) return fallback;
  final ext = name.substring(dot + 1).toLowerCase();
  if (ext.length > 8 || ext.contains('?')) return fallback;
  return ext;
}

Future<void> _loadIndex() async {
  final file = File(p.join(_dir!.path, _indexFile));
  if (!await file.exists()) {
    _index = {};
    return;
  }
  try {
    final raw = jsonDecode(await file.readAsString()) as Map<String, dynamic>;
    _index = {
      for (final e in raw.entries)
        e.key: _MediaCacheEntry.fromJson(
          Map<String, dynamic>.from(e.value as Map),
        ),
    };
  } catch (_) {
    _index = {};
  }
}

Future<void> _saveIndex() async {
  final file = File(p.join(_dir!.path, _indexFile));
  final map = {for (final e in _index!.entries) e.key: e.value.toJson()};
  await file.writeAsString(jsonEncode(map));
}

File _fileFor(String hash, String extension) =>
    File(p.join(_dir!.path, '$hash.$extension'));

Future<File?> getFile(String url) async {
  if (!isSupported) return null;
  await _ensureReady();
  final hash = _hashKey(keyForUrl(url));
  final entry = _index![hash];
  if (entry == null) return null;
  final file = _fileFor(hash, entry.extension);
  if (!await file.exists()) {
    _index!.remove(hash);
    await _saveIndex();
    return null;
  }
  entry.lastAccessMs = DateTime.now().millisecondsSinceEpoch;
  await _saveIndex();
  return file;
```

```
}

Future<File?> put(String url, Uint8List bytes, String extension) async {
  if (!isSupported || bytes.isEmpty) return null;
  await _ensureReady();
  final hash = _hashKey(keyForUrl(url));
  final ext = extension.replaceAll('.', '').toLowerCase();
  final file = _fileFor(hash, ext);

  final existing = _index![hash];
  if (existing != null && existing.extension != ext) {
    final old = _fileFor(hash, existing.extension);
    if (await old.exists()) await old.delete();
  }

  await file.writeAsBytes(bytes, flush: true);
  _index![hash] = _MediaCacheEntry(
    extension: ext,
    sizeBytes: bytes.length,
    lastAccessMs: DateTime.now().millisecondsSinceEpoch,
  );
  await _evictIfNeeded();
  await _saveIndex();
  return file;
}

Future<File?> getOrFetch(String url, {String? preferredExtension}) {
  final key = keyForUrl(url);
  return _inflight.putIfAbsent(key, () async {
    try {
      final existing = await getFile(url);
      if (existing != null) return existing;
      final response = await http.get(Uri.parse(url));
      if (response.statusCode < 200 || response.statusCode >= 300) {
        return null;
      }
      final bytes = response.bodyBytes;
      final ext = preferredExtension ?? extensionForUrl(url, fallback: 'bin');
      return put(url, Uint8List.fromList(bytes), ext);
    } finally {
      _inflight.remove(key);
    }
  });
}

Future<void> prefetch(String url, {String? preferredExtension}) async {
  try {
    await getOrFetch(url, preferredExtension: preferredExtension);
  } catch (_) {}
}

Future<void> _evictIfNeeded() async {
  var entries = _index!.entries.toList()
    ..sort((a, b) => a.value.lastAccessMs.compareTo(b.value.lastAccessMs));
  var total = entries.fold<int>(0, (s, e) => s + e.value.sizeBytes);

  while (entries.length > maxEntries || total > maxTotalBytes) {
    if (entries.isEmpty) break;
    final oldest = entries.removeAt(0);
    _index!.remove(oldest.key);
    total -= oldest.value.sizeBytes;
    final file = _fileFor(oldest.key, oldest.value.extension);
    if (await file.exists()) {
      try {
        await file.delete();
      } catch (_) {}
    }
  }
}

Future<int> totalBytes() async {
  if (!isSupported) return 0;
  await _ensureReady();
```

```
    return _index!.values.fold<int>(0, (s, e) => s + e.sizeBytes);
  }

  Future<void> clear() async {
    if (!isSupported) return;
    await _ensureReady();
    if (_dir != null && await _dir!.exists()) {
      try {
        await _dir!.delete(recursive: true);
      } catch (_) {}
    }
    _dir = null;
    _index = null;
    _initFuture = null;
    _inflight.clear();
  }
}

class _MediaCacheEntry {
  _MediaCacheEntry({
    required this.extension,
    required this.sizeBytes,
    required this.lastAccessMs,
  });

  final String extension;
  final int sizeBytes;
  int lastAccessMs;

  factory _MediaCacheEntry.fromJson(Map<String, dynamic> json) {
    return _MediaCacheEntry(
      extension: json['ext'] as String? ?? 'bin',
      sizeBytes: (json['size'] as num?)?.toInt() ?? 0,
      lastAccessMs: (json['at'] as num?)?.toInt() ?? 0,
    );
  }

  Map<String, dynamic> toJson() => {
    'ext': extension,
    'size': sizeBytes,
    'at': lastAccessMs,
  };
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/services/chat_media_cache_service.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/services/chat_video_thumbnail_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (chat video thumbnail service) is one concrete piece of that layer.

Line count: 118. Import count: 9. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:convert';
- import 'dart:io';
- import 'package:crypto/crypto.dart';
- import 'package:flutter/foundation.dart';
- import 'package:mu_study/core/services/chat_media_cache_service.dart';
- import 'package:path/path.dart' as p;
- import 'package:path_provider/path_provider.dart';
- import 'package:get_thumbnail_video/index.dart';
- import 'package:get_thumbnail_video/video_thumbnail.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/services/chat_video_thumbnail_service.dart**

```
import 'dart:convert';
import 'dart:io';

import 'package:crypto/crypto.dart';
import 'package:flutter/foundation.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:get_thumbnail_video/index.dart';
import 'package:get_thumbnail_video/video_thumbnail.dart';

class ChatVideoThumbnailService {
  ChatVideoThumbnailService._();

  static final ChatVideoThumbnailService instance =
      ChatVideoThumbnailService._();

  static const _folder = 'chat_video_thumbs';
  static const maxWidth = 440;

  Directory? _dir;
  Future<void>? _initFuture;
  final _inflight = <String, Future<File?>>{};

  bool get isSupported => !kIsWeb;

  Future<void> _ensureReady() async {
    if (!isSupported) return;
    _initFuture ??= _init();
    await _initFuture;
  }

  Future<void> _init() async {
    final root = await getApplicationCacheDirectory();
    _dir = Directory(p.join(root.path, _folder));
    if (!await _dir!.exists()) {
```

```
    await _dir!.create(recursive: true);
  }
}

static String _hashKey(String url) => sha1
    .convert(utf8.encode(ChatMediaCacheService.keyForUrl(url)))
    .toString();

File _thumbFile(String hash) => File(p.join(_dir!.path, '$hash.jpg'));

Future<File?> getThumbnail(String url) {
  final key = ChatMediaCacheService.keyForUrl(url);
  return _inflight.putIfAbsent(key, () async {
    try {
      if (!isSupported) return null;
      await _ensureReady();
      final hash = _hashKey(url);
      final cached = _thumbFile(hash);
      if (await cached.exists() && await cached.length() > 0) {
        return cached;
      }

      final videoFile = await ChatMediaCacheService.instance.getOrFetch(
        url,
        preferredExtension: ChatMediaCacheService.extensionForUrl(
          url,
          fallback: 'mp4',
        ),
      );

      Uint8List? bytes;
      if (videoFile != null) {
        try {
          bytes = await VideoThumbnail.thumbnailData(
            video: videoFile.path,
            imageFormat: ImageFormat.JPEG,
            maxWidth: maxWidth,
            quality: 75,
            timeMs: 0,
          );
        } catch (_) {}
      }
      if (bytes == null || bytes.isEmpty) {
        bytes = await VideoThumbnail.thumbnailData(
          video: url,
          imageFormat: ImageFormat.JPEG,
          maxWidth: maxWidth,
          quality: 75,
          timeMs: 0,
        );
      }

      if (bytes.isEmpty) return null;
      await cached.writeAsBytes(bytes, flush: true);
      return cached;
    } catch (_) {
      return null;
    } finally {
      _inflight.remove(key);
    }
  });
}

Future<void> prefetch(String url) async {
  try {
    await getThumbnail(url);
  } catch (_) {}
}

Future<void> clear() async {
  if (!isSupported) return;
  await _ensureReady();
  if (_dir != null && await _dir!.exists()) {
```

```
      try {
        await _dir!.delete(recursive: true);
      } catch (_) {}
    }
    _dir = null;
    _initFuture = null;
    _inflight.clear();
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/services/chat_video_thumbnail_service.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/services/voice_cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (voice cache service) is one concrete piece of that layer.

Line count: 209. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:convert';
- import 'dart:io';
- import 'package:crypto/crypto.dart';
- import 'package:flutter/foundation.dart';
- import 'package:path/path.dart' as p;
- import 'package:path_provider/path_provider.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/services/voice_cache_service.dart**

```
import 'dart:convert';
import 'dart:io';

import 'package:crypto/crypto.dart';
import 'package:flutter/foundation.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

class VoiceCacheService {
  VoiceCacheService._();

  static final VoiceCacheService instance = VoiceCacheService._();

  static const _folder = 'voice_cache';
  static const _indexFile = 'index.json';
  static const maxEntries = 100;
  static const maxTotalBytes = 100 * 1024 * 1024; // 100 MB

  Directory? _dir;
  Map<String, _VoiceCacheEntry>? _index;
  Future<void>? _initFuture;

  bool get isSupported => !kIsWeb;

  Future<void> _ensureReady() async {
    if (!isSupported) return;
    _initFuture ??= _init();
    await _initFuture;
  }

  Future<void> _init() async {
    final root = await getApplicationCacheDirectory();
    _dir = Directory(p.join(root.path, _folder));
    if (!await _dir!.exists()) {
      await _dir!.create(recursive: true);
    }
    await _loadIndex();
  }

  static String keyForUrl(String url) {
```

```
    final uri = Uri.tryParse(url);
    if (uri == null) return url;
    final path = uri.path;
    final signIdx = path.indexOf('/object/sign/');
    if (signIdx >= 0) {
      return path.substring(signIdx + '/object/sign/'.length);
    }
    final publicIdx = path.indexOf('/object/public/');
    if (publicIdx >= 0) {
      return path.substring(publicIdx + '/object/public/'.length);
    }
    return path.isEmpty ? url : path;
  }

  static String _hashKey(String key) =>
      sha1.convert(utf8.encode(key)).toString();

  Future<void> _loadIndex() async {
    final file = File(p.join(_dir!.path, _indexFile));
    if (!await file.exists()) {
      _index = {};
      return;
    }
    try {
      final raw = jsonDecode(await file.readAsString()) as Map<String, dynamic>;
      _index = {
        for (final e in raw.entries)
          e.key: _VoiceCacheEntry.fromJson(
            Map<String, dynamic>.from(e.value as Map),
          ),
      };
    } catch (_) {
      _index = {};
    }
  }

  Future<void> _saveIndex() async {
    final file = File(p.join(_dir!.path, _indexFile));
    final map = {for (final e in _index!.entries) e.key: e.value.toJson()};
    await file.writeAsString(jsonEncode(map));
  }

  File _fileFor(String hash, String extension) =>
      File(p.join(_dir!.path, '$hash.$extension'));

  Future<Uint8List?> readBytes(String url) async {
    final file = await getFile(url);
    if (file == null) return null;
    return Uint8List.fromList(await file.readAsBytes());
  }

  Future<File?> getFile(String url) async {
    if (!isSupported) return null;
    await _ensureReady();
    final key = keyForUrl(url);
    final hash = _hashKey(key);
    final entry = _index![hash];
    if (entry == null) return null;

    final file = _fileFor(hash, entry.extension);
    if (!await file.exists()) {
      _index!.remove(hash);
      await _saveIndex();
      return null;
    }

    entry.lastAccessMs = DateTime.now().millisecondsSinceEpoch;
    await _saveIndex();
    return file;
  }

  Future<File?> put(String url, Uint8List bytes, String extension) async {
    if (!isSupported) return null;
```

```
    if (bytes.isEmpty) return null;
    await _ensureReady();

    final key = keyForUrl(url);
    final hash = _hashKey(key);
    final ext = extension.replaceAll('.', '').toLowerCase();
    final file = _fileFor(hash, ext);

    final existing = _index![hash];
    if (existing != null && existing.extension != ext) {
      final old = _fileFor(hash, existing.extension);
      if (await old.exists()) await old.delete();
    }

    await file.writeAsBytes(bytes, flush: true);
    _index![hash] = _VoiceCacheEntry(
      extension: ext,
      sizeBytes: bytes.length,
      lastAccessMs: DateTime.now().millisecondsSinceEpoch,
    );
    await _evictIfNeeded();
    await _saveIndex();
    return file;
  }

  Future<void> _evictIfNeeded() async {
    var entries = _index!.entries.toList()
      ..sort((a, b) => a.value.lastAccessMs.compareTo(b.value.lastAccessMs));

    var total = entries.fold<int>(0, (s, e) => s + e.value.sizeBytes);

    while (entries.length > maxEntries || total > maxTotalBytes) {
      if (entries.isEmpty) break;
      final oldest = entries.removeAt(0);
      _index!.remove(oldest.key);
      total -= oldest.value.sizeBytes;
      final file = _fileFor(oldest.key, oldest.value.extension);
      if (await file.exists()) {
        try {
          await file.delete();
        } catch (_) {}
      }
    }
  }

  Future<int> totalBytes() async {
    if (!isSupported) return 0;
    await _ensureReady();
    return _index!.values.fold<int>(0, (s, e) => s + e.sizeBytes);
  }

  Future<int> entryCount() async {
    if (!isSupported) return 0;
    await _ensureReady();
    return _index!.length;
  }

  Future<void> clear() async {
    if (!isSupported) return;
    await _ensureReady();
    if (_dir != null && await _dir!.exists()) {
      try {
        await _dir!.delete(recursive: true);
      } catch (_) {}
    }
    _dir = null;
    _index = null;
    _initFuture = null;
  }
}

class _VoiceCacheEntry {
  _VoiceCacheEntry({
```

```
    required this.extension,
    required this.sizeBytes,
    required this.lastAccessMs,
  });

  final String extension;
  final int sizeBytes;
  int lastAccessMs;

  factory _VoiceCacheEntry.fromJson(Map<String, dynamic> json) {
    return _VoiceCacheEntry(
      extension: json['ext'] as String? ?? 'm4a',
      sizeBytes: (json['size'] as num?)?.toInt() ?? 0,
      lastAccessMs: (json['at'] as num?)?.toInt() ?? 0,
    );
  }

  Map<String, dynamic> toJson() => {
    'ext': extension,
    'size': sizeBytes,
    'at': lastAccessMs,
  };
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/services/voice_cache_service.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/theme/app_colors.dart

Four visual styles × light/dark Material 3 themes. This file (app colors) is one concrete piece of that layer.

Line count: 132. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/app_colors.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

class AppColors {
  static const darkChocolate = Color(
    0xFF2D2422,
  ); // Slightly darker for Linear feel
  static const coffeeBrown = Color(0xFF4A3731);
  static const warmBeige = Color(0xFFD7CCC8);
  static const cream = Color(0xFFFBFBF9); // Cleaner cream for Notion feel
  static const sand = Color(0xFFEFEBE9);

  static const mutedGold = Color(0xFFC5A059);
  static const warmAmber = Color(0xFFD4813E);
  static const softOlive = Color(0xFF8D917A);
  static const successGreen = Color(0xFF6B8E6B);
  static const errorRed = Color(0xFFAA5B5B);

  static const surfaceDark = Color(0xFF1E1E1E);
  static const surfaceLight = Color(0xFFFFFFFF);

  static const darkBackground = Color(0xFF0F0F0F);
  static const darkSurface = Color(0xFF1A1A1A);
  static const darkSurfaceElevated = Color(0xFF2A2A2A);
  static const darkSurfaceHighest = Color(0xFF343434);
  static const darkBorder = Color(0xFF525252);
  static const darkTextPrimary = Color(0xFFF5F5F5);
  static const darkTextSecondary = Color(0xFFD0D0D0); // ~40% brighter secondary
  static const darkTextTertiary = Color(0xFFB8B8B8);
  static const darkAccent = Color(0xFFC5A059); // mutedGold for highlights

  static const mathAccent = coffeeBrown;
  static const englishAccent = mutedGold;

  static Color accent(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    if (tokens != null) return tokens.accent;
    return Theme.of(context).brightness == Brightness.dark
        ? mutedGold
        : coffeeBrown;
  }

  static Color resolveAccent(BuildContext context, Color color) {
    final tokens = AppStyleTokens.maybeOf(context);
    if (tokens?.isMonochromeWorkspace ?? false) return ink(context);
    // Modern coffee/math browns vanish on Nature Paper night cards.
```

```
  if ((tokens?.isNaturePaper ?? false) &&
      Theme.of(context).brightness == Brightness.dark &&
      color.computeLuminance() < 0.2) {
    return NaturePaperColors.moonBeige;
  }
  return color;
}

static Color onAccent(BuildContext context) {
  final mono =
      AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
  final isDark = Theme.of(context).brightness == Brightness.dark;
  if (mono) return isDark ? Colors.black : Colors.white;
  return isDark ? const Color(0xFF1A1408) : Colors.white;
}

static Color success(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.success ?? successGreen;

static Color warning(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.warning ?? warmAmber;

static Color danger(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.danger ?? errorRed;

static Color goldAccent(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.gold ?? mutedGold;

static Color ink(BuildContext context) =>
    Theme.of(context).colorScheme.onSurface;

static Color inkAlpha(BuildContext context, double alpha) {
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final base =
      AppStyleTokens.maybeOf(context)?.inkBase ??
      (isDark ? darkTextPrimary : darkChocolate);
  if (!isDark) {
    return base.withValues(alpha: alpha);
  }

  final darkAlpha = switch (alpha) {
    <= 0.05 => 0.16,
    <= 0.08 => 0.22,
    <= 0.12 => 0.28,
    <= 0.2 => 0.36,
    <= 0.25 => 0.80,
    <= 0.35 => 0.84,
    <= 0.45 => 0.88,
    <= 0.55 => 0.90,
    <= 0.7 => 0.92,
    _ => alpha.clamp(0.80, 1.0),
  };
  return base.withValues(alpha: darkAlpha);
}

static Color controlSurface(BuildContext context) {
  final tokens = AppStyleTokens.maybeOf(context);
  if (tokens != null) return tokens.controlSurface;
  final isDark = Theme.of(context).brightness == Brightness.dark;
  return isDark ? darkSurfaceHighest : Colors.white;
}

static Color softWash(BuildContext context, {double lightAlpha = 0.5}) {
  final tokens = AppStyleTokens.maybeOf(context);
  final isDark = Theme.of(context).brightness == Brightness.dark;
  if (isDark) {
    return tokens?.softWashColor ?? Colors.white.withValues(alpha: 0.10);
  }
  return (tokens?.softWashColor ?? sand).withValues(alpha: lightAlpha);
}

static Color softSurface(BuildContext context) {
  final tokens = AppStyleTokens.maybeOf(context);
```

```
    if (tokens != null) return tokens.softSurface;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return isDark ? darkSurfaceHighest : sand;
  }

  static Color sheetSurface(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    if (tokens != null) return tokens.sheetSurface;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return isDark ? darkSurfaceElevated : Colors.white;
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/app_colors.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/core/theme/app_style.dart

Four visual styles × light/dark Material 3 themes. This file (app style) is one concrete piece of that layer.

Line count: 16. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/theme/app_style.dart**

```
enum AppStyle {
  modern,
  naturePaper,
  softNeumorphism,
  monochromeWorkspace;

  String get code => name;

  static AppStyle? tryParse(String? code) {
    if (code == null || code.isEmpty) return null;
    for (final v in AppStyle.values) {
      if (v.code == code) return v;
    }
    return null;
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/app_style.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/theme/app_style_tokens.dart

Four visual styles × light/dark Material 3 themes. This file (app style tokens) is one concrete piece of that layer.

Line count: 374. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style.dart';
- import 'package:mu_study/core/theme/monochrome_workspace_colors.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/app_style_tokens.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style.dart';
import 'package:mu_study/core/theme/monochrome_workspace_colors.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

class AppStyleTokens extends ThemeExtension<AppStyleTokens> {
  const AppStyleTokens({
    required this.style,
    required this.accent,
    required this.inkBase,
    required this.success,
    required this.warning,
    required this.danger,
    required this.gold,
    required this.controlSurface,
    required this.softSurface,
    required this.softWashColor,
    required this.sheetSurface,
    required this.cardSurface,
    required this.cardBorder,
    required this.cardRadius,
    required this.navSurface,
    required this.navGlow,
    required this.headerGlow,
    this.chartPositive,
    this.chartWarning,
    this.chartDanger,
  });

  final AppStyle style;

  final Color accent;

  final Color inkBase;

  final Color success;
  final Color warning;
  final Color danger;
```

```
final Color gold;

final Color controlSurface;

final Color softSurface;

final Color softWashColor;

final Color sheetSurface;

final Color cardSurface;

final Color cardBorder;

final double cardRadius;

final Color navSurface;

final Color navGlow;

final Color headerGlow;

final LinearGradient? chartPositive;
final LinearGradient? chartWarning;
final LinearGradient? chartDanger;

bool get isNaturePaper => style == AppStyle.naturePaper;

bool get isSoftNeumorphism => style == AppStyle.softNeumorphism;

bool get isMonochromeWorkspace => style == AppStyle.monochromeWorkspace;

bool get usesAmbientBackdrop => isNaturePaper || isSoftNeumorphism;

bool get prefersChartLabelsInside => isNaturePaper || isSoftNeumorphism;

static AppStyleTokens? maybeOf(BuildContext context) =>
    Theme.of(context).extension<AppStyleTokens>();

static const modernLight = AppStyleTokens(
  style: AppStyle.modern,
  accent: AppColors.coffeeBrown,
  inkBase: AppColors.darkChocolate,
  success: AppColors.successGreen,
  warning: AppColors.warmAmber,
  danger: AppColors.errorRed,
  gold: AppColors.mutedGold,
  controlSurface: Colors.white,
  softSurface: AppColors.sand,
  softWashColor: AppColors.sand,
  sheetSurface: Colors.white,
  cardSurface: Colors.white,
  cardBorder: Color(0x0F2D2422), // darkChocolate @ 6%
  cardRadius: 16,
  navSurface: Color(0xE0FBFBF9), // cream @ 88%
  navGlow: Colors.transparent,
  headerGlow: Colors.transparent,
);

static const modernDark = AppStyleTokens(
  style: AppStyle.modern,
  accent: AppColors.mutedGold,
  inkBase: AppColors.darkTextPrimary,
  success: AppColors.successGreen,
  warning: AppColors.warmAmber,
  danger: Color(0xFFE57373),
  gold: AppColors.mutedGold,
  controlSurface: AppColors.darkSurfaceHighest,
  softSurface: AppColors.darkSurfaceHighest,
  softWashColor: Color(0x1AFFFFFF), // white @ 10%, used as-is in dark
  sheetSurface: AppColors.darkSurfaceElevated,
  cardSurface: AppColors.darkSurfaceElevated,
  cardBorder: AppColors.darkBorder,
```

```
    cardRadius: 16,
    navSurface: Color(0xF5343434), // darkSurfaceHighest @ 96%
    navGlow: Colors.transparent,
    headerGlow: Colors.transparent,
  );

  static const natureLight = AppStyleTokens(
    style: AppStyle.naturePaper,
    accent: NaturePaperColors.forest,
    inkBase: NaturePaperColors.dayInk,
    success: NaturePaperColors.sage,
    warning: NaturePaperColors.softOrange,
    danger: NaturePaperColors.dayDanger,
    gold: NaturePaperColors.softOrange,
    controlSurface: NaturePaperColors.dayCard,
    softSurface: NaturePaperColors.paperBeige,
    softWashColor: NaturePaperColors.paperBeige,
    sheetSurface: NaturePaperColors.dayCard,
    cardSurface: NaturePaperColors.dayCard,
    cardBorder: Color(0x2E8A6A4A), // warmBrown @ 18%
    cardRadius: 24,
    navSurface: Color(0xE6FFFCF4), // dayCard @ 90%
    navGlow: NaturePaperColors.sunGlow,
    headerGlow: NaturePaperColors.sunGlow,
    chartPositive: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [NaturePaperColors.forest, Color(0xFFC9C36A)],
    ),
    chartWarning: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [NaturePaperColors.warmBrown, NaturePaperColors.softOrange],
    ),
    chartDanger: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF7E4437), NaturePaperColors.dayDanger],
    ),
  );

  static const natureDark = AppStyleTokens(
    style: AppStyle.naturePaper,
    accent: NaturePaperColors.amber,
    inkBase: NaturePaperColors.nightInk,
    success: NaturePaperColors.sageNight,
    warning: NaturePaperColors.amber,
    danger: NaturePaperColors.dustyRose,
    gold: NaturePaperColors.amber,
    controlSurface: NaturePaperColors.nightCardHigh,
    softSurface: NaturePaperColors.nightCardHigh,
    softWashColor: Color(0x14E6DCC8), // moonBeige @ 8%, used as-is in dark
    sheetSurface: NaturePaperColors.nightCardHigh,
    cardSurface: NaturePaperColors.nightCard,
    cardBorder: NaturePaperColors.nightBorder,
    cardRadius: 24,
    navSurface: Color(0xF0283630), // nightCardHigh @ 94%
    navGlow: NaturePaperColors.moonGlow,
    headerGlow: NaturePaperColors.moonGlow,
    chartPositive: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF4C6A50), NaturePaperColors.sageNight],
    ),
    chartWarning: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF8A6434), NaturePaperColors.amber],
    ),
    chartDanger: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF6E4444), NaturePaperColors.dustyRose],
```

```
    ),
  );

  static const softNeuLight = AppStyleTokens(
    style: AppStyle.softNeumorphism,
    accent: SoftNeumorphismColors.sage,
    inkBase: SoftNeumorphismColors.morningInk,
    success: SoftNeumorphismColors.sage,
    warning: SoftNeumorphismColors.amber,
    danger: SoftNeumorphismColors.morningDanger,
    gold: SoftNeumorphismColors.amber,
    controlSurface: SoftNeumorphismColors.morningSurfaceHigh,
    softSurface: SoftNeumorphismColors.morningInset,
    softWashColor: SoftNeumorphismColors.morningInset,
    sheetSurface: SoftNeumorphismColors.morningSurfaceHigh,
    cardSurface: SoftNeumorphismColors.morningSurface,
    cardBorder: Color(0x00000000),
    cardRadius: 18,
    navSurface: SoftNeumorphismColors.morningSurface,
    navGlow: SoftNeumorphismColors.sage,
    headerGlow: SoftNeumorphismColors.sage,
    chartPositive: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [SoftNeumorphismColors.sageDeep, SoftNeumorphismColors.sage],
    ),
    chartWarning: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFFB8844A), SoftNeumorphismColors.amber],
    ),
    chartDanger: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF8A5050), SoftNeumorphismColors.morningDanger],
    ),
  );

  static const softNeuDark = AppStyleTokens(
    style: AppStyle.softNeumorphism,
    accent: SoftNeumorphismColors.emerald,
    inkBase: SoftNeumorphismColors.nightInk,
    success: SoftNeumorphismColors.emerald,
    warning: SoftNeumorphismColors.amber,
    danger: SoftNeumorphismColors.nightDanger,
    gold: SoftNeumorphismColors.amber,
    controlSurface: SoftNeumorphismColors.nightSurfaceHigh,
    softSurface: SoftNeumorphismColors.nightInset,
    softWashColor: Color(0x14F2F0EB), // nightInk @ 8%
    sheetSurface: SoftNeumorphismColors.nightSurfaceHigh,
    cardSurface: SoftNeumorphismColors.nightSurface,
    cardBorder: Color(0x00000000),
    cardRadius: 18,
    navSurface: SoftNeumorphismColors.nightSurface,
    navGlow: SoftNeumorphismColors.emeraldGlow,
    headerGlow: SoftNeumorphismColors.emeraldGlow,
    chartPositive: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF3D6E55), SoftNeumorphismColors.emeraldGlow],
    ),
    chartWarning: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF8A6434), SoftNeumorphismColors.amber],
    ),
    chartDanger: LinearGradient(
      begin: Alignment.bottomCenter,
      end: Alignment.topCenter,
      colors: [Color(0xFF6E4444), SoftNeumorphismColors.nightDanger],
    ),
  );
```

```
  static const monoLight = AppStyleTokens(
    style: AppStyle.monochromeWorkspace,
    accent: MonochromeWorkspaceColors.lightAccent,
    inkBase: MonochromeWorkspaceColors.lightInk,
    success: MonochromeWorkspaceColors.lightSuccess,
    warning: MonochromeWorkspaceColors.lightWarning,
    danger: MonochromeWorkspaceColors.lightDanger,
    gold: MonochromeWorkspaceColors.lightGold,
    controlSurface: MonochromeWorkspaceColors.lightControl,
    softSurface: MonochromeWorkspaceColors.lightWash,
    softWashColor: MonochromeWorkspaceColors.lightWash,
    sheetSurface: MonochromeWorkspaceColors.lightCard,
    cardSurface: MonochromeWorkspaceColors.lightCard,
    cardBorder: MonochromeWorkspaceColors.lightBorder,
    cardRadius: 4,
    navSurface: Color(0xF5FFFFFF), // white @ 96%
    navGlow: Colors.transparent,
    headerGlow: Colors.transparent,
  );

  static const monoDark = AppStyleTokens(
    style: AppStyle.monochromeWorkspace,
    accent: MonochromeWorkspaceColors.darkAccent,
    inkBase: MonochromeWorkspaceColors.darkInk,
    success: MonochromeWorkspaceColors.darkSuccess,
    warning: MonochromeWorkspaceColors.darkWarning,
    danger: MonochromeWorkspaceColors.darkDanger,
    gold: MonochromeWorkspaceColors.darkGold,
    controlSurface: MonochromeWorkspaceColors.darkControl,
    softSurface: MonochromeWorkspaceColors.darkWash,
    softWashColor: Color(0x14FFFFFF), // white @ 8%
    sheetSurface: MonochromeWorkspaceColors.darkControl,
    cardSurface: MonochromeWorkspaceColors.darkCard,
    cardBorder: MonochromeWorkspaceColors.darkBorder,
    cardRadius: 4,
    navSurface: Color(0xF51A1A1A), // darkCard @ 96%
    navGlow: Colors.transparent,
    headerGlow: Colors.transparent,
  );

  @override
  AppStyleTokens copyWith({
    AppStyle? style,
    Color? accent,
    Color? inkBase,
    Color? success,
    Color? warning,
    Color? danger,
    Color? gold,
    Color? controlSurface,
    Color? softSurface,
    Color? softWashColor,
    Color? sheetSurface,
    Color? cardSurface,
    Color? cardBorder,
    double? cardRadius,
    Color? navSurface,
    Color? navGlow,
    Color? headerGlow,
    LinearGradient? chartPositive,
    LinearGradient? chartWarning,
    LinearGradient? chartDanger,
  }) {
    return AppStyleTokens(
      style: style ?? this.style,
      accent: accent ?? this.accent,
      inkBase: inkBase ?? this.inkBase,
      success: success ?? this.success,
      warning: warning ?? this.warning,
      danger: danger ?? this.danger,
      gold: gold ?? this.gold,
      controlSurface: controlSurface ?? this.controlSurface,
      softSurface: softSurface ?? this.softSurface,
```

```
      softWashColor: softWashColor ?? this.softWashColor,
      sheetSurface: sheetSurface ?? this.sheetSurface,
      cardSurface: cardSurface ?? this.cardSurface,
      cardBorder: cardBorder ?? this.cardBorder,
      cardRadius: cardRadius ?? this.cardRadius,
      navSurface: navSurface ?? this.navSurface,
      navGlow: navGlow ?? this.navGlow,
      headerGlow: headerGlow ?? this.headerGlow,
      chartPositive: chartPositive ?? this.chartPositive,
      chartWarning: chartWarning ?? this.chartWarning,
      chartDanger: chartDanger ?? this.chartDanger,
    );
  }

  @override
  AppStyleTokens lerp(ThemeExtension<AppStyleTokens>? other, double t) {
    if (other is! AppStyleTokens) return this;
    return AppStyleTokens(
      style: t < 0.5 ? style : other.style,
      accent: Color.lerp(accent, other.accent, t)!,
      inkBase: Color.lerp(inkBase, other.inkBase, t)!,
      success: Color.lerp(success, other.success, t)!,
      warning: Color.lerp(warning, other.warning, t)!,
      danger: Color.lerp(danger, other.danger, t)!,
      gold: Color.lerp(gold, other.gold, t)!,
      controlSurface: Color.lerp(controlSurface, other.controlSurface, t)!,
      softSurface: Color.lerp(softSurface, other.softSurface, t)!,
      softWashColor: Color.lerp(softWashColor, other.softWashColor, t)!,
      sheetSurface: Color.lerp(sheetSurface, other.sheetSurface, t)!,
      cardSurface: Color.lerp(cardSurface, other.cardSurface, t)!,
      cardBorder: Color.lerp(cardBorder, other.cardBorder, t)!,
      cardRadius: cardRadius + (other.cardRadius - cardRadius) * t,
      navSurface: Color.lerp(navSurface, other.navSurface, t)!,
      navGlow: Color.lerp(navGlow, other.navGlow, t)!,
      headerGlow: Color.lerp(headerGlow, other.headerGlow, t)!,
      chartPositive: t < 0.5 ? chartPositive : other.chartPositive,
      chartWarning: t < 0.5 ? chartWarning : other.chartWarning,
      chartDanger: t < 0.5 ? chartDanger : other.chartDanger,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/app_style_tokens.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/theme/app_text_theme.dart

Four visual styles × light/dark Material 3 themes. This file (app text theme) is one concrete piece of that layer.

Line count: 70. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:google_fonts/google_fonts.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/theme/app_text_theme.dart**

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

TextTheme buildJakartaTextTheme({
  required TextTheme base,
  required Color primary,
  required Color secondary,
  required Color tertiary,
}) {
  final jakarta = GoogleFonts.plusJakartaSansTextTheme(base);
  return jakarta.copyWith(
    displayLarge: jakarta.displayLarge?.copyWith(
      fontWeight: FontWeight.w800,
      color: primary,
      letterSpacing: -1,
    ),
    displayMedium: jakarta.displayMedium?.copyWith(
      fontWeight: FontWeight.w800,
      color: primary,
      letterSpacing: -0.8,
    ),
    displaySmall: jakarta.displaySmall?.copyWith(
      fontWeight: FontWeight.w700,
      color: primary,
      letterSpacing: -0.5,
    ),
    headlineLarge: jakarta.headlineLarge?.copyWith(
      fontWeight: FontWeight.w700,
      color: primary,
    ),
    headlineMedium: jakarta.headlineMedium?.copyWith(
      fontWeight: FontWeight.w700,
      color: primary,
    ),
    headlineSmall: jakarta.headlineSmall?.copyWith(
      fontWeight: FontWeight.w700,
      color: primary,
    ),
    titleLarge: jakarta.titleLarge?.copyWith(
      fontWeight: FontWeight.w700,
      color: primary,
    ),
    titleMedium: jakarta.titleMedium?.copyWith(
      fontWeight: FontWeight.w600,
      color: primary,
    ),
    titleSmall: jakarta.titleSmall?.copyWith(
      fontWeight: FontWeight.w600,
```

```
      color: primary,
    ),
    bodyLarge: jakarta.bodyLarge?.copyWith(
      color: primary.withValues(alpha: 0.92),
      fontSize: 16,
    ),
    bodyMedium: jakarta.bodyMedium?.copyWith(color: secondary, fontSize: 14),
    bodySmall: jakarta.bodySmall?.copyWith(color: tertiary, fontSize: 12),
    labelLarge: jakarta.labelLarge?.copyWith(
      fontWeight: FontWeight.w600,
      color: primary,
    ),
    labelMedium: jakarta.labelMedium?.copyWith(
      fontWeight: FontWeight.w600,
      color: secondary,
    ),
    labelSmall: jakarta.labelSmall?.copyWith(
      fontWeight: FontWeight.w600,
      color: tertiary,
    ),
  );
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/app_text_theme.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/theme/app_theme.dart

Four visual styles × light/dark Material 3 themes. This file (app theme) is one concrete piece of that layer.

Line count: 481. Import count: 8. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/app_text_theme.dart';
- import 'package:mu_study/core/theme/monochrome_workspace_theme.dart';
- import 'package:mu_study/core/theme/nature_paper_theme.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_theme.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/app_theme.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/app_text_theme.dart';
import 'package:mu_study/core/theme/monochrome_workspace_theme.dart';
import 'package:mu_study/core/theme/nature_paper_theme.dart';
import 'package:mu_study/core/theme/soft_neumorphism_theme.dart';

class AppTheme {
  /// Light theme for the given [style] (defaults to Modern Professional).
  static ThemeData light([AppStyle style = AppStyle.modern]) => switch (style) {
    AppStyle.modern => _modernLight(),
    AppStyle.naturePaper => NaturePaperTheme.light(),
    AppStyle.softNeumorphism => SoftNeumorphismTheme.light(),
    AppStyle.monochromeWorkspace => MonochromeWorkspaceTheme.light(),
  };

  /// Dark theme for the given [style] (defaults to Modern Professional).
  static ThemeData dark([AppStyle style = AppStyle.modern]) => switch (style) {
    AppStyle.modern => _modernDark(),
    AppStyle.naturePaper => NaturePaperTheme.dark(),
    AppStyle.softNeumorphism => SoftNeumorphismTheme.dark(),
    AppStyle.monochromeWorkspace => MonochromeWorkspaceTheme.dark(),
  };

  static TextTheme _jakartaTextTheme({
    required TextTheme base,
    required Color primary,
    required Color secondary,
    required Color tertiary,
  }) => buildJakartaTextTheme(
    base: base,
    primary: primary,
    secondary: secondary,
    tertiary: tertiary,
  );

  static ThemeData _modernLight() {
```

```
    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: ColorScheme.fromSeed(
        seedColor: AppColors.coffeeBrown,
        primary: AppColors.coffeeBrown,
        secondary: AppColors.mutedGold,
        surface: AppColors.cream,
        onSurface: AppColors.darkChocolate,
        surfaceContainerLow: AppColors.sand,
        surfaceContainer: Colors.white,
      ),
    );

    final textTheme = _jakartaTextTheme(
      base: base.textTheme,
      primary: AppColors.darkChocolate,
      secondary: AppColors.darkChocolate.withValues(alpha: 0.8),
      tertiary: AppColors.darkChocolate.withValues(alpha: 0.55),
    );

    return base.copyWith(
      scaffoldBackgroundColor: AppColors.cream,
      extensions: const [AppStyleTokens.modernLight],
      textTheme: textTheme,
      appBarTheme: AppBarTheme(
        elevation: 0,
        centerTitle: false,
        backgroundColor: Colors.transparent,
        foregroundColor: AppColors.darkChocolate,
        surfaceTintColor: Colors.transparent,
        titleTextStyle: textTheme.titleMedium?.copyWith(
          fontSize: 18,
          fontWeight: FontWeight.w700,
          color: AppColors.darkChocolate,
        ),
      ),
      cardTheme: CardThemeData(
        elevation: 0,
        color: Colors.white,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(16),
          side: BorderSide(
            color: AppColors.darkChocolate.withValues(alpha: 0.05),
          ),
        ),
        margin: const EdgeInsets.symmetric(vertical: 8),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: Colors.white,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: BorderSide(
            color: AppColors.darkChocolate.withValues(alpha: 0.1),
          ),
        ),
        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: BorderSide(
            color: AppColors.darkChocolate.withValues(alpha: 0.1),
          ),
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: const BorderSide(color: AppColors.coffeeBrown, width: 2),
        ),
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 16,
          vertical: 16,
        ),
        labelStyle: const TextStyle(color: AppColors.coffeeBrown),
      ),
```

```
elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: AppColors.coffeeBrown,
    foregroundColor: Colors.white,
    elevation: 0,
    padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    textStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
),
outlinedButtonTheme: OutlinedButtonThemeData(
  style: OutlinedButton.styleFrom(
    foregroundColor: AppColors.coffeeBrown,
    side: BorderSide(color: AppColors.coffeeBrown.withValues(alpha: 0.2)),
    padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(12),
    ),
    textStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
),
textButtonTheme: TextButtonThemeData(
  style: TextButton.styleFrom(
    foregroundColor: AppColors.coffeeBrown,
    textStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w600,
    ),
  ),
),
floatingActionButtonTheme: FloatingActionButtonThemeData(
  backgroundColor: AppColors.coffeeBrown,
  foregroundColor: Colors.white,
  elevation: 6,
  focusElevation: 8,
  hoverElevation: 8,
  highlightElevation: 12,
  shape: const StadiumBorder(),
  extendedTextStyle: textTheme.labelLarge?.copyWith(
    fontWeight: FontWeight.w700,
    fontSize: 15,
  ),
),
navigationBarTheme: NavigationBarThemeData(
  backgroundColor: Colors.white.withValues(alpha: 0.8),
  indicatorColor: AppColors.warmBeige.withValues(alpha: 0.3),
  elevation: 0,
  labelTextStyle: WidgetStateProperty.resolveWith((states) {
    final selected = states.contains(WidgetState.selected);
    return textTheme.labelSmall!.copyWith(
      fontSize: 11,
      fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
      color: selected
          ? AppColors.coffeeBrown
          : AppColors.darkChocolate.withValues(alpha: 0.5),
    );
  }),
  iconTheme: WidgetStateProperty.resolveWith((states) {
    return IconThemeData(
      size: 24,
      color: states.contains(WidgetState.selected)
          ? AppColors.coffeeBrown
          : AppColors.darkChocolate.withValues(alpha: 0.5),
    );
  }),
),
```

```
    );
  }

  static ThemeData _modernDark() {
    const onSurface = AppColors.darkTextPrimary;
    const onSurfaceVariant = AppColors.darkTextSecondary;
    const outline = AppColors.darkBorder;

    final colorScheme = ColorScheme(
      brightness: Brightness.dark,
      primary: AppColors.mutedGold,
      onPrimary: const Color(0xFF1A1408),
      primaryContainer: const Color(0xFF3D3420),
      onPrimaryContainer: AppColors.warmBeige,
      secondary: AppColors.warmBeige,
      onSecondary: const Color(0xFF1A1512),
      secondaryContainer: const Color(0xFF3A322E),
      onSecondaryContainer: AppColors.warmBeige,
      tertiary: AppColors.warmAmber,
      onTertiary: const Color(0xFF1A1208),
      error: const Color(0xFFE57373),
      onError: const Color(0xFF1A0A0A),
      surface: AppColors.darkSurface,
      onSurface: onSurface,
      onSurfaceVariant: onSurfaceVariant,
      surfaceContainerLowest: AppColors.darkBackground,
      surfaceContainerLow: const Color(0xFF151515),
      surfaceContainer: AppColors.darkSurface,
      surfaceContainerHigh: AppColors.darkSurfaceElevated,
      surfaceContainerHighest: AppColors.darkSurfaceHighest,
      outline: outline,
      outlineVariant: const Color(0xFF3D3D3D),
      shadow: Colors.black,
      scrim: Colors.black,
      inverseSurface: const Color(0xFFE8E8E8),
      onInverseSurface: const Color(0xFF1A1A1A),
      inversePrimary: AppColors.coffeeBrown,
    );

    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.dark,
      colorScheme: colorScheme,
    );

    final textTheme = _jakartaTextTheme(
      base: base.textTheme,
      primary: onSurface,
      secondary: onSurfaceVariant,
      tertiary: AppColors.darkTextTertiary,
    );

    return base.copyWith(
      scaffoldBackgroundColor: AppColors.darkBackground,
      canvasColor: AppColors.darkBackground,
      extensions: const [AppStyleTokens.modernDark],
      dividerColor: outline,
      disabledColor: AppColors.darkTextTertiary,
      textTheme: textTheme,
      iconTheme: const IconThemeData(color: onSurfaceVariant),
      primaryIconTheme: const IconThemeData(color: AppColors.mutedGold),
      dividerTheme: const DividerThemeData(
        color: outline,
        thickness: 1,
        space: 1,
      ),
      listTileTheme: ListTileThemeData(
        iconColor: onSurfaceVariant,
        textColor: onSurface,
        subtitleTextStyle: textTheme.bodySmall?.copyWith(
          color: onSurfaceVariant,
          fontSize: 13,
        ),
```

```
    ),
    chipTheme: ChipThemeData(
      backgroundColor: AppColors.mutedGold.withValues(alpha: 0.16),
      disabledColor: AppColors.darkSurface,
      selectedColor: AppColors.mutedGold.withValues(alpha: 0.32),
      secondarySelectedColor: AppColors.mutedGold.withValues(alpha: 0.32),
      labelStyle: textTheme.labelLarge!.copyWith(
        color: AppColors.warmBeige,
        fontWeight: FontWeight.w700,
      ),
      secondaryLabelStyle: textTheme.labelLarge!.copyWith(
        color: AppColors.warmBeige,
        fontWeight: FontWeight.w700,
      ),
      side: BorderSide(color: AppColors.mutedGold.withValues(alpha: 0.45)),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
    ),
    tabBarTheme: TabBarThemeData(
      labelColor: AppColors.mutedGold,
      unselectedLabelColor: onSurfaceVariant,
      indicatorColor: AppColors.mutedGold,
      dividerColor: outline,
      labelStyle: textTheme.labelLarge?.copyWith(fontWeight: FontWeight.w700),
      unselectedLabelStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w500,
      ),
    ),
    appBarTheme: AppBarTheme(
      elevation: 0,
      centerTitle: false,
      backgroundColor: AppColors.darkBackground,
      foregroundColor: onSurface,
      surfaceTintColor: Colors.transparent,
      iconTheme: const IconThemeData(color: onSurface),
      actionsIconTheme: const IconThemeData(color: onSurfaceVariant),
      titleTextStyle: textTheme.titleMedium?.copyWith(
        fontSize: 18,
        fontWeight: FontWeight.w700,
        color: onSurface,
      ),
    ),
    cardTheme: CardThemeData(
      elevation: 0,
      color: AppColors.darkSurfaceElevated,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black.withValues(alpha: 0.55),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: const BorderSide(color: outline, width: 1),
      ),
      margin: const EdgeInsets.symmetric(vertical: 8),
    ),
    dialogTheme: DialogThemeData(
      backgroundColor: AppColors.darkSurfaceElevated,
      surfaceTintColor: Colors.transparent,
      titleTextStyle: textTheme.titleMedium?.copyWith(
        fontSize: 18,
        fontWeight: FontWeight.w700,
        color: onSurface,
      ),
      contentTextStyle: textTheme.bodyMedium?.copyWith(
        color: onSurfaceVariant,
      ),
    ),
    bottomSheetTheme: const BottomSheetThemeData(
      backgroundColor: AppColors.darkSurfaceElevated,
      surfaceTintColor: Colors.transparent,
      modalBackgroundColor: AppColors.darkSurfaceElevated,
    ),
    snackBarTheme: SnackBarThemeData(
      backgroundColor: AppColors.darkSurfaceHighest,
      contentTextStyle: textTheme.bodyMedium?.copyWith(color: onSurface),
      actionTextColor: AppColors.mutedGold,
```

```
      ),
      popupMenuTheme: PopupMenuThemeData(
        color: AppColors.darkSurfaceHighest,
        surfaceTintColor: Colors.transparent,
        textStyle: textTheme.bodyMedium?.copyWith(color: onSurface),
      ),
      dropdownMenuTheme: DropdownMenuThemeData(
        textStyle: textTheme.bodyMedium?.copyWith(color: onSurface),
        inputDecorationTheme: InputDecorationTheme(
          filled: true,
          fillColor: AppColors.darkSurfaceHighest,
        ),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: AppColors.darkSurfaceHighest,
        hintStyle: textTheme.bodyMedium?.copyWith(
          color: AppColors.darkTextTertiary,
        ),
        labelStyle: textTheme.bodyMedium?.copyWith(color: onSurfaceVariant),
        floatingLabelStyle: textTheme.bodyMedium?.copyWith(
          color: AppColors.mutedGold,
        ),
        prefixIconColor: onSurfaceVariant,
        suffixIconColor: onSurfaceVariant,
        border: const OutlineInputBorder(
          borderRadius: BorderRadius.all(Radius.circular(12)),
          borderSide: BorderSide(color: outline),
        ),
        enabledBorder: const OutlineInputBorder(
          borderRadius: BorderRadius.all(Radius.circular(12)),
          borderSide: BorderSide(color: outline),
        ),
        focusedBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),
          borderSide: const BorderSide(color: AppColors.mutedGold, width: 2),
        ),
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 16,
          vertical: 16,
        ),
      ),
      elevatedButtonTheme: ElevatedButtonThemeData(
```

**lib/core/theme/app_theme.dart — continued (2/2)**

```
        style: ElevatedButton.styleFrom(
          backgroundColor: AppColors.mutedGold,
          foregroundColor: const Color(0xFF1A1408),
          disabledBackgroundColor: AppColors.darkSurfaceHighest,
          disabledForegroundColor: AppColors.darkTextTertiary,
          elevation: 0,
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
          textStyle: textTheme.labelLarge?.copyWith(
            fontWeight: FontWeight.w700,
            fontSize: 15,
          ),
        ),
      ),
      outlinedButtonTheme: OutlinedButtonThemeData(
        style: OutlinedButton.styleFrom(
          foregroundColor: AppColors.mutedGold,
          side: BorderSide(color: AppColors.mutedGold.withValues(alpha: 0.65)),
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(12),
          ),
          textStyle: textTheme.labelLarge?.copyWith(
            fontWeight: FontWeight.w700,
            fontSize: 15,
```

```
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: AppColors.mutedGold,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: AppColors.mutedGold,
    foregroundColor: const Color(0xFF1A1408),
    elevation: 6,
    focusElevation: 8,
    hoverElevation: 8,
    highlightElevation: 12,
    shape: const StadiumBorder(),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
  navigationBarTheme: NavigationBarThemeData(
    backgroundColor: AppColors.darkSurfaceElevated,
    indicatorColor: AppColors.mutedGold.withValues(alpha: 0.28),
    elevation: 0,
    surfaceTintColor: Colors.transparent,
    labelTextStyle: WidgetStateProperty.resolveWith((states) {
      final selected = states.contains(WidgetState.selected);
      return textTheme.labelSmall!.copyWith(
        fontSize: 11,
        fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
        color: selected ? AppColors.mutedGold : onSurfaceVariant,
      );
    }),
    iconTheme: WidgetStateProperty.resolveWith((states) {
      final selected = states.contains(WidgetState.selected);
      return IconThemeData(
        size: 24,
        color: selected ? AppColors.mutedGold : onSurfaceVariant,
      );
    }),
  ),
  navigationRailTheme: NavigationRailThemeData(
    backgroundColor: AppColors.darkSurface,
    indicatorColor: AppColors.mutedGold.withValues(alpha: 0.28),
    selectedIconTheme: const IconThemeData(color: AppColors.mutedGold),
    unselectedIconTheme: const IconThemeData(color: onSurfaceVariant),
    selectedLabelTextStyle: textTheme.labelLarge?.copyWith(
      color: AppColors.mutedGold,
      fontWeight: FontWeight.w700,
    ),
    unselectedLabelTextStyle: textTheme.labelLarge?.copyWith(
      color: onSurfaceVariant,
      fontWeight: FontWeight.w500,
    ),
  ),
  switchTheme: SwitchThemeData(
    thumbColor: WidgetStateProperty.resolveWith((states) {
      if (states.contains(WidgetState.selected)) {
        return AppColors.mutedGold;
      }
      return onSurfaceVariant;
    }),
    trackColor: WidgetStateProperty.resolveWith((states) {
      if (states.contains(WidgetState.selected)) {
        return AppColors.mutedGold.withValues(alpha: 0.42);
      }
      return AppColors.darkSurfaceHighest;
    }),
  ),
```

```
      progressIndicatorTheme: const ProgressIndicatorThemeData(
        color: AppColors.mutedGold,
        circularTrackColor: AppColors.darkSurfaceHighest,
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/app_theme.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/theme/monochrome_workspace_colors.dart

Four visual styles × light/dark Material 3 themes. This file (monochrome workspace colors) is one concrete piece of that layer.

Line count: 47. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/theme/monochrome_workspace_colors.dart**

```
import 'package:flutter/material.dart';

class MonochromeWorkspaceColors {
  static const lightBg = Color(0xFFFAFAFA);

  static const lightCard = Color(0xFFFFFFFF);

  static const lightControl = Color(0xFFFFFFFF);

  static const lightWash = Color(0xFFF2F2F2);

  static const lightInk = Color(0xFF111111);

  static const lightTextSecondary = Color(0xFF6B6B6B);
  static const lightTextTertiary = Color(0xFF8E8E8E);

  static const lightBorder = Color(0xFFE6E6E6);

  static const lightAccent = Color(0xFF111111);

  static const lightSuccess = Color(0xFF1A1A1A);
  static const lightWarning = Color(0xFF6B6B6B);
  static const lightDanger = Color(0xFF2E2E2E);
  static const lightGold = Color(0xFF4A4A4A);

  static const darkBg = Color(0xFF111111);

  static const darkCard = Color(0xFF1A1A1A);

  static const darkControl = Color(0xFF222222);

  static const darkWash = Color(0xFF181818);

  static const darkInk = Color(0xFFF5F5F5);

  static const darkTextSecondary = Color(0xFFA3A3A3);
  static const darkTextTertiary = Color(0xFF7A7A7A);

  static const darkBorder = Color(0xFF2E2E2E);

  static const darkAccent = Color(0xFFF5F5F5);

  static const darkSuccess = Color(0xFFE8E8E8);
  static const darkWarning = Color(0xFF9A9A9A);
  static const darkDanger = Color(0xFFD0D0D0);
  static const darkGold = Color(0xFFC0C0C0);
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/monochrome_workspace_colors.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/theme/monochrome_workspace_theme.dart

Four visual styles × light/dark Material 3 themes. This file (monochrome workspace theme) is one concrete piece of that layer.

Line count: 621. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/app_text_theme.dart';
- import 'package:mu_study/core/theme/monochrome_workspace_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/monochrome_workspace_theme.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/app_text_theme.dart';
import 'package:mu_study/core/theme/monochrome_workspace_colors.dart';

class MonochromeWorkspaceTheme {
  static const _fadeTransitions = PageTransitionsTheme(
    builders: {
      TargetPlatform.android: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.iOS: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.macOS: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.linux: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.windows: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.fuchsia: FadeUpwardsPageTransitionsBuilder(),
    },
  );

  static const _radius = 4.0;
  static const _controlRadius = 4.0;

  static ThemeData light() {
    const ink = MonochromeWorkspaceColors.lightInk;
    const secondary = MonochromeWorkspaceColors.lightTextSecondary;
    const tertiary = MonochromeWorkspaceColors.lightTextTertiary;
    const accent = MonochromeWorkspaceColors.lightAccent;
    const bg = MonochromeWorkspaceColors.lightBg;
    const card = MonochromeWorkspaceColors.lightCard;
    const border = MonochromeWorkspaceColors.lightBorder;

    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: ColorScheme.fromSeed(
        seedColor: accent,
        primary: accent,
        onPrimary: Colors.white,
        secondary: secondary,
        onSecondary: Colors.white,
        tertiary: tertiary,
        error: MonochromeWorkspaceColors.lightDanger,
        surface: card,
        onSurface: ink,
        surfaceContainerLowest: bg,
        surfaceContainerLow: MonochromeWorkspaceColors.lightWash,
```

```
      surfaceContainer: card,
      surfaceContainerHigh: card,
      surfaceContainerHighest: MonochromeWorkspaceColors.lightWash,
      outline: border,
    ),
  );

  final textTheme = _editorialTextTheme(
    base: base.textTheme,
    primary: ink,
    secondary: secondary,
    tertiary: tertiary,
  );

  return base.copyWith(
    scaffoldBackgroundColor: bg,
    canvasColor: bg,
    pageTransitionsTheme: _fadeTransitions,
    textTheme: textTheme,
    dividerColor: border,
    extensions: const [AppStyleTokens.monoLight],
    dividerTheme: const DividerThemeData(
      color: border,
      thickness: 1,
      space: 1,
    ),
    appBarTheme: AppBarTheme(
      elevation: 0,
      centerTitle: false,
      backgroundColor: Colors.transparent,
      foregroundColor: ink,
      surfaceTintColor: Colors.transparent,
      titleTextStyle: textTheme.titleLarge?.copyWith(
        fontSize: 20,
        fontWeight: FontWeight.w700,
        letterSpacing: -0.4,
        color: ink,
      ),
    ),
    cardTheme: CardThemeData(
      elevation: 0,
      color: card,
      surfaceTintColor: Colors.transparent,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_radius),
        side: const BorderSide(color: border),
      ),
      margin: const EdgeInsets.symmetric(vertical: 6),
    ),
    dialogTheme: DialogThemeData(
      backgroundColor: card,
      surfaceTintColor: Colors.transparent,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_radius),
        side: const BorderSide(color: border),
      ),
      titleTextStyle: textTheme.titleMedium?.copyWith(
        fontSize: 18,
        fontWeight: FontWeight.w700,
        color: ink,
      ),
      contentTextStyle: textTheme.bodyMedium?.copyWith(color: secondary),
    ),
    bottomSheetTheme: const BottomSheetThemeData(
      backgroundColor: card,
      surfaceTintColor: Colors.transparent,
      modalBackgroundColor: card,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(8)),
      ),
    ),
    snackBarTheme: SnackBarThemeData(
      backgroundColor: ink,
```

```
    contentTextStyle: textTheme.bodyMedium?.copyWith(color: Colors.white),
    actionTextColor: Colors.white,
    behavior: SnackBarBehavior.floating,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
    ),
  ),
  inputDecorationTheme: InputDecorationTheme(
    filled: true,
    fillColor: card,
    hintStyle: textTheme.bodyMedium?.copyWith(color: tertiary),
    labelStyle: const TextStyle(color: secondary),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: border),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: border),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: accent, width: 1.5),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: _monoButtonStyle(
      background: accent,
      foreground: Colors.white,
      textStyle: textTheme.labelLarge,
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: _monoButtonStyle(
      background: accent,
      foreground: Colors.white,
      textStyle: textTheme.labelLarge,
    ),
  ),
  outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
      foregroundColor: ink,
      side: const BorderSide(color: border, width: 1),
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_controlRadius),
      ),
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
        fontSize: 15,
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: ink,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: accent,
    foregroundColor: Colors.white,
    elevation: 0,
    focusElevation: 0,
    hoverElevation: 0,
    highlightElevation: 0,
    shape: RoundedRectangleBorder(
```

```
      borderRadius: BorderRadius.circular(_controlRadius),
    ),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
  chipTheme: ChipThemeData(
    backgroundColor: MonochromeWorkspaceColors.lightWash,
    selectedColor: ink.withValues(alpha: 0.10),
    labelStyle: textTheme.labelLarge!.copyWith(
      color: ink,
      fontWeight: FontWeight.w600,
    ),
    side: const BorderSide(color: border),
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
    ),
  ),
  switchTheme: SwitchThemeData(
    thumbColor: WidgetStateProperty.resolveWith((states) {
      if (states.contains(WidgetState.selected)) return Colors.white;
      return card;
    }),
    trackColor: WidgetStateProperty.resolveWith((states) {
      if (states.contains(WidgetState.selected)) return accent;
      return MonochromeWorkspaceColors.lightWash;
    }),
    trackOutlineColor: const WidgetStatePropertyAll(border),
  ),
  checkboxTheme: CheckboxThemeData(
    fillColor: WidgetStateProperty.resolveWith((states) {
      if (states.contains(WidgetState.selected)) return accent;
      return Colors.transparent;
    }),
    checkColor: const WidgetStatePropertyAll(Colors.white),
    side: const BorderSide(color: border, width: 1.4),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(3)),
  ),
  progressIndicatorTheme: ProgressIndicatorThemeData(
    color: accent,
    circularTrackColor: MonochromeWorkspaceColors.lightWash,
    linearTrackColor: MonochromeWorkspaceColors.lightWash,
  ),
  navigationBarTheme: NavigationBarThemeData(
    backgroundColor: card,
    indicatorColor: ink.withValues(alpha: 0.08),
    elevation: 0,
    labelTextStyle: WidgetStateProperty.resolveWith((states) {
      final selected = states.contains(WidgetState.selected);
      return textTheme.labelSmall!.copyWith(
        fontSize: 11,
        fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
        color: selected ? ink : tertiary,
      );
    }),
    iconTheme: WidgetStateProperty.resolveWith((states) {
      return IconThemeData(
        size: 22,
        color: states.contains(WidgetState.selected) ? ink : tertiary,
      );
    }),
  ),
  navigationRailTheme: NavigationRailThemeData(
    backgroundColor: card,
    indicatorColor: ink.withValues(alpha: 0.08),
    selectedIconTheme: const IconThemeData(color: ink),
    unselectedIconTheme: const IconThemeData(color: tertiary),
  ),
  listTileTheme: ListTileThemeData(
    iconColor: secondary,
    textColor: ink,
    subtitleTextStyle: textTheme.bodySmall?.copyWith(
```

```
          color: secondary,
          fontSize: 13,
        ),
      ),
    );
  }

  static ThemeData dark() {
    const ink = MonochromeWorkspaceColors.darkInk;
    const secondary = MonochromeWorkspaceColors.darkTextSecondary;
    const tertiary = MonochromeWorkspaceColors.darkTextTertiary;
    const accent = MonochromeWorkspaceColors.darkAccent;
    const bg = MonochromeWorkspaceColors.darkBg;
    const card = MonochromeWorkspaceColors.darkCard;
    const control = MonochromeWorkspaceColors.darkControl;
    const border = MonochromeWorkspaceColors.darkBorder;

    final colorScheme = ColorScheme(
      brightness: Brightness.dark,
      primary: accent,
      onPrimary: MonochromeWorkspaceColors.darkBg,
      primaryContainer: control,
      onPrimaryContainer: ink,
      secondary: secondary,
      onSecondary: bg,
      secondaryContainer: control,
      onSecondaryContainer: ink,
      tertiary: tertiary,
      onTertiary: bg,
      error: MonochromeWorkspaceColors.darkDanger,
      onError: bg,
      surface: card,
      onSurface: ink,
      onSurfaceVariant: secondary,
      surfaceContainerLowest: bg,
      surfaceContainerLow: MonochromeWorkspaceColors.darkWash,
      surfaceContainer: card,
      surfaceContainerHigh: control,
      surfaceContainerHighest: const Color(0xFF2A2A2A),
      outline: border,
      outlineVariant: const Color(0xFF3A3A3A),
      shadow: Colors.black,
      scrim: Colors.black,
      inverseSurface: MonochromeWorkspaceColors.lightCard,
      onInverseSurface: MonochromeWorkspaceColors.lightInk,
      inversePrimary: MonochromeWorkspaceColors.lightAccent,
    );

    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.dark,
      colorScheme: colorScheme,
    );

    final textTheme = _editorialTextTheme(
      base: base.textTheme,
      primary: ink,
      secondary: secondary,
      tertiary: tertiary,
    );

    return base.copyWith(
      scaffoldBackgroundColor: bg,
      canvasColor: bg,
      pageTransitionsTheme: _fadeTransitions,
      dividerColor: border,
      disabledColor: tertiary,
      textTheme: textTheme,
      iconTheme: const IconThemeData(color: secondary),
      primaryIconTheme: const IconThemeData(color: accent),
      extensions: const [AppStyleTokens.monoDark],
      dividerTheme: const DividerThemeData(
        color: border,
```

```
        thickness: 1,
        space: 1,
      ),
      listTileTheme: ListTileThemeData(
        iconColor: secondary,
        textColor: ink,
        subtitleTextStyle: textTheme.bodySmall?.copyWith(
          color: secondary,
          fontSize: 13,
        ),
      ),
      appBarTheme: AppBarTheme(
        elevation: 0,
        centerTitle: false,
        backgroundColor: Colors.transparent,
        foregroundColor: ink,
        surfaceTintColor: Colors.transparent,
        iconTheme: const IconThemeData(color: ink),
        actionsIconTheme: const IconThemeData(color: secondary),
        titleTextStyle: textTheme.titleLarge?.copyWith(
          fontSize: 20,
          fontWeight: FontWeight.w700,
          letterSpacing: -0.4,
          color: ink,
        ),
      ),
      cardTheme: CardThemeData(
        elevation: 0,
        color: card,
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(_radius),
          side: const BorderSide(color: border),
        ),
        margin: const EdgeInsets.symmetric(vertical: 6),
      ),
      dialogTheme: DialogThemeData(
        backgroundColor: control,
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(_radius),
          side: const BorderSide(color: border),
        ),
        titleTextStyle: textTheme.titleMedium?.copyWith(
          fontSize: 18,
          fontWeight: FontWeight.w700,
          color: ink,
        ),
        contentTextStyle: textTheme.bodyMedium?.copyWith(color: secondary),
      ),
      bottomSheetTheme: const BottomSheetThemeData(
        backgroundColor: control,
        surfaceTintColor: Colors.transparent,
        modalBackgroundColor: control,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.vertical(top: Radius.circular(8)),
        ),
      ),
      snackBarTheme: SnackBarThemeData(
        backgroundColor: control,
```

**lib/core/theme/monochrome_workspace_theme.dart — continued (2/2)**

```
        contentTextStyle: textTheme.bodyMedium?.copyWith(color: ink),
        actionTextColor: accent,
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(_controlRadius),
          side: const BorderSide(color: border),
        ),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
```

```
    fillColor: card,
    hintStyle: textTheme.bodyMedium?.copyWith(color: tertiary),
    labelStyle: const TextStyle(color: secondary),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: border),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: border),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: accent, width: 1.5),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: _monoButtonStyle(
      background: accent,
      foreground: bg,
      textStyle: textTheme.labelLarge,
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: _monoButtonStyle(
      background: accent,
      foreground: bg,
      textStyle: textTheme.labelLarge,
    ),
  ),
  outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
      foregroundColor: ink,
      side: const BorderSide(color: border, width: 1),
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_controlRadius),
      ),
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
        fontSize: 15,
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: ink,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: accent,
    foregroundColor: bg,
    elevation: 0,
    focusElevation: 0,
    hoverElevation: 0,
    highlightElevation: 0,
    shape: RoundedRectangleBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
    ),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
  chipTheme: ChipThemeData(
    backgroundColor: control,
```

```
      selectedColor: accent.withValues(alpha: 0.16),
      labelStyle: textTheme.labelLarge!.copyWith(
        color: ink,
        fontWeight: FontWeight.w600,
      ),
      side: const BorderSide(color: border),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_controlRadius),
      ),
    ),
    switchTheme: SwitchThemeData(
      thumbColor: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.selected)) return bg;
        return secondary;
      }),
      trackColor: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.selected)) return accent;
        return control;
      }),
      trackOutlineColor: const WidgetStatePropertyAll(border),
    ),
    checkboxTheme: CheckboxThemeData(
      fillColor: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.selected)) return accent;
        return Colors.transparent;
      }),
      checkColor: const WidgetStatePropertyAll(
        MonochromeWorkspaceColors.darkBg,
      ),
      side: const BorderSide(color: border, width: 1.4),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(3)),
    ),
    progressIndicatorTheme: ProgressIndicatorThemeData(
      color: accent,
      circularTrackColor: control,
      linearTrackColor: control,
    ),
    navigationBarTheme: NavigationBarThemeData(
      backgroundColor: card,
      indicatorColor: accent.withValues(alpha: 0.12),
      elevation: 0,
      labelTextStyle: WidgetStateProperty.resolveWith((states) {
        final selected = states.contains(WidgetState.selected);
        return textTheme.labelSmall!.copyWith(
          fontSize: 11,
          fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
          color: selected ? ink : tertiary,
        );
      }),
      iconTheme: WidgetStateProperty.resolveWith((states) {
        return IconThemeData(
          size: 22,
          color: states.contains(WidgetState.selected) ? ink : tertiary,
        );
      }),
    ),
    navigationRailTheme: NavigationRailThemeData(
      backgroundColor: card,
      indicatorColor: accent.withValues(alpha: 0.12),
      selectedIconTheme: const IconThemeData(color: ink),
      unselectedIconTheme: const IconThemeData(color: tertiary),
    ),
  );
}

static TextTheme _editorialTextTheme({
  required TextTheme base,
  required Color primary,
  required Color secondary,
  required Color tertiary,
}) {
  final jakarta = buildJakartaTextTheme(
    base: base,
```

```
      primary: primary,
      secondary: secondary,
      tertiary: tertiary,
    );
    return jakarta.copyWith(
      displayLarge: jakarta.displayLarge?.copyWith(
        fontSize: 40,
        height: 1.15,
        letterSpacing: -1.2,
        fontWeight: FontWeight.w700,
      ),
      displayMedium: jakarta.displayMedium?.copyWith(
        fontSize: 32,
        height: 1.18,
        letterSpacing: -0.9,
        fontWeight: FontWeight.w700,
      ),
      headlineLarge: jakarta.headlineLarge?.copyWith(
        fontSize: 28,
        height: 1.2,
        letterSpacing: -0.6,
        fontWeight: FontWeight.w700,
      ),
      headlineMedium: jakarta.headlineMedium?.copyWith(
        fontSize: 24,
        height: 1.25,
        letterSpacing: -0.4,
        fontWeight: FontWeight.w700,
      ),
      headlineSmall: jakarta.headlineSmall?.copyWith(
        fontSize: 20,
        height: 1.3,
        letterSpacing: -0.3,
        fontWeight: FontWeight.w700,
      ),
      titleLarge: jakarta.titleLarge?.copyWith(
        fontSize: 18,
        height: 1.35,
        fontWeight: FontWeight.w600,
      ),
      labelSmall: jakarta.labelSmall?.copyWith(
        fontSize: 11,
        letterSpacing: 1.1,
        fontWeight: FontWeight.w600,
        color: tertiary,
      ),
      bodyLarge: jakarta.bodyLarge?.copyWith(height: 1.55),
      bodyMedium: jakarta.bodyMedium?.copyWith(height: 1.5),
    );
  }

  static ButtonStyle _monoButtonStyle({
    required Color background,
    required Color foreground,
    required TextStyle? textStyle,
  }) {
    return ElevatedButton.styleFrom(
      backgroundColor: background,
      foregroundColor: foreground,
      elevation: 0,
      shadowColor: Colors.transparent,
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_controlRadius),
      ),
      textStyle: textStyle?.copyWith(fontWeight: FontWeight.w700, fontSize: 15),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/monochrome_workspace_theme.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/theme/nature_paper_colors.dart

Four visual styles × light/dark Material 3 themes. This file (nature paper colors) is one concrete piece of that layer.

Line count: 42. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/theme/nature_paper_colors.dart**

```
import 'package:flutter/material.dart';

class NaturePaperColors {
  static const cream = Color(0xFFF8F3E8);
  static const paperBeige = Color(0xFFEADFCB);
  static const sage = Color(0xFF8FA97D);
  static const forest = Color(0xFF5F7A61);
  static const softOrange = Color(0xFFD79A57);
  static const warmBrown = Color(0xFF8A6A4A);

  static const paleBlue = Color(0xFFDCE9EC);

  static const sunGlow = Color(0xFFF0C387);

  static const dayInk = Color(0xFF3B2F22);

  static const dayCard = Color(0xFFFFFCF4);

  static const dayCardEdge = Color(0xFFDFD2B6);

  static const deepForest = Color(0xFF465D48);

  static const dayDanger = Color(0xFFAA5B4C);

  static const midnight = Color(0xFF121824);
  static const forestNight = Color(0xFF18221B);
  static const moonBeige = Color(0xFFE6DCC8);
  static const sageNight = Color(0xFF7E9A78);
  static const amber = Color(0xFFD7A25A);
  static const dustyRose = Color(0xFFA56A6A);

  static const nightInk = Color(0xFFF1EBDC);
  static const nightTextSecondary = Color(0xFFCFC7B0);
  static const nightTextTertiary = Color(0xFFB3AA92);

  static const nightCard = Color(0xFF1E2924);
  static const nightCardHigh = Color(0xFF283630);
  static const nightCardHighest = Color(0xFF31403A);
  static const nightBorder = Color(0xFF43514A);

  static const moonGlow = Color(0xFFE8D9AE);
}
```

### Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/nature_paper_colors.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?

3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/theme/nature_paper_theme.dart

Four visual styles × light/dark Material 3 themes. This file (nature paper theme) is one concrete piece of that layer.

Line count: 577. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/app_text_theme.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/nature_paper_theme.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/app_text_theme.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

class NaturePaperTheme {
  static const _fadeTransitions = PageTransitionsTheme(
    builders: {
      TargetPlatform.android: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.iOS: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.macOS: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.linux: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.windows: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.fuchsia: FadeUpwardsPageTransitionsBuilder(),
    },
  );

  static ThemeData light() {
    const ink = NaturePaperColors.dayInk;
    const forest = NaturePaperColors.forest;
    const card = NaturePaperColors.dayCard;
    final border = NaturePaperColors.warmBrown.withValues(alpha: 0.18);

    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: ColorScheme.fromSeed(
        seedColor: forest,
        primary: forest,
        onPrimary: NaturePaperColors.cream,
        secondary: NaturePaperColors.sage,
        onSecondary: const Color(0xFF1F2A1E),
        tertiary: NaturePaperColors.softOrange,
        error: NaturePaperColors.dayDanger,
        surface: card,
        onSurface: ink,
        surfaceContainerLowest: NaturePaperColors.cream,
        surfaceContainerLow: NaturePaperColors.paperBeige,
        surfaceContainer: card,
        surfaceContainerHigh: const Color(0xFFF4ECD9),
        surfaceContainerHighest: NaturePaperColors.paperBeige,
        outline: border,
      ),
    );
```

```
    final textTheme = buildJakartaTextTheme(
      base: base.textTheme,
      primary: ink,
      secondary: ink.withValues(alpha: 0.78),
      tertiary: ink.withValues(alpha: 0.56),
    );

    return base.copyWith(
      scaffoldBackgroundColor: Colors.transparent,
      canvasColor: NaturePaperColors.cream,
      pageTransitionsTheme: _fadeTransitions,
      textTheme: textTheme,
      dividerColor: NaturePaperColors.warmBrown.withValues(alpha: 0.14),
      extensions: const [AppStyleTokens.natureLight],
      appBarTheme: AppBarTheme(
        elevation: 0,
        centerTitle: false,
        backgroundColor: Colors.transparent,
        foregroundColor: ink,
        surfaceTintColor: Colors.transparent,
        titleTextStyle: textTheme.titleMedium?.copyWith(
          fontSize: 18,
          fontWeight: FontWeight.w700,
          color: ink,
        ),
      ),
      cardTheme: CardThemeData(
        elevation: 0,
        color: card,
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(24),
          side: BorderSide(color: border),
        ),
        margin: const EdgeInsets.symmetric(vertical: 8),
      ),
      dialogTheme: DialogThemeData(
        backgroundColor: card,
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
        titleTextStyle: textTheme.titleMedium?.copyWith(
          fontSize: 18,
          fontWeight: FontWeight.w700,
          color: ink,
        ),
        contentTextStyle: textTheme.bodyMedium?.copyWith(
          color: ink.withValues(alpha: 0.8),
        ),
      ),
      bottomSheetTheme: const BottomSheetThemeData(
        backgroundColor: card,
        surfaceTintColor: Colors.transparent,
        modalBackgroundColor: card,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
        ),
      ),
      snackBarTheme: SnackBarThemeData(
        backgroundColor: const Color(0xFF4B3D2C),
        contentTextStyle: textTheme.bodyMedium?.copyWith(
          color: NaturePaperColors.cream,
        ),
        actionTextColor: NaturePaperColors.sunGlow,
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: card,
        hintStyle: textTheme.bodyMedium?.copyWith(
          color: ink.withValues(alpha: 0.45),
        ),
        labelStyle: TextStyle(color: forest.withValues(alpha: 0.9)),
```

```
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: BorderSide(color: border),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: BorderSide(color: border),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: forest, width: 2),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: _paperButtonStyle(
      background: forest,
      foreground: NaturePaperColors.cream,
      shadow: NaturePaperColors.warmBrown.withValues(alpha: 0.45),
      textStyle: textTheme.labelLarge,
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: _paperButtonStyle(
      background: forest,
      foreground: NaturePaperColors.cream,
      shadow: NaturePaperColors.warmBrown.withValues(alpha: 0.45),
      textStyle: textTheme.labelLarge,
    ),
  ),
  outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
      foregroundColor: forest,
      side: BorderSide(
        color: NaturePaperColors.sage.withValues(alpha: 0.65),
        width: 1.4,
      ),
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(18),
      ),
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w700,
        fontSize: 15,
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: forest,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: forest,
    foregroundColor: NaturePaperColors.cream,
    elevation: 6,
    focusElevation: 8,
    hoverElevation: 8,
    highlightElevation: 10,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
  chipTheme: ChipThemeData(
    backgroundColor: NaturePaperColors.sage.withValues(alpha: 0.16),
```

```
      selectedColor: NaturePaperColors.sage.withValues(alpha: 0.34),
      labelStyle: textTheme.labelLarge!.copyWith(
        color: NaturePaperColors.deepForest,
        fontWeight: FontWeight.w700,
      ),
      side: BorderSide(color: NaturePaperColors.sage.withValues(alpha: 0.4)),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ),
    navigationBarTheme: NavigationBarThemeData(
      backgroundColor: card.withValues(alpha: 0.9),
      indicatorColor: NaturePaperColors.sage.withValues(alpha: 0.28),
      elevation: 0,
      labelTextStyle: WidgetStateProperty.resolveWith((states) {
        final selected = states.contains(WidgetState.selected);
        return textTheme.labelSmall!.copyWith(
          fontSize: 11,
          fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
          color: selected ? forest : ink.withValues(alpha: 0.5),
        );
      }),
      iconTheme: WidgetStateProperty.resolveWith((states) {
        return IconThemeData(
          size: 24,
          color: states.contains(WidgetState.selected)
              ? forest
              : ink.withValues(alpha: 0.5),
        );
      }),
    ),
    navigationRailTheme: NavigationRailThemeData(
      backgroundColor: NaturePaperColors.cream.withValues(alpha: 0.86),
      indicatorColor: NaturePaperColors.sage.withValues(alpha: 0.28),
      selectedIconTheme: const IconThemeData(color: forest),
      unselectedIconTheme: IconThemeData(color: ink.withValues(alpha: 0.5)),
    ),
    progressIndicatorTheme: ProgressIndicatorThemeData(
      color: forest,
      circularTrackColor: NaturePaperColors.paperBeige.withValues(alpha: 0.7),
    ),
  );
}

static ThemeData dark() {
  const ink = NaturePaperColors.nightInk;
  const inkSecondary = NaturePaperColors.nightTextSecondary;
  const sage = NaturePaperColors.sageNight;
  const amber = NaturePaperColors.amber;
  const card = NaturePaperColors.nightCard;
  const cardHigh = NaturePaperColors.nightCardHigh;
  const border = NaturePaperColors.nightBorder;

  final colorScheme = ColorScheme(
    brightness: Brightness.dark,
    primary: sage,
    onPrimary: const Color(0xFF0F1A12),
    primaryContainer: const Color(0xFF32463A),
    onPrimaryContainer: NaturePaperColors.moonBeige,
    secondary: NaturePaperColors.moonBeige,
    onSecondary: const Color(0xFF1B1A12),
    secondaryContainer: const Color(0xFF3B4438),
    onSecondaryContainer: NaturePaperColors.moonBeige,
    tertiary: amber,
    onTertiary: const Color(0xFF221708),
    error: NaturePaperColors.dustyRose,
    onError: const Color(0xFF200F0F),
    surface: card,
    onSurface: ink,
    onSurfaceVariant: inkSecondary,
    surfaceContainerLowest: NaturePaperColors.midnight,
    surfaceContainerLow: NaturePaperColors.forestNight,
    surfaceContainer: card,
    surfaceContainerHigh: cardHigh,
    surfaceContainerHighest: NaturePaperColors.nightCardHighest,
```

```
    outline: border,
    outlineVariant: const Color(0xFF35413B),
    shadow: Colors.black,
    scrim: Colors.black,
    inverseSurface: NaturePaperColors.moonBeige,
    onInverseSurface: const Color(0xFF23201A),
    inversePrimary: NaturePaperColors.forest,
  );

  final base = ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: colorScheme,
  );

  final textTheme = buildJakartaTextTheme(
    base: base.textTheme,
    primary: ink,
    secondary: inkSecondary,
    tertiary: NaturePaperColors.nightTextTertiary,
  );

  return base.copyWith(
    scaffoldBackgroundColor: Colors.transparent,
    canvasColor: NaturePaperColors.midnight,
    pageTransitionsTheme: _fadeTransitions,
    dividerColor: border,
    disabledColor: NaturePaperColors.nightTextTertiary,
    textTheme: textTheme,
    iconTheme: const IconThemeData(color: inkSecondary),
    primaryIconTheme: const IconThemeData(color: amber),
    extensions: const [AppStyleTokens.natureDark],
    dividerTheme: const DividerThemeData(
      color: border,
      thickness: 1,
      space: 1,
    ),
    listTileTheme: ListTileThemeData(
      iconColor: inkSecondary,
      textColor: ink,
      subtitleTextStyle: textTheme.bodySmall?.copyWith(
        color: inkSecondary,
        fontSize: 13,
      ),
    ),
    chipTheme: ChipThemeData(
      backgroundColor: sage.withValues(alpha: 0.18),
      disabledColor: card,
      selectedColor: sage.withValues(alpha: 0.36),
      secondarySelectedColor: sage.withValues(alpha: 0.36),
      labelStyle: textTheme.labelLarge!.copyWith(
        color: NaturePaperColors.moonBeige,
        fontWeight: FontWeight.w700,
      ),
      secondaryLabelStyle: textTheme.labelLarge!.copyWith(
        color: NaturePaperColors.moonBeige,
        fontWeight: FontWeight.w700,
      ),
      side: BorderSide(color: sage.withValues(alpha: 0.45)),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ),
    tabBarTheme: TabBarThemeData(
      labelColor: amber,
      unselectedLabelColor: inkSecondary,
      indicatorColor: amber,
      dividerColor: border,
      labelStyle: textTheme.labelLarge?.copyWith(fontWeight: FontWeight.w700),
      unselectedLabelStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w500,
      ),
    ),
    appBarTheme: AppBarTheme(
      elevation: 0,
```

```
          centerTitle: false,
          backgroundColor: Colors.transparent,
          foregroundColor: ink,
          surfaceTintColor: Colors.transparent,
          iconTheme: const IconThemeData(color: ink),
          actionsIconTheme: const IconThemeData(color: inkSecondary),
          titleTextStyle: textTheme.titleMedium?.copyWith(
            fontSize: 18,
            fontWeight: FontWeight.w700,
            color: ink,
          ),
        ),
        cardTheme: CardThemeData(
          elevation: 0,
          color: card,
          surfaceTintColor: Colors.transparent,
          shadowColor: Colors.black.withValues(alpha: 0.5),
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(24),
            side: const BorderSide(color: border, width: 1),
          ),
          margin: const EdgeInsets.symmetric(vertical: 8),
        ),
        dialogTheme: DialogThemeData(
          backgroundColor: cardHigh,
          surfaceTintColor: Colors.transparent,
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),
          titleTextStyle: textTheme.titleMedium?.copyWith(
            fontSize: 18,
            fontWeight: FontWeight.w700,
            color: ink,
          ),
          contentTextStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),
        ),
        bottomSheetTheme: const BottomSheetThemeData(
          backgroundColor: cardHigh,
          surfaceTintColor: Colors.transparent,
          modalBackgroundColor: cardHigh,
```

**lib/core/theme/nature_paper_theme.dart — continued (2/2)**

```
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
          ),
        ),
        snackBarTheme: SnackBarThemeData(
          backgroundColor: NaturePaperColors.nightCardHighest,
          contentTextStyle: textTheme.bodyMedium?.copyWith(color: ink),
          actionTextColor: amber,
          behavior: SnackBarBehavior.floating,
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
        ),
        popupMenuTheme: PopupMenuThemeData(
          color: NaturePaperColors.nightCardHighest,
          surfaceTintColor: Colors.transparent,
          textStyle: textTheme.bodyMedium?.copyWith(color: ink),
        ),
        dropdownMenuTheme: DropdownMenuThemeData(
          textStyle: textTheme.bodyMedium?.copyWith(color: ink),
          inputDecorationTheme: const InputDecorationTheme(
            filled: true,
            fillColor: cardHigh,
          ),
        ),
        inputDecorationTheme: InputDecorationTheme(
          filled: true,
          fillColor: cardHigh,
          hintStyle: textTheme.bodyMedium?.copyWith(
            color: NaturePaperColors.nightTextTertiary,
          ),
          labelStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),
          floatingLabelStyle: textTheme.bodyMedium?.copyWith(color: amber),
          prefixIconColor: inkSecondary,
```

```
    suffixIconColor: inkSecondary,
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: border),
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: border),
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(16),
      borderSide: const BorderSide(color: amber, width: 2),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: _paperButtonStyle(
      background: sage,
      foreground: const Color(0xFF0F1A12),
      shadow: Colors.black.withValues(alpha: 0.6),
      textStyle: textTheme.labelLarge,
      disabledBackground: cardHigh,
      disabledForeground: NaturePaperColors.nightTextTertiary,
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: _paperButtonStyle(
      background: sage,
      foreground: const Color(0xFF0F1A12),
      shadow: Colors.black.withValues(alpha: 0.6),
      textStyle: textTheme.labelLarge,
      disabledBackground: cardHigh,
      disabledForeground: NaturePaperColors.nightTextTertiary,
    ),
  ),
  outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
      foregroundColor: NaturePaperColors.moonBeige,
      side: BorderSide(
        color: NaturePaperColors.moonBeige.withValues(alpha: 0.45),
        width: 1.4,
      ),
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(18),
      ),
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w700,
        fontSize: 15,
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: amber,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: sage,
    foregroundColor: const Color(0xFF0F1A12),
    elevation: 6,
    focusElevation: 8,
    hoverElevation: 8,
    highlightElevation: 10,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
```

```
          fontSize: 15,
        ),
      ),
      navigationBarTheme: NavigationBarThemeData(
        backgroundColor: cardHigh,
        indicatorColor: amber.withValues(alpha: 0.26),
        elevation: 0,
        surfaceTintColor: Colors.transparent,
        labelTextStyle: WidgetStateProperty.resolveWith((states) {
          final selected = states.contains(WidgetState.selected);
          return textTheme.labelSmall!.copyWith(
            fontSize: 11,
            fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
            color: selected ? amber : inkSecondary,
          );
        }),
        iconTheme: WidgetStateProperty.resolveWith((states) {
          final selected = states.contains(WidgetState.selected);
          return IconThemeData(
            size: 24,
            color: selected ? amber : inkSecondary,
          );
        }),
      ),
      navigationRailTheme: NavigationRailThemeData(
        backgroundColor: NaturePaperColors.forestNight.withValues(alpha: 0.88),
        indicatorColor: amber.withValues(alpha: 0.26),
        selectedIconTheme: const IconThemeData(color: amber),
        unselectedIconTheme: const IconThemeData(color: inkSecondary),
        selectedLabelTextStyle: textTheme.labelLarge?.copyWith(
          color: amber,
          fontWeight: FontWeight.w700,
        ),
        unselectedLabelTextStyle: textTheme.labelLarge?.copyWith(
          color: inkSecondary,
          fontWeight: FontWeight.w500,
        ),
      ),
      switchTheme: SwitchThemeData(
        thumbColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) return amber;
          return inkSecondary;
        }),
        trackColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return amber.withValues(alpha: 0.42);
          }
          return cardHigh;
        }),
      ),
      progressIndicatorTheme: const ProgressIndicatorThemeData(
        color: amber,
        circularTrackColor: cardHigh,
      ),
    );
  }

  static ButtonStyle _paperButtonStyle({
    required Color background,
    required Color foreground,
    required Color shadow,
    TextStyle? textStyle,
    Color? disabledBackground,
    Color? disabledForeground,
  }) {
    return ButtonStyle(
      backgroundColor: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.disabled)) {
          return disabledBackground ?? background.withValues(alpha: 0.4);
        }
        return background;
      }),
      foregroundColor: WidgetStateProperty.resolveWith((states) {
```

```
        if (states.contains(WidgetState.disabled)) {
          return disabledForeground ?? foreground.withValues(alpha: 0.5);
        }
        return foreground;
      }),
      shadowColor: WidgetStatePropertyAll(shadow),
      elevation: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.disabled)) return 0;
        if (states.contains(WidgetState.pressed)) return 7;
        if (states.contains(WidgetState.hovered)) return 5;
        return 3;
      }),
      padding: const WidgetStatePropertyAll(
        EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      ),
      shape: WidgetStatePropertyAll(
        RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
      ),
      textStyle: WidgetStatePropertyAll(
        textStyle?.copyWith(fontWeight: FontWeight.w700, fontSize: 15),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/nature_paper_theme.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/theme/neumorphic.dart

Four visual styles × light/dark Material 3 themes. This file (neumorphic) is one concrete piece of that layer.

Line count: 403. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/neumorphic.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

enum NeuElevation { flat, raised, floating, pressed }

class Neumorphic {
  const Neumorphic._();

  static bool isActive(BuildContext context) =>
      AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;

  static Color surface(
    BuildContext context, {
    NeuElevation elevation = NeuElevation.raised,
  }) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return switch (elevation) {
      NeuElevation.pressed =>
        isDark
            ? SoftNeumorphismColors.nightInset
            : SoftNeumorphismColors.morningInset,
      NeuElevation.floating =>
        isDark
            ? SoftNeumorphismColors.nightSurfaceHigh
            : SoftNeumorphismColors.morningSurfaceHigh,
      NeuElevation.flat || NeuElevation.raised =>
        isDark
            ? SoftNeumorphismColors.nightSurface
            : SoftNeumorphismColors.morningSurface,
    };
  }

  static Color shadowColor(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return isDark
        ? SoftNeumorphismColors.nightShadow
        : SoftNeumorphismColors.morningShadow;
  }
```

```
static Color highlightColor(BuildContext context) {
  final isDark = Theme.of(context).brightness == Brightness.dark;
  return isDark
      ? SoftNeumorphismColors.nightHighlight
      : SoftNeumorphismColors.morningHighlight;
}

static List<BoxShadow> shadows(
  BuildContext context, {
  NeuElevation elevation = NeuElevation.raised,
  double intensity = 1,
}) {
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final dark = shadowColor(context);
  final light = highlightColor(context);

  switch (elevation) {
    case NeuElevation.flat:
      return const [];
    case NeuElevation.pressed:
      return [
        BoxShadow(
          color: dark.withValues(alpha: (isDark ? 0.85 : 0.45) * intensity),
          offset: const Offset(4, 4),
          blurRadius: 8,
          spreadRadius: 1,
        ),
        BoxShadow(
          color: light.withValues(alpha: (isDark ? 0.55 : 0.90) * intensity),
          offset: const Offset(-3, -3),
          blurRadius: 6,
          spreadRadius: 0,
        ),
      ];
    case NeuElevation.raised:
      return [
        BoxShadow(
          color: dark.withValues(alpha: (isDark ? 0.75 : 0.38) * intensity),
          offset: const Offset(6, 6),
          blurRadius: 14,
        ),
        BoxShadow(
          color: light.withValues(alpha: (isDark ? 0.50 : 0.95) * intensity),
          offset: const Offset(-5, -5),
          blurRadius: 12,
        ),
      ];
    case NeuElevation.floating:
      return [
        BoxShadow(
          color: dark.withValues(alpha: (isDark ? 0.82 : 0.42) * intensity),
          offset: const Offset(10, 12),
          blurRadius: 24,
        ),
        BoxShadow(
          color: light.withValues(alpha: (isDark ? 0.55 : 1.0) * intensity),
          offset: const Offset(-6, -6),
          blurRadius: 16,
        ),
      ];
  }
}

static BoxDecoration decoration(
  BuildContext context, {
  NeuElevation elevation = NeuElevation.raised,
  BorderRadius? borderRadius,
  Color? color,
  Gradient? gradient,
  double intensity = 1,
}) {
  final radius =
      borderRadius ??
```

```
        BorderRadius.circular(
          AppStyleTokens.maybeOf(context)?.cardRadius ?? 18,
        );
    return BoxDecoration(
      color: gradient == null
          ? (color ?? surface(context, elevation: elevation))
          : null,
      gradient: gradient,
      borderRadius: radius,
      boxShadow: shadows(context, elevation: elevation, intensity: intensity),
    );
  }
}

class NeuIconWell extends StatelessWidget {
  const NeuIconWell({
    super.key,
    required this.icon,
    this.color,
    this.size = 22,
    this.padding = 12,
    this.elevation = NeuElevation.pressed,
    this.child,
  });

  final IconData icon;
  final Color? color;
  final double size;
  final double padding;

  final NeuElevation elevation;

  final Widget? child;

  @override
  Widget build(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    final tone = color ?? AppColors.accent(context);
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final iconWidget = child ?? PaperGlyph(icon: icon, size: size, color: tone);

    if (tokens?.isNaturePaper ?? false) {
      final side = size + padding * 2;
      return Container(
        width: side,
        height: side,
        alignment: Alignment.center,
        decoration: BoxDecoration(
          color: isDark
              ? NaturePaperColors.nightCardHighest
              : NaturePaperColors.paperBeige.withValues(alpha: 0.72),
          borderRadius: BorderRadius.circular(16),
          border: Border.all(
            color: isDark
                ? NaturePaperColors.moonGlow.withValues(alpha: 0.28)
                : NaturePaperColors.warmBrown.withValues(alpha: 0.28),
            width: 1.2,
          ),
          boxShadow: [
            BoxShadow(
              color: (isDark ? Colors.black : NaturePaperColors.warmBrown)
                  .withValues(alpha: isDark ? 0.35 : 0.12),
              blurRadius: 10,
              offset: const Offset(0, 3),
            ),
            if (!isDark)
              const BoxShadow(
                color: NaturePaperColors.dayCardEdge,
                offset: Offset(0, 2),
              ),
          ],
        ),
        child: iconWidget,
```

```
      );
    }

    if (tokens?.isSoftNeumorphism ?? false) {
      final side = size + padding * 2;
      return Container(
        width: side,
        height: side,
        alignment: Alignment.center,
        decoration: Neumorphic.decoration(
          context,
          elevation: elevation,
          borderRadius: BorderRadius.circular(14),
          intensity: 1.05,
        ),
        child: child ?? Icon(icon, size: size, color: tone),
      );
    }

    if (tokens?.isMonochromeWorkspace ?? false) {
      return Container(
        padding: EdgeInsets.all(padding),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(4),
          border: Border.all(color: tone.withValues(alpha: 0.18)),
          color: tone.withValues(alpha: 0.08),
        ),
        child: child ?? Icon(icon, size: size, color: tone),
      );
    }

    return Container(
      padding: EdgeInsets.all(padding),
      decoration: BoxDecoration(
        color: tone.withValues(alpha: 0.10),
        borderRadius: BorderRadius.circular(12),
      ),
      child: child ?? Icon(icon, size: size, color: tone),
    );
  }
}

class NeuAvatar extends StatelessWidget {
  const NeuAvatar({
    super.key,
    required this.child,
    this.radius = 26,
    this.elevation = NeuElevation.raised,
  });

  final Widget child;
  final double radius;
  final NeuElevation elevation;

  @override
  Widget build(BuildContext context) {
    if (!Neumorphic.isActive(context)) {
      return CircleAvatar(
        radius: radius,
        backgroundColor: AppColors.accent(context).withValues(alpha: 0.18),
        child: child,
      );
    }
    final side = radius * 2;
    return Container(
      width: side,
      height: side,
      alignment: Alignment.center,
      decoration: Neumorphic.decoration(
        context,
        elevation: elevation,
        borderRadius: BorderRadius.circular(radius),
        intensity: 1.1,
```

```
      ),
      child: child,
    );
  }
}

class NeuSearchField extends StatelessWidget {
  const NeuSearchField({
    super.key,
    required this.controller,
    required this.hintText,
    this.onChanged,
  });

  final TextEditingController controller;
  final String hintText;
  final ValueChanged<String>? onChanged;

  @override
  Widget build(BuildContext context) {
    final isNeu = Neumorphic.isActive(context);
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    final field = TextField(
      controller: controller,
      onChanged: onChanged,
      style: Theme.of(context).textTheme.bodyLarge,
      decoration: InputDecoration(
        hintText: hintText,
        prefixIcon: Icon(
          Icons.search_rounded,
          color: AppColors.inkAlpha(context, 0.45),
        ),
        filled: !isNeu,
        fillColor: isNeu
            ? Colors.transparent
            : (isDark
                  ? scheme.surfaceContainerHighest
                  : AppColors.controlSurface(context)),
        border: isNeu ? InputBorder.none : null,
        enabledBorder: isNeu ? InputBorder.none : null,
        focusedBorder: isNeu ? InputBorder.none : null,
        contentPadding: const EdgeInsets.symmetric(
          horizontal: 12,
          vertical: 14,
        ),
      ),
    );

    if (!isNeu) return field;

    return Container(
      decoration: Neumorphic.decoration(
        context,
        elevation: NeuElevation.pressed,
        borderRadius: BorderRadius.circular(16),
        intensity: 1.05,
      ),
      child: field,
    );
  }
}

class NeuPressable extends StatefulWidget {
  const NeuPressable({
    super.key,
    required this.child,
    this.onTap,
    this.borderRadius,
    this.padding,
    this.color,
    this.elevation = NeuElevation.raised,
```

```
    this.enabled = true,
  });

  final Widget child;
  final VoidCallback? onTap;
  final BorderRadius? borderRadius;
  final EdgeInsetsGeometry? padding;
  final Color? color;
  final NeuElevation elevation;
  final bool enabled;

  @override
  State<NeuPressable> createState() => _NeuPressableState();
}

class _NeuPressableState extends State<NeuPressable> {
  bool _pressed = false;

  @override
  Widget build(BuildContext context) {
    final neu = Neumorphic.isActive(context);
    final radius =
        widget.borderRadius ??
        BorderRadius.circular(
          AppStyleTokens.maybeOf(context)?.cardRadius ?? 18,
        );
    final elevation = (!widget.enabled || widget.onTap == null)
        ? NeuElevation.flat
        : (_pressed ? NeuElevation.pressed : widget.elevation);

    final body = AnimatedContainer(
      duration: const Duration(milliseconds: 140),
      curve: Curves.easeOutCubic,
      padding: widget.padding,
      transform: Matrix4.diagonal3Values(
        _pressed && neu ? 0.98 : 1,
        _pressed && neu ? 0.98 : 1,
        1,
      ),
      transformAlignment: Alignment.center,
      decoration: neu
          ? Neumorphic.decoration(
              context,
              elevation: elevation,
              borderRadius: radius,
              color: widget.color,
            )
          : BoxDecoration(
              color:
                  widget.color ?? AppStyleTokens.maybeOf(context)?.cardSurface,
              borderRadius: radius,
            ),
      child: widget.child,
    );

    if (widget.onTap == null || !widget.enabled) return body;

    return GestureDetector(
      onTapDown: (_) => setState(() => _pressed = true),
      onTapUp: (_) {
        setState(() => _pressed = false);
        widget.onTap?.call();
      },
      onTapCancel: () => setState(() => _pressed = false),
      child: body,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/neumorphic.dart?

2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/theme/soft_neumorphism_colors.dart

Four visual styles × light/dark Material 3 themes. This file (soft neumorphism colors) is one concrete piece of that layer.

Line count: 55. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/theme/soft_neumorphism_colors.dart**

```
import 'package:flutter/material.dart';

class SoftNeumorphismColors {
  static const morningBg = Color(0xFFE4E2DC);

  static const morningBgCool = Color(0xFFDEE1E4);

  static const morningSurface = Color(0xFFEAE8E2);

  static const morningSurfaceHigh = Color(0xFFF1EFE9);

  static const morningInset = Color(0xFFD6D4CE);

  static const sage = Color(0xFF6F9279);

  static const sageDeep = Color(0xFF54705C);

  static const amber = Color(0xFFC9965A);

  static const morningInk = Color(0xFF242422);

  static const morningTextSecondary = Color(0xFF5E5E5A);
  static const morningTextTertiary = Color(0xFF858580);

  static const morningShadow = Color(0xFF9A978F);

  static const morningHighlight = Color(0xFFFFFFFF);

  static const morningDanger = Color(0xFFB07070);

  static const nightBg = Color(0xFF1C1F24);

  static const nightBgCool = Color(0xFF181C22);

  static const nightSurface = Color(0xFF23272D);

  static const nightSurfaceHigh = Color(0xFF2A2F36);

  static const nightInset = Color(0xFF14171B);

  static const emerald = Color(0xFF5A9A78);

  static const emeraldGlow = Color(0xFF7AB894);

  static const nightInk = Color(0xFFF2F0EB);

  static const nightTextSecondary = Color(0xFFA8A6A0);
  static const nightTextTertiary = Color(0xFF8A8882);
```

```
  static const nightShadow = Color(0xFF050608);

  static const nightHighlight = Color(0xFF3E4550);

  static const nightDanger = Color(0xFFC08080);
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/soft_neumorphism_colors.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/core/theme/soft_neumorphism_theme.dart

Four visual styles × light/dark Material 3 themes. This file (soft neumorphism theme) is one concrete piece of that layer.

Line count: 593. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/app_text_theme.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/theme/soft_neumorphism_theme.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/app_text_theme.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

class SoftNeumorphismTheme {
  static const _fadeTransitions = PageTransitionsTheme(
    builders: {
      TargetPlatform.android: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.iOS: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.macOS: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.linux: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.windows: FadeUpwardsPageTransitionsBuilder(),
      TargetPlatform.fuchsia: FadeUpwardsPageTransitionsBuilder(),
    },
  );

  static const _radius = 18.0;
  static const _controlRadius = 14.0;

  static ThemeData light() {
    const ink = SoftNeumorphismColors.morningInk;
    const sage = SoftNeumorphismColors.sage;
    const surface = SoftNeumorphismColors.morningSurface;
    const bg = SoftNeumorphismColors.morningBg;

    final base = ThemeData(
      useMaterial3: true,
      brightness: Brightness.light,
      colorScheme: ColorScheme.fromSeed(
        seedColor: sage,
        primary: sage,
        onPrimary: SoftNeumorphismColors.morningSurfaceHigh,
        secondary: SoftNeumorphismColors.amber,
        onSecondary: const Color(0xFF2A2218),
        tertiary: SoftNeumorphismColors.sageDeep,
        error: SoftNeumorphismColors.morningDanger,
        surface: surface,
        onSurface: ink,
        surfaceContainerLowest: bg,
        surfaceContainerLow: SoftNeumorphismColors.morningInset,
        surfaceContainer: surface,
        surfaceContainerHigh: SoftNeumorphismColors.morningSurfaceHigh,
        surfaceContainerHighest: SoftNeumorphismColors.morningSurfaceHigh,
```

```
      outline: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.35),
    ),
  );

  final textTheme = buildJakartaTextTheme(
    base: base.textTheme,
    primary: ink,
    secondary: SoftNeumorphismColors.morningTextSecondary,
    tertiary: SoftNeumorphismColors.morningTextTertiary,
  );

  return base.copyWith(
    scaffoldBackgroundColor: Colors.transparent,
    canvasColor: bg,
    pageTransitionsTheme: _fadeTransitions,
    textTheme: textTheme,
    dividerColor: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.22),
    extensions: const [AppStyleTokens.softNeuLight],
    appBarTheme: AppBarTheme(
      elevation: 0,
      centerTitle: false,
      backgroundColor: Colors.transparent,
      foregroundColor: ink,
      surfaceTintColor: Colors.transparent,
      titleTextStyle: textTheme.titleMedium?.copyWith(
        fontSize: 18,
        fontWeight: FontWeight.w700,
        color: ink,
      ),
    ),
    cardTheme: CardThemeData(
      elevation: 0,
      color: surface,
      surfaceTintColor: Colors.transparent,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_radius),
      ),
      margin: const EdgeInsets.symmetric(vertical: 8),
    ),
    dialogTheme: DialogThemeData(
      backgroundColor: SoftNeumorphismColors.morningSurfaceHigh,
      surfaceTintColor: Colors.transparent,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_radius),
      ),
      titleTextStyle: textTheme.titleMedium?.copyWith(
        fontSize: 18,
        fontWeight: FontWeight.w700,
        color: ink,
      ),
      contentTextStyle: textTheme.bodyMedium?.copyWith(
        color: SoftNeumorphismColors.morningTextSecondary,
      ),
    ),
    bottomSheetTheme: const BottomSheetThemeData(
      backgroundColor: SoftNeumorphismColors.morningSurfaceHigh,
      surfaceTintColor: Colors.transparent,
      modalBackgroundColor: SoftNeumorphismColors.morningSurfaceHigh,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
      ),
    ),
    snackBarTheme: SnackBarThemeData(
      backgroundColor: SoftNeumorphismColors.sageDeep,
      contentTextStyle: textTheme.bodyMedium?.copyWith(
        color: SoftNeumorphismColors.morningSurfaceHigh,
      ),
      actionTextColor: SoftNeumorphismColors.amber,
      behavior: SnackBarBehavior.floating,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
```

```
    fillColor: SoftNeumorphismColors.morningInset,
    hintStyle: textTheme.bodyMedium?.copyWith(
      color: SoftNeumorphismColors.morningTextTertiary,
    ),
    labelStyle: TextStyle(
      color: SoftNeumorphismColors.morningTextSecondary,
    ),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: BorderSide.none,
    ),
    enabledBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: BorderSide.none,
    ),
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: sage, width: 1.5),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: _neuButtonStyle(
      background: sage,
      foreground: SoftNeumorphismColors.morningSurfaceHigh,
      shadow: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.35),
      textStyle: textTheme.labelLarge,
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: _neuButtonStyle(
      background: sage,
      foreground: SoftNeumorphismColors.morningSurfaceHigh,
      shadow: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.35),
      textStyle: textTheme.labelLarge,
    ),
  ),
  outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
      foregroundColor: SoftNeumorphismColors.sageDeep,
      side: BorderSide(color: sage.withValues(alpha: 0.45), width: 1.2),
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_controlRadius),
      ),
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w700,
        fontSize: 15,
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: SoftNeumorphismColors.sageDeep,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: sage,
    foregroundColor: SoftNeumorphismColors.morningSurfaceHigh,
    elevation: 4,
    focusElevation: 6,
    hoverElevation: 6,
    highlightElevation: 2,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
```

```
        ),
      ),
      chipTheme: ChipThemeData(
        backgroundColor: SoftNeumorphismColors.morningInset,
        selectedColor: sage.withValues(alpha: 0.28),
        labelStyle: textTheme.labelLarge!.copyWith(
          color: SoftNeumorphismColors.sageDeep,
          fontWeight: FontWeight.w700,
        ),
        side: BorderSide.none,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      ),
      switchTheme: SwitchThemeData(
        thumbColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return SoftNeumorphismColors.morningSurfaceHigh;
          }
          return SoftNeumorphismColors.morningSurfaceHigh;
        }),
        trackColor: WidgetStateProperty.resolveWith((states) {
          if (states.contains(WidgetState.selected)) {
            return sage;
          }
          return SoftNeumorphismColors.morningInset;
        }),
        trackOutlineColor: const WidgetStatePropertyAll(Colors.transparent),
      ),
      progressIndicatorTheme: ProgressIndicatorThemeData(
        color: sage,
        circularTrackColor: SoftNeumorphismColors.morningInset,
        linearTrackColor: SoftNeumorphismColors.morningInset,
      ),
      navigationBarTheme: NavigationBarThemeData(
        backgroundColor: SoftNeumorphismColors.morningSurfaceHigh,
        indicatorColor: sage.withValues(alpha: 0.22),
        elevation: 0,
        labelTextStyle: WidgetStateProperty.resolveWith((states) {
          final selected = states.contains(WidgetState.selected);
          return textTheme.labelSmall!.copyWith(
            fontSize: 11,
            fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
            color: selected
                ? SoftNeumorphismColors.sageDeep
                : SoftNeumorphismColors.morningTextTertiary,
          );
        }),
        iconTheme: WidgetStateProperty.resolveWith((states) {
          return IconThemeData(
            size: 24,
            color: states.contains(WidgetState.selected)
                ? SoftNeumorphismColors.sageDeep
                : SoftNeumorphismColors.morningTextTertiary,
          );
        }),
      ),
      navigationRailTheme: NavigationRailThemeData(
        backgroundColor: SoftNeumorphismColors.morningSurface.withValues(
          alpha: 0.92,
        ),
        indicatorColor: sage.withValues(alpha: 0.22),
        selectedIconTheme: const IconThemeData(
          color: SoftNeumorphismColors.sageDeep,
        ),
        unselectedIconTheme: const IconThemeData(
          color: SoftNeumorphismColors.morningTextTertiary,
        ),
      ),
    );
  }

  static ThemeData dark() {
    const ink = SoftNeumorphismColors.nightInk;
    const inkSecondary = SoftNeumorphismColors.nightTextSecondary;
```

```
const emerald = SoftNeumorphismColors.emerald;
const amber = SoftNeumorphismColors.amber;
const surface = SoftNeumorphismColors.nightSurface;
const surfaceHigh = SoftNeumorphismColors.nightSurfaceHigh;

final colorScheme = ColorScheme(
  brightness: Brightness.dark,
  primary: emerald,
  onPrimary: const Color(0xFF0E1812),
  primaryContainer: const Color(0xFF2A3F34),
  onPrimaryContainer: SoftNeumorphismColors.nightInk,
  secondary: amber,
  onSecondary: const Color(0xFF1A140C),
  secondaryContainer: const Color(0xFF3A3228),
  onSecondaryContainer: SoftNeumorphismColors.nightInk,
  tertiary: SoftNeumorphismColors.emeraldGlow,
  onTertiary: const Color(0xFF0E1812),
  error: SoftNeumorphismColors.nightDanger,
  onError: const Color(0xFF1A0C0C),
  surface: surface,
  onSurface: ink,
  onSurfaceVariant: inkSecondary,
  surfaceContainerLowest: SoftNeumorphismColors.nightBg,
  surfaceContainerLow: SoftNeumorphismColors.nightInset,
  surfaceContainer: surface,
  surfaceContainerHigh: surfaceHigh,
  surfaceContainerHighest: const Color(0xFF323840),
  outline: SoftNeumorphismColors.nightHighlight,
  outlineVariant: const Color(0xFF2E343C),
  shadow: SoftNeumorphismColors.nightShadow,
  scrim: Colors.black,
  inverseSurface: SoftNeumorphismColors.morningSurface,
  onInverseSurface: SoftNeumorphismColors.morningInk,
  inversePrimary: SoftNeumorphismColors.sage,
);

final base = ThemeData(
  useMaterial3: true,
  brightness: Brightness.dark,
  colorScheme: colorScheme,
);

final textTheme = buildJakartaTextTheme(
  base: base.textTheme,
  primary: ink,
  secondary: inkSecondary,
  tertiary: SoftNeumorphismColors.nightTextTertiary,
);

return base.copyWith(
  scaffoldBackgroundColor: Colors.transparent,
  canvasColor: SoftNeumorphismColors.nightBg,
  pageTransitionsTheme: _fadeTransitions,
  dividerColor: SoftNeumorphismColors.nightHighlight.withValues(
    alpha: 0.45,
  ),
  disabledColor: SoftNeumorphismColors.nightTextTertiary,
  textTheme: textTheme,
  iconTheme: const IconThemeData(color: inkSecondary),
  primaryIconTheme: const IconThemeData(color: emerald),
  extensions: const [AppStyleTokens.softNeuDark],
  dividerTheme: DividerThemeData(
    color: SoftNeumorphismColors.nightHighlight.withValues(alpha: 0.45),
    thickness: 1,
    space: 1,
  ),
  listTileTheme: ListTileThemeData(
    iconColor: inkSecondary,
    textColor: ink,
    subtitleTextStyle: textTheme.bodySmall?.copyWith(
      color: inkSecondary,
      fontSize: 13,
    ),
```

```
      ),
      appBarTheme: AppBarTheme(
        elevation: 0,
        centerTitle: false,
        backgroundColor: Colors.transparent,
        foregroundColor: ink,
        surfaceTintColor: Colors.transparent,
        iconTheme: const IconThemeData(color: ink),
        actionsIconTheme: const IconThemeData(color: inkSecondary),
        titleTextStyle: textTheme.titleMedium?.copyWith(
          fontSize: 18,
          fontWeight: FontWeight.w700,
          color: ink,
        ),
      ),
      cardTheme: CardThemeData(
        elevation: 0,
        color: surface,
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(_radius),
        ),
        margin: const EdgeInsets.symmetric(vertical: 8),
      ),
      dialogTheme: DialogThemeData(
        backgroundColor: surfaceHigh,
        surfaceTintColor: Colors.transparent,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(_radius),
        ),
        titleTextStyle: textTheme.titleMedium?.copyWith(
          fontSize: 18,
```

**lib/core/theme/soft_neumorphism_theme.dart — continued (2/2)**

```
          fontWeight: FontWeight.w700,
          color: ink,
        ),
        contentTextStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),
      ),
      bottomSheetTheme: const BottomSheetThemeData(
        backgroundColor: SoftNeumorphismColors.nightSurfaceHigh,
        surfaceTintColor: Colors.transparent,
        modalBackgroundColor: SoftNeumorphismColors.nightSurfaceHigh,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
        ),
      ),
      snackBarTheme: SnackBarThemeData(
        backgroundColor: surfaceHigh,
        contentTextStyle: textTheme.bodyMedium?.copyWith(color: ink),
        actionTextColor: amber,
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
      ),
      inputDecorationTheme: InputDecorationTheme(
        filled: true,
        fillColor: SoftNeumorphismColors.nightInset,
        hintStyle: textTheme.bodyMedium?.copyWith(
          color: SoftNeumorphismColors.nightTextTertiary,
        ),
        labelStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),
        floatingLabelStyle: textTheme.bodyMedium?.copyWith(color: emerald),
        prefixIconColor: inkSecondary,
        suffixIconColor: inkSecondary,
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(_controlRadius),
          borderSide: BorderSide.none,
        ),
        enabledBorder: OutlineInputBorder(
          borderRadius: BorderRadius.circular(_controlRadius),
          borderSide: BorderSide.none,
        ),
```

```
    focusedBorder: OutlineInputBorder(
      borderRadius: BorderRadius.circular(_controlRadius),
      borderSide: const BorderSide(color: emerald, width: 1.5),
    ),
    contentPadding: const EdgeInsets.symmetric(
      horizontal: 16,
      vertical: 16,
    ),
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: _neuButtonStyle(
      background: emerald,
      foreground: const Color(0xFF0E1812),
      shadow: SoftNeumorphismColors.nightShadow.withValues(alpha: 0.55),
      textStyle: textTheme.labelLarge,
    ),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: _neuButtonStyle(
      background: emerald,
      foreground: const Color(0xFF0E1812),
      shadow: SoftNeumorphismColors.nightShadow.withValues(alpha: 0.55),
      textStyle: textTheme.labelLarge,
    ),
  ),
  outlinedButtonTheme: OutlinedButtonThemeData(
    style: OutlinedButton.styleFrom(
      foregroundColor: SoftNeumorphismColors.emeraldGlow,
      side: BorderSide(color: emerald.withValues(alpha: 0.55), width: 1.2),
      padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(_controlRadius),
      ),
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w700,
        fontSize: 15,
      ),
    ),
  ),
  textButtonTheme: TextButtonThemeData(
    style: TextButton.styleFrom(
      foregroundColor: SoftNeumorphismColors.emeraldGlow,
      textStyle: textTheme.labelLarge?.copyWith(
        fontWeight: FontWeight.w600,
      ),
    ),
  ),
  floatingActionButtonTheme: FloatingActionButtonThemeData(
    backgroundColor: emerald,
    foregroundColor: const Color(0xFF0E1812),
    elevation: 4,
    focusElevation: 6,
    hoverElevation: 6,
    highlightElevation: 2,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),
    extendedTextStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w700,
      fontSize: 15,
    ),
  ),
  chipTheme: ChipThemeData(
    backgroundColor: SoftNeumorphismColors.nightInset,
    selectedColor: emerald.withValues(alpha: 0.32),
    labelStyle: textTheme.labelLarge!.copyWith(
      color: SoftNeumorphismColors.nightInk,
      fontWeight: FontWeight.w700,
    ),
    side: BorderSide.none,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
  ),
  switchTheme: SwitchThemeData(
    thumbColor: const WidgetStatePropertyAll(
      SoftNeumorphismColors.nightInk,
```

```
    ),
    trackColor: WidgetStateProperty.resolveWith((states) {
      if (states.contains(WidgetState.selected)) {
        return emerald;
      }
      return SoftNeumorphismColors.nightInset;
    }),
    trackOutlineColor: const WidgetStatePropertyAll(Colors.transparent),
  ),
  progressIndicatorTheme: ProgressIndicatorThemeData(
    color: emerald,
    circularTrackColor: SoftNeumorphismColors.nightInset,
    linearTrackColor: SoftNeumorphismColors.nightInset,
  ),
  navigationBarTheme: NavigationBarThemeData(
    backgroundColor: surfaceHigh,
    indicatorColor: emerald.withValues(alpha: 0.28),
    elevation: 0,
    surfaceTintColor: Colors.transparent,
    labelTextStyle: WidgetStateProperty.resolveWith((states) {
      final selected = states.contains(WidgetState.selected);
      return textTheme.labelSmall!.copyWith(
        fontSize: 11,
        fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
        color: selected
            ? SoftNeumorphismColors.emeraldGlow
            : SoftNeumorphismColors.nightTextTertiary,
      );
    }),
    iconTheme: WidgetStateProperty.resolveWith((states) {
      final selected = states.contains(WidgetState.selected);
      return IconThemeData(
        size: 24,
        color: selected
            ? SoftNeumorphismColors.emeraldGlow
            : SoftNeumorphismColors.nightTextTertiary,
      );
    }),
  ),
  navigationRailTheme: NavigationRailThemeData(
    backgroundColor: SoftNeumorphismColors.nightSurface,
    indicatorColor: emerald.withValues(alpha: 0.28),
    selectedIconTheme: const IconThemeData(
      color: SoftNeumorphismColors.emeraldGlow,
    ),
    unselectedIconTheme: const IconThemeData(
      color: SoftNeumorphismColors.nightTextTertiary,
    ),
    selectedLabelTextStyle: textTheme.labelLarge?.copyWith(
      color: SoftNeumorphismColors.emeraldGlow,
      fontWeight: FontWeight.w700,
    ),
    unselectedLabelTextStyle: textTheme.labelLarge?.copyWith(
      color: SoftNeumorphismColors.nightTextTertiary,
      fontWeight: FontWeight.w500,
    ),
  ),
  tabBarTheme: TabBarThemeData(
    labelColor: SoftNeumorphismColors.emeraldGlow,
    unselectedLabelColor: inkSecondary,
    indicatorColor: emerald,
    dividerColor: SoftNeumorphismColors.nightHighlight.withValues(
      alpha: 0.4,
    ),
    labelStyle: textTheme.labelLarge?.copyWith(fontWeight: FontWeight.w700),
    unselectedLabelStyle: textTheme.labelLarge?.copyWith(
      fontWeight: FontWeight.w500,
    ),
  ),
  popupMenuTheme: PopupMenuThemeData(
    color: surfaceHigh,
    surfaceTintColor: Colors.transparent,
    textStyle: textTheme.bodyMedium?.copyWith(color: ink),
```

```
      ),
    );
  }

  static ButtonStyle _neuButtonStyle({
    required Color background,
    required Color foreground,
    required Color shadow,
    TextStyle? textStyle,
  }) {
    return ButtonStyle(
      backgroundColor: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.disabled)) {
          return background.withValues(alpha: 0.40);
        }
        if (states.contains(WidgetState.pressed)) {
          return Color.lerp(background, Colors.black, 0.08);
        }
        return background;
      }),
      foregroundColor: WidgetStatePropertyAll(foreground),
      elevation: WidgetStateProperty.resolveWith((states) {
        if (states.contains(WidgetState.pressed)) return 1;
        if (states.contains(WidgetState.hovered)) return 5;
        return 3;
      }),
      shadowColor: WidgetStatePropertyAll(shadow),
      padding: const WidgetStatePropertyAll(
        EdgeInsets.symmetric(horizontal: 24, vertical: 16),
      ),
      shape: WidgetStatePropertyAll(
        RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(_controlRadius),
        ),
      ),
      textStyle: WidgetStatePropertyAll(
        textStyle?.copyWith(fontWeight: FontWeight.w700, fontSize: 15),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/theme/soft_neumorphism_theme.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/core/utils/responsive.dart

Responsive breakpoints and voice playback. This file (responsive) is one concrete piece of that layer.

Line count: 29. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:responsive_framework/responsive_framework.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/utils/responsive.dart**

```
import 'package:flutter/material.dart';
import 'package:responsive_framework/responsive_framework.dart';

class Breakpoints {
  static const mobile = 0;
  static const tablet = 600;
  static const desktop = 1100;
}

extension ResponsiveX on BuildContext {
  bool get isMobile => MediaQuery.sizeOf(this).width < Breakpoints.tablet;
  bool get isTablet =>
      MediaQuery.sizeOf(this).width >= Breakpoints.tablet &&
      MediaQuery.sizeOf(this).width < Breakpoints.desktop;
  bool get useRail => MediaQuery.sizeOf(this).width >= Breakpoints.tablet;

  double get contentMaxWidth {
    final w = MediaQuery.sizeOf(this).width;
    if (w >= Breakpoints.desktop) return 960;
    if (w >= Breakpoints.tablet) return 840;
    return w;
  }
}

List<Breakpoint> responsiveBreakpoints() => [
  const Breakpoint(start: 0, end: 599, name: MOBILE),
  const Breakpoint(start: 600, end: 1099, name: TABLET),
  const Breakpoint(start: 1100, end: double.infinity, name: DESKTOP),
];
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/utils/responsive.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/core/utils/time_of_day_greeting.dart

Responsive breakpoints and voice playback. This file (time of day greeting) is one concrete piece of that layer.

Line count: 8. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/l10n/app_localizations.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/core/utils/time_of_day_greeting.dart**

```
import 'package:mu_study/l10n/app_localizations.dart';

String timeOfDayGreeting(AppLocalizations l10n) {
  final hour = DateTime.now().hour;
  if (hour < 12) return l10n.greetingMorning;
  if (hour < 18) return l10n.greetingAfternoon;
  return l10n.greetingEvening;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/utils/time_of_day_greeting.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/core/utils/voice_playback.dart

Responsive breakpoints and voice playback. This file (voice playback) is one concrete piece of that layer.

Line count: 284. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'dart:io';
- import 'package:audioplayers/audioplayers.dart';
- import 'package:flutter/foundation.dart';
- import 'package:http/http.dart' as http;
- import 'package:mu_study/core/services/voice_cache_service.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/core/utils/voice_playback.dart**

```
import 'dart:async';
import 'dart:io';

import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;
import 'package:mu_study/core/services/voice_cache_service.dart';

class DetectedAudioFormat {
  const DetectedAudioFormat({required this.mimeType, required this.extension});

  final String mimeType;
  final String extension;
}

DetectedAudioFormat detectAudioFormat(Uint8List bytes) {
  if (bytes.length >= 12) {
    final isRiff =
        bytes[0] == 0x52 &&
        bytes[1] == 0x49 &&
        bytes[2] == 0x46 &&
        bytes[3] == 0x46;
    final isWave =
        bytes[8] == 0x57 &&
        bytes[9] == 0x41 &&
        bytes[10] == 0x56 &&
        bytes[11] == 0x45;
    if (isRiff && isWave) {
      return const DetectedAudioFormat(mimeType: 'audio/wav', extension: 'wav');
    }
  }

  if (bytes.length >= 8 &&
      bytes[4] == 0x66 &&
      bytes[5] == 0x74 &&
      bytes[6] == 0x79 &&
      bytes[7] == 0x70) {
    return const DetectedAudioFormat(mimeType: 'audio/mp4', extension: 'm4a');
  }

  if (bytes.length >= 3 &&
      bytes[0] == 0x49 &&
```

```
      bytes[1] == 0x44 &&
      bytes[2] == 0x33) {
    return const DetectedAudioFormat(mimeType: 'audio/mpeg', extension: 'mp3');
  }

  return const DetectedAudioFormat(mimeType: 'audio/mp4', extension: 'm4a');
}

String mimeTypeForExtension(String extension) {
  return switch (extension.toLowerCase().replaceAll('.', '')) {
    'wav' => 'audio/wav',
    'mp3' => 'audio/mpeg',
    'aac' => 'audio/aac',
    _ => 'audio/mp4',
  };
}

bool _audioContextReady = false;

Future<void> _ensureAudioContext() async {
  if (_audioContextReady) return;
  await AudioPlayer.global.setAudioContext(
    AudioContext(
      iOS: AudioContextIOS(
        category: AVAudioSessionCategory.playback,
        options: const {},
      ),
      android: const AudioContextAndroid(
        isSpeakerphoneOn: true,
        stayAwake: true,
        contentType: AndroidContentType.speech,
        usageType: AndroidUsageType.media,
        audioFocus: AndroidAudioFocus.gain,
      ),
    ),
  );
  _audioContextReady = true;
}

Future<Uint8List> _downloadVoiceBytes(String url) async {
  final response = await http.get(Uri.parse(url));
  if (response.statusCode < 200 || response.statusCode >= 300) {
    throw Exception('Could not download voice (${response.statusCode})');
  }
  final bytes = response.bodyBytes;
  if (bytes.isEmpty) {
    throw Exception('Voice file is empty');
  }
  if (bytes.length < 64) {
    final head = String.fromCharCodes(bytes.take(32));
    if (head.trimLeft().startsWith('{') || head.trimLeft().startsWith('<')) {
      throw Exception('Voice download returned an error payload');
    }
  }
  return Uint8List.fromList(bytes);
}

Future<({Uint8List bytes, String mimeType, File? file})> resolveVoiceSource(
  String url,
) async {
  final cache = VoiceCacheService.instance;
  final cachedFile = await cache.getFile(url);
  if (cachedFile != null) {
    final bytes = Uint8List.fromList(await cachedFile.readAsBytes());
    if (bytes.isNotEmpty) {
      final format = detectAudioFormat(bytes);
      final fileExt = cachedFile.path.split('.').last.toLowerCase();
      if (fileExt != format.extension) {
        final fixed = await cache.put(url, bytes, format.extension);
        return (bytes: bytes, mimeType: format.mimeType, file: fixed);
      }
      return (bytes: bytes, mimeType: format.mimeType, file: cachedFile);
    }
```

```
    }

    final bytes = await _downloadVoiceBytes(url);
    final format = detectAudioFormat(bytes);
    final file = await cache.put(url, bytes, format.extension);
    return (bytes: bytes, mimeType: format.mimeType, file: file);
  }

  class VoicePlaybackController extends ChangeNotifier {
    VoicePlaybackController._() {
      _attachPlayerListeners(_player);
    }

    static final VoicePlaybackController instance = VoicePlaybackController._();

    final AudioPlayer _player = AudioPlayer(playerId: 'mu_study_voice');
    String? _activeKey;
    bool _playing = false;
    bool _loading = false;
    Duration _position = Duration.zero;
    Duration _duration = Duration.zero;
    int _generation = 0;

    Future<void> _queue = Future.value();

    String? get activeKey => _activeKey;
    Duration get position => _position;
    Duration get duration => _duration;

    bool isPlaying(String key) => _playing && _activeKey == key;
    bool isLoading(String key) => _loading && _activeKey == key;

    void _attachPlayerListeners(AudioPlayer player) {
      player.onPlayerComplete.listen((_) {
        _playing = false;
        _loading = false;
        _position = Duration.zero;
        notifyListeners();
      });
      player.onPlayerStateChanged.listen((state) {
        if (state == PlayerState.stopped || state == PlayerState.completed) {
          if (_playing) {
            _playing = false;
            notifyListeners();
          }
        }
      });
      player.onPositionChanged.listen((pos) {
        if (!_playing && !_loading) return;
        _position = pos;
        notifyListeners();
      });
      player.onDurationChanged.listen((dur) {
        _duration = dur;
        notifyListeners();
      });
    }

    Future<void> stop() {
      return _enqueue(() async {
        _generation++;
        await _safeStopAndRelease();
        _playing = false;
        _loading = false;
        _activeKey = null;
        _position = Duration.zero;
        _duration = Duration.zero;
        notifyListeners();
      });
    }

    Future<void> toggleRemote(String url) => _toggle(url, remote: true);
```

```
Future<void> toggleLocal(String path) => _toggle(path, remote: false);

Future<void> _enqueue(Future<void> Function() action) {
  final run = _queue.then((_) => action());
  _queue = run.catchError((_) {});
  return run;
}

Future<void> _safeStopAndRelease() async {
  try {
    await _player.stop();
  } catch (_) {}
  try {
    await _player.release();
  } catch (_) {}
}

Future<void> _playResolved({
  required Uint8List bytes,
  required String mimeType,
  File? file,
}) async {
  if (file != null && await file.exists()) {
    await _player.play(DeviceFileSource(file.path, mimeType: mimeType));
    return;
  }
  await _player.play(BytesSource(bytes, mimeType: mimeType));
}

Future<void> _toggle(String key, {required bool remote}) {
  return _enqueue(() async {
    if (_loading && _activeKey == key) return;

    if (_activeKey == key && _playing) {
      _generation++;
      await _safeStopAndRelease();
      _playing = false;
      _loading = false;
      _activeKey = null;
      _position = Duration.zero;
      _duration = Duration.zero;
      notifyListeners();
      return;
    }

    final gen = ++_generation;
    _activeKey = key;
    _loading = true;
    _playing = false;
    _position = Duration.zero;
    _duration = Duration.zero;
    notifyListeners();

    try {
      await _ensureAudioContext();
      await _safeStopAndRelease();
      if (gen != _generation) return;

      if (remote) {
        final resolved = await resolveVoiceSource(key);
        if (gen != _generation) return;
        await _playResolved(
          bytes: resolved.bytes,
          mimeType: resolved.mimeType,
          file: resolved.file,
        );
      } else {
        final lower = key.toLowerCase();
        final mime = lower.endsWith('.wav')
            ? 'audio/wav'
            : lower.endsWith('.mp3')
            ? 'audio/mpeg'
            : 'audio/mp4';
```

```
          await _player.play(DeviceFileSource(key, mimeType: mime));
        }

        if (gen != _generation) {
          await _safeStopAndRelease();
          return;
        }
        _playing = true;
        _loading = false;
        notifyListeners();
      } catch (e) {
        if (gen != _generation) rethrow;
        _playing = false;
        _loading = false;
        _activeKey = null;
        _position = Duration.zero;
        _duration = Duration.zero;
        notifyListeners();
        rethrow;
      }
    });
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/core/utils/voice_playback.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/l10n/domain_labels.dart

ARB translations and generated AppLocalizations. This file (domain labels) is one concrete piece of that layer.

Line count: 52. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/l10n/domain_labels.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/models/models.dart';

extension MarkCategoryL10n on MarkCategory {
  String l10nLabel(AppLocalizations l10n) => switch (this) {
    MarkCategory.homework => l10n.categoryHomework,
    MarkCategory.classwork => l10n.categoryClasswork,
    MarkCategory.quiz => l10n.categoryQuiz,
    MarkCategory.test => l10n.categoryTest,
    MarkCategory.exam => l10n.categoryExam,
    MarkCategory.project => l10n.categoryProject,
    MarkCategory.oral => l10n.categoryOral,
    MarkCategory.other => l10n.categoryOther,
  };
}

extension AttendanceStatusL10n on AttendanceStatus {
  String l10nLabel(AppLocalizations l10n) => switch (this) {
    AttendanceStatus.present => l10n.attendancePresentWord,
    AttendanceStatus.absent => l10n.attendanceAbsentWord,
    AttendanceStatus.late => l10n.attendanceLateWord,
    AttendanceStatus.excused => l10n.attendanceExcusedWord,
  };
}

extension SphereL10n on Sphere {
  String l10nLabel(AppLocalizations l10n) => switch (this) {
    Sphere.math => l10n.sphereMath,
    Sphere.english => l10n.sphereEnglish,
  };
}

extension AppNotificationL10n on AppNotification {
  String kindLabel(AppLocalizations l10n) => switch (kind) {
    'assignment' => l10n.notifKindAssignment,
    'announcement' => l10n.notifKindAnnouncement,
    'graded' => l10n.notifKindGraded,
    'submission' => l10n.notifKindSubmission,
    'chat' => l10n.notifKindChat,
    _ => title.isNotEmpty ? title : l10n.notifKindSystem,
  };

  IconData get kindIcon => switch (kind) {
    'assignment' => Icons.assignment_outlined,
```

```
      'announcement' => Icons.campaign_outlined,
      'graded' => Icons.grade_outlined,
      'submission' => Icons.upload_file_outlined,
      'chat' => Icons.chat_bubble_outline_rounded,
      _ => Icons.notifications_outlined,
    };
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/l10n/domain_labels.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/l10n/l10n_x.dart

ARB translations and generated AppLocalizations. This file (l10n x) is one concrete piece of that layer.

Line count: 8. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/widgets.dart';
- import 'package:mu_study/l10n/app_localizations.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/l10n/l10n_x.dart**

```
import 'package:flutter/widgets.dart';
import 'package:mu_study/l10n/app_localizations.dart';

export 'package:mu_study/l10n/app_localizations.dart';

extension L10nX on BuildContext {
  AppLocalizations get l10n => AppLocalizations.of(this);
}
```

### Comprehension

1. What user role(s) can reach the behavior in lib/l10n/l10n_x.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/l10n/nav_labels.dart

ARB translations and generated AppLocalizations. This file (nav labels) is one concrete piece of that layer.

Line count: 19. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/l10n/app_localizations.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/l10n/nav_labels.dart**

```
import 'package:mu_study/l10n/app_localizations.dart';

String navLabel(AppLocalizations l10n, String key) {
  return switch (key) {
    'navDashboard' => l10n.navDashboard,
    'navAssignments' => l10n.navAssignments,
    'navMarks' => l10n.navMarks,
    'navAttendance' => l10n.navAttendance,
    'navNotifications' => l10n.navNotifications,
    'navProfile' => l10n.navProfile,
    'navUsers' => l10n.navUsers,
    'navGroups' => l10n.navGroups,
    'navAnalytics' => l10n.navAnalytics,
    'navGradebook' => l10n.navGradebook,
    'navChat' => l10n.navChat,
    'navFocus' => l10n.navFocus,
    _ => key,
  };
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/l10n/nav_labels.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/main.dart

Process entry: load env, init Supabase, wrap ProviderScope, run MaterialApp.router.

Line count: 102. Import count: 16. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_localizations/flutter_localizations.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/backgrounds/nature_paper_backdrop.dart';
- import 'package:mu_study/components/backgrounds/soft_neumorphism_backdrop.dart';
- import 'package:mu_study/core/constants/env.dart';
- import 'package:mu_study/core/theme/app_theme.dart';
- import 'package:mu_study/core/utils/responsive.dart';
- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/providers/settings/locale_provider.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/settings/theme_provider.dart';
- import 'package:mu_study/core/router/app_router.dart';
- import 'package:mu_study/screens/auth/setup_required_screen.dart';
- import 'package:responsive_framework/responsive_framework.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/main.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/backgrounds/nature_paper_backdrop.dart';
import 'package:mu_study/components/backgrounds/soft_neumorphism_backdrop.dart';
import 'package:mu_study/core/constants/env.dart';
import 'package:mu_study/core/theme/app_theme.dart';
import 'package:mu_study/core/utils/responsive.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/providers/settings/locale_provider.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/settings/theme_provider.dart';
import 'package:mu_study/core/router/app_router.dart';
import 'package:mu_study/screens/auth/setup_required_screen.dart';
import 'package:responsive_framework/responsive_framework.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Env.load();

  if (!Env.isConfigured) {
    runApp(const SetupRequiredScreen());
    return;
  }
```

```
  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabaseAnonKey,
  );

  final initialThemeMode = await loadSavedThemeMode();
  final initialStyle = await loadSavedAppStyle();

  runApp(
    ProviderScope(
      overrides: [
        appStyleProvider.overrideWith((ref) {
          final controller = AppStyleController(ref, initial: initialStyle);
          ref.listen(currentProfileProvider, (prev, next) {
            next.whenData((profile) {
              final code = profile?.preferredStyle;
              if (code != null) controller.applyFromProfile(code);
            });
          });
          return controller;
        }),
        themeModeProvider.overrideWith((ref) {
          final controller = ThemeModeController(
            ref,
            initial: initialThemeMode,
          );
          ref.listen(currentProfileProvider, (prev, next) {
            next.whenData((profile) {
              final code = profile?.preferredTheme;
              if (code != null) controller.applyFromProfile(code);
            });
          });
          return controller;
        }),
      ],
      child: const MuStudyApp(),
    ),
  );
}

class MuStudyApp extends ConsumerWidget {
  const MuStudyApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(routerProvider);
    final locale = ref.watch(localeProvider);
    final themeMode = ref.watch(themeModeProvider);
    final style = ref.watch(appStyleProvider);

    return MaterialApp.router(
      onGenerateTitle: (context) => AppLocalizations.of(context).appName,
      debugShowCheckedModeBanner: false,
      theme: AppTheme.light(style),
      darkTheme: AppTheme.dark(style),
      themeMode: themeMode,
      themeAnimationDuration: Duration.zero,
      locale: locale,
      supportedLocales: AppLocalizations.supportedLocales,
      localizationsDelegates: const [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      builder: (context, child) => ResponsiveBreakpoints.builder(
        child: NaturePaperBackdrop(
          child: SoftNeumorphismBackdrop(child: child!),
        ),
        breakpoints: responsiveBreakpoints(),
      ),
      routerConfig: router,
    );
```

```
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/main.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/models/assignment.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (assignment) is one concrete piece of that layer.

Line count: 135. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/enums.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/assignment.dart**

```
import 'package:mu_study/models/enums.dart';

class Assignment {
  const Assignment({
    required this.id,
    required this.teacherId,
    required this.groupId,
    required this.type,
    required this.title,
    this.description = '',
    this.dueDate,
    this.createdAt,
  });

  final String id;
  final String teacherId;
  final String groupId;
  final AssignmentType type;
  final String title;
  final String description;
  final DateTime? dueDate;
  final DateTime? createdAt;

  factory Assignment.fromJson(Map<String, dynamic> json) {
    return Assignment(
      id: json['id'] as String,
      teacherId: json['teacher_id'] as String,
      groupId: json['group_id'] as String,
      type: assignmentTypeFromString(json['type'] as String),
      title: json['title'] as String,
      description: json['description'] as String? ?? '',
      dueDate: json['due_date'] != null
          ? DateTime.parse(json['due_date'] as String)
          : null,
      createdAt: json['created_at'] != null
          ? DateTime.parse(json['created_at'] as String)
          : null,
    );
  }

  Map<String, dynamic> toInsert({
    required String teacherId,
    required String groupId,
  }) => {
    'teacher_id': teacherId,
    'group_id': groupId,
    'type': assignmentTypeToDb(type),
    'title': title,
```

```
      'description': description,
      'due_date': dueDate?.toIso8601String(),
    };
  }

  class TaskItem {
    const TaskItem({
      required this.id,
      required this.assignmentId,
      required this.title,
      this.description = '',
      this.attachmentUrl,
      this.linkUrl,
      this.sortOrder = 0,
    });

    final String id;
    final String assignmentId;
    final String title;
    final String description;
    final String? attachmentUrl;
    final String? linkUrl;
    final int sortOrder;

    factory TaskItem.fromJson(Map<String, dynamic> json) {
      return TaskItem(
        id: json['id'] as String,
        assignmentId: json['assignment_id'] as String,
        title: json['title'] as String,
        description: json['description'] as String? ?? '',
        attachmentUrl: json['attachment_url'] as String?,
        linkUrl: json['link_url'] as String?,
        sortOrder: json['sort_order'] as int? ?? 0,
      );
    }
  }

  class Submission {
    const Submission({
      required this.id,
      required this.taskId,
      required this.studentId,
      this.textAnswer,
      this.fileUrl,
      this.voiceUrl,
      this.status = SubmissionStatus.pending,
      this.score,
      this.feedback,
      this.isCompleted = false,
      this.submittedAt,
    });

    final String id;
    final String taskId;
    final String studentId;
    final String? textAnswer;
    final String? fileUrl;
    final String? voiceUrl;
    final SubmissionStatus status;
    final double? score;
    final String? feedback;
    final bool isCompleted;
    final DateTime? submittedAt;

    factory Submission.fromJson(Map<String, dynamic> json) {
      return Submission(
        id: json['id'] as String,
        taskId: json['task_id'] as String,
        studentId: json['student_id'] as String,
        textAnswer: json['text_answer'] as String?,
        fileUrl: json['file_url'] as String?,
        voiceUrl: json['voice_url'] as String?,
        status: SubmissionStatus.values.firstWhere(
```

```
        (e) => e.name == (json['status'] as String? ?? 'pending'),
        orElse: () => SubmissionStatus.pending,
      ),
      score: json['score'] != null
          ? double.tryParse(json['score'].toString())
          : null,
      feedback: json['feedback'] as String?,
      isCompleted: json['is_completed'] as bool? ?? false,
      submittedAt: json['submitted_at'] != null
          ? DateTime.parse(json['submitted_at'] as String)
          : null,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/assignment.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/models/attendance.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (attendance) is one concrete piece of that layer.

Line count: 84. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/enums.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/attendance.dart**

```
import 'package:mu_study/models/enums.dart';

class AttendanceRecord {
  const AttendanceRecord({
    required this.id,
    required this.groupId,
    required this.studentId,
    required this.status,
    required this.date,
    this.sessionId,
    this.studentName,
  });

  final String id;
  final String groupId;
  final String studentId;
  final AttendanceStatus status;
  final DateTime date;
  final String? sessionId;
  final String? studentName;

  factory AttendanceRecord.fromJson(Map<String, dynamic> json) {
    final student = json['students'] as Map?;
    final profile = student?['profiles'] as Map?;
    return AttendanceRecord(
      id: json['id'] as String,
      groupId: json['group_id'] as String,
      studentId: json['student_id'] as String,
      status: attendanceFromString(json['status'] as String),
      date: DateTime.parse(json['date'] as String),
      sessionId: json['session_id'] as String?,
      studentName: profile?['full_name'] as String?,
    );
  }
}

class AttendanceStats {
  const AttendanceStats({
    this.present = 0,
    this.absent = 0,
    this.late = 0,
    this.excused = 0,
  });

  final int present;
  final int absent;
  final int late;
  final int excused;
```

```
  int get total => present + absent + late + excused;

  double get presentPercent => total == 0 ? 0 : (present / total) * 100;

  factory AttendanceStats.fromCounts(Map<String, int> counts) {
    return AttendanceStats(
      present: counts['present'] ?? 0,
      absent: counts['absent'] ?? 0,
      late: counts['late'] ?? 0,
      excused: counts['excused'] ?? 0,
    );
  }

  factory AttendanceStats.fromRecords(Iterable<AttendanceRecord> records) {
    var present = 0, absent = 0, late = 0, excused = 0;
    for (final r in records) {
      switch (r.status) {
        case AttendanceStatus.present:
          present++;
        case AttendanceStatus.absent:
          absent++;
        case AttendanceStatus.late:
          late++;
        case AttendanceStatus.excused:
          excused++;
      }
    }
    return AttendanceStats(
      present: present,
      absent: absent,
      late: late,
      excused: excused,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/attendance.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/models/chat.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (chat) is one concrete piece of that layer.

Line count: 379. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/enums.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/chat.dart**

```
import 'package:mu_study/models/enums.dart';

class ChatRoomSummary {
  const ChatRoomSummary({
    required this.id,
    required this.groupId,
    required this.groupTitle,
    required this.teacherName,
    required this.memberCount,
    this.createdAt,
    this.lastMessageId,
    this.lastMessageType,
    this.lastMessageText,
    this.lastMessageAt,
    this.lastMessageDeleted = false,
    this.lastMessageSenderId,
    this.lastMessageSenderName,
    this.unreadCount = 0,
  });

  final String id;
  final String groupId;
  final String groupTitle;
  final String teacherName;
  final int memberCount;
  final DateTime? createdAt;
  final String? lastMessageId;
  final ChatMessageType? lastMessageType;
  final String? lastMessageText;
  final DateTime? lastMessageAt;
  final bool lastMessageDeleted;
  final String? lastMessageSenderId;
  final String? lastMessageSenderName;
  final int unreadCount;

  ChatRoomSummary copyWith({int? unreadCount}) {
    return ChatRoomSummary(
      id: id,
      groupId: groupId,
      groupTitle: groupTitle,
      teacherName: teacherName,
      memberCount: memberCount,
      createdAt: createdAt,
      lastMessageId: lastMessageId,
      lastMessageType: lastMessageType,
      lastMessageText: lastMessageText,
      lastMessageAt: lastMessageAt,
      lastMessageDeleted: lastMessageDeleted,
```

```
        lastMessageSenderId: lastMessageSenderId,
        lastMessageSenderName: lastMessageSenderName,
        unreadCount: unreadCount ?? this.unreadCount,
      );
    }

    factory ChatRoomSummary.fromJson(Map<String, dynamic> json) {
      final typeRaw = json['last_message_type'] as String?;
      return ChatRoomSummary(
        id: json['id'] as String,
        groupId: json['group_id'] as String,
        groupTitle: json['group_title'] as String? ?? '',
        teacherName: json['teacher_name'] as String? ?? '',
        memberCount: (json['member_count'] as num?)?.toInt() ?? 0,
        createdAt: json['created_at'] != null
            ? DateTime.parse(json['created_at'] as String)
            : null,
        lastMessageId: json['last_message_id'] as String?,
        lastMessageType: typeRaw == null
            ? null
            : chatMessageTypeFromString(typeRaw),
        lastMessageText: json['last_message_text'] as String?,
        lastMessageAt: json['last_message_at'] != null
            ? DateTime.parse(json['last_message_at'] as String)
            : null,
        lastMessageDeleted: json['last_message_deleted'] as bool? ?? false,
        lastMessageSenderId: json['last_message_sender_id'] as String?,
        lastMessageSenderName: json['last_message_sender_name'] as String?,
        unreadCount: (json['unread_count'] as num?)?.toInt() ?? 0,
      );
    }
  }

  class ChatMessage {
    const ChatMessage({
      required this.id,
      required this.roomId,
      required this.senderId,
      required this.messageType,
      this.textContent,
      this.mediaUrl,
      required this.createdAt,
      required this.updatedAt,
      this.isDeleted = false,
      this.senderName,
      this.senderUsername,
      this.senderAvatarUrl,
      this.senderRole,
      this.isReadByOthers = false,
      this.reactions = const [],
      this.replyToMessageId,
      this.replyTo,
    });

    final String id;
    final String roomId;
    final String senderId;
    final ChatMessageType messageType;
    final String? textContent;
    final String? mediaUrl;
    final DateTime createdAt;
    final DateTime updatedAt;
    final bool isDeleted;
    final String? senderName;
    final String? senderUsername;
    final String? senderAvatarUrl;
    final UserRole? senderRole;

    final bool isReadByOthers;

    final List<ChatReaction> reactions;

    final String? replyToMessageId;
```

```
final ChatReplyPreview? replyTo;

String get displayName {
  final name = senderName?.trim() ?? '';
  if (name.isNotEmpty) return name;
  final user = senderUsername?.trim() ?? '';
  if (user.isNotEmpty) {
    return user.startsWith('@') ? user.substring(1) : user;
  }
  return '';
}

bool get wasEdited =>
    !isDeleted && updatedAt.difference(createdAt).inSeconds.abs() > 1;

List<({String emoji, int count})> get reactionSummary {
  final counts = <String, int>{};
  final order = <String>[];
  for (final r in reactions) {
    if (!counts.containsKey(r.emoji)) order.add(r.emoji);
    counts[r.emoji] = (counts[r.emoji] ?? 0) + 1;
  }
  return [for (final emoji in order) (emoji: emoji, count: counts[emoji]!)];
}

String? myReactionEmoji(String? userId) {
  if (userId == null) return null;
  for (final r in reactions) {
    if (r.userId == userId) return r.emoji;
  }
  return null;
}

Map<String, dynamic> toCacheJson() {
  return {
    'id': id,
    'room_id': roomId,
    'sender_id': senderId,
    'message_type': messageType.name,
    'text_content': textContent,
    'media_url': mediaUrl,
    'created_at': createdAt.toIso8601String(),
    'updated_at': updatedAt.toIso8601String(),
    'is_deleted': isDeleted,
    'reply_to_message_id': replyToMessageId,
    if (replyTo != null) 'reply_to': replyTo!.toCacheJson(),
    'profiles': {
      'full_name': senderName,
      'username': senderUsername,
      'avatar_url': senderAvatarUrl,
      if (senderRole != null) 'role': senderRole!.name,
    },
    'message_reads': [
      if (isReadByOthers) {'user_id': 'cached_other'},
    ],
    'message_reactions': [
      for (final r in reactions)
        {
          'id': r.id,
          'message_id': r.messageId,
          'user_id': r.userId,
          'emoji': r.emoji,
        },
    ],
  };
}

ChatMessage copyWith({
  String? textContent,
  String? mediaUrl,
  DateTime? updatedAt,
  bool? isDeleted,
  bool? isReadByOthers,
```

```
    String? senderName,
    String? senderUsername,
    String? senderAvatarUrl,
    UserRole? senderRole,
    List<ChatReaction>? reactions,
    String? replyToMessageId,
    ChatReplyPreview? replyTo,
  }) {
    return ChatMessage(
      id: id,
      roomId: roomId,
      senderId: senderId,
      messageType: messageType,
      textContent: textContent ?? this.textContent,
      mediaUrl: mediaUrl ?? this.mediaUrl,
      createdAt: createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
      isDeleted: isDeleted ?? this.isDeleted,
      senderName: senderName ?? this.senderName,
      senderUsername: senderUsername ?? this.senderUsername,
      senderAvatarUrl: senderAvatarUrl ?? this.senderAvatarUrl,
      senderRole: senderRole ?? this.senderRole,
      isReadByOthers: isReadByOthers ?? this.isReadByOthers,
      reactions: reactions ?? this.reactions,
      replyToMessageId: replyToMessageId ?? this.replyToMessageId,
      replyTo: replyTo ?? this.replyTo,
    );
  }

  factory ChatMessage.fromJson(Map<String, dynamic> json) {
    final profile = json['profiles'] as Map?;
    final senderId = json['sender_id'] as String;
    final reads = json['message_reads'] as List?;
    final isReadByOthers =
        reads?.any((r) {
          if (r is! Map) return false;
          final readerId = r['user_id'] as String?;
          return readerId != null && readerId != senderId;
        }) ??
        false;

    final reactionRows = json['message_reactions'] as List?;
    final reactions =
        reactionRows
            ?.whereType<Map>()
            .map((r) => ChatReaction.fromJson(Map<String, dynamic>.from(r)))
            .toList() ??
        const <ChatReaction>[];

    final roleRaw = profile?['role'] as String?;

    ChatReplyPreview? replyTo;
    final replyRaw = json['reply_to'];
    if (replyRaw is Map) {
      replyTo = ChatReplyPreview.fromJson(Map<String, dynamic>.from(replyRaw));
    }

    return ChatMessage(
      id: json['id'] as String,
      roomId: json['room_id'] as String,
      senderId: senderId,
      messageType: chatMessageTypeFromString(json['message_type'] as String?),
      textContent: json['text_content'] as String?,
      mediaUrl: json['media_url'] as String?,
      createdAt: DateTime.parse(
        json['created_at'] as String? ?? DateTime.now().toIso8601String(),
      ),
      updatedAt: DateTime.parse(
        json['updated_at'] as String? ??
            json['created_at'] as String? ??
            DateTime.now().toIso8601String(),
      ),
      isDeleted: json['is_deleted'] as bool? ?? false,
```

```
      senderName: profile?['full_name'] as String?,
      senderUsername: profile?['username'] as String?,
      senderAvatarUrl: profile?['avatar_url'] as String?,
      senderRole: roleRaw != null ? userRoleFromString(roleRaw) : null,
      isReadByOthers: isReadByOthers,
      reactions: reactions,
      replyToMessageId: json['reply_to_message_id'] as String?,
      replyTo: replyTo,
    );
  }
}

class ChatReplyPreview {
  const ChatReplyPreview({
    required this.id,
    required this.messageType,
    this.textContent,
    this.mediaUrl,
    this.isDeleted = false,
    required this.senderId,
    this.senderName,
    this.senderUsername,
    this.senderAvatarUrl,
  });

  final String id;
  final ChatMessageType messageType;
  final String? textContent;
  final String? mediaUrl;
  final bool isDeleted;
  final String senderId;
  final String? senderName;
  final String? senderUsername;
  final String? senderAvatarUrl;

  String get displayName {
    final name = senderName?.trim() ?? '';
    if (name.isNotEmpty) return name;
    final user = senderUsername?.trim() ?? '';
    if (user.isNotEmpty) {
      return user.startsWith('@') ? user.substring(1) : user;
    }
    return '';
  }

  Map<String, dynamic> toCacheJson() => {
    'id': id,
    'message_type': messageType.name,
    'text_content': textContent,
    'media_url': mediaUrl,
    'is_deleted': isDeleted,
    'sender_id': senderId,
    'profiles': {
      'full_name': senderName,
      'username': senderUsername,
      'avatar_url': senderAvatarUrl,
    },
  };

  factory ChatReplyPreview.fromJson(Map<String, dynamic> json) {
    final profile = json['profiles'] as Map?;
    return ChatReplyPreview(
      id: json['id'] as String,
      messageType: chatMessageTypeFromString(json['message_type'] as String?),
      textContent: json['text_content'] as String?,
      mediaUrl: json['media_url'] as String?,
      isDeleted: json['is_deleted'] as bool? ?? false,
      senderId: json['sender_id'] as String? ?? '',
      senderName: profile?['full_name'] as String?,
      senderUsername: profile?['username'] as String?,
      senderAvatarUrl: profile?['avatar_url'] as String?,
    );
  }
```

```
  factory ChatReplyPreview.fromMessage(ChatMessage message) {
    return ChatReplyPreview(
      id: message.id,
      messageType: message.messageType,
      textContent: message.textContent,
      mediaUrl: message.mediaUrl,
      isDeleted: message.isDeleted,
      senderId: message.senderId,
      senderName: message.senderName,
      senderUsername: message.senderUsername,
      senderAvatarUrl: message.senderAvatarUrl,
    );
  }
}

class ChatReaction {
  const ChatReaction({
    required this.id,
    required this.messageId,
    required this.userId,
    required this.emoji,
  });

  final String id;
  final String messageId;
  final String userId;
  final String emoji;

  factory ChatReaction.fromJson(Map<String, dynamic> json) {
    return ChatReaction(
      id: json['id'] as String,
      messageId: json['message_id'] as String,
      userId: json['user_id'] as String,
      emoji: json['emoji'] as String,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/chat.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/models/enums.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (enums) is one concrete piece of that layer.

Line count: 102. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/enums.dart**

```
enum UserRole { student, teacher, admin }

enum Sphere { math, english }

enum AssignmentType { staticAssignment, homeworkPack }

enum AttendanceStatus { present, absent, late, excused }

enum SubmissionStatus { pending, submitted, reviewed }

enum ChatMessageType { text, voice, image, video, file }

ChatMessageType chatMessageTypeFromString(String? value) {
  return ChatMessageType.values.firstWhere(
    (e) => e.name == value,
    orElse: () => ChatMessageType.text,
  );
}

String chatMessageTypeToDb(ChatMessageType type) => type.name;

ChatMessageType chatMessageTypeForFileName(String fileName) {
  final lower = fileName.toLowerCase();
  const videos = ['.mp4', '.mov', '.webm', '.m4v', '.avi', '.mkv'];
  const images = [
    '.jpg',
    '.jpeg',
    '.png',
    '.gif',
    '.webp',
    '.heic',
    '.heif',
    '.bmp',
    '.tif',
    '.tiff',
  ];
  if (videos.any(lower.endsWith)) return ChatMessageType.video;
  if (images.any(lower.endsWith)) return ChatMessageType.image;
  return ChatMessageType.file;
}

enum MarkCategory {
  homework,
  classwork,
  quiz,
  test,
  exam,
  project,
  oral,
  other,
```

```
}

extension MarkCategoryLabel on MarkCategory {
  String get label => switch (this) {
    MarkCategory.homework => 'Homework',
    MarkCategory.classwork => 'Classwork',
    MarkCategory.quiz => 'Quiz',
    MarkCategory.test => 'Test',
    MarkCategory.exam => 'Exam',
    MarkCategory.project => 'Project',
    MarkCategory.oral => 'Oral',
    MarkCategory.other => 'Other',
  };
}

MarkCategory markCategoryFromString(String? value) {
  return MarkCategory.values.firstWhere(
    (e) => e.name == value,
    orElse: () => MarkCategory.other,
  );
}

UserRole userRoleFromString(String value) {
  return UserRole.values.firstWhere(
    (e) => e.name == value,
    orElse: () => UserRole.student,
  );
}

Sphere? sphereFromString(String? value) {
  if (value == null) return null;
  return Sphere.values.cast<Sphere?>().firstWhere(
    (e) => e?.name == value,
    orElse: () => null,
  );
}

AssignmentType assignmentTypeFromString(String value) {
  if (value == 'homework_pack') return AssignmentType.homeworkPack;
  return AssignmentType.staticAssignment;
}

String assignmentTypeToDb(AssignmentType type) {
  return type == AssignmentType.homeworkPack ? 'homework_pack' : 'static';
}

AttendanceStatus attendanceFromString(String value) {
  return AttendanceStatus.values.firstWhere(
    (e) => e.name == value,
    orElse: () => AttendanceStatus.present,
  );
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/enums.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/models/mark.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (mark) is one concrete piece of that layer.

Line count: 97. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/enums.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/mark.dart**

```
import 'package:mu_study/models/enums.dart';

class Mark {
  const Mark({
    required this.id,
    required this.groupId,
    required this.studentId,
    required this.teacherId,
    required this.title,
    required this.percentage,
    this.category = MarkCategory.other,
    this.weight = 1,
    this.comment = '',
    this.submissionId,
    required this.markedOn,
    this.studentName,
    this.groupTitle,
  });

  final String id;
  final String groupId;
  final String studentId;
  final String teacherId;
  final String title;
  final double percentage;
  final MarkCategory category;
  final double weight;
  final String comment;
  final String? submissionId;
  final DateTime markedOn;

  final String? studentName;

  final String? groupTitle;

  bool get isFromSubmission => submissionId != null;

  factory Mark.fromJson(Map<String, dynamic> json) {
    final student = json['students'] as Map?;
    final profile = student?['profiles'] as Map?;
    return Mark(
      id: json['id'] as String,
      groupId: json['group_id'] as String,
      studentId: json['student_id'] as String,
      teacherId: json['teacher_id'] as String,
      title: json['title'] as String? ?? '',
      percentage: double.tryParse('${json['percentage']}') ?? 0,
      category: markCategoryFromString(json['category'] as String?),
```

```
      weight: double.tryParse('${json['weight']}') ?? 1,
      comment: json['comment'] as String? ?? '',
      submissionId: json['submission_id'] as String?,
      markedOn:
          DateTime.tryParse(json['marked_on']?.toString() ?? '') ??
          DateTime.now(),
      studentName:
          profile?['full_name'] as String? ?? profile?['username'] as String?,
      groupTitle: (json['groups'] as Map?)?['title'] as String?,
    );
  }

  Map<String, dynamic> toWrite() => {
    'group_id': groupId,
    'student_id': studentId,
    'teacher_id': teacherId,
    'title': title,
    'category': category.name,
    'percentage': percentage,
    'weight': weight,
    'comment': comment,
    'marked_on': markedOn.toIso8601String().split('T').first,
  };
}

class GroupMember {
  const GroupMember({
    required this.studentId,
    required this.fullName,
    required this.username,
  });

  final String studentId;
  final String fullName;
  final String username;

  factory GroupMember.fromGroupStudentRow(Map<String, dynamic> json) {
    final student = json['students'] as Map;
    final profile = student['profiles'] as Map;
    final username = profile['username'] as String? ?? '';
    return GroupMember(
      studentId: student['id'] as String,
      fullName: (profile['full_name'] as String?)?.isNotEmpty == true
          ? profile['full_name'] as String
          : username,
      username: username,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/mark.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/models/models.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (models) is one concrete piece of that layer.

Line count: 7. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/models.dart**

```
export 'package:mu_study/models/enums.dart';
export 'package:mu_study/models/profile.dart';
export 'package:mu_study/models/assignment.dart';
export 'package:mu_study/models/mark.dart';
export 'package:mu_study/models/attendance.dart';
export 'package:mu_study/models/notification.dart';
export 'package:mu_study/models/chat.dart';
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/models.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/models/notification.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (notification) is one concrete piece of that layer.

Line count: 80. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/notification.dart**

```
class AppNotification {
  const AppNotification({
    required this.id,
    required this.userId,
    required this.title,
    this.body = '',
    this.isRead = false,
    this.kind = 'system',
    this.actionPath,
    this.refId,
    this.createdAt,
  });

  final String id;
  final String userId;
  final String title;
  final String body;
  final bool isRead;

  final String kind;
  final String? actionPath;
  final String? refId;
  final DateTime? createdAt;

  factory AppNotification.fromJson(Map<String, dynamic> json) {
    return AppNotification(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      title: json['title'] as String,
      body: json['body'] as String? ?? '',
      isRead: json['is_read'] as bool? ?? false,
      kind: json['kind'] as String? ?? 'system',
      actionPath: json['action_path'] as String?,
      refId: json['ref_id'] as String?,
      createdAt: json['created_at'] != null
          ? DateTime.parse(json['created_at'] as String)
          : null,
    );
  }
}

class Announcement {
  const Announcement({
    required this.id,
    this.teacherId,
    required this.groupId,
    required this.title,
    this.description = '',
    this.imageUrls = const [],
    this.fileUrls = const [],
```

```
      this.createdAt,
    });

    final String id;

    final String? teacherId;
    final String groupId;
    final String title;
    final String description;
    final List<String> imageUrls;
    final List<String> fileUrls;
    final DateTime? createdAt;

    factory Announcement.fromJson(Map<String, dynamic> json) {
      List<String> urls(dynamic v) =>
          (v as List<dynamic>? ?? []).map((e) => e.toString()).toList();
      return Announcement(
        id: json['id'] as String,
        teacherId: json['teacher_id'] as String?,
        groupId: json['group_id'] as String,
        title: json['title'] as String,
        description: json['description'] as String? ?? '',
        imageUrls: urls(json['image_urls']),
        fileUrls: urls(json['file_urls']),
        createdAt: json['created_at'] != null
            ? DateTime.parse(json['created_at'] as String)
            : null,
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/notification.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/models/profile.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (profile) is one concrete piece of that layer.

Line count: 112. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/enums.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/models/profile.dart**

```
import 'package:mu_study/models/enums.dart';

class Profile {
  const Profile({
    required this.id,
    required this.username,
    required this.fullName,
    required this.role,
    this.avatarUrl,
    this.isDisabled = false,
    this.preferredLanguage,
    this.preferredTheme,
    this.preferredStyle,
  });

  final String id;
  final String username;
  final String fullName;
  final UserRole role;
  final String? avatarUrl;
  final bool isDisabled;

  final String? preferredLanguage;

  final String? preferredTheme;

  final String? preferredStyle;

  bool get hasChosenLanguage =>
      preferredLanguage != null && preferredLanguage!.isNotEmpty;

  factory Profile.fromJson(Map<String, dynamic> json) {
    return Profile(
      id: json['id'] as String,
      username: json['username'] as String,
      fullName: json['full_name'] as String? ?? '',
      role: userRoleFromString(json['role'] as String),
      avatarUrl: json['avatar_url'] as String?,
      isDisabled: json['is_disabled'] as bool? ?? false,
      preferredLanguage: json['preferred_language'] as String?,
      preferredTheme: json['preferred_theme'] as String?,
      preferredStyle: json['preferred_style'] as String?,
    );
  }

  Map<String, dynamic> toJson() => {
    'full_name': fullName,
    'avatar_url': avatarUrl,
```

```
    'preferred_language': preferredLanguage,
    'preferred_theme': preferredTheme,
    'preferred_style': preferredStyle,
  };
}

class TeacherProfile {
  const TeacherProfile({
    required this.id,
    required this.profileId,
    required this.sphere,
  });

  final String id;
  final String profileId;
  final Sphere sphere;

  factory TeacherProfile.fromJson(Map<String, dynamic> json) {
    return TeacherProfile(
      id: json['id'] as String,
      profileId: json['profile_id'] as String,
      sphere: sphereFromString(json['sphere'] as String) ?? Sphere.math,
    );
  }
}

class StudentProfile {
  const StudentProfile({required this.id, required this.profileId});

  final String id;
  final String profileId;

  factory StudentProfile.fromJson(Map<String, dynamic> json) {
    return StudentProfile(
      id: json['id'] as String,
      profileId: json['profile_id'] as String,
    );
  }
}

class StudyGroup {
  const StudyGroup({
    required this.id,
    required this.teacherId,
    required this.title,
    this.createdAt,
  });

  final String id;
  final String teacherId;
  final String title;
  final DateTime? createdAt;

  factory StudyGroup.fromJson(Map<String, dynamic> json) {
    return StudyGroup(
      id: json['id'] as String,
      teacherId: json['teacher_id'] as String,
      title: json['title'] as String,
      createdAt: json['created_at'] != null
          ? DateTime.parse(json['created_at'] as String)
          : null,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/models/profile.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?

5. Name one widget or provider that would break if this file’s public API changed.

## lib/providers/admin/admin_providers.dart

Users, groups, analytics, academic years, mass generator. This file (admin providers) is one concrete piece of that layer.

Line count: 87. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';
- import 'package:mu_study/core/paging/page_result.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/providers/admin/admin_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/core/paging/page_result.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';

Future<int> _exactCount(String table) async {
  final result = await supabase
      .from(table)
      .select('id')
      .count(CountOption.exact);
  return result.count;
}

final adminPlatformStatsProvider = FutureProvider<Map<String, int>>((
  ref,
) async {
  final teachers = await _exactCount('teachers');
  final students = await _exactCount('students');
  final groups = await _exactCount('groups');
  final assignments = await _exactCount('assignments');
  final marks = await _exactCount('marks');
  return {
    'teachers': teachers,
    'students': students,
    'groups': groups,
    'assignments': assignments,
    'marks': marks,
  };
});

final adminGroupsProvider = FutureProvider<List<Map<String, dynamic>>>((
  ref,
) async {
  final rows = await supabase
      .from('groups')
      .select('id, title, teachers(sphere, profiles(full_name))')
      .order('title');
```

```
  return rows.map((e) => Map<String, dynamic>.from(e)).toList();
});

final adminAnnouncementsProvider = FutureProvider<List<Map<String, dynamic>>>((
  ref,
) async {
  final rows = await supabase
      .from('announcements')
      .select()
      .order('created_at', ascending: false);
  return rows.map((e) => Map<String, dynamic>.from(e)).toList();
});

final allProfilesProvider =
    AsyncNotifierProvider<AllProfilesNotifier, PageResult<Profile>>(
      AllProfilesNotifier.new,
    );

class AllProfilesNotifier extends AsyncNotifier<PageResult<Profile>> {
  @override
  Future<PageResult<Profile>> build() => _fetch(0);

  Future<PageResult<Profile>> _fetch(int offset) async {
    final limit = PageResult.fetchSize(PageResult.pageSize);
    final rows = await supabase
        .from('profiles')
        .select()
        .order('username')
        .range(offset, offset + limit - 1);
    return PageResult.fromFetch(
      rows.map((e) => Profile.fromJson(Map<String, dynamic>.from(e))).toList(),
    );
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/admin/admin_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/attendance/attendance_providers.dart

Lesson sessions + attendance upsert. This file (attendance providers) is one concrete piece of that layer.

Line count: 257. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/attendance/attendance_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';

String dateKey(DateTime date) => DateTime(
  date.year,
  date.month,
  date.day,
).toIso8601String().split('T').first;

DateTime calendarDay(DateTime date) =>
    DateTime(date.year, date.month, date.day);

bool isAttendanceWeekend(DateTime date) {
  final weekday = calendarDay(date).weekday;
  return weekday == DateTime.saturday || weekday == DateTime.sunday;
}

bool isAttendanceDateAllowed(DateTime date) {
  final day = calendarDay(date);
  final today = calendarDay(DateTime.now());
  if (day.isAfter(today)) return false;
  if (isAttendanceWeekend(day)) return false;
  return true;
}

class AttendanceFutureDateException implements Exception {
  const AttendanceFutureDateException();

  @override
  String toString() => 'AttendanceFutureDateException';
}

class AttendanceWeekendException implements Exception {
  const AttendanceWeekendException();

  @override
  String toString() => 'AttendanceWeekendException';
}

Never _rejectAttendanceDate(DateTime date) {
  final day = calendarDay(date);
  if (isAttendanceWeekend(day)) {
```

```
    throw const AttendanceWeekendException();
  }
  final today = calendarDay(DateTime.now());
  if (day.isAfter(today)) {
    throw const AttendanceFutureDateException();
  }
  throw StateError('Attendance date rejected unexpectedly: $day');
}

class AttendanceQuery {
  const AttendanceQuery({
    required this.groupId,
    this.studentId,
    required this.from,
    required this.to,
  });

  final String groupId;
  final String? studentId;
  final DateTime from;
  final DateTime to;

  @override
  bool operator ==(Object other) =>
      other is AttendanceQuery &&
      other.groupId == groupId &&
      other.studentId == studentId &&
      other.from == from &&
      other.to == to;

  @override
  int get hashCode => Object.hash(groupId, studentId, from, to);
}

final attendanceRangeProvider =
    FutureProvider.family<List<AttendanceRecord>, AttendanceQuery>((
      ref,
      query,
    ) async {
      var filtered = supabase
          .from('attendance')
          .select('*, students(profiles(full_name))')
          .eq('group_id', query.groupId)
          .gte('date', dateKey(query.from))
          .lte('date', dateKey(query.to));

      if (query.studentId != null) {
        filtered = filtered.eq('student_id', query.studentId!);
      }

      final rows = await filtered.order('date', ascending: true);
      return rows
          .map((e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)))
          .toList();
    });

final attendanceRangeStatsProvider =
    Provider.family<AsyncValue<AttendanceStats>, AttendanceQuery>((ref, query) {
      return ref
          .watch(attendanceRangeProvider(query))
          .whenData(AttendanceStats.fromRecords);
    });

final todayAttendanceProvider =
    FutureProvider.family<Map<String, AttendanceStatus>, String>((
      ref,
      groupId,
    ) async {
      final rows = await supabase
          .from('attendance')
          .select('student_id, status')
          .eq('group_id', groupId)
          .eq('date', dateKey(DateTime.now()));
```

```
      return {
        for (final row in rows)
          row['student_id'] as String: attendanceFromString(
            row['status'] as String,
          ),
      };
    });

class AttendanceWeekBucket {
  const AttendanceWeekBucket({
    this.present = 0,
    this.absent = 0,
    this.late = 0,
  });

  final int present;
  final int absent;
  final int late;

  int get max => [present, absent, late].fold(0, (a, b) => a > b ? a : b);
}

class AttendanceWeekTrend {
  const AttendanceWeekTrend({required this.thisWeek, required this.lastWeek});

  final AttendanceWeekBucket thisWeek;
  final AttendanceWeekBucket lastWeek;

  bool get isEmpty => thisWeek.max == 0 && lastWeek.max == 0;
}

final attendanceWeekTrendProvider =
    FutureProvider.family<AttendanceWeekTrend, String>((ref, groupId) async {
      final today = calendarDay(DateTime.now());
      final thisFrom = today.subtract(const Duration(days: 6));
      final lastTo = today.subtract(const Duration(days: 7));
      final lastFrom = today.subtract(const Duration(days: 13));

      final rows = await supabase
          .from('attendance')
          .select('status, date')
          .eq('group_id', groupId)
          .gte('date', dateKey(lastFrom))
          .lte('date', dateKey(today));

      var thisPresent = 0, thisAbsent = 0, thisLate = 0;
      var lastPresent = 0, lastAbsent = 0, lastLate = 0;

      for (final row in rows) {
        final day = calendarDay(DateTime.parse(row['date'] as String));
        final status = row['status'] as String;
        final inThisWeek = !day.isBefore(thisFrom) && !day.isAfter(today);
        final inLastWeek = !day.isBefore(lastFrom) && !day.isAfter(lastTo);

        if (inThisWeek) {
          switch (status) {
            case 'present':
              thisPresent++;
            case 'absent':
              thisAbsent++;
            case 'late':
              thisLate++;
          }
        } else if (inLastWeek) {
          switch (status) {
            case 'present':
              lastPresent++;
            case 'absent':
              lastAbsent++;
            case 'late':
              lastLate++;
          }
        }
```

```
      }

      return AttendanceWeekTrend(
        thisWeek: AttendanceWeekBucket(
          present: thisPresent,
          absent: thisAbsent,
          late: thisLate,
        ),
        lastWeek: AttendanceWeekBucket(
          present: lastPresent,
          absent: lastAbsent,
          late: lastLate,
        ),
      );
    });

Future<void> upsertAttendance({
  required String groupId,
  required String studentId,
  required AttendanceStatus status,
  required DateTime date,
}) async {
  if (!isAttendanceDateAllowed(date)) {
    _rejectAttendanceDate(date);
  }
  final key = dateKey(date);
  await supabase.rpc(
    'ensure_lesson_session',
    params: {'p_group_id': groupId, 'p_session_date': key},
  );
  await supabase.from('attendance').upsert({
    'group_id': groupId,
    'student_id': studentId,
    'status': status.name,
    'date': key,
  }, onConflict: 'group_id,student_id,date');
}

Future<void> upsertAttendanceBulk({
  required String groupId,
  required DateTime date,
  required Map<String, AttendanceStatus> byStudentId,
}) async {
  if (byStudentId.isEmpty) return;
  if (!isAttendanceDateAllowed(date)) {
    _rejectAttendanceDate(date);
  }
  final key = dateKey(date);
  await supabase.rpc(
    'ensure_lesson_session',
    params: {'p_group_id': groupId, 'p_session_date': key},
  );
  final rows = byStudentId.entries
      .map(
        (e) => {
          'group_id': groupId,
          'student_id': e.key,
          'status': e.value.name,
          'date': key,
        },
      )
      .toList();
  await supabase
      .from('attendance')
      .upsert(rows, onConflict: 'group_id,student_id,date');
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/attendance/attendance_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?

3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/auth/auth_provider.dart

Session stream, username login RPC, profile/teacher/student loaders. This file (auth provider) is one concrete piece of that layer.

Line count: 119. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/services/app_cache.dart';
- import 'package:mu_study/core/services/cache_service.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/auth/auth_provider.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/services/app_cache.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

final authStateProvider = StreamProvider<AuthState>((ref) {
  return supabase.auth.onAuthStateChange;
});

class AuthRepository {
  AuthRepository(this._cache);

  final CacheService? _cache;

  Future<String?> resolveAuthEmail(String username) async {
    final normalized = username.trim();
    final result = await supabase.rpc(
      'get_auth_email_by_username',
      params: {'p_username': normalized},
    );
    return result as String?;
  }

  Future<void> signIn(String username, String password) async {
    final email = await resolveAuthEmail(username);
    if (email == null) {
      throw const AuthException('Incorrect username or password');
    }
    await supabase.auth.signInWithPassword(email: email, password: password);
  }

  Future<void> changePassword({
    required String username,
    required String currentPassword,
    required String newPassword,
  }) async {
    await signIn(username, currentPassword);
    await supabase.auth.updateUser(UserAttributes(password: newPassword));
```

```
  }

  Future<void> signOut() async {
    await supabase.auth.signOut();
    await clearAllAppCaches(_cache);
  }

  User? get currentUser => supabase.auth.currentUser;
}

final authRepositoryProvider = Provider<AuthRepository>((ref) {
  return AuthRepository(ref.watch(cacheServiceProvider).valueOrNull);
});

class ProfileRepository {
  ProfileRepository(this._cache);

  final CacheService? _cache;

  Future<Profile?> fetchCurrentProfile() async {
    final uid = supabase.auth.currentUser?.id;
    if (uid == null) return null;

    try {
      final data = await supabase
          .from('profiles')
          .select()
          .eq('id', uid)
          .maybeSingle();
      if (data == null) return null;
      final profile = Profile.fromJson(Map<String, dynamic>.from(data));
      await _cache?.cacheProfile(data);
      return profile;
    } catch (_) {
      final cached = _cache?.cachedProfile;
      if (cached != null) return Profile.fromJson(cached);
      rethrow;
    }
  }

  Future<void> updateProfile(String id, Map<String, dynamic> updates) async {
    await supabase.from('profiles').update(updates).eq('id', id);
  }
}

final profileRepositoryProvider = Provider<ProfileRepository>((ref) {
  final cacheAsync = ref.watch(cacheServiceProvider);
  return ProfileRepository(cacheAsync.valueOrNull);
});

final currentProfileProvider = FutureProvider<Profile?>((ref) async {
  ref.watch(authStateProvider);
  final repo = ref.watch(profileRepositoryProvider);
  return repo.fetchCurrentProfile();
});

final teacherProfileProvider = FutureProvider<TeacherProfile?>((ref) async {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null || profile.role != UserRole.teacher) return null;
  final data = await supabase
      .from('teachers')
      .select()
      .eq('profile_id', profile.id)
      .maybeSingle();
  if (data == null) return null;
  return TeacherProfile.fromJson(Map<String, dynamic>.from(data));
});

final studentProfileProvider = FutureProvider<StudentProfile?>((ref) async {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null || profile.role != UserRole.student) return null;
  final data = await supabase
      .from('students')
```

```
        .select()
        .eq('profile_id', profile.id)
        .maybeSingle();
    if (data == null) return null;
    return StudentProfile.fromJson(Map<String, dynamic>.from(data));
  });
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/auth/auth_provider.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/providers/chat/chat_providers.dart

Room summaries, unread RPC, messages, reactions. This file (chat providers) is one concrete piece of that layer.

Line count: 802. Import count: 12. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'dart:typed_data';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/services/app_cache.dart';
- import 'package:mu_study/core/services/cache_service.dart';
- import 'package:mu_study/core/services/chat_media_cache_service.dart';
- import 'package:mu_study/core/services/voice_cache_service.dart';
- import 'package:mu_study/core/utils/voice_playback.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/chat/chat_providers.dart**

```
import 'dart:async';
import 'dart:typed_data';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/services/app_cache.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/core/services/voice_cache_service.dart';
import 'package:mu_study/core/utils/voice_playback.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const chatSignedUrlTtlSeconds = 60 * 60 * 24 * 365 * 10;
const _pageSize = 40;
const _chatMessageSelect =
    '*, profiles(full_name, avatar_url, username, role), message_reads(user_id),
message_reactions(id, message_id, user_id, emoji), reply_to:reply_to_message_id(id,
message_type, text_content, media_url, is_deleted, sender_id, profiles(full_name, username,
avatar_url))';

const chatReactionEmojis = ['❤■', '■', '■', '■', '■', '■'];

String chatBucketForType(ChatMessageType type) => switch (type) {
  ChatMessageType.image => 'chat-images',
  ChatMessageType.video => 'chat-videos',
  ChatMessageType.voice => 'chat-voices',
  ChatMessageType.file => 'chat-files',
  ChatMessageType.text => 'chat-images',
};
```

```
String chatContentType(ChatMessageType type, String fileName) {
  final lower = fileName.toLowerCase();
  return switch (type) {
    ChatMessageType.image =>
      lower.endsWith('.png')
          ? 'image/png'
          : lower.endsWith('.webp')
          ? 'image/webp'
          : lower.endsWith('.gif')
          ? 'image/gif'
          : 'image/jpeg',
    ChatMessageType.video =>
      lower.endsWith('.webm')
          ? 'video/webm'
          : lower.endsWith('.mov')
          ? 'video/quicktime'
          : 'video/mp4',
    ChatMessageType.voice =>
      lower.endsWith('.wav')
          ? 'audio/wav'
          : lower.endsWith('.mp3')
          ? 'audio/mpeg'
          : 'audio/mp4',
    ChatMessageType.file => _fileContentType(lower),
    ChatMessageType.text => 'text/plain',
  };
}

String _fileContentType(String lowerName) {
  if (lowerName.endsWith('.pdf')) return 'application/pdf';
  if (lowerName.endsWith('.zip')) return 'application/zip';
  if (lowerName.endsWith('.json')) return 'application/json';
  if (lowerName.endsWith('.csv')) return 'text/csv';
  if (lowerName.endsWith('.txt') ||
      lowerName.endsWith('.md') ||
      lowerName.endsWith('.py') ||
      lowerName.endsWith('.dart') ||
      lowerName.endsWith('.js') ||
      lowerName.endsWith('.ts') ||
      lowerName.endsWith('.html') ||
      lowerName.endsWith('.css') ||
      lowerName.endsWith('.sql')) {
    return 'text/plain';
  }
  if (lowerName.endsWith('.doc') || lowerName.endsWith('.docx')) {
    return 'application/msword';
  }
  if (lowerName.endsWith('.xls') || lowerName.endsWith('.xlsx')) {
    return 'application/vnd.ms-excel';
  }
  if (lowerName.endsWith('.ppt') || lowerName.endsWith('.pptx')) {
    return 'application/vnd.ms-powerpoint';
  }
  return 'application/octet-stream';
}

String chatVoiceContentType(Uint8List bytes, String fileName) {
  return detectAudioFormat(bytes).mimeType;
}

String chatVoiceFileName(Uint8List bytes, String fallbackName) {
  final format = detectAudioFormat(bytes);
  final stamp = DateTime.now().millisecondsSinceEpoch;
  final base = fallbackName.replaceAll(RegExp(r'[^\w.\-]+'), '_');
  if (base.toLowerCase().endsWith('.${format.extension}')) return base;
  final withoutExt = base.contains('.')
      ? base.substring(0, base.lastIndexOf('.'))
      : base;
  final safe = withoutExt.isEmpty ? 'chat_voice_$stamp' : withoutExt;
  return '$safe.${format.extension}';
}

final chatRoomsProvider = StreamProvider<List<ChatRoomSummary>>((ref) async* {
```

```
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null) {
    yield const [];
    return;
  }

  Future<List<ChatRoomSummary>> load() async {
    final rows = await supabase
        .from('chat_room_summaries')
        .select()
        .order('last_message_at', ascending: false, nullsFirst: false);

    final unreadRows =
        await supabase.rpc('chat_unread_counts') as List<dynamic>? ?? [];
    final unreadMap = <String, int>{};
    for (final row in unreadRows) {
      final map = Map<String, dynamic>.from(row as Map);
      final roomId = map['room_id'] as String?;
      if (roomId == null) continue;
      unreadMap[roomId] = (map['unread_count'] as num?)?.toInt() ?? 0;
    }

    final rooms = (rows as List)
        .map((e) => ChatRoomSummary.fromJson(Map<String, dynamic>.from(e)))
        .map((r) => r.copyWith(unreadCount: unreadMap[r.id] ?? 0))
        .toList();

    rooms.sort((a, b) {
      final aTime =
          a.lastMessageAt ??
          a.createdAt ??
          DateTime.fromMillisecondsSinceEpoch(0);
      final bTime =
          b.lastMessageAt ??
          b.createdAt ??
          DateTime.fromMillisecondsSinceEpoch(0);
      return bTime.compareTo(aTime);
    });
    return rooms;
  }

  final changes = StreamController<void>();
  final channel = supabase
      .channel('chat-rooms-${profile.id}')
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'chat_messages',
        callback: (_) {
          if (!changes.isClosed) changes.add(null);
        },
      )
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'message_reads',
        callback: (_) {
          if (!changes.isClosed) changes.add(null);
        },
      )
      .subscribe();

  ref.onDispose(() {
    supabase.removeChannel(channel);
    changes.close();
  });

  yield await load();
  await for (final _ in changes.stream) {
    yield await load();
  }
});
```

```
final unreadChatCountProvider = Provider<int>((ref) {
  final rooms = ref.watch(chatRoomsProvider);
  return rooms.maybeWhen(
    data: (list) => list.fold<int>(0, (sum, r) => sum + r.unreadCount),
    orElse: () => 0,
  );
});

final chatRoomProvider = FutureProvider.family<ChatRoomSummary?, String>((
  ref,
  roomId,
) async {
  final rooms = ref.watch(chatRoomsProvider).valueOrNull;
  if (rooms != null) {
    for (final room in rooms) {
      if (room.id == roomId) return room;
    }
  }
  final row = await supabase
      .from('chat_room_summaries')
      .select()
      .eq('id', roomId)
      .maybeSingle();
  if (row == null) return null;
  return ChatRoomSummary.fromJson(Map<String, dynamic>.from(row));
});

class ChatMessagesState {
  const ChatMessagesState({
    required this.messages,
    this.hasMore = false,
    this.loadingMore = false,
  });

  final List<ChatMessage> messages;
  final bool hasMore;
  final bool loadingMore;

  ChatMessagesState copyWith({
    List<ChatMessage>? messages,
    bool? hasMore,
    bool? loadingMore,
  }) {
    return ChatMessagesState(
      messages: messages ?? this.messages,
      hasMore: hasMore ?? this.hasMore,
      loadingMore: loadingMore ?? this.loadingMore,
    );
  }
}

class ChatMessagesNotifier
    extends FamilyAsyncNotifier<ChatMessagesState, String> {
  RealtimeChannel? _channel;
  final _profileCache = <String, Map<String, dynamic>>{};

  @override
  Future<ChatMessagesState> build(String roomId) async {
    final profile = await ref.watch(currentProfileProvider.future);
    if (profile == null) {
      return const ChatMessagesState(messages: []);
    }

    final cache = ref.read(cacheServiceProvider).valueOrNull;
    List<ChatMessage>? cachedMessages;
    try {
      final raw = cache?.cachedChatMessages(roomId);
      if (raw != null && raw.isNotEmpty) {
        cachedMessages = raw.map(ChatMessage.fromJson).toList();
      }
    } catch (_) {}

    try {
```

```
final initial = await _fetchPage(roomId, before: null);
await _persistMessages(roomId, initial.items);
prefetchChatMedia(initial.items);

_channel = supabase
    .channel('chat-room-$roomId')
    .onPostgresChanges(
      event: PostgresChangeEvent.insert,
      schema: 'public',
      table: 'chat_messages',
      filter: PostgresChangeFilter(
        type: PostgresChangeFilterType.eq,
        column: 'room_id',
        value: roomId,
      ),
      callback: (payload) => _onInsert(payload.newRecord),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.update,
      schema: 'public',
      table: 'chat_messages',
      filter: PostgresChangeFilter(
        type: PostgresChangeFilterType.eq,
        column: 'room_id',
        value: roomId,
      ),
      callback: (payload) => _onUpdate(payload.newRecord),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.delete,
      schema: 'public',
      table: 'chat_messages',
      filter: PostgresChangeFilter(
        type: PostgresChangeFilterType.eq,
        column: 'room_id',
        value: roomId,
      ),
      callback: (payload) => _onDelete(payload.oldRecord),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.insert,
      schema: 'public',
      table: 'message_reads',
      callback: (payload) => _onReadInsert(payload.newRecord),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.insert,
      schema: 'public',
      table: 'message_reactions',
      callback: (payload) => _onReactionUpsert(payload.newRecord),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.update,
      schema: 'public',
      table: 'message_reactions',
      callback: (payload) => _onReactionUpsert(payload.newRecord),
    )
    .onPostgresChanges(
      event: PostgresChangeEvent.delete,
      schema: 'public',
      table: 'message_reactions',
      callback: (payload) => _onReactionDelete(payload.oldRecord),
    )
    .subscribe();

ref.onDispose(() {
  final ch = _channel;
  if (ch != null) supabase.removeChannel(ch);
});

return ChatMessagesState(
  messages: initial.items,
  hasMore: initial.hasMore,
```

```
      );
    } catch (_) {
      if (cachedMessages != null) {
        prefetchChatMedia(cachedMessages);
        return ChatMessagesState(messages: cachedMessages, hasMore: true);
      }
      rethrow;
    }
  }

  Future<void> _persistMessages(
    String roomId,
    List<ChatMessage> messages,
  ) async {
    final cache = ref.read(cacheServiceProvider).valueOrNull;
    if (cache == null) return;
    try {
      await cache.cacheChatMessages(
        roomId,
        messages.map((m) => m.toCacheJson()).toList(),
      );
    } catch (_) {}
  }

  Future<void> _persistCurrent() async {
    final current = state.valueOrNull;
    if (current == null) return;
    await _persistMessages(arg, current.messages);
  }

  Future<({List<ChatMessage> items, bool hasMore})> _fetchPage(
    String roomId, {
    required DateTime? before,
  }) async {
    var filtered = supabase
        .from('chat_messages')
        .select(_chatMessageSelect)
        .eq('room_id', roomId);

    if (before != null) {
      filtered = filtered.lt('created_at', before.toIso8601String());
    }

    final rows = await filtered
        .order('created_at', ascending: false)
        .limit(_pageSize + 1);
    final list = (rows as List)
        .map((e) => ChatMessage.fromJson(Map<String, dynamic>.from(e)))
        .toList();
    final hasMore = list.length > _pageSize;
    final page = hasMore ? list.sublist(0, _pageSize) : list;
    return (items: page.reversed.toList(), hasMore: hasMore);
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    if (current.messages.isEmpty) return;

    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final oldest = current.messages.first.createdAt;
      final page = await _fetchPage(arg, before: oldest);
      final merged = [...page.items, ...current.messages];
      state = AsyncData(
        ChatMessagesState(
          messages: merged,
          hasMore: page.hasMore,
          loadingMore: false,
        ),
      );
      await _persistMessages(arg, merged);
      prefetchChatMedia(page.items);
```

```
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
    }
  }

  Future<void> _hydrateSender(Map<String, dynamic> row) async {
    final senderId = row['sender_id'] as String?;
    if (senderId == null || row['profiles'] != null) return;
    final cached = _profileCache[senderId];
    if (cached != null) {
      row['profiles'] = cached;
      return;
    }
    final profileRow = await supabase
        .from('profiles')
        .select('full_name, avatar_url, username, role')
        .eq('id', senderId)
        .maybeSingle();
    if (profileRow != null) {
      final map = Map<String, dynamic>.from(profileRow);
      _profileCache[senderId] = map;
      row['profiles'] = map;
    }
  }

  Future<void> _hydrateReply(Map<String, dynamic> row) async {
    final replyId = row['reply_to_message_id'] as String?;
    if (replyId == null || row['reply_to'] != null) return;

    final current = state.valueOrNull;
    final local = current?.messages.where((m) => m.id == replyId).firstOrNull;
    if (local != null) {
      row['reply_to'] = ChatReplyPreview.fromMessage(local).toCacheJson();
      return;
    }

    try {
      final data = await supabase
          .from('chat_messages')
          .select(
            'id, message_type, text_content, media_url, is_deleted, sender_id, profiles(
full_name, username, avatar_url)',
          )
          .eq('id', replyId)
          .maybeSingle();
```

**lib/providers/chat/chat_providers.dart — continued (2/2)**

```
      if (data != null) {
        row['reply_to'] = Map<String, dynamic>.from(data);
      }
    } catch (_) {}
  }

  Future<void> _onInsert(Map<String, dynamic> record) async {
    final row = Map<String, dynamic>.from(record);
    await _hydrateSender(row);
    await _hydrateReply(row);
    final message = ChatMessage.fromJson(row);
    final current = state.valueOrNull;
    if (current == null) return;
    if (current.messages.any((m) => m.id == message.id)) return;
    state = AsyncData(
      current.copyWith(messages: [...current.messages, message]),
    );
    prefetchChatMedia([message]);
    await _persistCurrent();
  }

  Future<void> _onUpdate(Map<String, dynamic> record) async {
    final row = Map<String, dynamic>.from(record);
    await _hydrateSender(row);
    final updated = ChatMessage.fromJson(row);
```

```
  final current = state.valueOrNull;
  if (current == null) return;
  final next = current.messages.map((m) {
    if (m.id != updated.id) return m;
    return updated.copyWith(
      senderName: updated.senderName ?? m.senderName,
      senderUsername: updated.senderUsername ?? m.senderUsername,
      senderAvatarUrl: updated.senderAvatarUrl ?? m.senderAvatarUrl,
      senderRole: updated.senderRole ?? m.senderRole,
      isReadByOthers: m.isReadByOthers || updated.isReadByOthers,
      reactions: updated.reactions.isEmpty ? m.reactions : updated.reactions,
      replyToMessageId: updated.replyToMessageId ?? m.replyToMessageId,
      replyTo: updated.replyTo ?? m.replyTo,
    );
  }).toList();
  state = AsyncData(current.copyWith(messages: next));
  await _persistCurrent();
}

void _onDelete(Map<String, dynamic> record) {
  final id = record['id'] as String?;
  if (id == null) return;
  final current = state.valueOrNull;
  if (current == null) return;
  state = AsyncData(
    current.copyWith(
      messages: current.messages.where((m) => m.id != id).toList(),
    ),
  );
  unawaited(_persistCurrent());
}

void _onReadInsert(Map<String, dynamic> record) {
  final messageId = record['message_id'] as String?;
  final readerId = record['user_id'] as String?;
  if (messageId == null || readerId == null) return;
  final current = state.valueOrNull;
  if (current == null) return;

  var changed = false;
  final next = current.messages.map((m) {
    if (m.id != messageId || m.isReadByOthers) return m;
    if (readerId == m.senderId) return m;
    changed = true;
    return m.copyWith(isReadByOthers: true);
  }).toList();

  if (changed) {
    state = AsyncData(current.copyWith(messages: next));
  }
}

void _onReactionUpsert(Map<String, dynamic> record) {
  final reaction = ChatReaction.fromJson(Map<String, dynamic>.from(record));
  final current = state.valueOrNull;
  if (current == null) return;
  if (!current.messages.any((m) => m.id == reaction.messageId)) return;

  final next = current.messages.map((m) {
    if (m.id != reaction.messageId) return m;
    final withoutUser = m.reactions
        .where((r) => r.userId != reaction.userId)
        .toList();
    return m.copyWith(reactions: [...withoutUser, reaction]);
  }).toList();
  state = AsyncData(current.copyWith(messages: next));
}

void _onReactionDelete(Map<String, dynamic> record) {
  final id = record['id'] as String?;
  final messageId = record['message_id'] as String?;
  final userId = record['user_id'] as String?;
  final current = state.valueOrNull;
```

```
    if (current == null) return;

    final next = current.messages.map((m) {
      if (messageId != null && m.id != messageId) return m;
      if (messageId == null &&
          id != null &&
          !m.reactions.any((r) => r.id == id)) {
        return m;
      }
      return m.copyWith(
        reactions: m.reactions.where((r) {
          if (id != null && r.id == id) return false;
          if (userId != null &&
              messageId != null &&
              r.userId == userId &&
              r.messageId == messageId) {
            return false;
          }
          return true;
        }).toList(),
      );
    }).toList();
    state = AsyncData(current.copyWith(messages: next));
  }

  Future<ChatMessage> sendText(String text, {String? replyToMessageId}) async {
    final profile = await ref.read(currentProfileProvider.future);
    if (profile == null) throw StateError('Not signed in');
    final trimmed = text.trim();
    if (trimmed.isEmpty) throw ArgumentError('Empty message');

    final row = await supabase
        .from('chat_messages')
        .insert({
          'room_id': arg,
          'sender_id': profile.id,
          'message_type': 'text',
          'text_content': trimmed,
          'reply_to_message_id': ?replyToMessageId,
        })
        .select(_chatMessageSelect)
        .single();

    final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));
    final current = state.valueOrNull;
    if (current != null && !current.messages.any((m) => m.id == message.id)) {
      state = AsyncData(
        current.copyWith(messages: [...current.messages, message]),
      );
      await _persistCurrent();
    }
    return message;
  }

  Future<ChatMessage> sendMedia({
    required ChatMessageType type,
    required Uint8List bytes,
    required String fileName,
    String? caption,
    String? replyToMessageId,
    void Function(double progress)? onProgress,
  }) async {
    final profile = await ref.read(currentProfileProvider.future);
    if (profile == null) throw StateError('Not signed in');
    if (type == ChatMessageType.text) {
      throw ArgumentError('Use sendText for text messages');
    }

    onProgress?.call(0.05);
    final stamp = DateTime.now().millisecondsSinceEpoch;
    final safeName = type == ChatMessageType.voice
        ? chatVoiceFileName(bytes, fileName)
        : fileName.replaceAll(RegExp(r'[^\w.\-]+'), '_');
```

```
    final path = '$arg/${profile.id}/${stamp}_$safeName';
    final bucket = chatBucketForType(type);
    final contentType = type == ChatMessageType.voice
        ? chatVoiceContentType(bytes, fileName)
        : chatContentType(type, fileName);

    await supabase.storage
        .from(bucket)
        .uploadBinary(
          path,
          bytes,
          fileOptions: FileOptions(upsert: false, contentType: contentType),
        );
    onProgress?.call(0.7);

    final url = await supabase.storage
        .from(bucket)
        .createSignedUrl(path, chatSignedUrlTtlSeconds);
    onProgress?.call(0.85);

    if (type == ChatMessageType.voice) {
      final format = detectAudioFormat(bytes);
      await VoiceCacheService.instance.put(url, bytes, format.extension);
    } else {
      final ext = ChatMediaCacheService.extensionForUrl(
        url,
        fallback: safeName.contains('.') ? safeName.split('.').last : 'bin',
      );
      await ChatMediaCacheService.instance.put(url, bytes, ext);
    }

    final captionText = caption?.trim();
    final row = await supabase
        .from('chat_messages')
        .insert({
          'room_id': arg,
          'sender_id': profile.id,
          'message_type': chatMessageTypeToDb(type),
          'text_content': (captionText == null || captionText.isEmpty)
              ? null
              : captionText,
          'media_url': url,
          'reply_to_message_id': ?replyToMessageId,
        })
        .select(_chatMessageSelect)
        .single();
    onProgress?.call(1);

    final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));
    final current = state.valueOrNull;
    if (current != null && !current.messages.any((m) => m.id == message.id)) {
      state = AsyncData(
        current.copyWith(messages: [...current.messages, message]),
      );
      await _persistCurrent();
    }
    return message;
  }

  Future<void> editMessage(String messageId, String newText) async {
    final trimmed = newText.trim();
    if (trimmed.isEmpty) throw ArgumentError('Empty message');
    await supabase
        .from('chat_messages')
        .update({'text_content': trimmed})
        .eq('id', messageId);
  }

  Future<void> deleteMessage(String messageId) async {
    await deleteMessages([messageId]);
  }

  Future<void> deleteMessages(Iterable<String> messageIds) async {
```

```
    final ids = messageIds.toList();
    if (ids.isEmpty) return;

    final current = state.valueOrNull;
    if (current != null) {
      final idSet = ids.toSet();
      state = AsyncData(
        current.copyWith(
          messages: [
            for (final m in current.messages)
              if (idSet.contains(m.id))
                m.copyWith(isDeleted: true)
              else
                m,
          ],
        ),
      );
      await _persistCurrent();
    }

    await supabase.from('chat_messages').update({
      'is_deleted': true,
      'text_content': null,
      'media_url': null,
    }).inFilter('id', ids);
  }

  Future<void> toggleReaction(String messageId, String emoji) async {
    final profile = await ref.read(currentProfileProvider.future);
    if (profile == null) throw StateError('Not signed in');

    final current = state.valueOrNull;
    ChatMessage? message;
    if (current != null) {
      for (final m in current.messages) {
        if (m.id == messageId) {
          message = m;
          break;
        }
      }
    }
    final existing = message?.myReactionEmoji(profile.id);

    if (current != null && message != null) {
      final nextReactions = <ChatReaction>[
        for (final r in message.reactions)
          if (r.userId != profile.id) r,
      ];
      if (existing != emoji) {
        nextReactions.add(
          ChatReaction(
            id: 'local-${profile.id}',
            messageId: messageId,
            userId: profile.id,
            emoji: emoji,
          ),
        );
      }
      state = AsyncData(
        current.copyWith(
          messages: [
            for (final m in current.messages)
              if (m.id == messageId)
                m.copyWith(reactions: nextReactions)
              else
                m,
          ],
        ),
      );
    }

    await supabase.rpc(
      'toggle_message_reaction',
```

```
      params: {'p_message_id': messageId, 'p_emoji': emoji},
    );
  }

  Future<void> markRoomRead() async {
    final profile = await ref.read(currentProfileProvider.future);
    if (profile == null) return;

    final unread = await supabase
        .from('chat_messages')
        .select('id')
        .eq('room_id', arg)
        .eq('is_deleted', false)
        .neq('sender_id', profile.id);

    final ids = (unread as List)
        .map((e) => (e as Map)['id'] as String)
        .toList();
    if (ids.isEmpty) return;

    final existing = await supabase
        .from('message_reads')
        .select('message_id')
        .eq('user_id', profile.id)
        .inFilter('message_id', ids);
    final already = {
      for (final e in existing as List) (e as Map)['message_id'] as String,
    };
    final toInsert = ids
        .where((id) => !already.contains(id))
        .map((id) => {'message_id': id, 'user_id': profile.id})
        .toList();
    if (toInsert.isEmpty) return;

    await supabase
        .from('message_reads')
        .upsert(toInsert, onConflict: 'message_id,user_id');
    ref.invalidate(chatRoomsProvider);
  }
}

final chatMessagesProvider =
    AsyncNotifierProvider.family<
      ChatMessagesNotifier,
      ChatMessagesState,
      String
    >(ChatMessagesNotifier.new);
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/chat/chat_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/providers/focus/focus_session_provider.dart

Local Pomodoro/free timer persisted in SharedPreferences. This file (focus session provider) is one concrete piece of that layer.

Line count: 245. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'dart:convert';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/focus/focus_time_up_cue.dart';
- import 'package:shared_preferences/shared_preferences.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/focus/focus_session_provider.dart**

```
import 'dart:async';
import 'dart:convert';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/focus/focus_time_up_cue.dart';
import 'package:shared_preferences/shared_preferences.dart';

const _prefsKey = 'focus_session_v1';

enum FocusTimerMode { pomodoro, free }

const kFocusPomodoroDuration = Duration(minutes: 25);

class FocusSession {
  const FocusSession({
    required this.assignmentId,
    required this.assignmentTitle,
    required this.mode,
    required this.startedAt,
    this.accumulated = Duration.zero,
    this.pausedAt,
  });

  final String assignmentId;
  final String assignmentTitle;
  final FocusTimerMode mode;
  final DateTime startedAt;

  final Duration accumulated;
  final DateTime? pausedAt;

  bool get isPaused => pausedAt != null;

  Duration elapsedAt([DateTime? now]) {
    final n = now ?? DateTime.now();
    if (isPaused) return accumulated;
    return accumulated + n.difference(startedAt);
  }

  Duration? remainingAt([DateTime? now]) {
    if (mode != FocusTimerMode.pomodoro) return null;
    final left = kFocusPomodoroDuration - elapsedAt(now);
```

```
    return left.isNegative ? Duration.zero : left;
  }

  bool isCompleteAt([DateTime? now]) {
    final left = remainingAt(now);
    return left != null && left == Duration.zero;
  }

  Map<String, dynamic> toJson() => {
    'assignmentId': assignmentId,
    'assignmentTitle': assignmentTitle,
    'mode': mode.name,
    'startedAt': startedAt.toIso8601String(),
    'accumulatedMs': accumulated.inMilliseconds,
    'pausedAt': pausedAt?.toIso8601String(),
  };

  factory FocusSession.fromJson(Map<String, dynamic> json) {
    return FocusSession(
      assignmentId: json['assignmentId'] as String,
      assignmentTitle: json['assignmentTitle'] as String? ?? '',
      mode: FocusTimerMode.values.firstWhere(
        (m) => m.name == json['mode'],
        orElse: () => FocusTimerMode.free,
      ),
      startedAt: DateTime.parse(json['startedAt'] as String),
      accumulated: Duration(milliseconds: json['accumulatedMs'] as int? ?? 0),
      pausedAt: json['pausedAt'] != null
          ? DateTime.parse(json['pausedAt'] as String)
          : null,
    );
  }

  FocusSession copyWith({
    DateTime? startedAt,
    Duration? accumulated,
    DateTime? pausedAt,
    bool clearPausedAt = false,
  }) {
    return FocusSession(
      assignmentId: assignmentId,
      assignmentTitle: assignmentTitle,
      mode: mode,
      startedAt: startedAt ?? this.startedAt,
      accumulated: accumulated ?? this.accumulated,
      pausedAt: clearPausedAt ? null : (pausedAt ?? this.pausedAt),
    );
  }
}

class FocusSummary {
  const FocusSummary({
    required this.assignmentId,
    required this.assignmentTitle,
    required this.focused,
    required this.mode,
    this.note,
    this.timedOut = false,
  });

  final String assignmentId;
  final String assignmentTitle;
  final Duration focused;
  final FocusTimerMode mode;
  final String? note;

  final bool timedOut;

  FocusSummary copyWith({String? note}) => FocusSummary(
    assignmentId: assignmentId,
    assignmentTitle: assignmentTitle,
    focused: focused,
    mode: mode,
```

```
      note: note ?? this.note,
      timedOut: timedOut,
    );
  }

  class FocusSessionState {
    const FocusSessionState({this.session, this.summary});

    final FocusSession? session;
    final FocusSummary? summary;

    bool get isActive => session != null;
    bool get showingSummary => summary != null && session == null;

    FocusSessionState copyWith({
      FocusSession? session,
      FocusSummary? summary,
      bool clearSession = false,
      bool clearSummary = false,
    }) {
      return FocusSessionState(
        session: clearSession ? null : (session ?? this.session),
        summary: clearSummary ? null : (summary ?? this.summary),
      );
    }
  }

  final focusSessionProvider =
      StateNotifierProvider<FocusSessionController, FocusSessionState>((ref) {
        return FocusSessionController();
      });

  final focusChatMutedProvider = Provider<bool>((ref) {
    return ref.watch(focusSessionProvider).isActive;
  });

  class FocusSessionController extends StateNotifier<FocusSessionState> {
    FocusSessionController() : super(const FocusSessionState()) {
      _restore();
    }

    Future<void> _restore() async {
      final prefs = await SharedPreferences.getInstance();
      final raw = prefs.getString(_prefsKey);
      if (raw == null || raw.isEmpty) return;
      try {
        final map = jsonDecode(raw) as Map<String, dynamic>;
        final session = FocusSession.fromJson(map);
        if (!mounted) return;
        if (session.isCompleteAt()) {
          await endSession(timedOut: true, playCue: false);
          return;
        }
        state = FocusSessionState(session: session);
      } catch (_) {
        await prefs.remove(_prefsKey);
      }
    }

    Future<void> _persist(FocusSession? session) async {
      final prefs = await SharedPreferences.getInstance();
      if (session == null) {
        await prefs.remove(_prefsKey);
      } else {
        await prefs.setString(_prefsKey, jsonEncode(session.toJson()));
      }
    }

    Future<void> start({
      required String assignmentId,
      required String assignmentTitle,
      required FocusTimerMode mode,
    }) async {
```

```
    final session = FocusSession(
      assignmentId: assignmentId,
      assignmentTitle: assignmentTitle,
      mode: mode,
      startedAt: DateTime.now(),
    );
    state = FocusSessionState(session: session);
    await _persist(session);
  }

  Future<void> pause() async {
    final s = state.session;
    if (s == null || s.isPaused) return;
    final now = DateTime.now();
    final next = s.copyWith(accumulated: s.elapsedAt(now), pausedAt: now);
    state = state.copyWith(session: next);
    await _persist(next);
  }

  Future<void> resume() async {
    final s = state.session;
    if (s == null || !s.isPaused) return;
    final next = s.copyWith(startedAt: DateTime.now(), clearPausedAt: true);
    state = state.copyWith(session: next);
    await _persist(next);
  }

  Future<void> endSession({bool timedOut = false, bool playCue = true}) async {
    final s = state.session;
    if (s == null) return;
    final focused = timedOut && s.mode == FocusTimerMode.pomodoro
        ? kFocusPomodoroDuration
        : s.elapsedAt();
    final summary = FocusSummary(
      assignmentId: s.assignmentId,
      assignmentTitle: s.assignmentTitle,
      focused: focused,
      mode: s.mode,
      timedOut: timedOut,
    );
    state = FocusSessionState(summary: summary);
    await _persist(null);
    if (timedOut && playCue) {
      await playFocusTimeUpCue();
    }
  }

  void setSummaryNote(String note) {
    final summary = state.summary;
    if (summary == null) return;
    state = state.copyWith(summary: summary.copyWith(note: note));
  }

  void dismissSummary() {
    state = const FocusSessionState();
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/focus/focus_session_provider.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/groups/group_providers.dart

Teacher-owned and student-membership groups. This file (group providers) is one concrete piece of that layer.

Line count: 69. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/providers/groups/group_providers.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final myGroupsProvider = FutureProvider<List<StudyGroup>>((ref) async {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null) return [];

  if (profile.role == UserRole.teacher) {
    final teacher = await ref.watch(teacherProfileProvider.future);
    if (teacher == null) return [];
    final rows = await supabase
        .from('groups')
        .select()
        .eq('teacher_id', teacher.id)
        .order('title');
    return rows
        .map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))
        .toList();
  }

  if (profile.role == UserRole.student) {
    final student = await ref.watch(studentProfileProvider.future);
    if (student == null) return [];
    final rows = await supabase
        .from('group_students')
        .select('groups(*)')
        .eq('student_id', student.id);
    return rows
        .map(
          (e) => StudyGroup.fromJson(
            Map<String, dynamic>.from(e['groups'] as Map),
          ),
        )
        .toList();
  }

  final rows = await supabase.from('groups').select().order('title');
  return rows
      .map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))
      .toList();
});
```

```
final selectedGroupIdProvider = StateProvider<String?>((ref) => null);

final activeGroupIdProvider = Provider<String?>((ref) {
  final selected = ref.watch(selectedGroupIdProvider);
  final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];
  if (selected != null && groups.any((g) => g.id == selected)) return selected;
  return groups.isEmpty ? null : groups.first.id;
});

final groupMembersProvider = FutureProvider.family<List<GroupMember>, String>((
  ref,
  groupId,
) async {
  final rows = await supabase
      .from('group_students')
      .select('students(id, profiles(full_name, username))')
      .eq('group_id', groupId);
  final members = rows
      .map((e) => GroupMember.fromGroupStudentRow(Map<String, dynamic>.from(e)))
      .toList();
  members.sort(
    (a, b) => a.fullName.toLowerCase().compareTo(b.fullName.toLowerCase()),
  );
  return members;
});
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/groups/group_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/homework/assignment_providers.dart

Assignments, tasks, teacher submissions queue. This file (assignment providers) is one concrete piece of that layer.

Line count: 112. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/paging/page_result.dart';
- import 'package:mu_study/core/services/cache_service.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/providers/homework/assignment_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/paging/page_result.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final assignmentsProvider =
    AsyncNotifierProvider.family<
      AssignmentsNotifier,
      PageResult<Assignment>,
      String?
    >(AssignmentsNotifier.new);

class AssignmentsNotifier
    extends FamilyAsyncNotifier<PageResult<Assignment>, String?> {
  @override
  Future<PageResult<Assignment>> build(String? groupId) => _fetch(0);

  Future<PageResult<Assignment>> _fetch(int offset) async {
    final cache = ref.watch(cacheServiceProvider).valueOrNull;
    final limit = PageResult.fetchSize(PageResult.pageSize);

    try {
      var query = supabase.from('assignments').select();
      if (arg != null) query = query.eq('group_id', arg!);
      final rows = await query
          .order('created_at', ascending: false)
          .range(offset, offset + limit - 1);
      final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();
      if (offset == 0) await cache?.cacheAssignments(list);
      return PageResult.fromFetch(list.map(Assignment.fromJson).toList());
    } catch (_) {
      if (offset == 0) {
        final cached = cache?.cachedAssignments;
        if (cached != null) {
          return PageResult(
```

```
            items: cached.map(Assignment.fromJson).toList(),
            hasMore: false,
          );
        }
      }
      rethrow;
    }
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}

final assignmentDetailProvider = FutureProvider.family<Assignment?, String>((
  ref,
  id,
) async {
  final data = await supabase
      .from('assignments')
      .select()
      .eq('id', id)
      .maybeSingle();
  if (data == null) return null;
  return Assignment.fromJson(Map<String, dynamic>.from(data));
});

final tasksProvider = FutureProvider.family<List<TaskItem>, String>((
  ref,
  assignmentId,
) async {
  final rows = await supabase
      .from('tasks')
      .select()
      .eq('assignment_id', assignmentId)
      .order('sort_order');
  return rows
      .map((e) => TaskItem.fromJson(Map<String, dynamic>.from(e)))
      .toList();
});

final assignmentProgressProvider = FutureProvider.family<double, String>((
  ref,
  assignmentId,
) async {
  final student = await ref.watch(studentProfileProvider.future);
  if (student == null) return 0.0;

  final tasks = await supabase
      .from('tasks')
      .select('id')
      .eq('assignment_id', assignmentId);
  if (tasks.isEmpty) return 0.0;

  final taskIds = tasks.map((t) => t['id'] as String).toList();
  final completed = await supabase
      .from('submissions')
      .select('task_id')
      .eq('student_id', student.id)
      .eq('is_completed', true)
      .inFilter('task_id', taskIds);

  return completed.length / tasks.length;
});
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/homework/assignment_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/providers/homework/teacher_submissions_provider.dart

Assignments, tasks, teacher submissions queue. This file (teacher submissions provider) is one concrete piece of that layer.

Line count: 54. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/homework/teacher_submissions_provider.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final teacherSubmissionsProvider = FutureProvider<List<Map<String, dynamic>>>((
  ref,
) async {
  final teacher = await ref.watch(teacherProfileProvider.future);
  if (teacher == null) return [];

  final assignments = await supabase
      .from('assignments')
      .select('id')
      .eq('teacher_id', teacher.id);
  final assignmentIds = assignments.map((e) => e['id'] as String).toList();
  if (assignmentIds.isEmpty) return [];

  final tasks = await supabase
      .from('tasks')
      .select('id')
      .inFilter('assignment_id', assignmentIds);
  final taskIds = tasks.map((e) => e['id'] as String).toList();
  if (taskIds.isEmpty) return [];

  final rows = await supabase
      .from('submissions')
      .select('''
            id, status, score, feedback, text_answer, file_url, voice_url,
            submitted_at,
            tasks(title, assignments(title)),
            students(profiles(full_name, username))
          ''')
      .inFilter('task_id', taskIds)
      .order('submitted_at', ascending: false);

  return rows.map((s) {
    final task = s['tasks'] as Map?;
    final assignment = task?['assignments'] as Map?;
    final profile = (s['students'] as Map?)?['profiles'] as Map?;
    return {
      'id': s['id'],
      'status': s['status'],
      'score': s['score'],
      'feedback': s['feedback'],
      'text_answer': s['text_answer'],
```

```
        'file_url': s['file_url'],
        'voice_url': s['voice_url'],
        'task_title': task?['title'],
        'assignment_title': assignment?['title'],
        'student_name':
            profile?['full_name'] ?? profile?['username'] ?? 'Student',
      };
    }).toList();
  });
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/homework/teacher_submissions_provider.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/marks/marks_providers.dart

Gradebook CRUD and student averages. This file (marks providers) is one concrete piece of that layer.

Line count: 256. Import count: 8. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/paging/page_result.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/marks/marks_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/paging/page_result.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

Future<List<Map<String, dynamic>>> _fetchMarksPage({
  required int offset,
  String? groupId,
  String? studentId,
  AcademicTerm term = AcademicTerm.all,
  String select = '*, students(profiles(full_name, username))',
}) async {
  final limit = PageResult.fetchSize(PageResult.pageSize);
  var query = supabase.from('marks').select(select);
  if (groupId != null) query = query.eq('group_id', groupId);
  if (studentId != null) query = query.eq('student_id', studentId);
  final start = term.startKey;
  final end = term.endKey;
  if (start != null) query = query.gte('marked_on', start);
  if (end != null) query = query.lte('marked_on', end);

  final rows = await query
      .order('marked_on', ascending: false)
      .order('created_at', ascending: false)
      .range(offset, offset + limit - 1);
  return rows.map((e) => Map<String, dynamic>.from(e)).toList();
}

Map<String, double> _weightedAverages(List<Map<String, dynamic>> rows) {
  final totals = <String, double>{};
  final weights = <String, double>{};
  for (final row in rows) {
    final id = row['student_id'] as String;
    final pct = double.tryParse('${row['percentage']}') ?? 0;
```

```
    final w = double.tryParse('${row['weight']}') ?? 1;
    totals[id] = (totals[id] ?? 0) + pct * w;
    weights[id] = (weights[id] ?? 0) + w;
  }
  return {
    for (final id in totals.keys)
      id: weights[id] == 0 ? 0 : totals[id]! / weights[id]!,
  };
}

final groupMarksProvider =
    AsyncNotifierProvider.family<GroupMarksNotifier, PageResult<Mark>, String>(
      GroupMarksNotifier.new,
    );

class GroupMarksNotifier extends FamilyAsyncNotifier<PageResult<Mark>, String> {
  @override
  Future<PageResult<Mark>> build(String groupId) async {
    ref.watch(selectedMarkTermProvider);
    return _fetch(0);
  }

  Future<PageResult<Mark>> _fetch(int offset) async {
    final term = ref.read(selectedMarkTermProvider);
    final rows = await _fetchMarksPage(
      offset: offset,
      groupId: arg,
      term: term,
    );
    return PageResult.fromFetch(rows.map(Mark.fromJson).toList());
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}

final groupMarkAveragesProvider =
    FutureProvider.family<Map<String, double>, String>((ref, groupId) async {
      final term = ref.watch(selectedMarkTermProvider);
      var query = supabase
          .from('marks')
          .select('student_id, percentage, weight')
          .eq('group_id', groupId);
      final start = term.startKey;
      final end = term.endKey;
      if (start != null) query = query.gte('marked_on', start);
      if (end != null) query = query.lte('marked_on', end);
      final rows = await query;
      return _weightedAverages(
        rows.map((e) => Map<String, dynamic>.from(e)).toList(),
      );
    });

final groupMarkCountProvider = FutureProvider.family<int, String>((
  ref,
  groupId,
) async {
  final term = ref.watch(selectedMarkTermProvider);
  var query = supabase.from('marks').select('id').eq('group_id', groupId);
  final start = term.startKey;
  final end = term.endKey;
  if (start != null) query = query.gte('marked_on', start);
  if (end != null) query = query.lte('marked_on', end);
```

```
  final result = await query.count(CountOption.exact);
  return result.count;
});

final memberMarksProvider =
    FutureProvider.family<List<Mark>, ({String groupId, String studentId})>((
      ref,
      key,
    ) async {
      final term = ref.watch(selectedMarkTermProvider);
      var query = supabase
          .from('marks')
          .select('*, students(profiles(full_name, username))')
          .eq('group_id', key.groupId)
          .eq('student_id', key.studentId);
      final start = term.startKey;
      final end = term.endKey;
      if (start != null) query = query.gte('marked_on', start);
      if (end != null) query = query.lte('marked_on', end);
      final rows = await query
          .order('marked_on', ascending: false)
          .order('created_at', ascending: false);
      return rows
          .map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))
          .toList();
    });

final myMarksProvider =
    AsyncNotifierProvider<MyMarksNotifier, PageResult<Mark>>(
      MyMarksNotifier.new,
    );

class MyMarksNotifier extends AsyncNotifier<PageResult<Mark>> {
  @override
  Future<PageResult<Mark>> build() async {
    ref.watch(selectedMarkTermProvider);
    ref.watch(studentProfileProvider);
    return _fetch(0);
  }

  Future<PageResult<Mark>> _fetch(int offset) async {
    final student = await ref.read(studentProfileProvider.future);
    if (student == null) {
      return const PageResult(items: [], hasMore: false);
    }
    final term = ref.read(selectedMarkTermProvider);
    final rows = await _fetchMarksPage(
      offset: offset,
      studentId: student.id,
      term: term,
      select: '*, groups(title)',
    );
    return PageResult.fromFetch(rows.map(Mark.fromJson).toList());
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}

final myMarkStatsProvider =
    FutureProvider<
      List<({MarkCategory category, double percentage, double weight})>
    >((ref) async {
```

```
      final student = await ref.watch(studentProfileProvider.future);
      if (student == null) return [];
      final term = ref.watch(selectedMarkTermProvider);
      var query = supabase
          .from('marks')
          .select('category, percentage, weight')
          .eq('student_id', student.id);
      final start = term.startKey;
      final end = term.endKey;
      if (start != null) query = query.gte('marked_on', start);
      if (end != null) query = query.lte('marked_on', end);
      final rows = await query;
      return rows
          .map(
            (e) => (
              category: markCategoryFromString(e['category'] as String?),
              percentage: double.tryParse('${e['percentage']}') ?? 0,
              weight: double.tryParse('${e['weight']}') ?? 1,
            ),
          )
          .toList();
    });

final myMarkAverageProvider = FutureProvider<double?>((ref) async {
  final stats = await ref.watch(myMarkStatsProvider.future);
  if (stats.isEmpty) return null;
  var total = 0.0;
  var weight = 0.0;
  for (final s in stats) {
    total += s.percentage * s.weight;
    weight += s.weight;
  }
  if (weight == 0) return null;
  return total / weight;
});

Future<void> saveMark({
  required String groupId,
  required String studentId,
  required String teacherId,
  required String title,
  required MarkCategory category,
  required double percentage,
  required double weight,
  required String comment,
  required DateTime markedOn,
  String? markId,
}) async {
  final payload = Mark(
    id: markId ?? '',
    groupId: groupId,
    studentId: studentId,
    teacherId: teacherId,
    title: title,
    percentage: percentage,
    category: category,
    weight: weight,
    comment: comment,
    markedOn: markedOn,
  ).toWrite();

  if (markId == null) {
    await supabase.from('marks').insert(payload);
  } else {
    await supabase.from('marks').update(payload).eq('id', markId);
  }
}

Future<void> deleteMark(String markId) async {
  await supabase.from('marks').delete().eq('id', markId);
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/marks/marks_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/providers/notifications/notification_providers.dart

Realtime inbox + announcements. This file (notification providers) is one concrete piece of that layer.

Line count: 91. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';
- import 'package:mu_study/core/services/cache_service.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/providers/notifications/notification_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final notificationsProvider = StreamProvider<List<AppNotification>>((
  ref,
) async* {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null) {
    yield [];
    return;
  }

  final cache = ref.watch(cacheServiceProvider).valueOrNull;

  Future<List<AppNotification>> load() async {
    final rows = await supabase
        .from('notifications')
        .select()
        .eq('user_id', profile.id)
        .order('created_at', ascending: false)
        .limit(50);
    final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();
    await cache?.cacheNotifications(list);
    return list.map(AppNotification.fromJson).toList();
  }

  final changes = StreamController<void>();
  final channel = supabase
      .channel('notifications-${profile.id}')
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'notifications',
        filter: PostgresChangeFilter(
```

```
          type: PostgresChangeFilterType.eq,
          column: 'user_id',
          value: profile.id,
        ),
        callback: (_) {
          if (!changes.isClosed) changes.add(null);
        },
      )
      .subscribe();

  ref.onDispose(() {
    supabase.removeChannel(channel);
    changes.close();
  });

  yield await load();

  await for (final _ in changes.stream) {
    yield await load();
  }
});

final unreadNotificationsCountProvider = Provider<int>((ref) {
  final notifications = ref.watch(notificationsProvider);
  return notifications.maybeWhen(
    data: (list) => list.where((n) => !n.isRead).length,
    orElse: () => 0,
  );
});

final announcementsProvider =
    FutureProvider.family<List<Announcement>, String?>((ref, groupId) async {
      var query = supabase.from('announcements').select();
      if (groupId != null) query = query.eq('group_id', groupId);
      final rows = await query.order('created_at', ascending: false);
      return rows
          .map((e) => Announcement.fromJson(Map<String, dynamic>.from(e)))
          .toList();
    });

Future<void> markNotificationRead(String id) async {
  await supabase.from('notifications').update({'is_read': true}).eq('id', id);
}

Future<void> markAllNotificationsRead(String userId) async {
  await supabase
      .from('notifications')
      .update({'is_read': true})
      .eq('user_id', userId)
      .eq('is_read', false);
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/notifications/notification_providers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/settings/locale_provider.dart

Locale, theme mode, AppStyle. This file (locale provider) is one concrete piece of that layer.

Line count: 65. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/scheduler.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/locale/app_locale.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:shared_preferences/shared_preferences.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/providers/settings/locale_provider.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/scheduler.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/locale/app_locale.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

const _prefsKey = 'preferred_language';

final localeProvider = StateNotifierProvider<LocaleController, Locale>((ref) {
  final controller = LocaleController(ref);
  ref.listen(currentProfileProvider, (prev, next) {
    next.whenData((profile) {
      final code = profile?.preferredLanguage;
      if (code != null) controller.applyFromProfile(code);
    });
  });
  return controller;
});

class LocaleController extends StateNotifier<Locale> {
  LocaleController(this._ref) : super(AppLanguage.english.locale) {
    _loadLocal();
  }

  final Ref _ref;

  Future<void> _loadLocal() async {
    final prefs = await SharedPreferences.getInstance();
    final code = prefs.getString(_prefsKey);
    final lang = AppLanguage.tryParse(code);
    if (lang != null) state = lang.locale;
  }

  void applyFromProfile(String code) {
    final lang = AppLanguage.tryParse(code);
    if (lang == null) return;
    if (state.languageCode == lang.code) return;
    state = lang.locale;
    SharedPreferences.getInstance().then(
      (prefs) => prefs.setString(_prefsKey, lang.code),
    );
```

```
  }

  Future<void> setLanguage(AppLanguage language) async {
    // `!semantics.parentDataDirty` assert storms in debug.
    await SchedulerBinding.instance.endOfFrame;
    if (!mounted) return;

    final localeChanged = state.languageCode != language.code;
    if (localeChanged) state = language.locale;

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_prefsKey, language.code);

    final profile = _ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    if (profile.preferredLanguage == language.code) return;

    await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {
      'preferred_language': language.code,
    });
    _ref.invalidate(currentProfileProvider);
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/settings/locale_provider.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/providers/settings/theme_provider.dart

Locale, theme mode, AppStyle. This file (theme provider) is one concrete piece of that layer.

Line count: 162. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/scheduler.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/theme/app_style.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:shared_preferences/shared_preferences.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/providers/settings/theme_provider.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/scheduler.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/theme/app_style.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

const _prefsKey = 'preferred_theme';
const _stylePrefsKey = 'preferred_style';

enum AppThemePreference {
  light,
  dark,
  system;

  String get code => name;

  ThemeMode get themeMode => switch (this) {
    AppThemePreference.light => ThemeMode.light,
    AppThemePreference.dark => ThemeMode.dark,
    AppThemePreference.system => ThemeMode.system,
  };

  static AppThemePreference? tryParse(String? code) {
    if (code == null || code.isEmpty) return null;
    for (final v in AppThemePreference.values) {
      if (v.code == code) return v;
    }
    return null;
  }
}

Future<ThemeMode> loadSavedThemeMode() async {
  final prefs = await SharedPreferences.getInstance();
  final pref = AppThemePreference.tryParse(prefs.getString(_prefsKey));
  return pref?.themeMode ?? ThemeMode.system;
}

final themeModeProvider = StateNotifierProvider<ThemeModeController, ThemeMode>(
  (ref) {
    final controller = ThemeModeController(ref);
    ref.listen(currentProfileProvider, (prev, next) {
```

```
      next.whenData((profile) {
        final code = profile?.preferredTheme;
        if (code != null) controller.applyFromProfile(code);
      });
    });
    return controller;
  },
);

class ThemeModeController extends StateNotifier<ThemeMode> {
  ThemeModeController(this._ref, {ThemeMode? initial})
    : super(initial ?? ThemeMode.system) {
    if (initial == null) {
      _loadLocal();
    }
  }

  final Ref _ref;

  Future<void> _loadLocal() async {
    final prefs = await SharedPreferences.getInstance();
    final pref = AppThemePreference.tryParse(prefs.getString(_prefsKey));
    if (pref != null && mounted) state = pref.themeMode;
  }

  void applyFromProfile(String code) {
    final pref = AppThemePreference.tryParse(code);
    if (pref == null) return;
    if (state == pref.themeMode) return;
    state = pref.themeMode;
    SharedPreferences.getInstance().then(
      (prefs) => prefs.setString(_prefsKey, pref.code),
    );
  }

  Future<void> setPreference(AppThemePreference preference) async {
    await SchedulerBinding.instance.endOfFrame;
    if (!mounted) return;

    if (state != preference.themeMode) {
      state = preference.themeMode;
    }

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_prefsKey, preference.code);

    final profile = _ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    if (profile.preferredTheme == preference.code) return;

    await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {
      'preferred_theme': preference.code,
    });
    _ref.invalidate(currentProfileProvider);
  }
}

Future<AppStyle> loadSavedAppStyle() async {
  final prefs = await SharedPreferences.getInstance();
  return AppStyle.tryParse(prefs.getString(_stylePrefsKey)) ?? AppStyle.modern;
}

final appStyleProvider = StateNotifierProvider<AppStyleController, AppStyle>((
  ref,
) {
  final controller = AppStyleController(ref);
  ref.listen(currentProfileProvider, (prev, next) {
    next.whenData((profile) {
      final code = profile?.preferredStyle;
      if (code != null) controller.applyFromProfile(code);
    });
  });
  return controller;
```

```
});

class AppStyleController extends StateNotifier<AppStyle> {
  AppStyleController(this._ref, {AppStyle? initial})
    : super(initial ?? AppStyle.modern) {
    if (initial == null) {
      _loadLocal();
    }
  }

  final Ref _ref;

  Future<void> _loadLocal() async {
    final prefs = await SharedPreferences.getInstance();
    final style = AppStyle.tryParse(prefs.getString(_stylePrefsKey));
    if (style != null && mounted) state = style;
  }

  void applyFromProfile(String code) {
    final style = AppStyle.tryParse(code);
    if (style == null) return;
    if (state == style) return;
    state = style;
    SharedPreferences.getInstance().then(
      (prefs) => prefs.setString(_stylePrefsKey, style.code),
    );
  }

  Future<void> setStyle(AppStyle style) async {
    await SchedulerBinding.instance.endOfFrame;
    if (!mounted) return;

    if (state != style) state = style;

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_stylePrefsKey, style.code);

    final profile = _ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    if (profile.preferredStyle == style.code) return;

    await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {
      'preferred_style': style.code,
    });
    _ref.invalidate(currentProfileProvider);
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/providers/settings/theme_provider.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/admin/academic_years/admin_academic_years_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 459. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/academic_years/admin_academic_years_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AdminAcademicYearsScreen extends ConsumerStatefulWidget {
  const AdminAcademicYearsScreen({super.key});

  @override
  ConsumerState<AdminAcademicYearsScreen> createState() =>
      _AdminAcademicYearsScreenState();
}

class _AdminAcademicYearsScreenState
    extends ConsumerState<AdminAcademicYearsScreen> {
  bool _busy = false;

  Future<void> _createNext() async {
    final l10n = context.l10n;
    setState(() => _busy = true);
    try {
      final row = await supabase.rpc('admin_create_next_academic_year');
      final label = (row as Map)['label'] as String? ?? '';
      ref.invalidate(academicYearsProvider);
      ref.invalidate(academicTermOptionsProvider);
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
```

```
      ).showSnackBar(SnackBar(content: Text(l10n.adminYearCreated(label))));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(e.toString())));
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  Future<void> _setCurrent(String yearId) async {
    setState(() => _busy = true);
    try {
      await supabase.rpc(
        'admin_set_current_academic_year',
        params: {'p_year_id': yearId},
      );
      ref.invalidate(academicYearsProvider);
      ref.invalidate(academicTermOptionsProvider);
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(e.toString())));
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  String _normalizeYearConfirm(String value) =>
      value.trim().replaceAll(RegExp(r'[-–—■]'), '-');

  Future<void> _deleteYear(AcademicYearRecord year) async {
    final l10n = context.l10n;
    if (year.isCurrent) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(l10n.adminDeleteYearCurrentBlocked)),
      );
      return;
    }

    final confirmController = TextEditingController();
    final expected = _normalizeYearConfirm(year.label);
    var matches = false;

    final confirmed = await showDialog<bool>(
      context: context,
      useRootNavigator: true,
      builder: (ctx) {
        return StatefulBuilder(
          builder: (ctx, setLocal) {
            return AlertDialog(
              title: Text(l10n.adminDeleteYearTitle(year.label)),
              content: Column(
                mainAxisSize: MainAxisSize.min,
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Text(l10n.adminDeleteYearBody(year.label)),
                  const SizedBox(height: 16),
                  TextField(
                    controller: confirmController,
                    autofocus: true,
                    decoration: InputDecoration(
                      hintText: l10n.adminDeleteYearConfirmHint(year.label),
                    ),
                    onChanged: (value) {
                      setLocal(() {
                        matches = _normalizeYearConfirm(value) == expected;
                      });
                    },
                  ),
                ],
```

```
            ),
            actions: [
              TextButton(
                onPressed: () => Navigator.pop(ctx, false),
                child: Text(l10n.commonCancel),
              ),
              TextButton(
                onPressed: matches ? () => Navigator.pop(ctx, true) : null,
                style: TextButton.styleFrom(
                  foregroundColor: Theme.of(ctx).colorScheme.error,
                ),
                child: Text(l10n.commonDelete),
              ),
            ],
          );
        },
      );
    },
  );
  confirmController.dispose();

  if (confirmed != true || !mounted) return;

  setState(() => _busy = true);
  try {
    await supabase.rpc(
      'admin_delete_academic_year',
      params: {'p_year_id': year.id},
    );
    ref.invalidate(academicYearsProvider);
    ref.invalidate(academicTermOptionsProvider);
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(l10n.adminYearDeleted(year.label))),
    );
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(e.toString())));
  } finally {
    if (mounted) setState(() => _busy = false);
  }
}

Future<void> _editTerm(AcademicTerm term) async {
  final termId = term.dbTermId;
  final semester = term.semester;
  final start = term.startInclusive;
  final end = term.endInclusive;
  if (termId == null || semester == null || start == null || end == null) {
    return;
  }

  final l10n = context.l10n;
  final locale = Localizations.localeOf(context).toString();
  final dateFmt = DateFormat.yMMMd(locale);
  var startsOn = start;
  var endsOn = end;

  final saved = await showDialog<bool>(
    context: context,
    useRootNavigator: true,
    builder: (ctx) {
      return StatefulBuilder(
        builder: (ctx, setLocal) {
          Future<void> pickStart() async {
            final picked = await showDatePicker(
              context: ctx,
              initialDate: startsOn,
              firstDate: DateTime(startsOn.year - 2),
              lastDate: DateTime(endsOn.year + 2),
            );
```

```
          if (picked != null) setLocal(() => startsOn = picked);
        }

        Future<void> pickEnd() async {
          final picked = await showDatePicker(
            context: ctx,
            initialDate: endsOn,
            firstDate: DateTime(startsOn.year - 2),
            lastDate: DateTime(endsOn.year + 2),
          );
          if (picked != null) setLocal(() => endsOn = picked);
        }

        return AlertDialog(
          title: Text(l10n.adminEditTermTitle(semester)),
          content: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              ListTile(
                contentPadding: EdgeInsets.zero,
                title: Text(l10n.adminTermStartsOn),
                subtitle: Text(dateFmt.format(startsOn)),
                trailing: const Icon(
                  Icons.calendar_today_outlined,
                  size: 18,
                ),
                onTap: pickStart,
              ),
              ListTile(
                contentPadding: EdgeInsets.zero,
                title: Text(l10n.adminTermEndsOn),
                subtitle: Text(dateFmt.format(endsOn)),
                trailing: const Icon(
                  Icons.calendar_today_outlined,
                  size: 18,
                ),
                onTap: pickEnd,
              ),
            ],
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(ctx, false),
              child: Text(l10n.commonCancel),
            ),
            ElevatedButton(
              onPressed: () {
                if (endsOn.isBefore(startsOn)) {
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(content: Text(l10n.adminTermDatesInvalid)),
                  );
                  return;
                }
                Navigator.pop(ctx, true);
              },
              child: Text(l10n.commonSave),
            ),
          ],
        );
      },
    );
  },
);

if (saved != true || !mounted) return;

setState(() => _busy = true);
try {
  await supabase.rpc(
    'admin_update_academic_term',
    params: {
      'p_term_id': termId,
      'p_starts_on': startsOn.toIso8601String().split('T').first,
```

```
          'p_ends_on': endsOn.toIso8601String().split('T').first,
        },
      );
      ref.invalidate(academicYearsProvider);
      ref.invalidate(academicTermOptionsProvider);
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.adminTermUpdated)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(e.toString())));
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final years = ref.watch(academicYearsProvider);
    final dateFmt = DateFormat.yMMMd(
      Localizations.localeOf(context).toString(),
    );

    return Scaffold(
      appBar: AppBar(title: Text(l10n.adminAcademicYearsTitle)),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _busy ? null : _createNext,
        icon: _busy
            ? const SizedBox(
                width: 18,
                height: 18,
                child: CircularProgressIndicator(strokeWidth: 2),
              )
            : const Icon(Icons.add),
        label: Text(l10n.adminCreateNextYear),
      ),
      floatingActionButtonLocation: shellFabLocation(context),
      body: years.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (list) {
          if (list.isEmpty) {
            return EmptyState(
              title: l10n.adminNoAcademicYears,
              subtitle: l10n.adminAcademicYearsHint,
              icon: Icons.calendar_month_outlined,
            );
          }
          return RefreshIndicator(
            onRefresh: () async {
              ref.invalidate(academicYearsProvider);
              await ref.read(academicYearsProvider.future);
            },
            child: ListView.builder(
              padding: EdgeInsets.fromLTRB(
                20,
                12,
                20,
                shellContentBottomPadding(context, hasFab: true),
              ),
              itemCount: list.length + 1,
              itemBuilder: (context, index) {
                if (index == 0) {
                  return Padding(
                    padding: const EdgeInsets.only(bottom: 12),
                    child: Text(
                      l10n.adminAcademicYearsHint,
                      style: TextStyle(
                        color: AppColors.inkAlpha(context, 0.5),
```

```
                      fontSize: 13,
                    ),
                  ),
                );
              }
              final year = list[index - 1];
              return PremiumCard(
                padding: const EdgeInsets.all(16),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Row(
                      children: [
                        Expanded(
                          child: Text(
                            year.label,
                            style: const TextStyle(
                              fontWeight: FontWeight.w800,
                              fontSize: 18,
                            ),
                          ),
                        ),
                        IconButton(
                          tooltip: l10n.adminDeleteYear,
                          onPressed: _busy ? null : () => _deleteYear(year),
                          icon: Icon(
                            Icons.delete_outline,
                            color: year.isCurrent
                                ? AppColors.inkAlpha(context, 0.25)
                                : Theme.of(context).colorScheme.error,
                          ),
                        ),
                        if (year.isCurrent)
                          Container(
                            padding: const EdgeInsets.symmetric(
                              horizontal: 10,
                              vertical: 4,
                            ),
                            decoration: BoxDecoration(
                              color: AppColors.accent(
                                context,
                              ).withValues(alpha: 0.18),
                              borderRadius: BorderRadius.circular(20),
                              border: Border.all(
                                color: AppColors.accent(
                                  context,
                                ).withValues(alpha: 0.45),
                              ),
                            ),
                            child: Text(
                              l10n.adminCurrentYearBadge,
                              style: TextStyle(
                                fontSize: 11,
                                fontWeight: FontWeight.w800,
                                color: AppColors.accent(context),
                              ),
                            ),
                          )
                        else
                          TextButton(
                            onPressed: _busy
                                ? null
                                : () => _setCurrent(year.id),
                            child: Text(l10n.adminSetCurrentYear),
```

**lib/screens/admin/academic_years/admin_academic_years_screen.dart — continued (2/2)**

```
                          ),
                      ],
                    ),
                    const SizedBox(height: 4),
                    Text(
                      l10n.adminYearSemesters,
```

```
                  style: TextStyle(
                    fontSize: 12,
                    color: AppColors.inkAlpha(context, 0.5),
                  ),
                ),
                const SizedBox(height: 12),
                for (final t in year.terms) ...[
                  Padding(
                    padding: const EdgeInsets.only(bottom: 4),
                    child: Material(
                      color: Colors.transparent,
                      child: InkWell(
                        borderRadius: BorderRadius.circular(10),
                        onTap: _busy ? null : () => _editTerm(t),
                        child: Padding(
                          padding: const EdgeInsets.symmetric(
                            vertical: 6,
                            horizontal: 2,
                          ),
                          child: Row(
                            children: [
                              Icon(
                                Icons.circle,
                                size: 8,
                                color: AppColors.accent(context),
                              ),
                              const SizedBox(width: 10),
                              Expanded(
                                child: Text(
                                  l10n.adminSemesterRange(
                                    t.semester ?? 0,
                                    dateFmt.format(t.startInclusive!),
                                    dateFmt.format(t.endInclusive!),
                                  ),
                                  style: TextStyle(
                                    fontSize: 13,
                                    color: AppColors.ink(context),
                                  ),
                                ),
                              ),
                              Icon(
                                Icons.edit_outlined,
                                size: 16,
                                color: AppColors.inkAlpha(context, 0.45),
                              ),
                            ],
                          ),
                        ),
                      ),
                    ),
                  ),
                ],
              ],
            ),
          );
        },
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/academic_years/admin_academic_years_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?

4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/admin/analytics/admin_analytics_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 85. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:syncfusion_flutter_charts/charts.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/analytics/admin_analytics_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:syncfusion_flutter_charts/charts.dart';

class AdminAnalyticsScreen extends ConsumerWidget {
  const AdminAnalyticsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final stats = ref.watch(adminPlatformStatsProvider);
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.adminAnalytics)),
      body: stats.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (s) => RefreshIndicator(
          onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),
          child: ListView(
            padding: EdgeInsets.fromLTRB(
              20,
              20,
              20,
              shellContentBottomPadding(context),
            ),
            children: [
```

```
              PremiumSectionHeader(
                title: l10n.adminActivityDistribution,
                subtitle: l10n.adminGlobalMetrics,
              ),
              PremiumCard(
                padding: const EdgeInsets.all(24),
                child: SfCartesianChart(
                  primaryXAxis: const CategoryAxis(),
                  series: <CartesianSeries>[
                    ColumnSeries<_AdminChartData, String>(
                      dataSource: [
                        _AdminChartData(
                          l10n.adminChartAssignments,
                          (s['assignments'] ?? 0).toDouble(),
                        ),
                        _AdminChartData(
                          l10n.adminChartMarks,
                          (s['marks'] ?? 0).toDouble(),
                        ),
                        _AdminChartData(
                          l10n.adminChartStudents,
                          (s['students'] ?? 0).toDouble(),
                        ),
                        _AdminChartData(
                          l10n.adminChartGroups,
                          (s['groups'] ?? 0).toDouble(),
                        ),
                      ],
                      xValueMapper: (d, _) => d.label,
                      yValueMapper: (d, _) => d.value,
                      color: AppColors.accent(context),
                      borderRadius: const BorderRadius.vertical(
                        top: Radius.circular(8),
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class _AdminChartData {
  _AdminChartData(this.label, this.value);
  final String label;
  final double value;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/analytics/admin_analytics_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/admin/announcements/admin_announcements_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 279. Import count: 11. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/notifications/notification_providers.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/admin/announcements/admin_announcements_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AdminAnnouncementsScreen extends ConsumerStatefulWidget {
  const AdminAnnouncementsScreen({super.key});

  @override
  ConsumerState<AdminAnnouncementsScreen> createState() =>
      _AdminAnnouncementsScreenState();
}

class _AdminAnnouncementsScreenState
    extends ConsumerState<AdminAnnouncementsScreen> {
  void _refresh() {
    ref.invalidate(adminAnnouncementsProvider);
    ref.invalidate(announcementsProvider);
  }

  Future<void> _compose({Map<String, dynamic>? existing}) async {
    final groups = await ref.read(adminGroupsProvider.future);
    if (groups.isEmpty || !mounted) return;
    final l10n = context.l10n;
    final title = TextEditingController(
```

```
  text: existing?['title'] as String? ?? '',
);
final body = TextEditingController(
  text: existing?['description'] as String? ?? '',
);
var groupId =
    existing?['group_id'] as String? ?? groups.first['id'] as String;

await showDialog<void>(
  context: context,
  builder: (ctx) => StatefulBuilder(
    builder: (ctx, setStateLocal) => AlertDialog(
      title: Text(
        existing == null ? l10n.announcementsNew : l10n.announcementsEdit,
      ),
      content: SingleChildScrollView(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            TextField(
              controller: title,
              decoration: InputDecoration(labelText: l10n.commonTitle),
            ),
            const SizedBox(height: 16),
            TextField(
              controller: body,
              decoration: InputDecoration(labelText: l10n.commonMessage),
              maxLines: 4,
            ),
            if (existing == null) ...[
              const SizedBox(height: 16),
              DropdownButtonFormField<String>(
                initialValue: groupId,
                decoration: InputDecoration(
                  labelText: l10n.teacherTargetGroup,
                ),
                items: groups
                    .map(
                      (g) => DropdownMenuItem(
                        value: g['id'] as String,
                        child: Text(g['title'] as String),
                      ),
                    )
                    .toList(),
                onChanged: (v) =>
                    setStateLocal(() => groupId = v ?? groupId),
              ),
            ],
          ],
        ),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx),
          child: Text(l10n.commonCancel),
        ),
        ElevatedButton(
          onPressed: () async {
            if (title.text.trim().isEmpty) return;
            if (existing == null) {
              await supabase.from('announcements').insert({
                'group_id': groupId,
                'title': title.text.trim(),
                'description': body.text.trim(),
              });
            } else {
              await supabase
                  .from('announcements')
                  .update({
                    'title': title.text.trim(),
                    'description': body.text.trim(),
                  })
                  .eq('id', existing['id'] as String);
```

```
              }
              if (ctx.mounted) Navigator.pop(ctx);
              _refresh();
            },
            child: Text(
              existing == null ? l10n.announcementsPost : l10n.commonSave,
            ),
          ),
        ],
      ),
    ),
  );
}

Future<void> _delete(Map<String, dynamic> row) async {
  final l10n = context.l10n;
  final confirmed = await showDialog<bool>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: Text(l10n.announcementsDeleteTitle),
      content: Text(l10n.announcementsDeleteBody),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          child: Text(l10n.commonCancel),
        ),
        FilledButton(
          style: FilledButton.styleFrom(
            backgroundColor: AppColors.danger(context),
          ),
          onPressed: () => Navigator.pop(ctx, true),
          child: Text(l10n.commonDelete),
        ),
      ],
    ),
  );
  if (confirmed != true) return;
  await supabase.from('announcements').delete().eq('id', row['id'] as String);
  _refresh();
}

@override
Widget build(BuildContext context) {
  final announcements = ref.watch(adminAnnouncementsProvider);
  final groupTitles = {
    for (final g in ref.watch(adminGroupsProvider).valueOrNull ?? [])
      g['id'] as String: g['title'] as String,
  };
  final l10n = context.l10n;

  return Scaffold(
    appBar: AppBar(title: Text(l10n.announcementsAll)),
    floatingActionButtonLocation: shellFabLocation(context),
    floatingActionButton: FloatingActionButton(
      onPressed: () => _compose(),
      child: const Icon(Icons.campaign_outlined),
    ),
    body: announcements.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
      data: (rows) => rows.isEmpty
          ? EmptyState(
              title: l10n.announcementsEmpty,
              subtitle: l10n.announcementsEmptyAdmin,
            )
          : RefreshIndicator(
              onRefresh: () async => _refresh(),
              child: ListView.builder(
                padding: EdgeInsets.fromLTRB(
                  20,
                  20,
                  20,
                  shellContentBottomPadding(context, hasFab: true),
```

```
),
itemCount: rows.length,
itemBuilder: (_, i) {
  final row = rows[i];
  final createdAt = DateTime.tryParse(
    row['created_at'] as String? ?? '',
  );
  final groupTitle = groupTitles[row['group_id'] as String?];
  final byAdmin = row['teacher_id'] == null;
  return PremiumCard(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Expanded(
              child: Text(
                row['title'] as String,
                style: const TextStyle(
                  fontWeight: FontWeight.w800,
                  fontSize: 16,
                ),
              ),
            ),
            PopupMenuButton<String>(
              padding: EdgeInsets.zero,
              icon: Icon(
                Icons.more_horiz_rounded,
                size: 20,
                color: AppColors.inkAlpha(context, 0.5),
              ),
              onSelected: (v) => v == 'edit'
                  ? _compose(existing: row)
                  : _delete(row),
              itemBuilder: (_) => [
                PopupMenuItem(
                  value: 'edit',
                  child: Text(l10n.commonEdit),
                ),
                PopupMenuItem(
                  value: 'delete',
                  child: Text(l10n.commonDelete),
                ),
              ],
            ),
          ],
        ),
        const SizedBox(height: 8),
        Text(
          row['description'] as String? ?? '',
          style: TextStyle(
            color: AppColors.inkAlpha(context, 0.7),
            height: 1.5,
          ),
        ),
        const SizedBox(height: 12),
        Wrap(
          spacing: 8,
          runSpacing: 4,
          crossAxisAlignment: WrapCrossAlignment.center,
          children: [
            if (createdAt != null)
              Text(
                DateFormat.yMMMd().format(createdAt),
                style: TextStyle(
                  fontSize: 11,
                  color: AppColors.inkAlpha(context, 0.4),
                ),
              ),
            if (groupTitle != null)
              Text(
```

```
                                '• $groupTitle',
                                style: TextStyle(
                                  fontSize: 11,
                                  fontWeight: FontWeight.w600,
                                  color: AppColors.inkAlpha(context, 0.55),
                                ),
                              ),
                            if (byAdmin)
                              Text(
                                '• ${l10n.announcementsByAdmin}',
                                style: TextStyle(
                                  fontSize: 11,
                                  fontWeight: FontWeight.w700,
                                  color: AppColors.accent(context),
                                ),
                              ),
                          ],
                        ),
                      ],
                    ),
                  );
                },
              ),
            ),
          ),
        ),
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/announcements/admin_announcements_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/admin/credentials/admin_credential_print_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 221. Import count: 12. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/admin/credential_slips_pdf.dart';
- import 'package:mu_study/core/admin/credential_slips_source.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/enums.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:share_plus/share_plus.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/credentials/admin_credential_print_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/admin/credential_slips_pdf.dart';
import 'package:mu_study/core/admin/credential_slips_source.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/enums.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:share_plus/share_plus.dart';

class AdminCredentialPrintScreen extends ConsumerStatefulWidget {
  const AdminCredentialPrintScreen({super.key});

  @override
  ConsumerState<AdminCredentialPrintScreen> createState() =>
      _AdminCredentialPrintScreenState();
}

class _AdminCredentialPrintScreenState
    extends ConsumerState<AdminCredentialPrintScreen> {
  UserRole _role = UserRole.student;
  String? _groupId;
  bool _exporting = false;

  Future<void> _export() async {
    if (_exporting) return;
    setState(() => _exporting = true);
    final l10n = context.l10n;
```

```
    try {
      final slips = await fetchCredentialSlips(
        role: _role,
        groupId: _role == UserRole.student ? _groupId : null,
      );
      if (slips.isEmpty) {
        throw Exception(l10n.adminCredentialNeedUsers);
      }
      final bytes = buildCredentialSlipsPdf(slips, l10n);
      const fileName = 'mu_study_login_slips.pdf';
      await SharePlus.instance.share(
        ShareParams(
          files: [
            XFile.fromData(bytes, mimeType: 'application/pdf', name: fileName),
          ],
          fileNameOverrides: [fileName],
          subject: l10n.adminCredentialPrintTitle,
          downloadFallbackEnabled: true,
        ),
      );
    } catch (e) {
      if (!mounted) return;
      final message = e is Exception
          ? e.toString().replaceFirst(RegExp(r'^Exception: '), '')
          : '$e';
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(message)));
    } finally {
      if (mounted) setState(() => _exporting = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final groups = ref.watch(adminGroupsProvider);

    return Scaffold(
      appBar: AppBar(title: Text(l10n.adminCredentialPrintTitle)),
      body: ListView(
        padding: EdgeInsets.fromLTRB(
          20,
          20,
          20,
          shellContentBottomPadding(context, extra: 24),
        ),
        children: [
          PremiumCard(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  l10n.adminCredentialPrintSubtitle,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    color: AppColors.inkAlpha(context, 0.7),
                  ),
                ),
                const SizedBox(height: 16),
                SegmentedButton<UserRole>(
                  segments: [
                    ButtonSegment(
                      value: UserRole.student,
                      label: Text(l10n.adminCredentialStudents),
                    ),
                    ButtonSegment(
                      value: UserRole.teacher,
                      label: Text(l10n.adminCredentialTeachers),
                    ),
                  ],
                  selected: {_role},
```

```
                  onSelectionChanged: (next) {
                    setState(() {
                      _role = next.first;
                      if (_role != UserRole.student) _groupId = null;
                    });
                  },
                ),
                if (_role == UserRole.student) ...[
                  const SizedBox(height: 16),
                  groups.when(
                    loading: () => const LinearProgressIndicator(),
                    error: (e, _) => Text('$e'),
                    data: (rows) => DropdownButtonFormField<String?>(
                      key: ValueKey(_groupId),
                      initialValue: _groupId,
                      decoration: InputDecoration(
                        labelText: l10n.adminGroups,
                      ),
                      items: [
                        DropdownMenuItem<String?>(
                          value: null,
                          child: Text(l10n.adminCredentialAllGroups),
                        ),
                        ...rows.map(
                          (g) => DropdownMenuItem<String?>(
                            value: g['id'] as String,
                            child: Text(g['title'] as String? ?? ''),
                          ),
                        ),
                      ],
                      onChanged: (value) => setState(() => _groupId = value),
                    ),
                  ),
                ],
                const SizedBox(height: 20),
                SizedBox(
                  height: 48,
                  width: double.infinity,
                  child: FilledButton.icon(
                    onPressed: _exporting ? null : _export,
                    icon: _exporting
                        ? const SizedBox(
                            width: 18,
                            height: 18,
                            child: CircularProgressIndicator(strokeWidth: 2),
                          )
                        : const Icon(Icons.picture_as_pdf_outlined),
                    label: Text(
                      _exporting
                          ? l10n.commonLoading
                          : l10n.adminCredentialExportPdf,
                    ),
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(height: 16),
          _PreviewList(
            role: _role,
            groupId: _role == UserRole.student ? _groupId : null,
          ),
        ],
      ),
    );
  }
}

class _PreviewList extends StatelessWidget {
  const _PreviewList({required this.role, required this.groupId});

  final UserRole role;
  final String? groupId;
```

```
  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return FutureBuilder(
      key: ValueKey('${role.name}-${groupId ?? 'all'}'),
      future: fetchCredentialSlipPreviews(role: role, groupId: groupId),
      builder: (context, snapshot) {
        if (snapshot.connectionState != ConnectionState.done) {
          return const Padding(
            padding: EdgeInsets.symmetric(vertical: 24),
            child: Center(child: CircularProgressIndicator()),
          );
        }
        if (snapshot.hasError) {
          return ErrorState(message: '${snapshot.error}');
        }
        final slips = snapshot.data ?? const [];
        if (slips.isEmpty) {
          return EmptyState(
            icon: Icons.print_outlined,
            title: l10n.adminCredentialNeedUsers,
          );
        }
        return PremiumCard(
          padding: const EdgeInsets.fromLTRB(8, 8, 8, 8),
          child: Column(
            children: [
              for (final slip in slips)
                ListTile(
                  dense: true,
                  title: Text(
                    slip.fullName.isEmpty ? slip.username : slip.fullName,
                    style: const TextStyle(fontWeight: FontWeight.w700),
                  ),
                  subtitle: Text(slip.username),
                ),
            ],
          ),
        );
      },
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/credentials/admin_credential_print_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/admin/dashboard/admin_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 346. Import count: 11. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:syncfusion_flutter_charts/charts.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/dashboard/admin_dashboard_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:syncfusion_flutter_charts/charts.dart';

class AdminDashboardScreen extends ConsumerWidget {
  const AdminDashboardScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final stats = ref.watch(adminPlatformStatsProvider);

    return stats.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
      data: (s) {
        final l10n = context.l10n;
        return RefreshIndicator(
          onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),
          child: ListView(
            padding: EdgeInsets.fromLTRB(
              20,
              24,
              20,
              shellContentBottomPadding(context, extra: 24),
```

```
),
children: [
  Text(
    l10n.adminWorkspace,
    style: Theme.of(context).textTheme.bodyLarge?.copyWith(
      color: AppColors.inkAlpha(context, 0.5),
      fontWeight: FontWeight.w700,
    ),
  ),
  const SizedBox(height: 8),
  Text(
    l10n.adminPlatformOverview,
    style: Theme.of(context).textTheme.displaySmall,
  ),
  SizedBox(height: 32),

  GridView.count(
    shrinkWrap: true,
    physics: const NeverScrollableScrollPhysics(),
    crossAxisCount: 2,
    crossAxisSpacing: 16,
    mainAxisSpacing: 16,
    childAspectRatio: 1.2,
    children: [
      _AdminStatCard(
        title: l10n.adminTeachers,
        value: s['teachers']!,
        icon: Icons.school_outlined,
        color: AppColors.accent(context),
      ),
      _AdminStatCard(
        title: l10n.adminStudents,
        value: s['students']!,
        icon: Icons.people_outline,
        color: AppColors.mutedGold,
      ),
      _AdminStatCard(
        title: l10n.adminGroupsLabel,
        value: s['groups']!,
        icon: Icons.groups_outlined,
        color: AppColors.successGreen,
      ),
      _AdminStatCard(
        title: l10n.adminMarksLabel,
        value: s['marks'] ?? 0,
        icon: Icons.grade_outlined,
        color: AppColors.warmAmber,
      ),
    ],
  ),
  const SizedBox(height: 32),

  PremiumSectionHeader(
    title: l10n.adminActivityInsights,
    subtitle: l10n.adminCurrentMetrics,
  ),
  PremiumCard(
    padding: const EdgeInsets.all(24),
    child: SizedBox(
      height: 200,
      child: SfCartesianChart(
        plotAreaBorderWidth: 0,
        primaryXAxis: const CategoryAxis(
          majorGridLines: MajorGridLines(width: 0),
          axisLine: AxisLine(width: 0),
        ),
        primaryYAxis: const NumericAxis(
          majorGridLines: MajorGridLines(
            width: 1,
            color: Color(0xFFF0F0F0),
          ),
          axisLine: AxisLine(width: 0),
        ),
```

```
        series: <CartesianSeries>[
          ColumnSeries<_AdminChartData, String>(
            dataSource: [
              _AdminChartData(
                l10n.adminChartTasks,
                s['assignments']!.toDouble(),
              ),
              _AdminChartData(
                l10n.adminChartGroups,
                s['groups']!.toDouble(),
              ),
              _AdminChartData(
                l10n.adminChartStaff,
                s['teachers']!.toDouble(),
              ),
            ],
            xValueMapper: (d, _) => d.label,
            yValueMapper: (d, _) => d.value,
            color: AppColors.accent(context),
            borderRadius: const BorderRadius.vertical(
              top: Radius.circular(6),
            ),
          ),
        ],
      ),
    ),
  ),
),
const SizedBox(height: 24),
PremiumSectionHeader(
  title: l10n.adminAcademicYears,
  subtitle: l10n.adminAcademicYearsSubtitle,
),
PremiumCard(
  onTap: () => context.push('/admin/academic-years'),
  child: Row(
    children: [
      Container(
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: AppColors.softWash(context),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Icon(
          Icons.calendar_month_outlined,
          color: AppColors.accent(context),
        ),
      ),
      const SizedBox(width: 16),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              l10n.adminAcademicYearsTitle,
              style: const TextStyle(
                fontWeight: FontWeight.w800,
                fontSize: 16,
              ),
            ),
            const SizedBox(height: 2),
            Text(
              l10n.adminCreateNextYear,
              style: TextStyle(
                fontSize: 13,
                color: AppColors.inkAlpha(context, 0.5),
              ),
            ),
          ],
        ),
      ),
      Icon(
        Icons.chevron_right_rounded,
        color: AppColors.inkAlpha(context, 0.35),
```

```
      ),
    ],
  ),
),
const SizedBox(height: 16),
PremiumCard(
  onTap: () => context.push('/admin/credential-print'),
  child: Row(
    children: [
      Container(
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: AppColors.softWash(context),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Icon(
          Icons.print_outlined,
          color: AppColors.accent(context),
        ),
      ),
      const SizedBox(width: 16),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              l10n.adminCredentialPrintTitle,
              style: const TextStyle(
                fontWeight: FontWeight.w800,
                fontSize: 16,
              ),
            ),
            const SizedBox(height: 2),
            Text(
              l10n.adminCredentialPrintSubtitle,
              style: TextStyle(
                fontSize: 13,
                color: AppColors.inkAlpha(context, 0.5),
              ),
            ),
          ],
        ),
      ),
      Icon(
        Icons.chevron_right_rounded,
        color: AppColors.inkAlpha(context, 0.35),
      ),
    ],
  ),
),
const SizedBox(height: 16),
PremiumCard(
  onTap: () => context.push('/admin/mass-generator'),
  child: Row(
    children: [
      Container(
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: AppColors.softWash(context),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Icon(
          Icons.dataset_outlined,
          color: AppColors.accent(context),
        ),
      ),
      const SizedBox(width: 16),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Mass Data Generator',
```

```
                            style: const TextStyle(
                              fontWeight: FontWeight.w800,
                              fontSize: 16,
                            ),
                          ),
                          const SizedBox(height: 2),
                          Text(
                            'Paste teachers, groups and students; generate in one go.',
                            style: TextStyle(
                              fontSize: 13,
                              color: AppColors.inkAlpha(context, 0.5),
                            ),
                          ),
                        ],
                      ),
                    ),
                    Icon(
                      Icons.chevron_right_rounded,
                      color: AppColors.inkAlpha(context, 0.35),
                    ),
                  ],
                ),
              ),
              const SizedBox(height: 100),
            ],
          ),
        );
      },
    );
  }
}

class _AdminStatCard extends StatelessWidget {
  const _AdminStatCard({
    required this.title,
    required this.value,
    required this.icon,
    required this.color,
  });

  final String title;
  final int value;
  final IconData icon;
  final Color color;

  @override
  Widget build(BuildContext context) {
    final tone = AppColors.resolveAccent(context, color);
    return PremiumCard(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Container(
                padding: const EdgeInsets.all(8),
                decoration: BoxDecoration(
                  color: tone.withValues(alpha: 0.1),
                  borderRadius: BorderRadius.circular(8),
                ),
                child: Icon(icon, size: 18, color: tone),
              ),
            ],
          ),
          const Spacer(),
          Text(
            '$value',
            style: Theme.of(context).textTheme.headlineMedium?.copyWith(
              fontWeight: FontWeight.w800,
              color: AppColors.ink(context),
```

```
              ),
            ),
            const SizedBox(height: 4),
            Text(
              title,
              style: Theme.of(context).textTheme.labelSmall?.copyWith(
                letterSpacing: 1.0,
                fontWeight: FontWeight.w800,
                color: AppColors.inkAlpha(context, 0.4),
              ),
            ),
          ],
        ),
      );
  }
}

class _AdminChartData {
  _AdminChartData(this.label, this.value);
  final String label;
  final double value;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/dashboard/admin_dashboard_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 292. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/models/enums.dart';
- import 'package:mu_study/core/admin/credential_slips_source.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/models/enums.dart';
import 'package:mu_study/core/admin/credential_slips_source.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class AdminMassDataGeneratorScreen extends ConsumerStatefulWidget {
  const AdminMassDataGeneratorScreen({super.key});

  @override
  ConsumerState<AdminMassDataGeneratorScreen> createState() =>
      _AdminMassDataGeneratorScreenState();
}

class _AdminMassDataGeneratorScreenState
    extends ConsumerState<AdminMassDataGeneratorScreen> {
  final _teachersCtrl = TextEditingController();
  final _studentsCtrl = TextEditingController();
  final _groupsCtrl = TextEditingController();
  final _assignmentsCtrl = TextEditingController();

  bool _running = false;

  @override
  void dispose() {
    _teachersCtrl.dispose();
    _studentsCtrl.dispose();
```

```
  _groupsCtrl.dispose();
  _assignmentsCtrl.dispose();
  super.dispose();
}

static List<String> _splitLines(String raw) => raw
    .split('\n')
    .map((e) => e.trim())
    .where((e) => e.isNotEmpty)
    .toList();

/// Format: `username|password|full_name|sphere` (sphere: math|english)
List<Map<String, dynamic>> _parseTeachers() {
  final out = <Map<String, dynamic>>[];
  for (final line in _splitLines(_teachersCtrl.text)) {
    final parts = line.split('|').map((e) => e.trim()).toList();
    if (parts.length != 4) {
      throw ArgumentError(
        'Teacher line must be: username|password|full_name|sphere. Bad: "$line"',
      );
    }

    final sphereRaw = parts[3].toLowerCase();
    final sphere = Sphere.values.firstWhere(
      (s) => s.name == sphereRaw,
      orElse: () => Sphere.math,
    );

    out.add({
      'username': parts[0],
      'password': parts[1],
      'full_name': parts[2],
      'sphere': sphere.name,
    });
  }
  return out;
}

/// Format: `username|password|full_name`
List<Map<String, dynamic>> _parseStudents() {
  final out = <Map<String, dynamic>>[];
  for (final line in _splitLines(_studentsCtrl.text)) {
    final parts = line.split('|').map((e) => e.trim()).toList();
    if (parts.length != 3) {
      throw ArgumentError(
        'Student line must be: username|password|full_name. Bad: "$line"',
      );
    }

    out.add({
      'username': parts[0],
      'password': parts[1],
      'full_name': parts[2],
    });
  }
  return out;
}

/// Format: `teacher_username|group_title`
List<Map<String, dynamic>> _parseGroups() {
  final out = <Map<String, dynamic>>[];
  for (final line in _splitLines(_groupsCtrl.text)) {
    final parts = line.split('|').map((e) => e.trim()).toList();
    if (parts.length != 2) {
      throw ArgumentError(
        'Group line must be: teacher_username|group_title. Bad: "$line"',
      );
    }
    out.add({
      'teacher_username': parts[0],
      'title': parts[1],
    });
  }
```

```
    return out;
  }

  /// Format: `teacher_username|group_title|student_username`
  List<Map<String, dynamic>> _parseAssignments() {
    final out = <Map<String, dynamic>>[];
    for (final line in _splitLines(_assignmentsCtrl.text)) {
      final parts = line.split('|').map((e) => e.trim()).toList();
      if (parts.length != 3) {
        throw ArgumentError(
          'Assignment line must be: teacher_username|group_title|student_username. Bad:
"$line"',
        );
      }
      out.add({
        'teacher_username': parts[0],
        'group_title': parts[1],
        'student_username': parts[2],
      });
    }
    return out;
  }

  Future<void> _run() async {
    if (_running) return;
    setState(() => _running = true);
    try {
      final teachers = _parseTeachers();
      final students = _parseStudents();
      final groups = _parseGroups();
      final assignments = _parseAssignments();

      await supabase.rpc(
        'admin_mass_generate_school_data',
        params: {
          'p_teachers': teachers,
          'p_students': students,
          'p_groups': groups,
          'p_assignments': assignments,
        },
      );
      await saveLoginPasswordsByUsername([...teachers, ...students]);

      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(
            'Generated successfully.',
          ),
        ),
      );

      ref.invalidate(adminPlatformStatsProvider);
      ref.invalidate(adminGroupsProvider);
      ref.invalidate(allProfilesProvider);
    } on ArgumentError catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('$e')),
      );
    } on PostgrestException catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.message)),
      );
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Failed: $e')),
      );
    } finally {
      if (mounted) setState(() => _running = false);
    }
```

```
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Mass Data Generator'),
    ),
    body: ListView(
      padding: EdgeInsets.fromLTRB(
        20,
        20,
        20,
        shellContentBottomPadding(context, extra: 24),
      ),
      children: [
        PremiumCard(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text(
                'Paste data (use "|" delimiter).',
                style: TextStyle(fontWeight: FontWeight.w800),
              ),
              const SizedBox(height: 12),
              Text(
                '1) Teachers: username|password|full_name|sphere',
                style: TextStyle(color: AppColors.inkAlpha(context, 0.55)),
              ),
              const SizedBox(height: 8),
              TextField(
                controller: _teachersCtrl,
                minLines: 6,
                maxLines: 10,
                keyboardType: TextInputType.multiline,
                decoration: const InputDecoration(
                  border: OutlineInputBorder(),
                  hintText: '@teacher1|password123|Teacher One|math',
                ),
              ),
              const SizedBox(height: 16),
              const Text(
                '2) Students: username|password|full_name',
              ),
              const SizedBox(height: 8),
              TextField(
                controller: _studentsCtrl,
                minLines: 6,
                maxLines: 10,
                keyboardType: TextInputType.multiline,
                decoration: const InputDecoration(
                  border: OutlineInputBorder(),
                  hintText: '@student1|password123|Student One',
                ),
              ),
              const SizedBox(height: 16),
              const Text(
                '3) Groups: teacher_username|group_title',
              ),
              const SizedBox(height: 8),
              TextField(
                controller: _groupsCtrl,
                minLines: 4,
                maxLines: 8,
                keyboardType: TextInputType.multiline,
                decoration: const InputDecoration(
                  border: OutlineInputBorder(),
                  hintText: '@teacher1|English 7A',
                ),
              ),
              const SizedBox(height: 16),
              const Text(
```

```
                  '4) Assignments: teacher_username|group_title|student_username',
                ),
                const SizedBox(height: 8),
                TextField(
                  controller: _assignmentsCtrl,
                  minLines: 4,
                  maxLines: 10,
                  keyboardType: TextInputType.multiline,
                  decoration: const InputDecoration(
                    border: OutlineInputBorder(),
                    hintText: '@teacher1|English 7A|@student1',
                  ),
                ),
                const SizedBox(height: 18),
                SizedBox(
                  height: 48,
                  child: FilledButton.icon(
                    onPressed: _running ? null : _run,
                    icon: _running
                        ? const SizedBox(
                            width: 18,
                            height: 18,
                            child: CircularProgressIndicator(strokeWidth: 2),
                          )
                        : const Icon(Icons.auto_fix_high_rounded),
                    label: Text(_running ? 'Generating...' : 'Generate'),
                  ),
                ),
                const SizedBox(height: 8),
                Text(
                  'Tip: usernames should match exactly the ones used in Teachers/Students.',
                  style: TextStyle(color: AppColors.inkAlpha(context, 0.55)),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/admin/groups/admin_groups_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 235. Import count: 11. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:mu_study/screens/admin/helpers/admin_labels.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/groups/admin_groups_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/screens/admin/helpers/admin_labels.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AdminGroupsScreen extends ConsumerStatefulWidget {
  const AdminGroupsScreen({super.key});

  @override
  ConsumerState<AdminGroupsScreen> createState() => _AdminGroupsScreenState();
}

class _AdminGroupsScreenState extends ConsumerState<AdminGroupsScreen> {
  Future<void> _createGroup() async {
    final title = TextEditingController();
    String? selectedTeacherId;
    var submitting = false;

    final teachersRes = await supabase
        .from('teachers')
        .select('id, profiles(full_name, username)');
    final teachers = teachersRes as List<dynamic>;

    if (!mounted) return;

    await showDialog<void>(
```

```
        context: context,
        builder: (ctx) => StatefulBuilder(
          builder: (ctx, setLocal) {
            final l10n = ctx.l10n;
            return AlertDialog(
              title: Text(l10n.adminCreateGroup),
              content: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  TextField(
                    controller: title,
                    decoration: InputDecoration(
                      labelText: l10n.commonGroup,
                      hintText: l10n.adminGroupTitleHint,
                    ),
                  ),
                  const SizedBox(height: 16),
                  DropdownButtonFormField<String>(
                    initialValue: selectedTeacherId,
                    decoration: InputDecoration(
                      labelText: l10n.adminAssignTeacher,
                    ),
                    items: teachers.map((t) {
                      final p = t['profiles'] as Map;
                      final name =
                          p['full_name'] as String? ?? p['username'] as String;
                      return DropdownMenuItem(
                        value: t['id'] as String,
                        child: Text(name),
                      );
                    }).toList(),
                    onChanged: submitting
                        ? null
                        : (v) => setLocal(() => selectedTeacherId = v),
                  ),
                ],
              ),
              actions: [
                TextButton(
                  onPressed: submitting ? null : () => Navigator.pop(ctx),
                  child: Text(l10n.commonCancel),
                ),
                ElevatedButton(
                  onPressed: submitting
                      ? null
                      : () async {
                          final groupTitle = title.text.trim();
                          if (groupTitle.isEmpty || selectedTeacherId == null) {
                            return;
                          }
                          setLocal(() => submitting = true);
                          try {
                            await supabase.from('groups').insert({
                              'title': groupTitle,
                              'teacher_id': selectedTeacherId,
                            });
                            if (ctx.mounted) Navigator.pop(ctx);
                            ref.invalidate(adminGroupsProvider);
                          } catch (e) {
                            if (ctx.mounted) {
                              setLocal(() => submitting = false);
                              final message = '$e'.contains(
                                    'groups_teacher_title_uidx',
                                  )
                                  ? l10n.adminGroupTitleExists
                                  : l10n.commonErrorWithDetails('$e');
                              ScaffoldMessenger.of(ctx).showSnackBar(
                                SnackBar(content: Text(message)),
                              );
                            }
                          }
                        },
                  child: Text(l10n.adminCreateGroupButton),
```

```
            ),
          ],
        );
      },
    ),
  );
}

Future<void> _assignStudent(String groupId) async {
  final students = await supabase
      .from('students')
      .select('id, profiles(full_name, username)');
  if (!mounted) return;
  String? studentId;
  await showDialog<void>(
    context: context,
    builder: (ctx) {
      final l10n = ctx.l10n;
      return AlertDialog(
        title: Text(l10n.adminAssignStudent),
        content: DropdownButtonFormField<String>(
          decoration: InputDecoration(labelText: l10n.adminSelectStudent),
          items: students.map((s) {
            final p = s['profiles'] as Map;
            return DropdownMenuItem(
              value: s['id'] as String,
              child: Text(
                p['full_name'] as String? ?? p['username'] as String,
              ),
            );
          }).toList(),
          onChanged: (v) => studentId = v,
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx),
            child: Text(l10n.commonCancel),
          ),
          ElevatedButton(
            onPressed: () async {
              if (studentId == null) return;
              await supabase.from('group_students').upsert({
                'group_id': groupId,
                'student_id': studentId,
              }, onConflict: 'group_id,student_id');
              if (ctx.mounted) Navigator.pop(ctx);
              ref.invalidate(adminGroupsProvider);
              ref.invalidate(groupMembersProvider(groupId));
            },
            child: Text(l10n.commonAssign),
          ),
        ],
      );
    },
  );
}

@override
Widget build(BuildContext context) {
  final groups = ref.watch(adminGroupsProvider);
  final l10n = context.l10n;

  return Scaffold(
    appBar: AppBar(title: Text(l10n.adminGroups)),
    floatingActionButtonLocation: shellFabLocation(context),
    floatingActionButton: FloatingActionButton(
      onPressed: _createGroup,
      child: const Icon(Icons.group_add_outlined),
    ),
    body: groups.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
      data: (rows) => RefreshIndicator(
```

```
            onRefresh: () async => ref.invalidate(adminGroupsProvider),
            child: ListView.builder(
              padding: EdgeInsets.fromLTRB(
                20,
                20,
                20,
                shellContentBottomPadding(context, hasFab: true),
              ),
              itemCount: rows.length,
              itemBuilder: (_, i) {
                final g = rows[i];
                final teacher = g['teachers'] as Map?;
                final profile = teacher?['profiles'] as Map?;
                return PremiumCard(
                  padding: const EdgeInsets.all(16),
                  child: Row(
                    children: [
                      Expanded(
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Text(
                              g['title'] as String,
                              style: const TextStyle(
                                fontWeight: FontWeight.w800,
                                fontSize: 16,
                              ),
                            ),
                            Text(
                              '${l10n.adminTeacherLine(profile?['full_name'] as String? ?? '—')} '
                              '(${localizedSphere(l10n, teacher?['sphere'])})',
                              style: TextStyle(
                                color: AppColors.inkAlpha(context, 0.5),
                                fontSize: 13,
                              ),
                            ),
                          ],
                        ),
                      ),
                      IconButton(
                        icon: Icon(
                          Icons.person_add_alt,
                          color: AppColors.accent(context),
                        ),
                        onPressed: () => _assignStudent(g['id'] as String),
                      ),
                    ],
                  ),
                );
              },
            ),
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/groups/admin_groups_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/admin/helpers/admin_labels.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 15. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/l10n/app_localizations.dart';
- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/admin/helpers/admin_labels.dart**

```
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/models/models.dart';

String localizedRole(AppLocalizations l10n, UserRole role) => switch (role) {
  UserRole.student => l10n.commonStudent,
  UserRole.teacher => l10n.commonTeacher,
  UserRole.admin => l10n.commonAdmin,
};

String localizedSphere(AppLocalizations l10n, Object? raw) {
  final name = raw?.toString();
  if (name == Sphere.math.name) return l10n.sphereMath;
  if (name == Sphere.english.name) return l10n.sphereEnglish;
  return name ?? '—';
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/helpers/admin_labels.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/admin/notifications/admin_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 22. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/admin/notifications/admin_notifications_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class AdminNotificationsScreen extends ConsumerWidget {
  const AdminNotificationsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final l10n = context.l10n;
    return Scaffold(
      body: Center(
        child: EmptyState(
          title: l10n.adminActivityLog,
          subtitle: l10n.adminActivityLogSubtitle,
          icon: Icons.history,
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/notifications/admin_notifications_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/admin/profile/admin_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 12. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/screens/profile/app_profile_screen.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/admin/profile/admin_profile_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/profile/app_profile_screen.dart';

class AdminProfileScreen extends ConsumerWidget {
  const AdminProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const AppProfileScreen();
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/profile/admin_profile_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/admin/users/admin_users_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 319. Import count: 15. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/inputs/load_more_button.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/admin/admin_providers.dart';
- import 'package:mu_study/core/admin/credential_slips_source.dart';
- import 'package:mu_study/screens/admin/helpers/admin_labels.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/admin/users/admin_users_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/core/admin/credential_slips_source.dart';
import 'package:mu_study/screens/admin/helpers/admin_labels.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class AdminUsersScreen extends ConsumerStatefulWidget {
  const AdminUsersScreen({super.key});

  @override
  ConsumerState<AdminUsersScreen> createState() => _AdminUsersScreenState();
}

class _AdminUsersScreenState extends ConsumerState<AdminUsersScreen> {
  Future<void> _createUser() async {
    final username = TextEditingController(text: '@');
```

```
final password = TextEditingController();
final name = TextEditingController();
var role = UserRole.student;
var sphere = Sphere.math;
String? selectedGroupId;

final groupsRes = await supabase.from('groups').select('id, title');
final groups = groupsRes as List<dynamic>;

if (!mounted) return;

await showDialog<void>(
  context: context,
  builder: (ctx) => StatefulBuilder(
    builder: (ctx, setLocal) {
      final l10n = ctx.l10n;
      return AlertDialog(
        title: Text(l10n.adminCreateUser),
        content: SingleChildScrollView(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              TextField(
                controller: username,
                decoration: InputDecoration(
                  labelText: l10n.loginUsername,
                  hintText: '@username',
                ),
              ),
              const SizedBox(height: 16),
              TextField(
                controller: password,
                decoration: InputDecoration(labelText: l10n.adminPassword),
                obscureText: true,
              ),
              const SizedBox(height: 16),
              TextField(
                controller: name,
                decoration: InputDecoration(labelText: l10n.adminFullName),
              ),
              const SizedBox(height: 16),
              DropdownButtonFormField<UserRole>(
                initialValue: role,
                decoration: InputDecoration(labelText: l10n.adminRole),
                items: UserRole.values
                    .map(
                      (r) => DropdownMenuItem(
                        value: r,
                        child: Text(localizedRole(l10n, r).toUpperCase()),
                      ),
                    )
                    .toList(),
                onChanged: (v) =>
                    setLocal(() => role = v ?? UserRole.student),
              ),
              if (role == UserRole.teacher) ...[
                const SizedBox(height: 16),
                DropdownButtonFormField<Sphere>(
                  initialValue: sphere,
                  decoration: InputDecoration(labelText: l10n.adminSphere),
                  items: Sphere.values
                      .map(
                        (s) => DropdownMenuItem(
                          value: s,
                          child: Text(s.l10nLabel(l10n)),
                        ),
                      )
                      .toList(),
                  onChanged: (v) =>
                      setLocal(() => sphere = v ?? Sphere.math),
                ),
              ],
              if (role == UserRole.student) ...[
```

```
          const SizedBox(height: 16),
          DropdownButtonFormField<String>(
            initialValue: selectedGroupId,
            decoration: InputDecoration(
              labelText: l10n.adminInitialGroupOptional,
            ),
            items: groups
                .map(
                  (g) => DropdownMenuItem(
                    value: g['id'] as String,
                    child: Text(g['title'] as String),
                  ),
                )
                .toList(),
            onChanged: (v) => setLocal(() => selectedGroupId = v),
          ),
        ],
      ],
    ),
  ),
  actions: [
    TextButton(
      onPressed: () => Navigator.pop(ctx),
      child: Text(l10n.commonCancel),
    ),
    ElevatedButton(
      onPressed: () async {
        try {
          final userId = await supabase.rpc(
            'admin_create_user',
            params: {
              'p_username': username.text.trim(),
              'p_password': password.text,
              'p_full_name': name.text.trim(),
              'p_role': role.name,
              if (role == UserRole.teacher) 'p_sphere': sphere.name,
            },
          );

          if (role == UserRole.student && selectedGroupId != null) {
            final studentRes = await supabase
                .from('students')
                .select('id')
                .eq('profile_id', userId)
                .single();
            final studentId = studentRes['id'] as String;

            await supabase.from('group_students').insert({
              'group_id': selectedGroupId,
              'student_id': studentId,
            });
          }

          await saveLoginPassword(
            userId: '$userId',
            password: password.text,
          );

          if (ctx.mounted) Navigator.pop(ctx);
          ref.invalidate(allProfilesProvider);
          if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(content: Text(l10n.adminUserCreated)),
            );
          }
        } on PostgrestException catch (e) {
          if (ctx.mounted) {
            ScaffoldMessenger.of(
              ctx,
            ).showSnackBar(SnackBar(content: Text(e.message)));
          }
        }
      },
```

```
              child: Text(l10n.adminCreateUserButton),
            ),
          ],
        );
      },
    ),
  );
}

Future<void> _resetPassword(Profile profile) async {
  final password = TextEditingController();
  await showDialog<void>(
    context: context,
    builder: (ctx) {
      final l10n = ctx.l10n;
      return AlertDialog(
        title: Text(l10n.adminResetPasswordTitle(profile.username)),
        content: TextField(
          controller: password,
          decoration: InputDecoration(labelText: l10n.adminPassword),
          obscureText: true,
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx),
            child: Text(l10n.commonCancel),
          ),
          ElevatedButton(
            onPressed: () async {
              await supabase.rpc(
                'admin_reset_password',
                params: {
                  'p_user_id': profile.id,
                  'p_password': password.text,
                },
              );
              await saveLoginPassword(
                userId: profile.id,
                password: password.text,
              );
              if (ctx.mounted) Navigator.pop(ctx);
            },
            child: Text(l10n.commonReset),
          ),
        ],
      );
    },
  );
}

Future<void> _toggleDisable(Profile profile) async {
  await supabase
      .from('profiles')
      .update({'is_disabled': !profile.isDisabled})
      .eq('id', profile.id);
  ref.invalidate(allProfilesProvider);
}

@override
Widget build(BuildContext context) {
  final profiles = ref.watch(allProfilesProvider);
  final l10n = context.l10n;

  return Scaffold(
    appBar: AppBar(title: Text(l10n.adminUsers)),
    floatingActionButtonLocation: shellFabLocation(context),
    floatingActionButton: FloatingActionButton(
      onPressed: _createUser,
      child: const Icon(Icons.person_add_alt_1),
    ),
    body: profiles.when(
      data: (page) {
        final list = page.items;
```

```
          return ListView.builder(
            padding: EdgeInsets.fromLTRB(
              20,
              20,
              20,
              shellContentBottomPadding(context, hasFab: true),
            ),
            itemCount: list.length + (page.hasMore ? 1 : 0),
            itemBuilder: (_, i) {
              if (i == list.length) {
                return LoadMoreButton(
                  hasMore: page.hasMore,
                  loading: page.loadingMore,
                  onPressed: () =>
                      ref.read(allProfilesProvider.notifier).loadMore(),
                );
              }
              final p = list[i];
              return PremiumCard(
                padding: const EdgeInsets.symmetric(
                  horizontal: 16,
                  vertical: 8,
                ),
                child: ListTile(
                  contentPadding: EdgeInsets.zero,
                  leading: CircleAvatar(
                    backgroundColor: p.isDisabled
                        ? Colors.grey
                        : AppColors.coffeeBrown,
                    child: const Icon(
                      Icons.person,
                      color: Colors.white,
                      size: 20,
                    ),
                  ),
                  title: Text(
                    p.fullName.isNotEmpty ? p.fullName : p.username,
                    style: const TextStyle(fontWeight: FontWeight.w700),
                  ),
                  subtitle: Text(
                    l10n.adminRoleLine(
                      localizedRole(l10n, p.role).toUpperCase(),
                      p.username,
                    ),
                  ),
                  trailing: PopupMenuButton<String>(
                    onSelected: (v) {
                      if (v == 'reset') _resetPassword(p);
                      if (v == 'disable') _toggleDisable(p);
                    },
                    itemBuilder: (_) => [
                      PopupMenuItem(
                        value: 'reset',
                        child: Text(l10n.adminResetPassword),
                      ),
                      PopupMenuItem(
                        value: 'disable',
                        child: Text(
                          p.isDisabled ? l10n.adminEnable : l10n.adminDisable,
                        ),
                      ),
                    ],
                  ),
                ),
              );
            },
          );
        },
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
      ),
    );
  }
```

```
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/admin/users/admin_users_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/auth/language_selection_screen.dart

Splash, login, language, setup-required, disabled account. This file (language selection screen) is one concrete piece of that layer.

Line count: 266. Import count: 8. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/core/locale/app_locale.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/settings/locale_provider.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/auth/language_selection_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/core/locale/app_locale.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/settings/locale_provider.dart';

class LanguageSelectionScreen extends ConsumerStatefulWidget {
  const LanguageSelectionScreen({super.key});

  @override
  ConsumerState<LanguageSelectionScreen> createState() =>
      _LanguageSelectionScreenState();
}

class _LanguageSelectionScreenState
    extends ConsumerState<LanguageSelectionScreen> {
  AppLanguage? _selected;
  bool _saving = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;
      setState(() {
        _selected = AppLanguage.fromLocale(ref.read(localeProvider));
      });
    });
  }

  Future<void> _continue() async {
    final lang = _selected;
    if (lang == null || _saving) return;
```

```
    setState(() => _saving = true);
    try {
      await ref.read(localeProvider.notifier).setLanguage(lang);
      if (!mounted) return;
      context.go('/boot');
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final profile = ref.watch(currentProfileProvider).valueOrNull;

    return Scaffold(
      backgroundColor: AppColors.cream,
      body: SafeArea(
        child: Center(
          child: ConstrainedBox(
            constraints: const BoxConstraints(maxWidth: 480),
            child: Padding(
              padding: const EdgeInsets.all(28),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  const Spacer(),
                  Text(
                    l10n.appName,
                    textAlign: TextAlign.center,
                    style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                      fontWeight: FontWeight.w800,
                      color: AppColors.accent(context),
                    ),
                  ),
                  const SizedBox(height: 12),
                  Text(
                    l10n.languageTitle,
                    textAlign: TextAlign.center,
                    style: Theme.of(context).textTheme.titleLarge?.copyWith(
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                  const SizedBox(height: 8),
                  Text(
                    l10n.languageSubtitle,
                    textAlign: TextAlign.center,
                    style: TextStyle(
                      color: AppColors.inkAlpha(context, 0.55),
                      height: 1.4,
                    ),
                  ),
                  if (profile != null) ...[
                    const SizedBox(height: 8),
                    Text(
                      profile.fullName.isNotEmpty
                          ? profile.fullName
                          : profile.username,
                      textAlign: TextAlign.center,
                      style: TextStyle(
                        fontWeight: FontWeight.w600,
                        color: AppColors.inkAlpha(context, 0.4),
                      ),
                    ),
                  ],
                  const SizedBox(height: 32),
                  _LanguageCard(
                    language: AppLanguage.english,
                    title: 'English',
                    subtitle: l10n.languageEnglish,
                    flag: 'EN',
                    selected: _selected == AppLanguage.english,
                    onTap: () =>
```

```
                      setState(() => _selected = AppLanguage.english),
                ),
                const SizedBox(height: 12),
                _LanguageCard(
                  language: AppLanguage.russian,
                  title: '[redacted]',
                  subtitle: l10n.languageRussian,
                  flag: 'RU',
                  selected: _selected == AppLanguage.russian,
                  onTap: () =>
                      setState(() => _selected = AppLanguage.russian),
                ),
                const SizedBox(height: 12),
                _LanguageCard(
                  language: AppLanguage.uzbek,
                  title: 'O‘zbekcha',
                  subtitle: l10n.languageUzbek,
                  flag: 'UZ',
                  selected: _selected == AppLanguage.uzbek,
                  onTap: () => setState(() => _selected = AppLanguage.uzbek),
                ),
                const Spacer(),
                FilledButton(
                  onPressed: _saving || _selected == null ? null : _continue,
                  style: FilledButton.styleFrom(
                    backgroundColor: AppColors.coffeeBrown,
                    padding: const EdgeInsets.symmetric(vertical: 16),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(14),
                    ),
                  ),
                  child: _saving
                      ? const SizedBox(
                          width: 22,
                          height: 22,
                          child: CircularProgressIndicator(
                            strokeWidth: 2,
                            color: Colors.white,
                          ),
                        )
                      : Text(l10n.languageContinue),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}

class _LanguageCard extends StatelessWidget {
  const _LanguageCard({
    required this.language,
    required this.title,
    required this.subtitle,
    required this.flag,
    required this.selected,
    required this.onTap,
  });

  final AppLanguage language;
  final String title;
  final String subtitle;
  final String flag;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
```

```
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(16),
        child: Ink(
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
          decoration: BoxDecoration(
            color: AppColors.controlSurface(context),
            borderRadius: BorderRadius.circular(16),
            border: Border.all(
              color: selected
                  ? AppColors.accent(context)
                  : AppColors.inkAlpha(context, 0.08),
              width: selected ? 2 : 1,
            ),
            boxShadow: selected
                ? [
                    BoxShadow(
                      color: AppColors.accent(context).withValues(alpha: 0.12),
                      blurRadius: 16,
                      offset: const Offset(0, 6),
                    ),
                  ]
                : null,
          ),
          child: Row(
            children: [
              Container(
                width: 44,
                height: 44,
                alignment: Alignment.center,
                decoration: BoxDecoration(
                  color: selected
                      ? AppColors.accent(context)
                      : AppColors.softWash(context, lightAlpha: 0.7),
                  borderRadius: BorderRadius.circular(12),
                ),
                child: Text(
                  flag,
                  style: TextStyle(
                    fontWeight: FontWeight.w800,
                    fontSize: 13,
                    color: selected ? Colors.white : AppColors.accent(context),
                  ),
                ),
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      title,
                      style: const TextStyle(
                        fontWeight: FontWeight.w800,
                        fontSize: 16,
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      subtitle,
                      style: TextStyle(
                        fontSize: 12,
                        color: AppColors.inkAlpha(context, 0.5),
                      ),
                    ),
                  ],
                ),
              ),
              Icon(
                selected
                    ? Icons.radio_button_checked_rounded
                    : Icons.radio_button_off_rounded,
                color: selected
```

```
                    ? AppColors.accent(context)
                    : AppColors.inkAlpha(context, 0.25),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/auth/language_selection_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/auth/login_screen.dart

Splash, login, language, setup-required, disabled account. This file (login screen) is one concrete piece of that layer.

Line count: 450. Import count: 13. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math' as math;
- import 'package:flutter/foundation.dart';
- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';
- import 'package:mu_study/components/brand/app_logo.dart';
- import 'package:mu_study/components/buttons/primary_button.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/constants/env.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/auth/login_screen.dart**

```
import 'dart:math' as math;

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/components/brand/app_logo.dart';
import 'package:mu_study/components/buttons/primary_button.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/constants/env.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';

class LoginScreen extends ConsumerStatefulWidget {
  const LoginScreen({super.key});

  @override
  ConsumerState<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends ConsumerState<LoginScreen> {
  final _usernameController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _loading = false;
  String? _error;

  @override
```

```
  void dispose() {
    _usernameController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    final l10n = context.l10n;
    if (_usernameController.text.isEmpty || _passwordController.text.isEmpty) {
      setState(() => _error = l10n.loginMissingFields);
      return;
    }

    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      await ref
          .read(authRepositoryProvider)
          .signIn(_usernameController.text.trim(), _passwordController.text);
      ref.invalidate(currentProfileProvider);
      if (!mounted) return;
      context.go('/boot');
    } on AuthException catch (_) {
      setState(() {
        _error = l10n.loginIncorrect;
      });
    } on PostgrestException catch (e) {
      setState(() {
        _error = _messageForPostgrest(e, l10n.loginConfigError);
      });
    } catch (e) {
      setState(() {
        _error = kDebugMode ? 'Sign in error: $e' : l10n.loginFailed;
      });
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  String _messageForPostgrest(PostgrestException e, String configError) {
    final msg = e.message.toLowerCase();
    if (msg.contains('jwt') ||
        msg.contains('api key') ||
        msg.contains('invalid') && msg.contains('key')) {
      return configError;
    }
    return e.message;
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return Scaffold(
      backgroundColor: Theme.of(context).scaffoldBackgroundColor,
      body: Stack(
        children: [
          Positioned(
            top: -100,
            right: -100,
            child: Container(
              width: 300,
              height: 300,
              decoration: BoxDecoration(
                color: AppColors.softWash(context, lightAlpha: 0.1),
                shape: BoxShape.circle,
              ),
            ),
          ),
          Positioned(
            bottom: -50,
            left: -50,
```

```
          child: Container(
            width: 200,
            height: 200,
            decoration: BoxDecoration(
              color: AppColors.accent(context).withValues(alpha: 0.05),
              shape: BoxShape.circle,
            ),
          ),
        ),
        const _LoginNatureScenery(),
        SafeArea(
          child: Center(
            child: SingleChildScrollView(
              padding: const EdgeInsets.symmetric(horizontal: 32),
              child: ConstrainedBox(
                constraints: const BoxConstraints(maxWidth: 400),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    const Center(child: AppLogoImage(size: 88)),
                    const SizedBox(height: 32),
                    Center(
                      child: Text(
                        l10n.appName,
                        style: Theme.of(
                          context,
                        ).textTheme.displaySmall?.copyWith(letterSpacing: -1),
                      ),
                    ),
                    const SizedBox(height: 8),
                    Center(
                      child: Text(
                        l10n.loginSubtitle,
                        textAlign: TextAlign.center,
                        style: Theme.of(context).textTheme.bodyLarge
                            ?.copyWith(
                              color: AppColors.inkAlpha(context, 0.5),
                              fontWeight: FontWeight.w500,
                            ),
                      ),
                    ),
                    const SizedBox(height: 48),
                    Text(
                      l10n.loginUsername.toUpperCase(),
                      style: Theme.of(context).textTheme.labelSmall?.copyWith(
                        letterSpacing: 1.2,
                        fontWeight: FontWeight.w800,
                        color: AppColors.accent(
                          context,
                        ).withValues(alpha: 0.6),
                      ),
                    ),
                    const SizedBox(height: 8),
                    TextField(
                      controller: _usernameController,
                      decoration: const InputDecoration(
                        hintText: '@username',
                        prefixIcon: Icon(Icons.alternate_email, size: 20),
                      ),
                      textInputAction: TextInputAction.next,
                    ),
                    const SizedBox(height: 24),
                    Text(
                      l10n.loginPassword.toUpperCase(),
                      style: Theme.of(context).textTheme.labelSmall?.copyWith(
                        letterSpacing: 1.2,
                        fontWeight: FontWeight.w800,
                        color: AppColors.accent(
                          context,
                        ).withValues(alpha: 0.6),
                      ),
                    ),
                    const SizedBox(height: 8),
```

```
                    TextField(
                      controller: _passwordController,
                      decoration: const InputDecoration(
                        hintText: '••••••••',
                        prefixIcon: Icon(Icons.lock_outline, size: 20),
                      ),
                      obscureText: true,
                      onSubmitted: (_) => _login(),
                    ),
                    if (_error != null) ...[
                      const SizedBox(height: 16),
                      Container(
                        padding: const EdgeInsets.symmetric(
                          horizontal: 16,
                          vertical: 12,
                        ),
                        decoration: BoxDecoration(
                          color: AppColors.errorRed.withValues(alpha: 0.1),
                          borderRadius: BorderRadius.circular(8),
                          border: Border.all(
                            color: AppColors.errorRed.withValues(alpha: 0.2),
                          ),
                        ),
                        child: Row(
                          children: [
                            const Icon(
                              Icons.error_outline,
                              color: AppColors.errorRed,
                              size: 18,
                            ),
                            const SizedBox(width: 12),
                            Expanded(
                              child: Text(
                                _error!,
                                style: const TextStyle(
                                  color: AppColors.errorRed,
                                  fontSize: 13,
                                  fontWeight: FontWeight.w600,
                                ),
                              ),
                            ),
                          ],
                        ),
                      ),
                    ],
                    if (!Env.isConfigured) ...[
                      const SizedBox(height: 16),
                      Text(
                        context.l10n.loginConfigMissing,
                        style: Theme.of(context).textTheme.bodySmall
                            ?.copyWith(
                              color: AppColors.mutedGold,
                              fontWeight: FontWeight.w600,
                            ),
                      ),
                    ],
                    const SizedBox(height: 40),
                    PrimaryButton(
                      label: l10n.loginSignIn,
                      loading: _loading,
                      onPressed: _login,
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

```
class _LoginNatureScenery extends StatefulWidget {
  const _LoginNatureScenery();

  @override
  State<_LoginNatureScenery> createState() => _LoginNatureSceneryState();
}

class _LoginCloud {
  const _LoginCloud({
    required this.asset,
    required this.top,
    required this.size,
    required this.speed,
    required this.phase,
    required this.opacity,
  });

  final String asset;
  final double top;
  final double size;
  final double speed;
  final double phase;
  final double opacity;
}

class _LoginNatureSceneryState extends State<_LoginNatureScenery>
    with SingleTickerProviderStateMixin {
  static const _clouds = [
    _LoginCloud(
      asset: PaperAssets.whiteCloud,
      top: 72,
      size: 64,
      speed: 1.15,
      phase: 0.05,
      opacity: 0.72,
    ),
    _LoginCloud(
      asset: PaperAssets.flatCloud,
      top: 130,
      size: 48,
      speed: 0.75,
      phase: 0.42,
      opacity: 0.48,
    ),
    _LoginCloud(
      asset: PaperAssets.grayCloud,
      top: 168,
      size: 40,
      speed: 1.35,
      phase: 0.7,
      opacity: 0.32,
    ),
    _LoginCloud(
      asset: PaperAssets.whiteCloud,
      top: 210,
      size: 36,
      speed: 0.95,
      phase: 0.22,
      opacity: 0.4,
    ),
    _LoginCloud(
      asset: PaperAssets.flatCloud,
      top: 54,
      size: 30,
      speed: 1.55,
      phase: 0.88,
      opacity: 0.28,
    ),
  ];

  AnimationController? _ctrl;

  void _syncController(bool nature) {
```

```
  if (nature && _ctrl == null) {
    _ctrl = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 10),
    )..repeat();
  } else if (!nature && _ctrl != null) {
    _ctrl!.dispose();
    _ctrl = null;
  }
}

@override
void didChangeDependencies() {
  super.didChangeDependencies();
  _syncController(isNaturePaper(context));
}

@override
void dispose() {
  _ctrl?.dispose();
  _ctrl = null;
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final nature = isNaturePaper(context);
  _syncController(nature);
  final ctrl = _ctrl;
  if (!nature || ctrl == null) return const SizedBox.shrink();
  final width = MediaQuery.sizeOf(context).width;

  return IgnorePointer(
    child: AnimatedBuilder(
      animation: ctrl,
      builder: (context, _) {
        final t = ctrl.value;
        final angle = t * 2 * math.pi;
        final tealX = 28 + math.sin(angle * 0.5) * 18;
        final tealY = 110 + math.sin(angle) * 12;
        final tealTilt = -0.08 + math.sin(angle) * 0.08;
        final creamX = 40 + math.sin(angle * 0.4 + 1.8) * 14;
        final creamY = 82 + math.sin(angle + 1.2) * 10;
        final creamTilt = 0.1 + math.sin(angle + 1.2) * 0.07;

        return Stack(
          children: [
            for (final c in _clouds)
              Positioned(
                left:
                    width -
                    (((t * c.speed + c.phase) % 1.0) * (width + c.size + 40)),
                top: c.top,
                child: Image.asset(
                  c.asset,
                  width: c.size,
                  height: c.size,
                  opacity: AlwaysStoppedAnimation(c.opacity),
                  filterQuality: FilterQuality.medium,
                ),
              ),
            Positioned(
              left: tealX,
              top: tealY,
              child: Transform.rotate(
                angle: tealTilt,
                child: Image.asset(
                  PaperAssets.tealBird,
                  width: 40,
                  height: 40,
                  filterQuality: FilterQuality.medium,
                ),
              ),
```

```
                ),
                Positioned(
                  right: creamX,
                  top: creamY,
                  child: Transform.rotate(
                    angle: creamTilt,
                    child: Image.asset(
                      PaperAssets.creamBird,
                      width: 32,
                      height: 32,
                      filterQuality: FilterQuality.medium,
                    ),
                  ),
                ),
                Positioned(
                  bottom: 12,
                  left: 8,
                  child: Image.asset(
                    PaperAssets.singleTree,
                    height: 96,
                    opacity: const AlwaysStoppedAnimation(0.75),
                    filterQuality: FilterQuality.medium,
                  ),
                ),
                Positioned(
                  bottom: 8,
                  right: 8,
                  child: Image.asset(
                    PaperAssets.twoTrees,
                    height: 110,
```

**lib/screens/auth/login_screen.dart — continued (2/2)**

```
                    opacity: const AlwaysStoppedAnimation(0.75),
                    filterQuality: FilterQuality.medium,
                  ),
                ),
                Positioned(
                  bottom: 96,
                  right: 52,
                  child: Transform.rotate(
                    angle: 0.3,
                    child: Image.asset(
                      PaperAssets.whiteDaisy,
                      width: 24,
                      height: 24,
                      opacity: const AlwaysStoppedAnimation(0.85),
                      filterQuality: FilterQuality.medium,
                    ),
                  ),
                ),
              ],
            );
          },
        ),
      );
    }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/auth/login_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/auth/setup_required_screen.dart

Splash, login, language, setup-required, disabled account. This file (setup required screen) is one concrete piece of that layer.

Line count: 81. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_localizations/flutter_localizations.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_theme.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/auth/setup_required_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_theme.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class SetupRequiredScreen extends StatelessWidget {
  const SetupRequiredScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      onGenerateTitle: (ctx) => AppLocalizations.of(ctx).appName,
      theme: AppTheme.light(),
      supportedLocales: AppLocalizations.supportedLocales,
      localizationsDelegates: const [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      home: Builder(
        builder: (context) {
          final l10n = context.l10n;
          return Scaffold(
            body: SafeArea(
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      l10n.appName,
                      style: Theme.of(context).textTheme.headlineMedium
                          ?.copyWith(
                            fontWeight: FontWeight.w700,
                            color: AppColors.darkChocolate,
                          ),
                    ),
                    const SizedBox(height: 12),
                    Text(
                      l10n.setupRequiredTitle,
```

```
                    style: Theme.of(context).textTheme.titleMedium?.copyWith(
                      fontWeight: FontWeight.w700,
                    ),
                  ),
                  const SizedBox(height: 8),
                  Text(
                    l10n.setupRequiredBody,
                    style: Theme.of(context).textTheme.bodyLarge,
                  ),
                  const SizedBox(height: 24),
                  Text(
                    l10n.setupOption1,
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                  const SizedBox(height: 8),
                  const SelectableText(
                    'cp .env.example .env\n'
                    '# Edit .env and set SUPABASE_ANON_KEY from Supabase → Settings → API\n'
                    'flutter run',
                  ),
                  const SizedBox(height: 20),
                  Text(
                    l10n.setupOption2,
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                  const SizedBox(height: 8),
                  const SelectableText(
                    'flutter run --dart-define-from-file=.env',
                  ),
                ],
              ),
            ),
          ),
        ),
      );
    },
  ),
);
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/auth/setup_required_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/auth/splash_screen.dart

Splash, login, language, setup-required, disabled account. This file (splash screen) is one concrete piece of that layer.

Line count: 70. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:flutter_animate/flutter_animate.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/brand/app_logo.dart';
- import 'package:mu_study/core/theme/app_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/auth/splash_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_animate/flutter_animate.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/brand/app_logo.dart';
import 'package:mu_study/core/theme/app_colors.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    Future<void>.delayed(const Duration(milliseconds: 1200), () {
      if (mounted) context.go('/boot');
    });
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;
    final bg = theme.scaffoldBackgroundColor;
    final fg = isDark ? AppColors.darkTextPrimary : AppColors.darkChocolate;

    final overlay = isDark
        ? SystemUiOverlayStyle.light
        : SystemUiOverlayStyle.dark;

    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: overlay.copyWith(
        statusBarColor: Colors.transparent,
        systemNavigationBarColor: bg,
        systemNavigationBarIconBrightness: isDark
            ? Brightness.light
```

```
            : Brightness.dark,
      ),
      child: Scaffold(
        backgroundColor: bg,
        body: Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const AppLogoImage(size: 88)
                  .animate(onPlay: (c) => c.repeat(reverse: true))
                  .scale(
                    begin: const Offset(0.95, 0.95),
                    end: const Offset(1.05, 1.05),
                    duration: 900.ms,
                  ),
              const SizedBox(height: 16),
              Text(
                'MU Study',
                style: theme.textTheme.headlineMedium?.copyWith(
                  fontWeight: FontWeight.w700,
                  color: fg,
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/auth/splash_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/chat/list/chat_list_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 345. Import count: 15. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_animate/flutter_animate.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/components/layouts/shell_nav_bar.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/chat/chat_providers.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/list/chat_list_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_animate/flutter_animate.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/layouts/shell_nav_bar.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';

class ChatListScreen extends ConsumerStatefulWidget {
  const ChatListScreen({super.key, required this.basePath});

  final String basePath;

  @override
  ConsumerState<ChatListScreen> createState() => _ChatListScreenState();
}

class _ChatListScreenState extends ConsumerState<ChatListScreen> {
```

```
final _search = TextEditingController();
String _query = '';

@override
void dispose() {
  _search.dispose();
  super.dispose();
}

String _preview(ChatRoomSummary room, AppLocalizations l10n) {
  if (room.lastMessageId == null) return l10n.chatNoMessagesYet;
  if (room.lastMessageDeleted) return l10n.chatMessageDeleted;
  return switch (room.lastMessageType) {
    ChatMessageType.image => l10n.chatPreviewImage,
    ChatMessageType.video => l10n.chatPreviewVideo,
    ChatMessageType.voice => l10n.chatPreviewVoice,
    ChatMessageType.file =>
      room.lastMessageText?.trim().isNotEmpty == true
          ? room.lastMessageText!.trim()
          : l10n.chatPreviewFile,
    ChatMessageType.text || null =>
      room.lastMessageText?.trim().isNotEmpty == true
          ? room.lastMessageText!.trim()
          : l10n.chatNoMessagesYet,
  };
}

String _formatTime(DateTime? dt) {
  if (dt == null) return '';
  final local = dt.toLocal();
  final now = DateTime.now();
  if (local.year == now.year &&
      local.month == now.month &&
      local.day == now.day) {
    return DateFormat.Hm().format(local);
  }
  if (now.difference(local).inDays < 7) {
    return DateFormat.E().format(local);
  }
  return DateFormat.MMMd().format(local);
}

@override
Widget build(BuildContext context) {
  final l10n = context.l10n;
  final roomsAsync = ref.watch(chatRoomsProvider);
  final isNeu = Neumorphic.isActive(context);

  return Scaffold(
    appBar: AppBar(title: Text(l10n.chatTitle)),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.fromLTRB(16, 8, 16, 12),
          child: Row(
            children: [
              Expanded(
                child: NeuSearchField(
                  controller: _search,
                  hintText: l10n.chatSearchHint,
                  onChanged: (v) =>
                      setState(() => _query = v.trim().toLowerCase()),
                ),
              ),
              if (isNaturePaper(context))
                const Padding(
                  padding: EdgeInsets.only(left: 10),
                  child: PaperDecor(
                    PaperAssets.tealBird,
                    size: 36,
                    angle: -0.06,
                  ),
                ),
```

```
              ],
            ),
          ),
          Expanded(
            child: roomsAsync.when(
              loading: () => const Center(child: CircularProgressIndicator()),
              error: (e, _) =>
                  Center(child: Text(l10n.commonErrorWithDetails('$e'))),
              data: (rooms) {
                final filtered = _query.isEmpty
                    ? rooms
                    : rooms
                          .where(
                            (r) =>
                                r.groupTitle.toLowerCase().contains(_query) ||
                                r.teacherName.toLowerCase().contains(_query),
                          )
                          .toList();

                if (filtered.isEmpty) {
                  return EmptyState(
                    icon: Icons.chat_bubble_outline_rounded,
                    title: l10n.chatEmptyTitle,
                    subtitle: l10n.chatEmptySubtitle,
                  );
                }

                return ListView.separated(
                  clipBehavior: isNeu ? Clip.none : Clip.hardEdge,
                  padding: EdgeInsets.fromLTRB(
                    isNeu ? 16 : 12,
                    isNeu ? 4 : 0,
                    isNeu ? 16 : 12,
                    shellContentBottomPadding(context),
                  ),
                  itemCount: filtered.length,
                  separatorBuilder: (_, _) => SizedBox(height: isNeu ? 4 : 4),
                  itemBuilder: (context, index) {
                    final room = filtered[index];
                    return _ChatRoomTile(
                          room: room,
                          preview: _preview(room, l10n),
                          timeLabel: _formatTime(room.lastMessageAt),
                          onTap: () =>
                              context.push('${widget.basePath}/${room.id}'),
                        )
                        .animate()
                        .fadeIn(duration: 220.ms, delay: (20 * index).ms)
                        .slideY(
                          begin: 0.04,
                          end: 0,
                          duration: 280.ms,
                          delay: (20 * index).ms,
                          curve: Curves.easeOutCubic,
                        );
                  },
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}

class _ChatRoomTile extends ConsumerWidget {
  const _ChatRoomTile({
    required this.room,
    required this.preview,
    required this.timeLabel,
    required this.onTap,
  });
```

```
final ChatRoomSummary room;
final String preview;
final String timeLabel;
final VoidCallback onTap;

@override
Widget build(BuildContext context, WidgetRef ref) {
  final scheme = Theme.of(context).colorScheme;
  final me = ref.watch(currentProfileProvider).valueOrNull;
  final unread = room.unreadCount;
  final l10n = context.l10n;
  final isNeu = Neumorphic.isActive(context);
  final accent = AppColors.accent(context);

  final avatarLetter = Text(
    room.groupTitle.isNotEmpty
        ? room.groupTitle.characters.first.toUpperCase()
        : '?',
    style: TextStyle(
      fontWeight: FontWeight.w800,
      color: accent,
      fontSize: 18,
    ),
  );

  final content = Padding(
    padding: EdgeInsets.symmetric(
      horizontal: isNeu ? 4 : 0,
      vertical: isNeu ? 2 : 0,
    ),
    child: Row(
      children: [
        if (isNeu)
          NeuAvatar(child: avatarLetter)
        else
          CircleAvatar(
            radius: 26,
            backgroundColor: accent.withValues(alpha: 0.18),
            child: avatarLetter,
          ),
        const SizedBox(width: 12),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Row(
                children: [
                  Expanded(
                    child: Text(
                      room.groupTitle,
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                      style: Theme.of(context).textTheme.titleMedium
                          ?.copyWith(
                            fontWeight: unread > 0
                                ? FontWeight.w800
                                : FontWeight.w700,
                          ),
                    ),
                  ),
                  if (timeLabel.isNotEmpty)
                    Text(
                      timeLabel,
                      style: Theme.of(context).textTheme.bodySmall?.copyWith(
                        color: unread > 0
                            ? AppColors.goldAccent(context)
                            : scheme.onSurface.withValues(alpha: 0.45),
                        fontWeight: unread > 0
                            ? FontWeight.w700
                            : FontWeight.w500,
                      ),
                    ),
                ],
```

```
            ),
            const SizedBox(height: 2),
            Text(
              l10n.chatTeacherMembers(room.teacherName, room.memberCount),
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: Theme.of(context).textTheme.bodySmall?.copyWith(
                color: AppColors.inkAlpha(context, 0.5),
              ),
            ),
            const SizedBox(height: 4),
            Row(
              children: [
                Expanded(
                  child: Text(
                    room.lastMessageSenderId != null &&
                            room.lastMessageSenderId == me?.id
                        ? '${l10n.chatYou}: $preview'
                        : preview,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: Theme.of(context).textTheme.bodyMedium?.copyWith(
                      fontWeight: unread > 0
                          ? FontWeight.w600
                          : FontWeight.w400,
                      color: scheme.onSurface.withValues(
                        alpha: unread > 0 ? 0.9 : 0.65,
                      ),
                    ),
                  ),
                ),
                if (unread > 0) ...[
                  const SizedBox(width: 8),
                  Container(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 8,
                      vertical: 2,
                    ),
                    decoration: BoxDecoration(
                      color: AppColors.goldAccent(context),
                      borderRadius: BorderRadius.circular(999),
                      boxShadow: isNeu
                          ? [
                              BoxShadow(
                                color: Neumorphic.shadowColor(
                                  context,
                                ).withValues(alpha: 0.35),
                                offset: const Offset(2, 2),
                                blurRadius: 4,
                              ),
                            ]
                          : null,
                    ),
                    child: Text(
                      unread > 99 ? '99+' : '$unread',
                      style: TextStyle(
                        color:
                            Theme.of(context).brightness == Brightness.dark
                            ? const Color(0xFF1A1408)
                            : Colors.white,
                        fontSize: 11,
                        fontWeight: FontWeight.w800,
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    ],
  ),
```

```
    );

    if (isNeu) {
      return PremiumCard(
        onTap: onTap,
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
        elevation: NeuElevation.raised,
        child: content,
      );
    }

    final isDark = Theme.of(context).brightness == Brightness.dark;
    return Material(
      color: isDark ? scheme.surfaceContainerHigh : Colors.white,
      borderRadius: BorderRadius.circular(16),
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(16),
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
          child: content,
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/list/chat_list_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/chat/room/chat_room_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 1605. Import count: 27. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:file_picker/file_picker.dart';
- import 'package:flutter/foundation.dart';
- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:image_picker/image_picker.dart';
- import 'package:mu_study/components/chat/chat_media_gallery.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/core/utils/responsive.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/chat/chat_providers.dart';
- import 'package:mu_study/screens/chat/room/helpers/chat_list_entries.dart';
- import 'package:mu_study/screens/chat/room/widgets/attach_chip.dart';
- … and 9 more imports

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/room/chat_room_screen.dart**

```
import 'dart:async';

import 'package:file_picker/file_picker.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:image_picker/image_picker.dart';
import 'package:mu_study/components/chat/chat_media_gallery.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/utils/responsive.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';
import 'package:mu_study/screens/chat/room/helpers/chat_list_entries.dart';
import 'package:mu_study/screens/chat/room/widgets/attach_chip.dart';
import 'package:mu_study/screens/chat/room/widgets/chat_date_chip.dart';
```

```
import 'package:mu_study/screens/chat/room/widgets/edit_message_dialog.dart';
import 'package:mu_study/screens/chat/room/widgets/message_bubble.dart';
import 'package:mu_study/screens/chat/room/widgets/reply_composer_banner.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:record/record.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class ChatRoomScreen extends ConsumerStatefulWidget {
  const ChatRoomScreen({super.key, required this.roomId});

  final String roomId;

  @override
  ConsumerState<ChatRoomScreen> createState() => _ChatRoomScreenState();
}

class _ChatRoomScreenState extends ConsumerState<ChatRoomScreen>
    with WidgetsBindingObserver {
  static const _nearBottomThreshold = 120.0;

  static const _recentActivityWindow = Duration(minutes: 3);

  final _text = TextEditingController();
  final _scroll = ScrollController();
  final _recorder = AudioRecorder();
  final _focus = FocusNode();

  bool _sending = false;
  bool _attachOpen = false;
  bool _recording = false;
  double _uploadProgress = 0;
  String? _uploadLabel;
  Duration _recordElapsed = Duration.zero;
  Timer? _recordTick;
  Timer? _typingIdle;
  Timer? _presenceHeartbeat;
  Timer? _presencePoll;
  RealtimeChannel? _typingChannel;
  List<String> _typingNames = const [];
  final Set<String> _presenceOnlineIds = {};
  final Set<String> _recentlyActiveIds = {};
  int _onlineCount = 0;
  bool _markedRead = false;
  bool _presenceSetupInFlight = false;

  bool _stickToBottom = true;
  bool _showScrollDown = false;
  bool _didInitialScroll = false;
  String? _lastTailMessageId;
  int _lastMessageCount = 0;
  bool _loadMoreQueued = false;
  Timer? _scrollFabDebounce;
  Timer? _highlightTimer;
  ChatMessage? _replyTo;
  String? _highlightedMessageId;
  final _messageKeys = <String, GlobalKey>{};
  bool _jumpingToReply = false;
  bool _selectionMode = false;
  Set<String> _selectedMessageIds = {};

  GlobalKey _keyForMessage(String id) =>
      _messageKeys.putIfAbsent(id, GlobalKey.new);

  bool _canModerateMessage(ChatMessage message, String? myId, bool isAdmin) {
    if (message.isDeleted) return false;
    if (isAdmin) return true;
    return myId != null && message.senderId == myId;
  }

  /// Students/teachers: own messages only. Admins: any message in the room.
  bool _canDeleteMessage(ChatMessage message, String? myId, bool isAdmin) {
```

```
  return _canModerateMessage(message, myId, isAdmin);
}

Iterable<ChatMessage> _deletableMessages(
  List<ChatMessage> messages,
  String? myId,
  bool isAdmin,
) {
  return messages.where((m) => _canDeleteMessage(m, myId, isAdmin));
}

void _enterSelectionMode({String? messageId}) {
  setState(() {
    _selectionMode = true;
    _selectedMessageIds = messageId == null ? {} : {messageId};
    _attachOpen = false;
  });
  _focus.unfocus();
}

void _exitSelectionMode() {
  setState(() {
    _selectionMode = false;
    _selectedMessageIds = {};
  });
}

void _toggleMessageSelection(String messageId) {
  setState(() {
    if (_selectedMessageIds.contains(messageId)) {
      _selectedMessageIds.remove(messageId);
    } else {
      _selectedMessageIds = {..._selectedMessageIds, messageId};
    }
  });
}

void _selectAllMessages(
  List<ChatMessage> messages,
  String? myId,
  bool isAdmin,
) {
  setState(() {
    _selectedMessageIds = _deletableMessages(messages, myId, isAdmin)
        .map((m) => m.id)
        .toSet();
  });
}

Future<void> _deleteSelected() async {
  if (_selectedMessageIds.isEmpty) return;
  final count = _selectedMessageIds.length;
  final ok = await showDialog<bool>(
    context: context,
    builder: (context) => AlertDialog(
      title: Text(context.l10n.chatDeleteSelectedTitle(count)),
      content: Text(context.l10n.chatDeleteSelectedBody),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: Text(context.l10n.commonCancel),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(context, true),
          child: Text(context.l10n.commonDelete),
        ),
      ],
    ),
  );
  if (ok != true) return;
  final me = ref.read(currentProfileProvider).valueOrNull;
  final isAdmin = me?.role == UserRole.admin;
  final current = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;
```

```
  final ids = current == null
      ? _selectedMessageIds.toList()
      : _deletableMessages(current.messages, me?.id, isAdmin)
          .where((m) => _selectedMessageIds.contains(m.id))
          .map((m) => m.id)
          .toList();
  if (ids.isEmpty) return;
  try {
    await ref
        .read(chatMessagesProvider(widget.roomId).notifier)
        .deleteMessages(ids);
    if (mounted) _exitSelectionMode();
  } catch (e) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),
      );
    }
  }
}

@override
void initState() {
  super.initState();
  WidgetsBinding.instance.addObserver(this);
  _text.addListener(_onTextChanged);
  WidgetsBinding.instance.addPostFrameCallback((_) {
    if (!mounted) return;
    _setupTyping();
    // Don't open the keyboard until the user taps the field.
    _focus.unfocus();
    FocusManager.instance.primaryFocus?.unfocus();
  });
}

@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this);
  _typingIdle?.cancel();
  _recordTick?.cancel();
  _presenceHeartbeat?.cancel();
  _presencePoll?.cancel();
  _scrollFabDebounce?.cancel();
  _highlightTimer?.cancel();
  _scroll.dispose();
  _text.dispose();
  _focus.dispose();
  _recorder.dispose();
  final ch = _typingChannel;
  _typingChannel = null;
  if (ch != null) {
    () async {
      try {
        await ch.untrack();
      } catch (_) {}
      try {
        await supabase.removeChannel(ch);
      } catch (_) {}
    }();
  }
  super.dispose();
}

@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.resumed) {
    _retrackPresence();
  }
}

void _commitOnlineCount({List<String>? typingNames}) {
  final me = ref.read(currentProfileProvider).valueOrNull?.id;
  final merged = <String>{..._presenceOnlineIds, ..._recentlyActiveIds, ?me};
```

```
  final nextCount = merged.length;
  final nextTyping = typingNames ?? _typingNames;
  if (!mounted) return;
  if (nextCount == _onlineCount &&
      typingNames == null &&
      identical(nextTyping, _typingNames)) {
    return;
  }
  setState(() {
    _onlineCount = nextCount;
    if (typingNames != null) _typingNames = typingNames;
  });
}

void _refreshPresence(RealtimeChannel channel, String myId) {
  final onlineIds = <String>{};
  final typingNames = <String>{};

  for (final entry in channel.presenceState()) {
    final keyId = entry.key.trim();
    if (keyId.isNotEmpty) onlineIds.add(keyId);

    for (final presence in entry.presences) {
      final payload = presence.payload;
      final id = (payload['user_id'] as String?)?.trim();
      if (id != null && id.isNotEmpty) onlineIds.add(id);

      final typing = payload['typing'] == true;
      final name = payload['full_name'] as String?;
      final who = id ?? (keyId.isEmpty ? null : keyId);
      if (typing && who != null && who != myId && name != null) {
        typingNames.add(name);
      }
    }
  }

  onlineIds.add(myId);
  _presenceOnlineIds
    ..clear()
    ..addAll(onlineIds);
  _commitOnlineCount(typingNames: typingNames.toList()..sort());
}

void _notePresenceJoin(RealtimePresenceJoinPayload payload) {
  final key = payload.key.trim();
  if (key.isNotEmpty) _presenceOnlineIds.add(key);
  for (final presence in payload.newPresences) {
    final id = (presence.payload['user_id'] as String?)?.trim();
    if (id != null && id.isNotEmpty) _presenceOnlineIds.add(id);
  }
  _commitOnlineCount();
}

void _notePresenceLeave(RealtimePresenceLeavePayload payload) {
  final key = payload.key.trim();
  if (key.isNotEmpty) _presenceOnlineIds.remove(key);
  for (final presence in payload.leftPresences) {
    final id = (presence.payload['user_id'] as String?)?.trim();
    if (id != null && id.isNotEmpty) _presenceOnlineIds.remove(id);
  }
  _commitOnlineCount();
}

void _updateRecentlyActive(List<ChatMessage> messages) {
  final cutoff = DateTime.now().subtract(_recentActivityWindow);
  final active = <String>{};
  for (final m in messages) {
    if (m.isDeleted) continue;
    if (m.createdAt.isBefore(cutoff)) continue;
    if (m.senderId.isNotEmpty) active.add(m.senderId);
  }
  if (active.length == _recentlyActiveIds.length &&
      active.containsAll(_recentlyActiveIds)) {
```

```
      return;
    }
    _recentlyActiveIds
      ..clear()
      ..addAll(active);
    _commitOnlineCount();
  }

  Future<void> _retrackPresence() async {
    final profile = ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    final channel = _typingChannel;
    if (channel == null) {
      await _setupTyping();
      return;
    }
    try {
      await channel.track({
        'user_id': profile.id,
        'full_name': profile.fullName,
        'typing': _text.text.trim().isNotEmpty,
      });
    } catch (_) {}
    if (mounted) _refreshPresence(channel, profile.id);
  }

  Future<void> _setupTyping() async {
    if (_presenceSetupInFlight || _typingChannel != null) return;
    final profile = ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;

    _presenceSetupInFlight = true;
    _presenceHeartbeat?.cancel();
    _presencePoll?.cancel();

    try {
      final previous = _typingChannel;
      _typingChannel = null;
      if (previous != null) {
        try {
          await previous.untrack();
        } catch (_) {}
        try {
          await supabase.removeChannel(previous);
        } catch (_) {}
      }

      final channel = supabase.channel(
        'typing-${widget.roomId}',
        opts: RealtimeChannelConfig(key: profile.id),
      );

      channel
        ..onPresenceSync((_) {
          if (!mounted) return;
          _refreshPresence(channel, profile.id);
        })
        ..onPresenceJoin((payload) {
          if (!mounted) return;
          _notePresenceJoin(payload);
          scheduleMicrotask(() {
            if (mounted) _refreshPresence(channel, profile.id);
          });
        })
        ..onPresenceLeave((payload) {
          if (!mounted) return;
          _notePresenceLeave(payload);
          scheduleMicrotask(() {
            if (mounted) _refreshPresence(channel, profile.id);
          });
        });

      channel.subscribe((status, [_]) async {
```

```
        if (status == RealtimeSubscribeStatus.subscribed) {
          await channel.track({
            'user_id': profile.id,
            'full_name': profile.fullName,
            'typing': false,
          });
          if (mounted) _refreshPresence(channel, profile.id);
        }
      });

      _presenceHeartbeat = Timer.periodic(const Duration(seconds: 20), (_) {
        if (!mounted || _typingChannel == null) return;
        _retrackPresence();
      });

      _presencePoll = Timer.periodic(const Duration(seconds: 4), (_) {
        if (!mounted || _typingChannel == null) return;
        _refreshPresence(_typingChannel!, profile.id);
        final msgs = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;
        if (msgs != null) _updateRecentlyActive(msgs.messages);
      });

      if (mounted) _typingChannel = channel;
    } finally {
      _presenceSetupInFlight = false;
    }
  }

  void _onTextChanged() {
    if (mounted) setState(() {});
    final profile = ref.read(currentProfileProvider).valueOrNull;
    if (profile == null || _typingChannel == null) return;
    _typingChannel!.track({
      'user_id': profile.id,
      'full_name': profile.fullName,
      'typing': _text.text.trim().isNotEmpty,
    });
    _typingIdle?.cancel();
    _typingIdle = Timer(const Duration(seconds: 3), () {
      _typingChannel?.track({
        'user_id': profile.id,
        'full_name': profile.fullName,
        'typing': false,
      });
    });
  }

  bool get _isNearBottom {
    if (!_scroll.hasClients) return true;
    final pos = _scroll.position;
    if (pos.maxScrollExtent <= 0) return true;
    return pos.pixels <= _nearBottomThreshold;
  }

  void _onUserScrolled() {
    if (!_scroll.hasClients) return;
    final nearBottom = _isNearBottom;
    _stickToBottom = nearBottom;
    final show = !nearBottom;
    if (show == _showScrollDown) return;
```

**lib/screens/chat/room/chat_room_screen.dart — continued (2/5)**

```
    _scrollFabDebounce?.cancel();
    _scrollFabDebounce = Timer(const Duration(milliseconds: 60), () {
      if (!mounted) return;
      final next = !_isNearBottom;
      if (next != _showScrollDown) {
        setState(() => _showScrollDown = next);
      }
    });
  }

  void _maybeLoadMoreFromScroll() {
```

```
    if (!_scroll.hasClients) return;
    final pos = _scroll.position;
    if (pos.maxScrollExtent > pos.viewportDimension * 0.5 &&
        pos.pixels >= pos.maxScrollExtent - 80) {
      _requestLoadMore();
    }
  }

  bool _onScrollNotification(ScrollNotification notification) {
    if (notification.metrics.axis != Axis.vertical) return false;

    if (notification is ScrollUpdateNotification ||
        notification is OverscrollNotification) {
      _maybeLoadMoreFromScroll();
    }

    if (_stickToBottom &&
        notification is ScrollMetricsNotification &&
        notification.metrics.axis == Axis.vertical) {
      final m = notification.metrics;
      if (m.pixels > 1) {
        _scroll.jumpTo(0);
      }
    }

    if (notification is UserScrollNotification) {
      _onUserScrolled();
    }
    return false;
  }

  void _requestLoadMore() {
    if (_loadMoreQueued) return;
    final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;
    if (state == null || !state.hasMore || state.loadingMore) return;
    _loadMoreQueued = true;
    final beforeExtent = _scroll.hasClients
        ? _scroll.position.maxScrollExtent
        : 0.0;
    final beforePixels = _scroll.hasClients ? _scroll.position.pixels : 0.0;
    ref
        .read(chatMessagesProvider(widget.roomId).notifier)
        .loadMore()
        .whenComplete(() {
          _loadMoreQueued = false;
          if (!mounted || !_scroll.hasClients) return;
          WidgetsBinding.instance.addPostFrameCallback((_) {
            if (!mounted || !_scroll.hasClients) return;
            final afterExtent = _scroll.position.maxScrollExtent;
            final delta = afterExtent - beforeExtent;
            if (delta > 0) {
              _scroll.jumpTo(beforePixels + delta);
            }
          });
        });
  }

  void _maybeMarkRead(AsyncValue<ChatMessagesState> messages) {
    if (_markedRead) return;
    messages.whenData((_) {
      _markedRead = true;
      ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();
    });
  }

  void _onMessagesUpdated(ChatMessagesState state) {
    _updateRecentlyActive(state.messages);

    final tailId = state.messages.isEmpty ? null : state.messages.last.id;
    final count = state.messages.length;

    if (!_didInitialScroll && state.messages.isNotEmpty) {
      _didInitialScroll = true;
```

```
      _lastTailMessageId = tailId;
      _lastMessageCount = count;
      _scrollToBottom(animated: false);
      return;
    }

    final hasNewTail = tailId != null && tailId != _lastTailMessageId;
    final prepended = count > _lastMessageCount && !hasNewTail;
    _lastTailMessageId = tailId;
    _lastMessageCount = count;

    if (prepended) return;
    if (!hasNewTail) return;

    if (_stickToBottom) {
      _scrollToBottom();
      ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();
    } else if (!_showScrollDown && mounted) {
      setState(() => _showScrollDown = true);
    }
  }

  void _onScrollDownTap() {
    setState(() {
      _stickToBottom = true;
      _showScrollDown = false;
    });
    _scrollToBottom();
    ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();
  }

  void _beginReply(ChatMessage message) {
    if (message.isDeleted) return;
    setState(() => _replyTo = message);
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;
      _focus.requestFocus();
    });
  }

  void _clearReply() {
    if (_replyTo == null) return;
    setState(() => _replyTo = null);
  }

  void _flashMessage(String messageId) {
    _highlightTimer?.cancel();
    setState(() => _highlightedMessageId = messageId);
    _highlightTimer = Timer(const Duration(milliseconds: 1400), () {
      if (!mounted) return;
      if (_highlightedMessageId == messageId) {
        setState(() => _highlightedMessageId = null);
      }
    });
  }

  int? _reverseIndexForMessage(String messageId, List<ChatMessage> messages) {
    final entries = chatEntriesFor(messages);
    final entryIndex = entries.indexWhere((e) => e.message?.id == messageId);
    if (entryIndex < 0) return null;
    return entries.length - 1 - entryIndex;
  }

  Future<void> _awaitLoadMoreSettled({required int previousCount}) async {
    for (var i = 0; i < 40; i++) {
      await Future<void>.delayed(const Duration(milliseconds: 50));
      if (!mounted) return;
      final s = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;
      if (s == null) return;
      if (!s.loadingMore &&
          (s.messages.length != previousCount || !s.hasMore)) {
        return;
      }
```

```
  }
}

Future<void> _jumpNearReverseIndex(int reverseIndex) async {
  if (!_scroll.hasClients) return;
  const avgExtent = 160.0;
  for (var pass = 0; pass < 3; pass++) {
    if (!_scroll.hasClients) return;
    final max = _scroll.position.maxScrollExtent;
    final target = (reverseIndex * avgExtent).clamp(0.0, max);
    _scroll.jumpTo(target);
    await Future<void>.delayed(const Duration(milliseconds: 32));
  }
}

Future<bool> _ensureVisibleMessage(String messageId) async {
  final key = _keyForMessage(messageId);
  for (var frame = 0; frame < 12; frame++) {
    final ctx = key.currentContext;
    if (ctx != null && ctx.mounted) {
      await Scrollable.ensureVisible(
        ctx,
        duration: const Duration(milliseconds: 280),
        curve: Curves.easeOutCubic,
        alignment: 0.35,
      );
      return true;
    }
    await Future<void>.delayed(const Duration(milliseconds: 40));
  }
  return false;
}

Future<void> _scrollToRepliedMessage(String messageId) async {
  if (_jumpingToReply || messageId.isEmpty) return;
  _jumpingToReply = true;
  _stickToBottom = false;
  setState(() => _showScrollDown = true);
  final notFoundLabel = context.l10n.chatReplyNotFound;
  final messenger = ScaffoldMessenger.of(context);
  try {
    for (var attempt = 0; attempt < 30; attempt++) {
      if (!mounted) return;
      final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;
      if (state == null) return;

      ChatMessage? hit;
      for (final m in state.messages) {
        if (m.id == messageId) {
          hit = m;
          break;
        }
      }
      if (hit != null) {
        if (hit.isDeleted) {
          messenger.showSnackBar(
            SnackBar(content: Text(context.l10n.chatReplyDeleted)),
          );
          return;
        }
        break;
      }

      if (!state.hasMore) {
        messenger.showSnackBar(SnackBar(content: Text(notFoundLabel)));
        return;
      }

      if (state.loadingMore) {
        await _awaitLoadMoreSettled(previousCount: state.messages.length);
        continue;
      }
```

```
      final before = state.messages.length;
      await ref.read(chatMessagesProvider(widget.roomId).notifier).loadMore();
      await _awaitLoadMoreSettled(previousCount: before);
    }

    if (!mounted) return;
    final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;
    ChatMessage? target;
    if (state != null) {
      for (final m in state.messages) {
        if (m.id == messageId) {
          target = m;
          break;
        }
      }
    }
    if (state == null || target == null || target.isDeleted) {
      messenger.showSnackBar(
        SnackBar(
          content: Text(
            target?.isDeleted == true
                ? context.l10n.chatReplyDeleted
                : notFoundLabel,
          ),
        ),
      );
      return;
    }

    final reverseIndex = _reverseIndexForMessage(messageId, state.messages);
    if (reverseIndex != null) {
      await _jumpNearReverseIndex(reverseIndex);
    }

    if (!mounted) return;
    var visible = await _ensureVisibleMessage(messageId);
    if (!visible && reverseIndex != null && mounted) {
      await _jumpNearReverseIndex(reverseIndex + 2);
      visible = await _ensureVisibleMessage(messageId);
    }

    if (visible && mounted) {
      _flashMessage(messageId);
      return;
    }

    if (reverseIndex != null && _scroll.hasClients) {
      await _jumpNearReverseIndex(reverseIndex);
      if (await _ensureVisibleMessage(messageId) && mounted) {
        _flashMessage(messageId);
        return;
      }
    }

    messenger.showSnackBar(SnackBar(content: Text(notFoundLabel)));
  } finally {
    _jumpingToReply = false;
  }
}

Future<void> _sendText() async {
  final value = _text.text.trim();
  if (value.isEmpty || _sending) return;
  final replyId = _replyTo?.id;
  setState(() => _sending = true);
  try {
    _text.clear();
    await ref
        .read(chatMessagesProvider(widget.roomId).notifier)
        .sendText(value, replyToMessageId: replyId);
    setState(() {
      _stickToBottom = true;
      _showScrollDown = false;
```

```
        _replyTo = null;
      });
      _scrollToBottom();
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.chatSendFailed('$e'))),
        );
      }
    } finally {
      if (mounted) setState(() => _sending = false);
    }
  }

  void _scrollToBottom({bool animated = true}) {
    void attempt([int framesLeft = 3]) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!mounted) return;
        if (!_scroll.hasClients) {
          if (framesLeft > 0) attempt(framesLeft - 1);
          return;
        }
        final pos = _scroll.position;
        if (pos.pixels <= 1) {
          if (framesLeft > 0) attempt(framesLeft - 1);
          return;
        }
        if (!animated || framesLeft > 0) {
          _scroll.jumpTo(0);
          if (framesLeft > 0) attempt(framesLeft - 1);
          return;
        }
        _scroll.animateTo(
          0,
          duration: const Duration(milliseconds: 220),
          curve: Curves.easeOut,
        );
      });
    }

    attempt(animated ? 1 : 3);
  }

  Future<void> _pickFile() async {
    final result = await FilePicker.platform.pickFiles(withData: true);
    if (result == null || result.files.isEmpty) return;
    final file = result.files.first;
    if (file.bytes == null) return;
    final type = chatMessageTypeForFileName(file.name);
    await _sendMedia(
      type,
      file.bytes!,
      file.name,
      caption: type == ChatMessageType.file ? file.name : null,
    );
  }

  Future<void> _sendMedia(
    ChatMessageType type,
    Uint8List bytes,
    String fileName, {
    String? caption,
  }) async {
    final replyId = _replyTo?.id;
    setState(() {
      _sending = true;
      _uploadProgress = 0.02;
      _uploadLabel = fileName;
      _attachOpen = false;
    });
    try {
      await ref
          .read(chatMessagesProvider(widget.roomId).notifier)
```

```
          .sendMedia(
            type: type,
            bytes: bytes,
            fileName: fileName,
            caption: caption,
            replyToMessageId: replyId,
            onProgress: (p) {
              if (mounted) setState(() => _uploadProgress = p);
            },
          );
      setState(() {
        _stickToBottom = true;
        _showScrollDown = false;
        _replyTo = null;
      });
      _scrollToBottom();
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.chatSendFailed('$e'))),
        );
      }
    } finally {
      if (mounted) {
        setState(() {
          _sending = false;
          _uploadProgress = 0;
          _uploadLabel = null;
        });
      }
    }
  }

  Future<void> _pickImage() async {
    final picker = ImagePicker();
    final image = await picker.pickImage(
      source: ImageSource.gallery,
      imageQuality: 85,
    );
    if (image == null) return;
    final bytes = await image.readAsBytes();
    await _sendMedia(ChatMessageType.image, bytes, image.name);
  }

  Future<void> _pickVideo() async {
    final picker = ImagePicker();
    final video = await picker.pickVideo(source: ImageSource.gallery);
    if (video == null) return;
    final bytes = await video.readAsBytes();
    await _sendMedia(ChatMessageType.video, bytes, video.name);
  }

  Future<String> _recordingPath() async {
    final stamp = DateTime.now().millisecondsSinceEpoch;
    final ext = kIsWeb ? 'wav' : 'm4a';
    final name = 'chat_voice_$stamp.$ext';
    if (kIsWeb) return name;
    final dir = await getTemporaryDirectory();
    return p.join(dir.path, name);
  }

  Future<void> _startRecording() async {
    if (_recording || _sending) return;
    final allowed = await _recorder.hasPermission();
    if (!allowed) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(context.l10n.chatMicDenied)));
      }
      return;
    }
    final path = await _recordingPath();
```

```
      await _recorder.start(
        RecordConfig(encoder: kIsWeb ? AudioEncoder.wav : AudioEncoder.aacLc),
        path: path,
      );
      setState(() {
        _recording = true;
        _recordElapsed = Duration.zero;
      });
      _recordTick = Timer.periodic(const Duration(seconds: 1), (_) {
        if (!mounted) return;
        setState(() => _recordElapsed += const Duration(seconds: 1));
      });
    }

    Future<void> _stopRecording({required bool send}) async {
      _recordTick?.cancel();
      final path = await _recorder.stop();
      setState(() => _recording = false);
      if (!send || path == null) return;

      try {
        final bytes = await XFile(path).readAsBytes();
```

**lib/screens/chat/room/chat_room_screen.dart — continued (3/5)**

```
        final name = kIsWeb
            ? 'chat_voice_${DateTime.now().millisecondsSinceEpoch}.wav'
            : p.basename(path);
        await _sendMedia(ChatMessageType.voice, bytes, name);
      } catch (e) {
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(context.l10n.chatSendFailed('$e'))),
          );
        }
      }
    }

    Future<void> _editMessage(ChatMessage message) async {
      final result = await showDialog<String>(
        context: context,
        useRootNavigator: true,
        builder: (context) =>
            EditMessageDialog(initialText: message.textContent ?? ''),
      );
      if (!mounted || result == null || result.isEmpty) return;
      try {
        await ref
            .read(chatMessagesProvider(widget.roomId).notifier)
            .editMessage(message.id, result);
      } catch (e) {
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),
          );
        }
      }
    }

    Future<void> _deleteMessage(ChatMessage message) async {
      final ok = await showDialog<bool>(
        context: context,
        builder: (context) => AlertDialog(
          title: Text(context.l10n.chatDeleteTitle),
          content: Text(context.l10n.chatDeleteBody),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context, false),
              child: Text(context.l10n.commonCancel),
            ),
            FilledButton(
              onPressed: () => Navigator.pop(context, true),
              child: Text(context.l10n.commonDelete),
```

```
          ),
        ],
      ),
    );
    if (ok != true) return;
    try {
      await ref
          .read(chatMessagesProvider(widget.roomId).notifier)
          .deleteMessage(message.id);
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),
        );
      }
    }
  }

  Future<void> _react(ChatMessage message, String emoji) async {
    try {
      await ref
          .read(chatMessagesProvider(widget.roomId).notifier)
          .toggleReaction(message.id, emoji);
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),
        );
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final room = ref.watch(chatRoomProvider(widget.roomId));
    final messages = ref.watch(chatMessagesProvider(widget.roomId));
    final me = ref.watch(currentProfileProvider).valueOrNull;
    final isAdmin = me?.role == UserRole.admin;
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    ref.listen(chatMessagesProvider(widget.roomId), (prev, next) {
      next.whenData(_onMessagesUpdated);
    });
    ref.listen(currentProfileProvider, (prev, next) {
      if (next.valueOrNull != null && _typingChannel == null) {
        _setupTyping();
      }
    });

    _maybeMarkRead(messages);

    final roomData = room.valueOrNull;
    final title = roomData?.groupTitle ?? l10n.chatTitle;

    final keyboardInset = MediaQuery.viewInsetsOf(context).bottom;
    final keyboardOpen = keyboardInset > 0;

    return PopScope(
      canPop: !_selectionMode,
      onPopInvokedWithResult: (didPop, result) {
        if (!didPop && _selectionMode) _exitSelectionMode();
      },
      child: Scaffold(
      // Keep viewInsets so the composer can sit flush on the keyboard.
      // Otherwise shell nav padding (from extendBody) leaves a large gap.
      resizeToAvoidBottomInset: false,
      appBar: _selectionMode
          ? AppBar(
              leading: IconButton(
                icon: const Icon(Icons.close),
                onPressed: _exitSelectionMode,
```

```
        ),
        title: Text(l10n.chatSelectedCount(_selectedMessageIds.length)),
        actions: [
          TextButton(
            onPressed: messages.maybeWhen(
              data: (state) => state.messages.isEmpty
                  ? null
                  : () => _selectAllMessages(
                      state.messages,
                      me?.id,
                      isAdmin,
                    ),
              orElse: () => null,
            ),
            child: Text(l10n.commonSelectAll),
          ),
          IconButton(
            tooltip: l10n.commonDelete,
            icon: const Icon(Icons.delete_outline),
            onPressed: _selectedMessageIds.isEmpty
                ? null
                : _deleteSelected,
          ),
        ],
      )
    : AppBar(
  titleSpacing: 0,
  title: Row(
    children: [
      Builder(
        builder: (context) {
          final isMono =
              AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??
              false;
          final accent = AppColors.accent(context);
          return Container(
            width: 36,
            height: 36,
            alignment: Alignment.center,
            decoration: BoxDecoration(
              color: accent.withValues(alpha: 0.12),
              borderRadius: BorderRadius.circular(isMono ? 4 : 18),
              border: isMono
                  ? Border.all(color: accent.withValues(alpha: 0.22))
                  : null,
            ),
            child: Icon(Icons.groups_rounded, size: 20, color: accent),
          );
        },
      ),
      const SizedBox(width: 10),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              title,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: const TextStyle(
                fontWeight: FontWeight.w700,
                fontSize: 16,
              ),
            ),
            if (roomData != null)
              Text.rich(
                TextSpan(
                  style: Theme.of(context).textTheme.bodySmall?.copyWith(
                    color: AppColors.inkAlpha(context, 0.55),
                    fontSize: 12,
                  ),
                  children: [
                    TextSpan(
```

```
                      text: l10n.chatTeacherMembers(
                        roomData.teacherName,
                        roomData.memberCount,
                      ),
                    ),
                    if (_onlineCount > 0) ...[
                      const TextSpan(text: ' · '),
                      TextSpan(
                        text: '● ${l10n.chatOnlineCount(_onlineCount)}',
                        style: TextStyle(
                          color:
                              (AppStyleTokens.maybeOf(
                                    context,
                                  )?.isMonochromeWorkspace ??
                                  false)
                              ? AppColors.ink(context)
                              : const Color(0xFF4CAF50),
                          fontWeight: FontWeight.w600,
                        ),
                      ),
                    ],
                  ],
                ),
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
              ),
            ],
          ),
        ),
      ],
    ),
    actions: [
      IconButton(
        tooltip: l10n.chatSelectMessages,
        icon: const Icon(Icons.checklist_rounded),
        onPressed: () => _enterSelectionMode(),
      ),
    ],
  ),
  body: Column(
    children: [
      if (_uploadProgress > 0 && _uploadProgress < 1)
        LinearProgressIndicator(value: _uploadProgress),
      if (_uploadLabel != null && _uploadProgress > 0)
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
          child: Text(
            l10n.chatUploading(_uploadLabel!),
            style: Theme.of(context).textTheme.bodySmall,
          ),
        ),
      Expanded(
        child: messages.when(
          loading: () => const Center(child: CircularProgressIndicator()),
          error: (e, _) =>
              Center(child: Text(l10n.commonErrorWithDetails('$e'))),
          data: (state) {
            if (state.messages.isEmpty) {
              return Center(
                child: Text(
                  l10n.chatNoMessagesYet,
                  style: Theme.of(context).textTheme.bodyLarge?.copyWith(
                    color: AppColors.inkAlpha(context, 0.5),
                  ),
                ),
              );
            }
            final entries = chatEntriesFor(state.messages);
            final loadMoreOffset = state.hasMore ? 1 : 0;
            final galleryItems = chatMediaItemsFromMessages(state.messages);

            return Stack(
              children: [
```

```
NotificationListener<ScrollNotification>(
  onNotification: _onScrollNotification,
  child: ListView.builder(
    reverse: true,
    controller: _scroll,
    cacheExtent: 1200,
    padding: EdgeInsets.fromLTRB(
      12,
      8,
      12,
      _selectionMode
          ? shellContentBottomPadding(context)
          : 12,
    ),
    itemCount: entries.length + loadMoreOffset,
    itemBuilder: (context, index) {
      if (state.hasMore && index == entries.length) {
        return Padding(
          padding: const EdgeInsets.only(top: 12),
          child: Center(
            child: state.loadingMore
                ? const SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(
                      strokeWidth: 2,
                    ),
                  )
                : TextButton(
                    onPressed: () => ref
                        .read(
                          chatMessagesProvider(
                            widget.roomId,
                          ).notifier,
                        )
                        .loadMore(),
                    child: Text(l10n.commonLoadMore),
                  ),
          ),
        );
      }
      final entry = entries[entries.length - 1 - index];
      if (entry.date != null) {
        return ChatDateChip(date: entry.date!);
      }
      final message = entry.message!;
      final mine = message.senderId == me?.id;
      final canModerate = _canModerateMessage(
        message,
        me?.id,
        isAdmin,
      );
      final canDelete = _canDeleteMessage(
        message,
        me?.id,
        isAdmin,
      );
      return KeyedSubtree(
        key: _keyForMessage(message.id),
        child: MessageBubble(
          message: message,
          isMine: mine,
          myUserId: me?.id,
          galleryItems: galleryItems,
          highlighted: _highlightedMessageId == message.id,
          selectionMode: _selectionMode,
          selected: _selectedMessageIds.contains(
            message.id,
          ),
          selectable: canDelete,
          onSelectionToggle: canDelete
              ? () => _toggleMessageSelection(message.id)
              : null,
```

```
                                onSelect: canDelete
                                    ? () => _enterSelectionMode(
                                        messageId: message.id,
                                      )
                                    : null,
                                onJumpToReply: _selectionMode
                                    ? null
                                    : (id) => _scrollToRepliedMessage(id),
                                onReply: _selectionMode || message.isDeleted
                                    ? null
                                    : () => _beginReply(message),
                                onReact: _selectionMode || message.isDeleted
                                    ? null
```

**lib/screens/chat/room/chat_room_screen.dart — continued (4/5)**

```
                                    : (emoji) => _react(message, emoji),
                                onEdit:
                                    !_selectionMode &&
                                        canModerate &&
                                        message.messageType ==
                                            ChatMessageType.text &&
                                        !message.isDeleted
                                    ? () => _editMessage(message)
                                    : null,
                                onDelete:
                                    !_selectionMode && canDelete
                                    ? () => _deleteMessage(message)
                                    : null,
                              ),
                            );
                          },
                        ),
                      ),
                      Positioned(
                        right: 16,
                        bottom: 12,
                        child: IgnorePointer(
                          ignoring: !_showScrollDown,
                          child: AnimatedScale(
                            scale: _showScrollDown ? 1 : 0,
                            duration: const Duration(milliseconds: 180),
                            curve: Curves.easeOutBack,
                            child: AnimatedOpacity(
                              opacity: _showScrollDown ? 1 : 0,
                              duration: const Duration(milliseconds: 160),
                              child: Material(
                                color: isDark
                                    ? scheme.surfaceContainerHighest
                                    : Colors.white,
                                elevation: 4,
                                shadowColor: AppColors.inkAlpha(context, 0.18),
                                shape: const CircleBorder(),
                                child: InkWell(
                                  customBorder: const CircleBorder(),
                                  onTap: _onScrollDownTap,
                                  child: SizedBox(
                                    width: 44,
                                    height: 44,
                                    child: Icon(
                                      Icons.keyboard_arrow_down_rounded,
                                      color: AppColors.accent(context),
                                      size: 28,
                                    ),
                                  ),
                                ),
                              ),
                            ),
                          ),
                        ),
                      ),
                    ],
                  );
```

```
      },
    ),
  ),
  if (!_selectionMode && _typingNames.isNotEmpty)
    Padding(
      padding: const EdgeInsets.fromLTRB(16, 0, 16, 4),
      child: Align(
        alignment: Alignment.centerLeft,
        child: Text(
          l10n.chatTyping(_typingNames.join(', ')),
          style: Theme.of(context).textTheme.bodySmall?.copyWith(
            color: AppColors.warmAmber,
            fontStyle: FontStyle.italic,
          ),
        ),
      ),
    ),
  if (!_selectionMode && _recording)
    Container(
      width: double.infinity,
      color: AppColors.errorRed.withValues(alpha: 0.08),
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 10),
      child: Row(
        children: [
          const Icon(Icons.mic, color: AppColors.errorRed),
          const SizedBox(width: 8),
          Text(
            l10n.chatRecording(
              '${_recordElapsed.inMinutes.toString().padLeft(2, '0')}:'
              '${(_recordElapsed.inSeconds % 60).toString().padLeft(2, '0')}',
            ),
          ),
          const Spacer(),
          TextButton(
            onPressed: () => _stopRecording(send: false),
            child: Text(l10n.commonCancel),
          ),
          FilledButton(
            onPressed: () => _stopRecording(send: true),
            child: Text(l10n.commonSubmit),
          ),
        ],
      ),
    ),
  if (!_selectionMode)
    Padding(
    padding: EdgeInsets.fromLTRB(
      context.isMobile ? 10 : 16,
      6,
      context.isMobile ? 10 : 16,
      keyboardInset +
          (keyboardOpen
              ? 8
              : (context.isMobile ? 10 : 16) +
                    MediaQuery.paddingOf(context).bottom),
    ),
    child: Builder(
      builder: (context) {
        final dockTokens = AppStyleTokens.maybeOf(context);
        final isNeuDock = dockTokens?.isSoftNeumorphism ?? false;
        final isMonoDock = dockTokens?.isMonochromeWorkspace ?? false;
        final dockRadius = BorderRadius.circular(
          isMonoDock ? 4 : (isNeuDock ? 24 : 28),
        );
        final dockChild = Padding(
          padding: const EdgeInsets.fromLTRB(6, 8, 8, 8),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              AnimatedSize(
                duration: const Duration(milliseconds: 220),
                curve: Curves.easeOutCubic,
                child: _attachOpen
```

```
        ? Padding(
            padding: const EdgeInsets.only(bottom: 8),
            child: Row(
              mainAxisAlignment:
                  MainAxisAlignment.spaceEvenly,
              children: [
                AttachChip(
                  icon: Icons.image_outlined,
                  label: l10n.chatAttachImage,
                  onTap: _pickImage,
                ),
                AttachChip(
                  icon: Icons.videocam_outlined,
                  label: l10n.chatAttachVideo,
                  onTap: _pickVideo,
                ),
                AttachChip(
                  icon: Icons.attach_file_rounded,
                  label: l10n.chatAttachFile,
                  onTap: _pickFile,
                ),
              ],
            ),
          )
        : const SizedBox.shrink(),
  ),
  AnimatedSize(
    duration: const Duration(milliseconds: 180),
    curve: Curves.easeOutCubic,
    child: _replyTo == null
        ? const SizedBox.shrink()
        : Padding(
            padding: const EdgeInsets.fromLTRB(8, 0, 8, 8),
            child: ReplyComposerBanner(
              message: _replyTo!,
              onClose: _clearReply,
            ),
          ),
  ),
  Row(
    crossAxisAlignment: CrossAxisAlignment.end,
    children: [
      IconButton(
        onPressed: _sending
            ? null
            : () => setState(
                () => _attachOpen = !_attachOpen,
              ),
        icon: AnimatedRotation(
          turns: _attachOpen ? 0.125 : 0,
          duration: const Duration(milliseconds: 200),
          child: const Icon(Icons.add_circle_outline),
        ),
      ),
      Expanded(
        child: TextField(
          controller: _text,
          focusNode: _focus,
          autofocus: false,
          minLines: 1,
          maxLines: 5,
          textInputAction: TextInputAction.send,
          onSubmitted: (_) => _sendText(),
          style: Theme.of(context).textTheme.bodyLarge,
          decoration: InputDecoration(
            hintText: l10n.chatMessageHint,
            filled: false,
            isDense: true,
            border: InputBorder.none,
            enabledBorder: InputBorder.none,
            focusedBorder: InputBorder.none,
            disabledBorder: InputBorder.none,
            errorBorder: InputBorder.none,
```

```
        focusedErrorBorder: InputBorder.none,
        contentPadding: const EdgeInsets.fromLTRB(
          4,
          10,
          8,
          10,
        ),
      ),
    ),
  ),
),
if (_text.text.trim().isEmpty)
  GestureDetector(
    onLongPressStart: (_) => _startRecording(),
    onLongPressEnd: (_) => _stopRecording(send: true),
    child: Container(
      width: 40,
      height: 40,
      decoration: BoxDecoration(
        color: _recording
            ? AppColors.danger(context)
            : AppColors.accent(context),
        shape: isMonoDock
            ? BoxShape.rectangle
            : BoxShape.circle,
        borderRadius: isMonoDock
            ? BorderRadius.circular(4)
            : null,
        boxShadow: isNeuDock
            ? [
                BoxShadow(
                  color: Neumorphic.shadowColor(
                    context,
                  ).withValues(alpha: 0.35),
                  offset: const Offset(2, 3),
                  blurRadius: 6,
                ),
              ]
            : null,
      ),
      child: Icon(
        _recording
            ? Icons.stop_rounded
            : Icons.mic_none,
        color: (isDark && (isNeuDock || isMonoDock))
            ? const Color(0xFF0E1812)
            : Colors.white,
        size: 22,
      ),
    ),
  )
else
  IconButton.filled(
    onPressed: _sending ? null : _sendText,
    style: IconButton.styleFrom(
      backgroundColor: AppColors.accent(context),
      foregroundColor:
          (isDark && (isNeuDock || isMonoDock))
          ? const Color(0xFF0E1812)
          : Colors.white,
      fixedSize: const Size(40, 40),
      padding: EdgeInsets.zero,
      shape: isMonoDock
          ? RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(4),
            )
          : null,
    ),
    icon: _sending
        ? SizedBox(
            width: 18,
            height: 18,
            child: CircularProgressIndicator(
              strokeWidth: 2,
```

```
                                        color:
                                            (isDark &&
                                                (isNeuDock || isMonoDock))
                                            ? const Color(0xFF0E1812)
                                            : Colors.white,
                                      ),
                                    )
                                  : const Icon(Icons.send_rounded, size: 20),
                            ),
                          ],
                        ),
                      ],
                    ),
                  );

                  if (isNeuDock) {
                    return Container(
```

**lib/screens/chat/room/chat_room_screen.dart — continued (5/5)**

```
                      decoration: Neumorphic.decoration(
                        context,
                        elevation: NeuElevation.floating,
                        borderRadius: dockRadius,
                      ),
                      clipBehavior: Clip.antiAlias,
                      child: dockChild,
                    );
                  }

                  return Material(
                    color: isDark
                        ? scheme.surfaceContainerHigh.withValues(alpha: 0.96)
                        : Colors.white.withValues(alpha: 0.94),
                    elevation: isMonoDock ? 0 : 10,
                    shadowColor: isMonoDock
                        ? Colors.transparent
                        : AppColors.inkAlpha(context, 0.18),
                    surfaceTintColor: Colors.transparent,
                    borderRadius: dockRadius,
                    clipBehavior: Clip.antiAlias,
                    child: DecoratedBox(
                      decoration: BoxDecoration(
                        borderRadius: dockRadius,
                        border: Border.all(
                          color: AppColors.inkAlpha(
                            context,
                            isMonoDock ? 0.12 : 0.08,
                          ),
                        ),
                      ),
                      child: dockChild,
                    ),
                  );
                },
              ),
            ),
          ),
        ],
      ),
    ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/chat_room_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/chat/room/helpers/chat_list_entries.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 29. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/room/helpers/chat_list_entries.dart**

```
import 'package:mu_study/models/models.dart';

class ChatListEntry {
  const ChatListEntry._({this.date, this.message});

  factory ChatListEntry.date(DateTime date) => ChatListEntry._(date: date);

  factory ChatListEntry.message(ChatMessage message) =>
      ChatListEntry._(message: message);

  final DateTime? date;
  final ChatMessage? message;
}

List<ChatListEntry> chatEntriesFor(List<ChatMessage> messages) {
  final entries = <ChatListEntry>[];
  DateTime? lastDay;
  for (final message in messages) {
    if (message.isDeleted) continue;
    final local = message.createdAt.toLocal();
    final day = DateTime(local.year, local.month, local.day);
    if (lastDay == null || day != lastDay) {
      entries.add(ChatListEntry.date(day));
      lastDay = day;
    }
    entries.add(ChatListEntry.message(message));
  }
  return entries;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/helpers/chat_list_entries.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/chat/room/helpers/reply_snippets.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 25. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/room/helpers/reply_snippets.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';

String chatReplySnippet(BuildContext context, ChatReplyPreview? reply) {
  final l10n = context.l10n;
  if (reply == null || reply.isDeleted) return l10n.chatReplyDeleted;
  return switch (reply.messageType) {
    ChatMessageType.text =>
      (reply.textContent ?? '').trim().isEmpty
          ? l10n.chatMessageDeleted
          : reply.textContent!.trim(),
    ChatMessageType.image => l10n.chatPreviewImage,
    ChatMessageType.video => l10n.chatPreviewVideo,
    ChatMessageType.voice => l10n.chatPreviewVoice,
    ChatMessageType.file =>
      (reply.textContent?.trim().isNotEmpty == true)
          ? reply.textContent!.trim()
          : l10n.chatPreviewFile,
  };
}

String chatReplySnippetFromMessage(BuildContext context, ChatMessage message) {
  return chatReplySnippet(context, ChatReplyPreview.fromMessage(message));
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/helpers/reply_snippets.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/chat/room/widgets/attach_chip.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 50. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/attach_chip.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';

class AttachChip extends StatelessWidget {
  const AttachChip({
    super.key,
    required this.icon,
    required this.label,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    final radius = isMono ? 4.0 : 14.0;
    return InkWell(
      onTap: onTap,
      borderRadius: BorderRadius.circular(radius),
      child: Container(
        width: 96,
        padding: const EdgeInsets.symmetric(vertical: 12),
        decoration: BoxDecoration(
          color: AppColors.accent(context).withValues(alpha: 0.08),
          borderRadius: BorderRadius.circular(radius),
          border: isMono
              ? Border.all(color: AppColors.inkAlpha(context, 0.14))
              : null,
        ),
        child: Column(
          children: [
            Icon(icon, color: AppColors.accent(context)),
            const SizedBox(height: 4),
            Text(
              label,
              style: Theme.of(
                context,
              ).textTheme.labelSmall?.copyWith(fontWeight: FontWeight.w600),
            ),
          ],
```

```
          ),
        ),
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/attach_chip.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/chat/room/widgets/chat_date_chip.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 73. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/chat_date_chip.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class ChatDateChip extends StatelessWidget {
  const ChatDateChip({super.key, required this.date});

  final DateTime date;

  String _label(BuildContext context) {
    final l10n = context.l10n;
    final now = DateTime.now();
    final today = DateTime(now.year, now.month, now.day);
    final yesterday = today.subtract(const Duration(days: 1));
    final day = DateTime(date.year, date.month, date.day);

    if (day == today) return l10n.chatDateToday;
    if (day == yesterday) return l10n.chatDateYesterday;

    final locale = Localizations.localeOf(context).toString();
    if (day.year == today.year) {
      return DateFormat.MMMMd(locale).format(day);
    }
    return DateFormat.yMMMMd(locale).format(day);
  }

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 12),
      child: Center(
        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),
          decoration: BoxDecoration(
            color: isDark
```

```
                ? scheme.surfaceContainerHighest.withValues(alpha: 0.9)
                : AppColors.sand.withValues(alpha: 0.95),
            borderRadius: BorderRadius.circular(isMono ? 4 : 999),
            border: isMono
                ? Border.all(color: AppColors.inkAlpha(context, 0.12))
                : null,
            boxShadow: isMono
                ? null
                : [
                    BoxShadow(
                      color: AppColors.inkAlpha(context, 0.06),
                      blurRadius: 8,
                      offset: const Offset(0, 2),
                    ),
                  ],
          ),
          child: Text(
            _label(context),
            style: Theme.of(context).textTheme.labelMedium?.copyWith(
              fontWeight: FontWeight.w700,
              color: AppColors.inkAlpha(context, 0.7),
              fontFeatures: isMono
                  ? const [FontFeature.tabularFigures()]
                  : null,
            ),
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/chat_date_chip.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/chat/room/widgets/chat_file.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 166. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:io';
- import 'package:flutter/foundation.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/core/services/chat_media_cache_service.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:path/path.dart' as p;
- import 'package:path_provider/path_provider.dart';
- import 'package:share_plus/share_plus.dart';
- import 'package:url_launcher/url_launcher.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/chat_file.dart**

```
import 'dart:io';

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:share_plus/share_plus.dart';
import 'package:url_launcher/url_launcher.dart';

class ChatFile extends StatelessWidget {
  const ChatFile({
    super.key,
    required this.url,
    required this.fileName,
    required this.color,
  });

  final String url;
  final String fileName;
  final Color color;

  static String _mimeForName(String name) {
    final ext = name.contains('.') ? name.split('.').last.toLowerCase() : '';
    return switch (ext) {
      'csv' => 'text/csv',
      'txt' => 'text/plain',
      'pdf' => 'application/pdf',
      'json' => 'application/json',
      'doc' => 'application/msword',
      'docx' =>
        'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
```

```
      'xls' => 'application/vnd.ms-excel',
      'xlsx' =>
        'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
      'ppt' => 'application/vnd.ms-powerpoint',
      'pptx' =>
        'application/vnd.openxmlformats-officedocument.presentationml.presentation',
      'zip' => 'application/zip',
      'png' => 'image/png',
      'jpg' || 'jpeg' => 'image/jpeg',
      'gif' => 'image/gif',
      'webp' => 'image/webp',
      'mp4' => 'video/mp4',
      'mov' => 'video/quicktime',
      'mp3' => 'audio/mpeg',
      'm4a' => 'audio/mp4',
      'wav' => 'audio/wav',
      'py' => 'text/x-python',
      _ => 'application/octet-stream',
    };
  }

  Future<void> _open(BuildContext context) async {
    final l10n = context.l10n;
    try {
      if (kIsWeb) {
        final uri = Uri.tryParse(url);
        if (uri != null &&
            await launchUrl(uri, mode: LaunchMode.platformDefault)) {
          return;
        }
        throw StateError('launch failed');
      }

      final ext = fileName.contains('.')
          ? fileName.split('.').last
          : ChatMediaCacheService.extensionForUrl(url, fallback: 'bin');
      final cached = await ChatMediaCacheService.instance.getOrFetch(
        url,
        preferredExtension: ext,
      );
      if (cached == null) {
        final uri = Uri.tryParse(url);
        if (uri != null &&
            await launchUrl(uri, mode: LaunchMode.externalApplication)) {
          return;
        }
        throw StateError('download failed');
      }

      final safeName = fileName.replaceAll(RegExp(r'[^\w.\-]+'), '_');
      final tmp = File(p.join((await getTemporaryDirectory()).path, safeName));
      await cached.copy(tmp.path);

      await SharePlus.instance.share(
        ShareParams(
          files: [
            XFile(tmp.path, mimeType: _mimeForName(safeName), name: safeName),
          ],
          fileNameOverrides: [safeName],
          downloadFallbackEnabled: true,
        ),
      );
    } catch (_) {
      if (!context.mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.chatOpenFileFailed)));
    }
  }

  @override
  Widget build(BuildContext context) {
    final isMono =
```

```
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    final radius = isMono ? 4.0 : 12.0;
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: () => _open(context),
        borderRadius: BorderRadius.circular(radius),
        child: Container(
          constraints: const BoxConstraints(minWidth: 180, maxWidth: 260),
          padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 10),
          decoration: BoxDecoration(
            color: color.withValues(alpha: 0.08),
            borderRadius: BorderRadius.circular(radius),
            border: Border.all(color: color.withValues(alpha: 0.18)),
          ),
          child: Row(
            mainAxisSize: MainAxisSize.min,
            children: [
              Icon(Icons.insert_drive_file_rounded, color: color, size: 28),
              const SizedBox(width: 10),
              Flexible(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      fileName,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                      style: TextStyle(
                        color: color,
                        fontWeight: FontWeight.w700,
                        fontSize: 13,
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      context.l10n.chatOpenFile,
                      style: TextStyle(
                        color: color.withValues(alpha: 0.7),
                        fontSize: 11,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ],
                ),
              ),
              const SizedBox(width: 4),
              Icon(
                Icons.download_rounded,
                color: color.withValues(alpha: 0.75),
                size: 20,
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/chat_file.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/chat/room/widgets/chat_image.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 121. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:io';
- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/foundation.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/components/chat/chat_media_gallery.dart';
- import 'package:mu_study/core/services/chat_media_cache_service.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/chat_image.dart**

```
import 'dart:io';

import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/components/chat/chat_media_gallery.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';

class ChatImage extends StatefulWidget {
  const ChatImage({
    super.key,
    required this.url,
    this.galleryItems = const [],
    this.galleryIndex = -1,
  });

  final String url;
  final List<ChatMediaItem> galleryItems;
  final int galleryIndex;

  @override
  State<ChatImage> createState() => _ChatImageState();
}

class _ChatImageState extends State<ChatImage> {
  File? _cachedFile;
  bool _resolving = true;

  @override
  void initState() {
    super.initState();
    _resolve();
  }

  @override
  void didUpdateWidget(covariant ChatImage oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.url != widget.url) {
```

```
    _cachedFile = null;
    _resolving = true;
    _resolve();
  }
}

Future<void> _resolve() async {
  if (kIsWeb || !ChatMediaCacheService.instance.isSupported) {
    if (mounted) setState(() => _resolving = false);
    return;
  }
  final file = await ChatMediaCacheService.instance.getOrFetch(
    widget.url,
    preferredExtension: ChatMediaCacheService.extensionForUrl(
      widget.url,
      fallback: 'jpg',
    ),
  );
  if (!mounted) return;
  setState(() {
    _cachedFile = file;
    _resolving = false;
  });
}

Widget _buildImage({required BoxFit fit, double? width, double? height}) {
  final file = _cachedFile;
  if (file != null) {
    return Image.file(
      file,
      fit: fit,
      width: width,
      height: height,
      errorBuilder: (_, _, _) => const Icon(Icons.broken_image_outlined),
    );
  }
  return CachedNetworkImage(
    imageUrl: widget.url,
    width: width,
    height: height,
    fit: fit,
    placeholder: (_, _) => SizedBox(
      width: width ?? 220,
      height: height ?? 180,
      child: const Center(child: CircularProgressIndicator(strokeWidth: 2)),
    ),
    errorWidget: (_, _, _) => const Icon(Icons.broken_image_outlined),
  );
}

@override
Widget build(BuildContext context) {
  if (_resolving && _cachedFile == null) {
    return const SizedBox(
      width: 220,
      height: 180,
      child: Center(child: CircularProgressIndicator(strokeWidth: 2)),
    );
  }
  final isMono =
      AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
  return ClipRRect(
    borderRadius: BorderRadius.circular(isMono ? 4 : 12),
    child: GestureDetector(
      onTap: () {
        final items = widget.galleryItems.isNotEmpty
            ? widget.galleryItems
            : [
                ChatMediaItem(
                  id: widget.url,
                  url: widget.url,
                  kind: ChatMediaKind.image,
                ),
```

```
                ];
            final index = widget.galleryIndex >= 0 ? widget.galleryIndex : 0;
            showChatMediaGallery(context, items: items, initialIndex: index);
          },
          child: _buildImage(fit: BoxFit.cover, width: 220, height: 180),
        ),
      );
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/chat_image.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/chat/room/widgets/chat_video.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 149. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:io';
- import 'package:flutter/material.dart';
- import 'package:mu_study/components/chat/chat_media_gallery.dart';
- import 'package:mu_study/core/services/chat_video_thumbnail_service.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/room/widgets/chat_video.dart**

```
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:mu_study/components/chat/chat_media_gallery.dart';
import 'package:mu_study/core/services/chat_video_thumbnail_service.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class ChatVideo extends StatefulWidget {
  const ChatVideo({
    super.key,
    required this.url,
    this.galleryItems = const [],
    this.galleryIndex = -1,
  });

  final String url;
  final List<ChatMediaItem> galleryItems;
  final int galleryIndex;

  @override
  State<ChatVideo> createState() => _ChatVideoState();
}

class _ChatVideoState extends State<ChatVideo> {
  File? _thumb;
  bool _loading = true;

  @override
  void initState() {
    super.initState();
    _loadThumb();
  }

  @override
  void didUpdateWidget(covariant ChatVideo oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.url != widget.url) {
      _thumb = null;
      _loading = true;
```

```
      _loadThumb();
    }
  }

  Future<void> _loadThumb() async {
    final file = await ChatVideoThumbnailService.instance.getThumbnail(
      widget.url,
    );
    if (!mounted) return;
    setState(() {
      _thumb = file;
      _loading = false;
    });
  }

  void _openFullscreen() {
    final items = widget.galleryItems.isNotEmpty
        ? widget.galleryItems
        : [
            ChatMediaItem(
              id: widget.url,
              url: widget.url,
              kind: ChatMediaKind.video,
            ),
          ];
    final index = widget.galleryIndex >= 0 ? widget.galleryIndex : 0;
    showChatMediaGallery(context, items: items, initialIndex: index);
  }

  @override
  Widget build(BuildContext context) {
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    return ClipRRect(
      borderRadius: BorderRadius.circular(isMono ? 4 : 12),
      child: SizedBox(
        width: 220,
        height: 180,
        child: Material(
          color: const Color(0xFF1C1C1E),
          child: InkWell(
            onTap: _openFullscreen,
            child: Stack(
              fit: StackFit.expand,
              alignment: Alignment.center,
              children: [
                if (_thumb != null)
                  Image.file(
                    _thumb!,
                    fit: BoxFit.cover,
                    errorBuilder: (_, _, _) =>
                        const ColoredBox(color: Color(0xFF2C2C2E)),
                  )
                else
                  ColoredBox(
                    color: const Color(0xFF2C2C2E),
                    child: _loading
                        ? const Center(
                            child: SizedBox(
                              width: 22,
                              height: 22,
                              child: CircularProgressIndicator(
                                strokeWidth: 2,
                                color: Colors.white54,
                              ),
                            ),
                          )
                        : Icon(
                            Icons.videocam_rounded,
                            size: 44,
                            color: Colors.white.withValues(alpha: 0.22),
                          ),
                  ),
```

```
                const ColoredBox(color: Color(0x33000000)),
                const Icon(
                  Icons.play_circle_filled_rounded,
                  color: Colors.white,
                  size: 52,
                ),
                Positioned(
                  right: 8,
                  bottom: 8,
                  child: Container(
                    padding: const EdgeInsets.symmetric(
                      horizontal: 8,
                      vertical: 4,
                    ),
                    decoration: BoxDecoration(
                      color: Colors.black54,
                      borderRadius: BorderRadius.circular(8),
                    ),
                    child: Text(
                      context.l10n.chatPreviewVideo,
                      style: const TextStyle(
                        color: Colors.white,
                        fontSize: 11,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/chat_video.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/chat/room/widgets/chat_voice.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 256. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/utils/voice_playback.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/room/widgets/chat_voice.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/utils/voice_playback.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class ChatVoice extends StatefulWidget {
  const ChatVoice({
    super.key,
    required this.url,
    required this.timeLabel,
    required this.isMine,
    required this.isReadByOthers,
  });

  final String url;
  final String timeLabel;
  final bool isMine;
  final bool isReadByOthers;

  @override
  State<ChatVoice> createState() => _ChatVoiceState();
}

class _ChatVoiceState extends State<ChatVoice> {
  final _voice = VoicePlaybackController.instance;
  late final List<double> _bars = _waveformFor(widget.url);

  static const _bg = Color(0xFF1C1C1E);
  static const _playCircle = Color(0xFF2C2C2E);
  static const _played = Color(0xFFB8B0E8);
  static const _unplayed = Color(0xFF8E8E93);
  static const _meta = Color(0xFFAEAEB2);

  static List<double> _waveformFor(String seed) {
    var hash = 2166136261;
    for (final unit in seed.codeUnits) {
      hash ^= unit;
      hash = (hash * 16777619) & 0x7fffffff;
    }
    final bars = <double>[];
    var x = hash == 0 ? 1 : hash;
    for (var i = 0; i < 36; i++) {
      x = (1103515245 * x + 12345) & 0x7fffffff;
      final n = (x % 1000) / 1000.0;
```

```
    final envelope = 0.55 + 0.45 * (1 - (((i / 35) - 0.5).abs() * 1.4));
    bars.add((0.22 + n * 0.78) * envelope.clamp(0.35, 1.0));
  }
  return bars;
}

static String _formatDuration(Duration d) {
  final total = d.inSeconds.clamp(0, 99 * 60);
  final m = total ~/ 60;
  final s = total % 60;
  return '$m:${s.toString().padLeft(2, '0')}';
}

@override
void initState() {
  super.initState();
  _voice.addListener(_onVoiceChanged);
}

@override
void dispose() {
  _voice.removeListener(_onVoiceChanged);
  super.dispose();
}

void _onVoiceChanged() {
  if (mounted) setState(() {});
}

Future<void> _toggle() async {
  try {
    await _voice.toggleRemote(widget.url);
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),
    );
  }
}

@override
Widget build(BuildContext context) {
  final active = _voice.activeKey == widget.url;
  final playing = _voice.isPlaying(widget.url);
  final loading = _voice.isLoading(widget.url);
  final duration = active ? _voice.duration : Duration.zero;
  final position = active ? _voice.position : Duration.zero;
  final progress = (!active || duration.inMilliseconds <= 0)
      ? 0.0
      : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);
  final durationLabel = duration > Duration.zero
      ? _formatDuration(playing ? duration - position : duration)
      : '0:00';
  final isMono =
      AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
  final playRadius = BorderRadius.circular(isMono ? 4 : 20);
  final playedColor = isMono ? const Color(0xFFE8E8E8) : _played;
  final unplayedColor = isMono ? const Color(0xFF6E6E73) : _unplayed;

  return Container(
    constraints: const BoxConstraints(minWidth: 220, maxWidth: 280),
    padding: const EdgeInsets.fromLTRB(10, 10, 12, 10),
    decoration: BoxDecoration(
      color: _bg,
      borderRadius: BorderRadius.circular(isMono ? 6 : 999),
      border: isMono
          ? Border.all(color: Colors.white.withValues(alpha: 0.10))
          : null,
    ),
    child: Row(
      children: [
        Material(
          color: isMono ? Colors.white : _playCircle,
```

```
  borderRadius: playRadius,
  child: InkWell(
    borderRadius: playRadius,
    onTap: loading ? null : _toggle,
    child: SizedBox(
      width: 40,
      height: 40,
      child: Center(
        child: loading
            ? SizedBox(
                width: 18,
                height: 18,
                child: CircularProgressIndicator(
                  strokeWidth: 2,
                  color: isMono ? Colors.black : Colors.white,
                ),
              )
            : Icon(
                playing
                    ? Icons.pause_rounded
                    : Icons.play_arrow_rounded,
                color: isMono ? Colors.black : Colors.white,
                size: playing ? 22 : 26,
              ),
      ),
    ),
  ),
),
const SizedBox(width: 10),
Expanded(
  child: SizedBox(
    height: 28,
    child: CustomPaint(
      painter: VoiceWaveformPainter(
        bars: _bars,
        progress: progress,
        playedColor: playedColor,
        unplayedColor: unplayedColor,
      ),
    ),
  ),
),
const SizedBox(width: 10),
Column(
  crossAxisAlignment: CrossAxisAlignment.end,
  mainAxisSize: MainAxisSize.min,
  children: [
    Text(
      durationLabel,
      style: const TextStyle(
        color: _meta,
        fontSize: 11,
        fontWeight: FontWeight.w600,
        height: 1.1,
        fontFeatures: [FontFeature.tabularFigures()],
      ),
    ),
    const SizedBox(height: 2),
    Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text(
          widget.timeLabel,
          style: const TextStyle(
            color: _meta,
            fontSize: 11,
            fontWeight: FontWeight.w500,
            height: 1.1,
            fontFeatures: [FontFeature.tabularFigures()],
          ),
        ),
        if (widget.isMine) ...[
          const SizedBox(width: 3),
```

```
                    Icon(
                      widget.isReadByOthers
                          ? Icons.done_all_rounded
                          : Icons.done_rounded,
                      size: 13,
                      color: widget.isReadByOthers
                          ? (isMono
                                ? Colors.white.withValues(alpha: 0.85)
                                : const Color(0xFF9ED0FF))
                          : _meta,
                    ),
                  ],
                ],
              ),
            ],
          ),
        ],
      ),
    );
  }
}

class VoiceWaveformPainter extends CustomPainter {
  const VoiceWaveformPainter({
    required this.bars,
    required this.progress,
    required this.playedColor,
    required this.unplayedColor,
  });

  final List<double> bars;
  final double progress;
  final Color playedColor;
  final Color unplayedColor;

  @override
  void paint(Canvas canvas, Size size) {
    if (bars.isEmpty || size.width <= 0 || size.height <= 0) return;
    const gap = 2.0;
    final barWidth = ((size.width - gap * (bars.length - 1)) / bars.length)
        .clamp(1.5, 3.5);
    final total = bars.length * barWidth + (bars.length - 1) * gap;
    var x = (size.width - total) / 2;
    final playedUntil = progress * bars.length;

    for (var i = 0; i < bars.length; i++) {
      final h = (bars[i] * size.height).clamp(4.0, size.height);
      final top = (size.height - h) / 2;
      final rrect = RRect.fromRectAndRadius(
        Rect.fromLTWH(x, top, barWidth, h),
        const Radius.circular(999),
      );
      final paint = Paint()
        ..color = i + 1 <= playedUntil ? playedColor : unplayedColor;
      canvas.drawRRect(rrect, paint);
      x += barWidth + gap;
    }
  }

  @override
  bool shouldRepaint(covariant VoiceWaveformPainter oldDelegate) =>
      oldDelegate.progress != progress ||
      oldDelegate.playedColor != playedColor ||
      oldDelegate.unplayedColor != unplayedColor ||
      !identical(oldDelegate.bars, bars);
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/chat_voice.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?

3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/chat/room/widgets/edit_message_dialog.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 46. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/l10n/l10n_x.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/chat/room/widgets/edit_message_dialog.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class EditMessageDialog extends StatefulWidget {
  const EditMessageDialog({super.key, required this.initialText});

  final String initialText;

  @override
  State<EditMessageDialog> createState() => _EditMessageDialogState();
}

class _EditMessageDialogState extends State<EditMessageDialog> {
  late final TextEditingController _controller;

  @override
  void initState() {
    super.initState();
    _controller = TextEditingController(text: widget.initialText);
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return AlertDialog(
      title: Text(l10n.chatEditTitle),
      content: TextField(controller: _controller, maxLines: 4, autofocus: true),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: Text(l10n.commonCancel),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(context, _controller.text.trim()),
          child: Text(l10n.commonSave),
        ),
      ],
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/edit_message_dialog.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

# lib/screens/chat/room/widgets/message_bubble.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 1232. Import count: 22. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math' as math;
- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/chat/animated_emoji_message.dart';
- import 'package:mu_study/components/chat/chat_lottie_emoji.dart';
- import 'package:mu_study/components/chat/chat_media_gallery.dart';
- import 'package:mu_study/core/chat/emoji_message.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/chat/chat_providers.dart';
- import 'package:mu_study/screens/chat/room/widgets/chat_file.dart';
- import 'package:mu_study/screens/chat/room/widgets/chat_image.dart';
- … and 4 more imports

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/message_bubble.dart**

```
import 'dart:math' as math;

import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/chat/animated_emoji_message.dart';
import 'package:mu_study/components/chat/chat_lottie_emoji.dart';
import 'package:mu_study/components/chat/chat_media_gallery.dart';
import 'package:mu_study/core/chat/emoji_message.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';
import 'package:mu_study/screens/chat/room/widgets/chat_file.dart';
import 'package:mu_study/screens/chat/room/widgets/chat_image.dart';
import 'package:mu_study/screens/chat/room/widgets/chat_video.dart';
```

```
import 'package:mu_study/screens/chat/room/widgets/chat_voice.dart';
import 'package:mu_study/screens/chat/room/widgets/reply_quote.dart';
import 'package:mu_study/screens/chat/room/widgets/swipe_to_reply.dart';

class MessageBubble extends StatelessWidget {
  const MessageBubble({
    super.key,
    required this.message,
    required this.isMine,
    this.myUserId,
    this.galleryItems = const [],
    this.highlighted = false,
    this.selectionMode = false,
    this.selected = false,
    this.selectable = false,
    this.onSelectionToggle,
    this.onSelect,
    this.onJumpToReply,
    this.onReply,
    this.onReact,
    this.onEdit,
    this.onDelete,
  });

  final ChatMessage message;
  final bool isMine;
  final String? myUserId;
  final List<ChatMediaItem> galleryItems;
  final bool highlighted;
  final bool selectionMode;
  final bool selected;
  final bool selectable;
  final VoidCallback? onSelectionToggle;
  final VoidCallback? onSelect;
  final ValueChanged<String>? onJumpToReply;
  final VoidCallback? onReply;
  final ValueChanged<String>? onReact;
  final VoidCallback? onEdit;
  final VoidCallback? onDelete;

  static const _nameColors = [
    Color(0xFFE57373),
    Color(0xFF5B9BD5),
    Color(0xFF7CB342),
    Color(0xFFAB47BC),
    Color(0xFFFFB300),
    Color(0xFF26A69A),
    Color(0xFFEC407A),
    Color(0xFF42A5F5),
  ];

  static const _monoNameColors = [
    Color(0xFF111111),
    Color(0xFF3A3A3A),
    Color(0xFF5A5A5A),
    Color(0xFF7A7A7A),
    Color(0xFF2E2E2E),
    Color(0xFF4E4E4E),
  ];

  static const _monoNameColorsDark = [
    Color(0xFFF5F5F5),
    Color(0xFFD0D0D0),
    Color(0xFFB0B0B0),
    Color(0xFF909090),
    Color(0xFFE0E0E0),
    Color(0xFFC0C0C0),
  ];

  Color _senderColor(
    String senderId, {
    required bool monochrome,
    required bool isDark,
```

```
}) {
  var hash = 0;
  for (final c in senderId.codeUnits) {
    hash = (hash + c) & 0x7fffffff;
  }
  final palette = monochrome
      ? (isDark ? _monoNameColorsDark : _monoNameColors)
      : _nameColors;
  return palette[hash % palette.length];
}

@override
Widget build(BuildContext context) {
  final scheme = Theme.of(context).colorScheme;
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final tokens = AppStyleTokens.maybeOf(context);
  final isNeu = tokens?.isSoftNeumorphism ?? false;
  final isMono = tokens?.isMonochromeWorkspace ?? false;
  final time = DateFormat.Hm().format(message.createdAt.toLocal());
  final Color bubbleColor;
  final Color textColor;
  if (isNeu) {
    bubbleColor = isMine
        ? (isDark
              ? SoftNeumorphismColors.emerald
              : SoftNeumorphismColors.sage)
        : Neumorphic.surface(context, elevation: NeuElevation.raised);
    textColor = isMine
        ? (isDark
              ? const Color(0xFF0E1812)
              : SoftNeumorphismColors.morningSurfaceHigh)
        : scheme.onSurface;
  } else if (isMono) {
    bubbleColor = isMine
        ? AppColors.ink(context)
        : (isDark
              ? Colors.white.withValues(alpha: 0.06)
              : Colors.black.withValues(alpha: 0.04));
    textColor = isMine
        ? (isDark ? Colors.black : Colors.white)
        : scheme.onSurface;
  } else {
    bubbleColor = isMine
        ? (isDark ? const Color(0xFF3E2C23) : AppColors.coffeeBrown)
        : (isDark ? AppColors.darkSurfaceElevated : Colors.white);
    textColor = isMine
        ? (isDark ? AppColors.darkTextPrimary : Colors.white)
        : scheme.onSurface;
  }
  final myEmoji = message.myReactionEmoji(myUserId);
  final summary = message.reactionSummary;
  final hasReactions = summary.isNotEmpty;
  final isAnimatedEmoji =
      !message.isDeleted &&
      message.messageType == ChatMessageType.text &&
      isSingleAnimatedEmojiMessage(message.textContent);
  final isVoiceMessage =
      !message.isDeleted && message.messageType == ChatMessageType.voice;
  final voiceHasCaption =
      isVoiceMessage && (message.textContent?.trim().isNotEmpty ?? false);
  final voiceHasReply =
      isVoiceMessage &&
      (message.replyTo != null || message.replyToMessageId != null);
  final isStandaloneVoice =
      isVoiceMessage && !voiceHasCaption && !voiceHasReply;
  final metaColor = (isAnimatedEmoji || isStandaloneVoice)
      ? AppColors.inkAlpha(context, 0.45)
      : textColor.withValues(alpha: 0.65);

  Widget messageColumn({required Color bodyTextColor}) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      mainAxisSize: MainAxisSize.min,
```

```
children: [
  if (!isMine)
    Padding(
      padding: const EdgeInsets.only(bottom: 4),
      child: SenderHeader(
        message: message,
        nameColor: _senderColor(
          message.senderId,
          monochrome: isMono,
          isDark: isDark,
        ),
      ),
    ),
  if (message.isDeleted)
    Text(
      context.l10n.chatMessageDeleted,
      style: TextStyle(
        fontStyle: FontStyle.italic,
        color: bodyTextColor.withValues(alpha: 0.7),
      ),
    )
  else ...[
    if (message.replyTo != null || message.replyToMessageId != null)
      Padding(
        padding: const EdgeInsets.only(bottom: 6),
        child: ReplyQuote(
          reply: message.replyTo,
          accent: isMono
              ? bodyTextColor.withValues(alpha: 0.72)
              : (isMine
                    ? (isDark
                          ? const Color(0xFF9ED0FF)
                          : Colors.white.withValues(alpha: 0.9))
                    : AppColors.accent(context)),
          onSurface: bodyTextColor,
          sharp: isMono,
          onTap: () {
            final id = message.replyTo?.id ?? message.replyToMessageId;
            if (id != null) onJumpToReply?.call(id);
          },
        ),
      ),
    MessageBody(
      message: message,
      textColor: bodyTextColor,
      galleryItems: galleryItems,
      timeLabel: time,
      isMine: isMine,
    ),
    if (message.textContent != null &&
        message.textContent!.isNotEmpty &&
        message.messageType != ChatMessageType.text &&
        message.messageType != ChatMessageType.file)
      Padding(
        padding: const EdgeInsets.only(top: 6),
        child: Text(
          message.textContent!,
          style: TextStyle(color: bodyTextColor, height: 1.35),
        ),
      ),
  ],
  if (!isVoiceMessage || voiceHasCaption) ...[
    const SizedBox(height: 4),
    Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        if (message.wasEdited)
          Padding(
            padding: const EdgeInsets.only(right: 6),
            child: Text(
              context.l10n.chatEdited,
              style: TextStyle(fontSize: 10, color: metaColor),
            ),
```

```
              ),
            Text(time, style: TextStyle(fontSize: 10, color: metaColor)),
            if (isMine && !message.isDeleted) ...[
              const SizedBox(width: 4),
              Icon(
                message.isReadByOthers
                    ? Icons.done_all_rounded
                    : Icons.done_rounded,
                size: 14,
                color: message.isReadByOthers
                    ? (isMono
                          ? bodyTextColor.withValues(alpha: 0.72)
                          : const Color(0xFF9ED0FF))
                    : metaColor,
              ),
            ],
          ],
        ),
      ],
    ],
  );
}

final highlightAccent = AppColors.accent(context).withValues(alpha: 0.85);
final highlightGlow = AppColors.accent(context).withValues(alpha: 0.35);

Widget bubbleBody;
if (isAnimatedEmoji || isStandaloneVoice) {
  final radius = BorderRadius.circular(
    isMono ? 6 : (isStandaloneVoice ? 999 : 24),
  );
  bubbleBody = AnimatedScale(
    scale: highlighted ? (isMono ? 1.02 : 1.06) : 1.0,
    duration: const Duration(milliseconds: 240),
    curve: isMono ? Curves.easeOutCubic : Curves.easeOutBack,
    child: AnimatedContainer(
      duration: const Duration(milliseconds: 240),
      curve: Curves.easeOut,
      padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 2),
      decoration: BoxDecoration(
        borderRadius: radius,
        border: Border.all(
          color: highlighted ? highlightAccent : Colors.transparent,
          width: 1.5,
        ),
        boxShadow: highlighted && !isMono
            ? [
                BoxShadow(
                  color: highlightGlow,
                  blurRadius: 16,
                  offset: const Offset(0, 2),
                ),
              ]
            : null,
      ),
      child: messageColumn(bodyTextColor: textColor),
    ),
  );
} else {
  final radius = isMono
      ? BorderRadius.only(
          topLeft: const Radius.circular(6),
          topRight: const Radius.circular(6),
          bottomLeft: Radius.circular(isMine ? 6 : 2),
          bottomRight: Radius.circular(isMine ? 2 : 6),
        )
      : BorderRadius.only(
          topLeft: const Radius.circular(20),
          topRight: const Radius.circular(20),
          bottomLeft: Radius.circular(isMine ? 20 : 6),
          bottomRight: Radius.circular(isMine ? 6 : 20),
        );
  final List<BoxShadow> bubbleShadows;
```

```
  if (isMono) {
    bubbleShadows = const [];
  } else if (highlighted) {
    bubbleShadows = [
      BoxShadow(
        color: highlightGlow,
        blurRadius: 16,
        offset: const Offset(0, 2),
      ),
    ];
  } else if (isNeu) {
    bubbleShadows = isMine
        ? [
            BoxShadow(
              color: Neumorphic.shadowColor(
                context,
              ).withValues(alpha: isDark ? 0.45 : 0.20),
              offset: const Offset(3, 4),
              blurRadius: 10,
            ),
          ]
        : Neumorphic.shadows(
            context,
            elevation: NeuElevation.raised,
            intensity: 0.75,
          );
  } else {
    bubbleShadows = [
      BoxShadow(
        color: Colors.black.withValues(alpha: isDark ? 0.25 : 0.04),
        blurRadius: 8,
        offset: const Offset(0, 2),
      ),
    ];
  }
  final Color bubbleBorder;
  if (highlighted) {
    bubbleBorder = highlightAccent;
  } else if (isMono && !isMine) {
    bubbleBorder = AppColors.inkAlpha(context, 0.12);
  } else {
    bubbleBorder = Colors.transparent;
  }
  bubbleBody = AnimatedScale(
    scale: highlighted ? (isMono ? 1.01 : 1.03) : 1.0,
    duration: const Duration(milliseconds: 240),
    curve: isMono ? Curves.easeOutCubic : Curves.easeOutBack,
    child: AnimatedContainer(
      duration: const Duration(milliseconds: 240),
      curve: Curves.easeOut,
      padding: EdgeInsets.fromLTRB(
        isMono ? 12 : 12,
        isMono ? 9 : 8,
        isMono ? 12 : 12,
        isMono ? 7 : 6,
      ),
      decoration: BoxDecoration(
        color: bubbleColor,
        borderRadius: radius,
        border: Border.all(
          color: bubbleBorder,
          width: highlighted ? 1.5 : 1,
        ),
        boxShadow: bubbleShadows,
      ),
      child: messageColumn(bodyTextColor: textColor),
    ),
  );
}

final bubble = GestureDetector(
  onTap: selectionMode
      ? (selectable ? onSelectionToggle : null)
```

```
          : null,
      onLongPress: message.isDeleted
          ? null
          : selectionMode
          ? (selectable ? onSelectionToggle : null)
          : () {
              HapticFeedback.mediumImpact();
              _showActions(context);
            },
      child: bubbleBody,
    );

    final bubbleWithReactions = Column(
      crossAxisAlignment: isMine
          ? CrossAxisAlignment.end
          : CrossAxisAlignment.start,
      children: [
        bubble,
        if (hasReactions)
          Transform.translate(
```

**lib/screens/chat/room/widgets/message_bubble.dart — continued (2/3)**

```
            offset: const Offset(0, -6),
            child: Padding(
              padding: EdgeInsets.only(
                left: isMine ? 0 : 4,
                right: isMine ? 4 : 0,
              ),
              child: Wrap(
                spacing: 4,
                runSpacing: 4,
                children: [
                  for (final item in summary)
                    ReactionPill(
                      emoji: item.emoji,
                      count: item.count,
                      mine: myEmoji == item.emoji,
                      onTap: onReact == null
                          ? null
                          : () => onReact!(item.emoji),
                    ),
                ],
              ),
            ),
          ),
      ],
    );

    return LayoutBuilder(
      builder: (context, constraints) {
        final mineMax = (constraints.maxWidth * 0.78).clamp(
          160.0,
          constraints.maxWidth,
        );
        final incomingMax = (constraints.maxWidth - 36).clamp(
          120.0,
          constraints.maxWidth * 0.85,
        );

        if (isMine) {
          final child = Align(
            alignment: Alignment.centerRight,
            child: ConstrainedBox(
              constraints: BoxConstraints(maxWidth: mineMax),
              child: Padding(
                padding: EdgeInsets.only(top: 4, bottom: hasReactions ? 10 : 4),
                child: bubbleWithReactions,
              ),
            ),
          );
          final wrapped = selectionMode && selectable
              ? _selectionRow(context, child)
```

```
      : child;
  if (onReply == null || selectionMode) return wrapped;
  return SwipeToReply(onReply: onReply!, child: wrapped);
}

final initials = message.displayName
    .trim()
    .split(RegExp(r'\s+'))
    .where((p) => p.isNotEmpty)
    .take(2)
    .map((p) => p[0].toUpperCase())
    .join();

final senderTone = _senderColor(
  message.senderId,
  monochrome: isMono,
  isDark: isDark,
);
final incomingAvatar = isMono
    ? Container(
        width: 28,
        height: 28,
        alignment: Alignment.center,
        clipBehavior: Clip.antiAlias,
        decoration: BoxDecoration(
          color: senderTone.withValues(alpha: 0.16),
          borderRadius: BorderRadius.circular(4),
          border: Border.all(color: senderTone.withValues(alpha: 0.28)),
          image:
              message.senderAvatarUrl != null &&
                  message.senderAvatarUrl!.isNotEmpty
              ? DecorationImage(
                  image: CachedNetworkImageProvider(
                    message.senderAvatarUrl!,
                  ),
                  fit: BoxFit.cover,
                )
              : null,
        ),
        child:
            message.senderAvatarUrl == null ||
                message.senderAvatarUrl!.isEmpty
            ? Text(
                initials.isEmpty ? '?' : initials,
                style: TextStyle(
                  fontSize: 11,
                  fontWeight: FontWeight.w700,
                  color: senderTone,
                ),
              )
            : null,
      )
    : CircleAvatar(
        radius: 14,
        backgroundColor: senderTone.withValues(alpha: 0.22),
        backgroundImage:
            message.senderAvatarUrl != null &&
                message.senderAvatarUrl!.isNotEmpty
            ? CachedNetworkImageProvider(message.senderAvatarUrl!)
            : null,
        child:
            message.senderAvatarUrl == null ||
                message.senderAvatarUrl!.isEmpty
            ? Text(
                initials.isEmpty ? '?' : initials,
                style: TextStyle(
                  fontSize: 11,
                  fontWeight: FontWeight.w700,
                  color: senderTone,
                ),
              )
            : null,
      );
```

```
      final incoming = Padding(
        padding: EdgeInsets.only(top: 4, bottom: hasReactions ? 10 : 4),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.end,
          children: [
            incomingAvatar,
            const SizedBox(width: 8),
            Flexible(
              child: Align(
                alignment: Alignment.centerLeft,
                child: ConstrainedBox(
                  constraints: BoxConstraints(maxWidth: incomingMax),
                  child: bubbleWithReactions,
                ),
              ),
            ),
          ],
        ),
      );
      final wrapped = selectionMode && selectable
          ? _selectionRow(context, incoming)
          : incoming;
      if (onReply == null || selectionMode) return wrapped;
      return SwipeToReply(onReply: onReply!, child: wrapped);
    },
  );
}

Widget _selectionRow(BuildContext context, Widget child) {
  final accent = AppColors.accent(context);
  final muted = AppColors.inkAlpha(context, 0.35);
  return GestureDetector(
    behavior: HitTestBehavior.opaque,
    onTap: selectable ? onSelectionToggle : null,
    child: AnimatedContainer(
      duration: const Duration(milliseconds: 160),
      color: selected ? accent.withValues(alpha: 0.08) : Colors.transparent,
      padding: const EdgeInsets.symmetric(horizontal: 4),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.center,
        children: [
          Icon(
            selected ? Icons.check_circle_rounded : Icons.circle_outlined,
            size: 22,
            color: selectable
                ? (selected ? accent : muted)
                : muted.withValues(alpha: 0.45),
          ),
          const SizedBox(width: 6),
          // Ignore media InkWells (files/images/video/voice) so taps select
          // the message instead of opening/downloading the attachment.
          Expanded(
            child: IgnorePointer(
              ignoring: selectable,
              child: child,
            ),
          ),
        ],
      ),
    ),
  );
}

void _showActions(BuildContext context) {
  final box = context.findRenderObject() as RenderBox?;
  if (box == null || !box.hasSize) return;

  final overlay = Overlay.maybeOf(context, rootOverlay: true);
  if (overlay == null) return;

  final origin = box.localToGlobal(Offset.zero);
  final rect = origin & box.size;
  final textToCopy = message.isDeleted ? null : message.textContent?.trim();
```

```
final canCopy = textToCopy != null && textToCopy.isNotEmpty;

late OverlayEntry entry;
void dismiss() {
  if (entry.mounted) entry.remove();
}

entry = OverlayEntry(
  builder: (ctx) {
    return Material(
      type: MaterialType.transparency,
      child: Stack(
        children: [
          Positioned.fill(
            child: GestureDetector(
              behavior: HitTestBehavior.opaque,
              onTap: dismiss,
              child: ColoredBox(
                color: Colors.black.withValues(alpha: 0.42),
              ),
            ),
          ),
          MessageActionPopover(
            anchorRect: rect,
            isMine: isMine,
            emojis: onReact == null ? const [] : chatReactionEmojis,
            onEmoji: onReact == null
                ? null
                : (emoji) {
                    dismiss();
                    onReact!(emoji);
                  },
            onReply: onReply == null
                ? null
                : () {
                    dismiss();
                    onReply!();
                  },
            onCopy: !canCopy
                ? null
                : () async {
                    await Clipboard.setData(ClipboardData(text: textToCopy));
                    dismiss();
                    if (context.mounted) {
                      ScaffoldMessenger.of(context).showSnackBar(
                        SnackBar(content: Text(context.l10n.chatCopied)),
                      );
                    }
                  },
            onEdit: onEdit == null
                ? null
                : () {
                    dismiss();
                    onEdit!();
                  },
            onSelect: onSelect == null
                ? null
                : () {
                    dismiss();
                    onSelect!();
                  },
            onDelete: onDelete == null
                ? null
                : () {
                    dismiss();
                    onDelete!();
                  },
          ),
        ],
      ),
    );
  },
);
```

```
    overlay.insert(entry);
  }
}

class MessageActionPopover extends StatelessWidget {
  const MessageActionPopover({
    super.key,
    required this.anchorRect,
    required this.isMine,
    required this.emojis,
    this.onEmoji,
    this.onReply,
    this.onCopy,
    this.onEdit,
    this.onSelect,
    this.onDelete,
  });

  final Rect anchorRect;
  final bool isMine;
  final List<String> emojis;
  final ValueChanged<String>? onEmoji;
  final VoidCallback? onReply;
  final VoidCallback? onCopy;
  final VoidCallback? onEdit;
  final VoidCallback? onSelect;
  final VoidCallback? onDelete;

  static const _menuWidth = 220.0;
  static const _reactionHeight = 60.0;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final media = MediaQuery.of(context);
    final screen = media.size;
    final pad = media.padding;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    final surface = isDark
        ? const Color(0xE62A2F36)
        : Colors.white.withValues(alpha: 0.96);
    final dividerColor = isDark
        ? Colors.white.withValues(alpha: 0.08)
        : Colors.black.withValues(alpha: 0.08);
    final itemColor = isDark ? Colors.white : AppColors.darkChocolate;

    final reactionWidth = emojis.isEmpty ? 0.0 : (emojis.length * 48.0) + 20;
    final panelWidth = [
      if (emojis.isNotEmpty) reactionWidth,
      _menuWidth,
    ].fold<double>(0, (a, b) => a > b ? a : b);

    final itemCount = [
      onReply,
      onCopy,
      onEdit,
      onSelect,
      onDelete,
    ].where((callback) => callback != null).length;
    final menuHeight = itemCount * 44.0 + 8;
    final panelHeight =
        (emojis.isEmpty ? 0.0 : _reactionHeight + 8) + menuHeight;
    final bottomReserve = pad.bottom + media.viewInsets.bottom + 108;
    final spaceBelow = screen.height - bottomReserve - anchorRect.bottom;
    final spaceAbove = anchorRect.top - pad.top - 8;
    final placeAbove =
        spaceBelow < panelHeight + 12 && spaceAbove >= panelHeight * 0.45;
    final stackTop = placeAbove
        ? (anchorRect.top - panelHeight - 10).clamp(
            pad.top + 8,
            math.max(pad.top + 8, screen.height - panelHeight - bottomReserve),
          )
```

```
    : (anchorRect.bottom + 10).clamp(
        pad.top + 8,
        math.max(pad.top + 8, screen.height - panelHeight - bottomReserve),
      );

var left = isMine ? anchorRect.right - panelWidth : anchorRect.left;
left = left.clamp(12.0, screen.width - panelWidth - 12);

Widget menuDivider() =>
    Divider(height: 1, thickness: 1, color: dividerColor);

final menuItems = <Widget>[
  if (onReply != null)
    PopoverMenuItem(
      icon: Icons.reply_rounded,
      label: l10n.chatReply,
      color: itemColor,
      onTap: onReply!,
    ),
  if (onReply != null &&
      (onCopy != null ||
          onEdit != null ||
          onSelect != null ||
          onDelete != null))
    menuDivider(),
  if (onCopy != null)
    PopoverMenuItem(
      icon: Icons.copy_rounded,
      label: l10n.chatCopyText,
      color: itemColor,
      onTap: onCopy!,
    ),
  if (onEdit != null)
    PopoverMenuItem(
      icon: Icons.edit_outlined,
      label: l10n.commonEdit,
      color: itemColor,
      onTap: onEdit!,
    ),
  if (onSelect != null)
    PopoverMenuItem(
      icon: Icons.check_circle_outline_rounded,
      label: l10n.chatSelect,
      color: itemColor,
      onTap: onSelect!,
    ),
  if (onDelete != null) ...[
    if (onReply != null ||
        onCopy != null ||
        onEdit != null ||
        onSelect != null)
      menuDivider(),
    PopoverMenuItem(
      icon: Icons.delete_outline_rounded,
      label: l10n.commonDelete,
      color: AppColors.errorRed,
      onTap: onDelete!,
    ),
  ],
];

return Positioned(
  left: left,
  top: stackTop.toDouble(),
  child: Material(
    type: MaterialType.transparency,
    child: Column(
      crossAxisAlignment: isMine
          ? CrossAxisAlignment.end
          : CrossAxisAlignment.start,
      mainAxisSize: MainAxisSize.min,
      children: [
        if (emojis.isNotEmpty)
```

```
              ReactionBar(
                emojis: emojis,
                surface: surface,
                onEmoji: onEmoji!,
                showCaretBelow: placeAbove,
                isMine: isMine,
              ),
            if (emojis.isNotEmpty && menuItems.isNotEmpty)
```

**lib/screens/chat/room/widgets/message_bubble.dart — continued (3/3)**

```
              const SizedBox(height: 8),
            if (menuItems.isNotEmpty)
              Builder(
                builder: (context) {
                  final isMono =
                      AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??
                      false;
                  return Container(
                    width: _menuWidth,
                    decoration: BoxDecoration(
                      color: surface,
                      borderRadius: BorderRadius.circular(isMono ? 6 : 16),
                      border: isMono
                          ? Border.all(color: AppColors.inkAlpha(context, 0.12))
                          : null,
                      boxShadow: isMono
                          ? null
                          : [
                              BoxShadow(
                                color: Colors.black.withValues(alpha: 0.28),
                                blurRadius: 24,
                                offset: const Offset(0, 10),
                              ),
                            ],
                    ),
                    clipBehavior: Clip.antiAlias,
                    child: Column(
                      mainAxisSize: MainAxisSize.min,
                      children: menuItems,
                    ),
                  );
                },
              ),
          ],
        ),
      ),
    );
  }
}

class ReactionBar extends StatelessWidget {
  const ReactionBar({
    super.key,
    required this.emojis,
    required this.surface,
    required this.onEmoji,
    required this.showCaretBelow,
    required this.isMine,
  });

  final List<String> emojis;
  final Color surface;
  final ValueChanged<String> onEmoji;
  final bool showCaretBelow;
  final bool isMine;

  @override
  Widget build(BuildContext context) {
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    final barRadius = isMono ? 6.0 : 999.0;
    return Column(
```

```
        crossAxisAlignment: isMine
            ? CrossAxisAlignment.end
            : CrossAxisAlignment.start,
        mainAxisSize: MainAxisSize.min,
        children: [
          if (!showCaretBelow)
            Padding(
              padding: EdgeInsets.only(
                left: isMine ? 0 : 22,
                right: isMine ? 22 : 0,
              ),
              child: CustomPaint(
                size: const Size(14, 7),
                painter: CaretPainter(color: surface, pointUp: true),
              ),
            ),
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 4),
            decoration: BoxDecoration(
              color: surface,
              borderRadius: BorderRadius.circular(barRadius),
              border: isMono
                  ? Border.all(color: AppColors.inkAlpha(context, 0.12))
                  : null,
              boxShadow: isMono
                  ? null
                  : [
                      BoxShadow(
                        color: Colors.black.withValues(alpha: 0.28),
                        blurRadius: 20,
                        offset: const Offset(0, 8),
                      ),
                    ],
            ),
            child: Row(
              mainAxisSize: MainAxisSize.min,
              children: [
                for (final emoji in emojis)
                  InkWell(
                    borderRadius: BorderRadius.circular(isMono ? 4 : 20),
                    onTap: () => onEmoji(emoji),
                    child: Padding(
                      padding: const EdgeInsets.symmetric(
                        horizontal: 6,
                        vertical: 4,
                      ),
                      child: ChatLottieEmoji(
                        emoji: emoji,
                        size: 32,
                        playOnTap: false,
                        repeat: true,
                      ),
                    ),
                  ),
              ],
            ),
          ),
          if (showCaretBelow)
            Padding(
              padding: EdgeInsets.only(
                left: isMine ? 0 : 22,
                right: isMine ? 22 : 0,
              ),
              child: CustomPaint(
                size: const Size(14, 7),
                painter: CaretPainter(color: surface, pointUp: false),
              ),
            ),
        ],
      );
    }
  }
```

```
class CaretPainter extends CustomPainter {
  const CaretPainter({required this.color, required this.pointUp});

  final Color color;
  final bool pointUp;

  @override
  void paint(Canvas canvas, Size size) {
    final path = Path();
    if (pointUp) {
      path.moveTo(0, size.height);
      path.lineTo(size.width / 2, 0);
      path.lineTo(size.width, size.height);
    } else {
      path.moveTo(0, 0);
      path.lineTo(size.width / 2, size.height);
      path.lineTo(size.width, 0);
    }
    path.close();
    canvas.drawPath(path, Paint()..color = color);
  }

  @override
  bool shouldRepaint(covariant CaretPainter oldDelegate) =>
      oldDelegate.color != color || oldDelegate.pointUp != pointUp;
}

class PopoverMenuItem extends StatelessWidget {
  const PopoverMenuItem({
    super.key,
    required this.icon,
    required this.label,
    required this.color,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final Color color;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: onTap,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
        child: Row(
          children: [
            Icon(icon, size: 20, color: color),
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                label,
                style: TextStyle(
                  color: color,
                  fontSize: 15,
                  fontWeight: FontWeight.w500,
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class ReactionPill extends StatelessWidget {
  const ReactionPill({
    super.key,
    required this.emoji,
    required this.count,
```

```
    required this.mine,
    this.onTap,
  });

  final String emoji;
  final int count;
  final bool mine;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    final Color fill;
    final Color border;
    if (mine) {
      fill = AppColors.accent(context).withValues(alpha: isDark ? 0.32 : 0.20);
      border = AppColors.accent(context).withValues(alpha: 0.85);
    } else {
      fill = isDark
          ? AppColors.darkSurfaceHighest
          : AppColors.sand.withValues(alpha: 0.95);
      border = AppColors.inkAlpha(context, 0.12);
    }
    final chipRadius = isMono ? 4.0 : 999.0;
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onTap,
        borderRadius: BorderRadius.circular(chipRadius),
        child: Container(
          padding: const EdgeInsets.fromLTRB(6, 3, 8, 3),
          decoration: BoxDecoration(
            color: fill,
            borderRadius: BorderRadius.circular(chipRadius),
            border: Border.all(color: border),
          ),
          child: Row(
            mainAxisSize: MainAxisSize.min,
            children: [
              ChatLottieEmoji(emoji: emoji, size: 20, repeat: false),
              const SizedBox(width: 4),
              Text(
                '$count',
                style: TextStyle(
                  fontSize: 12,
                  fontWeight: FontWeight.w700,
                  color: mine
                      ? AppColors.accent(context)
                      : AppColors.ink(context),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class SenderHeader extends StatelessWidget {
  const SenderHeader({
    super.key,
    required this.message,
    required this.nameColor,
  });

  final ChatMessage message;
  final Color nameColor;

  @override
  Widget build(BuildContext context) {
```

```
    final l10n = context.l10n;
    final name = message.displayName;
    if (name.isEmpty) return const SizedBox.shrink();

    final role = message.senderRole;
    final tagLabel = switch (role) {
      UserRole.admin => l10n.chatRoleAdmin,
      UserRole.teacher => l10n.chatRoleTeacher,
      _ => null,
    };

    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        Flexible(
          child: Text(
            name,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: TextStyle(
              fontWeight: FontWeight.w700,
              fontSize: 12,
              color: nameColor,
            ),
          ),
        ),
        if (tagLabel != null) ...[
          const SizedBox(width: 6),
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 7, vertical: 2),
            decoration: BoxDecoration(
              color: nameColor.withValues(alpha: 0.16),
              borderRadius: BorderRadius.circular(
                (AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??
                        false)
                    ? 3
                    : 8,
              ),
            ),
            child: Text(
              tagLabel,
              style: TextStyle(
                fontSize: 10,
                fontWeight: FontWeight.w700,
                color: nameColor.withValues(alpha: 0.95),
                height: 1.1,
              ),
            ),
          ),
        ],
      ],
    );
  }
}

class MessageBody extends StatelessWidget {
  const MessageBody({
    super.key,
    required this.message,
    required this.textColor,
    this.galleryItems = const [],
    this.timeLabel = '',
    this.isMine = false,
  });

  final ChatMessage message;
  final Color textColor;
  final List<ChatMediaItem> galleryItems;
  final String timeLabel;
  final bool isMine;

  int get _galleryIndex =>
      galleryItems.indexWhere((item) => item.id == message.id);
```

```
  @override
  Widget build(BuildContext context) {
    return switch (message.messageType) {
      ChatMessageType.text => _buildTextBody(),
      ChatMessageType.image => ChatImage(
        url: message.mediaUrl!,
        galleryItems: galleryItems,
        galleryIndex: _galleryIndex,
      ),
      ChatMessageType.video => ChatVideo(
        url: message.mediaUrl!,
        galleryItems: galleryItems,
        galleryIndex: _galleryIndex,
      ),
      ChatMessageType.voice => ChatVoice(
        url: message.mediaUrl!,
        timeLabel: timeLabel,
        isMine: isMine,
        isReadByOthers: message.isReadByOthers,
      ),
      ChatMessageType.file => ChatFile(
        url: message.mediaUrl!,
        fileName: message.textContent?.trim().isNotEmpty == true
            ? message.textContent!.trim()
            : 'file',
        color: textColor,
      ),
    };
  }

  Widget _buildTextBody() {
    final text = message.textContent ?? '';
    final asset = singleAnimatedEmojiAsset(text);
    final emoji = singleAnimatedEmojiGrapheme(text);
    if (asset != null && emoji != null) {
      return AnimatedEmojiMessage(
        key: ValueKey('anim-emoji-${message.id}'),
        assetPath: asset,
        emoji: emoji,
        size: 144,
      );
    }
    return Text(
      text,
      style: TextStyle(color: textColor, height: 1.35, fontSize: 15),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/message_bubble.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

# lib/screens/chat/room/widgets/reply_composer_banner.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 105. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/screens/chat/room/helpers/reply_snippets.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/reply_composer_banner.dart**

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/screens/chat/room/helpers/reply_snippets.dart';

class ReplyComposerBanner extends StatelessWidget {
  const ReplyComposerBanner({
    super.key,
    required this.message,
    required this.onClose,
  });

  final ChatMessage message;
  final VoidCallback onClose;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final accent = AppColors.accent(context);
    final name = message.displayName.isEmpty ? '…' : message.displayName;
    final snippet = chatReplySnippetFromMessage(context, message);
    final showThumb =
        message.messageType == ChatMessageType.image &&
        message.mediaUrl != null &&
        message.mediaUrl!.isNotEmpty;

    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    final radius = isMono ? 4.0 : 14.0;
    return Material(
      color: AppColors.controlSurface(context),
      borderRadius: BorderRadius.circular(radius),
      child: Padding(
        padding: const EdgeInsets.fromLTRB(10, 8, 4, 8),
        child: Row(
          children: [
```

```
            Container(
              width: 3,
              height: 40,
              decoration: BoxDecoration(
                color: accent,
                borderRadius: BorderRadius.circular(isMono ? 1 : 2),
              ),
            ),
            const SizedBox(width: 8),
            if (showThumb) ...[
              ClipRRect(
                borderRadius: BorderRadius.circular(isMono ? 3 : 6),
                child: CachedNetworkImage(
                  imageUrl: message.mediaUrl!,
                  width: 36,
                  height: 36,
                  fit: BoxFit.cover,
                  errorWidget: (_, _, _) =>
                      const SizedBox(width: 36, height: 36),
                ),
              ),
              const SizedBox(width: 8),
            ],
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text(
                    l10n.chatReplyTo(name),
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: TextStyle(
                      color: accent,
                      fontWeight: FontWeight.w700,
                      fontSize: 13,
                    ),
                  ),
                  const SizedBox(height: 2),
                  Text(
                    snippet,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: TextStyle(
                      color: AppColors.inkAlpha(context, 0.75),
                      fontSize: 13,
                    ),
                  ),
                ],
              ),
            ),
            IconButton(
              onPressed: onClose,
              visualDensity: VisualDensity.compact,
              icon: Icon(
                Icons.close_rounded,
                size: 20,
                color: AppColors.inkAlpha(context, 0.55),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/reply_composer_banner.dart?

2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/chat/room/widgets/reply_quote.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 105. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/screens/chat/room/helpers/reply_snippets.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/reply_quote.dart**

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/screens/chat/room/helpers/reply_snippets.dart';

class ReplyQuote extends StatelessWidget {
  const ReplyQuote({
    super.key,
    required this.reply,
    required this.accent,
    required this.onSurface,
    this.sharp = false,
    this.onTap,
  });

  final ChatReplyPreview? reply;
  final Color accent;
  final Color onSurface;
  final bool sharp;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final name = (reply?.displayName ?? '').trim().isEmpty
        ? '…'
        : reply!.displayName;
    final snippet = chatReplySnippet(context, reply);
    final showThumb =
        reply != null &&
        !reply!.isDeleted &&
        reply!.messageType == ChatMessageType.image &&
        reply!.mediaUrl != null &&
        reply!.mediaUrl!.isNotEmpty;
    final radius = sharp ? 4.0 : 10.0;

    return Material(
      color: onSurface.withValues(alpha: 0.08),
      borderRadius: BorderRadius.circular(radius),
      child: InkWell(
        onTap: onTap,
```

```
          borderRadius: BorderRadius.circular(radius),
          child: Padding(
            padding: const EdgeInsets.fromLTRB(8, 6, 8, 6),
            child: Row(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Container(
                  width: 3,
                  height: showThumb ? 36 : 32,
                  decoration: BoxDecoration(
                    color: accent,
                    borderRadius: BorderRadius.circular(sharp ? 1 : 2),
                  ),
                ),
                const SizedBox(width: 8),
                if (showThumb) ...[
                  ClipRRect(
                    borderRadius: BorderRadius.circular(sharp ? 3 : 5),
                    child: CachedNetworkImage(
                      imageUrl: reply!.mediaUrl!,
                      width: 32,
                      height: 32,
                      fit: BoxFit.cover,
                      errorWidget: (_, _, _) =>
                          const SizedBox(width: 32, height: 32),
                    ),
                  ),
                  const SizedBox(width: 8),
                ],
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        l10n.chatReplyTo(name),
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
                        style: TextStyle(
                          color: accent,
                          fontWeight: FontWeight.w700,
                          fontSize: 12,
                        ),
                      ),
                      const SizedBox(height: 1),
                      Text(
                        snippet,
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
                        style: TextStyle(
                          color: onSurface.withValues(alpha: 0.85),
                          fontSize: 12,
                        ),
                      ),
                    ],
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/reply_quote.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?

5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/chat/room/widgets/swipe_to_reply.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 70. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:mu_study/core/theme/app_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/chat/room/widgets/swipe_to_reply.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:mu_study/core/theme/app_colors.dart';

class SwipeToReply extends StatefulWidget {
  const SwipeToReply({super.key, required this.child, required this.onReply});

  final Widget child;
  final VoidCallback onReply;

  @override
  State<SwipeToReply> createState() => _SwipeToReplyState();
}

class _SwipeToReplyState extends State<SwipeToReply> {
  static const _threshold = 56.0;
  double _dx = 0;
  bool _triggered = false;

  @override
  Widget build(BuildContext context) {
    final progress = (_dx / _threshold).clamp(0.0, 1.0);
    return GestureDetector(
      behavior: HitTestBehavior.translucent,
      onHorizontalDragUpdate: (details) {
        final next = (_dx + details.delta.dx).clamp(0.0, _threshold * 1.35);
        setState(() => _dx = next);
        if (!_triggered && _dx >= _threshold) {
          _triggered = true;
          HapticFeedback.selectionClick();
        }
      },
      onHorizontalDragEnd: (_) {
        final shouldReply = _dx >= _threshold;
        setState(() {
          _dx = 0;
          _triggered = false;
        });
        if (shouldReply) widget.onReply();
      },
      onHorizontalDragCancel: () {
        setState(() {
          _dx = 0;
          _triggered = false;
        });
```

```
      },
      child: Stack(
        alignment: Alignment.centerLeft,
        children: [
          if (progress > 0)
            Opacity(
              opacity: progress,
              child: Padding(
                padding: const EdgeInsets.only(left: 8),
                child: Icon(
                  Icons.reply_rounded,
                  color: AppColors.accent(context),
                  size: 22 + (progress * 4),
                ),
              ),
            ),
          Transform.translate(
            offset: Offset(_dx * 0.85, 0),
            child: widget.child,
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/chat/room/widgets/swipe_to_reply.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/profile/app_profile_screen.dart

Shared profile (theme, language, password, logout). This file (app profile screen) is one concrete piece of that layer.

Line count: 747. Import count: 28. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:image_picker/image_picker.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/settings/language_settings_tile.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/progress/progress_report_share.dart';
- import 'package:mu_study/core/services/app_cache.dart';
- import 'package:mu_study/core/services/cache_service.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- … and 10 more imports

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/profile/app_profile_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:image_picker/image_picker.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/settings/language_settings_tile.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/progress/progress_report_share.dart';
import 'package:mu_study/core/services/app_cache.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
```

```
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/profile/widgets/profile_hero_card.dart';
import 'package:mu_study/screens/profile/widgets/profile_settings_row.dart';
import 'package:mu_study/screens/profile/widgets/profile_theme_selector.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:package_info_plus/package_info_plus.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class AppProfileScreen extends ConsumerWidget {
  const AppProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final profile = ref.watch(currentProfileProvider);
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.profileTitle)),
      body: profile.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (p) {
          if (p == null) {
            return EmptyState(title: l10n.profileNotSignedIn);
          }
          return _ProfileBody(profile: p);
        },
      ),
    );
  }
}

class _ProfileBody extends ConsumerStatefulWidget {
  const _ProfileBody({required this.profile});

  final Profile profile;

  @override
  ConsumerState<_ProfileBody> createState() => _ProfileBodyState();
}

class _ProfileBodyState extends ConsumerState<_ProfileBody> {
  String _version = '—';
  bool _uploadingAvatar = false;

  @override
  void initState() {
    super.initState();
    PackageInfo.fromPlatform()
        .then((info) {
          if (mounted) setState(() => _version = info.version);
        })
        .catchError((_) {});
  }

  String get _roleBasePath => switch (widget.profile.role) {
    UserRole.student => '/student',
    UserRole.teacher => '/teacher',
    UserRole.admin => '/admin',
  };

  List<_Shortcut> get _shortcuts {
    final l10n = context.l10n;
    final base = _roleBasePath;
    return switch (widget.profile.role) {
      UserRole.student => [
        _Shortcut(
          Icons.self_improvement_outlined,
          l10n.navFocus,
          '$base/focus',
        ),
        _Shortcut(Icons.grade_outlined, l10n.marksTitle, '$base/marks'),
```

```
        _Shortcut(
          Icons.assignment_outlined,
          l10n.assignmentsTitle,
          '$base/assignments',
        ),
        _Shortcut(
          Icons.event_available_outlined,
          l10n.attendanceTitle,
          '$base/attendance',
        ),
        _Shortcut(Icons.chat_outlined, l10n.navChat, '$base/chat'),
      ],
      UserRole.teacher => [
        _Shortcut(
          Icons.event_available_outlined,
          l10n.attendanceTitle,
          '$base/attendance',
        ),
        _Shortcut(Icons.grade_outlined, l10n.teacherGradebook, '$base/marks'),
        _Shortcut(
          Icons.assignment_outlined,
          l10n.assignmentsTitle,
          '$base/assignments',
        ),
        _Shortcut(Icons.chat_outlined, l10n.navChat, '$base/chat'),
      ],
      UserRole.admin => [
        _Shortcut(Icons.people_outline, l10n.navUsers, '$base/users'),
        _Shortcut(Icons.groups_outlined, l10n.navGroups, '$base/groups'),
        _Shortcut(
          Icons.calendar_month_outlined,
          l10n.adminAcademicYears,
          '$base/academic-years',
        ),
        _Shortcut(
          Icons.campaign_outlined,
          l10n.announcementsTitle,
          '$base/announcements',
        ),
        _Shortcut(
          Icons.notifications_outlined,
          l10n.profileNotifications,
          '$base/notifications',
        ),
      ],
    };
  }

  Future<void> _pickAvatar() async {
    final picker = ImagePicker();
    final file = await picker.pickImage(
      source: ImageSource.gallery,
      maxWidth: 1024,
      maxHeight: 1024,
      imageQuality: 85,
    );
    if (file == null || !mounted) return;

    setState(() => _uploadingAvatar = true);
    final l10n = context.l10n;
    try {
      final bytes = await file.readAsBytes();
      final path = '${widget.profile.id}/avatar.jpg';
      await supabase.storage
          .from('avatars')
          .uploadBinary(
            path,
            bytes,
            fileOptions: const FileOptions(
              upsert: true,
              contentType: 'image/jpeg',
            ),
          );
```

```
      final publicUrl = supabase.storage.from('avatars').getPublicUrl(path);
      final stamped = '$publicUrl?t=${DateTime.now().millisecondsSinceEpoch}';
      await ref.read(profileRepositoryProvider).updateProfile(
        widget.profile.id,
        {'avatar_url': stamped},
      );
      ref.invalidate(currentProfileProvider);
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.profilePhotoUpdated)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    } finally {
      if (mounted) setState(() => _uploadingAvatar = false);
    }
  }

  Future<void> _editName() async {
    final l10n = context.l10n;
    final controller = TextEditingController(text: widget.profile.fullName);
    final saved = await showDialog<String>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: Text(l10n.profileEditName),
        content: TextField(
          controller: controller,
          autofocus: true,
          textCapitalization: TextCapitalization.words,
          decoration: InputDecoration(labelText: l10n.profileFullName),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx),
            child: Text(l10n.commonCancel),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(ctx, controller.text.trim()),
            child: Text(l10n.commonSave),
          ),
        ],
      ),
    );
    if (saved == null || saved.isEmpty || !mounted) return;
    if (saved == widget.profile.fullName) return;

    try {
      await ref.read(profileRepositoryProvider).updateProfile(
        widget.profile.id,
        {'full_name': saved},
      );
      ref.invalidate(currentProfileProvider);
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.profileNameUpdated)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _changePassword() async {
    final l10n = context.l10n;
    final current = TextEditingController();
    final next = TextEditingController();
    final confirm = TextEditingController();
    var saving = false;
```

```
      await showDialog<void>(
        context: context,
        builder: (ctx) => StatefulBuilder(
          builder: (ctx, setLocal) => AlertDialog(
            title: Text(l10n.profileChangePassword),
            content: SingleChildScrollView(
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  TextField(
                    controller: current,
                    obscureText: true,
                    decoration: InputDecoration(
                      labelText: l10n.profileCurrentPassword,
                    ),
                  ),
                  const SizedBox(height: 12),
                  TextField(
                    controller: next,
                    obscureText: true,
                    decoration: InputDecoration(
                      labelText: l10n.profileNewPassword,
                    ),
                  ),
                  const SizedBox(height: 12),
                  TextField(
                    controller: confirm,
                    obscureText: true,
                    decoration: InputDecoration(
                      labelText: l10n.profileConfirmPassword,
                    ),
                  ),
                  const SizedBox(height: 8),
                  Text(
                    l10n.profilePasswordHint,
                    style: TextStyle(
                      fontSize: 12,
                      color: Theme.of(
                        ctx,
                      ).colorScheme.onSurface.withValues(alpha: 0.5),
                    ),
                  ),
                ],
              ),
            ),
            actions: [
              TextButton(
                onPressed: saving ? null : () => Navigator.pop(ctx),
                child: Text(l10n.commonCancel),
              ),
              FilledButton(
                onPressed: saving
                    ? null
                    : () async {
                        final n = next.text;
                        if (n.length < 8) {
                          ScaffoldMessenger.of(context).showSnackBar(
                            SnackBar(content: Text(l10n.profilePasswordTooShort)),
                          );
                          return;
                        }
                        if (n != confirm.text) {
                          ScaffoldMessenger.of(context).showSnackBar(
                            SnackBar(content: Text(l10n.profilePasswordMismatch)),
                          );
                          return;
                        }
                        setLocal(() => saving = true);
                        try {
                          await ref
                              .read(authRepositoryProvider)
                              .changePassword(
                                username: widget.profile.username,
```

```
                          currentPassword: current.text,
                          newPassword: n,
                        );
                    if (ctx.mounted) Navigator.pop(ctx);
                    if (!mounted) return;
                    ScaffoldMessenger.of(context).showSnackBar(
                      SnackBar(content: Text(l10n.profilePasswordChanged)),
                    );
                  } catch (e) {
                    setLocal(() => saving = false);
                    if (!mounted) return;
                    ScaffoldMessenger.of(context).showSnackBar(
                      SnackBar(
                        content: Text(l10n.commonCouldNotSave('$e')),
                      ),
                    );
                  }
                },
            child: saving
                ? const SizedBox(
                    width: 18,
                    height: 18,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  )
                : Text(l10n.commonSave),
          ),
        ],
      ),
    ),
  );
}

Future<void> _clearCachedData() async {
  final l10n = context.l10n;

  final first = await showDialog<bool>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: Text(l10n.profileClearCacheWarn1Title),
      content: Text(l10n.profileClearCacheWarn1Body),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          child: Text(l10n.commonCancel),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(ctx, true),
          style: FilledButton.styleFrom(backgroundColor: AppColors.errorRed),
          child: Text(l10n.profileClearCacheConfirm),
        ),
      ],
    ),
  );
  if (first != true || !mounted) return;

  final second = await showDialog<bool>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: Text(l10n.profileClearCacheWarn2Title),
      content: Text(l10n.profileClearCacheWarn2Body),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          child: Text(l10n.commonCancel),
        ),
        FilledButton(
          onPressed: () => Navigator.pop(ctx, true),
          style: FilledButton.styleFrom(backgroundColor: AppColors.errorRed),
          child: Text(l10n.profileClearCacheConfirm),
        ),
      ],
    ),
  );
```

```
    if (second != true || !mounted) return;

    try {
      final cache = ref.read(cacheServiceProvider).valueOrNull;
      await clearChatAndMediaCaches(cache);
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.profileClearCacheDone)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _shareProgress() async {
    final l10n = context.l10n;
    final groupId = ref.read(activeGroupIdProvider);
    final student = await ref.read(studentProfileProvider.future);
    if (groupId == null || student == null) {
```

**lib/screens/profile/app_profile_screen.dart — continued (2/2)**

```
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.progressReportNeedGroup)));
      return;
    }
    final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];
    final groupTitle =
        groups.where((g) => g.id == groupId).map((g) => g.title).firstOrNull ??
        'group';
    try {
      await shareProgressReport(
        studentId: student.id,
        groupId: groupId,
        studentName: widget.profile.fullName.isNotEmpty
            ? widget.profile.fullName
            : widget.profile.username,
        groupTitle: groupTitle,
        term: ref.read(selectedMarkTermProvider),
        l10n: l10n,
      );
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.progressReportFailed('$e'))));
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];
    final teacher = ref.watch(teacherProfileProvider).valueOrNull;
    final student = ref.watch(studentProfileProvider).valueOrNull;
    final average = widget.profile.role == UserRole.student
        ? ref.watch(myMarkAverageProvider).valueOrNull
        : null;
    final groupId = ref.watch(activeGroupIdProvider);
    AttendanceStats? attStats;
    if (widget.profile.role == UserRole.student &&
        student != null &&
        groupId != null) {
      final now = DateTime.now();
      final query = AttendanceQuery(
        groupId: groupId,
        studentId: student.id,
        from: DateTime(
```

```
      now.year,
      now.month,
      now.day,
    ).subtract(const Duration(days: 120)),
    to: DateTime(now.year, now.month, now.day),
  );
  attStats = ref.watch(attendanceRangeStatsProvider(query)).valueOrNull;
}

final roleLabel = switch (widget.profile.role) {
  UserRole.student => l10n.commonStudent,
  UserRole.teacher => l10n.commonTeacher,
  UserRole.admin => l10n.commonAdmin,
};

return ListView(
  padding: EdgeInsets.fromLTRB(
    20,
    12,
    20,
    shellContentBottomPadding(context, extra: 24),
  ),
  children: [
    ProfileHeroCard(
      profile: widget.profile,
      roleLabel: roleLabel,
      sphereLabel: teacher == null
          ? null
          : '${l10n.profileSubject}: ${teacher.sphere.l10nLabel(l10n)}',
      uploading: _uploadingAvatar,
      onEditPhoto: _pickAvatar,
      onEditName: _editName,
    ),
    const SizedBox(height: 20),
    if (widget.profile.role != UserRole.admin) ...[
      PremiumSectionHeader(
        title: l10n.profileYourGroups,
        subtitle: groups.isEmpty
            ? l10n.profileNoGroups
            : l10n.profileGroupsCount(groups.length),
        decorAsset: PaperAssets.bookshelfDouble,
      ),
      PremiumCard(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
        child: groups.isEmpty
            ? Text(
                l10n.profileNoGroups,
                style: TextStyle(color: AppColors.inkAlpha(context, 0.5)),
              )
            : Column(
                children: [
                  for (var i = 0; i < groups.length; i++) ...[
                    if (i > 0) const Divider(height: 16),
                    Row(
                      children: [
                        Icon(
                          Icons.groups_outlined,
                          size: 18,
                          color: AppColors.accent(context),
                        ),
                        const SizedBox(width: 12),
                        Expanded(
                          child: Text(
                            groups[i].title,
                            style: const TextStyle(
                              fontWeight: FontWeight.w600,
                            ),
                          ),
                        ),
                        if (groupId == groups[i].id)
                          Icon(
                            Icons.check_circle,
                            size: 16,
```

```
                        color: AppColors.successGreen,
                      ),
                    ],
                  ),
                ],
              ],
            ),
    ),
    if (widget.profile.role == UserRole.student) ...[
      const SizedBox(height: 8),
      Row(
        children: [
          Expanded(
            child: _StatChip(
              label: l10n.profileAverageLabel,
              value: average == null
                  ? '—'
                  : '${average.toStringAsFixed(average % 1 == 0 ? 0 : 1)}%',
            ),
          ),
          const SizedBox(width: 10),
          Expanded(
            child: _StatChip(
              label: l10n.profileAttendanceLabel,
              value: attStats == null || attStats.total == 0
                  ? '—'
                  : '${attStats.presentPercent.toStringAsFixed(0)}%',
            ),
          ),
        ],
      ),
      SizedBox(height: 8),
      PremiumCard(
        onTap: _shareProgress,
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
        child: Row(
          children: [
            Icon(
              Icons.ios_share_rounded,
              color: AppColors.accent(context),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Text(
                l10n.profileShareProgress,
                style: const TextStyle(fontWeight: FontWeight.w600),
              ),
            ),
            Icon(
              Icons.chevron_right,
              color: AppColors.inkAlpha(context, 0.35),
            ),
          ],
        ),
      ),
    ),
  ],
  const SizedBox(height: 12),
],
PremiumSectionHeader(
  title: l10n.profileShortcuts,
  decorAsset: PaperAssets.puzzlePieces,
),
GridView.count(
  crossAxisCount: 2,
  shrinkWrap: true,
  physics: const NeverScrollableScrollPhysics(),
  mainAxisSpacing: 10,
  crossAxisSpacing: 10,
  childAspectRatio: 2.4,
  children: [
    for (final s in _shortcuts)
      PremiumCard(
        onTap: () => context.push(s.path),
```

```
        padding: const EdgeInsets.symmetric(
          horizontal: 14,
          vertical: 12,
        ),
        child: Row(
          children: [
            PaperGlyph(
              icon: s.icon,
              color: AppColors.accent(context),
              size: 22,
            ),
            const SizedBox(width: 10),
            Expanded(
              child: Text(
                s.label,
                maxLines: 2,
                overflow: TextOverflow.ellipsis,
                style: const TextStyle(
                  fontWeight: FontWeight.w700,
                  fontSize: 13,
                ),
              ),
            ),
          ],
        ),
      ),
  ],
),
const SizedBox(height: 20),
PremiumSectionHeader(
  title: l10n.profileSettings,
  decorAsset: PaperAssets.settingsGear,
),
PremiumCard(
  padding: EdgeInsets.zero,
  child: Column(
    children: [
      Padding(
        padding: const EdgeInsets.fromLTRB(16, 16, 16, 8),
        child: LanguageSettingsTile(embedded: true),
      ),
      const Divider(height: 1),
      Padding(
        padding: const EdgeInsets.fromLTRB(16, 12, 16, 16),
        child: ProfileThemeSelector(),
      ),
      const Divider(height: 1),
      ProfileSettingsRow(
        icon: Icons.notifications_outlined,
        label: l10n.profileNotifications,
        onTap: () => context.push('$_roleBasePath/notifications'),
      ),
      const Divider(height: 1),
      ProfileSettingsRow(
        icon: Icons.badge_outlined,
        label: l10n.profileEditName,
        onTap: _editName,
      ),
      const Divider(height: 1),
      ProfileSettingsRow(
        icon: Icons.lock_outline,
        label: l10n.profileChangePassword,
        onTap: _changePassword,
      ),
      const Divider(height: 1),
      ProfileSettingsRow(
        icon: Icons.cleaning_services_outlined,
        label: l10n.profileClearCache,
        onTap: _clearCachedData,
      ),
    ],
  ),
),
```

```
        const SizedBox(height: 24),
        Center(
          child: TextButton.icon(
            onPressed: () => ref.read(authRepositoryProvider).signOut(),
            style: TextButton.styleFrom(
              foregroundColor: AppColors.errorRed.withValues(alpha: 0.9),
            ),
            icon: const Icon(Icons.logout, size: 18),
            label: Text(
              widget.profile.role == UserRole.admin
                  ? l10n.profileSignOutAdmin
                  : l10n.commonSignOut,
              style: const TextStyle(fontWeight: FontWeight.w600),
            ),
          ),
        ),
        const SizedBox(height: 8),
        Center(
          child: Text(
            l10n.profileAppVersion(_version),
            style: TextStyle(
              fontSize: 12,
              color: AppColors.inkAlpha(context, 0.4),
            ),
          ),
        ),
      ],
    );
  }
}

class _Shortcut {
  const _Shortcut(this.icon, this.label, this.path);
  final IconData icon;
  final String label;
  final String path;
}

class _StatChip extends StatelessWidget {
  const _StatChip({required this.label, required this.value});
  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return PremiumCard(
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: AppColors.inkAlpha(context, 0.5),
            ),
          ),
          const SizedBox(height: 4),
          Text(
            value,
            style: TextStyle(
              fontWeight: FontWeight.w800,
              fontSize: 20,
              color: AppColors.ink(context),
            ),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/profile/app_profile_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/profile/widgets/profile_hero_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 201. Import count: 5. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:cached_network_image/cached_network_image.dart';
- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/models/models.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/profile/widgets/profile_hero_card.dart**

```
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/models/models.dart';

class ProfileHeroCard extends StatelessWidget {
  const ProfileHeroCard({
    super.key,
    required this.profile,
    required this.roleLabel,
    required this.onEditPhoto,
    required this.onEditName,
    this.sphereLabel,
    this.uploading = false,
  });

  final Profile profile;
  final String roleLabel;
  final String? sphereLabel;
  final VoidCallback onEditPhoto;
  final VoidCallback onEditName;
  final bool uploading;

  String get _initials {
    final parts = profile.fullName
        .trim()
        .split(RegExp(r'\s+'))
        .where((p) => p.isNotEmpty)
        .toList();
    if (parts.isEmpty) {
      final u = profile.username;
      return u.isEmpty ? '?' : u.substring(0, 1).toUpperCase();
    }
    if (parts.length == 1) {
      return parts.first.substring(0, 1).toUpperCase();
    }
    return (parts[0].substring(0, 1) + parts[1].substring(0, 1)).toUpperCase();
  }

  @override
  Widget build(BuildContext context) {
```

```
final theme = Theme.of(context);
final isDark = theme.brightness == Brightness.dark;

return Container(
  padding: const EdgeInsets.fromLTRB(20, 24, 20, 20),
  decoration: BoxDecoration(
    borderRadius: BorderRadius.circular(24),
    gradient: LinearGradient(
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
      colors: isDark
          ? const [Color(0xFF2A2420), Color(0xFF1C1816)]
          : const [Color(0xFFE8D9C4), Color(0xFFF5EEE4)],
    ),
    border: Border.all(
      color: theme.colorScheme.onSurface.withValues(alpha: 0.06),
    ),
  ),
  child: Column(
    children: [
      Stack(
        alignment: Alignment.bottomRight,
        children: [
          PaperScallopedAvatar(
            avatarSize: 88,
            child: CircleAvatar(
              radius: 44,
              backgroundColor: AppColors.accent(context),
              backgroundImage:
                  profile.avatarUrl != null && profile.avatarUrl!.isNotEmpty
                  ? CachedNetworkImageProvider(profile.avatarUrl!)
                  : null,
              child: uploading
                  ? SizedBox(
                      width: 28,
                      height: 28,
                      child: CircularProgressIndicator(
                        strokeWidth: 2,
                        color:
                            Theme.of(context).brightness == Brightness.dark
                            ? Colors.black
                            : Colors.white,
                      ),
                    )
                  : (profile.avatarUrl == null ||
                        profile.avatarUrl!.isEmpty)
                  ? Text(
                      _initials,
                      style: TextStyle(
                        color:
                            Theme.of(context).brightness == Brightness.dark
                            ? Colors.black
                            : Colors.white,
                        fontWeight: FontWeight.w800,
                        fontSize: 28,
                      ),
                    )
                  : null,
            ),
          ),
          Material(
            color: theme.colorScheme.surface,
            shape: const CircleBorder(),
            child: InkWell(
              customBorder: const CircleBorder(),
              onTap: uploading ? null : onEditPhoto,
              child: Padding(
                padding: EdgeInsets.all(6),
                child: Icon(
                  Icons.camera_alt_outlined,
                  size: 16,
                  color: AppColors.accent(context),
                ),
```

```
                ),
              ),
            ),
          ],
        ),
        const SizedBox(height: 14),
        InkWell(
          onTap: onEditName,
          borderRadius: BorderRadius.circular(8),
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
            child: Row(
              mainAxisSize: MainAxisSize.min,
              children: [
                Flexible(
                  child: Text(
                    profile.fullName.isNotEmpty
                        ? profile.fullName
                        : profile.username,
                    textAlign: TextAlign.center,
                    style: theme.textTheme.titleLarge?.copyWith(
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                ),
                const SizedBox(width: 6),
                Icon(
                  Icons.edit_outlined,
                  size: 16,
                  color: AppColors.inkAlpha(context, 0.4),
                ),
              ],
            ),
          ),
        ),
        const SizedBox(height: 4),
        Text(
          profile.username.startsWith('@')
              ? profile.username
              : '@${profile.username}',
          style: TextStyle(color: AppColors.inkAlpha(context, 0.5)),
        ),
        const SizedBox(height: 12),
        Wrap(
          spacing: 8,
          runSpacing: 8,
          alignment: WrapAlignment.center,
          children: [
            _ProfileChip(label: roleLabel),
            if (sphereLabel != null) _ProfileChip(label: sphereLabel!),
          ],
        ),
      ],
    ),
  );
 }
}

class _ProfileChip extends StatelessWidget {
  const _ProfileChip({required this.label});
  final String label;

  @override
  Widget build(BuildContext context) {
    final accent = AppColors.accent(context);
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      decoration: BoxDecoration(
        color: accent.withValues(alpha: isDark ? 0.22 : 0.12),
        borderRadius: BorderRadius.circular(20),
        border: Border.all(
          color: accent.withValues(alpha: isDark ? 0.55 : 0.2),
```

```
          ),
        ),
        child: Text(
          label,
          style: TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.w700,
            color: AppColors.accent(context),
          ),
        ),
      );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/profile/widgets/profile_hero_card.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/profile/widgets/profile_settings_row.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 33. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/profile/widgets/profile_settings_row.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';

class ProfileSettingsRow extends StatelessWidget {
  const ProfileSettingsRow({
    super.key,
    required this.icon,
    required this.label,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return ListTile(
      leading: PaperGlyph(
        icon: icon,
        size: 24,
        color: AppColors.accent(context),
      ),
      title: Text(label, style: const TextStyle(fontWeight: FontWeight.w600)),
      trailing: Icon(
        Icons.chevron_right,
        color: AppColors.inkAlpha(context, 0.35),
      ),
      onTap: onTap,
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/profile/widgets/profile_settings_row.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/profile/widgets/profile_theme_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 242. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style.dart';
- import 'package:mu_study/core/theme/monochrome_workspace_colors.dart';
- import 'package:mu_study/core/theme/nature_paper_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/settings/theme_provider.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/profile/widgets/profile_theme_selector.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style.dart';
import 'package:mu_study/core/theme/monochrome_workspace_colors.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/settings/theme_provider.dart';

class ProfileThemeSelector extends ConsumerWidget {
  const ProfileThemeSelector({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final l10n = context.l10n;
    final mode = ref.watch(themeModeProvider);
    final style = ref.watch(appStyleProvider);
    final selected = switch (mode) {
      ThemeMode.light => AppThemePreference.light,
      ThemeMode.dark => AppThemePreference.dark,
      ThemeMode.system => AppThemePreference.system,
    };

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          l10n.profileAppearance,
          style: Theme.of(
            context,
          ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
        ),
```

```
const SizedBox(height: 10),
SegmentedButton<AppThemePreference>(
  segments: [
    ButtonSegment(
      value: AppThemePreference.light,
      label: Text(l10n.profileThemeLight),
      icon: const Icon(Icons.light_mode_outlined, size: 16),
    ),
    ButtonSegment(
      value: AppThemePreference.dark,
      label: Text(l10n.profileThemeDark),
      icon: const Icon(Icons.dark_mode_outlined, size: 16),
    ),
    ButtonSegment(
      value: AppThemePreference.system,
      label: Text(l10n.profileThemeSystem),
      icon: const Icon(Icons.brightness_auto_outlined, size: 16),
    ),
  ],
  selected: {selected},
  onSelectionChanged: (set) {
    final next = set.first;
    WidgetsBinding.instance.addPostFrameCallback((_) {
      ref.read(themeModeProvider.notifier).setPreference(next);
    });
  },
),
const SizedBox(height: 16),
Text(
  l10n.profileStyle,
  style: Theme.of(
    context,
  ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
),
const SizedBox(height: 10),
LayoutBuilder(
  builder: (context, constraints) {
    final tileWidth = (constraints.maxWidth - 10) / 2;
    return Wrap(
      spacing: 10,
      runSpacing: 10,
      children: [
        SizedBox(
          width: tileWidth,
          child: _StyleOptionCard(
            label: l10n.profileStyleModern,
            icon: Icons.business_center_outlined,
            previewColors: const [
              AppColors.cream,
              AppColors.warmBeige,
              AppColors.coffeeBrown,
            ],
            isSelected: style == AppStyle.modern,
            onTap: () => ref
                .read(appStyleProvider.notifier)
                .setStyle(AppStyle.modern),
          ),
        ),
        SizedBox(
          width: tileWidth,
          child: _StyleOptionCard(
            label: l10n.profileStyleNature,
            icon: Icons.eco_outlined,
            previewColors: const [
              NaturePaperColors.cream,
              NaturePaperColors.sage,
              NaturePaperColors.forest,
            ],
            isSelected: style == AppStyle.naturePaper,
            onTap: () => ref
                .read(appStyleProvider.notifier)
                .setStyle(AppStyle.naturePaper),
          ),
```

```
                ),
                SizedBox(
                  width: tileWidth,
                  child: _StyleOptionCard(
                    label: l10n.profileStyleSoftNeu,
                    icon: Icons.layers_outlined,
                    previewColors: const [
                      SoftNeumorphismColors.morningBg,
                      SoftNeumorphismColors.sage,
                      SoftNeumorphismColors.amber,
                    ],
                    isSelected: style == AppStyle.softNeumorphism,
                    onTap: () => ref
                        .read(appStyleProvider.notifier)
                        .setStyle(AppStyle.softNeumorphism),
                  ),
                ),
                SizedBox(
                  width: tileWidth,
                  child: _StyleOptionCard(
                    label: l10n.profileStyleMonochrome,
                    icon: Icons.contrast_outlined,
                    previewColors: const [
                      MonochromeWorkspaceColors.lightBg,
                      MonochromeWorkspaceColors.lightBorder,
                      MonochromeWorkspaceColors.lightInk,
                    ],
                    isSelected: style == AppStyle.monochromeWorkspace,
                    onTap: () => ref
                        .read(appStyleProvider.notifier)
                        .setStyle(AppStyle.monochromeWorkspace),
                  ),
                ),
              ],
            );
          },
        ),
      ],
    );
  }
}

class _StyleOptionCard extends StatelessWidget {
  const _StyleOptionCard({
    required this.label,
    required this.icon,
    required this.previewColors,
    required this.isSelected,
    required this.onTap,
  });

  final String label;
  final IconData icon;
  final List<Color> previewColors;
  final bool isSelected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final accent = AppColors.accent(context);
    final isNeu = Neumorphic.isActive(context);
    final radius = BorderRadius.circular(isNeu ? 18 : 16);

    return InkWell(
      onTap: onTap,
      borderRadius: radius,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        curve: Curves.easeOutCubic,
        padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 12),
        decoration: isNeu
            ? Neumorphic.decoration(
                context,
```

```
                elevation: isSelected
                    ? NeuElevation.pressed
                    : NeuElevation.raised,
                borderRadius: radius,
                color: isSelected ? accent.withValues(alpha: 0.12) : null,
              )
            : BoxDecoration(
                color: isSelected
                    ? accent.withValues(alpha: 0.10)
                    : AppColors.softWash(context, lightAlpha: 0.35),
                borderRadius: radius,
                border: Border.all(
                  color: isSelected
                      ? accent
                      : AppColors.inkAlpha(context, 0.12),
                  width: isSelected ? 1.6 : 1,
                ),
              ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                for (final c in previewColors)
                  Padding(
                    padding: const EdgeInsets.only(right: 4),
                    child: Container(
                      width: 14,
                      height: 14,
                      decoration: BoxDecoration(
                        color: c,
                        shape: BoxShape.circle,
                        border: Border.all(
                          color: AppColors.inkAlpha(context, 0.12),
                        ),
                      ),
                    ),
                  ),
                const Spacer(),
                if (isSelected)
                  Icon(Icons.check_circle, size: 16, color: accent)
                else
                  Icon(icon, size: 16, color: AppColors.inkAlpha(context, 0.4)),
              ],
            ),
            const SizedBox(height: 8),
            Text(
              label,
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: TextStyle(
                fontSize: 12,
                fontWeight: FontWeight.w700,
                color: isSelected ? accent : AppColors.inkAlpha(context, 0.75),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/profile/widgets/profile_theme_selector.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/attendance/student_attendance_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 159. Import count: 14. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/attendance/attendance_calendar_grid.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/attendance/student_attendance_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/attendance/attendance_calendar_grid.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';

class StudentAttendanceScreen extends ConsumerStatefulWidget {
  const StudentAttendanceScreen({super.key});

  @override
  ConsumerState<StudentAttendanceScreen> createState() =>
      _StudentAttendanceScreenState();
}

class _StudentAttendanceScreenState
    extends ConsumerState<StudentAttendanceScreen> {
  late DateTime _month;

  @override
```

```
  void initState() {
    super.initState();
    final now = DateTime.now();
    _month = DateTime(now.year, now.month);
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final groupId = ref.watch(activeGroupIdProvider);
    final student = ref.watch(studentProfileProvider);

    if (groupId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.attendanceTitle)),
        body: EmptyState(title: l10n.attendanceNoGroup),
      );
    }

    final studentId = student.valueOrNull?.id;
    if (studentId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.attendanceTitle)),
        body: student.when(
          loading: () => const Center(child: CircularProgressIndicator()),
          error: (e, _) => ErrorState(message: e.toString()),
          data: (_) => EmptyState(title: l10n.attendanceNoGroup),
        ),
      );
    }

    final from = DateTime(_month.year, _month.month, 1);
    final to = DateTime(_month.year, _month.month + 1, 0);
    final query = AttendanceQuery(
      groupId: groupId,
      studentId: studentId,
      from: from,
      to: to,
    );
    final records = ref.watch(attendanceRangeProvider(query));
    final stats = ref.watch(attendanceRangeStatsProvider(query));

    return Scaffold(
      appBar: AppBar(title: Text(l10n.attendanceTitle)),
      body: RefreshIndicator(
        onRefresh: () async {
          ref.invalidate(attendanceRangeProvider(query));
        },
        child: ListView(
          padding: EdgeInsets.fromLTRB(
            16,
            8,
            16,
            shellContentBottomPadding(context, extra: 16),
          ),
          children: [
            const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
            AttendanceMonthHeader(
              month: _month,
              onPrev: () => setState(() {
                _month = DateTime(_month.year, _month.month - 1);
              }),
              onNext: () => setState(() {
                _month = DateTime(_month.year, _month.month + 1);
              }),
            ),
            const SizedBox(height: 8),
            stats.when(
              data: (s) => AttendanceStatsBar(stats: s),
              loading: () => const SizedBox(height: 56),
              error: (_, _) => const SizedBox.shrink(),
            ),
            const SizedBox(height: 16),
```

```
            PremiumCard(
              child: records.when(
                loading: () => const Padding(
                  padding: EdgeInsets.all(32),
                  child: Center(child: CircularProgressIndicator()),
                ),
                error: (e, _) => ErrorState(message: e.toString()),
                data: (list) {
                  final byDate = <DateTime, AttendanceStatus>{
                    for (final r in list)
                      DateTime(r.date.year, r.date.month, r.date.day): r.status,
                  };
                  return Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Row(
                        children: [
                          Expanded(
                            child: Text(
                              l10n.attendanceHistory,
                              style: Theme.of(context).textTheme.titleMedium
                                  ?.copyWith(fontWeight: FontWeight.w800),
                            ),
                          ),
                          PaperDecor(
                            PaperAssets.calendarWithClock,
                            size: 34,
                            angle: 0.08,
                          ),
                        ],
                      ),
                      const SizedBox(height: 12),
                      AttendanceCalendarGrid(
                        month: _month,
                        statusByDate: byDate,
                      ),
                      const SizedBox(height: 16),
                      const AttendanceLegendRow(),
                      if (list.isEmpty) ...[
                        const SizedBox(height: 16),
                        Text(
                          l10n.attendanceCalendarEmpty,
                          style: TextStyle(
                            color: AppColors.inkAlpha(context, 0.5),
                          ),
                        ),
                      ],
                    ],
                  );
                },
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/attendance/student_attendance_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/student/focus/focus_format.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 10. Import count: 0. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/student/focus/focus_format.dart**

```
String formatFocusDuration(Duration d) {
  final total = d.inSeconds.clamp(0, 99 * 3600);
  final h = total ~/ 3600;
  final m = (total % 3600) ~/ 60;
  final s = total % 60;
  if (h > 0) {
    return '$h:${m.toString().padLeft(2, '0')}:${s.toString().padLeft(2, '0')}';
  }
  return '${m.toString().padLeft(2, '0')}:${s.toString().padLeft(2, '0')}';
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/focus_format.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/focus/student_focus_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 160. Import count: 12. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:async';
- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/focus/focus_session_provider.dart';
- import 'package:mu_study/screens/student/focus/focus_format.dart';
- import 'package:mu_study/screens/student/focus/widgets/active_session_view.dart';
- import 'package:mu_study/screens/student/focus/widgets/focus_home_view.dart';
- import 'package:mu_study/screens/student/focus/widgets/focus_summary_view.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/focus/student_focus_screen.dart**

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/focus/focus_session_provider.dart';
import 'package:mu_study/screens/student/focus/focus_format.dart';
import 'package:mu_study/screens/student/focus/widgets/active_session_view.dart';
import 'package:mu_study/screens/student/focus/widgets/focus_home_view.dart';
import 'package:mu_study/screens/student/focus/widgets/focus_summary_view.dart';

class StudentFocusScreen extends ConsumerStatefulWidget {
  const StudentFocusScreen({super.key});

  @override
  ConsumerState<StudentFocusScreen> createState() => _StudentFocusScreenState();
}

class _StudentFocusScreenState extends ConsumerState<StudentFocusScreen>
    with WidgetsBindingObserver {
  Timer? _tick;
  final _noteController = TextEditingController();

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _tick = Timer.periodic(const Duration(seconds: 1), (_) {
```

```
    if (!mounted) return;
    final session = ref.read(focusSessionProvider).session;
    if (session == null || session.isPaused) return;
    setState(() {});
    _finishIfTimedOut(session);
  });
}

@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this);
  _tick?.cancel();
  _noteController.dispose();
  super.dispose();
}

@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.resumed) {
    final session = ref.read(focusSessionProvider).session;
    if (session != null) _finishIfTimedOut(session);
  }
}

void _finishIfTimedOut(FocusSession session) {
  if (!session.isCompleteAt()) return;
  ref.read(focusSessionProvider.notifier).endSession(timedOut: true);
}

Future<void> _pickModeAndStart(Assignment a) async {
  final l10n = context.l10n;
  final mode = await showModalBottomSheet<FocusTimerMode>(
    context: context,
    showDragHandle: true,
    builder: (ctx) {
      return SafeArea(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(20, 8, 20, 24),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                l10n.focusChooseMode,
                style: Theme.of(ctx).textTheme.titleMedium?.copyWith(
                  fontWeight: FontWeight.w800,
                ),
              ),
              const SizedBox(height: 6),
              Text(
                a.title,
                style: TextStyle(
                  color: AppColors.inkAlpha(ctx, 0.6),
                  fontWeight: FontWeight.w600,
                ),
              ),
              const SizedBox(height: 16),
              ListTile(
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(14),
                ),
                leading: Icon(
                  Icons.timer_outlined,
                  color: AppColors.accent(ctx),
                ),
                title: Text(l10n.focusModePomodoro),
                subtitle: Text(l10n.focusModePomodoroHint),
                onTap: () => Navigator.pop(ctx, FocusTimerMode.pomodoro),
              ),
              ListTile(
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(14),
                ),
```

```
                  leading: Icon(
                    Icons.all_inclusive_rounded,
                    color: AppColors.accent(ctx),
                  ),
                  title: Text(l10n.focusModeFree),
                  subtitle: Text(l10n.focusModeFreeHint),
                  onTap: () => Navigator.pop(ctx, FocusTimerMode.free),
                ),
              ],
            ),
          ),
        );
      },
    );
    if (mode == null || !mounted) return;
    await ref
        .read(focusSessionProvider.notifier)
        .start(assignmentId: a.id, assignmentTitle: a.title, mode: mode);
  }

  @override
  Widget build(BuildContext context) {
    final focus = ref.watch(focusSessionProvider);
    if (focus.session != null) {
      return ActiveSessionView(
        session: focus.session!,
        formatDuration: formatFocusDuration,
        onPause: () => ref.read(focusSessionProvider.notifier).pause(),
        onResume: () => ref.read(focusSessionProvider.notifier).resume(),
        onEnd: () => ref.read(focusSessionProvider.notifier).endSession(),
        onOpenTask: () =>
            context.push('/student/assignments/${focus.session!.assignmentId}'),
      );
    }
    if (focus.summary != null) {
      final summary = focus.summary!;
      if (_noteController.text != (summary.note ?? '') &&
          summary.note != null) {
        _noteController.text = summary.note!;
      }
      return FocusSummaryView(
        summary: summary,
        formatDuration: formatFocusDuration,
        noteController: _noteController,
        onNoteChanged: (v) =>
            ref.read(focusSessionProvider.notifier).setSummaryNote(v),
        onOpenTask: () {
          ref.read(focusSessionProvider.notifier).dismissSummary();
          context.push('/student/assignments/${summary.assignmentId}');
        },
        onDone: () => ref.read(focusSessionProvider.notifier).dismissSummary(),
      );
    }
    return FocusHomeView(onStart: _pickModeAndStart);
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/student_focus_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/student/focus/widgets/active_session_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 194. Import count: 6. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/focus/focus_session_provider.dart';
- import 'package:mu_study/screens/student/focus/widgets/focus_ambient_decor.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/focus/widgets/active_session_view.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/focus/focus_session_provider.dart';
import 'package:mu_study/screens/student/focus/widgets/focus_ambient_decor.dart';

class ActiveSessionView extends StatelessWidget {
  const ActiveSessionView({
    super.key,
    required this.session,
    required this.formatDuration,
    required this.onPause,
    required this.onResume,
    required this.onEnd,
    required this.onOpenTask,
  });

  final FocusSession session;
  final String Function(Duration) formatDuration;
  final VoidCallback onPause;
  final VoidCallback onResume;
  final VoidCallback onEnd;
  final VoidCallback onOpenTask;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final nature = isNaturePaper(context);
    final elapsed = session.elapsedAt();
    final remaining = session.remainingAt();
    final display = remaining ?? elapsed;
    final progress = session.mode == FocusTimerMode.pomodoro
        ? (1 -
                  (remaining?.inMilliseconds ?? 0) /
                      kFocusPomodoroDuration.inMilliseconds)
              .clamp(0.0, 1.0)
        : null;

    return Scaffold(
```

```
backgroundColor: nature
    ? Colors.transparent
    : Theme.of(context).scaffoldBackgroundColor,
body: Stack(
  fit: StackFit.expand,
  children: [
    if (nature) const FocusAmbientDecor(),
    SafeArea(
      child: Padding(
        padding: const EdgeInsets.fromLTRB(24, 16, 24, 24),
        child: Column(
          children: [
            Row(
              children: [
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 10,
                    vertical: 5,
                  ),
                  decoration: BoxDecoration(
                    color: AppColors.accent(
                      context,
                    ).withValues(alpha: 0.14),
                    borderRadius: BorderRadius.circular(999),
                  ),
                  child: Text(
                    l10n.focusDoNotDisturb,
                    style: TextStyle(
                      fontSize: 12,
                      fontWeight: FontWeight.w700,
                      color: AppColors.accent(context),
                    ),
                  ),
                ),
                const Spacer(),
                TextButton(
                  onPressed: onEnd,
                  child: Text(l10n.focusEndSession),
                ),
              ],
            ),
            const Spacer(flex: 2),
            Text(
              session.assignmentTitle,
              textAlign: TextAlign.center,
              style: Theme.of(context).textTheme.titleMedium?.copyWith(
                fontWeight: FontWeight.w700,
                color: AppColors.inkAlpha(context, 0.75),
              ),
            ),
            const SizedBox(height: 8),
            Text(
              session.mode == FocusTimerMode.pomodoro
                  ? l10n.focusModePomodoro
                  : l10n.focusModeFree,
              style: TextStyle(
                fontSize: 13,
                fontWeight: FontWeight.w600,
                color: AppColors.inkAlpha(context, 0.45),
              ),
            ),
            const SizedBox(height: 28),
            SizedBox(
              width: 220,
              height: 220,
              child: Stack(
                alignment: Alignment.center,
                children: [
                  if (progress != null)
                    SizedBox.expand(
                      child: CircularProgressIndicator(
                        value: progress,
                        strokeWidth: 6,
```

```
                backgroundColor: AppColors.inkAlpha(
                  context,
                  0.08,
                ),
                color: AppColors.accent(context),
              ),
            )
          else
            Container(
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                border: Border.all(
                  color: AppColors.inkAlpha(context, 0.08),
                  width: 6,
                ),
              ),
            ),
          Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text(
                formatDuration(display),
                style: Theme.of(context).textTheme.displayMedium
                    ?.copyWith(
                      fontWeight: FontWeight.w300,
                      letterSpacing: 1,
                      fontFeatures: const [
                        FontFeature.tabularFigures(),
                      ],
                    ),
              ),
              if (session.isPaused)
                Text(
                  l10n.focusPaused,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    color: AppColors.warning(context),
                  ),
                ),
            ],
          ),
        ],
      ),
    ),
    const Spacer(flex: 2),
    Row(
      children: [
        Expanded(
          child: OutlinedButton.icon(
            onPressed: session.isPaused ? onResume : onPause,
            icon: Icon(
              session.isPaused
                  ? Icons.play_arrow_rounded
                  : Icons.pause_rounded,
            ),
            label: Text(
              session.isPaused
                  ? l10n.focusResume
                  : l10n.focusPause,
            ),
          ),
        ),
        const SizedBox(width: 12),
        Expanded(
          child: FilledButton.icon(
            onPressed: onOpenTask,
            icon: const Icon(Icons.open_in_new_rounded, size: 18),
            label: Text(l10n.focusOpenAssignment),
          ),
        ),
      ],
    ),
  ],
```

```
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/widgets/active_session_view.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/student/focus/widgets/focus_ambient_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 56. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/screens/student/focus/widgets/soft_fireflies.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/focus/widgets/focus_ambient_decor.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/screens/student/focus/widgets/soft_fireflies.dart';

class FocusAmbientDecor extends StatelessWidget {
  const FocusAmbientDecor({super.key});

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.shrink();
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return IgnorePointer(
      child: Stack(
        children: [
          Positioned(
            top: 80,
            left: -20,
            child: PaperDecor(
              isDark ? PaperAssets.grayCloud : PaperAssets.whiteCloud,
              size: 90,
              opacity: isDark ? 0.25 : 0.45,
            ),
          ),
          Positioned(
            top: 140,
            right: 12,
            child: PaperDecor(
              PaperAssets.flatCloud,
              size: 64,
              opacity: isDark ? 0.2 : 0.35,
            ),
          ),
          Positioned(
            bottom: 40,
            left: 8,
            child: PaperDecor(
              PaperAssets.singleTree,
              height: 100,
              opacity: isDark ? 0.35 : 0.5,
            ),
          ),
          Positioned(
            bottom: 28,
            right: 4,
            child: PaperDecor(
```

```
              PaperAssets.twoTrees,
              height: 120,
              opacity: isDark ? 0.4 : 0.55,
            ),
          ),
          if (isDark) const Positioned.fill(child: SoftFireflies()),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/widgets/focus_ambient_decor.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/focus/widgets/focus_home_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 234. Import count: 16. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/buttons/primary_button.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/homework/due_date_label.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/homework/assignment_due.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/homework/assignment_providers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/focus/widgets/focus_home_view.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/buttons/primary_button.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';

class FocusHomeView extends ConsumerWidget {
  const FocusHomeView({super.key, required this.onStart});

  final Future<void> Function(Assignment a) onStart;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
```

```
final l10n = context.l10n;
final groupId = ref.watch(activeGroupIdProvider);
final assignments = ref.watch(assignmentsProvider(groupId));
final nature = isNaturePaper(context);

return Scaffold(
  body: RefreshIndicator(
    onRefresh: () async {
      if (groupId != null) ref.invalidate(assignmentsProvider(groupId));
    },
    child: ListView(
      padding: EdgeInsets.fromLTRB(
        20,
        MediaQuery.paddingOf(context).top + 20,
        20,
        shellContentBottomPadding(context),
      ),
      children: [
        Row(
          children: [
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    l10n.focusTitle,
                    style: Theme.of(context).textTheme.headlineSmall
                        ?.copyWith(
                          fontWeight: FontWeight.w800,
                          letterSpacing: -0.5,
                        ),
                  ),
                  const SizedBox(height: 4),
                  Text(
                    l10n.focusSubtitle,
                    style: TextStyle(
                      color: AppColors.inkAlpha(context, 0.55),
                      fontWeight: FontWeight.w500,
                    ),
                  ),
                ],
              ),
            ),
            if (nature)
              const PaperDecor(
                PaperAssets.whiteCloud,
                size: 48,
                opacity: 0.7,
              ),
          ],
        ),
        const SizedBox(height: 24),
        PremiumSectionHeader(
          title: l10n.focusTodaysFocus,
          subtitle: l10n.focusTodaysFocusSubtitle,
          decorAsset: PaperAssets.yellowPencil,
        ),
        assignments.when(
          loading: () => const Center(
            child: Padding(
              padding: EdgeInsets.all(32),
              child: CircularProgressIndicator(),
            ),
          ),
          error: (e, _) => ErrorState(message: e.toString()),
          data: (page) {
            final focusItems = assignmentsForFocusSession(
              page.items,
            ).take(3).toList();
            if (focusItems.isEmpty) {
              return EmptyState(
                title: l10n.focusEmptyTitle,
                subtitle: l10n.focusEmptySubtitle,
```

```
      icon: Icons.self_improvement_outlined,
    );
  }
  final primary = focusItems.first;
  return Column(
    children: [
      PremiumCard(
        padding: const EdgeInsets.all(18),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                PaperGlyph(
                  icon: Icons.task_alt_rounded,
                  size: 28,
                  color: AppColors.accent(context),
                ),
                const SizedBox(width: 12),
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        primary.title,
                        style: const TextStyle(
                          fontWeight: FontWeight.w800,
                          fontSize: 17,
                        ),
                      ),
                      const SizedBox(height: 4),
                      DueDateLabel(
                        assignment: primary,
                        compact: true,
                      ),
                    ],
                  ),
                ),
              ],
            ),
            const SizedBox(height: 16),
            PrimaryButton(
              label: l10n.focusContinueHomework,
              onPressed: () => onStart(primary),
            ),
            const SizedBox(height: 8),
            TextButton(
              onPressed: () => context.push(
                '/student/assignments/${primary.id}',
              ),
              child: Text(l10n.focusOpenAssignment),
            ),
          ],
        ),
      ),
      if (focusItems.length > 1) ...[
        const SizedBox(height: 16),
        PremiumSectionHeader(
          title: l10n.focusAlsoDue,
          decorAsset: PaperAssets.stickyNotesPair,
        ),
        for (final a in focusItems.skip(1))
          PremiumCard(
            onTap: () => onStart(a),
            padding: const EdgeInsets.symmetric(
              horizontal: 16,
              vertical: 14,
            ),
            child: Row(
              children: [
                PaperGlyph(
                  icon: assignmentIsOverdue(a)
                      ? Icons.warning_amber_rounded
```

```
                                : Icons.assignment_outlined,
                            color: assignmentIsOverdue(a)
                                ? AppColors.danger(context)
                                : AppColors.accent(context),
                            size: 22,
                          ),
                          const SizedBox(width: 12),
                          Expanded(
                            child: Column(
                              crossAxisAlignment: CrossAxisAlignment.start,
                              children: [
                                Text(
                                  a.title,
                                  style: const TextStyle(
                                    fontWeight: FontWeight.w700,
                                  ),
                                ),
                                DueDateLabel(assignment: a, compact: true),
                              ],
                            ),
                          ),
                          Icon(
                            Icons.play_circle_outline_rounded,
                            color: AppColors.inkAlpha(context, 0.35),
                          ),
                        ],
                      ),
                    ),
                  ),
                ],
              ],
            );
          },
        ),
        const SizedBox(height: 20),
        PremiumCard(
          padding: const EdgeInsets.all(16),
          child: Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Icon(
                Icons.chat_bubble_outline_rounded,
                size: 20,
                color: AppColors.inkAlpha(context, 0.45),
              ),
              const SizedBox(width: 12),
              Expanded(
                child: Text(
                  l10n.focusChatMuteHint,
                  style: TextStyle(
                    fontSize: 13,
                    height: 1.4,
                    color: AppColors.inkAlpha(context, 0.6),
                  ),
                ),
              ),
            ],
          ),
        ),
      ],
    ),
  ),
 ),
 );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/widgets/focus_home_view.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?

5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/student/focus/widgets/focus_summary_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 125. Import count: 8. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/components/buttons/primary_button.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/focus/focus_session_provider.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/student/focus/widgets/focus_summary_view.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/buttons/primary_button.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/focus/focus_session_provider.dart';

class FocusSummaryView extends StatelessWidget {
  const FocusSummaryView({
    super.key,
    required this.summary,
    required this.formatDuration,
    required this.noteController,
    required this.onNoteChanged,
    required this.onOpenTask,
    required this.onDone,
  });

  final FocusSummary summary;
  final String Function(Duration) formatDuration;
  final TextEditingController noteController;
  final ValueChanged<String> onNoteChanged;
  final VoidCallback onOpenTask;
  final VoidCallback onDone;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final minutes = summary.focused.inMinutes;
    final seconds = summary.focused.inSeconds % 60;

    return Scaffold(
      body: SafeArea(
        child: ListView(
          padding: EdgeInsets.fromLTRB(
```

```
          20,
          24,
          20,
          shellContentBottomPadding(context),
        ),
        children: [
          if (isNaturePaper(context))
            const Align(
              alignment: Alignment.centerRight,
              child: PaperDecor(
                PaperAssets.beigeLeaf,
                size: 36,
                angle: 0.3,
                opacity: 0.75,
              ),
            ),
          Text(
            summary.timedOut ? l10n.focusTimeUp : l10n.focusSessionComplete,
            style: Theme.of(
              context,
            ).textTheme.headlineSmall?.copyWith(fontWeight: FontWeight.w800),
          ),
          const SizedBox(height: 8),
          Text(
            summary.timedOut
                ? l10n.focusTimeUpSubtitle
                : l10n.focusSessionCompleteSubtitle,
            style: TextStyle(color: AppColors.inkAlpha(context, 0.55)),
          ),
          const SizedBox(height: 20),
          PaperTaped(
            child: PremiumCard(
              padding: const EdgeInsets.all(20),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    summary.assignmentTitle,
                    style: const TextStyle(
                      fontWeight: FontWeight.w800,
                      fontSize: 17,
                    ),
                  ),
                  const SizedBox(height: 12),
                  Text(
                    l10n.focusMinutesFocused(
                      minutes > 0 ? '$minutes min' : '${seconds}s',
                    ),
                    style: TextStyle(
                      fontSize: 28,
                      fontWeight: FontWeight.w300,
                      color: AppColors.accent(context),
                    ),
                  ),
                  const SizedBox(height: 4),
                  Text(
                    formatDuration(summary.focused),
                    style: TextStyle(
                      color: AppColors.inkAlpha(context, 0.45),
                      fontFeatures: const [FontFeature.tabularFigures()],
                    ),
                  ),
                ],
              ),
            ),
          ),
          const SizedBox(height: 20),
          Text(
            l10n.focusOptionalNote,
            style: const TextStyle(fontWeight: FontWeight.w700),
          ),
          const SizedBox(height: 8),
          TextField(
```

```
              controller: noteController,
              onChanged: onNoteChanged,
              maxLines: 3,
              decoration: InputDecoration(hintText: l10n.focusOptionalNoteHint),
            ),
            const SizedBox(height: 24),
            PrimaryButton(label: l10n.focusSaveAndOpen, onPressed: onOpenTask),
            const SizedBox(height: 10),
            TextButton(onPressed: onDone, child: Text(l10n.focusDone)),
          ],
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/widgets/focus_summary_view.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/student/focus/widgets/soft_fireflies.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 67. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math' as math;
- import 'package:flutter/material.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/student/focus/widgets/soft_fireflies.dart**

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

class SoftFireflies extends StatefulWidget {
  const SoftFireflies({super.key});

  @override
  State<SoftFireflies> createState() => _SoftFirefliesState();
}

class _SoftFirefliesState extends State<SoftFireflies>
    with SingleTickerProviderStateMixin {
  late final AnimationController _c = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 12),
  )..repeat();

  @override
  void dispose() {
    _c.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: _c,
      builder: (_, _) => CustomPaint(painter: SoftFirefliesPainter(_c.value)),
    );
  }
}

class SoftFirefliesPainter extends CustomPainter {
  SoftFirefliesPainter(this.t);
  final double t;

  static const _pts = [
    Offset(0.2, 0.35),
    Offset(0.55, 0.28),
    Offset(0.78, 0.42),
    Offset(0.35, 0.55),
  ];

  @override
  void paint(Canvas canvas, Size size) {
    final glow = Paint()
```

```
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 5);
    final core = Paint();
    for (var i = 0; i < _pts.length; i++) {
      final phase = i * 1.4;
      final a = t * 2 * math.pi;
      final pulse = 0.3 + 0.7 * (0.5 + 0.5 * math.sin(a * 2 + phase));
      final c = Offset(
        _pts[i].dx * size.width + math.sin(a + phase) * 8,
        _pts[i].dy * size.height + math.cos(a + phase) * 5,
      );
      glow.color = const Color(0xFFE8C57F).withValues(alpha: 0.35 * pulse);
      core.color = const Color(0xFFF6E3B4).withValues(alpha: 0.85 * pulse);
      canvas.drawCircle(c, 3.2, glow);
      canvas.drawCircle(c, 1.1, core);
    }
  }

  @override
  bool shouldRepaint(covariant SoftFirefliesPainter old) => old.t != t;
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/focus/widgets/soft_fireflies.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/home/student_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 682. Import count: 25. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:math';
- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/homework/due_date_label.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/homework/assignment_due.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/core/utils/time_of_day_greeting.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/components/marks/mark_average_card.dart';
- … and 7 more imports

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/home/student_dashboard_screen.dart**

```
import 'dart:math';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/utils/time_of_day_greeting.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/components/marks/mark_average_card.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
```

```
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/student/home/widgets/dashboard_announcements.dart';
import 'package:skeletonizer/skeletonizer.dart';

class StudentDashboardScreen extends ConsumerStatefulWidget {
  const StudentDashboardScreen({super.key});

  @override
  ConsumerState<StudentDashboardScreen> createState() =>
      _StudentDashboardScreenState();
}

class _StudentDashboardScreenState
    extends ConsumerState<StudentDashboardScreen> {
  late final int _dailyGreetingIndex;

  @override
  void initState() {
    super.initState();
    _dailyGreetingIndex = Random().nextInt(4);
  }

  String _dailyMotivation(AppLocalizations l10n) {
    final lines = [
      l10n.dailyMotivation1,
      l10n.dailyMotivation2,
      l10n.dailyMotivation3,
      l10n.dailyMotivation4,
    ];
    return lines[_dailyGreetingIndex];
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final profile = ref.watch(currentProfileProvider);
    final groupId = ref.watch(activeGroupIdProvider);
    final assignments = ref.watch(assignmentsProvider(groupId));
    final notifications = ref.watch(notificationsProvider);
    final announcements = ref.watch(announcementsProvider(groupId));
    final marks = ref.watch(myMarksProvider);
    final average = ref.watch(myMarkAverageProvider).valueOrNull;

    return RefreshIndicator(
      onRefresh: () async {
        ref.invalidate(myGroupsProvider);
        ref.invalidate(myMarksProvider);
        ref.invalidate(myMarkAverageProvider);
        if (groupId != null) {
          ref.invalidate(assignmentsProvider(groupId));
          ref.invalidate(announcementsProvider(groupId));
        }
      },
      child: ListView(
        padding: EdgeInsets.only(
          left: 20,
          right: 20,
          top: MediaQuery.of(context).padding.top + 24,
          bottom: shellContentBottomPadding(context),
        ),
        children: [
          profile.when(
            data: (p) => Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  crossAxisAlignment: CrossAxisAlignment.center,
                  children: [
                    CircleAvatar(
                      radius: 24,
                      backgroundColor: AppColors.accent(context),
```

```
                      child: Text(
                        (p?.fullName.isNotEmpty == true
                                ? p!.fullName
                                : p?.username ?? 'U')[0]
                            .toUpperCase(),
                        style: TextStyle(
                          color: Theme.of(context).brightness == Brightness.dark
                              ? Colors.black
                              : Colors.white,
                          fontWeight: FontWeight.w800,
                          fontSize: 20,
                        ),
                      ),
                    ),
                    const SizedBox(width: 16),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            '${l10n.greetingWithName(timeOfDayGreeting(l10n), p?.fullName.split(
' ').first ?? p?.username ?? '')} ■',
                            style: Theme.of(context).textTheme.headlineSmall
                                ?.copyWith(
                                  fontWeight: FontWeight.w800,
                                  letterSpacing: -0.5,
                                ),
                          ),
                          const SizedBox(height: 4),
                          Text(
                            _dailyMotivation(l10n),
                            style: TextStyle(
                              color: AppColors.inkAlpha(context, 0.5),
                              fontSize: 14,
                            ),
                          ),
                        ],
                      ),
                    ),
                    PaperDecor(PaperAssets.whiteCloud, size: 44, opacity: 0.8),
                  ],
                ),
              ],
            ),
            loading: () => Skeletonizer(
              enabled: true,
              child: ListTile(
                leading: const CircleAvatar(radius: 24),
                title: Text(l10n.greetingLoadingTitle),
                subtitle: Text(l10n.greetingLoadingSubtitle),
              ),
            ),
            error: (_, _) => const SizedBox.shrink(),
          ),
          const SizedBox(height: 24),

          MarkAverageCard(
            average: average,
            markCount: marks.valueOrNull?.items.length ?? 0,
            onTap: () => context.push('/student/marks'),
          ),
          const SizedBox(height: 16),

          PremiumSectionHeader(
            title: l10n.continueLearning,
            subtitle: l10n.continueLearningSubtitle,
            decorAsset: PaperAssets.yellowPencil,
          ),
          assignments.when(
            data: (page) {
              final list = sortAssignmentsByDue(page.items);
              if (list.isEmpty) {
                return PremiumCard(
```

```
    padding: const EdgeInsets.all(32),
    child: Column(
      children: [
        if (isNaturePaper(context))
          const PaperEmptyIllustration(
            asset: PaperAssets.envelopeSuccess,
          )
        else
          Icon(
            Icons.check_circle_outline_rounded,
            size: 48,
            color: AppColors.accent(
              context,
            ).withValues(alpha: 0.85),
          ),
        const SizedBox(height: 16),
        Text(
          l10n.everythingCompleted,
          style: const TextStyle(
            fontWeight: FontWeight.w800,
            fontSize: 18,
          ),
        ),
        const SizedBox(height: 4),
        Text(
          l10n.caughtUpToday,
          style: TextStyle(
            color: AppColors.inkAlpha(context, 0.6),
          ),
        ),
      ],
    ),
  );
}
final a = list.first;
final isMath = a.title.toLowerCase().contains('math');
final accent = assignmentIsOverdue(a)
    ? AppColors.danger(context)
    : isMath
    ? AppColors.accent(context)
    : AppColors.goldAccent(context);
final heroRadius =
    AppStyleTokens.maybeOf(context)?.cardRadius ?? 16.0;

return PremiumCard(
  onTap: () => context.push('/student/assignments/${a.id}'),
  padding: EdgeInsets.zero,
  elevation: NeuElevation.floating,
  child: Container(
    decoration: BoxDecoration(
      borderRadius: BorderRadius.circular(heroRadius),
      gradient: LinearGradient(
        colors: [accent, accent.withValues(alpha: 0.82)],
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
      ),
    ),
    child: Stack(
      children: [
        Positioned(
          right: -20,
          top: -20,
          child: isNaturePaper(context)
              ? Opacity(
                  opacity: 0.16,
                  child: Transform.rotate(
                    angle: 0.18,
                    child: Image.asset(
                      isMath
                          ? PaperAssets.linedPaperPencil
                          : PaperAssets.openBook,
                      width: 130,
                      filterQuality: FilterQuality.medium,
```

```
              ),
            ),
          )
        : Icon(
            isMath ? Icons.calculate : Icons.menu_book,
            size: 140,
            color: Colors.white.withValues(alpha: 0.1),
          ),
  ),
  Padding(
    padding: const EdgeInsets.all(24),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Wrap(
          spacing: 8,
          runSpacing: 8,
          crossAxisAlignment: WrapCrossAlignment.center,
          children: [
            Container(
              padding: const EdgeInsets.symmetric(
                horizontal: 12,
                vertical: 6,
              ),
              decoration: BoxDecoration(
                color: Colors.white.withValues(alpha: 0.2),
                borderRadius: BorderRadius.circular(20),
              ),
              child: Text(
                a.type == AssignmentType.homeworkPack
                    ? l10n.homeworkPack
                    : l10n.staticTask,
                style: const TextStyle(
                  color: Colors.white,
                  fontSize: 11,
                  fontWeight: FontWeight.w700,
                  letterSpacing: 0.5,
                ),
              ),
            ),
            DueDateLabel(assignment: a, onDark: true),
          ],
        ),
        const SizedBox(height: 16),
        Text(
          a.title,
          style: const TextStyle(
            color: Colors.white,
            fontSize: 22,
            fontWeight: FontWeight.w800,
            letterSpacing: -0.5,
          ),
        ),
        if (a.description.isNotEmpty) ...[
          const SizedBox(height: 8),
          Text(
            a.description,
            maxLines: 2,
            overflow: TextOverflow.ellipsis,
            style: TextStyle(
              color: Colors.white.withValues(alpha: 0.8),
              fontSize: 14,
            ),
          ),
        ],
        const SizedBox(height: 24),
        Consumer(
          builder: (context, ref, _) {
            final progressAsync = ref.watch(
              assignmentProgressProvider(a.id),
            );
            final progress =
                progressAsync.valueOrNull ?? 0.0;
```

```
                                return Row(
                                  children: [
                                    Expanded(
                                      child: Column(
                                        crossAxisAlignment:
                                            CrossAxisAlignment.start,
                                        children: [
                                          Row(
                                            mainAxisAlignment:
                                                MainAxisAlignment.spaceBetween,
                                            children: [
```

**lib/screens/student/home/student_dashboard_screen.dart — continued (2/3)**

```
                                              Text(
                                                l10n.commonProgress,
                                                style: TextStyle(
                                                  color: Colors.white
                                                      .withValues(alpha: 0.8),
                                                  fontSize: 12,
                                                  fontWeight: FontWeight.w600,
                                                ),
                                              ),
                                              Text(
                                                '${(progress * 100).toInt()}%',
                                                style: const TextStyle(
                                                  color: Colors.white,
                                                  fontSize: 12,
                                                  fontWeight: FontWeight.w800,
                                                ),
                                              ),
                                            ],
                                          ),
                                          const SizedBox(height: 8),
                                          ClipRRect(
                                            borderRadius: BorderRadius.circular(
                                              4,
                                            ),
                                            child: TweenAnimationBuilder<double>(
                                              tween: Tween<double>(
                                                begin: 0,
                                                end: progress,
                                              ),
                                              duration: const Duration(
                                                milliseconds: 1000,
                                              ),
                                              curve: Curves.easeOutCubic,
                                              builder: (context, value, _) {
                                                return LinearProgressIndicator(
                                                  value: value,
                                                  backgroundColor:
                                                      Colors.black26,
                                                  valueColor:
                                                      const AlwaysStoppedAnimation(
                                                        Colors.white,
                                                      ),
                                                  minHeight: 6,
                                                );
                                              },
                                            ),
                                          ),
                                        ],
                                      ),
                                    ),
                                    const SizedBox(width: 24),
                                    Container(
                                      padding: const EdgeInsets.all(12),
                                      decoration: const BoxDecoration(
                                        color: Colors.white,
                                        shape: BoxShape.circle,
                                      ),
                                      child: Icon(
                                        Icons.play_arrow_rounded,
```

```
                                color: accent,
                                size: 24,
                              ),
                            ),
                          ],
                        );
                      },
                    ),
                  ],
                ),
              ),
            ],
          ),
        ),
      );
  },
  loading: () => const Skeletonizer(
    enabled: true,
    child: PremiumCard(
      child: SizedBox(height: 200, width: double.infinity),
    ),
  ),
  error: (e, _) => ErrorState(message: e.toString()),
),
const SizedBox(height: 24),

PremiumCard(
  onTap: () => context.go('/student/focus'),
  padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
  child: Row(
    children: [
      PaperGlyph(
        icon: Icons.self_improvement_outlined,
        size: 26,
        color: AppColors.accent(context),
      ),
      const SizedBox(width: 14),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              l10n.focusTitle,
              style: const TextStyle(
                fontWeight: FontWeight.w800,
                fontSize: 15,
              ),
            ),
            const SizedBox(height: 2),
            Text(
              l10n.focusSubtitle,
              style: TextStyle(
                fontSize: 12,
                color: AppColors.inkAlpha(context, 0.5),
              ),
            ),
          ],
        ),
      ),
      Icon(
        Icons.chevron_right_rounded,
        color: AppColors.inkAlpha(context, 0.3),
      ),
    ],
  ),
),
const SizedBox(height: 24),

PremiumSectionHeader(
  title: l10n.homeworkDueThisWeek,
  subtitle: l10n.homeworkDueThisWeekSubtitle,
  decorAsset: PaperAssets.stickyNotesPair,
),
```

```
assignments.when(
  data: (page) {
    final dueFocus = assignmentsDueThisWeekFocus(page.items);
    if (dueFocus.isEmpty) {
      return PremiumCard(
        padding: const EdgeInsets.symmetric(
          horizontal: 20,
          vertical: 18,
        ),
        child: Row(
          children: [
            PaperGlyph(
              icon: Icons.event_available_rounded,
              size: 26,
              color: AppColors.success(
                context,
              ).withValues(alpha: 0.85),
            ),
            const SizedBox(width: 14),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    l10n.homeworkNoDueThisWeek,
                    style: const TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 15,
                    ),
                  ),
                  const SizedBox(height: 2),
                  Text(
                    l10n.homeworkNoDueThisWeekSubtitle,
                    style: TextStyle(
                      fontSize: 12,
                      color: AppColors.inkAlpha(context, 0.5),
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      );
    }
    return Column(
      children: dueFocus.take(5).map((a) {
        final isHomeworkPack = a.type == AssignmentType.homeworkPack;
        final overdue = assignmentIsOverdue(a);
        return PremiumCard(
          onTap: () => context.push('/student/assignments/${a.id}'),
          padding: const EdgeInsets.symmetric(
            horizontal: 16,
            vertical: 16,
          ),
          child: Row(
            children: [
              Builder(
                builder: (context) {
                  final tone = AppColors.resolveAccent(
                    context,
                    overdue
                        ? AppColors.danger(context)
                        : isHomeworkPack
                        ? AppColors.goldAccent(context)
                        : AppColors.accent(context),
                  );
                  final iconData = overdue
                      ? Icons.warning_amber_rounded
                      : isHomeworkPack
                      ? Icons.inventory_2_rounded
                      : Icons.task_alt_rounded;
                  if (Neumorphic.isActive(context)) {
```

```
                    return NeuIconWell(
                      icon: iconData,
                      color: tone,
                      size: 20,
                    );
                  }
                  final isMono =
                      AppStyleTokens.maybeOf(
                        context,
                      )?.isMonochromeWorkspace ??
                      false;
                  return Container(
                    padding: const EdgeInsets.all(12),
                    decoration: BoxDecoration(
                      color: tone.withValues(
                        alpha: isMono ? 0.08 : 0.1,
                      ),
                      borderRadius: BorderRadius.circular(
                        AppStyleTokens.maybeOf(context)?.cardRadius ??
                            12,
                      ),
                      border: isMono
                          ? Border.all(
                              color: tone.withValues(alpha: 0.18),
                            )
                          : null,
                    ),
                    child: PaperGlyph(
                      icon: iconData,
                      color: tone,
                      size: 20,
                    ),
                  );
                },
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      a.title,
                      style: const TextStyle(
                        fontWeight: FontWeight.w700,
                        fontSize: 15,
                      ),
                    ),
                    const SizedBox(height: 4),
                    DueDateLabel(assignment: a, compact: true),
                  ],
                ),
              ),
              Icon(
                Icons.chevron_right_rounded,
                size: 20,
                color: AppColors.inkAlpha(context, 0.3),
              ),
            ],
          ),
        );
      }).toList(),
    );
  },
  loading: () => const SizedBox.shrink(),
  error: (_, _) => const SizedBox.shrink(),
),

const SizedBox(height: 24),
PremiumSectionHeader(
  title: context.l10n.announcementsTitle,
  subtitle: context.l10n.announcementsSubtitle,
  decorAsset: PaperAssets.tanNoteGreenTab,
),
```

```
          announcements.when(
            data: (list) => list.isEmpty
                ? EmptyState(
                    title: context.l10n.announcementsEmpty,
                    icon: Icons.campaign_rounded,
                  )
                : DashboardAnnouncements(announcements: list),
            loading: () => Skeletonizer(
              enabled: true,
              child: PremiumCard(child: Text(l10n.loadingAnnouncements)),
            ),
            error: (_, _) => const SizedBox.shrink(),
          ),

          const SizedBox(height: 24),
          notifications.when(
            data: (list) {
              final unread = list.where((n) => !n.isRead).length;
              return PremiumCard(
                onTap: () => context.go('/student/notifications'),
                child: Row(
                  children: [
                    NeuIconWell(
                      icon: Icons.notifications_none_rounded,
                      color: AppColors.accent(context),
                      size: 20,
                      child: Badge(
                        isLabelVisible: unread > 0,
                        label: Text(
                          '$unread',
                          style: const TextStyle(fontSize: 10),
                        ),
                        backgroundColor: AppColors.danger(context),
                        child: PaperGlyph(
                          icon: Icons.notifications_none_rounded,
                          color: AppColors.accent(context),
                          size: 20,
                        ),
```

**lib/screens/student/home/student_dashboard_screen.dart — continued (3/3)**

```
                      ),
                    ),
                    const SizedBox(width: 16),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            l10n.notificationsCardTitle,
                            style: const TextStyle(
                              fontWeight: FontWeight.w700,
                              fontSize: 15,
                            ),
                          ),
                          Text(
                            unread > 0
                                ? l10n.unreadAlerts(unread)
                                : l10n.upToDate,
                            style: TextStyle(
                              color: AppColors.inkAlpha(context, 0.5),
                              fontSize: 13,
                            ),
                          ),
                        ],
                      ),
                    ),
                    Icon(
                      Icons.chevron_right_rounded,
                      size: 20,
                      color: AppColors.inkAlpha(context, 0.3),
                    ),
                  ],
```

```
                ),
              );
            },
            loading: () => const SizedBox.shrink(),
            error: (_, _) => const SizedBox.shrink(),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/home/student_dashboard_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/home/widgets/dashboard_announcements.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 68. Import count: 4. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/screens/student/home/widgets/expandable_announcement_card.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/home/widgets/dashboard_announcements.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/screens/student/home/widgets/expandable_announcement_card.dart';

class DashboardAnnouncements extends StatefulWidget {
  const DashboardAnnouncements({super.key, required this.announcements});

  final List<Announcement> announcements;

  static const _previewCount = 3;

  @override
  State<DashboardAnnouncements> createState() => _DashboardAnnouncementsState();
}

class _DashboardAnnouncementsState extends State<DashboardAnnouncements> {
  bool _showAll = false;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final sorted = [...widget.announcements]
      ..sort((a, b) {
        final da = a.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0);
        final db = b.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0);
        return db.compareTo(da);
      });
    final canExpand = sorted.length > DashboardAnnouncements._previewCount;
    final visible = _showAll || !canExpand
        ? sorted
        : sorted.take(DashboardAnnouncements._previewCount).toList();

    return Column(
      children: [
        AnimatedSize(
          duration: const Duration(milliseconds: 280),
          curve: Curves.easeOutCubic,
          alignment: Alignment.topCenter,
          child: Column(
            children: [
              for (final a in visible)
                ExpandableAnnouncementCard(announcement: a),
            ],
```

```
          ),
        ),
        if (canExpand)
          Padding(
            padding: const EdgeInsets.only(top: 4),
            child: TextButton.icon(
              onPressed: () => setState(() => _showAll = !_showAll),
              icon: Icon(
                _showAll
                    ? Icons.expand_less_rounded
                    : Icons.expand_more_rounded,
                size: 20,
              ),
              label: Text(
                _showAll
                    ? l10n.announcementsShowLess
                    : l10n.announcementsViewAll(sorted.length),
              ),
            ),
          ),
      ],
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/home/widgets/dashboard_announcements.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/home/widgets/expandable_announcement_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 153. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:intl/intl.dart' hide TextDirection;
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/home/widgets/expandable_announcement_card.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart' hide TextDirection;
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';

class ExpandableAnnouncementCard extends StatefulWidget {
  const ExpandableAnnouncementCard({super.key, required this.announcement});

  final Announcement announcement;

  @override
  State<ExpandableAnnouncementCard> createState() =>
      _ExpandableAnnouncementCardState();
}

class _ExpandableAnnouncementCardState
    extends State<ExpandableAnnouncementCard> {
  static const _collapsedLines = 3;

  bool _expanded = false;

  TextStyle get _bodyStyle => TextStyle(
    fontSize: 14,
    color: AppColors.inkAlpha(context, 0.6),
    height: 1.4,
  );

  bool _exceedsCollapsedLines(double maxWidth, TextDirection direction) {
    final text = widget.announcement.description.trim();
    if (text.isEmpty) return false;
    final painter = TextPainter(
      text: TextSpan(text: text, style: _bodyStyle),
      maxLines: _collapsedLines,
      textDirection: direction,
    )..layout(maxWidth: maxWidth);
    return painter.didExceedMaxLines;
```

```
}

@override
Widget build(BuildContext context) {
  final a = widget.announcement;
  final description = a.description.trim();

  return PaperTaped(
    alignment: -0.55,
    child: PremiumCard(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              CircleAvatar(
                radius: 12,
                backgroundColor: AppColors.softSurface(context),
                child: Icon(
                  Icons.person,
                  size: 12,
                  color: AppColors.accent(context),
                ),
              ),
              const SizedBox(width: 8),
              Text(
                context.l10n.commonTeacher,
                style: TextStyle(
                  fontSize: 12,
                  fontWeight: FontWeight.w700,
                  color: AppColors.inkAlpha(context, 0.7),
                ),
              ),
              const Spacer(),
              Text(
                DateFormat.MMMd().format(a.createdAt ?? DateTime.now()),
                style: TextStyle(
                  fontSize: 11,
                  color: AppColors.inkAlpha(context, 0.4),
                ),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Text(
            a.title,
            style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 15),
          ),
          if (description.isNotEmpty) ...[
            const SizedBox(height: 4),
            LayoutBuilder(
              builder: (context, constraints) {
                final canExpand = _exceedsCollapsedLines(
                  constraints.maxWidth,
                  Directionality.of(context),
                );
                return Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    AnimatedSize(
                      duration: const Duration(milliseconds: 220),
                      curve: Curves.easeInOut,
                      alignment: Alignment.topCenter,
                      child: Text(
                        description,
                        maxLines: _expanded ? null : _collapsedLines,
                        overflow: _expanded
                            ? TextOverflow.visible
                            : TextOverflow.ellipsis,
                        style: _bodyStyle,
                      ),
                    ),
```

```
                    if (canExpand) ...[
                      SizedBox(height: 6),
                      GestureDetector(
                        onTap: () => setState(() => _expanded = !_expanded),
                        behavior: HitTestBehavior.opaque,
                        child: Row(
                          mainAxisSize: MainAxisSize.min,
                          children: [
                            Text(
                              _expanded
                                  ? context.l10n.announcementsShowLess
                                  : context.l10n.announcementsReadMore,
                              style: TextStyle(
                                fontSize: 13,
                                fontWeight: FontWeight.w700,
                                color: AppColors.accent(context),
                              ),
                            ),
                            SizedBox(width: 2),
                            Icon(
                              _expanded
                                  ? Icons.expand_less_rounded
                                  : Icons.expand_more_rounded,
                              size: 18,
                              color: AppColors.accent(context),
                            ),
                          ],
                        ),
                      ),
                    ],
                  ],
                );
              },
            ),
          ],
        ],
      ),
    ),
  );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/home/widgets/expandable_announcement_card.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/homework/student_assignments_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 371. Import count: 17. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/homework/due_date_label.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/inputs/load_more_button.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/homework/assignment_due.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/homework/assignment_providers.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/student/homework/student_assignments_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';

class StudentAssignmentsScreen extends ConsumerWidget {
  const StudentAssignmentsScreen({super.key});

  void _open(BuildContext context, Assignment a) {
    context.push('/student/assignments/${a.id}');
```

```
}

@override
Widget build(BuildContext context, WidgetRef ref) {
  final groupId = ref.watch(activeGroupIdProvider);
  final assignments = ref.watch(assignmentsProvider(groupId));

  final l10n = context.l10n;
  return Scaffold(
    extendBody: true,
    appBar: AppBar(title: Text(l10n.assignmentsTitle)),
    body: assignments.when(
      data: (page) {
        final list = sortAssignmentsByDue(page.items);
        if (list.isEmpty) {
          return ListView(
            padding: EdgeInsets.only(
              left: 20,
              right: 20,
              top: 12,
              bottom: shellContentBottomPadding(context),
            ),
            children: [
              const GroupSelector(padding: EdgeInsets.only(bottom: 12)),
              EmptyState(
                title: l10n.assignmentsEmpty,
                subtitle: l10n.assignmentsEmptySubtitle,
                icon: Icons.task_alt_rounded,
              ),
            ],
          );
        }
        final isNeu = Neumorphic.isActive(context);
        return ListView.builder(
          clipBehavior: isNeu ? Clip.none : Clip.hardEdge,
          padding: EdgeInsets.only(
            left: 20,
            right: 20,
            top: 12,
            bottom: shellContentBottomPadding(context),
          ),
          itemCount: list.length + 1 + (page.hasMore ? 1 : 0),
          itemBuilder: (_, i) {
            if (i == 0) {
              return Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
                  PremiumSectionHeader(
                    title: l10n.assignmentsLearningPath,
                    subtitle: l10n.assignmentsActive,
                  ),
                ],
              );
            }
            if (i == list.length + 1) {
              return LoadMoreButton(
                hasMore: page.hasMore,
                loading: page.loadingMore,
                onPressed: () => ref
                    .read(assignmentsProvider(groupId).notifier)
                    .loadMore(),
              );
            }
            final a = list[i - 1];
            final isPack = a.type == AssignmentType.homeworkPack;
            final isMath = a.title.toLowerCase().contains('math');
            final overdue = assignmentIsOverdue(a);
            final accent = AppColors.resolveAccent(
              context,
              overdue
                  ? AppColors.danger(context)
                  : (isMath ? AppColors.mathAccent : AppColors.englishAccent),
```

```
);
final iconData = overdue
    ? Icons.warning_amber_rounded
    : isMath
    ? Icons.calculate_rounded
    : Icons.menu_book_rounded;

return PremiumCard(
  onTap: () => _open(context, a),
  padding: const EdgeInsets.symmetric(
    horizontal: 16,
    vertical: 16,
  ),
  child: Row(
    children: [
      NeuIconWell(icon: iconData, color: accent, size: 24),
      const SizedBox(width: 16),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Expanded(
                  child: Text(
                    a.title,
                    style: const TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 16,
                    ),
                  ),
                ),
                if (overdue) ...[
                  const SizedBox(width: 8),
                  Text(
                    l10n.homeworkLateBadge,
                    style: TextStyle(
                      color: AppColors.danger(context),
                      fontSize: 11,
                      fontWeight: FontWeight.w800,
                    ),
                  ),
                ],
              ],
            ),
            const SizedBox(height: 6),
            DueDateLabel(assignment: a, compact: true),
            if (a.description.isNotEmpty &&
                a.dueDate == null) ...[
              const SizedBox(height: 2),
              Text(
                a.description,
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: TextStyle(
                  color: AppColors.inkAlpha(context, 0.5),
                  fontSize: 13,
                ),
              ),
            ],
            if (isPack) ...[
              const SizedBox(height: 2),
              Text(
                l10n.homeworkPack,
                style: TextStyle(
                  color: AppColors.inkAlpha(context, 0.4),
                  fontSize: 11,
                  fontWeight: FontWeight.w600,
                ),
              ),
            ],
          ],
        ),
      ),
```

```
                    ),
                    Icon(
                      Icons.chevron_right_rounded,
                      color: AppColors.inkAlpha(context, 0.3),
                      size: 20,
                    ),
                  ],
                ),
              );
            },
          );
        },
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
      ),
    );
  }
}

class StudentAssignmentDetailScreen extends ConsumerWidget {
  const StudentAssignmentDetailScreen({super.key, required this.assignmentId});

  final String assignmentId;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final assignment = ref.watch(assignmentDetailProvider(assignmentId));
    final tasks = ref.watch(tasksProvider(assignmentId));

    final l10n = context.l10n;
    return Scaffold(
      appBar: AppBar(title: Text(l10n.assignmentDetails)),
      body: assignment.when(
        data: (a) {
          if (a == null) {
            return EmptyState(title: l10n.assignmentNotFound);
          }
          return RefreshIndicator(
            onRefresh: () async {
              ref.invalidate(assignmentDetailProvider(assignmentId));
              ref.invalidate(tasksProvider(assignmentId));
            },
            child: ListView(
              padding: EdgeInsets.fromLTRB(
                20,
                24,
                20,
                shellContentBottomPadding(context, extra: 24),
              ),
              children: [
                Container(
                  padding: const EdgeInsets.all(24),
                  decoration: BoxDecoration(
                    color: AppColors.accent(context),
                    borderRadius: BorderRadius.circular(20),
                    boxShadow: [
                      BoxShadow(
                        color: AppColors.accent(context).withValues(alpha: 0.2),
                        blurRadius: 20,
                        offset: const Offset(0, 10),
                      ),
                    ],
                  ),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        (a.type == AssignmentType.homeworkPack
                                ? l10n.homeworkPack
                                : l10n.staticTask)
                            .toUpperCase(),
                        style: const TextStyle(
                          color: Colors.white70,
```

```
            fontSize: 10,
            fontWeight: FontWeight.w800,
            letterSpacing: 1.2,
          ),
        ),
        const SizedBox(height: 8),
        Text(
          a.title,
          style: Theme.of(context).textTheme.headlineSmall
              ?.copyWith(
                color: Colors.white,
                fontWeight: FontWeight.w800,
              ),
        ),
        const SizedBox(height: 12),
        DueDateLabel(assignment: a, onDark: true),
        if (a.description.isNotEmpty) ...[
          const SizedBox(height: 12),
          Text(
            a.description,
            style: TextStyle(
              color: Colors.white.withValues(alpha: 0.8),
              fontSize: 14,
              height: 1.5,
            ),
          ),
        ],
      ],
    ),
  ),
),
const SizedBox(height: 32),
PremiumSectionHeader(
  title: l10n.assignmentTasks,
  subtitle: l10n.assignmentTasksSubtitle,
),
tasks.when(
  data: (items) {
    if (items.isEmpty) {
      return EmptyState(
        title: l10n.assignmentsEmpty,
        icon: Icons.task_alt_rounded,
      );
    }
    return ListView.builder(
      shrinkWrap: true,
      physics: const NeverScrollableScrollPhysics(),
      itemCount: items.length,
      itemBuilder: (context, index) {
        final t = items[index];
        return PremiumCard(
          onTap: () => context.push('/student/tasks/${t.id}'),
          padding: const EdgeInsets.all(16),
          child: Row(
            children: [
              Container(
                width: 32,
                height: 32,
                decoration: BoxDecoration(
                  color: AppColors.softWash(
                    context,
                    lightAlpha: 0.3,
                  ),
                  shape: BoxShape.circle,
                ),
                child: Center(
                  child: Text(
                    '${index + 1}',
                    style: TextStyle(
                      fontWeight: FontWeight.w800,
                      color: AppColors.accent(context),
                      fontSize: 12,
                    ),
                  ),
```

```
                                ),
                              ),
                              const SizedBox(width: 16),
                              Expanded(
                                child: Column(
                                  crossAxisAlignment: CrossAxisAlignment.start,
                                  children: [
                                    Text(
                                      t.title,
                                      style: const TextStyle(
                                        fontWeight: FontWeight.w700,
                                        fontSize: 16,
                                      ),
                                    ),
                                    if (t.description.isNotEmpty)
                                      Text(
                                        t.description,
                                        maxLines: 1,
                                        overflow: TextOverflow.ellipsis,
                                        style: TextStyle(
```

**lib/screens/student/homework/student_assignments_screen.dart — continued (2/2)**

```
                                          color: AppColors.inkAlpha(
                                            context,
                                            0.5,
                                          ),
                                          fontSize: 13,
                                        ),
                                      ),
                                  ],
                                ),
                              ),
                              const Icon(
                                Icons.arrow_forward_rounded,
                                size: 18,
                                color: Colors.grey,
                              ),
                            ],
                          ),
                        );
                      },
                    );
                  },
                  loading: () =>
                      const Center(child: CircularProgressIndicator()),
                  error: (e, _) => ErrorState(message: e.toString()),
                ),
                const SizedBox(height: 100),
              ],
            ),
          );
        },
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/homework/student_assignments_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/homework/student_task_submit_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 506. Import count: 18. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:typed_data';
- import 'package:file_picker/file_picker.dart';
- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:image_picker/image_picker.dart';
- import 'package:mu_study/components/attachments/attachment_preview.dart';
- import 'package:mu_study/components/attachments/voice_note_field.dart';
- import 'package:mu_study/components/buttons/primary_button.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/dialogs/pdf_viewer_sheet.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/homework/assignment_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:supabase_flutter/supabase_flutter.dart';
- import 'package:url_launcher/url_launcher.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/student/homework/student_task_submit_screen.dart**

```
import 'dart:typed_data';

import 'package:file_picker/file_picker.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:image_picker/image_picker.dart';
import 'package:mu_study/components/attachments/attachment_preview.dart';
import 'package:mu_study/components/attachments/voice_note_field.dart';
import 'package:mu_study/components/buttons/primary_button.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/dialogs/pdf_viewer_sheet.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:url_launcher/url_launcher.dart';

class StudentTaskSubmitScreen extends ConsumerStatefulWidget {
```

```
  const StudentTaskSubmitScreen({super.key, required this.taskId});

  final String taskId;

  @override
  ConsumerState<StudentTaskSubmitScreen> createState() =>
      _StudentTaskSubmitScreenState();
}

const _signedUrlTtlSeconds = 60 * 60 * 24 * 365 * 10;

class _StudentTaskSubmitScreenState
    extends ConsumerState<StudentTaskSubmitScreen> {
  final _textController = TextEditingController();
  bool _loading = false;
  bool _completed = false;
  String? _fileUrl;
  String? _fileName;
  String? _storagePath;
  Uint8List? _localBytes;
  String? _voiceUrl;
  String? _voiceStoragePath;
  Map<String, dynamic>? _task;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    final task = await supabase
        .from('tasks')
        .select()
        .eq('id', widget.taskId)
        .maybeSingle();
    if (task != null) setState(() => _task = Map<String, dynamic>.from(task));

    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;
    final sub = await supabase
        .from('submissions')
        .select()
        .eq('task_id', widget.taskId)
        .eq('student_id', student.id)
        .maybeSingle();
    if (sub != null && mounted) {
      final url = sub['file_url'] as String?;
      final voice = sub['voice_url'] as String?;
      setState(() {
        _textController.text = sub['text_answer'] as String? ?? '';
        _fileUrl = url;
        _fileName = url == null ? null : attachmentFileName(url);
        _storagePath = url == null ? null : storagePathFromSignedUrl(url);
        _localBytes = null;
        _voiceUrl = voice;
        _voiceStoragePath = voice == null
            ? null
            : storagePathFromSignedUrl(voice);
        _completed = sub['is_completed'] as bool? ?? false;
      });
    }
  }

  Future<void> _storeAttachment({
    required String studentId,
    required String fileName,
    required Uint8List bytes,
  }) async {
    final path = '$studentId/${widget.taskId}/$fileName';
    await supabase.storage
        .from('submissions')
        .uploadBinary(
```

```
          path,
          bytes,
          fileOptions: const FileOptions(upsert: true),
        );
    final url = await supabase.storage
        .from('submissions')
        .createSignedUrl(path, _signedUrlTtlSeconds);
    if (!mounted) return;
    setState(() {
      _fileUrl = url;
      _fileName = fileName;
      _storagePath = path;
      _localBytes = bytes;
    });
  }

  Future<void> _uploadFile() async {
    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;

    final result = await FilePicker.platform.pickFiles(withData: true);
    if (result == null || result.files.isEmpty) return;
    final file = result.files.first;
    if (file.bytes == null) return;

    setState(() => _loading = true);
    try {
      await _storeAttachment(
        studentId: student.id,
        fileName: file.name,
        bytes: file.bytes!,
      );
    } catch (e) {
      if (!mounted) return;
      _showError(context.l10n.taskUploadFailed('$e'));
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _pickImage() async {
    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;
    final picker = ImagePicker();
    final image = await picker.pickImage(
      source: ImageSource.gallery,
      imageQuality: 88,
    );
    if (image == null) return;
    final bytes = await image.readAsBytes();
    setState(() => _loading = true);
    try {
      await _storeAttachment(
        studentId: student.id,
        fileName: image.name,
        bytes: bytes,
      );
    } catch (e) {
      if (!mounted) return;
      _showError(context.l10n.taskUploadFailed('$e'));
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _removeAttachment() async {
    final path = _storagePath;
    setState(() {
      _fileUrl = null;
      _fileName = null;
      _storagePath = null;
      _localBytes = null;
    });
```

```
    if (path != null) {
      try {
        await supabase.storage.from('submissions').remove([path]);
      } catch (_) {}
    }

    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;
    try {
      await supabase
          .from('submissions')
          .update({'file_url': null})
          .eq('task_id', widget.taskId)
          .eq('student_id', student.id);
    } catch (_) {}
  }

  Future<void> _uploadVoice(Uint8List bytes, String fileName) async {
    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;

    setState(() => _loading = true);
    try {
      final path = '${student.id}/${widget.taskId}/voice/$fileName';
      await supabase.storage
          .from('submissions')
          .uploadBinary(
            path,
            bytes,
            fileOptions: FileOptions(
              upsert: true,
              contentType: fileName.endsWith('.wav')
                  ? 'audio/wav'
                  : 'audio/mp4',
            ),
          );
      final url = await supabase.storage
          .from('submissions')
          .createSignedUrl(path, _signedUrlTtlSeconds);
      if (!mounted) return;
      setState(() {
        _voiceUrl = url;
        _voiceStoragePath = path;
      });
    } catch (e) {
      if (mounted) _showError(context.l10n.taskVoiceUploadFailed('$e'));
      rethrow;
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _removeVoice() async {
    final path = _voiceStoragePath;
    setState(() {
      _voiceUrl = null;
      _voiceStoragePath = null;
    });

    if (path != null) {
      try {
        await supabase.storage.from('submissions').remove([path]);
      } catch (_) {}
    }

    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;
    try {
      await supabase
          .from('submissions')
          .update({'voice_url': null})
          .eq('task_id', widget.taskId)
          .eq('student_id', student.id);
```

```
    } catch (_) {}
  }

  void _showError(String message) {
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(message)));
  }

  Future<void> _save({required bool submit}) async {
    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;
    setState(() => _loading = true);
    try {
      await supabase.from('submissions').upsert({
        'task_id': widget.taskId,
        'student_id': student.id,
        'text_answer': _textController.text.trim(),
        'file_url': _fileUrl,
        'voice_url': _voiceUrl,
        'is_completed': _completed,
        if (submit) 'status': 'submitted',
        if (submit) 'submitted_at': DateTime.now().toIso8601String(),
      }, onConflict: 'task_id,student_id');
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(
              submit ? context.l10n.taskSubmitted : context.l10n.taskSaved,
            ),
          ),
        );

        ref.invalidate(assignmentProgressProvider);
      }
    } catch (e) {
      if (!mounted) return;
      _showError(context.l10n.taskCouldNotSave('$e'));
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  void dispose() {
    _textController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final task = _task;
    final l10n = context.l10n;
    return Scaffold(
      appBar: AppBar(title: Text(task?['title'] as String? ?? l10n.taskTitle)),
      body: task == null
          ? const Center(child: CircularProgressIndicator())
          : ListView(
              padding: EdgeInsets.fromLTRB(
                16,
                16,
                16,
                shellContentBottomPadding(context, extra: 16),
              ),
              children: [
                PremiumCard(child: Text(task['description'] as String? ?? '')),
                if (task['attachment_url'] != null) ...[
                  const SizedBox(height: 8),
                  OutlinedButton.icon(
                    onPressed: () => openPdfViewer(
                      context,
                      task['attachment_url'] as String,
```

```
      title: task['title'] as String?,
    ),
    icon: const Icon(Icons.picture_as_pdf_outlined),
    label: Text(l10n.taskViewPdf),
  ),
],
if (task['link_url'] != null) ...[
  const SizedBox(height: 8),
  TextButton(
    onPressed: () =>
        launchUrl(Uri.parse(task['link_url'] as String)),
    child: Text(l10n.taskOpenLink),
  ),
],
const SizedBox(height: 12),
SwitchListTile(
  title: Text(l10n.taskMarkCompleted),
  value: _completed,
  onChanged: (v) => setState(() => _completed = v),
),
TextField(
  controller: _textController,
  maxLines: 5,
  decoration: InputDecoration(labelText: l10n.taskYourAnswer),
),
const SizedBox(height: 16),
Text(
  l10n.taskAttachment,
  style: Theme.of(
    context,
  ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w700),
),
const SizedBox(height: 4),
Text(
  _fileUrl == null
      ? l10n.taskAttachmentHint
      : l10n.taskAttachmentHintReady,
  style: TextStyle(
    fontSize: 12,
    color: AppColors.inkAlpha(context, 0.5),
  ),
),
const SizedBox(height: 12),
if (_fileUrl != null)
  AttachmentPreview(
    url: _fileUrl!,
    fileName: _fileName,
    bytes: _localBytes,
    enabled: !_loading,
    onRemove: _loading ? null : _removeAttachment,
  )
else
  Row(
    children: [
      Expanded(
        child: _AttachAction(
          icon: Icons.image_outlined,
          label: l10n.taskImage,
          subtitle: l10n.taskImageSubtitle,
          onPressed: _loading ? null : _pickImage,
        ),
      ),
      const SizedBox(width: 10),
      Expanded(
        child: _AttachAction(
          icon: Icons.upload_file_outlined,
          label: l10n.taskFile,
          subtitle: l10n.taskFileSubtitle,
          onPressed: _loading ? null : _uploadFile,
        ),
      ),
    ],
  ),
```

```
                if (_fileUrl != null) ...[
                  const SizedBox(height: 10),
                  Row(
                    children: [
                      TextButton.icon(
                        onPressed: _loading ? null : _pickImage,
                        icon: const Icon(Icons.image_outlined, size: 18),
                        label: Text(l10n.taskReplaceImage),
                      ),
                      TextButton.icon(
                        onPressed: _loading ? null : _uploadFile,
                        icon: const Icon(Icons.upload_file_outlined, size: 18),
                        label: Text(l10n.taskReplaceFile),
                      ),
                    ],
                  ),
                ],
                const SizedBox(height: 20),
                Text(
                  l10n.taskVoice,
                  style: Theme.of(
                    context,
                  ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w700),
                ),
                const SizedBox(height: 4),
                Text(
                  l10n.taskVoiceHint,
                  style: TextStyle(
                    fontSize: 12,
                    color: AppColors.inkAlpha(context, 0.5),
                  ),
                ),
                const SizedBox(height: 12),
                VoiceNoteField(
                  voiceUrl: _voiceUrl,
                  enabled: !_loading,
                  onUpload: _uploadVoice,
                  onRemove: _loading ? null : _removeVoice,
                ),
```

**lib/screens/student/homework/student_task_submit_screen.dart — continued (2/2)**

```
                const SizedBox(height: 20),
                PrimaryButton(
                  label: l10n.commonSaveDraft,
                  loading: _loading,
                  onPressed: () => _save(submit: false),
                ),
                const SizedBox(height: 8),
                PrimaryButton(
                  label: l10n.commonSubmit,
                  loading: _loading,
                  onPressed: () => _save(submit: true),
                ),
              ],
            ),
      );
    }
  }

  class _AttachAction extends StatelessWidget {
    const _AttachAction({
      required this.icon,
      required this.label,
      required this.subtitle,
      required this.onPressed,
    });

    final IconData icon;
    final String label;
    final String subtitle;
    final VoidCallback? onPressed;

    @override
```

```
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onPressed,
        borderRadius: BorderRadius.circular(16),
        child: Ink(
          padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 16),
          decoration: BoxDecoration(
            color: AppColors.controlSurface(context),
            borderRadius: BorderRadius.circular(16),
            border: Border.all(color: AppColors.inkAlpha(context, 0.08)),
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Container(
                width: 40,
                height: 40,
                decoration: BoxDecoration(
                  color: AppColors.accent(context).withValues(alpha: 0.1),
                  borderRadius: BorderRadius.circular(12),
                ),
                child: Icon(icon, color: AppColors.accent(context), size: 20),
              ),
              const SizedBox(height: 12),
              Text(
                label,
                style: const TextStyle(
                  fontWeight: FontWeight.w700,
                  fontSize: 14,
                ),
              ),
              const SizedBox(height: 2),
              Text(
                subtitle,
                style: TextStyle(
                  fontSize: 11,
                  color: AppColors.inkAlpha(context, 0.45),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/homework/student_task_submit_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/student/marks/student_marks_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 287. Import count: 21. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/inputs/load_more_button.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/components/marks/mark_average_card.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/components/marks/term_selector.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/progress/progress_report_share.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- … and 3 more imports

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/student/marks/student_marks_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/mark_average_card.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/components/marks/term_selector.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/progress/progress_report_share.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
```

```
import 'package:mu_study/providers/marks/marks_providers.dart';

class StudentMarksScreen extends ConsumerWidget {
  const StudentMarksScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final marks = ref.watch(myMarksProvider);
    final stats = ref.watch(myMarkStatsProvider);
    final average = ref.watch(myMarkAverageProvider).valueOrNull;
    final markCount = stats.valueOrNull?.length ?? 0;
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.marksTitle),
        actions: [
          IconButton(
            tooltip: l10n.progressReportShare,
            onPressed: () => _shareProgress(context, ref),
            icon: const Icon(Icons.ios_share_rounded),
          ),
        ],
      ),
      body: RefreshIndicator(
        onRefresh: () async {
          ref.invalidate(myMarksProvider);
          ref.invalidate(myMarkStatsProvider);
          ref.invalidate(myMarkAverageProvider);
        },
        child: marks.when(
          data: (page) => ListView(
            padding: EdgeInsets.only(
              left: 20,
              right: 20,
              top: 12,
              bottom: shellContentBottomPadding(context),
            ),
            children: [
              const TermSelector(padding: EdgeInsets.only(bottom: 12)),
              MarkAverageCard(average: average, markCount: markCount),
              if (markCount > 0) ...[
                PremiumSectionHeader(
                  title: context.l10n.marksByCategory,
                  subtitle: context.l10n.marksWhereStrongest,
                  decorAsset: PaperAssets.brainVerified,
                ),
                _CategoryBreakdown(stats: stats.valueOrNull ?? const []),
                PremiumSectionHeader(
                  title: context.l10n.marksAll,
                  subtitle: context.l10n.marksNewestFirst,
                  decorAsset: PaperAssets.documentLayout,
                ),
                if (page.items.isEmpty)
                  EmptyState(
                    title: context.l10n.marksEmptyTitle,
                    subtitle: context.l10n.marksEmptySubtitle,
                    icon: Icons.grade_outlined,
                  )
                else ...[
                  ...page.items.map((m) => _StudentMarkTile(mark: m)),
                  LoadMoreButton(
                    hasMore: page.hasMore,
                    loading: page.loadingMore,
                    onPressed: () =>
                        ref.read(myMarksProvider.notifier).loadMore(),
                  ),
                ],
              ] else
                EmptyState(
                  title: context.l10n.marksEmptyTitle,
                  subtitle: context.l10n.marksEmptySubtitle,
                  icon: Icons.grade_outlined,
```

```
                ),
              ],
            ),
            loading: () => const Center(child: CircularProgressIndicator()),
            error: (e, _) => ErrorState(message: e.toString()),
          ),
        ),
      );
    }

    Future<void> _shareProgress(BuildContext context, WidgetRef ref) async {
      final l10n = context.l10n;
      final groupId = ref.read(activeGroupIdProvider);
      if (groupId == null) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(l10n.progressReportNeedGroup)));
        return;
      }

      final student = await ref.read(studentProfileProvider.future);
      final profile = await ref.read(currentProfileProvider.future);
      if (student == null || profile == null) {
        if (context.mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(l10n.progressReportFailed('No profile'))),
          );
        }
        return;
      }

      final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];
      final groupTitle =
          groups.where((g) => g.id == groupId).map((g) => g.title).firstOrNull ??
          'group';
      final term = ref.read(selectedMarkTermProvider);

      if (context.mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(l10n.progressReportSharing)));
      }

      try {
        await shareProgressReport(
          studentId: student.id,
          groupId: groupId,
          studentName: profile.fullName.isNotEmpty
              ? profile.fullName
              : profile.username,
          groupTitle: groupTitle,
          term: term,
          l10n: l10n,
        );
        if (context.mounted) {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(SnackBar(content: Text(l10n.progressReportDone)));
        }
      } catch (e) {
        if (context.mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(l10n.progressReportFailed('$e'))),
          );
        }
      }
    }
  }

  class _CategoryBreakdown extends StatelessWidget {
    const _CategoryBreakdown({required this.stats});

    final List<({MarkCategory category, double percentage, double weight})> stats;
```

```
@override
Widget build(BuildContext context) {
  final l10n = context.l10n;
  final byCategory =
      <MarkCategory, List<({double percentage, double weight})>>{};
  for (final s in stats) {
    byCategory.putIfAbsent(s.category, () => []).add((
      percentage: s.percentage,
      weight: s.weight,
    ));
  }

  final rows = byCategory.entries.map((entry) {
    final totalWeight = entry.value.fold<double>(0, (s, m) => s + m.weight);
    final weighted = entry.value.fold<double>(
      0,
      (s, m) => s + m.percentage * m.weight,
    );
    return (entry.key, totalWeight == 0 ? 0.0 : weighted / totalWeight);
  }).toList()..sort((a, b) => b.$2.compareTo(a.$2));

  return PremiumCard(
    padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 8),
    child: Column(
      children: rows
          .map(
            (row) => Padding(
              padding: const EdgeInsets.symmetric(vertical: 10),
              child: Row(
                children: [
                  Expanded(
                    flex: 2,
                    child: Text(
                      row.$1.l10nLabel(l10n),
                      style: const TextStyle(
                        fontWeight: FontWeight.w600,
                        fontSize: 14,
                      ),
                    ),
                  ),
                  Expanded(
                    flex: 3,
                    child: ClipRRect(
                      borderRadius: BorderRadius.circular(4),
                      child: LinearProgressIndicator(
                        value: row.$2 / 100,
                        minHeight: 6,
                        backgroundColor: AppColors.softSurface(context),
                        valueColor: AlwaysStoppedAnimation(
                          percentColor(row.$2, context: context),
                        ),
                      ),
                    ),
                  ),
                  const SizedBox(width: 12),
                  SizedBox(
                    width: 48,
                    child: Text(
                      formatPercent(row.$2),
                      textAlign: TextAlign.end,
                      style: const TextStyle(
                        fontWeight: FontWeight.w800,
                        fontSize: 13,
                      ),
                    ),
                  ),
                ],
              ),
            ),
          )
          .toList(),
    ),
  );
```

```
  }
}

class _StudentMarkTile extends StatelessWidget {
  const _StudentMarkTile({required this.mark});

  final Mark mark;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return PremiumCard(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  mark.title,
                  style: const TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 15,
                  ),
                ),
                const SizedBox(height: 4),
                Text(
                  [
                    mark.category.l10nLabel(l10n),
                    if (mark.groupTitle != null) mark.groupTitle!,
                    DateFormat.yMMMd().format(mark.markedOn),
                  ].join(' · '),
                  style: TextStyle(
                    fontSize: 12,
                    color: AppColors.inkAlpha(context, 0.5),
                  ),
                ),
              ],
            ),
          ),
          PercentBadge(percent: mark.percentage),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/marks/student_marks_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/student/notifications/student_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 132. Import count: 14. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/notifications/notification_providers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/notifications/student_notifications_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';

class StudentNotificationsScreen extends ConsumerStatefulWidget {
  const StudentNotificationsScreen({super.key});

  @override
  ConsumerState<StudentNotificationsScreen> createState() =>
      _StudentNotificationsScreenState();
}

class _StudentNotificationsScreenState
    extends ConsumerState<StudentNotificationsScreen> {
  @override
  void initState() {
    super.initState();
```

```
    WidgetsBinding.instance.addPostFrameCallback((_) => _markAllRead());
  }

  Future<void> _markAllRead() async {
    if (!mounted) return;
    final profile = ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    await markAllNotificationsRead(profile.id);
    ref.invalidate(notificationsProvider);
  }

  void _open(AppNotification n) {
    final path = n.actionPath?.trim();
    if (path == null || path.isEmpty) return;
    context.push(path);
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final notifications = ref.watch(notificationsProvider);

    return Scaffold(
      appBar: AppBar(title: Text(l10n.notificationsTitle)),
      body: notifications.when(
        data: (list) {
          if (list.isEmpty) {
            return EmptyState(
              title: l10n.notificationsEmpty,
              subtitle: l10n.notificationsEmptySubtitle,
            );
          }
          return ListView.separated(
            padding: EdgeInsets.fromLTRB(
              16,
              16,
              16,
              shellContentBottomPadding(context, extra: 16),
            ),
            itemCount: list.length,
            separatorBuilder: (_, _) => const SizedBox(height: 8),
            itemBuilder: (context, index) {
              final n = list[index];
              final when = n.createdAt;
              final timeLabel = when == null
                  ? null
                  : DateFormat.MMMd().add_jm().format(when.toLocal());
              final canOpen =
                  n.actionPath != null && n.actionPath!.trim().isNotEmpty;

              return PremiumCard(
                onTap: canOpen ? () => _open(n) : null,
                child: ListTile(
                  contentPadding: EdgeInsets.zero,
                  leading: PaperGlyph(
                    icon: n.kindIcon,
                    size: 26,
                    opacity: n.isRead ? 0.45 : 1,
                    color: n.isRead
                        ? AppColors.inkAlpha(context, 0.35)
                        : AppColors.accent(context),
                  ),
                  title: Text(
                    n.kindLabel(l10n),
                    style: TextStyle(
                      fontWeight: n.isRead ? FontWeight.w500 : FontWeight.w800,
                    ),
                  ),
                  subtitle: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      if (n.body.isNotEmpty) ...[
                        const SizedBox(height: 4),
```

```
                      Text(n.body),
                    ],
                    if (timeLabel != null) ...[
                      const SizedBox(height: 6),
                      Text(
                        timeLabel,
                        style: TextStyle(
                          fontSize: 11,
                          color: AppColors.inkAlpha(context, 0.45),
                        ),
                      ),
                    ],
                  ],
                ),
                trailing: canOpen
                    ? Icon(
                        Icons.chevron_right_rounded,
                        color: AppColors.inkAlpha(context, 0.3),
                      )
                    : null,
              ),
            );
          },
        );
      },
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
    ),
  );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/notifications/student_notifications_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

# lib/screens/student/profile/student_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 12. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/screens/profile/app_profile_screen.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/student/profile/student_profile_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/profile/app_profile_screen.dart';

class StudentProfileScreen extends ConsumerWidget {
  const StudentProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const AppProfileScreen();
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/student/profile/student_profile_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/analytics/teacher_analytics_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 101. Import count: 13. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/analytics/teacher_analytics_charts.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';
- import 'package:mu_study/providers/marks/marks_providers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/analytics/teacher_analytics_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/analytics/teacher_analytics_charts.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';

class TeacherAnalyticsScreen extends ConsumerWidget {
  const TeacherAnalyticsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groupId = ref.watch(activeGroupIdProvider);
    final l10n = context.l10n;

    if (groupId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.navAnalytics)),
        body: EmptyState(title: l10n.teacherNoData),
      );
    }
```

```
    final attendance = ref.watch(attendanceWeekTrendProvider(groupId));
    final averages = ref.watch(groupMarkAveragesProvider(groupId));

    return Scaffold(
      appBar: AppBar(title: Text(l10n.navAnalytics)),
      body: RefreshIndicator(
        onRefresh: () async {
          ref.invalidate(attendanceWeekTrendProvider(groupId));
          ref.invalidate(groupMarkAveragesProvider(groupId));
        },
        child: ListView(
          padding: EdgeInsets.fromLTRB(
            20,
            8,
            20,
            shellContentBottomPadding(context, extra: 8),
          ),
          children: [
            const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
            PremiumSectionHeader(
              title: l10n.teacherMarkDistribution,
              subtitle: l10n.teacherStudentsByAverage,
              decorAsset: PaperAssets.texturedTanStar,
            ),
            averages.when(
              data: (map) {
                final bands = markBandPoints(
                  context,
                  l10n,
                  map.values.toList(),
                );
                if (bands.isEmpty) {
                  return EmptyState(
                    title: l10n.marksNoRecorded,
                    subtitle: l10n.teacherNoMarksTrends,
                    icon: Icons.grade_outlined,
                  );
                }
                return PaperTaped(
                  alignment: 0.45,
                  child: PremiumCard(
                    padding: const EdgeInsets.fromLTRB(16, 20, 16, 20),
                    child: MarkDistributionDoughnut(points: bands),
                  ),
                );
              },
              loading: () => const Center(child: CircularProgressIndicator()),
              error: (e, _) => ErrorState(message: e.toString()),
            ),
            const SizedBox(height: 8),
            PremiumSectionHeader(
              title: l10n.attendanceTrend,
              subtitle: l10n.attendanceOverall,
              decorAsset: PaperAssets.calendar,
            ),
            attendance.when(
              data: (trend) => PaperTaped(
                alignment: -0.45,
                child: PremiumCard(
                  padding: const EdgeInsets.fromLTRB(12, 16, 12, 8),
                  child: AttendanceTrendBars(trend: trend),
                ),
              ),
              loading: () => const Center(child: CircularProgressIndicator()),
              error: (e, _) => ErrorState(message: e.toString()),
            ),
          ],
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/analytics/teacher_analytics_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

# lib/screens/teacher/announcements/teacher_announcements_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 298. Import count: 14. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/notifications/notification_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/announcements/teacher_announcements_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class TeacherAnnouncementsScreen extends ConsumerStatefulWidget {
  const TeacherAnnouncementsScreen({super.key});

  @override
  ConsumerState<TeacherAnnouncementsScreen> createState() =>
      _TeacherAnnouncementsScreenState();
}

class _TeacherAnnouncementsScreenState
    extends ConsumerState<TeacherAnnouncementsScreen> {
  Future<void> _create() async {
```

```
  final teacher = await ref.read(teacherProfileProvider.future);
  final groups = await ref.read(myGroupsProvider.future);
  if (teacher == null || groups.isEmpty) return;
  final title = TextEditingController();
  final body = TextEditingController();
  var groupId = ref.read(activeGroupIdProvider) ?? groups.first.id;

  if (!mounted) return;
  final l10n = context.l10n;
  await showDialog<void>(
    context: context,
    builder: (ctx) => StatefulBuilder(
      builder: (ctx, setStateLocal) => AlertDialog(
        title: Text(l10n.announcementsNew),
        content: SingleChildScrollView(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              TextField(
                controller: title,
                decoration: InputDecoration(labelText: l10n.commonTitle),
              ),
              const SizedBox(height: 16),
              TextField(
                controller: body,
                decoration: InputDecoration(labelText: l10n.commonMessage),
                maxLines: 4,
              ),
              const SizedBox(height: 16),
              DropdownButtonFormField<String>(
                initialValue: groupId,
                decoration: InputDecoration(
                  labelText: l10n.teacherTargetGroup,
                ),
                items: groups
                    .map(
                      (g) =>
                          DropdownMenuItem(value: g.id, child: Text(g.title)),
                    )
                    .toList(),
                onChanged: (v) => setStateLocal(() => groupId = v ?? groupId),
              ),
            ],
          ),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx),
            child: Text(l10n.commonCancel),
          ),
          ElevatedButton(
            onPressed: () async {
              if (title.text.trim().isEmpty) return;
              await supabase.from('announcements').insert({
                'teacher_id': teacher.id,
                'group_id': groupId,
                'title': title.text.trim(),
                'description': body.text.trim(),
              });
              if (ctx.mounted) Navigator.pop(ctx);
              ref.invalidate(announcementsProvider);
            },
            child: Text(l10n.announcementsPost),
          ),
        ],
      ),
    ),
  );
}

Future<void> _edit(Announcement a) async {
  final l10n = context.l10n;
  final title = TextEditingController(text: a.title);
```

```
  final body = TextEditingController(text: a.description);
  await showDialog<void>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: Text(l10n.announcementsEdit),
      content: SingleChildScrollView(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            TextField(
              controller: title,
              decoration: InputDecoration(labelText: l10n.commonTitle),
            ),
            const SizedBox(height: 16),
            TextField(
              controller: body,
              decoration: InputDecoration(labelText: l10n.commonMessage),
              maxLines: 4,
            ),
          ],
        ),
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx),
          child: Text(l10n.commonCancel),
        ),
        ElevatedButton(
          onPressed: () async {
            if (title.text.trim().isEmpty) return;
            await supabase
                .from('announcements')
                .update({
                  'title': title.text.trim(),
                  'description': body.text.trim(),
                })
                .eq('id', a.id);
            if (ctx.mounted) Navigator.pop(ctx);
            ref.invalidate(announcementsProvider);
          },
          child: Text(l10n.commonSave),
        ),
      ],
    ),
  );
}

Future<void> _delete(Announcement a) async {
  final l10n = context.l10n;
  final confirmed = await showDialog<bool>(
    context: context,
    builder: (ctx) => AlertDialog(
      title: Text(l10n.announcementsDeleteTitle),
      content: Text(l10n.announcementsDeleteBody),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(ctx, false),
          child: Text(l10n.commonCancel),
        ),
        FilledButton(
          style: FilledButton.styleFrom(
            backgroundColor: AppColors.danger(context),
          ),
          onPressed: () => Navigator.pop(ctx, true),
          child: Text(l10n.commonDelete),
        ),
      ],
    ),
  );
  if (confirmed != true) return;
  await supabase.from('announcements').delete().eq('id', a.id);
  ref.invalidate(announcementsProvider);
}
```

```
@override
Widget build(BuildContext context) {
  final groupId = ref.watch(activeGroupIdProvider);
  final items = ref.watch(announcementsProvider(groupId));
  final myTeacherId = ref.watch(teacherProfileProvider).valueOrNull?.id;
  final l10n = context.l10n;

  return Scaffold(
    appBar: AppBar(title: Text(l10n.announcementsTitle)),
    floatingActionButtonLocation: shellFabLocation(context),
    floatingActionButton: FloatingActionButton(
      onPressed: _create,
      child: const Icon(Icons.campaign_outlined),
    ),
    body: items.when(
      data: (list) => ListView.builder(
        padding: EdgeInsets.fromLTRB(
          20,
          20,
          20,
          shellContentBottomPadding(context, hasFab: true),
        ),
        itemCount: list.length + 1,
        itemBuilder: (_, index) {
          if (index == 0) {
            return Column(
              children: [
                const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
                if (list.isEmpty)
                  EmptyState(
                    title: l10n.announcementsEmpty,
                    subtitle: l10n.announcementsEmptyTeacher,
                  ),
              ],
            );
          }
          final i = index - 1;
          final announcement = list[i];
          final mine =
              myTeacherId != null && announcement.teacherId == myTeacherId;
          return PremiumCard(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Expanded(
                      child: Text(
                        announcement.title,
                        style: const TextStyle(
                          fontWeight: FontWeight.w800,
                          fontSize: 16,
                        ),
                      ),
                    ),
                    if (mine)
                      PopupMenuButton<String>(
                        padding: EdgeInsets.zero,
                        icon: Icon(
                          Icons.more_horiz_rounded,
                          size: 20,
                          color: AppColors.inkAlpha(context, 0.5),
                        ),
                        onSelected: (v) => v == 'edit'
                            ? _edit(announcement)
                            : _delete(announcement),
                        itemBuilder: (_) => [
                          PopupMenuItem(
                            value: 'edit',
                            child: Text(l10n.commonEdit),
                          ),
```

```
                        PopupMenuItem(
                          value: 'delete',
                          child: Text(l10n.commonDelete),
                        ),
                      ],
                    ),
                  ],
                ),
                const SizedBox(height: 8),
                Text(
                  announcement.description,
                  style: TextStyle(
                    color: AppColors.inkAlpha(context, 0.7),
                    height: 1.5,
                  ),
                ),
                const SizedBox(height: 12),
                Row(
                  children: [
                    Text(
                      DateFormat.yMMMd().format(
                        announcement.createdAt ?? DateTime.now(),
                      ),
                      style: TextStyle(
                        fontSize: 11,
                        color: AppColors.inkAlpha(context, 0.4),
                      ),
                    ),
                    if (announcement.teacherId == null) ...[
                      const SizedBox(width: 8),
                      Text(
                        '• ${l10n.announcementsByAdmin}',
                        style: TextStyle(
                          fontSize: 11,
                          fontWeight: FontWeight.w700,
                          color: AppColors.accent(context),
                        ),
                      ),
                    ],
                  ],
                ),
              ],
            ),
          );
        },
      ),
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
    ),
  );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/announcements/teacher_announcements_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/assignments/teacher_assignments_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 874. Import count: 21. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/homework/due_date_label.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/inputs/load_more_button.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/homework/assignment_due.dart';
- import 'package:mu_study/core/homework/assignment_templates.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- … and 3 more imports

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/assignments/teacher_assignments_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/homework/assignment_templates.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
```

```
import 'package:mu_study/supabase/supabase_client.dart';

class TeacherAssignmentsScreen extends ConsumerStatefulWidget {
  const TeacherAssignmentsScreen({super.key});

  @override
  ConsumerState<TeacherAssignmentsScreen> createState() =>
      _TeacherAssignmentsScreenState();
}

class _TeacherAssignmentsScreenState
    extends ConsumerState<TeacherAssignmentsScreen> {
  void _openPack(BuildContext context, Assignment a) {
    context.push('/teacher/assignments/${a.id}');
  }

  Future<void> _createAssignment({AssignmentTemplate? fromTemplate}) async {
    final teacher = await ref.read(teacherProfileProvider.future);
    final groups = await ref.read(myGroupsProvider.future);
    if (teacher == null || groups.isEmpty) return;

    var targetGroupId = ref.read(activeGroupIdProvider) ?? groups.first.id;
    final titleController = TextEditingController(
      text: fromTemplate?.title ?? '',
    );
    final descController = TextEditingController(
      text: fromTemplate?.description ?? '',
    );
    var type = fromTemplate?.type ?? AssignmentType.staticAssignment;
    DateTime? dueDate;

    if (!mounted) return;
    final l10n = context.l10n;
    await showDialog<void>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setStateLocal) => AlertDialog(
          title: Text(
            fromTemplate == null
                ? l10n.assignmentNew
                : l10n.teacherCreateFromTemplate,
          ),
          content: SingleChildScrollView(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (fromTemplate != null && fromTemplate.tasks.isNotEmpty)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 12),
                    child: Text(
                      l10n.teacherTasksCopied(fromTemplate.tasks.length),
                      style: TextStyle(
                        color: AppColors.inkAlpha(context, 0.55),
                        fontSize: 13,
                      ),
                    ),
                  ),
                TextField(
                  controller: titleController,
                  textCapitalization: TextCapitalization.sentences,
                  decoration: InputDecoration(labelText: l10n.commonTitle),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: descController,
                  textCapitalization: TextCapitalization.sentences,
                  decoration: InputDecoration(
                    labelText: l10n.commonDescription,
                  ),
                ),
                const SizedBox(height: 16),
                if (fromTemplate == null) ...[
```

```
  DropdownButtonFormField<AssignmentType>(
    initialValue: type,
    decoration: InputDecoration(labelText: l10n.commonType),
    items: [
      DropdownMenuItem(
        value: AssignmentType.staticAssignment,
        child: Text(l10n.staticAssignment),
      ),
      DropdownMenuItem(
        value: AssignmentType.homeworkPack,
        child: Text(l10n.homeworkPack),
      ),
    ],
    onChanged: (v) => setStateLocal(
      () => type = v ?? AssignmentType.staticAssignment,
    ),
  ),
  const SizedBox(height: 8),
  Text(
    type == AssignmentType.homeworkPack
        ? l10n.assignmentTypeHintPack
        : l10n.assignmentTypeHintStatic,
    style: TextStyle(
      fontSize: 12,
      height: 1.35,
      color: AppColors.inkAlpha(context, 0.55),
    ),
  ),
] else
  InputDecorator(
    decoration: InputDecoration(labelText: l10n.commonType),
    child: Text(
      type == AssignmentType.homeworkPack
          ? l10n.homeworkPack
          : l10n.staticAssignment,
    ),
  ),
const SizedBox(height: 16),
DropdownButtonFormField<String>(
  initialValue: targetGroupId,
  decoration: InputDecoration(labelText: l10n.commonGroup),
  items: groups
      .map(
        (g) =>
            DropdownMenuItem(value: g.id, child: Text(g.title)),
      )
      .toList(),
  onChanged: (v) =>
      setStateLocal(() => targetGroupId = v ?? targetGroupId),
),
const SizedBox(height: 8),
ListTile(
  contentPadding: EdgeInsets.zero,
  title: Text(
    dueDate == null
        ? l10n.commonNoDueDate
        : l10n.commonDueDate(
            DateFormat.yMMMd().format(dueDate!),
          ),
  ),
  trailing: const Icon(Icons.calendar_today_outlined, size: 18),
  onTap: () async {
    final picked = await showDatePicker(
      context: ctx,
      initialDate: dueDate ?? DateTime.now(),
      firstDate: DateTime.now().subtract(
        const Duration(days: 30),
      ),
      lastDate: DateTime.now().add(const Duration(days: 365)),
    );
    if (picked != null) setStateLocal(() => dueDate = picked);
  },
),
```

```
              ],
            ),
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(ctx),
              child: Text(l10n.commonCancel),
            ),
            ElevatedButton(
              onPressed: () async {
                if (titleController.text.trim().isEmpty) return;
                Assignment? created;
                if (fromTemplate != null) {
                  created = await createAssignmentFromTemplate(
                    template: fromTemplate,
                    teacherId: teacher.id,
                    groupId: targetGroupId,
                    dueDate: dueDate,
                    titleOverride: titleController.text.trim(),
                    descriptionOverride: descController.text.trim(),
                  );
                } else {
                  final title = titleController.text.trim();
                  final description = descController.text.trim();
                  final row = await supabase
                      .from('assignments')
                      .insert({
                        'teacher_id': teacher.id,
                        'group_id': targetGroupId,
                        'type': assignmentTypeToDb(type),
                        'title': title,
                        'description': description,
                        'due_date': dueDate?.toIso8601String(),
                      })
                      .select()
                      .single();
                  created = Assignment.fromJson(Map<String, dynamic>.from(row));
                  if (type == AssignmentType.staticAssignment) {
                    await supabase.from('tasks').insert({
                      'assignment_id': created.id,
                      'title': title,
                      'description': description,
                      'sort_order': 0,
                    });
                  }
                }
                if (ctx.mounted) Navigator.pop(ctx);
                ref.invalidate(assignmentsProvider(targetGroupId));
                if (mounted && created.type == AssignmentType.homeworkPack) {
                  _openPack(context, created);
                }
              },
              child: Text(l10n.commonCreate),
            ),
          ],
        ),
      ),
    );
  }

  Future<void> _editAssignment(Assignment a) async {
    final l10n = context.l10n;
    final titleController = TextEditingController(text: a.title);
    final descController = TextEditingController(text: a.description);
    DateTime? dueDate = a.dueDate;

    final saved = await showDialog<bool>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setStateLocal) => AlertDialog(
          title: Text(l10n.assignmentEdit),
          content: SingleChildScrollView(
            child: Column(
```

```
            mainAxisSize: MainAxisSize.min,
            children: [
              TextField(
                controller: titleController,
                textCapitalization: TextCapitalization.sentences,
                decoration: InputDecoration(labelText: l10n.commonTitle),
              ),
              const SizedBox(height: 16),
              TextField(
                controller: descController,
                textCapitalization: TextCapitalization.sentences,
                decoration: InputDecoration(
                  labelText: l10n.commonDescription,
                ),
                maxLines: 3,
              ),
              const SizedBox(height: 8),
              ListTile(
                contentPadding: EdgeInsets.zero,
                title: Text(
                  dueDate == null
                      ? l10n.commonNoDueDate
                      : l10n.commonDueDate(
                          DateFormat.yMMMd().format(dueDate!),
                        ),
                ),
                trailing: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    if (dueDate != null)
                      IconButton(
                        tooltip: l10n.commonNoDueDate,
                        onPressed: () => setStateLocal(() => dueDate = null),
                        icon: const Icon(Icons.clear, size: 18),
                      ),
                    const Icon(Icons.calendar_today_outlined, size: 18),
                  ],
                ),
                onTap: () async {
                  final picked = await showDatePicker(
                    context: ctx,
                    initialDate: dueDate ?? DateTime.now(),
                    firstDate: DateTime.now().subtract(
                      const Duration(days: 365),
                    ),
                    lastDate: DateTime.now().add(const Duration(days: 365)),
                  );
                  if (picked != null) setStateLocal(() => dueDate = picked);
                },
              ),
            ],
          ),
        ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: Text(l10n.commonCancel),
          ),
          ElevatedButton(
            onPressed: () => Navigator.pop(ctx, true),
            child: Text(l10n.commonSave),
          ),
        ],
      ),
    ),
  );

  if (saved != true || titleController.text.trim().isEmpty) return;
  final title = titleController.text.trim();
  final description = descController.text.trim();
  try {
    await supabase
        .from('assignments')
```

```
          .update({
            'title': title,
            'description': description,
            'due_date': dueDate?.toIso8601String(),
          })
          .eq('id', a.id);
      if (a.type == AssignmentType.staticAssignment) {
        final tasks = await supabase
            .from('tasks')
            .select('id')
            .eq('assignment_id', a.id)
            .order('sort_order')
            .limit(1);
        if (tasks.isEmpty) {
          await supabase.from('tasks').insert({
            'assignment_id': a.id,
            'title': title,
            'description': description,
            'sort_order': 0,
          });
        } else {
          await supabase
              .from('tasks')
              .update({'title': title, 'description': description})
              .eq('id', tasks.first['id'] as String);
        }
      }
      ref.invalidate(assignmentsProvider(a.groupId));
      ref.invalidate(assignmentDetailProvider(a.id));
      ref.invalidate(tasksProvider(a.id));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }
```

**lib/screens/teacher/assignments/teacher_assignments_screen.dart — continued (2/3)**

```
  Future<void> _deleteAssignment(Assignment a) async {
    final l10n = context.l10n;
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: Text(l10n.assignmentDeleteTitle),
        content: Text(l10n.assignmentDeleteBody),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: Text(l10n.commonCancel),
          ),
          FilledButton(
            style: FilledButton.styleFrom(
              backgroundColor: AppColors.danger(context),
            ),
            onPressed: () => Navigator.pop(ctx, true),
            child: Text(l10n.commonDelete),
          ),
        ],
      ),
    );
    if (confirmed != true) return;
    try {
      await supabase.from('assignments').delete().eq('id', a.id);
      ref.invalidate(assignmentsProvider(a.groupId));
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.assignmentDeleted)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
```

```
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _duplicateAssignment(Assignment source) async {
    final teacher = await ref.read(teacherProfileProvider.future);
    final groups = await ref.read(myGroupsProvider.future);
    if (teacher == null || groups.isEmpty || !mounted) return;

    var targetGroupId = ref.read(activeGroupIdProvider) ?? source.groupId;
    final titleController = TextEditingController(
      text: '${source.title} (copy)',
    );
    DateTime? dueDate = source.dueDate;
    final l10n = context.l10n;

    final confirmed = await showDialog<bool>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setStateLocal) => AlertDialog(
          title: Text(l10n.teacherDuplicateTitle),
          content: SingleChildScrollView(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                TextField(
                  controller: titleController,
                  textCapitalization: TextCapitalization.sentences,
                  decoration: InputDecoration(labelText: l10n.commonTitle),
                ),
                const SizedBox(height: 16),
                DropdownButtonFormField<String>(
                  initialValue: targetGroupId,
                  decoration: InputDecoration(labelText: l10n.commonGroup),
                  items: groups
                      .map(
                        (g) =>
                            DropdownMenuItem(value: g.id, child: Text(g.title)),
                      )
                      .toList(),
                  onChanged: (v) =>
                      setStateLocal(() => targetGroupId = v ?? targetGroupId),
                ),
                ListTile(
                  contentPadding: EdgeInsets.zero,
                  title: Text(
                    dueDate == null
                        ? l10n.commonNoDueDate
                        : l10n.commonDueDate(
                            DateFormat.yMMMd().format(dueDate!),
                          ),
                  ),
                  trailing: const Icon(Icons.calendar_today_outlined, size: 18),
                  onTap: () async {
                    final picked = await showDatePicker(
                      context: ctx,
                      initialDate: dueDate ?? DateTime.now(),
                      firstDate: DateTime.now().subtract(
                        const Duration(days: 30),
                      ),
                      lastDate: DateTime.now().add(const Duration(days: 365)),
                    );
                    if (picked != null) setStateLocal(() => dueDate = picked);
                  },
                ),
              ],
            ),
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(ctx, false),
              child: Text(l10n.commonCancel),
```

```
            ),
            ElevatedButton(
              onPressed: () => Navigator.pop(ctx, true),
              child: Text(l10n.teacherDuplicateAssignment),
            ),
          ],
        ),
      ),
    );

    if (confirmed != true || !mounted) return;

    try {
      final created = await duplicateAssignment(
        source: source,
        teacherId: teacher.id,
        groupId: targetGroupId,
        dueDate: dueDate,
        titleOverride: titleController.text.trim(),
      );
      ref.invalidate(assignmentsProvider(targetGroupId));
      if (source.groupId != targetGroupId) {
        ref.invalidate(assignmentsProvider(source.groupId));
      }
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.teacherDuplicated)));
      if (created.type == AssignmentType.homeworkPack) {
        _openPack(context, created);
      }
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _saveAsTemplate(Assignment assignment) async {
    final teacher = await ref.read(teacherProfileProvider.future);
    if (teacher == null || !mounted) return;
    final l10n = context.l10n;
    try {
      await saveAssignmentAsTemplate(
        teacherId: teacher.id,
        assignment: assignment,
      );
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.teacherTemplateSaved)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _openTemplates() async {
    final teacher = await ref.read(teacherProfileProvider.future);
    if (teacher == null || !mounted) return;

    await showModalBottomSheet<void>(
      context: context,
      isScrollControlled: true,
      useRootNavigator: true,
      backgroundColor: Colors.transparent,
      builder: (ctx) {
        return _TemplatesSheet(
          teacherId: teacher.id,
          onUse: (template) {
```

```
            Navigator.pop(ctx);
            _createAssignment(fromTemplate: template);
          },
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    final groupId = ref.watch(activeGroupIdProvider);
    final assignments = ref.watch(assignmentsProvider(groupId));
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.assignmentsTitle),
        actions: [
          IconButton(
            tooltip: l10n.teacherTemplates,
            onPressed: _openTemplates,
            icon: const Icon(Icons.dashboard_customize_outlined),
          ),
        ],
      ),
      floatingActionButtonLocation: shellFabLocation(context),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _createAssignment(),
        child: const Icon(Icons.add),
      ),
      body: assignments.when(
        data: (page) {
          final list = sortAssignmentsByDue(page.items);
          final isNeu = Neumorphic.isActive(context);
          return ListView.builder(
            clipBehavior: isNeu ? Clip.none : Clip.hardEdge,
            padding: EdgeInsets.fromLTRB(
              20,
              20,
              20,
              shellContentBottomPadding(context, hasFab: true),
            ),
            itemCount: list.length + 1 + (page.hasMore ? 1 : 0),
            itemBuilder: (_, index) {
              if (index == 0) {
                return Column(
                  children: [
                    const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
                    if (list.isEmpty)
                      EmptyState(
                        title: l10n.assignmentsEmpty,
                        subtitle: l10n.assignmentsEmptyTeacher,
                      ),
                  ],
                );
              }
              if (index == list.length + 1) {
                return LoadMoreButton(
                  hasMore: page.hasMore,
                  loading: page.loadingMore,
                  onPressed: () => ref
                      .read(assignmentsProvider(groupId).notifier)
                      .loadMore(),
                );
              }
              final a = list[index - 1];
              final isPack = a.type == AssignmentType.homeworkPack;
              final overdue = assignmentIsOverdue(a);
              final tone = AppColors.resolveAccent(
                context,
                overdue
                    ? AppColors.danger(context)
                    : isPack
```

```
      ? AppColors.goldAccent(context)
      : AppColors.accent(context),
);
final iconData = overdue
    ? Icons.warning_amber_rounded
    : isPack
    ? Icons.inventory_2_outlined
    : Icons.description_outlined;

return PremiumCard(
  onTap: () =>
      isPack ? _openPack(context, a) : _editAssignment(a),
  padding: const EdgeInsets.symmetric(
    horizontal: 16,
    vertical: 14,
  ),
  child: Row(
    children: [
      NeuIconWell(
        icon: iconData,
        color: tone,
        size: 20,
        padding: 10,
      ),
      const SizedBox(width: 16),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              a.title,
              style: const TextStyle(
                fontWeight: FontWeight.w700,
                fontSize: 16,
              ),
            ),
            const SizedBox(height: 4),
            DueDateLabel(assignment: a, compact: true),
            Text(
              isPack ? l10n.homeworkPack : l10n.staticTask,
              style: TextStyle(
                color: AppColors.inkAlpha(context, 0.45),
                fontSize: 12,
              ),
            ),
          ],
        ),
      ),
      PopupMenuButton<String>(
        onSelected: (value) {
          switch (value) {
            case 'edit':
              _editAssignment(a);
            case 'delete':
              _deleteAssignment(a);
            case 'duplicate':
              _duplicateAssignment(a);
            case 'template':
              _saveAsTemplate(a);
            case 'open':
              _openPack(context, a);
          }
        },
        itemBuilder: (_) => [
          if (isPack)
            PopupMenuItem(
              value: 'open',
              child: Text(l10n.teacherManagePack),
            ),
          PopupMenuItem(
            value: 'edit',
            child: Text(l10n.commonEdit),
          ),
```

```
                        PopupMenuItem(
                          value: 'duplicate',
                          child: Text(l10n.teacherDuplicateAssignment),
                        ),
                        PopupMenuItem(
                          value: 'template',
                          child: Text(l10n.teacherSaveAsTemplate),
                        ),
                        PopupMenuItem(
                          value: 'delete',
                          child: Text(l10n.commonDelete),
                        ),
                      ],
                    ),
                  ],
                ),
              );
            },
          );
        },
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
      ),
    );
  }
}

class _TemplatesSheet extends StatefulWidget {
  const _TemplatesSheet({required this.teacherId, required this.onUse});

  final String teacherId;
  final ValueChanged<AssignmentTemplate> onUse;

  @override
  State<_TemplatesSheet> createState() => _TemplatesSheetState();
}

class _TemplatesSheetState extends State<_TemplatesSheet> {
  late Future<List<AssignmentTemplate>> _future;

  @override
  void initState() {
    super.initState();
    _future = fetchAssignmentTemplates(widget.teacherId);
  }

  Future<void> _reload() async {
    setState(() {
      _future = fetchAssignmentTemplates(widget.teacherId);
    });
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return Padding(
      padding: EdgeInsets.only(
        bottom: MediaQuery.of(context).viewInsets.bottom,
      ),
      child: Container(
        constraints: BoxConstraints(
          maxHeight: MediaQuery.sizeOf(context).height * 0.75,
        ),
        decoration: BoxDecoration(
          color: AppColors.sheetSurface(context),
          borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),
        ),
        child: Column(
          children: [
            const SizedBox(height: 12),
            Container(
              width: 40,
              height: 4,
```

```
              decoration: BoxDecoration(
                color: AppColors.softSurface(context),
                borderRadius: BorderRadius.circular(2),
              ),
```

**lib/screens/teacher/assignments/teacher_assignments_screen.dart — continued (3/3)**

```
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 12, 20, 8),
              child: PremiumSectionHeader(
                title: l10n.teacherTemplates,
                subtitle: l10n.teacherTemplatesEmptySubtitle,
              ),
            ),
            Expanded(
              child: FutureBuilder<List<AssignmentTemplate>>(
                future: _future,
                builder: (context, snap) {
                  if (snap.connectionState != ConnectionState.done) {
                    return const Center(child: CircularProgressIndicator());
                  }
                  if (snap.hasError) {
                    return ErrorState(message: '${snap.error}');
                  }
                  final items = snap.data ?? const [];
                  if (items.isEmpty) {
                    return EmptyState(
                      title: l10n.teacherTemplatesEmpty,
                      subtitle: l10n.teacherTemplatesEmptySubtitle,
                      icon: Icons.dashboard_customize_outlined,
                    );
                  }
                  return ListView.separated(
                    padding: EdgeInsets.fromLTRB(
                      20,
                      0,
                      20,
                      MediaQuery.of(context).padding.bottom + 20,
                    ),
                    itemCount: items.length,
                    separatorBuilder: (context, index) =>
                        const SizedBox(height: 8),
                    itemBuilder: (_, i) {
                      final t = items[i];
                      final isPack = t.type == AssignmentType.homeworkPack;
                      return PremiumCard(
                        padding: const EdgeInsets.symmetric(
                          horizontal: 14,
                          vertical: 12,
                        ),
                        child: Row(
                          children: [
                            Expanded(
                              child: Column(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                children: [
                                  Text(
                                    t.title,
                                    style: const TextStyle(
                                      fontWeight: FontWeight.w700,
                                      fontSize: 15,
                                    ),
                                  ),
                                  const SizedBox(height: 2),
                                  Text(
                                    '${isPack ? l10n.homeworkPack : l10n.staticTask}'
                                    '${t.tasks.isEmpty ? '' : ' · ${l10n.teacherTasksCopied(t.
tasks.length)}'}',
                                    style: TextStyle(
                                      fontSize: 12,
                                      color: AppColors.inkAlpha(context, 0.5),
                                    ),
```

```
                                ),
                              ],
                            ),
                          ),
                          TextButton(
                            onPressed: () => widget.onUse(t),
                            child: Text(l10n.teacherUseTemplate),
                          ),
                          IconButton(
                            tooltip: l10n.commonDelete,
                            onPressed: () async {
                              final ok = await showDialog<bool>(
                                context: context,
                                builder: (c) => AlertDialog(
                                  title: Text(l10n.teacherDeleteTemplate),
                                  content: Text(
                                    l10n.teacherDeleteTemplateBody(t.title),
                                  ),
                                  actions: [
                                    TextButton(
                                      onPressed: () =>
                                          Navigator.pop(c, false),
                                      child: Text(l10n.commonCancel),
                                    ),
                                    TextButton(
                                      onPressed: () => Navigator.pop(c, true),
                                      child: Text(
                                        l10n.commonDelete,
                                        style: const TextStyle(
                                          color: Colors.red,
                                        ),
                                      ),
                                    ),
                                  ],
                                ),
                              );
                              if (ok == true) {
                                await deleteAssignmentTemplate(t.id);
                                await _reload();
                              }
                            },
                            icon: const Icon(Icons.delete_outline),
                          ),
                        ],
                      ),
                    );
                  },
                );
              },
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/assignments/teacher_assignments_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/attendance/teacher_attendance_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 819. Import count: 14. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/attendance/attendance_calendar_grid.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/attendance/attendance_providers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/attendance/teacher_attendance_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/attendance/attendance_calendar_grid.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';

class TeacherAttendanceScreen extends ConsumerStatefulWidget {
  const TeacherAttendanceScreen({super.key});

  @override
  ConsumerState<TeacherAttendanceScreen> createState() =>
      _TeacherAttendanceScreenState();
}

class _TeacherAttendanceScreenState
    extends ConsumerState<TeacherAttendanceScreen>
    with SingleTickerProviderStateMixin {
  final Map<String, AttendanceStatus> _edits = {};
  String? _editedGroupId;
```

```
  bool _saving = false;
  late final TabController _tabs;
  late DateTime _month;
  String? _historyStudentId;

  @override
  void initState() {
    super.initState();
    _tabs = TabController(length: 2, vsync: this);
    final now = DateTime.now();
    _month = DateTime(now.year, now.month);
  }

  @override
  void dispose() {
    _tabs.dispose();
    super.dispose();
  }

  AttendanceStatus _statusFor(
    String studentId,
    Map<String, AttendanceStatus> saved,
  ) {
    return _edits[studentId] ?? saved[studentId] ?? AttendanceStatus.present;
  }

  void _resetEditsIfGroupChanged(String groupId) {
    if (_editedGroupId != groupId) {
      _edits.clear();
      _editedGroupId = groupId;
      _historyStudentId = null;
    }
  }

  void _markAll(List<GroupMember> members, AttendanceStatus status) {
    setState(() {
      for (final m in members) {
        _edits[m.studentId] = status;
      }
    });
  }

  Future<void> _saveAll(
    String groupId,
    List<GroupMember> members,
    Map<String, AttendanceStatus> saved,
  ) async {
    if (members.isEmpty) return;
    setState(() => _saving = true);

    try {
      final byStudent = {
        for (final m in members) m.studentId: _statusFor(m.studentId, saved),
      };
      await upsertAttendanceBulk(
        groupId: groupId,
        date: DateTime.now(),
        byStudentId: byStudent,
      );

      ref.invalidate(todayAttendanceProvider(groupId));
      ref.invalidate(attendanceWeekTrendProvider(groupId));
      ref.invalidate(attendanceRangeProvider);
      _edits.clear();

      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(context.l10n.attendanceSaved)));
      }
    } on AttendanceWeekendException {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
```

```
          SnackBar(content: Text(context.l10n.attendanceWeekendNotAllowed)),
        );
      }
    } on AttendanceFutureDateException {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.attendanceFutureNotAllowed)),
        );
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.commonErrorWithDetails('$e'))),
        );
      }
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  Future<void> _cycleHistoryDay({
    required String groupId,
    required String studentId,
    required DateTime day,
    required AttendanceStatus? current,
  }) async {
    if (!isAttendanceDateAllowed(day)) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(
              isAttendanceWeekend(day)
                  ? context.l10n.attendanceWeekendNotAllowed
                  : context.l10n.attendanceFutureNotAllowed,
            ),
          ),
        );
      }
      return;
    }
    final next = nextAttendanceStatus(current);
    try {
      await upsertAttendance(
        groupId: groupId,
        studentId: studentId,
        status: next,
        date: day,
      );
      ref.invalidate(todayAttendanceProvider(groupId));
      ref.invalidate(attendanceWeekTrendProvider(groupId));
      ref.invalidate(attendanceRangeProvider);
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(context.l10n.attendanceUpdated)));
      }
    } on AttendanceWeekendException {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.attendanceWeekendNotAllowed)),
        );
      }
    } on AttendanceFutureDateException {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.attendanceFutureNotAllowed)),
        );
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.commonErrorWithDetails('$e'))),
        );
```

```
    }
  }
}

@override
Widget build(BuildContext context) {
  final groupId = ref.watch(activeGroupIdProvider);
  final l10n = context.l10n;
  final canMarkToday = isAttendanceDateAllowed(DateTime.now());

  if (groupId == null) {
    return Scaffold(
      appBar: AppBar(title: Text(l10n.attendanceMark)),
      body: EmptyState(title: l10n.attendanceNoGroups),
    );
  }

  _resetEditsIfGroupChanged(groupId);

  final members = ref.watch(groupMembersProvider(groupId));
  final saved = ref.watch(todayAttendanceProvider(groupId));
  final memberList = members.valueOrNull ?? const <GroupMember>[];
  final historyStudentId =
      _historyStudentId ??
      (memberList.isEmpty ? null : memberList.first.studentId);

  return Scaffold(
    appBar: AppBar(
      title: Text(l10n.attendanceTitle),
      actions: [
        if (_tabs.index == 0)
          Padding(
            padding: const EdgeInsets.only(right: 16, top: 8, bottom: 8),
            child: Builder(
              builder: (context) {
                final isMono =
                    AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??
                    false;
                return FilledButton(
                  onPressed:
                      !canMarkToday ||
                          _saving ||
                          members.valueOrNull == null
                      ? null
                      : () => _saveAll(
                          groupId,
                          members.value!,
                          saved.valueOrNull ?? const {},
                        ),
                  style: FilledButton.styleFrom(
                    backgroundColor: AppColors.accent(context),
                    foregroundColor: AppColors.onAccent(context),
                    disabledBackgroundColor: AppColors.inkAlpha(
                      context,
                      0.12,
                    ),
                    disabledForegroundColor: AppColors.inkAlpha(
                      context,
                      0.35,
                    ),
                    padding: const EdgeInsets.symmetric(horizontal: 24),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(isMono ? 4 : 12),
                    ),
                  ),
                  child: _saving
                      ? SizedBox(
                          width: 20,
                          height: 20,
                          child: CircularProgressIndicator(
                            strokeWidth: 2,
                            color: AppColors.onAccent(context),
                          ),
```

```
                    )
                  : Text(l10n.commonSave),
            );
          },
        ),
      ),
  ],
  bottom: TabBar(
    controller: _tabs,
    onTap: (_) => setState(() {}),
    tabs: [
      Tab(text: l10n.attendanceMark),
      Tab(text: l10n.attendanceHistory),
    ],
  ),
),
body: TabBarView(
  controller: _tabs,
  children: [
    members.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
      data: (list) {
        final savedMap =
            saved.valueOrNull ?? const <String, AttendanceStatus>{};
        return Column(
          children: [
            const GroupSelector(
              padding: EdgeInsets.fromLTRB(20, 16, 20, 0),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 20, 20, 12),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                children: [
                  Text(
                    DateFormat.yMMMMd().format(DateTime.now()),
                    style: TextStyle(
                      color: AppColors.inkAlpha(context, 0.5),
                      fontSize: 13,
                    ),
                  ),
                  if (!canMarkToday) ...[
                    const SizedBox(height: 8),
                    Text(
                      l10n.attendanceWeekendNotAllowed,
                      style: TextStyle(
                        color: AppColors.inkAlpha(context, 0.55),
                        fontSize: 13,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ],
                  const SizedBox(height: 8),
                  _BulkActions(
                    onPresent: !canMarkToday
                        ? null
                        : () =>
                              _markAll(list, AttendanceStatus.present),
                    onLate: !canMarkToday
                        ? null
                        : () => _markAll(list, AttendanceStatus.late),
                    onAbsent: !canMarkToday
                        ? null
                        : () =>
                              _markAll(list, AttendanceStatus.absent),
                    onExcused: !canMarkToday
                        ? null
                        : () =>
                              _markAll(list, AttendanceStatus.excused),
                  ),
                ],
              ),
```

```
                ),
                const Divider(height: 1),
                Expanded(
                  child: list.isEmpty
                      ? EmptyState(
                          title: l10n.marksNoStudents,
                          icon: Icons.person_off_outlined,
                        )
                      : ListView(
                          padding: EdgeInsets.fromLTRB(
                            20,
                            20,
                            20,
                            shellContentBottomPadding(context),
                          ),
                          children: [
                            PremiumCard(
                              padding: EdgeInsets.zero,
                              child: Column(
                                children: List.generate(list.length, (index) {
                                  final member = list[index];
                                  return Column(
                                    children: [
                                      Padding(
                                        padding: const EdgeInsets.symmetric(
                                          horizontal: 20,
                                          vertical: 16,
                                        ),
                                        child: Row(
                                          children: [
                                            Text(
                                              '${index + 1}',
                                              style: TextStyle(
                                                color: AppColors.inkAlpha(
                                                  context,
                                                  0.3,
                                                ),
                                                fontWeight: FontWeight.bold,
                                              ),
                                            ),
                                            const SizedBox(width: 20),
                                            Expanded(
                                              child: Column(
                                                crossAxisAlignment:
                                                    CrossAxisAlignment.start,
                                                children: [
                                                  Text(
                                                    member.fullName,
                                                    style: TextStyle(
                                                      fontWeight:
                                                          FontWeight.bold,
                                                      fontSize: 15,
                                                      color: AppColors.ink(
```

**lib/screens/teacher/attendance/teacher_attendance_screen.dart — continued (2/3)**

```
                                                        context,
                                                      ),
                                                    ),
                                                  ),
                                                  Text(
                                                    member.username,
                                                    style: TextStyle(
                                                      color:
                                                          AppColors.inkAlpha(
                                                            context,
                                                            0.4,
                                                          ),
                                                      fontSize: 12,
                                                    ),
                                                  ),
                                                ],
                                              ),
```

```
                                    ),
                                    _AttendanceToggle(
                                      status: _statusFor(
                                        member.studentId,
                                        savedMap,
                                      ),
                                      onChanged: !canMarkToday
                                          ? null
                                          : (s) => setState(
                                              () => _edits[member
                                                      .studentId] =
                                                  s,
                                            ),
                                    ),
                                  ],
                                ),
                              ),
                              if (index < list.length - 1)
                                Divider(
                                  height: 1,
                                  indent: 20,
                                  endIndent: 20,
                                  color: AppColors.inkAlpha(
                                    context,
                                    0.12,
                                  ),
                                ),
                            ],
                          );
                        }),
                      ),
                    ),
                    const SizedBox(height: 16),
                    Wrap(
                      spacing: 16,
                      runSpacing: 8,
                      children: [
                        _LegendItem(
                          label: l10n.attendancePresent,
                          color: AppColors.successGreen,
                        ),
                        _LegendItem(
                          label: l10n.attendanceLate,
                          color: AppColors.warmAmber,
                        ),
                        _LegendItem(
                          label: l10n.attendanceAbsent,
                          color: AppColors.errorRed,
                        ),
                        _LegendItem(
                          label: l10n.attendanceExcused,
                          color: const Color(0xFF5B8DB8),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
            ],
          );
        },
      ),
      _TeacherAttendanceHistoryTab(
        groupId: groupId,
        members: memberList,
        month: _month,
        selectedStudentId: historyStudentId,
        onMonthChanged: (m) => setState(() => _month = m),
        onStudentChanged: (id) => setState(() => _historyStudentId = id),
        onDayTap: (day, current) {
          if (historyStudentId == null) return;
          _cycleHistoryDay(
            groupId: groupId,
```

```
                studentId: historyStudentId,
                day: day,
                current: current,
              );
            },
          ),
        ],
      ),
    );
  }
}

class _TeacherAttendanceHistoryTab extends ConsumerWidget {
  const _TeacherAttendanceHistoryTab({
    required this.groupId,
    required this.members,
    required this.month,
    required this.selectedStudentId,
    required this.onMonthChanged,
    required this.onStudentChanged,
    required this.onDayTap,
  });

  final String groupId;
  final List<GroupMember> members;
  final DateTime month;
  final String? selectedStudentId;
  final ValueChanged<DateTime> onMonthChanged;
  final ValueChanged<String?> onStudentChanged;
  final void Function(DateTime day, AttendanceStatus? current) onDayTap;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final l10n = context.l10n;
    if (members.isEmpty) {
      return EmptyState(
        title: l10n.marksNoStudents,
        icon: Icons.person_off_outlined,
      );
    }

    final from = DateTime(month.year, month.month, 1);
    final to = DateTime(month.year, month.month + 1, 0);
    final query = AttendanceQuery(
      groupId: groupId,
      studentId: selectedStudentId,
      from: from,
      to: to,
    );
    final records = ref.watch(attendanceRangeProvider(query));
    final stats = ref.watch(attendanceRangeStatsProvider(query));

    return ListView(
      padding: EdgeInsets.fromLTRB(
        16,
        12,
        16,
        shellContentBottomPadding(context, extra: 16),
      ),
      children: [
        const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
        DropdownButtonFormField<String>(
          // ignore: deprecated_member_use
          value: selectedStudentId,
          decoration: InputDecoration(labelText: l10n.attendanceSelectStudent),
          items: [
            for (final m in members)
              DropdownMenuItem(value: m.studentId, child: Text(m.fullName)),
          ],
          onChanged: onStudentChanged,
        ),
        const SizedBox(height: 8),
        AttendanceMonthHeader(
```

```
          month: month,
          onPrev: () => onMonthChanged(DateTime(month.year, month.month - 1)),
          onNext: () => onMonthChanged(DateTime(month.year, month.month + 1)),
        ),
        const SizedBox(height: 8),
        stats.when(
          data: (s) => AttendanceStatsBar(stats: s),
          loading: () => const SizedBox(height: 56),
          error: (_, _) => const SizedBox.shrink(),
        ),
        const SizedBox(height: 12),
        Text(
          l10n.attendanceTapToCycle,
          style: TextStyle(
            color: AppColors.inkAlpha(context, 0.5),
            fontSize: 12,
          ),
        ),
        const SizedBox(height: 8),
        PremiumCard(
          child: records.when(
            loading: () => const Padding(
              padding: EdgeInsets.all(32),
              child: Center(child: CircularProgressIndicator()),
            ),
            error: (e, _) => ErrorState(message: e.toString()),
            data: (list) {
              final byDate = <DateTime, AttendanceStatus>{
                for (final r in list)
                  DateTime(r.date.year, r.date.month, r.date.day): r.status,
              };
              return Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  AttendanceCalendarGrid(
                    month: month,
                    statusByDate: byDate,
                    editable: selectedStudentId != null,
                    onDayTap: selectedStudentId == null
                        ? null
                        : (day) => onDayTap(
                            day,
                            byDate[DateTime(day.year, day.month, day.day)],
                          ),
                  ),
                  const SizedBox(height: 16),
                  const AttendanceLegendRow(),
                ],
              );
            },
          ),
        ),
      ],
    );
  }
}

class _BulkActions extends StatelessWidget {
  const _BulkActions({
    required this.onPresent,
    required this.onLate,
    required this.onAbsent,
    required this.onExcused,
  });

  final VoidCallback? onPresent;
  final VoidCallback? onLate;
  final VoidCallback? onAbsent;
  final VoidCallback? onExcused;

  static const _excusedBlue = Color(0xFF5B8DB8);

  @override
```

```
Widget build(BuildContext context) {
  final l10n = context.l10n;
  return Wrap(
    spacing: 4,
    runSpacing: 0,
    children: [
      TextButton.icon(
        onPressed: onPresent,
        icon: const Icon(
          Icons.check_circle_outline,
          size: 18,
          color: AppColors.successGreen,
        ),
        label: Text(
          l10n.attendanceAllPresent,
          style: const TextStyle(
            color: AppColors.successGreen,
            fontWeight: FontWeight.bold,
            fontSize: 12,
          ),
        ),
      ),
      TextButton.icon(
        onPressed: onLate,
        icon: const Icon(
          Icons.schedule,
          size: 18,
          color: AppColors.warmAmber,
        ),
        label: Text(
          l10n.attendanceAllLate,
          style: const TextStyle(
            color: AppColors.warmAmber,
            fontWeight: FontWeight.bold,
            fontSize: 12,
          ),
        ),
      ),
      TextButton.icon(
        onPressed: onAbsent,
        icon: const Icon(
          Icons.highlight_off,
          size: 18,
          color: AppColors.errorRed,
        ),
        label: Text(
          l10n.attendanceAllAbsent,
          style: const TextStyle(
            color: AppColors.errorRed,
            fontWeight: FontWeight.bold,
            fontSize: 12,
          ),
        ),
      ),
      TextButton.icon(
        onPressed: onExcused,
        icon: const Icon(
          Icons.event_available_outlined,
          size: 18,
          color: _excusedBlue,
        ),
        label: Text(
          l10n.attendanceAllExcused,
          style: const TextStyle(
            color: _excusedBlue,
            fontWeight: FontWeight.bold,
            fontSize: 12,
          ),
        ),
      ),
    ],
  );
}
```

```
}

class _AttendanceToggle extends StatelessWidget {
  const _AttendanceToggle({required this.status, required this.onChanged});

  final AttendanceStatus? status;
  final ValueChanged<AttendanceStatus>? onChanged;

  static const _excusedBlue = Color(0xFF5B8DB8);

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        _StatusButton(
          label: context.l10n.attendancePresentShort,
          isSelected: status == AttendanceStatus.present,
          activeColor: AppColors.successGreen,
          onTap: onChanged == null
              ? null
              : () => onChanged!(AttendanceStatus.present),
        ),
        const SizedBox(width: 6),
        _StatusButton(
          label: context.l10n.attendanceLateShort,
          isSelected: status == AttendanceStatus.late,
          activeColor: AppColors.warmAmber,
          onTap: onChanged == null
              ? null
              : () => onChanged!(AttendanceStatus.late),
        ),
        const SizedBox(width: 6),
        _StatusButton(
          label: context.l10n.attendanceAbsentShort,
          isSelected: status == AttendanceStatus.absent,
          activeColor: AppColors.errorRed,
          onTap: onChanged == null
              ? null
              : () => onChanged!(AttendanceStatus.absent),
        ),
        const SizedBox(width: 6),
        _StatusButton(
          label: context.l10n.attendanceExcusedShort,
          isSelected: status == AttendanceStatus.excused,
          activeColor: _excusedBlue,
          onTap: onChanged == null
              ? null
              : () => onChanged!(AttendanceStatus.excused),
        ),
      ],
    );
  }
}

class _StatusButton extends StatelessWidget {
  const _StatusButton({
    required this.label,
    required this.isSelected,
    required this.activeColor,
    required this.onTap,
  });

  final String label;
  final bool isSelected;
  final Color activeColor;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: AnimatedContainer(
```

```
        duration: const Duration(milliseconds: 200),
        width: 36,
        height: 36,
        decoration: BoxDecoration(
          color: isSelected ? activeColor : AppColors.controlSurface(context),
          shape: BoxShape.circle,
          border: Border.all(
            color: isSelected ? activeColor : AppColors.inkAlpha(context, 0.25),
            width: 1.5,
          ),
          boxShadow: isSelected
              ? [
                  BoxShadow(
                    color: activeColor.withValues(alpha: 0.3),
                    blurRadius: 8,
                    offset: const Offset(0, 4),
                  ),
                ]
              : [],
        ),
        child: Center(
          child: Text(
            label,
            style: TextStyle(
              color: isSelected
                  ? Colors.white
```

**lib/screens/teacher/attendance/teacher_attendance_screen.dart — continued (3/3)**

```
                  : AppColors.inkAlpha(context, 0.55),
              fontWeight: FontWeight.w800,
              fontSize: 14,
            ),
          ),
        ),
      ),
    );
  }
}

class _LegendItem extends StatelessWidget {
  const _LegendItem({required this.label, required this.color});
  final String label;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Container(
          width: 8,
          height: 8,
          decoration: BoxDecoration(color: color, shape: BoxShape.circle),
        ),
        const SizedBox(width: 6),
        Text(
          label,
          style: TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.w600,
            color: AppColors.inkAlpha(context, 0.6),
          ),
        ),
      ],
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/attendance/teacher_attendance_screen.dart?

2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/dashboard/teacher_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 476. Import count: 16. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:go_router/go_router.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/decor/paper_decor.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/core/theme/app_style_tokens.dart';
- import 'package:mu_study/core/theme/neumorphic.dart';
- import 'package:mu_study/core/utils/time_of_day_greeting.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/notifications/notification_providers.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/teacher/dashboard/teacher_dashboard_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/utils/time_of_day_greeting.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';

class TeacherDashboardScreen extends ConsumerWidget {
  const TeacherDashboardScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final profile = ref.watch(currentProfileProvider);
    final teacher = ref.watch(teacherProfileProvider);
```

```
final groups = ref.watch(myGroupsProvider);
final sphere = teacher.valueOrNull?.sphere;
final l10n = context.l10n;

return Scaffold(
  extendBody: true,
  floatingActionButtonLocation: shellFabLocation(context),
  floatingActionButton: FloatingActionButton(
    child: const Icon(Icons.add),
    onPressed: () {
      showModalBottomSheet(
        context: context,
        // The shell's nav bar floats above its own body, so sheets opened
        // on the shell navigator would slide out underneath it.
        useRootNavigator: true,
        backgroundColor: Colors.transparent,
        builder: (ctx) {
          final isMono =
              AppStyleTokens.maybeOf(ctx)?.isMonochromeWorkspace ?? false;
          final sheetRadius = isMono ? 8.0 : 24.0;
          final iconRadius = isMono ? 4.0 : 20.0;

          Widget actionIcon({
            required Color color,
            required IconData icon,
          }) {
            final fill = AppColors.resolveAccent(ctx, color);
            final isNeu = Neumorphic.isActive(ctx);
            if (isNeu) {
              return NeuIconWell(
                icon: icon,
                color: fill,
                size: 20,
                padding: 10,
                elevation: NeuElevation.raised,
              );
            }
            return Container(
              width: 40,
              height: 40,
              alignment: Alignment.center,
              decoration: BoxDecoration(
                color: fill,
                borderRadius: BorderRadius.circular(iconRadius),
              ),
              child: Icon(icon, color: AppColors.onAccent(ctx), size: 20),
            );
          }

          return Container(
            margin: EdgeInsets.fromLTRB(
              16,
              16,
              16,
              16 + MediaQuery.viewPaddingOf(ctx).bottom,
            ),
            padding: const EdgeInsets.all(24),
            decoration: BoxDecoration(
              color: AppColors.sheetSurface(ctx),
              borderRadius: BorderRadius.circular(sheetRadius),
              border: isMono
                  ? Border.all(color: AppColors.inkAlpha(ctx, 0.12))
                  : null,
            ),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                PremiumSectionHeader(
                  title: l10n.teacherQuickActions,
                  subtitle: l10n.teacherCreateItems,
                ),
                ListTile(
                  leading: actionIcon(
```

```
                  color: AppColors.coffeeBrown,
                  icon: Icons.assignment_outlined,
                ),
                title: Text(
                  l10n.teacherNewAssignment,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    color: AppColors.ink(ctx),
                  ),
                ),
                onTap: () {
                  Navigator.pop(ctx);
                  context.push('/teacher/assignments');
                },
              ),
              ListTile(
                leading: actionIcon(
                  color: AppColors.mutedGold,
                  icon: Icons.campaign_outlined,
                ),
                title: Text(
                  l10n.teacherNewAnnouncement,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    color: AppColors.ink(ctx),
                  ),
                ),
                onTap: () {
                  Navigator.pop(ctx);
                  context.push('/teacher/announcements');
                },
              ),
            ],
          ),
        );
      },
    );
  },
),
body: RefreshIndicator(
  onRefresh: () async {
    ref.invalidate(currentProfileProvider);
    ref.invalidate(teacherProfileProvider);
    ref.invalidate(myGroupsProvider);
  },
  child: ListView(
    padding: EdgeInsets.only(
      left: 20,
      right: 20,
      top: MediaQuery.of(context).padding.top + 24,
      bottom: shellContentBottomPadding(context, hasFab: true),
    ),
    children: [
      profile.when(
        data: (p) => Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Builder(
                  builder: (context) {
                    final sphereColor = AppColors.resolveAccent(
                      context,
                      sphere == Sphere.english
                          ? AppColors.englishAccent
                          : AppColors.mathAccent,
                    );
                    return Container(
                      padding: const EdgeInsets.all(8),
                      decoration: BoxDecoration(
                        color: sphereColor.withValues(alpha: 0.1),
                        borderRadius: BorderRadius.circular(
                          AppStyleTokens.maybeOf(
```

```
                            context,
                          )?.isMonochromeWorkspace ==
                          true
                      ? 6
                      : 8,
                ),
                border:
                    (AppStyleTokens.maybeOf(
                          context,
                        )?.isMonochromeWorkspace ??
                        false)
                    ? Border.all(
                        color: sphereColor.withValues(alpha: 0.2),
                      )
                    : null,
              ),
              child: PaperGlyph(
                icon: sphere == Sphere.english
                    ? Icons.menu_book
                    : Icons.calculate,
                color: sphereColor,
                size: 20,
              ),
            );
          },
        ),
        const SizedBox(width: 12),
        Text(
          l10n.teacherWorkspace,
          style: Theme.of(context).textTheme.bodyLarge?.copyWith(
            color: AppColors.inkAlpha(context, 0.5),
            fontWeight: FontWeight.w700,
          ),
        ),
      ],
    ),
    const SizedBox(height: 8),
    Text(
      l10n.greetingWithName(
        timeOfDayGreeting(l10n),
        p?.fullName.split(' ').first ?? p?.username ?? '',
      ),
      style: Theme.of(context).textTheme.headlineSmall?.copyWith(
        fontWeight: FontWeight.w800,
        letterSpacing: -0.5,
      ),
    ),
  ),
],
),
loading: () => Center(child: CircularProgressIndicator()),
error: (_, _) => SizedBox.shrink(),
),
SizedBox(height: 32),
groups.when(
  data: (g) => Row(
    children: [
      _StatCard(
        label: l10n.teacherActiveGroups,
        value: '${g.length}',
        icon: Icons.groups_outlined,
        color: AppColors.accent(context),
      ),
      const SizedBox(width: 12),
      _StatCard(
        label: l10n.teacherSubject,
        value: sphere?.name.toUpperCase() ?? '—',
        icon: Icons.subject_outlined,
        color: AppColors.mutedGold,
      ),
    ],
  ),
  loading: () => const SizedBox.shrink(),
  error: (_, _) => const SizedBox.shrink(),
```

```
),
const SizedBox(height: 24),
ref
    .watch(notificationsProvider)
    .when(
      data: (list) {
        final unread = list.where((n) => !n.isRead).length;
        return PremiumCard(
          onTap: () => context.push('/teacher/notifications'),
          child: Row(
            children: [
              NeuIconWell(
                icon: Icons.notifications_outlined,
                color: AppColors.accent(context),
                child: Badge(
                  isLabelVisible: unread > 0,
                  label: Text(
                    '$unread',
                    style: const TextStyle(fontSize: 10),
                  ),
                  backgroundColor: AppColors.danger(context),
                  child: Icon(
                    Icons.notifications_outlined,
                    color: AppColors.accent(context),
                  ),
                ),
              ),
              const SizedBox(width: 16),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      l10n.notificationsCardTitle,
                      style: const TextStyle(
                        fontWeight: FontWeight.w800,
                        fontSize: 16,
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      unread > 0
                          ? l10n.unreadAlerts(unread)
                          : l10n.upToDate,
                      style: TextStyle(
                        fontSize: 13,
                        color: AppColors.inkAlpha(context, 0.55),
                      ),
                    ),
                  ],
                ),
              ),
              Icon(
                Icons.chevron_right_rounded,
                color: AppColors.inkAlpha(context, 0.3),
              ),
            ],
          ),
        );
      },
      loading: () => const SizedBox.shrink(),
      error: (_, _) => const SizedBox.shrink(),
    ),
const SizedBox(height: 24),
PremiumSectionHeader(
  title: l10n.teacherDailyTasks,
  subtitle: l10n.teacherNeedsAttention,
),
_QuickActionCard(
  title: l10n.attendanceMark,
  subtitle: l10n.attendanceDailyTracking,
  icon: Icons.how_to_reg_rounded,
  onTap: () => context.push('/teacher/attendance'),
```

```
              accentColor: AppColors.mutedGold,
            ),
            _QuickActionCard(
              title: l10n.teacherGradebook,
              subtitle: l10n.teacherRecordMarks,
              icon: Icons.grade_rounded,
              onTap: () => context.push('/teacher/marks'),
              accentColor: AppColors.mathAccent,
            ),
            _QuickActionCard(
              title: l10n.teacherReviewSubmissions,
              subtitle: l10n.teacherCheckAssignments,
              icon: Icons.fact_check_rounded,
              onTap: () => context.push('/teacher/submissions'),
              accentColor: AppColors.successGreen,
            ),
            _QuickActionCard(
              title: l10n.assignmentManage,
              subtitle: l10n.assignmentManageSubtitle,
```

**lib/screens/teacher/dashboard/teacher_dashboard_screen.dart — continued (2/2)**

```
              icon: Icons.assignment_rounded,
              onTap: () => context.push('/teacher/assignments'),
              accentColor: AppColors.coffeeBrown,
            ),
            _QuickActionCard(
              title: l10n.announcementsTitle,
              subtitle: l10n.announcementsShareUpdates,
              icon: Icons.campaign_rounded,
              onTap: () => context.push('/teacher/announcements'),
              accentColor: AppColors.englishAccent,
            ),
            _QuickActionCard(
              title: l10n.teacherPerformanceAnalytics,
              subtitle: l10n.teacherTrackProgress,
              icon: Icons.analytics_rounded,
              onTap: () => context.push('/teacher/analytics'),
              accentColor: AppColors.warmAmber,
            ),
          ],
        ),
      ),
    );
  }
}

class _StatCard extends StatelessWidget {
  const _StatCard({
    required this.label,
    required this.value,
    required this.icon,
    required this.color,
  });

  final String label;
  final String value;
  final IconData icon;
  final Color color;

  @override
  Widget build(BuildContext context) {
    final tone = AppColors.resolveAccent(context, color);
    final isNeu = Neumorphic.isActive(context);
    return Expanded(
      child: PremiumCard(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
```

```
                Text(
                  label,
                  style: Theme.of(context).textTheme.labelSmall?.copyWith(
                    letterSpacing: 1.0,
                    fontWeight: FontWeight.w800,
                    color: AppColors.inkAlpha(context, 0.4),
                  ),
                ),
                if (isNeu)
                  NeuIconWell(icon: icon, color: tone, size: 18, padding: 8)
                else
                  PaperGlyph(
                    icon: icon,
                    size: 18,
                    color: tone.withValues(alpha: 0.5),
                  ),
              ],
            ),
            const SizedBox(height: 12),
            Text(
              value,
              style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                fontWeight: FontWeight.w800,
                color: tone,
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _QuickActionCard extends StatelessWidget {
  const _QuickActionCard({
    required this.title,
    required this.subtitle,
    required this.icon,
    required this.onTap,
    required this.accentColor,
  });

  final String title;
  final String subtitle;
  final IconData icon;
  final VoidCallback onTap;
  final Color accentColor;

  @override
  Widget build(BuildContext context) {
    final tone = AppColors.resolveAccent(context, accentColor);

    return PremiumCard(
      onTap: onTap,
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
      child: Row(
        children: [
          NeuIconWell(icon: icon, color: tone, size: 24),
          const SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  title,
                  style: const TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 16,
                  ),
                ),
                Text(
                  subtitle,
                  style: TextStyle(
```

```
                    color: AppColors.inkAlpha(context, 0.5),
                    fontSize: 13,
                  ),
                ),
              ],
            ),
          ),
          Icon(
            Icons.chevron_right_rounded,
            size: 20,
            color: AppColors.inkAlpha(context, 0.2),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/dashboard/teacher_dashboard_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/homework/teacher_homework_pack_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 420. Import count: 13. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/core/homework/assignment_templates.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/homework/assignment_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/homework/teacher_homework_pack_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_templates.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class TeacherHomeworkPackScreen extends ConsumerStatefulWidget {
  const TeacherHomeworkPackScreen({super.key, required this.assignmentId});

  final String assignmentId;

  @override
  ConsumerState<TeacherHomeworkPackScreen> createState() =>
      _TeacherHomeworkPackScreenState();
}

class _TeacherHomeworkPackScreenState
    extends ConsumerState<TeacherHomeworkPackScreen> {
  Future<void> _saveAsTemplate(Assignment? assignment) async {
    if (assignment == null) return;
    final teacher = await ref.read(teacherProfileProvider.future);
```

```
    if (teacher == null || !mounted) return;
    final l10n = context.l10n;
    try {
      await saveAssignmentAsTemplate(
        teacherId: teacher.id,
        assignment: assignment,
      );
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.teacherTemplateSaved)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _showTaskDialog({TaskItem? task, int currentCount = 0}) async {
    final assignment = ref
        .read(assignmentDetailProvider(widget.assignmentId))
        .valueOrNull;
    if (assignment?.type == AssignmentType.staticAssignment) return;

    final isEditing = task != null;
    final titleController = TextEditingController(text: task?.title ?? '');
    final descController = TextEditingController(text: task?.description ?? '');
    final linkController = TextEditingController(text: task?.linkUrl ?? '');
    final attachmentController = TextEditingController(
      text: task?.attachmentUrl ?? '',
    );

    if (!mounted) return;
    final l10n = context.l10n;

    await showDialog<void>(
      context: context,
      builder: (ctx) => AlertDialog(
        title: Text(isEditing ? l10n.teacherEditTask : l10n.teacherAddTask),
        content: SingleChildScrollView(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              TextField(
                controller: titleController,
                decoration: InputDecoration(
                  labelText: l10n.commonTitle,
                  hintText: l10n.teacherTaskTitleHint,
                ),
              ),
              const SizedBox(height: 16),
              TextField(
                controller: descController,
                decoration: InputDecoration(labelText: l10n.commonDescription),
                maxLines: 3,
              ),
              const SizedBox(height: 16),
              TextField(
                controller: linkController,
                decoration: InputDecoration(
                  labelText: l10n.teacherReferenceLinkOptional,
                  hintText: l10n.teacherLinkHint,
                  prefixIcon: const Icon(Icons.link),
                ),
              ),
              const SizedBox(height: 16),
              TextField(
                controller: attachmentController,
                decoration: InputDecoration(
                  labelText: l10n.teacherAttachmentUrlOptional,
                  hintText: l10n.teacherAttachmentHint,
                  prefixIcon: const Icon(Icons.attachment),
```

```
            ),
          ),
        ],
      ),
    ),
    actions: [
      if (isEditing)
        TextButton(
          onPressed: () async {
            final confirm = await showDialog<bool>(
              context: ctx,
              builder: (c) => AlertDialog(
                title: Text(l10n.teacherDeleteTask),
                content: Text(l10n.commonCannotUndo),
                actions: [
                  TextButton(
                    onPressed: () => Navigator.pop(c, false),
                    child: Text(l10n.commonCancel),
                  ),
                  TextButton(
                    onPressed: () => Navigator.pop(c, true),
                    child: Text(
                      l10n.commonDelete,
                      style: const TextStyle(color: Colors.red),
                    ),
                  ),
                ],
              ),
            );
            if (confirm == true) {
              await supabase.from('tasks').delete().eq('id', task.id);
              if (ctx.mounted) Navigator.pop(ctx);
              ref.invalidate(tasksProvider(widget.assignmentId));
            }
          },
          child: Text(
            l10n.commonDelete,
            style: const TextStyle(color: Colors.red),
          ),
        ),
      TextButton(
        onPressed: () => Navigator.pop(ctx),
        child: Text(l10n.commonCancel),
      ),
      ElevatedButton(
        onPressed: () async {
          if (titleController.text.trim().isEmpty) return;

          final Map<String, dynamic> data = {
            'assignment_id': widget.assignmentId,
            'title': titleController.text.trim(),
            'description': descController.text.trim(),
            'link_url': linkController.text.trim().isEmpty
                ? null
                : linkController.text.trim(),
            'attachment_url': attachmentController.text.trim().isEmpty
                ? null
                : attachmentController.text.trim(),
          };

          if (isEditing) {
            await supabase.from('tasks').update(data).eq('id', task.id);
          } else {
            data['sort_order'] = currentCount;
            await supabase.from('tasks').insert(data);
          }

          if (ctx.mounted) Navigator.pop(ctx);
          ref.invalidate(tasksProvider(widget.assignmentId));
        },
        child: Text(isEditing ? l10n.commonSave : l10n.teacherAddTask),
      ),
    ],
```

```
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final assignment = ref.watch(assignmentDetailProvider(widget.assignmentId));
    final tasks = ref.watch(tasksProvider(widget.assignmentId));
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.teacherHomeworkPackTasks),
        actions: [
          IconButton(
            tooltip: l10n.teacherSaveAsTemplate,
            onPressed: () => _saveAsTemplate(assignment.valueOrNull),
            icon: const Icon(Icons.bookmark_add_outlined),
          ),
        ],
      ),
      floatingActionButtonLocation: shellFabLocation(context),
      floatingActionButton: assignment.maybeWhen(
        data: (a) {
          if (a == null || a.type != AssignmentType.homeworkPack) {
            return null;
          }
          return tasks.maybeWhen(
            data: (items) => FloatingActionButton.extended(
              onPressed: () => _showTaskDialog(currentCount: items.length),
              icon: const Icon(Icons.add),
              label: Text(l10n.teacherAddTask),
            ),
            orElse: () => null,
          );
        },
        orElse: () => null,
      ),
      body: assignment.when(
        data: (a) {
          if (a == null) {
            return EmptyState(title: l10n.assignmentNotFound);
          }
          if (a.type != AssignmentType.homeworkPack) {
            return EmptyState(
              title: l10n.staticAssignment,
              subtitle: l10n.assignmentTypeHintStatic,
            );
          }
          return RefreshIndicator(
            onRefresh: () async {
              ref.invalidate(assignmentDetailProvider(widget.assignmentId));
              ref.invalidate(tasksProvider(widget.assignmentId));
            },
            child: ListView(
              padding: EdgeInsets.fromLTRB(
                20,
                24,
                20,
                shellContentBottomPadding(context, hasFab: true, extra: 24),
              ),
              children: [
                Container(
                  padding: const EdgeInsets.all(24),
                  decoration: BoxDecoration(
                    color: AppColors.mutedGold,
                    borderRadius: BorderRadius.circular(20),
                    boxShadow: [
                      BoxShadow(
                        color: AppColors.mutedGold.withValues(alpha: 0.2),
                        blurRadius: 20,
                        offset: const Offset(0, 10),
                      ),
```

```
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            l10n.homeworkPack.toUpperCase(),
            style: const TextStyle(
              color: Colors.white,
              fontSize: 10,
              fontWeight: FontWeight.w800,
              letterSpacing: 1.2,
            ),
          ),
          const SizedBox(height: 8),
          Text(
            a.title,
            style: Theme.of(context).textTheme.headlineSmall
                ?.copyWith(
                  color: AppColors.ink(context),
                  fontWeight: FontWeight.w800,
                ),
          ),
          if (a.description.isNotEmpty) ...[
            const SizedBox(height: 12),
            Text(
              a.description,
              style: TextStyle(
                color: AppColors.inkAlpha(context, 0.8),
                fontSize: 14,
                height: 1.5,
              ),
            ),
          ],
        ],
      ),
    ),
  ),
  const SizedBox(height: 32),
  PremiumSectionHeader(
    title: l10n.teacherTasksContent,
    subtitle: l10n.teacherManagePack,
  ),
  tasks.when(
    data: (items) {
      if (items.isEmpty) {
        return Padding(
          padding: const EdgeInsets.only(top: 24.0),
          child: Text(
            l10n.teacherNoTasksYet,
            style: TextStyle(
              color: AppColors.inkAlpha(context, 0.5),
              fontStyle: FontStyle.italic,
            ),
          ),
        );
      }
      return ListView.builder(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        itemCount: items.length,
        itemBuilder: (context, index) {
          final t = items[index];
          final hasLink =
              t.linkUrl != null && t.linkUrl!.isNotEmpty;
          final hasAttachment =
              t.attachmentUrl != null &&
              t.attachmentUrl!.isNotEmpty;

          return PremiumCard(
            onTap: () => _showTaskDialog(task: t),
            padding: const EdgeInsets.all(16),
            child: Row(
              children: [
```

```
                        Container(
                          width: 32,
                          height: 32,
                          decoration: BoxDecoration(
                            color: AppColors.softWash(
                              context,
                              lightAlpha: 0.5,
                            ),
                            shape: BoxShape.circle,
                          ),
                          child: Center(
                            child: Text(
                              '${index + 1}',
                              style: TextStyle(
                                fontWeight: FontWeight.w800,
                                color: AppColors.ink(context),
                                fontSize: 12,
                              ),
                            ),
                          ),
                        ),
                        const SizedBox(width: 16),
                        Expanded(
                          child: Column(
                            crossAxisAlignment: CrossAxisAlignment.start,
                            children: [
                              Text(
                                t.title,
                                style: const TextStyle(
                                  fontWeight: FontWeight.w700,
                                  fontSize: 16,
                                ),
                              ),
```

**lib/screens/teacher/homework/teacher_homework_pack_screen.dart — continued (2/2)**

```
                              if (t.description.isNotEmpty)
                                Text(
                                  t.description,
                                  maxLines: 2,
                                  overflow: TextOverflow.ellipsis,
                                  style: TextStyle(
                                    color: AppColors.inkAlpha(
                                      context,
                                      0.6,
                                    ),
                                    fontSize: 13,
                                  ),
                                ),
                              if (hasLink || hasAttachment)
                                Padding(
                                  padding: const EdgeInsets.only(
                                    top: 8.0,
                                  ),
                                  child: Row(
                                    children: [
                                      if (hasLink)
                                        const Icon(
                                          Icons.link,
                                          size: 14,
                                          color: AppColors.successGreen,
                                        ),
                                      if (hasLink && hasAttachment)
                                        const SizedBox(width: 8),
                                      if (hasAttachment)
                                        const Icon(
                                          Icons.attachment,
                                          size: 14,
                                          color: AppColors.successGreen,
                                        ),
                                    ],
                                  ),
                                ),
```

```
                                ],
                              ),
                            ),
                            Icon(
                              Icons.edit_outlined,
                              size: 18,
                              color: AppColors.inkAlpha(context, 0.3),
                            ),
                          ],
                        ),
                      );
                    },
                  );
                },
                loading: () =>
                    const Center(child: CircularProgressIndicator()),
                error: (e, _) => ErrorState(message: e.toString()),
              ),
              const SizedBox(height: 100),
            ],
          ),
        );
      },
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
    ),
  );
 }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/homework/teacher_homework_pack_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/homework/teacher_submissions_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 402. Import count: 17. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/attachments/voice_note_field.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/core/homework/feedback_reuse.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/homework/teacher_submissions_provider.dart';
- import 'package:mu_study/providers/marks/marks_providers.dart';
- import 'package:mu_study/supabase/supabase_client.dart';
- import 'package:url_launcher/url_launcher.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/teacher/homework/teacher_submissions_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/attachments/voice_note_field.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/homework/feedback_reuse.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/teacher_submissions_provider.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:url_launcher/url_launcher.dart';

class TeacherSubmissionsScreen extends ConsumerWidget {
  const TeacherSubmissionsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
```

```
    final submissions = ref.watch(teacherSubmissionsProvider);
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.teacherSubmissions)),
      body: submissions.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (rows) {
          if (rows.isEmpty) {
            return EmptyState(
              title: l10n.teacherNoSubmissions,
              subtitle: l10n.teacherNoSubmissionsSubtitle,
              icon: Icons.fact_check_outlined,
            );
          }
          return RefreshIndicator(
            onRefresh: () async => ref.invalidate(teacherSubmissionsProvider),
            child: ListView.builder(
              padding: EdgeInsets.fromLTRB(
                20,
                20,
                20,
                shellContentBottomPadding(context),
              ),
              itemCount: rows.length,
              itemBuilder: (_, i) {
                final row = rows[i];
                final score = row['score'] == null
                    ? null
                    : double.tryParse('${row['score']}');
                return PremiumCard(
                  onTap: () => _review(context, ref, row),
                  padding: const EdgeInsets.symmetric(
                    horizontal: 16,
                    vertical: 14,
                  ),
                  child: Row(
                    children: [
                      Expanded(
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: [
                            Text(
                              row['task_title'] as String? ?? l10n.taskTitle,
                              style: const TextStyle(
                                fontWeight: FontWeight.w700,
                                fontSize: 15,
                              ),
                            ),
                            const SizedBox(height: 2),
                            Text(
                              '${row['student_name']} · ${row['status']}',
                              style: TextStyle(
                                fontSize: 12,
                                color: AppColors.inkAlpha(context, 0.5),
                              ),
                            ),
                          ],
                        ),
                      ),
                      PercentBadge(percent: score, compact: true),
                    ],
                  ),
                );
              },
            ),
          );
        },
      ),
    );
  }
```

```
  Future<void> _review(
    BuildContext context,
    WidgetRef ref,
    Map<String, dynamic> row,
  ) async {
    final teacher = await ref.read(teacherProfileProvider.future);
    if (!context.mounted) return;
    final saved = await showModalBottomSheet<bool>(
      context: context,
      useRootNavigator: true,
      isScrollControlled: true,
      showDragHandle: true,
      builder: (ctx) {
        final bottom = MediaQuery.viewInsetsOf(ctx).bottom;
        return Padding(
          padding: EdgeInsets.only(bottom: bottom),
          child: _ReviewSubmissionDialog(row: row, teacherId: teacher?.id),
        );
      },
    );

    if (saved != true || !context.mounted) return;

    ref.invalidate(teacherSubmissionsProvider);
    final groupId = ref.read(activeGroupIdProvider);
    if (groupId != null) {
      ref.invalidate(groupMarksProvider(groupId));
      ref.invalidate(groupMarkAveragesProvider(groupId));
    }
  }
}

class _ReviewSubmissionDialog extends StatefulWidget {
  const _ReviewSubmissionDialog({required this.row, this.teacherId});

  final Map<String, dynamic> row;
  final String? teacherId;

  @override
  State<_ReviewSubmissionDialog> createState() =>
      _ReviewSubmissionDialogState();
}

class _ReviewSubmissionDialogState extends State<_ReviewSubmissionDialog> {
  late final TextEditingController _scoreController;
  late final TextEditingController _feedbackController;
  late final FocusNode _scoreFocus;
  bool _saving = false;
  bool _applyToOthers = false;
  List<String> _snippets = const [];

  @override
  void initState() {
    super.initState();
    final score = widget.row['score'];
    _scoreController = TextEditingController(
      text: score == null ? '' : '$score',
    );
    _feedbackController = TextEditingController(
      text: widget.row['feedback'] as String? ?? '',
    );
    _scoreFocus = FocusNode();
    _loadSnippets();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (mounted) _scoreFocus.requestFocus();
    });
  }

  Future<void> _loadSnippets() async {
    final teacherId = widget.teacherId;
    if (teacherId == null) return;
    final snippets = await loadFeedbackSnippets(teacherId: teacherId);
    if (!mounted) return;
```

```
    setState(() => _snippets = snippets);
  }

  @override
  void dispose() {
    _scoreFocus.dispose();
    _scoreController.dispose();
    _feedbackController.dispose();
    super.dispose();
  }

  Future<void> _save() async {
    if (_saving) return;

    final raw = _scoreController.text.trim().replaceAll(',', '.');
    final score = raw.isEmpty ? null : double.tryParse(raw);
    if (raw.isNotEmpty && (score == null || score < 0 || score > 100)) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(context.l10n.teacherScoreRange)));
      return;
    }

    setState(() => _saving = true);
    final feedback = _feedbackController.text.trim();
    try {
      await supabase
          .from('submissions')
          .update({
            'score': score,
            'feedback': feedback,
            'status': score == null ? 'submitted' : 'reviewed',
          })
          .eq('id', widget.row['id']);

      final teacherId = widget.teacherId;
      if (teacherId != null && feedback.isNotEmpty) {
        await rememberFeedbackSnippet(teacherId: teacherId, feedback: feedback);
      }

      var applied = 0;
      if (_applyToOthers && feedback.isNotEmpty) {
        applied = await applyFeedbackToOtherPendingForSameTask(
          submissionId: widget.row['id'] as String,
          feedback: feedback,
        );
      }

      if (!mounted) return;
      if (applied > 0) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(context.l10n.teacherFeedbackAppliedCount(applied)),
          ),
        );
      }
      Navigator.of(context).pop(true);
    } catch (e) {
      if (!mounted) return;
      setState(() => _saving = false);
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final fileUrl = widget.row['file_url'] as String?;
    final voiceUrl = widget.row['voice_url'] as String?;
    final scheme = Theme.of(context).colorScheme;
```

```
    return SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.fromLTRB(20, 8, 20, 20),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text(
              widget.row['student_name'] as String? ??
                  l10n.teacherReviewSubmission,
              style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 18),
            ),
            const SizedBox(height: 4),
            Text(
              widget.row['task_title'] as String? ?? '',
              style: TextStyle(
                fontWeight: FontWeight.w600,
                color: AppColors.inkAlpha(context, 0.6),
              ),
            ),
            const SizedBox(height: 14),
            Text(
              widget.row['text_answer'] as String? ??
                  l10n.teacherNoWrittenAnswer,
              style: TextStyle(
                color: AppColors.inkAlpha(context, 0.75),
                height: 1.4,
              ),
            ),
            if (voiceUrl != null && voiceUrl.isNotEmpty) ...[
              const SizedBox(height: 12),
              VoiceNotePlayer(url: voiceUrl),
            ],
            if (fileUrl != null) ...[
              const SizedBox(height: 12),
              OutlinedButton.icon(
                onPressed: () => launchUrl(
                  Uri.parse(fileUrl),
                  mode: LaunchMode.externalApplication,
                ),
                icon: const Icon(Icons.attach_file, size: 18),
                label: Text(l10n.teacherOpenAttachment),
              ),
            ],
            const SizedBox(height: 16),
            TextField(
              controller: _scoreController,
              focusNode: _scoreFocus,
              enabled: !_saving,
              keyboardType: const TextInputType.numberWithOptions(
                decimal: true,
              ),
              textInputAction: TextInputAction.next,
              inputFormatters: [
                FilteringTextInputFormatter.allow(RegExp(r'[0-9.,]')),
                LengthLimitingTextInputFormatter(6),
              ],
              decoration: InputDecoration(
                labelText: l10n.commonScore,
                suffixText: '%',
                helperText: l10n.teacherScoreHelper,
              ),
            ),
            const SizedBox(height: 12),
            TextField(
              controller: _feedbackController,
              enabled: !_saving,
              maxLines: 3,
              textCapitalization: TextCapitalization.sentences,
              decoration: InputDecoration(labelText: l10n.commonFeedback),
            ),
            if (_snippets.isNotEmpty) ...[
              const SizedBox(height: 10),
```

```
  Text(
    l10n.teacherRecentFeedback,
    style: TextStyle(
      fontSize: 12,
      fontWeight: FontWeight.w600,
      color: AppColors.inkAlpha(context, 0.55),
    ),
  ),
  const SizedBox(height: 6),
  Wrap(
    spacing: 6,
    runSpacing: 6,
    children: _snippets.take(8).map((s) {
      final label = s.length > 42 ? '${s.substring(0, 42)}…' : s;
      return ActionChip(
        label: Text(label, style: const TextStyle(fontSize: 12)),
        onPressed: _saving
            ? null
            : () {
                _feedbackController.text = s;
                _feedbackController.selection =
                    TextSelection.collapsed(offset: s.length);
              },
      );
    }).toList(),
  ),
],
const SizedBox(height: 4),
CheckboxListTile(
  contentPadding: EdgeInsets.zero,
  dense: true,
  value: _applyToOthers,
  onChanged: _saving
      ? null
      : (v) => setState(() => _applyToOthers = v ?? false),
  controlAffinity: ListTileControlAffinity.leading,
  title: Text(
    l10n.teacherApplyFeedbackToOthers,
    style: const TextStyle(fontSize: 13),
  ),
),
const SizedBox(height: 12),
Row(
  children: [
    Expanded(
      child: TextButton(
        onPressed: _saving
            ? null
            : () => Navigator.of(context).pop(false),
        child: Text(l10n.commonCancel),
      ),
    ),
    const SizedBox(width: 8),
    Expanded(
      child: FilledButton(
        onPressed: _saving ? null : _save,
        style: FilledButton.styleFrom(
          backgroundColor: AppColors.accent(context),
          foregroundColor:
              Theme.of(context).brightness == Brightness.dark
              ? AppColors.darkChocolate
              : Colors.white,
          padding: const EdgeInsets.symmetric(vertical: 14),
        ),
        child: _saving
            ? SizedBox(
                width: 18,
                height: 18,
                child: CircularProgressIndicator(
                  strokeWidth: 2,
                  color: scheme.onPrimary,
                ),
              )
```

```
                        : Text(
                            l10n.commonSave,
```

**lib/screens/teacher/homework/teacher_submissions_screen.dart — continued (2/2)**

```
                              style: const TextStyle(fontWeight: FontWeight.w700),
                            ),
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
        );
    }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/homework/teacher_submissions_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/marks/teacher_marks_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 481. Import count: 21. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/layouts/app_shell.dart';
- import 'package:mu_study/components/inputs/group_selector.dart';
- import 'package:mu_study/components/inputs/load_more_button.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/components/marks/term_selector.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/marks/marks_csv_export.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/marks/marks_providers.dart';
- import 'package:mu_study/screens/teacher/marks/widgets/bulk_mark_sheet.dart';
- … and 3 more imports

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/teacher/marks/teacher_marks_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/components/marks/term_selector.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/marks/marks_csv_export.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/teacher/marks/widgets/bulk_mark_sheet.dart';
import 'package:mu_study/screens/teacher/marks/widgets/mark_editor_sheet.dart';
import 'package:mu_study/screens/teacher/marks/widgets/mark_row.dart';
```

```
import 'package:mu_study/screens/teacher/marks/widgets/student_marks_sheet.dart';

class TeacherMarksScreen extends ConsumerWidget {
  const TeacherMarksScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groupId = ref.watch(activeGroupIdProvider);
    final l10n = context.l10n;

    if (groupId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.teacherGradebook)),
        body: EmptyState(
          title: l10n.teacherNoGroups,
          subtitle: l10n.teacherNoGroupsSubtitle,
          icon: Icons.groups_outlined,
        ),
      );
    }

    final members = ref.watch(groupMembersProvider(groupId));
    final averages = ref.watch(groupMarkAveragesProvider(groupId));
    final marks = ref.watch(groupMarksProvider(groupId));
    final markCount =
        ref.watch(groupMarkCountProvider(groupId)).valueOrNull ?? 0;
    final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];
    final groupTitle = groups
        .where((g) => g.id == groupId)
        .map((g) => g.title)
        .firstOrNull;

    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.teacherGradebook),
        actions: [
          IconButton(
            tooltip: l10n.marksExportCsv,
            onPressed: () =>
                _export(context, ref, groupId, groupTitle ?? 'group'),
            icon: const Icon(Icons.ios_share_rounded),
          ),
        ],
      ),
      floatingActionButtonLocation: shellFabLocation(context),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _openBulkEntry(context, ref, groupId),
        icon: const Icon(Icons.playlist_add_rounded),
        label: Text(l10n.teacherGradeClass),
      ),
      body: RefreshIndicator(
        onRefresh: () async => _refresh(ref, groupId),
        child: ListView(
          padding: EdgeInsets.only(
            left: 20,
            right: 20,
            top: 8,
            bottom: shellContentBottomPadding(context, hasFab: true),
          ),
          children: [
            const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
            const TermSelector(padding: EdgeInsets.only(bottom: 12)),
            _ClassSummaryCard(
              averages: averages.valueOrNull ?? const {},
              studentCount: members.valueOrNull?.length ?? 0,
              markCount: markCount,
            ),
            PremiumSectionHeader(
              title: l10n.marksStudentsHeader,
              subtitle: l10n.marksStudentsSubtitle,
            ),
            members.when(
              data: (list) {
```

```
        if (list.isEmpty) {
          return EmptyState(
            title: l10n.marksNoStudents,
            subtitle: l10n.marksNoStudentsSubtitle,
            icon: Icons.person_off_outlined,
          );
        }
        final avg = averages.valueOrNull ?? const <String, double>{};
        return Column(
          children: list
              .map(
                (m) => _StudentRow(
                  member: m,
                  average: avg[m.studentId],
                  onTap: () =>
                      _openStudentSheet(context, ref, groupId, m),
                ),
              )
              .toList(),
        );
      },
      loading: () => const Center(
        child: Padding(
          padding: EdgeInsets.all(24),
          child: CircularProgressIndicator(),
        ),
      ),
      error: (e, _) => ErrorState(message: e.toString()),
    ),
    const SizedBox(height: 8),
    PremiumSectionHeader(
      title: l10n.marksRecent,
      subtitle: l10n.marksRecentSubtitle,
    ),
    marks.when(
      data: (page) {
        if (page.items.isEmpty) {
          return EmptyState(
            title: l10n.marksNoRecorded,
            subtitle: l10n.marksNoRecordedSubtitle,
            icon: Icons.grade_outlined,
          );
        }
        return Column(
          children: [
            ...page.items.map(
              (mark) => MarkRow(
                mark: mark,
                onEdit: () => _openMarkEditor(
                  context,
                  ref,
                  groupId: groupId,
                  studentId: mark.studentId,
                  studentName: mark.studentName ?? l10n.commonStudent,
                  existing: mark,
                ),
                onDelete: () => _confirmDelete(context, ref, mark),
              ),
            ),
            LoadMoreButton(
              hasMore: page.hasMore,
              loading: page.loadingMore,
              onPressed: () => ref
                  .read(groupMarksProvider(groupId).notifier)
                  .loadMore(),
            ),
          ],
        );
      },
      loading: () => const SizedBox.shrink(),
      error: (e, _) => ErrorState(message: e.toString()),
    ),
  ],
```

```
        ),
      ),
    );
  }

  void _refresh(WidgetRef ref, String groupId) {
    ref.invalidate(groupMembersProvider(groupId));
    ref.invalidate(groupMarkAveragesProvider(groupId));
    ref.invalidate(groupMarkCountProvider(groupId));
    ref.invalidate(groupMarksProvider(groupId));
    ref.invalidate(memberMarksProvider);
  }

  Future<void> _export(
    BuildContext context,
    WidgetRef ref,
    String groupId,
    String groupTitle,
  ) async {
    final l10n = context.l10n;
    try {
      await exportGroupMarksCsv(
        groupId: groupId,
        groupTitle: groupTitle,
        term: ref.read(selectedMarkTermProvider),
      );
      if (context.mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(l10n.marksExportDone)));
      }
    } catch (e) {
      if (context.mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(l10n.commonExportFailed('$e'))));
      }
    }
  }

  Future<void> _openStudentSheet(
    BuildContext context,
    WidgetRef ref,
    String groupId,
    GroupMember member,
  ) async {
    final result = await showModalBottomSheet<Object?>(
      context: context,
      isScrollControlled: true,
      // The shell's nav bar floats above its own body, so a sheet opened on
      // the shell navigator would come up underneath it.
      useRootNavigator: true,
      backgroundColor: Colors.transparent,
      builder: (_) => StudentMarksSheet(groupId: groupId, member: member),
    );

    if (context.mounted && result != null) {
      await _openMarkEditor(
        context,
        ref,
        groupId: groupId,
        studentId: member.studentId,
        studentName: member.fullName,
        existing: result is Mark ? result : null,
      );
    }
    _refresh(ref, groupId);
  }

  Future<void> _openBulkEntry(
    BuildContext context,
    WidgetRef ref,
    String groupId,
```

```
) async {
  final members = await ref.read(groupMembersProvider(groupId).future);
  if (!context.mounted) return;
  if (members.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(context.l10n.marksGroupHasNoStudents)),
    );
    return;
  }
  await showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    useRootNavigator: true,
    backgroundColor: Colors.transparent,
    builder: (_) => BulkMarkSheet(groupId: groupId, members: members),
  );
  _refresh(ref, groupId);
}

Future<void> _openMarkEditor(
  BuildContext context,
  WidgetRef ref, {
  required String groupId,
  required String studentId,
  required String studentName,
  Mark? existing,
}) async {
  await showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    useRootNavigator: true,
    backgroundColor: Colors.transparent,
    builder: (_) => MarkEditorSheet(
      groupId: groupId,
      studentId: studentId,
      studentName: studentName,
      existing: existing,
    ),
  );
  _refresh(ref, groupId);
}

Future<void> _confirmDelete(
  BuildContext context,
  WidgetRef ref,
  Mark mark,
) async {
  if (mark.isFromSubmission) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(context.l10n.marksFromSubmissionDelete)),
    );
    return;
  }

  final confirmed = await showDialog<bool>(
    context: context,
    builder: (ctx) {
      final l10n = context.l10n;
      return AlertDialog(
        title: Text(l10n.marksDeleteTitle),
        content: Text(l10n.marksDeleteBody(mark.title)),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(ctx, false),
            child: Text(l10n.commonCancel),
          ),
          FilledButton(
            style: FilledButton.styleFrom(
              backgroundColor: AppColors.errorRed,
            ),
            onPressed: () => Navigator.pop(ctx, true),
            child: Text(l10n.commonDelete),
          ),
```

```
          ],
        );
      },
    );

    if (confirmed != true) return;
    await deleteMark(mark.id);
    _refresh(ref, mark.groupId);
  }
}

class _ClassSummaryCard extends StatelessWidget {
  const _ClassSummaryCard({
    required this.averages,
    required this.studentCount,
    required this.markCount,
  });

  final Map<String, double> averages;
  final int studentCount;
  final int markCount;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final values = averages.values.toList();
    final classAverage = values.isEmpty
        ? null
        : values.reduce((a, b) => a + b) / values.length;

    return PremiumCard(
      padding: const EdgeInsets.all(24),
      child: Row(
        children: [
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                l10n.marksOverallAverage.toUpperCase(),
                style: Theme.of(context).textTheme.labelSmall?.copyWith(
                  letterSpacing: 1.0,
                  fontWeight: FontWeight.w800,
                  color: AppColors.inkAlpha(context, 0.4),
                ),
              ),
              const SizedBox(height: 8),
              Text(
                formatPercent(classAverage),
                style: Theme.of(context).textTheme.displaySmall?.copyWith(
                  fontWeight: FontWeight.w800,
                  color: classAverage == null
                      ? AppColors.inkAlpha(context, 0.3)
                      : percentColor(classAverage, context: context),
                ),
              ),
            ],
          ),
          const Spacer(),
          Column(
            crossAxisAlignment: CrossAxisAlignment.end,
            children: [
              _SummaryStat(
                label: l10n.marksSummaryStudents,
                value: '$studentCount',
              ),
              const SizedBox(height: 8),
              _SummaryStat(label: l10n.marksSummaryMarks, value: '$markCount'),
            ],
          ),
        ],
      ),
    );
  }
```

```
}

class _SummaryStat extends StatelessWidget {
  const _SummaryStat({required this.label, required this.value});

  final String label;
  final String value;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text(
          label,
          style: TextStyle(
            fontSize: 13,
            color: AppColors.inkAlpha(context, 0.5),
          ),
        ),
        const SizedBox(width: 8),
        Text(
          value,
          style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 15),
        ),
      ],
    );
  }
}

class _StudentRow extends StatelessWidget {
  const _StudentRow({
    required this.member,
    required this.average,
    required this.onTap,
  });

  final GroupMember member;
  final double? average;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return PremiumCard(
      onTap: onTap,
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Row(
        children: [
          CircleAvatar(
            radius: 18,
            backgroundColor: AppColors.softSurface(context),
```

**lib/screens/teacher/marks/teacher_marks_screen.dart — continued (2/2)**

```
            child: Text(
              member.fullName.isEmpty ? '?' : member.fullName[0].toUpperCase(),
              style: TextStyle(
                color: AppColors.accent(context),
                fontWeight: FontWeight.w800,
              ),
            ),
          ),
          const SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  member.fullName,
                  style: const TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 15,
```

```
                  ),
                ),
                Text(
                  average == null
                      ? l10n.marksNoMarksYetShort
                      : percentLabel(average ?? 0, l10n),
                  style: TextStyle(
                    fontSize: 12,
                    color: AppColors.inkAlpha(context, 0.5),
                  ),
                ),
              ],
            ),
          ),
          PercentBadge(percent: average),
          const SizedBox(width: 4),
          Icon(
            Icons.chevron_right_rounded,
            size: 20,
            color: AppColors.inkAlpha(context, 0.25),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/marks/teacher_marks_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 240. Import count: 9. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/marks/marks_providers.dart';
- import 'package:mu_study/screens/teacher/marks/widgets/gradebook_helpers.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/teacher/marks/widgets/gradebook_helpers.dart';

class BulkMarkSheet extends ConsumerStatefulWidget {
  const BulkMarkSheet({
    super.key,
    required this.groupId,
    required this.members,
  });

  final String groupId;
  final List<GroupMember> members;

  @override
  ConsumerState<BulkMarkSheet> createState() => _BulkMarkSheetState();
}

class _BulkMarkSheetState extends ConsumerState<BulkMarkSheet> {
  final _titleController = TextEditingController();
  final _weightController = TextEditingController(text: '1');
  final _commentController = TextEditingController();
  late final Map<String, TextEditingController> _percentControllers = {
    for (final m in widget.members) m.studentId: TextEditingController(),
  };

  MarkCategory _category = MarkCategory.test;
  DateTime _date = DateTime.now();
  bool _saving = false;
```

```
@override
void dispose() {
  _titleController.dispose();
  _weightController.dispose();
  _commentController.dispose();
  for (final c in _percentControllers.values) {
    c.dispose();
  }
  super.dispose();
}

Future<void> _save() async {
  final l10n = context.l10n;
  final title = _titleController.text.trim();
  if (title.isEmpty) {
    _toast(l10n.marksNeedTitle);
    return;
  }

  final weight = double.tryParse(_weightController.text.trim()) ?? 1;
  if (weight <= 0) {
    _toast(l10n.marksWeightPositive);
    return;
  }

  final entries = <String, double>{};
  for (final entry in _percentControllers.entries) {
    final raw = entry.value.text.trim();
    if (raw.isEmpty) continue;
    final value = double.tryParse(raw);
    if (value == null || value < 0 || value > 100) {
      _toast(l10n.teacherScoreRange);
      return;
    }
    entries[entry.key] = value;
  }

  if (entries.isEmpty) {
    _toast(l10n.marksNeedAtLeastOne);
    return;
  }

  final teacher = await ref.read(teacherProfileProvider.future);
  if (teacher == null) return;

  final comment = _commentController.text.trim();
  setState(() => _saving = true);
  try {
    for (final entry in entries.entries) {
      await saveMark(
        groupId: widget.groupId,
        studentId: entry.key,
        teacherId: teacher.id,
        title: title,
        category: _category,
        percentage: entry.value,
        weight: weight,
        comment: comment,
        markedOn: _date,
      );
    }
    if (mounted) {
      Navigator.pop(context);
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(l10n.marksSavedCount(entries.length))),
      );
    }
  } catch (e) {
    _toast(l10n.commonCouldNotSave('$e'));
  } finally {
    if (mounted) setState(() => _saving = false);
  }
}
```

```
void _toast(String message) {
  if (!mounted) return;
  ScaffoldMessenger.of(
    context,
  ).showSnackBar(SnackBar(content: Text(message)));
}

@override
Widget build(BuildContext context) {
  final l10n = context.l10n;
  return GradebookSheetScaffold(
    title: l10n.marksGradeTheClass,
    subtitle: l10n.marksOnePercentEach,
    action: FilledButton(
      onPressed: _saving ? null : _save,
      style: gradebookPrimaryButtonStyle(context),
      child: _saving
          ? SizedBox(
              width: 20,
              height: 20,
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: Theme.of(context).brightness == Brightness.dark
                    ? AppColors.darkChocolate
                    : Colors.white,
              ),
            )
          : Text(
              l10n.marksSaveMarks,
              style: const TextStyle(fontWeight: FontWeight.w700),
            ),
    ),
    child: ListView(
      shrinkWrap: true,
      padding: const EdgeInsets.symmetric(horizontal: 20),
      children: [
        TextField(
          controller: _titleController,
          textCapitalization: TextCapitalization.sentences,
          decoration: InputDecoration(
            labelText: l10n.commonTitle,
            hintText: l10n.marksTitleHint,
          ),
        ),
        const SizedBox(height: 12),
        TextField(
          controller: _commentController,
          textCapitalization: TextCapitalization.sentences,
          maxLines: 2,
          decoration: InputDecoration(
            labelText: l10n.teacherSharedComment,
            hintText: l10n.teacherSharedCommentHint,
          ),
        ),
        const SizedBox(height: 16),
        Row(
          children: [
            Expanded(
              child: GradebookCategoryDropdown(
                value: _category,
                onChanged: (v) => setState(() => _category = v),
              ),
            ),
            const SizedBox(width: 12),
            SizedBox(
              width: 90,
              child: TextField(
                controller: _weightController,
                keyboardType: const TextInputType.numberWithOptions(
                  decimal: true,
                ),
                decoration: InputDecoration(labelText: l10n.marksWeight),
              ),
```

```
              ),
            ],
          ),
          const SizedBox(height: 16),
          GradebookDateField(
            value: _date,
            onChanged: (v) => setState(() => _date = v),
          ),
          const SizedBox(height: 24),
          Text(
            l10n.marksLeaveBlankSkip,
            style: TextStyle(
              fontSize: 12,
              color: AppColors.inkAlpha(context, 0.5),
            ),
          ),
          const SizedBox(height: 8),
          ...widget.members.map(
            (m) => Padding(
              padding: const EdgeInsets.symmetric(vertical: 6),
              child: Row(
                children: [
                  Expanded(
                    child: Text(
                      m.fullName,
                      style: const TextStyle(fontWeight: FontWeight.w600),
                    ),
                  ),
                  SizedBox(
                    width: 96,
                    child: TextField(
                      controller: _percentControllers[m.studentId],
                      textAlign: TextAlign.center,
                      keyboardType: const TextInputType.numberWithOptions(
                        decimal: true,
                      ),
                      inputFormatters: [
                        FilteringTextInputFormatter.allow(
                          RegExp(r'^\d{0,3}(\.\d{0,2})?'),
                        ),
                      ],
                      decoration: const InputDecoration(
                        hintText: '—',
                        suffixText: '%',
                        isDense: true,
                      ),
                    ),
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/marks/widgets/gradebook_helpers.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 147. Import count: 7. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/layouts/premium_section_header.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/teacher/marks/widgets/gradebook_helpers.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';

ButtonStyle gradebookPrimaryButtonStyle(BuildContext context) {
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final fg = isDark ? AppColors.darkChocolate : Colors.white;
  return FilledButton.styleFrom(
    backgroundColor: AppColors.accent(context),
    foregroundColor: fg,
    disabledBackgroundColor: AppColors.accent(context).withValues(alpha: 0.35),
    disabledForegroundColor: fg.withValues(alpha: 0.55),
    padding: const EdgeInsets.symmetric(vertical: 16),
  );
}

class GradebookSheetScaffold extends StatelessWidget {
  const GradebookSheetScaffold({
    super.key,
    required this.title,
    required this.subtitle,
    required this.child,
    required this.action,
  });

  final String title;
  final String subtitle;
  final Widget child;
  final Widget action;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: EdgeInsets.only(
        bottom: MediaQuery.of(context).viewInsets.bottom,
```

```
      ),
      child: Container(
        constraints: BoxConstraints(
          maxHeight: MediaQuery.sizeOf(context).height * 0.9,
        ),
        decoration: BoxDecoration(
          color: AppColors.sheetSurface(context),
          borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),
        ),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const SizedBox(height: 12),
            Container(
              width: 40,
              height: 4,
              decoration: BoxDecoration(
                color: AppColors.softSurface(context),
                borderRadius: BorderRadius.circular(2),
              ),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(20, 12, 20, 0),
              child: PremiumSectionHeader(title: title, subtitle: subtitle),
            ),
            Flexible(child: child),
            Padding(
              padding: EdgeInsets.fromLTRB(
                20,
                12,
                20,
                MediaQuery.of(context).padding.bottom + 20,
              ),
              child: SizedBox(width: double.infinity, child: action),
            ),
          ],
        ),
      ),
    );
  }
}

class GradebookCategoryDropdown extends StatelessWidget {
  const GradebookCategoryDropdown({
    super.key,
    required this.value,
    required this.onChanged,
  });

  final MarkCategory value;
  final ValueChanged<MarkCategory> onChanged;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return DropdownButtonFormField<MarkCategory>(
      initialValue: value,
      decoration: InputDecoration(labelText: l10n.marksCategory),
      items: MarkCategory.values
          .map(
            (c) => DropdownMenuItem(value: c, child: Text(c.l10nLabel(l10n))),
          )
          .toList(),
      onChanged: (v) => onChanged(v ?? MarkCategory.other),
    );
  }
}

class GradebookDateField extends StatelessWidget {
  const GradebookDateField({
    super.key,
    required this.value,
    required this.onChanged,
```

```
  });

  final DateTime value;
  final ValueChanged<DateTime> onChanged;

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: () async {
        final picked = await showDatePicker(
          context: context,
          initialDate: value,
          firstDate: DateTime(2020),
          lastDate: DateTime.now().add(const Duration(days: 365)),
        );
        if (picked != null) onChanged(picked);
      },
      borderRadius: BorderRadius.circular(12),
      child: InputDecorator(
        decoration: InputDecoration(labelText: context.l10n.marksDate),
        child: Row(
          children: [
            Text(DateFormat.yMMMd().format(value)),
            const Spacer(),
            Icon(
              Icons.calendar_today_outlined,
              size: 16,
              color: AppColors.inkAlpha(context, 0.4),
            ),
          ],
        ),
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/marks/widgets/gradebook_helpers.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/screens/teacher/marks/widgets/mark_editor_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 213. Import count: 10. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter/services.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/auth/auth_provider.dart';
- import 'package:mu_study/providers/marks/marks_providers.dart';
- import 'package:mu_study/screens/teacher/marks/widgets/gradebook_helpers.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/teacher/marks/widgets/mark_editor_sheet.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/teacher/marks/widgets/gradebook_helpers.dart';

class MarkEditorSheet extends ConsumerStatefulWidget {
  const MarkEditorSheet({
    super.key,
    required this.groupId,
    required this.studentId,
    required this.studentName,
    this.existing,
  });

  final String groupId;
  final String studentId;
  final String studentName;
  final Mark? existing;

  @override
  ConsumerState<MarkEditorSheet> createState() => _MarkEditorSheetState();
}

class _MarkEditorSheetState extends ConsumerState<MarkEditorSheet> {
  late final _titleController = TextEditingController(
    text: widget.existing?.title ?? '',
  );
  late final _percentController = TextEditingController(
```

```
    text: widget.existing == null
        ? ''
        : formatPercent(widget.existing!.percentage).replaceAll('%', ''),
  );
  late final _weightController = TextEditingController(
    text: '${widget.existing?.weight ?? 1}',
  );
  late final _commentController = TextEditingController(
    text: widget.existing?.comment ?? '',
  );

  late MarkCategory _category = widget.existing?.category ?? MarkCategory.other;
  late DateTime _date = widget.existing?.markedOn ?? DateTime.now();
  bool _saving = false;

  @override
  void dispose() {
    _titleController.dispose();
    _percentController.dispose();
    _weightController.dispose();
    _commentController.dispose();
    super.dispose();
  }

  Future<void> _save() async {
    final l10n = context.l10n;
    final title = _titleController.text.trim();
    final percent = double.tryParse(_percentController.text.trim());
    final weight = double.tryParse(_weightController.text.trim()) ?? 1;

    if (title.isEmpty) {
      _toast(l10n.marksNeedTitle);
      return;
    }
    if (percent == null || percent < 0 || percent > 100) {
      _toast(l10n.teacherScoreRange);
      return;
    }
    if (weight <= 0) {
      _toast(l10n.marksWeightPositive);
      return;
    }

    final teacher = await ref.read(teacherProfileProvider.future);
    if (teacher == null) return;

    setState(() => _saving = true);
    try {
      await saveMark(
        groupId: widget.groupId,
        studentId: widget.studentId,
        teacherId: teacher.id,
        title: title,
        category: _category,
        percentage: percent,
        weight: weight,
        comment: _commentController.text.trim(),
        markedOn: _date,
        markId: widget.existing?.id,
      );
      if (mounted) Navigator.pop(context);
    } catch (e) {
      _toast(l10n.commonCouldNotSave('$e'));
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  void _toast(String message) {
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(message)));
```

```
}

@override
Widget build(BuildContext context) {
  final l10n = context.l10n;
  final linked = widget.existing?.isFromSubmission ?? false;

  return GradebookSheetScaffold(
    title: widget.existing == null ? l10n.marksNewMark : l10n.marksEditMark,
    subtitle: widget.studentName,
    action: FilledButton(
      onPressed: _saving ? null : _save,
      style: gradebookPrimaryButtonStyle(context),
      child: _saving
          ? SizedBox(
              width: 20,
              height: 20,
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: Theme.of(context).brightness == Brightness.dark
                    ? AppColors.darkChocolate
                    : Colors.white,
              ),
            )
          : Text(
              l10n.marksSaveMark,
              style: const TextStyle(fontWeight: FontWeight.w700),
            ),
    ),
    child: ListView(
      shrinkWrap: true,
      padding: const EdgeInsets.symmetric(horizontal: 20),
      children: [
        if (linked)
          Padding(
            padding: const EdgeInsets.only(bottom: 16),
            child: Text(
              l10n.marksFromSubmissionEdit,
              style: const TextStyle(
                fontSize: 12,
                color: AppColors.warmAmber,
                height: 1.4,
              ),
            ),
          ),
        TextField(
          controller: _titleController,
          textCapitalization: TextCapitalization.sentences,
          decoration: InputDecoration(labelText: l10n.commonTitle),
        ),
        const SizedBox(height: 16),
        Row(
          children: [
            SizedBox(
              width: 110,
              child: TextField(
                controller: _percentController,
                keyboardType: const TextInputType.numberWithOptions(
                  decimal: true,
                ),
                inputFormatters: [
                  FilteringTextInputFormatter.allow(
                    RegExp(r'^\d{0,3}(\.\d{0,2})?'),
                  ),
                ],
                decoration: InputDecoration(
                  labelText: l10n.commonScore,
                  suffixText: '%',
                ),
              ),
            ),
            const SizedBox(width: 12),
            SizedBox(
```

```
                width: 90,
                child: TextField(
                  controller: _weightController,
                  keyboardType: const TextInputType.numberWithOptions(
                    decimal: true,
                  ),
                  decoration: InputDecoration(labelText: l10n.marksWeight),
                ),
              ),
            ],
          ),
          const SizedBox(height: 16),
          GradebookCategoryDropdown(
            value: _category,
            onChanged: (v) => setState(() => _category = v),
          ),
          const SizedBox(height: 16),
          GradebookDateField(
            value: _date,
            onChanged: (v) => setState(() => _date = v),
          ),
          const SizedBox(height: 16),
          TextField(
            controller: _commentController,
            maxLines: 3,
            textCapitalization: TextCapitalization.sentences,
            decoration: InputDecoration(labelText: l10n.marksCommentOptional),
          ),
        ],
      ),
    );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/marks/widgets/mark_editor_sheet.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file’s public API changed.

## lib/screens/teacher/marks/widgets/mark_row.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 90. Import count: 8. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:intl/intl.dart';
- import 'package:mu_study/components/cards/premium_card.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/core/theme/app_colors.dart';
- import 'package:mu_study/l10n/domain_labels.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/marks/widgets/mark_row.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';

class MarkRow extends StatelessWidget {
  const MarkRow({
    super.key,
    required this.mark,
    required this.onEdit,
    this.onDelete,
    this.showStudent = true,
  });

  final Mark mark;
  final VoidCallback onEdit;
  final VoidCallback? onDelete;
  final bool showStudent;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final subtitle = [
      mark.category.l10nLabel(l10n),
      if (showStudent && mark.studentName != null) mark.studentName!,
      DateFormat.MMMd().format(mark.markedOn),
      if (mark.weight != 1)
        '×${mark.weight.toStringAsFixed(mark.weight.truncateToDouble() == mark.weight ? 0 :
1)}',
    ].join(' · ');

    return PremiumCard(
      onTap: onEdit,
```

```
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
        child: Row(
          children: [
            PercentBadge(percent: mark.percentage, compact: true),
            const SizedBox(width: 14),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    mark.title,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: const TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 15,
                    ),
                  ),
                  Text(
                    subtitle,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: TextStyle(
                      fontSize: 12,
                      color: AppColors.inkAlpha(context, 0.5),
                    ),
                  ),
                ],
              ),
            ),
            if (mark.isFromSubmission)
              Tooltip(
                message: l10n.marksFromSubmission,
                child: Icon(
                  Icons.link_rounded,
                  size: 16,
                  color: AppColors.inkAlpha(context, 0.3),
                ),
              ),
            if (onDelete != null)
              IconButton(
                tooltip: context.l10n.commonDelete,
                icon: Icon(
                  Icons.delete_outline_rounded,
                  size: 20,
                  color: AppColors.inkAlpha(context, 0.35),
                ),
                onPressed: onDelete,
              ),
          ],
        ),
      );
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/marks/widgets/mark_row.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/teacher/marks/widgets/student_marks_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 129. Import count: 12. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/components/layouts/empty_state.dart';
- import 'package:mu_study/components/marks/percent_badge.dart';
- import 'package:mu_study/core/marks/academic_term.dart';
- import 'package:mu_study/core/progress/progress_report_share.dart';
- import 'package:mu_study/l10n/l10n_x.dart';
- import 'package:mu_study/models/models.dart';
- import 'package:mu_study/providers/groups/group_providers.dart';
- import 'package:mu_study/providers/marks/marks_providers.dart';
- import 'package:mu_study/screens/teacher/marks/widgets/gradebook_helpers.dart';
- import 'package:mu_study/screens/teacher/marks/widgets/mark_row.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/screens/teacher/marks/widgets/student_marks_sheet.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/progress/progress_report_share.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/teacher/marks/widgets/gradebook_helpers.dart';
import 'package:mu_study/screens/teacher/marks/widgets/mark_row.dart';

const studentMarksSheetAdd = 'add';

class StudentMarksSheet extends ConsumerWidget {
  const StudentMarksSheet({
    super.key,
    required this.groupId,
    required this.member,
  });

  final String groupId;
  final GroupMember member;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final marks = ref.watch(
      memberMarksProvider((groupId: groupId, studentId: member.studentId)),
    );
    final mine = marks.valueOrNull ?? const <Mark>[];
```

```
  final average = ref
      .watch(groupMarkAveragesProvider(groupId))
      .valueOrNull?[member.studentId];
  final l10n = context.l10n;

  return GradebookSheetScaffold(
    title: member.fullName,
    subtitle: average == null
        ? l10n.marksNoMarksYetShort
        : l10n.marksAverageLine(
            formatPercent(average),
            percentLabel(average, l10n),
          ),
    action: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        OutlinedButton.icon(
          onPressed: () => _shareProgress(context, ref),
          icon: const Icon(Icons.ios_share_rounded),
          label: Text(l10n.progressReportShare),
        ),
        const SizedBox(height: 8),
        FilledButton.icon(
          style: gradebookPrimaryButtonStyle(context),
          onPressed: () => Navigator.pop(context, studentMarksSheetAdd),
          icon: const Icon(Icons.add_rounded),
          label: Text(
            l10n.marksAddMark,
            style: const TextStyle(fontWeight: FontWeight.w700),
          ),
        ),
      ],
    ),
    child: marks.isLoading
        ? const Padding(
            padding: EdgeInsets.all(32),
            child: Center(child: CircularProgressIndicator()),
          )
        : mine.isEmpty
        ? Padding(
            padding: const EdgeInsets.symmetric(horizontal: 20),
            child: EmptyState(
              title: l10n.marksNothingYet,
              subtitle: l10n.marksAddFirst,
              icon: Icons.grade_outlined,
            ),
          )
        : ListView(
            shrinkWrap: true,
            padding: const EdgeInsets.symmetric(horizontal: 20),
            children: mine
                .map(
                  (mark) => MarkRow(
                    mark: mark,
                    showStudent: false,
                    onEdit: () => Navigator.pop(context, mark),
                  ),
                )
                .toList(),
          ),
  );
}

Future<void> _shareProgress(BuildContext context, WidgetRef ref) async {
  final l10n = context.l10n;
  final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];
  final groupTitle =
      groups.where((g) => g.id == groupId).map((g) => g.title).firstOrNull ??
      'group';
  final term = ref.read(selectedMarkTermProvider);

  ScaffoldMessenger.of(
    context,
```

```
      ).showSnackBar(SnackBar(content: Text(l10n.progressReportSharing)));

      try {
        await shareProgressReport(
          studentId: member.studentId,
          groupId: groupId,
          studentName: member.fullName,
          groupTitle: groupTitle,
          term: term,
          l10n: l10n,
        );
        if (context.mounted) {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(SnackBar(content: Text(l10n.progressReportDone)));
        }
      } catch (e) {
        if (context.mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text(l10n.progressReportFailed('$e'))),
          );
        }
      }
    }
  }
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/marks/widgets/student_marks_sheet.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/teacher/notifications/teacher_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 12. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/screens/student/notifications/student_notifications_screen.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/notifications/teacher_notifications_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/student/notifications/student_notifications_screen.dart';

class TeacherNotificationsScreen extends ConsumerWidget {
  const TeacherNotificationsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const StudentNotificationsScreen();
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/notifications/teacher_notifications_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/screens/teacher/profile/teacher_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 12. Import count: 3. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:flutter/material.dart';
- import 'package:flutter_riverpod/flutter_riverpod.dart';
- import 'package:mu_study/screens/profile/app_profile_screen.dart';

**How to study this file**
1) Name the layer (screen / provider / model / widget / service).
2) List the tables or RPCs it touches (search for .from( and .rpc().
3) List the providers it watches or invalidates.
4) Predict what breaks if this file is deleted.
5) Only then read the listing below.

**lib/screens/teacher/profile/teacher_profile_screen.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/profile/app_profile_screen.dart';

class TeacherProfileScreen extends ConsumerWidget {
  const TeacherProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const AppProfileScreen();
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/screens/teacher/profile/teacher_profile_screen.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# lib/supabase/supabase_client.dart

Thin accessor: supabase = Supabase.instance.client. This file (supabase client) is one concrete piece of that layer.

Line count: 3. Import count: 1. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'package:supabase_flutter/supabase_flutter.dart';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/supabase/supabase_client.dart**

```
import 'package:supabase_flutter/supabase_flutter.dart';

SupabaseClient get supabase => Supabase.instance.client;
```

## Comprehension

1. What user role(s) can reach the behavior in lib/supabase/supabase_client.dart?
2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

## lib/tools/generate_school_sql.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

Line count: 261. Import count: 2. Read imports to learn dependencies, then scan for class / provider names, then read the first public method end-to-end.

- import 'dart:io';
- import 'dart:math';

> **How to study this file**
> 1) Name the layer (screen / provider / model / widget / service).
> 2) List the tables or RPCs it touches (search for .from( and .rpc().
> 3) List the providers it watches or invalidates.
> 4) Predict what breaks if this file is deleted.
> 5) Only then read the listing below.

**lib/tools/generate_school_sql.dart**

```
import 'dart:io';
import 'dart:math';

final teachers = <Map<String, String>>[
  {
    'first_name': 'James',
    'last_name': 'English',
    'username': '@james',
    'sphere': 'english',
  },
  {
    'first_name': 'Malika',
    'last_name': 'Math',
    'username': '@malika',
    'sphere': 'math',
  },
];

final students = <Map<String, String>>[
  {'first_name': 'John', 'last_name': 'Jones', 'username': '@john_jones'},
  {'first_name': 'Someone', 'last_name': 'Someone', 'username': '@someone'},
  {'first_name': 'Amina', 'last_name': 'Karimova', 'username': '@amina01'},
];

final groups = <Map<String, String>>[
  {'teacher_username': '@james', 'title': 'English Group C'},
  {'teacher_username': '@malika', 'title': 'Math Group A'},
];

final assignments = <Map<String, String>>[
  {
    'teacher_username': '@james',
    'group_title': 'English Group C',
    'student_username': '@john_jones',
  },
  {
    'teacher_username': '@james',
    'group_title': 'English Group C',
    'student_username': '@someone',
  },
  {
    'teacher_username': '@malika',
    'group_title': 'Math Group A',
    'student_username': '@amina01',
  },
];
```

```
final usedPasswords = <String>{};
final rng = Random.secure();

String sqlEscape(String value) => value.replaceAll("'", "''");

String sqlString(String value) => "'${sqlEscape(value)}'";

String generatePassword() {
  const alphabet =
      'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-_';
  String next() =>
      List.generate(20, (_) => alphabet[rng.nextInt(alphabet.length)]).join();

  var password = next();
  while (usedPasswords.contains(password)) {
    password = next();
  }
  usedPasswords.add(password);
  return password;
}

String normalizeUsername(String raw) {
  final trimmed = raw.trim();
  if (trimmed.isEmpty) {
    throw ArgumentError('username is required');
  }
  return trimmed.startsWith('@') ? trimmed : '@$trimmed';
}

String authEmail(String username) {
  final local = username.replaceAll('@', '').toLowerCase().trim();
  return '$local@mustudy.internal';
}

String fullName(Map<String, String> user) {
  final first = (user['first_name'] ?? '').trim();
  final last = (user['last_name'] ?? '').trim();
  return '$first $last'.trim();
}

final printedCredentials = <String>[];

void printCreateUser({
  required Map<String, String> user,
  required String role,
  String? sphere,
}) {
  final username = normalizeUsername(user['username'] ?? '');
  final name = fullName(user);
  final password = generatePassword();
  final email = authEmail(username);
  final sphereSql = sphere == null ? 'NULL' : sqlString(sphere);
  printedCredentials.add(
    '$username|$password|$name|$role',
  );

  stdout.writeln(
    '-- $username / $password  ($role${sphere == null ? '' : ', $sphere'})',
  );
  stdout.writeln('''
DO \$\$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT := ${sqlString(email)};
  v_username TEXT := ${sqlString(username)};
  v_full_name TEXT := ${sqlString(name)};
  v_password TEXT := ${sqlString(password)};
  v_role public.user_role := ${sqlString(role)}::public.user_role;
  v_sphere public.sphere_type := ${sphere == null ? 'NULL' : "$sphereSql::public.sphere_type"};
  v_teacher_id UUID;
BEGIN
  IF EXISTS (SELECT 1 FROM public.profiles WHERE username = v_username) THEN
    RAISE NOTICE 'skip existing user %', v_username;
```

```
    RETURN;
  END IF;

  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000',
    v_user_id,
    'authenticated',
    'authenticated',
    v_email,
    extensions.crypt(v_password, extensions.gen_salt('bf')),
    NOW(),
    jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),
    jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role', v_role::text),
    NOW(), NOW(), '', '', '', '', false, false
  );

  INSERT INTO auth.identities (
    id, user_id, identity_data, provider, provider_id, last_sign_in_at, created_at, updated_at
  ) VALUES (
    gen_random_uuid(),
    v_user_id,
    jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
    'email',
    v_email,
    NOW(), NOW(), NOW()
  );

  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, v_username, v_full_name, v_role, v_email);

  INSERT INTO public.login_credentials (profile_id, password, updated_at)
  VALUES (v_user_id, v_password, NOW())
  ON CONFLICT (profile_id) DO UPDATE
    SET password = excluded.password,
        updated_at = NOW();

  IF v_role = 'teacher' THEN
    INSERT INTO public.teachers (profile_id, sphere)
    VALUES (v_user_id, v_sphere)
    RETURNING id INTO v_teacher_id;
  ELSIF v_role = 'student' THEN
    INSERT INTO public.students (profile_id) VALUES (v_user_id);
  END IF;
END
\$\$;
''');
}

void main() {
  stdout.writeln('-- Generated by lib/tools/generate_school_sql.dart');
  stdout.writeln('-- Copy everything below into Supabase SQL Editor and run.');
  stdout.writeln(
    '-- Login passwords are in the comments above each user block.',
  );
  stdout.writeln();
  stdout.writeln('BEGIN;');
  stdout.writeln();
  stdout.writeln('''
CREATE TABLE IF NOT EXISTS public.login_credentials (
  profile_id uuid PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
  password text NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now()
);
''');

  stdout.writeln('-- ========================');
  stdout.writeln('-- Teachers');
  stdout.writeln('-- ========================');
```

```
  for (final teacher in teachers) {
    printCreateUser(
      user: teacher,
      role: 'teacher',
      sphere: (teacher['sphere'] ?? 'math').toLowerCase(),
    );
  }

  stdout.writeln('-- ========================');
  stdout.writeln('-- Students');
  stdout.writeln('-- ========================');
  for (final student in students) {
    printCreateUser(user: student, role: 'student');
  }

  stdout.writeln('-- ========================');
  stdout.writeln('-- Groups');
  stdout.writeln('-- ========================');
  for (final group in groups) {
    final teacherUsername = normalizeUsername(group['teacher_username'] ?? '');
    final title = (group['title'] ?? '').trim();
    stdout.writeln('''
INSERT INTO public.groups (teacher_id, title)
SELECT t.id, ${sqlString(title)}
FROM public.teachers t
JOIN public.profiles p ON p.id = t.profile_id
WHERE p.username = ${sqlString(teacherUsername)}
  AND NOT EXISTS (
    SELECT 1 FROM public.groups g
    WHERE g.teacher_id = t.id AND g.title = ${sqlString(title)}
  );
''');
  }

  stdout.writeln('-- ========================');
  stdout.writeln('-- Assign students to groups');
  stdout.writeln('-- ========================');
  for (final assignment in assignments) {
    final teacherUsername = normalizeUsername(
      assignment['teacher_username'] ?? '',
    );
    final groupTitle = (assignment['group_title'] ?? '').trim();
    final studentUsername = normalizeUsername(
      assignment['student_username'] ?? '',
    );
    stdout.writeln('''
INSERT INTO public.group_students (group_id, student_id)
SELECT g.id, s.id
FROM public.groups g
JOIN public.teachers t ON t.id = g.teacher_id
JOIN public.profiles tp ON tp.id = t.profile_id
JOIN public.students s ON TRUE
JOIN public.profiles sp ON sp.id = s.profile_id
WHERE tp.username = ${sqlString(teacherUsername)}
  AND g.title = ${sqlString(groupTitle)}
  AND sp.username = ${sqlString(studentUsername)}
ON CONFLICT (group_id, student_id) DO NOTHING;
''');
  }

  stdout.writeln('COMMIT;');
  stdout.writeln();
  stdout.writeln('-- ========================');
  stdout.writeln('-- ========================');
  for (final line in printedCredentials) {
    stdout.writeln('$line');
  }
}
```

## Comprehension

1. What user role(s) can reach the behavior in lib/tools/generate_school_sql.dart?

2. Which database table or RPC would you inspect in Supabase to debug it?
3. Does it hold UI, state, or both? Why was that choice made?
4. What happens offline — cache, error, or empty?
5. Name one widget or provider that would break if this file's public API changed.

# Annotated source — line lessons for every Dart file

The source atlas printed raw files. This chapter slows down: every class and provider is named, and every line that talks to Supabase, Riverpod, or the router is explained in plain English. Use it when a tutorial says “open this file” and you do not know which line matters.

# Line lesson — lib/components/analytics/teacher_analytics_charts.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class AnalyticsChartPoint {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L15.** `class MarkDistributionDoughnut extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L114.** `class AttendanceTrendBars extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L319.** `class _WeekCategory {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L327.** `class _LegendPill extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/analytics/teacher_analytics_charts.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/attachments/attachment_preview.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L83.** `class AttachmentPreview extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L137.** `class _ImagePreviewCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L320.** `class _FilePreviewCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L424.** `class _RemoveButton extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L452.** `class _ImageLightbox extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/attachments/attachment_preview.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/attachments/voice_note_field.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L13.** `class VoiceNoteField extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L31.** `class _VoiceNoteFieldState extends State<VoiceNoteField> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L390.** `class VoiceNotePlayer extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L400.** `class _VoiceNotePlayerState extends State<VoiceNotePlayer> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L505.** `class _VoiceShell extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L547.** `class _RoundAction extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L576.** `class _PulsingDot extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L585.** `class _PulsingDotState extends State<_PulsingDot>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L586.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/components/attachments/voice_note_field.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/attendance/attendance_calendar_grid.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L46.** `class AttendanceCalendarGrid extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L127.** `class _DayCell extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L206.** `class AttendanceMonthHeader extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L248.** `class AttendanceStatsBar extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L294.** `class _StatChip extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L340.** `class AttendanceLegendRow extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L369.** `class _Legend extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/attendance/attendance_calendar_grid.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/backgrounds/nature_paper_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class NaturePaperBackdrop extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18.** `class _NaturePaperBackdropState extends State<NaturePaperBackdrop>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L86.** `class _DayScene extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L93.** `class _DaySceneState extends State<_DayScene>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L94.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L226.** `class _NightScene extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L233.** `class _NightSceneState extends State<_NightScene>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L234.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L339.** `class _NightSkyPainter extends CustomPainter {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L396.** `class _PaperGrainPainter extends CustomPainter {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L423.** `class _TransitionLeaves extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L475.** `class _CloudSpec {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L491.** `class _LeafSpec {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**TASK — Predict a break in lib/components/backgrounds/nature_paper_backdrop.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/backgrounds/soft_neumorphism_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class SoftNeumorphismBackdrop extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L44.** `class _MorningAmbient extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L108.** `class _NightAmbient extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/backgrounds/soft_neumorphism_backdrop.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/brand/app_logo.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class AppLogo {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7.** `class AppLogoImage extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/brand/app_logo.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/buttons/primary_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class PrimaryButton extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/buttons/primary_button.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/cards/premium_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class PremiumCard extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L34.** `class _PremiumCardState extends State<PremiumCard>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L35.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/components/cards/premium_card.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/chat/animated_emoji_message.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class AnimatedEmojiMessage extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L23.** `class _AnimatedEmojiMessageState extends State<AnimatedEmojiMessage>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24.** `with TickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/components/chat/animated_emoji_message.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/chat/chat_lottie_emoji.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class ChatLottieEmoji extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26.** `class _ChatLottieEmojiState extends State<ChatLottieEmoji>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/components/chat/chat_lottie_emoji.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/chat/chat_media_gallery.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `enum ChatMediaKind { image, video }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L18.** `class ChatMediaItem {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L85.** `class ChatMediaGalleryPage extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L99.** `class _ChatMediaGalleryPageState extends State<ChatMediaGalleryPage> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L219.** `class _GalleryImagePage extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L227.** `class _GalleryImagePageState extends State<_GalleryImagePage>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L293.** `class _GalleryVideoPage extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L303.** `class _GalleryVideoPageState extends State<_GalleryVideoPage>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L565.** `class _ControlsBar extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L686.** `class _ControlButton extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/chat/chat_media_gallery.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/decor/paper_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class PaperAssets {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L232.** `class PaperDecor extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L266.** `class PaperGlyph extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L329.** `class PaperEmptyIllustration extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L397.** `class PaperTaped extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L443.** `class PaperDivider extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L468.** `class PaperScallopedAvatar extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/decor/paper_decor.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/decor/sidebar_grove.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class _CloudSpec {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24.** `class SidebarSkyClouds extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L31.** `class _SidebarSkyCloudsState extends State<SidebarSkyClouds>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L32.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L125.** `class SidebarGrove extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L132.** `class _SidebarGroveState extends State<SidebarGrove>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L133.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L313.** `class _FirefliesPainter extends CustomPainter {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/decor/sidebar_grove.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/dialogs/pdf_viewer_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/components/dialogs/pdf_viewer_sheet.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/homework/due_date_label.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class DueDateLabel extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/homework/due_date_label.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/inputs/group_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L7.** `class GroupSelector extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14.** `final groups = ref.watch(myGroupsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L15.** `final activeId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L57.** `ref.read(selectedGroupIdProvider.notifier).state = value;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/components/inputs/group_selector.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/inputs/load_more_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class LoadMoreButton extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/inputs/load_more_button.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/layouts/app_shell.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L20.** `class AppShell extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L33.** `ConsumerState<AppShell> createState() => _AppShellState();`

Widget that can watch providers. Required for profile/theme/locale.

**L36.** `class _AppShellState extends ConsumerState<AppShell> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L47.** `context.go(widget.destinations[index].path);`

Replace the navigation stack location. Login uses /boot so redirect can settle.

**L54.** `final unreadNotifications = ref.watch(unreadNotificationsCountProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L55.** `final unreadChat = ref.watch(focusChatMutedProvider)`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L57.** `: ref.watch(unreadChatCountProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

> **TASK — Predict a break in lib/components/layouts/app_shell.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/layouts/empty_state.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L7.** `class EmptyState extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L85.** `class ErrorState extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/layouts/empty_state.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/layouts/premium_section_header.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class PremiumSectionHeader extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/layouts/premium_section_header.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/layouts/shell_nav_bar.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L32.** `enum ShellBadgeKind { none, notifications, chat }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L34.** `class ShellDestination {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L84.** `class ShellFabLocation extends StandardFabLocation with FabEndOffsetX {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L134.** `class ShellNavBar extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L268.** `class _ShellNavItem extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/layouts/shell_nav_bar.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/marks/mark_average_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class MarkAverageCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/marks/mark_average_card.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/marks/percent_badge.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L47.** `class PercentBadge extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/components/marks/percent_badge.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/marks/term_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class TermSelector extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L15.** `final current = ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L16.** `final optionsAsync = ref.watch(academicTermOptionsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L82.** `.read(selectedMarkTermProvider.notifier)`

Riverpod binding. Name it after the value it exposes.

**L83.** `.select(next);`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

> **TASK — Predict a break in lib/components/marks/term_selector.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/components/settings/language_settings_tile.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class LanguageSettingsTile extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L29.** `final current = AppLanguage.fromLocale(ref.watch(localeProvider));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L129.** `ref.read(localeProvider.notifier).setLanguage(lang);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/components/settings/language_settings_tile.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/admin/credential_slip.dart

Credential slip PDF generation for printed login cards. This file (credential slip) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L1.** `class CredentialSlip {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/admin/credential_slip.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/admin/credential_slips_pdf.dart

Credential slip PDF generation for printed login cards. This file (credential slips pdf) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/admin/credential_slips_pdf.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/admin/credential_slips_source.dart

Credential slip PDF generation for printed login cards. This file (credential slips source) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L18.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L36.** `.from('profiles')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L37.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L38.** `.eq('username', username)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L61.** `final rows = await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L71.** `final row = Map<String, dynamic>.from(raw as Map);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L93.** `.from('group_students')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L94.** `.select(`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L97.** `.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L108.** `.from('profiles')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L109.** `.select('id, username, full_name, role, is_disabled')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L110.** `.eq('role', role.name)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L111.** `.eq('is_disabled', false)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L166.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**TASK — Predict a break in lib/core/admin/credential_slips_source.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/chat/animated_emoji_registry.dart

Emoji helpers and animated emoji registry. This file (animated emoji registry) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/chat/animated_emoji_registry.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/chat/emoji_message.dart

Emoji helpers and animated emoji registry. This file (emoji message) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/chat/emoji_message.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/constants/env.dart

Compile-time and runtime configuration (Env). This file (env) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class Env {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/constants/env.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/focus/focus_time_up_cue.dart

Focus-mode audio/visual cues (local, not a database table). This file (focus time up cue) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/focus/focus_time_up_cue.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/homework/assignment_due.dart

Due-date helpers and assignment template JSON. This file (assignment due) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `enum AssignmentDueStatus { none, upcoming, dueSoon, dueToday, overdue }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

> **TASK — Predict a break in lib/core/homework/assignment_due.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/homework/assignment_templates.dart

Due-date helpers and assignment template JSON. This file (assignment templates) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L4.** `class TemplateTaskDraft {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L48.** `class AssignmentTemplate {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L76.** `TemplateTaskDraft.fromJson(Map<String, dynamic>.from(item)),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L102.** `.from('assignment_templates')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L103.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L104.** `.eq('teacher_id', teacherId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L107.** `.map((e) => AssignmentTemplate.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L116.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L117.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L118.** `.eq('assignment_id', assignment.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L123.** `TaskItem.fromJson(Map<String, dynamic>.from(e)),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L149.** `.from('assignment_templates')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L150.** `.insert({`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L158.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L161.** `return AssignmentTemplate.fromJson(Map<String, dynamic>.from(row));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L165.** `await supabase.from('assignment_templates').delete().eq('id', templateId);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L177.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L178.** `.insert({`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L188.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L191.** `final assignment = Assignment.fromJson(Map<String, dynamic>.from(inserted));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L211.** `await supabase.from('tasks').insert([`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L235.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L236.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L237.** `.eq('assignment_id', source.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L245.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L246.** `.insert({`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L254.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L257.** `final assignment = Assignment.fromJson(Map<String, dynamic>.from(inserted));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L261.** `await supabase.from('tasks').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L268.** `final t = TaskItem.fromJson(Map<String, dynamic>.from(taskRows.first));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L269.** `await supabase.from('tasks').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L281.** `TaskItem.fromJson(Map<String, dynamic>.from(row)),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L283.** `await supabase.from('tasks').insert([`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**TASK — Predict a break in lib/core/homework/assignment_templates.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/homework/feedback_reuse.dart

Due-date helpers and assignment template JSON. This file (feedback reuse) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L41.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L42.** `.select('task_id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L43.** `.eq('id', submissionId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L49.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L50.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L51.** `.eq('task_id', taskId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L59.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L60.** `.update({'feedback': text})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L67.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L68.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L69.** `.eq('teacher_id', teacherId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L74.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L75.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L81.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82.** `.select('feedback')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**TASK — Predict a break in lib/core/homework/feedback_reuse.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/locale/app_locale.dart

Supported locales EN/RU/UZ. This file (app locale) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `enum AppLanguage {`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

> **TASK — Predict a break in lib/core/locale/app_locale.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/marks/academic_term.dart

Academic term math and CSV export. This file (academic term) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class AcademicTerm {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L150.** `class AcademicYearRecord {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L201.** `.from('academic_years')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L202.** `.select('id, start_year, label, is_current, created_at')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L209.** `.from('academic_terms')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L210.** `.select('id, academic_year_id, semester, starts_on, ends_on')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L226.** `final academicTermOptionsProvider = FutureProvider<List<AcademicTerm>>((`

Riverpod binding. Name it after the value it exposes.

**L233.** `final academicYearsProvider = FutureProvider<List<AcademicYearRecord>>((`

Riverpod binding. Name it after the value it exposes.

**L239.** `class SelectedMarkTermNotifier extends Notifier<AcademicTerm> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L244.** `ref.listen(academicTermOptionsProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L275.** `final selectedMarkTermProvider =`

Riverpod binding. Name it after the value it exposes.

**L276.** `NotifierProvider<SelectedMarkTermNotifier, AcademicTerm>(`

Riverpod binding. Name it after the value it exposes.

**L277.** `SelectedMarkTermNotifier.new,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **TASK — Predict a break in lib/core/marks/academic_term.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/marks/marks_csv_export.dart

Academic term math and CSV export. This file (marks csv export) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L26.** `.from('marks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L27.** `.select('*, students(profiles(full_name, username))')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L28.** `.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L54.** `final mark = Mark.fromJson(Map<String, dynamic>.from(row));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **TASK — Predict a break in lib/core/marks/marks_csv_export.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/paging/page_result.dart

Generic page-result type for list pagination. This file (page result) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L1.** `class PageResult<T> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/paging/page_result.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/progress/progress_report.dart

Student progress report model + PDF + share. This file (progress report) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class ProgressReport {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L45.** `.from('marks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L46.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L47.** `.eq('student_id', studentId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L48.** `.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L58.** `.map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L97.** `.from('attendance')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L98.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L99.** `.eq('group_id', groupId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L100.** `.eq('student_id', studentId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L107.** `(e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **TASK — Predict a break in lib/core/progress/progress_report.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/progress/progress_report_pdf.dart

Student progress report model + PDF + share. This file (progress report pdf) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/progress/progress_report_pdf.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/progress/progress_report_share.dart

Student progress report model + PDF + share. This file (progress report share) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/progress/progress_report_share.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/router/app_router.dart

go_router route table and auth/language/disabled redirects. This file (app router) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L43.** `final routerProvider = Provider<GoRouter>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L44.** `final notifier = _RouterNotifier(ref);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48.** `refreshListenable: notifier,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49.** `redirect: notifier.redirect,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74.** `class _RouterNotifier extends ChangeNotifier {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L75.** `_RouterNotifier(this._ref) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76.** `_ref.listen(authStateProvider, (_, _) => notifyListeners());`

Riverpod binding. Name it after the value it exposes.

**L77.** `_ref.listen(currentProfileProvider, (_, _) => notifyListeners());`

Riverpod binding. Name it after the value it exposes.

**L91.** `final profileAsync = _ref.read(currentProfileProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/core/router/app_router.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/services/app_cache.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (app cache) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/services/app_cache.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/services/cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (cache service) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class CacheService {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L38.** `.map((e) => Map<String, dynamic>.from(e as Map))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L95.** `final cacheServiceProvider = FutureProvider<CacheService>((ref) async {`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/core/services/cache_service.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/services/chat_media_cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (chat media cache service) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `class ChatMediaCacheService {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L81.** `Map<String, dynamic>.from(e.value as Map),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L125.** `if (await old.exists()) await old.delete();`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L177.** `await file.delete();`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L194.** `await _dir!.delete(recursive: true);`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L204.** `class _MediaCacheEntry {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/services/chat_media_cache_service.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/services/chat_video_thumbnail_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (chat video thumbnail service) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L12.** `class ChatVideoThumbnailService {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L111.** `await _dir!.delete(recursive: true);`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

> **TASK — Predict a break in lib/core/services/chat_video_thumbnail_service.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/services/voice_cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (voice cache service) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class VoiceCacheService {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L69.** `Map<String, dynamic>.from(e.value as Map),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L125.** `if (await old.exists()) await old.delete();`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L153.** `await file.delete();`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L176.** `await _dir!.delete(recursive: true);`

Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L185.** `class _VoiceCacheEntry {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/services/voice_cache_service.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/app_colors.dart

Four visual styles × light/dark Material 3 themes. This file (app colors) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class AppColors {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/app_colors.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/app_style.dart

Four visual styles × light/dark Material 3 themes. This file (app style) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L1.** `enum AppStyle {`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

> **TASK — Predict a break in lib/core/theme/app_style.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/app_style_tokens.dart

Four visual styles × light/dark Material 3 themes. This file (app style tokens) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class AppStyleTokens extends ThemeExtension<AppStyleTokens> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/app_style_tokens.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/app_text_theme.dart

Four visual styles × light/dark Material 3 themes. This file (app text theme) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/theme/app_text_theme.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/app_theme.dart

Four visual styles × light/dark Material 3 themes. This file (app theme) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `class AppTheme {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/app_theme.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/monochrome_workspace_colors.dart

Four visual styles × light/dark Material 3 themes. This file (monochrome workspace colors) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class MonochromeWorkspaceColors {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/monochrome_workspace_colors.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/monochrome_workspace_theme.dart

Four visual styles × light/dark Material 3 themes. This file (monochrome workspace theme) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class MonochromeWorkspaceTheme {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/monochrome_workspace_theme.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/nature_paper_colors.dart

Four visual styles × light/dark Material 3 themes. This file (nature paper colors) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class NaturePaperColors {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/nature_paper_colors.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/nature_paper_theme.dart

Four visual styles × light/dark Material 3 themes. This file (nature paper theme) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class NaturePaperTheme {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/nature_paper_theme.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/neumorphic.dart

Four visual styles × light/dark Material 3 themes. This file (neumorphic) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `enum NeuElevation { flat, raised, floating, pressed }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L10.** `class Neumorphic {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L131.** `class NeuIconWell extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L232.** `class NeuAvatar extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L269.** `class NeuSearchField extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L327.** `class NeuPressable extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L351.** `class _NeuPressableState extends State<NeuPressable> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/neumorphic.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/soft_neumorphism_colors.dart

Four visual styles × light/dark Material 3 themes. This file (soft neumorphism colors) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class SoftNeumorphismColors {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/soft_neumorphism_colors.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/theme/soft_neumorphism_theme.dart

Four visual styles × light/dark Material 3 themes. This file (soft neumorphism theme) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class SoftNeumorphismTheme {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/theme/soft_neumorphism_theme.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/utils/responsive.dart

Responsive breakpoints and voice playback. This file (responsive) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L4.** `class Breakpoints {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/utils/responsive.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/utils/time_of_day_greeting.dart

Responsive breakpoints and voice playback. This file (time of day greeting) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/core/utils/time_of_day_greeting.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/core/utils/voice_playback.dart

Responsive breakpoints and voice playback. This file (voice playback) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class DetectedAudioFormat {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L124.** `class VoicePlaybackController extends ChangeNotifier {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/core/utils/voice_playback.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/l10n/domain_labels.dart

ARB translations and generated AppLocalizations. This file (domain labels) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/l10n/domain_labels.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/l10n/l10n_x.dart

ARB translations and generated AppLocalizations. This file (l10n x) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/l10n/l10n_x.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/l10n/nav_labels.dart

ARB translations and generated AppLocalizations. This file (nav labels) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/l10n/nav_labels.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/main.dart

Process entry: load env, init Supabase, wrap ProviderScope, run MaterialApp.router.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L37.** `ProviderScope(`

Riverpod binding. Name it after the value it exposes.

**L39.** `appStyleProvider.overrideWith((ref) {`

Riverpod binding. Name it after the value it exposes.

**L41.** `ref.listen(currentProfileProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L49.** `themeModeProvider.overrideWith((ref) {`

Riverpod binding. Name it after the value it exposes.

**L54.** `ref.listen(currentProfileProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L68.** `class MuStudyApp extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L73.** `final router = ref.watch(routerProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L74.** `final locale = ref.watch(localeProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L75.** `final themeMode = ref.watch(themeModeProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L76.** `final style = ref.watch(appStyleProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

> **TASK — Predict a break in lib/main.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/assignment.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (assignment) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class Assignment {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L54.** `class TaskItem {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L86.** `class Submission {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/models/assignment.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/attendance.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (attendance) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class AttendanceRecord {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L37.** `class AttendanceStats {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/models/attendance.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/chat.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (chat) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class ChatRoomSummary {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L82.** `class ChatMessage {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L241.** `.map((r) => ChatReaction.fromJson(Map<String, dynamic>.from(r)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L250.** `replyTo = ChatReplyPreview.fromJson(Map<String, dynamic>.from(replyRaw));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L281.** `class ChatReplyPreview {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L358.** `class ChatReaction {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/models/chat.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/enums.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (enums) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L1.** `enum UserRole { student, teacher, admin }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L3.** `enum Sphere { math, english }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L5.** `enum AssignmentType { staticAssignment, homeworkPack }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L7.** `enum AttendanceStatus { present, absent, late, excused }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L9.** `enum SubmissionStatus { pending, submitted, reviewed }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L11.** `enum ChatMessageType { text, voice, image, video, file }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L42.** `enum MarkCategory {`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

> **TASK — Predict a break in lib/models/enums.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/mark.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (mark) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class Mark {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L74.** `class GroupMember {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/models/mark.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/models.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (models) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/models/models.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/notification.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (notification) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L1.** `class AppNotification {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L42.** `class Announcement {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/models/notification.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/models/profile.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (profile) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class Profile {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L55.** `class TeacherProfile {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L75.** `class StudentProfile {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L89.** `class StudyGroup {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/models/profile.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/admin/admin_providers.dart

Users, groups, analytics, academic years, mass generator. This file (admin providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L11.** `.from(table)`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L12.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L17.** `final adminPlatformStatsProvider = FutureProvider<Map<String, int>>((`

Riverpod binding. Name it after the value it exposes.

**L34.** `final adminGroupsProvider = FutureProvider<List<Map<String, dynamic>>>((`

Riverpod binding. Name it after the value it exposes.

**L38.** `.from('groups')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L39.** `.select('id, title, teachers(sphere, profiles(full_name))')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L41.** `return rows.map((e) => Map<String, dynamic>.from(e)).toList();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L44.** `final adminAnnouncementsProvider = FutureProvider<List<Map<String, dynamic>>>((`

Riverpod binding. Name it after the value it exposes.

**L48.** `.from('announcements')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L49.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L51.** `return rows.map((e) => Map<String, dynamic>.from(e)).toList();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L54.** `final allProfilesProvider =`

Riverpod binding. Name it after the value it exposes.

**L55.** `AsyncNotifierProvider<AllProfilesNotifier, PageResult<Profile>>(`

Riverpod binding. Name it after the value it exposes.

**L56.** `AllProfilesNotifier.new,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59.** `class AllProfilesNotifier extends AsyncNotifier<PageResult<Profile>> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L66.** `.from('profiles')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L67.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L71.** `rows.map((e) => Profile.fromJson(Map<String, dynamic>.from(e))).toList(),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **TASK — Predict a break in lib/providers/admin/admin_providers.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/attendance/attendance_providers.dart

Lesson sessions + attendance upsert. This file (attendance providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L29.** `class AttendanceFutureDateException implements Exception {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L36.** `class AttendanceWeekendException implements Exception {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L55.** `class AttendanceQuery {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L59.** `required this.from,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73.** `other.from == from &&`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80.** `final attendanceRangeProvider =`

Riverpod binding. Name it after the value it exposes.

**L81.** `FutureProvider.family<List<AttendanceRecord>, AttendanceQuery>((`

Riverpod binding. Name it after the value it exposes.

**L86.** `.from('attendance')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L87.** `.select('*, students(profiles(full_name))')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L88.** `.eq('group_id', query.groupId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L89.** `.gte('date', dateKey(query.from))`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93.** `filtered = filtered.eq('student_id', query.studentId!);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L98.** `.map((e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L102.** `final attendanceRangeStatsProvider =`

Riverpod binding. Name it after the value it exposes.

**L103.** `Provider.family<AsyncValue<AttendanceStats>, AttendanceQuery>((ref, query) {`

Riverpod binding. Name it after the value it exposes.

**L105.** `.watch(attendanceRangeProvider(query))`

Riverpod binding. Name it after the value it exposes.

**L109.** `final todayAttendanceProvider =`

Riverpod binding. Name it after the value it exposes.

**L110.** `FutureProvider.family<Map<String, AttendanceStatus>, String>((`

Riverpod binding. Name it after the value it exposes.

**L115.** `.from('attendance')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L116.** `.select('student_id, status')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L117.** `.eq('group_id', groupId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L118.** `.eq('date', dateKey(DateTime.now()));`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L127.** `class AttendanceWeekBucket {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L141.** `class AttendanceWeekTrend {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L150.** `final attendanceWeekTrendProvider =`

Riverpod binding. Name it after the value it exposes.

**L151.** `FutureProvider.family<AttendanceWeekTrend, String>((ref, groupId) async {`

Riverpod binding. Name it after the value it exposes.

**L158.** `.from('attendance')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L159.** `.select('status, date')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L160.** `.eq('group_id', groupId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L218.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L222.** `await supabase.from('attendance').upsert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L240.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L255.** `.from('attendance')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L256.** `.upsert(rows, onConflict: 'group_id,student_id,date');`

Insert-or-update. Submissions and attendance use this so retrying is safe.

**TASK — Predict a break in lib/providers/attendance/attendance_providers.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/auth/auth_provider.dart

Session stream, username login RPC, profile/teacher/student loaders. This file (auth provider) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `final authStateProvider = StreamProvider<AuthState>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L12.** `class AuthRepository {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19.** `final result = await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L51.** `final authRepositoryProvider = Provider<AuthRepository>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L52.** `return AuthRepository(ref.watch(cacheServiceProvider).valueOrNull);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L55.** `class ProfileRepository {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L66.** `.from('profiles')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L67.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L68.** `.eq('id', uid)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L71.** `final profile = Profile.fromJson(Map<String, dynamic>.from(data));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82.** `await supabase.from('profiles').update(updates).eq('id', id);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L86.** `final profileRepositoryProvider = Provider<ProfileRepository>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L87.** `final cacheAsync = ref.watch(cacheServiceProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L91.** `final currentProfileProvider = FutureProvider<Profile?>((ref) async {`

Riverpod binding. Name it after the value it exposes.

**L92.** `ref.watch(authStateProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L93.** `final repo = ref.watch(profileRepositoryProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L97.** `final teacherProfileProvider = FutureProvider<TeacherProfile?>((ref) async {`

Riverpod binding. Name it after the value it exposes.

**L98.** `final profile = await ref.watch(currentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L101.** `.from('teachers')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L102.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L103.** `.eq('profile_id', profile.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L106.** `return TeacherProfile.fromJson(Map<String, dynamic>.from(data));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L109.** `final studentProfileProvider = FutureProvider<StudentProfile?>((ref) async {`

Riverpod binding. Name it after the value it exposes.

**L110.** `final profile = await ref.watch(currentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L113.** `.from('students')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L114.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L115.** `.eq('profile_id', profile.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L118.** `return StudentProfile.fromJson(Map<String, dynamic>.from(data));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **TASK — Predict a break in lib/providers/auth/auth_provider.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/chat/chat_providers.dart

Room summaries, unread RPC, messages, reactions. This file (chat providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L102.** `final chatRoomsProvider = StreamProvider<List<ChatRoomSummary>>((ref) async* {`

Riverpod binding. Name it after the value it exposes.

**L103.** `final profile = await ref.watch(currentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L111.** `.from('chat_room_summaries')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L112.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L116.** `await supabase.rpc('chat_unread_counts') as List<dynamic>? ?? [];`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L119.** `final map = Map<String, dynamic>.from(row as Map);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L126.** `.map((e) => ChatRoomSummary.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L146.** `.channel('chat-rooms-${profile.id}')`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L147.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L155.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L176.** `final unreadChatCountProvider = Provider<int>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L177.** `final rooms = ref.watch(chatRoomsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L184.** `final chatRoomProvider = FutureProvider.family<ChatRoomSummary?, String>((`

Riverpod binding. Name it after the value it exposes.

**L188.** `final rooms = ref.watch(chatRoomsProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L195.** `.from('chat_room_summaries')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L196.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L197.** `.eq('id', roomId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L200.** `return ChatRoomSummary.fromJson(Map<String, dynamic>.from(row));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L203.** `class ChatMessagesState {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L227.** `class ChatMessagesNotifier`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L228.** `extends FamilyAsyncNotifier<ChatMessagesState, String> {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234.** `final profile = await ref.watch(currentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L239.** `final cache = ref.read(cacheServiceProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L254.** `.channel('chat-room-$roomId')`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L255.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L256.** `event: PostgresChangeEvent.insert,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260.** `type: PostgresChangeFilterType.eq,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L267.** `event: PostgresChangeEvent.update,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271.** `type: PostgresChangeFilterType.eq,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L278.** `event: PostgresChangeEvent.delete,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282.** `type: PostgresChangeFilterType.eq,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L289.** `event: PostgresChangeEvent.insert,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L295.** `event: PostgresChangeEvent.insert,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L301.** `event: PostgresChangeEvent.update,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L307.** `event: PostgresChangeEvent.delete,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336.** `final cache = ref.read(cacheServiceProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L357.** `.from('chat_messages')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L358.** `.select(_chatMessageSelect)`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L359.** `.eq('room_id', roomId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L369.** `.map((e) => ChatMessage.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L409.** `.from('profiles')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L410.** `.select('full_name, avatar_url, username, role')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L411.** `.eq('id', senderId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L414.** `final map = Map<String, dynamic>.from(profileRow);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L433.** `.from('chat_messages')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L434.** `.select(`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L437.** `.eq('id', replyId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L440.** `row['reply_to'] = Map<String, dynamic>.from(data);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L446.** `final row = Map<String, dynamic>.from(record);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L461.** `final row = Map<String, dynamic>.from(record);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L517.** `final reaction = ChatReaction.fromJson(Map<String, dynamic>.from(record));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L563.** `final profile = await ref.read(currentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L569.** `.from('chat_messages')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L570.** `.insert({`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L577.** `.select(_chatMessageSelect)`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L580.** `final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L599.** `final profile = await ref.read(currentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L617.** `.from(bucket)`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L626.** `.from(bucket)`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L627.** `.createSignedUrl(path, chatSignedUrlTtlSeconds);`

Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L643.** `.from('chat_messages')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L644.** `.insert({`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L654.** `.select(_chatMessageSelect)`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L658.** `final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L673.** `.from('chat_messages')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L674.** `.update({'text_content': trimmed})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L675.** `.eq('id', messageId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L703.** `await supabase.from('chat_messages').update({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L711.** `final profile = await ref.read(currentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L754.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L761.** `final profile = await ref.read(currentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L765.** `.from('chat_messages')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L766.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L767.** `.eq('room_id', arg)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L768.** `.eq('is_deleted', false)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L777.** `.from('message_reads')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L778.** `.select('message_id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L779.** `.eq('user_id', profile.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L791.** `.from('message_reads')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L792.** `.upsert(toInsert, onConflict: 'message_id,user_id');`

Insert-or-update. Submissions and attendance use this so retrying is safe.

**L793.** `ref.invalidate(chatRoomsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L797.** `final chatMessagesProvider =`

Riverpod binding. Name it after the value it exposes.

**L798.** `AsyncNotifierProvider.family<`

Riverpod binding. Name it after the value it exposes.

**L799.** `ChatMessagesNotifier,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L802.** `>(ChatMessagesNotifier.new);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **TASK — Predict a break in lib/providers/chat/chat_providers.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/focus/focus_session_provider.dart

Local Pomodoro/free timer persisted in SharedPreferences. This file (focus session provider) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `enum FocusTimerMode { pomodoro, free }`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L14.** `class FocusSession {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L93.** `class FocusSummary {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L121.** `class FocusSessionState {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L143.** `final focusSessionProvider =`

Riverpod binding. Name it after the value it exposes.

**L144.** `StateNotifierProvider<FocusSessionController, FocusSessionState>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L148.** `final focusChatMutedProvider = Provider<bool>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L149.** `return ref.watch(focusSessionProvider).isActive;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L152.** `class FocusSessionController extends StateNotifier<FocusSessionState> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/providers/focus/focus_session_provider.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/groups/group_providers.dart

Teacher-owned and student-membership groups. This file (group providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `final myGroupsProvider = FutureProvider<List<StudyGroup>>((ref) async {`

Riverpod binding. Name it after the value it exposes.

**L7.** `final profile = await ref.watch(currentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L11.** `final teacher = await ref.watch(teacherProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L14.** `.from('groups')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L15.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L16.** `.eq('teacher_id', teacher.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L19.** `.map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L24.** `final student = await ref.watch(studentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L27.** `.from('group_students')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L28.** `.select('groups(*)')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L29.** `.eq('student_id', student.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L33.** `Map<String, dynamic>.from(e['groups'] as Map),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L39.** `final rows = await supabase.from('groups').select().order('title');`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L41.** `.map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L45.** `final selectedGroupIdProvider = StateProvider<String?>((ref) => null);`

Riverpod binding. Name it after the value it exposes.

**L47.** `final activeGroupIdProvider = Provider<String?>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L48.** `final selected = ref.watch(selectedGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L49.** `final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L54.** `final groupMembersProvider = FutureProvider.family<List<GroupMember>, String>((`

Riverpod binding. Name it after the value it exposes.

**L59.** `.from('group_students')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L60.** `.select('students(id, profiles(full_name, username))')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L61.** `.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L63.** `.map((e) => GroupMember.fromGroupStudentRow(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **TASK — Predict a break in lib/providers/groups/group_providers.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/homework/assignment_providers.dart

Assignments, tasks, teacher submissions queue. This file (assignment providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `final assignmentsProvider =`

Riverpod binding. Name it after the value it exposes.

**L11.** `AsyncNotifierProvider.family<`

Riverpod binding. Name it after the value it exposes.

**L12.** `AssignmentsNotifier,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15.** `>(AssignmentsNotifier.new);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17.** `class AssignmentsNotifier`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18.** `extends FamilyAsyncNotifier<PageResult<Assignment>, String?> {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23.** `final cache = ref.watch(cacheServiceProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L27.** `var query = supabase.from('assignments').select();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L28.** `if (arg != null) query = query.eq('group_id', arg!);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L32.** `final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L63.** `final assignmentDetailProvider = FutureProvider.family<Assignment?, String>((`

Riverpod binding. Name it after the value it exposes.

**L68.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L69.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L70.** `.eq('id', id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L73.** `return Assignment.fromJson(Map<String, dynamic>.from(data));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L76.** `final tasksProvider = FutureProvider.family<List<TaskItem>, String>((`

Riverpod binding. Name it after the value it exposes.

**L81.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L83.** `.eq('assignment_id', assignmentId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L86.** `.map((e) => TaskItem.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L90.** `final assignmentProgressProvider = FutureProvider.family<double, String>((`

Riverpod binding. Name it after the value it exposes.

**L94.** `final student = await ref.watch(studentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L98.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L99.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L100.** `.eq('assignment_id', assignmentId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L105.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L106.** `.select('task_id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L107.** `.eq('student_id', student.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L108.** `.eq('is_completed', true)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

> **TASK — Predict a break in lib/providers/homework/assignment_providers.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/homework/teacher_submissions_provider.dart

Assignments, tasks, teacher submissions queue. This file (teacher submissions provider) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `final teacherSubmissionsProvider = FutureProvider<List<Map<String, dynamic>>>((`

Riverpod binding. Name it after the value it exposes.

**L8.** `final teacher = await ref.watch(teacherProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L12.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L13.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L14.** `.eq('teacher_id', teacher.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L19.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L20.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L26.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L27.** `.select('''`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

> **TASK — Predict a break in lib/providers/homework/teacher_submissions_provider.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/marks/marks_providers.dart

Gradebook CRUD and student averages. This file (marks providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L19.** `var query = supabase.from('marks').select(select);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L20.** `if (groupId != null) query = query.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L21.** `if (studentId != null) query = query.eq('student_id', studentId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L31.** `return rows.map((e) => Map<String, dynamic>.from(e)).toList();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L50.** `final groupMarksProvider =`

Riverpod binding. Name it after the value it exposes.

**L51.** `AsyncNotifierProvider.family<GroupMarksNotifier, PageResult<Mark>, String>(`

Riverpod binding. Name it after the value it exposes.

**L52.** `GroupMarksNotifier.new,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55.** `class GroupMarksNotifier extends FamilyAsyncNotifier<PageResult<Mark>, String> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L58.** `ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L63.** `final term = ref.read(selectedMarkTermProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L86.** `final groupMarkAveragesProvider =`

Riverpod binding. Name it after the value it exposes.

**L87.** `FutureProvider.family<Map<String, double>, String>((ref, groupId) async {`

Riverpod binding. Name it after the value it exposes.

**L88.** `final term = ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L90.** `.from('marks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L91.** `.select('student_id, percentage, weight')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L92.** `.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L99.** `rows.map((e) => Map<String, dynamic>.from(e)).toList(),`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L103.** `final groupMarkCountProvider = FutureProvider.family<int, String>((`

Riverpod binding. Name it after the value it exposes.

**L107.** `final term = ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L108.** `var query = supabase.from('marks').select('id').eq('group_id', groupId);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L117.** `final memberMarksProvider =`

Riverpod binding. Name it after the value it exposes.

**L118.** `FutureProvider.family<List<Mark>, ({String groupId, String studentId})>((`

Riverpod binding. Name it after the value it exposes.

**L122.** `final term = ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L124.** `.from('marks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L125.** `.select('*, students(profiles(full_name, username))')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L126.** `.eq('group_id', key.groupId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L127.** `.eq('student_id', key.studentId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L136.** `.map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L140.** `final myMarksProvider =`

Riverpod binding. Name it after the value it exposes.

**L141.** `AsyncNotifierProvider<MyMarksNotifier, PageResult<Mark>>(`

Riverpod binding. Name it after the value it exposes.

**L142.** `MyMarksNotifier.new,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145.** `class MyMarksNotifier extends AsyncNotifier<PageResult<Mark>> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L148.** `ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L149.** `ref.watch(studentProfileProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L154.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L158.** `final term = ref.read(selectedMarkTermProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L182.** `final myMarkStatsProvider =`

Riverpod binding. Name it after the value it exposes.

**L183.** `FutureProvider<`

Riverpod binding. Name it after the value it exposes.

**L186.** `final student = await ref.watch(studentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L188.** `final term = ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L190.** `.from('marks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L191.** `.select('category, percentage, weight')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L192.** `.eq('student_id', student.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L209.** `final myMarkAverageProvider = FutureProvider<double?>((ref) async {`

Riverpod binding. Name it after the value it exposes.

**L210.** `final stats = await ref.watch(myMarkStatsProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L248.** `await supabase.from('marks').insert(payload);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L250.** `await supabase.from('marks').update(payload).eq('id', markId);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L255.** `await supabase.from('marks').delete().eq('id', markId);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **TASK — Predict a break in lib/providers/marks/marks_providers.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/notifications/notification_providers.dart

Realtime inbox + announcements. This file (notification providers) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `final notificationsProvider = StreamProvider<List<AppNotification>>((`

Riverpod binding. Name it after the value it exposes.

**L13.** `final profile = await ref.watch(currentProfileProvider.future);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L19.** `final cache = ref.watch(cacheServiceProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L23.** `.from('notifications')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L24.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L25.** `.eq('user_id', profile.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L28.** `final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L35.** `.channel('notifications-${profile.id}')`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L36.** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L41.** `type: PostgresChangeFilterType.eq,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63.** `final unreadNotificationsCountProvider = Provider<int>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L64.** `final notifications = ref.watch(notificationsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L71.** `final announcementsProvider =`

Riverpod binding. Name it after the value it exposes.

**L72.** `FutureProvider.family<List<Announcement>, String?>((ref, groupId) async {`

Riverpod binding. Name it after the value it exposes.

**L73.** `var query = supabase.from('announcements').select();`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L74.** `if (groupId != null) query = query.eq('group_id', groupId);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L77.** `.map((e) => Announcement.fromJson(Map<String, dynamic>.from(e)))`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82.** `await supabase.from('notifications').update({'is_read': true}).eq('id', id);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L87.** `.from('notifications')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L88.** `.update({'is_read': true})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L89.** `.eq('user_id', userId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L90.** `.eq('is_read', false);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**TASK — Predict a break in lib/providers/notifications/notification_providers.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/settings/locale_provider.dart

Locale, theme mode, AppStyle. This file (locale provider) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `final localeProvider = StateNotifierProvider<LocaleController, Locale>((ref) {`

Riverpod binding. Name it after the value it exposes.

**L12.** `ref.listen(currentProfileProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L21.** `class LocaleController extends StateNotifier<Locale> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L56.** `final profile = _ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L60.** `await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L63.** `_ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/providers/settings/locale_provider.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/providers/settings/theme_provider.dart

Locale, theme mode, AppStyle. This file (theme provider) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L11.** `enum AppThemePreference {`

Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L39.** `final themeModeProvider = StateNotifierProvider<ThemeModeController, ThemeMode>(`

Riverpod binding. Name it after the value it exposes.

**L42.** `ref.listen(currentProfileProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L52.** `class ThemeModeController extends StateNotifier<ThemeMode> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L89.** `final profile = _ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L93.** `await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L96.** `_ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L105.** `final appStyleProvider = StateNotifierProvider<AppStyleController, AppStyle>((`

Riverpod binding. Name it after the value it exposes.

**L109.** `ref.listen(currentProfileProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L118.** `class AppStyleController extends StateNotifier<AppStyle> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L153.** `final profile = _ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L157.** `await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L160.** `_ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/providers/settings/theme_provider.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/academic_years/admin_academic_years_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L12.** `class AdminAcademicYearsScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16.** `ConsumerState<AdminAcademicYearsScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L20.** `class _AdminAcademicYearsScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21.** `extends ConsumerState<AdminAcademicYearsScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L28.** `final row = await supabase.rpc('admin_create_next_academic_year');`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L30.** `ref.invalidate(academicYearsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L31.** `ref.invalidate(academicTermOptionsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L49.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L53.** `ref.invalidate(academicYearsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L54.** `ref.invalidate(academicTermOptionsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L133.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L137.** `ref.invalidate(academicYearsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L138.** `ref.invalidate(academicTermOptionsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L249.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L257.** `ref.invalidate(academicYearsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L258.** `ref.invalidate(academicTermOptionsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L276.** `final years = ref.watch(academicYearsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L308.** `ref.invalidate(academicYearsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L309.** `await ref.read(academicYearsProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/screens/admin/academic_years/admin_academic_years_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/analytics/admin_analytics_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L12.** `class AdminAnalyticsScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17.** `final stats = ref.watch(adminPlatformStatsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L26.** `onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L81.** `class _AdminChartData {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/admin/analytics/admin_analytics_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/announcements/admin_announcements_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L13.** `class AdminAnnouncementsScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17.** `ConsumerState<AdminAnnouncementsScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L21.** `class _AdminAnnouncementsScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L22.** `extends ConsumerState<AdminAnnouncementsScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L24.** `ref.invalidate(adminAnnouncementsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L25.** `ref.invalidate(announcementsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L29.** `final groups = await ref.read(adminGroupsProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L93.** `await supabase.from('announcements').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L100.** `.from('announcements')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L101.** `.update({`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L105.** `.eq('id', existing['id'] as String);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L143.** `await supabase.from('announcements').delete().eq('id', row['id'] as String);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L149.** `final announcements = ref.watch(adminAnnouncementsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L151.** `for (final g in ref.watch(adminGroupsProvider).valueOrNull ?? [])`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**TASK — Predict a break in lib/screens/admin/announcements/admin_announcements_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/credentials/admin_credential_print_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L14.** `class AdminCredentialPrintScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18.** `ConsumerState<AdminCredentialPrintScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L22.** `class _AdminCredentialPrintScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L23.** `extends ConsumerState<AdminCredentialPrintScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L68.** `final groups = ref.watch(adminGroupsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L173.** `class _PreviewList extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/admin/credentials/admin_credential_print_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/dashboard/admin_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L13.** `class AdminDashboardScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18.** `final stats = ref.watch(adminPlatformStatsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L26.** `onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L284.** `class _AdminStatCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L342.** `class _AdminChartData {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/admin/dashboard/admin_dashboard_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L12.** `class AdminMassDataGeneratorScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16.** `ConsumerState<AdminMassDataGeneratorScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L20.** `class _AdminMassDataGeneratorScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21.** `extends ConsumerState<AdminMassDataGeneratorScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L137.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L157.** `ref.invalidate(adminPlatformStatsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L158.** `ref.invalidate(adminGroupsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L159.** `ref.invalidate(allProfilesProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/groups/admin_groups_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L13.** `class AdminGroupsScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17.** `ConsumerState<AdminGroupsScreen> createState() => _AdminGroupsScreenState();`

Widget that can watch providers. Required for profile/theme/locale.

**L20.** `class _AdminGroupsScreenState extends ConsumerState<AdminGroupsScreen> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27.** `.from('teachers')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L28.** `.select('id, profiles(full_name, username)');`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L86.** `await supabase.from('groups').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L91.** `ref.invalidate(adminGroupsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L117.** `.from('students')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L118.** `.select('id, profiles(full_name, username)');`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L148.** `await supabase.from('group_students').upsert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L153.** `ref.invalidate(adminGroupsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L154.** `ref.invalidate(groupMembersProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L166.** `final groups = ref.watch(adminGroupsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L180.** `onRefresh: () async => ref.invalidate(adminGroupsProvider),`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**TASK — Predict a break in lib/screens/admin/groups/admin_groups_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/helpers/admin_labels.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/screens/admin/helpers/admin_labels.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/notifications/admin_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class AdminNotificationsScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/admin/notifications/admin_notifications_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/profile/admin_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class AdminProfileScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/admin/profile/admin_profile_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/admin/users/admin_users_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L17.** `class AdminUsersScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21.** `ConsumerState<AdminUsersScreen> createState() => _AdminUsersScreenState();`

Widget that can watch providers. Required for profile/theme/locale.

**L24.** `class _AdminUsersScreenState extends ConsumerState<AdminUsersScreen> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L33.** `final groupsRes = await supabase.from('groups').select('id, title');`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L128.** `final userId = await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L141.** `.from('students')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L142.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L143.** `.eq('profile_id', userId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L147.** `await supabase.from('group_students').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L159.** `ref.invalidate(allProfilesProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L202.** `await supabase.rpc(`

Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L225.** `.from('profiles')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L226.** `.update({'is_disabled': !profile.isDisabled})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L227.** `.eq('id', profile.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L228.** `ref.invalidate(allProfilesProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L233.** `final profiles = ref.watch(allProfilesProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L260.** `ref.read(allProfilesProvider.notifier).loadMore(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/screens/admin/users/admin_users_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/auth/language_selection_screen.dart

Splash, login, language, setup-required, disabled account. This file (language selection screen) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `class LanguageSelectionScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14.** `ConsumerState<LanguageSelectionScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L18.** `class _LanguageSelectionScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19.** `extends ConsumerState<LanguageSelectionScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L29.** `_selected = AppLanguage.fromLocale(ref.read(localeProvider));`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L40.** `await ref.read(localeProvider.notifier).setLanguage(lang);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L51.** `final profile = ref.watch(currentProfileProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L163.** `class _LanguageCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/auth/language_selection_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/auth/login_screen.dart

Splash, login, language, setup-required, disabled account. This file (login screen) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `class LoginScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `ConsumerState<LoginScreen> createState() => _LoginScreenState();`

Widget that can watch providers. Required for profile/theme/locale.

**L23.** `class _LoginScreenState extends ConsumerState<LoginScreen> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L49.** `.read(authRepositoryProvider)`

Riverpod binding. Name it after the value it exposes.

**L51.** `ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L250.** `class _LoginNatureScenery extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L257.** `class _LoginCloud {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L275.** `class _LoginNatureSceneryState extends State<_LoginNatureScenery>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L276.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/screens/auth/login_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/auth/setup_required_screen.dart

Splash, login, language, setup-required, disabled account. This file (setup required screen) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L7.** `class SetupRequiredScreen extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/auth/setup_required_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/auth/splash_screen.dart

Splash, login, language, setup-required, disabled account. This file (splash screen) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class SplashScreen extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L15.** `class _SplashScreenState extends State<SplashScreen> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/auth/splash_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/list/chat_list_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L17.** `class ChatListScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L23.** `ConsumerState<ChatListScreen> createState() => _ChatListScreenState();`

Widget that can watch providers. Required for profile/theme/locale.

**L26.** `class _ChatListScreenState extends ConsumerState<ChatListScreen> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L72.** `final roomsAsync = ref.watch(chatRoomsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L166.** `class _ChatRoomTile extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L182.** `final me = ref.watch(currentProfileProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

> **TASK — Predict a break in lib/screens/chat/list/chat_list_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/chat_room_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L30.** `class ChatRoomScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L36.** `ConsumerState<ChatRoomScreen> createState() => _ChatRoomScreenState();`

Widget that can watch providers. Required for profile/theme/locale.

**L39.** `class _ChatRoomScreenState extends ConsumerState<ChatRoomScreen>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L164.** `final me = ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L166.** `final current = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L176.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L238.** `final me = ref.read(currentProfileProvider).valueOrNull?.id;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L322.** `final profile = ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L341.** `final profile = ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L360.** `final channel = supabase.channel(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L404.** `final msgs = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L416.** `final profile = ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L490.** `final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L498.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L518.** `ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L546.** `ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L558.** `ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L597.** `final s = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L646.** `final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L677.** `await ref.read(chatMessagesProvider(widget.roomId).notifier).loadMore();`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L682.** `final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L744.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L821.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L940.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L972.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L986.** `.read(chatMessagesProvider(widget.roomId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L1000.** `final room = ref.watch(chatRoomProvider(widget.roomId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L1001.** `final messages = ref.watch(chatMessagesProvider(widget.roomId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L1002.** `final me = ref.watch(currentProfileProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L1007.** `ref.listen(chatMessagesProvider(widget.roomId), (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L1010.** `ref.listen(currentProfileProvider, (prev, next) {`

Riverpod binding. Name it after the value it exposes.

**L1214.** `.read(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1215.** `chatMessagesProvider(`

Riverpod binding. Name it after the value it exposes.

**L1217.** `).notifier,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**TASK — Predict a break in lib/screens/chat/room/chat_room_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/helpers/chat_list_entries.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L3.** `class ChatListEntry {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/helpers/chat_list_entries.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/helpers/reply_snippets.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/screens/chat/room/helpers/reply_snippets.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/attach_chip.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class AttachChip extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/attach_chip.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/chat_date_chip.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L7.** `class ChatDateChip extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/chat_date_chip.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/chat_file.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L13.** `class ChatFile extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/chat_file.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/chat_image.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `class ChatImage extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26.** `class _ChatImageState extends State<ChatImage> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/chat_image.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/chat_video.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class ChatVideo extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25.** `class _ChatVideoState extends State<ChatVideo> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/chat_video.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/chat_voice.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class ChatVoice extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24.** `class _ChatVoiceState extends State<ChatVoice> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L213.** `class VoiceWaveformPainter extends CustomPainter {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/chat_voice.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/edit_message_dialog.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L4.** `class EditMessageDialog extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13.** `class _EditMessageDialogState extends State<EditMessageDialog> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/edit_message_dialog.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/message_bubble.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L25.** `class MessageBubble extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L488.** `image: CachedNetworkImageProvider(`

Riverpod binding. Name it after the value it exposes.

**L514.** `? CachedNetworkImageProvider(message.senderAvatarUrl!)`

Riverpod binding. Name it after the value it exposes.

**L676.** `overlay.insert(entry);`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L680.** `class MessageActionPopover extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L870.** `class ReactionBar extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L965.** `class CaretPainter extends CustomPainter {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L992.** `class PopoverMenuItem extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L1033.** `class ReactionPill extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L1099.** `class SenderHeader extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L1166.** `class MessageBody extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/message_bubble.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/reply_composer_banner.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class ReplyComposerBanner extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/reply_composer_banner.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/reply_quote.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L7.** `class ReplyQuote extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/reply_quote.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/chat/room/widgets/swipe_to_reply.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class SwipeToReply extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L15.** `class _SwipeToReplyState extends State<SwipeToReply> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/chat/room/widgets/swipe_to_reply.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/profile/app_profile_screen.dart

Shared profile (theme, language, password, logout). This file (app profile screen) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L30.** `class AppProfileScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L35.** `final profile = ref.watch(currentProfileProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L54.** `class _ProfileBody extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L60.** `ConsumerState<_ProfileBody> createState() => _ProfileBodyState();`

Widget that can watch providers. Required for profile/theme/locale.

**L63.** `class _ProfileBodyState extends ConsumerState<_ProfileBody> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L158.** `.from('avatars')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L167.** `final publicUrl = supabase.storage.from('avatars').getPublicUrl(path);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L169.** `await ref.read(profileRepositoryProvider).updateProfile(`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L173.** `ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L217.** `await ref.read(profileRepositoryProvider).updateProfile(`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L221.** `ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L311.** `.read(authRepositoryProvider)`

Riverpod binding. Name it after the value it exposes.

**L390.** `final cache = ref.read(cacheServiceProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L406.** `final groupId = ref.read(activeGroupIdProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L407.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L415.** `final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L427.** `term: ref.read(selectedMarkTermProvider),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L441.** `final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L442.** `final teacher = ref.watch(teacherProfileProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L443.** `final student = ref.watch(studentProfileProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L445.** `? ref.watch(myMarkAverageProvider).valueOrNull`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L447.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L463.** `attStats = ref.watch(attendanceRangeStatsProvider(query)).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L602.** `onTap: () => context.push(s.path),`

Push a route the user can pop (assignment detail, chat room).

**L679.** `onPressed: () => ref.read(authRepositoryProvider).signOut(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L707.** `class _Shortcut {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L714.** `class _StatChip extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/profile/app_profile_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/profile/widgets/profile_hero_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L7.** `class ProfileHeroCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L73.** `? CachedNetworkImageProvider(profile.avatarUrl!)`

Riverpod binding. Name it after the value it exposes.

**L174.** `class _ProfileChip extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/profile/widgets/profile_hero_card.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/profile/widgets/profile_settings_row.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class ProfileSettingsRow extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/profile/widgets/profile_settings_row.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/profile/widgets/profile_theme_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L12.** `class ProfileThemeSelector extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18.** `final mode = ref.watch(themeModeProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L19.** `final style = ref.watch(appStyleProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L58.** `ref.read(themeModeProvider.notifier).setPreference(next);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L89.** `.read(appStyleProvider.notifier)`

Riverpod binding. Name it after the value it exposes.

**L105.** `.read(appStyleProvider.notifier)`

Riverpod binding. Name it after the value it exposes.

**L121.** `.read(appStyleProvider.notifier)`

Riverpod binding. Name it after the value it exposes.

**L137.** `.read(appStyleProvider.notifier)`

Riverpod binding. Name it after the value it exposes.

**L150.** `class _StyleOptionCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/profile/widgets/profile_theme_selector.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/attendance/student_attendance_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `class StudentAttendanceScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `ConsumerState<StudentAttendanceScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L24.** `class _StudentAttendanceScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25.** `extends ConsumerState<StudentAttendanceScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L38.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L39.** `final student = ref.watch(studentProfileProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L68.** `final records = ref.watch(attendanceRangeProvider(query));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L69.** `final stats = ref.watch(attendanceRangeStatsProvider(query));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L75.** `ref.invalidate(attendanceRangeProvider(query));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/screens/student/attendance/student_attendance_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/focus_format.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/screens/student/focus/focus_format.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/student_focus_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L15.** `class StudentFocusScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19.** `ConsumerState<StudentFocusScreen> createState() => _StudentFocusScreenState();`

Widget that can watch providers. Required for profile/theme/locale.

**L22.** `class _StudentFocusScreenState extends ConsumerState<StudentFocusScreen>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L33.** `final session = ref.read(focusSessionProvider).session;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L51.** `final session = ref.read(focusSessionProvider).session;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L58.** `ref.read(focusSessionProvider.notifier).endSession(timedOut: true);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L121.** `.read(focusSessionProvider.notifier)`

Riverpod binding. Name it after the value it exposes.

**L127.** `final focus = ref.watch(focusSessionProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L132.** `onPause: () => ref.read(focusSessionProvider.notifier).pause(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L133.** `onResume: () => ref.read(focusSessionProvider.notifier).resume(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L134.** `onEnd: () => ref.read(focusSessionProvider.notifier).endSession(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L150.** `ref.read(focusSessionProvider.notifier).setSummaryNote(v),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L152.** `ref.read(focusSessionProvider.notifier).dismissSummary();`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L155.** `onDone: () => ref.read(focusSessionProvider.notifier).dismissSummary(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**TASK — Predict a break in lib/screens/student/focus/student_focus_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/widgets/active_session_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L8.** `class ActiveSessionView extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/focus/widgets/active_session_view.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/widgets/focus_ambient_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class FocusAmbientDecor extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/focus/widgets/focus_ambient_decor.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/widgets/focus_home_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L18.** `class FocusHomeView extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L27.** `final assignments = ref.watch(assignmentsProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L33.** `if (groupId != null) ref.invalidate(assignmentsProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/screens/student/focus/widgets/focus_home_view.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/widgets/focus_summary_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `class FocusSummaryView extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/focus/widgets/focus_summary_view.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/focus/widgets/soft_fireflies.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class SoftFireflies extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L12.** `class _SoftFirefliesState extends State<SoftFireflies>`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L34.** `class SoftFirefliesPainter extends CustomPainter {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/focus/widgets/soft_fireflies.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/home/student_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L27.** `class StudentDashboardScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L31.** `ConsumerState<StudentDashboardScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L35.** `class _StudentDashboardScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L36.** `extends ConsumerState<StudentDashboardScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L58.** `final profile = ref.watch(currentProfileProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L59.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L60.** `final assignments = ref.watch(assignmentsProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L61.** `final notifications = ref.watch(notificationsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L62.** `final announcements = ref.watch(announcementsProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L63.** `final marks = ref.watch(myMarksProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L64.** `final average = ref.watch(myMarkAverageProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L68.** `ref.invalidate(myGroupsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L69.** `ref.invalidate(myMarksProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L70.** `ref.invalidate(myMarkAverageProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L72.** `ref.invalidate(assignmentsProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L73.** `ref.invalidate(announcementsProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L305.** `Consumer(`

Widget that can watch providers. Required for profile/theme/locale.

**L307.** `final progressAsync = ref.watch(`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L308.** `assignmentProgressProvider(a.id),`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/screens/student/home/student_dashboard_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/home/widgets/dashboard_announcements.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L6.** `class DashboardAnnouncements extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17.** `class _DashboardAnnouncementsState extends State<DashboardAnnouncements> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/home/widgets/dashboard_announcements.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/home/widgets/expandable_announcement_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L9.** `class ExpandableAnnouncementCard extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19.** `class _ExpandableAnnouncementCardState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `extends State<ExpandableAnnouncementCard> {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **TASK — Predict a break in lib/screens/student/home/widgets/expandable_announcement_card.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/homework/student_assignments_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L19.** `class StudentAssignmentsScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L28.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29.** `final assignments = ref.watch(assignmentsProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L84.** `.read(assignmentsProvider(groupId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L189.** `class StudentAssignmentDetailScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L196.** `final assignment = ref.watch(assignmentDetailProvider(assignmentId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L197.** `final tasks = ref.watch(tasksProvider(assignmentId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L209.** `ref.invalidate(assignmentDetailProvider(assignmentId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L210.** `ref.invalidate(tasksProvider(assignmentId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/screens/student/homework/student_assignments_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/homework/student_task_submit_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L21.** `class StudentTaskSubmitScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27.** `ConsumerState<StudentTaskSubmitScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L33.** `class _StudentTaskSubmitScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L34.** `extends ConsumerState<StudentTaskSubmitScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L54.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L55.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L56.** `.eq('id', widget.taskId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L58.** `if (task != null) setState(() => _task = Map<String, dynamic>.from(task));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L60.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L63.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L64.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L65.** `.eq('task_id', widget.taskId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L66.** `.eq('student_id', student.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L93.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L100.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L101.** `.createSignedUrl(path, _signedUrlTtlSeconds);`

Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L112.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L136.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L171.** `await supabase.storage.from('submissions').remove([path]);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L175.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L179.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L180.** `.update({'file_url': null})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L181.** `.eq('task_id', widget.taskId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L182.** `.eq('student_id', student.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L187.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L194.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L206.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L207.** `.createSignedUrl(path, _signedUrlTtlSeconds);`

Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L230.** `await supabase.storage.from('submissions').remove([path]);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L234.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L238.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L239.** `.update({'voice_url': null})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L240.** `.eq('task_id', widget.taskId)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L241.** `.eq('student_id', student.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L253.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L257.** `await supabase.from('submissions').upsert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L276.** `ref.invalidate(assignmentProgressProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L445.** `class _AttachAction extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/homework/student_task_submit_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/marks/student_marks_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L23.** `class StudentMarksScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L28.** `final marks = ref.watch(myMarksProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29.** `final stats = ref.watch(myMarkStatsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L30.** `final average = ref.watch(myMarkAverageProvider).valueOrNull;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L47.** `ref.invalidate(myMarksProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L48.** `ref.invalidate(myMarkStatsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L49.** `ref.invalidate(myMarkAverageProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L86.** `ref.read(myMarksProvider.notifier).loadMore(),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L106.** `final groupId = ref.read(activeGroupIdProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L114.** `final student = await ref.read(studentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L115.** `final profile = await ref.read(currentProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L125.** `final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L129.** `final term = ref.read(selectedMarkTermProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L163.** `class _CategoryBreakdown extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L244.** `class _StudentMarkTile extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**TASK — Predict a break in lib/screens/student/marks/student_marks_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/notifications/student_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `class StudentNotificationsScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `ConsumerState<StudentNotificationsScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L24.** `class _StudentNotificationsScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25.** `extends ConsumerState<StudentNotificationsScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L34.** `final profile = ref.read(currentProfileProvider).valueOrNull;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L37.** `ref.invalidate(notificationsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L43.** `context.push(path);`

Push a route the user can pop (assignment detail, chat room).

**L49.** `final notifications = ref.watch(notificationsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

> **TASK — Predict a break in lib/screens/student/notifications/student_notifications_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/student/profile/student_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class StudentProfileScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/student/profile/student_profile_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/analytics/teacher_analytics_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L15.** `class TeacherAnalyticsScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L30.** `final attendance = ref.watch(attendanceWeekTrendProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L31.** `final averages = ref.watch(groupMarkAveragesProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L37.** `ref.invalidate(attendanceWeekTrendProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L38.** `ref.invalidate(groupMarkAveragesProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **TASK — Predict a break in lib/screens/teacher/analytics/teacher_analytics_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/announcements/teacher_announcements_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `class TeacherAnnouncementsScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `ConsumerState<TeacherAnnouncementsScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L24.** `class _TeacherAnnouncementsScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25.** `extends ConsumerState<TeacherAnnouncementsScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L27.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L28.** `final groups = await ref.read(myGroupsProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L32.** `var groupId = ref.read(activeGroupIdProvider) ?? groups.first.id;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L80.** `await supabase.from('announcements').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L87.** `ref.invalidate(announcementsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L131.** `.from('announcements')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L132.** `.update({`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L136.** `.eq('id', a.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L138.** `ref.invalidate(announcementsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L170.** `await supabase.from('announcements').delete().eq('id', a.id);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L171.** `ref.invalidate(announcementsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L176.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L177.** `final items = ref.watch(announcementsProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L178.** `final myTeacherId = ref.watch(teacherProfileProvider).valueOrNull?.id;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**TASK — Predict a break in lib/screens/teacher/announcements/teacher_announcements_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/assignments/teacher_assignments_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L23.** `class TeacherAssignmentsScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27.** `ConsumerState<TeacherAssignmentsScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L31.** `class _TeacherAssignmentsScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L32.** `extends ConsumerState<TeacherAssignmentsScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L38.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L39.** `final groups = await ref.read(myGroupsProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L42.** `var targetGroupId = ref.read(activeGroupIdProvider) ?? groups.first.id;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L192.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L193.** `.insert({`

Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L201.** `.select()`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L203.** `created = Assignment.fromJson(Map<String, dynamic>.from(row));`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L205.** `await supabase.from('tasks').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L214.** `ref.invalidate(assignmentsProvider(targetGroupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L312.** `.from('assignments')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L313.** `.update({`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L318.** `.eq('id', a.id);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L321.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L322.** `.select('id')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L323.** `.eq('assignment_id', a.id)`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L327.** `await supabase.from('tasks').insert({`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L335.** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L336.** `.update({'title': title, 'description': description})`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L337.** `.eq('id', tasks.first['id'] as String);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L340.** `ref.invalidate(assignmentsProvider(a.groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L341.** `ref.invalidate(assignmentDetailProvider(a.id));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L342.** `ref.invalidate(tasksProvider(a.id));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L375.** `await supabase.from('assignments').delete().eq('id', a.id);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L376.** `ref.invalidate(assignmentsProvider(a.groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L390.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L391.** `final groups = await ref.read(myGroupsProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L394.** `var targetGroupId = ref.read(activeGroupIdProvider) ?? source.groupId;`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L477.** `ref.invalidate(assignmentsProvider(targetGroupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L479.** `ref.invalidate(assignmentsProvider(source.groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L497.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L518.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L540.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L541.** `final assignments = ref.watch(assignmentsProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L591.** `.read(assignmentsProvider(groupId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L703.** `class _TemplatesSheet extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L713.** `class _TemplatesSheetState extends State<_TemplatesSheet> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/assignments/teacher_assignments_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/attendance/teacher_attendance_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `class TeacherAttendanceScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20.** `ConsumerState<TeacherAttendanceScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L24.** `class _TeacherAttendanceScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25.** `extends ConsumerState<TeacherAttendanceScreen>`

Widget that can watch providers. Required for profile/theme/locale.

**L26.** `with SingleTickerProviderStateMixin {`

Riverpod binding. Name it after the value it exposes.

**L89.** `ref.invalidate(todayAttendanceProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L90.** `ref.invalidate(attendanceWeekTrendProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L91.** `ref.invalidate(attendanceRangeProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L150.** `ref.invalidate(todayAttendanceProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L151.** `ref.invalidate(attendanceWeekTrendProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L152.** `ref.invalidate(attendanceRangeProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L181.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L194.** `final members = ref.watch(groupMembersProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L195.** `final saved = ref.watch(todayAttendanceProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L476.** `class _TeacherAttendanceHistoryTab extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L513.** `final records = ref.watch(attendanceRangeProvider(query));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L514.** `final stats = ref.watch(attendanceRangeStatsProvider(query));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L594.** `class _BulkActions extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L685.** `class _AttendanceToggle extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L738.** `class _StatusButton extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L793.** `class _LegendItem extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/attendance/teacher_attendance_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/dashboard/teacher_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L18.** `class TeacherDashboardScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L23.** `final profile = ref.watch(currentProfileProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L24.** `final teacher = ref.watch(teacherProfileProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L25.** `final groups = ref.watch(myGroupsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L139.** `ref.invalidate(currentProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L140.** `ref.invalidate(teacherProfileProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L141.** `ref.invalidate(myGroupsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L247.** `.watch(notificationsProvider)`

Riverpod binding. Name it after the value it exposes.

**L361.** `class _StatCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L420.** `class _QuickActionCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/dashboard/teacher_dashboard_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/homework/teacher_homework_pack_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L15.** `class TeacherHomeworkPackScreen extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21.** `ConsumerState<TeacherHomeworkPackScreen> createState() =>`

Widget that can watch providers. Required for profile/theme/locale.

**L25.** `class _TeacherHomeworkPackScreenState`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26.** `extends ConsumerState<TeacherHomeworkPackScreen> {`

Widget that can watch providers. Required for profile/theme/locale.

**L29.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L51.** `.read(assignmentDetailProvider(widget.assignmentId))`

Riverpod binding. Name it after the value it exposes.

**L133.** `await supabase.from('tasks').delete().eq('id', task.id);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L135.** `ref.invalidate(tasksProvider(widget.assignmentId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L164.** `await supabase.from('tasks').update(data).eq('id', task.id);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L167.** `await supabase.from('tasks').insert(data);`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L171.** `ref.invalidate(tasksProvider(widget.assignmentId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L182.** `final assignment = ref.watch(assignmentDetailProvider(widget.assignmentId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L183.** `final tasks = ref.watch(tasksProvider(widget.assignmentId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L227.** `ref.invalidate(assignmentDetailProvider(widget.assignmentId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L228.** `ref.invalidate(tasksProvider(widget.assignmentId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**TASK — Predict a break in lib/screens/teacher/homework/teacher_homework_pack_screen.dart**
Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/homework/teacher_submissions_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L19.** `class TeacherSubmissionsScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24.** `final submissions = ref.watch(teacherSubmissionsProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L41.** `onRefresh: () async => ref.invalidate(teacherSubmissionsProvider),`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L102.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L120.** `ref.invalidate(teacherSubmissionsProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L121.** `final groupId = ref.read(activeGroupIdProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L123.** `ref.invalidate(groupMarksProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L124.** `ref.invalidate(groupMarkAveragesProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L129.** `class _ReviewSubmissionDialog extends StatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L140.** `class _ReviewSubmissionDialogState extends State<_ReviewSubmissionDialog> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L197.** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L198.** `.update({`

Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L203.** `.eq('id', widget.row['id']);`

Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

> **TASK — Predict a break in lib/screens/teacher/homework/teacher_submissions_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/marks/teacher_marks_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L23.** `class TeacherMarksScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L28.** `final groupId = ref.watch(activeGroupIdProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L42.** `final members = ref.watch(groupMembersProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L43.** `final averages = ref.watch(groupMarkAveragesProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L44.** `final marks = ref.watch(groupMarksProvider(groupId));`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L46.** `ref.watch(groupMarkCountProvider(groupId)).valueOrNull ?? 0;`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L47.** `final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L157.** `.read(groupMarksProvider(groupId).notifier)`

Riverpod binding. Name it after the value it exposes.

**L173.** `ref.invalidate(groupMembersProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L174.** `ref.invalidate(groupMarkAveragesProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L175.** `ref.invalidate(groupMarkCountProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L176.** `ref.invalidate(groupMarksProvider(groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L177.** `ref.invalidate(memberMarksProvider);`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L191.** `term: ref.read(selectedMarkTermProvider),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L241.** `final members = await ref.read(groupMembersProvider(groupId).future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L324.** `class _ClassSummaryCard extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L388.** `class _SummaryStat extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L416.** `class _StudentRow extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/marks/teacher_marks_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L11.** `class BulkMarkSheet extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L22.** `ConsumerState<BulkMarkSheet> createState() => _BulkMarkSheetState();`

Widget that can watch providers. Required for profile/theme/locale.

**L25.** `class _BulkMarkSheetState extends ConsumerState<BulkMarkSheet> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L79.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/marks/widgets/gradebook_helpers.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L21.** `class GradebookSheetScaffold extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L82.** `class GradebookCategoryDropdown extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L108.** `class GradebookDateField extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/marks/widgets/gradebook_helpers.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/marks/widgets/mark_editor_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L12.** `class MarkEditorSheet extends ConsumerStatefulWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27.** `ConsumerState<MarkEditorSheet> createState() => _MarkEditorSheetState();`

Widget that can watch providers. Required for profile/theme/locale.

**L30.** `class _MarkEditorSheetState extends ConsumerState<MarkEditorSheet> {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L78.** `final teacher = await ref.read(teacherProfileProvider.future);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/screens/teacher/marks/widgets/mark_editor_sheet.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/marks/widgets/mark_row.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L10.** `class MarkRow extends StatelessWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/marks/widgets/mark_row.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/marks/widgets/student_marks_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L16.** `class StudentMarksSheet extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L28.** `final marks = ref.watch(`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29.** `memberMarksProvider((groupId: groupId, studentId: member.studentId)),`

Riverpod binding. Name it after the value it exposes.

**L33.** `.watch(groupMarkAveragesProvider(groupId))`

Riverpod binding. Name it after the value it exposes.

**L97.** `final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L101.** `final term = ref.read(selectedMarkTermProvider);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

> **TASK — Predict a break in lib/screens/teacher/marks/widgets/student_marks_sheet.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/notifications/teacher_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class TeacherNotificationsScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/notifications/teacher_notifications_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/screens/teacher/profile/teacher_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L5.** `class TeacherProfileScreen extends ConsumerWidget {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

> **TASK — Predict a break in lib/screens/teacher/profile/teacher_profile_screen.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/supabase/supabase_client.dart

Thin accessor: supabase = Supabase.instance.client. This file (supabase client) is one concrete piece of that layer.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

This file is mostly layout, colors, or pure functions. That is still product code: a theme token file is how Nature Paper stays consistent. Read it top to bottom once, then return when a screen imports it.

> **TASK — Predict a break in lib/supabase/supabase_client.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# Line lesson — lib/tools/generate_school_sql.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

No top-level class/provider parsed (the file may be a barrel export or use syntax the lesson parser skips). Read it as glue code.

## Why these lines exist

**L137.** `jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),`

Riverpod binding. Name it after the value it exposes.

**L143.** `id, user_id, identity_data, provider, provider_id, last_sign_in_at, created_at, updated_at`

Riverpod binding. Name it after the value it exposes.

> **TASK — Predict a break in lib/tools/generate_school_sql.dart**
> Name one line above. Describe the user-visible bug if you delete it. Then restore it in your mind — do not actually break a working tree unless you are on a branch.

# SQL function atlas — every exported function

Each file under docs/tutorial/export/fn_*.sql is a live function body recovered from the MU Study database. Read the name, then the LANGUAGE and SECURITY clause, then the body. SECURITY DEFINER functions run as the owner — they must check is_admin() or membership themselves because RLS on tables they touch may be bypassed.

## fn__demo_insert_user.sql

Function family: **_demo_insert_user**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn__demo_insert_user.sql**

```
CREATE OR REPLACE FUNCTION public._demo_insert_user(p_username text, p_password text,
p_full_name text, p_role user_role)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE v_user_id UUID := gen_random_uuid(); v_email TEXT;
BEGIN
  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';
  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000', v_user_id, 'authenticated', 'authenticated',
    v_email, extensions.crypt(p_password, extensions.gen_salt('bf')), NOW(),
    jsonb_build_object('provider','email','providers',ARRAY['email']),
    jsonb_build_object('username', p_username, 'full_name', p_full_name, 'role', p_role::text),
    NOW(), NOW(), '', '', '', '', false, false
  );
  INSERT INTO auth.identities (id, user_id, identity_data, provider, provider_id,
last_sign_in_at, created_at, updated_at)
  VALUES (
    gen_random_uuid(), v_user_id,
    jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
    'email', v_email, NOW(), NOW(), NOW()
  );
  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);
  IF p_role = 'student' THEN INSERT INTO public.students (profile_id) VALUES (v_user_id); END
IF;
  RETURN v_user_id;
END; $function$
```

> **CHECKPOINT — Explain fn__demo_insert_user.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_academic_semester_bounds.sql

Function family: **academic_semester_bounds**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

**fn_academic_semester_bounds.sql**

```
CREATE OR REPLACE FUNCTION public.academic_semester_bounds(p_start_year integer, p_semester
integer)
 RETURNS TABLE(starts_on date, ends_on date)
 LANGUAGE plpgsql
 IMMUTABLE
AS $function$
BEGIN
  IF p_semester < 1 OR p_semester > 4 THEN
    RAISE EXCEPTION 'semester must be 1–4';
  END IF;

  RETURN QUERY
  SELECT
    CASE p_semester
      WHEN 1 THEN make_date(p_start_year, 8, 1)
      WHEN 2 THEN make_date(p_start_year, 11, 11)
      WHEN 3 THEN make_date(p_start_year + 1, 1, 11)
      ELSE make_date(p_start_year + 1, 3, 28)
    END,
    CASE p_semester
      WHEN 1 THEN make_date(p_start_year, 11, 2)
      WHEN 2 THEN make_date(p_start_year, 12, 28)
      WHEN 3 THEN make_date(p_start_year + 1, 3, 20)
      ELSE make_date(p_start_year + 1, 6, 19)
    END;
END;
$function$
```

> **CHECKPOINT — Explain fn_academic_semester_bounds.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_academic_year_label.sql

Function family: **academic_year_label**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

**fn_academic_year_label.sql**

```
CREATE OR REPLACE FUNCTION public.academic_year_label(p_start_year integer)
 RETURNS text
 LANGUAGE sql
 IMMUTABLE
AS $function$
  SELECT p_start_year::TEXT || '–' || lpad(((p_start_year + 1) % 100)::TEXT, 2, '0');
$function$
```

> **CHECKPOINT — Explain fn_academic_year_label.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_create_next_academic_year.sql

Function family: **admin_create_next_academic_year**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_create_next_academic_year.sql**

```
CREATE OR REPLACE FUNCTION public.admin_create_next_academic_year()
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_next INT;
  v_now DATE := CURRENT_DATE;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can create academic years';
  END IF;

  SELECT COALESCE(MAX(start_year), CASE
    WHEN EXTRACT(MONTH FROM v_now) >= 8 THEN EXTRACT(YEAR FROM v_now)::INT - 1
    ELSE EXTRACT(YEAR FROM v_now)::INT - 2
  END) + 1
  INTO v_next
  FROM public.academic_years;

  RETURN public.create_academic_year_with_terms(
    v_next,
    FALSE,
    auth.uid()
  );
END;
$function$
```

> **CHECKPOINT — Explain fn_admin_create_next_academic_year.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_create_user.sql

Function family: **admin_create_user**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_create_user.sql**

```
CREATE OR REPLACE FUNCTION public.admin_create_user(p_username text, p_password text,
p_full_name text, p_role user_role, p_sphere sphere_type DEFAULT NULL::sphere_type)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';

  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000',
    v_user_id,
    'authenticated',
    'authenticated',
    v_email,
    extensions.crypt(p_password, extensions.gen_salt('bf')),
    NOW(),
    jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),
    jsonb_build_object(
      'username', p_username,
      'full_name', p_full_name,
      'role', p_role::text
    ),
    NOW(),
    NOW(),
    '',
    '',
    '',
    '',
    false,
    false
  );

  INSERT INTO auth.identities (
    id, user_id, identity_data, provider, provider_id,
    last_sign_in_at, created_at, updated_at
  ) VALUES (
    gen_random_uuid(),
    v_user_id,
    jsonb_build_object(
      'sub', v_user_id::text,
      'email', v_email,
      'email_verified', true
    ),
```

```
    'email',
    v_email,
    NOW(),
    NOW(),
    NOW()
  );

  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);

  IF EXISTS (
    SELECT 1
    FROM information_schema.tables
    WHERE table_schema = 'public' AND table_name = 'login_credentials'
  ) THEN
    INSERT INTO public.login_credentials (profile_id, password, updated_at)
    VALUES (v_user_id, p_password, NOW())
    ON CONFLICT (profile_id) DO UPDATE
      SET password = excluded.password,
          updated_at = NOW();
  END IF;

  IF p_role = 'teacher' THEN
    IF p_sphere IS NULL THEN
      RAISE EXCEPTION 'Sphere required for teacher';
    END IF;
    -- Do NOT auto-create Group 1/2/3. Admins add groups manually.
    INSERT INTO public.teachers (profile_id, sphere)
    VALUES (v_user_id, p_sphere);
  ELSIF p_role = 'student' THEN
    INSERT INTO public.students (profile_id) VALUES (v_user_id);
  END IF;

  RETURN v_user_id;
END;
$function$
```

**CHECKPOINT — Explain fn_admin_create_user.sql**
In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_delete_academic_year.sql

Function family: **admin_delete_academic_year**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_delete_academic_year.sql**

```
CREATE OR REPLACE FUNCTION public.admin_delete_academic_year(p_year_id uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
  v_remaining INT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can delete academic years';
  END IF;

  SELECT * INTO v_year
  FROM public.academic_years
  WHERE id = p_year_id;

  IF v_year.id IS NULL THEN
    RAISE EXCEPTION 'Academic year not found';
  END IF;

  IF v_year.is_current THEN
    RAISE EXCEPTION 'Cannot delete the current academic year. Set another year as current first.
';
  END IF;

  SELECT COUNT(*) INTO v_remaining FROM public.academic_years;
  IF v_remaining <= 1 THEN
    RAISE EXCEPTION 'Cannot delete the last academic year';
  END IF;

  DELETE FROM public.academic_years
  WHERE id = p_year_id
  RETURNING * INTO v_year;

  RETURN v_year;
END;
$function$
```

> **CHECKPOINT — Explain fn_admin_delete_academic_year.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_mass_generate_school_data.sql

Function family: **admin_mass_generate_school_data**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_mass_generate_school_data.sql**

```
CREATE OR REPLACE FUNCTION public.admin_mass_generate_school_data(p_teachers jsonb, p_students
jsonb, p_groups jsonb, p_assignments jsonb)
 RETURNS jsonb
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE
  v_errors TEXT[] := ARRAY[]::TEXT[];

  v_teacher_upserted INT := 0;
  v_student_upserted INT := 0;
  v_groups_created INT := 0;
  v_assignments_inserted INT := 0;

  -- Auth/profile ids
  v_profile_id UUID;
  v_user_id UUID;
  v_teacher_id UUID;
  v_student_id UUID;
  v_group_id UUID;

  -- temp strings
  v_username TEXT;
  v_password TEXT;
  v_full_name TEXT;
  v_sphere TEXT;
  v_email TEXT;

  v_teacher_username TEXT;
  v_group_title TEXT;
  v_student_username TEXT;

  v_row JSONB;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  -- ==========================
  -- Teachers + teacher rows
  -- ==========================
  IF p_teachers IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_teachers)
    LOOP
      v_username := trim(both from v_row->>'username');
      v_password := v_row->>'password';
      v_full_name := trim(both from v_row->>'full_name');
      v_sphere := lower(trim(both from v_row->>'sphere'));

      IF v_username IS NULL OR v_username = '' OR v_password IS NULL OR v_password = '' THEN
        v_errors := v_errors || format('Teacher missing username/password: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      IF v_sphere IS NULL OR (v_sphere <> 'math' AND v_sphere <> 'english') THEN
```

```
        v_errors := v_errors || format('Teacher invalid sphere for %s: %s', v_username,
COALESCE(v_sphere,'<null>'));
        CONTINUE;
      END IF;

      v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

      -- Existing profile?
      SELECT id INTO v_profile_id
      FROM public.profiles
      WHERE username = v_username
      LIMIT 1;

      IF v_profile_id IS NULL THEN
        v_user_id := gen_random_uuid();

        INSERT INTO auth.users (
          instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
          raw_app_meta_data, raw_user_meta_data,
          created_at, updated_at,
          confirmation_token, recovery_token, email_change_token_new, email_change,
          is_sso_user, is_anonymous
        ) VALUES (
          '00000000-0000-0000-0000-000000000000',
          v_user_id,
          'authenticated',
          'authenticated',
          v_email,
          extensions.crypt(v_password, extensions.gen_salt('bf')),
          NOW(),
          jsonb_build_object('provider','email','providers',ARRAY['email']),
          jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role',
'teacher'),
          NOW(),
          NOW(),
          '', '', '', '',
          false, false
        );

        INSERT INTO auth.identities (
          id, user_id, identity_data, provider, provider_id,
          last_sign_in_at, created_at, updated_at
        ) VALUES (
          gen_random_uuid(),
          v_user_id,
          jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
          'email',
          v_user_id::text,
          NOW(), NOW(), NOW()
        );

        INSERT INTO public.profiles (id, username, full_name, role, auth_email)
        VALUES (v_user_id, v_username, COALESCE(v_full_name,'') , 'teacher', v_email);

        v_profile_id := v_user_id;
      ELSE
        -- Update password + basic profile fields
        v_user_id := v_profile_id;
        UPDATE auth.users
        SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
            updated_at = NOW()
        WHERE id = v_user_id;

        UPDATE public.profiles
        SET full_name = COALESCE(v_full_name,''),
            role = 'teacher',
            auth_email = v_email
        WHERE id = v_user_id;
      END IF;

      -- Upsert teachers row
      INSERT INTO public.teachers (profile_id, sphere)
      VALUES (
```

```
        v_profile_id,
        CASE WHEN v_sphere = 'math' THEN 'math'::public.sphere_type ELSE 'english'::public.
sphere_type END
      )
      ON CONFLICT (profile_id) DO UPDATE
      SET sphere = EXCLUDED.sphere;

      v_teacher_upserted := v_teacher_upserted + 1;
    END LOOP;
  END IF;

  -- ==========================
  -- Students + student rows
  -- ==========================
  IF p_students IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_students)
    LOOP
      v_username := trim(both from v_row->>'username');
      v_password := v_row->>'password';
      v_full_name := trim(both from v_row->>'full_name');

      IF v_username IS NULL OR v_username = '' OR v_password IS NULL OR v_password = '' THEN
        v_errors := v_errors || format('Student missing username/password: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

      SELECT id INTO v_profile_id
      FROM public.profiles
      WHERE username = v_username
      LIMIT 1;

      IF v_profile_id IS NULL THEN
        v_user_id := gen_random_uuid();

        INSERT INTO auth.users (
          instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
          raw_app_meta_data, raw_user_meta_data,
          created_at, updated_at,
          confirmation_token, recovery_token, email_change_token_new, email_change,
          is_sso_user, is_anonymous
        ) VALUES (
          '00000000-0000-0000-0000-000000000000',
          v_user_id,
          'authenticated',
          'authenticated',
          v_email,
          extensions.crypt(v_password, extensions.gen_salt('bf')),
          NOW(),
          jsonb_build_object('provider','email','providers',ARRAY['email']),
          jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role',
'student'),
          NOW(),
          NOW(),
          '', '', '', '',
          false, false
        );

        INSERT INTO auth.identities (
          id, user_id, identity_data, provider, provider_id,
          last_sign_in_at, created_at, updated_at
        ) VALUES (
          gen_random_uuid(),
          v_user_id,
          jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
          'email',
          v_user_id::text,
          NOW(), NOW(), NOW()
        );

        INSERT INTO public.profiles (id, username, full_name, role, auth_email)
        VALUES (v_user_id, v_username, COALESCE(v_full_name,'') , 'student', v_email);
```

```
        v_profile_id := v_user_id;
      ELSE
        v_user_id := v_profile_id;
        UPDATE auth.users
        SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
            updated_at = NOW()
        WHERE id = v_user_id;

        UPDATE public.profiles
        SET full_name = COALESCE(v_full_name,''),
            role = 'student',
            auth_email = v_email
        WHERE id = v_user_id;
      END IF;

      INSERT INTO public.students (profile_id)
      VALUES (v_profile_id)
      ON CONFLICT (profile_id) DO NOTHING;

      v_student_upserted := v_student_upserted + 1;
    END LOOP;
  END IF;

  -- ==========================
  -- Groups (exact titles)
  -- ==========================
  IF p_groups IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_groups)
    LOOP
      v_teacher_username := trim(both from v_row->>'teacher_username');
      v_group_title := trim(both from v_row->>'title');

      IF v_teacher_username IS NULL OR v_teacher_username = '' OR v_group_title IS NULL OR
v_group_title = '' THEN
        v_errors := v_errors || format('Group missing teacher_username/title: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      SELECT t.id INTO v_teacher_id
      FROM public.teachers t
      JOIN public.profiles p ON p.id = t.profile_id
      WHERE p.username = v_teacher_username
        AND p.role = 'teacher'
      LIMIT 1;

      IF v_teacher_id IS NULL THEN
        v_errors := v_errors || format('Unknown teacher for group %s (teacher_username=%s)',
v_group_title, v_teacher_username);
        CONTINUE;
      END IF;

      SELECT g.id INTO v_group_id
      FROM public.groups g
      WHERE g.teacher_id = v_teacher_id
        AND g.title = v_group_title
      LIMIT 1;

      IF v_group_id IS NULL THEN
        INSERT INTO public.groups (teacher_id, title)
        VALUES (v_teacher_id, v_group_title);
        v_groups_created := v_groups_created + 1;
      END IF;
    END LOOP;
  END IF;

  -- ==========================
  -- Assign students -> groups
  -- ==========================
  IF p_assignments IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_assignments)
    LOOP
      v_teacher_username := trim(both from v_row->>'teacher_username');
      v_group_title := trim(both from v_row->>'group_title');
```

```
      v_student_username := trim(both from v_row->>'student_username');

      IF v_teacher_username IS NULL OR v_teacher_username = '' OR
         v_group_title IS NULL OR v_group_title = '' OR
         v_student_username IS NULL OR v_student_username = '' THEN
        v_errors := v_errors || format('Assignment missing teacher_username/group_title/
student_username: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      -- Resolve group_id by teacher_username + group_title
      SELECT g.id INTO v_group_id
      FROM public.groups g
      JOIN public.teachers t ON t.id = g.teacher_id
      JOIN public.profiles p ON p.id = t.profile_id
      WHERE p.username = v_teacher_username
        AND p.role = 'teacher'
        AND g.title = v_group_title
      LIMIT 1;

      IF v_group_id IS NULL THEN
        v_errors := v_errors || format('Unknown group for assignment %s -> (teacher=%s,
group_title=%s)', v_student_username, v_teacher_username, v_group_title);
        CONTINUE;
      END IF;

      -- Resolve student_id
      SELECT s.id INTO v_student_id
      FROM public.students s
      JOIN public.profiles p ON p.id = s.profile_id
      WHERE p.username = v_student_username
        AND p.role = 'student'
      LIMIT 1;

      IF v_student_id IS NULL THEN
        v_errors := v_errors || format('Unknown student username: %s', v_student_username);
        CONTINUE;
      END IF;

      INSERT INTO public.group_students (group_id, student_id)
      VALUES (v_group_id, v_student_id)
      ON CONFLICT (group_id, student_id) DO NOTHING;

      -- We don't have a portable "rowcount" here without extra GET DIAGNOSTICS.
      -- Still count by checking existence after insert.
      IF EXISTS (
        SELECT 1 FROM public.group_students gs
        WHERE gs.group_id = v_group_id AND gs.student_id = v_student_id
      ) THEN
        v_assignments_inserted := v_assignments_inserted + 1;
      END IF;
    END LOOP;
  END IF;

  RETURN jsonb_build_object(
    'teachers_upserted', v_teacher_upserted,
    'students_upserted', v_student_upserted,
    'groups_created', v_groups_created,
    'assignments_inserted', v_assignments_inserted,
    'errors', to_jsonb(v_errors)
  );
END;
$function$
```

**CHECKPOINT — Explain fn_admin_mass_generate_school_data.sql**
In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_reset_password.sql

Function family: **admin_reset_password**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_reset_password.sql**

```
CREATE OR REPLACE FUNCTION public.admin_reset_password(p_user_id uuid, p_password text)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'auth', 'public', 'extensions'
AS $function$
BEGIN
  IF NOT public.is_admin() THEN RAISE EXCEPTION 'Not authorized'; END IF;
  UPDATE auth.users SET encrypted_password = extensions.crypt(p_password, extensions.gen_salt(
'bf')), updated_at = NOW() WHERE id = p_user_id;
END; $function$
```

> **CHECKPOINT — Explain fn_admin_reset_password.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_set_current_academic_year.sql

Function family: **admin_set_current_academic_year**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_set_current_academic_year.sql**

```
CREATE OR REPLACE FUNCTION public.admin_set_current_academic_year(p_year_id uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can set the current academic year';
  END IF;

  UPDATE public.academic_years SET is_current = FALSE WHERE is_current;

  UPDATE public.academic_years
  SET is_current = TRUE
  WHERE id = p_year_id
  RETURNING * INTO v_year;

  IF v_year.id IS NULL THEN
    RAISE EXCEPTION 'Academic year not found';
  END IF;

  RETURN v_year;
END;
$function$
```

> **CHECKPOINT — Explain fn_admin_set_current_academic_year.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_admin_update_academic_term.sql

Function family: **admin_update_academic_term**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_admin_update_academic_term.sql**

```
CREATE OR REPLACE FUNCTION public.admin_update_academic_term(p_term_id uuid, p_starts_on date,
p_ends_on date)
 RETURNS academic_terms
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_term public.academic_terms;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can edit term dates';
  END IF;

  IF p_ends_on < p_starts_on THEN
    RAISE EXCEPTION 'End date must be on or after start date';
  END IF;

  UPDATE public.academic_terms
  SET starts_on = p_starts_on, ends_on = p_ends_on
  WHERE id = p_term_id
  RETURNING * INTO v_term;

  IF v_term.id IS NULL THEN
    RAISE EXCEPTION 'Academic term not found';
  END IF;

  RETURN v_term;
END;
$function$
```

> **CHECKPOINT — Explain fn_admin_update_academic_term.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_attendance_sync_session.sql

Function family: **attendance_sync_session**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_attendance_sync_session.sql**

```
CREATE OR REPLACE FUNCTION public.attendance_sync_session()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  IF NEW.date IS NOT NULL AND NEW.date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot mark attendance for a future date'
      USING ERRCODE = 'check_violation';
  END IF;

  IF NEW.session_id IS NULL AND NEW.group_id IS NOT NULL AND NEW.date IS NOT NULL THEN
    NEW.session_id := public.ensure_lesson_session(NEW.group_id, NEW.date::date);
  ELSIF NEW.session_id IS NOT NULL THEN
    SELECT ls.group_id, ls.session_date
      INTO NEW.group_id, NEW.date
    FROM public.lesson_sessions ls
    WHERE ls.id = NEW.session_id;
  END IF;

  IF NEW.date IS NOT NULL AND NEW.date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot mark attendance for a future date'
      USING ERRCODE = 'check_violation';
  END IF;

  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_attendance_sync_session.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_can_access_chat_room.sql

Function family: **can_access_chat_room**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_can_access_chat_room.sql**

```
CREATE OR REPLACE FUNCTION public.can_access_chat_room(rid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT public.is_admin()
    OR EXISTS (
      SELECT 1
      FROM public.chat_rooms cr
      WHERE cr.id = rid
        AND (
          public.is_teacher_of_group(cr.group_id)
          OR public.is_student_in_group(cr.group_id)
        )
    );
$function$
```

> **CHECKPOINT — Explain fn_can_access_chat_room.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_chat_messages_reply_same_room.sql

Function family: **chat_messages_reply_same_room**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

**fn_chat_messages_reply_same_room.sql**

```
CREATE OR REPLACE FUNCTION public.chat_messages_reply_same_room()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
  v_room UUID;
BEGIN
  IF NEW.reply_to_message_id IS NULL THEN
    RETURN NEW;
  END IF;
  IF NEW.reply_to_message_id = NEW.id THEN
    RAISE EXCEPTION 'message cannot reply to itself';
  END IF;
  SELECT room_id INTO v_room
  FROM public.chat_messages
  WHERE id = NEW.reply_to_message_id;
  IF v_room IS NULL THEN
    RAISE EXCEPTION 'reply target not found';
  END IF;
  IF v_room <> NEW.room_id THEN
    RAISE EXCEPTION 'reply must be in the same chat room';
  END IF;
  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_chat_messages_reply_same_room.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_chat_storage_room_id.sql

Function family: **chat_storage_room_id**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

**fn_chat_storage_room_id.sql**

```
CREATE OR REPLACE FUNCTION public.chat_storage_room_id(object_name text)
 RETURNS uuid
 LANGUAGE plpgsql
 IMMUTABLE
 SET search_path TO 'public'
AS $function$
DECLARE
  part TEXT := split_part(object_name, '/', 1);
BEGIN
  IF part ~* '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$' THEN
    RETURN part::UUID;
  END IF;
  RETURN NULL;
END;
$function$
```

> **CHECKPOINT — Explain fn_chat_storage_room_id.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_chat_unread_counts.sql

Function family: **chat_unread_counts**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_chat_unread_counts.sql**

```
CREATE OR REPLACE FUNCTION public.chat_unread_counts()
 RETURNS TABLE(room_id uuid, unread_count bigint)
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT
    m.room_id,
    COUNT(*)::BIGINT AS unread_count
  FROM public.chat_messages m
  WHERE m.is_deleted = FALSE
    AND m.sender_id <> auth.uid()
    AND public.can_access_chat_room(m.room_id)
    AND NOT EXISTS (
      SELECT 1
      FROM public.message_reads r
      WHERE r.message_id = m.id
        AND r.user_id = auth.uid()
    )
  GROUP BY m.room_id;
$function$
```

> **CHECKPOINT — Explain fn_chat_unread_counts.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_create_academic_year_with_terms.sql

Function family: **create_academic_year_with_terms**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_create_academic_year_with_terms.sql**

```
CREATE OR REPLACE FUNCTION public.create_academic_year_with_terms(p_start_year integer,
p_is_current boolean DEFAULT false, p_created_by uuid DEFAULT NULL::uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
  v_sem INT;
  v_bounds RECORD;
BEGIN
  IF p_is_current THEN
    UPDATE public.academic_years SET is_current = FALSE WHERE is_current;
  END IF;

  INSERT INTO public.academic_years (start_year, label, is_current, created_by)
  VALUES (
    p_start_year,
    public.academic_year_label(p_start_year),
    p_is_current,
    p_created_by
  )
  RETURNING * INTO v_year;

  FOR v_sem IN 1..4 LOOP
    SELECT * INTO v_bounds
    FROM public.academic_semester_bounds(p_start_year, v_sem);

    INSERT INTO public.academic_terms (
      academic_year_id, semester, starts_on, ends_on
    ) VALUES (
      v_year.id, v_sem, v_bounds.starts_on, v_bounds.ends_on
    );
  END LOOP;

  RETURN v_year;
END;
$function$
```

> **CHECKPOINT — Explain fn_create_academic_year_with_terms.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_current_profile_role.sql

Function family: **current_profile_role**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_current_profile_role.sql**

```
CREATE OR REPLACE FUNCTION public.current_profile_role()
 RETURNS user_role
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$ SELECT role FROM public.profiles WHERE id = auth.uid(); $function$
```

> **CHECKPOINT — Explain fn_current_profile_role.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_ensure_chat_room_for_group.sql

Function family: **ensure_chat_room_for_group**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_ensure_chat_room_for_group.sql**

```
CREATE OR REPLACE FUNCTION public.ensure_chat_room_for_group()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  INSERT INTO public.chat_rooms (group_id)
  VALUES (NEW.id)
  ON CONFLICT (group_id) DO NOTHING;
  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_ensure_chat_room_for_group.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_ensure_lesson_session.sql

Function family: **ensure_lesson_session**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_ensure_lesson_session.sql**

```
CREATE OR REPLACE FUNCTION public.ensure_lesson_session(p_group_id uuid, p_session_date date)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  sid UUID;
BEGIN
  IF p_session_date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot create a lesson session for a future date'
      USING ERRCODE = 'check_violation';
  END IF;

  INSERT INTO public.lesson_sessions (group_id, session_date)
  VALUES (p_group_id, p_session_date)
  ON CONFLICT (group_id, session_date)
  DO UPDATE SET group_id = EXCLUDED.group_id
  RETURNING id INTO sid;
  RETURN sid;
END;
$function$
```

> **CHECKPOINT — Explain fn_ensure_lesson_session.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_get_auth_email_by_username.sql

Function family: **get_auth_email_by_username**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_get_auth_email_by_username.sql**

```
CREATE OR REPLACE FUNCTION public.get_auth_email_by_username(p_username text)
 RETURNS text
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT auth_email FROM public.profiles
  WHERE lower(username) = lower(trim(p_username)) AND is_disabled = FALSE LIMIT 1;
$function$
```

> **CHECKPOINT — Explain fn_get_auth_email_by_username.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_is_admin.sql

Function family: **is_admin**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_is_admin.sql**

```
CREATE OR REPLACE FUNCTION public.is_admin()
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin' AND is_disabled =
FALSE
  );
$function$
```

> **CHECKPOINT — Explain fn_is_admin.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_is_student_in_group.sql

Function family: **is_student_in_group**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_is_student_in_group.sql**

```
CREATE OR REPLACE FUNCTION public.is_student_in_group(gid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = gid AND s.profile_id = auth.uid()
  );
$function$
```

> **CHECKPOINT — Explain fn_is_student_in_group.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_is_teacher_of_group.sql

Function family: **is_teacher_of_group**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_is_teacher_of_group.sql**

```
CREATE OR REPLACE FUNCTION public.is_teacher_of_group(gid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.groups g
    JOIN public.teachers t ON t.id = g.teacher_id
    WHERE g.id = gid AND t.profile_id = auth.uid()
  );
$function$
```

> **CHECKPOINT — Explain fn_is_teacher_of_group.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_notify_chat_message.sql

Function family: **notify_chat_message**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_notify_chat_message.sql**

```
CREATE OR REPLACE FUNCTION public.notify_chat_message(p_user_id uuid, p_title text, p_body text,
 p_action_path text, p_room_id uuid)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_id UUID;
BEGIN
  SELECT id INTO v_id
  FROM public.notifications
  WHERE user_id = p_user_id
    AND kind = 'chat'
    AND ref_id = p_room_id
    AND is_read = FALSE
  ORDER BY created_at DESC
  LIMIT 1;

  IF v_id IS NOT NULL THEN
    UPDATE public.notifications
    SET title = p_title,
        body = COALESCE(p_body, ''),
        action_path = p_action_path,
        created_at = NOW()
    WHERE id = v_id;
  ELSE
    PERFORM public.notify_user(
      p_user_id, p_title, p_body, 'chat', p_action_path, p_room_id
    );
  END IF;
END;
$function$
```

> **CHECKPOINT — Explain fn_notify_chat_message.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_notify_user.sql

Function family: **notify_user**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_notify_user.sql**

```
CREATE OR REPLACE FUNCTION public.notify_user(p_user_id uuid, p_title text, p_body text, p_kind
text DEFAULT 'system'::text, p_action_path text DEFAULT NULL::text, p_ref_id uuid DEFAULT NULL::
uuid)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  INSERT INTO public.notifications (user_id, title, body, kind, action_path, ref_id)
  VALUES (
    p_user_id,
    p_title,
    COALESCE(p_body, ''),
    COALESCE(NULLIF(trim(p_kind), ''), 'system'),
    p_action_path,
    p_ref_id
  );
END;
$function$
```

> **CHECKPOINT — Explain fn_notify_user.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_profile_route_prefix.sql

Function family: **profile_route_prefix**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_profile_route_prefix.sql**

```
CREATE OR REPLACE FUNCTION public.profile_route_prefix(p_user_id uuid)
 RETURNS text
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT CASE role
    WHEN 'teacher' THEN '/teacher'
    WHEN 'admin' THEN '/admin'
    ELSE '/student'
  END
  FROM public.profiles
  WHERE id = p_user_id;
$function$
```

> **CHECKPOINT — Explain fn_profile_route_prefix.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_seed_demo_data.sql

Function family: **seed_demo_data**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_seed_demo_data.sql**

```
CREATE OR REPLACE FUNCTION public.seed_demo_data()
 RETURNS text
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth'
AS $function$
DECLARE
  v_admin UUID; v_math UUID; v_eng UUID; v_student UUID;
  v_math_tid UUID; v_eng_tid UUID; v_student_sid UUID;
  v_group UUID; v_pack UUID;
BEGIN
  IF EXISTS (SELECT 1 FROM public.profiles LIMIT 1) THEN RETURN 'already_seeded'; END IF;

  v_admin := public._demo_insert_user('@admin', 'password123', 'Platform Admin', 'admin');
  v_math := public._demo_insert_user('@math_teacher', 'password123', 'Amina Math', 'teacher');
  v_eng := public._demo_insert_user('@english_teacher', 'password123', 'James English',
'teacher');
  v_student := public._demo_insert_user('@test_user', 'password123', 'Demo Student', 'student');

  INSERT INTO public.teachers (profile_id, sphere) VALUES (v_math, 'math') RETURNING id INTO
v_math_tid;
  INSERT INTO public.teachers (profile_id, sphere) VALUES (v_eng, 'english') RETURNING id INTO
v_eng_tid;

  INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group A') RETURNING
id INTO v_group;
  INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group B');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group C');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group A');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group B');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group C');

  SELECT id INTO v_student_sid FROM public.students WHERE profile_id = v_student;
  INSERT INTO public.group_students (group_id, student_id) VALUES (v_group, v_student_sid);

  INSERT INTO public.assignments (teacher_id, group_id, type, title, description, due_date)
  VALUES (v_math_tid, v_group, 'static', 'Read page 12', 'Review chapter on linear equations.',
NOW() + INTERVAL '2 days');

  INSERT INTO public.assignments (teacher_id, group_id, type, title, description, due_date)
  VALUES (v_math_tid, v_group, 'homework_pack', 'Algebra Week 1 Pack', 'Complete all tasks
below.', NOW() + INTERVAL '5 days')
  RETURNING id INTO v_pack;

  INSERT INTO public.tasks (assignment_id, title, description, sort_order, link_url)
  VALUES (v_pack, 'Practice set A', 'Solve exercises 1-10.', 1, 'https://www.khanacademy.org/
math/algebra');
  INSERT INTO public.tasks (assignment_id, title, description, sort_order)
  VALUES (v_pack, 'Written reflection', 'Explain how you checked your answers.', 2);

  INSERT INTO public.announcements (teacher_id, group_id, title, description)
  VALUES (v_math_tid, v_group, 'Welcome to Math Group A', 'Office hours every Wednesday.');

  INSERT INTO public.attendance (group_id, student_id, status, date)
  VALUES (v_group, v_student_sid, 'present', CURRENT_DATE);

  INSERT INTO public.notifications (user_id, title, body)
```

```
  VALUES (v_student, 'Welcome to MU Study', 'Your demo account is ready.');

  RETURN 'seeded';
END; $function$
```

**CHECKPOINT — Explain fn_seed_demo_data.sql**
In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_sync_submission_mark.sql

Function family: **sync_submission_mark**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_sync_submission_mark.sql**

```
CREATE OR REPLACE FUNCTION public.sync_submission_mark()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE v_group_id UUID; v_teacher_id UUID; v_title TEXT;
BEGIN
  IF NEW.score IS NULL THEN
    DELETE FROM public.marks WHERE submission_id = NEW.id;
    RETURN NEW;
  END IF;

  SELECT a.group_id, a.teacher_id, tk.title
    INTO v_group_id, v_teacher_id, v_title
  FROM public.tasks tk
  JOIN public.assignments a ON a.id = tk.assignment_id
  WHERE tk.id = NEW.task_id;

  IF v_group_id IS NULL THEN RETURN NEW; END IF;

  INSERT INTO public.marks (
    group_id, student_id, teacher_id, submission_id, title, category, percentage, comment
  ) VALUES (
    v_group_id, NEW.student_id, v_teacher_id, NEW.id,
    COALESCE(v_title, 'Homework'), 'homework', NEW.score, COALESCE(NEW.feedback, '')
  )
  ON CONFLICT (submission_id) DO UPDATE SET
    percentage = EXCLUDED.percentage,
    title = EXCLUDED.title,
    comment = EXCLUDED.comment,
    updated_at = NOW();

  RETURN NEW;
END; $function$
```

> **CHECKPOINT — Explain fn_sync_submission_mark.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_toggle_message_reaction.sql

Function family: **toggle_message_reaction**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_toggle_message_reaction.sql**

```
CREATE OR REPLACE FUNCTION public.toggle_message_reaction(p_message_id uuid, p_emoji text)
 RETURNS message_reactions
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_uid UUID := auth.uid();
  v_existing public.message_reactions;
  v_result public.message_reactions;
BEGIN
  IF v_uid IS NULL THEN
    RAISE EXCEPTION 'Not authenticated';
  END IF;

  IF p_emoji IS NULL OR char_length(trim(p_emoji)) < 1 THEN
    RAISE EXCEPTION 'Emoji required';
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM public.chat_messages m
    WHERE m.id = p_message_id
      AND m.is_deleted = FALSE
      AND public.can_access_chat_room(m.room_id)
  ) THEN
    RAISE EXCEPTION 'Message not found or inaccessible';
  END IF;

  SELECT * INTO v_existing
  FROM public.message_reactions
  WHERE message_id = p_message_id AND user_id = v_uid;

  IF v_existing.id IS NOT NULL AND v_existing.emoji = p_emoji THEN
    DELETE FROM public.message_reactions WHERE id = v_existing.id;
    RETURN v_existing;
  END IF;

  INSERT INTO public.message_reactions (message_id, user_id, emoji)
  VALUES (p_message_id, v_uid, p_emoji)
  ON CONFLICT (message_id, user_id)
  DO UPDATE SET emoji = EXCLUDED.emoji, created_at = NOW()
  RETURNING * INTO v_result;

  RETURN v_result;
END;
$function$
```

> **CHECKPOINT — Explain fn_toggle_message_reaction.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_touch_chat_message_updated_at.sql

Function family: **touch_chat_message_updated_at**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_touch_chat_message_updated_at.sql**

```
CREATE OR REPLACE FUNCTION public.touch_chat_message_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  NEW.updated_at := NOW();
  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_touch_chat_message_updated_at.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_touch_updated_at.sql

Function family: **touch_updated_at**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

**fn_touch_updated_at.sql**

```
CREATE OR REPLACE FUNCTION public.touch_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END; $function$
```

> **CHECKPOINT — Explain fn_touch_updated_at.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_trg_announcement_notify.sql

Function family: **trg_announcement_notify**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_trg_announcement_notify.sql**

```
CREATE OR REPLACE FUNCTION public.trg_announcement_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_prefix TEXT;
BEGIN
  FOR r IN
    SELECT s.profile_id AS uid
    FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = NEW.group_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    PERFORM public.notify_user(
      r.uid,
      'Announcement',
      NEW.title,
      'announcement',
      v_prefix || '/dashboard',
      NEW.id
    );
  END LOOP;
  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_trg_announcement_notify.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_trg_assignment_notify.sql

Function family: **trg_assignment_notify**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_trg_assignment_notify.sql**

```
CREATE OR REPLACE FUNCTION public.trg_assignment_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_prefix TEXT;
BEGIN
  FOR r IN
    SELECT s.profile_id AS uid
    FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = NEW.group_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    PERFORM public.notify_user(
      r.uid,
      'New homework',
      NEW.title,
      'assignment',
      v_prefix || '/assignments/' || NEW.id::text,
      NEW.id
    );
  END LOOP;
  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_trg_assignment_notify.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_trg_chat_message_notify.sql

Function family: **trg_chat_message_notify**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_trg_chat_message_notify.sql**

```
CREATE OR REPLACE FUNCTION public.trg_chat_message_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_group_id UUID;
  v_group_title TEXT;
  v_sender_name TEXT;
  v_preview TEXT;
  v_prefix TEXT;
  v_path TEXT;
BEGIN
  IF NEW.is_deleted THEN
    RETURN NEW;
  END IF;

  SELECT cr.group_id, g.title
    INTO v_group_id, v_group_title
  FROM public.chat_rooms cr
  JOIN public.groups g ON g.id = cr.group_id
  WHERE cr.id = NEW.room_id;

  IF v_group_id IS NULL THEN RETURN NEW; END IF;

  SELECT COALESCE(full_name, username, 'Someone')
    INTO v_sender_name
  FROM public.profiles
  WHERE id = NEW.sender_id;

  v_preview := CASE NEW.message_type
    WHEN 'text' THEN left(COALESCE(NEW.text_content, ''), 120)
    WHEN 'image' THEN 'Photo'
    WHEN 'video' THEN 'Video'
    WHEN 'voice' THEN 'Voice message'
    ELSE 'New message'
  END;

  FOR r IN
    SELECT DISTINCT uid FROM (
      SELECT s.profile_id AS uid
      FROM public.group_students gs
      JOIN public.students s ON s.id = gs.student_id
      WHERE gs.group_id = v_group_id
      UNION
      SELECT t.profile_id AS uid
      FROM public.groups g
      JOIN public.teachers t ON t.id = g.teacher_id
      WHERE g.id = v_group_id
    ) members
    WHERE uid IS DISTINCT FROM NEW.sender_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    v_path := v_prefix || '/chat/' || NEW.room_id::text;
```

```
    PERFORM public.notify_chat_message(
      r.uid,
      COALESCE(v_group_title, 'Chat'),
      COALESCE(v_sender_name, 'Someone') || ': ' || v_preview,
      v_path,
      NEW.room_id
    );
  END LOOP;

  RETURN NEW;
END;
$function$
```

**CHECKPOINT — Explain fn_trg_chat_message_notify.sql**
In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_trg_mark_notify.sql

Function family: **trg_mark_notify**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_trg_mark_notify.sql**

```
CREATE OR REPLACE FUNCTION public.trg_mark_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_uid UUID;
  v_prefix TEXT;
  v_body TEXT;
BEGIN
  IF TG_OP = 'UPDATE'
     AND NEW.percentage IS NOT DISTINCT FROM OLD.percentage
     AND NEW.comment IS NOT DISTINCT FROM OLD.comment THEN
    RETURN NEW;
  END IF;

  SELECT profile_id INTO v_uid FROM public.students WHERE id = NEW.student_id;
  IF v_uid IS NULL THEN RETURN NEW; END IF;

  v_prefix := COALESCE(public.profile_route_prefix(v_uid), '/student');
  v_body := NEW.title || ' — ' || ROUND(NEW.percentage)::TEXT || '%';
  IF NEW.comment IS NOT NULL AND length(trim(NEW.comment)) > 0 THEN
    v_body := v_body || E'\n' || left(trim(NEW.comment), 160);
  END IF;

  PERFORM public.notify_user(
    v_uid,
    'Work graded',
    v_body,
    'graded',
    v_prefix || '/marks',
    NEW.id
  );
  RETURN NEW;
END;
$function$
```

> **CHECKPOINT — Explain fn_trg_mark_notify.sql**
> In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

## fn_trg_submission_notify.sql

Function family: **trg_submission_notify**. If the name starts with admin_, only an admin session should succeed. If it starts with trg_, it is a trigger helper. If it starts with is_ or can_, it is a boolean used inside RLS policies.

> **WARNING — SECURITY DEFINER**
> This function runs with the privileges of its owner. Confirm it checks auth.uid() / is_admin() / membership before writing rows. Never expose a definer that inserts into auth.users without an admin gate.

**fn_trg_submission_notify.sql**

```
CREATE OR REPLACE FUNCTION public.trg_submission_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_teacher_profile UUID;
  v_student_name TEXT;
  v_task_title TEXT;
  v_assignment_id UUID;
  v_prefix TEXT;
  v_is_submit BOOLEAN;
BEGIN
  v_is_submit :=
    (TG_OP = 'INSERT' AND (NEW.file_url IS NOT NULL OR NEW.text_answer IS NOT NULL OR NEW.
submitted_at IS NOT NULL))
    OR (
      TG_OP = 'UPDATE'
      AND (
        (OLD.file_url IS DISTINCT FROM NEW.file_url AND NEW.file_url IS NOT NULL)
        OR (OLD.text_answer IS DISTINCT FROM NEW.text_answer AND NEW.text_answer IS NOT NULL)
        OR (OLD.submitted_at IS DISTINCT FROM NEW.submitted_at AND NEW.submitted_at IS NOT NULL)
        OR (OLD.is_completed IS DISTINCT FROM NEW.is_completed AND NEW.is_completed IS TRUE)
      )
    );

  IF NOT v_is_submit THEN
    RETURN NEW;
  END IF;

  IF TG_OP = 'UPDATE'
     AND OLD.score IS DISTINCT FROM NEW.score
     AND OLD.file_url IS NOT DISTINCT FROM NEW.file_url
     AND OLD.text_answer IS NOT DISTINCT FROM NEW.text_answer
     AND OLD.submitted_at IS NOT DISTINCT FROM NEW.submitted_at
     AND OLD.is_completed IS NOT DISTINCT FROM NEW.is_completed THEN
    RETURN NEW;
  END IF;

  SELECT t.profile_id, tk.title, a.id
    INTO v_teacher_profile, v_task_title, v_assignment_id
  FROM public.tasks tk
  JOIN public.assignments a ON a.id = tk.assignment_id
  JOIN public.teachers t ON t.id = a.teacher_id
  WHERE tk.id = NEW.task_id;

  IF v_teacher_profile IS NULL THEN RETURN NEW; END IF;

  SELECT COALESCE(p.full_name, p.username, 'Student')
    INTO v_student_name
  FROM public.students s
  JOIN public.profiles p ON p.id = s.profile_id
  WHERE s.id = NEW.student_id;

  v_prefix := COALESCE(public.profile_route_prefix(v_teacher_profile), '/teacher');
```

```
  PERFORM public.notify_user(
    v_teacher_profile,
    'New submission',
    COALESCE(v_student_name, 'Student') || ' · ' || COALESCE(v_task_title, 'Task'),
    'submission',
    v_prefix || '/assignments/' || v_assignment_id::text,
    NEW.id
  );
  RETURN NEW;
END;
$function$
```

**CHECKPOINT — Explain fn_trg_submission_notify.sql**
In one sentence: who may call this, what table it writes or reads, and what exception it raises on unauthorized use. If you cannot answer, re-read the first 20 lines of the body.

# Annotated SQL — every statement in COMPLETE_BACKEND.sql

SQL is not decoration. Each statement either creates a type, a table, a view, a function, a trigger, or a grant. This chapter walks the complete script so you can narrate it without scrolling blindly.

## COMPLETE_BACKEND.sql lines 1–7

**COMPLETE_BACKEND.sql:1-7**

```
-- ============================================================
-- MU Study — Complete Backend Rebuild Script
-- Source: live Supabase project nmmdvdprczukaskxxdrs (EXACT)
-- Run in SQL Editor on a brand-new project, top to bottom.
-- ============================================================
```

Section comment in the recovered script. Use it as a bookmark when applying SQL.

> **CHECKPOINT — Explain lines 1–7**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 8–11

**COMPLETE_BACKEND.sql:8-11**

```
CREATE EXTENSION IF NOT EXISTS pgcrypto WITH SCHEMA extensions;

-- ---------- ENUMS ----------
CREATE TYPE public.user_role AS ENUM ('student', 'teacher', 'admin');
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 8–11**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 12–15

**COMPLETE_BACKEND.sql:12-15**

```
CREATE TYPE public.sphere_type AS ENUM ('math', 'english');
CREATE TYPE public.assignment_type AS ENUM ('static', 'homework_pack');
CREATE TYPE public.attendance_status AS ENUM ('present', 'absent', 'late', 'excused');
CREATE TYPE public.submission_status AS ENUM ('pending', 'submitted', 'reviewed');
```

Enum or composite type. Flutter models must use the same labels (homework_pack, not homeworkPack).

> **CHECKPOINT — Explain lines 12–15**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 16–21

**COMPLETE_BACKEND.sql:16-21**

```
CREATE TYPE public.chat_message_type AS ENUM ('text', 'voice', 'image', 'video', 'file');
CREATE TYPE public.mark_category AS ENUM (
  'homework', 'classwork', 'quiz', 'test', 'exam', 'project', 'oral', 'other'
);

-- ---------- TABLES ----------
```

Enum or composite type. Flutter models must use the same labels (homework_pack, not homeworkPack).

> **CHECKPOINT — Explain lines 16–21**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 22–43

**COMPLETE_BACKEND.sql:22-43**

```
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  username TEXT NOT NULL UNIQUE,
  full_name TEXT NOT NULL DEFAULT '',
  role public.user_role NOT NULL,
  auth_email TEXT NOT NULL UNIQUE,
  avatar_url TEXT,
  is_disabled BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  preferred_language TEXT,
  preferred_theme TEXT,
  preferred_style TEXT,
  CONSTRAINT profiles_preferred_language_check CHECK (
    preferred_language IS NULL OR preferred_language = ANY (ARRAY['en'::text, 'ru'::text, 'uz'::
text])
  ),
  CONSTRAINT profiles_preferred_theme_check CHECK (
    preferred_theme IS NULL OR preferred_theme = ANY (ARRAY['light'::text, 'dark'::text,
'system'::text])
  ),
  CONSTRAINT profiles_preferred_style_check CHECK (
    preferred_style IS NULL OR preferred_style = ANY (ARRAY['modern'::text, 'naturePaper'::text,
 'softNeumorphism'::text, 'monochromeWorkspace'::text])
  )
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 22–43**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 44–52

**COMPLETE_BACKEND.sql:44-52**

```
CREATE INDEX idx_profiles_username ON public.profiles USING btree (username);

CREATE TABLE public.teachers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  profile_id UUID NOT NULL UNIQUE REFERENCES public.profiles(id) ON DELETE CASCADE,
  sphere public.sphere_type NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 44–52**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 53–58

**COMPLETE_BACKEND.sql:53-58**

```
CREATE TABLE public.students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  profile_id UUID NOT NULL UNIQUE REFERENCES public.profiles(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 53–58**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 59–64

**COMPLETE_BACKEND.sql:59-64**

```
CREATE TABLE public.groups (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 59–64**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 65–75

**COMPLETE_BACKEND.sql:65-75**

```
CREATE UNIQUE INDEX groups_teacher_title_uidx ON public.groups USING btree (teacher_id, title);
CREATE INDEX idx_groups_teacher ON public.groups USING btree (teacher_id);

CREATE TABLE public.group_students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (group_id, student_id)
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 65–75**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 76–85

**COMPLETE_BACKEND.sql:76-85**

```
CREATE TABLE public.assignments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
```

```
  type public.assignment_type NOT NULL DEFAULT 'static',
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  due_date TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 76–85**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 86–98

**COMPLETE_BACKEND.sql:86-98**

```
CREATE INDEX idx_assignments_group ON public.assignments USING btree (group_id);

CREATE TABLE public.tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  assignment_id UUID NOT NULL REFERENCES public.assignments(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  attachment_url TEXT,
  link_url TEXT,
  sort_order INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 86–98**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 99–116

**COMPLETE_BACKEND.sql:99-116**

```
CREATE TABLE public.submissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID NOT NULL REFERENCES public.tasks(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  text_answer TEXT,
  file_url TEXT,
  status public.submission_status NOT NULL DEFAULT 'pending',
  score NUMERIC,
  feedback TEXT,
  is_completed BOOLEAN NOT NULL DEFAULT false,
  submitted_at TIMESTAMPTZ,
  voice_url TEXT,
  UNIQUE (task_id, student_id),
  CONSTRAINT submissions_score_percent_check CHECK (
    score IS NULL OR (score >= 0 AND score <= 100)
  )
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 99–116**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 117–123

**COMPLETE_BACKEND.sql:117-123**

```
CREATE TABLE public.lesson_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  session_date DATE NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (group_id, session_date)
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 117–123**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 124–137

**COMPLETE_BACKEND.sql:124-137**

```
CREATE INDEX idx_lesson_sessions_group_date ON public.lesson_sessions USING btree (group_id,
session_date DESC);

CREATE TABLE public.attendance (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  status public.attendance_status NOT NULL,
  date DATE NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  session_id UUID NOT NULL REFERENCES public.lesson_sessions(id) ON DELETE CASCADE,
  UNIQUE (group_id, student_id, date),
  UNIQUE (session_id, student_id)
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 124–137**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 138–154

**COMPLETE_BACKEND.sql:138-154**

```
CREATE TABLE public.marks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  submission_id UUID UNIQUE REFERENCES public.submissions(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  category public.mark_category NOT NULL DEFAULT 'other',
  percentage NUMERIC NOT NULL,
  weight NUMERIC NOT NULL DEFAULT 1,
  comment TEXT NOT NULL DEFAULT '',
```

```
  marked_on DATE NOT NULL DEFAULT CURRENT_DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT marks_percentage_check CHECK (percentage >= 0 AND percentage <= 100),
  CONSTRAINT marks_weight_check CHECK (weight > 0)
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 138–154**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 155–168

**COMPLETE_BACKEND.sql:155-168**

```
CREATE INDEX idx_marks_group ON public.marks USING btree (group_id, marked_on DESC);
CREATE INDEX idx_marks_student ON public.marks USING btree (student_id, marked_on DESC);

CREATE TABLE public.announcements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID REFERENCES public.teachers(id) ON DELETE CASCADE,
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  image_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
  file_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 155–168**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 169–179

**COMPLETE_BACKEND.sql:169-179**

```
CREATE TABLE public.notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  body TEXT NOT NULL DEFAULT '',
  is_read BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  kind TEXT NOT NULL DEFAULT 'system',
  action_path TEXT,
  ref_id UUID
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 169–179**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 180–192

**COMPLETE_BACKEND.sql:180-192**

```
CREATE INDEX idx_notifications_user ON public.notifications USING btree (user_id, is_read,
created_at DESC);
CREATE INDEX idx_notifications_user_kind_unread ON public.notifications USING btree (user_id,
kind, is_read) WHERE (is_read = false);

CREATE TABLE public.assignment_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  type public.assignment_type NOT NULL DEFAULT 'static',
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  tasks JSONB NOT NULL DEFAULT '[]'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 180–192**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 193–203

**COMPLETE_BACKEND.sql:193-203**

```
CREATE INDEX assignment_templates_teacher_idx ON public.assignment_templates USING btree (
teacher_id, updated_at DESC);

CREATE TABLE public.academic_years (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  start_year INTEGER NOT NULL UNIQUE,
  label TEXT NOT NULL,
  is_current BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by UUID REFERENCES auth.users(id) ON DELETE SET NULL,
  CONSTRAINT academic_years_start_year_check CHECK (start_year >= 2000 AND start_year <= 2100)
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 193–203**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 204–215

**COMPLETE_BACKEND.sql:204-215**

```
CREATE UNIQUE INDEX academic_years_one_current ON public.academic_years USING btree ((true))
WHERE is_current;

CREATE TABLE public.academic_terms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  academic_year_id UUID NOT NULL REFERENCES public.academic_years(id) ON DELETE CASCADE,
  semester INTEGER NOT NULL,
  starts_on DATE NOT NULL,
  ends_on DATE NOT NULL,
  UNIQUE (academic_year_id, semester),
  CONSTRAINT academic_terms_semester_check CHECK (semester >= 1 AND semester <= 4),
  CONSTRAINT academic_terms_range_check CHECK (ends_on >= starts_on)
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 204–215**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 216–223

**COMPLETE_BACKEND.sql:216-223**

```
CREATE INDEX idx_academic_terms_dates ON public.academic_terms USING btree (starts_on, ends_on);
CREATE INDEX idx_academic_terms_year ON public.academic_terms USING btree (academic_year_id);

CREATE TABLE public.chat_rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL UNIQUE REFERENCES public.groups(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 216–223**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 224–243

**COMPLETE_BACKEND.sql:224-243**

```
CREATE INDEX idx_chat_rooms_group ON public.chat_rooms USING btree (group_id);

CREATE TABLE public.chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES public.chat_rooms(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  message_type public.chat_message_type NOT NULL DEFAULT 'text',
  text_content TEXT,
  media_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  is_deleted BOOLEAN NOT NULL DEFAULT false,
  reply_to_message_id UUID REFERENCES public.chat_messages(id) ON DELETE SET NULL,
  CONSTRAINT chat_messages_content_check CHECK (
    is_deleted OR (
      ((message_type = 'text'::chat_message_type) AND (text_content IS NOT NULL) AND (length(
TRIM(BOTH FROM text_content)) > 0))
      OR ((message_type <> 'text'::chat_message_type) AND (media_url IS NOT NULL) AND (length(
TRIM(BOTH FROM media_url)) > 0))
    )
  )
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 224–243**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 244–247

**COMPLETE_BACKEND.sql:244-247**

```
CREATE INDEX idx_chat_messages_room_created ON public.chat_messages USING btree (room_id,
created_at DESC);
CREATE INDEX idx_chat_messages_sender ON public.chat_messages USING btree (sender_id);
CREATE INDEX idx_chat_messages_reply_to ON public.chat_messages USING btree (
reply_to_message_id) WHERE (reply_to_message_id IS NOT NULL);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 244–247**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 248–254

**COMPLETE_BACKEND.sql:248-254**

```
CREATE TABLE public.message_reads (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES public.chat_messages(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  read_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (message_id, user_id)
);
```

Base table. After create, enable RLS and add policies or the API returns empty sets for authenticated users.

> **CHECKPOINT — Explain lines 248–254**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 255–265

**COMPLETE_BACKEND.sql:255-265**

```
CREATE INDEX idx_message_reads_user ON public.message_reads USING btree (user_id, message_id);

CREATE TABLE public.message_reactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES public.chat_messages(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  emoji TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (message_id, user_id),
  CONSTRAINT message_reactions_emoji_len CHECK (char_length(emoji) >= 1 AND char_length(emoji)
<= 16)
);
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 255–265**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 266–270

**COMPLETE_BACKEND.sql:266-270**

```
CREATE INDEX idx_message_reactions_message ON public.message_reactions USING btree (message_id);
ALTER TABLE public.message_reactions REPLICA IDENTITY FULL;


-- ---------- VIEWS ----------
```

Index. Unique indexes are product rules (one group title per teacher; one current academic year).

> **CHECKPOINT — Explain lines 266–270**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 271–292

**COMPLETE_BACKEND.sql:271-292**

```
 CREATE OR REPLACE VIEW public.chat_room_summaries AS
  SELECT cr.id,
     cr.group_id,
     cr.created_at,
     g.title AS group_title,
     tp.full_name AS teacher_name,
     (( SELECT (count(*))::integer AS count
            FROM group_students gs
           WHERE (gs.group_id = g.id)) + 1) AS member_count,
     lm.id AS last_message_id,
     lm.message_type AS last_message_type,
     lm.text_content AS last_message_text,
     lm.media_url AS last_message_media_url,
     lm.created_at AS last_message_at,
     lm.is_deleted AS last_message_deleted,
     lm.sender_id AS last_message_sender_id,
     sp.full_name AS last_message_sender_name
    FROM (((((chat_rooms cr
      JOIN groups g ON ((g.id = cr.group_id)))
      JOIN teachers t ON ((t.id = g.teacher_id)))
      JOIN profiles tp ON ((tp.id = t.profile_id)))
      LEFT JOIN LATERAL ( SELECT m.id,
```

View. Treat it as a stored SELECT. Chat list and mark averages depend on these.

> **CHECKPOINT — Explain lines 271–292**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 293–306

**COMPLETE_BACKEND.sql:293-306**

```
             m.room_id,
             m.sender_id,
             m.message_type,
             m.text_content,
             m.media_url,
             m.created_at,
             m.updated_at,
             m.is_deleted
            FROM chat_messages m
           WHERE (m.room_id = cr.id)
           ORDER BY m.created_at DESC
          LIMIT 1) lm ON (true))
     LEFT JOIN profiles sp ON ((sp.id = lm.sender_id)));
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 293–306**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 307–314

**COMPLETE_BACKEND.sql:307-314**

```
CREATE OR REPLACE VIEW public.student_mark_averages AS
 SELECT student_id,
    round((sum((percentage * weight)) / NULLIF(sum(weight), (0)::numeric)), 2) AS
average_percent,
    (count(*))::integer AS marks_count,
    max(marked_on) AS last_marked_on
   FROM marks
  GROUP BY student_id;
```

View. Treat it as a stored SELECT. Chat list and mark averages depend on these.

> **CHECKPOINT — Explain lines 307–314**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 315–324

**COMPLETE_BACKEND.sql:315-324**

```
CREATE OR REPLACE VIEW public.student_group_mark_averages AS
 SELECT student_id,
    group_id,
    round((sum((percentage * weight)) / NULLIF(sum(weight), (0)::numeric)), 2) AS
average_percent,
    (count(*))::integer AS marks_count,
    max(marked_on) AS last_marked_on
   FROM marks
  GROUP BY student_id, group_id;

-- FILE: fn__demo_insert_user.sql
```

View. Treat it as a stored SELECT. Chat list and mark averages depend on these.

> **CHECKPOINT — Explain lines 315–324**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 325–346

**COMPLETE_BACKEND.sql:325-346**

```
CREATE OR REPLACE FUNCTION public._demo_insert_user(p_username text, p_password text,
p_full_name text, p_role user_role)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE v_user_id UUID := gen_random_uuid(); v_email TEXT;
BEGIN
  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';
  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000', v_user_id, 'authenticated', 'authenticated',
    v_email, extensions.crypt(p_password, extensions.gen_salt('bf')), NOW(),
    jsonb_build_object('provider','email','providers',ARRAY['email']),
    jsonb_build_object('username', p_username, 'full_name', p_full_name, 'role', p_role::text),
    NOW(), NOW(), '', '', '', '', false, false
```

```
  );
  INSERT INTO auth.identities (id, user_id, identity_data, provider, provider_id,
last_sign_in_at, created_at, updated_at)
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 325–346**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 347–358

**COMPLETE_BACKEND.sql:347-358**

```
  VALUES (
    gen_random_uuid(), v_user_id,
    jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
    'email', v_email, NOW(), NOW(), NOW()
  );
  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);
  IF p_role = 'student' THEN INSERT INTO public.students (profile_id) VALUES (v_user_id); END
IF;
  RETURN v_user_id;
END; $function$

-- FILE: fn_academic_semester_bounds.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 347–358**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 359–380

**COMPLETE_BACKEND.sql:359-380**

```
CREATE OR REPLACE FUNCTION public.academic_semester_bounds(p_start_year integer, p_semester
integer)
 RETURNS TABLE(starts_on date, ends_on date)
 LANGUAGE plpgsql
 IMMUTABLE
AS $function$
BEGIN
  IF p_semester < 1 OR p_semester > 4 THEN
    RAISE EXCEPTION 'semester must be 1–4';
  END IF;

  RETURN QUERY
  SELECT
    CASE p_semester
      WHEN 1 THEN make_date(p_start_year, 8, 1)
      WHEN 2 THEN make_date(p_start_year, 11, 11)
      WHEN 3 THEN make_date(p_start_year + 1, 1, 11)
      ELSE make_date(p_start_year + 1, 3, 28)
    END,
    CASE p_semester
      WHEN 1 THEN make_date(p_start_year, 11, 2)
      WHEN 2 THEN make_date(p_start_year, 12, 28)
      WHEN 3 THEN make_date(p_start_year + 1, 3, 20)
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 359–380**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 381–386

**COMPLETE_BACKEND.sql:381-386**

```
      ELSE make_date(p_start_year + 1, 6, 19)
    END;
END;
$function$

-- FILE: fn_academic_year_label.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 381–386**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 387–395

**COMPLETE_BACKEND.sql:387-395**

```
CREATE OR REPLACE FUNCTION public.academic_year_label(p_start_year integer)
 RETURNS text
 LANGUAGE sql
 IMMUTABLE
AS $function$
  SELECT p_start_year::TEXT || '–' || lpad(((p_start_year + 1) % 100)::TEXT, 2, '0');
$function$

-- FILE: fn_admin_create_next_academic_year.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 387–395**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 396–417

**COMPLETE_BACKEND.sql:396-417**

```
CREATE OR REPLACE FUNCTION public.admin_create_next_academic_year()
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_next INT;
  v_now DATE := CURRENT_DATE;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can create academic years';
  END IF;

  SELECT COALESCE(MAX(start_year), CASE
    WHEN EXTRACT(MONTH FROM v_now) >= 8 THEN EXTRACT(YEAR FROM v_now)::INT - 1
    ELSE EXTRACT(YEAR FROM v_now)::INT - 2
```

```
  END) + 1
  INTO v_next
  FROM public.academic_years;

  RETURN public.create_academic_year_with_terms(
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 396–417**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 418–425

**COMPLETE_BACKEND.sql:418-425**

```
    v_next,
    FALSE,
    auth.uid()
  );
END;
$function$

-- FILE: fn_admin_create_user.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 418–425**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 426–447

**COMPLETE_BACKEND.sql:426-447**

```
CREATE OR REPLACE FUNCTION public.admin_create_user(p_username text, p_password text,
p_full_name text, p_role user_role, p_sphere sphere_type DEFAULT NULL::sphere_type)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';

  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 426–447**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 448–469

**COMPLETE_BACKEND.sql:448-469**

```
    '00000000-0000-0000-0000-000000000000',
    v_user_id,
    'authenticated',
    'authenticated',
    v_email,
    extensions.crypt(p_password, extensions.gen_salt('bf')),
    NOW(),
    jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),
    jsonb_build_object(
      'username', p_username,
      'full_name', p_full_name,
      'role', p_role::text
    ),
    NOW(),
    NOW(),
    '',
    '',
    '',
    '',
    false,
    false
  );
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 448–469**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 470–491

**COMPLETE_BACKEND.sql:470-491**

```
  INSERT INTO auth.identities (
    id, user_id, identity_data, provider, provider_id,
    last_sign_in_at, created_at, updated_at
  ) VALUES (
    gen_random_uuid(),
    v_user_id,
    jsonb_build_object(
      'sub', v_user_id::text,
      'email', v_email,
      'email_verified', true
    ),
    'email',
    v_email,
    NOW(),
    NOW(),
    NOW()
  );

  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 470–491**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 492–513

**COMPLETE_BACKEND.sql:492-513**

```
   IF EXISTS (
     SELECT 1
     FROM information_schema.tables
     WHERE table_schema = 'public' AND table_name = 'login_credentials'
   ) THEN
     INSERT INTO public.login_credentials (profile_id, password, updated_at)
     VALUES (v_user_id, p_password, NOW())
     ON CONFLICT (profile_id) DO UPDATE
       SET password = excluded.password,
           updated_at = NOW();
   END IF;

   IF p_role = 'teacher' THEN
     IF p_sphere IS NULL THEN
       RAISE EXCEPTION 'Sphere required for teacher';
     END IF;
     -- Do NOT auto-create Group 1/2/3. Admins add groups manually.
     INSERT INTO public.teachers (profile_id, sphere)
     VALUES (v_user_id, p_sphere);
   ELSIF p_role = 'student' THEN
     INSERT INTO public.students (profile_id) VALUES (v_user_id);
   END IF;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 492–513**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 514–519

**COMPLETE_BACKEND.sql:514-519**

```
   RETURN v_user_id;
 END;
 $function$

 -- FILE: fn_admin_delete_academic_year.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 514–519**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 520–541

**COMPLETE_BACKEND.sql:520-541**

```
 CREATE OR REPLACE FUNCTION public.admin_delete_academic_year(p_year_id uuid)
  RETURNS academic_years
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
```

```
AS $function$
DECLARE
  v_year public.academic_years;
  v_remaining INT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can delete academic years';
  END IF;

  SELECT * INTO v_year
  FROM public.academic_years
  WHERE id = p_year_id;

  IF v_year.id IS NULL THEN
    RAISE EXCEPTION 'Academic year not found';
  END IF;
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 520–541**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 542–559

**COMPLETE_BACKEND.sql:542-559**

```
  IF v_year.is_current THEN
    RAISE EXCEPTION 'Cannot delete the current academic year. Set another year as current first.
';
  END IF;

  SELECT COUNT(*) INTO v_remaining FROM public.academic_years;
  IF v_remaining <= 1 THEN
    RAISE EXCEPTION 'Cannot delete the last academic year';
  END IF;

  DELETE FROM public.academic_years
  WHERE id = p_year_id
  RETURNING * INTO v_year;

  RETURN v_year;
END;
$function$

-- FILE: fn_admin_mass_generate_school_data.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 542–559**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 560–581

**COMPLETE_BACKEND.sql:560-581**

```
CREATE OR REPLACE FUNCTION public.admin_mass_generate_school_data(p_teachers jsonb, p_students
jsonb, p_groups jsonb, p_assignments jsonb)
 RETURNS jsonb
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE
  v_errors TEXT[] := ARRAY[]::TEXT[];
```

```
  v_teacher_upserted INT := 0;
  v_student_upserted INT := 0;
  v_groups_created INT := 0;
  v_assignments_inserted INT := 0;

  -- Auth/profile ids
  v_profile_id UUID;
  v_user_id UUID;
  v_teacher_id UUID;
  v_student_id UUID;
  v_group_id UUID;

  -- temp strings
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 560–581**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 582–603

**COMPLETE_BACKEND.sql:582-603**

```
  v_username TEXT;
  v_password TEXT;
  v_full_name TEXT;
  v_sphere TEXT;
  v_email TEXT;

  v_teacher_username TEXT;
  v_group_title TEXT;
  v_student_username TEXT;

  v_row JSONB;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  -- ==========================
  -- Teachers + teacher rows
  -- ==========================
  IF p_teachers IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_teachers)
    LOOP
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 582–603**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 604–625

**COMPLETE_BACKEND.sql:604-625**

```
      v_username := trim(both from v_row->>'username');
      v_password := v_row->>'password';
      v_full_name := trim(both from v_row->>'full_name');
      v_sphere := lower(trim(both from v_row->>'sphere'));

      IF v_username IS NULL OR v_username = '' OR v_password IS NULL OR v_password = '' THEN
        v_errors := v_errors || format('Teacher missing username/password: %s', v_row::TEXT);
        CONTINUE;
      END IF;
```

```
      IF v_sphere IS NULL OR (v_sphere <> 'math' AND v_sphere <> 'english') THEN
        v_errors := v_errors || format('Teacher invalid sphere for %s: %s', v_username,
 COALESCE(v_sphere,'<null>'));
        CONTINUE;
      END IF;

      v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

      -- Existing profile?
      SELECT id INTO v_profile_id
      FROM public.profiles
      WHERE username = v_username
      LIMIT 1;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 604–625**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 626–647

**COMPLETE_BACKEND.sql:626-647**

```
      IF v_profile_id IS NULL THEN
        v_user_id := gen_random_uuid();

        INSERT INTO auth.users (
          instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
          raw_app_meta_data, raw_user_meta_data,
          created_at, updated_at,
          confirmation_token, recovery_token, email_change_token_new, email_change,
          is_sso_user, is_anonymous
        ) VALUES (
          '00000000-0000-0000-0000-000000000000',
          v_user_id,
          'authenticated',
          'authenticated',
          v_email,
          extensions.crypt(v_password, extensions.gen_salt('bf')),
          NOW(),
          jsonb_build_object('provider','email','providers',ARRAY['email']),
          jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role',
 'teacher'),
          NOW(),
          NOW(),
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 626–647**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 648–669

**COMPLETE_BACKEND.sql:648-669**

```
          '', '', '', '',
          false, false
        );

        INSERT INTO auth.identities (
          id, user_id, identity_data, provider, provider_id,
          last_sign_in_at, created_at, updated_at
        ) VALUES (
          gen_random_uuid(),
```

```
          v_user_id,
          jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
          'email',
          v_user_id::text,
          NOW(), NOW(), NOW()
        );

        INSERT INTO public.profiles (id, username, full_name, role, auth_email)
        VALUES (v_user_id, v_username, COALESCE(v_full_name,'') , 'teacher', v_email);

        v_profile_id := v_user_id;
      ELSE
        -- Update password + basic profile fields
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 648–669**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 670–691

**COMPLETE_BACKEND.sql:670-691**

```
        v_user_id := v_profile_id;
        UPDATE auth.users
        SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
            updated_at = NOW()
        WHERE id = v_user_id;

        UPDATE public.profiles
        SET full_name = COALESCE(v_full_name,''),
            role = 'teacher',
            auth_email = v_email
        WHERE id = v_user_id;
      END IF;

      -- Upsert teachers row
      INSERT INTO public.teachers (profile_id, sphere)
      VALUES (
        v_profile_id,
        CASE WHEN v_sphere = 'math' THEN 'math'::public.sphere_type ELSE 'english'::public.
 sphere_type END
      )
      ON CONFLICT (profile_id) DO UPDATE
      SET sphere = EXCLUDED.sphere;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 670–691**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 692–713

**COMPLETE_BACKEND.sql:692-713**

```
      v_teacher_upserted := v_teacher_upserted + 1;
    END LOOP;
  END IF;

  -- ==========================
  -- Students + student rows
  -- ==========================
  IF p_students IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_students)
```

```
    LOOP
      v_username := trim(both from v_row->>'username');
      v_password := v_row->>'password';
      v_full_name := trim(both from v_row->>'full_name');

      IF v_username IS NULL OR v_username = '' OR v_password IS NULL OR v_password = '' THEN
        v_errors := v_errors || format('Student missing username/password: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

      SELECT id INTO v_profile_id
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 692–713**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 714–735

**COMPLETE_BACKEND.sql:714-735**

```
      FROM public.profiles
      WHERE username = v_username
      LIMIT 1;

      IF v_profile_id IS NULL THEN
        v_user_id := gen_random_uuid();

        INSERT INTO auth.users (
          instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
          raw_app_meta_data, raw_user_meta_data,
          created_at, updated_at,
          confirmation_token, recovery_token, email_change_token_new, email_change,
          is_sso_user, is_anonymous
        ) VALUES (
          '00000000-0000-0000-0000-000000000000',
          v_user_id,
          'authenticated',
          'authenticated',
          v_email,
          extensions.crypt(v_password, extensions.gen_salt('bf')),
          NOW(),
          jsonb_build_object('provider','email','providers',ARRAY['email']),
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 714–735**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 736–757

**COMPLETE_BACKEND.sql:736-757**

```
          jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role',
 'student'),
          NOW(),
          NOW(),
          '', '', '', '',
          false, false
        );

        INSERT INTO auth.identities (
```

```
          id, user_id, identity_data, provider, provider_id,
          last_sign_in_at, created_at, updated_at
        ) VALUES (
          gen_random_uuid(),
          v_user_id,
          jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
          'email',
          v_user_id::text,
          NOW(), NOW(), NOW()
        );

        INSERT INTO public.profiles (id, username, full_name, role, auth_email)
        VALUES (v_user_id, v_username, COALESCE(v_full_name,'') , 'student', v_email);
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 736–757**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 758–779

**COMPLETE_BACKEND.sql:758-779**

```
        v_profile_id := v_user_id;
      ELSE
        v_user_id := v_profile_id;
        UPDATE auth.users
        SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
            updated_at = NOW()
        WHERE id = v_user_id;

        UPDATE public.profiles
        SET full_name = COALESCE(v_full_name,''),
            role = 'student',
            auth_email = v_email
        WHERE id = v_user_id;
      END IF;

      INSERT INTO public.students (profile_id)
      VALUES (v_profile_id)
      ON CONFLICT (profile_id) DO NOTHING;

      v_student_upserted := v_student_upserted + 1;
    END LOOP;
  END IF;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 758–779**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 780–801

**COMPLETE_BACKEND.sql:780-801**

```
  -- ==========================
  -- Groups (exact titles)
  -- ==========================
  IF p_groups IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_groups)
    LOOP
      v_teacher_username := trim(both from v_row->>'teacher_username');
      v_group_title := trim(both from v_row->>'title');
```

```
      IF v_teacher_username IS NULL OR v_teacher_username = '' OR v_group_title IS NULL OR
 v_group_title = '' THEN
        v_errors := v_errors || format('Group missing teacher_username/title: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      SELECT t.id INTO v_teacher_id
      FROM public.teachers t
      JOIN public.profiles p ON p.id = t.profile_id
      WHERE p.username = v_teacher_username
        AND p.role = 'teacher'
      LIMIT 1;
```

Section comment in the recovered script. Use it as a bookmark when applying SQL.

> **CHECKPOINT — Explain lines 780–801**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 802–823

**COMPLETE_BACKEND.sql:802-823**

```
      IF v_teacher_id IS NULL THEN
        v_errors := v_errors || format('Unknown teacher for group %s (teacher_username=%s)',
 v_group_title, v_teacher_username);
        CONTINUE;
      END IF;

      SELECT g.id INTO v_group_id
      FROM public.groups g
      WHERE g.teacher_id = v_teacher_id
        AND g.title = v_group_title
      LIMIT 1;

      IF v_group_id IS NULL THEN
        INSERT INTO public.groups (teacher_id, title)
        VALUES (v_teacher_id, v_group_title);
        v_groups_created := v_groups_created + 1;
      END IF;
    END LOOP;
  END IF;

  -- ============================
  -- Assign students -> groups
  -- ============================
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 802–823**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 824–845

**COMPLETE_BACKEND.sql:824-845**

```
  IF p_assignments IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_assignments)
    LOOP
      v_teacher_username := trim(both from v_row->>'teacher_username');
      v_group_title := trim(both from v_row->>'group_title');
      v_student_username := trim(both from v_row->>'student_username');

      IF v_teacher_username IS NULL OR v_teacher_username = '' OR
         v_group_title IS NULL OR v_group_title = '' OR
```

```
         v_student_username IS NULL OR v_student_username = '' THEN
        v_errors := v_errors || format('Assignment missing teacher_username/group_title/
 student_username: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      -- Resolve group_id by teacher_username + group_title
      SELECT g.id INTO v_group_id
      FROM public.groups g
      JOIN public.teachers t ON t.id = g.teacher_id
      JOIN public.profiles p ON p.id = t.profile_id
      WHERE p.username = v_teacher_username
        AND p.role = 'teacher'
        AND g.title = v_group_title
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 824–845**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 846–867

**COMPLETE_BACKEND.sql:846-867**

```
      LIMIT 1;

      IF v_group_id IS NULL THEN
        v_errors := v_errors || format('Unknown group for assignment %s -> (teacher=%s,
 group_title=%s)', v_student_username, v_teacher_username, v_group_title);
        CONTINUE;
      END IF;

      -- Resolve student_id
      SELECT s.id INTO v_student_id
      FROM public.students s
      JOIN public.profiles p ON p.id = s.profile_id
      WHERE p.username = v_student_username
        AND p.role = 'student'
      LIMIT 1;

      IF v_student_id IS NULL THEN
        v_errors := v_errors || format('Unknown student username: %s', v_student_username);
        CONTINUE;
      END IF;

      INSERT INTO public.group_students (group_id, student_id)
      VALUES (v_group_id, v_student_id)
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 846–867**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 868–889

**COMPLETE_BACKEND.sql:868-889**

```
      ON CONFLICT (group_id, student_id) DO NOTHING;

      -- We don't have a portable "rowcount" here without extra GET DIAGNOSTICS.
      -- Still count by checking existence after insert.
      IF EXISTS (
        SELECT 1 FROM public.group_students gs
        WHERE gs.group_id = v_group_id AND gs.student_id = v_student_id
```

```
      ) THEN
        v_assignments_inserted := v_assignments_inserted + 1;
      END IF;
    END LOOP;
  END IF;

  RETURN jsonb_build_object(
    'teachers_upserted', v_teacher_upserted,
    'students_upserted', v_student_upserted,
    'groups_created', v_groups_created,
    'assignments_inserted', v_assignments_inserted,
    'errors', to_jsonb(v_errors)
  );
END;
$function$
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 868–889**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 890–903

**COMPLETE_BACKEND.sql:890-903**

```
-- FILE: fn_admin_reset_password.sql
CREATE OR REPLACE FUNCTION public.admin_reset_password(p_user_id uuid, p_password text)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'auth', 'public', 'extensions'
AS $function$
BEGIN
  IF NOT public.is_admin() THEN RAISE EXCEPTION 'Not authorized'; END IF;
  UPDATE auth.users SET encrypted_password = extensions.crypt(p_password, extensions.gen_salt(
'bf')), updated_at = NOW() WHERE id = p_user_id;
END; $function$

-- FILE: fn_admin_set_current_academic_year.sql
```

Section comment in the recovered script. Use it as a bookmark when applying SQL.

> **CHECKPOINT — Explain lines 890–903**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 904–925

**COMPLETE_BACKEND.sql:904-925**

```
CREATE OR REPLACE FUNCTION public.admin_set_current_academic_year(p_year_id uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can set the current academic year';
  END IF;

  UPDATE public.academic_years SET is_current = FALSE WHERE is_current;
```

```
  UPDATE public.academic_years
  SET is_current = TRUE
  WHERE id = p_year_id
  RETURNING * INTO v_year;

  IF v_year.id IS NULL THEN
    RAISE EXCEPTION 'Academic year not found';
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 904–925**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 926–932

**COMPLETE_BACKEND.sql:926-932**

```
  END IF;

  RETURN v_year;
END;
$function$

-- FILE: fn_admin_update_academic_term.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 926–932**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 933–954

**COMPLETE_BACKEND.sql:933-954**

```
CREATE OR REPLACE FUNCTION public.admin_update_academic_term(p_term_id uuid, p_starts_on date,
p_ends_on date)
 RETURNS academic_terms
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_term public.academic_terms;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can edit term dates';
  END IF;

  IF p_ends_on < p_starts_on THEN
    RAISE EXCEPTION 'End date must be on or after start date';
  END IF;

  UPDATE public.academic_terms
  SET starts_on = p_starts_on, ends_on = p_ends_on
  WHERE id = p_term_id
  RETURNING * INTO v_term;
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 933–954**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 955–963

**COMPLETE_BACKEND.sql:955-963**

```
   IF v_term.id IS NULL THEN
     RAISE EXCEPTION 'Academic term not found';
   END IF;

   RETURN v_term;
 END;
 $function$

 -- FILE: fn_attendance_sync_session.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 955–963**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 964–985

**COMPLETE_BACKEND.sql:964-985**

```
 CREATE OR REPLACE FUNCTION public.attendance_sync_session()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
 AS $function$
 BEGIN
   IF NEW.date IS NOT NULL AND NEW.date > CURRENT_DATE THEN
     RAISE EXCEPTION 'Cannot mark attendance for a future date'
       USING ERRCODE = 'check_violation';
   END IF;

   IF NEW.session_id IS NULL AND NEW.group_id IS NOT NULL AND NEW.date IS NOT NULL THEN
     NEW.session_id := public.ensure_lesson_session(NEW.group_id, NEW.date::date);
   ELSIF NEW.session_id IS NOT NULL THEN
     SELECT ls.group_id, ls.session_date
       INTO NEW.group_id, NEW.date
     FROM public.lesson_sessions ls
     WHERE ls.id = NEW.session_id;
   END IF;

   IF NEW.date IS NOT NULL AND NEW.date > CURRENT_DATE THEN
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 964–985**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 986–994

**COMPLETE_BACKEND.sql:986-994**

```
     RAISE EXCEPTION 'Cannot mark attendance for a future date'
```

```
      USING ERRCODE = 'check_violation';
  END IF;

  RETURN NEW;
END;
$function$

-- FILE: fn_can_access_chat_room.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 986–994**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 995–1013

**COMPLETE_BACKEND.sql:995-1013**

```
CREATE OR REPLACE FUNCTION public.can_access_chat_room(rid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT public.is_admin()
    OR EXISTS (
      SELECT 1
      FROM public.chat_rooms cr
      WHERE cr.id = rid
        AND (
          public.is_teacher_of_group(cr.group_id)
          OR public.is_student_in_group(cr.group_id)
        )
    );
$function$

-- FILE: fn_chat_messages_reply_same_room.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 995–1013**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1014–1035

**COMPLETE_BACKEND.sql:1014-1035**

```
CREATE OR REPLACE FUNCTION public.chat_messages_reply_same_room()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
  v_room UUID;
BEGIN
  IF NEW.reply_to_message_id IS NULL THEN
    RETURN NEW;
  END IF;
  IF NEW.reply_to_message_id = NEW.id THEN
    RAISE EXCEPTION 'message cannot reply to itself';
  END IF;
  SELECT room_id INTO v_room
  FROM public.chat_messages
  WHERE id = NEW.reply_to_message_id;
  IF v_room IS NULL THEN
```

```
    RAISE EXCEPTION 'reply target not found';
  END IF;
  IF v_room <> NEW.room_id THEN
    RAISE EXCEPTION 'reply must be in the same chat room';
  END IF;
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1014–1035**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1036–1040

**COMPLETE_BACKEND.sql:1036-1040**

```
  RETURN NEW;
END;
$function$

-- FILE: fn_chat_storage_room_id.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1036–1040**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1041–1057

**COMPLETE_BACKEND.sql:1041-1057**

```
CREATE OR REPLACE FUNCTION public.chat_storage_room_id(object_name text)
 RETURNS uuid
 LANGUAGE plpgsql
 IMMUTABLE
 SET search_path TO 'public'
AS $function$
DECLARE
  part TEXT := split_part(object_name, '/', 1);
BEGIN
  IF part ~* '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$' THEN
    RETURN part::UUID;
  END IF;
  RETURN NULL;
END;
$function$

-- FILE: fn_chat_unread_counts.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1041–1057**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1058–1079

**COMPLETE_BACKEND.sql:1058-1079**

```
CREATE OR REPLACE FUNCTION public.chat_unread_counts()
 RETURNS TABLE(room_id uuid, unread_count bigint)
```

```
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT
    m.room_id,
    COUNT(*)::BIGINT AS unread_count
  FROM public.chat_messages m
  WHERE m.is_deleted = FALSE
    AND m.sender_id <> auth.uid()
    AND public.can_access_chat_room(m.room_id)
    AND NOT EXISTS (
      SELECT 1
      FROM public.message_reads r
      WHERE r.message_id = m.id
        AND r.user_id = auth.uid()
    )
  GROUP BY m.room_id;
$function$
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1058–1079**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1080–1101

**COMPLETE_BACKEND.sql:1080-1101**

```
-- FILE: fn_create_academic_year_with_terms.sql
CREATE OR REPLACE FUNCTION public.create_academic_year_with_terms(p_start_year integer,
p_is_current boolean DEFAULT false, p_created_by uuid DEFAULT NULL::uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
  v_sem INT;
  v_bounds RECORD;
BEGIN
  IF p_is_current THEN
    UPDATE public.academic_years SET is_current = FALSE WHERE is_current;
  END IF;

  INSERT INTO public.academic_years (start_year, label, is_current, created_by)
  VALUES (
    p_start_year,
    public.academic_year_label(p_start_year),
    p_is_current,
    p_created_by
```

Section comment in the recovered script. Use it as a bookmark when applying SQL.

> **CHECKPOINT — Explain lines 1080–1101**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1102–1120

**COMPLETE_BACKEND.sql:1102-1120**

```
  )
  RETURNING * INTO v_year;
```

```
  FOR v_sem IN 1..4 LOOP
    SELECT * INTO v_bounds
    FROM public.academic_semester_bounds(p_start_year, v_sem);

    INSERT INTO public.academic_terms (
      academic_year_id, semester, starts_on, ends_on
    ) VALUES (
      v_year.id, v_sem, v_bounds.starts_on, v_bounds.ends_on
    );
  END LOOP;

  RETURN v_year;
END;
$function$

-- FILE: fn_current_profile_role.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1102–1120**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1121–1128

**COMPLETE_BACKEND.sql:1121-1128**

```
CREATE OR REPLACE FUNCTION public.current_profile_role()
 RETURNS user_role
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$ SELECT role FROM public.profiles WHERE id = auth.uid(); $function$

-- FILE: fn_ensure_chat_room_for_group.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1121–1128**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1129–1143

**COMPLETE_BACKEND.sql:1129-1143**

```
CREATE OR REPLACE FUNCTION public.ensure_chat_room_for_group()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  INSERT INTO public.chat_rooms (group_id)
  VALUES (NEW.id)
  ON CONFLICT (group_id) DO NOTHING;
  RETURN NEW;
END;
$function$

-- FILE: fn_ensure_lesson_session.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1129–1143**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1144–1165

**COMPLETE_BACKEND.sql:1144-1165**

```
CREATE OR REPLACE FUNCTION public.ensure_lesson_session(p_group_id uuid, p_session_date date)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  sid UUID;
BEGIN
  IF p_session_date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot create a lesson session for a future date'
      USING ERRCODE = 'check_violation';
  END IF;

  INSERT INTO public.lesson_sessions (group_id, session_date)
  VALUES (p_group_id, p_session_date)
  ON CONFLICT (group_id, session_date)
  DO UPDATE SET group_id = EXCLUDED.group_id
  RETURNING id INTO sid;
  RETURN sid;
END;
$function$
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1144–1165**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1166–1178

**COMPLETE_BACKEND.sql:1166-1178**

```
-- FILE: fn_get_auth_email_by_username.sql
CREATE OR REPLACE FUNCTION public.get_auth_email_by_username(p_username text)
 RETURNS text
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT auth_email FROM public.profiles
  WHERE lower(username) = lower(trim(p_username)) AND is_disabled = FALSE LIMIT 1;
$function$

-- FILE: fn_is_admin.sql
```

Section comment in the recovered script. Use it as a bookmark when applying SQL.

> **CHECKPOINT — Explain lines 1166–1178**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1179–1190

**COMPLETE_BACKEND.sql:1179-1190**

```
CREATE OR REPLACE FUNCTION public.is_admin()
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin' AND is_disabled =
FALSE
  );
$function$

-- FILE: fn_is_student_in_group.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1179–1190**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1191–1204

**COMPLETE_BACKEND.sql:1191-1204**

```
CREATE OR REPLACE FUNCTION public.is_student_in_group(gid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = gid AND s.profile_id = auth.uid()
  );
$function$

-- FILE: fn_is_teacher_of_group.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1191–1204**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1205–1218

**COMPLETE_BACKEND.sql:1205-1218**

```
CREATE OR REPLACE FUNCTION public.is_teacher_of_group(gid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.groups g
    JOIN public.teachers t ON t.id = g.teacher_id
    WHERE g.id = gid AND t.profile_id = auth.uid()
  );
$function$
```

```
-- FILE: fn_notify_chat_message.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1205–1218**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1219–1240

**COMPLETE_BACKEND.sql:1219-1240**

```
CREATE OR REPLACE FUNCTION public.notify_chat_message(p_user_id uuid, p_title text, p_body text,
 p_action_path text, p_room_id uuid)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_id UUID;
BEGIN
  SELECT id INTO v_id
  FROM public.notifications
  WHERE user_id = p_user_id
    AND kind = 'chat'
    AND ref_id = p_room_id
    AND is_read = FALSE
  ORDER BY created_at DESC
  LIMIT 1;

  IF v_id IS NOT NULL THEN
    UPDATE public.notifications
    SET title = p_title,
        body = COALESCE(p_body, ''),
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1219–1240**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1241–1252

**COMPLETE_BACKEND.sql:1241-1252**

```
        action_path = p_action_path,
        created_at = NOW()
    WHERE id = v_id;
  ELSE
    PERFORM public.notify_user(
      p_user_id, p_title, p_body, 'chat', p_action_path, p_room_id
    );
  END IF;
END;
$function$

-- FILE: fn_notify_user.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1241–1252**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1253–1272

**COMPLETE_BACKEND.sql:1253-1272**

```
CREATE OR REPLACE FUNCTION public.notify_user(p_user_id uuid, p_title text, p_body text, p_kind
text DEFAULT 'system'::text, p_action_path text DEFAULT NULL::text, p_ref_id uuid DEFAULT NULL::
uuid)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  INSERT INTO public.notifications (user_id, title, body, kind, action_path, ref_id)
  VALUES (
    p_user_id,
    p_title,
    COALESCE(p_body, ''),
    COALESCE(NULLIF(trim(p_kind), ''), 'system'),
    p_action_path,
    p_ref_id
  );
END;
$function$

-- FILE: fn_profile_route_prefix.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1253–1272**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1273–1288

**COMPLETE_BACKEND.sql:1273-1288**

```
CREATE OR REPLACE FUNCTION public.profile_route_prefix(p_user_id uuid)
 RETURNS text
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT CASE role
    WHEN 'teacher' THEN '/teacher'
    WHEN 'admin' THEN '/admin'
    ELSE '/student'
  END
  FROM public.profiles
  WHERE id = p_user_id;
$function$

-- FILE: fn_seed_demo_data.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1273–1288**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1289–1310

**COMPLETE_BACKEND.sql:1289-1310**

```
 CREATE OR REPLACE FUNCTION public.seed_demo_data()
  RETURNS text
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public', 'auth'
 AS $function$
 DECLARE
   v_admin UUID; v_math UUID; v_eng UUID; v_student UUID;
   v_math_tid UUID; v_eng_tid UUID; v_student_sid UUID;
   v_group UUID; v_pack UUID;
 BEGIN
   IF EXISTS (SELECT 1 FROM public.profiles LIMIT 1) THEN RETURN 'already_seeded'; END IF;

   v_admin := public._demo_insert_user('@admin', 'password123', 'Platform Admin', 'admin');
   v_math := public._demo_insert_user('@math_teacher', 'password123', 'Amina Math', 'teacher');
   v_eng := public._demo_insert_user('@english_teacher', 'password123', 'James English',
 'teacher');
   v_student := public._demo_insert_user('@test_user', 'password123', 'Demo Student', 'student');

   INSERT INTO public.teachers (profile_id, sphere) VALUES (v_math, 'math') RETURNING id INTO
 v_math_tid;
   INSERT INTO public.teachers (profile_id, sphere) VALUES (v_eng, 'english') RETURNING id INTO
 v_eng_tid;

   INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group A') RETURNING
 id INTO v_group;
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1289–1310**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1311–1332

**COMPLETE_BACKEND.sql:1311-1332**

```
   INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group B');
   INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group C');
   INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group A');
   INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group B');
   INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group C');

   SELECT id INTO v_student_sid FROM public.students WHERE profile_id = v_student;
   INSERT INTO public.group_students (group_id, student_id) VALUES (v_group, v_student_sid);

   INSERT INTO public.assignments (teacher_id, group_id, type, title, description, due_date)
   VALUES (v_math_tid, v_group, 'static', 'Read page 12', 'Review chapter on linear equations.',
 NOW() + INTERVAL '2 days');

   INSERT INTO public.assignments (teacher_id, group_id, type, title, description, due_date)
   VALUES (v_math_tid, v_group, 'homework_pack', 'Algebra Week 1 Pack', 'Complete all tasks
 below.', NOW() + INTERVAL '5 days')
   RETURNING id INTO v_pack;

   INSERT INTO public.tasks (assignment_id, title, description, sort_order, link_url)
   VALUES (v_pack, 'Practice set A', 'Solve exercises 1-10.', 1, 'https://www.khanacademy.org/
 math/algebra');
```

```
  INSERT INTO public.tasks (assignment_id, title, description, sort_order)
  VALUES (v_pack, 'Written reflection', 'Explain how you checked your answers.', 2);

  INSERT INTO public.announcements (teacher_id, group_id, title, description)
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1311–1332**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1333–1344

**COMPLETE_BACKEND.sql:1333-1344**

```
  VALUES (v_math_tid, v_group, 'Welcome to Math Group A', 'Office hours every Wednesday.');

  INSERT INTO public.attendance (group_id, student_id, status, date)
  VALUES (v_group, v_student_sid, 'present', CURRENT_DATE);

  INSERT INTO public.notifications (user_id, title, body)
  VALUES (v_student, 'Welcome to MU Study', 'Your demo account is ready.');

  RETURN 'seeded';
END; $function$

-- FILE: fn_sync_submission_mark.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1333–1344**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1345–1366

**COMPLETE_BACKEND.sql:1345-1366**

```
CREATE OR REPLACE FUNCTION public.sync_submission_mark()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE v_group_id UUID; v_teacher_id UUID; v_title TEXT;
BEGIN
  IF NEW.score IS NULL THEN
    DELETE FROM public.marks WHERE submission_id = NEW.id;
    RETURN NEW;
  END IF;

  SELECT a.group_id, a.teacher_id, tk.title
    INTO v_group_id, v_teacher_id, v_title
  FROM public.tasks tk
  JOIN public.assignments a ON a.id = tk.assignment_id
  WHERE tk.id = NEW.task_id;

  IF v_group_id IS NULL THEN RETURN NEW; END IF;

  INSERT INTO public.marks (
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1345–1366**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1367–1381

**COMPLETE_BACKEND.sql:1367-1381**

```
    group_id, student_id, teacher_id, submission_id, title, category, percentage, comment
  ) VALUES (
    v_group_id, NEW.student_id, v_teacher_id, NEW.id,
    COALESCE(v_title, 'Homework'), 'homework', NEW.score, COALESCE(NEW.feedback, '')
  )
  ON CONFLICT (submission_id) DO UPDATE SET
    percentage = EXCLUDED.percentage,
    title = EXCLUDED.title,
    comment = EXCLUDED.comment,
    updated_at = NOW();

  RETURN NEW;
END; $function$

-- FILE: fn_toggle_message_reaction.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1367–1381**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1382–1403

**COMPLETE_BACKEND.sql:1382-1403**

```
CREATE OR REPLACE FUNCTION public.toggle_message_reaction(p_message_id uuid, p_emoji text)
 RETURNS message_reactions
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_uid UUID := auth.uid();
  v_existing public.message_reactions;
  v_result public.message_reactions;
BEGIN
  IF v_uid IS NULL THEN
    RAISE EXCEPTION 'Not authenticated';
  END IF;

  IF p_emoji IS NULL OR char_length(trim(p_emoji)) < 1 THEN
    RAISE EXCEPTION 'Emoji required';
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM public.chat_messages m
    WHERE m.id = p_message_id
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1382–1403**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1404–1425

**COMPLETE_BACKEND.sql:1404-1425**

```
      AND m.is_deleted = FALSE
      AND public.can_access_chat_room(m.room_id)
  ) THEN
    RAISE EXCEPTION 'Message not found or inaccessible';
  END IF;

  SELECT * INTO v_existing
  FROM public.message_reactions
  WHERE message_id = p_message_id AND user_id = v_uid;

  IF v_existing.id IS NOT NULL AND v_existing.emoji = p_emoji THEN
    DELETE FROM public.message_reactions WHERE id = v_existing.id;
    RETURN v_existing;
  END IF;

  INSERT INTO public.message_reactions (message_id, user_id, emoji)
  VALUES (p_message_id, v_uid, p_emoji)
  ON CONFLICT (message_id, user_id)
  DO UPDATE SET emoji = EXCLUDED.emoji, created_at = NOW()
  RETURNING * INTO v_result;

  RETURN v_result;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1404–1425**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1426–1429

**COMPLETE_BACKEND.sql:1426-1429**

```
END;
$function$

-- FILE: fn_touch_chat_message_updated_at.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1426–1429**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1430–1442

**COMPLETE_BACKEND.sql:1430-1442**

```
CREATE OR REPLACE FUNCTION public.touch_chat_message_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  NEW.updated_at := NOW();
  RETURN NEW;
END;
$function$

-- FILE: fn_touch_updated_at.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1430–1442**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1443–1450

**COMPLETE_BACKEND.sql:1443-1450**

```
CREATE OR REPLACE FUNCTION public.touch_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END; $function$

-- FILE: fn_trg_announcement_notify.sql
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1443–1450**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1451–1472

**COMPLETE_BACKEND.sql:1451-1472**

```
CREATE OR REPLACE FUNCTION public.trg_announcement_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_prefix TEXT;
BEGIN
  FOR r IN
    SELECT s.profile_id AS uid
    FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = NEW.group_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    PERFORM public.notify_user(
      r.uid,
      'Announcement',
      NEW.title,
      'announcement',
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1451–1472**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1473–1481

**COMPLETE_BACKEND.sql:1473-1481**

```
      v_prefix || '/dashboard',
      NEW.id
    );
  END LOOP;
  RETURN NEW;
END;
$function$

-- FILE: fn_trg_assignment_notify.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1473–1481**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1482–1503

**COMPLETE_BACKEND.sql:1482-1503**

```
CREATE OR REPLACE FUNCTION public.trg_assignment_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_prefix TEXT;
BEGIN
  FOR r IN
    SELECT s.profile_id AS uid
    FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = NEW.group_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    PERFORM public.notify_user(
      r.uid,
      'New homework',
      NEW.title,
      'assignment',
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1482–1503**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1504–1512

**COMPLETE_BACKEND.sql:1504-1512**

```
      v_prefix || '/assignments/' || NEW.id::text,
      NEW.id
    );
  END LOOP;
  RETURN NEW;
END;
$function$

-- FILE: fn_trg_chat_message_notify.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1504–1512**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1513–1534

**COMPLETE_BACKEND.sql:1513-1534**

```
 CREATE OR REPLACE FUNCTION public.trg_chat_message_notify()
  RETURNS trigger
  LANGUAGE plpgsql
  SECURITY DEFINER
  SET search_path TO 'public'
 AS $function$
 DECLARE
   r RECORD;
   v_group_id UUID;
   v_group_title TEXT;
   v_sender_name TEXT;
   v_preview TEXT;
   v_prefix TEXT;
   v_path TEXT;
 BEGIN
   IF NEW.is_deleted THEN
     RETURN NEW;
   END IF;

   SELECT cr.group_id, g.title
     INTO v_group_id, v_group_title
   FROM public.chat_rooms cr
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1513–1534**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1535–1556

**COMPLETE_BACKEND.sql:1535-1556**

```
   JOIN public.groups g ON g.id = cr.group_id
   WHERE cr.id = NEW.room_id;

   IF v_group_id IS NULL THEN RETURN NEW; END IF;

   SELECT COALESCE(full_name, username, 'Someone')
     INTO v_sender_name
   FROM public.profiles
   WHERE id = NEW.sender_id;

   v_preview := CASE NEW.message_type
     WHEN 'text' THEN left(COALESCE(NEW.text_content, ''), 120)
     WHEN 'image' THEN 'Photo'
     WHEN 'video' THEN 'Video'
     WHEN 'voice' THEN 'Voice message'
     ELSE 'New message'
   END;

   FOR r IN
     SELECT DISTINCT uid FROM (
       SELECT s.profile_id AS uid
       FROM public.group_students gs
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1535–1556**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1557–1578

**COMPLETE_BACKEND.sql:1557-1578**

```
      JOIN public.students s ON s.id = gs.student_id
      WHERE gs.group_id = v_group_id
      UNION
      SELECT t.profile_id AS uid
      FROM public.groups g
      JOIN public.teachers t ON t.id = g.teacher_id
      WHERE g.id = v_group_id
    ) members
    WHERE uid IS DISTINCT FROM NEW.sender_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    v_path := v_prefix || '/chat/' || NEW.room_id::text;
    PERFORM public.notify_chat_message(
      r.uid,
      COALESCE(v_group_title, 'Chat'),
      COALESCE(v_sender_name, 'Someone') || ': ' || v_preview,
      v_path,
      NEW.room_id
    );
  END LOOP;

  RETURN NEW;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1557–1578**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1579–1582

**COMPLETE_BACKEND.sql:1579-1582**

```
END;
$function$

-- FILE: fn_trg_mark_notify.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1579–1582**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1583–1604

**COMPLETE_BACKEND.sql:1583-1604**

```
CREATE OR REPLACE FUNCTION public.trg_mark_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
```

```
DECLARE
  v_uid UUID;
  v_prefix TEXT;
  v_body TEXT;
BEGIN
  IF TG_OP = 'UPDATE'
     AND NEW.percentage IS NOT DISTINCT FROM OLD.percentage
     AND NEW.comment IS NOT DISTINCT FROM OLD.comment THEN
    RETURN NEW;
  END IF;

  SELECT profile_id INTO v_uid FROM public.students WHERE id = NEW.student_id;
  IF v_uid IS NULL THEN RETURN NEW; END IF;

  v_prefix := COALESCE(public.profile_route_prefix(v_uid), '/student');
  v_body := NEW.title || ' — ' || ROUND(NEW.percentage)::TEXT || '%';
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1583–1604**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1605–1621

**COMPLETE_BACKEND.sql:1605-1621**

```
  IF NEW.comment IS NOT NULL AND length(trim(NEW.comment)) > 0 THEN
    v_body := v_body || E'\n' || left(trim(NEW.comment), 160);
  END IF;

  PERFORM public.notify_user(
    v_uid,
    'Work graded',
    v_body,
    'graded',
    v_prefix || '/marks',
    NEW.id
  );
  RETURN NEW;
END;
$function$

-- FILE: fn_trg_submission_notify.sql
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1605–1621**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1622–1643

**COMPLETE_BACKEND.sql:1622-1643**

```
CREATE OR REPLACE FUNCTION public.trg_submission_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_teacher_profile UUID;
  v_student_name TEXT;
  v_task_title TEXT;
  v_assignment_id UUID;
```

```
  v_prefix TEXT;
  v_is_submit BOOLEAN;
BEGIN
  v_is_submit :=
    (TG_OP = 'INSERT' AND (NEW.file_url IS NOT NULL OR NEW.text_answer IS NOT NULL OR NEW.
submitted_at IS NOT NULL))
    OR (
      TG_OP = 'UPDATE'
      AND (
        (OLD.file_url IS DISTINCT FROM NEW.file_url AND NEW.file_url IS NOT NULL)
        OR (OLD.text_answer IS DISTINCT FROM NEW.text_answer AND NEW.text_answer IS NOT NULL)
        OR (OLD.submitted_at IS DISTINCT FROM NEW.submitted_at AND NEW.submitted_at IS NOT NULL)
```

Function. Note LANGUAGE and SECURITY. DEFINER must check is_admin or membership.

> **CHECKPOINT — Explain lines 1622–1643**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1644–1665

**COMPLETE_BACKEND.sql:1644-1665**

```
        OR (OLD.is_completed IS DISTINCT FROM NEW.is_completed AND NEW.is_completed IS TRUE)
      )
    );

  IF NOT v_is_submit THEN
    RETURN NEW;
  END IF;

  IF TG_OP = 'UPDATE'
     AND OLD.score IS DISTINCT FROM NEW.score
     AND OLD.file_url IS NOT DISTINCT FROM NEW.file_url
     AND OLD.text_answer IS NOT DISTINCT FROM NEW.text_answer
     AND OLD.submitted_at IS NOT DISTINCT FROM NEW.submitted_at
     AND OLD.is_completed IS NOT DISTINCT FROM NEW.is_completed THEN
    RETURN NEW;
  END IF;

  SELECT t.profile_id, tk.title, a.id
    INTO v_teacher_profile, v_task_title, v_assignment_id
  FROM public.tasks tk
  JOIN public.assignments a ON a.id = tk.assignment_id
  JOIN public.teachers t ON t.id = a.teacher_id
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1644–1665**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1666–1687

**COMPLETE_BACKEND.sql:1666-1687**

```
  WHERE tk.id = NEW.task_id;

  IF v_teacher_profile IS NULL THEN RETURN NEW; END IF;

  SELECT COALESCE(p.full_name, p.username, 'Student')
    INTO v_student_name
  FROM public.students s
  JOIN public.profiles p ON p.id = s.profile_id
  WHERE s.id = NEW.student_id;
```

```
  v_prefix := COALESCE(public.profile_route_prefix(v_teacher_profile), '/teacher');

  PERFORM public.notify_user(
    v_teacher_profile,
    'New submission',
    COALESCE(v_student_name, 'Student') || ' · ' || COALESCE(v_task_title, 'Task'),
    'submission',
    v_prefix || '/assignments/' || v_assignment_id::text,
    NEW.id
  );
  RETURN NEW;
END;
```

Supporting SQL (constraint, comment, or continuation). Keep it when you recreate the database.

> **CHECKPOINT — Explain lines 1666–1687**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1688–1691

**COMPLETE_BACKEND.sql:1688-1691**

```
$function$


-- ---------- TRIGGERS ----------
```

Trigger. Side effects (notify, touch updated_at, ensure chat room) happen here, not in Flutter.

> **CHECKPOINT — Explain lines 1688–1691**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1692–1695

**COMPLETE_BACKEND.sql:1692-1695**

```
CREATE TRIGGER announcement_created_notify AFTER INSERT ON announcements FOR EACH ROW EXECUTE
FUNCTION trg_announcement_notify();
CREATE TRIGGER assignment_created_notify AFTER INSERT ON assignments FOR EACH ROW EXECUTE
FUNCTION trg_assignment_notify();
CREATE TRIGGER tr_attendance_sync_session BEFORE INSERT OR UPDATE ON attendance FOR EACH ROW
EXECUTE FUNCTION attendance_sync_session();
CREATE TRIGGER tr_chat_message_notify AFTER INSERT ON chat_messages FOR EACH ROW EXECUTE
FUNCTION trg_chat_message_notify();
```

Trigger. Side effects (notify, touch updated_at, ensure chat room) happen here, not in Flutter.

> **CHECKPOINT — Explain lines 1692–1695**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1696–1699

**COMPLETE_BACKEND.sql:1696-1699**

```
CREATE TRIGGER tr_chat_messages_reply_same_room BEFORE INSERT OR UPDATE OF reply_to_message_id,
room_id ON chat_messages FOR EACH ROW EXECUTE FUNCTION chat_messages_reply_same_room();
CREATE TRIGGER tr_chat_messages_updated_at BEFORE UPDATE ON chat_messages FOR EACH ROW EXECUTE
FUNCTION touch_chat_message_updated_at();
CREATE TRIGGER tr_groups_create_chat_room AFTER INSERT ON groups FOR EACH ROW EXECUTE FUNCTION
```

```
ensure_chat_room_for_group();
CREATE TRIGGER tr_mark_notify AFTER INSERT OR UPDATE ON marks FOR EACH ROW EXECUTE FUNCTION
trg_mark_notify();
```

Trigger. Side effects (notify, touch updated_at, ensure chat room) happen here, not in Flutter.

> **CHECKPOINT — Explain lines 1696–1699**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1700–1710

**COMPLETE_BACKEND.sql:1700-1710**

```
CREATE TRIGGER tr_marks_touch BEFORE UPDATE ON marks FOR EACH ROW EXECUTE FUNCTION
touch_updated_at();
CREATE TRIGGER tr_submission_mark AFTER INSERT OR UPDATE OF score, feedback ON submissions FOR
EACH ROW EXECUTE FUNCTION sync_submission_mark();
CREATE TRIGGER tr_submission_notify AFTER INSERT OR UPDATE ON submissions FOR EACH ROW EXECUTE
FUNCTION trg_submission_notify();



-- MU Study — Complete RLS Policies (EXACT from live project)
-- Run AFTER helper functions (is_admin, is_teacher_of_group, is_student_in_group,
-- can_access_chat_room, current_profile_role) exist.
```

Trigger. Side effects (notify, touch updated_at, ensure chat room) happen here, not in Flutter.

> **CHECKPOINT — Explain lines 1700–1710**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1711–1714

**COMPLETE_BACKEND.sql:1711-1714**

```
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.teachers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.groups ENABLE ROW LEVEL SECURITY;
```

RLS on. Without policies this table is invisible to the app role.

> **CHECKPOINT — Explain lines 1711–1714**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1715–1718

**COMPLETE_BACKEND.sql:1715-1718**

```
ALTER TABLE public.group_students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.submissions ENABLE ROW LEVEL SECURITY;
```

RLS on. Without policies this table is invisible to the app role.

> **CHECKPOINT — Explain lines 1715–1718**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1719–1722

**COMPLETE_BACKEND.sql:1719-1722**

```
ALTER TABLE public.attendance ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.lesson_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.marks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.announcements ENABLE ROW LEVEL SECURITY;
```

RLS on. Without policies this table is invisible to the app role.

> **CHECKPOINT — Explain lines 1719–1722**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1723–1726

**COMPLETE_BACKEND.sql:1723-1726**

```
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignment_templates ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.academic_years ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.academic_terms ENABLE ROW LEVEL SECURITY;
```

RLS on. Without policies this table is invisible to the app role.

> **CHECKPOINT — Explain lines 1723–1726**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1727–1732

**COMPLETE_BACKEND.sql:1727-1732**

```
ALTER TABLE public.chat_rooms ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reads ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reactions ENABLE ROW LEVEL SECURITY;

-- profiles
```

RLS on. Without policies this table is invisible to the app role.

> **CHECKPOINT — Explain lines 1727–1732**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1733–1736

**COMPLETE_BACKEND.sql:1733-1736**

```
CREATE POLICY profiles_admin_all ON public.profiles FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY profiles_update_self ON public.profiles FOR UPDATE TO authenticated
```

```
  USING ((id = auth.uid()) AND (is_disabled = false)) WITH CHECK (id = auth.uid());
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1733–1736**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1737–1745

**COMPLETE_BACKEND.sql:1737-1745**

```
CREATE POLICY profiles_select ON public.profiles FOR SELECT TO authenticated USING (
  (id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students st ON ((st.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (st.profile_id = profiles.id))
  ))
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1737–1745**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1746–1766

**COMPLETE_BACKEND.sql:1746-1766**

```
CREATE POLICY profiles_select_groupmates ON public.profiles FOR SELECT TO authenticated USING (
  (EXISTS (
    SELECT 1 FROM (((group_students gs_self
      JOIN students s_self ON ((s_self.id = gs_self.student_id)))
      JOIN group_students gs_other ON ((gs_other.group_id = gs_self.group_id)))
      JOIN students s_other ON ((s_other.id = gs_other.student_id)))
    WHERE ((s_self.profile_id = auth.uid()) AND (s_other.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((group_students gs
      JOIN students s ON ((s.id = gs.student_id)))
      JOIN groups g ON ((g.id = gs.group_id)))
      JOIN teachers t ON ((t.id = g.teacher_id)))
    WHERE ((s.profile_id = auth.uid()) AND (t.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students s ON ((s.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (s.profile_id = profiles.id))
  ))
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1746–1766**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1767–1774

**COMPLETE_BACKEND.sql:1767-1774**

```
CREATE POLICY profiles_select_chat_message_senders ON public.profiles FOR SELECT TO
authenticated USING (
  EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE ((m.sender_id = profiles.id) AND can_access_chat_room(m.room_id))
  )
);

-- teachers / students
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1767–1774**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1775–1779

**COMPLETE_BACKEND.sql:1775-1779**

```
CREATE POLICY teachers_admin ON public.teachers FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY teachers_select ON public.teachers FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'student'::user_role)
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1775–1779**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1780–1786

**COMPLETE_BACKEND.sql:1780-1786**

```
CREATE POLICY students_admin ON public.students FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY students_select ON public.students FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'teacher'::user_role)
);

-- groups
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1780–1786**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1787–1793

**COMPLETE_BACKEND.sql:1787-1793**

```
CREATE POLICY groups_select ON public.groups FOR SELECT TO authenticated USING (
  is_teacher_of_group(id) OR is_student_in_group(id) OR is_admin()
);
CREATE POLICY groups_teacher_manage ON public.groups FOR ALL TO authenticated
```

```
  USING (is_teacher_of_group(id) OR is_admin())
  WITH CHECK (is_teacher_of_group(id) OR is_admin());
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1787–1793**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1794–1801

**COMPLETE_BACKEND.sql:1794-1801**

```
CREATE POLICY group_students_select ON public.group_students FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY group_students_manage ON public.group_students FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- assignments / tasks
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1794–1801**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1802–1808

**COMPLETE_BACKEND.sql:1802-1808**

```
CREATE POLICY assignments_select ON public.assignments FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY assignments_teacher ON public.assignments FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.profile_id
= auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1802–1808**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1809–1815

**COMPLETE_BACKEND.sql:1809-1815**

```
CREATE POLICY tasks_select ON public.tasks FOR SELECT TO authenticated USING (
  EXISTS (
    SELECT 1 FROM assignments a
    WHERE a.id = tasks.assignment_id
      AND (is_teacher_of_group(a.group_id) OR is_student_in_group(a.group_id) OR is_admin())
  )
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1809–1815**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1816–1826

**COMPLETE_BACKEND.sql:1816-1826**

```
CREATE POLICY tasks_teacher ON public.tasks FOR ALL TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin());

-- submissions
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1816–1826**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1827–1834

**COMPLETE_BACKEND.sql:1827-1834**

```
CREATE POLICY submissions_select ON public.submissions FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id = auth.
uid()))
  OR (EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  ))
  OR is_admin()
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1827–1834**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1835–1848

**COMPLETE_BACKEND.sql:1835-1848**

```
CREATE POLICY submissions_student ON public.submissions FOR ALL TO authenticated
  USING (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id
= auth.uid()))
  WITH CHECK (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.
profile_id = auth.uid()));
CREATE POLICY submissions_teacher_update ON public.submissions FOR UPDATE TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
```

```
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin());

-- attendance / sessions
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1835–1848**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1849–1854

**COMPLETE_BACKEND.sql:1849-1854**

```
CREATE POLICY attendance_select ON public.attendance FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY attendance_teacher ON public.attendance FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1849–1854**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1855–1862

**COMPLETE_BACKEND.sql:1855-1862**

```
CREATE POLICY lesson_sessions_select ON public.lesson_sessions FOR SELECT TO authenticated
USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY lesson_sessions_manage ON public.lesson_sessions FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- marks
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1855–1862**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1863–1866

**COMPLETE_BACKEND.sql:1863-1866**

```
CREATE POLICY marks_select ON public.marks FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = marks.student_id AND s.profile_id = auth.uid(
)))
  OR is_teacher_of_group(group_id) OR is_admin()
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1863–1866**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1867–1871

**COMPLETE_BACKEND.sql:1867-1871**

```
CREATE POLICY marks_teacher_manage ON public.marks FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- announcements / templates / notifications
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1867–1871**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1872–1877

**COMPLETE_BACKEND.sql:1872-1877**

```
CREATE POLICY announcements_select ON public.announcements FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY announcements_teacher ON public.announcements FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1872–1877**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1878–1884

**COMPLETE_BACKEND.sql:1878-1884**

```
CREATE POLICY assignment_templates_teacher ON public.assignment_templates FOR ALL TO
authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id
AND t.profile_id = auth.uid())) OR is_admin());
CREATE POLICY notifications_own ON public.notifications FOR ALL TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());

-- academic
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1878–1884**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1885–1888

**COMPLETE_BACKEND.sql:1885-1888**

```
CREATE POLICY academic_years_select ON public.academic_years FOR SELECT TO authenticated USING (
true);
CREATE POLICY academic_years_admin ON public.academic_years FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY academic_terms_select ON public.academic_terms FOR SELECT TO authenticated USING (
true);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1885–1888**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1889–1892

**COMPLETE_BACKEND.sql:1889-1892**

```
CREATE POLICY academic_terms_admin ON public.academic_terms FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- chat
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1889–1892**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1893–1897

**COMPLETE_BACKEND.sql:1893-1897**

```
CREATE POLICY chat_rooms_select ON public.chat_rooms FOR SELECT TO authenticated
  USING (can_access_chat_room(id));
CREATE POLICY chat_rooms_admin_insert ON public.chat_rooms FOR INSERT TO authenticated
  WITH CHECK (is_admin() OR is_teacher_of_group(group_id));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1893–1897**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1898–1901

**COMPLETE_BACKEND.sql:1898-1901**

```
CREATE POLICY chat_messages_select ON public.chat_messages FOR SELECT TO authenticated
  USING (can_access_chat_room(room_id));
CREATE POLICY chat_messages_insert ON public.chat_messages FOR INSERT TO authenticated
```

```
  WITH CHECK ((sender_id = auth.uid()) AND can_access_chat_room(room_id));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1898–1901**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1902–1907

**COMPLETE_BACKEND.sql:1902-1907**

```
CREATE POLICY chat_messages_update ON public.chat_messages FOR UPDATE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)))
  WITH CHECK (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
CREATE POLICY chat_messages_delete ON public.chat_messages FOR DELETE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1902–1907**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1908–1912

**COMPLETE_BACKEND.sql:1908-1912**

```
CREATE POLICY message_reads_select ON public.message_reads FOR SELECT TO authenticated USING (
  (user_id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  ))
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1908–1912**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1913–1916

**COMPLETE_BACKEND.sql:1913-1916**

```
CREATE POLICY message_reads_insert ON public.message_reads FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  )));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1913–1916**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1917–1922

**COMPLETE_BACKEND.sql:1917-1922**

```
CREATE POLICY message_reads_delete ON public.message_reads FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());

CREATE POLICY message_reactions_select ON public.message_reactions FOR SELECT TO authenticated
USING (
  EXISTS (SELECT 1 FROM chat_messages m WHERE m.id = message_reactions.message_id AND
can_access_chat_room(m.room_id))
);
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1917–1922**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1923–1927

**COMPLETE_BACKEND.sql:1923-1927**

```
CREATE POLICY message_reactions_insert ON public.message_reactions FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE m.id = message_reactions.message_id AND m.is_deleted = false AND can_access_chat_room(
m.room_id)
  )));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1923–1927**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1928–1946

**COMPLETE_BACKEND.sql:1928-1946**

```
CREATE POLICY message_reactions_update ON public.message_reactions FOR UPDATE TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());
CREATE POLICY message_reactions_delete ON public.message_reactions FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());


-- ---------- STORAGE BUCKETS ----------
INSERT INTO storage.buckets (id, name, public) VALUES
  ('avatars', 'avatars', true),
  ('submissions', 'submissions', false),
  ('assignments', 'assignments', false),
  ('announcements', 'announcements', false),
  ('chat-images', 'chat-images', false),
  ('chat-videos', 'chat-videos', false),
  ('chat-voices', 'chat-voices', false),
  ('chat-files', 'chat-files', false)
ON CONFLICT (id) DO NOTHING;

-- Storage policies (EXACT from live)
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1928–1946**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1947–1950

**COMPLETE_BACKEND.sql:1947-1950**

```
CREATE POLICY storage_avatars_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'avatars') WITH CHECK (bucket_id = 'avatars');
CREATE POLICY storage_avatars_read ON storage.objects FOR SELECT TO public
  USING (bucket_id = 'avatars');
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1947–1950**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1951–1954

**COMPLETE_BACKEND.sql:1951-1954**

```
CREATE POLICY storage_submissions_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'submissions') WITH CHECK (bucket_id = 'submissions');
CREATE POLICY storage_assignments_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'assignments') WITH CHECK (bucket_id = 'assignments');
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1951–1954**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1955–1959

**COMPLETE_BACKEND.sql:1955-1959**

```
CREATE POLICY storage_announcements_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'announcements') WITH CHECK (bucket_id = 'announcements');

CREATE POLICY storage_chat_images_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-images' AND can_access_chat_room(chat_storage_room_id(name)));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1955–1959**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1960–1964

**COMPLETE_BACKEND.sql:1960-1964**

```
CREATE POLICY storage_chat_images_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-images' AND can_access_chat_room(chat_storage_room_id(name))
AND (storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_images_update ON storage.objects FOR UPDATE TO authenticated
```

```
  USING (bucket_id = 'chat-images' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-images' AND (is_admin() OR (storage.foldername(name))[2] = (
auth.uid())::text));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1960–1964**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1965–1969

**COMPLETE_BACKEND.sql:1965-1969**

```
CREATE POLICY storage_chat_images_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-images' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));

CREATE POLICY storage_chat_videos_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-videos' AND can_access_chat_room(chat_storage_room_id(name)));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1965–1969**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1970–1974

**COMPLETE_BACKEND.sql:1970-1974**

```
CREATE POLICY storage_chat_videos_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-videos' AND can_access_chat_room(chat_storage_room_id(name))
AND (storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_videos_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-videos' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-videos' AND (is_admin() OR (storage.foldername(name))[2] = (
auth.uid())::text));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1970–1974**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1975–1979

**COMPLETE_BACKEND.sql:1975-1979**

```
CREATE POLICY storage_chat_videos_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-videos' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));

CREATE POLICY storage_chat_voices_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-voices' AND can_access_chat_room(chat_storage_room_id(name)));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1975–1979**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1980–1984

**COMPLETE_BACKEND.sql:1980-1984**

```
CREATE POLICY storage_chat_voices_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-voices' AND can_access_chat_room(chat_storage_room_id(name))
AND (storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_voices_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-voices' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-voices' AND (is_admin() OR (storage.foldername(name))[2] = (
auth.uid())::text));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1980–1984**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1985–1989

**COMPLETE_BACKEND.sql:1985-1989**

```
CREATE POLICY storage_chat_voices_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-voices' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));

CREATE POLICY storage_chat_files_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-files' AND can_access_chat_room(chat_storage_room_id(name)));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1985–1989**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1990–1994

**COMPLETE_BACKEND.sql:1990-1994**

```
CREATE POLICY storage_chat_files_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-files' AND can_access_chat_room(chat_storage_room_id(name)) AND
(storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_files_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-files' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-files' AND (is_admin() OR (storage.foldername(name))[2] = (auth.
uid())::text));
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1990–1994**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 1995–2008

**COMPLETE_BACKEND.sql:1995-2008**

```
CREATE POLICY storage_chat_files_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-files' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));


-- ---------- REALTIME ----------
-- Add tables to supabase_realtime publication (Dashboard or):
ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;
ALTER PUBLICATION supabase_realtime ADD TABLE public.chat_messages;
ALTER PUBLICATION supabase_realtime ADD TABLE public.message_reads;
ALTER PUBLICATION supabase_realtime ADD TABLE public.message_reactions;


-- ---------- GRANTS (recommended hardening vs live) ----------
-- EXACT live: many RPCs executable by anon; harden for production:
```

Access rule. USING is for existing rows; WITH CHECK is for new/changed rows.

> **CHECKPOINT — Explain lines 1995–2008**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 2009–2012

**COMPLETE_BACKEND.sql:2009-2012**

```
REVOKE EXECUTE ON FUNCTION public.seed_demo_data() FROM anon, authenticated;
REVOKE EXECUTE ON FUNCTION public._demo_insert_user(text, text, text, user_role) FROM anon,
authenticated;
GRANT EXECUTE ON FUNCTION public.get_auth_email_by_username(text) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.admin_create_user(text, text, text, user_role, sphere_type) TO
authenticated;
```

Privilege. Demo seed functions must stay revoked from anon/authenticated on real schools.

> **CHECKPOINT — Explain lines 2009–2012**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 2013–2016

**COMPLETE_BACKEND.sql:2013-2016**

```
GRANT EXECUTE ON FUNCTION public.admin_reset_password(uuid, text) TO authenticated;
GRANT EXECUTE ON FUNCTION public.chat_unread_counts() TO authenticated;
GRANT EXECUTE ON FUNCTION public.toggle_message_reaction(uuid, text) TO authenticated;
GRANT EXECUTE ON FUNCTION public.ensure_lesson_session(uuid, date) TO authenticated;
```

Privilege. Demo seed functions must stay revoked from anon/authenticated on real schools.

> **CHECKPOINT — Explain lines 2013–2016**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

## COMPLETE_BACKEND.sql lines 2017–2017

**COMPLETE_BACKEND.sql:2017-2017**

```
GRANT EXECUTE ON FUNCTION public.can_access_chat_room(uuid) TO authenticated;
```

Privilege. Demo seed functions must stay revoked from anon/authenticated on real schools.

> **CHECKPOINT — Explain lines 2017–2017**
> In one sentence: what object appears in Postgres after this chunk, and which Flutter file would break if you skipped it.

# Screen manual — every destination

A reconstruction is finished only when each screen exists, routes, and talks to the correct provider. Work this list with the running app.

## SplashScreen

| Field | Value |
|---|---|
| Route | /splash |
| File | lib/screens/auth/splash_screen.dart |
| Role gate | all |
| Purpose | Brand pause while session restores. |

Source length: 70 lines. Study build() first, then event handlers.

**lib/screens/auth/splash_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_animate/flutter_animate.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/brand/app_logo.dart';
import 'package:mu_study/core/theme/app_colors.dart';

class SplashScreen extends StatefulWidget {
  const SplashScreen({super.key});

  @override
  State<SplashScreen> createState() => _SplashScreenState();
}

class _SplashScreenState extends State<SplashScreen> {
  @override
  void initState() {
    super.initState();
    Future<void>.delayed(const Duration(milliseconds: 1200), () {
      if (mounted) context.go('/boot');
    });
  }

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final isDark = theme.brightness == Brightness.dark;
    final bg = theme.scaffoldBackgroundColor;
    final fg = isDark ? AppColors.darkTextPrimary : AppColors.darkChocolate;

    final overlay = isDark
        ? SystemUiOverlayStyle.light
        : SystemUiOverlayStyle.dark;

    return AnnotatedRegion<SystemUiOverlayStyle>(
      value: overlay.copyWith(
        statusBarColor: Colors.transparent,
        systemNavigationBarColor: bg,
        systemNavigationBarIconBrightness: isDark
            ? Brightness.light
            : Brightness.dark,
      ),
      child: Scaffold(
        backgroundColor: bg,
        body: Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const AppLogoImage(size: 88)
                  .animate(onPlay: (c) => c.repeat(reverse: true))
```

```
                .scale(
                  begin: const Offset(0.95, 0.95),
                  end: const Offset(1.05, 1.05),
                  duration: 900.ms,
                ),
            const SizedBox(height: 16),
            Text(
              'MU Study',
              style: theme.textTheme.headlineMedium?.copyWith(
                fontWeight: FontWeight.w700,
                color: fg,
              ),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /splash as role all. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — SplashScreen**
> Screenshot /splash. Annotate one widget and the table behind it. File the screenshot in private notes.

# SetupRequiredScreen

| Field | Value |
|---|---|
| Route | (no route) |
| File | lib/screens/auth/setup_required_screen.dart |
| Role gate | misconfig |
| Purpose | Shown when Env.isConfigured is false. |

Source length: 81 lines. Study build() first, then event handlers.

**lib/screens/auth/setup_required_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_theme.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class SetupRequiredScreen extends StatelessWidget {
  const SetupRequiredScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      onGenerateTitle: (ctx) => AppLocalizations.of(ctx).appName,
      theme: AppTheme.light(),
      supportedLocales: AppLocalizations.supportedLocales,
      localizationsDelegates: const [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      home: Builder(
        builder: (context) {
          final l10n = context.l10n;
          return Scaffold(
            body: SafeArea(
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      l10n.appName,
                      style: Theme.of(context).textTheme.headlineMedium
                          ?.copyWith(
                            fontWeight: FontWeight.w700,
                            color: AppColors.darkChocolate,
                          ),
                    ),
                    const SizedBox(height: 12),
                    Text(
                      l10n.setupRequiredTitle,
                      style: Theme.of(context).textTheme.titleMedium?.copyWith(
                        fontWeight: FontWeight.w700,
                      ),
                    ),
                    const SizedBox(height: 8),
                    Text(
                      l10n.setupRequiredBody,
                      style: Theme.of(context).textTheme.bodyLarge,
                    ),
                    const SizedBox(height: 24),
                    Text(
                      l10n.setupOption1,
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                    const SizedBox(height: 8),
```

```
                    const SelectableText(
                      'cp .env.example .env\n'
                      '# Edit .env and set SUPABASE_ANON_KEY from Supabase → Settings → API\n'
                      'flutter run',
                    ),
                    const SizedBox(height: 20),
                    Text(
                      l10n.setupOption2,
                      style: Theme.of(context).textTheme.titleMedium,
                    ),
                    const SizedBox(height: 8),
                    const SelectableText(
                      'flutter run --dart-define-from-file=.env',
                    ),
                  ],
                ),
              ),
            ),
          ),
        );
      },
    ),
  );
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to (no route) as role misconfig. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — SetupRequiredScreen**
> Screenshot (no route). Annotate one widget and the table behind it. File the screenshot in private notes.

## LoginScreen

| Field | Value |
|---|---|
| Route | /login |
| File | lib/screens/auth/login_screen.dart |
| Role gate | guest |
| Purpose | Username + password → AuthRepository. |

Source length: 450 lines. Study build() first, then event handlers.

**lib/screens/auth/login_screen.dart (opening)**

```
import 'dart:math' as math;

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/components/brand/app_logo.dart';
import 'package:mu_study/components/buttons/primary_button.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/constants/env.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';

class LoginScreen extends ConsumerStatefulWidget {
  const LoginScreen({super.key});

  @override
  ConsumerState<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends ConsumerState<LoginScreen> {
  final _usernameController = TextEditingController();
  final _passwordController = TextEditingController();
  bool _loading = false;
  String? _error;

  @override
  void dispose() {
    _usernameController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  Future<void> _login() async {
    final l10n = context.l10n;
    if (_usernameController.text.isEmpty || _passwordController.text.isEmpty) {
      setState(() => _error = l10n.loginMissingFields);
      return;
    }

    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      await ref
          .read(authRepositoryProvider)
          .signIn(_usernameController.text.trim(), _passwordController.text);
      ref.invalidate(currentProfileProvider);
      if (!mounted) return;
      context.go('/boot');
    } on AuthException catch (_) {
      setState(() {
        _error = l10n.loginIncorrect;
      });
```

```
    } on PostgrestException catch (e) {
      setState(() {
        _error = _messageForPostgrest(e, l10n.loginConfigError);
      });
    } catch (e) {
      setState(() {
        _error = kDebugMode ? 'Sign in error: $e' : l10n.loginFailed;
      });
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  String _messageForPostgrest(PostgrestException e, String configError) {
    final msg = e.message.toLowerCase();
    if (msg.contains('jwt') ||
        msg.contains('api key') ||
        msg.contains('invalid') && msg.contains('key')) {
      return configError;
    }
    return e.message;
  }
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /login as role guest. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — LoginScreen**
> Screenshot /login. Annotate one widget and the table behind it. File the screenshot in private notes.

# LanguageSelectionScreen

| Field | Value |
|---|---|
| Route | /language |
| File | lib/screens/auth/language_selection_screen.dart |
| Role gate | authed |
| Purpose | Forces EN/RU/UZ before any shell. |

Source length: 266 lines. Study build() first, then event handlers.

**lib/screens/auth/language_selection_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/core/locale/app_locale.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/settings/locale_provider.dart';

class LanguageSelectionScreen extends ConsumerStatefulWidget {
  const LanguageSelectionScreen({super.key});

  @override
  ConsumerState<LanguageSelectionScreen> createState() =>
      _LanguageSelectionScreenState();
}

class _LanguageSelectionScreenState
    extends ConsumerState<LanguageSelectionScreen> {
  AppLanguage? _selected;
  bool _saving = false;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted) return;
      setState(() {
        _selected = AppLanguage.fromLocale(ref.read(localeProvider));
      });
    });
  }

  Future<void> _continue() async {
    final lang = _selected;
    if (lang == null || _saving) return;

    setState(() => _saving = true);
    try {
      await ref.read(localeProvider.notifier).setLanguage(lang);
      if (!mounted) return;
      context.go('/boot');
    } finally {
      if (mounted) setState(() => _saving = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final profile = ref.watch(currentProfileProvider).valueOrNull;

    return Scaffold(
      backgroundColor: AppColors.cream,
      body: SafeArea(
        child: Center(
          child: ConstrainedBox(
```

```
constraints: const BoxConstraints(maxWidth: 480),
child: Padding(
  padding: const EdgeInsets.all(28),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      const Spacer(),
      Text(
        l10n.appName,
        textAlign: TextAlign.center,
        style: Theme.of(context).textTheme.headlineMedium?.copyWith(
          fontWeight: FontWeight.w800,
          color: AppColors.accent(context),
        ),
      ),
      const SizedBox(height: 12),
      Text(
        l10n.languageTitle,
        textAlign: TextAlign.center,
        style: Theme.of(context).textTheme.titleLarge?.copyWith(
          fontWeight: FontWeight.w800,
        ),
      ),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /language as role authed. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — LanguageSelectionScreen**
> Screenshot /language. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentDashboardScreen

| Field | Value |
|---|---|
| Route | /student/dashboard |
| File | lib/screens/student/home/student_dashboard_screen.dart |
| Role gate | student |
| Purpose | Home + expandable announcements. |

Source length: 682 lines. Study build() first, then event handlers.

**lib/screens/student/home/student_dashboard_screen.dart (opening)**

```
import 'dart:math';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/utils/time_of_day_greeting.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/components/marks/mark_average_card.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/student/home/widgets/dashboard_announcements.dart';
import 'package:skeletonizer/skeletonizer.dart';

class StudentDashboardScreen extends ConsumerStatefulWidget {
  const StudentDashboardScreen({super.key});

  @override
  ConsumerState<StudentDashboardScreen> createState() =>
      _StudentDashboardScreenState();
}

class _StudentDashboardScreenState
    extends ConsumerState<StudentDashboardScreen> {
  late final int _dailyGreetingIndex;

  @override
  void initState() {
    super.initState();
    _dailyGreetingIndex = Random().nextInt(4);
  }

  String _dailyMotivation(AppLocalizations l10n) {
    final lines = [
      l10n.dailyMotivation1,
      l10n.dailyMotivation2,
      l10n.dailyMotivation3,
      l10n.dailyMotivation4,
    ];
    return lines[_dailyGreetingIndex];
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
```

```
final profile = ref.watch(currentProfileProvider);
final groupId = ref.watch(activeGroupIdProvider);
final assignments = ref.watch(assignmentsProvider(groupId));
final notifications = ref.watch(notificationsProvider);
final announcements = ref.watch(announcementsProvider(groupId));
final marks = ref.watch(myMarksProvider);
final average = ref.watch(myMarkAverageProvider).valueOrNull;

return RefreshIndicator(
  onRefresh: () async {
    ref.invalidate(myGroupsProvider);
    ref.invalidate(myMarksProvider);
    ref.invalidate(myMarkAverageProvider);
    if (groupId != null) {
      ref.invalidate(assignmentsProvider(groupId));
      ref.invalidate(announcementsProvider(groupId));
    }
  },
  child: ListView(
    padding: EdgeInsets.only(
      left: 20,
      right: 20,
      top: MediaQuery.of(context).padding.top + 24,
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/dashboard as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentDashboardScreen**
> Screenshot /student/dashboard. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentAssignmentsScreen

| Field | Value |
|---|---|
| Route | /student/assignments |
| File | lib/screens/student/homework/student_assignments_screen.dart |
| Role gate | student |
| Purpose | Static vs pack list. |

Source length: 371 lines. Study build() first, then event handlers.

**lib/screens/student/homework/student_assignments_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';

class StudentAssignmentsScreen extends ConsumerWidget {
  const StudentAssignmentsScreen({super.key});

  void _open(BuildContext context, Assignment a) {
    context.push('/student/assignments/${a.id}');
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groupId = ref.watch(activeGroupIdProvider);
    final assignments = ref.watch(assignmentsProvider(groupId));

    final l10n = context.l10n;
    return Scaffold(
      extendBody: true,
      appBar: AppBar(title: Text(l10n.assignmentsTitle)),
      body: assignments.when(
        data: (page) {
          final list = sortAssignmentsByDue(page.items);
          if (list.isEmpty) {
            return ListView(
              padding: EdgeInsets.only(
                left: 20,
                right: 20,
                top: 12,
                bottom: shellContentBottomPadding(context),
              ),
              children: [
                const GroupSelector(padding: EdgeInsets.only(bottom: 12)),
                EmptyState(
                  title: l10n.assignmentsEmpty,
                  subtitle: l10n.assignmentsEmptySubtitle,
                  icon: Icons.task_alt_rounded,
                ),
              ],
            );
          }
          final isNeu = Neumorphic.isActive(context);
          return ListView.builder(
```

```
clipBehavior: isNeu ? Clip.none : Clip.hardEdge,
padding: EdgeInsets.only(
  left: 20,
  right: 20,
  top: 12,
  bottom: shellContentBottomPadding(context),
),
itemCount: list.length + 1 + (page.hasMore ? 1 : 0),
itemBuilder: (_, i) {
  if (i == 0) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
        PremiumSectionHeader(
          title: l10n.assignmentsLearningPath,
          subtitle: l10n.assignmentsActive,
        ),
      ],
    );
  }
  if (i == list.length + 1) {
    return LoadMoreButton(
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/assignments as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentAssignmentsScreen**
> Screenshot /student/assignments. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentTaskSubmitScreen

| Field | Value |
|---|---|
| Route | /student/tasks/:taskId |
| File | lib/screens/student/homework/student_task_submit_screen.dart |
| Role gate | student |
| Purpose | Text / file / voice submit. |

Source length: 506 lines. Study build() first, then event handlers.

**lib/screens/student/homework/student_task_submit_screen.dart (opening)**

```
import 'dart:typed_data';

import 'package:file_picker/file_picker.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:image_picker/image_picker.dart';
import 'package:mu_study/components/attachments/attachment_preview.dart';
import 'package:mu_study/components/attachments/voice_note_field.dart';
import 'package:mu_study/components/buttons/primary_button.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/dialogs/pdf_viewer_sheet.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:url_launcher/url_launcher.dart';

class StudentTaskSubmitScreen extends ConsumerStatefulWidget {
  const StudentTaskSubmitScreen({super.key, required this.taskId});

  final String taskId;

  @override
  ConsumerState<StudentTaskSubmitScreen> createState() =>
      _StudentTaskSubmitScreenState();
}

const _signedUrlTtlSeconds = 60 * 60 * 24 * 365 * 10;

class _StudentTaskSubmitScreenState
    extends ConsumerState<StudentTaskSubmitScreen> {
  final _textController = TextEditingController();
  bool _loading = false;
  bool _completed = false;
  String? _fileUrl;
  String? _fileName;
  String? _storagePath;
  Uint8List? _localBytes;
  String? _voiceUrl;
  String? _voiceStoragePath;
  Map<String, dynamic>? _task;

  @override
  void initState() {
    super.initState();
    _load();
  }

  Future<void> _load() async {
    final task = await supabase
        .from('tasks')
        .select()
        .eq('id', widget.taskId)
        .maybeSingle();
```

```
    if (task != null) setState(() => _task = Map<String, dynamic>.from(task));

    final student = await ref.read(studentProfileProvider.future);
    if (student == null) return;
    final sub = await supabase
        .from('submissions')
        .select()
        .eq('task_id', widget.taskId)
        .eq('student_id', student.id)
        .maybeSingle();
    if (sub != null && mounted) {
      final url = sub['file_url'] as String?;
      final voice = sub['voice_url'] as String?;
      setState(() {
        _textController.text = sub['text_answer'] as String? ?? '';
        _fileUrl = url;
        _fileName = url == null ? null : attachmentFileName(url);
        _storagePath = url == null ? null : storagePathFromSignedUrl(url);
        _localBytes = null;
        _voiceUrl = voice;
        _voiceStoragePath = voice == null
            ? null
            : storagePathFromSignedUrl(voice);
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/tasks/:taskId as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentTaskSubmitScreen**
> Screenshot /student/tasks/:taskId. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentFocusScreen

| Field | Value |
|---|---|
| Route | /student/focus |
| File | lib/screens/student/focus/student_focus_screen.dart |
| Role gate | student |
| Purpose | Pomodoro or free timer (local). |

Source length: 160 lines. Study build() first, then event handlers.

**lib/screens/student/focus/student_focus_screen.dart (opening)**

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/focus/focus_session_provider.dart';
import 'package:mu_study/screens/student/focus/focus_format.dart';
import 'package:mu_study/screens/student/focus/widgets/active_session_view.dart';
import 'package:mu_study/screens/student/focus/widgets/focus_home_view.dart';
import 'package:mu_study/screens/student/focus/widgets/focus_summary_view.dart';

class StudentFocusScreen extends ConsumerStatefulWidget {
  const StudentFocusScreen({super.key});

  @override
  ConsumerState<StudentFocusScreen> createState() => _StudentFocusScreenState();
}

class _StudentFocusScreenState extends ConsumerState<StudentFocusScreen>
    with WidgetsBindingObserver {
  Timer? _tick;
  final _noteController = TextEditingController();

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _tick = Timer.periodic(const Duration(seconds: 1), (_) {
      if (!mounted) return;
      final session = ref.read(focusSessionProvider).session;
      if (session == null || session.isPaused) return;
      setState(() {});
      _finishIfTimedOut(session);
    });
  }

  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _tick?.cancel();
    _noteController.dispose();
    super.dispose();
  }

  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    if (state == AppLifecycleState.resumed) {
      final session = ref.read(focusSessionProvider).session;
      if (session != null) _finishIfTimedOut(session);
    }
  }

  void _finishIfTimedOut(FocusSession session) {
    if (!session.isCompleteAt()) return;
```

```
  ref.read(focusSessionProvider.notifier).endSession(timedOut: true);
}

Future<void> _pickModeAndStart(Assignment a) async {
  final l10n = context.l10n;
  final mode = await showModalBottomSheet<FocusTimerMode>(
    context: context,
    showDragHandle: true,
    builder: (ctx) {
      return SafeArea(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(20, 8, 20, 24),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                l10n.focusChooseMode,
                style: Theme.of(ctx).textTheme.titleMedium?.copyWith(
                  fontWeight: FontWeight.w800,
                ),
              ),
              const SizedBox(height: 6),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/focus as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentFocusScreen**
> Screenshot /student/focus. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentMarksScreen

| Field | Value |
|---|---|
| Route | /student/marks |
| File | lib/screens/student/marks/student_marks_screen.dart |
| Role gate | student |
| Purpose | Own marks and averages. |

Source length: 287 lines. Study build() first, then event handlers.

**lib/screens/student/marks/student_marks_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/mark_average_card.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/components/marks/term_selector.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/progress/progress_report_share.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';

class StudentMarksScreen extends ConsumerWidget {
  const StudentMarksScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final marks = ref.watch(myMarksProvider);
    final stats = ref.watch(myMarkStatsProvider);
    final average = ref.watch(myMarkAverageProvider).valueOrNull;
    final markCount = stats.valueOrNull?.length ?? 0;
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.marksTitle),
        actions: [
          IconButton(
            tooltip: l10n.progressReportShare,
            onPressed: () => _shareProgress(context, ref),
            icon: const Icon(Icons.ios_share_rounded),
          ),
        ],
      ),
      body: RefreshIndicator(
        onRefresh: () async {
          ref.invalidate(myMarksProvider);
          ref.invalidate(myMarkStatsProvider);
          ref.invalidate(myMarkAverageProvider);
        },
        child: marks.when(
          data: (page) => ListView(
            padding: EdgeInsets.only(
              left: 20,
              right: 20,
              top: 12,
              bottom: shellContentBottomPadding(context),
```

```
),
children: [
  const TermSelector(padding: EdgeInsets.only(bottom: 12)),
  MarkAverageCard(average: average, markCount: markCount),
  if (markCount > 0) ...[
    PremiumSectionHeader(
      title: context.l10n.marksByCategory,
      subtitle: context.l10n.marksWhereStrongest,
      decorAsset: PaperAssets.brainVerified,
    ),
    _CategoryBreakdown(stats: stats.valueOrNull ?? const []),
    PremiumSectionHeader(
      title: context.l10n.marksAll,
      subtitle: context.l10n.marksNewestFirst,
      decorAsset: PaperAssets.documentLayout,
    ),
    if (page.items.isEmpty)
      EmptyState(
        title: context.l10n.marksEmptyTitle,
        subtitle: context.l10n.marksEmptySubtitle,
        icon: Icons.grade_outlined,
      )
    else ...[
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/marks as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentMarksScreen**
> Screenshot /student/marks. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentAttendanceScreen

| Field | Value |
|---|---|
| Route | /student/attendance |
| File | lib/screens/student/attendance/student_attendance_screen.dart |
| Role gate | student |
| Purpose | Calendar of statuses. |

Source length: 159 lines. Study build() first, then event handlers.

**lib/screens/student/attendance/student_attendance_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/attendance/attendance_calendar_grid.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';

class StudentAttendanceScreen extends ConsumerStatefulWidget {
  const StudentAttendanceScreen({super.key});

  @override
  ConsumerState<StudentAttendanceScreen> createState() =>
      _StudentAttendanceScreenState();
}

class _StudentAttendanceScreenState
    extends ConsumerState<StudentAttendanceScreen> {
  late DateTime _month;

  @override
  void initState() {
    super.initState();
    final now = DateTime.now();
    _month = DateTime(now.year, now.month);
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final groupId = ref.watch(activeGroupIdProvider);
    final student = ref.watch(studentProfileProvider);

    if (groupId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.attendanceTitle)),
        body: EmptyState(title: l10n.attendanceNoGroup),
      );
    }

    final studentId = student.valueOrNull?.id;
    if (studentId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.attendanceTitle)),
        body: student.when(
          loading: () => const Center(child: CircularProgressIndicator()),
          error: (e, _) => ErrorState(message: e.toString()),
          data: (_) => EmptyState(title: l10n.attendanceNoGroup),
        ),
      );
```

```
}

final from = DateTime(_month.year, _month.month, 1);
final to = DateTime(_month.year, _month.month + 1, 0);
final query = AttendanceQuery(
  groupId: groupId,
  studentId: studentId,
  from: from,
  to: to,
);
final records = ref.watch(attendanceRangeProvider(query));
final stats = ref.watch(attendanceRangeStatsProvider(query));

return Scaffold(
  appBar: AppBar(title: Text(l10n.attendanceTitle)),
  body: RefreshIndicator(
    onRefresh: () async {
      ref.invalidate(attendanceRangeProvider(query));
    },
    child: ListView(
      padding: EdgeInsets.fromLTRB(
        16,
        8,
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/attendance as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentAttendanceScreen**
> Screenshot /student/attendance. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentNotificationsScreen

| Field | Value |
|---|---|
| Route | /student/notifications |
| File | lib/screens/student/notifications/student_notifications_screen.dart |
| Role gate | student |
| Purpose | Inbox. |

Source length: 132 lines. Study build() first, then event handlers.

**lib/screens/student/notifications/student_notifications_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';

class StudentNotificationsScreen extends ConsumerStatefulWidget {
  const StudentNotificationsScreen({super.key});

  @override
  ConsumerState<StudentNotificationsScreen> createState() =>
      _StudentNotificationsScreenState();
}

class _StudentNotificationsScreenState
    extends ConsumerState<StudentNotificationsScreen> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) => _markAllRead());
  }

  Future<void> _markAllRead() async {
    if (!mounted) return;
    final profile = ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    await markAllNotificationsRead(profile.id);
    ref.invalidate(notificationsProvider);
  }

  void _open(AppNotification n) {
    final path = n.actionPath?.trim();
    if (path == null || path.isEmpty) return;
    context.push(path);
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final notifications = ref.watch(notificationsProvider);

    return Scaffold(
      appBar: AppBar(title: Text(l10n.notificationsTitle)),
      body: notifications.when(
        data: (list) {
          if (list.isEmpty) {
            return EmptyState(
              title: l10n.notificationsEmpty,
```

```
      subtitle: l10n.notificationsEmptySubtitle,
    );
  }
  return ListView.separated(
    padding: EdgeInsets.fromLTRB(
      16,
      16,
      16,
      shellContentBottomPadding(context, extra: 16),
    ),
    itemCount: list.length,
    separatorBuilder: (_, _) => const SizedBox(height: 8),
    itemBuilder: (context, index) {
      final n = list[index];
      final when = n.createdAt;
      final timeLabel = when == null
          ? null
          : DateFormat.MMMd().add_jm().format(when.toLocal());
      final canOpen =
          n.actionPath != null && n.actionPath!.trim().isNotEmpty;

      return PremiumCard(
        onTap: canOpen ? () => _open(n) : null,
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/notifications as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentNotificationsScreen**
> Screenshot /student/notifications. Annotate one widget and the table behind it. File the screenshot in private notes.

## StudentProfileScreen

| Field | Value |
|---|---|
| Route | /student/profile |
| File | lib/screens/student/profile/student_profile_screen.dart |
| Role gate | student |
| Purpose | Wrapper → AppProfileScreen. |

Source length: 12 lines. Study build() first, then event handlers.

**lib/screens/student/profile/student_profile_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/profile/app_profile_screen.dart';

class StudentProfileScreen extends ConsumerWidget {
  const StudentProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const AppProfileScreen();
  }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /student/profile as role student. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — StudentProfileScreen**
> Screenshot /student/profile. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherDashboardScreen

| Field | Value |
|---|---|
| Route | /teacher/dashboard |
| File | lib/screens/teacher/dashboard/teacher_dashboard_screen.dart |
| Role gate | teacher |
| Purpose | Shortcuts and group context. |

Source length: 476 lines. Study build() first, then event handlers.

**lib/screens/teacher/dashboard/teacher_dashboard_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/utils/time_of_day_greeting.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';

class TeacherDashboardScreen extends ConsumerWidget {
  const TeacherDashboardScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final profile = ref.watch(currentProfileProvider);
    final teacher = ref.watch(teacherProfileProvider);
    final groups = ref.watch(myGroupsProvider);
    final sphere = teacher.valueOrNull?.sphere;
    final l10n = context.l10n;

    return Scaffold(
      extendBody: true,
      floatingActionButtonLocation: shellFabLocation(context),
      floatingActionButton: FloatingActionButton(
        child: const Icon(Icons.add),
        onPressed: () {
          showModalBottomSheet(
            context: context,
            // The shell's nav bar floats above its own body, so sheets opened
            // on the shell navigator would slide out underneath it.
            useRootNavigator: true,
            backgroundColor: Colors.transparent,
            builder: (ctx) {
              final isMono =
                  AppStyleTokens.maybeOf(ctx)?.isMonochromeWorkspace ?? false;
              final sheetRadius = isMono ? 8.0 : 24.0;
              final iconRadius = isMono ? 4.0 : 20.0;

              Widget actionIcon({
                required Color color,
                required IconData icon,
              }) {
                final fill = AppColors.resolveAccent(ctx, color);
                final isNeu = Neumorphic.isActive(ctx);
                if (isNeu) {
                  return NeuIconWell(
                    icon: icon,
                    color: fill,
                    size: 20,
```

```
      padding: 10,
      elevation: NeuElevation.raised,
    );
  }
  return Container(
    width: 40,
    height: 40,
    alignment: Alignment.center,
    decoration: BoxDecoration(
      color: fill,
      borderRadius: BorderRadius.circular(iconRadius),
    ),
    child: Icon(icon, color: AppColors.onAccent(ctx), size: 20),
  );
}

return Container(
  margin: EdgeInsets.fromLTRB(
    16,
    16,
    16,
    16 + MediaQuery.viewPaddingOf(ctx).bottom,
  ),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/dashboard as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherDashboardScreen**
> Screenshot /teacher/dashboard. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherAssignmentsScreen

| Field | Value |
|---|---|
| Route | /teacher/assignments |
| File | lib/screens/teacher/assignments/teacher_assignments_screen.dart |
| Role gate | teacher |
| Purpose | Create/list assignments. |

Source length: 874 lines. Study build() first, then event handlers.

**lib/screens/teacher/assignments/teacher_assignments_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/homework/due_date_label.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/homework/assignment_templates.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class TeacherAssignmentsScreen extends ConsumerStatefulWidget {
  const TeacherAssignmentsScreen({super.key});

  @override
  ConsumerState<TeacherAssignmentsScreen> createState() =>
      _TeacherAssignmentsScreenState();
}

class _TeacherAssignmentsScreenState
    extends ConsumerState<TeacherAssignmentsScreen> {
  void _openPack(BuildContext context, Assignment a) {
    context.push('/teacher/assignments/${a.id}');
  }

  Future<void> _createAssignment({AssignmentTemplate? fromTemplate}) async {
    final teacher = await ref.read(teacherProfileProvider.future);
    final groups = await ref.read(myGroupsProvider.future);
    if (teacher == null || groups.isEmpty) return;

    var targetGroupId = ref.read(activeGroupIdProvider) ?? groups.first.id;
    final titleController = TextEditingController(
      text: fromTemplate?.title ?? '',
    );
    final descController = TextEditingController(
      text: fromTemplate?.description ?? '',
    );
    var type = fromTemplate?.type ?? AssignmentType.staticAssignment;
    DateTime? dueDate;

    if (!mounted) return;
    final l10n = context.l10n;
    await showDialog<void>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setStateLocal) => AlertDialog(
```

```
title: Text(
  fromTemplate == null
      ? l10n.assignmentNew
      : l10n.teacherCreateFromTemplate,
),
content: SingleChildScrollView(
  child: Column(
    mainAxisSize: MainAxisSize.min,
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      if (fromTemplate != null && fromTemplate.tasks.isNotEmpty)
        Padding(
          padding: const EdgeInsets.only(bottom: 12),
          child: Text(
            l10n.teacherTasksCopied(fromTemplate.tasks.length),
            style: TextStyle(
              color: AppColors.inkAlpha(context, 0.55),
              fontSize: 13,
            ),
          ),
        ),
      TextField(
        controller: titleController,
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/assignments as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherAssignmentsScreen**
> Screenshot /teacher/assignments. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherHomeworkPackScreen

| Field | Value |
|---|---|
| Route | /teacher/assignments/:id |
| File | lib/screens/teacher/homework/teacher_homework_pack_screen.dart |
| Role gate | teacher |
| Purpose | Tasks and pack progress. |

Source length: 420 lines. Study build() first, then event handlers.

**lib/screens/teacher/homework/teacher_homework_pack_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/homework/assignment_templates.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/homework/assignment_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class TeacherHomeworkPackScreen extends ConsumerStatefulWidget {
  const TeacherHomeworkPackScreen({super.key, required this.assignmentId});

  final String assignmentId;

  @override
  ConsumerState<TeacherHomeworkPackScreen> createState() =>
      _TeacherHomeworkPackScreenState();
}

class _TeacherHomeworkPackScreenState
    extends ConsumerState<TeacherHomeworkPackScreen> {
  Future<void> _saveAsTemplate(Assignment? assignment) async {
    if (assignment == null) return;
    final teacher = await ref.read(teacherProfileProvider.future);
    if (teacher == null || !mounted) return;
    final l10n = context.l10n;
    try {
      await saveAssignmentAsTemplate(
        teacherId: teacher.id,
        assignment: assignment,
      );
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.teacherTemplateSaved)));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));
    }
  }

  Future<void> _showTaskDialog({TaskItem? task, int currentCount = 0}) async {
    final assignment = ref
        .read(assignmentDetailProvider(widget.assignmentId))
        .valueOrNull;
    if (assignment?.type == AssignmentType.staticAssignment) return;

    final isEditing = task != null;
    final titleController = TextEditingController(text: task?.title ?? '');
    final descController = TextEditingController(text: task?.description ?? '');
```

```
final linkController = TextEditingController(text: task?.linkUrl ?? '');
final attachmentController = TextEditingController(
  text: task?.attachmentUrl ?? '',
);

if (!mounted) return;
final l10n = context.l10n;

await showDialog<void>(
  context: context,
  builder: (ctx) => AlertDialog(
    title: Text(isEditing ? l10n.teacherEditTask : l10n.teacherAddTask),
    content: SingleChildScrollView(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          TextField(
            controller: titleController,
            decoration: InputDecoration(
              labelText: l10n.commonTitle,
              hintText: l10n.teacherTaskTitleHint,
            ),
          ),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/assignments/:id as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherHomeworkPackScreen**
> Screenshot /teacher/assignments/:id. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherSubmissionsScreen

| Field | Value |
|---|---|
| Route | /teacher/submissions |
| File | lib/screens/teacher/homework/teacher_submissions_screen.dart |
| Role gate | teacher |
| Purpose | Review queue. |

Source length: 402 lines. Study build() first, then event handlers.

**lib/screens/teacher/homework/teacher_submissions_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/attachments/voice_note_field.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/homework/feedback_reuse.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/homework/teacher_submissions_provider.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:url_launcher/url_launcher.dart';

class TeacherSubmissionsScreen extends ConsumerWidget {
  const TeacherSubmissionsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final submissions = ref.watch(teacherSubmissionsProvider);
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.teacherSubmissions)),
      body: submissions.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (rows) {
          if (rows.isEmpty) {
            return EmptyState(
              title: l10n.teacherNoSubmissions,
              subtitle: l10n.teacherNoSubmissionsSubtitle,
              icon: Icons.fact_check_outlined,
            );
          }
          return RefreshIndicator(
            onRefresh: () async => ref.invalidate(teacherSubmissionsProvider),
            child: ListView.builder(
              padding: EdgeInsets.fromLTRB(
                20,
                20,
                20,
                shellContentBottomPadding(context),
              ),
              itemCount: rows.length,
              itemBuilder: (_, i) {
                final row = rows[i];
                final score = row['score'] == null
                    ? null
                    : double.tryParse('${row['score']}');
                return PremiumCard(
                  onTap: () => _review(context, ref, row),
                  padding: const EdgeInsets.symmetric(
```

```
      horizontal: 16,
      vertical: 14,
    ),
    child: Row(
      children: [
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                row['task_title'] as String? ?? l10n.taskTitle,
                style: const TextStyle(
                  fontWeight: FontWeight.w700,
                  fontSize: 15,
                ),
              ),
              const SizedBox(height: 2),
              Text(
                '${row['student_name']} · ${row['status']}',
                style: TextStyle(
                  fontSize: 12,
                  color: AppColors.inkAlpha(context, 0.5),
                ),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/submissions as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherSubmissionsScreen**
> Screenshot /teacher/submissions. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherAttendanceScreen

| Field | Value |
|---|---|
| Route | /teacher/attendance |
| File | lib/screens/teacher/attendance/teacher_attendance_screen.dart |
| Role gate | teacher |
| Purpose | Roll grid + ensure_lesson_session. |

Source length: 819 lines. Study build() first, then event handlers.

**lib/screens/teacher/attendance/teacher_attendance_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/attendance/attendance_calendar_grid.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';

class TeacherAttendanceScreen extends ConsumerStatefulWidget {
  const TeacherAttendanceScreen({super.key});

  @override
  ConsumerState<TeacherAttendanceScreen> createState() =>
      _TeacherAttendanceScreenState();
}

class _TeacherAttendanceScreenState
    extends ConsumerState<TeacherAttendanceScreen>
    with SingleTickerProviderStateMixin {
  final Map<String, AttendanceStatus> _edits = {};
  String? _editedGroupId;
  bool _saving = false;
  late final TabController _tabs;
  late DateTime _month;
  String? _historyStudentId;

  @override
  void initState() {
    super.initState();
    _tabs = TabController(length: 2, vsync: this);
    final now = DateTime.now();
    _month = DateTime(now.year, now.month);
  }

  @override
  void dispose() {
    _tabs.dispose();
    super.dispose();
  }

  AttendanceStatus _statusFor(
    String studentId,
    Map<String, AttendanceStatus> saved,
  ) {
    return _edits[studentId] ?? saved[studentId] ?? AttendanceStatus.present;
  }

  void _resetEditsIfGroupChanged(String groupId) {
    if (_editedGroupId != groupId) {
      _edits.clear();
```

```
      _editedGroupId = groupId;
      _historyStudentId = null;
    }
  }

  void _markAll(List<GroupMember> members, AttendanceStatus status) {
    setState(() {
      for (final m in members) {
        _edits[m.studentId] = status;
      }
    });
  }

  Future<void> _saveAll(
    String groupId,
    List<GroupMember> members,
    Map<String, AttendanceStatus> saved,
  ) async {
    if (members.isEmpty) return;
    setState(() => _saving = true);

    try {
      final byStudent = {
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/attendance as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherAttendanceScreen**
> Screenshot /teacher/attendance. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherMarksScreen

| Field | Value |
|---|---|
| Route | /teacher/marks |
| File | lib/screens/teacher/marks/teacher_marks_screen.dart |
| Role gate | teacher |
| Purpose | Gradebook. |

Source length: 481 lines. Study build() first, then event handlers.

**lib/screens/teacher/marks/teacher_marks_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/components/marks/term_selector.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/marks/marks_csv_export.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/teacher/marks/widgets/bulk_mark_sheet.dart';
import 'package:mu_study/screens/teacher/marks/widgets/mark_editor_sheet.dart';
import 'package:mu_study/screens/teacher/marks/widgets/mark_row.dart';
import 'package:mu_study/screens/teacher/marks/widgets/student_marks_sheet.dart';

class TeacherMarksScreen extends ConsumerWidget {
  const TeacherMarksScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groupId = ref.watch(activeGroupIdProvider);
    final l10n = context.l10n;

    if (groupId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.teacherGradebook)),
        body: EmptyState(
          title: l10n.teacherNoGroups,
          subtitle: l10n.teacherNoGroupsSubtitle,
          icon: Icons.groups_outlined,
        ),
      );
    }

    final members = ref.watch(groupMembersProvider(groupId));
    final averages = ref.watch(groupMarkAveragesProvider(groupId));
    final marks = ref.watch(groupMarksProvider(groupId));
    final markCount =
        ref.watch(groupMarkCountProvider(groupId)).valueOrNull ?? 0;
    final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];
    final groupTitle = groups
        .where((g) => g.id == groupId)
        .map((g) => g.title)
        .firstOrNull;

    return Scaffold(
      appBar: AppBar(
        title: Text(l10n.teacherGradebook),
        actions: [
          IconButton(
```

```
            tooltip: l10n.marksExportCsv,
            onPressed: () =>
                _export(context, ref, groupId, groupTitle ?? 'group'),
            icon: const Icon(Icons.ios_share_rounded),
          ),
        ],
      ),
      floatingActionButtonLocation: shellFabLocation(context),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _openBulkEntry(context, ref, groupId),
        icon: const Icon(Icons.playlist_add_rounded),
        label: Text(l10n.teacherGradeClass),
      ),
      body: RefreshIndicator(
        onRefresh: () async => _refresh(ref, groupId),
        child: ListView(
          padding: EdgeInsets.only(
            left: 20,
            right: 20,
            top: 8,
            bottom: shellContentBottomPadding(context, hasFab: true),
          ),
          children: [
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/marks as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherMarksScreen**
> Screenshot /teacher/marks. Annotate one widget and the table behind it. File the screenshot in private notes.

## TeacherAnalyticsScreen

| Field | Value |
|---|---|
| Route | /teacher/analytics |
| File | lib/screens/teacher/analytics/teacher_analytics_screen.dart |
| Role gate | teacher |
| Purpose | Charts from aggregates. |

Source length: 101 lines. Study build() first, then event handlers.

**lib/screens/teacher/analytics/teacher_analytics_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/analytics/teacher_analytics_charts.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';

class TeacherAnalyticsScreen extends ConsumerWidget {
  const TeacherAnalyticsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groupId = ref.watch(activeGroupIdProvider);
    final l10n = context.l10n;

    if (groupId == null) {
      return Scaffold(
        appBar: AppBar(title: Text(l10n.navAnalytics)),
        body: EmptyState(title: l10n.teacherNoData),
      );
    }

    final attendance = ref.watch(attendanceWeekTrendProvider(groupId));
    final averages = ref.watch(groupMarkAveragesProvider(groupId));

    return Scaffold(
      appBar: AppBar(title: Text(l10n.navAnalytics)),
      body: RefreshIndicator(
        onRefresh: () async {
          ref.invalidate(attendanceWeekTrendProvider(groupId));
          ref.invalidate(groupMarkAveragesProvider(groupId));
        },
        child: ListView(
          padding: EdgeInsets.fromLTRB(
            20,
            8,
            20,
            shellContentBottomPadding(context, extra: 8),
          ),
          children: [
            const GroupSelector(padding: EdgeInsets.only(bottom: 8)),
            PremiumSectionHeader(
              title: l10n.teacherMarkDistribution,
              subtitle: l10n.teacherStudentsByAverage,
              decorAsset: PaperAssets.texturedTanStar,
            ),
            averages.when(
              data: (map) {
                final bands = markBandPoints(
                  context,
```

```
        l10n,
        map.values.toList(),
      );
      if (bands.isEmpty) {
        return EmptyState(
          title: l10n.marksNoRecorded,
          subtitle: l10n.teacherNoMarksTrends,
          icon: Icons.grade_outlined,
        );
      }
      return PaperTaped(
        alignment: 0.45,
        child: PremiumCard(
          padding: const EdgeInsets.fromLTRB(16, 20, 16, 20),
          child: MarkDistributionDoughnut(points: bands),
        ),
      );
    },
    loading: () => const Center(child: CircularProgressIndicator()),
    error: (e, _) => ErrorState(message: e.toString()),
  ),
  const SizedBox(height: 8),
  PremiumSectionHeader(
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/analytics as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherAnalyticsScreen**
> Screenshot /teacher/analytics. Annotate one widget and the table behind it. File the screenshot in private notes.

# TeacherAnnouncementsScreen

| Field | Value |
|---|---|
| Route | /teacher/announcements |
| File | lib/screens/teacher/announcements/teacher_announcements_screen.dart |
| Role gate | teacher |
| Purpose | Group posts. |

Source length: 298 lines. Study build() first, then event handlers.

**lib/screens/teacher/announcements/teacher_announcements_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/inputs/group_selector.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class TeacherAnnouncementsScreen extends ConsumerStatefulWidget {
  const TeacherAnnouncementsScreen({super.key});

  @override
  ConsumerState<TeacherAnnouncementsScreen> createState() =>
      _TeacherAnnouncementsScreenState();
}

class _TeacherAnnouncementsScreenState
    extends ConsumerState<TeacherAnnouncementsScreen> {
  Future<void> _create() async {
    final teacher = await ref.read(teacherProfileProvider.future);
    final groups = await ref.read(myGroupsProvider.future);
    if (teacher == null || groups.isEmpty) return;
    final title = TextEditingController();
    final body = TextEditingController();
    var groupId = ref.read(activeGroupIdProvider) ?? groups.first.id;

    if (!mounted) return;
    final l10n = context.l10n;
    await showDialog<void>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setStateLocal) => AlertDialog(
          title: Text(l10n.announcementsNew),
          content: SingleChildScrollView(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                TextField(
                  controller: title,
                  decoration: InputDecoration(labelText: l10n.commonTitle),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: body,
                  decoration: InputDecoration(labelText: l10n.commonMessage),
                  maxLines: 4,
                ),
                const SizedBox(height: 16),
                DropdownButtonFormField<String>(
                  initialValue: groupId,
```

```
            decoration: InputDecoration(
              labelText: l10n.teacherTargetGroup,
            ),
            items: groups
                .map(
                  (g) =>
                      DropdownMenuItem(value: g.id, child: Text(g.title)),
                )
                .toList(),
            onChanged: (v) => setStateLocal(() => groupId = v ?? groupId),
          ),
        ],
      ),
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.pop(ctx),
        child: Text(l10n.commonCancel),
      ),
      ElevatedButton(
        onPressed: () async {
          if (title.text.trim().isEmpty) return;
          await supabase.from('announcements').insert({
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/announcements as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherAnnouncementsScreen**
> Screenshot /teacher/announcements. Annotate one widget and the table behind it. File the screenshot in private notes.

## TeacherNotificationsScreen

| Field | Value |
|---|---|
| Route | /teacher/notifications |
| File | lib/screens/teacher/notifications/teacher_notifications_screen.dart |
| Role gate | teacher |
| Purpose | Reuses inbox UI. |

Source length: 12 lines. Study build() first, then event handlers.

**lib/screens/teacher/notifications/teacher_notifications_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/student/notifications/student_notifications_screen.dart';

class TeacherNotificationsScreen extends ConsumerWidget {
  const TeacherNotificationsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const StudentNotificationsScreen();
  }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/notifications as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherNotificationsScreen**
> Screenshot /teacher/notifications. Annotate one widget and the table behind it. File the screenshot in private notes.

## TeacherProfileScreen

| Field | Value |
|---|---|
| Route | /teacher/profile |
| File | lib/screens/teacher/profile/teacher_profile_screen.dart |
| Role gate | teacher |
| Purpose | Wrapper → AppProfileScreen. |

Source length: 12 lines. Study build() first, then event handlers.

**lib/screens/teacher/profile/teacher_profile_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/profile/app_profile_screen.dart';

class TeacherProfileScreen extends ConsumerWidget {
  const TeacherProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const AppProfileScreen();
  }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /teacher/profile as role teacher. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — TeacherProfileScreen**
> Screenshot /teacher/profile. Annotate one widget and the table behind it. File the screenshot in private notes.

## AdminDashboardScreen

| Field | Value |
|---|---|
| Route | /admin/dashboard |
| File | lib/screens/admin/dashboard/admin_dashboard_screen.dart |
| Role gate | admin |
| Purpose | Platform counts. |

Source length: 346 lines. Study build() first, then event handlers.

**lib/screens/admin/dashboard/admin_dashboard_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:syncfusion_flutter_charts/charts.dart';

class AdminDashboardScreen extends ConsumerWidget {
  const AdminDashboardScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final stats = ref.watch(adminPlatformStatsProvider);

    return stats.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => ErrorState(message: e.toString()),
      data: (s) {
        final l10n = context.l10n;
        return RefreshIndicator(
          onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),
          child: ListView(
            padding: EdgeInsets.fromLTRB(
              20,
              24,
              20,
              shellContentBottomPadding(context, extra: 24),
            ),
            children: [
              Text(
                l10n.adminWorkspace,
                style: Theme.of(context).textTheme.bodyLarge?.copyWith(
                  color: AppColors.inkAlpha(context, 0.5),
                  fontWeight: FontWeight.w700,
                ),
              ),
              const SizedBox(height: 8),
              Text(
                l10n.adminPlatformOverview,
                style: Theme.of(context).textTheme.displaySmall,
              ),
              SizedBox(height: 32),

              GridView.count(
                shrinkWrap: true,
                physics: const NeverScrollableScrollPhysics(),
                crossAxisCount: 2,
                crossAxisSpacing: 16,
                mainAxisSpacing: 16,
                childAspectRatio: 1.2,
                children: [
                  _AdminStatCard(
```

```
      title: l10n.adminTeachers,
      value: s['teachers']!,
      icon: Icons.school_outlined,
      color: AppColors.accent(context),
    ),
    _AdminStatCard(
      title: l10n.adminStudents,
      value: s['students']!,
      icon: Icons.people_outline,
      color: AppColors.mutedGold,
    ),
    _AdminStatCard(
      title: l10n.adminGroupsLabel,
      value: s['groups']!,
      icon: Icons.groups_outlined,
      color: AppColors.successGreen,
    ),
    _AdminStatCard(
      title: l10n.adminMarksLabel,
      value: s['marks'] ?? 0,
      icon: Icons.grade_outlined,
      color: AppColors.warmAmber,
    ),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/dashboard as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminDashboardScreen**
> Screenshot /admin/dashboard. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminUsersScreen

| Field | Value |
|---|---|
| Route | /admin/users |
| File | lib/screens/admin/users/admin_users_screen.dart |
| Role gate | admin |
| Purpose | admin_create_user + disable. |

Source length: 319 lines. Study build() first, then event handlers.

**lib/screens/admin/users/admin_users_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/inputs/load_more_button.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/core/admin/credential_slips_source.dart';
import 'package:mu_study/screens/admin/helpers/admin_labels.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class AdminUsersScreen extends ConsumerStatefulWidget {
  const AdminUsersScreen({super.key});

  @override
  ConsumerState<AdminUsersScreen> createState() => _AdminUsersScreenState();
}

class _AdminUsersScreenState extends ConsumerState<AdminUsersScreen> {
  Future<void> _createUser() async {
    final username = TextEditingController(text: '@');
    final password = TextEditingController();
    final name = TextEditingController();
    var role = UserRole.student;
    var sphere = Sphere.math;
    String? selectedGroupId;

    final groupsRes = await supabase.from('groups').select('id, title');
    final groups = groupsRes as List<dynamic>;

    if (!mounted) return;

    await showDialog<void>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setLocal) {
          final l10n = ctx.l10n;
          return AlertDialog(
            title: Text(l10n.adminCreateUser),
            content: SingleChildScrollView(
              child: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  TextField(
                    controller: username,
                    decoration: InputDecoration(
                      labelText: l10n.loginUsername,
                      hintText: '@username',
                    ),
                  ),
                  const SizedBox(height: 16),
                  TextField(
```

```
  controller: password,
  decoration: InputDecoration(labelText: l10n.adminPassword),
  obscureText: true,
),
const SizedBox(height: 16),
TextField(
  controller: name,
  decoration: InputDecoration(labelText: l10n.adminFullName),
),
const SizedBox(height: 16),
DropdownButtonFormField<UserRole>(
  initialValue: role,
  decoration: InputDecoration(labelText: l10n.adminRole),
  items: UserRole.values
      .map(
        (r) => DropdownMenuItem(
          value: r,
          child: Text(localizedRole(l10n, r).toUpperCase()),
        ),
      )
      .toList(),
  onChanged: (v) =>
      setLocal(() => role = v ?? UserRole.student),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/users as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminUsersScreen**
> Screenshot /admin/users. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminGroupsScreen

| Field | Value |
|---|---|
| Route | /admin/groups |
| File | lib/screens/admin/groups/admin_groups_screen.dart |
| Role gate | admin |
| Purpose | Groups + membership; chat trigger. |

Source length: 235 lines. Study build() first, then event handlers.

**lib/screens/admin/groups/admin_groups_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/screens/admin/helpers/admin_labels.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AdminGroupsScreen extends ConsumerStatefulWidget {
  const AdminGroupsScreen({super.key});

  @override
  ConsumerState<AdminGroupsScreen> createState() => _AdminGroupsScreenState();
}

class _AdminGroupsScreenState extends ConsumerState<AdminGroupsScreen> {
  Future<void> _createGroup() async {
    final title = TextEditingController();
    String? selectedTeacherId;
    var submitting = false;

    final teachersRes = await supabase
        .from('teachers')
        .select('id, profiles(full_name, username)');
    final teachers = teachersRes as List<dynamic>;

    if (!mounted) return;

    await showDialog<void>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setLocal) {
          final l10n = ctx.l10n;
          return AlertDialog(
            title: Text(l10n.adminCreateGroup),
            content: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                TextField(
                  controller: title,
                  decoration: InputDecoration(
                    labelText: l10n.commonGroup,
                    hintText: l10n.adminGroupTitleHint,
                  ),
                ),
                const SizedBox(height: 16),
                DropdownButtonFormField<String>(
                  initialValue: selectedTeacherId,
                  decoration: InputDecoration(
                    labelText: l10n.adminAssignTeacher,
                  ),
                  items: teachers.map((t) {
                    final p = t['profiles'] as Map;
```

```
            final name =
                p['full_name'] as String? ?? p['username'] as String;
            return DropdownMenuItem(
              value: t['id'] as String,
              child: Text(name),
            );
          }).toList(),
          onChanged: submitting
              ? null
              : (v) => setLocal(() => selectedTeacherId = v),
        ),
      ],
    ),
    actions: [
      TextButton(
        onPressed: submitting ? null : () => Navigator.pop(ctx),
        child: Text(l10n.commonCancel),
      ),
      ElevatedButton(
        onPressed: submitting
            ? null
            : () async {
                final groupTitle = title.text.trim();
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/groups as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminGroupsScreen**
> Screenshot /admin/groups. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminAnalyticsScreen

| Field | Value |
|---|---|
| Route | /admin/analytics |
| File | lib/screens/admin/analytics/admin_analytics_screen.dart |
| Role gate | admin |
| Purpose | School-wide charts. |

Source length: 85 lines. Study build() first, then event handlers.

**lib/screens/admin/analytics/admin_analytics_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:syncfusion_flutter_charts/charts.dart';

class AdminAnalyticsScreen extends ConsumerWidget {
  const AdminAnalyticsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final stats = ref.watch(adminPlatformStatsProvider);
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.adminAnalytics)),
      body: stats.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (s) => RefreshIndicator(
          onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),
          child: ListView(
            padding: EdgeInsets.fromLTRB(
              20,
              20,
              20,
              shellContentBottomPadding(context),
            ),
            children: [
              PremiumSectionHeader(
                title: l10n.adminActivityDistribution,
                subtitle: l10n.adminGlobalMetrics,
              ),
              PremiumCard(
                padding: const EdgeInsets.all(24),
                child: SfCartesianChart(
                  primaryXAxis: const CategoryAxis(),
                  series: <CartesianSeries>[
                    ColumnSeries<_AdminChartData, String>(
                      dataSource: [
                        _AdminChartData(
                          l10n.adminChartAssignments,
                          (s['assignments'] ?? 0).toDouble(),
                        ),
                        _AdminChartData(
                          l10n.adminChartMarks,
                          (s['marks'] ?? 0).toDouble(),
                        ),
                        _AdminChartData(
                          l10n.adminChartStudents,
                          (s['students'] ?? 0).toDouble(),
                        ),
```

```
                      _AdminChartData(
                        l10n.adminChartGroups,
                        (s['groups'] ?? 0).toDouble(),
                      ),
                    ],
                    xValueMapper: (d, _) => d.label,
                    yValueMapper: (d, _) => d.value,
                    color: AppColors.accent(context),
                    borderRadius: const BorderRadius.vertical(
                      top: Radius.circular(8),
                    ),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  ),
),
);
  }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/analytics as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminAnalyticsScreen**
> Screenshot /admin/analytics. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminAcademicYearsScreen

| Field | Value |
|---|---|
| Route | /admin/academic-years |
| File | lib/screens/admin/academic_years/admin_academic_years_screen.dart |
| Role gate | admin |
| Purpose | Years and four terms. |

Source length: 459 lines. Study build() first, then event handlers.

**lib/screens/admin/academic_years/admin_academic_years_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AdminAcademicYearsScreen extends ConsumerStatefulWidget {
  const AdminAcademicYearsScreen({super.key});

  @override
  ConsumerState<AdminAcademicYearsScreen> createState() =>
      _AdminAcademicYearsScreenState();
}

class _AdminAcademicYearsScreenState
    extends ConsumerState<AdminAcademicYearsScreen> {
  bool _busy = false;

  Future<void> _createNext() async {
    final l10n = context.l10n;
    setState(() => _busy = true);
    try {
      final row = await supabase.rpc('admin_create_next_academic_year');
      final label = (row as Map)['label'] as String? ?? '';
      ref.invalidate(academicYearsProvider);
      ref.invalidate(academicTermOptionsProvider);
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(l10n.adminYearCreated(label))));
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(e.toString())));
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  Future<void> _setCurrent(String yearId) async {
    setState(() => _busy = true);
    try {
      await supabase.rpc(
        'admin_set_current_academic_year',
        params: {'p_year_id': yearId},
      );
      ref.invalidate(academicYearsProvider);
      ref.invalidate(academicTermOptionsProvider);
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(
```

```
        context,
      ).showSnackBar(SnackBar(content: Text(e.toString())));
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  String _normalizeYearConfirm(String value) =>
      value.trim().replaceAll(RegExp(r'[-–—■]'), '-');

  Future<void> _deleteYear(AcademicYearRecord year) async {
    final l10n = context.l10n;
    if (year.isCurrent) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(l10n.adminDeleteYearCurrentBlocked)),
      );
      return;
    }

    final confirmController = TextEditingController();
    final expected = _normalizeYearConfirm(year.label);
    var matches = false;
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/academic-years as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminAcademicYearsScreen**
> Screenshot /admin/academic-years. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminAnnouncementsScreen

| Field | Value |
|---|---|
| Route | /admin/announcements |
| File | lib/screens/admin/announcements/admin_announcements_screen.dart |
| Role gate | admin |
| Purpose | Admin posts. |

Source length: 279 lines. Study build() first, then event handlers.

**lib/screens/admin/announcements/admin_announcements_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';

class AdminAnnouncementsScreen extends ConsumerStatefulWidget {
  const AdminAnnouncementsScreen({super.key});

  @override
  ConsumerState<AdminAnnouncementsScreen> createState() =>
      _AdminAnnouncementsScreenState();
}

class _AdminAnnouncementsScreenState
    extends ConsumerState<AdminAnnouncementsScreen> {
  void _refresh() {
    ref.invalidate(adminAnnouncementsProvider);
    ref.invalidate(announcementsProvider);
  }

  Future<void> _compose({Map<String, dynamic>? existing}) async {
    final groups = await ref.read(adminGroupsProvider.future);
    if (groups.isEmpty || !mounted) return;
    final l10n = context.l10n;
    final title = TextEditingController(
      text: existing?['title'] as String? ?? '',
    );
    final body = TextEditingController(
      text: existing?['description'] as String? ?? '',
    );
    var groupId =
        existing?['group_id'] as String? ?? groups.first['id'] as String;

    await showDialog<void>(
      context: context,
      builder: (ctx) => StatefulBuilder(
        builder: (ctx, setStateLocal) => AlertDialog(
          title: Text(
            existing == null ? l10n.announcementsNew : l10n.announcementsEdit,
          ),
          content: SingleChildScrollView(
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                TextField(
                  controller: title,
                  decoration: InputDecoration(labelText: l10n.commonTitle),
                ),
                const SizedBox(height: 16),
                TextField(
```

```
    controller: body,
    decoration: InputDecoration(labelText: l10n.commonMessage),
    maxLines: 4,
  ),
  if (existing == null) ...[
    const SizedBox(height: 16),
    DropdownButtonFormField<String>(
      initialValue: groupId,
      decoration: InputDecoration(
        labelText: l10n.teacherTargetGroup,
      ),
      items: groups
          .map(
            (g) => DropdownMenuItem(
              value: g['id'] as String,
              child: Text(g['title'] as String),
            ),
          )
          .toList(),
      onChanged: (v) =>
          setStateLocal(() => groupId = v ?? groupId),
    ),
  ],
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/announcements as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminAnnouncementsScreen**
> Screenshot /admin/announcements. Annotate one widget and the table behind it. File the screenshot in private notes.

## AdminNotificationsScreen

| Field | Value |
|---|---|
| Route | /admin/notifications |
| File | lib/screens/admin/notifications/admin_notifications_screen.dart |
| Role gate | admin |
| Purpose | Placeholder / empty. |

Source length: 22 lines. Study build() first, then event handlers.

**lib/screens/admin/notifications/admin_notifications_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class AdminNotificationsScreen extends ConsumerWidget {
  const AdminNotificationsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final l10n = context.l10n;
    return Scaffold(
      body: Center(
        child: EmptyState(
          title: l10n.adminActivityLog,
          subtitle: l10n.adminActivityLogSubtitle,
          icon: Icons.history,
        ),
      ),
    );
  }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/notifications as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminNotificationsScreen**
> Screenshot /admin/notifications. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminCredentialPrintScreen

| Field | Value |
|---|---|
| Route | /admin/credential-print |
| File | lib/screens/admin/credentials/admin_credential_print_screen.dart |
| Role gate | admin |
| Purpose | Printable login slips PDF. |

Source length: 221 lines. Study build() first, then event handlers.

**lib/screens/admin/credentials/admin_credential_print_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/core/admin/credential_slips_pdf.dart';
import 'package:mu_study/core/admin/credential_slips_source.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/enums.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:share_plus/share_plus.dart';

class AdminCredentialPrintScreen extends ConsumerStatefulWidget {
  const AdminCredentialPrintScreen({super.key});

  @override
  ConsumerState<AdminCredentialPrintScreen> createState() =>
      _AdminCredentialPrintScreenState();
}

class _AdminCredentialPrintScreenState
    extends ConsumerState<AdminCredentialPrintScreen> {
  UserRole _role = UserRole.student;
  String? _groupId;
  bool _exporting = false;

  Future<void> _export() async {
    if (_exporting) return;
    setState(() => _exporting = true);
    final l10n = context.l10n;
    try {
      final slips = await fetchCredentialSlips(
        role: _role,
        groupId: _role == UserRole.student ? _groupId : null,
      );
      if (slips.isEmpty) {
        throw Exception(l10n.adminCredentialNeedUsers);
      }
      final bytes = buildCredentialSlipsPdf(slips, l10n);
      const fileName = 'mu_study_login_slips.pdf';
      await SharePlus.instance.share(
        ShareParams(
          files: [
            XFile.fromData(bytes, mimeType: 'application/pdf', name: fileName),
          ],
          fileNameOverrides: [fileName],
          subject: l10n.adminCredentialPrintTitle,
          downloadFallbackEnabled: true,
        ),
      );
    } catch (e) {
      if (!mounted) return;
      final message = e is Exception
          ? e.toString().replaceFirst(RegExp(r'^Exception: '), '')
          : '$e';
      ScaffoldMessenger.of(
```

```
      context,
    ).showSnackBar(SnackBar(content: Text(message)));
  } finally {
    if (mounted) setState(() => _exporting = false);
  }
}

@override
Widget build(BuildContext context) {
  final l10n = context.l10n;
  final groups = ref.watch(adminGroupsProvider);

  return Scaffold(
    appBar: AppBar(title: Text(l10n.adminCredentialPrintTitle)),
    body: ListView(
      padding: EdgeInsets.fromLTRB(
        20,
        20,
        20,
        shellContentBottomPadding(context, extra: 24),
      ),
      children: [
        PremiumCard(
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/credential-print as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminCredentialPrintScreen**
> Screenshot /admin/credential-print. Annotate one widget and the table behind it. File the screenshot in private notes.

# AdminMassDataGeneratorScreen

| Field | Value |
|---|---|
| Route | /admin/mass-generator |
| File | lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart |
| Role gate | admin |
| Purpose | admin_mass_generate_school_data. |

Source length: 292 lines. Study build() first, then event handlers.

**lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/models/enums.dart';
import 'package:mu_study/core/admin/credential_slips_source.dart';
import 'package:mu_study/providers/admin/admin_providers.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class AdminMassDataGeneratorScreen extends ConsumerStatefulWidget {
  const AdminMassDataGeneratorScreen({super.key});

  @override
  ConsumerState<AdminMassDataGeneratorScreen> createState() =>
      _AdminMassDataGeneratorScreenState();
}

class _AdminMassDataGeneratorScreenState
    extends ConsumerState<AdminMassDataGeneratorScreen> {
  final _teachersCtrl = TextEditingController();
  final _studentsCtrl = TextEditingController();
  final _groupsCtrl = TextEditingController();
  final _assignmentsCtrl = TextEditingController();

  bool _running = false;

  @override
  void dispose() {
    _teachersCtrl.dispose();
    _studentsCtrl.dispose();
    _groupsCtrl.dispose();
    _assignmentsCtrl.dispose();
    super.dispose();
  }

  static List<String> _splitLines(String raw) => raw
      .split('\n')
      .map((e) => e.trim())
      .where((e) => e.isNotEmpty)
      .toList();

  /// Format: `username|password|full_name|sphere` (sphere: math|english)
  List<Map<String, dynamic>> _parseTeachers() {
    final out = <Map<String, dynamic>>[];
    for (final line in _splitLines(_teachersCtrl.text)) {
      final parts = line.split('|').map((e) => e.trim()).toList();
      if (parts.length != 4) {
        throw ArgumentError(
          'Teacher line must be: username|password|full_name|sphere. Bad: "$line"',
        );
      }

      final sphereRaw = parts[3].toLowerCase();
      final sphere = Sphere.values.firstWhere(
        (s) => s.name == sphereRaw,
```

```
        orElse: () => Sphere.math,
      );

      out.add({
        'username': parts[0],
        'password': parts[1],
        'full_name': parts[2],
        'sphere': sphere.name,
      });
    }
    return out;
  }

  /// Format: `username|password|full_name`
  List<Map<String, dynamic>> _parseStudents() {
    final out = <Map<String, dynamic>>[];
    for (final line in _splitLines(_studentsCtrl.text)) {
      final parts = line.split('|').map((e) => e.trim()).toList();
      if (parts.length != 3) {
        throw ArgumentError(
          'Student line must be: username|password|full_name. Bad: "$line"',
        );
      }
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/mass-generator as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminMassDataGeneratorScreen**
> Screenshot /admin/mass-generator. Annotate one widget and the table behind it. File the screenshot in private notes.

## AdminProfileScreen

| Field | Value |
|---|---|
| Route | /admin/profile |
| File | lib/screens/admin/profile/admin_profile_screen.dart |
| Role gate | admin |
| Purpose | Wrapper → AppProfileScreen. |

Source length: 12 lines. Study build() first, then event handlers.

**lib/screens/admin/profile/admin_profile_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/screens/profile/app_profile_screen.dart';

class AdminProfileScreen extends ConsumerWidget {
  const AdminProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return const AppProfileScreen();
  }
}
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /admin/profile as role admin. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AdminProfileScreen**
> Screenshot /admin/profile. Annotate one widget and the table behind it. File the screenshot in private notes.

## ChatListScreen

| Field | Value |
|---|---|
| Route | /*/chat |
| File | lib/screens/chat/list/chat_list_screen.dart |
| Role gate | all roles |
| Purpose | Rooms + unread. |

Source length: 345 lines. Study build() first, then event handlers.

**lib/screens/chat/list/chat_list_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_animate/flutter_animate.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/layouts/shell_nav_bar.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';

class ChatListScreen extends ConsumerStatefulWidget {
  const ChatListScreen({super.key, required this.basePath});

  final String basePath;

  @override
  ConsumerState<ChatListScreen> createState() => _ChatListScreenState();
}

class _ChatListScreenState extends ConsumerState<ChatListScreen> {
  final _search = TextEditingController();
  String _query = '';

  @override
  void dispose() {
    _search.dispose();
    super.dispose();
  }

  String _preview(ChatRoomSummary room, AppLocalizations l10n) {
    if (room.lastMessageId == null) return l10n.chatNoMessagesYet;
    if (room.lastMessageDeleted) return l10n.chatMessageDeleted;
    return switch (room.lastMessageType) {
      ChatMessageType.image => l10n.chatPreviewImage,
      ChatMessageType.video => l10n.chatPreviewVideo,
      ChatMessageType.voice => l10n.chatPreviewVoice,
      ChatMessageType.file =>
        room.lastMessageText?.trim().isNotEmpty == true
            ? room.lastMessageText!.trim()
            : l10n.chatPreviewFile,
      ChatMessageType.text || null =>
        room.lastMessageText?.trim().isNotEmpty == true
            ? room.lastMessageText!.trim()
            : l10n.chatNoMessagesYet,
    };
  }

  String _formatTime(DateTime? dt) {
    if (dt == null) return '';
    final local = dt.toLocal();
    final now = DateTime.now();
```

```
    if (local.year == now.year &&
        local.month == now.month &&
        local.day == now.day) {
      return DateFormat.Hm().format(local);
    }
    if (now.difference(local).inDays < 7) {
      return DateFormat.E().format(local);
    }
    return DateFormat.MMMd().format(local);
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final roomsAsync = ref.watch(chatRoomsProvider);
    final isNeu = Neumorphic.isActive(context);

    return Scaffold(
      appBar: AppBar(title: Text(l10n.chatTitle)),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 8, 16, 12),
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /*/chat as role all roles. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — ChatListScreen**
> Screenshot /*/chat. Annotate one widget and the table behind it. File the screenshot in private notes.

## ChatRoomScreen

| Field | Value |
|---|---|
| Route | /*/chat/:roomId |
| File | lib/screens/chat/room/chat_room_screen.dart |
| Role gate | all roles |
| Purpose | Messages, media, reactions. |

Source length: 1605 lines. Study build() first, then event handlers.

**lib/screens/chat/room/chat_room_screen.dart (opening)**

```
import 'dart:async';

import 'package:file_picker/file_picker.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:image_picker/image_picker.dart';
import 'package:mu_study/components/chat/chat_media_gallery.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/core/utils/responsive.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';
import 'package:mu_study/screens/chat/room/helpers/chat_list_entries.dart';
import 'package:mu_study/screens/chat/room/widgets/attach_chip.dart';
import 'package:mu_study/screens/chat/room/widgets/chat_date_chip.dart';
import 'package:mu_study/screens/chat/room/widgets/edit_message_dialog.dart';
import 'package:mu_study/screens/chat/room/widgets/message_bubble.dart';
import 'package:mu_study/screens/chat/room/widgets/reply_composer_banner.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:record/record.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class ChatRoomScreen extends ConsumerStatefulWidget {
  const ChatRoomScreen({super.key, required this.roomId});

  final String roomId;

  @override
  ConsumerState<ChatRoomScreen> createState() => _ChatRoomScreenState();
}

class _ChatRoomScreenState extends ConsumerState<ChatRoomScreen>
    with WidgetsBindingObserver {
  static const _nearBottomThreshold = 120.0;

  static const _recentActivityWindow = Duration(minutes: 3);

  final _text = TextEditingController();
  final _scroll = ScrollController();
  final _recorder = AudioRecorder();
  final _focus = FocusNode();

  bool _sending = false;
  bool _attachOpen = false;
  bool _recording = false;
  double _uploadProgress = 0;
  String? _uploadLabel;
  Duration _recordElapsed = Duration.zero;
  Timer? _recordTick;
  Timer? _typingIdle;
```

```
Timer? _presenceHeartbeat;
Timer? _presencePoll;
RealtimeChannel? _typingChannel;
List<String> _typingNames = const [];
final Set<String> _presenceOnlineIds = {};
final Set<String> _recentlyActiveIds = {};
int _onlineCount = 0;
bool _markedRead = false;
bool _presenceSetupInFlight = false;

bool _stickToBottom = true;
bool _showScrollDown = false;
bool _didInitialScroll = false;
String? _lastTailMessageId;
int _lastMessageCount = 0;
bool _loadMoreQueued = false;
Timer? _scrollFabDebounce;
Timer? _highlightTimer;
ChatMessage? _replyTo;
String? _highlightedMessageId;
final _messageKeys = <String, GlobalKey>{};
bool _jumpingToReply = false;
bool _selectionMode = false;
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to /*/chat/:roomId as role all roles. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — ChatRoomScreen**
> Screenshot /*/chat/:roomId. Annotate one widget and the table behind it. File the screenshot in private notes.

# AppProfileScreen

| Field | Value |
|---|---|
| Route | via wrappers |
| File | lib/screens/profile/app_profile_screen.dart |
| Role gate | all |
| Purpose | Theme, language, password, logout. |

Source length: 747 lines. Study build() first, then event handlers.

**lib/screens/profile/app_profile_screen.dart (opening)**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:image_picker/image_picker.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/components/layouts/empty_state.dart';
import 'package:mu_study/components/layouts/premium_section_header.dart';
import 'package:mu_study/components/settings/language_settings_tile.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/progress/progress_report_share.dart';
import 'package:mu_study/core/services/app_cache.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/domain_labels.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/providers/groups/group_providers.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:mu_study/providers/marks/marks_providers.dart';
import 'package:mu_study/screens/profile/widgets/profile_hero_card.dart';
import 'package:mu_study/screens/profile/widgets/profile_settings_row.dart';
import 'package:mu_study/screens/profile/widgets/profile_theme_selector.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:package_info_plus/package_info_plus.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

class AppProfileScreen extends ConsumerWidget {
  const AppProfileScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final profile = ref.watch(currentProfileProvider);
    final l10n = context.l10n;

    return Scaffold(
      appBar: AppBar(title: Text(l10n.profileTitle)),
      body: profile.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => ErrorState(message: e.toString()),
        data: (p) {
          if (p == null) {
            return EmptyState(title: l10n.profileNotSignedIn);
          }
          return _ProfileBody(profile: p);
        },
      ),
    );
  }
}

class _ProfileBody extends ConsumerStatefulWidget {
  const _ProfileBody({required this.profile});

  final Profile profile;
```

```
  @override
  ConsumerState<_ProfileBody> createState() => _ProfileBodyState();
}

class _ProfileBodyState extends ConsumerState<_ProfileBody> {
  String _version = '—';
  bool _uploadingAvatar = false;

  @override
  void initState() {
    super.initState();
    PackageInfo.fromPlatform()
        .then((info) {
          if (mounted) setState(() => _version = info.version);
        })
        .catchError((_) {});
  }

  String get _roleBasePath => switch (widget.profile.role) {
    UserRole.student => '/student',
    UserRole.teacher => '/teacher',
    UserRole.admin => '/admin',
```

The opening shows imports — those are your dependency map. If you see auth_provider, the screen is session-aware. If you see l10n_x, strings are translated. If you see supabase_client directly, prefer moving that call into a provider when you rebuild.

1. Navigate to via wrappers as role all. If redirect sends you away, write which gate fired.
2. Name the provider(s) watched in build().
3. Perform the primary action (submit, save, send, create).
4. Confirm the Table Editor row or Storage object.
5. Repeat as a role that should be denied — you should bounce or see empty data.

> **TASK — Manual QA — AppProfileScreen**
> Screenshot via wrappers. Annotate one widget and the table behind it. File the screenshot in private notes.

# Provider manual — every state module

## lib/providers/admin/admin_providers.dart

Users, groups, analytics, academic years, mass generator. This file (admin providers) is one concrete piece of that layer.

Providers declared: adminPlatformStatsProvider, adminGroupsProvider, adminAnnouncementsProvider, allProfilesProvider.

**lib/providers/admin/admin_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/core/paging/page_result.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';

Future<int> _exactCount(String table) async {
  final result = await supabase
      .from(table)
      .select('id')
      .count(CountOption.exact);
  return result.count;
}

final adminPlatformStatsProvider = FutureProvider<Map<String, int>>((
  ref,
) async {
  final teachers = await _exactCount('teachers');
  final students = await _exactCount('students');
  final groups = await _exactCount('groups');
  final assignments = await _exactCount('assignments');
  final marks = await _exactCount('marks');
  return {
    'teachers': teachers,
    'students': students,
    'groups': groups,
    'assignments': assignments,
    'marks': marks,
  };
});

final adminGroupsProvider = FutureProvider<List<Map<String, dynamic>>>((
  ref,
) async {
  final rows = await supabase
      .from('groups')
      .select('id, title, teachers(sphere, profiles(full_name))')
      .order('title');
  return rows.map((e) => Map<String, dynamic>.from(e)).toList();
});

final adminAnnouncementsProvider = FutureProvider<List<Map<String, dynamic>>>((
  ref,
) async {
  final rows = await supabase
      .from('announcements')
      .select()
      .order('created_at', ascending: false);
  return rows.map((e) => Map<String, dynamic>.from(e)).toList();
});

final allProfilesProvider =
    AsyncNotifierProvider<AllProfilesNotifier, PageResult<Profile>>(
      AllProfilesNotifier.new,
    );
```

```
class AllProfilesNotifier extends AsyncNotifier<PageResult<Profile>> {
  @override
  Future<PageResult<Profile>> build() => _fetch(0);

  Future<PageResult<Profile>> _fetch(int offset) async {
    final limit = PageResult.fetchSize(PageResult.pageSize);
    final rows = await supabase
        .from('profiles')
        .select()
        .order('username')
        .range(offset, offset + limit - 1);
    return PageResult.fromFetch(
      rows.map((e) => Profile.fromJson(Map<String, dynamic>.from(e))).toList(),
    );
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}
```

**TASK — Trace lib/providers/admin/admin_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/attendance/attendance_providers.dart

Lesson sessions + attendance upsert. This file (attendance providers) is one concrete piece of that layer.

Providers declared: attendanceRangeProvider, attendanceRangeStatsProvider, todayAttendanceProvider, attendanceWeekTrendProvider.

**lib/providers/attendance/attendance_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';

String dateKey(DateTime date) => DateTime(
  date.year,
  date.month,
  date.day,
).toIso8601String().split('T').first;

DateTime calendarDay(DateTime date) =>
    DateTime(date.year, date.month, date.day);

bool isAttendanceWeekend(DateTime date) {
  final weekday = calendarDay(date).weekday;
  return weekday == DateTime.saturday || weekday == DateTime.sunday;
}

bool isAttendanceDateAllowed(DateTime date) {
  final day = calendarDay(date);
  final today = calendarDay(DateTime.now());
  if (day.isAfter(today)) return false;
  if (isAttendanceWeekend(day)) return false;
  return true;
}

class AttendanceFutureDateException implements Exception {
  const AttendanceFutureDateException();

  @override
  String toString() => 'AttendanceFutureDateException';
}

class AttendanceWeekendException implements Exception {
  const AttendanceWeekendException();

  @override
  String toString() => 'AttendanceWeekendException';
}

Never _rejectAttendanceDate(DateTime date) {
  final day = calendarDay(date);
  if (isAttendanceWeekend(day)) {
    throw const AttendanceWeekendException();
  }
  final today = calendarDay(DateTime.now());
  if (day.isAfter(today)) {
    throw const AttendanceFutureDateException();
  }
  throw StateError('Attendance date rejected unexpectedly: $day');
}

class AttendanceQuery {
  const AttendanceQuery({
    required this.groupId,
    this.studentId,
    required this.from,
    required this.to,
  });

  final String groupId;
```

```
  final String? studentId;
  final DateTime from;
  final DateTime to;

  @override
  bool operator ==(Object other) =>
      other is AttendanceQuery &&
      other.groupId == groupId &&
      other.studentId == studentId &&
      other.from == from &&
      other.to == to;

  @override
  int get hashCode => Object.hash(groupId, studentId, from, to);
}

final attendanceRangeProvider =
    FutureProvider.family<List<AttendanceRecord>, AttendanceQuery>((
      ref,
      query,
    ) async {
      var filtered = supabase
          .from('attendance')
          .select('*, students(profiles(full_name))')
          .eq('group_id', query.groupId)
          .gte('date', dateKey(query.from))
          .lte('date', dateKey(query.to));

      if (query.studentId != null) {
        filtered = filtered.eq('student_id', query.studentId!);
      }

      final rows = await filtered.order('date', ascending: true);
      return rows
          .map((e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)))
          .toList();
    });

final attendanceRangeStatsProvider =
    Provider.family<AsyncValue<AttendanceStats>, AttendanceQuery>((ref, query) {
      return ref
          .watch(attendanceRangeProvider(query))
          .whenData(AttendanceStats.fromRecords);
    });

final todayAttendanceProvider =
    FutureProvider.family<Map<String, AttendanceStatus>, String>((
      ref,
      groupId,
    ) async {
      final rows = await supabase
          .from('attendance')
          .select('student_id, status')
          .eq('group_id', groupId)
          .eq('date', dateKey(DateTime.now()));
      return {
        for (final row in rows)
          row['student_id'] as String: attendanceFromString(
            row['status'] as String,
          ),
      };
    });

class AttendanceWeekBucket {
  const AttendanceWeekBucket({
    this.present = 0,
    this.absent = 0,
    this.late = 0,
  });

  final int present;
  final int absent;
  final int late;
```

```
  int get max => [present, absent, late].fold(0, (a, b) => a > b ? a : b);
}

class AttendanceWeekTrend {
  const AttendanceWeekTrend({required this.thisWeek, required this.lastWeek});

  final AttendanceWeekBucket thisWeek;
  final AttendanceWeekBucket lastWeek;

  bool get isEmpty => thisWeek.max == 0 && lastWeek.max == 0;
}

final attendanceWeekTrendProvider =
    FutureProvider.family<AttendanceWeekTrend, String>((ref, groupId) async {
      final today = calendarDay(DateTime.now());
      final thisFrom = today.subtract(const Duration(days: 6));
      final lastTo = today.subtract(const Duration(days: 7));
      final lastFrom = today.subtract(const Duration(days: 13));

      final rows = await supabase
          .from('attendance')
          .select('status, date')
          .eq('group_id', groupId)
          .gte('date', dateKey(lastFrom))
          .lte('date', dateKey(today));

      var thisPresent = 0, thisAbsent = 0, thisLate = 0;
      var lastPresent = 0, lastAbsent = 0, lastLate = 0;

      for (final row in rows) {
        final day = calendarDay(DateTime.parse(row['date'] as String));
        final status = row['status'] as String;
        final inThisWeek = !day.isBefore(thisFrom) && !day.isAfter(today);
        final inLastWeek = !day.isBefore(lastFrom) && !day.isAfter(lastTo);

        if (inThisWeek) {
          switch (status) {
            case 'present':
              thisPresent++;
            case 'absent':
              thisAbsent++;
            case 'late':
              thisLate++;
          }
        } else if (inLastWeek) {
          switch (status) {
            case 'present':
              lastPresent++;
            case 'absent':
              lastAbsent++;
            case 'late':
              lastLate++;
          }
        }
      }

      return AttendanceWeekTrend(
        thisWeek: AttendanceWeekBucket(
          present: thisPresent,
          absent: thisAbsent,
          late: thisLate,
        ),
        lastWeek: AttendanceWeekBucket(
          present: lastPresent,
          absent: lastAbsent,
          late: lastLate,
        ),
      );
    });

Future<void> upsertAttendance({
  required String groupId,
  required String studentId,
```

```
    required AttendanceStatus status,
    required DateTime date,
  }) async {
    if (!isAttendanceDateAllowed(date)) {
      _rejectAttendanceDate(date);
    }
    final key = dateKey(date);
    await supabase.rpc(
      'ensure_lesson_session',
      params: {'p_group_id': groupId, 'p_session_date': key},
    );
    await supabase.from('attendance').upsert({
      'group_id': groupId,
      'student_id': studentId,
      'status': status.name,
      'date': key,
    }, onConflict: 'group_id,student_id,date');
  }

  Future<void> upsertAttendanceBulk({
    required String groupId,
    required DateTime date,
    required Map<String, AttendanceStatus> byStudentId,
  }) async {
    if (byStudentId.isEmpty) return;
    if (!isAttendanceDateAllowed(date)) {
      _rejectAttendanceDate(date);
    }
    final key = dateKey(date);
    await supabase.rpc(
      'ensure_lesson_session',
      params: {'p_group_id': groupId, 'p_session_date': key},
    );
    final rows = byStudentId.entries
        .map(
          (e) => {
            'group_id': groupId,
            'student_id': e.key,
            'status': e.value.name,
            'date': key,
          },
        )
        .toList();
    await supabase
        .from('attendance')
        .upsert(rows, onConflict: 'group_id,student_id,date');
  }
```

**TASK — Trace lib/providers/attendance/attendance_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/auth/auth_provider.dart

Session stream, username login RPC, profile/teacher/student loaders. This file (auth provider) is one concrete piece of that layer.

Providers declared: authStateProvider, authRepositoryProvider, profileRepositoryProvider, currentProfileProvider, teacherProfileProvider, studentProfileProvider.

**lib/providers/auth/auth_provider.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/services/app_cache.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

final authStateProvider = StreamProvider<AuthState>((ref) {
  return supabase.auth.onAuthStateChange;
});

class AuthRepository {
  AuthRepository(this._cache);

  final CacheService? _cache;

  Future<String?> resolveAuthEmail(String username) async {
    final normalized = username.trim();
    final result = await supabase.rpc(
      'get_auth_email_by_username',
      params: {'p_username': normalized},
    );
    return result as String?;
  }

  Future<void> signIn(String username, String password) async {
    final email = await resolveAuthEmail(username);
    if (email == null) {
      throw const AuthException('Incorrect username or password');
    }
    await supabase.auth.signInWithPassword(email: email, password: password);
  }

  Future<void> changePassword({
    required String username,
    required String currentPassword,
    required String newPassword,
  }) async {
    await signIn(username, currentPassword);
    await supabase.auth.updateUser(UserAttributes(password: newPassword));
  }

  Future<void> signOut() async {
    await supabase.auth.signOut();
    await clearAllAppCaches(_cache);
  }

  User? get currentUser => supabase.auth.currentUser;
}

final authRepositoryProvider = Provider<AuthRepository>((ref) {
  return AuthRepository(ref.watch(cacheServiceProvider).valueOrNull);
});

class ProfileRepository {
  ProfileRepository(this._cache);

  final CacheService? _cache;

  Future<Profile?> fetchCurrentProfile() async {
    final uid = supabase.auth.currentUser?.id;
```

```
    if (uid == null) return null;

    try {
      final data = await supabase
          .from('profiles')
          .select()
          .eq('id', uid)
          .maybeSingle();
      if (data == null) return null;
      final profile = Profile.fromJson(Map<String, dynamic>.from(data));
      await _cache?.cacheProfile(data);
      return profile;
    } catch (_) {
      final cached = _cache?.cachedProfile;
      if (cached != null) return Profile.fromJson(cached);
      rethrow;
    }
  }

  Future<void> updateProfile(String id, Map<String, dynamic> updates) async {
    await supabase.from('profiles').update(updates).eq('id', id);
  }
}

final profileRepositoryProvider = Provider<ProfileRepository>((ref) {
  final cacheAsync = ref.watch(cacheServiceProvider);
  return ProfileRepository(cacheAsync.valueOrNull);
});

final currentProfileProvider = FutureProvider<Profile?>((ref) async {
  ref.watch(authStateProvider);
  final repo = ref.watch(profileRepositoryProvider);
  return repo.fetchCurrentProfile();
});

final teacherProfileProvider = FutureProvider<TeacherProfile?>((ref) async {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null || profile.role != UserRole.teacher) return null;
  final data = await supabase
      .from('teachers')
      .select()
      .eq('profile_id', profile.id)
      .maybeSingle();
  if (data == null) return null;
  return TeacherProfile.fromJson(Map<String, dynamic>.from(data));
});

final studentProfileProvider = FutureProvider<StudentProfile?>((ref) async {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null || profile.role != UserRole.student) return null;
  final data = await supabase
      .from('students')
      .select()
      .eq('profile_id', profile.id)
      .maybeSingle();
  if (data == null) return null;
  return StudentProfile.fromJson(Map<String, dynamic>.from(data));
});
```

**TASK — Trace lib/providers/auth/auth_provider.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/chat/chat_providers.dart

Room summaries, unread RPC, messages, reactions. This file (chat providers) is one concrete piece of that layer.

Providers declared: chatRoomsProvider, unreadChatCountProvider, chatRoomProvider, chatMessagesProvider.

**lib/providers/chat/chat_providers.dart**

```
import 'dart:async';
import 'dart:typed_data';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/services/app_cache.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/core/services/voice_cache_service.dart';
import 'package:mu_study/core/utils/voice_playback.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const chatSignedUrlTtlSeconds = 60 * 60 * 24 * 365 * 10;
const _pageSize = 40;
const _chatMessageSelect =
    '*, profiles(full_name, avatar_url, username, role), message_reads(user_id),
message_reactions(id, message_id, user_id, emoji), reply_to:reply_to_message_id(id,
message_type, text_content, media_url, is_deleted, sender_id, profiles(full_name, username,
avatar_url))';

const chatReactionEmojis = ['❤■', '■', '■', '■', '■', '■'];

String chatBucketForType(ChatMessageType type) => switch (type) {
  ChatMessageType.image => 'chat-images',
  ChatMessageType.video => 'chat-videos',
  ChatMessageType.voice => 'chat-voices',
  ChatMessageType.file => 'chat-files',
  ChatMessageType.text => 'chat-images',
};

String chatContentType(ChatMessageType type, String fileName) {
  final lower = fileName.toLowerCase();
  return switch (type) {
    ChatMessageType.image =>
      lower.endsWith('.png')
          ? 'image/png'
          : lower.endsWith('.webp')
          ? 'image/webp'
          : lower.endsWith('.gif')
          ? 'image/gif'
          : 'image/jpeg',
    ChatMessageType.video =>
      lower.endsWith('.webm')
          ? 'video/webm'
          : lower.endsWith('.mov')
          ? 'video/quicktime'
          : 'video/mp4',
    ChatMessageType.voice =>
      lower.endsWith('.wav')
          ? 'audio/wav'
          : lower.endsWith('.mp3')
          ? 'audio/mpeg'
          : 'audio/mp4',
    ChatMessageType.file => _fileContentType(lower),
    ChatMessageType.text => 'text/plain',
  };
}

String _fileContentType(String lowerName) {
```

```
  if (lowerName.endsWith('.pdf')) return 'application/pdf';
  if (lowerName.endsWith('.zip')) return 'application/zip';
  if (lowerName.endsWith('.json')) return 'application/json';
  if (lowerName.endsWith('.csv')) return 'text/csv';
  if (lowerName.endsWith('.txt') ||
      lowerName.endsWith('.md') ||
      lowerName.endsWith('.py') ||
      lowerName.endsWith('.dart') ||
      lowerName.endsWith('.js') ||
      lowerName.endsWith('.ts') ||
      lowerName.endsWith('.html') ||
      lowerName.endsWith('.css') ||
      lowerName.endsWith('.sql')) {
    return 'text/plain';
  }
  if (lowerName.endsWith('.doc') || lowerName.endsWith('.docx')) {
    return 'application/msword';
  }
  if (lowerName.endsWith('.xls') || lowerName.endsWith('.xlsx')) {
    return 'application/vnd.ms-excel';
  }
  if (lowerName.endsWith('.ppt') || lowerName.endsWith('.pptx')) {
    return 'application/vnd.ms-powerpoint';
  }
  return 'application/octet-stream';
}

String chatVoiceContentType(Uint8List bytes, String fileName) {
  return detectAudioFormat(bytes).mimeType;
}

String chatVoiceFileName(Uint8List bytes, String fallbackName) {
  final format = detectAudioFormat(bytes);
  final stamp = DateTime.now().millisecondsSinceEpoch;
  final base = fallbackName.replaceAll(RegExp(r'[^\w.\-]+'), '_');
  if (base.toLowerCase().endsWith('.${format.extension}')) return base;
  final withoutExt = base.contains('.')
      ? base.substring(0, base.lastIndexOf('.'))
      : base;
  final safe = withoutExt.isEmpty ? 'chat_voice_$stamp' : withoutExt;
  return '$safe.${format.extension}';
}

final chatRoomsProvider = StreamProvider<List<ChatRoomSummary>>((ref) async* {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null) {
    yield const [];
    return;
  }

  Future<List<ChatRoomSummary>> load() async {
    final rows = await supabase
        .from('chat_room_summaries')
        .select()
        .order('last_message_at', ascending: false, nullsFirst: false);

    final unreadRows =
        await supabase.rpc('chat_unread_counts') as List<dynamic>? ?? [];
    final unreadMap = <String, int>{};
    for (final row in unreadRows) {
      final map = Map<String, dynamic>.from(row as Map);
      final roomId = map['room_id'] as String?;
      if (roomId == null) continue;
      unreadMap[roomId] = (map['unread_count'] as num?)?.toInt() ?? 0;
    }

    final rooms = (rows as List)
        .map((e) => ChatRoomSummary.fromJson(Map<String, dynamic>.from(e)))
        .map((r) => r.copyWith(unreadCount: unreadMap[r.id] ?? 0))
        .toList();

    rooms.sort((a, b) {
      final aTime =
```

```
          a.lastMessageAt ??
          a.createdAt ??
          DateTime.fromMillisecondsSinceEpoch(0);
      final bTime =
          b.lastMessageAt ??
          b.createdAt ??
          DateTime.fromMillisecondsSinceEpoch(0);
      return bTime.compareTo(aTime);
    });
    return rooms;
  }

  final changes = StreamController<void>();
  final channel = supabase
      .channel('chat-rooms-${profile.id}')
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'chat_messages',
        callback: (_) {
          if (!changes.isClosed) changes.add(null);
        },
      )
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'message_reads',
        callback: (_) {
          if (!changes.isClosed) changes.add(null);
        },
      )
      .subscribe();

  ref.onDispose(() {
    supabase.removeChannel(channel);
    changes.close();
  });

  yield await load();
  await for (final _ in changes.stream) {
    yield await load();
  }
});

final unreadChatCountProvider = Provider<int>((ref) {
  final rooms = ref.watch(chatRoomsProvider);
  return rooms.maybeWhen(
    data: (list) => list.fold<int>(0, (sum, r) => sum + r.unreadCount),
    orElse: () => 0,
  );
});

final chatRoomProvider = FutureProvider.family<ChatRoomSummary?, String>((
  ref,
  roomId,
) async {
  final rooms = ref.watch(chatRoomsProvider).valueOrNull;
  if (rooms != null) {
    for (final room in rooms) {
      if (room.id == roomId) return room;
    }
  }
  final row = await supabase
      .from('chat_room_summaries')
      .select()
      .eq('id', roomId)
      .maybeSingle();
  if (row == null) return null;
  return ChatRoomSummary.fromJson(Map<String, dynamic>.from(row));
});

class ChatMessagesState {
  const ChatMessagesState({
```

```
    required this.messages,
    this.hasMore = false,
    this.loadingMore = false,
  });

  final List<ChatMessage> messages;
  final bool hasMore;
  final bool loadingMore;

  ChatMessagesState copyWith({
    List<ChatMessage>? messages,
    bool? hasMore,
    bool? loadingMore,
  }) {
    return ChatMessagesState(
      messages: messages ?? this.messages,
      hasMore: hasMore ?? this.hasMore,
      loadingMore: loadingMore ?? this.loadingMore,
    );
  }
}

class ChatMessagesNotifier
    extends FamilyAsyncNotifier<ChatMessagesState, String> {
  RealtimeChannel? _channel;
  final _profileCache = <String, Map<String, dynamic>>{};

  @override
  Future<ChatMessagesState> build(String roomId) async {
    final profile = await ref.watch(currentProfileProvider.future);
    if (profile == null) {
      return const ChatMessagesState(messages: []);
    }

    final cache = ref.read(cacheServiceProvider).valueOrNull;
    List<ChatMessage>? cachedMessages;
    try {
      final raw = cache?.cachedChatMessages(roomId);
      if (raw != null && raw.isNotEmpty) {
        cachedMessages = raw.map(ChatMessage.fromJson).toList();
      }
    } catch (_) {}

    try {
      final initial = await _fetchPage(roomId, before: null);
      await _persistMessages(roomId, initial.items);
      prefetchChatMedia(initial.items);

      _channel = supabase
          .channel('chat-room-$roomId')
          .onPostgresChanges(
            event: PostgresChangeEvent.insert,
            schema: 'public',
            table: 'chat_messages',
            filter: PostgresChangeFilter(
              type: PostgresChangeFilterType.eq,
              column: 'room_id',
              value: roomId,
            ),
            callback: (payload) => _onInsert(payload.newRecord),
          )
          .onPostgresChanges(
            event: PostgresChangeEvent.update,
            schema: 'public',
            table: 'chat_messages',
            filter: PostgresChangeFilter(
              type: PostgresChangeFilterType.eq,
              column: 'room_id',
              value: roomId,
            ),
            callback: (payload) => _onUpdate(payload.newRecord),
          )
          .onPostgresChanges(
```

```
            event: PostgresChangeEvent.delete,
            schema: 'public',
            table: 'chat_messages',
            filter: PostgresChangeFilter(
              type: PostgresChangeFilterType.eq,
              column: 'room_id',
              value: roomId,
            ),
            callback: (payload) => _onDelete(payload.oldRecord),
          )
          .onPostgresChanges(
            event: PostgresChangeEvent.insert,
            schema: 'public',
            table: 'message_reads',
            callback: (payload) => _onReadInsert(payload.newRecord),
          )
          .onPostgresChanges(
            event: PostgresChangeEvent.insert,
            schema: 'public',
            table: 'message_reactions',
            callback: (payload) => _onReactionUpsert(payload.newRecord),
          )
          .onPostgresChanges(
            event: PostgresChangeEvent.update,
            schema: 'public',
            table: 'message_reactions',
            callback: (payload) => _onReactionUpsert(payload.newRecord),
          )
          .onPostgresChanges(
            event: PostgresChangeEvent.delete,
            schema: 'public',
            table: 'message_reactions',
            callback: (payload) => _onReactionDelete(payload.oldRecord),
          )
          .subscribe();

      ref.onDispose(() {
        final ch = _channel;
        if (ch != null) supabase.removeChannel(ch);
      });

      return ChatMessagesState(
        messages: initial.items,
        hasMore: initial.hasMore,
      );
    } catch (_) {
      if (cachedMessages != null) {
        prefetchChatMedia(cachedMessages);
        return ChatMessagesState(messages: cachedMessages, hasMore: true);
      }
      rethrow;
    }
  }

  Future<void> _persistMessages(
    String roomId,
    List<ChatMessage> messages,
  ) async {
    final cache = ref.read(cacheServiceProvider).valueOrNull;
    if (cache == null) return;
    try {
      await cache.cacheChatMessages(
        roomId,
        messages.map((m) => m.toCacheJson()).toList(),
      );
    } catch (_) {}
  }

  Future<void> _persistCurrent() async {
    final current = state.valueOrNull;
    if (current == null) return;
    await _persistMessages(arg, current.messages);
  }
```

```
Future<({List<ChatMessage> items, bool hasMore})> _fetchPage(
  String roomId, {
  required DateTime? before,
}) async {
  var filtered = supabase
      .from('chat_messages')
      .select(_chatMessageSelect)
      .eq('room_id', roomId);

  if (before != null) {
    filtered = filtered.lt('created_at', before.toIso8601String());
  }

  final rows = await filtered
      .order('created_at', ascending: false)
      .limit(_pageSize + 1);
  final list = (rows as List)
      .map((e) => ChatMessage.fromJson(Map<String, dynamic>.from(e)))
      .toList();
  final hasMore = list.length > _pageSize;
  final page = hasMore ? list.sublist(0, _pageSize) : list;
  return (items: page.reversed.toList(), hasMore: hasMore);
}

Future<void> loadMore() async {
  final current = state.valueOrNull;
  if (current == null || !current.hasMore || current.loadingMore) return;
  if (current.messages.isEmpty) return;

  state = AsyncData(current.copyWith(loadingMore: true));
  try {
    final oldest = current.messages.first.createdAt;
    final page = await _fetchPage(arg, before: oldest);
    final merged = [...page.items, ...current.messages];
    state = AsyncData(
      ChatMessagesState(
        messages: merged,
        hasMore: page.hasMore,
        loadingMore: false,
      ),
    );
    await _persistMessages(arg, merged);
    prefetchChatMedia(page.items);
  } catch (_) {
    state = AsyncData(current.copyWith(loadingMore: false));
  }
}

Future<void> _hydrateSender(Map<String, dynamic> row) async {
  final senderId = row['sender_id'] as String?;
  if (senderId == null || row['profiles'] != null) return;
  final cached = _profileCache[senderId];
  if (cached != null) {
    row['profiles'] = cached;
    return;
  }
  final profileRow = await supabase
      .from('profiles')
      .select('full_name, avatar_url, username, role')
      .eq('id', senderId)
      .maybeSingle();
  if (profileRow != null) {
    final map = Map<String, dynamic>.from(profileRow);
    _profileCache[senderId] = map;
    row['profiles'] = map;
  }
}

Future<void> _hydrateReply(Map<String, dynamic> row) async {
  final replyId = row['reply_to_message_id'] as String?;
  if (replyId == null || row['reply_to'] != null) return;

  final current = state.valueOrNull;
```

```
    final local = current?.messages.where((m) => m.id == replyId).firstOrNull;
    if (local != null) {
      row['reply_to'] = ChatReplyPreview.fromMessage(local).toCacheJson();
      return;
    }

    try {
      final data = await supabase
          .from('chat_messages')
          .select(
            'id, message_type, text_content, media_url, is_deleted, sender_id, profiles(
full_name, username, avatar_url)',
          )
          .eq('id', replyId)
          .maybeSingle();
```

**lib/providers/chat/chat_providers.dart — continued (2/2)**

```
      if (data != null) {
        row['reply_to'] = Map<String, dynamic>.from(data);
      }
    } catch (_) {}
  }

  Future<void> _onInsert(Map<String, dynamic> record) async {
    final row = Map<String, dynamic>.from(record);
    await _hydrateSender(row);
    await _hydrateReply(row);
    final message = ChatMessage.fromJson(row);
    final current = state.valueOrNull;
    if (current == null) return;
    if (current.messages.any((m) => m.id == message.id)) return;
    state = AsyncData(
      current.copyWith(messages: [...current.messages, message]),
    );
    prefetchChatMedia([message]);
    await _persistCurrent();
  }

  Future<void> _onUpdate(Map<String, dynamic> record) async {
    final row = Map<String, dynamic>.from(record);
    await _hydrateSender(row);
    final updated = ChatMessage.fromJson(row);
    final current = state.valueOrNull;
    if (current == null) return;
    final next = current.messages.map((m) {
      if (m.id != updated.id) return m;
      return updated.copyWith(
        senderName: updated.senderName ?? m.senderName,
        senderUsername: updated.senderUsername ?? m.senderUsername,
        senderAvatarUrl: updated.senderAvatarUrl ?? m.senderAvatarUrl,
        senderRole: updated.senderRole ?? m.senderRole,
        isReadByOthers: m.isReadByOthers || updated.isReadByOthers,
        reactions: updated.reactions.isEmpty ? m.reactions : updated.reactions,
        replyToMessageId: updated.replyToMessageId ?? m.replyToMessageId,
        replyTo: updated.replyTo ?? m.replyTo,
      );
    }).toList();
    state = AsyncData(current.copyWith(messages: next));
    await _persistCurrent();
  }

  void _onDelete(Map<String, dynamic> record) {
    final id = record['id'] as String?;
    if (id == null) return;
    final current = state.valueOrNull;
    if (current == null) return;
    state = AsyncData(
      current.copyWith(
        messages: current.messages.where((m) => m.id != id).toList(),
      ),
    );
    unawaited(_persistCurrent());
```

```
}

void _onReadInsert(Map<String, dynamic> record) {
  final messageId = record['message_id'] as String?;
  final readerId = record['user_id'] as String?;
  if (messageId == null || readerId == null) return;
  final current = state.valueOrNull;
  if (current == null) return;

  var changed = false;
  final next = current.messages.map((m) {
    if (m.id != messageId || m.isReadByOthers) return m;
    if (readerId == m.senderId) return m;
    changed = true;
    return m.copyWith(isReadByOthers: true);
  }).toList();

  if (changed) {
    state = AsyncData(current.copyWith(messages: next));
  }
}

void _onReactionUpsert(Map<String, dynamic> record) {
  final reaction = ChatReaction.fromJson(Map<String, dynamic>.from(record));
  final current = state.valueOrNull;
  if (current == null) return;
  if (!current.messages.any((m) => m.id == reaction.messageId)) return;

  final next = current.messages.map((m) {
    if (m.id != reaction.messageId) return m;
    final withoutUser = m.reactions
        .where((r) => r.userId != reaction.userId)
        .toList();
    return m.copyWith(reactions: [...withoutUser, reaction]);
  }).toList();
  state = AsyncData(current.copyWith(messages: next));
}

void _onReactionDelete(Map<String, dynamic> record) {
  final id = record['id'] as String?;
  final messageId = record['message_id'] as String?;
  final userId = record['user_id'] as String?;
  final current = state.valueOrNull;
  if (current == null) return;

  final next = current.messages.map((m) {
    if (messageId != null && m.id != messageId) return m;
    if (messageId == null &&
        id != null &&
        !m.reactions.any((r) => r.id == id)) {
      return m;
    }
    return m.copyWith(
      reactions: m.reactions.where((r) {
        if (id != null && r.id == id) return false;
        if (userId != null &&
            messageId != null &&
            r.userId == userId &&
            r.messageId == messageId) {
          return false;
        }
        return true;
      }).toList(),
    );
  }).toList();
  state = AsyncData(current.copyWith(messages: next));
}

Future<ChatMessage> sendText(String text, {String? replyToMessageId}) async {
  final profile = await ref.read(currentProfileProvider.future);
  if (profile == null) throw StateError('Not signed in');
  final trimmed = text.trim();
  if (trimmed.isEmpty) throw ArgumentError('Empty message');
```

```
    final row = await supabase
        .from('chat_messages')
        .insert({
          'room_id': arg,
          'sender_id': profile.id,
          'message_type': 'text',
          'text_content': trimmed,
          'reply_to_message_id': ?replyToMessageId,
        })
        .select(_chatMessageSelect)
        .single();

    final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));
    final current = state.valueOrNull;
    if (current != null && !current.messages.any((m) => m.id == message.id)) {
      state = AsyncData(
        current.copyWith(messages: [...current.messages, message]),
      );
      await _persistCurrent();
    }
    return message;
  }

  Future<ChatMessage> sendMedia({
    required ChatMessageType type,
    required Uint8List bytes,
    required String fileName,
    String? caption,
    String? replyToMessageId,
    void Function(double progress)? onProgress,
  }) async {
    final profile = await ref.read(currentProfileProvider.future);
    if (profile == null) throw StateError('Not signed in');
    if (type == ChatMessageType.text) {
      throw ArgumentError('Use sendText for text messages');
    }

    onProgress?.call(0.05);
    final stamp = DateTime.now().millisecondsSinceEpoch;
    final safeName = type == ChatMessageType.voice
        ? chatVoiceFileName(bytes, fileName)
        : fileName.replaceAll(RegExp(r'[^\w.\-]+'), '_');
    final path = '$arg/${profile.id}/${stamp}_$safeName';
    final bucket = chatBucketForType(type);
    final contentType = type == ChatMessageType.voice
        ? chatVoiceContentType(bytes, fileName)
        : chatContentType(type, fileName);

    await supabase.storage
        .from(bucket)
        .uploadBinary(
          path,
          bytes,
          fileOptions: FileOptions(upsert: false, contentType: contentType),
        );
    onProgress?.call(0.7);

    final url = await supabase.storage
        .from(bucket)
        .createSignedUrl(path, chatSignedUrlTtlSeconds);
    onProgress?.call(0.85);

    if (type == ChatMessageType.voice) {
      final format = detectAudioFormat(bytes);
      await VoiceCacheService.instance.put(url, bytes, format.extension);
    } else {
      final ext = ChatMediaCacheService.extensionForUrl(
        url,
        fallback: safeName.contains('.') ? safeName.split('.').last : 'bin',
      );
      await ChatMediaCacheService.instance.put(url, bytes, ext);
    }
```

```
  final captionText = caption?.trim();
  final row = await supabase
      .from('chat_messages')
      .insert({
        'room_id': arg,
        'sender_id': profile.id,
        'message_type': chatMessageTypeToDb(type),
        'text_content': (captionText == null || captionText.isEmpty)
            ? null
            : captionText,
        'media_url': url,
        'reply_to_message_id': ?replyToMessageId,
      })
      .select(_chatMessageSelect)
      .single();
  onProgress?.call(1);

  final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));
  final current = state.valueOrNull;
  if (current != null && !current.messages.any((m) => m.id == message.id)) {
    state = AsyncData(
      current.copyWith(messages: [...current.messages, message]),
    );
    await _persistCurrent();
  }
  return message;
}

Future<void> editMessage(String messageId, String newText) async {
  final trimmed = newText.trim();
  if (trimmed.isEmpty) throw ArgumentError('Empty message');
  await supabase
      .from('chat_messages')
      .update({'text_content': trimmed})
      .eq('id', messageId);
}

Future<void> deleteMessage(String messageId) async {
  await deleteMessages([messageId]);
}

Future<void> deleteMessages(Iterable<String> messageIds) async {
  final ids = messageIds.toList();
  if (ids.isEmpty) return;

  final current = state.valueOrNull;
  if (current != null) {
    final idSet = ids.toSet();
    state = AsyncData(
      current.copyWith(
        messages: [
          for (final m in current.messages)
            if (idSet.contains(m.id))
              m.copyWith(isDeleted: true)
            else
              m,
        ],
      ),
    );
    await _persistCurrent();
  }

  await supabase.from('chat_messages').update({
    'is_deleted': true,
    'text_content': null,
    'media_url': null,
  }).inFilter('id', ids);
}

Future<void> toggleReaction(String messageId, String emoji) async {
  final profile = await ref.read(currentProfileProvider.future);
  if (profile == null) throw StateError('Not signed in');
```

```
    final current = state.valueOrNull;
    ChatMessage? message;
    if (current != null) {
      for (final m in current.messages) {
        if (m.id == messageId) {
          message = m;
          break;
        }
      }
    }
    final existing = message?.myReactionEmoji(profile.id);

    if (current != null && message != null) {
      final nextReactions = <ChatReaction>[
        for (final r in message.reactions)
          if (r.userId != profile.id) r,
      ];
      if (existing != emoji) {
        nextReactions.add(
          ChatReaction(
            id: 'local-${profile.id}',
            messageId: messageId,
            userId: profile.id,
            emoji: emoji,
          ),
        );
      }
      state = AsyncData(
        current.copyWith(
          messages: [
            for (final m in current.messages)
              if (m.id == messageId)
                m.copyWith(reactions: nextReactions)
              else
                m,
          ],
        ),
      );
    }

    await supabase.rpc(
      'toggle_message_reaction',
      params: {'p_message_id': messageId, 'p_emoji': emoji},
    );
  }

  Future<void> markRoomRead() async {
    final profile = await ref.read(currentProfileProvider.future);
    if (profile == null) return;

    final unread = await supabase
        .from('chat_messages')
        .select('id')
        .eq('room_id', arg)
        .eq('is_deleted', false)
        .neq('sender_id', profile.id);

    final ids = (unread as List)
        .map((e) => (e as Map)['id'] as String)
        .toList();
    if (ids.isEmpty) return;

    final existing = await supabase
        .from('message_reads')
        .select('message_id')
        .eq('user_id', profile.id)
        .inFilter('message_id', ids);
    final already = {
      for (final e in existing as List) (e as Map)['message_id'] as String,
    };
    final toInsert = ids
        .where((id) => !already.contains(id))
        .map((id) => {'message_id': id, 'user_id': profile.id})
```

```
        .toList();
    if (toInsert.isEmpty) return;

    await supabase
        .from('message_reads')
        .upsert(toInsert, onConflict: 'message_id,user_id');
    ref.invalidate(chatRoomsProvider);
  }
}

final chatMessagesProvider =
    AsyncNotifierProvider.family<
      ChatMessagesNotifier,
      ChatMessagesState,
      String
    >(ChatMessagesNotifier.new);
```

**TASK — Trace lib/providers/chat/chat_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/focus/focus_session_provider.dart

Local Pomodoro/free timer persisted in SharedPreferences. This file (focus session provider) is one concrete piece of that layer.

Providers declared: focusSessionProvider, focusChatMutedProvider.

**lib/providers/focus/focus_session_provider.dart**

```
import 'dart:async';
import 'dart:convert';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/focus/focus_time_up_cue.dart';
import 'package:shared_preferences/shared_preferences.dart';

const _prefsKey = 'focus_session_v1';

enum FocusTimerMode { pomodoro, free }

const kFocusPomodoroDuration = Duration(minutes: 25);

class FocusSession {
  const FocusSession({
    required this.assignmentId,
    required this.assignmentTitle,
    required this.mode,
    required this.startedAt,
    this.accumulated = Duration.zero,
    this.pausedAt,
  });

  final String assignmentId;
  final String assignmentTitle;
  final FocusTimerMode mode;
  final DateTime startedAt;

  final Duration accumulated;
  final DateTime? pausedAt;

  bool get isPaused => pausedAt != null;

  Duration elapsedAt([DateTime? now]) {
    final n = now ?? DateTime.now();
    if (isPaused) return accumulated;
    return accumulated + n.difference(startedAt);
  }

  Duration? remainingAt([DateTime? now]) {
    if (mode != FocusTimerMode.pomodoro) return null;
    final left = kFocusPomodoroDuration - elapsedAt(now);
    return left.isNegative ? Duration.zero : left;
  }

  bool isCompleteAt([DateTime? now]) {
    final left = remainingAt(now);
    return left != null && left == Duration.zero;
  }

  Map<String, dynamic> toJson() => {
    'assignmentId': assignmentId,
    'assignmentTitle': assignmentTitle,
    'mode': mode.name,
    'startedAt': startedAt.toIso8601String(),
    'accumulatedMs': accumulated.inMilliseconds,
    'pausedAt': pausedAt?.toIso8601String(),
  };

  factory FocusSession.fromJson(Map<String, dynamic> json) {
    return FocusSession(
      assignmentId: json['assignmentId'] as String,
      assignmentTitle: json['assignmentTitle'] as String? ?? '',
```

```
      mode: FocusTimerMode.values.firstWhere(
        (m) => m.name == json['mode'],
        orElse: () => FocusTimerMode.free,
      ),
      startedAt: DateTime.parse(json['startedAt'] as String),
      accumulated: Duration(milliseconds: json['accumulatedMs'] as int? ?? 0),
      pausedAt: json['pausedAt'] != null
          ? DateTime.parse(json['pausedAt'] as String)
          : null,
    );
  }

  FocusSession copyWith({
    DateTime? startedAt,
    Duration? accumulated,
    DateTime? pausedAt,
    bool clearPausedAt = false,
  }) {
    return FocusSession(
      assignmentId: assignmentId,
      assignmentTitle: assignmentTitle,
      mode: mode,
      startedAt: startedAt ?? this.startedAt,
      accumulated: accumulated ?? this.accumulated,
      pausedAt: clearPausedAt ? null : (pausedAt ?? this.pausedAt),
    );
  }
}

class FocusSummary {
  const FocusSummary({
    required this.assignmentId,
    required this.assignmentTitle,
    required this.focused,
    required this.mode,
    this.note,
    this.timedOut = false,
  });

  final String assignmentId;
  final String assignmentTitle;
  final Duration focused;
  final FocusTimerMode mode;
  final String? note;

  final bool timedOut;

  FocusSummary copyWith({String? note}) => FocusSummary(
    assignmentId: assignmentId,
    assignmentTitle: assignmentTitle,
    focused: focused,
    mode: mode,
    note: note ?? this.note,
    timedOut: timedOut,
  );
}

class FocusSessionState {
  const FocusSessionState({this.session, this.summary});

  final FocusSession? session;
  final FocusSummary? summary;

  bool get isActive => session != null;
  bool get showingSummary => summary != null && session == null;

  FocusSessionState copyWith({
    FocusSession? session,
    FocusSummary? summary,
    bool clearSession = false,
    bool clearSummary = false,
  }) {
    return FocusSessionState(
```

```
      session: clearSession ? null : (session ?? this.session),
      summary: clearSummary ? null : (summary ?? this.summary),
    );
  }
}

final focusSessionProvider =
    StateNotifierProvider<FocusSessionController, FocusSessionState>((ref) {
      return FocusSessionController();
    });

final focusChatMutedProvider = Provider<bool>((ref) {
  return ref.watch(focusSessionProvider).isActive;
});

class FocusSessionController extends StateNotifier<FocusSessionState> {
  FocusSessionController() : super(const FocusSessionState()) {
    _restore();
  }

  Future<void> _restore() async {
    final prefs = await SharedPreferences.getInstance();
    final raw = prefs.getString(_prefsKey);
    if (raw == null || raw.isEmpty) return;
    try {
      final map = jsonDecode(raw) as Map<String, dynamic>;
      final session = FocusSession.fromJson(map);
      if (!mounted) return;
      if (session.isCompleteAt()) {
        await endSession(timedOut: true, playCue: false);
        return;
      }
      state = FocusSessionState(session: session);
    } catch (_) {
      await prefs.remove(_prefsKey);
    }
  }

  Future<void> _persist(FocusSession? session) async {
    final prefs = await SharedPreferences.getInstance();
    if (session == null) {
      await prefs.remove(_prefsKey);
    } else {
      await prefs.setString(_prefsKey, jsonEncode(session.toJson()));
    }
  }

  Future<void> start({
    required String assignmentId,
    required String assignmentTitle,
    required FocusTimerMode mode,
  }) async {
    final session = FocusSession(
      assignmentId: assignmentId,
      assignmentTitle: assignmentTitle,
      mode: mode,
      startedAt: DateTime.now(),
    );
    state = FocusSessionState(session: session);
    await _persist(session);
  }

  Future<void> pause() async {
    final s = state.session;
    if (s == null || s.isPaused) return;
    final now = DateTime.now();
    final next = s.copyWith(accumulated: s.elapsedAt(now), pausedAt: now);
    state = state.copyWith(session: next);
    await _persist(next);
  }

  Future<void> resume() async {
    final s = state.session;
```

```
    if (s == null || !s.isPaused) return;
    final next = s.copyWith(startedAt: DateTime.now(), clearPausedAt: true);
    state = state.copyWith(session: next);
    await _persist(next);
  }

  Future<void> endSession({bool timedOut = false, bool playCue = true}) async {
    final s = state.session;
    if (s == null) return;
    final focused = timedOut && s.mode == FocusTimerMode.pomodoro
        ? kFocusPomodoroDuration
        : s.elapsedAt();
    final summary = FocusSummary(
      assignmentId: s.assignmentId,
      assignmentTitle: s.assignmentTitle,
      focused: focused,
      mode: s.mode,
      timedOut: timedOut,
    );
    state = FocusSessionState(summary: summary);
    await _persist(null);
    if (timedOut && playCue) {
      await playFocusTimeUpCue();
    }
  }

  void setSummaryNote(String note) {
    final summary = state.summary;
    if (summary == null) return;
    state = state.copyWith(summary: summary.copyWith(note: note));
  }

  void dismissSummary() {
    state = const FocusSessionState();
  }
}
```

**TASK — Trace lib/providers/focus/focus_session_provider.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/groups/group_providers.dart

Teacher-owned and student-membership groups. This file (group providers) is one concrete piece of that layer.

Providers declared: myGroupsProvider, selectedGroupIdProvider, activeGroupIdProvider, groupMembersProvider.

**lib/providers/groups/group_providers.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final myGroupsProvider = FutureProvider<List<StudyGroup>>((ref) async {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null) return [];

  if (profile.role == UserRole.teacher) {
    final teacher = await ref.watch(teacherProfileProvider.future);
    if (teacher == null) return [];
    final rows = await supabase
        .from('groups')
        .select()
        .eq('teacher_id', teacher.id)
        .order('title');
    return rows
        .map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))
        .toList();
  }

  if (profile.role == UserRole.student) {
    final student = await ref.watch(studentProfileProvider.future);
    if (student == null) return [];
    final rows = await supabase
        .from('group_students')
        .select('groups(*)')
        .eq('student_id', student.id);
    return rows
        .map(
          (e) => StudyGroup.fromJson(
            Map<String, dynamic>.from(e['groups'] as Map),
          ),
        )
        .toList();
  }

  final rows = await supabase.from('groups').select().order('title');
  return rows
      .map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))
      .toList();
});

final selectedGroupIdProvider = StateProvider<String?>((ref) => null);

final activeGroupIdProvider = Provider<String?>((ref) {
  final selected = ref.watch(selectedGroupIdProvider);
  final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];
  if (selected != null && groups.any((g) => g.id == selected)) return selected;
  return groups.isEmpty ? null : groups.first.id;
});

final groupMembersProvider = FutureProvider.family<List<GroupMember>, String>((
  ref,
  groupId,
) async {
  final rows = await supabase
      .from('group_students')
      .select('students(id, profiles(full_name, username))')
      .eq('group_id', groupId);
```

```
  final members = rows
      .map((e) => GroupMember.fromGroupStudentRow(Map<String, dynamic>.from(e)))
      .toList();
  members.sort(
    (a, b) => a.fullName.toLowerCase().compareTo(b.fullName.toLowerCase()),
  );
  return members;
});
```

**TASK — Trace lib/providers/groups/group_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/homework/assignment_providers.dart

Assignments, tasks, teacher submissions queue. This file (assignment providers) is one concrete piece of that layer.

Providers declared: assignmentsProvider, assignmentDetailProvider, tasksProvider, assignmentProgressProvider.

**lib/providers/homework/assignment_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/paging/page_result.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final assignmentsProvider =
    AsyncNotifierProvider.family<
      AssignmentsNotifier,
      PageResult<Assignment>,
      String?
    >(AssignmentsNotifier.new);

class AssignmentsNotifier
    extends FamilyAsyncNotifier<PageResult<Assignment>, String?> {
  @override
  Future<PageResult<Assignment>> build(String? groupId) => _fetch(0);

  Future<PageResult<Assignment>> _fetch(int offset) async {
    final cache = ref.watch(cacheServiceProvider).valueOrNull;
    final limit = PageResult.fetchSize(PageResult.pageSize);

    try {
      var query = supabase.from('assignments').select();
      if (arg != null) query = query.eq('group_id', arg!);
      final rows = await query
          .order('created_at', ascending: false)
          .range(offset, offset + limit - 1);
      final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();
      if (offset == 0) await cache?.cacheAssignments(list);
      return PageResult.fromFetch(list.map(Assignment.fromJson).toList());
    } catch (_) {
      if (offset == 0) {
        final cached = cache?.cachedAssignments;
        if (cached != null) {
          return PageResult(
            items: cached.map(Assignment.fromJson).toList(),
            hasMore: false,
          );
        }
      }
      rethrow;
    }
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}
```

```
final assignmentDetailProvider = FutureProvider.family<Assignment?, String>((
  ref,
  id,
) async {
  final data = await supabase
      .from('assignments')
      .select()
      .eq('id', id)
      .maybeSingle();
  if (data == null) return null;
  return Assignment.fromJson(Map<String, dynamic>.from(data));
});

final tasksProvider = FutureProvider.family<List<TaskItem>, String>((
  ref,
  assignmentId,
) async {
  final rows = await supabase
      .from('tasks')
      .select()
      .eq('assignment_id', assignmentId)
      .order('sort_order');
  return rows
      .map((e) => TaskItem.fromJson(Map<String, dynamic>.from(e)))
      .toList();
});

final assignmentProgressProvider = FutureProvider.family<double, String>((
  ref,
  assignmentId,
) async {
  final student = await ref.watch(studentProfileProvider.future);
  if (student == null) return 0.0;

  final tasks = await supabase
      .from('tasks')
      .select('id')
      .eq('assignment_id', assignmentId);
  if (tasks.isEmpty) return 0.0;

  final taskIds = tasks.map((t) => t['id'] as String).toList();
  final completed = await supabase
      .from('submissions')
      .select('task_id')
      .eq('student_id', student.id)
      .eq('is_completed', true)
      .inFilter('task_id', taskIds);

  return completed.length / tasks.length;
});
```

**TASK — Trace lib/providers/homework/assignment_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/homework/teacher_submissions_provider.dart

Assignments, tasks, teacher submissions queue. This file (teacher submissions provider) is one concrete piece of that layer.

Providers declared: teacherSubmissionsProvider.

**lib/providers/homework/teacher_submissions_provider.dart**

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final teacherSubmissionsProvider = FutureProvider<List<Map<String, dynamic>>>((
  ref,
) async {
  final teacher = await ref.watch(teacherProfileProvider.future);
  if (teacher == null) return [];

  final assignments = await supabase
      .from('assignments')
      .select('id')
      .eq('teacher_id', teacher.id);
  final assignmentIds = assignments.map((e) => e['id'] as String).toList();
  if (assignmentIds.isEmpty) return [];

  final tasks = await supabase
      .from('tasks')
      .select('id')
      .inFilter('assignment_id', assignmentIds);
  final taskIds = tasks.map((e) => e['id'] as String).toList();
  if (taskIds.isEmpty) return [];

  final rows = await supabase
      .from('submissions')
      .select('''
            id, status, score, feedback, text_answer, file_url, voice_url,
            submitted_at,
            tasks(title, assignments(title)),
            students(profiles(full_name, username))
          ''')
      .inFilter('task_id', taskIds)
      .order('submitted_at', ascending: false);

  return rows.map((s) {
    final task = s['tasks'] as Map?;
    final assignment = task?['assignments'] as Map?;
    final profile = (s['students'] as Map?)?['profiles'] as Map?;
    return {
      'id': s['id'],
      'status': s['status'],
      'score': s['score'],
      'feedback': s['feedback'],
      'text_answer': s['text_answer'],
      'file_url': s['file_url'],
      'voice_url': s['voice_url'],
      'task_title': task?['title'],
      'assignment_title': assignment?['title'],
      'student_name':
          profile?['full_name'] ?? profile?['username'] ?? 'Student',
    };
  }).toList();
});
```

> **TASK — Trace lib/providers/homework/teacher_submissions_provider.dart**
> Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/marks/marks_providers.dart

Gradebook CRUD and student averages. This file (marks providers) is one concrete piece of that layer.

Providers declared: groupMarksProvider, groupMarkAveragesProvider, groupMarkCountProvider, memberMarksProvider, myMarksProvider, myMarkStatsProvider, myMarkAverageProvider.

**lib/providers/marks/marks_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/paging/page_result.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

Future<List<Map<String, dynamic>>> _fetchMarksPage({
  required int offset,
  String? groupId,
  String? studentId,
  AcademicTerm term = AcademicTerm.all,
  String select = '*, students(profiles(full_name, username))',
}) async {
  final limit = PageResult.fetchSize(PageResult.pageSize);
  var query = supabase.from('marks').select(select);
  if (groupId != null) query = query.eq('group_id', groupId);
  if (studentId != null) query = query.eq('student_id', studentId);
  final start = term.startKey;
  final end = term.endKey;
  if (start != null) query = query.gte('marked_on', start);
  if (end != null) query = query.lte('marked_on', end);

  final rows = await query
      .order('marked_on', ascending: false)
      .order('created_at', ascending: false)
      .range(offset, offset + limit - 1);
  return rows.map((e) => Map<String, dynamic>.from(e)).toList();
}

Map<String, double> _weightedAverages(List<Map<String, dynamic>> rows) {
  final totals = <String, double>{};
  final weights = <String, double>{};
  for (final row in rows) {
    final id = row['student_id'] as String;
    final pct = double.tryParse('${row['percentage']}') ?? 0;
    final w = double.tryParse('${row['weight']}') ?? 1;
    totals[id] = (totals[id] ?? 0) + pct * w;
    weights[id] = (weights[id] ?? 0) + w;
  }
  return {
    for (final id in totals.keys)
      id: weights[id] == 0 ? 0 : totals[id]! / weights[id]!,
  };
}

final groupMarksProvider =
    AsyncNotifierProvider.family<GroupMarksNotifier, PageResult<Mark>, String>(
      GroupMarksNotifier.new,
    );

class GroupMarksNotifier extends FamilyAsyncNotifier<PageResult<Mark>, String> {
  @override
  Future<PageResult<Mark>> build(String groupId) async {
    ref.watch(selectedMarkTermProvider);
    return _fetch(0);
  }

  Future<PageResult<Mark>> _fetch(int offset) async {
    final term = ref.read(selectedMarkTermProvider);
```

```
    final rows = await _fetchMarksPage(
      offset: offset,
      groupId: arg,
      term: term,
    );
    return PageResult.fromFetch(rows.map(Mark.fromJson).toList());
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}

final groupMarkAveragesProvider =
    FutureProvider.family<Map<String, double>, String>((ref, groupId) async {
      final term = ref.watch(selectedMarkTermProvider);
      var query = supabase
          .from('marks')
          .select('student_id, percentage, weight')
          .eq('group_id', groupId);
      final start = term.startKey;
      final end = term.endKey;
      if (start != null) query = query.gte('marked_on', start);
      if (end != null) query = query.lte('marked_on', end);
      final rows = await query;
      return _weightedAverages(
        rows.map((e) => Map<String, dynamic>.from(e)).toList(),
      );
    });

final groupMarkCountProvider = FutureProvider.family<int, String>((
  ref,
  groupId,
) async {
  final term = ref.watch(selectedMarkTermProvider);
  var query = supabase.from('marks').select('id').eq('group_id', groupId);
  final start = term.startKey;
  final end = term.endKey;
  if (start != null) query = query.gte('marked_on', start);
  if (end != null) query = query.lte('marked_on', end);
  final result = await query.count(CountOption.exact);
  return result.count;
});

final memberMarksProvider =
    FutureProvider.family<List<Mark>, ({String groupId, String studentId})>((
      ref,
      key,
    ) async {
      final term = ref.watch(selectedMarkTermProvider);
      var query = supabase
          .from('marks')
          .select('*, students(profiles(full_name, username))')
          .eq('group_id', key.groupId)
          .eq('student_id', key.studentId);
      final start = term.startKey;
      final end = term.endKey;
      if (start != null) query = query.gte('marked_on', start);
      if (end != null) query = query.lte('marked_on', end);
      final rows = await query
          .order('marked_on', ascending: false)
          .order('created_at', ascending: false);
      return rows
          .map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))
```

```
          .toList();
    });

final myMarksProvider =
    AsyncNotifierProvider<MyMarksNotifier, PageResult<Mark>>(
      MyMarksNotifier.new,
    );

class MyMarksNotifier extends AsyncNotifier<PageResult<Mark>> {
  @override
  Future<PageResult<Mark>> build() async {
    ref.watch(selectedMarkTermProvider);
    ref.watch(studentProfileProvider);
    return _fetch(0);
  }

  Future<PageResult<Mark>> _fetch(int offset) async {
    final student = await ref.read(studentProfileProvider.future);
    if (student == null) {
      return const PageResult(items: [], hasMore: false);
    }
    final term = ref.read(selectedMarkTermProvider);
    final rows = await _fetchMarksPage(
      offset: offset,
      studentId: student.id,
      term: term,
      select: '*, groups(title)',
    );
    return PageResult.fromFetch(rows.map(Mark.fromJson).toList());
  }

  Future<void> loadMore() async {
    final current = state.valueOrNull;
    if (current == null || !current.hasMore || current.loadingMore) return;
    state = AsyncData(current.copyWith(loadingMore: true));
    try {
      final next = await _fetch(current.items.length);
      state = AsyncData(current.append(next.items, hasMore: next.hasMore));
    } catch (_) {
      state = AsyncData(current.copyWith(loadingMore: false));
      rethrow;
    }
  }
}

final myMarkStatsProvider =
    FutureProvider<
      List<({MarkCategory category, double percentage, double weight})>
    >((ref) async {
      final student = await ref.watch(studentProfileProvider.future);
      if (student == null) return [];
      final term = ref.watch(selectedMarkTermProvider);
      var query = supabase
          .from('marks')
          .select('category, percentage, weight')
          .eq('student_id', student.id);
      final start = term.startKey;
      final end = term.endKey;
      if (start != null) query = query.gte('marked_on', start);
      if (end != null) query = query.lte('marked_on', end);
      final rows = await query;
      return rows
          .map(
            (e) => (
              category: markCategoryFromString(e['category'] as String?),
              percentage: double.tryParse('${e['percentage']}') ?? 0,
              weight: double.tryParse('${e['weight']}') ?? 1,
            ),
          )
          .toList();
    });

final myMarkAverageProvider = FutureProvider<double?>((ref) async {
```

```
  final stats = await ref.watch(myMarkStatsProvider.future);
  if (stats.isEmpty) return null;
  var total = 0.0;
  var weight = 0.0;
  for (final s in stats) {
    total += s.percentage * s.weight;
    weight += s.weight;
  }
  if (weight == 0) return null;
  return total / weight;
});

Future<void> saveMark({
  required String groupId,
  required String studentId,
  required String teacherId,
  required String title,
  required MarkCategory category,
  required double percentage,
  required double weight,
  required String comment,
  required DateTime markedOn,
  String? markId,
}) async {
  final payload = Mark(
    id: markId ?? '',
    groupId: groupId,
    studentId: studentId,
    teacherId: teacherId,
    title: title,
    percentage: percentage,
    category: category,
    weight: weight,
    comment: comment,
    markedOn: markedOn,
  ).toWrite();

  if (markId == null) {
    await supabase.from('marks').insert(payload);
  } else {
    await supabase.from('marks').update(payload).eq('id', markId);
  }
}

Future<void> deleteMark(String markId) async {
  await supabase.from('marks').delete().eq('id', markId);
}
```

**TASK — Trace lib/providers/marks/marks_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/notifications/notification_providers.dart

Realtime inbox + announcements. This file (notification providers) is one concrete piece of that layer.

Providers declared: notificationsProvider, unreadNotificationsCountProvider, announcementsProvider.

**lib/providers/notifications/notification_providers.dart**

```
import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mu_study/core/services/cache_service.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:mu_study/supabase/supabase_client.dart';

final notificationsProvider = StreamProvider<List<AppNotification>>((
  ref,
) async* {
  final profile = await ref.watch(currentProfileProvider.future);
  if (profile == null) {
    yield [];
    return;
  }

  final cache = ref.watch(cacheServiceProvider).valueOrNull;

  Future<List<AppNotification>> load() async {
    final rows = await supabase
        .from('notifications')
        .select()
        .eq('user_id', profile.id)
        .order('created_at', ascending: false)
        .limit(50);
    final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();
    await cache?.cacheNotifications(list);
    return list.map(AppNotification.fromJson).toList();
  }

  final changes = StreamController<void>();
  final channel = supabase
      .channel('notifications-${profile.id}')
      .onPostgresChanges(
        event: PostgresChangeEvent.all,
        schema: 'public',
        table: 'notifications',
        filter: PostgresChangeFilter(
          type: PostgresChangeFilterType.eq,
          column: 'user_id',
          value: profile.id,
        ),
        callback: (_) {
          if (!changes.isClosed) changes.add(null);
        },
      )
      .subscribe();

  ref.onDispose(() {
    supabase.removeChannel(channel);
    changes.close();
  });

  yield await load();

  await for (final _ in changes.stream) {
    yield await load();
  }
});

final unreadNotificationsCountProvider = Provider<int>((ref) {
  final notifications = ref.watch(notificationsProvider);
```

```
  return notifications.maybeWhen(
    data: (list) => list.where((n) => !n.isRead).length,
    orElse: () => 0,
  );
});

final announcementsProvider =
    FutureProvider.family<List<Announcement>, String?>((ref, groupId) async {
      var query = supabase.from('announcements').select();
      if (groupId != null) query = query.eq('group_id', groupId);
      final rows = await query.order('created_at', ascending: false);
      return rows
          .map((e) => Announcement.fromJson(Map<String, dynamic>.from(e)))
          .toList();
    });

Future<void> markNotificationRead(String id) async {
  await supabase.from('notifications').update({'is_read': true}).eq('id', id);
}

Future<void> markAllNotificationsRead(String userId) async {
  await supabase
      .from('notifications')
      .update({'is_read': true})
      .eq('user_id', userId)
      .eq('is_read', false);
}
```

**TASK — Trace lib/providers/notifications/notification_providers.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/settings/locale_provider.dart

Locale, theme mode, AppStyle. This file (locale provider) is one concrete piece of that layer.

Providers declared: localeProvider.

**lib/providers/settings/locale_provider.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/scheduler.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/locale/app_locale.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

const _prefsKey = 'preferred_language';

final localeProvider = StateNotifierProvider<LocaleController, Locale>((ref) {
  final controller = LocaleController(ref);
  ref.listen(currentProfileProvider, (prev, next) {
    next.whenData((profile) {
      final code = profile?.preferredLanguage;
      if (code != null) controller.applyFromProfile(code);
    });
  });
  return controller;
});

class LocaleController extends StateNotifier<Locale> {
  LocaleController(this._ref) : super(AppLanguage.english.locale) {
    _loadLocal();
  }

  final Ref _ref;

  Future<void> _loadLocal() async {
    final prefs = await SharedPreferences.getInstance();
    final code = prefs.getString(_prefsKey);
    final lang = AppLanguage.tryParse(code);
    if (lang != null) state = lang.locale;
  }

  void applyFromProfile(String code) {
    final lang = AppLanguage.tryParse(code);
    if (lang == null) return;
    if (state.languageCode == lang.code) return;
    state = lang.locale;
    SharedPreferences.getInstance().then(
      (prefs) => prefs.setString(_prefsKey, lang.code),
    );
  }

  Future<void> setLanguage(AppLanguage language) async {
    // `!semantics.parentDataDirty` assert storms in debug.
    await SchedulerBinding.instance.endOfFrame;
    if (!mounted) return;

    final localeChanged = state.languageCode != language.code;
    if (localeChanged) state = language.locale;

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_prefsKey, language.code);

    final profile = _ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    if (profile.preferredLanguage == language.code) return;

    await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {
      'preferred_language': language.code,
    });
    _ref.invalidate(currentProfileProvider);
  }
```

```
}
```

**TASK — Trace lib/providers/settings/locale_provider.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

## lib/providers/settings/theme_provider.dart

Locale, theme mode, AppStyle. This file (theme provider) is one concrete piece of that layer.

Providers declared: themeModeProvider, appStyleProvider.

**lib/providers/settings/theme_provider.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/scheduler.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/core/theme/app_style.dart';
import 'package:mu_study/providers/auth/auth_provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

const _prefsKey = 'preferred_theme';
const _stylePrefsKey = 'preferred_style';

enum AppThemePreference {
  light,
  dark,
  system;

  String get code => name;

  ThemeMode get themeMode => switch (this) {
    AppThemePreference.light => ThemeMode.light,
    AppThemePreference.dark => ThemeMode.dark,
    AppThemePreference.system => ThemeMode.system,
  };

  static AppThemePreference? tryParse(String? code) {
    if (code == null || code.isEmpty) return null;
    for (final v in AppThemePreference.values) {
      if (v.code == code) return v;
    }
    return null;
  }
}

Future<ThemeMode> loadSavedThemeMode() async {
  final prefs = await SharedPreferences.getInstance();
  final pref = AppThemePreference.tryParse(prefs.getString(_prefsKey));
  return pref?.themeMode ?? ThemeMode.system;
}

final themeModeProvider = StateNotifierProvider<ThemeModeController, ThemeMode>(
  (ref) {
    final controller = ThemeModeController(ref);
    ref.listen(currentProfileProvider, (prev, next) {
      next.whenData((profile) {
        final code = profile?.preferredTheme;
        if (code != null) controller.applyFromProfile(code);
      });
    });
    return controller;
  },
);

class ThemeModeController extends StateNotifier<ThemeMode> {
  ThemeModeController(this._ref, {ThemeMode? initial})
    : super(initial ?? ThemeMode.system) {
    if (initial == null) {
      _loadLocal();
    }
  }

  final Ref _ref;

  Future<void> _loadLocal() async {
    final prefs = await SharedPreferences.getInstance();
    final pref = AppThemePreference.tryParse(prefs.getString(_prefsKey));
```

```
    if (pref != null && mounted) state = pref.themeMode;
  }

  void applyFromProfile(String code) {
    final pref = AppThemePreference.tryParse(code);
    if (pref == null) return;
    if (state == pref.themeMode) return;
    state = pref.themeMode;
    SharedPreferences.getInstance().then(
      (prefs) => prefs.setString(_prefsKey, pref.code),
    );
  }

  Future<void> setPreference(AppThemePreference preference) async {
    await SchedulerBinding.instance.endOfFrame;
    if (!mounted) return;

    if (state != preference.themeMode) {
      state = preference.themeMode;
    }

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_prefsKey, preference.code);

    final profile = _ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    if (profile.preferredTheme == preference.code) return;

    await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {
      'preferred_theme': preference.code,
    });
    _ref.invalidate(currentProfileProvider);
  }
}

Future<AppStyle> loadSavedAppStyle() async {
  final prefs = await SharedPreferences.getInstance();
  return AppStyle.tryParse(prefs.getString(_stylePrefsKey)) ?? AppStyle.modern;
}

final appStyleProvider = StateNotifierProvider<AppStyleController, AppStyle>((
  ref,
) {
  final controller = AppStyleController(ref);
  ref.listen(currentProfileProvider, (prev, next) {
    next.whenData((profile) {
      final code = profile?.preferredStyle;
      if (code != null) controller.applyFromProfile(code);
    });
  });
  return controller;
});

class AppStyleController extends StateNotifier<AppStyle> {
  AppStyleController(this._ref, {AppStyle? initial})
    : super(initial ?? AppStyle.modern) {
    if (initial == null) {
      _loadLocal();
    }
  }

  final Ref _ref;

  Future<void> _loadLocal() async {
    final prefs = await SharedPreferences.getInstance();
    final style = AppStyle.tryParse(prefs.getString(_stylePrefsKey));
    if (style != null && mounted) state = style;
  }

  void applyFromProfile(String code) {
    final style = AppStyle.tryParse(code);
    if (style == null) return;
    if (state == style) return;
```

```
    state = style;
    SharedPreferences.getInstance().then(
      (prefs) => prefs.setString(_stylePrefsKey, style.code),
    );
  }

  Future<void> setStyle(AppStyle style) async {
    await SchedulerBinding.instance.endOfFrame;
    if (!mounted) return;

    if (state != style) state = style;

    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_stylePrefsKey, style.code);

    final profile = _ref.read(currentProfileProvider).valueOrNull;
    if (profile == null) return;
    if (profile.preferredStyle == style.code) return;

    await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {
      'preferred_style': style.code,
    });
    _ref.invalidate(currentProfileProvider);
  }
}
```

**TASK — Trace lib/providers/settings/theme_provider.dart**
Write the table/RPC list. Write which screen watches each provider. Write what invalidate you must call after a successful mutation.

# Component catalog — every reusable widget file

## lib/components/analytics/teacher_analytics_charts.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type AnalyticsChartPoint
- Type MarkDistributionDoughnut
- Type AttendanceTrendBars
- Type _WeekCategory
- Type _LegendPill

**lib/components/analytics/teacher_analytics_charts.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';
import 'package:syncfusion_flutter_charts/charts.dart';

class AnalyticsChartPoint {
  const AnalyticsChartPoint(this.label, this.value, this.color);
  final String label;
  final double value;
  final Color color;
}

class MarkDistributionDoughnut extends StatelessWidget {
  const MarkDistributionDoughnut({super.key, required this.points});

  final List<AnalyticsChartPoint> points;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final scheme = Theme.of(context).colorScheme;
    final total = points.fold<double>(0, (sum, p) => sum + p.value).round();
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final labelsInside =
        AppStyleTokens.maybeOf(context)?.prefersChartLabelsInside ?? false;

    return Column(
      children: [
        SizedBox(
          height: 280,
          child: SfCircularChart(
            margin: EdgeInsets.zero,
            annotations: <CircularChartAnnotation>[
              CircularChartAnnotation(
                widget: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
                      l10n.analyticsTotalMarks(total),
                      textAlign: TextAlign.center,
                      style: TextStyle(
                        color: scheme.onSurface,
                        fontWeight: FontWeight.w700,
                        fontSize: 14,
                        height: 1.25,
                      ),
                    ),
                  ],
                ),
              ),
            ],
```

```
            series: <CircularSeries<AnalyticsChartPoint, String>>[
              DoughnutSeries<AnalyticsChartPoint, String>(
                dataSource: points,
                xValueMapper: (d, _) => d.label,
                yValueMapper: (d, _) => d.value,
                pointColorMapper: (d, _) => d.color,
                innerRadius: '62%',
                radius: '78%',
                strokeWidth: isDark ? 2 : 1,
                strokeColor: isDark ? const Color(0xFF1A1A1A) : scheme.surface,
                animationDuration: 1100,
                animationDelay: 80,
                dataLabelMapper: (d, _) {
                  final pct = total == 0
                      ? 0
                      : ((d.value / total) * 100).round();
                  return '${d.value.round()}\n$pct%';
                },
                dataLabelSettings: DataLabelSettings(
                  isVisible: true,
                  labelPosition: labelsInside
                      ? ChartDataLabelPosition.inside
                      : ChartDataLabelPosition.outside,
                  connectorLineSettings: ConnectorLineSettings(
                    type: ConnectorType.curve,
                    length: '12%',
                    width: labelsInside ? 0 : 1.2,
                    color: scheme.onSurface.withValues(alpha: 0.35),
                  ),
                  textStyle: TextStyle(
                    color: labelsInside ? Colors.white : scheme.onSurface,
                    fontSize: 11,
                    fontWeight: labelsInside
                        ? FontWeight.w700
                        : FontWeight.w600,
                    height: 1.2,
                  ),
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 8),
        Wrap(
          spacing: 8,
          runSpacing: 8,
          alignment: WrapAlignment.center,
          children: [
            for (final p in points)
              _LegendPill(
                color: p.color,
                label: '${p.label}: ${p.value.round()}',
              ),
          ],
        ),
      ],
    );
  }
}

class AttendanceTrendBars extends StatelessWidget {
  const AttendanceTrendBars({super.key, required this.trend});

  final AttendanceWeekTrend trend;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final tokens = AppStyleTokens.maybeOf(context);
    final isMono = tokens?.isMonochromeWorkspace ?? false;
    final barRadius = isMono
```

```
      ? const BorderRadius.vertical(top: Radius.circular(2))
      : const BorderRadius.vertical(top: Radius.circular(6));
  final labelOnBar = isMono && !isDark
      ? AppColors.ink(context)
      : Colors.white;
  final categories = <_WeekCategory>[
    _WeekCategory(
      l10n.analyticsThisWeek,
      trend.thisWeek.present.toDouble(),
      trend.thisWeek.absent.toDouble(),
      trend.thisWeek.late.toDouble(),
    ),
    _WeekCategory(
      l10n.analyticsLastWeek,
      trend.lastWeek.present.toDouble(),
      trend.lastWeek.absent.toDouble(),
      trend.lastWeek.late.toDouble(),
    ),
  ];
  final maxY = [
    trend.thisWeek.max,
    trend.lastWeek.max,
    1,
  ].reduce((a, b) => a > b ? a : b);

  return Column(
    children: [
      SizedBox(
        height: 260,
        child: SfCartesianChart(
          margin: const EdgeInsets.only(top: 8, right: 8),
          plotAreaBorderWidth: 0,
          primaryXAxis: CategoryAxis(
            majorGridLines: const MajorGridLines(width: 0),
            axisLine: const AxisLine(width: 0),
            majorTickLines: const MajorTickLines(size: 0),
            labelStyle: TextStyle(
              color: scheme.onSurface.withValues(alpha: 0.75),
              fontWeight: FontWeight.w600,
              fontSize: 12,
            ),
          ),
          primaryYAxis: NumericAxis(
            minimum: 0,
            maximum: (maxY + 1).toDouble(),
            interval: maxY <= 5 ? 1 : null,
            majorGridLines: MajorGridLines(
              width: 1,
              color: scheme.onSurface.withValues(alpha: isDark ? 0.12 : 0.08),
            ),
            axisLine: const AxisLine(width: 0),
            majorTickLines: const MajorTickLines(size: 0),
            labelStyle: TextStyle(
              color: scheme.onSurface.withValues(alpha: 0.45),
              fontSize: 11,
            ),
          ),
          legend: Legend(
            isVisible: true,
            position: LegendPosition.bottom,
            overflowMode: LegendItemOverflowMode.wrap,
            iconHeight: 10,
            iconWidth: 10,
            padding: 0,
            itemPadding: 12,
            textStyle: TextStyle(
              color: scheme.onSurface.withValues(alpha: 0.8),
              fontSize: 12,
            ),
          ),
          series: <CartesianSeries<_WeekCategory, String>>[
            ColumnSeries<_WeekCategory, String>(
              name: l10n.attendancePresentWord,
```

```
            dataSource: categories,
            xValueMapper: (d, _) => d.label,
            yValueMapper: (d, _) => d.present,
            dataLabelMapper: (d, _) =>
                d.present == 0 ? '' : '${d.present.round()}',
            color: AppColors.success(context),
            gradient: isMono ? null : tokens?.chartPositive,
            borderRadius: barRadius,
            width: isMono ? 0.55 : 0.7,
            spacing: 0.18,
            animationDuration: 1000,
            animationDelay: 60,
            dataLabelSettings: DataLabelSettings(
              isVisible: true,
              labelAlignment: ChartDataLabelAlignment.middle,
              textStyle: TextStyle(
                color: isMono && !isDark ? Colors.white : labelOnBar,
                fontSize: 11,
                fontWeight: FontWeight.w700,
              ),
            ),
          ),
          ColumnSeries<_WeekCategory, String>(
            name: l10n.attendanceAbsentWord,
            dataSource: categories,
            xValueMapper: (d, _) => d.label,
            yValueMapper: (d, _) => d.absent,
            dataLabelMapper: (d, _) =>
                d.absent == 0 ? '' : '${d.absent.round()}',
            color: AppColors.danger(context),
            gradient: isMono ? null : tokens?.chartDanger,
            borderRadius: barRadius,
            width: isMono ? 0.55 : 0.7,
            spacing: 0.18,
            animationDuration: 1100,
            animationDelay: 160,
            dataLabelSettings: DataLabelSettings(
              isVisible: true,
              labelAlignment: ChartDataLabelAlignment.middle,
              textStyle: TextStyle(
                color: isMono && !isDark ? Colors.white : labelOnBar,
                fontSize: 11,
                fontWeight: FontWeight.w700,
              ),
            ),
          ),
          ColumnSeries<_WeekCategory, String>(
            name: l10n.attendanceLateWord,
            dataSource: categories,
            xValueMapper: (d, _) => d.label,
            yValueMapper: (d, _) => d.late,
            dataLabelMapper: (d, _) =>
                d.late == 0 ? '' : '${d.late.round()}',
            color: AppColors.warning(context),
            gradient: isMono ? null : tokens?.chartWarning,
            borderRadius: barRadius,
            width: isMono ? 0.55 : 0.7,
            spacing: 0.18,
            animationDuration: 1200,
            animationDelay: 260,
            dataLabelSettings: DataLabelSettings(
              isVisible: true,
              labelAlignment: ChartDataLabelAlignment.middle,
              textStyle: TextStyle(
                color: isMono && !isDark ? Colors.white : labelOnBar,
                fontSize: 11,
                fontWeight: FontWeight.w700,
              ),
            ),
          ),
        ],
      ),
    ),
```

```
      ],
    );
  }
}

List<AnalyticsChartPoint> markBandPoints(
  BuildContext context,
  AppLocalizations l10n,
  List<double> averages,
) {
  var excellent = 0, good = 0, needsWork = 0, atRisk = 0;
  for (final value in averages) {
    if (value >= 85) {
      excellent++;
    } else if (value >= 70) {
      good++;
    } else if (value >= 50) {
      needsWork++;
    } else {
      atRisk++;
    }
  }
  return [
    AnalyticsChartPoint(
      l10n.chartBandExcellent,
      excellent.toDouble(),
      AppColors.success(context),
    ),
    AnalyticsChartPoint(
      l10n.chartBandGood,
      good.toDouble(),
      AppColors.goldAccent(context),
    ),
    AnalyticsChartPoint(
      l10n.chartBandNeedsWork,
      needsWork.toDouble(),
      AppColors.warning(context),
    ),
    AnalyticsChartPoint(
      l10n.chartBandAtRisk,
      atRisk.toDouble(),
      AppColors.danger(context),
    ),
  ].where((p) => p.value > 0).toList();
}

class _WeekCategory {
  const _WeekCategory(this.label, this.present, this.absent, this.late);
  final String label;
  final double present;
  final double absent;
  final double late;
}

class _LegendPill extends StatelessWidget {
  const _LegendPill({required this.color, required this.label});

  final Color color;
  final String label;

  @override
  Widget build(BuildContext context) {
    final scheme = Theme.of(context).colorScheme;
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final isMono =
        AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
      decoration: BoxDecoration(
        color: isDark
            ? Colors.white.withValues(alpha: 0.06)
            : scheme.onSurface.withValues(alpha: 0.05),
        borderRadius: BorderRadius.circular(isMono ? 4 : 20),
```

```
          border: isMono
              ? Border.all(color: scheme.onSurface.withValues(alpha: 0.08))
              : null,
        ),
        child: Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            Container(
              width: 8,
              height: 8,
              decoration: BoxDecoration(
                color: color,
                shape: BoxShape.circle,
                boxShadow: [
                  BoxShadow(color: color.withValues(alpha: 0.55), blurRadius: 6),
                ],
              ),
            ),
            const SizedBox(width: 6),
            Text(
              label,
              style: TextStyle(
                color: scheme.onSurface.withValues(alpha: 0.85),
                fontSize: 12,
                fontWeight: FontWeight.w600,
              ),
            ),
          ],
        ),
      );
  }
}
```

**WARNING — Reuse rule**
If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/attachments/attachment_preview.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type AttachmentPreview
- Type _ImagePreviewCard
- Type _FilePreviewCard
- Type _RemoveButton
- Type _ImageLightbox

**lib/components/attachments/attachment_preview.dart**

```
import 'dart:typed_data';

import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/components/dialogs/pdf_viewer_sheet.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:url_launcher/url_launcher.dart';

const _imageExtensions = {'jpg', 'jpeg', 'png', 'gif', 'webp', 'heic', 'heif'};

bool isImageAttachment(String? nameOrUrl) {
  final ext = attachmentExtension(nameOrUrl);
  return ext != null && _imageExtensions.contains(ext);
}

bool isPdfAttachment(String? nameOrUrl) {
  return attachmentExtension(nameOrUrl) == 'pdf';
}

String? attachmentExtension(String? nameOrUrl) {
  if (nameOrUrl == null || nameOrUrl.isEmpty) return null;
  final clean = Uri.tryParse(nameOrUrl)?.path ?? nameOrUrl;
  final base = clean.split('/').last.split('?').first;
  final dot = base.lastIndexOf('.');
  if (dot < 0 || dot == base.length - 1) return null;
  return base.substring(dot + 1).toLowerCase();
}

String attachmentFileName(String? nameOrUrl, {String fallback = 'Attachment'}) {
  if (nameOrUrl == null || nameOrUrl.isEmpty) return fallback;
  final clean = Uri.tryParse(nameOrUrl)?.path ?? nameOrUrl;
  final base = clean.split('/').last.split('?').first;
  return base.isEmpty ? fallback : Uri.decodeComponent(base);
}

String? storagePathFromSignedUrl(String url, {String? bucket}) {
  final uri = Uri.tryParse(url);
  if (uri == null) return null;
  final segments = uri.pathSegments;
  final signIdx = segments.indexOf('sign');
  if (signIdx >= 0 && signIdx + 2 < segments.length) {
    final bucketName = segments[signIdx + 1];
    if (bucket != null && bucketName != bucket) return null;
    return segments.sublist(signIdx + 2).map(Uri.decodeComponent).join('/');
  }
  return null;
}

Future<void> openAttachment(
  BuildContext context, {
  required String url,
  String? fileName,
  Uint8List? bytes,
}) async {
  final name = fileName ?? attachmentFileName(url);
```

```
    if (isImageAttachment(name)) {
      await showDialog<void>(
        context: context,
        useRootNavigator: true,
        barrierColor: Colors.black.withValues(alpha: 0.92),
        builder: (ctx) => _ImageLightbox(url: url, bytes: bytes, title: name),
      );
      return;
    }

    if (isPdfAttachment(name)) {
      await openPdfViewer(context, url, title: name);
      return;
    }

    final uri = Uri.tryParse(url);
    if (uri != null && await canLaunchUrl(uri)) {
      await launchUrl(uri, mode: LaunchMode.externalApplication);
    } else if (context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.attachmentCouldNotOpen)),
      );
    }
  }

  class AttachmentPreview extends StatelessWidget {
    const AttachmentPreview({
      super.key,
      required this.url,
      this.fileName,
      this.bytes,
      this.onRemove,
      this.enabled = true,
    });

    final String url;
    final String? fileName;
    final Uint8List? bytes;
    final VoidCallback? onRemove;
    final bool enabled;

    @override
    Widget build(BuildContext context) {
      final l10n = context.l10n;
      final name =
          fileName ?? attachmentFileName(url, fallback: l10n.taskAttachment);
      final isImage = isImageAttachment(name);

      return isImage
          ? _ImagePreviewCard(
              url: url,
              bytes: bytes,
              fileName: name,
              subtitle: l10n.attachmentImage,
              tapPreview: l10n.attachmentTapPreview,
              onOpen: enabled
                  ? () => openAttachment(
                      context,
                      url: url,
                      fileName: name,
                      bytes: bytes,
                    )
                  : null,
              onRemove: enabled ? onRemove : null,
            )
          : _FilePreviewCard(
              fileName: name,
              kindLabel: isPdfAttachment(name)
                  ? l10n.attachmentPdf
                  : l10n.attachmentFile,
              tapOpen: l10n.attachmentTapOpen,
              onOpen: enabled
                  ? () => openAttachment(context, url: url, fileName: name)
```

```
                : null,
            onRemove: enabled ? onRemove : null,
          );
  }
}

class _ImagePreviewCard extends StatelessWidget {
  const _ImagePreviewCard({
    required this.url,
    required this.fileName,
    required this.subtitle,
    required this.tapPreview,
    this.bytes,
    this.onOpen,
    this.onRemove,
  });

  final String url;
  final String fileName;
  final String subtitle;
  final String tapPreview;
  final Uint8List? bytes;
  final VoidCallback? onOpen;
  final VoidCallback? onRemove;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onOpen,
        borderRadius: BorderRadius.circular(16),
        child: Ink(
          decoration: BoxDecoration(
            color: AppColors.controlSurface(context),
            borderRadius: BorderRadius.circular(16),
            border: Border.all(color: AppColors.inkAlpha(context, 0.06)),
            boxShadow: [
              BoxShadow(
                color: AppColors.inkAlpha(context, 0.06),
                blurRadius: 16,
                offset: const Offset(0, 6),
              ),
            ],
          ),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              ClipRRect(
                borderRadius: const BorderRadius.vertical(
                  top: Radius.circular(16),
                ),
                child: AspectRatio(
                  aspectRatio: 16 / 10,
                  child: Stack(
                    fit: StackFit.expand,
                    children: [
                      ColoredBox(
                        color: AppColors.softWash(context, lightAlpha: 0.5),
                        child: bytes != null
                            ? Image.memory(bytes!, fit: BoxFit.cover)
                            : CachedNetworkImage(
                                imageUrl: url,
                                fit: BoxFit.cover,
                                placeholder: (_, _) => Center(
                                  child: SizedBox(
                                    width: 24,
                                    height: 24,
                                    child: CircularProgressIndicator(
                                      strokeWidth: 2,
                                    ),
                                  ),
                                ),
```

```
                    errorWidget: (_, _, _) => Center(
                      child: Icon(
                        Icons.broken_image_outlined,
                        color: AppColors.accent(context),
                        size: 32,
                      ),
                    ),
                  ),
            ),
            Positioned(
              left: 0,
              right: 0,
              bottom: 0,
              child: DecoratedBox(
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    begin: Alignment.topCenter,
                    end: Alignment.bottomCenter,
                    colors: [
                      Colors.transparent,
                      AppColors.inkAlpha(context, 0.45),
                    ],
                  ),
                ),
                child: const SizedBox(height: 48),
              ),
            ),
            Positioned(
              left: 12,
              bottom: 10,
              child: Row(
                children: [
                  Icon(
                    Icons.zoom_in_rounded,
                    size: 16,
                    color: Colors.white.withValues(alpha: 0.95),
                  ),
                  const SizedBox(width: 6),
                  Text(
                    tapPreview,
                    style: TextStyle(
                      color: Colors.white.withValues(alpha: 0.95),
                      fontSize: 12,
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                ],
              ),
            ),
            if (onRemove != null)
              Positioned(
                top: 8,
                right: 8,
                child: _RemoveButton(onPressed: onRemove!),
              ),
          ],
        ),
      ),
    ),
    Padding(
      padding: const EdgeInsets.fromLTRB(14, 12, 14, 14),
      child: Row(
        children: [
          Container(
            width: 36,
            height: 36,
            decoration: BoxDecoration(
              color: AppColors.mutedGold.withValues(alpha: 0.15),
              borderRadius: BorderRadius.circular(10),
            ),
            child: const Icon(
              Icons.image_outlined,
              size: 18,
```

```
                        color: AppColors.mutedGold,
                      ),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            fileName,
                            maxLines: 1,
                            overflow: TextOverflow.ellipsis,
                            style: const TextStyle(
                              fontWeight: FontWeight.w700,
                              fontSize: 14,
                            ),
                          ),
                          const SizedBox(height: 2),
                          Text(
                            subtitle,
                            style: TextStyle(
                              fontSize: 12,
                              color: AppColors.inkAlpha(context, 0.45),
                            ),
                          ),
                        ],
                      ),
                    ),
                    Icon(
                      Icons.chevron_right_rounded,
                      color: AppColors.inkAlpha(context, 0.3),
                    ),
                  ],
                ),
              ),
            ),
          ],
        ),
      ),
    ),
  );
 }
}

class _FilePreviewCard extends StatelessWidget {
  const _FilePreviewCard({
    required this.fileName,
    required this.kindLabel,
    required this.tapOpen,
    this.onOpen,
    this.onRemove,
  });

  final String fileName;
  final String kindLabel;
  final String tapOpen;
  final VoidCallback? onOpen;
  final VoidCallback? onRemove;

  @override
  Widget build(BuildContext context) {
    final isPdf = isPdfAttachment(fileName);
    final accent = isPdf ? AppColors.coffeeBrown : AppColors.softOlive;
    final icon = isPdf
        ? Icons.picture_as_pdf_rounded
        : Icons.insert_drive_file_outlined;

    return Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: onOpen,
        borderRadius: BorderRadius.circular(16),
        child: Ink(
          padding: const EdgeInsets.all(14),
```

```
          decoration: BoxDecoration(
            color: AppColors.controlSurface(context),
            borderRadius: BorderRadius.circular(16),
            border: Border.all(color: AppColors.inkAlpha(context, 0.06)),
            boxShadow: [
              BoxShadow(
                color: AppColors.inkAlpha(context, 0.06),
                blurRadius: 16,
                offset: const Offset(0, 6),
              ),
            ],
          ),
          child: Row(
            children: [
              Container(
                width: 52,
                height: 52,
                decoration: BoxDecoration(
                  color: accent.withValues(alpha: 0.12),
                  borderRadius: BorderRadius.circular(14),
                ),
                child: Icon(icon, color: accent, size: 26),
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      fileName,
                      maxLines: 2,
                      overflow: TextOverflow.ellipsis,
                      style: const TextStyle(
                        fontWeight: FontWeight.w700,
                        fontSize: 14,
                        height: 1.25,
                      ),
                    ),
                    const SizedBox(height: 4),
                    Text(
                      kindLabel,
                      style: TextStyle(
                        fontSize: 12,
                        color: AppColors.inkAlpha(context, 0.45),
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      tapOpen,
                      style: TextStyle(
                        fontSize: 11,
                        fontWeight: FontWeight.w600,
                        color: accent.withValues(alpha: 0.9),
                      ),
                    ),
                  ],
                ),
              ),
              if (onRemove != null) ...[
                const SizedBox(width: 4),
```

**lib/components/attachments/attachment_preview.dart — continued (2/2)**

```
                _RemoveButton(onPressed: onRemove!),
              ] else
                Icon(
                  Icons.chevron_right_rounded,
                  color: AppColors.inkAlpha(context, 0.3),
                ),
            ],
          ),
        ),
      ),
```

```
    );
  }
}

class _RemoveButton extends StatelessWidget {
  const _RemoveButton({required this.onPressed});

  final VoidCallback onPressed;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: Colors.white.withValues(alpha: 0.95),
      shape: const CircleBorder(),
      elevation: 1,
      shadowColor: AppColors.inkAlpha(context, 0.2),
      child: InkWell(
        customBorder: const CircleBorder(),
        onTap: onPressed,
        child: const Padding(
          padding: EdgeInsets.all(8),
          child: Icon(
            Icons.delete_outline_rounded,
            size: 18,
            color: AppColors.errorRed,
          ),
        ),
      ),
    );
  }
}

class _ImageLightbox extends StatelessWidget {
  const _ImageLightbox({required this.url, required this.title, this.bytes});

  final String url;
  final String title;
  final Uint8List? bytes;

  @override
  Widget build(BuildContext context) {
    return Dialog.fullscreen(
      backgroundColor: Colors.black,
      child: SafeArea(
        child: Stack(
          children: [
            Center(
              child: InteractiveViewer(
                minScale: 0.8,
                maxScale: 4,
                child: bytes != null
                    ? Image.memory(bytes!)
                    : CachedNetworkImage(
                        imageUrl: url,
                        fit: BoxFit.contain,
                        placeholder: (_, _) => const CircularProgressIndicator(
                          color: Colors.white,
                        ),
                        errorWidget: (_, _, _) => const Icon(
                          Icons.broken_image_outlined,
                          color: Colors.white70,
                          size: 48,
                        ),
                      ),
              ),
            ),
            Positioned(
              top: 4,
              left: 8,
              right: 8,
              child: Row(
                children: [
                  Expanded(
```

```
                    child: Text(
                      title,
                      maxLines: 1,
                      overflow: TextOverflow.ellipsis,
                      style: const TextStyle(
                        color: Colors.white,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                  IconButton(
                    onPressed: () => Navigator.of(context).pop(),
                    icon: const Icon(Icons.close_rounded, color: Colors.white),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/attachments/voice_note_field.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type VoiceNoteField
- Type _VoiceNoteFieldState
- Type VoiceNotePlayer
- Type _VoiceNotePlayerState
- Type _VoiceShell
- Type _RoundAction
- Type _PulsingDot
- Type _PulsingDotState

**lib/components/attachments/voice_note_field.dart**

```
import 'dart:async';

import 'package:cross_file/cross_file.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/utils/voice_playback.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';
import 'package:record/record.dart';

class VoiceNoteField extends StatefulWidget {
  const VoiceNoteField({
    super.key,
    this.voiceUrl,
    this.enabled = true,
    required this.onUpload,
    this.onRemove,
  });

  final String? voiceUrl;
  final bool enabled;
  final Future<void> Function(Uint8List bytes, String fileName) onUpload;
  final VoidCallback? onRemove;

  @override
  State<VoiceNoteField> createState() => _VoiceNoteFieldState();
}

class _VoiceNoteFieldState extends State<VoiceNoteField> {
  final AudioRecorder _recorder = AudioRecorder();
  final _voice = VoicePlaybackController.instance;

  bool _recording = false;
  bool _busy = false;
  String? _localPath;
  Duration _elapsed = Duration.zero;
  Timer? _tick;

  bool get _hasNote =>
      (widget.voiceUrl != null && widget.voiceUrl!.isNotEmpty) ||
      _localPath != null;

  String? get _playKey => _localPath ?? widget.voiceUrl;

  @override
  void initState() {
    super.initState();
    _voice.addListener(_onVoiceChanged);
  }
```

```
  @override
  void dispose() {
    _tick?.cancel();
    _voice.removeListener(_onVoiceChanged);
    _recorder.dispose();
    super.dispose();
  }

  void _onVoiceChanged() {
    if (mounted) setState(() {});
  }

  String _format(Duration d) {
    final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');
    final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');
    return '$m:$s';
  }

  Future<String> _recordingPath() async {
    final stamp = DateTime.now().millisecondsSinceEpoch;
    final ext = kIsWeb ? 'wav' : 'm4a';
    final name = 'voice_$stamp.$ext';
    if (kIsWeb) return name;
    final dir = await getTemporaryDirectory();
    return p.join(dir.path, name);
  }

  Future<void> _startRecording() async {
    if (!widget.enabled || _busy || _recording) return;

    final allowed = await _recorder.hasPermission();
    if (!allowed) {
      if (!mounted) return;
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(SnackBar(content: Text(context.l10n.voiceMicPermission)));
      return;
    }

    try {
      await _voice.stop();
      final path = await _recordingPath();
      await _recorder.start(
        RecordConfig(
          encoder: kIsWeb ? AudioEncoder.wav : AudioEncoder.aacLc,
          bitRate: 128000,
          sampleRate: 44100,
        ),
        path: path,
      );
      if (!mounted) return;
      setState(() {
        _recording = true;
        _elapsed = Duration.zero;
        _localPath = null;
      });
      _tick?.cancel();
      _tick = Timer.periodic(const Duration(seconds: 1), (_) {
        if (!mounted) return;
        setState(() => _elapsed += const Duration(seconds: 1));
      });
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.voiceCouldNotStart('$e'))),
      );
    }
  }

  Future<void> _stopRecording() async {
    if (!_recording) return;
    _tick?.cancel();
    setState(() {
```

```
      _recording = false;
      _busy = true;
    });

    try {
      final path = await _recorder.stop();
      if (path == null || path.isEmpty) {
        throw Exception('No audio was captured.');
      }
      final bytes = await XFile(path).readAsBytes();
      if (bytes.isEmpty) throw Exception('Recording was empty.');

      final fileName = kIsWeb
          ? 'voice_${DateTime.now().millisecondsSinceEpoch}.wav'
          : p.basename(path);

      setState(() => _localPath = path);
      await widget.onUpload(bytes, fileName);
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text(context.l10n.voiceCouldNotSave('$e'))),
        );
      }
      setState(() => _localPath = null);
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  Future<void> _cancelRecording() async {
    _tick?.cancel();
    try {
      await _recorder.cancel();
    } catch (_) {}
    if (!mounted) return;
    setState(() {
      _recording = false;
      _elapsed = Duration.zero;
    });
  }

  Future<void> _togglePlay() async {
    if (_busy || _recording || !_hasNote) return;
    final key = _playKey;
    if (key == null) return;

    try {
      if (_localPath != null) {
        await _voice.toggleLocal(_localPath!);
      } else if (widget.voiceUrl != null) {
        await _voice.toggleRemote(widget.voiceUrl!);
      }
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),
      );
    }
  }

  Future<void> _remove() async {
    await _voice.stop();
    setState(() {
      _localPath = null;
    });
    widget.onRemove?.call();
  }

  @override
  Widget build(BuildContext context) {
    if (_recording) return _buildRecordingCard();
    if (_hasNote) return _buildPlaybackCard();
```

```
    return _buildIdleCard();
  }

  Widget _buildIdleCard() {
    return _VoiceShell(
      child: Row(
        children: [
          _RoundAction(
            color: AppColors.accent(context),
            icon: Icons.mic_rounded,
            onPressed: widget.enabled && !_busy ? _startRecording : null,
          ),
          const SizedBox(width: 14),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  context.l10n.taskVoice,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 14,
                    color: AppColors.ink(context),
                  ),
                ),
                const SizedBox(height: 2),
                Text(
                  context.l10n.voiceTapToRecord,
                  style: TextStyle(
                    fontSize: 12,
                    color: AppColors.inkAlpha(context, 0.55),
                  ),
                ),
              ],
            ),
          ),
          if (_busy)
            const SizedBox(
              width: 20,
              height: 20,
              child: CircularProgressIndicator(strokeWidth: 2),
            ),
        ],
      ),
    );
  }

  Widget _buildRecordingCard() {
    return _VoiceShell(
      accent: AppColors.errorRed.withValues(alpha: 0.12),
      child: Row(
        children: [
          _PulsingDot(active: _recording),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  context.l10n.voiceRecording,
                  style: const TextStyle(
                    fontWeight: FontWeight.w800,
                    fontSize: 14,
                    color: AppColors.errorRed,
                  ),
                ),
                const SizedBox(height: 2),
                Text(
                  _format(_elapsed),
                  style: TextStyle(
                    fontSize: 13,
                    fontFeatures: const [FontFeature.tabularFigures()],
                    color: AppColors.inkAlpha(context, 0.7),
```

```
                ),
              ),
            ],
          ),
        ),
        IconButton(
          tooltip: context.l10n.commonCancel,
          onPressed: _cancelRecording,
          icon: const Icon(Icons.close_rounded),
          color: AppColors.inkAlpha(context, 0.5),
        ),
        const SizedBox(width: 4),
        _RoundAction(
          color: AppColors.errorRed,
          icon: Icons.stop_rounded,
          onPressed: _stopRecording,
        ),
      ],
    ),
  );
}

Widget _buildPlaybackCard() {
  final key = _playKey ?? '';
  final playing = _voice.isPlaying(key);
  final loading = _voice.isLoading(key);
  final position = playing || loading ? _voice.position : Duration.zero;
  final duration = playing || loading ? _voice.duration : Duration.zero;
  final progress = duration.inMilliseconds == 0
      ? 0.0
      : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);

  return _VoiceShell(
    child: Column(
      children: [
        Row(
          children: [
            _RoundAction(
              color: AppColors.mutedGold,
              icon: loading
                  ? Icons.hourglass_top_rounded
                  : playing
                  ? Icons.stop_rounded
                  : Icons.play_arrow_rounded,
              onPressed: widget.enabled && !_busy ? _togglePlay : null,
            ),
            const SizedBox(width: 14),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    context.l10n.voiceNote,
                    style: TextStyle(
                      fontWeight: FontWeight.w700,
                      fontSize: 14,
                      color: AppColors.ink(context),
                    ),
                  ),
                  const SizedBox(height: 2),
                  Text(
                    duration == Duration.zero
                        ? context.l10n.voiceTapPlay
                        : '${_format(position)} / ${_format(duration)}',
                    style: TextStyle(
                      fontSize: 12,
                      fontFeatures: const [FontFeature.tabularFigures()],
                      color: AppColors.inkAlpha(context, 0.55),
                    ),
                  ),
                ],
              ),
            ),
          ),
```

```
              if (_busy)
                const Padding(
                  padding: EdgeInsets.only(right: 8),
                  child: SizedBox(
                    width: 20,
                    height: 20,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  ),
                ),
              if (widget.onRemove != null && widget.enabled)
                IconButton(
                  tooltip: context.l10n.commonDelete,
                  onPressed: _busy ? null : _remove,
                  icon: const Icon(Icons.delete_outline_rounded),
                  color: AppColors.errorRed,
                ),
            ],
          ),
          const SizedBox(height: 10),
          ClipRRect(
            borderRadius: BorderRadius.circular(4),
            child: LinearProgressIndicator(
              value: playing || position > Duration.zero ? progress : 0,
              minHeight: 4,
              backgroundColor: AppColors.softSurface(context),
              color: AppColors.mutedGold,
            ),
          ),
          if (widget.enabled && widget.onRemove != null) ...[
            const SizedBox(height: 8),
            Align(
              alignment: Alignment.centerLeft,
              child: TextButton.icon(
                onPressed: _busy || _recording ? null : _startRecording,
                icon: const Icon(Icons.mic_none_rounded, size: 18),
                label: Text(context.l10n.voiceRerecord),
              ),
            ),
          ],
        ],
      ),
    );
  }
}

class VoiceNotePlayer extends StatefulWidget {
  const VoiceNotePlayer({super.key, required this.url, this.title});

  final String url;
  final String? title;

  @override
  State<VoiceNotePlayer> createState() => _VoiceNotePlayerState();
}

class _VoiceNotePlayerState extends State<VoiceNotePlayer> {
  final _voice = VoicePlaybackController.instance;

  @override
  void initState() {
    super.initState();
    _voice.addListener(_onVoiceChanged);
  }

  @override
  void dispose() {
    _voice.removeListener(_onVoiceChanged);
    super.dispose();
  }

  void _onVoiceChanged() {
    if (mounted) setState(() {});
  }
```

```
  String _format(Duration d) {
    final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');
    final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');
    return '$m:$s';
  }

  Future<void> _toggle() async {
    try {
      await _voice.toggleRemote(widget.url);
    } catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final playing = _voice.isPlaying(widget.url);
    final loading = _voice.isLoading(widget.url);
    final position = playing || loading ? _voice.position : Duration.zero;
    final duration = playing || loading ? _voice.duration : Duration.zero;
    final progress = duration.inMilliseconds == 0
        ? 0.0
        : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);

    return _VoiceShell(
      child: Column(
        children: [
          Row(
            children: [
              _RoundAction(
                color: AppColors.mutedGold,
                icon: loading
                    ? Icons.hourglass_top_rounded
                    : playing
                    ? Icons.stop_rounded
                    : Icons.play_arrow_rounded,
                onPressed: _toggle,
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(
                      widget.title ?? l10n.voiceNote,
                      style: TextStyle(
                        fontWeight: FontWeight.w700,
                        fontSize: 14,
```

**lib/components/attachments/voice_note_field.dart — continued (2/2)**

```
                        color: AppColors.ink(context),
                      ),
                    ),
                    const SizedBox(height: 2),
                    Text(
                      duration == Duration.zero
                          ? l10n.voiceStudentAnswer
                          : '${_format(position)} / ${_format(duration)}',
                      style: TextStyle(
                        fontSize: 12,
                        color: AppColors.inkAlpha(context, 0.55),
                      ),
                    ),
                  ],
                ),
              ),
            ],
          ),
```

```
          const SizedBox(height: 10),
          ClipRRect(
            borderRadius: BorderRadius.circular(4),
            child: LinearProgressIndicator(
              value: playing || position > Duration.zero ? progress : 0,
              minHeight: 4,
              backgroundColor: AppColors.softSurface(context),
              color: AppColors.mutedGold,
            ),
          ),
        ],
      ),
    );
  }
}

class _VoiceShell extends StatelessWidget {
  const _VoiceShell({required this.child, this.accent});

  final Widget child;
  final Color? accent;

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final bg =
        accent ??
        (isDark
            ? AppColors.darkSurfaceElevated
            : AppColors.sand.withValues(alpha: 0.85));

    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(14),
      decoration: BoxDecoration(
        color: bg,
        borderRadius: BorderRadius.circular(16),
        border: Border.all(
          color: isDark
              ? AppColors.darkBorder.withValues(alpha: 0.55)
              : AppColors.inkAlpha(context, 0.08),
        ),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: isDark ? 0.28 : 0.06),
            blurRadius: 16,
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: DefaultTextStyle.merge(
        style: TextStyle(color: AppColors.ink(context)),
        child: child,
      ),
    );
  }
}

class _RoundAction extends StatelessWidget {
  const _RoundAction({
    required this.color,
    required this.icon,
    required this.onPressed,
  });

  final Color color;
  final IconData icon;
  final VoidCallback? onPressed;

  @override
  Widget build(BuildContext context) {
    return Material(
      color: color.withValues(alpha: onPressed == null ? 0.35 : 1),
```

```
      shape: const CircleBorder(),
      child: InkWell(
        customBorder: const CircleBorder(),
        onTap: onPressed,
        child: SizedBox(
          width: 48,
          height: 48,
          child: Icon(icon, color: Colors.white, size: 26),
        ),
      ),
    );
  }
}

class _PulsingDot extends StatefulWidget {
  const _PulsingDot({required this.active});

  final bool active;

  @override
  State<_PulsingDot> createState() => _PulsingDotState();
}

class _PulsingDotState extends State<_PulsingDot>
    with SingleTickerProviderStateMixin {
  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 900),
  )..repeat(reverse: true);

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return ScaleTransition(
      scale: Tween(
        begin: 0.85,
        end: 1.15,
      ).animate(CurvedAnimation(parent: _controller, curve: Curves.easeInOut)),
      child: Container(
        width: 14,
        height: 14,
        decoration: const BoxDecoration(
          color: AppColors.errorRed,
          shape: BoxShape.circle,
        ),
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/attendance/attendance_calendar_grid.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type AttendanceCalendarGrid
- Type _DayCell
- Type AttendanceMonthHeader
- Type AttendanceStatsBar
- Type _StatChip
- Type AttendanceLegendRow
- Type _Legend

**lib/components/attendance/attendance_calendar_grid.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/attendance/attendance_providers.dart';

Color attendanceStatusColor(AttendanceStatus? status, {BuildContext? context}) {
  return switch (status) {
    AttendanceStatus.present => AppColors.successGreen,
    AttendanceStatus.late => AppColors.warmAmber,
    AttendanceStatus.absent => AppColors.errorRed,
    AttendanceStatus.excused => const Color(0xFF5B8DB8),
    null =>
      context != null
          ? AppColors.inkAlpha(context, 0.08)
          : const Color(0xFF808080).withValues(alpha: 0.2),
  };
}

List<List<DateTime?>> buildMonthWeekRows(DateTime month) {
  final first = DateTime(month.year, month.month, 1);
  final daysInMonth = DateTime(month.year, month.month + 1, 0).day;
  final startOffset = first.weekday - 1;
  final cells = <DateTime?>[];
  for (var i = 0; i < startOffset; i++) {
    cells.add(null);
  }
  for (var d = 1; d <= daysInMonth; d++) {
    cells.add(DateTime(month.year, month.month, d));
  }
  while (cells.length % 7 != 0) {
    cells.add(null);
  }

  final weeks = <List<DateTime?>>[];
  for (var i = 0; i < cells.length; i += 7) {
    weeks.add(cells.sublist(i, i + 5));
  }
  while (weeks.isNotEmpty && weeks.last.every((d) => d == null)) {
    weeks.removeLast();
  }
  return weeks;
}

class AttendanceCalendarGrid extends StatelessWidget {
  const AttendanceCalendarGrid({
    super.key,
    required this.month,
    required this.statusByDate,
    this.onDayTap,
    this.editable = false,
  });
```

```
  final DateTime month;

  final Map<DateTime, AttendanceStatus> statusByDate;
  final ValueChanged<DateTime>? onDayTap;
  final bool editable;

  DateTime _dayKey(DateTime d) => DateTime(d.year, d.month, d.day);

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    final weeks = buildMonthWeekRows(month);
    final labels = [
      l10n.attendanceWeekdayMon,
      l10n.attendanceWeekdayTue,
      l10n.attendanceWeekdayWed,
      l10n.attendanceWeekdayThu,
      l10n.attendanceWeekdayFri,
    ];

    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Row(
          children: [
            for (final label in labels)
              Expanded(
                child: Center(
                  child: Text(
                    label,
                    style: Theme.of(context).textTheme.labelSmall?.copyWith(
                      fontWeight: FontWeight.w700,
                      color: AppColors.inkAlpha(context, 0.45),
                    ),
                  ),
                ),
              ),
          ],
        ),
        const SizedBox(height: 8),
        for (final week in weeks) ...[
          Row(
            children: [
              for (final day in week)
                Expanded(
                  child: Padding(
                    padding: const EdgeInsets.all(3),
                    child: _DayCell(
                      day: day,
                      status: day == null ? null : statusByDate[_dayKey(day)],
                      editable:
                          editable &&
                          day != null &&
                          isAttendanceDateAllowed(day),
                      isFuture: day != null && !isAttendanceDateAllowed(day),
                      onTap:
                          day == null ||
                              onDayTap == null ||
                              !isAttendanceDateAllowed(day)
                          ? null
                          : () => onDayTap!(day),
                    ),
                  ),
                ),
            ],
          ),
        ],
      ],
    );
  }
}

class _DayCell extends StatelessWidget {
```

```
const _DayCell({
  required this.day,
  required this.status,
  required this.editable,
  this.isFuture = false,
  this.onTap,
});

final DateTime? day;
final AttendanceStatus? status;
final bool editable;
final bool isFuture;
final VoidCallback? onTap;

@override
Widget build(BuildContext context) {
  if (day == null) {
    return AspectRatio(
      aspectRatio: 1,
      child: DecoratedBox(
        decoration: BoxDecoration(
          color: AppColors.inkAlpha(context, 0.03),
          borderRadius: BorderRadius.circular(10),
        ),
      ),
    );
  }

  final color = attendanceStatusColor(status, context: context);
  final hasStatus = status != null;
  final today = DateTime.now();
  final isToday =
      day!.year == today.year &&
      day!.month == today.month &&
      day!.day == today.day;

  final bg = isFuture
      ? AppColors.inkAlpha(context, 0.04)
      : hasStatus
      ? color.withValues(alpha: 0.92)
      : AppColors.inkAlpha(context, 0.06);

  return AspectRatio(
    aspectRatio: 1,
    child: Material(
      color: bg,
      borderRadius: BorderRadius.circular(10),
      child: InkWell(
        onTap: editable ? onTap : null,
        borderRadius: BorderRadius.circular(10),
        child: DecoratedBox(
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(10),
            border: isToday
                ? Border.all(color: AppColors.accent(context), width: 1.5)
                : null,
          ),
          child: Center(
            child: Text(
              '${day!.day}',
              style: TextStyle(
                fontWeight: FontWeight.w700,
                fontSize: 12,
                color: isFuture
                    ? AppColors.inkAlpha(context, 0.22)
                    : hasStatus
                    ? Colors.white
                    : AppColors.inkAlpha(context, 0.45),
              ),
            ),
          ),
        ),
      ),
    ),
```

```
      ),
    );
  }
}

class AttendanceMonthHeader extends StatelessWidget {
  const AttendanceMonthHeader({
    super.key,
    required this.month,
    required this.onPrev,
    required this.onNext,
  });

  final DateTime month;
  final VoidCallback onPrev;
  final VoidCallback onNext;

  @override
  Widget build(BuildContext context) {
    final label = DateFormat.yMMMM(
      Localizations.localeOf(context).toString(),
    ).format(month);

    return Row(
      children: [
        IconButton(
          onPressed: onPrev,
          icon: const Icon(Icons.chevron_left_rounded),
        ),
        Expanded(
          child: Text(
            label,
            textAlign: TextAlign.center,
            style: Theme.of(
              context,
            ).textTheme.titleMedium?.copyWith(fontWeight: FontWeight.w800),
          ),
        ),
        IconButton(
          onPressed: onNext,
          icon: const Icon(Icons.chevron_right_rounded),
        ),
      ],
    );
  }
}

class AttendanceStatsBar extends StatelessWidget {
  const AttendanceStatsBar({super.key, required this.stats});

  final AttendanceStats stats;

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return Row(
      children: [
        Expanded(
          child: _StatChip(
            label: l10n.attendancePercentLabel,
            value: '${stats.presentPercent.toStringAsFixed(0)}%',
            color: AppColors.accent(context),
          ),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: _StatChip(
            label: l10n.attendancePresentWord,
            value: '${stats.present}',
            color: AppColors.successGreen,
          ),
        ),
        const SizedBox(width: 8),
```

```
        Expanded(
          child: _StatChip(
            label: l10n.attendanceAbsentWord,
            value: '${stats.absent}',
            color: AppColors.errorRed,
          ),
        ),
        const SizedBox(width: 8),
        Expanded(
          child: _StatChip(
            label: l10n.attendanceLateWord,
            value: '${stats.late}',
            color: AppColors.warmAmber,
          ),
        ),
      ],
    );
  }
}

class _StatChip extends StatelessWidget {
  const _StatChip({
    required this.label,
    required this.value,
    required this.color,
  });

  final String label;
  final String value;
  final Color color;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 10),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.1),
        borderRadius: BorderRadius.circular(12),
      ),
      child: Column(
        children: [
          Text(
            value,
            style: TextStyle(
              fontWeight: FontWeight.w800,
              fontSize: 16,
              color: color,
            ),
          ),
          const SizedBox(height: 2),
          Text(
            label,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: TextStyle(
              fontSize: 10,
              fontWeight: FontWeight.w600,
              color: AppColors.inkAlpha(context, 0.55),
            ),
          ),
        ],
      ),
    );
  }
}

class AttendanceLegendRow extends StatelessWidget {
  const AttendanceLegendRow({super.key});

  @override
  Widget build(BuildContext context) {
    final l10n = context.l10n;
    return Wrap(
```

```
      spacing: 12,
      runSpacing: 8,
      children: [
        _Legend(
          color: AppColors.successGreen,
          label: l10n.attendancePresentWord,
        ),
        _Legend(color: AppColors.warmAmber, label: l10n.attendanceLateWord),
        _Legend(color: AppColors.errorRed, label: l10n.attendanceAbsentWord),
        _Legend(
          color: const Color(0xFF5B8DB8),
          label: l10n.attendanceExcusedWord,
        ),
        _Legend(
          color: AppColors.inkAlpha(context, 0.12),
          label: l10n.attendanceNoLesson,
        ),
      ],
    );
  }
}

class _Legend extends StatelessWidget {
  const _Legend({required this.color, required this.label});
  final Color color;
  final String label;

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        Container(
          width: 12,
          height: 12,
          decoration: BoxDecoration(
            color: color,
            borderRadius: BorderRadius.circular(3),
          ),
        ),
        const SizedBox(width: 6),
        Text(
          label,
          style: TextStyle(
            fontSize: 12,
            fontWeight: FontWeight.w600,
            color: AppColors.inkAlpha(context, 0.6),
          ),
        ),
      ],
    );
  }
}

AttendanceStatus nextAttendanceStatus(AttendanceStatus? current) {
  return switch (current) {
    null => AttendanceStatus.present,
    AttendanceStatus.present => AttendanceStatus.late,
    AttendanceStatus.late => AttendanceStatus.absent,
    AttendanceStatus.absent => AttendanceStatus.excused,
    AttendanceStatus.excused => AttendanceStatus.present,
  };
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/backgrounds/nature_paper_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type NaturePaperBackdrop
- Type _NaturePaperBackdropState
- Type _DayScene
- Type _DaySceneState
- Type _NightScene
- Type _NightSceneState
- Type _NightSkyPainter
- Type _PaperGrainPainter
- Type _TransitionLeaves
- Type _CloudSpec
- Type _LeafSpec

**lib/components/backgrounds/nature_paper_backdrop.dart**

```
import 'dart:math' as math;

import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

const _kAssetDir = 'assets/images/nature-paper';

class NaturePaperBackdrop extends StatefulWidget {
  const NaturePaperBackdrop({super.key, required this.child});

  final Widget child;

  @override
  State<NaturePaperBackdrop> createState() => _NaturePaperBackdropState();
}

class _NaturePaperBackdropState extends State<NaturePaperBackdrop>
    with SingleTickerProviderStateMixin {
  late final AnimationController _transition;
  bool? _wasNightScene;

  @override
  void initState() {
    super.initState();
    _transition = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1700),
    );
  }

  @override
  void dispose() {
    _transition.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    final isNature = tokens?.isNaturePaper ?? false;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    final bool? nightScene = isNature ? isDark : null;
    if (nightScene != null &&
        _wasNightScene != null &&
        nightScene != _wasNightScene) {
```

```
      _transition.forward(from: 0);
    }
    _wasNightScene = nightScene;

    return Stack(
      children: [
        Positioned.fill(
          child: RepaintBoundary(
            child: ExcludeSemantics(
              child: isNature
                  ? AnimatedSwitcher(
                      duration: const Duration(milliseconds: 650),
                      switchInCurve: Curves.easeOut,
                      switchOutCurve: Curves.easeIn,
                      transitionBuilder: _fadeWithoutChildKey,
                      child: isDark
                          ? const _NightScene(key: ValueKey('nature-night'))
                          : const _DayScene(key: ValueKey('nature-day')),
                    )
                  : const SizedBox.shrink(),
            ),
          ),
        ),
        widget.child,
        Positioned.fill(
          child: IgnorePointer(
            child: _TransitionLeaves(animation: _transition),
          ),
        ),
      ],
    );
  }
}

Widget _fadeWithoutChildKey(Widget child, Animation<double> animation) {
  return FadeTransition(opacity: animation, child: child);
}

class _DayScene extends StatefulWidget {
  const _DayScene({super.key});

  @override
  State<_DayScene> createState() => _DaySceneState();
}

class _DaySceneState extends State<_DayScene>
    with SingleTickerProviderStateMixin {
  late final AnimationController _drift;

  static const _clouds = [
    _CloudSpec('white-cloud.png', y: 0.07, size: 0.30, speed: 1.7, phase: 0.1),
    _CloudSpec('flat-cloud.png', y: 0.16, size: 0.22, speed: 1.15, phase: 0.55),
    _CloudSpec('gray-cloud.png', y: 0.26, size: 0.17, speed: 1.4, phase: 0.85),
  ];

  static const _leaves = [
    _LeafSpec('beige-leaf.png', x: 0.12, size: 26, speed: 0.9, phase: 0.0),
    _LeafSpec('tan-seed.png', x: 0.32, size: 18, speed: 1.3, phase: 0.35),
    _LeafSpec('white-daisy.png', x: 0.55, size: 22, speed: 0.7, phase: 0.6),
    _LeafSpec('beige-leaf.png', x: 0.74, size: 20, speed: 1.1, phase: 0.2),
    _LeafSpec('salmon-flower.png', x: 0.88, size: 24, speed: 0.8, phase: 0.8),
  ];

  @override
  void initState() {
    super.initState();
    _drift = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 36),
    )..repeat();
  }

  @override
```

```
void dispose() {
  _drift.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return LayoutBuilder(
    builder: (context, constraints) {
      final w = constraints.maxWidth;
      final h = constraints.maxHeight;
      return Stack(
        fit: StackFit.expand,
        children: [
          const DecoratedBox(
            decoration: BoxDecoration(
              gradient: LinearGradient(
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
                stops: [0.0, 0.52, 1.0],
                colors: [
                  NaturePaperColors.cream,
                  Color(0xFFEFE4CD),
                  NaturePaperColors.paleBlue,
                ],
              ),
            ),
          ),
          Positioned(
            top: -h * 0.12,
            left: -w * 0.15,
            child: Container(
              width: w * 0.9,
              height: w * 0.9,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: RadialGradient(
                  colors: [
                    NaturePaperColors.sunGlow.withValues(alpha: 0.30),
                    NaturePaperColors.sunGlow.withValues(alpha: 0.0),
                  ],
                ),
              ),
            ),
          ),
          const CustomPaint(painter: _PaperGrainPainter(isDark: false)),
          Positioned(
            left: 0,
            right: 0,
            bottom: 0,
            child: Image.asset(
              '$_kAssetDir/landscape-hills-footer.png',
              width: w,
              fit: BoxFit.fitWidth,
              opacity: const AlwaysStoppedAnimation(0.4),
              filterQuality: FilterQuality.medium,
            ),
          ),
          AnimatedBuilder(
            animation: _drift,
            builder: (context, _) {
              final t = _drift.value;
              return Stack(
                children: [
                  for (final c in _clouds)
                    Positioned(
                      left: (((t * c.speed + c.phase) % 1.2) - 0.15) * w,
                      top: c.y * h,
                      child: Image.asset(
                        '$_kAssetDir/${c.asset}',
                        width: (w * c.size).clamp(70.0, 220.0),
                        opacity: const AlwaysStoppedAnimation(0.55),
                        filterQuality: FilterQuality.medium,
```

```
                      ),
                    ),
                  for (final leaf in _leaves)
                    Positioned(
                      left:
                          leaf.x * w +
                          math.sin(t * math.pi * 8 + leaf.phase * 7) * 18,
                      top:
                          (((t * leaf.speed * 2 + leaf.phase) % 1.1) - 0.05) *
                          h,
                      child: Transform.rotate(
                        angle:
                            math.sin(t * math.pi * 6 + leaf.phase * 9) * 0.5,
                        child: Image.asset(
                          '$_kAssetDir/${leaf.asset}',
                          width: leaf.size,
                          opacity: const AlwaysStoppedAnimation(0.45),
                          filterQuality: FilterQuality.medium,
                        ),
                      ),
                    ),
                ],
              );
            },
          ),
        ],
      );
    },
  );
  }
}

class _NightScene extends StatefulWidget {
  const _NightScene({super.key});

  @override
  State<_NightScene> createState() => _NightSceneState();
}

class _NightSceneState extends State<_NightScene>
    with SingleTickerProviderStateMixin {
  late final AnimationController _drift;

  @override
  void initState() {
    super.initState();
    _drift = AnimationController(
      vsync: this,
      duration: const Duration(seconds: 40),
    )..repeat();
  }

  @override
  void dispose() {
    _drift.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final w = constraints.maxWidth;
        final h = constraints.maxHeight;
        return Stack(
          fit: StackFit.expand,
          children: [
            const DecoratedBox(
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  begin: Alignment.topCenter,
                  end: Alignment.bottomCenter,
                  stops: [0.0, 1.0],
```

```
                  colors: [
                    NaturePaperColors.midnight,
                    NaturePaperColors.forestNight,
                  ],
                ),
              ),
            ),
            Positioned(
              top: -h * 0.10,
              right: -w * 0.20,
              child: Container(
                width: w * 0.85,
                height: w * 0.85,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      NaturePaperColors.moonGlow.withValues(alpha: 0.16),
                      NaturePaperColors.moonGlow.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
            const CustomPaint(painter: _PaperGrainPainter(isDark: true)),
            AnimatedBuilder(
              animation: _drift,
              builder: (context, _) =>
                  CustomPaint(painter: _NightSkyPainter(t: _drift.value)),
            ),
            Positioned(
              left: 0,
              right: 0,
              bottom: h * 0.03,
              child: ColorFiltered(
                colorFilter: const ColorFilter.mode(
                  Color(0xFF1C2822),
                  BlendMode.srcIn,
                ),
                child: Image.asset(
                  '$_kAssetDir/landscape-hills-footer.png',
                  width: w,
                  fit: BoxFit.fitWidth,
                  opacity: const AlwaysStoppedAnimation(0.9),
                  filterQuality: FilterQuality.medium,
                ),
              ),
            ),
            Positioned(
              left: 0,
              right: 0,
              bottom: 0,
              child: ColorFiltered(
                colorFilter: const ColorFilter.mode(
                  Color(0xFF121B16),
                  BlendMode.srcIn,
                ),
                child: Image.asset(
                  '$_kAssetDir/landscape-hills-footer.png',
                  width: w,
                  fit: BoxFit.fitWidth,
                  filterQuality: FilterQuality.medium,
                ),
              ),
            ),
          ],
        );
      },
    );
  }
}

class _NightSkyPainter extends CustomPainter {
```

```
  const _NightSkyPainter({required this.t});

  final double t;

  static const _starCount = 70;
  static const _fireflyCount = 9;

  @override
  void paint(Canvas canvas, Size size) {
    final rng = math.Random(7);
    final starPaint = Paint();
    for (var i = 0; i < _starCount; i++) {
      final x = rng.nextDouble() * size.width;
      final y = rng.nextDouble() * size.height * 0.55;
      final r = 0.5 + rng.nextDouble() * 0.9;
      final baseAlpha = 0.10 + rng.nextDouble() * 0.25;
      final twinkle = i % 3 == 0
          ? 0.5 + 0.5 * math.sin(t * math.pi * 10 + i * 1.7)
          : 1.0;
      starPaint.color = NaturePaperColors.moonBeige.withValues(
        alpha: baseAlpha * twinkle,
      );
      canvas.drawCircle(Offset(x, y), r, starPaint);
    }

    final glowPaint = Paint()
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 5);
    final corePaint = Paint();
    final fly = math.Random(23);
    for (var i = 0; i < _fireflyCount; i++) {
      final baseX = fly.nextDouble();
      final baseY = 0.35 + fly.nextDouble() * 0.55;
      final phase = fly.nextDouble() * math.pi * 2;
      final x =
          (baseX +
              0.05 * math.sin(t * math.pi * 4 + phase) +
              0.02 * math.sin(t * math.pi * 14 + phase * 3)) *
          size.width;
      final y =
          (baseY + 0.03 * math.cos(t * math.pi * 6 + phase * 2)) * size.height;
      final pulse = (0.5 + 0.5 * math.sin(t * math.pi * 8 + phase * 5)).clamp(
        0.0,
        1.0,
      );
      if (pulse < 0.15) continue;
      glowPaint.color = NaturePaperColors.amber.withValues(alpha: 0.35 * pulse);
      corePaint.color = const Color(0xFFF4D9A0).withValues(alpha: 0.85 * pulse);
      canvas.drawCircle(Offset(x, y), 5, glowPaint);
      canvas.drawCircle(Offset(x, y), 1.6, corePaint);
    }
  }

  @override
  bool shouldRepaint(_NightSkyPainter oldDelegate) => oldDelegate.t != t;
}

class _PaperGrainPainter extends CustomPainter {
  const _PaperGrainPainter({required this.isDark});

  final bool isDark;

  @override
  void paint(Canvas canvas, Size size) {
    final rng = math.Random(42);
    final paint = Paint()
      ..color = isDark
          ? NaturePaperColors.moonBeige.withValues(alpha: 0.025)
          : NaturePaperColors.warmBrown.withValues(alpha: 0.035);
    final count = (size.width * size.height / 3200).round().clamp(120, 700);
    for (var i = 0; i < count; i++) {
      canvas.drawCircle(
        Offset(rng.nextDouble() * size.width, rng.nextDouble() * size.height),
        0.6 + rng.nextDouble() * 0.7,
```

```
        paint,
      );
    }
  }

  @override
  bool shouldRepaint(_PaperGrainPainter oldDelegate) =>
      oldDelegate.isDark != isDark;
}

class _TransitionLeaves extends StatelessWidget {
  const _TransitionLeaves({required this.animation});

  final Animation<double> animation;

  static const _sprites = [
    _LeafSpec('beige-leaf.png', x: 0.08, size: 30, speed: 1.0, phase: 0.0),
    _LeafSpec('white-daisy.png', x: 0.25, size: 24, speed: 1.3, phase: 0.3),
    _LeafSpec('tan-seed.png', x: 0.42, size: 20, speed: 0.9, phase: 0.6),
    _LeafSpec('beige-leaf.png', x: 0.58, size: 26, speed: 1.2, phase: 0.15),
    _LeafSpec('salmon-flower.png', x: 0.72, size: 24, speed: 1.0, phase: 0.45),
    _LeafSpec('beige-leaf.png', x: 0.88, size: 22, speed: 1.4, phase: 0.75),
  ];

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
```

**lib/components/backgrounds/nature_paper_backdrop.dart — continued (2/2)**

```
      animation: animation,
      builder: (context, _) {
        final t = animation.value;
        if (t == 0 || animation.isCompleted) return const SizedBox.shrink();
        final size = MediaQuery.sizeOf(context);
        final opacity = math.sin(t * math.pi).clamp(0.0, 1.0) * 0.85;
        return Stack(
          children: [
            for (final leaf in _sprites)
              Positioned(
                left:
                    leaf.x * size.width +
                    math.sin(t * math.pi * 3 + leaf.phase * 8) * 26,
                top:
                    (t * leaf.speed * 1.1 - 0.08 + leaf.phase * 0.1) *
                    size.height,
                child: Opacity(
                  opacity: opacity,
                  child: Transform.rotate(
                    angle: t * math.pi * 2 * leaf.speed + leaf.phase * 5,
                    child: Image.asset(
                      '$_kAssetDir/${leaf.asset}',
                      width: leaf.size,
                      filterQuality: FilterQuality.medium,
                    ),
                  ),
                ),
              ),
          ],
        );
      },
    );
  }
}

class _CloudSpec {
  const _CloudSpec(
    this.asset, {
    required this.y,
    required this.size,
    required this.speed,
    required this.phase,
  });
```

```
  final String asset;
  final double y;
  final double size;
  final double speed;
  final double phase;
}

class _LeafSpec {
  const _LeafSpec(
    this.asset, {
    required this.x,
    required this.size,
    required this.speed,
    required this.phase,
  });

  final String asset;
  final double x;
  final double size;
  final double speed;
  final double phase;
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/backgrounds/soft_neumorphism_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type SoftNeumorphismBackdrop
- Type _MorningAmbient
- Type _NightAmbient

**lib/components/backgrounds/soft_neumorphism_backdrop.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/soft_neumorphism_colors.dart';

class SoftNeumorphismBackdrop extends StatelessWidget {
  const SoftNeumorphismBackdrop({super.key, required this.child});

  final Widget child;

  @override
  Widget build(BuildContext context) {
    final isNeu = AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    return Stack(
      children: [
        Positioned.fill(
          child: RepaintBoundary(
            child: ExcludeSemantics(
              child: isNeu
                  ? AnimatedSwitcher(
                      duration: const Duration(milliseconds: 550),
                      switchInCurve: Curves.easeOut,
                      switchOutCurve: Curves.easeIn,
                      transitionBuilder: _fadeWithoutChildKey,
                      child: isDark
                          ? const _NightAmbient(key: ValueKey('neu-night'))
                          : const _MorningAmbient(key: ValueKey('neu-day')),
                    )
                  : const SizedBox.shrink(),
            ),
          ),
        ),
        child,
      ],
    );
  }
}

Widget _fadeWithoutChildKey(Widget child, Animation<double> animation) {
  return FadeTransition(opacity: animation, child: child);
}

class _MorningAmbient extends StatelessWidget {
  const _MorningAmbient({super.key});

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topLeft,
          end: Alignment.bottomRight,
          colors: [
            SoftNeumorphismColors.morningBg,
            SoftNeumorphismColors.morningBgCool,
            Color(0xFFECE8E0),
          ],
          stops: [0.0, 0.55, 1.0],
        ),
```

```
      ),
      child: Stack(
        children: [
          Positioned(
            top: -80,
            right: -60,
            child: IgnorePointer(
              child: Container(
                width: 280,
                height: 280,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.sage.withValues(alpha: 0.10),
                      SoftNeumorphismColors.sage.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
          Positioned(
            bottom: -40,
            left: -40,
            child: IgnorePointer(
              child: Container(
                width: 240,
                height: 240,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.amber.withValues(alpha: 0.08),
                      SoftNeumorphismColors.amber.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _NightAmbient extends StatelessWidget {
  const _NightAmbient({super.key});

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [
            SoftNeumorphismColors.nightBgCool,
            SoftNeumorphismColors.nightBg,
            Color(0xFF1C2220),
          ],
          stops: [0.0, 0.5, 1.0],
        ),
      ),
      child: Stack(
        children: [
          Positioned(
            top: -100,
            left: -40,
            child: IgnorePointer(
              child: Container(
                width: 300,
```

```
                height: 300,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.emerald.withValues(alpha: 0.09),
                      SoftNeumorphismColors.emerald.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
          Positioned(
            bottom: -60,
            right: -50,
            child: IgnorePointer(
              child: Container(
                width: 260,
                height: 260,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  gradient: RadialGradient(
                    colors: [
                      SoftNeumorphismColors.amber.withValues(alpha: 0.06),
                      SoftNeumorphismColors.amber.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

**WARNING — Reuse rule**
If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/brand/app_logo.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type AppLogo
- Type AppLogoImage

**lib/components/brand/app_logo.dart**

```
import 'package:flutter/material.dart';

class AppLogo {
  static const asset = 'assets/images/logo/logo.jpeg';
}

class AppLogoImage extends StatelessWidget {
  const AppLogoImage({
    super.key,
    this.size = 40,
    this.borderRadius,
    this.fit = BoxFit.cover,
  });

  final double size;
  final BorderRadius? borderRadius;
  final BoxFit fit;

  @override
  Widget build(BuildContext context) {
    final radius = borderRadius ?? BorderRadius.circular(size * 0.22);
    return ClipRRect(
      borderRadius: radius,
      child: Image.asset(
        AppLogo.asset,
        width: size,
        height: size,
        fit: fit,
        filterQuality: FilterQuality.medium,
        semanticLabel: 'MU Study',
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/buttons/primary_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type PrimaryButton

**lib/components/buttons/primary_button.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

class PrimaryButton extends StatelessWidget {
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.loading = false,
    this.icon,
    this.backgroundColor,
    this.foregroundColor,
  });

  final String label;
  final VoidCallback? onPressed;
  final bool loading;
  final IconData? icon;
  final Color? backgroundColor;
  final Color? foregroundColor;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final nature = isNaturePaper(context);
    final bg =
        backgroundColor ??
        (nature ? AppColors.accent(context) : AppColors.coffeeBrown);
    final fg =
        foregroundColor ??
        (nature ? theme.colorScheme.onPrimary : Colors.white);

    final ButtonStyle style;
    if (nature && backgroundColor == null && foregroundColor == null) {
      style = (theme.elevatedButtonTheme.style ?? const ButtonStyle()).merge(
        ElevatedButton.styleFrom(minimumSize: const Size.fromHeight(56)),
      );
    } else {
      style = ElevatedButton.styleFrom(
        backgroundColor: bg,
        foregroundColor: fg,
        minimumSize: const Size.fromHeight(56),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(nature ? 18 : 14),
        ),
        elevation: nature ? 3 : 0,
        shadowColor: nature
            ? NaturePaperColors.warmBrown.withValues(alpha: 0.45)
            : null,
      );
    }

    return ElevatedButton(
      onPressed: loading ? null : onPressed,
      style: style,
      child: loading
          ? SizedBox(
              height: 24,
              width: 24,
              child: CircularProgressIndicator(strokeWidth: 2.5, color: fg),
            )
```

```
            : Row(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  if (icon != null) ...[
                    PaperGlyph(icon: icon!, size: 20, color: fg),
                    const SizedBox(width: 10),
                  ],
                  Text(
                    label,
                    style: theme.textTheme.labelLarge?.copyWith(
                      fontWeight: FontWeight.w700,
                      color: fg,
                      fontSize: 16,
                    ),
                  ),
                ],
              ),
      );
    }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/cards/premium_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type PremiumCard
- Type _PremiumCardState

**lib/components/cards/premium_card.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_animate/flutter_animate.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';

class PremiumCard extends StatefulWidget {
  const PremiumCard({
    super.key,
    required this.child,
    this.onTap,
    this.padding = const EdgeInsets.all(20),
    this.color,
    this.borderColor,
    this.borderRadius,
    this.elevation,
  });

  final Widget child;
  final VoidCallback? onTap;
  final EdgeInsets padding;
  final Color? color;
  final Color? borderColor;

  final double? borderRadius;

  final NeuElevation? elevation;

  @override
  State<PremiumCard> createState() => _PremiumCardState();
}

class _PremiumCardState extends State<PremiumCard>
    with SingleTickerProviderStateMixin {
  late final AnimationController _hoverController;
  late final Animation<double> _scaleAnimation;
  late final Animation<double> _elevationAnimation;
  bool _pressed = false;

  @override
  void initState() {
    super.initState();
    _hoverController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 200),
    );
    _scaleAnimation = Tween<double>(begin: 1.0, end: 1.01).animate(
      CurvedAnimation(parent: _hoverController, curve: Curves.easeOutCubic),
    );
    _elevationAnimation = Tween<double>(begin: 4.0, end: 12.0).animate(
      CurvedAnimation(parent: _hoverController, curve: Curves.easeOutCubic),
    );
  }

  @override
  void dispose() {
    _hoverController.dispose();
    super.dispose();
  }
```

```
@override
Widget build(BuildContext context) {
  final scheme = Theme.of(context).colorScheme;
  final isDark = Theme.of(context).brightness == Brightness.dark;
  final tokens = AppStyleTokens.maybeOf(context);
  final isNature = tokens?.isNaturePaper ?? false;
  final isNeu = tokens?.isSoftNeumorphism ?? false;
  final isMono = tokens?.isMonochromeWorkspace ?? false;
  final radius = widget.borderRadius ?? tokens?.cardRadius ?? 16.0;

  return AnimatedBuilder(
        animation: _hoverController,
        builder: (context, child) {
          final hover = _hoverController.value;
          final cardColor =
              widget.color ??
              tokens?.cardSurface ??
              (isDark ? AppColors.darkSurfaceElevated : scheme.surface);
          final borderColor =
              widget.borderColor ??
              tokens?.cardBorder ??
              (isDark
                  ? AppColors.darkBorder
                  : AppColors.inkAlpha(context, 0.06));
          final shadowColor = isDark
              ? Colors.black.withValues(alpha: 0.55 + hover * 0.25)
              : scheme.onSurface.withValues(alpha: 0.02 + hover * 0.03);

          final List<BoxShadow> shadows;
          final bool showBorder;
          Color fillColor = cardColor;

          if (isNeu) {
            final baseElev = widget.elevation ?? NeuElevation.raised;
            final elev = _pressed
                ? NeuElevation.pressed
                : (hover > 0.5 ? NeuElevation.floating : baseElev);
            fillColor =
                widget.color ?? Neumorphic.surface(context, elevation: elev);
            shadows = Neumorphic.shadows(
              context,
              elevation: elev,
              intensity: 1.15 + hover * 0.2,
            );
            showBorder = false;
          } else if (isMono) {
            shadows = const [];
            showBorder = true;
          } else if (isNature && !isDark) {
            fillColor = widget.color ?? NaturePaperColors.dayCard;
            shadows = [
              BoxShadow(
                color: NaturePaperColors.warmBrown.withValues(
                  alpha: 0.12 + hover * 0.06,
                ),
                blurRadius: 16 + hover * 8,
                offset: Offset(0, 6 + hover * 2),
              ),
              const BoxShadow(
                color: NaturePaperColors.dayCardEdge,
                offset: Offset(0, 2),
              ),
            ];
            showBorder = true;
          } else if (isNature && isDark) {
            fillColor = widget.color ?? NaturePaperColors.nightCardHigh;
            shadows = [
              BoxShadow(
                color: Colors.black.withValues(alpha: 0.45 + hover * 0.12),
                blurRadius: 18,
                offset: const Offset(0, 8),
              ),
              BoxShadow(
```

```
      color: NaturePaperColors.moonGlow.withValues(
        alpha: 0.05 + hover * 0.08,
      ),
      blurRadius: 20 + hover * 10,
    ),
  ];
  showBorder = true;
} else {
  shadows = [
    BoxShadow(
      color: shadowColor,
      blurRadius: _elevationAnimation.value + (isDark ? 6 : 0),
      offset: Offset(0, _elevationAnimation.value / 2),
    ),
  ];
  showBorder = true;
}

final Color hoverBorder;
if (isMono) {
  hoverBorder = hover > 0
      ? AppColors.inkAlpha(context, isDark ? 0.45 : 0.28)
      : borderColor;
} else if (isNature) {
  hoverBorder = isDark
      ? NaturePaperColors.moonGlow.withValues(
          alpha: 0.30 + hover * 0.18,
        )
      : NaturePaperColors.warmBrown.withValues(
          alpha: 0.30 + hover * 0.10,
        );
} else if (isDark && hover > 0 && !isNeu) {
  hoverBorder = AppColors.mutedGold.withValues(
    alpha: 0.35 + hover * 0.25,
  );
} else {
  hoverBorder = borderColor;
}

final scale = isMono
    ? 1.0
    : (isNeu
          ? (_pressed ? 0.985 : (1.0 + hover * 0.012))
          : _scaleAnimation.value);

return Transform.scale(
  scale: scale,
  child: AnimatedContainer(
    duration: const Duration(milliseconds: 160),
    curve: Curves.easeOutCubic,
    margin: const EdgeInsets.symmetric(vertical: 8),
    decoration: BoxDecoration(
      color: fillColor,
      borderRadius: BorderRadius.circular(radius),
      border: showBorder
          ? Border.all(
              color: hoverBorder,
              width: isNature ? 1.4 : 1.0,
            )
          : null,
      boxShadow: shadows,
    ),
    child: Material(
      color: Colors.transparent,
      child: InkWell(
        onTap: widget.onTap,
        onTapDown: widget.onTap == null
            ? null
            : (_) => setState(() => _pressed = true),
        onTapUp: widget.onTap == null
            ? null
            : (_) => setState(() => _pressed = false),
        onTapCancel: widget.onTap == null
```

```
                    ? null
                    : () => setState(() => _pressed = false),
                onHover: (isHovered) {
                  if (widget.onTap != null) {
                    if (isHovered) {
                      _hoverController.forward();
                    } else {
                      _hoverController.reverse();
                    }
                  }
                },
                borderRadius: BorderRadius.circular(radius),
                highlightColor: AppColors.accent(
                  context,
                ).withValues(alpha: isDark ? 0.12 : 0.05),
                splashColor: AppColors.accent(
                  context,
                ).withValues(alpha: isDark ? 0.16 : 0.05),
                child: Padding(
                  padding: widget.padding,
                  child: widget.child,
                ),
              ),
            ),
          ),
        ),
      );
    },
  )
  .animate()
  .fadeIn(duration: 400.ms, curve: Curves.easeOutCubic)
  .slideY(
    begin: 0.02,
    end: 0,
    duration: 400.ms,
    curve: Curves.easeOutCubic,
  );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/chat/animated_emoji_message.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type AnimatedEmojiMessage
- Type _AnimatedEmojiMessageState

**lib/components/chat/animated_emoji_message.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:lottie/lottie.dart';

class AnimatedEmojiMessage extends StatefulWidget {
  const AnimatedEmojiMessage({
    super.key,
    required this.assetPath,
    required this.emoji,
    this.size = 144,
  });

  final String assetPath;

  final String emoji;

  final double size;

  @override
  State<AnimatedEmojiMessage> createState() => _AnimatedEmojiMessageState();
}

class _AnimatedEmojiMessageState extends State<AnimatedEmojiMessage>
    with TickerProviderStateMixin {
  LottieComposition? _composition;
  bool _failed = false;
  late final AnimationController _lottieController;
  late final AnimationController _bounceController;
  late final Animation<double> _bounceScale;

  @override
  void initState() {
    super.initState();
    _lottieController = AnimationController(vsync: this);
    _lottieController.addStatusListener(_onLottieStatus);
    _bounceController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 280),
    );
    _bounceScale = TweenSequence<double>([
      TweenSequenceItem(
        tween: Tween(
          begin: 1.0,
          end: 1.14,
        ).chain(CurveTween(curve: Curves.easeOut)),
        weight: 40,
      ),
      TweenSequenceItem(
        tween: Tween(
          begin: 1.14,
          end: 1.0,
        ).chain(CurveTween(curve: Curves.easeInOut)),
        weight: 60,
      ),
    ]).animate(_bounceController);
    _load();
  }

  void _onLottieStatus(AnimationStatus status) {
    if (status != AnimationStatus.completed) return;
    _holdLastVisibleFrame();
```

```
}

void _holdLastVisibleFrame() {
  final composition = _composition;
  if (composition == null || !mounted) return;
  final span = composition.endFrame - composition.startFrame;
  if (span <= 0) return;
  const epsilonFrames = 0.5;
  final hold = ((span - epsilonFrames) / span).clamp(0.0, 0.999);
  if (_lottieController.value != hold) {
    _lottieController.value = hold;
  }
}

@override
void didUpdateWidget(covariant AnimatedEmojiMessage oldWidget) {
  super.didUpdateWidget(oldWidget);
  if (oldWidget.assetPath != widget.assetPath) {
    _composition = null;
    _failed = false;
    _lottieController
      ..stop()
      ..reset();
    _load();
  }
}

Future<void> _load() async {
  try {
    final composition = await AssetLottie(widget.assetPath).load();
    if (!mounted) return;
    setState(() {
      _composition = composition;
      _failed = false;
      _lottieController.duration = composition.duration;
      _lottieController.value = 0;
    });
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (!mounted || _composition == null) return;
      _lottieController.forward(from: 0);
    });
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _failed = true;
      _composition = null;
    });
  }
}

void _replay() {
  if (_composition == null) return;
  HapticFeedback.selectionClick();
  _bounceController.forward(from: 0);
  _lottieController.forward(from: 0);
}

@override
void dispose() {
  _lottieController.removeStatusListener(_onLottieStatus);
  _lottieController.dispose();
  _bounceController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return GestureDetector(
    behavior: HitTestBehavior.opaque,
    onTap: _composition != null ? _replay : null,
    child: SizedBox(
      width: widget.size,
      height: widget.size,
```

```
          child: RepaintBoundary(
            child: ScaleTransition(scale: _bounceScale, child: _buildBody()),
          ),
        ),
      );
    }

    Widget _buildBody() {
      if (_failed) {
        return Center(
          child: Text(
            widget.emoji,
            style: TextStyle(fontSize: widget.size * 0.72, height: 1),
          ),
        );
      }
      final composition = _composition;
      if (composition == null) {
        return const SizedBox.expand();
      }
      return Lottie(
        composition: composition,
        controller: _lottieController,
        width: widget.size,
        height: widget.size,
        fit: BoxFit.contain,
        repeat: false,
      );
    }
  }
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/chat/chat_lottie_emoji.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type ChatLottieEmoji
- Type _ChatLottieEmojiState

**lib/components/chat/chat_lottie_emoji.dart**

```
import 'package:flutter/material.dart';
import 'package:lottie/lottie.dart';
import 'package:mu_study/core/chat/animated_emoji_registry.dart';

class ChatLottieEmoji extends StatefulWidget {
  const ChatLottieEmoji({
    super.key,
    required this.emoji,
    this.size = 22,
    this.playOnTap = true,
    this.autoplay = true,
    this.repeat = false,
  });

  final String emoji;
  final double size;
  final bool playOnTap;
  final bool autoplay;

  final bool repeat;

  @override
  State<ChatLottieEmoji> createState() => _ChatLottieEmojiState();
}

class _ChatLottieEmojiState extends State<ChatLottieEmoji>
    with SingleTickerProviderStateMixin {
  LottieComposition? _composition;
  bool _failed = false;
  late final AnimationController _controller;
  String? _assetPath;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this);
    _controller.addStatusListener(_onStatus);
    _assetPath = animatedEmojiAssetPath(widget.emoji);
    _load();
  }

  @override
  void didUpdateWidget(covariant ChatLottieEmoji oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.emoji != widget.emoji) {
      _composition = null;
      _failed = false;
      _controller
        ..stop()
        ..reset();
      _assetPath = animatedEmojiAssetPath(widget.emoji);
      _load();
    }
  }

  void _onStatus(AnimationStatus status) {
    if (widget.repeat) return;
    if (status != AnimationStatus.completed) return;
    _holdLastVisibleFrame();
  }
```

```
void _holdLastVisibleFrame() {
  final composition = _composition;
  if (composition == null || !mounted) return;
  final span = composition.endFrame - composition.startFrame;
  if (span <= 0) return;
  const epsilonFrames = 0.5;
  final hold = ((span - epsilonFrames) / span).clamp(0.0, 0.999);
  if (_controller.value != hold) _controller.value = hold;
}

Future<void> _load() async {
  final path = _assetPath;
  if (path == null) {
    if (mounted) setState(() => _failed = true);
    return;
  }
  try {
    final composition = await AssetLottie(path).load();
    if (!mounted) return;
    setState(() {
      _composition = composition;
      _failed = false;
      _controller.duration = composition.duration;
      _controller.value = 0;
    });
    if (widget.autoplay) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (!mounted || _composition == null) return;
        _startPlayback();
      });
    } else if (!widget.repeat) {
      _holdLastVisibleFrame();
    }
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _failed = true;
      _composition = null;
    });
  }
}

void _startPlayback() {
  if (_composition == null) return;
  if (widget.repeat) {
    _controller.repeat(min: 0, max: _loopMax);
  } else {
    _controller.forward(from: 0);
  }
}

double get _loopMax {
  final composition = _composition;
  if (composition == null) return 0.999;
  final span = composition.endFrame - composition.startFrame;
  if (span <= 0) return 0.999;
  return ((span - 0.5) / span).clamp(0.0, 0.999);
}

void _replay() => _startPlayback();

@override
void dispose() {
  _controller.removeStatusListener(_onStatus);
  _controller.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final child = SizedBox(
    width: widget.size,
    height: widget.size,
```

```
      child: _buildBody(),
    );
    if (!widget.playOnTap || _composition == null) return child;
    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTap: _replay,
      child: child,
    );
  }

  Widget _buildBody() {
    if (_failed || _assetPath == null) {
      return Center(
        child: Text(
          widget.emoji,
          style: TextStyle(fontSize: widget.size * 0.85, height: 1),
        ),
      );
    }
    final composition = _composition;
    if (composition == null) return const SizedBox.expand();
    return Lottie(
      composition: composition,
      controller: _controller,
      width: widget.size,
      height: widget.size,
      fit: BoxFit.contain,
      repeat: false,
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/chat/chat_media_gallery.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type ChatMediaItem
- Type ChatMediaGalleryPage
- Type _ChatMediaGalleryPageState
- Type _GalleryImagePage
- Type _GalleryImagePageState
- Type _GalleryVideoPage
- Type _GalleryVideoPageState
- Type _ControlsBar
- Type _ControlButton

**lib/components/chat/chat_media_gallery.dart**

```
import 'dart:async';
import 'dart:io';

import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:mu_study/core/services/chat_media_cache_service.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/models/models.dart';
import 'package:video_player/video_player.dart';

const _speedOptions = <double>[0.5, 0.75, 1.0, 1.25, 1.5, 2.0];
const _dismissDistance = 120.0;

enum ChatMediaKind { image, video }

class ChatMediaItem {
  const ChatMediaItem({
    required this.id,
    required this.url,
    required this.kind,
  });

  final String id;
  final String url;
  final ChatMediaKind kind;
}

List<ChatMediaItem> chatMediaItemsFromMessages(Iterable<ChatMessage> messages) {
  final items = <ChatMediaItem>[];
  for (final m in messages) {
    if (m.isDeleted) continue;
    final url = m.mediaUrl;
    if (url == null || url.isEmpty) continue;
    final kind = switch (m.messageType) {
      ChatMessageType.image => ChatMediaKind.image,
      ChatMessageType.video => ChatMediaKind.video,
      _ => null,
    };
    if (kind == null) continue;
    items.add(ChatMediaItem(id: m.id, url: url, kind: kind));
  }
  return items;
}

Future<void> showChatMediaGallery(
  BuildContext context, {
  required List<ChatMediaItem> items,
  required int initialIndex,
```

```
}) {
  assert(items.isNotEmpty);
  final index = initialIndex.clamp(0, items.length - 1);
  return showGeneralDialog<void>(
    context: context,
    barrierDismissible: true,
    barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,
    barrierColor: Colors.black,
    transitionDuration: const Duration(milliseconds: 220),
    pageBuilder: (context, animation, secondaryAnimation) {
      return ChatMediaGalleryPage(items: items, initialIndex: index);
    },
    transitionBuilder: (context, animation, secondaryAnimation, child) {
      return FadeTransition(
        opacity: animation,
        child: ScaleTransition(
          scale: Tween<double>(begin: 0.96, end: 1).animate(
            CurvedAnimation(parent: animation, curve: Curves.easeOutCubic),
          ),
          child: child,
        ),
      );
    },
  );
}

Future<void> showChatVideoPlayer(BuildContext context, {required String url}) {
  return showChatMediaGallery(
    context,
    items: [ChatMediaItem(id: url, url: url, kind: ChatMediaKind.video)],
    initialIndex: 0,
  );
}

class ChatMediaGalleryPage extends StatefulWidget {
  const ChatMediaGalleryPage({
    super.key,
    required this.items,
    required this.initialIndex,
  });

  final List<ChatMediaItem> items;
  final int initialIndex;

  @override
  State<ChatMediaGalleryPage> createState() => _ChatMediaGalleryPageState();
}

class _ChatMediaGalleryPageState extends State<ChatMediaGalleryPage> {
  late final PageController _pageController;
  late int _index;
  double _dragY = 0;
  bool _dragging = false;

  @override
  void initState() {
    super.initState();
    _index = widget.initialIndex;
    _pageController = PageController(initialPage: _index);
    SystemChrome.setEnabledSystemUIMode(SystemUiMode.immersiveSticky);
  }

  @override
  void dispose() {
    _pageController.dispose();
    SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);
    super.dispose();
  }

  void _onVerticalDragUpdate(DragUpdateDetails details) {
    final dy = details.delta.dy;
    setState(() {
      _dragging = true;
```

```
      _dragY = (_dragY + dy).clamp(0.0, 400.0);
    });
  }

  void _onVerticalDragEnd(DragEndDetails details) {
    final fling = details.primaryVelocity ?? 0;
    final shouldClose = _dragY > _dismissDistance || fling > 900;
    if (shouldClose) {
      Navigator.of(context).maybePop();
      return;
    }
    setState(() {
      _dragging = false;
      _dragY = 0;
    });
  }

  void _onVerticalDragCancel() {
    setState(() {
      _dragging = false;
      _dragY = 0;
    });
  }

  @override
  Widget build(BuildContext context) {
    final progress = (_dragY / _dismissDistance).clamp(0.0, 1.0);
    final bgOpacity = (1.0 - progress * 0.55).clamp(0.35, 1.0);
    final scale = (1.0 - progress * 0.08).clamp(0.9, 1.0);

    return Material(
      color: Colors.black.withValues(alpha: bgOpacity),
      child: GestureDetector(
        onVerticalDragUpdate: _onVerticalDragUpdate,
        onVerticalDragEnd: _onVerticalDragEnd,
        onVerticalDragCancel: _onVerticalDragCancel,
        child: AnimatedContainer(
          duration: _dragging
              ? Duration.zero
              : const Duration(milliseconds: 180),
          curve: Curves.easeOutCubic,
          transform: Matrix4.identity()
            ..translateByDouble(0, _dragging ? _dragY : 0, 0, 1)
            ..scaleByDouble(scale, scale, 1, 1),
          transformAlignment: Alignment.center,
          child: SafeArea(
            child: Stack(
              fit: StackFit.expand,
              children: [
                PageView.builder(
                  controller: _pageController,
                  itemCount: widget.items.length,
                  onPageChanged: (i) => setState(() => _index = i),
                  itemBuilder: (context, i) {
                    final item = widget.items[i];
                    final active = i == _index && _dragY < 24;
                    return switch (item.kind) {
                      ChatMediaKind.image => _GalleryImagePage(url: item.url),
                      ChatMediaKind.video => _GalleryVideoPage(
                        url: item.url,
                        isActive: active,
                      ),
                    };
                  },
                ),
                Positioned(
                  top: 4,
                  left: 4,
                  child: IconButton(
                    onPressed: () => Navigator.of(context).maybePop(),
                    icon: const Icon(Icons.close_rounded, color: Colors.white),
                  ),
                ),
```

```
                if (widget.items.length > 1)
                  Positioned(
                    top: 14,
                    right: 16,
                    child: Text(
                      '${_index + 1} / ${widget.items.length}',
                      style: const TextStyle(
                        color: Colors.white70,
                        fontWeight: FontWeight.w600,
                        fontSize: 13,
                      ),
                    ),
                  ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}

class _GalleryImagePage extends StatefulWidget {
  const _GalleryImagePage({required this.url});
  final String url;

  @override
  State<_GalleryImagePage> createState() => _GalleryImagePageState();
}

class _GalleryImagePageState extends State<_GalleryImagePage>
    with AutomaticKeepAliveClientMixin {
  File? _file;
  bool _loading = true;

  @override
  bool get wantKeepAlive => true;

  @override
  void initState() {
    super.initState();
    _resolve();
  }

  Future<void> _resolve() async {
    if (!kIsWeb && ChatMediaCacheService.instance.isSupported) {
      final file = await ChatMediaCacheService.instance.getOrFetch(
        widget.url,
        preferredExtension: ChatMediaCacheService.extensionForUrl(
          widget.url,
          fallback: 'jpg',
        ),
      );
      if (!mounted) return;
      setState(() {
        _file = file;
        _loading = false;
      });
      return;
    }
    if (mounted) setState(() => _loading = false);
  }

  @override
  Widget build(BuildContext context) {
    super.build(context);
    if (_loading) {
      return const Center(
        child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
      );
    }
    final child = _file != null
        ? Image.file(_file!, fit: BoxFit.contain)
```

```
        : CachedNetworkImage(
            imageUrl: widget.url,
            fit: BoxFit.contain,
            placeholder: (_, _) => const Center(
              child: CircularProgressIndicator(
                strokeWidth: 2,
                color: Colors.white,
              ),
            ),
            errorWidget: (_, _, _) => const Icon(
              Icons.broken_image_outlined,
              color: Colors.white54,
              size: 48,
            ),
          );
    return InteractiveViewer(
      minScale: 1,
      maxScale: 4,
      child: Center(child: child),
    );
  }
}

class _GalleryVideoPage extends StatefulWidget {
  const _GalleryVideoPage({required this.url, required this.isActive});

  final String url;
  final bool isActive;

  @override
  State<_GalleryVideoPage> createState() => _GalleryVideoPageState();
}

class _GalleryVideoPageState extends State<_GalleryVideoPage>
    with AutomaticKeepAliveClientMixin {
  VideoPlayerController? _controller;
  bool _ready = false;
  bool _failed = false;
  bool _controlsVisible = true;
  double _speed = 1.0;
  Timer? _hideControls;

  @override
  bool get wantKeepAlive => true;

  @override
  void initState() {
    super.initState();
    _open();
  }

  @override
  void didUpdateWidget(covariant _GalleryVideoPage oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.isActive != widget.isActive) {
      _syncPlayback();
    }
  }

  Future<void> _open() async {
    VideoPlayerController controller;
    if (!kIsWeb && ChatMediaCacheService.instance.isSupported) {
      final file = await ChatMediaCacheService.instance.getOrFetch(
        widget.url,
        preferredExtension: ChatMediaCacheService.extensionForUrl(
          widget.url,
          fallback: 'mp4',
        ),
      );
      if (!mounted) return;
      controller = file != null
          ? VideoPlayerController.file(file)
          : VideoPlayerController.networkUrl(Uri.parse(widget.url));
```

```
    } else {
      controller = VideoPlayerController.networkUrl(Uri.parse(widget.url));
    }

    _controller = controller;
    controller.addListener(_onTick);
    try {
      await controller.initialize();
      await controller.setPlaybackSpeed(_speed);
      if (!mounted) return;
      setState(() => _ready = true);
      await _syncPlayback();
    } catch (_) {
      if (mounted) setState(() => _failed = true);
    }
  }

  Future<void> _syncPlayback() async {
    final c = _controller;
    if (c == null || !c.value.isInitialized) return;
    if (widget.isActive) {
      await c.play();
      _scheduleHideControls();
    } else {
      await c.pause();
      _hideControls?.cancel();
      if (mounted) setState(() => _controlsVisible = true);
    }
  }

  void _onTick() {
    if (mounted) setState(() {});
  }

  void _scheduleHideControls() {
    _hideControls?.cancel();
    if (!(_controller?.value.isPlaying ?? false)) return;
    _hideControls = Timer(const Duration(seconds: 3), () {
      if (mounted && (_controller?.value.isPlaying ?? false)) {
        setState(() => _controlsVisible = false);
      }
    });
  }

  void _toggleControls() {
    setState(() => _controlsVisible = !_controlsVisible);
    if (_controlsVisible) _scheduleHideControls();
  }

  Future<void> _togglePlay() async {
    final c = _controller;
    if (c == null || !c.value.isInitialized) return;
    if (c.value.isPlaying) {
      await c.pause();
      _hideControls?.cancel();
      setState(() => _controlsVisible = true);
    } else {
      await c.play();
      _scheduleHideControls();
    }
  }

  Future<void> _seekBy(Duration delta) async {
    final c = _controller;
    if (c == null || !c.value.isInitialized) return;
    final total = c.value.duration;
    var next = c.value.position + delta;
    if (next < Duration.zero) next = Duration.zero;
    if (next > total) next = total;
    await c.seekTo(next);
    _scheduleHideControls();
  }
```

```
  Future<void> _setSpeed(double speed) async {
    final c = _controller;
    if (c == null) return;
    setState(() => _speed = speed);
    await c.setPlaybackSpeed(speed);
    _scheduleHideControls();
  }

  Future<void> _showSpeedSheet() async {
    _hideControls?.cancel();
    setState(() => _controlsVisible = true);
    final selected = await showModalBottomSheet<double>(
      context: context,
      backgroundColor: const Color(0xFF1C1C1E),
      builder: (ctx) {
        final l10n = context.l10n;
        return SafeArea(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              Padding(
                padding: const EdgeInsets.fromLTRB(20, 16, 20, 8),
                child: Align(
                  alignment: Alignment.centerLeft,
                  child: Text(
                    l10n.chatVideoPlaybackSpeed,
                    style: const TextStyle(
                      color: Colors.white,
                      fontWeight: FontWeight.w700,
                      fontSize: 16,
                    ),
                  ),
                ),
              ),
              for (final speed in _speedOptions)
                ListTile(
                  selected: speed == _speed,
                  selectedColor: Colors.white,
                  selectedTileColor: Colors.white12,
                  title: Text(
                    speed == 1.0
                        ? l10n.chatVideoSpeedNormal
                        : l10n.chatVideoSpeedValue(speed),
                    style: const TextStyle(color: Colors.white),
                  ),
                  trailing: speed == _speed
                      ? const Icon(Icons.check, color: Colors.white)
                      : null,
                  onTap: () => Navigator.pop(ctx, speed),
                ),
              const SizedBox(height: 8),
            ],
          ),
        );
      },
    );
    if (selected != null) await _setSpeed(selected);
    _scheduleHideControls();
  }

  String _fmt(Duration d) {
    final total = d.inSeconds;
    final m = (total ~/ 60).toString().padLeft(2, '0');
    final s = (total % 60).toString().padLeft(2, '0');
```

**lib/components/chat/chat_media_gallery.dart — continued (2/2)**

```
    return '$m:$s';
  }

  @override
  void dispose() {
    _hideControls?.cancel();
```

```
    _controller?.removeListener(_onTick);
    _controller?.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    super.build(context);
    final l10n = context.l10n;
    final c = _controller;

    return Stack(
      fit: StackFit.expand,
      children: [
        if (_failed)
          Center(
            child: Text(
              l10n.chatVideoCouldNotPlay,
              style: const TextStyle(color: Colors.white70),
            ),
          )
        else if (!_ready || c == null)
          const Center(
            child: CircularProgressIndicator(
              strokeWidth: 2,
              color: Colors.white,
            ),
          )
        else
          GestureDetector(
            behavior: HitTestBehavior.opaque,
            onTap: _toggleControls,
            child: Center(
              child: AspectRatio(
                aspectRatio: c.value.aspectRatio == 0
                    ? 16 / 9
                    : c.value.aspectRatio,
                child: VideoPlayer(c),
              ),
            ),
          ),
        if (_ready && c != null)
          Positioned(
            left: 0,
            right: 0,
            bottom: 0,
            child: AnimatedOpacity(
              opacity: _controlsVisible ? 1 : 0,
              duration: const Duration(milliseconds: 180),
              child: IgnorePointer(
                ignoring: !_controlsVisible,
                child: _ControlsBar(
                  controller: c,
                  onSeekBack: () => _seekBy(const Duration(seconds: -10)),
                  onSeekForward: () => _seekBy(const Duration(seconds: 10)),
                  onTogglePlay: _togglePlay,
                  onSpeedTap: _showSpeedSheet,
                  onSeek: (value) async {
                    final total = c.value.duration.inMilliseconds;
                    await c.seekTo(
                      Duration(milliseconds: (value * total).round()),
                    );
                    _scheduleHideControls();
                  },
                  format: _fmt,
                  speedLabel: _speed == 1.0
                      ? l10n.chatVideoSpeedNormal
                      : l10n.chatVideoSpeedValue(_speed),
                  rewindLabel: l10n.chatVideoRewind10,
                  forwardLabel: l10n.chatVideoForward10,
                ),
              ),
            ),
          ),
```

```
          ),
      ],
    );
  }
}

class _ControlsBar extends StatelessWidget {
  const _ControlsBar({
    required this.controller,
    required this.onSeekBack,
    required this.onSeekForward,
    required this.onTogglePlay,
    required this.onSpeedTap,
    required this.onSeek,
    required this.format,
    required this.speedLabel,
    required this.rewindLabel,
    required this.forwardLabel,
  });

  final VideoPlayerController controller;
  final VoidCallback onSeekBack;
  final VoidCallback onSeekForward;
  final VoidCallback onTogglePlay;
  final VoidCallback onSpeedTap;
  final ValueChanged<double> onSeek;
  final String Function(Duration) format;
  final String speedLabel;
  final String rewindLabel;
  final String forwardLabel;

  @override
  Widget build(BuildContext context) {
    final value = controller.value;
    final duration = value.duration;
    final position = value.position;
    final progress = duration.inMilliseconds == 0
        ? 0.0
        : (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);

    return Container(
      padding: const EdgeInsets.fromLTRB(12, 28, 12, 16),
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Colors.transparent, Colors.black87],
        ),
      ),
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Row(
            children: [
              Text(
                format(position),
                style: const TextStyle(color: Colors.white70, fontSize: 12),
              ),
              Expanded(
                child: SliderTheme(
                  data: SliderTheme.of(context).copyWith(
                    trackHeight: 2.5,
                    thumbShape: const RoundSliderThumbShape(
                      enabledThumbRadius: 7,
                    ),
                    overlayShape: const RoundSliderOverlayShape(
                      overlayRadius: 14,
                    ),
                    activeTrackColor: Colors.white,
                    inactiveTrackColor: Colors.white24,
                    thumbColor: Colors.white,
                  ),
                  child: Slider(value: progress, onChanged: onSeek),
```

```
                ),
              ),
              Text(
                format(duration),
                style: const TextStyle(color: Colors.white70, fontSize: 12),
              ),
            ],
          ),
          const SizedBox(height: 4),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              _ControlButton(
                onPressed: onSeekBack,
                icon: Icons.replay_10_rounded,
                label: rewindLabel,
              ),
              IconButton(
                onPressed: onTogglePlay,
                iconSize: 56,
                icon: Icon(
                  value.isPlaying
                      ? Icons.pause_circle_filled_rounded
                      : Icons.play_circle_filled_rounded,
                  color: Colors.white,
                ),
              ),
              _ControlButton(
                onPressed: onSeekForward,
                icon: Icons.forward_10_rounded,
                label: forwardLabel,
              ),
              TextButton(
                onPressed: onSpeedTap,
                style: TextButton.styleFrom(
                  foregroundColor: Colors.white,
                  padding: const EdgeInsets.symmetric(horizontal: 10),
                ),
                child: Text(
                  speedLabel,
                  style: const TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 14,
                  ),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }
}

class _ControlButton extends StatelessWidget {
  const _ControlButton({
    required this.onPressed,
    required this.icon,
    required this.label,
  });

  final VoidCallback onPressed;
  final IconData icon;
  final String label;

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        IconButton(
          onPressed: onPressed,
          icon: Icon(icon, color: Colors.white, size: 32),
```

```
            tooltip: label,
          ),
          Text(
            label,
            style: const TextStyle(color: Colors.white70, fontSize: 11),
          ),
        ],
      );
    }
  }
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/decor/paper_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type PaperAssets
- Type PaperDecor
- Type PaperGlyph
- Type PaperEmptyIllustration
- Type PaperTaped
- Type PaperDivider
- Type PaperScallopedAvatar

**lib/components/decor/paper_decor.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';

class PaperAssets {
  static const _dir = 'assets/images/nature-paper';

  static const avatarSilhouette = '$_dir/avatar-silhouette.png';
  static const beigeLeaf = '$_dir/beige-leaf.png';
  static const bookshelfSingle = '$_dir/bookshelf-single-shelf.png';
  static const bookshelfDouble = '$_dir/bookshelf-two-shelves.png';
  static const brainVerified = '$_dir/brain-verified.png';
  static const browserDashboard = '$_dir/browser-dashboard.png';
  static const calendar = '$_dir/calendar.png';
  static const calendarWithClock = '$_dir/calendar-with-clock.png';
  static const cardboardStar = '$_dir/cardboard-star.png';
  static const chatBubbles = '$_dir/chat-bubbles.png';
  static const clipboardChecklist = '$_dir/clipboard-checklist.png';
  static const clipboardProfile = '$_dir/clipboard-profile.png';
  static const clipboardWithPencil = '$_dir/clipboard-with-pencil.png';
  static const closeButton = '$_dir/close-button.png';
  static const creamBird = '$_dir/cream-bird.png';
  static const creamPaperGreenTape = '$_dir/cream-paper-green-tape.png';
  static const divisionButton = '$_dir/division-button.png';
  static const documentLayout = '$_dir/document-layout.png';
  static const downloadInbox = '$_dir/download-inbox.png';
  static const dustyRoseFlower = '$_dir/dusty-rose-flower.png';
  static const envelopeSuccess = '$_dir/envelope-success.png';
  static const flatCloud = '$_dir/flat-cloud.png';
  static const grayCloud = '$_dir/gray-cloud.png';
  static const groupChat = '$_dir/group-chat.png';
  static const hangingGarland = '$_dir/hanging-garland.png';
  static const homeHouse = '$_dir/home-house.png';
  static const kraftPaperStrip = '$_dir/kraft-paper-strip.png';
  static const landscapeHills = '$_dir/landscape-hills-footer.png';
  static const linedPaperPencil = '$_dir/lined-paper-pencil.png';
  static const magnifyingGlass = '$_dir/magnifying-glass.png';
  static const manilaFolder = '$_dir/manila-folder.png';
  static const marketStall = '$_dir/market-stall.png';
  static const minusButton = '$_dir/minus-button.png';
  static const notificationBell = '$_dir/notification-bell-alert.png';
  static const openBook = '$_dir/open-book.png';
  static const paperFunnel = '$_dir/paper-funnel.png';
  static const puzzlePieces = '$_dir/puzzle-pieces.png';
  static const roundedTanStar = '$_dir/rounded-tan-star.png';
  static const sageScallopedFrame = '$_dir/sage-scalloped-frame.png';
  static const salmonFlower = '$_dir/salmon-flower.png';
  static const settingsGear = '$_dir/settings-gear.png';
  static const singleTree = '$_dir/single-tree.png';
  static const spiralNotebook = '$_dir/spiral-notebook.png';
  static const stickyNotesPair = '$_dir/sticky-notes-pair.png';
  static const stripedWashiTape = '$_dir/striped-washi-tape.png';
```

```
  static const tanNoteGreenTab = '$_dir/tan-note-green-tab.png';
  static const tanSeed = '$_dir/tan-seed.png';
  static const teacherChalkboard = '$_dir/teacher-chalkboard.png';
  static const tealBird = '$_dir/teal-bird.png';
  static const terracottaScallopedFrame =
      '$_dir/terracotta-scalloped-frame.png';
  static const texturedTanStar = '$_dir/textured-tan-star.png';
  static const tornPaperScrap = '$_dir/torn-paper-scrap.png';
  static const twoTrees = '$_dir/two-trees.png';
  static const wallClock = '$_dir/wall-clock.png';
  static const whiteCloud = '$_dir/white-cloud.png';
  static const whiteDaisy = '$_dir/white-daisy.png';
  static const yellowPencil = '$_dir/yellow-pencil.png';
}

bool isNaturePaper(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.isNaturePaper ?? false;

bool isSoftNeumorphism(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;

bool isMonochromeWorkspace(BuildContext context) =>
    AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;

String? paperAssetForIcon(IconData? icon) {
  if (icon == null) return null;
  if (icon == Icons.inbox_outlined || icon == Icons.inbox) {
    return PaperAssets.downloadInbox;
  }
  if (icon == Icons.grade_outlined ||
      icon == Icons.grade ||
      icon == Icons.star_outline ||
      icon == Icons.star ||
      icon == Icons.star_rounded) {
    return PaperAssets.cardboardStar;
  }
  if (icon == Icons.assignment_outlined ||
      icon == Icons.assignment ||
      icon == Icons.task_outlined ||
      icon == Icons.assignment_turned_in_outlined) {
    return PaperAssets.spiralNotebook;
  }
  if (icon == Icons.task_alt_rounded ||
      icon == Icons.edit_outlined ||
      icon == Icons.edit ||
      icon == Icons.add ||
      icon == Icons.add_rounded) {
    return PaperAssets.yellowPencil;
  }
  if (icon == Icons.inventory_2_rounded || icon == Icons.inventory_2_outlined) {
    return PaperAssets.manilaFolder;
  }
  if (icon == Icons.chat_bubble_outline_rounded ||
      icon == Icons.chat_outlined ||
      icon == Icons.chat ||
      icon == Icons.forum_outlined ||
      icon == Icons.chat_bubble_outline) {
    return PaperAssets.chatBubbles;
  }
  if (icon == Icons.campaign_rounded || icon == Icons.campaign_outlined) {
    return PaperAssets.envelopeSuccess;
  }
  if (icon == Icons.event_available_outlined ||
      icon == Icons.event_available_rounded ||
      icon == Icons.event_outlined) {
    return PaperAssets.clipboardChecklist;
  }
  if (icon == Icons.calendar_today_outlined ||
      icon == Icons.calendar_month_outlined ||
      icon == Icons.calendar_today) {
    return PaperAssets.calendar;
  }
  if (icon == Icons.people_outline ||
```

```
        icon == Icons.groups_outlined ||
        icon == Icons.groups ||
        icon == Icons.people_alt_outlined) {
      return PaperAssets.groupChat;
    }
    if (icon == Icons.notifications_outlined ||
        icon == Icons.notifications_none_rounded ||
        icon == Icons.notifications_active_outlined ||
        icon == Icons.notifications) {
      return PaperAssets.notificationBell;
    }
    if (icon == Icons.search || icon == Icons.search_rounded) {
      return PaperAssets.magnifyingGlass;
    }
    if (icon == Icons.menu_book ||
        icon == Icons.subject_outlined ||
        icon == Icons.language_rounded ||
        icon == Icons.language) {
      return PaperAssets.openBook;
    }
    if (icon == Icons.calculate || icon == Icons.calculate_outlined) {
      return PaperAssets.divisionButton;
    }
    if (icon == Icons.badge_outlined ||
        icon == Icons.person_outline ||
        icon == Icons.person ||
        icon == Icons.person_rounded ||
        icon == Icons.account_circle_outlined) {
      return PaperAssets.clipboardProfile;
    }
    if (icon == Icons.settings_outlined ||
        icon == Icons.settings ||
        icon == Icons.tune_rounded) {
      return PaperAssets.settingsGear;
    }
    if (icon == Icons.home_outlined || icon == Icons.home) {
      return PaperAssets.homeHouse;
    }
    if (icon == Icons.dashboard_outlined ||
        icon == Icons.dashboard ||
        icon == Icons.insights_outlined ||
        icon == Icons.analytics_outlined ||
        icon == Icons.analytics_rounded) {
      return PaperAssets.browserDashboard;
    }
    if (icon == Icons.warning_amber_rounded ||
        icon == Icons.cleaning_services_outlined ||
        icon == Icons.error_outline) {
      return PaperAssets.tornPaperScrap;
    }
    if (icon == Icons.schedule ||
        icon == Icons.access_time_rounded ||
        icon == Icons.timer_outlined) {
      return PaperAssets.wallClock;
    }
    if (icon == Icons.extension_outlined) return PaperAssets.puzzlePieces;
    if (icon == Icons.school || icon == Icons.school_outlined) {
      return PaperAssets.teacherChalkboard;
    }
    if (icon == Icons.self_improvement_outlined ||
        icon == Icons.self_improvement) {
      return PaperAssets.openBook;
    }
    if (icon == Icons.folder_outlined || icon == Icons.folder) {
      return PaperAssets.manilaFolder;
    }
    if (icon == Icons.psychology_outlined || icon == Icons.psychology) {
      return PaperAssets.brainVerified;
    }
    return null;
  }

  double paperControlRadius(BuildContext context) {
```

```
  if (isNaturePaper(context)) return 18;
  if (isMonochromeWorkspace(context)) return 6;
  return 14;
}

BoxDecoration paperControlDecoration(
  BuildContext context, {
  Color? color,
  double? radius,
}) {
  final r = radius ?? paperControlRadius(context);
  final nature = isNaturePaper(context);
  return BoxDecoration(
    color: color ?? AppColors.controlSurface(context),
    borderRadius: BorderRadius.circular(r),
    border: Border.all(
      color: nature
          ? AppColors.inkAlpha(context, 0.12)
          : AppColors.inkAlpha(context, 0.08),
    ),
    boxShadow: nature && Theme.of(context).brightness == Brightness.light
        ? [
            BoxShadow(
              color: const Color(0xFF8A6A4A).withValues(alpha: 0.08),
              blurRadius: 10,
              offset: const Offset(0, 3),
            ),
          ]
        : null,
  );
}

class PaperDecor extends StatelessWidget {
  const PaperDecor(
    this.asset, {
    super.key,
    this.size,
    this.height,
    this.angle = 0,
    this.opacity = 1,
    this.fit = BoxFit.contain,
  });

  final String asset;
  final double? size;
  final double? height;
  final double angle;
  final double opacity;
  final BoxFit fit;

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.shrink();
    Widget img = Image.asset(
      asset,
      width: size,
      height: height ?? size,
      fit: fit,
      opacity: AlwaysStoppedAnimation(opacity),
      filterQuality: FilterQuality.medium,
    );
    if (angle != 0) img = Transform.rotate(angle: angle, child: img);
    return ExcludeSemantics(child: img);
  }
}

class PaperGlyph extends StatelessWidget {
  const PaperGlyph({
    super.key,
    required this.icon,
    this.asset,
    this.size = 22,
    this.color,
```

```
    this.opacity = 1,
    this.retintInDark = true,
  });

  final IconData icon;

  final String? asset;
  final double size;
  final Color? color;
  final double opacity;

  /// When false, keep original pack colors (e.g. shell bottom nav).
  final bool retintInDark;

  @override
  Widget build(BuildContext context) {
    final resolved = asset ?? paperAssetForIcon(icon);
    if (!isNaturePaper(context) || resolved == null) {
      return Icon(
        icon,
        size: size,
        color: opacity == 1 ? color : color?.withValues(alpha: opacity),
      );
    }

    Widget img = Image.asset(
      resolved,
      width: size,
      height: size,
      fit: BoxFit.contain,
      filterQuality: FilterQuality.medium,
    );

    // Pack sprites are painted for cream paper; retint on night cards so
    // dark brown assets (stars, notebooks) stay readable.
    final isDark = Theme.of(context).brightness == Brightness.dark;
    if (isDark && retintInDark) {
      final provided = color;
      final tint =
          (provided != null && provided.computeLuminance() > 0.38)
          ? provided
          : NaturePaperColors.moonBeige;
      img = ColorFiltered(
        colorFilter: ColorFilter.mode(tint, BlendMode.srcIn),
        child: img,
      );
    }

    if (opacity < 1) {
      img = Opacity(opacity: opacity, child: img);
    }

    return ExcludeSemantics(child: img);
  }
}

class PaperEmptyIllustration extends StatelessWidget {
  const PaperEmptyIllustration({super.key, required this.asset});

  final String asset;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: 128,
      height: 104,
      child: Stack(
        alignment: Alignment.center,
        children: [
          Positioned(
            left: 0,
            top: 10,
            child: Transform.rotate(
```

```
              angle: -0.5,
              child: Image.asset(
                PaperAssets.beigeLeaf,
                width: 26,
                opacity: const AlwaysStoppedAnimation(0.8),
              ),
            ),
          ),
          Positioned(
            right: 4,
            top: 0,
            child: Transform.rotate(
              angle: 0.35,
              child: Image.asset(
                PaperAssets.roundedTanStar,
                width: 20,
                opacity: const AlwaysStoppedAnimation(0.85),
              ),
            ),
          ),
          Positioned(
            right: 0,
            bottom: 2,
            child: Image.asset(
              PaperAssets.whiteDaisy,
              width: 22,
              opacity: const AlwaysStoppedAnimation(0.9),
            ),
          ),
          Positioned(
            left: 8,
            bottom: 0,
            child: Image.asset(
              PaperAssets.tanSeed,
              width: 14,
              opacity: const AlwaysStoppedAnimation(0.7),
            ),
          ),
          Image.asset(
            asset,
            width: 76,
            height: 76,
            fit: BoxFit.contain,
            filterQuality: FilterQuality.medium,
          ),
        ],
      ),
    );
  }
}

class PaperTaped extends StatelessWidget {
  const PaperTaped({
    super.key,
    required this.child,
    this.angle = -0.07,
    this.tapeWidth = 88,
    this.alignment = 0,
  });

  final Widget child;
  final double angle;
  final double tapeWidth;

  final double alignment;

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return child;
    return Stack(
      clipBehavior: Clip.none,
      children: [
        child,
```

```
          Positioned(
            top: 0,
            left: 0,
            right: 0,
            child: Align(
              alignment: Alignment(alignment, 0),
              child: ExcludeSemantics(
                child: Transform.rotate(
                  angle: angle,
                  child: Image.asset(
                    PaperAssets.stripedWashiTape,
                    width: tapeWidth,
                    opacity: const AlwaysStoppedAnimation(0.92),
```

**lib/components/decor/paper_decor.dart — continued (2/2)**

```
                    filterQuality: FilterQuality.medium,
                  ),
                ),
              ),
            ),
          ),
        ],
      );
    }
  }

  class PaperDivider extends StatelessWidget {
    const PaperDivider({super.key, this.width = 180});

    final double width;

    @override
    Widget build(BuildContext context) {
      if (!isNaturePaper(context)) return const SizedBox.shrink();
      return Padding(
        padding: const EdgeInsets.symmetric(vertical: 10),
        child: Center(
          child: ExcludeSemantics(
            child: Image.asset(
              PaperAssets.kraftPaperStrip,
              width: width,
              height: 14,
              fit: BoxFit.fill,
              opacity: const AlwaysStoppedAnimation(0.75),
            ),
          ),
        ),
      );
    }
  }

  class PaperScallopedAvatar extends StatelessWidget {
    const PaperScallopedAvatar({
      super.key,
      required this.child,
      required this.avatarSize,
    });

    final Widget child;

    final double avatarSize;

    @override
    Widget build(BuildContext context) {
      if (!isNaturePaper(context)) return child;
      final frameSize = avatarSize * 1.30;
      return SizedBox(
        width: frameSize,
        height: frameSize,
        child: Stack(
          alignment: Alignment.center,
          children: [
```

```
            ExcludeSemantics(
              child: Image.asset(
                PaperAssets.sageScallopedFrame,
                width: frameSize,
                height: frameSize,
                fit: BoxFit.contain,
                filterQuality: FilterQuality.medium,
              ),
            ),
            child,
          ],
        ),
      );
  }
}

String? paperStarForPercent(double? percent) {
  if (percent == null) return null;
  if (percent >= 85) return PaperAssets.cardboardStar;
  if (percent >= 70) return PaperAssets.texturedTanStar;
  return null;
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/decor/sidebar_grove.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type _CloudSpec
- Type SidebarSkyClouds
- Type _SidebarSkyCloudsState
- Type SidebarGrove
- Type _SidebarGroveState
- Type _FirefliesPainter

**lib/components/decor/sidebar_grove.dart**

```
import 'dart:math' as math;

import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';

class _CloudSpec {
  const _CloudSpec({
    required this.asset,
    required this.y,
    required this.height,
    required this.speed,
    required this.phase,
    required this.opacity,
  });

  final String asset;
  final double y;
  final double height;
  final double speed;
  final double phase;
  final double opacity;
}

class SidebarSkyClouds extends StatefulWidget {
  const SidebarSkyClouds({super.key});

  @override
  State<SidebarSkyClouds> createState() => _SidebarSkyCloudsState();
}

class _SidebarSkyCloudsState extends State<SidebarSkyClouds>
    with SingleTickerProviderStateMixin {
  static const _clouds = [
    _CloudSpec(
      asset: PaperAssets.whiteCloud,
      y: 0.12,
      height: 28,
      speed: 0.55,
      phase: 0.05,
      opacity: 0.55,
    ),
    _CloudSpec(
      asset: PaperAssets.flatCloud,
      y: 0.38,
      height: 22,
      speed: 0.35,
      phase: 0.4,
      opacity: 0.4,
    ),
    _CloudSpec(
      asset: PaperAssets.grayCloud,
      y: 0.62,
      height: 18,
      speed: 0.7,
      phase: 0.72,
```

```
      opacity: 0.28,
    ),
    _CloudSpec(
      asset: PaperAssets.whiteCloud,
      y: 0.82,
      height: 16,
      speed: 0.45,
      phase: 0.2,
      opacity: 0.35,
    ),
  ];

  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 22),
  )..repeat();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.expand();
    if (Theme.of(context).brightness == Brightness.dark) {
      return const SizedBox.expand();
    }

    return IgnorePointer(
      child: RepaintBoundary(
        child: ClipRect(
          child: LayoutBuilder(
            builder: (context, constraints) {
              final w = constraints.maxWidth;
              final h = constraints.maxHeight;
              if (h < 24) return const SizedBox.expand();
              return AnimatedBuilder(
                animation: _controller,
                builder: (context, _) {
                  final t = _controller.value;
                  return Stack(
                    clipBehavior: Clip.hardEdge,
                    children: [
                      for (final c in _clouds)
                        Positioned(
                          left:
                              w - (((t * c.speed + c.phase) % 1.0) * (w + 100)),
                          top: c.y * h,
                          child: Image.asset(
                            c.asset,
                            height: c.height,
                            opacity: AlwaysStoppedAnimation(c.opacity),
                            filterQuality: FilterQuality.medium,
                          ),
                        ),
                    ],
                  );
                },
              );
            },
          ),
        ),
      ),
    );
  }
}

class SidebarGrove extends StatefulWidget {
  const SidebarGrove({super.key});

  @override
```

```
  State<SidebarGrove> createState() => _SidebarGroveState();
}

class _SidebarGroveState extends State<SidebarGrove>
    with SingleTickerProviderStateMixin {
  static const _clouds = [
    _CloudSpec(
      asset: PaperAssets.whiteCloud,
      y: 4,
      height: 24,
      speed: 3.0,
      phase: 0.0,
      opacity: 0.7,
    ),
    _CloudSpec(
      asset: PaperAssets.flatCloud,
      y: 22,
      height: 18,
      speed: 1.8,
      phase: 0.45,
      opacity: 0.45,
    ),
    _CloudSpec(
      asset: PaperAssets.grayCloud,
      y: 10,
      height: 14,
      speed: 2.4,
      phase: 0.78,
      opacity: 0.32,
    ),
  ];

  late final AnimationController _controller = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 14),
  )..repeat();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Widget _swaying({
    required double t,
    required double phase,
    required Widget child,
  }) {
    final angle = math.sin(t * 2 * math.pi + phase) * 0.022;
    return Transform.rotate(
      angle: angle,
      alignment: Alignment.bottomCenter,
      child: child,
    );
  }

  @override
  Widget build(BuildContext context) {
    if (!isNaturePaper(context)) return const SizedBox.shrink();
    final isDark = Theme.of(context).brightness == Brightness.dark;

    return RepaintBoundary(
      child: ClipRect(
        child: SizedBox(
          height: 148,
          width: double.infinity,
          child: LayoutBuilder(
            builder: (context, constraints) {
              final width = constraints.maxWidth;
              return AnimatedBuilder(
                animation: _controller,
                builder: (context, _) {
                  final t = _controller.value;
```

```
final travel = width + 90;
return Stack(
  clipBehavior: Clip.hardEdge,
  children: [
    if (!isDark)
      for (final c in _clouds)
        Positioned(
          left:
              width -
              (((t * c.speed + c.phase) % 1.0) * travel),
          top: c.y,
          child: Image.asset(
            c.asset,
            height: c.height,
            opacity: AlwaysStoppedAnimation(c.opacity),
            filterQuality: FilterQuality.medium,
          ),
        ),
    Positioned(
      left: 0,
      right: 0,
      bottom: 0,
      child: Image.asset(
        PaperAssets.landscapeHills,
        fit: BoxFit.fitWidth,
        opacity: AlwaysStoppedAnimation(isDark ? 0.5 : 0.85),
      ),
    ),
    Positioned(
      left: 22,
      bottom: 26,
      child: _swaying(
        t: t,
        phase: 1.9,
        child: Image.asset(
          PaperAssets.singleTree,
          height: 46,
          opacity: AlwaysStoppedAnimation(isDark ? 0.85 : 1),
        ),
      ),
    ),
    Positioned(
      right: 22,
      bottom: 12,
      child: _swaying(
        t: t,
        phase: 0,
        child: Image.asset(
          PaperAssets.twoTrees,
          height: 78,
          opacity: AlwaysStoppedAnimation(isDark ? 0.9 : 1),
        ),
      ),
    ),
    Positioned(
      left: 78,
      bottom: 16,
      child: Image.asset(
        PaperAssets.whiteDaisy,
        height: 15,
        opacity: AlwaysStoppedAnimation(isDark ? 0.55 : 0.9),
      ),
    ),
    Positioned(
      right: 96,
      bottom: 8,
      child: Image.asset(
        PaperAssets.salmonFlower,
        height: 17,
        opacity: AlwaysStoppedAnimation(isDark ? 0.55 : 0.9),
      ),
    ),
    Positioned(
```

```
                        left: 46,
                        bottom: 6,
                        child: Image.asset(
                          PaperAssets.tanSeed,
                          height: 10,
                          opacity: AlwaysStoppedAnimation(isDark ? 0.45 : 0.7),
                        ),
                      ),
                      if (!isDark)
                        Positioned(
                          left: 96,
                          bottom: 64,
                          child: _swaying(
                            t: t,
                            phase: 3.4,
                            child: Image.asset(
                              PaperAssets.tealBird,
                              height: 18,
                              opacity: const AlwaysStoppedAnimation(0.9),
                            ),
                          ),
                        ),
                      if (isDark)
                        Positioned.fill(
                          child: CustomPaint(painter: _FirefliesPainter(t)),
                        ),
                    ],
                  );
                },
              );
            },
          ),
        ),
      ),
    );
  }
}

class _FirefliesPainter extends CustomPainter {
  _FirefliesPainter(this.t);

  final double t;

  static const _anchors = [
    Offset(0.16, 0.42),
    Offset(0.34, 0.62),
    Offset(0.52, 0.30),
    Offset(0.68, 0.55),
    Offset(0.82, 0.28),
    Offset(0.90, 0.60),
  ];

  @override
  void paint(Canvas canvas, Size size) {
    final glow = Paint()
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4);
    final core = Paint();

    for (var i = 0; i < _anchors.length; i++) {
      final phase = i * 1.7;
      final angle = t * 2 * math.pi;
      final dx = math.sin(angle + phase) * 7;
      final dy = math.sin((angle + phase) * 2) * 4;
      final center = Offset(
        _anchors[i].dx * size.width + dx,
        _anchors[i].dy * size.height + dy,
      );
      final pulse = 0.25 + 0.75 * (0.5 + 0.5 * math.sin(angle * 3 + phase * 2));
      glow.color = const Color(0xFFE8C57F).withValues(alpha: 0.45 * pulse);
      core.color = const Color(0xFFF6E3B4).withValues(alpha: 0.9 * pulse);
      canvas.drawCircle(center, 3.4, glow);
      canvas.drawCircle(center, 1.1, core);
    }
```

```
  }

  @override
  bool shouldRepaint(_FirefliesPainter oldDelegate) => oldDelegate.t != t;
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/dialogs/pdf_viewer_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**lib/components/dialogs/pdf_viewer_sheet.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:syncfusion_flutter_pdfviewer/pdfviewer.dart';

Future<void> openPdfViewer(BuildContext context, String url, {String? title}) {
  return showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    useSafeArea: true,
    // The shell's nav bar floats above its own body, so a sheet opened on the
    // shell navigator would come up underneath it.
    useRootNavigator: true,
    builder: (ctx) => DraggableScrollableSheet(
      expand: false,
      initialChildSize: 0.92,
      maxChildSize: 0.98,
      minChildSize: 0.5,
      builder: (_, controller) => Column(
        children: [
          ListTile(
            title: Text(title ?? ctx.l10n.pdfDefaultTitle),
            trailing: IconButton(
              icon: const Icon(Icons.close),
              onPressed: () => Navigator.pop(ctx),
            ),
          ),
          Expanded(
            child: SfPdfViewer.network(
              url,
              scrollDirection: PdfScrollDirection.vertical,
            ),
          ),
        ],
      ),
    ),
  );
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/homework/due_date_label.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type DueDateLabel

**lib/components/homework/due_date_label.dart**

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:mu_study/core/homework/assignment_due.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/models/models.dart';

class DueDateLabel extends StatelessWidget {
  const DueDateLabel({
    super.key,
    required this.assignment,
    this.compact = false,
    this.onDark = false,
  });

  final Assignment assignment;
  final bool compact;
  final bool onDark;

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context);
    final status = assignmentDueStatus(assignment);
    final due = assignment.dueDate;
    final muted = onDark
        ? Colors.white.withValues(alpha: 0.85)
        : AppColors.inkAlpha(context, 0.55);

    if (status == AssignmentDueStatus.none || due == null) {
      return Text(
        l10n.commonNoDueDate,
        style: TextStyle(color: muted, fontSize: compact ? 12 : 13),
      );
    }

    final (label, color) = _labelAndColor(context, l10n, status, due, onDark);

    if (compact) {
      return Text(
        label,
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
        style: TextStyle(
          color: color,
          fontSize: 12,
          fontWeight: FontWeight.w600,
        ),
      );
    }

    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
      decoration: BoxDecoration(
        color: color.withValues(alpha: onDark ? 0.22 : 0.12),
        borderRadius: BorderRadius.circular(20),
      ),
      child: Text(
        label,
        style: TextStyle(
          color: onDark ? Colors.white : color,
          fontSize: 12,
          fontWeight: FontWeight.w700,
        ),
```

```
        ),
      );
    }

    static (String, Color) _labelAndColor(
      BuildContext context,
      AppLocalizations l10n,
      AssignmentDueStatus status,
      DateTime due,
      bool onDark,
    ) {
      final dateStr = DateFormat.MMMd().format(due);
      final isDark = Theme.of(context).brightness == Brightness.dark;
      switch (status) {
        case AssignmentDueStatus.overdue:
          return (
            l10n.homeworkOverdue(dateStr),
            onDark
                ? const Color(0xFFFFCDD2)
                : (isDark ? const Color(0xFFE57373) : AppColors.errorRed),
          );
        case AssignmentDueStatus.dueToday:
          return (
            l10n.homeworkDueToday,
            onDark
                ? const Color(0xFFFFE0B2)
                : (isDark ? const Color(0xFFFFB74D) : AppColors.warmAmber),
          );
        case AssignmentDueStatus.dueSoon:
          final today = assignmentCalendarDay(DateTime.now());
          final days = assignmentCalendarDay(due).difference(today).inDays;
          final label = days == 1
              ? l10n.homeworkDueTomorrow
              : l10n.homeworkDueInDays(days);
          return (label, onDark ? const Color(0xFFFFE0B2) : AppColors.mutedGold);
        case AssignmentDueStatus.upcoming:
          return (
            l10n.commonDueDate(dateStr),
            onDark ? Colors.white : AppColors.inkAlpha(context, 0.7),
          );
        case AssignmentDueStatus.none:
          return (
            l10n.commonNoDueDate,
            onDark ? Colors.white70 : AppColors.inkAlpha(context, 0.55),
          );
      }
    }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/inputs/group_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type GroupSelector

**lib/components/inputs/group_selector.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/providers/groups/group_providers.dart';

class GroupSelector extends ConsumerWidget {
  const GroupSelector({super.key, this.padding = const EdgeInsets.all(20)});

  final EdgeInsets padding;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final groups = ref.watch(myGroupsProvider);
    final activeId = ref.watch(activeGroupIdProvider);
    final list = groups.valueOrNull ?? const [];

    if (list.length < 2) return const SizedBox.shrink();

    final radius = paperControlRadius(context);

    return Padding(
      padding: padding,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
        decoration: paperControlDecoration(context),
        child: Row(
          children: [
            PaperGlyph(
              icon: Icons.groups_outlined,
              size: 18,
              color: AppColors.accent(context),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: DropdownButtonHideUnderline(
                child: DropdownButton<String>(
                  value: activeId,
                  isExpanded: true,
                  borderRadius: BorderRadius.circular(radius),
                  dropdownColor: AppColors.controlSurface(context),
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    fontSize: 15,
                    color: AppColors.ink(context),
                  ),
                  items: list
                      .map(
                        (g) => DropdownMenuItem(
                          value: g.id,
                          child: Text(g.title, overflow: TextOverflow.ellipsis),
                        ),
                      )
                      .toList(),
                  onChanged: (value) {
                    if (value == null) return;
                    ref.read(selectedGroupIdProvider.notifier).state = value;
                  },
                ),
              ),
            ),
          ],
        ),
```

```
        ),
      );
    }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/inputs/load_more_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type LoadMoreButton

**lib/components/inputs/load_more_button.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class LoadMoreButton extends StatelessWidget {
  const LoadMoreButton({
    super.key,
    required this.hasMore,
    required this.loading,
    required this.onPressed,
  });

  final bool hasMore;
  final bool loading;
  final VoidCallback onPressed;

  @override
  Widget build(BuildContext context) {
    if (!hasMore) return const SizedBox.shrink();

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 12),
      child: Center(
        child: loading
            ? SizedBox(
                width: 24,
                height: 24,
                child: CircularProgressIndicator(strokeWidth: 2),
              )
            : TextButton(
                onPressed: onPressed,
                child: Text(
                  context.l10n.commonLoadMore,
                  style: TextStyle(
                    fontWeight: FontWeight.w700,
                    color: AppColors.accent(context),
                  ),
                ),
              ),
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/layouts/app_shell.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type AppShell
- Type _AppShellState

**lib/components/layouts/app_shell.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/brand/app_logo.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/decor/sidebar_grove.dart';
import 'package:mu_study/components/layouts/shell_nav_bar.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/utils/responsive.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/l10n/nav_labels.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/chat/chat_providers.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';
import 'package:mu_study/providers/focus/focus_session_provider.dart';

export 'package:mu_study/components/layouts/shell_nav_bar.dart';

class AppShell extends ConsumerStatefulWidget {
  const AppShell({
    super.key,
    required this.role,
    required this.child,
    required this.destinations,
  });

  final UserRole role;
  final Widget child;
  final List<ShellDestination> destinations;

  @override
  ConsumerState<AppShell> createState() => _AppShellState();
}

class _AppShellState extends ConsumerState<AppShell> {
  bool _isSidebarExpanded = true;

  int _indexFromLocation(String location) {
    for (var i = 0; i < widget.destinations.length; i++) {
      if (location.startsWith(widget.destinations[i].path)) return i;
    }
    return 0;
  }

  void _go(int index) {
    context.go(widget.destinations[index].path);
  }

  @override
  Widget build(BuildContext context) {
    final location = GoRouterState.of(context).uri.toString();
    final index = _indexFromLocation(location);
    final unreadNotifications = ref.watch(unreadNotificationsCountProvider);
    final unreadChat = ref.watch(focusChatMutedProvider)
        ? 0
        : ref.watch(unreadChatCountProvider);
    final isDesktop = MediaQuery.sizeOf(context).width >= Breakpoints.desktop;
    final isTablet = context.isTablet;
    final l10n = context.l10n;
    String labelOf(ShellDestination d) => navLabel(l10n, d.labelKey);
```

```
int badgeCount(ShellDestination d) => switch (d.badge) {
  ShellBadgeKind.notifications => unreadNotifications,
  ShellBadgeKind.chat => unreadChat,
  ShellBadgeKind.none => 0,
};

Widget icon(
  ShellDestination d, {
  bool isSelected = false,
  bool usePaperAssets = true,
}) {
  final isMono =
      AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;
  final tone = isSelected
      ? AppColors.accent(context)
      : AppColors.inkAlpha(context, 0.5);
  final Widget base = usePaperAssets
      ? PaperGlyph(
          icon: d.icon,
          size: 24,
          color: tone,
          opacity: isSelected || !isNaturePaper(context) ? 1 : 0.72,
        )
      : Icon(d.icon, size: 24, color: tone);
  final count = badgeCount(d);
  if (count > 0) {
    return Badge(
      label: Text(
        count > 99 ? '99+' : '$count',
        style: TextStyle(
          fontSize: 10,
          color: isMono
              ? (Theme.of(context).brightness == Brightness.dark
                    ? Colors.black
                    : Colors.white)
              : null,
        ),
      ),
      backgroundColor: isMono
          ? AppColors.ink(context)
          : AppColors.warmAmber,
      child: base,
    );
  }
  return base;
}

if (isDesktop) {
  return Scaffold(
    body: Row(
      children: [
        AnimatedContainer(
          duration: const Duration(milliseconds: 300),
          curve: Curves.easeInOut,
          width: _isSidebarExpanded ? 260 : 80,
          decoration: BoxDecoration(
            color: Theme.of(context).colorScheme.surface,
            border: Border(
              right: BorderSide(
                color: Theme.of(
                  context,
                ).colorScheme.onSurface.withValues(alpha: 0.05),
              ),
            ),
          ),
          child: ClipRect(
            child: Column(
              children: [
                const SizedBox(height: 24),
                Padding(
                  padding: const EdgeInsets.symmetric(horizontal: 16),
                  child: Row(
                    mainAxisAlignment: _isSidebarExpanded
```

```
          ? MainAxisAlignment.spaceBetween
          : MainAxisAlignment.center,
      children: [
        if (_isSidebarExpanded)
          Expanded(
            child: ClipRect(
              child: Row(
                children: [
                  const AppLogoImage(size: 28),
                  const SizedBox(width: 12),
                  Text(
                    l10n.shellMuStudy,
                    maxLines: 1,
                    overflow: TextOverflow.clip,
                    softWrap: false,
                    style: Theme.of(context)
                        .textTheme
                        .titleMedium
                        ?.copyWith(
                          fontWeight: FontWeight.w800,
                          letterSpacing: -0.5,
                        ),
                  ),
                ],
              ),
            ),
          ),
        IconButton(
          onPressed: () => setState(
            () => _isSidebarExpanded = !_isSidebarExpanded,
          ),
          icon: Icon(
            _isSidebarExpanded ? Icons.menu_open : Icons.menu,
            color: AppColors.inkAlpha(context, 0.5),
          ),
        ),
      ],
    ),
  ),
  if (_isSidebarExpanded)
    const Padding(
      padding: EdgeInsets.only(top: 12),
      child: SizedBox(
        width: double.infinity,
        child: PaperDecor(
          PaperAssets.hangingGarland,
          height: 52,
          fit: BoxFit.fitWidth,
          opacity: 0.95,
        ),
      ),
    ),
  const SizedBox(height: 20),
  Padding(
    padding: const EdgeInsets.symmetric(horizontal: 12),
    child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        for (var i = 0; i < widget.destinations.length; i++)
          Builder(
            builder: (context) {
              final d = widget.destinations[i];
              final isSelected = index == i;
              return Padding(
                padding: const EdgeInsets.only(bottom: 4),
                child: InkWell(
                  onTap: () => _go(i),
                  borderRadius: BorderRadius.circular(8),
                  child: AnimatedContainer(
                    duration: const Duration(
                      milliseconds: 200,
                    ),
                    padding: const EdgeInsets.symmetric(
```

```
                        horizontal: 12,
                        vertical: 10,
                      ),
                      decoration: BoxDecoration(
                        color: isSelected
                            ? AppColors.accent(
                                context,
                              ).withValues(alpha: 0.12)
                            : Colors.transparent,
                        borderRadius: BorderRadius.circular(
                          isNaturePaper(context) ? 16 : 8,
                        ),
                      ),
                      child: ClipRect(
                        child: Row(
                          mainAxisAlignment: _isSidebarExpanded
                              ? MainAxisAlignment.start
                              : MainAxisAlignment.center,
                          children: [
                            icon(d, isSelected: isSelected),
                            if (_isSidebarExpanded) ...[
                              const SizedBox(width: 12),
                              Expanded(
                                child: Text(
                                  labelOf(d),
                                  maxLines: 1,
                                  overflow: TextOverflow.clip,
                                  softWrap: false,
                                  style: Theme.of(context)
                                      .textTheme
                                      .bodyMedium
                                      ?.copyWith(
                                        fontWeight: isSelected
                                            ? FontWeight.w700
                                            : FontWeight.w500,
                                        color: isSelected
                                            ? AppColors.accent(
                                                context,
                                              )
                                            : AppColors.inkAlpha(
                                                context,
                                                0.7,
                                              ),
                                      ),
                                ),
                              ),
                            ],
                          ],
                        ),
                      ),
                    ),
                  ),
                ),
              );
            },
          ),
      ],
    ),
  ),
  if (_isSidebarExpanded)
    const Expanded(child: SidebarSkyClouds())
  else
    const Spacer(),
  if (_isSidebarExpanded) const SidebarGrove(),
  if (_isSidebarExpanded)
    Padding(
      padding: const EdgeInsets.all(16),
      child: Container(
        padding: const EdgeInsets.all(12),
        decoration: paperControlDecoration(
          context,
          color: AppColors.softWash(context, lightAlpha: 0.2),
          radius: isNaturePaper(context) ? 16 : 12,
        ),
```

```
                          child: Row(
                            children: [
                              if (isNaturePaper(context))
                                const PaperGlyph(
                                  icon: Icons.person_outline,
                                  size: 28,
                                )
                              else
                                CircleAvatar(
                                  radius: 16,
                                  backgroundColor: AppColors.accent(context),
                                  child: Icon(
                                    Icons.person,
                                    size: 16,
                                    color:
                                        Theme.of(context).brightness ==
                                            Brightness.dark
                                        ? Colors.black
                                        : Colors.white,
                                  ),
                                ),
                              const SizedBox(width: 12),
                              Expanded(
                                child: Column(
                                  crossAxisAlignment: CrossAxisAlignment.start,
                                  children: [
                                    Text(
                                      switch (widget.role) {
                                        UserRole.student => l10n.commonStudent,
```

**lib/components/layouts/app_shell.dart — continued (2/2)**

```
                                        UserRole.teacher => l10n.commonTeacher,
                                        UserRole.admin => l10n.commonAdmin,
                                      }.toUpperCase(),
                                      style: TextStyle(
                                        fontSize: 10,
                                        fontWeight: FontWeight.w800,
                                        color: AppColors.accent(context),
                                      ),
                                    ),
                                    Text(
                                      l10n.commonWorkspace,
                                      style: const TextStyle(
                                        fontSize: 12,
                                        fontWeight: FontWeight.w500,
                                      ),
                                    ),
                                  ],
                                ),
                              ),
                            ],
                          ),
                        ),
                      ),
                    ),
                  ),
                  const SizedBox(height: 16),
                ],
              ),
            ),
          ),
          Expanded(
            child: Center(
              child: ConstrainedBox(
                constraints: BoxConstraints(
                  maxWidth: context.contentMaxWidth,
                ),
                child: widget.child,
              ),
            ),
          ),
        ],
      ),
    );
```

```
    }

    if (isTablet) {
      return Scaffold(
        body: Row(
          children: [
            SafeArea(
              right: false,
              child: NavigationRail(
                selectedIndex: index,
                onDestinationSelected: _go,
                labelType: NavigationRailLabelType.none,
                backgroundColor: Theme.of(context).colorScheme.surface,
                indicatorColor: AppColors.accent(
                  context,
                ).withValues(alpha: 0.15),
                leading: const Padding(
                  padding: EdgeInsets.symmetric(vertical: 24),
                  child: AppLogoImage(size: 28),
                ),
                destinations: widget.destinations
                    .map(
                      (d) => NavigationRailDestination(
                        icon: icon(d),
                        selectedIcon: icon(d, isSelected: true),
                        label: Text(labelOf(d)),
                      ),
                    )
                    .toList(),
              ),
            ),
            const VerticalDivider(width: 1, thickness: 1),
            Expanded(
              child: Center(
                child: ConstrainedBox(
                  constraints: BoxConstraints(
                    maxWidth: context.contentMaxWidth,
                  ),
                  child: widget.child,
                ),
              ),
            ),
          ],
        ),
      );
    }

    return Scaffold(
      extendBody: true,
      resizeToAvoidBottomInset: false,
      body: Center(
        child: ConstrainedBox(
          constraints: BoxConstraints(maxWidth: context.contentMaxWidth),
          child: widget.child,
        ),
      ),
      bottomNavigationBar: ShellNavBar(
        destinations: widget.destinations,
        selectedIndex: index,
        onDestinationSelected: _go,
        iconBuilder: (d, isSelected) =>
            icon(d, isSelected: isSelected, usePaperAssets: false),
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/layouts/empty_state.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type EmptyState
- Type ErrorState

**lib/components/layouts/empty_state.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class EmptyState extends StatelessWidget {
  const EmptyState({
    super.key,
    required this.title,
    this.subtitle,
    this.icon = Icons.inbox_outlined,
  });

  final String title;
  final String? subtitle;
  final IconData icon;

  @override
  Widget build(BuildContext context) {
    final paperAsset = isNaturePaper(context)
        ? (paperAssetForIcon(icon) ?? PaperAssets.tornPaperScrap)
        : null;
    final isNeu = isSoftNeumorphism(context);
    final isMono = isMonochromeWorkspace(context);

    Widget leading;
    if (paperAsset != null) {
      leading = PaperEmptyIllustration(asset: paperAsset);
    } else if (isNeu) {
      leading = NeuIconWell(
        icon: icon,
        color: AppColors.accent(context),
        size: 36,
        padding: 26,
        elevation: NeuElevation.raised,
      );
    } else if (isMono) {
      leading = Container(
        width: 72,
        height: 72,
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(6),
          border: Border.all(color: AppColors.inkAlpha(context, 0.12)),
        ),
        child: Icon(icon, size: 28, color: AppColors.inkAlpha(context, 0.45)),
      );
    } else {
      leading = Icon(icon, size: 56, color: AppColors.inkAlpha(context, 0.25));
    }

    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            leading,
            const SizedBox(height: 16),
            Text(
              title,
```

```
              textAlign: TextAlign.center,
              style: Theme.of(context).textTheme.titleMedium?.copyWith(
                fontWeight: FontWeight.w600,
                color: AppColors.ink(context),
              ),
            ),
            if (subtitle != null) ...[
              const SizedBox(height: 8),
              Text(
                subtitle!,
                textAlign: TextAlign.center,
                style: Theme.of(context).textTheme.bodyMedium?.copyWith(
                  color: AppColors.inkAlpha(context, 0.55),
                ),
              ),
            ],
          ],
        ),
      ),
    );
  }
}

class ErrorState extends StatelessWidget {
  const ErrorState({super.key, required this.message, this.onRetry});

  final String message;
  final VoidCallback? onRetry;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            if (isNaturePaper(context))
              const PaperEmptyIllustration(asset: PaperAssets.tornPaperScrap)
            else
              Icon(
                Icons.error_outline,
                color: AppColors.danger(context),
                size: 48,
              ),
            const SizedBox(height: 12),
            Text(message, textAlign: TextAlign.center),
            if (onRetry != null) ...[
              const SizedBox(height: 16),
              TextButton(
                onPressed: onRetry,
                child: Text(context.l10n.commonTryAgain),
              ),
            ],
          ],
        ),
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/layouts/premium_section_header.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type PremiumSectionHeader

**lib/components/layouts/premium_section_header.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';

class PremiumSectionHeader extends StatelessWidget {
  const PremiumSectionHeader({
    super.key,
    required this.title,
    this.subtitle,
    this.trailing,
    this.horizontalPadding = 0,
    this.decorAsset,
  });

  final String title;
  final String? subtitle;
  final Widget? trailing;
  final double horizontalPadding;

  final String? decorAsset;

  @override
  Widget build(BuildContext context) {
    final tokens = AppStyleTokens.maybeOf(context);
    final headerGlow = tokens?.headerGlow ?? Colors.transparent;
    final isDark = Theme.of(context).brightness == Brightness.dark;

    Widget content = Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        if (subtitle != null)
          Padding(
            padding: const EdgeInsets.only(bottom: 4),
            child: Text(
              subtitle!.toUpperCase(),
              style: Theme.of(context).textTheme.labelSmall?.copyWith(
                letterSpacing: tokens?.isMonochromeWorkspace == true
                    ? 1.4
                    : 1.2,
                fontWeight: FontWeight.w700,
                color: tokens?.isMonochromeWorkspace == true
                    ? AppColors.inkAlpha(context, 0.45)
                    : AppColors.accent(context).withValues(alpha: 0.92),
              ),
            ),
          ),
        Row(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            Flexible(
              child: Text(
                title,
                style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                  fontWeight: FontWeight.w800,
                  letterSpacing: tokens?.isMonochromeWorkspace == true
                      ? -0.7
                      : -0.5,
                  fontSize: tokens?.isMonochromeWorkspace == true ? 26 : null,
                ),
              ),
            ),
          ),
```

```
            const SizedBox(width: 8),
            PaperDecor(
              decorAsset ?? PaperAssets.beigeLeaf,
              size: decorAsset == null ? 16 : 22,
              angle: decorAsset == null ? 0.5 : 0.1,
              opacity: 0.9,
            ),
          ],
        ),
      ],
    );

    if (headerGlow != Colors.transparent) {
      content = Stack(
        clipBehavior: Clip.none,
        children: [
          Positioned(
            left: -28,
            top: -20,
            child: IgnorePointer(
              child: Container(
                width: 190,
                height: 84,
                decoration: BoxDecoration(
                  borderRadius: BorderRadius.circular(48),
                  gradient: RadialGradient(
                    colors: [
                      headerGlow.withValues(alpha: isDark ? 0.10 : 0.26),
                      headerGlow.withValues(alpha: 0.0),
                    ],
                  ),
                ),
              ),
            ),
          ),
          content,
        ],
      );
    }

    return Padding(
      padding: EdgeInsets.symmetric(
        horizontal: horizontalPadding,
        vertical: 12,
      ),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.end,
        children: [
          Expanded(child: content),
          ?trailing,
        ],
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/layouts/shell_nav_bar.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type ShellDestination
- Type ShellFabLocation
- Type ShellNavBar
- Type _ShellNavItem

**lib/components/layouts/shell_nav_bar.dart**

```
import 'dart:math' as math;
import 'dart:ui';

import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/core/theme/nature_paper_colors.dart';
import 'package:mu_study/core/theme/neumorphic.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/l10n/nav_labels.dart';

/// Height of the floating navigation pill, excluding the safe area beneath it.
const double kShellNavBarHeight = 64;

/// Inset of the pill from the left and right edges of the screen.
const double kShellNavBarSideMargin = 16;

/// Gap between the pill and the bottom of the screen on devices without a home
/// indicator or gesture bar. Devices that have one use that inset instead, so
/// the pill never sits inside the system gesture area.
const double kShellNavBarMinBottomGap = 16;

/// Gap between a floating action button and the top of the pill.
const double kShellFabGap = 16;

/// Height of a standard [FloatingActionButton]; extended ones are shorter.
const double kShellFabHeight = 56;

/// Duration of the nav bar's selection animations.
const Duration kShellNavBarAnimationDuration = Duration(milliseconds: 240);

enum ShellBadgeKind { none, notifications, chat }

class ShellDestination {
  const ShellDestination({
    required this.labelKey,
    required this.icon,
    required this.path,
    this.badge = ShellBadgeKind.none,
  });

  /// Key into [AppLocalizations] nav_* getters (e.g. `navDashboard`).
  final String labelKey;
  final IconData icon;
  final String path;
  final ShellBadgeKind badge;
}

/// Space the shell's navigation bar occupies at the bottom of the screen,
/// including the safe area beneath it.
///
/// Screens inside the shell have their own `Scaffold`, which knows nothing
/// about the shell's floating pill and would otherwise place floating action
/// buttons and trailing content underneath it.
///
/// The shell renders the pill with `Scaffold.extendBody`, which folds its
/// height into the bottom padding of the body and zeroes the body's bottom
/// `viewPadding` — so the padding alone is the whole story. Outside the shell,
```

```
/// on tablet, on desktop and on routes pushed above it, this is the plain
/// system safe area, which is what those layouts need anyway.
double shellBottomInset(BuildContext context) =>
    MediaQuery.paddingOf(context).bottom;

/// Bottom padding for scrollable content inside the shell so the last item
/// clears the navigation bar and, when [hasFab] is set, the floating action
/// button above it.
double shellContentBottomPadding(
  BuildContext context, {
  bool hasFab = false,
  double extra = 20,
}) {
  final double base = shellBottomInset(context) + extra;
  return hasFab ? base + kShellFabHeight + kShellFabGap : base;
}

/// Positions a [Scaffold.floatingActionButton] above the shell's navigation
/// bar, the safe area and the keyboard.
FloatingActionButtonLocation shellFabLocation(BuildContext context) =>
    ShellFabLocation(bottomInset: shellBottomInset(context));

/// End-aligned floating action button location that floats a fixed distance
/// above the shell's navigation bar rather than above the scaffold's own
/// bottom edge.
class ShellFabLocation extends StandardFabLocation with FabEndOffsetX {
  const ShellFabLocation({required this.bottomInset});

  /// Result of [shellBottomInset] for the screen this button belongs to.
  final double bottomInset;

  @override
  double getOffsetY(ScaffoldPrelayoutGeometry geometry, double adjustment) {
    final double fabHeight = geometry.floatingActionButtonSize.height;
    final double keyboard = geometry.minInsets.bottom;

    // The keyboard covers the nav bar rather than moving it, so the reserved
    // space shrinks by however much of it the keyboard hides. The button holds
    // still until the keyboard reaches it, then rides up with it.
    final double margin = math.max(
      kFloatingActionButtonMargin,
      bottomInset + kShellFabGap - keyboard,
    );

    double y = geometry.contentBottom - fabHeight - margin;
    final double snackBarHeight = geometry.snackBarSize.height;
    if (snackBarHeight > 0) {
      y = math.min(
        y,
        geometry.contentBottom -
            snackBarHeight -
            fabHeight -
            kFloatingActionButtonMargin,
      );
    }
    return y + adjustment;
  }

  @override
  bool operator ==(Object other) =>
      other is ShellFabLocation && other.bottomInset == bottomInset;

  @override
  int get hashCode => bottomInset.hashCode;

  @override
  String toString() => 'ShellFabLocation(bottomInset: $bottomInset)';
}

/// Frosted navigation pill used as the shell's `bottomNavigationBar` on mobile.
///
/// The pill is [kShellNavBarHeight] tall and sits on top of the larger of the
/// bottom safe area and [kShellNavBarMinBottomGap], so it clears home
```

```
/// indicators and gesture bars on any device. That total is what screens read
/// back through [shellBottomInset].
class ShellNavBar extends StatelessWidget {
  const ShellNavBar({
    super.key,
    required this.destinations,
    required this.selectedIndex,
    required this.onDestinationSelected,
    required this.iconBuilder,
  });

  /// Identifies the pill itself, so tests can measure its bounds.
  static const Key pillKey = Key('shellNavBarPill');

  final List<ShellDestination> destinations;
  final int selectedIndex;
  final ValueChanged<int> onDestinationSelected;
  final Widget Function(ShellDestination destination, bool isSelected)
  iconBuilder;

  @override
  Widget build(BuildContext context) {
    // viewPadding rather than padding: the bar sits below the keyboard, so it
    // must not resize while the keyboard is up.
    final EdgeInsets viewPadding = MediaQuery.viewPaddingOf(context);

    return Padding(
      padding: EdgeInsets.only(
        left: kShellNavBarSideMargin + viewPadding.left,
        right: kShellNavBarSideMargin + viewPadding.right,
        bottom: math.max(viewPadding.bottom, kShellNavBarMinBottomGap),
      ),
      child: _buildPill(context),
    );
  }

  Widget _buildPill(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    final tokens = AppStyleTokens.maybeOf(context);
    final isNature = tokens?.isNaturePaper ?? false;
    final isNeu = tokens?.isSoftNeumorphism ?? false;
    final isMono = tokens?.isMonochromeWorkspace ?? false;

    // Monochrome: compact rectangular dock. Others keep the stadium pill.
    final BorderRadius radius = BorderRadius.all(
      Radius.circular(isMono ? 12 : kShellNavBarHeight / 2),
    );

    final Color fill =
        tokens?.navSurface ??
        (isDark
            ? AppColors.darkSurfaceHighest.withValues(alpha: 0.96)
            : Theme.of(context).colorScheme.surface.withValues(alpha: 0.88));
    final Color borderColor;
    if (isNeu) {
      borderColor = Colors.transparent;
    } else if (isMono) {
      borderColor = isDark
          ? AppColors.inkAlpha(context, 0.18)
          : AppColors.inkAlpha(context, 0.10);
    } else if (isNature) {
      borderColor = isDark
          ? NaturePaperColors.moonGlow.withValues(alpha: 0.22)
          : NaturePaperColors.warmBrown.withValues(alpha: 0.18);
    } else {
      borderColor = isDark ? AppColors.darkBorder : Colors.transparent;
    }

    final List<BoxShadow> shadows;
    if (isNeu) {
      // Floating extruded dock — dual soft shadows, no glass blur.
      shadows = Neumorphic.shadows(context, elevation: NeuElevation.floating);
    } else if (isMono) {
```

```
      // Editorial: no soft shadow — border alone defines the dock.
      shadows = const [];
    } else {
      shadows = [
        BoxShadow(
          color: isDark
              ? Colors.black.withValues(alpha: 0.55)
              : (isNature
                    ? NaturePaperColors.warmBrown.withValues(alpha: 0.28)
                    : AppColors.inkAlpha(context, 0.12)),
          blurRadius: 24,
          offset: const Offset(0, 8),
        ),
        if (isNature && isDark)
          BoxShadow(
            color: NaturePaperColors.moonGlow.withValues(alpha: 0.08),
            blurRadius: 28,
          ),
      ];
    }

    final navRow = Material(
      color: fill,
      shape: RoundedRectangleBorder(
        borderRadius: radius,
        side: BorderSide(color: borderColor),
      ),
      clipBehavior: Clip.antiAlias,
      child: Row(
        children: [
          for (var i = 0; i < destinations.length; i++)
            Expanded(
              child: _ShellNavItem(
                destination: destinations[i],
                isSelected: i == selectedIndex,
                icon: iconBuilder(destinations[i], i == selectedIndex),
                onTap: () => onDestinationSelected(i),
                pressedActive: isNeu,
                editorialActive: isMono,
              ),
            ),
        ],
      ),
    );

    return Container(
      key: pillKey,
      height: kShellNavBarHeight,
      decoration: BoxDecoration(borderRadius: radius, boxShadow: shadows),
      child: ClipRRect(
        borderRadius: radius,
        // Soft Neu + Monochrome avoid glassmorphism; Modern/Nature frost.
        child: (isNeu || isMono)
            ? navRow
            : BackdropFilter(
                filter: ImageFilter.blur(sigmaX: 18, sigmaY: 18),
                child: navRow,
              ),
      ),
    );
  }
}

class _ShellNavItem extends StatelessWidget {
  const _ShellNavItem({
    required this.destination,
    required this.isSelected,
    required this.icon,
    required this.onTap,
    this.pressedActive = false,
    this.editorialActive = false,
  });
```

```
final ShellDestination destination;
final bool isSelected;
final Widget icon;
final VoidCallback onTap;

/// Neumorphism: active tab looks physically pressed into the dock.
final bool pressedActive;

/// Monochrome Workspace: filled ink well + thin underline indicator.
final bool editorialActive;

@override
Widget build(BuildContext context) {
  final glow = AppStyleTokens.maybeOf(context)?.navGlow ?? Colors.transparent;
  final ink = AppColors.ink(context);
  final isDark = Theme.of(context).brightness == Brightness.dark;

  return Semantics(
    button: true,
    selected: isSelected,
    label: navLabel(context.l10n, destination.labelKey),
    child: InkWell(
      onTap: onTap,
      customBorder: editorialActive
          ? RoundedRectangleBorder(borderRadius: BorderRadius.circular(10))
          : const StadiumBorder(),
      child: SizedBox(
        height: kShellNavBarHeight,
        child: Stack(
          alignment: Alignment.center,
          children: [
            // Soft Neu: pressed well for the active destination.
            if (pressedActive)
              AnimatedOpacity(
                duration: kShellNavBarAnimationDuration,
                curve: Curves.easeOutCubic,
                opacity: isSelected ? 1 : 0,
                child: Container(
                  width: 48,
                  height: 40,
                  decoration: BoxDecoration(
                    color: Neumorphic.surface(
                      context,
                      elevation: NeuElevation.pressed,
                    ),
                    borderRadius: BorderRadius.circular(14),
                    boxShadow: Neumorphic.shadows(
                      context,
                      elevation: NeuElevation.pressed,
                    ),
                  ),
                ),
              ),
            // Monochrome: solid ink fill behind the active icon.
            if (editorialActive)
              AnimatedOpacity(
                duration: kShellNavBarAnimationDuration,
                curve: Curves.easeOutCubic,
                opacity: isSelected ? 1 : 0,
                child: Container(
                  width: 40,
                  height: 36,
                  decoration: BoxDecoration(
                    color: ink,
                    borderRadius: BorderRadius.circular(8),
                  ),
                ),
              ),
            if (glow != Colors.transparent)
              AnimatedOpacity(
                duration: kShellNavBarAnimationDuration,
                curve: Curves.easeOutCubic,
                opacity: isSelected ? 1 : 0,
```

```
                  child: Container(
                    width: 44,
                    height: 44,
                    decoration: BoxDecoration(
                      shape: BoxShape.circle,
                      gradient: RadialGradient(
                        colors: [
                          glow.withValues(alpha: pressedActive ? 0.28 : 0.40),
                          glow.withValues(alpha: 0.0),
                        ],
                      ),
                    ),
                  ),
                ),
              ),
              // The icon is centred on the pill and the indicator floats
              // underneath it, so selecting a tab never shifts the icon.
              AnimatedScale(
                duration: kShellNavBarAnimationDuration,
                curve: Curves.easeOutCubic,
                scale: isSelected
                    ? (pressedActive || editorialActive ? 1.0 : 1.08)
                    : 1,
                child: editorialActive && isSelected
                    ? ColorFiltered(
                        colorFilter: ColorFilter.mode(
                          isDark ? Colors.black : Colors.white,
                          BlendMode.srcIn,
                        ),
                        child: icon,
                      )
                    : icon,
              ),
              if (!pressedActive)
                Positioned(
                  bottom: editorialActive ? 8 : 12,
                  child: AnimatedContainer(
                    duration: kShellNavBarAnimationDuration,
```

**lib/components/layouts/shell_nav_bar.dart — continued (2/2)**

```
                    curve: Curves.easeOutCubic,
                    height: editorialActive ? 2 : 4,
                    width: isSelected ? (editorialActive ? 12 : 16) : 0,
                    decoration: BoxDecoration(
                      color: editorialActive
                          ? ink.withValues(alpha: 0.35)
                          : AppColors.accent(context),
                      borderRadius: BorderRadius.circular(1),
                    ),
                  ),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/marks/mark_average_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type MarkAverageCard

**lib/components/marks/mark_average_card.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/components/cards/premium_card.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/components/marks/percent_badge.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class MarkAverageCard extends StatelessWidget {
  const MarkAverageCard({
    super.key,
    required this.average,
    required this.markCount,
    this.onTap,
  });

  final double? average;
  final int markCount;
  final VoidCallback? onTap;

  @override
  Widget build(BuildContext context) {
    final value = average;
    final color = value == null
        ? AppColors.accent(context)
        : percentColor(value, context: context);
    final starAsset = paperStarForPercent(value);

    return PremiumCard(
      onTap: onTap,
      padding: const EdgeInsets.all(24),
      child: Row(
        children: [
          SizedBox(
            width: 76,
            height: 76,
            child: TweenAnimationBuilder<double>(
              tween: Tween<double>(begin: 0, end: (value ?? 0) / 100),
              duration: const Duration(milliseconds: 900),
              curve: Curves.easeOutCubic,
              builder: (context, progress, _) {
                const stroke = 8.0;
                const labelInset = stroke * 2 + 10;
                return Stack(
                  alignment: Alignment.center,
                  children: [
                    Positioned.fill(
                      child: CircularProgressIndicator(
                        value: progress,
                        backgroundColor: AppColors.softSurface(context),
                        valueColor: AlwaysStoppedAnimation(color),
                        strokeWidth: stroke,
                        strokeCap: StrokeCap.round,
                      ),
                    ),
                    Padding(
                      padding: const EdgeInsets.all(labelInset / 2),
                      child: FittedBox(
                        fit: BoxFit.scaleDown,
                        child: Text(
                          formatPercent(value),
                          maxLines: 1,
                          textAlign: TextAlign.center,
                          style: const TextStyle(
```

```
                          fontWeight: FontWeight.w800,
                          fontSize: 16,
                          height: 1,
                          letterSpacing: -0.4,
                        ),
                      ),
                    ),
                  ),
                ],
              );
            },
          ),
        ),
        const SizedBox(width: 20),
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                context.l10n.marksOverallAverage,
                style: const TextStyle(
                  fontWeight: FontWeight.w800,
                  fontSize: 16,
                ),
              ),
              const SizedBox(height: 4),
              Text(
                value == null
                    ? context.l10n.marksNoneYet
                    : context.l10n.marksBasedOn(
                        percentLabel(value, context.l10n),
                        markCount,
                      ),
                style: TextStyle(
                  color: AppColors.inkAlpha(context, 0.6),
                  fontSize: 13,
                  height: 1.4,
                ),
              ),
            ],
          ),
        ),
        if (starAsset != null)
          Padding(
            padding: const EdgeInsets.only(left: 8),
            child: PaperDecor(starAsset, size: 42, angle: 0.14),
          ),
        if (onTap != null)
          Icon(
            Icons.chevron_right_rounded,
            size: 20,
            color: AppColors.inkAlpha(context, 0.3),
          ),
      ],
    ),
  );
 }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

# lib/components/marks/percent_badge.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type PercentBadge

**lib/components/marks/percent_badge.dart**

```
import 'package:flutter/material.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/core/theme/app_style_tokens.dart';
import 'package:mu_study/l10n/app_localizations.dart';

Color percentColor(double percent, {BuildContext? context}) {
  final mono =
      context != null &&
      (AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false);
  final isDark =
      context != null && Theme.of(context).brightness == Brightness.dark;

  if (mono) {
    if (isDark) {
      if (percent >= 85) return const Color(0xFFF5F5F5);
      if (percent >= 70) return const Color(0xFFC8C8C8);
      if (percent >= 50) return const Color(0xFF9A9A9A);
      return const Color(0xFF707070);
    }
    if (percent >= 85) return const Color(0xFF111111);
    if (percent >= 70) return const Color(0xFF3A3A3A);
    if (percent >= 50) return const Color(0xFF6B6B6B);
    return const Color(0xFF8E8E8E);
  }

  if (percent >= 85) return AppColors.successGreen;
  if (percent >= 70) return AppColors.mutedGold;
  if (percent >= 50) return AppColors.warmAmber;
  return AppColors.errorRed;
}

String percentLabel(double percent, [AppLocalizations? l10n]) {
  if (percent >= 85) return l10n?.percentExcellent ?? 'Excellent';
  if (percent >= 70) return l10n?.percentGood ?? 'Good';
  if (percent >= 50) return l10n?.percentNeedsWork ?? 'Needs work';
  return l10n?.percentAtRisk ?? 'At risk';
}

String formatPercent(double? percent) {
  if (percent == null) return '—';
  final rounded = percent.roundToDouble();
  return percent == rounded
      ? '${rounded.toInt()}%'
      : '${percent.toStringAsFixed(1)}%';
}

class PercentBadge extends StatelessWidget {
  const PercentBadge({super.key, required this.percent, this.compact = false});

  final double? percent;
  final bool compact;

  @override
  Widget build(BuildContext context) {
    final value = percent;
    final color = value == null
        ? AppColors.inkAlpha(context, 0.3)
        : percentColor(value, context: context);
    final tokens = AppStyleTokens.maybeOf(context);
    final isNature = tokens?.isNaturePaper ?? false;
    final radius = tokens?.isMonochromeWorkspace == true
        ? (compact ? 4.0 : 6.0)
        : isNature
```

```
        ? (compact ? 12.0 : 16.0)
        : (compact ? 8.0 : 12.0);

    return Container(
      padding: EdgeInsets.symmetric(
        horizontal: compact ? 10 : 14,
        vertical: compact ? 5 : 8,
      ),
      decoration: BoxDecoration(
        color: color.withValues(alpha: isNature ? 0.14 : 0.12),
        borderRadius: BorderRadius.circular(radius),
        border: tokens?.isMonochromeWorkspace == true
            ? Border.all(color: color.withValues(alpha: 0.28))
            : isNature
            ? Border.all(color: color.withValues(alpha: 0.22))
            : null,
      ),
      child: Text(
        formatPercent(value),
        style: TextStyle(
          color: color,
          fontWeight: FontWeight.w800,
          fontSize: compact ? 13 : 16,
          letterSpacing: -0.3,
          fontFeatures: tokens?.isMonochromeWorkspace == true
              ? const [FontFeature.tabularFigures()]
              : null,
        ),
      ),
    );
  }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/marks/term_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type TermSelector

**lib/components/marks/term_selector.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/marks/academic_term.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';

class TermSelector extends ConsumerWidget {
  const TermSelector({super.key, this.padding = EdgeInsets.zero});

  final EdgeInsets padding;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final current = ref.watch(selectedMarkTermProvider);
    final optionsAsync = ref.watch(academicTermOptionsProvider);
    final radius = paperControlRadius(context);

    return optionsAsync.when(
      loading: () => Padding(
        padding: padding,
        child: const SizedBox(
          height: 48,
          child: Center(child: CircularProgressIndicator(strokeWidth: 2)),
        ),
      ),
      error: (e, _) => Padding(
        padding: padding,
        child: Text(
          e.toString(),
          style: TextStyle(color: AppColors.errorRed, fontSize: 12),
        ),
      ),
      data: (options) {
        final selected = options.cast<AcademicTerm?>().firstWhere(
          (t) => t!.id == current.id,
          orElse: () => current,
        )!;

        return Padding(
          padding: padding,
          child: Container(
            padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 2),
            decoration: paperControlDecoration(context),
            child: Row(
              children: [
                PaperGlyph(
                  icon: Icons.calendar_month_outlined,
                  size: 18,
                  color: AppColors.accent(context),
                ),
                const SizedBox(width: 10),
                Expanded(
                  child: DropdownButtonHideUnderline(
                    child: DropdownButton<String>(
                      value: options.any((t) => t.id == selected.id)
                          ? selected.id
                          : options.first.id,
                      isExpanded: true,
                      borderRadius: BorderRadius.circular(radius),
                      dropdownColor: AppColors.controlSurface(context),
                      style: TextStyle(
                        fontWeight: FontWeight.w700,
```

```
                    fontSize: 14,
                    color: AppColors.ink(context),
                  ),
                  items: options
                      .map(
                        (t) => DropdownMenuItem(
                          value: t.id,
                          child: Text(
                            t.label(context.l10n),
                            overflow: TextOverflow.ellipsis,
                          ),
                        ),
                      )
                      .toList(),
                  onChanged: (id) {
                    if (id == null) return;
                    final next = options.firstWhere((t) => t.id == id);
                    ref
                        .read(selectedMarkTermProvider.notifier)
                        .select(next);
                  },
                ),
              ),
            ),
          ),
        ],
      ),
    ),
  );
 },
 );
 }
}
```

**WARNING — Reuse rule**
If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

## lib/components/settings/language_settings_tile.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

- Type LanguageSettingsTile

**lib/components/settings/language_settings_tile.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:mu_study/components/decor/paper_decor.dart';
import 'package:mu_study/core/locale/app_locale.dart';
import 'package:mu_study/core/theme/app_colors.dart';
import 'package:mu_study/l10n/l10n_x.dart';
import 'package:mu_study/providers/settings/locale_provider.dart';

class LanguageSettingsTile extends ConsumerWidget {
  const LanguageSettingsTile({super.key, this.embedded = false});

  final bool embedded;

  String _nativeName(AppLanguage lang) => switch (lang) {
    AppLanguage.english => 'English',
    AppLanguage.russian => '[redacted]',
    AppLanguage.uzbek => 'Oʻzbekcha',
  };

  String _codeLabel(AppLanguage lang) => switch (lang) {
    AppLanguage.english => 'EN',
    AppLanguage.russian => 'RU',
    AppLanguage.uzbek => 'UZ',
  };

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final l10n = context.l10n;
    final current = AppLanguage.fromLocale(ref.watch(localeProvider));
    final onSurface = Theme.of(context).colorScheme.onSurface;

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        if (!embedded) ...[
          Text(
            l10n.languageSection,
            style: Theme.of(
              context,
            ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
          ),
          const SizedBox(height: 4),
          Text(
            l10n.languageHint,
            style: TextStyle(
              fontSize: 12,
              color: AppColors.inkAlpha(context, 0.5),
            ),
          ),
          const SizedBox(height: 12),
        ] else ...[
          Text(
            l10n.languageSection,
            style: Theme.of(
              context,
            ).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),
          ),
          const SizedBox(height: 10),
        ],
        Container(
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
          decoration: paperControlDecoration(
            context,
```

```
  color: embedded
      ? AppColors.inkAlpha(context, 0.05)
      : AppColors.controlSurface(context),
),
child: Row(
  children: [
    PaperGlyph(
      icon: Icons.language_rounded,
      size: 18,
      color: AppColors.accent(context),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: DropdownButtonHideUnderline(
        child: DropdownButton<AppLanguage>(
          value: current,
          isExpanded: true,
          borderRadius: BorderRadius.circular(
            paperControlRadius(context),
          ),
          dropdownColor: AppColors.controlSurface(context),
          icon: Icon(
            Icons.keyboard_arrow_down_rounded,
            color: AppColors.inkAlpha(context, 0.45),
          ),
          style: TextStyle(
            fontWeight: FontWeight.w700,
            fontSize: 15,
            color: onSurface,
          ),
          items: AppLanguage.values
              .map(
                (lang) => DropdownMenuItem(
                  value: lang,
                  child: Row(
                    children: [
                      SizedBox(
                        width: 28,
                        child: Text(
                          _codeLabel(lang),
                          style: TextStyle(
                            fontWeight: FontWeight.w800,
                            fontSize: 12,
                            color: lang == current
                                ? AppColors.accent(context)
                                : AppColors.inkAlpha(context, 0.45),
                          ),
                        ),
                      ),
                      const SizedBox(width: 8),
                      Expanded(
                        child: Text(
                          _nativeName(lang),
                          overflow: TextOverflow.ellipsis,
                        ),
                      ),
                    ],
                  ),
                ),
              )
              .toList(),
          onChanged: (lang) {
            if (lang == null || lang == current) return;
            // rebuild (avoids semantics.parentDataDirty).
            WidgetsBinding.instance.addPostFrameCallback((_) {
              ref.read(localeProvider.notifier).setLanguage(lang);
            });
          },
        ),
      ),
    ),
  ],
),
```

```
          ),
        ],
      );
    }
}
```

> **WARNING — Reuse rule**
> If this widget starts calling supabase, it is no longer a dumb component. Push I/O back to a provider so tests and other screens stay simple.

# Every-line Dart commentary

This is the slowest chapter on purpose. Every non-trivial line of application Dart is printed with a one-sentence reason. Skip it while racing through tutorials; use it when a file feels like noise. Imports and lone braces are omitted.

# Every line — lib/components/analytics/teacher_analytics_charts.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `class AnalyticsChartPoint {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const AnalyticsChartPoint(this.label, this.value, this.color);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final double value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `class MarkDistributionDoughnut extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16** `const MarkDistributionDoughnut({super.key, required this.points});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final List<AnalyticsChartPoint> points;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final total = points.fold<double>(0, (sum, p) => sum + p.value).round();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final labelsInside =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `AppStyleTokens.maybeOf(context)?.prefersChartLabelsInside ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `height: 280,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `child: SfCircularChart(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `margin: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `annotations: <CircularChartAnnotation>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `CircularChartAnnotation(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `widget: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `l10n.analyticsTotalMarks(total),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `color: scheme.onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `height: 1.25,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `series: <CircularSeries<AnalyticsChartPoint, String>>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `DoughnutSeries<AnalyticsChartPoint, String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `dataSource: points,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `xValueMapper: (d, _) => d.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `yValueMapper: (d, _) => d.value,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `pointColorMapper: (d, _) => d.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `innerRadius: '62%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `radius: '78%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `strokeWidth: isDark ? 2 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `strokeColor: isDark ? const Color(0xFF1A1A1A) : scheme.surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `animationDuration: 1100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `animationDelay: 80,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `dataLabelMapper: (d, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final pct = total == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `? 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `: ((d.value / total) * 100).round();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `return '${d.value.round()}\n$pct%';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `dataLabelSettings: DataLabelSettings(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `isVisible: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `labelPosition: labelsInside`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `? ChartDataLabelPosition.inside`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `: ChartDataLabelPosition.outside,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `connectorLineSettings: ConnectorLineSettings(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `type: ConnectorType.curve,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `length: '12%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `width: labelsInside ? 0 : 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `color: scheme.onSurface.withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `textStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `color: labelsInside ? Colors.white : scheme.onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `fontWeight: labelsInside`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `? FontWeight.w700`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `height: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `spacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `runSpacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `alignment: WrapAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `for (final p in points)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `_LegendPill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `color: p.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `label: '${p.label}: ${p.value.round()}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `class AttendanceTrendBars extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L115** `const AttendanceTrendBars({super.key, required this.trend});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `final AttendanceWeekTrend trend;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `final isMono = tokens?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `final barRadius = isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `? const BorderRadius.vertical(top: Radius.circular(2))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `: const BorderRadius.vertical(top: Radius.circular(6));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `final labelOnBar = isMono && !isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `? AppColors.ink(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `: Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `final categories = <_WeekCategory>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `_WeekCategory(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `l10n.analyticsThisWeek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `trend.thisWeek.present.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `trend.thisWeek.absent.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `trend.thisWeek.late.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `_WeekCategory(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `l10n.analyticsLastWeek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `trend.lastWeek.present.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `trend.lastWeek.absent.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `trend.lastWeek.late.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `final maxY = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `trend.thisWeek.max,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `trend.lastWeek.max,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `].reduce((a, b) => a > b ? a : b);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `height: 260,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `child: SfCartesianChart(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `margin: const EdgeInsets.only(top: 8, right: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `plotAreaBorderWidth: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `primaryXAxis: CategoryAxis(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `majorGridLines: const MajorGridLines(width: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `axisLine: const AxisLine(width: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `majorTickLines: const MajorTickLines(size: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `labelStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `color: scheme.onSurface.withValues(alpha: 0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `primaryYAxis: NumericAxis(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `minimum: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `maximum: (maxY + 1).toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `interval: maxY <= 5 ? 1 : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `majorGridLines: MajorGridLines(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `width: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `color: scheme.onSurface.withValues(alpha: isDark ? 0.12 : 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `axisLine: const AxisLine(width: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `majorTickLines: const MajorTickLines(size: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `labelStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `color: scheme.onSurface.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `legend: Legend(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `isVisible: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `position: LegendPosition.bottom,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `overflowMode: LegendItemOverflowMode.wrap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `iconHeight: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `iconWidth: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `padding: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `itemPadding: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `textStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `color: scheme.onSurface.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `series: <CartesianSeries<_WeekCategory, String>>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `ColumnSeries<_WeekCategory, String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `name: l10n.attendancePresentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `dataSource: categories,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `xValueMapper: (d, _) => d.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `yValueMapper: (d, _) => d.present,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `dataLabelMapper: (d, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `d.present == 0 ? '' : '${d.present.round()}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `color: AppColors.success(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `gradient: isMono ? null : tokens?.chartPositive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `borderRadius: barRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `width: isMono ? 0.55 : 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `spacing: 0.18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `animationDuration: 1000,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `animationDelay: 60,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `dataLabelSettings: DataLabelSettings(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `isVisible: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `labelAlignment: ChartDataLabelAlignment.middle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `textStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `color: isMono && !isDark ? Colors.white : labelOnBar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `ColumnSeries<_WeekCategory, String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `name: l10n.attendanceAbsentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `dataSource: categories,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `xValueMapper: (d, _) => d.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `yValueMapper: (d, _) => d.absent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `dataLabelMapper: (d, _) =>`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `d.absent == 0 ? '' : '${d.absent.round()}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `color: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `gradient: isMono ? null : tokens?.chartDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `borderRadius: barRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `width: isMono ? 0.55 : 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `spacing: 0.18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `animationDuration: 1100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `animationDelay: 160,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `dataLabelSettings: DataLabelSettings(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `isVisible: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `labelAlignment: ChartDataLabelAlignment.middle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `textStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `color: isMono && !isDark ? Colors.white : labelOnBar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `ColumnSeries<_WeekCategory, String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `name: l10n.attendanceLateWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `dataSource: categories,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `xValueMapper: (d, _) => d.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `yValueMapper: (d, _) => d.late,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `dataLabelMapper: (d, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `d.late == 0 ? '' : '${d.late.round()}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `color: AppColors.warning(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `gradient: isMono ? null : tokens?.chartWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `borderRadius: barRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `width: isMono ? 0.55 : 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `spacing: 0.18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `animationDuration: 1200,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `animationDelay: 260,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `dataLabelSettings: DataLabelSettings(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `isVisible: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `labelAlignment: ChartDataLabelAlignment.middle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `textStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `color: isMono && !isDark ? Colors.white : labelOnBar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `List<AnalyticsChartPoint> markBandPoints(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `AppLocalizations l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `List<double> averages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `var excellent = 0, good = 0, needsWork = 0, atRisk = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `for (final value in averages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `if (value >= 85) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `excellent++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `} else if (value >= 70) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `good++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `} else if (value >= 50) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `needsWork++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `atRisk++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `return [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `AnalyticsChartPoint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `l10n.chartBandExcellent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `excellent.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `AppColors.success(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `AnalyticsChartPoint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `l10n.chartBandGood,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `good.toDouble(),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `AppColors.goldAccent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `AnalyticsChartPoint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `l10n.chartBandNeedsWork,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `needsWork.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `AppColors.warning(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `AnalyticsChartPoint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `l10n.chartBandAtRisk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `atRisk.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `].where((p) => p.value > 0).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `class _WeekCategory {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L320** `const _WeekCategory(this.label, this.present, this.absent, this.late);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `final double present;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `final double absent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `final double late;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `class _LegendPill extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L328** `const _LegendPill({required this.color, required this.label});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `? Colors.white.withValues(alpha: 0.06)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `: scheme.onSurface.withValues(alpha: 0.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `borderRadius: BorderRadius.circular(isMono ? 4 : 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `? Border.all(color: scheme.onSurface.withValues(alpha: 0.08))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `width: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `height: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `BoxShadow(color: color.withValues(alpha: 0.55), blurRadius: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `color: scheme.onSurface.withValues(alpha: 0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/analytics/teacher_analytics_charts.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/attachments/attachment_preview.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L10** `const _imageExtensions = {'jpg', 'jpeg', 'png', 'gif', 'webp', 'heic', 'heif'};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `bool isImageAttachment(String? nameOrUrl) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final ext = attachmentExtension(nameOrUrl);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `return ext != null && _imageExtensions.contains(ext);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `bool isPdfAttachment(String? nameOrUrl) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `return attachmentExtension(nameOrUrl) == 'pdf';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `String? attachmentExtension(String? nameOrUrl) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `if (nameOrUrl == null || nameOrUrl.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final clean = Uri.tryParse(nameOrUrl)?.path ?? nameOrUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final base = clean.split('/').last.split('?').first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final dot = base.lastIndexOf('.');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `if (dot < 0 || dot == base.length - 1) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `return base.substring(dot + 1).toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `String attachmentFileName(String? nameOrUrl, {String fallback = 'Attachment'}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (nameOrUrl == null || nameOrUrl.isEmpty) return fallback;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final clean = Uri.tryParse(nameOrUrl)?.path ?? nameOrUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final base = clean.split('/').last.split('?').first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return base.isEmpty ? fallback : Uri.decodeComponent(base);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `String? storagePathFromSignedUrl(String url, {String? bucket}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final uri = Uri.tryParse(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `if (uri == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final segments = uri.pathSegments;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `final signIdx = segments.indexOf('sign');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `if (signIdx >= 0 && signIdx + 2 < segments.length) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `final bucketName = segments[signIdx + 1];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `if (bucket != null && bucketName != bucket) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `return segments.sublist(signIdx + 2).map(Uri.decodeComponent).join('/');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `Future<void> openAttachment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `required String url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `String? fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `Uint8List? bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final name = fileName ?? attachmentFileName(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (isImageAttachment(name)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `barrierColor: Colors.black.withValues(alpha: 0.92),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `builder: (ctx) => _ImageLightbox(url: url, bytes: bytes, title: name),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (isPdfAttachment(name)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `await openPdfViewer(context, url, title: name);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final uri = Uri.tryParse(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (uri != null && await canLaunchUrl(uri)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `await launchUrl(uri, mode: LaunchMode.externalApplication);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `} else if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `SnackBar(content: Text(context.l10n.attachmentCouldNotOpen)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `class AttachmentPreview extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L84** `const AttachmentPreview({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `this.fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `this.bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `this.onRemove,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `this.enabled = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final String? fileName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final Uint8List? bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `final VoidCallback? onRemove;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final bool enabled;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final name =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `fileName ?? attachmentFileName(url, fallback: l10n.taskAttachment);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final isImage = isImageAttachment(name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `return isImage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `? _ImagePreviewCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `url: url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `bytes: bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `fileName: name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `subtitle: l10n.attachmentImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `tapPreview: l10n.attachmentTapPreview,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `onOpen: enabled`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `? () => openAttachment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `url: url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `fileName: name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `bytes: bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `onRemove: enabled ? onRemove : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `: _FilePreviewCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `fileName: name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `kindLabel: isPdfAttachment(name)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `? l10n.attachmentPdf`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `: l10n.attachmentFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `tapOpen: l10n.attachmentTapOpen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `onOpen: enabled`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `? () => openAttachment(context, url: url, fileName: name)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `onRemove: enabled ? onRemove : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `class _ImagePreviewCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L138** `const _ImagePreviewCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `required this.fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `required this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `required this.tapPreview,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `this.bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `this.onOpen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `this.onRemove,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `final String fileName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final String subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `final String tapPreview;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `final Uint8List? bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `final VoidCallback? onOpen;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final VoidCallback? onRemove;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `onTap: onOpen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `child: Ink(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `color: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `border: Border.all(color: AppColors.inkAlpha(context, 0.06)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `color: AppColors.inkAlpha(context, 0.06),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `offset: const Offset(0, 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `borderRadius: const BorderRadius.vertical(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `top: Radius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `child: AspectRatio(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `aspectRatio: 16 / 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `ColoredBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `color: AppColors.softWash(context, lightAlpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `child: bytes != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `? Image.memory(bytes!, fit: BoxFit.cover)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `: CachedNetworkImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `imageUrl: url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `fit: BoxFit.cover,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `placeholder: (_, _) => Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `width: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `height: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `errorWidget: (_, _, _) => Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `Icons.broken_image_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `size: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `bottom: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `child: DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `begin: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `end: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `child: const SizedBox(height: 48),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `left: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `bottom: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `Icons.zoom_in_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `color: Colors.white.withValues(alpha: 0.95),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `tapPreview,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `color: Colors.white.withValues(alpha: 0.95),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `if (onRemove != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `top: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `right: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `child: _RemoveButton(onPressed: onRemove!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `padding: const EdgeInsets.fromLTRB(14, 12, 14, 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `width: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `height: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `color: AppColors.mutedGold.withValues(alpha: 0.15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `borderRadius: BorderRadius.circular(10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `child: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `Icons.image_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `class _FilePreviewCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L321** `const _FilePreviewCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `required this.fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `required this.kindLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `required this.tapOpen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `this.onOpen,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `this.onRemove,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `final String fileName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `final String kindLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `final String tapOpen;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `final VoidCallback? onOpen;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `final VoidCallback? onRemove;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `final isPdf = isPdfAttachment(fileName);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `final accent = isPdf ? AppColors.coffeeBrown : AppColors.softOlive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `final icon = isPdf`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `? Icons.picture_as_pdf_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `: Icons.insert_drive_file_outlined;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `onTap: onOpen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `child: Ink(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `padding: const EdgeInsets.all(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `color: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `border: Border.all(color: AppColors.inkAlpha(context, 0.06)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `color: AppColors.inkAlpha(context, 0.06),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `offset: const Offset(0, 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `width: 52,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `height: 52,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `color: accent.withValues(alpha: 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `borderRadius: BorderRadius.circular(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `child: Icon(icon, color: accent, size: 26),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `height: 1.25,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `kindLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `tapOpen,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `color: accent.withValues(alpha: 0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `if (onRemove != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `const SizedBox(width: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `_RemoveButton(onPressed: onRemove!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `] else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `class _RemoveButton extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L425** `const _RemoveButton({required this.onPressed});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `final VoidCallback onPressed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `color: Colors.white.withValues(alpha: 0.95),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `shape: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `elevation: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `shadowColor: AppColors.inkAlpha(context, 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `customBorder: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `onTap: onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `child: const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `padding: EdgeInsets.all(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `Icons.delete_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `class _ImageLightbox extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L453** `const _ImageLightbox({required this.url, required this.title, this.bytes});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `final Uint8List? bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `return Dialog.fullscreen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `backgroundColor: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `child: SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `child: InteractiveViewer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `minScale: 0.8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `maxScale: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `child: bytes != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `? Image.memory(bytes!)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `: CachedNetworkImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `imageUrl: url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `placeholder: (_, _) => const CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `errorWidget: (_, _, _) => const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `Icons.broken_image_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `color: Colors.white70,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `size: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `top: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `left: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `right: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `onPressed: () => Navigator.of(context).pop(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `icon: const Icon(Icons.close_rounded, color: Colors.white),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/attachments/attachment_preview.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/attachments/voice_note_field.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `class VoiceNoteField extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14** `const VoiceNoteField({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `this.voiceUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `this.enabled = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required this.onUpload,`
Storage write. Path folders must satisfy bucket policies.

**L19** `this.onRemove,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String? voiceUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final bool enabled;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final Future<void> Function(Uint8List bytes, String fileName) onUpload;`
Storage write. Path folders must satisfy bucket policies.

**L25** `final VoidCallback? onRemove;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `State<VoiceNoteField> createState() => _VoiceNoteFieldState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `class _VoiceNoteFieldState extends State<VoiceNoteField> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L32** `final AudioRecorder _recorder = AudioRecorder();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final _voice = VoicePlaybackController.instance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `bool _recording = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `bool _busy = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `String? _localPath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `Duration _elapsed = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `Timer? _tick;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `bool get _hasNote =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `(widget.voiceUrl != null && widget.voiceUrl!.isNotEmpty) ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `_localPath != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `String? get _playKey => _localPath ?? widget.voiceUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `_voice.addListener(_onVoiceChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `_tick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `_voice.removeListener(_onVoiceChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `_recorder.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `void _onVoiceChanged() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `if (mounted) setState(() {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `String _format(Duration d) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `return '$m:$s';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `Future<String> _recordingPath() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final stamp = DateTime.now().millisecondsSinceEpoch;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final ext = kIsWeb ? 'wav' : 'm4a';`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `final name = 'voice_$stamp.$ext';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `if (kIsWeb) return name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final dir = await getTemporaryDirectory();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `return p.join(dir.path, name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `Future<void> _startRecording() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `if (!widget.enabled || _busy || _recording) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `final allowed = await _recorder.hasPermission();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `if (!allowed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `).showSnackBar(SnackBar(content: Text(context.l10n.voiceMicPermission)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `await _voice.stop();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final path = await _recordingPath();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `await _recorder.start(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `RecordConfig(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `encoder: kIsWeb ? AudioEncoder.wav : AudioEncoder.aacLc,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `bitRate: 128000,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `sampleRate: 44100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `path: path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `_recording = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `_elapsed = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_localPath = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `_tick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `_tick = Timer.periodic(const Duration(seconds: 1), (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `setState(() => _elapsed += const Duration(seconds: 1));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `SnackBar(content: Text(context.l10n.voiceCouldNotStart('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `Future<void> _stopRecording() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `if (!_recording) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `_tick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `_recording = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `_busy = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `final path = await _recorder.stop();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `if (path == null || path.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `throw Exception('No audio was captured.');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `final bytes = await XFile(path).readAsBytes();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `if (bytes.isEmpty) throw Exception('Recording was empty.');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `final fileName = kIsWeb`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `? 'voice_${DateTime.now().millisecondsSinceEpoch}.wav'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `: p.basename(path);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `setState(() => _localPath = path);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `await widget.onUpload(bytes, fileName);`
Storage write. Path folders must satisfy bucket policies.

**L144** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `SnackBar(content: Text(context.l10n.voiceCouldNotSave('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `setState(() => _localPath = null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `if (mounted) setState(() => _busy = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `Future<void> _cancelRecording() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `_tick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `await _recorder.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `_recording = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `_elapsed = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `Future<void> _togglePlay() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `if (_busy || _recording || !_hasNote) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `final key = _playKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `if (key == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `if (_localPath != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `await _voice.toggleLocal(_localPath!);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `} else if (widget.voiceUrl != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `await _voice.toggleRemote(widget.voiceUrl!);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `Future<void> _remove() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `await _voice.stop();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `_localPath = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `widget.onRemove?.call();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `if (_recording) return _buildRecordingCard();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `if (_hasNote) return _buildPlaybackCard();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `return _buildIdleCard();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `Widget _buildIdleCard() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `return _VoiceShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `_RoundAction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `icon: Icons.mic_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `onPressed: widget.enabled && !_busy ? _startRecording : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `context.l10n.taskVoice,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `context.l10n.voiceTapToRecord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `if (_busy)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `height: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `Widget _buildRecordingCard() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `return _VoiceShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `accent: AppColors.errorRed.withValues(alpha: 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `_PulsingDot(active: _recording),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `context.l10n.voiceRecording,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `_format(_elapsed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `fontFeatures: const [FontFeature.tabularFigures()],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `color: AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `tooltip: context.l10n.commonCancel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `onPressed: _cancelRecording,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `icon: const Icon(Icons.close_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `const SizedBox(width: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `_RoundAction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `icon: Icons.stop_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `onPressed: _stopRecording,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `Widget _buildPlaybackCard() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `final key = _playKey ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `final playing = _voice.isPlaying(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `final loading = _voice.isLoading(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `final position = playing || loading ? _voice.position : Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `final duration = playing || loading ? _voice.duration : Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `final progress = duration.inMilliseconds == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `? 0.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `: (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `return _VoiceShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `_RoundAction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `icon: loading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `? Icons.hourglass_top_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `: playing`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `? Icons.stop_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `: Icons.play_arrow_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `onPressed: widget.enabled && !_busy ? _togglePlay : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `context.l10n.voiceNote,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `duration == Duration.zero`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `? context.l10n.voiceTapPlay`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `: '${_format(position)} / ${_format(duration)}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `fontFeatures: const [FontFeature.tabularFigures()],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `if (_busy)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `padding: EdgeInsets.only(right: 8),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `height: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `if (widget.onRemove != null && widget.enabled)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `tooltip: context.l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `onPressed: _busy ? null : _remove,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `icon: const Icon(Icons.delete_outline_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `borderRadius: BorderRadius.circular(4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `child: LinearProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `value: playing || position > Duration.zero ? progress : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `minHeight: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `backgroundColor: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `if (widget.enabled && widget.onRemove != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `alignment: Alignment.centerLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `child: TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `onPressed: _busy || _recording ? null : _startRecording,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `icon: const Icon(Icons.mic_none_rounded, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `label: Text(context.l10n.voiceRerecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `class VoiceNotePlayer extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L391** `const VoiceNotePlayer({super.key, required this.url, this.title});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `final String? title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `State<VoiceNotePlayer> createState() => _VoiceNotePlayerState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `class _VoiceNotePlayerState extends State<VoiceNotePlayer> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L401** `final _voice = VoicePlaybackController.instance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `_voice.addListener(_onVoiceChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `_voice.removeListener(_onVoiceChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `void _onVoiceChanged() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `if (mounted) setState(() {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `String _format(Duration d) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `final m = d.inMinutes.remainder(60).toString().padLeft(2, '0');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `final s = d.inSeconds.remainder(60).toString().padLeft(2, '0');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `return '$m:$s';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `Future<void> _toggle() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `await _voice.toggleRemote(widget.url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `final playing = _voice.isPlaying(widget.url);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `final loading = _voice.isLoading(widget.url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `final position = playing || loading ? _voice.position : Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `final duration = playing || loading ? _voice.duration : Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `final progress = duration.inMilliseconds == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `? 0.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `: (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `return _VoiceShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `_RoundAction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `icon: loading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `? Icons.hourglass_top_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `: playing`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `? Icons.stop_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `: Icons.play_arrow_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `onPressed: _toggle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `widget.title ?? l10n.voiceNote,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `duration == Duration.zero`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `? l10n.voiceStudentAnswer`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `: '${_format(position)} / ${_format(duration)}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `borderRadius: BorderRadius.circular(4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `child: LinearProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `value: playing || position > Duration.zero ? progress : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `minHeight: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `backgroundColor: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `class _VoiceShell extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L506** `const _VoiceShell({required this.child, this.accent});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `final Color? accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `final bg =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `accent ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `(isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `? AppColors.darkSurfaceElevated`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `: AppColors.sand.withValues(alpha: 0.85));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `width: double.infinity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `padding: const EdgeInsets.all(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `color: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `? AppColors.darkBorder.withValues(alpha: 0.55)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `: AppColors.inkAlpha(context, 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `color: Colors.black.withValues(alpha: isDark ? 0.28 : 0.06),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `offset: const Offset(0, 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `child: DefaultTextStyle.merge(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `style: TextStyle(color: AppColors.ink(context)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `class _RoundAction extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L548** `const _RoundAction({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `required this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `required this.onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `final VoidCallback? onPressed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `color: color.withValues(alpha: onPressed == null ? 0.35 : 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `shape: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `customBorder: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `onTap: onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `width: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `height: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `child: Icon(icon, color: Colors.white, size: 26),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `class _PulsingDot extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L577** `const _PulsingDot({required this.active});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `final bool active;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `State<_PulsingDot> createState() => _PulsingDotState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L585** `class _PulsingDotState extends State<_PulsingDot>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L586** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L587** `late final AnimationController _controller = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `duration: const Duration(milliseconds: 900),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `)..repeat(reverse: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L592** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `_controller.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `return ScaleTransition(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `scale: Tween(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `begin: 0.85,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L603** `end: 1.15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `).animate(CurvedAnimation(parent: _controller, curve: Curves.easeInOut)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `child: Container(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `width: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `height: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `decoration: const BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/attachments/voice_note_field.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/attendance/attendance_calendar_grid.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `Color attendanceStatusColor(AttendanceStatus? status, {BuildContext? context}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `return switch (status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `AttendanceStatus.present => AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `AttendanceStatus.late => AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `AttendanceStatus.absent => AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `AttendanceStatus.excused => const Color(0xFF5B8DB8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `null =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `context != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `? AppColors.inkAlpha(context, 0.08)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `: const Color(0xFF808080).withValues(alpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `List<List<DateTime?>> buildMonthWeekRows(DateTime month) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final first = DateTime(month.year, month.month, 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final daysInMonth = DateTime(month.year, month.month + 1, 0).day;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final startOffset = first.weekday - 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final cells = <DateTime?>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `for (var i = 0; i < startOffset; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `cells.add(null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `for (var d = 1; d <= daysInMonth; d++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `cells.add(DateTime(month.year, month.month, d));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `while (cells.length % 7 != 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `cells.add(null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final weeks = <List<DateTime?>>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `for (var i = 0; i < cells.length; i += 7) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `weeks.add(cells.sublist(i, i + 5));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `while (weeks.isNotEmpty && weeks.last.every((d) => d == null)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `weeks.removeLast();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `return weeks;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `class AttendanceCalendarGrid extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L47** `const AttendanceCalendarGrid({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `required this.month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `required this.statusByDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `this.onDayTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `this.editable = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `final DateTime month;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final Map<DateTime, AttendanceStatus> statusByDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final ValueChanged<DateTime>? onDayTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final bool editable;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `DateTime _dayKey(DateTime d) => DateTime(d.year, d.month, d.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final weeks = buildMonthWeekRows(month);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final labels = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `l10n.attendanceWeekdayMon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `l10n.attendanceWeekdayTue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `l10n.attendanceWeekdayWed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `l10n.attendanceWeekdayThu,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `l10n.attendanceWeekdayFri,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `for (final label in labels)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `for (final week in weeks) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `for (final day in week)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `padding: const EdgeInsets.all(3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `child: _DayCell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `day: day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `status: day == null ? null : statusByDate[_dayKey(day)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `editable:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `editable &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `day != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `isAttendanceDateAllowed(day),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `isFuture: day != null && !isAttendanceDateAllowed(day),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `onTap:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `day == null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `onDayTap == null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `!isAttendanceDateAllowed(day)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `: () => onDayTap!(day),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `class _DayCell extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L128** `const _DayCell({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `required this.day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `required this.status,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `required this.editable,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `this.isFuture = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `final DateTime? day;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `final AttendanceStatus? status;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `final bool editable;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `final bool isFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `if (day == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `return AspectRatio(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `aspectRatio: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `child: DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `color: AppColors.inkAlpha(context, 0.03),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `borderRadius: BorderRadius.circular(10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `final color = attendanceStatusColor(status, context: context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `final hasStatus = status != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `final today = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final isToday =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `day!.year == today.year &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `day!.month == today.month &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `day!.day == today.day;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `final bg = isFuture`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `? AppColors.inkAlpha(context, 0.04)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `: hasStatus`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `? color.withValues(alpha: 0.92)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `: AppColors.inkAlpha(context, 0.06);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `return AspectRatio(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `aspectRatio: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `child: Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `color: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `borderRadius: BorderRadius.circular(10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `onTap: editable ? onTap : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `borderRadius: BorderRadius.circular(10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `child: DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `borderRadius: BorderRadius.circular(10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `border: isToday`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `? Border.all(color: AppColors.accent(context), width: 1.5)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `'${day!.day}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `color: isFuture`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `? AppColors.inkAlpha(context, 0.22)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `: hasStatus`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `? Colors.white`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `class AttendanceMonthHeader extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L207** `const AttendanceMonthHeader({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `required this.month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `required this.onPrev,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `required this.onNext,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `final DateTime month;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `final VoidCallback onPrev;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `final VoidCallback onNext;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `final label = DateFormat.yMMMM(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `Localizations.localeOf(context).toString(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `).format(month);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `onPressed: onPrev,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `icon: const Icon(Icons.chevron_left_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `).textTheme.titleMedium?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `onPressed: onNext,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `icon: const Icon(Icons.chevron_right_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `class AttendanceStatsBar extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L249** `const AttendanceStatsBar({super.key, required this.stats});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `final AttendanceStats stats;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `child: _StatChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `label: l10n.attendancePercentLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `value: '${stats.presentPercent.toStringAsFixed(0)}%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `child: _StatChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `label: l10n.attendancePresentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `value: '${stats.present}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `child: _StatChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `label: l10n.attendanceAbsentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `value: '${stats.absent}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `child: _StatChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `label: l10n.attendanceLateWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `value: '${stats.late}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `class _StatChip extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L295** `const _StatChip({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `required this.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `required this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `final String value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `color: color.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `fontSize: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `class AttendanceLegendRow extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L341** `const AttendanceLegendRow({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `return Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `spacing: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `runSpacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `_Legend(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `label: l10n.attendancePresentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `_Legend(color: AppColors.warmAmber, label: l10n.attendanceLateWord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `_Legend(color: AppColors.errorRed, label: l10n.attendanceAbsentWord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `_Legend(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `color: const Color(0xFF5B8DB8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `label: l10n.attendanceExcusedWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `_Legend(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `color: AppColors.inkAlpha(context, 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `label: l10n.attendanceNoLesson,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `class _Legend extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L370** `const _Legend({required this.color, required this.label});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `width: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `height: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `borderRadius: BorderRadius.circular(3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `AttendanceStatus nextAttendanceStatus(AttendanceStatus? current) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `return switch (current) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `null => AttendanceStatus.present,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `AttendanceStatus.present => AttendanceStatus.late,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `AttendanceStatus.late => AttendanceStatus.absent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `AttendanceStatus.absent => AttendanceStatus.excused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `AttendanceStatus.excused => AttendanceStatus.present,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/attendance/attendance_calendar_grid.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/backgrounds/nature_paper_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L7** `const _kAssetDir = 'assets/images/nature-paper';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `class NaturePaperBackdrop extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `const NaturePaperBackdrop({super.key, required this.child});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `State<NaturePaperBackdrop> createState() => _NaturePaperBackdropState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `class _NaturePaperBackdropState extends State<NaturePaperBackdrop>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L20** `late final AnimationController _transition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `bool? _wasNightScene;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `_transition = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `duration: const Duration(milliseconds: 1700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `_transition.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `final isNature = tokens?.isNaturePaper ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final bool? nightScene = isNature ? isDark : null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `if (nightScene != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `_wasNightScene != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `nightScene != _wasNightScene) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `_transition.forward(from: 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `_wasNightScene = nightScene;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `Positioned.fill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `child: RepaintBoundary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `child: ExcludeSemantics(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `child: isNature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `? AnimatedSwitcher(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `duration: const Duration(milliseconds: 650),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `switchInCurve: Curves.easeOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `switchOutCurve: Curves.easeIn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `transitionBuilder: _fadeWithoutChildKey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `child: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `? const _NightScene(key: ValueKey('nature-night'))`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `: const _DayScene(key: ValueKey('nature-day')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `: const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `Positioned.fill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `child: _TransitionLeaves(animation: _transition),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Widget _fadeWithoutChildKey(Widget child, Animation<double> animation) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `return FadeTransition(opacity: animation, child: child);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `class _DayScene extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L87** `const _DayScene({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `State<_DayScene> createState() => _DaySceneState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `class _DaySceneState extends State<_DayScene>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L94** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L95** `late final AnimationController _drift;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `static const _clouds = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `_CloudSpec('white-cloud.png', y: 0.07, size: 0.30, speed: 1.7, phase: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `_CloudSpec('flat-cloud.png', y: 0.16, size: 0.22, speed: 1.15, phase: 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `_CloudSpec('gray-cloud.png', y: 0.26, size: 0.17, speed: 1.4, phase: 0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `static const _leaves = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `_LeafSpec('beige-leaf.png', x: 0.12, size: 26, speed: 0.9, phase: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `_LeafSpec('tan-seed.png', x: 0.32, size: 18, speed: 1.3, phase: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `_LeafSpec('white-daisy.png', x: 0.55, size: 22, speed: 0.7, phase: 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_LeafSpec('beige-leaf.png', x: 0.74, size: 20, speed: 1.1, phase: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `_LeafSpec('salmon-flower.png', x: 0.88, size: 24, speed: 0.8, phase: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `_drift = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `duration: const Duration(seconds: 36),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `)..repeat();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `_drift.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `return LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `final w = constraints.maxWidth;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `final h = constraints.maxHeight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `const DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `begin: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `end: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `stops: [0.0, 0.52, 1.0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `Color(0xFFEFE4CD),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `NaturePaperColors.paleBlue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `top: -h * 0.12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `left: -w * 0.15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `width: w * 0.9,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `height: w * 0.9,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `NaturePaperColors.sunGlow.withValues(alpha: 0.30),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `NaturePaperColors.sunGlow.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `const CustomPaint(painter: _PaperGrainPainter(isDark: false)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `bottom: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `'$_kAssetDir/landscape-hills-footer.png',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `width: w,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `fit: BoxFit.fitWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `opacity: const AlwaysStoppedAnimation(0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `animation: _drift,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `builder: (context, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `final t = _drift.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `for (final c in _clouds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `left: (((t * c.speed + c.phase) % 1.2) - 0.15) * w,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `top: c.y * h,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `'$_kAssetDir/${c.asset}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `width: (w * c.size).clamp(70.0, 220.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `opacity: const AlwaysStoppedAnimation(0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `for (final leaf in _leaves)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `left:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `leaf.x * w +`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `math.sin(t * math.pi * 8 + leaf.phase * 7) * 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `top:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `(((t * leaf.speed * 2 + leaf.phase) % 1.1) - 0.05) *`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `h,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `angle:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `math.sin(t * math.pi * 6 + leaf.phase * 9) * 0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `'$_kAssetDir/${leaf.asset}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `width: leaf.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `opacity: const AlwaysStoppedAnimation(0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `class _NightScene extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L227** `const _NightScene({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `State<_NightScene> createState() => _NightSceneState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `class _NightSceneState extends State<_NightScene>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L234** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L235** `late final AnimationController _drift;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `_drift = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `duration: const Duration(seconds: 40),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `)..repeat();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `_drift.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `return LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `final w = constraints.maxWidth;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `final h = constraints.maxHeight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `const DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `begin: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `end: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `stops: [0.0, 1.0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `NaturePaperColors.midnight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `NaturePaperColors.forestNight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `top: -h * 0.10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `right: -w * 0.20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `width: w * 0.85,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `height: w * 0.85,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `NaturePaperColors.moonGlow.withValues(alpha: 0.16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `NaturePaperColors.moonGlow.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `const CustomPaint(painter: _PaperGrainPainter(isDark: true)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `animation: _drift,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `builder: (context, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `CustomPaint(painter: _NightSkyPainter(t: _drift.value)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `bottom: h * 0.03,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `child: ColorFiltered(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `colorFilter: const ColorFilter.mode(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `Color(0xFF1C2822),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `BlendMode.srcIn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `'$_kAssetDir/landscape-hills-footer.png',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `width: w,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `fit: BoxFit.fitWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `opacity: const AlwaysStoppedAnimation(0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `bottom: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `child: ColorFiltered(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `colorFilter: const ColorFilter.mode(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `Color(0xFF121B16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `BlendMode.srcIn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `'$_kAssetDir/landscape-hills-footer.png',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `width: w,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `fit: BoxFit.fitWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `class _NightSkyPainter extends CustomPainter {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L340** `const _NightSkyPainter({required this.t});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `final double t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `static const _starCount = 70;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `static const _fireflyCount = 9;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `void paint(Canvas canvas, Size size) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `final rng = math.Random(7);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `final starPaint = Paint();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `for (var i = 0; i < _starCount; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `final x = rng.nextDouble() * size.width;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `final y = rng.nextDouble() * size.height * 0.55;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `final r = 0.5 + rng.nextDouble() * 0.9;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `final baseAlpha = 0.10 + rng.nextDouble() * 0.25;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `final twinkle = i % 3 == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `? 0.5 + 0.5 * math.sin(t * math.pi * 10 + i * 1.7)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `: 1.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `starPaint.color = NaturePaperColors.moonBeige.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `alpha: baseAlpha * twinkle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `canvas.drawCircle(Offset(x, y), r, starPaint);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `final glowPaint = Paint()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `..maskFilter = const MaskFilter.blur(BlurStyle.normal, 5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `final corePaint = Paint();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `final fly = math.Random(23);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `for (var i = 0; i < _fireflyCount; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `final baseX = fly.nextDouble();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `final baseY = 0.35 + fly.nextDouble() * 0.55;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `final phase = fly.nextDouble() * math.pi * 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `final x =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `(baseX +`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `0.05 * math.sin(t * math.pi * 4 + phase) +`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `0.02 * math.sin(t * math.pi * 14 + phase * 3)) *`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `size.width;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `final y =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `(baseY + 0.03 * math.cos(t * math.pi * 6 + phase * 2)) * size.height;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `final pulse = (0.5 + 0.5 * math.sin(t * math.pi * 8 + phase * 5)).clamp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `0.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `if (pulse < 0.15) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `glowPaint.color = NaturePaperColors.amber.withValues(alpha: 0.35 * pulse);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `corePaint.color = const Color(0xFFF4D9A0).withValues(alpha: 0.85 * pulse);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `canvas.drawCircle(Offset(x, y), 5, glowPaint);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `canvas.drawCircle(Offset(x, y), 1.6, corePaint);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `bool shouldRepaint(_NightSkyPainter oldDelegate) => oldDelegate.t != t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `class _PaperGrainPainter extends CustomPainter {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L397** `const _PaperGrainPainter({required this.isDark});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `final bool isDark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `void paint(Canvas canvas, Size size) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `final rng = math.Random(42);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `final paint = Paint()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `..color = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `? NaturePaperColors.moonBeige.withValues(alpha: 0.025)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `: NaturePaperColors.warmBrown.withValues(alpha: 0.035);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `final count = (size.width * size.height / 3200).round().clamp(120, 700);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `for (var i = 0; i < count; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `canvas.drawCircle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `Offset(rng.nextDouble() * size.width, rng.nextDouble() * size.height),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `0.6 + rng.nextDouble() * 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `paint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `bool shouldRepaint(_PaperGrainPainter oldDelegate) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `oldDelegate.isDark != isDark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `class _TransitionLeaves extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L424** `const _TransitionLeaves({required this.animation});`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `final Animation<double> animation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `static const _sprites = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `_LeafSpec('beige-leaf.png', x: 0.08, size: 30, speed: 1.0, phase: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `_LeafSpec('white-daisy.png', x: 0.25, size: 24, speed: 1.3, phase: 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `_LeafSpec('tan-seed.png', x: 0.42, size: 20, speed: 0.9, phase: 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `_LeafSpec('beige-leaf.png', x: 0.58, size: 26, speed: 1.2, phase: 0.15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `_LeafSpec('salmon-flower.png', x: 0.72, size: 24, speed: 1.0, phase: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `_LeafSpec('beige-leaf.png', x: 0.88, size: 22, speed: 1.4, phase: 0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `return AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `animation: animation,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `builder: (context, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `final t = animation.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `if (t == 0 || animation.isCompleted) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `final size = MediaQuery.sizeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `final opacity = math.sin(t * math.pi).clamp(0.0, 1.0) * 0.85;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `for (final leaf in _sprites)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `left:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `leaf.x * size.width +`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `math.sin(t * math.pi * 3 + leaf.phase * 8) * 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `top:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `(t * leaf.speed * 1.1 - 0.08 + leaf.phase * 0.1) *`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `size.height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `child: Opacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `opacity: opacity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `angle: t * math.pi * 2 * leaf.speed + leaf.phase * 5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `'$_kAssetDir/${leaf.asset}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `width: leaf.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `class _CloudSpec {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L476** `const _CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `this.asset, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `required this.y,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `required this.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `required this.speed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `required this.phase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `final String asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `final double y;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `final double speed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `final double phase;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `class _LeafSpec {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L492** `const _LeafSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `this.asset, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `required this.x,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `required this.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `required this.speed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `required this.phase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `final String asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `final double x;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `final double speed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `final double phase;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/backgrounds/nature_paper_backdrop.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/backgrounds/soft_neumorphism_backdrop.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class SoftNeumorphismBackdrop extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const SoftNeumorphismBackdrop({super.key, required this.child});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final isNeu = AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `Positioned.fill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `child: RepaintBoundary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `child: ExcludeSemantics(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `child: isNeu`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `? AnimatedSwitcher(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `duration: const Duration(milliseconds: 550),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `switchInCurve: Curves.easeOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `switchOutCurve: Curves.easeIn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `transitionBuilder: _fadeWithoutChildKey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `child: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `? const _NightAmbient(key: ValueKey('neu-night'))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `: const _MorningAmbient(key: ValueKey('neu-day')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `: const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `Widget _fadeWithoutChildKey(Widget child, Animation<double> animation) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `return FadeTransition(opacity: animation, child: child);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `class _MorningAmbient extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L45** `const _MorningAmbient({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `return DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `decoration: const BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `begin: Alignment.topLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `end: Alignment.bottomRight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `SoftNeumorphismColors.morningBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `SoftNeumorphismColors.morningBgCool,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `Color(0xFFECE8E0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `stops: [0.0, 0.55, 1.0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `top: -80,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `right: -60,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `width: 280,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `height: 280,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `SoftNeumorphismColors.sage.withValues(alpha: 0.10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `SoftNeumorphismColors.sage.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `bottom: -40,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `left: -40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `width: 240,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `height: 240,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `SoftNeumorphismColors.amber.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `SoftNeumorphismColors.amber.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `class _NightAmbient extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L109** `const _NightAmbient({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `return DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `decoration: const BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `begin: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `end: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `SoftNeumorphismColors.nightBgCool,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `SoftNeumorphismColors.nightBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `Color(0xFF1C2220),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `stops: [0.0, 0.5, 1.0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `top: -100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `left: -40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `width: 300,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `height: 300,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `SoftNeumorphismColors.emerald.withValues(alpha: 0.09),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `SoftNeumorphismColors.emerald.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `bottom: -60,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `right: -50,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `width: 260,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `height: 260,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `SoftNeumorphismColors.amber.withValues(alpha: 0.06),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `SoftNeumorphismColors.amber.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/backgrounds/soft_neumorphism_backdrop.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/brand/app_logo.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L3** `class AppLogo {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `static const asset = 'assets/images/logo/logo.jpeg';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `class AppLogoImage extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const AppLogoImage({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.size = 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.borderRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.fit = BoxFit.cover,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final BorderRadius? borderRadius;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final BoxFit fit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final radius = borderRadius ?? BorderRadius.circular(size * 0.22);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `return ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `AppLogo.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `width: size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `height: size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `fit: fit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `semanticLabel: 'MU Study',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/brand/app_logo.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/buttons/primary_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class PrimaryButton extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `const PrimaryButton({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.loading = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.backgroundColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.foregroundColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final VoidCallback? onPressed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final bool loading;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final IconData? icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final Color? backgroundColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final Color? foregroundColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final theme = Theme.of(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final nature = isNaturePaper(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final bg =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `backgroundColor ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `(nature ? AppColors.accent(context) : AppColors.coffeeBrown);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final fg =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `foregroundColor ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `(nature ? theme.colorScheme.onPrimary : Colors.white);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final ButtonStyle style;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (nature && backgroundColor == null && foregroundColor == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `style = (theme.elevatedButtonTheme.style ?? const ButtonStyle()).merge(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `ElevatedButton.styleFrom(minimumSize: const Size.fromHeight(56)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `style = ElevatedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `backgroundColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `foregroundColor: fg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `minimumSize: const Size.fromHeight(56),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `borderRadius: BorderRadius.circular(nature ? 18 : 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `elevation: nature ? 3 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `shadowColor: nature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `? NaturePaperColors.warmBrown.withValues(alpha: 0.45)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `return ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `onPressed: loading ? null : onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `style: style,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `child: loading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `height: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `width: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `child: CircularProgressIndicator(strokeWidth: 2.5, color: fg),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `mainAxisAlignment: MainAxisAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `if (icon != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `PaperGlyph(icon: icon!, size: 20, color: fg),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `style: theme.textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `color: fg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/buttons/primary_button.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/cards/premium_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `class PremiumCard extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const PremiumCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.padding = const EdgeInsets.all(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `this.borderColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `this.borderRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `this.elevation,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final EdgeInsets padding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final Color? color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final Color? borderColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final double? borderRadius;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final NeuElevation? elevation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `State<PremiumCard> createState() => _PremiumCardState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `class _PremiumCardState extends State<PremiumCard>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L35** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L36** `late final AnimationController _hoverController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `late final Animation<double> _scaleAnimation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `late final Animation<double> _elevationAnimation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `bool _pressed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `_hoverController = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `duration: const Duration(milliseconds: 200),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `_scaleAnimation = Tween<double>(begin: 1.0, end: 1.01).animate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `CurvedAnimation(parent: _hoverController, curve: Curves.easeOutCubic),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `_elevationAnimation = Tween<double>(begin: 4.0, end: 12.0).animate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `CurvedAnimation(parent: _hoverController, curve: Curves.easeOutCubic),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `_hoverController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final tokens = AppStyleTokens.maybeOf(context);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final isNature = tokens?.isNaturePaper ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `final isNeu = tokens?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `final isMono = tokens?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `final radius = widget.borderRadius ?? tokens?.cardRadius ?? 16.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `return AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `animation: _hoverController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `builder: (context, child) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `final hover = _hoverController.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final cardColor =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `widget.color ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `tokens?.cardSurface ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `(isDark ? AppColors.darkSurfaceElevated : scheme.surface);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final borderColor =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `widget.borderColor ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `tokens?.cardBorder ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `(isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `? AppColors.darkBorder`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `: AppColors.inkAlpha(context, 0.06));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `final shadowColor = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `? Colors.black.withValues(alpha: 0.55 + hover * 0.25)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `: scheme.onSurface.withValues(alpha: 0.02 + hover * 0.03);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `final List<BoxShadow> shadows;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final bool showBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `Color fillColor = cardColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final baseElev = widget.elevation ?? NeuElevation.raised;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `final elev = _pressed`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `? NeuElevation.pressed`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `: (hover > 0.5 ? NeuElevation.floating : baseElev);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `fillColor =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `widget.color ?? Neumorphic.surface(context, elevation: elev);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `shadows = Neumorphic.shadows(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `elevation: elev,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `intensity: 1.15 + hover * 0.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `showBorder = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `} else if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `shadows = const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `showBorder = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `} else if (isNature && !isDark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `fillColor = widget.color ?? NaturePaperColors.dayCard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `shadows = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `color: NaturePaperColors.warmBrown.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `alpha: 0.12 + hover * 0.06,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `blurRadius: 16 + hover * 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `offset: Offset(0, 6 + hover * 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `const BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `color: NaturePaperColors.dayCardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `offset: Offset(0, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `showBorder = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `} else if (isNature && isDark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `fillColor = widget.color ?? NaturePaperColors.nightCardHigh;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `shadows = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `color: Colors.black.withValues(alpha: 0.45 + hover * 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `blurRadius: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `offset: const Offset(0, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `color: NaturePaperColors.moonGlow.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `alpha: 0.05 + hover * 0.08,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `blurRadius: 20 + hover * 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `showBorder = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `shadows = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `color: shadowColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `blurRadius: _elevationAnimation.value + (isDark ? 6 : 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `offset: Offset(0, _elevationAnimation.value / 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `showBorder = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `final Color hoverBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `hoverBorder = hover > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `? AppColors.inkAlpha(context, isDark ? 0.45 : 0.28)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `: borderColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `} else if (isNature) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `hoverBorder = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `? NaturePaperColors.moonGlow.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `alpha: 0.30 + hover * 0.18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `: NaturePaperColors.warmBrown.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `alpha: 0.30 + hover * 0.10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `} else if (isDark && hover > 0 && !isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `hoverBorder = AppColors.mutedGold.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `alpha: 0.35 + hover * 0.25,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `hoverBorder = borderColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `final scale = isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `? 1.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `: (isNeu`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `? (_pressed ? 0.985 : (1.0 + hover * 0.012))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `: _scaleAnimation.value);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `return Transform.scale(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `scale: scale,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `duration: const Duration(milliseconds: 160),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `color: fillColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `border: showBorder`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `? Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `color: hoverBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `width: isNature ? 1.4 : 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `boxShadow: shadows,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `child: Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `onTap: widget.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `onTapDown: widget.onTap == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `: (_) => setState(() => _pressed = true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `onTapUp: widget.onTap == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `: (_) => setState(() => _pressed = false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `onTapCancel: widget.onTap == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `: () => setState(() => _pressed = false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `onHover: (isHovered) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `if (widget.onTap != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `if (isHovered) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `_hoverController.forward();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `_hoverController.reverse();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `highlightColor: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `).withValues(alpha: isDark ? 0.12 : 0.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `splashColor: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `).withValues(alpha: isDark ? 0.16 : 0.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `padding: widget.padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `child: widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `.animate()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `.fadeIn(duration: 400.ms, curve: Curves.easeOutCubic)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `.slideY(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `begin: 0.02,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `end: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `duration: 400.ms,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/cards/premium_card.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/chat/animated_emoji_message.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class AnimatedEmojiMessage extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const AnimatedEmojiMessage({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.assetPath,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.size = 144,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final String assetPath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String emoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `State<AnimatedEmojiMessage> createState() => _AnimatedEmojiMessageState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `class _AnimatedEmojiMessageState extends State<AnimatedEmojiMessage>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24** `with TickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L25** `LottieComposition? _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `bool _failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `late final AnimationController _lottieController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `late final AnimationController _bounceController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `late final Animation<double> _bounceScale;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `_lottieController = AnimationController(vsync: this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `_lottieController.addStatusListener(_onLottieStatus);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `_bounceController = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `duration: const Duration(milliseconds: 280),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `_bounceScale = TweenSequence<double>([`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `TweenSequenceItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `tween: Tween(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `begin: 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `end: 1.14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `).chain(CurveTween(curve: Curves.easeOut)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `weight: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `TweenSequenceItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `tween: Tween(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `begin: 1.14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `end: 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `).chain(CurveTween(curve: Curves.easeInOut)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `weight: 60,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `]).animate(_bounceController);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `_load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `void _onLottieStatus(AnimationStatus status) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `if (status != AnimationStatus.completed) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `_holdLastVisibleFrame();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `void _holdLastVisibleFrame() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final composition = _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `if (composition == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final span = composition.endFrame - composition.startFrame;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (span <= 0) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `const epsilonFrames = 0.5;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `final hold = ((span - epsilonFrames) / span).clamp(0.0, 0.999);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `if (_lottieController.value != hold) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `_lottieController.value = hold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `void didUpdateWidget(covariant AnimatedEmojiMessage oldWidget) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `super.didUpdateWidget(oldWidget);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `if (oldWidget.assetPath != widget.assetPath) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `_composition = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `_failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `_lottieController`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `..stop()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `..reset();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `_load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `Future<void> _load() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final composition = await AssetLottie(widget.assetPath).load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `_composition = composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `_failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `_lottieController.duration = composition.duration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `_lottieController.value = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `if (!mounted || _composition == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `_lottieController.forward(from: 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `_failed = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_composition = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `void _replay() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (_composition == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `HapticFeedback.selectionClick();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `_bounceController.forward(from: 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `_lottieController.forward(from: 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `_lottieController.removeStatusListener(_onLottieStatus);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `_lottieController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `_bounceController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `return GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `behavior: HitTestBehavior.opaque,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `onTap: _composition != null ? _replay : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `width: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `height: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `child: RepaintBoundary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `child: ScaleTransition(scale: _bounceScale, child: _buildBody()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Widget _buildBody() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `if (_failed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `return Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `widget.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `style: TextStyle(fontSize: widget.size * 0.72, height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `final composition = _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `if (composition == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `return const SizedBox.expand();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `return Lottie(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `composition: composition,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `controller: _lottieController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `width: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `height: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `repeat: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/chat/animated_emoji_message.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/chat/chat_lottie_emoji.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class ChatLottieEmoji extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const ChatLottieEmoji({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `this.size = 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.playOnTap = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.autoplay = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.repeat = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String emoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final bool playOnTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final bool autoplay;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final bool repeat;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `State<ChatLottieEmoji> createState() => _ChatLottieEmojiState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `class _ChatLottieEmojiState extends State<ChatLottieEmoji>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L28** `LottieComposition? _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `bool _failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `late final AnimationController _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `String? _assetPath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `_controller = AnimationController(vsync: this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `_controller.addStatusListener(_onStatus);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `_assetPath = animatedEmojiAssetPath(widget.emoji);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `_load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `void didUpdateWidget(covariant ChatLottieEmoji oldWidget) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `super.didUpdateWidget(oldWidget);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `if (oldWidget.emoji != widget.emoji) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `_composition = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `_failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `_controller`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `..stop()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `..reset();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `_assetPath = animatedEmojiAssetPath(widget.emoji);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `_load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `void _onStatus(AnimationStatus status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `if (widget.repeat) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (status != AnimationStatus.completed) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `_holdLastVisibleFrame();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `void _holdLastVisibleFrame() {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final composition = _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `if (composition == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final span = composition.endFrame - composition.startFrame;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `if (span <= 0) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `const epsilonFrames = 0.5;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `final hold = ((span - epsilonFrames) / span).clamp(0.0, 0.999);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `if (_controller.value != hold) _controller.value = hold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `Future<void> _load() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final path = _assetPath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (path == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `if (mounted) setState(() => _failed = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `final composition = await AssetLottie(path).load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `_composition = composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `_failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `_controller.duration = composition.duration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `_controller.value = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `if (widget.autoplay) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `if (!mounted || _composition == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `_startPlayback();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `} else if (!widget.repeat) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `_holdLastVisibleFrame();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `_failed = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `_composition = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `void _startPlayback() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (_composition == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `if (widget.repeat) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_controller.repeat(min: 0, max: _loopMax);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `_controller.forward(from: 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `double get _loopMax {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final composition = _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `if (composition == null) return 0.999;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `final span = composition.endFrame - composition.startFrame;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `if (span <= 0) return 0.999;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `return ((span - 0.5) / span).clamp(0.0, 0.999);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `void _replay() => _startPlayback();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `_controller.removeStatusListener(_onStatus);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `_controller.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `final child = SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `width: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `height: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `child: _buildBody(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `if (!widget.playOnTap || _composition == null) return child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `return GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `behavior: HitTestBehavior.opaque,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `onTap: _replay,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `Widget _buildBody() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `if (_failed || _assetPath == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `return Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `widget.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `style: TextStyle(fontSize: widget.size * 0.85, height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final composition = _composition;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `if (composition == null) return const SizedBox.expand();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `return Lottie(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `composition: composition,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `controller: _controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `width: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `height: widget.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `repeat: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/chat/chat_lottie_emoji.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/chat/chat_media_gallery.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `const _speedOptions = <double>[0.5, 0.75, 1.0, 1.25, 1.5, 2.0];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `const _dismissDistance = 120.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `enum ChatMediaKind { image, video }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L18** `class ChatMediaItem {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19** `const ChatMediaItem({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `required this.kind,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final ChatMediaKind kind;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `List<ChatMediaItem> chatMediaItemsFromMessages(Iterable<ChatMessage> messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final items = <ChatMediaItem>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `for (final m in messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (m.isDeleted) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final url = m.mediaUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `if (url == null || url.isEmpty) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final kind = switch (m.messageType) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `ChatMessageType.image => ChatMediaKind.image,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `ChatMessageType.video => ChatMediaKind.video,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `_ => null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `if (kind == null) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `items.add(ChatMediaItem(id: m.id, url: url, kind: kind));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `return items;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `Future<void> showChatMediaGallery(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `required List<ChatMediaItem> items,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `required int initialIndex,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `assert(items.isNotEmpty);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final index = initialIndex.clamp(0, items.length - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `return showGeneralDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `barrierDismissible: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `barrierColor: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `transitionDuration: const Duration(milliseconds: 220),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `pageBuilder: (context, animation, secondaryAnimation) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `return ChatMediaGalleryPage(items: items, initialIndex: index);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `transitionBuilder: (context, animation, secondaryAnimation, child) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `return FadeTransition(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `opacity: animation,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `child: ScaleTransition(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `scale: Tween<double>(begin: 0.96, end: 1).animate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `CurvedAnimation(parent: animation, curve: Curves.easeOutCubic),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `Future<void> showChatVideoPlayer(BuildContext context, {required String url}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `return showChatMediaGallery(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `items: [ChatMediaItem(id: url, url: url, kind: ChatMediaKind.video)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `initialIndex: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `class ChatMediaGalleryPage extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L86** `const ChatMediaGalleryPage({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `required this.items,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `required this.initialIndex,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final List<ChatMediaItem> items;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final int initialIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `State<ChatMediaGalleryPage> createState() => _ChatMediaGalleryPageState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `class _ChatMediaGalleryPageState extends State<ChatMediaGalleryPage> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L100** `late final PageController _pageController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `late int _index;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `double _dragY = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `bool _dragging = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `_index = widget.initialIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `_pageController = PageController(initialPage: _index);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `SystemChrome.setEnabledSystemUIMode(SystemUiMode.immersiveSticky);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `_pageController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `SystemChrome.setEnabledSystemUIMode(SystemUiMode.edgeToEdge);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `void _onVerticalDragUpdate(DragUpdateDetails details) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `final dy = details.delta.dy;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `_dragging = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `_dragY = (_dragY + dy).clamp(0.0, 400.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `void _onVerticalDragEnd(DragEndDetails details) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `final fling = details.primaryVelocity ?? 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `final shouldClose = _dragY > _dismissDistance || fling > 900;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `if (shouldClose) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `Navigator.of(context).maybePop();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `_dragging = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `_dragY = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `void _onVerticalDragCancel() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `_dragging = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `_dragY = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final progress = (_dragY / _dismissDistance).clamp(0.0, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `final bgOpacity = (1.0 - progress * 0.55).clamp(0.35, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `final scale = (1.0 - progress * 0.08).clamp(0.9, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `color: Colors.black.withValues(alpha: bgOpacity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `child: GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `onVerticalDragUpdate: _onVerticalDragUpdate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `onVerticalDragEnd: _onVerticalDragEnd,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `onVerticalDragCancel: _onVerticalDragCancel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `duration: _dragging`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `? Duration.zero`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `: const Duration(milliseconds: 180),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `transform: Matrix4.identity()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `..translateByDouble(0, _dragging ? _dragY : 0, 0, 1)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `..scaleByDouble(scale, scale, 1, 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `transformAlignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `child: SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `PageView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `controller: _pageController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `itemCount: widget.items.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `onPageChanged: (i) => setState(() => _index = i),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `itemBuilder: (context, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `final item = widget.items[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `final active = i == _index && _dragY < 24;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `return switch (item.kind) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `ChatMediaKind.image => _GalleryImagePage(url: item.url),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `ChatMediaKind.video => _GalleryVideoPage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `url: item.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `isActive: active,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `top: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `left: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `child: IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `onPressed: () => Navigator.of(context).maybePop(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `icon: const Icon(Icons.close_rounded, color: Colors.white),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `if (widget.items.length > 1)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `top: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `right: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `'${_index + 1} / ${widget.items.length}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `color: Colors.white70,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `class _GalleryImagePage extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L220** `const _GalleryImagePage({required this.url});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `State<_GalleryImagePage> createState() => _GalleryImagePageState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `class _GalleryImagePageState extends State<_GalleryImagePage>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L228** `with AutomaticKeepAliveClientMixin {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `File? _file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `bool _loading = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `bool get wantKeepAlive => true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `_resolve();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `Future<void> _resolve() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `if (!kIsWeb && ChatMediaCacheService.instance.isSupported) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `final file = await ChatMediaCacheService.instance.getOrFetch(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `preferredExtension: ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `fallback: 'jpg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `_file = file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `if (mounted) setState(() => _loading = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `super.build(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `if (_loading) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `return const Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `final child = _file != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `? Image.file(_file!, fit: BoxFit.contain)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `: CachedNetworkImage(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `imageUrl: widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `placeholder: (_, _) => const Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `errorWidget: (_, _, _) => const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `Icons.broken_image_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `color: Colors.white54,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `size: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `return InteractiveViewer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `minScale: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `maxScale: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `child: Center(child: child),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `class _GalleryVideoPage extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L294** `const _GalleryVideoPage({required this.url, required this.isActive});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `final bool isActive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `State<_GalleryVideoPage> createState() => _GalleryVideoPageState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `class _GalleryVideoPageState extends State<_GalleryVideoPage>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L304** `with AutomaticKeepAliveClientMixin {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `VideoPlayerController? _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `bool _ready = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `bool _failed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `bool _controlsVisible = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `double _speed = 1.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `Timer? _hideControls;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `bool get wantKeepAlive => true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `_open();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `void didUpdateWidget(covariant _GalleryVideoPage oldWidget) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `super.didUpdateWidget(oldWidget);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `if (oldWidget.isActive != widget.isActive) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `_syncPlayback();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `Future<void> _open() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `VideoPlayerController controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `if (!kIsWeb && ChatMediaCacheService.instance.isSupported) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `final file = await ChatMediaCacheService.instance.getOrFetch(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `preferredExtension: ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `fallback: 'mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `controller = file != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `? VideoPlayerController.file(file)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `: VideoPlayerController.networkUrl(Uri.parse(widget.url));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `controller = VideoPlayerController.networkUrl(Uri.parse(widget.url));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `_controller = controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `controller.addListener(_onTick);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `await controller.initialize();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `await controller.setPlaybackSpeed(_speed);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `setState(() => _ready = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `await _syncPlayback();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `if (mounted) setState(() => _failed = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `Future<void> _syncPlayback() async {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `final c = _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `if (c == null || !c.value.isInitialized) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `if (widget.isActive) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `await c.play();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `_scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `await c.pause();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `_hideControls?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `if (mounted) setState(() => _controlsVisible = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `void _onTick() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `if (mounted) setState(() {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `void _scheduleHideControls() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `_hideControls?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `if (!(_controller?.value.isPlaying ?? false)) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `_hideControls = Timer(const Duration(seconds: 3), () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `if (mounted && (_controller?.value.isPlaying ?? false)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `setState(() => _controlsVisible = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `void _toggleControls() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `setState(() => _controlsVisible = !_controlsVisible);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `if (_controlsVisible) _scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `Future<void> _togglePlay() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `final c = _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `if (c == null || !c.value.isInitialized) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `if (c.value.isPlaying) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `await c.pause();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `_hideControls?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `setState(() => _controlsVisible = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `await c.play();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `_scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `Future<void> _seekBy(Duration delta) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `final c = _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `if (c == null || !c.value.isInitialized) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `final total = c.value.duration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `var next = c.value.position + delta;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `if (next < Duration.zero) next = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `if (next > total) next = total;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `await c.seekTo(next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `_scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `Future<void> _setSpeed(double speed) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `final c = _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `if (c == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `setState(() => _speed = speed);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `await c.setPlaybackSpeed(speed);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `_scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `Future<void> _showSpeedSheet() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `_hideControls?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `setState(() => _controlsVisible = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `final selected = await showModalBottomSheet<double>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `backgroundColor: const Color(0xFF1C1C1E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `return SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `padding: const EdgeInsets.fromLTRB(20, 16, 20, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `child: Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `alignment: Alignment.centerLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `l10n.chatVideoPlaybackSpeed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `for (final speed in _speedOptions)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `selected: speed == _speed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `selectedColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `selectedTileColor: Colors.white12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `speed == 1.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `? l10n.chatVideoSpeedNormal`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `: l10n.chatVideoSpeedValue(speed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `style: const TextStyle(color: Colors.white),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `trailing: speed == _speed`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `? const Icon(Icons.check, color: Colors.white)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `onTap: () => Navigator.pop(ctx, speed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `if (selected != null) await _setSpeed(selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `_scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `String _fmt(Duration d) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `final total = d.inSeconds;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `final m = (total ~/ 60).toString().padLeft(2, '0');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `final s = (total % 60).toString().padLeft(2, '0');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `return '$m:$s';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `_hideControls?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `_controller?.removeListener(_onTick);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `_controller?.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `super.build(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `final c = _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `if (_failed)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `l10n.chatVideoCouldNotPlay,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `style: const TextStyle(color: Colors.white70),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `else if (!_ready || c == null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `const Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `behavior: HitTestBehavior.opaque,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `onTap: _toggleControls,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `child: AspectRatio(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `aspectRatio: c.value.aspectRatio == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `? 16 / 9`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `: c.value.aspectRatio,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `child: VideoPlayer(c),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `if (_ready && c != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `bottom: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `child: AnimatedOpacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `opacity: _controlsVisible ? 1 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `duration: const Duration(milliseconds: 180),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `ignoring: !_controlsVisible,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `child: _ControlsBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `controller: c,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `onSeekBack: () => _seekBy(const Duration(seconds: -10)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `onSeekForward: () => _seekBy(const Duration(seconds: 10)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `onTogglePlay: _togglePlay,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `onSpeedTap: _showSpeedSheet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `onSeek: (value) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `final total = c.value.duration.inMilliseconds;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `await c.seekTo(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `Duration(milliseconds: (value * total).round()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `_scheduleHideControls();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `format: _fmt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `speedLabel: _speed == 1.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `? l10n.chatVideoSpeedNormal`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `: l10n.chatVideoSpeedValue(_speed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `rewindLabel: l10n.chatVideoRewind10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `forwardLabel: l10n.chatVideoForward10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `class _ControlsBar extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L566** `const _ControlsBar({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `required this.controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `required this.onSeekBack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `required this.onSeekForward,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `required this.onTogglePlay,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `required this.onSpeedTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `required this.onSeek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `required this.format,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `required this.speedLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `required this.rewindLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `required this.forwardLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `final VideoPlayerController controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `final VoidCallback onSeekBack;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `final VoidCallback onSeekForward;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `final VoidCallback onTogglePlay;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `final VoidCallback onSpeedTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `final ValueChanged<double> onSeek;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L585** `final String Function(Duration) format;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `final String speedLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `final String rewindLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `final String forwardLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L591** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L592** `final value = controller.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `final duration = value.duration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `final position = value.position;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `final progress = duration.inMilliseconds == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `? 0.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `: (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `padding: const EdgeInsets.fromLTRB(12, 28, 12, 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `decoration: const BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L603** `begin: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `end: Alignment.bottomCenter,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `colors: [Colors.transparent, Colors.black87],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `format(position),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L615** `style: const TextStyle(color: Colors.white70, fontSize: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `child: SliderTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `data: SliderTheme.of(context).copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `trackHeight: 2.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `thumbShape: const RoundSliderThumbShape(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L622** `enabledThumbRadius: 7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `overlayShape: const RoundSliderOverlayShape(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `overlayRadius: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `activeTrackColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L628** `inactiveTrackColor: Colors.white24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L629** `thumbColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `child: Slider(value: progress, onChanged: onSeek),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `format(duration),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `style: const TextStyle(color: Colors.white70, fontSize: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `mainAxisAlignment: MainAxisAlignment.spaceEvenly,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `_ControlButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `onPressed: onSeekBack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `icon: Icons.replay_10_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `label: rewindLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L648** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `onPressed: onTogglePlay,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `iconSize: 56,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `value.isPlaying`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L654** `? Icons.pause_circle_filled_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L655** `: Icons.play_circle_filled_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `_ControlButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `onPressed: onSeekForward,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `icon: Icons.forward_10_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `label: forwardLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L663** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L664** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L665** `onPressed: onSpeedTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `foregroundColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `padding: const EdgeInsets.symmetric(horizontal: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L671** `speedLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L673** `fontWeight: FontWeight.w700,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L674** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L675** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L677** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L679** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L681** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L686** `class _ControlButton extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L687** `const _ControlButton({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L688** `required this.onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L689** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L690** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L693** `final VoidCallback onPressed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L694** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L697** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L699** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L700** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L701** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L702** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L703** `onPressed: onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L704** `icon: Icon(icon, color: Colors.white, size: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L705** `tooltip: label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L706** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L708** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L709** `style: const TextStyle(color: Colors.white70, fontSize: 11),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/chat/chat_media_gallery.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/decor/paper_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class PaperAssets {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `static const _dir = 'assets/images/nature-paper';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `static const avatarSilhouette = '$_dir/avatar-silhouette.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `static const beigeLeaf = '$_dir/beige-leaf.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `static const bookshelfSingle = '$_dir/bookshelf-single-shelf.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static const bookshelfDouble = '$_dir/bookshelf-two-shelves.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `static const brainVerified = '$_dir/brain-verified.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static const browserDashboard = '$_dir/browser-dashboard.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const calendar = '$_dir/calendar.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static const calendarWithClock = '$_dir/calendar-with-clock.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static const cardboardStar = '$_dir/cardboard-star.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const chatBubbles = '$_dir/chat-bubbles.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `static const clipboardChecklist = '$_dir/clipboard-checklist.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `static const clipboardProfile = '$_dir/clipboard-profile.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static const clipboardWithPencil = '$_dir/clipboard-with-pencil.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `static const closeButton = '$_dir/close-button.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `static const creamBird = '$_dir/cream-bird.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `static const creamPaperGreenTape = '$_dir/cream-paper-green-tape.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `static const divisionButton = '$_dir/division-button.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `static const documentLayout = '$_dir/document-layout.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `static const downloadInbox = '$_dir/download-inbox.png';`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L28** `static const dustyRoseFlower = '$_dir/dusty-rose-flower.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `static const envelopeSuccess = '$_dir/envelope-success.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `static const flatCloud = '$_dir/flat-cloud.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `static const grayCloud = '$_dir/gray-cloud.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `static const groupChat = '$_dir/group-chat.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `static const hangingGarland = '$_dir/hanging-garland.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `static const homeHouse = '$_dir/home-house.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `static const kraftPaperStrip = '$_dir/kraft-paper-strip.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `static const landscapeHills = '$_dir/landscape-hills-footer.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `static const linedPaperPencil = '$_dir/lined-paper-pencil.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `static const magnifyingGlass = '$_dir/magnifying-glass.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `static const manilaFolder = '$_dir/manila-folder.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `static const marketStall = '$_dir/market-stall.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `static const minusButton = '$_dir/minus-button.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `static const notificationBell = '$_dir/notification-bell-alert.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `static const openBook = '$_dir/open-book.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `static const paperFunnel = '$_dir/paper-funnel.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `static const puzzlePieces = '$_dir/puzzle-pieces.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `static const roundedTanStar = '$_dir/rounded-tan-star.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `static const sageScallopedFrame = '$_dir/sage-scalloped-frame.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `static const salmonFlower = '$_dir/salmon-flower.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `static const settingsGear = '$_dir/settings-gear.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `static const singleTree = '$_dir/single-tree.png';`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `static const spiralNotebook = '$_dir/spiral-notebook.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `static const stickyNotesPair = '$_dir/sticky-notes-pair.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `static const stripedWashiTape = '$_dir/striped-washi-tape.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `static const tanNoteGreenTab = '$_dir/tan-note-green-tab.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `static const tanSeed = '$_dir/tan-seed.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `static const teacherChalkboard = '$_dir/teacher-chalkboard.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `static const tealBird = '$_dir/teal-bird.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `static const terracottaScallopedFrame =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `'$_dir/terracotta-scalloped-frame.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `static const texturedTanStar = '$_dir/textured-tan-star.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `static const tornPaperScrap = '$_dir/torn-paper-scrap.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `static const twoTrees = '$_dir/two-trees.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `static const wallClock = '$_dir/wall-clock.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `static const whiteCloud = '$_dir/white-cloud.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `static const whiteDaisy = '$_dir/white-daisy.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `static const yellowPencil = '$_dir/yellow-pencil.png';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `bool isNaturePaper(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `AppStyleTokens.maybeOf(context)?.isNaturePaper ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `bool isSoftNeumorphism(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `bool isMonochromeWorkspace(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `String? paperAssetForIcon(IconData? icon) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `if (icon == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `if (icon == Icons.inbox_outlined || icon == Icons.inbox) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `return PaperAssets.downloadInbox;`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L83** `if (icon == Icons.grade_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `icon == Icons.grade ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `icon == Icons.star_outline ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `icon == Icons.star ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `icon == Icons.star_rounded) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `return PaperAssets.cardboardStar;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `if (icon == Icons.assignment_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `icon == Icons.assignment ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `icon == Icons.task_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `icon == Icons.assignment_turned_in_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `return PaperAssets.spiralNotebook;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `if (icon == Icons.task_alt_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `icon == Icons.edit_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `icon == Icons.edit ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `icon == Icons.add ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `icon == Icons.add_rounded) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `return PaperAssets.yellowPencil;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `if (icon == Icons.inventory_2_rounded || icon == Icons.inventory_2_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return PaperAssets.manilaFolder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `if (icon == Icons.chat_bubble_outline_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `icon == Icons.chat_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `icon == Icons.chat ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `icon == Icons.forum_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `icon == Icons.chat_bubble_outline) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `return PaperAssets.chatBubbles;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (icon == Icons.campaign_rounded || icon == Icons.campaign_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return PaperAssets.envelopeSuccess;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `if (icon == Icons.event_available_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `icon == Icons.event_available_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `icon == Icons.event_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `return PaperAssets.clipboardChecklist;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `if (icon == Icons.calendar_today_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `icon == Icons.calendar_month_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `icon == Icons.calendar_today) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `return PaperAssets.calendar;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `if (icon == Icons.people_outline ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `icon == Icons.groups_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `icon == Icons.groups ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `icon == Icons.people_alt_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `return PaperAssets.groupChat;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `if (icon == Icons.notifications_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `icon == Icons.notifications_none_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `icon == Icons.notifications_active_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `icon == Icons.notifications) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `return PaperAssets.notificationBell;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `if (icon == Icons.search || icon == Icons.search_rounded) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `return PaperAssets.magnifyingGlass;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `if (icon == Icons.menu_book ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `icon == Icons.subject_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `icon == Icons.language_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `icon == Icons.language) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `return PaperAssets.openBook;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `if (icon == Icons.calculate || icon == Icons.calculate_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `return PaperAssets.divisionButton;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `if (icon == Icons.badge_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `icon == Icons.person_outline ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `icon == Icons.person ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `icon == Icons.person_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `icon == Icons.account_circle_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `return PaperAssets.clipboardProfile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `if (icon == Icons.settings_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `icon == Icons.settings ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `icon == Icons.tune_rounded) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `return PaperAssets.settingsGear;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `if (icon == Icons.home_outlined || icon == Icons.home) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `return PaperAssets.homeHouse;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `if (icon == Icons.dashboard_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `icon == Icons.dashboard ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `icon == Icons.insights_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `icon == Icons.analytics_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `icon == Icons.analytics_rounded) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `return PaperAssets.browserDashboard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `if (icon == Icons.warning_amber_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `icon == Icons.cleaning_services_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `icon == Icons.error_outline) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `return PaperAssets.tornPaperScrap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `if (icon == Icons.schedule ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `icon == Icons.access_time_rounded ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `icon == Icons.timer_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `return PaperAssets.wallClock;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `if (icon == Icons.extension_outlined) return PaperAssets.puzzlePieces;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `if (icon == Icons.school || icon == Icons.school_outlined) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `return PaperAssets.teacherChalkboard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `if (icon == Icons.self_improvement_outlined ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `icon == Icons.self_improvement) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `return PaperAssets.openBook;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `if (icon == Icons.folder_outlined || icon == Icons.folder) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `return PaperAssets.manilaFolder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `if (icon == Icons.psychology_outlined || icon == Icons.psychology) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `return PaperAssets.brainVerified;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `double paperControlRadius(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `if (isNaturePaper(context)) return 18;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `if (isMonochromeWorkspace(context)) return 6;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `return 14;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `BoxDecoration paperControlDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `Color? color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `double? radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `final r = radius ?? paperControlRadius(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `final nature = isNaturePaper(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `return BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `color: color ?? AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `borderRadius: BorderRadius.circular(r),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `color: nature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `? AppColors.inkAlpha(context, 0.12)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `: AppColors.inkAlpha(context, 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `boxShadow: nature && Theme.of(context).brightness == Brightness.light`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `color: const Color(0xFF8A6A4A).withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `blurRadius: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `offset: const Offset(0, 3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `class PaperDecor extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L233** `const PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `this.asset, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `this.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `this.height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `this.angle = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `this.opacity = 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `this.fit = BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `final String asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `final double? size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `final double? height;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `final double angle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `final double opacity;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `final BoxFit fit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `if (!isNaturePaper(context)) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `Widget img = Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `width: size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `height: height ?? size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `fit: fit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `opacity: AlwaysStoppedAnimation(opacity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `if (angle != 0) img = Transform.rotate(angle: angle, child: img);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `return ExcludeSemantics(child: img);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `class PaperGlyph extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L267** `const PaperGlyph({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `this.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `this.size = 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `this.opacity = 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `this.retintInDark = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `final String? asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `final Color? color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `final double opacity;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `final bool retintInDark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `final resolved = asset ?? paperAssetForIcon(icon);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `if (!isNaturePaper(context) || resolved == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `return Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `size: size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `color: opacity == 1 ? color : color?.withValues(alpha: opacity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `Widget img = Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `resolved,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `width: size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `height: size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `if (isDark && retintInDark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `final provided = color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `final tint =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `(provided != null && provided.computeLuminance() > 0.38)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `? provided`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `: NaturePaperColors.moonBeige;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `img = ColorFiltered(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `colorFilter: ColorFilter.mode(tint, BlendMode.srcIn),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `child: img,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `if (opacity < 1) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `img = Opacity(opacity: opacity, child: img);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `return ExcludeSemantics(child: img);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `class PaperEmptyIllustration extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L330** `const PaperEmptyIllustration({super.key, required this.asset});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `final String asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `return SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `width: 128,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `height: 104,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `left: 0,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `top: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `angle: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `PaperAssets.beigeLeaf,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `width: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `opacity: const AlwaysStoppedAnimation(0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `right: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `top: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `angle: 0.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `PaperAssets.roundedTanStar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `opacity: const AlwaysStoppedAnimation(0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `bottom: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `PaperAssets.whiteDaisy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `width: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `opacity: const AlwaysStoppedAnimation(0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `left: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `bottom: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `PaperAssets.tanSeed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `width: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `opacity: const AlwaysStoppedAnimation(0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `width: 76,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `height: 76,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `class PaperTaped extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L398** `const PaperTaped({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `this.angle = -0.07,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `this.tapeWidth = 88,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `this.alignment = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `final double angle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `final double tapeWidth;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `final double alignment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `if (!isNaturePaper(context)) return child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `clipBehavior: Clip.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `top: 0,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `child: Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `alignment: Alignment(alignment, 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `child: ExcludeSemantics(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `angle: angle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `PaperAssets.stripedWashiTape,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `width: tapeWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `opacity: const AlwaysStoppedAnimation(0.92),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `class PaperDivider extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L444** `const PaperDivider({super.key, this.width = 180});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `final double width;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `if (!isNaturePaper(context)) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `padding: const EdgeInsets.symmetric(vertical: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `child: ExcludeSemantics(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `PaperAssets.kraftPaperStrip,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `width: width,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `height: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `fit: BoxFit.fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `opacity: const AlwaysStoppedAnimation(0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `class PaperScallopedAvatar extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L469** `const PaperScallopedAvatar({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `required this.avatarSize,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `final double avatarSize;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `if (!isNaturePaper(context)) return child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `final frameSize = avatarSize * 1.30;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `return SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `width: frameSize,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `height: frameSize,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `ExcludeSemantics(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `PaperAssets.sageScallopedFrame,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `width: frameSize,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `height: frameSize,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `fit: BoxFit.contain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `String? paperStarForPercent(double? percent) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `if (percent == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `if (percent >= 85) return PaperAssets.cardboardStar;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `if (percent >= 70) return PaperAssets.texturedTanStar;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/decor/paper_decor.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/decor/sidebar_grove.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class _CloudSpec {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `const _CloudSpec({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.y,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.speed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.phase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.opacity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final double y;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final double height;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final double speed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final double phase;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final double opacity;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `class SidebarSkyClouds extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `const SidebarSkyClouds({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `State<SidebarSkyClouds> createState() => _SidebarSkyCloudsState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `class _SidebarSkyCloudsState extends State<SidebarSkyClouds>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L32** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L33** `static const _clouds = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `asset: PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `y: 0.12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `height: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `speed: 0.55,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `phase: 0.05,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `opacity: 0.55,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `asset: PaperAssets.flatCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `y: 0.38,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `height: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `speed: 0.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `phase: 0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `opacity: 0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `asset: PaperAssets.grayCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `y: 0.62,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `speed: 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `phase: 0.72,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `opacity: 0.28,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `asset: PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `y: 0.82,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `height: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `speed: 0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `phase: 0.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `opacity: 0.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `late final AnimationController _controller = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `duration: const Duration(seconds: 22),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `)..repeat();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `_controller.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `if (!isNaturePaper(context)) return const SizedBox.expand();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `if (Theme.of(context).brightness == Brightness.dark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `return const SizedBox.expand();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `return IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `child: RepaintBoundary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `child: ClipRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `child: LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final w = constraints.maxWidth;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final h = constraints.maxHeight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `if (h < 24) return const SizedBox.expand();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `return AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `animation: _controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `builder: (context, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final t = _controller.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `clipBehavior: Clip.hardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `for (final c in _clouds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `left:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `w - (((t * c.speed + c.phase) % 1.0) * (w + 100)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `top: c.y * h,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `c.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `height: c.height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `opacity: AlwaysStoppedAnimation(c.opacity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `class SidebarGrove extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L126** `const SidebarGrove({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `State<SidebarGrove> createState() => _SidebarGroveState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `class _SidebarGroveState extends State<SidebarGrove>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L133** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L134** `static const _clouds = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `asset: PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `y: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `height: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `speed: 3.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `phase: 0.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `opacity: 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `asset: PaperAssets.flatCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `y: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `speed: 1.8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `phase: 0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `opacity: 0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `_CloudSpec(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `asset: PaperAssets.grayCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `y: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `height: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `speed: 2.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `phase: 0.78,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `opacity: 0.32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `late final AnimationController _controller = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `duration: const Duration(seconds: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `)..repeat();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `_controller.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `Widget _swaying({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `required double t,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `required double phase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `required Widget child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `final angle = math.sin(t * 2 * math.pi + phase) * 0.022;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `return Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `angle: angle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `alignment: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `if (!isNaturePaper(context)) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `return RepaintBoundary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `child: ClipRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `height: 148,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `width: double.infinity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `child: LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `final width = constraints.maxWidth;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `return AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `animation: _controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `builder: (context, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `final t = _controller.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `final travel = width + 90;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `clipBehavior: Clip.hardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `if (!isDark)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `for (final c in _clouds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `left:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `width -`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `(((t * c.speed + c.phase) % 1.0) * travel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `top: c.y,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `c.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `height: c.height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `opacity: AlwaysStoppedAnimation(c.opacity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `left: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `right: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `bottom: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `PaperAssets.landscapeHills,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `fit: BoxFit.fitWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `opacity: AlwaysStoppedAnimation(isDark ? 0.5 : 0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `left: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `bottom: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `child: _swaying(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `t: t,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `phase: 1.9,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `PaperAssets.singleTree,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `height: 46,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `opacity: AlwaysStoppedAnimation(isDark ? 0.85 : 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `right: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `bottom: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `child: _swaying(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `t: t,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `phase: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `PaperAssets.twoTrees,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `height: 78,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `opacity: AlwaysStoppedAnimation(isDark ? 0.9 : 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `left: 78,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `bottom: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `PaperAssets.whiteDaisy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `height: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `opacity: AlwaysStoppedAnimation(isDark ? 0.55 : 0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `right: 96,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `bottom: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `PaperAssets.salmonFlower,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `height: 17,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `opacity: AlwaysStoppedAnimation(isDark ? 0.55 : 0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `left: 46,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `bottom: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `PaperAssets.tanSeed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `height: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `opacity: AlwaysStoppedAnimation(isDark ? 0.45 : 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `if (!isDark)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `left: 96,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `bottom: 64,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `child: _swaying(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `t: t,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `phase: 3.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `PaperAssets.tealBird,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `opacity: const AlwaysStoppedAnimation(0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `if (isDark)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `Positioned.fill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `child: CustomPaint(painter: _FirefliesPainter(t)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `class _FirefliesPainter extends CustomPainter {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L314** `_FirefliesPainter(this.t);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `final double t;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `static const _anchors = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `Offset(0.16, 0.42),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `Offset(0.34, 0.62),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `Offset(0.52, 0.30),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `Offset(0.68, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `Offset(0.82, 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `Offset(0.90, 0.60),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `void paint(Canvas canvas, Size size) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `final glow = Paint()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `final core = Paint();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `for (var i = 0; i < _anchors.length; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `final phase = i * 1.7;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `final angle = t * 2 * math.pi;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `final dx = math.sin(angle + phase) * 7;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `final dy = math.sin((angle + phase) * 2) * 4;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `final center = Offset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `_anchors[i].dx * size.width + dx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `_anchors[i].dy * size.height + dy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `final pulse = 0.25 + 0.75 * (0.5 + 0.5 * math.sin(angle * 3 + phase * 2));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `glow.color = const Color(0xFFE8C57F).withValues(alpha: 0.45 * pulse);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `core.color = const Color(0xFFF6E3B4).withValues(alpha: 0.9 * pulse);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `canvas.drawCircle(center, 3.4, glow);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `canvas.drawCircle(center, 1.1, core);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `bool shouldRepaint(_FirefliesPainter oldDelegate) => oldDelegate.t != t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/decor/sidebar_grove.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/dialogs/pdf_viewer_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `Future<void> openPdfViewer(BuildContext context, String url, {String? title}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `return showModalBottomSheet<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `isScrollControlled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `useSafeArea: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `builder: (ctx) => DraggableScrollableSheet(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `expand: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `initialChildSize: 0.92,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `maxChildSize: 0.98,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `minChildSize: 0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `builder: (_, controller) => Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `title: Text(title ?? ctx.l10n.pdfDefaultTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `trailing: IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `icon: const Icon(Icons.close),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `child: SfPdfViewer.network(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `scrollDirection: PdfScrollDirection.vertical,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/dialogs/pdf_viewer_sheet.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/homework/due_date_label.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `class DueDateLabel extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const DueDateLabel({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.assignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.compact = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.onDark = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final Assignment assignment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final bool compact;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final bool onDark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final l10n = AppLocalizations.of(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final status = assignmentDueStatus(assignment);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final due = assignment.dueDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final muted = onDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `? Colors.white.withValues(alpha: 0.85)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `: AppColors.inkAlpha(context, 0.55);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `if (status == AssignmentDueStatus.none || due == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `return Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `l10n.commonNoDueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `style: TextStyle(color: muted, fontSize: compact ? 12 : 13),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final (label, color) = _labelAndColor(context, l10n, status, due, onDark);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (compact) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `color: color.withValues(alpha: onDark ? 0.22 : 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `borderRadius: BorderRadius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `color: onDark ? Colors.white : color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `fontWeight: FontWeight.w700,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `static (String, Color) _labelAndColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `AppLocalizations l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `AssignmentDueStatus status,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `DateTime due,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `bool onDark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `final dateStr = DateFormat.MMMd().format(due);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `switch (status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `case AssignmentDueStatus.overdue:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `return (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `l10n.homeworkOverdue(dateStr),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `onDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `? const Color(0xFFFFCDD2)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `: (isDark ? const Color(0xFFE57373) : AppColors.errorRed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `case AssignmentDueStatus.dueToday:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `return (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `l10n.homeworkDueToday,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `onDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `? const Color(0xFFFFE0B2)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `: (isDark ? const Color(0xFFFFB74D) : AppColors.warmAmber),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `case AssignmentDueStatus.dueSoon:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final today = assignmentCalendarDay(DateTime.now());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final days = assignmentCalendarDay(due).difference(today).inDays;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final label = days == 1`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `? l10n.homeworkDueTomorrow`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `: l10n.homeworkDueInDays(days);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `return (label, onDark ? const Color(0xFFFFE0B2) : AppColors.mutedGold);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `case AssignmentDueStatus.upcoming:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `return (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `l10n.commonDueDate(dateStr),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `onDark ? Colors.white : AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `case AssignmentDueStatus.none:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `return (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `l10n.commonNoDueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `onDark ? Colors.white70 : AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/homework/due_date_label.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/inputs/group_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L7** `class GroupSelector extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const GroupSelector({super.key, this.padding = const EdgeInsets.all(20)});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final EdgeInsets padding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final groups = ref.watch(myGroupsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L15** `final activeId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L16** `final list = groups.valueOrNull ?? const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `if (list.length < 2) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final radius = paperControlRadius(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `padding: padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `decoration: paperControlDecoration(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `icon: Icons.groups_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `child: DropdownButtonHideUnderline(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `child: DropdownButton<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `value: activeId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `isExpanded: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `dropdownColor: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `items: list`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `(g) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `value: g.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `child: Text(g.title, overflow: TextOverflow.ellipsis),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `onChanged: (value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `if (value == null) return;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `ref.read(selectedGroupIdProvider.notifier).state = value;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L58** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/inputs/group_selector.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/inputs/load_more_button.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class LoadMoreButton extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const LoadMoreButton({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.loading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final bool hasMore;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final bool loading;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final VoidCallback onPressed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `if (!hasMore) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `padding: const EdgeInsets.symmetric(vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `child: loading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `width: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `height: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `: TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `onPressed: onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `context.l10n.commonLoadMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/inputs/load_more_button.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/layouts/app_shell.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L20** `class AppShell extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21** `const AppShell({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `required this.role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `required this.destinations,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final UserRole role;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final List<ShellDestination> destinations;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `ConsumerState<AppShell> createState() => _AppShellState();`
Widget that can watch providers. Required for profile/theme/locale.

**L36** `class _AppShellState extends ConsumerState<AppShell> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L37** `bool _isSidebarExpanded = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `int _indexFromLocation(String location) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `for (var i = 0; i < widget.destinations.length; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `if (location.startsWith(widget.destinations[i].path)) return i;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `void _go(int index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `context.go(widget.destinations[index].path);`
Replace the navigation stack location. Login uses /boot so redirect can settle.

**L50** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final location = GoRouterState.of(context).uri.toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final index = _indexFromLocation(location);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final unreadNotifications = ref.watch(unreadNotificationsCountProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L55** `final unreadChat = ref.watch(focusChatMutedProvider)`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L56** `? 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `: ref.watch(unreadChatCountProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L58** `final isDesktop = MediaQuery.sizeOf(context).width >= Breakpoints.desktop;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final isTablet = context.isTablet;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `String labelOf(ShellDestination d) => navLabel(l10n, d.labelKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `int badgeCount(ShellDestination d) => switch (d.badge) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `ShellBadgeKind.notifications => unreadNotifications,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `ShellBadgeKind.chat => unreadChat,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `ShellBadgeKind.none => 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `Widget icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `ShellDestination d, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `bool isSelected = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `bool usePaperAssets = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final tone = isSelected`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `: AppColors.inkAlpha(context, 0.5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `final Widget base = usePaperAssets`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `? PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `icon: d.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `color: tone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `opacity: isSelected || !isNaturePaper(context) ? 1 : 0.72,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `: Icon(d.icon, size: 24, color: tone);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final count = badgeCount(d);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `if (count > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `return Badge(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `count > 99 ? '99+' : '$count',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `fontSize: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `color: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `? (Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `? Colors.black`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `: Colors.white)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `backgroundColor: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `? AppColors.ink(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `child: base,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `return base;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `if (isDesktop) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `body: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `duration: const Duration(milliseconds: 300),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `curve: Curves.easeInOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `width: _isSidebarExpanded ? 260 : 80,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `color: Theme.of(context).colorScheme.surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `border: Border(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `right: BorderSide(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `color: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `).colorScheme.onSurface.withValues(alpha: 0.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `child: ClipRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `padding: const EdgeInsets.symmetric(horizontal: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `mainAxisAlignment: _isSidebarExpanded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `? MainAxisAlignment.spaceBetween`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `: MainAxisAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `if (_isSidebarExpanded)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `child: ClipRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `const AppLogoImage(size: 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `l10n.shellMuStudy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `overflow: TextOverflow.clip,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `softWrap: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `style: Theme.of(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `.textTheme`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `.titleMedium`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `letterSpacing: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `onPressed: () => setState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `() => _isSidebarExpanded = !_isSidebarExpanded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `_isSidebarExpanded ? Icons.menu_open : Icons.menu,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `if (_isSidebarExpanded)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `padding: EdgeInsets.only(top: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `width: double.infinity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `PaperAssets.hangingGarland,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `height: 52,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `fit: BoxFit.fitWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `opacity: 0.95,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `padding: const EdgeInsets.symmetric(horizontal: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `for (var i = 0; i < widget.destinations.length; i++)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `final d = widget.destinations[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `final isSelected = index == i;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `padding: const EdgeInsets.only(bottom: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `onTap: () => _go(i),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `borderRadius: BorderRadius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `duration: const Duration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `milliseconds: 200,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `horizontal: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `vertical: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `color: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `? AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `).withValues(alpha: 0.12)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `borderRadius: BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `isNaturePaper(context) ? 16 : 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `child: ClipRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `mainAxisAlignment: _isSidebarExpanded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `? MainAxisAlignment.start`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `: MainAxisAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `icon(d, isSelected: isSelected),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `if (_isSidebarExpanded) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `labelOf(d),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `overflow: TextOverflow.clip,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `softWrap: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `style: Theme.of(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `.textTheme`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `.bodyMedium`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `fontWeight: isSelected`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `? FontWeight.w700`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `color: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `? AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `if (_isSidebarExpanded)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `const Expanded(child: SidebarSkyClouds())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `if (_isSidebarExpanded) const SidebarGrove(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `if (_isSidebarExpanded)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `padding: const EdgeInsets.all(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `decoration: paperControlDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `color: AppColors.softWash(context, lightAlpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `radius: isNaturePaper(context) ? 16 : 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `if (isNaturePaper(context))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `const PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `icon: Icons.person_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `size: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `radius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `Icons.person,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `Theme.of(context).brightness ==`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `? Colors.black`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `switch (widget.role) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `UserRole.student => l10n.commonStudent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `UserRole.teacher => l10n.commonTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `UserRole.admin => l10n.commonAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `}.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `fontSize: 10,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `l10n.commonWorkspace,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `constraints: BoxConstraints(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `maxWidth: context.contentMaxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `child: widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `if (isTablet) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `body: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `right: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `child: NavigationRail(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `selectedIndex: index,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `onDestinationSelected: _go,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `labelType: NavigationRailLabelType.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `backgroundColor: Theme.of(context).colorScheme.surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `indicatorColor: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `).withValues(alpha: 0.15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `leading: const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `padding: EdgeInsets.symmetric(vertical: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `child: AppLogoImage(size: 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `destinations: widget.destinations`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `(d) => NavigationRailDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `icon: icon(d),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `selectedIcon: icon(d, isSelected: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `label: Text(labelOf(d)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `const VerticalDivider(width: 1, thickness: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `constraints: BoxConstraints(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `maxWidth: context.contentMaxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `child: widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `extendBody: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `resizeToAvoidBottomInset: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `body: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `constraints: BoxConstraints(maxWidth: context.contentMaxWidth),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `child: widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `bottomNavigationBar: ShellNavBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `destinations: widget.destinations,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `selectedIndex: index,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `onDestinationSelected: _go,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `iconBuilder: (d, isSelected) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `icon(d, isSelected: isSelected, usePaperAssets: false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/components/layouts/app_shell.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/layouts/empty_state.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L7** `class EmptyState extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const EmptyState({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.icon = Icons.inbox_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String? subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final paperAsset = isNaturePaper(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `? (paperAssetForIcon(icon) ?? PaperAssets.tornPaperScrap)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `: null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final isNeu = isSoftNeumorphism(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final isMono = isMonochromeWorkspace(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget leading;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (paperAsset != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `leading = PaperEmptyIllustration(asset: paperAsset);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `} else if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `leading = NeuIconWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `icon: icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `size: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `padding: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `elevation: NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `} else if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `leading = Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `width: 72,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `height: 72,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `borderRadius: BorderRadius.circular(6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `border: Border.all(color: AppColors.inkAlpha(context, 0.12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `child: Icon(icon, size: 28, color: AppColors.inkAlpha(context, 0.45)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `leading = Icon(icon, size: 56, color: AppColors.inkAlpha(context, 0.25));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `padding: const EdgeInsets.all(32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `leading,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `style: Theme.of(context).textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (subtitle != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `subtitle!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `style: Theme.of(context).textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `class ErrorState extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L86** `const ErrorState({super.key, required this.message, this.onRetry});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final String message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final VoidCallback? onRetry;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `return Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `if (isNaturePaper(context))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `const PaperEmptyIllustration(asset: PaperAssets.tornPaperScrap)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `Icons.error_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `color: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `size: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `Text(message, textAlign: TextAlign.center),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `if (onRetry != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `onPressed: onRetry,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `child: Text(context.l10n.commonTryAgain),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/layouts/empty_state.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/layouts/premium_section_header.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class PremiumSectionHeader extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `const PremiumSectionHeader({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.trailing,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.horizontalPadding = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.decorAsset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String? subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final Widget? trailing;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final double horizontalPadding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final String? decorAsset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final headerGlow = tokens?.headerGlow ?? Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `Widget content = Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `if (subtitle != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `padding: const EdgeInsets.only(bottom: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `subtitle!.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `letterSpacing: tokens?.isMonochromeWorkspace == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `? 1.4`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `color: tokens?.isMonochromeWorkspace == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `? AppColors.inkAlpha(context, 0.45)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `: AppColors.accent(context).withValues(alpha: 0.92),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `crossAxisAlignment: CrossAxisAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `Flexible(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `title,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `style: Theme.of(context).textTheme.headlineSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `letterSpacing: tokens?.isMonochromeWorkspace == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `? -0.7`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `fontSize: tokens?.isMonochromeWorkspace == true ? 26 : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `decorAsset ?? PaperAssets.beigeLeaf,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `size: decorAsset == null ? 16 : 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `angle: decorAsset == null ? 0.5 : 0.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `opacity: 0.9,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `if (headerGlow != Colors.transparent) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `content = Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `clipBehavior: Clip.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `left: -28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `top: -20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `width: 190,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `height: 84,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `borderRadius: BorderRadius.circular(48),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `headerGlow.withValues(alpha: isDark ? 0.10 : 0.26),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `headerGlow.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `content,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `padding: EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `horizontal: horizontalPadding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `vertical: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `crossAxisAlignment: CrossAxisAlignment.end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Expanded(child: content),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `?trailing,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/layouts/premium_section_header.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/layouts/shell_nav_bar.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `const double kShellNavBarHeight = 64;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `const double kShellNavBarSideMargin = 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `const double kShellNavBarMinBottomGap = 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `const double kShellFabGap = 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `const double kShellFabHeight = 56;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `const Duration kShellNavBarAnimationDuration = Duration(milliseconds: 240);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `enum ShellBadgeKind { none, notifications, chat }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L34** `class ShellDestination {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L35** `const ShellDestination({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `required this.labelKey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `required this.path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `this.badge = ShellBadgeKind.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `final String labelKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final String path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final ShellBadgeKind badge;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `double shellBottomInset(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `MediaQuery.paddingOf(context).bottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `double shellContentBottomPadding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `bool hasFab = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `double extra = 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final double base = shellBottomInset(context) + extra;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `return hasFab ? base + kShellFabHeight + kShellFabGap : base;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `FloatingActionButtonLocation shellFabLocation(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `ShellFabLocation(bottomInset: shellBottomInset(context));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `class ShellFabLocation extends StandardFabLocation with FabEndOffsetX {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L85** `const ShellFabLocation({required this.bottomInset});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final double bottomInset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `double getOffsetY(ScaffoldPrelayoutGeometry geometry, double adjustment) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final double fabHeight = geometry.floatingActionButtonSize.height;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final double keyboard = geometry.minInsets.bottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final double margin = math.max(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `kFloatingActionButtonMargin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `bottomInset + kShellFabGap - keyboard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `double y = geometry.contentBottom - fabHeight - margin;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final double snackBarHeight = geometry.snackBarSize.height;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (snackBarHeight > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `y = math.min(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `y,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `geometry.contentBottom -`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `snackBarHeight -`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `fabHeight -`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `kFloatingActionButtonMargin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return y + adjustment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `bool operator ==(Object other) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `other is ShellFabLocation && other.bottomInset == bottomInset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `int get hashCode => bottomInset.hashCode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `String toString() => 'ShellFabLocation(bottomInset: $bottomInset)';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `class ShellNavBar extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L135** `const ShellNavBar({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `required this.destinations,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `required this.selectedIndex,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `required this.onDestinationSelected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `required this.iconBuilder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `static const Key pillKey = Key('shellNavBarPill');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `final List<ShellDestination> destinations;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `final int selectedIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `final ValueChanged<int> onDestinationSelected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `final Widget Function(ShellDestination destination, bool isSelected)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `iconBuilder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `final EdgeInsets viewPadding = MediaQuery.viewPaddingOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `left: kShellNavBarSideMargin + viewPadding.left,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `right: kShellNavBarSideMargin + viewPadding.right,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `bottom: math.max(viewPadding.bottom, kShellNavBarMinBottomGap),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `child: _buildPill(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `Widget _buildPill(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `final isNature = tokens?.isNaturePaper ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `final isNeu = tokens?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `final isMono = tokens?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final BorderRadius radius = BorderRadius.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `Radius.circular(isMono ? 12 : kShellNavBarHeight / 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `final Color fill =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `tokens?.navSurface ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `(isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `? AppColors.darkSurfaceHighest.withValues(alpha: 0.96)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `: Theme.of(context).colorScheme.surface.withValues(alpha: 0.88));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `final Color borderColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `borderColor = Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `} else if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `borderColor = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `? AppColors.inkAlpha(context, 0.18)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `: AppColors.inkAlpha(context, 0.10);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `} else if (isNature) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `borderColor = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `? NaturePaperColors.moonGlow.withValues(alpha: 0.22)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `: NaturePaperColors.warmBrown.withValues(alpha: 0.18);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `borderColor = isDark ? AppColors.darkBorder : Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `final List<BoxShadow> shadows;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `shadows = Neumorphic.shadows(context, elevation: NeuElevation.floating);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `} else if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `shadows = const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `shadows = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `? Colors.black.withValues(alpha: 0.55)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `: (isNature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `? NaturePaperColors.warmBrown.withValues(alpha: 0.28)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `: AppColors.inkAlpha(context, 0.12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `blurRadius: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `offset: const Offset(0, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `if (isNature && isDark)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `color: NaturePaperColors.moonGlow.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `blurRadius: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `final navRow = Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `color: fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `side: BorderSide(color: borderColor),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `clipBehavior: Clip.antiAlias,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `for (var i = 0; i < destinations.length; i++)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `child: _ShellNavItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `destination: destinations[i],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `isSelected: i == selectedIndex,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `icon: iconBuilder(destinations[i], i == selectedIndex),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `onTap: () => onDestinationSelected(i),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `pressedActive: isNeu,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `editorialActive: isMono,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `key: pillKey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `height: kShellNavBarHeight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `decoration: BoxDecoration(borderRadius: radius, boxShadow: shadows),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `child: ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `child: (isNeu || isMono)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `? navRow`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `: BackdropFilter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `filter: ImageFilter.blur(sigmaX: 18, sigmaY: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `child: navRow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `class _ShellNavItem extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L269** `const _ShellNavItem({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `required this.destination,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `required this.isSelected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `this.pressedActive = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `this.editorialActive = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `final ShellDestination destination;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `final bool isSelected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `final Widget icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `final bool pressedActive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `final bool editorialActive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `final glow = AppStyleTokens.maybeOf(context)?.navGlow ?? Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `final ink = AppColors.ink(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `return Semantics(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `button: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `selected: isSelected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `label: navLabel(context.l10n, destination.labelKey),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `customBorder: editorialActive`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `? RoundedRectangleBorder(borderRadius: BorderRadius.circular(10))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `: const StadiumBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `height: kShellNavBarHeight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `if (pressedActive)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `AnimatedOpacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `duration: kShellNavBarAnimationDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `opacity: isSelected ? 1 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `width: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `color: Neumorphic.surface(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `elevation: NeuElevation.pressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `borderRadius: BorderRadius.circular(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `boxShadow: Neumorphic.shadows(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `elevation: NeuElevation.pressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `if (editorialActive)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `AnimatedOpacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `duration: kShellNavBarAnimationDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `opacity: isSelected ? 1 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `height: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `borderRadius: BorderRadius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `if (glow != Colors.transparent)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `AnimatedOpacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `duration: kShellNavBarAnimationDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `opacity: isSelected ? 1 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `width: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `height: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `gradient: RadialGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `colors: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `glow.withValues(alpha: pressedActive ? 0.28 : 0.40),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `glow.withValues(alpha: 0.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `AnimatedScale(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `duration: kShellNavBarAnimationDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `scale: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `? (pressedActive || editorialActive ? 1.0 : 1.08)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `child: editorialActive && isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `? ColorFiltered(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `colorFilter: ColorFilter.mode(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `isDark ? Colors.black : Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `BlendMode.srcIn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `child: icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `: icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `if (!pressedActive)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `bottom: editorialActive ? 8 : 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `duration: kShellNavBarAnimationDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `height: editorialActive ? 2 : 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `width: isSelected ? (editorialActive ? 12 : 16) : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `color: editorialActive`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `? ink.withValues(alpha: 0.35)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `borderRadius: BorderRadius.circular(1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/layouts/shell_nav_bar.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/marks/mark_average_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `class MarkAverageCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const MarkAverageCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.average,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.markCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final double? average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final int markCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final value = average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final color = value == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `: percentColor(value, context: context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final starAsset = paperStarForPercent(value);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `width: 76,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `height: 76,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `child: TweenAnimationBuilder<double>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `tween: Tween<double>(begin: 0, end: (value ?? 0) / 100),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `duration: const Duration(milliseconds: 900),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `builder: (context, progress, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `const stroke = 8.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `const labelInset = stroke * 2 + 10;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `Positioned.fill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `value: progress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `backgroundColor: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `valueColor: AlwaysStoppedAnimation(color),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `strokeWidth: stroke,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `strokeCap: StrokeCap.round,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `Padding(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `padding: const EdgeInsets.all(labelInset / 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `child: FittedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `fit: BoxFit.scaleDown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `formatPercent(value),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `height: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `letterSpacing: -0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `const SizedBox(width: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `context.l10n.marksOverallAverage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `value == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `? context.l10n.marksNoneYet`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `: context.l10n.marksBasedOn(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `percentLabel(value, context.l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `markCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `height: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `if (starAsset != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `padding: const EdgeInsets.only(left: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `child: PaperDecor(starAsset, size: 42, angle: 0.14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `if (onTap != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/marks/mark_average_card.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/marks/percent_badge.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `Color percentColor(double percent, {BuildContext? context}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `final mono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `context != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `(AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final isDark =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `context != null && Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `if (mono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `if (isDark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `if (percent >= 85) return const Color(0xFFF5F5F5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `if (percent >= 70) return const Color(0xFFC8C8C8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `if (percent >= 50) return const Color(0xFF9A9A9A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `return const Color(0xFF707070);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `if (percent >= 85) return const Color(0xFF111111);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `if (percent >= 70) return const Color(0xFF3A3A3A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `if (percent >= 50) return const Color(0xFF6B6B6B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `return const Color(0xFF8E8E8E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `if (percent >= 85) return AppColors.successGreen;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `if (percent >= 70) return AppColors.mutedGold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (percent >= 50) return AppColors.warmAmber;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `return AppColors.errorRed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `String percentLabel(double percent, [AppLocalizations? l10n]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (percent >= 85) return l10n?.percentExcellent ?? 'Excellent';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `if (percent >= 70) return l10n?.percentGood ?? 'Good';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `if (percent >= 50) return l10n?.percentNeedsWork ?? 'Needs work';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `return l10n?.percentAtRisk ?? 'At risk';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `String formatPercent(double? percent) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `if (percent == null) return '—';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `final rounded = percent.roundToDouble();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `return percent == rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `? '${rounded.toInt()}%'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `: '${percent.toStringAsFixed(1)}%';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `class PercentBadge extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L48** `const PercentBadge({super.key, required this.percent, this.compact = false});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final double? percent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final bool compact;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `final value = percent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final color = value == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `? AppColors.inkAlpha(context, 0.3)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `: percentColor(value, context: context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final isNature = tokens?.isNaturePaper ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final radius = tokens?.isMonochromeWorkspace == true`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `? (compact ? 4.0 : 6.0)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `: isNature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `? (compact ? 12.0 : 16.0)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `: (compact ? 8.0 : 12.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `padding: EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `horizontal: compact ? 10 : 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `vertical: compact ? 5 : 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `color: color.withValues(alpha: isNature ? 0.14 : 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `border: tokens?.isMonochromeWorkspace == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `? Border.all(color: color.withValues(alpha: 0.28))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `: isNature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `? Border.all(color: color.withValues(alpha: 0.22))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `formatPercent(value),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `fontSize: compact ? 13 : 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `letterSpacing: -0.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `fontFeatures: tokens?.isMonochromeWorkspace == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `? const [FontFeature.tabularFigures()]`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/marks/percent_badge.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/marks/term_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `class TermSelector extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const TermSelector({super.key, this.padding = EdgeInsets.zero});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final EdgeInsets padding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final current = ref.watch(selectedMarkTermProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L16** `final optionsAsync = ref.watch(academicTermOptionsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L17** `final radius = paperControlRadius(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `return optionsAsync.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `loading: () => Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `padding: padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `child: const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `height: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `child: Center(child: CircularProgressIndicator(strokeWidth: 2)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `error: (e, _) => Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `padding: padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `e.toString(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `style: TextStyle(color: AppColors.errorRed, fontSize: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `data: (options) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final selected = options.cast<AcademicTerm?>().firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `(t) => t!.id == current.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `orElse: () => current,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `)!;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `padding: padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `decoration: paperControlDecoration(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `icon: Icons.calendar_month_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `child: DropdownButtonHideUnderline(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `child: DropdownButton<String>(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `value: options.any((t) => t.id == selected.id)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `? selected.id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `: options.first.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `isExpanded: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `dropdownColor: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `items: options`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `(t) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `value: t.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `t.label(context.l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `onChanged: (id) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `if (id == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final next = options.firstWhere((t) => t.id == id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `.read(selectedMarkTermProvider.notifier)`
Riverpod binding. Name it after the value it exposes.

**L83** `.select(next);`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L84** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/marks/term_selector.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/components/settings/language_settings_tile.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L9** `class LanguageSettingsTile extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `const LanguageSettingsTile({super.key, this.embedded = false});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final bool embedded;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `String _nativeName(AppLanguage lang) => switch (lang) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `AppLanguage.english => 'English',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `AppLanguage.russian => '[redacted]',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `AppLanguage.uzbek => 'O‘zbekcha',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `String _codeLabel(AppLanguage lang) => switch (lang) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `AppLanguage.english => 'EN',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `AppLanguage.russian => 'RU',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `AppLanguage.uzbek => 'UZ',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final current = AppLanguage.fromLocale(ref.watch(localeProvider));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L30** `final onSurface = Theme.of(context).colorScheme.onSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `if (!embedded) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `l10n.languageSection,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `l10n.languageHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `] else ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `l10n.languageSection,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `decoration: paperControlDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `color: embedded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `? AppColors.inkAlpha(context, 0.05)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `icon: Icons.language_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `child: DropdownButtonHideUnderline(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `child: DropdownButton<AppLanguage>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `value: current,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `isExpanded: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `borderRadius: BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `paperControlRadius(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `dropdownColor: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `Icons.keyboard_arrow_down_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `color: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `items: AppLanguage.values`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `(lang) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `value: lang,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `width: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `_codeLabel(lang),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `color: lang == current`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `_nativeName(lang),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `onChanged: (lang) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `if (lang == null || lang == current) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `ref.read(localeProvider.notifier).setLanguage(lang);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L131** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/components/settings/language_settings_tile.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/admin/credential_slip.dart

Credential slip PDF generation for printed login cards. This file (credential slip) is one concrete piece of that layer.

**L1** `class CredentialSlip {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L2** `const CredentialSlip({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L3** `this.profileId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `required this.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final String? profileId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final String username;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final String fullName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final String password;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `CredentialSlip copyWith({String? password}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `return CredentialSlip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `profileId: profileId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `username: username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `fullName: fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `password: password ?? this.password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `List<CredentialSlip> uniqueCredentialSlips(Iterable<CredentialSlip> slips) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final seen = <String>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final unique = <CredentialSlip>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `for (final slip in slips) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final key = (slip.profileId ?? slip.username).trim().toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `if (key.isEmpty || !seen.add(key)) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `unique.add(slip);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `return unique;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/admin/credential_slip.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/admin/credential_slips_pdf.dart

Credential slip PDF generation for printed login cards. This file (credential slips pdf) is one concrete piece of that layer.

**L8** `const _slipsPerPage = 3;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `Uint8List buildCredentialSlipsPdf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `List<CredentialSlip> slips,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `AppLocalizations l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final unique = uniqueCredentialSlips(slips);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `if (unique.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `throw ArgumentError(l10n.adminCredentialNeedUsers);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final document = PdfDocument();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `document.pageSettings.size = PdfPageSize.a4;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `document.pageSettings.margins.all = 28;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final muFont = PdfStandardFont(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `PdfFontFamily.helvetica,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `style: PdfFontStyle.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final studyFont = PdfStandardFont(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `PdfFontFamily.helvetica,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `style: PdfFontStyle.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final disclaimerFont = PdfStandardFont(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `PdfFontFamily.helvetica,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `style: PdfFontStyle.italic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final labelFont = PdfStandardFont(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `PdfFontFamily.helvetica,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `style: PdfFontStyle.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final valueFont = PdfStandardFont(PdfFontFamily.helvetica, 11);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final muInk = PdfColor(45, 36, 22);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final gold = PdfColor(197, 160, 89);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final valueInk = PdfColor(20, 20, 20);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final muted = PdfColor(120, 120, 120);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final border = PdfColor(210, 210, 210);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final fill = PdfColor(255, 255, 255);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final linePen = PdfPen(PdfColor(230, 230, 230), width: 0.8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `for (var i = 0; i < unique.length; i += _slipsPerPage) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final page = document.pages.add();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final size = page.getClientSize();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final g = page.graphics;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `const gap = 18.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final slipHeight = (size.height - gap * (_slipsPerPage - 1)) / _slipsPerPage;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final batch = unique.skip(i).take(_slipsPerPage).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `for (var s = 0; s < batch.length; s++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final top = s * (slipHeight + gap);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final bounds = Rect.fromLTWH(0, top, size.width, slipHeight);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `_drawSlip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `g,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `bounds: bounds,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `slip: batch[s],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `l10n: l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `muFont: muFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `studyFont: studyFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `disclaimerFont: disclaimerFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `labelFont: labelFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `valueFont: valueFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `muInk: muInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `gold: gold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `valueInk: valueInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `muted: muted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `border: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `fill: fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `linePen: linePen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final bytes = Uint8List.fromList(document.saveSync());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `document.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `return bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `void _drawSlip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `PdfGraphics g, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `required Rect bounds,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `required CredentialSlip slip,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `required AppLocalizations l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `required PdfFont muFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `required PdfFont studyFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `required PdfFont disclaimerFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `required PdfFont labelFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `required PdfFont valueFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `required PdfColor muInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `required PdfColor gold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `required PdfColor valueInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `required PdfColor muted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `required PdfColor border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `required PdfColor fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `required PdfPen linePen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `const radius = 10.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final card = _roundedRect(bounds, radius);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `g.drawPath(card, brush: PdfSolidBrush(fill), pen: PdfPen(border, width: 1));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `const barWidth = 7.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `g.drawRectangle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `brush: PdfSolidBrush(gold),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `bounds: Rect.fromLTWH(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `bounds.left,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `bounds.top + radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `barWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `bounds.height - radius * 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `final left = bounds.left + 28;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `final width = bounds.width - 48;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `var y = bounds.top + 22;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `'MU',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `muFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `brush: PdfSolidBrush(muInk),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `bounds: Rect.fromLTWH(left, y, 40, 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `'Study',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `studyFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `brush: PdfSolidBrush(gold),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `bounds: Rect.fromLTWH(left + 34, y, 80, 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `y += 28;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `g.drawLine(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `linePen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `Offset(left, y),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `Offset(bounds.right - 20, y),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `y += 10;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `l10n.adminCredentialConfidential,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `disclaimerFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `brush: PdfSolidBrush(muted),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `bounds: Rect.fromLTWH(left, y, width, 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `y += 36;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `void row(String label, String value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `'$label:',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `labelFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `brush: PdfSolidBrush(muInk),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `bounds: Rect.fromLTWH(left, y, 100, 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `valueFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `brush: PdfSolidBrush(valueInk),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `bounds: Rect.fromLTWH(left + 108, y, width - 108, 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `y += 22;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `row(l10n.loginUsername, slip.username);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `row(l10n.adminFullName, slip.fullName);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `row(l10n.adminPassword, slip.password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `PdfPath _roundedRect(Rect r, double radius) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `final path = PdfPath();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `final d = radius * 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `path.addArc(Rect.fromLTWH(r.left, r.top, d, d), 180, 90);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `path.addArc(Rect.fromLTWH(r.right - d, r.top, d, d), 270, 90);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `path.addArc(Rect.fromLTWH(r.right - d, r.bottom - d, d, d), 0, 90);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `path.addArc(Rect.fromLTWH(r.left, r.bottom - d, d, d), 90, 90);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `path.closeFigure();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `return path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/admin/credential_slips_pdf.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/admin/credential_slips_source.dart

Credential slip PDF generation for printed login cards. This file (credential slips source) is one concrete piece of that layer.

**L9** `const _passwordStorePrefix = 'login_slip_pw_';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `Future<void> saveLoginPassword({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required String userId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required String password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `if (userId.isEmpty || password.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `await _writeStoredPassword(userId, password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L19** `'admin_save_login_password',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `params: {'p_user_id': userId, 'p_password': password},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Future<void> saveLoginPasswordsByUsername(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `Iterable<Map<String, dynamic>> rows,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `for (final row in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final username = (row['username'] as String?)?.trim() ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final password = (row['password'] as String?) ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (username.isEmpty || password.isEmpty) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final profile = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `.from('profiles')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L37** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L38** `.eq('username', username)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L39** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final id = profile?['id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `if (id == null) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `await saveLoginPassword(userId: id, password: password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `Future<List<CredentialSlip>> fetchCredentialSlips({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `required UserRole role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `String? groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `var slips = await _fetchFromRpc(role: role, groupId: groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `slips ??= await fetchCredentialSlipPreviews(role: role, groupId: groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `return uniqueCredentialSlips(await _ensurePrintablePasswords(slips));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `Future<List<CredentialSlip>?> _fetchFromRpc({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `required UserRole role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `String? groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final rows = await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L62** `'admin_list_login_slips',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `params: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `'p_role': role.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `'p_group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (rows is! List) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `return uniqueCredentialSlips(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `rows.map((raw) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final row = Map<String, dynamic>.from(raw as Map);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L72** `return CredentialSlip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `profileId: row['profile_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `username: (row['username'] as String? ?? '').trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `fullName: (row['full_name'] as String? ?? '').trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `password: (row['password'] as String? ?? '').trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `} on PostgrestException {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `Future<List<CredentialSlip>> fetchCredentialSlipPreviews({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `required UserRole role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `String? groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final slips = <CredentialSlip>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `if (role == UserRole.student && groupId != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final rows = await supabase`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `.from('group_students')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L94** `.select(`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L95** `'students(profile_id, profiles(id, username, full_name, role, is_disabled))',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L98** `for (final raw in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final student = raw['students'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `final profile = student?['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final slip = _slipFromProfile(profile);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `if (slip != null) slips.add(slip);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return uniqueCredentialSlips(slips);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `.from('profiles')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L109** `.select('id, username, full_name, role, is_disabled')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L110** `.eq('role', role.name)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L111** `.eq('is_disabled', false)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L112** `.order('full_name');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `for (final raw in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final slip = _slipFromProfile(raw);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `if (slip != null) slips.add(slip);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `return uniqueCredentialSlips(slips);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `CredentialSlip? _slipFromProfile(Map<dynamic, dynamic>? json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `if (json == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `if (json['is_disabled'] == true) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `final username = (json['username'] as String? ?? '').trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `if (username.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `return CredentialSlip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `profileId: json['id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `username: username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `fullName: (json['full_name'] as String? ?? '').trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `password: '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `Future<List<CredentialSlip>> _ensurePrintablePasswords(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `List<CredentialSlip> slips,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `final used = <String>{`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `for (final slip in slips)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `if (slip.password.isNotEmpty) slip.password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final out = <CredentialSlip>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `for (final slip in slips) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `if (slip.password.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final id = slip.profileId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `if (id != null) await _writeStoredPassword(id, slip.password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `out.add(slip);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final id = slip.profileId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `if (id == null || id.isEmpty) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `final stored = await _readStoredPassword(id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `if (stored != null && stored.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `used.add(stored);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `out.add(slip.copyWith(password: stored));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `var password = _newPassword();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `while (used.contains(password)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `password = _newPassword();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `used.add(password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L167** `'admin_reset_password',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `params: {'p_user_id': id, 'p_password': password},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `await saveLoginPassword(userId: id, password: password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `out.add(slip.copyWith(password: password));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `Future<String?> _readStoredPassword(String userId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `return prefs.getString('$_passwordStorePrefix$userId');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `Future<void> _writeStoredPassword(String userId, String password) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `await prefs.setString('$_passwordStorePrefix$userId', password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `String _newPassword() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `const alphabet =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `final rng = Random.secure();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `return List.generate(12, (_) => alphabet[rng.nextInt(alphabet.length)]).join();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/admin/credential_slips_source.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/chat/animated_emoji_registry.dart

Emoji helpers and animated emoji registry. This file (animated emoji registry) is one concrete piece of that layer.

**L1** `library;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L3** `const String kAnimatedEmojiAssetPrefix = 'assets/lottie/';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `const Map<String, String> kAnimatedEmojiAssets = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `'❤■': 'Heart.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `'❤': 'Heart.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `'■': 'Heart eyes.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `'■': 'Emoji 30 - Lovely Kiss.json', // face + hearts (kiss), not ■ mark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `'■': 'Emoji 30 - Lovely Kiss.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `'■': 'LMAO-2 side tears.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `'■': 'LMAO.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `'■': 'smiley emoji.json', // composition: Grinning Face`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `'■': 'Cool emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `'■': 'Winking Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `'■': 'Zany tongue.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `'■': 'Calm.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `'■': 'Yummy.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `'■': 'Crying emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `'■': 'Sad emoji to Crying emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `'■': 'Pensive.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `'■': 'Sad Emoji.json', // composition is pensive-like; closest sad we have`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `'■': 'Holding back tears.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `'■': 'Angry Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `'■': 'Angry.json', // composition: emoji_pouting`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `'■': 'Mad emoji.json', // composition: steam from nose`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `'■': 'Angry Face with Horns.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `'■': 'Angry Face with Symbols on Mouth Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `'■': 'Confounded Face.json', // filename; layers read as confused mouth`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `'■': 'Confounded Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `'■': 'Puke Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `'■': 'Sick face with thermometer.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `'■': 'Sick face with thermometer.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `'■': 'injury head face emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `'■': 'Dizzy Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `'■': 'Cold Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `'■': 'High Temperature.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `'■': 'Sleepy.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `'■': 'Yawn Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `'■': 'Drooling Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'■': 'Zipper mouth Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `'■': 'Rolling Eyes Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `'■': 'Neutral face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'■': 'Thinking.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'■': 'Anxious Face with Sweat.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `'■': 'Astonished Face.json',`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `'■': 'Astonished Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `'■': 'Party.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `'■': 'Confetti emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `'■': 'Confetti emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `'■': 'Sparkles Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `'■': 'Star Strike Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `'■': 'Sparkles Emoji.json', // sparkles, not a star-struck face`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `'■': 'Thumb.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `'■': 'Clapping Hands.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `'■': 'Muscle.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `'■': 'Please.json', // U+1F64F folded hands`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `'■': 'Crossed Fingers.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `'■■': 'Index finger.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `'■': 'Backhand Index Pointing Up.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `'■': 'Backhand Index Pointing Down.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `'■': 'Backhand Index Pointing Right.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `'■': 'Call Me Hand.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `'■': 'Hand.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `'■': 'Hand.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `'■': 'Middle finger.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `'■': 'Fire.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `'■': 'check-mark.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `'✔■': 'check-mark.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `'✔': 'check-mark.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `'■■': 'check-mark.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `'■': 'check-mark.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `'■': 'X.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `'■': 'Eyes.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `'■': 'Rocket.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `'■': 'Money.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `'■': 'FlyingMoney.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `'■': 'Unicorn Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `'■': 'Alien Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `'■': 'Animated Clown Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `'■': 'Cowboy Hat Face.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `'■': 'Halo.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `'■': 'Taxi.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `'■': 'Laptop.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `'■': 'Hourglass.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `'■': 'Hourglass.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `'■': '100% Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `'■': 'Bomb Emoji.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `'■': 'emoji head explosion.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `'■': 'emoji head explosion.json', // exploding head`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `'■': 'Medal Animations.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `'■': 'Medal Animations.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `'■■': 'High Temperature.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `'■': 'Emoji Walking.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `'■■■■': 'Emoji Walking.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `'■■■■': 'Emoji Walking.json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `String? animatedEmojiAssetPath(String emoji) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final direct = kAnimatedEmojiAssets[emoji];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `if (direct != null) return '$kAnimatedEmojiAssetPrefix$direct';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final stripped = emoji.replaceAll('\uFE0F', '').replaceAll('\uFE0E', '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `if (stripped != emoji) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final alt = kAnimatedEmojiAssets[stripped];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `if (alt != null) return '$kAnimatedEmojiAssetPrefix$alt';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final withVs16 = '$stripped\uFE0F';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `final vs = kAnimatedEmojiAssets[withVs16];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `if (vs != null) return '$kAnimatedEmojiAssetPrefix$vs';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `bool isAnimatedEmojiSupported(String emoji) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `animatedEmojiAssetPath(emoji) != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `Set<String> get registeredAnimatedEmojiFilenames =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `kAnimatedEmojiAssets.values.toSet();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `const Set<String> kAnimatedEmojiRedundantFilenames = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `'Starstruck.json', // same star-struck face as Star Strike Emoji.json →` ■
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/chat/animated_emoji_registry.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/chat/emoji_message.dart

Emoji helpers and animated emoji registry. This file (emoji message) is one concrete piece of that layer.

**L4** `String? singleAnimatedEmojiAsset(String? text) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `if (text == null || text.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `final trimmed = text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `if (trimmed.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `if (_containsWhitespace(trimmed)) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final graphemes = trimmed.characters;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `if (graphemes.length != 1) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final emoji = graphemes.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `return animatedEmojiAssetPath(emoji) ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `animatedEmojiAssetPath(stripEmojiVariationSelectors(emoji));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `bool isSingleAnimatedEmojiMessage(String? text) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `singleAnimatedEmojiAsset(text) != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `String? singleAnimatedEmojiGrapheme(String? text) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `if (!isSingleAnimatedEmojiMessage(text)) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return text!.trim().characters.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `String stripEmojiVariationSelectors(String emoji) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final buffer = StringBuffer();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `for (final r in emoji.runes) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (r == 0xFE0F || r == 0xFE0E) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `buffer.writeCharCode(r);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return buffer.toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `bool _containsWhitespace(String text) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `for (final unit in text.runes) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `if (unit == 0x20 ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `unit == 0x09 ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `unit == 0x0A ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `unit == 0x0D ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `unit == 0x200B ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `unit == 0x00A0 ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `unit == 0xFEFF ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `unit == 0x2060) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/chat/emoji_message.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/constants/env.dart

Compile-time and runtime configuration (Env). This file (env) is one concrete piece of that layer.

**L3** `class Env {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `static const _defaultUrl = 'https://nmmdvdprczukaskxxdrs.supabase.co';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `static const _placeholderKey = 'your_anon_key_here';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `static String supabaseUrl = _defaultUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `static String supabaseAnonKey = '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `static bool get isConfigured => supabaseAnonKey.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static void _sanitizeKey() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final key = supabaseAnonKey.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `if (key.isEmpty ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `key == _placeholderKey ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `key.startsWith('your_') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `key.contains('paste_your') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `key.startsWith('sb_secret_')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `supabaseAnonKey = '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `if (!key.startsWith('eyJ') && !key.startsWith('sb_publishable_')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `supabaseAnonKey = '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `static Future<void> load() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `const defineUrl = String.fromEnvironment('SUPABASE_URL');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `const defineKey = String.fromEnvironment('SUPABASE_ANON_KEY');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (defineKey.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `supabaseAnonKey = defineKey;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (defineUrl.isNotEmpty) supabaseUrl = defineUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `_sanitizeKey();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `await dotenv.load(fileName: '.env');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `supabaseAnonKey =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `(dotenv.env['SUPABASE_PUBLISHABLE_KEY'] ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `dotenv.env['SUPABASE_ANON_KEY'])`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `?.trim() ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final url = dotenv.env['SUPABASE_URL']?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `if (url != null && url.isNotEmpty) supabaseUrl = url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `_sanitizeKey();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/constants/env.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

## Every line — lib/core/focus/focus_time_up_cue.dart

Focus-mode audio/visual cues (local, not a database table). This file (focus time up cue) is one concrete piece of that layer.

**L3** `Future<void> playFocusTimeUpCue() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `await HapticFeedback.heavyImpact();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `await Future<void>.delayed(const Duration(milliseconds: 140));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `await HapticFeedback.mediumImpact();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `await SystemSound.play(SystemSoundType.alert);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/focus/focus_time_up_cue.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/homework/assignment_due.dart

Due-date helpers and assignment template JSON. This file (assignment due) is one concrete piece of that layer.

**L3** `enum AssignmentDueStatus { none, upcoming, dueSoon, dueToday, overdue }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L5** `DateTime assignmentCalendarDay(DateTime date) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `DateTime(date.year, date.month, date.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `DateTime? assignmentDueDay(Assignment a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final d = a.dueDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `if (d == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `return assignmentCalendarDay(d);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `DateTime startOfWeek(DateTime day) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final d = assignmentCalendarDay(day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `return d.subtract(Duration(days: d.weekday - DateTime.monday));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `DateTime endOfWeek(DateTime day) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `startOfWeek(day).add(const Duration(days: 6));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `AssignmentDueStatus assignmentDueStatus(Assignment a, [DateTime? now]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final due = assignmentDueDay(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `if (due == null) return AssignmentDueStatus.none;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final today = assignmentCalendarDay(now ?? DateTime.now());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `if (due.isBefore(today)) return AssignmentDueStatus.overdue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `if (due == today) return AssignmentDueStatus.dueToday;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final days = due.difference(today).inDays;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `if (days <= 2) return AssignmentDueStatus.dueSoon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `return AssignmentDueStatus.upcoming;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `bool assignmentIsDueThisWeek(Assignment a, [DateTime? now]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final due = assignmentDueDay(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `if (due == null) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final n = now ?? DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final start = startOfWeek(n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final end = endOfWeek(n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return !due.isBefore(start) && !due.isAfter(end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `bool assignmentIsOverdue(Assignment a, [DateTime? now]) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `assignmentDueStatus(a, now) == AssignmentDueStatus.overdue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `List<Assignment> sortAssignmentsByDue(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `Iterable<Assignment> items, [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `DateTime? now,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final list = items.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final n = now ?? DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `list.sort((a, b) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final da = assignmentDueDay(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final db = assignmentDueDay(b);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `if (da == null && db == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `return (b.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0)).compareTo(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `a.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `if (da == null) return 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `if (db == null) return -1;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final today = assignmentCalendarDay(n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final aOver = da.isBefore(today);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final bOver = db.isBefore(today);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `if (aOver != bOver) return aOver ? -1 : 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `return da.compareTo(db);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return list;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `List<Assignment> assignmentsDueThisWeekFocus(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `Iterable<Assignment> items, [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `DateTime? now,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `final n = now ?? DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `final filtered = items.where((a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `if (assignmentIsOverdue(a, n)) return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `return assignmentIsDueThisWeek(a, n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `return sortAssignmentsByDue(filtered, n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `List<Assignment> assignmentsForFocusSession(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `Iterable<Assignment> items, [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `DateTime? now,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `]) => sortAssignmentsByDue(items, now);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/homework/assignment_due.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/homework/assignment_templates.dart

Due-date helpers and assignment template JSON. This file (assignment templates) is one concrete piece of that layer.

**L4** `class TemplateTaskDraft {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L5** `const TemplateTaskDraft({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `this.description = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `this.attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `this.linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.sortOrder = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final String description;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String? attachmentUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String? linkUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final int sortOrder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `factory TemplateTaskDraft.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `return TemplateTaskDraft(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `title: json['title'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `description: json['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `attachmentUrl: json['attachment_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `linkUrl: json['link_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `sortOrder: json['sort_order'] as int? ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `factory TemplateTaskDraft.fromTask(TaskItem task) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `return TemplateTaskDraft(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `title: task.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `description: task.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `attachmentUrl: task.attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `linkUrl: task.linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `sortOrder: task.sortOrder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `Map<String, dynamic> toJson() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `'description': description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `'attachment_url': attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `'link_url': linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'sort_order': sortOrder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `class AssignmentTemplate {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L49** `const AssignmentTemplate({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `required this.teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `required this.type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `this.description = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `this.tasks = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `this.updatedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final String id;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final String teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final AssignmentType type;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final String description;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final List<TemplateTaskDraft> tasks;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final DateTime? createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final DateTime? updatedAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `factory AssignmentTemplate.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `final rawTasks = json['tasks'];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final tasks = <TemplateTaskDraft>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `if (rawTasks is List) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `for (final item in rawTasks) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (item is Map) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `tasks.add(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `TemplateTaskDraft.fromJson(Map<String, dynamic>.from(item)),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L81** `return AssignmentTemplate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `teacherId: json['teacher_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `type: assignmentTypeFromString(json['type'] as String? ?? 'static'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `title: json['title'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `description: json['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `tasks: tasks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `createdAt: json['created_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `? DateTime.tryParse(json['created_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `updatedAt: json['updated_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `? DateTime.tryParse(json['updated_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `Future<List<AssignmentTemplate>> fetchAssignmentTemplates(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `.from('assignment_templates')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L103** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L104** `.eq('teacher_id', teacherId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L105** `.order('updated_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `.map((e) => AssignmentTemplate.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L108** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `Future<AssignmentTemplate> saveAssignmentAsTemplate({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `required String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `required Assignment assignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final taskRows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L117** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L118** `.eq('assignment_id', assignment.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L119** `.order('sort_order', ascending: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `var drafts = taskRows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `(e) => TemplateTaskDraft.fromTask(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `TaskItem.fromJson(Map<String, dynamic>.from(e)),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `if (assignment.type == AssignmentType.staticAssignment) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `if (drafts.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `drafts = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `TemplateTaskDraft(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `title: assignment.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `description: assignment.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `final first = drafts.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `drafts = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `TemplateTaskDraft(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `title: first.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `description: first.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `attachmentUrl: first.attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `linkUrl: first.linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `final row = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `.from('assignment_templates')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L150** `.insert({`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L151** `'teacher_id': teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `'type': assignmentTypeToDb(assignment.type),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `'title': assignment.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `'description': assignment.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `'tasks': drafts.map((d) => d.toJson()).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `'updated_at': DateTime.now().toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L159** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `return AssignmentTemplate.fromJson(Map<String, dynamic>.from(row));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L164** `Future<void> deleteAssignmentTemplate(String templateId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `await supabase.from('assignment_templates').delete().eq('id', templateId);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L168** `Future<Assignment> createAssignmentFromTemplate({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `required AssignmentTemplate template,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `required String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `DateTime? dueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `String? titleOverride,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `String? descriptionOverride,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final inserted = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L178** `.insert({`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L179** `'teacher_id': teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `'type': assignmentTypeToDb(template.type),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `'title': titleOverride?.trim().isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `? titleOverride!.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `: template.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `'description': descriptionOverride ?? template.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `'due_date': dueDate?.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L189** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `final assignment = Assignment.fromJson(Map<String, dynamic>.from(inserted));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L193** `final title = assignment.title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final description = assignment.description;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `List<TemplateTaskDraft> tasksToInsert;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `if (template.type == AssignmentType.staticAssignment) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `final first = template.tasks.isNotEmpty ? template.tasks.first : null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `tasksToInsert = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `TemplateTaskDraft(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `title: first?.title.isNotEmpty == true ? first!.title : title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `description: first?.description ?? description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `attachmentUrl: first?.attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `linkUrl: first?.linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `tasksToInsert = template.tasks;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `if (tasksToInsert.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `await supabase.from('tasks').insert([`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L212** `for (var i = 0; i < tasksToInsert.length; i++)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `'assignment_id': assignment.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `'title': tasksToInsert[i].title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `'description': tasksToInsert[i].description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `'attachment_url': tasksToInsert[i].attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `'link_url': tasksToInsert[i].linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `'sort_order': i,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `return assignment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `Future<Assignment> duplicateAssignment({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `required Assignment source,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `required String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `DateTime? dueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `String? titleOverride,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `final taskRows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L236** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L237** `.eq('assignment_id', source.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L238** `.order('sort_order', ascending: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `final title = titleOverride?.trim().isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `? titleOverride!.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `: '${source.title} (copy)';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `final inserted = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L246** `.insert({`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L247** `'teacher_id': teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `'type': assignmentTypeToDb(source.type),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `'description': source.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `'due_date': dueDate?.toIso8601String(),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L255** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `final assignment = Assignment.fromJson(Map<String, dynamic>.from(inserted));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L259** `if (source.type == AssignmentType.staticAssignment) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `if (taskRows.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `await supabase.from('tasks').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L262** `'assignment_id': assignment.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `'description': source.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `'sort_order': 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `final t = TaskItem.fromJson(Map<String, dynamic>.from(taskRows.first));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L269** `await supabase.from('tasks').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L270** `'assignment_id': assignment.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `'title': t.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `'description': t.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `'attachment_url': t.attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `'link_url': t.linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `'sort_order': 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `} else if (taskRows.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `final tasks = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `for (final row in taskRows)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `TaskItem.fromJson(Map<String, dynamic>.from(row)),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L283** `await supabase.from('tasks').insert([`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L284** `for (var i = 0; i < tasks.length; i++)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `'assignment_id': assignment.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `'title': tasks[i].title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `'description': tasks[i].description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `'attachment_url': tasks[i].attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `'link_url': tasks[i].linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `'sort_order': i,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `return assignment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/homework/assignment_templates.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/homework/feedback_reuse.dart

Due-date helpers and assignment template JSON. This file (feedback reuse) is one concrete piece of that layer.

**L6** `const _snippetsPrefsKey = 'teacher_feedback_snippets';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `const _maxSnippets = 20;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `Future<List<String>> loadFeedbackSnippets({required String teacherId}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final local = _readLocal(prefs, teacherId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final remote = await _fetchRecentFeedbackFromDb(teacherId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `return _mergeUnique(local, remote);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `Future<void> rememberFeedbackSnippet({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `required String feedback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final text = feedback.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `if (text.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final current = _readLocal(prefs, teacherId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final next = _mergeUnique([text], current).take(_maxSnippets).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final all = _readAll(prefs);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `all[teacherId] = next;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `await prefs.setString(_snippetsPrefsKey, jsonEncode(all));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Future<int> applyFeedbackToOtherPendingForSameTask({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `required String submissionId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `required String feedback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final text = feedback.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (text.isEmpty) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final row = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L42** `.select('task_id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L43** `.eq('id', submissionId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L44** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final taskId = row?['task_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `if (taskId == null) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final siblings = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L50** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L51** `.eq('task_id', taskId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L52** `.neq('id', submissionId)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `.inFilter('status', ['pending', 'submitted']);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `if (siblings.isEmpty) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final ids = siblings.map((e) => e['id'] as String).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L60** `.update({'feedback': text})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L61** `.inFilter('id', ids);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `return ids.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `Future<List<String>> _fetchRecentFeedbackFromDb(String teacherId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final assignments = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L68** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L69** `.eq('teacher_id', teacherId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L70** `final assignmentIds = assignments.map((e) => e['id'] as String).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `if (assignmentIds.isEmpty) return const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final tasks = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L75** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L76** `.inFilter('assignment_id', assignmentIds);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `final taskIds = tasks.map((e) => e['id'] as String).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `if (taskIds.isEmpty) return const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82** `.select('feedback')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L83** `.inFilter('task_id', taskIds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `.order('submitted_at', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `.limit(40);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final out = <String>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `for (final row in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final f = (row['feedback'] as String?)?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `if (f == null || f.isEmpty) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `if (!out.contains(f)) out.add(f);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `if (out.length >= _maxSnippets) break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `List<String> _readLocal(SharedPreferences prefs, String teacherId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final all = _readAll(prefs);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final list = all[teacherId];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `if (list is! List) return const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `return list`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `.whereType<String>()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `.map((e) => e.trim())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `.where((e) => e.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `Map<String, dynamic> _readAll(SharedPreferences prefs) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final raw = prefs.getString(_snippetsPrefsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `if (raw == null || raw.isEmpty) return {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final decoded = jsonDecode(raw);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (decoded is Map<String, dynamic>) return decoded;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `if (decoded is Map) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `return decoded.map((k, v) => MapEntry(k.toString(), v));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `return {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `List<String> _mergeUnique(List<String> primary, List<String> secondary) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final seen = <String>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `final out = <String>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `for (final item in [...primary, ...secondary]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `final t = item.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `if (t.isEmpty || seen.contains(t)) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `seen.add(t);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `out.add(t);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/homework/feedback_reuse.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/locale/app_locale.dart

Supported locales EN/RU/UZ. This file (app locale) is one concrete piece of that layer.

**L3** `enum AppLanguage {`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L4** `english('en'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `russian('ru'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `uzbek('uz');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `const AppLanguage(this.code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final String code;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `Locale get locale => Locale(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `static AppLanguage? tryParse(String? code) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `if (code == null || code.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `for (final lang in AppLanguage.values) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `if (lang.code == code) return lang;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static AppLanguage fromLocale(Locale locale) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `return tryParse(locale.languageCode) ?? AppLanguage.english;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/locale/app_locale.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/marks/academic_term.dart

Academic term math and CSV export. This file (academic term) is one concrete piece of that layer.

**L5** `class AcademicTerm {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const AcademicTerm({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.academicYearStart,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.semester,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.startInclusive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.endInclusive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.yearIsCurrent = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.dbTermId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.dbYearId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final int? academicYearStart;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final int? semester;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final DateTime? startInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final DateTime? endInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final bool yearIsCurrent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final String? dbTermId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final String? dbYearId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `bool get isAll => id == 'all';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `static const AcademicTerm all = AcademicTerm(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `id: 'all',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `academicYearStart: null,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `semester: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `static (DateTime, DateTime) bounds(int academicYearStart, int semester) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `assert(semester >= 1 && semester <= 4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `return switch (semester) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `1 => (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `DateTime(academicYearStart, 8, 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `DateTime(academicYearStart, 11, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `2 => (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `DateTime(academicYearStart, 11, 11),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `DateTime(academicYearStart, 12, 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `3 => (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `DateTime(academicYearStart + 1, 1, 11),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `DateTime(academicYearStart + 1, 3, 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `_ => (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `DateTime(academicYearStart + 1, 3, 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `DateTime(academicYearStart + 1, 6, 19),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `static AcademicTerm semesterOf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `int academicYearStart,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `int semester, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `bool yearIsCurrent = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `String? dbTermId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `String? dbYearId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `DateTime? startInclusive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `DateTime? endInclusive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `assert(semester >= 1 && semester <= 4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final (start, end) = bounds(academicYearStart, semester);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `return AcademicTerm(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `id: '$academicYearStart-$semester',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `academicYearStart: academicYearStart,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `semester: semester,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `startInclusive: startInclusive ?? start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `endInclusive: endInclusive ?? end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `yearIsCurrent: yearIsCurrent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `dbTermId: dbTermId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `dbYearId: dbYearId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `static int academicYearFor(DateTime now) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `now.month >= 8 ? now.year : now.year - 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `static int semesterFor(DateTime now) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final ay = academicYearFor(now);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final day = DateTime(now.year, now.month, now.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `for (var s = 1; s <= 4; s++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final (start, end) = bounds(ay, s);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `if (!day.isBefore(start) && !day.isAfter(end)) return s;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `for (var s = 1; s <= 4; s++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final (start, _) = bounds(ay, s);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `if (day.isBefore(start)) return s;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `return 4;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `static AcademicTerm currentFrom(List<AcademicTerm> options, [DateTime? now]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final n = now ?? DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `final day = DateTime(n.year, n.month, n.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final terms = options.where((t) => !t.isAll).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (terms.isEmpty) return all;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final currentYearTerms = terms.where((t) => t.yearIsCurrent).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `final pool = currentYearTerms.isNotEmpty ? currentYearTerms : terms;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final matching = pool.where((t) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final start = t.startInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final end = t.endInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (start == null || end == null) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return !day.isBefore(start) && !day.isAfter(end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `}).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `if (matching.isNotEmpty) return matching.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `pool.sort((a, b) => (a.semester ?? 0).compareTo(b.semester ?? 0));`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `final first = pool.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `final last = pool.last;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `if (first.startInclusive != null && day.isBefore(first.startInclusive!)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `return all;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `if (last.endInclusive != null && day.isAfter(last.endInclusive!)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `return all;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `for (final t in pool) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `final start = t.startInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `if (start != null && day.isBefore(start)) return t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `return last;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `String label(AppLocalizations l10n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `if (isAll) return l10n.marksTermAll;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `final start = academicYearStart!;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final endShort = (start + 1) % 100;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `final yearRange = '$start–${endShort.toString().padLeft(2, '0')}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `return l10n.marksTermSemester(yearRange, semester!);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `String? get startKey => startInclusive?.toIso8601String().split('T').first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `String? get endKey => endInclusive?.toIso8601String().split('T').first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `class AcademicYearRecord {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L151** `const AcademicYearRecord({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `required this.startYear,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `required this.isCurrent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `required this.terms,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `final int startYear;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `final bool isCurrent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `final List<AcademicTerm> terms;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `factory AcademicYearRecord.fromJson(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `Map<String, dynamic> json,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `List<Map<String, dynamic>> termRows,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `final yearId = json['id'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `final startYear = json['start_year'] as int;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `final isCurrent = json['is_current'] as bool? ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `final terms = termRows.map((t) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `final semester = t['semester'] as int;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `final start = DateTime.parse(t['starts_on'] as String);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `final end = DateTime.parse(t['ends_on'] as String);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `return AcademicTerm.semesterOf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `startYear,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `semester,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `yearIsCurrent: isCurrent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `dbTermId: t['id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `dbYearId: yearId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `startInclusive: DateTime(start.year, start.month, start.day),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `endInclusive: DateTime(end.year, end.month, end.day),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `}).toList()..sort((a, b) => (a.semester ?? 0).compareTo(b.semester ?? 0));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `return AcademicYearRecord(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `id: yearId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `startYear: startYear,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `label:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `json['label'] as String? ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `'$startYear–${((startYear + 1) % 100).toString().padLeft(2, '0')}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `isCurrent: isCurrent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `terms: terms,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `Future<List<AcademicYearRecord>> fetchAcademicYears() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `final years = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `.from('academic_years')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L202** `.select('id, start_year, label, is_current, created_at')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L203** `.order('start_year', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `final yearList = (years as List).cast<Map<String, dynamic>>();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `if (yearList.isEmpty) return const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `final terms = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `.from('academic_terms')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L210** `.select('id, academic_year_id, semester, starts_on, ends_on')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L211** `.order('semester');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `final termList = (terms as List).cast<Map<String, dynamic>>();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `final byYear = <String, List<Map<String, dynamic>>>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `for (final t in termList) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `final yid = t['academic_year_id'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `byYear.putIfAbsent(yid, () => []).add(t);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `return [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `for (final y in yearList)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `AcademicYearRecord.fromJson(y, byYear[y['id'] as String] ?? const []),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `final academicTermOptionsProvider = FutureProvider<List<AcademicTerm>>((`
Riverpod binding. Name it after the value it exposes.

**L227** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `final years = await fetchAcademicYears();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `return [AcademicTerm.all, for (final y in years) ...y.terms];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `final academicYearsProvider = FutureProvider<List<AcademicYearRecord>>((`
Riverpod binding. Name it after the value it exposes.

**L234** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `return fetchAcademicYears();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `class SelectedMarkTermNotifier extends Notifier<AcademicTerm> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L240** `bool _initializedFromDb = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `AcademicTerm build() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `ref.listen(academicTermOptionsProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L245** `next.whenData((options) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `if (options.length <= 1) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `final current = state;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `AcademicTerm? match;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `for (final t in options) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `if (t.id == current.id) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `match = t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `if (!_initializedFromDb || match == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `_initializedFromDb = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `state = AcademicTerm.currentFrom(options);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `if (match.startKey != current.startKey ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `match.endKey != current.endKey ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `match.yearIsCurrent != current.yearIsCurrent) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `state = match;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `return AcademicTerm.all;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `void select(AcademicTerm term) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `state = term;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `final selectedMarkTermProvider =`
Riverpod binding. Name it after the value it exposes.

**L276** `NotifierProvider<SelectedMarkTermNotifier, AcademicTerm>(`
Riverpod binding. Name it after the value it exposes.

**L277** `SelectedMarkTermNotifier.new,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/marks/academic_term.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/marks/marks_csv_export.dart

Academic term math and CSV export. This file (marks csv export) is one concrete piece of that layer.

**L9** `String _csvCell(Object? value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final s = '${value ?? ''}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `if (s.contains(',') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `s.contains('"') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `s.contains('\n') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `s.contains('\r')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `return '"${s.replaceAll('"', '""')}"';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `return s;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Future<void> exportGroupMarksCsv({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `required String groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `required AcademicTerm term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `var query = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `.from('marks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L27** `.select('*, students(profiles(full_name, username))')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L28** `.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L30** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `if (start != null) query = query.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (end != null) query = query.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final rows = await query`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `.order('marked_on', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `.order('created_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final buffer = StringBuffer()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `..writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `'Student',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `'Username',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'Title',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `'Category',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `'Percentage',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'Weight',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'Date',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `'Comment',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `].map(_csvCell).join(','),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `for (final row in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final mark = Mark.fromJson(Map<String, dynamic>.from(row));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L55** `final student = row['students'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final profile = student?['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `buffer.writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `profile?['full_name'] ?? mark.studentName ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `profile?['username'] ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `mark.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `mark.category.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `mark.percentage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `mark.weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `dateKey(mark.markedOn),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `mark.comment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `].map(_csvCell).join(','),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final safeTitle = groupTitle`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `.toLowerCase()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `.replaceAll(RegExp(r'[^a-z0-9]+'), '_')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `.replaceAll(RegExp(r'^_|_$'), '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `final fileName =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `'marks_${safeTitle.isEmpty ? groupId : safeTitle}_${term.id}.csv';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `final bytes = utf8.encode(buffer.toString());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `await SharePlus.instance.share(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `ShareParams(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `files: [XFile.fromData(bytes, mimeType: 'text/csv', name: fileName)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `fileNameOverrides: [fileName],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `subject: 'Marks — $groupTitle',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `downloadFallbackEnabled: true,`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/marks/marks_csv_export.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/paging/page_result.dart

Generic page-result type for list pagination. This file (page result) is one concrete piece of that layer.

**L1** `class PageResult<T> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L2** `const PageResult({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L3** `required this.items,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `required this.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `this.loadingMore = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `final List<T> items;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final bool hasMore;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final bool loadingMore;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static const int pageSize = 40;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static int fetchSize(int pageSize) => pageSize + 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static PageResult<T> fromFetch<T>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `List<T> fetched, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `int pageSize = PageResult.pageSize,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final hasMore = fetched.length > pageSize;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final items = hasMore ? fetched.sublist(0, pageSize) : fetched;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `return PageResult(items: items, hasMore: hasMore);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `PageResult<T> copyWith({List<T>? items, bool? hasMore, bool? loadingMore}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `return PageResult(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `items: items ?? this.items,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `hasMore: hasMore ?? this.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `loadingMore: loadingMore ?? this.loadingMore,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `PageResult<T> append(List<T> next, {required bool hasMore}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return PageResult(items: [...items, ...next], hasMore: hasMore);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/paging/page_result.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/progress/progress_report.dart

Student progress report model + PDF + share. This file (progress report) is one concrete piece of that layer.

**L8** `class ProgressReport {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const ProgressReport({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.termLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `required this.average,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.markCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.attendance,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `required this.categoryAverages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required this.recentMarks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `this.generatedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String studentName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final String groupTitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final String termLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final AcademicTerm term;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final double? average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final int markCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final AttendanceStats attendance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final List<({MarkCategory category, double average})> categoryAverages;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final List<Mark> recentMarks;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final DateTime? generatedAt;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `double get attendanceRate => attendance.presentPercent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `Future<ProgressReport> fetchProgressReport({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `required String studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `required String groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `required AcademicTerm term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `required AppLocalizations l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `var marksQuery = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `.from('marks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L46** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L47** `.eq('student_id', studentId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L48** `.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L49** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (start != null) marksQuery = marksQuery.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `if (end != null) marksQuery = marksQuery.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final markRows = await marksQuery`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `.order('marked_on', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `.order('created_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final marks = markRows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `.map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L59** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `double? average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `if (marks.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `var total = 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `var weight = 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `for (final m in marks) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `total += m.percentage * m.weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `weight += m.weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `if (weight > 0) average = total / weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final byCategory = <MarkCategory, List<Mark>>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `for (final m in marks) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `byCategory.putIfAbsent(m.category, () => []).add(m);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final categoryAverages = byCategory.entries.map((e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `var total = 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `var weight = 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `for (final m in e.value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `total += m.percentage * m.weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `weight += m.weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `return (category: e.key, average: weight == 0 ? 0.0 : total / weight);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `}).toList()..sort((a, b) => b.average.compareTo(a.average));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final from =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `term.startInclusive ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `DateTime(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `now.year,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `now.month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `now.day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `).subtract(const Duration(days: 120));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final to = term.endInclusive ?? DateTime(now.year, now.month, now.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `final attendanceRows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `.from('attendance')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L98** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L99** `.eq('group_id', groupId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L100** `.eq('student_id', studentId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L101** `.gte('date', dateKey(from))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `.lte('date', dateKey(to))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `.order('date', ascending: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final attendance = AttendanceStats.fromRecords(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `attendanceRows.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `(e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `return ProgressReport(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `studentName: studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `termLabel: term.label(l10n),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `term: term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `average: average,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `markCount: marks.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `attendance: attendance,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `categoryAverages: categoryAverages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `recentMarks: marks.take(12).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `generatedAt: DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `String progressReportShareText(ProgressReport report, AppLocalizations l10n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `final avg = report.average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `final avgLine = avg == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `? l10n.marksNoneYet`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `: '${avg.toStringAsFixed(avg % 1 == 0 ? 0 : 1)}%';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `final att = report.attendance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `final buf = StringBuffer()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `..writeln(l10n.progressReportTitle)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `..writeln('${report.studentName} · ${report.groupTitle}')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `..writeln(report.termLabel)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `..writeln()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `..writeln('${l10n.marksOverallAverage}: $avgLine')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `..writeln('${l10n.progressReportMarksCounted}: ${report.markCount}')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `..writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `'${l10n.progressReportAttendanceRate}: '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `'${att.total == 0 ? '—' : '${report.attendanceRate.toStringAsFixed(0)}%'}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `if (att.total > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `buf.writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `'${l10n.attendancePresentWord}: ${att.present} · '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `'${l10n.attendanceAbsentWord}: ${att.absent} · '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `'${l10n.attendanceLateWord}: ${att.late} · '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `'${l10n.attendanceExcusedWord}: ${att.excused}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `buf`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `..writeln()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `..writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `l10n.progressReportGeneratedOn(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `DateFormat.yMMMd().add_jm().format(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `report.generatedAt ?? DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `return buf.toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/progress/progress_report.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/progress/progress_report_pdf.dart

Student progress report model + PDF + share. This file (progress report pdf) is one concrete piece of that layer.

**L10** `Uint8List buildProgressReportPdf(ProgressReport report, AppLocalizations l10n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final document = PdfDocument();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final page = document.pages.add();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final size = page.getClientSize();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final g = page.graphics;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final titleFont = PdfStandardFont(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `PdfFontFamily.helvetica,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `style: PdfFontStyle.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final headingFont = PdfStandardFont(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `PdfFontFamily.helvetica,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `style: PdfFontStyle.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final bodyFont = PdfStandardFont(PdfFontFamily.helvetica, 11);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final smallFont = PdfStandardFont(PdfFontFamily.helvetica, 9);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final ink = PdfColor(44, 36, 22);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final muted = PdfColor(110, 100, 88);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final accent = PdfColor(107, 79, 53);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final line = PdfPen(PdfColor(220, 210, 195), width: 0.8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `var y = 28.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `void drawText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `String text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `PdfFont font, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `PdfColor? color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `double? x,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `double? maxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `font,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `brush: PdfSolidBrush(color ?? ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `bounds: Rect.fromLTWH(x ?? 28, y, maxWidth ?? (size.width - 56), 40),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `drawText('MU Study', titleFont, color: accent);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `y += 26;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `drawText(l10n.progressReportTitle, headingFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `y += 22;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `drawText('${report.studentName} · ${report.groupTitle}', bodyFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `y += 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `drawText(report.termLabel, bodyFont, color: muted);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `y += 14;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `g.drawLine(line, Offset(28, y), Offset(size.width - 28, y));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `y += 18;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `drawText(l10n.marksOverallAverage, headingFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `y += 18;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final avg = report.average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final avgText = avg == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `? l10n.marksNoneYet`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `: '${avg.toStringAsFixed(avg % 1 == 0 ? 0 : 1)}% (${l10n.progressReportMarksCounted}: ${report.markCount})';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `drawText(avgText, bodyFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `y += 22;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `if (report.categoryAverages.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `drawText(l10n.marksByCategory, headingFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `y += 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `for (final row in report.categoryAverages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `drawText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `'${row.category.l10nLabel(l10n)} — ${row.average.toStringAsFixed(1)}%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `bodyFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `y += 14;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `if (y > size.height - 120) break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `y += 8;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `drawText(l10n.progressReportAttendance, headingFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `y += 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final att = report.attendance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `if (att.total == 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `drawText(l10n.progressReportNoAttendance, bodyFont, color: muted);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `y += 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `drawText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `'${l10n.progressReportAttendanceRate}: ${report.attendanceRate.toStringAsFixed(0)}% (${att.total} ${l10n.progressReportSessions})',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `bodyFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `y += 14;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `drawText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `'${l10n.attendancePresentWord}: ${att.present} '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `'${l10n.attendanceAbsentWord}: ${att.absent} '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `'${l10n.attendanceLateWord}: ${att.late} '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `'${l10n.attendanceExcusedWord}: ${att.excused}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `bodyFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `y += 20;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `if (report.recentMarks.isNotEmpty && y < size.height - 100) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `drawText(l10n.progressReportRecentMarks, headingFont);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `y += 16;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `for (final m in report.recentMarks) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `if (y > size.height - 60) break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final date = DateFormat.yMMMd().format(m.markedOn);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `drawText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `'$date · ${m.title} · ${m.percentage.toStringAsFixed(0)}%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `smallFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `color: muted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `y += 12;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final generated = DateFormat.yMMMd().add_jm().format(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `report.generatedAt ?? DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `g.drawString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `l10n.progressReportGeneratedOn(generated),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `smallFont,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `brush: PdfSolidBrush(muted),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `bounds: Rect.fromLTWH(28, size.height - 36, size.width - 56, 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `final bytes = Uint8List.fromList(document.saveSync());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `document.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `return bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/progress/progress_report_pdf.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/progress/progress_report_share.dart

Student progress report model + PDF + share. This file (progress report share) is one concrete piece of that layer.

**L7** `Future<void> shareProgressReport({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required String studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required String groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required AcademicTerm term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required AppLocalizations l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final report = await fetchProgressReport(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `studentId: studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `studentName: studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `term: term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `l10n: l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final pdfBytes = buildProgressReportPdf(report, l10n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final text = progressReportShareText(report, l10n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final safeStudent = studentName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `.toLowerCase()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `.replaceAll(RegExp(r'[^a-z0-9]+'), '_')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `.replaceAll(RegExp(r'^_|_$'), '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final fileName =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `'progress_${safeStudent.isEmpty ? studentId : safeStudent}_${term.id}.pdf';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `await SharePlus.instance.share(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `ShareParams(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `text: text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `subject: '${l10n.progressReportTitle} — $studentName',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `files: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `XFile.fromData(pdfBytes, mimeType: 'application/pdf', name: fileName),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `fileNameOverrides: [fileName],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `downloadFallbackEnabled: true,`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L43** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/progress/progress_report_share.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/router/app_router.dart

go_router route table and auth/language/disabled redirects. This file (app router) is one concrete piece of that layer.

**L43** `final routerProvider = Provider<GoRouter>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L44** `final notifier = _RouterNotifier(ref);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `return GoRouter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `initialLocation: '/splash',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `refreshListenable: notifier,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `redirect: notifier.redirect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `GoRoute(path: '/splash', builder: (_, _) => const SplashScreen()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `GoRoute(path: '/login', builder: (_, _) => const LoginScreen()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `path: '/language',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `builder: (_, _) => const LanguageSelectionScreen(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `path: '/boot',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `builder: (_, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `const Scaffold(body: Center(child: CircularProgressIndicator())),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `path: '/account-disabled',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `builder: (context, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `Scaffold(body: Center(child: Text(context.l10n.accountDisabled))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `_studentShell(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `_teacherShell(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `_adminShell(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `class _RouterNotifier extends ChangeNotifier {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L75** `_RouterNotifier(this._ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `_ref.listen(authStateProvider, (_, _) => notifyListeners());`
Riverpod binding. Name it after the value it exposes.

**L77** `_ref.listen(currentProfileProvider, (_, _) => notifyListeners());`
Riverpod binding. Name it after the value it exposes.

**L80** `final Ref _ref;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `String? redirect(BuildContext context, GoRouterState state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `final loc = state.matchedLocation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `if (loc == '/splash') return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `final session = Supabase.instance.client.auth.currentSession;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `if (session == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `return loc == '/login' ? null : '/login';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final profileAsync = _ref.read(currentProfileProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L96** `if (!profileAsync.hasValue &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `(profileAsync.isLoading || profileAsync.isRefreshing)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `if (loc == '/boot' || loc == '/language') return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `return '/boot';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final profile = profileAsync.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `if (profile == null) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return loc == '/login' ? null : '/login';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `if (profile.isDisabled) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `return loc == '/account-disabled' ? null : '/account-disabled';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final home = switch (profile.role) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `UserRole.student => '/student/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `UserRole.teacher => '/teacher/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `UserRole.admin => '/admin/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `if (!profile.hasChosenLanguage) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `return loc == '/language' ? null : '/language';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `if (loc == '/login' || loc == '/boot' || loc == '/language') {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `return home;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `ShellRoute _studentShell() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `const destinations = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `labelKey: 'navDashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `icon: Icons.dashboard_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `path: '/student/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `labelKey: 'navAssignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `icon: Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `path: '/student/assignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `labelKey: 'navFocus',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `icon: Icons.self_improvement_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `path: '/student/focus',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `labelKey: 'navMarks',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `path: '/student/marks',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `labelKey: 'navChat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `icon: Icons.chat_bubble_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `path: '/student/chat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `badge: ShellBadgeKind.chat,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `labelKey: 'navAttendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `icon: Icons.event_available_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `path: '/student/attendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `labelKey: 'navProfile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `icon: Icons.person_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `path: '/student/profile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `return ShellRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `builder: (context, state, child) => AppShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `role: UserRole.student,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `destinations: destinations,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `path: '/student/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `pageBuilder: (_, s) => _fade(const StudentDashboardScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `path: '/student/assignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `pageBuilder: (_, s) => _fade(const StudentAssignmentsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `path: ':id',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `builder: (_, s) => StudentAssignmentDetailScreen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `assignmentId: s.pathParameters['id']!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `path: '/student/tasks/:taskId',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `builder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `StudentTaskSubmitScreen(taskId: s.pathParameters['taskId']!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `path: '/student/focus',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `pageBuilder: (_, s) => _fade(const StudentFocusScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `path: '/student/marks',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `pageBuilder: (_, s) => _fade(const StudentMarksScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `path: '/student/chat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `pageBuilder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `_fade(const ChatListScreen(basePath: '/student/chat'), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `path: ':roomId',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `builder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `ChatRoomScreen(roomId: s.pathParameters['roomId']!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `path: '/student/attendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `pageBuilder: (_, s) => _fade(const StudentAttendanceScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `path: '/student/notifications',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `pageBuilder: (_, s) => _fade(const StudentNotificationsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `path: '/student/profile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `pageBuilder: (_, s) => _fade(const StudentProfileScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `ShellRoute _teacherShell() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `const destinations = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `labelKey: 'navDashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `icon: Icons.dashboard_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `path: '/teacher/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `labelKey: 'navAssignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `icon: Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `path: '/teacher/assignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `labelKey: 'navChat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `icon: Icons.chat_bubble_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `path: '/teacher/chat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `badge: ShellBadgeKind.chat,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `labelKey: 'navAttendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `icon: Icons.event_available_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `path: '/teacher/attendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `labelKey: 'navGradebook',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `path: '/teacher/marks',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `labelKey: 'navProfile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `icon: Icons.person_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `path: '/teacher/profile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `return ShellRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `builder: (context, state, child) => AppShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `role: UserRole.teacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `destinations: destinations,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `path: '/teacher/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `pageBuilder: (_, s) => _fade(const TeacherDashboardScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `path: '/teacher/submissions',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `builder: (_, _) => const TeacherSubmissionsScreen(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `GoRoute(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `path: '/teacher/assignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `pageBuilder: (_, s) => _fade(const TeacherAssignmentsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `path: ':id',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `builder: (_, s) => TeacherHomeworkPackScreen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `assignmentId: s.pathParameters['id']!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `path: '/teacher/chat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `pageBuilder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `_fade(const ChatListScreen(basePath: '/teacher/chat'), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `path: ':roomId',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `builder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `ChatRoomScreen(roomId: s.pathParameters['roomId']!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `path: '/teacher/attendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `pageBuilder: (_, s) => _fade(const TeacherAttendanceScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `path: '/teacher/marks',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `pageBuilder: (_, s) => _fade(const TeacherMarksScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `path: '/teacher/analytics',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `builder: (_, _) => const TeacherAnalyticsScreen(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `path: '/teacher/announcements',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `pageBuilder: (_, s) => _fade(const TeacherAnnouncementsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `path: '/teacher/notifications',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `pageBuilder: (_, s) => _fade(const TeacherNotificationsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `path: '/teacher/profile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `pageBuilder: (_, s) => _fade(const TeacherProfileScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `ShellRoute _adminShell() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `const destinations = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `labelKey: 'navDashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `icon: Icons.dashboard_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `path: '/admin/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `labelKey: 'navUsers',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `icon: Icons.people_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `path: '/admin/users',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `labelKey: 'navGroups',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `icon: Icons.groups_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `path: '/admin/groups',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `labelKey: 'navChat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `icon: Icons.chat_bubble_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `path: '/admin/chat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `badge: ShellBadgeKind.chat,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `labelKey: 'navAnalytics',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `icon: Icons.insights_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `path: '/admin/analytics',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `ShellDestination(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `labelKey: 'navProfile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `icon: Icons.person_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `path: '/admin/profile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `return ShellRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `builder: (context, state, child) => AppShell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `role: UserRole.admin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `destinations: destinations,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `path: '/admin/dashboard',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `pageBuilder: (_, s) => _fade(const AdminDashboardScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `path: '/admin/users',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `pageBuilder: (_, s) => _fade(const AdminUsersScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `path: '/admin/groups',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `pageBuilder: (_, s) => _fade(const AdminGroupsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `path: '/admin/chat',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `pageBuilder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `_fade(const ChatListScreen(basePath: '/admin/chat'), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `routes: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `path: ':roomId',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `builder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `ChatRoomScreen(roomId: s.pathParameters['roomId']!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `path: '/admin/analytics',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `pageBuilder: (_, s) => _fade(const AdminAnalyticsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `path: '/admin/academic-years',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `pageBuilder: (_, s) => _fade(const AdminAcademicYearsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `path: '/admin/announcements',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `pageBuilder: (_, s) => _fade(const AdminAnnouncementsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `path: '/admin/notifications',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `pageBuilder: (_, s) => _fade(const AdminNotificationsScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `path: '/admin/profile',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `pageBuilder: (_, s) => _fade(const AdminProfileScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `GoRoute(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `path: '/admin/mass-generator',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `pageBuilder: (_, s) => _fade(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `const AdminMassDataGeneratorScreen(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `s,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `GoRoute(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `path: '/admin/credential-print',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `pageBuilder: (_, s) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `_fade(const AdminCredentialPrintScreen(), s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `CustomTransitionPage<void> _fade(Widget child, GoRouterState state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `return CustomTransitionPage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `key: state.pageKey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `transitionsBuilder: (_, animation, _, child) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `FadeTransition(opacity: animation, child: child),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/router/app_router.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/services/app_cache.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (app cache) is one concrete piece of that layer.

**L8** `Future<void> clearChatAndMediaCaches(CacheService? prefsCache) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `await prefsCache?.clearChatMessageCaches();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `await VoiceCacheService.instance.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `await ChatMediaCacheService.instance.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `await ChatVideoThumbnailService.instance.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `await DefaultCacheManager().emptyCache();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `Future<void> clearAllAppCaches(CacheService? prefsCache) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `await prefsCache?.clearAll();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `await VoiceCacheService.instance.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `await ChatMediaCacheService.instance.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `await ChatVideoThumbnailService.instance.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `await DefaultCacheManager().emptyCache();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `void prefetchChatMedia(Iterable<ChatMessage> messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `for (final m in messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final url = m.mediaUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (url == null || url.isEmpty) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `if (m.messageType == ChatMessageType.voice) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final ext = switch (m.messageType) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `ChatMessageType.image => ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `fallback: 'jpg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `ChatMessageType.video => ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `fallback: 'mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `ChatMessageType.file => ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `fallback: (m.textContent ?? 'bin').split('.').last,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `_ => 'bin',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `ChatMediaCacheService.instance.prefetch(url, preferredExtension: ext);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (m.messageType == ChatMessageType.video) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `ChatVideoThumbnailService.instance.prefetch(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/services/app_cache.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/services/cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (cache service) is one concrete piece of that layer.

**L6** `class CacheService {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `CacheService(this._prefs);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final SharedPreferences _prefs;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `static const _profileKey = 'cached_profile';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static const _notificationsKey = 'cached_notifications';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `static const _assignmentsKey = 'cached_assignments';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static const _dashboardKey = 'cached_dashboard';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const _chatRoomIdsKey = 'cached_chat_room_ids';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static String _chatMessagesKey(String roomId) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `'cached_chat_messages_$roomId';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Future<void> saveJson(String key, Map<String, dynamic> data) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `await _prefs.setString(key, jsonEncode(data));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Map<String, dynamic>? readJson(String key) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final raw = _prefs.getString(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `if (raw == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `return jsonDecode(raw) as Map<String, dynamic>;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `Future<void> saveList(String key, List<Map<String, dynamic>> items) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `await _prefs.setString(key, jsonEncode(items));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `List<Map<String, dynamic>>? readList(String key) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final raw = _prefs.getString(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (raw == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `return (jsonDecode(raw) as List<dynamic>)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `.map((e) => Map<String, dynamic>.from(e as Map))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L39** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `Future<void> cacheProfile(Map<String, dynamic> profile) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `saveJson(_profileKey, profile);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `Map<String, dynamic>? get cachedProfile => readJson(_profileKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `Future<void> cacheNotifications(List<Map<String, dynamic>> items) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `saveList(_notificationsKey, items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `List<Map<String, dynamic>>? get cachedNotifications =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `readList(_notificationsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `Future<void> cacheAssignments(List<Map<String, dynamic>> items) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `saveList(_assignmentsKey, items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `List<Map<String, dynamic>>? get cachedAssignments =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `readList(_assignmentsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `Future<void> cacheDashboard(Map<String, dynamic> data) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `saveJson(_dashboardKey, data);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `Map<String, dynamic>? get cachedDashboard => readJson(_dashboardKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `Future<void> cacheChatMessages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `String roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `List<Map<String, dynamic>> rows,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `await saveList(_chatMessagesKey(roomId), rows);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `final ids = _prefs.getStringList(_chatRoomIdsKey) ?? <String>[];`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `if (!ids.contains(roomId)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `await _prefs.setStringList(_chatRoomIdsKey, [...ids, roomId]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `List<Map<String, dynamic>>? cachedChatMessages(String roomId) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `readList(_chatMessagesKey(roomId));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `Future<void> clearChatMessageCaches() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `final ids = _prefs.getStringList(_chatRoomIdsKey) ?? <String>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `for (final id in ids) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `await _prefs.remove(_chatMessagesKey(id));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `await _prefs.remove(_chatRoomIdsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `Future<void> clearAll() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `await _prefs.remove(_profileKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `await _prefs.remove(_notificationsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `await _prefs.remove(_assignmentsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `await _prefs.remove(_dashboardKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `await clearChatMessageCaches();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final cacheServiceProvider = FutureProvider<CacheService>((ref) async {`
Riverpod binding. Name it after the value it exposes.

**L96** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `return CacheService(prefs);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/services/cache_service.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/services/chat_media_cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (chat media cache service) is one concrete piece of that layer.

**L10** `class ChatMediaCacheService {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L11** `ChatMediaCacheService._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `static final ChatMediaCacheService instance = ChatMediaCacheService._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const _folder = 'chat_media_cache';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static const _indexFile = 'index.json';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static const maxEntries = 200;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const maxTotalBytes = 400 * 1024 * 1024; // 400 MB`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Directory? _dir;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Map<String, _MediaCacheEntry>? _index;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `Future<void>? _initFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final _inflight = <String, Future<File?>>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `bool get isSupported => !kIsWeb;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Future<void> _ensureReady() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (!isSupported) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `_initFuture ??= _init();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `await _initFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Future<void> _init() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final root = await getApplicationCacheDirectory();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `_dir = Directory(p.join(root.path, _folder));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (!await _dir!.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `await _dir!.create(recursive: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `await _loadIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `static String keyForUrl(String url) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `final uri = Uri.tryParse(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `if (uri == null) return url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final path = uri.path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final signIdx = path.indexOf('/object/sign/');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `if (signIdx >= 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `return path.substring(signIdx + '/object/sign/'.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final publicIdx = path.indexOf('/object/public/');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (publicIdx >= 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return path.substring(publicIdx + '/object/public/'.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `return path.isEmpty ? url : path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `static String _hashKey(String key) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `sha1.convert(utf8.encode(key)).toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `static String extensionForUrl(String url, {String fallback = 'bin'}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final key = keyForUrl(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final name = key.split('/').last;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final dot = name.lastIndexOf('.');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `if (dot <= 0 || dot == name.length - 1) return fallback;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final ext = name.substring(dot + 1).toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `if (ext.length > 8 || ext.contains('?')) return fallback;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return ext;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `Future<void> _loadIndex() async {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final file = File(p.join(_dir!.path, _indexFile));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `if (!await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `_index = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `final raw = jsonDecode(await file.readAsString()) as Map<String, dynamic>;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `_index = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `for (final e in raw.entries)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `e.key: _MediaCacheEntry.fromJson(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `Map<String, dynamic>.from(e.value as Map),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `_index = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `Future<void> _saveIndex() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `final file = File(p.join(_dir!.path, _indexFile));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final map = {for (final e in _index!.entries) e.key: e.value.toJson()};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `await file.writeAsString(jsonEncode(map));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `File _fileFor(String hash, String extension) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `File(p.join(_dir!.path, '$hash.$extension'));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `Future<File?> getFile(String url) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `if (!isSupported) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final hash = _hashKey(keyForUrl(url));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final entry = _index![hash];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `if (entry == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final file = _fileFor(hash, entry.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (!await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `_index!.remove(hash);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `await _saveIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `entry.lastAccessMs = DateTime.now().millisecondsSinceEpoch;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `await _saveIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `return file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `Future<File?> put(String url, Uint8List bytes, String extension) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `if (!isSupported || bytes.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `final hash = _hashKey(keyForUrl(url));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `final ext = extension.replaceAll('.', '').toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `final file = _fileFor(hash, ext);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final existing = _index![hash];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `if (existing != null && existing.extension != ext) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `final old = _fileFor(hash, existing.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `if (await old.exists()) await old.delete();`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L128** `await file.writeAsBytes(bytes, flush: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `_index![hash] = _MediaCacheEntry(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `extension: ext,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `sizeBytes: bytes.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `lastAccessMs: DateTime.now().millisecondsSinceEpoch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `await _evictIfNeeded();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `await _saveIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `return file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `Future<File?> getOrFetch(String url, {String? preferredExtension}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final key = keyForUrl(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `return _inflight.putIfAbsent(key, () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `final existing = await getFile(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `if (existing != null) return existing;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final response = await http.get(Uri.parse(url));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `if (response.statusCode < 200 || response.statusCode >= 300) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `final bytes = response.bodyBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final ext = preferredExtension ?? extensionForUrl(url, fallback: 'bin');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `return put(url, Uint8List.fromList(bytes), ext);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `_inflight.remove(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `Future<void> prefetch(String url, {String? preferredExtension}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `await getOrFetch(url, preferredExtension: preferredExtension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `Future<void> _evictIfNeeded() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `var entries = _index!.entries.toList()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `..sort((a, b) => a.value.lastAccessMs.compareTo(b.value.lastAccessMs));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `var total = entries.fold<int>(0, (s, e) => s + e.value.sizeBytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `while (entries.length > maxEntries || total > maxTotalBytes) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `if (entries.isEmpty) break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `final oldest = entries.removeAt(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `_index!.remove(oldest.key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `total -= oldest.value.sizeBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `final file = _fileFor(oldest.key, oldest.value.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `if (await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `await file.delete();`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L178** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `Future<int> totalBytes() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `if (!isSupported) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `return _index!.values.fold<int>(0, (s, e) => s + e.sizeBytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `Future<void> clear() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `if (!isSupported) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `if (_dir != null && await _dir!.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `await _dir!.delete(recursive: true);`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L195** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `_dir = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `_index = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `_initFuture = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `_inflight.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `class _MediaCacheEntry {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L205** `_MediaCacheEntry({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `required this.extension,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `required this.sizeBytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `required this.lastAccessMs,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `final String extension;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `final int sizeBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `int lastAccessMs;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `factory _MediaCacheEntry.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `return _MediaCacheEntry(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `extension: json['ext'] as String? ?? 'bin',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `sizeBytes: (json['size'] as num?)?.toInt() ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `lastAccessMs: (json['at'] as num?)?.toInt() ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `Map<String, dynamic> toJson() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `'ext': extension,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `'size': sizeBytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `'at': lastAccessMs,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/services/chat_media_cache_service.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/services/chat_video_thumbnail_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (chat video thumbnail service) is one concrete piece of that layer.

**L12** `class ChatVideoThumbnailService {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `ChatVideoThumbnailService._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static final ChatVideoThumbnailService instance =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `ChatVideoThumbnailService._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const _folder = 'chat_video_thumbs';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `static const maxWidth = 440;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Directory? _dir;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `Future<void>? _initFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final _inflight = <String, Future<File?>>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `bool get isSupported => !kIsWeb;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Future<void> _ensureReady() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (!isSupported) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `_initFuture ??= _init();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `await _initFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Future<void> _init() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final root = await getApplicationCacheDirectory();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `_dir = Directory(p.join(root.path, _folder));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (!await _dir!.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `await _dir!.create(recursive: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `static String _hashKey(String url) => sha1`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `.convert(utf8.encode(ChatMediaCacheService.keyForUrl(url)))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `.toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `File _thumbFile(String hash) => File(p.join(_dir!.path, '$hash.jpg'));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `Future<File?> getThumbnail(String url) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final key = ChatMediaCacheService.keyForUrl(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `return _inflight.putIfAbsent(key, () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (!isSupported) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final hash = _hashKey(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final cached = _thumbFile(hash);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `if (await cached.exists() && await cached.length() > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `return cached;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final videoFile = await ChatMediaCacheService.instance.getOrFetch(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `preferredExtension: ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `fallback: 'mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `Uint8List? bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (videoFile != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `bytes = await VideoThumbnail.thumbnailData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `video: videoFile.path,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `imageFormat: ImageFormat.JPEG,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `maxWidth: maxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `quality: 75,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `timeMs: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `if (bytes == null || bytes.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `bytes = await VideoThumbnail.thumbnailData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `video: url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `imageFormat: ImageFormat.JPEG,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `maxWidth: maxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `quality: 75,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `timeMs: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `if (bytes.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `await cached.writeAsBytes(bytes, flush: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `return cached;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `_inflight.remove(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `Future<void> prefetch(String url) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `await getThumbnail(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `Future<void> clear() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `if (!isSupported) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `if (_dir != null && await _dir!.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `await _dir!.delete(recursive: true);`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L112** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `_dir = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `_initFuture = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `_inflight.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/services/chat_video_thumbnail_service.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/services/voice_cache_service.dart

Caches, voice, chat media, SharedPreferences wrappers. This file (voice cache service) is one concrete piece of that layer.

**L9** `class VoiceCacheService {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `VoiceCacheService._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static final VoiceCacheService instance = VoiceCacheService._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static const _folder = 'voice_cache';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const _indexFile = 'index.json';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static const maxEntries = 100;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static const maxTotalBytes = 100 * 1024 * 1024; // 100 MB`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `Directory? _dir;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Map<String, _VoiceCacheEntry>? _index;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Future<void>? _initFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `bool get isSupported => !kIsWeb;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `Future<void> _ensureReady() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `if (!isSupported) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `_initFuture ??= _init();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `await _initFuture;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `Future<void> _init() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final root = await getApplicationCacheDirectory();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `_dir = Directory(p.join(root.path, _folder));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `if (!await _dir!.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `await _dir!.create(recursive: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `await _loadIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `static String keyForUrl(String url) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `final uri = Uri.tryParse(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `if (uri == null) return url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `final path = uri.path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final signIdx = path.indexOf('/object/sign/');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `if (signIdx >= 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `return path.substring(signIdx + '/object/sign/'.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final publicIdx = path.indexOf('/object/public/');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `if (publicIdx >= 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `return path.substring(publicIdx + '/object/public/'.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return path.isEmpty ? url : path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `static String _hashKey(String key) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `sha1.convert(utf8.encode(key)).toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `Future<void> _loadIndex() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final file = File(p.join(_dir!.path, _indexFile));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `if (!await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `_index = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final raw = jsonDecode(await file.readAsString()) as Map<String, dynamic>;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `_index = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `for (final e in raw.entries)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `e.key: _VoiceCacheEntry.fromJson(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `Map<String, dynamic>.from(e.value as Map),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `_index = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `Future<void> _saveIndex() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `final file = File(p.join(_dir!.path, _indexFile));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `final map = {for (final e in _index!.entries) e.key: e.value.toJson()};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `await file.writeAsString(jsonEncode(map));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `File _fileFor(String hash, String extension) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `File(p.join(_dir!.path, '$hash.$extension'));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `Future<Uint8List?> readBytes(String url) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final file = await getFile(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `if (file == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `return Uint8List.fromList(await file.readAsBytes());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `Future<File?> getFile(String url) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `if (!isSupported) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final key = keyForUrl(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `final hash = _hashKey(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final entry = _index![hash];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `if (entry == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `final file = _fileFor(hash, entry.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `if (!await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `_index!.remove(hash);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `await _saveIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `entry.lastAccessMs = DateTime.now().millisecondsSinceEpoch;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `await _saveIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `return file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Future<File?> put(String url, Uint8List bytes, String extension) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (!isSupported) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `if (bytes.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `final key = keyForUrl(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `final hash = _hashKey(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `final ext = extension.replaceAll('.', '').toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `final file = _fileFor(hash, ext);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final existing = _index![hash];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `if (existing != null && existing.extension != ext) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `final old = _fileFor(hash, existing.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `if (await old.exists()) await old.delete();`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L128** `await file.writeAsBytes(bytes, flush: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `_index![hash] = _VoiceCacheEntry(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `extension: ext,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `sizeBytes: bytes.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `lastAccessMs: DateTime.now().millisecondsSinceEpoch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `await _evictIfNeeded();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `await _saveIndex();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `return file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `Future<void> _evictIfNeeded() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `var entries = _index!.entries.toList()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `..sort((a, b) => a.value.lastAccessMs.compareTo(b.value.lastAccessMs));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `var total = entries.fold<int>(0, (s, e) => s + e.value.sizeBytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `while (entries.length > maxEntries || total > maxTotalBytes) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `if (entries.isEmpty) break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `final oldest = entries.removeAt(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `_index!.remove(oldest.key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `total -= oldest.value.sizeBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final file = _fileFor(oldest.key, oldest.value.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `if (await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `await file.delete();`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L154** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `Future<int> totalBytes() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `if (!isSupported) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `return _index!.values.fold<int>(0, (s, e) => s + e.sizeBytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `Future<int> entryCount() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `if (!isSupported) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `return _index!.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `Future<void> clear() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `if (!isSupported) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `await _ensureReady();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `if (_dir != null && await _dir!.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `await _dir!.delete(recursive: true);`
Removes matching rows. CASCADE on FKs may delete children — know the blast radius.

**L177** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `_dir = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `_index = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `_initFuture = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `class _VoiceCacheEntry {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L186** `_VoiceCacheEntry({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `required this.extension,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `required this.sizeBytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `required this.lastAccessMs,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `final String extension;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `final int sizeBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `int lastAccessMs;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `factory _VoiceCacheEntry.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `return _VoiceCacheEntry(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `extension: json['ext'] as String? ?? 'm4a',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `sizeBytes: (json['size'] as num?)?.toInt() ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `lastAccessMs: (json['at'] as num?)?.toInt() ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `Map<String, dynamic> toJson() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `'ext': extension,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `'size': sizeBytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `'at': lastAccessMs,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/services/voice_cache_service.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/app_colors.dart

Four visual styles × light/dark Material 3 themes. This file (app colors) is one concrete piece of that layer.

**L5** `class AppColors {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `static const darkChocolate = Color(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `0xFF2D2422,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `); // Slightly darker for Linear feel`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `static const coffeeBrown = Color(0xFF4A3731);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `static const warmBeige = Color(0xFFD7CCC8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `static const cream = Color(0xFFFBFBF9); // Cleaner cream for Notion feel`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static const sand = Color(0xFFEFEBE9);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static const mutedGold = Color(0xFFC5A059);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const warmAmber = Color(0xFFD4813E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static const softOlive = Color(0xFF8D917A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static const successGreen = Color(0xFF6B8E6B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const errorRed = Color(0xFFAA5B5B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `static const surfaceDark = Color(0xFF1E1E1E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static const surfaceLight = Color(0xFFFFFFFF);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `static const darkBackground = Color(0xFF0F0F0F);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `static const darkSurface = Color(0xFF1A1A1A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `static const darkSurfaceElevated = Color(0xFF2A2A2A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `static const darkSurfaceHighest = Color(0xFF343434);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `static const darkBorder = Color(0xFF525252);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `static const darkTextPrimary = Color(0xFFF5F5F5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `static const darkTextSecondary = Color(0xFFD0D0D0); // ~40% brighter secondary`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `static const darkTextTertiary = Color(0xFFB8B8B8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `static const darkAccent = Color(0xFFC5A059); // mutedGold for highlights`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `static const mathAccent = coffeeBrown;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `static const englishAccent = mutedGold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `static Color accent(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (tokens != null) return tokens.accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `? mutedGold`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `: coffeeBrown;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `static Color resolveAccent(BuildContext context, Color color) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `if (tokens?.isMonochromeWorkspace ?? false) return ink(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `if ((tokens?.isNaturePaper ?? false) &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `Theme.of(context).brightness == Brightness.dark &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `color.computeLuminance() < 0.2) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `return NaturePaperColors.moonBeige;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `return color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `static Color onAccent(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final mono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `if (mono) return isDark ? Colors.black : Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `return isDark ? const Color(0xFF1A1408) : Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `static Color success(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `AppStyleTokens.maybeOf(context)?.success ?? successGreen;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `static Color warning(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `AppStyleTokens.maybeOf(context)?.warning ?? warmAmber;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `static Color danger(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `AppStyleTokens.maybeOf(context)?.danger ?? errorRed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `static Color goldAccent(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `AppStyleTokens.maybeOf(context)?.gold ?? mutedGold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `static Color ink(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `Theme.of(context).colorScheme.onSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `static Color inkAlpha(BuildContext context, double alpha) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `final base =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `AppStyleTokens.maybeOf(context)?.inkBase ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `(isDark ? darkTextPrimary : darkChocolate);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `if (!isDark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `return base.withValues(alpha: alpha);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final darkAlpha = switch (alpha) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `<= 0.05 => 0.16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `<= 0.08 => 0.22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `<= 0.12 => 0.28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `<= 0.2 => 0.36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `<= 0.25 => 0.80,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `<= 0.35 => 0.84,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `<= 0.45 => 0.88,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `<= 0.55 => 0.90,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `<= 0.7 => 0.92,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `_ => alpha.clamp(0.80, 1.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `return base.withValues(alpha: darkAlpha);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `static Color controlSurface(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (tokens != null) return tokens.controlSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `return isDark ? darkSurfaceHighest : Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `static Color softWash(BuildContext context, {double lightAlpha = 0.5}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (isDark) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return tokens?.softWashColor ?? Colors.white.withValues(alpha: 0.10);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `return (tokens?.softWashColor ?? sand).withValues(alpha: lightAlpha);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `static Color softSurface(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `if (tokens != null) return tokens.softSurface;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `return isDark ? darkSurfaceHighest : sand;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `static Color sheetSurface(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `if (tokens != null) return tokens.sheetSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `return isDark ? darkSurfaceElevated : Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/theme/app_colors.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/app_style.dart

Four visual styles × light/dark Material 3 themes. This file (app style) is one concrete piece of that layer.

**L1** `enum AppStyle {`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L2** `modern,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L3** `naturePaper,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `softNeumorphism,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `monochromeWorkspace;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `String get code => name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `static AppStyle? tryParse(String? code) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `if (code == null || code.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `for (final v in AppStyle.values) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `if (v.code == code) return v;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/app_style.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/app_style_tokens.dart

Four visual styles × light/dark Material 3 themes. This file (app style tokens) is one concrete piece of that layer.

**L8** `class AppStyleTokens extends ThemeExtension<AppStyleTokens> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const AppStyleTokens({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.style,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.inkBase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.success,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `required this.warning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.danger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.gold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `required this.controlSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required this.softSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `required this.softWashColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `required this.sheetSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `required this.cardSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `required this.cardBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `required this.cardRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `required this.navSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `required this.navGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `required this.headerGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `this.chartPositive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `this.chartWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `this.chartDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final AppStyle style;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final Color accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final Color inkBase;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final Color success;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final Color warning;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final Color danger;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `final Color gold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final Color controlSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final Color softSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final Color softWashColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final Color sheetSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final Color cardSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final Color cardBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final double cardRadius;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final Color navSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final Color navGlow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final Color headerGlow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final LinearGradient? chartPositive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final LinearGradient? chartWarning;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final LinearGradient? chartDanger;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `bool get isNaturePaper => style == AppStyle.naturePaper;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `bool get isSoftNeumorphism => style == AppStyle.softNeumorphism;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `bool get isMonochromeWorkspace => style == AppStyle.monochromeWorkspace;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `bool get usesAmbientBackdrop => isNaturePaper || isSoftNeumorphism;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `bool get prefersChartLabelsInside => isNaturePaper || isSoftNeumorphism;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `static AppStyleTokens? maybeOf(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `Theme.of(context).extension<AppStyleTokens>();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `static const modernLight = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `style: AppStyle.modern,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `accent: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `inkBase: AppColors.darkChocolate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `success: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `warning: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `danger: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `gold: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `controlSurface: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `softSurface: AppColors.sand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `softWashColor: AppColors.sand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `sheetSurface: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `cardSurface: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `cardBorder: Color(0x0F2D2422), // darkChocolate @ 6%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `cardRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `navSurface: Color(0xE0FBFBF9), // cream @ 88%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `navGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `headerGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `static const modernDark = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `style: AppStyle.modern,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `accent: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `inkBase: AppColors.darkTextPrimary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `success: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `warning: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `danger: Color(0xFFE57373),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `gold: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `controlSurface: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `softSurface: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `softWashColor: Color(0x1AFFFFFF), // white @ 10%, used as-is in dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `sheetSurface: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `cardSurface: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `cardBorder: AppColors.darkBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `cardRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `navSurface: Color(0xF5343434), // darkSurfaceHighest @ 96%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `navGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `headerGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `static const natureLight = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `style: AppStyle.naturePaper,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `accent: NaturePaperColors.forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `inkBase: NaturePaperColors.dayInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `success: NaturePaperColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `warning: NaturePaperColors.softOrange,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `danger: NaturePaperColors.dayDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `gold: NaturePaperColors.softOrange,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `controlSurface: NaturePaperColors.dayCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `softSurface: NaturePaperColors.paperBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `softWashColor: NaturePaperColors.paperBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `sheetSurface: NaturePaperColors.dayCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `cardSurface: NaturePaperColors.dayCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `cardBorder: Color(0x2E8A6A4A), // warmBrown @ 18%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `cardRadius: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `navSurface: Color(0xE6FFFCF4), // dayCard @ 90%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `navGlow: NaturePaperColors.sunGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `headerGlow: NaturePaperColors.sunGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `chartPositive: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `colors: [NaturePaperColors.forest, Color(0xFFC9C36A)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `chartWarning: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `colors: [NaturePaperColors.warmBrown, NaturePaperColors.softOrange],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `chartDanger: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `colors: [Color(0xFF7E4437), NaturePaperColors.dayDanger],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `static const natureDark = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `style: AppStyle.naturePaper,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `accent: NaturePaperColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `inkBase: NaturePaperColors.nightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `success: NaturePaperColors.sageNight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `warning: NaturePaperColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `danger: NaturePaperColors.dustyRose,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `gold: NaturePaperColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `controlSurface: NaturePaperColors.nightCardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `softSurface: NaturePaperColors.nightCardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `softWashColor: Color(0x14E6DCC8), // moonBeige @ 8%, used as-is in dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `sheetSurface: NaturePaperColors.nightCardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `cardSurface: NaturePaperColors.nightCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `cardBorder: NaturePaperColors.nightBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `cardRadius: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `navSurface: Color(0xF0283630), // nightCardHigh @ 94%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `navGlow: NaturePaperColors.moonGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `headerGlow: NaturePaperColors.moonGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `chartPositive: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `colors: [Color(0xFF4C6A50), NaturePaperColors.sageNight],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `chartWarning: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `colors: [Color(0xFF8A6434), NaturePaperColors.amber],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `chartDanger: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `colors: [Color(0xFF6E4444), NaturePaperColors.dustyRose],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `static const softNeuLight = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `style: AppStyle.softNeumorphism,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `accent: SoftNeumorphismColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `inkBase: SoftNeumorphismColors.morningInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `success: SoftNeumorphismColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `warning: SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `danger: SoftNeumorphismColors.morningDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `gold: SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `controlSurface: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `softSurface: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `softWashColor: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `sheetSurface: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `cardSurface: SoftNeumorphismColors.morningSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `cardBorder: Color(0x00000000),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `cardRadius: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `navSurface: SoftNeumorphismColors.morningSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `navGlow: SoftNeumorphismColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `headerGlow: SoftNeumorphismColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `chartPositive: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `colors: [SoftNeumorphismColors.sageDeep, SoftNeumorphismColors.sage],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `chartWarning: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `colors: [Color(0xFFB8844A), SoftNeumorphismColors.amber],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `chartDanger: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `colors: [Color(0xFF8A5050), SoftNeumorphismColors.morningDanger],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `static const softNeuDark = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `style: AppStyle.softNeumorphism,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `accent: SoftNeumorphismColors.emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `inkBase: SoftNeumorphismColors.nightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `success: SoftNeumorphismColors.emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `warning: SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `danger: SoftNeumorphismColors.nightDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `gold: SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `controlSurface: SoftNeumorphismColors.nightSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `softSurface: SoftNeumorphismColors.nightInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `softWashColor: Color(0x14F2F0EB), // nightInk @ 8%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `sheetSurface: SoftNeumorphismColors.nightSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `cardSurface: SoftNeumorphismColors.nightSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `cardBorder: Color(0x00000000),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `cardRadius: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `navSurface: SoftNeumorphismColors.nightSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `navGlow: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `headerGlow: SoftNeumorphismColors.emeraldGlow,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `chartPositive: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `colors: [Color(0xFF3D6E55), SoftNeumorphismColors.emeraldGlow],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `chartWarning: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `colors: [Color(0xFF8A6434), SoftNeumorphismColors.amber],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `chartDanger: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `begin: Alignment.bottomCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `end: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `colors: [Color(0xFF6E4444), SoftNeumorphismColors.nightDanger],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `static const monoLight = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `style: AppStyle.monochromeWorkspace,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `accent: MonochromeWorkspaceColors.lightAccent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `inkBase: MonochromeWorkspaceColors.lightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `success: MonochromeWorkspaceColors.lightSuccess,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `warning: MonochromeWorkspaceColors.lightWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `danger: MonochromeWorkspaceColors.lightDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `gold: MonochromeWorkspaceColors.lightGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `controlSurface: MonochromeWorkspaceColors.lightControl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `softSurface: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `softWashColor: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `sheetSurface: MonochromeWorkspaceColors.lightCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `cardSurface: MonochromeWorkspaceColors.lightCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `cardBorder: MonochromeWorkspaceColors.lightBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `cardRadius: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `navSurface: Color(0xF5FFFFFF), // white @ 96%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `navGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `headerGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `static const monoDark = AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `style: AppStyle.monochromeWorkspace,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `accent: MonochromeWorkspaceColors.darkAccent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `inkBase: MonochromeWorkspaceColors.darkInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `success: MonochromeWorkspaceColors.darkSuccess,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `warning: MonochromeWorkspaceColors.darkWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `danger: MonochromeWorkspaceColors.darkDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `gold: MonochromeWorkspaceColors.darkGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `controlSurface: MonochromeWorkspaceColors.darkControl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `softSurface: MonochromeWorkspaceColors.darkWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `softWashColor: Color(0x14FFFFFF), // white @ 8%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `sheetSurface: MonochromeWorkspaceColors.darkControl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `cardSurface: MonochromeWorkspaceColors.darkCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `cardBorder: MonochromeWorkspaceColors.darkBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `cardRadius: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `navSurface: Color(0xF51A1A1A), // darkCard @ 96%`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `navGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `headerGlow: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `AppStyleTokens copyWith({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `AppStyle? style,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `Color? accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `Color? inkBase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `Color? success,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `Color? warning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `Color? danger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `Color? gold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `Color? controlSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `Color? softSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `Color? softWashColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `Color? sheetSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `Color? cardSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `Color? cardBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `double? cardRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `Color? navSurface,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `Color? navGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `Color? headerGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `LinearGradient? chartPositive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `LinearGradient? chartWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `LinearGradient? chartDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `return AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `style: style ?? this.style,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `accent: accent ?? this.accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `inkBase: inkBase ?? this.inkBase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `success: success ?? this.success,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `warning: warning ?? this.warning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `danger: danger ?? this.danger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `gold: gold ?? this.gold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `controlSurface: controlSurface ?? this.controlSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `softSurface: softSurface ?? this.softSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `softWashColor: softWashColor ?? this.softWashColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `sheetSurface: sheetSurface ?? this.sheetSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `cardSurface: cardSurface ?? this.cardSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `cardBorder: cardBorder ?? this.cardBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `cardRadius: cardRadius ?? this.cardRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `navSurface: navSurface ?? this.navSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `navGlow: navGlow ?? this.navGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `headerGlow: headerGlow ?? this.headerGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `chartPositive: chartPositive ?? this.chartPositive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `chartWarning: chartWarning ?? this.chartWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `chartDanger: chartDanger ?? this.chartDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `AppStyleTokens lerp(ThemeExtension<AppStyleTokens>? other, double t) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `if (other is! AppStyleTokens) return this;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `return AppStyleTokens(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `style: t < 0.5 ? style : other.style,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `accent: Color.lerp(accent, other.accent, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `inkBase: Color.lerp(inkBase, other.inkBase, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `success: Color.lerp(success, other.success, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `warning: Color.lerp(warning, other.warning, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `danger: Color.lerp(danger, other.danger, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `gold: Color.lerp(gold, other.gold, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `controlSurface: Color.lerp(controlSurface, other.controlSurface, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `softSurface: Color.lerp(softSurface, other.softSurface, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `softWashColor: Color.lerp(softWashColor, other.softWashColor, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `sheetSurface: Color.lerp(sheetSurface, other.sheetSurface, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `cardSurface: Color.lerp(cardSurface, other.cardSurface, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `cardBorder: Color.lerp(cardBorder, other.cardBorder, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `cardRadius: cardRadius + (other.cardRadius - cardRadius) * t,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `navSurface: Color.lerp(navSurface, other.navSurface, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `navGlow: Color.lerp(navGlow, other.navGlow, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `headerGlow: Color.lerp(headerGlow, other.headerGlow, t)!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `chartPositive: t < 0.5 ? chartPositive : other.chartPositive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `chartWarning: t < 0.5 ? chartWarning : other.chartWarning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `chartDanger: t < 0.5 ? chartDanger : other.chartDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/theme/app_style_tokens.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/app_text_theme.dart

Four visual styles × light/dark Material 3 themes. This file (app text theme) is one concrete piece of that layer.

**L4** `TextTheme buildJakartaTextTheme({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required TextTheme base,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required Color primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required Color secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required Color tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final jakarta = GoogleFonts.plusJakartaSansTextTheme(base);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `return jakarta.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `displayLarge: jakarta.displayLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `letterSpacing: -1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `displayMedium: jakarta.displayMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `letterSpacing: -0.8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `displaySmall: jakarta.displaySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `letterSpacing: -0.5,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `headlineLarge: jakarta.headlineLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `headlineMedium: jakarta.headlineMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `headlineSmall: jakarta.headlineSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `titleLarge: jakarta.titleLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `titleMedium: jakarta.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `titleSmall: jakarta.titleSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `bodyLarge: jakarta.bodyLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `color: primary.withValues(alpha: 0.92),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `bodyMedium: jakarta.bodyMedium?.copyWith(color: secondary, fontSize: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `bodySmall: jakarta.bodySmall?.copyWith(color: tertiary, fontSize: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `labelLarge: jakarta.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `color: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `labelMedium: jakarta.labelMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `color: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `labelSmall: jakarta.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `color: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/app_text_theme.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/app_theme.dart

Four visual styles × light/dark Material 3 themes. This file (app theme) is one concrete piece of that layer.

**L10** `class AppTheme {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L12** `static ThemeData light([AppStyle style = AppStyle.modern]) => switch (style) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `AppStyle.modern => _modernLight(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `AppStyle.naturePaper => NaturePaperTheme.light(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `AppStyle.softNeumorphism => SoftNeumorphismTheme.light(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `AppStyle.monochromeWorkspace => MonochromeWorkspaceTheme.light(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `static ThemeData dark([AppStyle style = AppStyle.modern]) => switch (style) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `AppStyle.modern => _modernDark(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `AppStyle.naturePaper => NaturePaperTheme.dark(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `AppStyle.softNeumorphism => SoftNeumorphismTheme.dark(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `AppStyle.monochromeWorkspace => MonochromeWorkspaceTheme.dark(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `static TextTheme _jakartaTextTheme({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `required TextTheme base,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `required Color primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `required Color secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `required Color tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `}) => buildJakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `base: base,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `primary: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `secondary: secondary,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `tertiary: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `static ThemeData _modernLight() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `brightness: Brightness.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `colorScheme: ColorScheme.fromSeed(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `seedColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `primary: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `secondary: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `surface: AppColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `onSurface: AppColors.darkChocolate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `surfaceContainerLow: AppColors.sand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `surfaceContainer: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final textTheme = _jakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `primary: AppColors.darkChocolate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `secondary: AppColors.darkChocolate.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `tertiary: AppColors.darkChocolate.withValues(alpha: 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `scaffoldBackgroundColor: AppColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `extensions: const [AppStyleTokens.modernLight],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `foregroundColor: AppColors.darkChocolate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `color: AppColors.darkChocolate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `side: BorderSide(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `color: AppColors.darkChocolate.withValues(alpha: 0.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `fillColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `borderSide: BorderSide(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `color: AppColors.darkChocolate.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `borderSide: BorderSide(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `color: AppColors.darkChocolate.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `borderSide: const BorderSide(color: AppColors.coffeeBrown, width: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `vertical: 16,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `labelStyle: const TextStyle(color: AppColors.coffeeBrown),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `style: ElevatedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `backgroundColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `foregroundColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `foregroundColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `side: BorderSide(color: AppColors.coffeeBrown.withValues(alpha: 0.2)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `foregroundColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `backgroundColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `foregroundColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `elevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `focusElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `hoverElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `highlightElevation: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `shape: const StadiumBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `backgroundColor: Colors.white.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `indicatorColor: AppColors.warmBeige.withValues(alpha: 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `? AppColors.coffeeBrown`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `: AppColors.darkChocolate.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `return IconThemeData(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `color: states.contains(WidgetState.selected)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `? AppColors.coffeeBrown`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `: AppColors.darkChocolate.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `static ThemeData _modernDark() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `const onSurface = AppColors.darkTextPrimary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `const onSurfaceVariant = AppColors.darkTextSecondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `const outline = AppColors.darkBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final colorScheme = ColorScheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `primary: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `onPrimary: const Color(0xFF1A1408),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `primaryContainer: const Color(0xFF3D3420),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `onPrimaryContainer: AppColors.warmBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `secondary: AppColors.warmBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `onSecondary: const Color(0xFF1A1512),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `secondaryContainer: const Color(0xFF3A322E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `onSecondaryContainer: AppColors.warmBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `tertiary: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `onTertiary: const Color(0xFF1A1208),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `error: const Color(0xFFE57373),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `onError: const Color(0xFF1A0A0A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `surface: AppColors.darkSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `onSurface: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `onSurfaceVariant: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `surfaceContainerLowest: AppColors.darkBackground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `surfaceContainerLow: const Color(0xFF151515),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `surfaceContainer: AppColors.darkSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `surfaceContainerHigh: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `surfaceContainerHighest: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `outline: outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `outlineVariant: const Color(0xFF3D3D3D),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `shadow: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `scrim: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `inverseSurface: const Color(0xFFE8E8E8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `onInverseSurface: const Color(0xFF1A1A1A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `inversePrimary: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `colorScheme: colorScheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `final textTheme = _jakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `primary: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `secondary: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `tertiary: AppColors.darkTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `scaffoldBackgroundColor: AppColors.darkBackground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `canvasColor: AppColors.darkBackground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `extensions: const [AppStyleTokens.modernDark],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `dividerColor: outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `disabledColor: AppColors.darkTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `iconTheme: const IconThemeData(color: onSurfaceVariant),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `primaryIconTheme: const IconThemeData(color: AppColors.mutedGold),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `dividerTheme: const DividerThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `color: outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `thickness: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `space: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `listTileTheme: ListTileThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `iconColor: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `textColor: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `subtitleTextStyle: textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `color: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `fontSize: 13,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `backgroundColor: AppColors.mutedGold.withValues(alpha: 0.16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `disabledColor: AppColors.darkSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `selectedColor: AppColors.mutedGold.withValues(alpha: 0.32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `secondarySelectedColor: AppColors.mutedGold.withValues(alpha: 0.32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `color: AppColors.warmBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `secondaryLabelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `color: AppColors.warmBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `side: BorderSide(color: AppColors.mutedGold.withValues(alpha: 0.45)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `tabBarTheme: TabBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `labelColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `unselectedLabelColor: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `indicatorColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `dividerColor: outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `labelStyle: textTheme.labelLarge?.copyWith(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `unselectedLabelStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `backgroundColor: AppColors.darkBackground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `foregroundColor: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `iconTheme: const IconThemeData(color: onSurface),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `actionsIconTheme: const IconThemeData(color: onSurfaceVariant),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `color: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `color: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `shadowColor: Colors.black.withValues(alpha: 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `side: const BorderSide(color: outline, width: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `backgroundColor: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `color: onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `contentTextStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `color: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `backgroundColor: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `surfaceTintColor: Colors.transparent,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `modalBackgroundColor: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `backgroundColor: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: onSurface),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `actionTextColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `popupMenuTheme: PopupMenuThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `color: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `textStyle: textTheme.bodyMedium?.copyWith(color: onSurface),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `dropdownMenuTheme: DropdownMenuThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `textStyle: textTheme.bodyMedium?.copyWith(color: onSurface),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `fillColor: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `fillColor: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `hintStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `color: AppColors.darkTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `labelStyle: textTheme.bodyMedium?.copyWith(color: onSurfaceVariant),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `floatingLabelStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `prefixIconColor: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `suffixIconColor: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `border: const OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `borderRadius: BorderRadius.all(Radius.circular(12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `borderSide: BorderSide(color: outline),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `enabledBorder: const OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `borderRadius: BorderRadius.all(Radius.circular(12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `borderSide: BorderSide(color: outline),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `borderSide: const BorderSide(color: AppColors.mutedGold, width: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `style: ElevatedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `backgroundColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `foregroundColor: const Color(0xFF1A1408),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `disabledBackgroundColor: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `disabledForegroundColor: AppColors.darkTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `foregroundColor: AppColors.mutedGold,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `side: BorderSide(color: AppColors.mutedGold.withValues(alpha: 0.65)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `foregroundColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `backgroundColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `foregroundColor: const Color(0xFF1A1408),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `elevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `focusElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `hoverElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `highlightElevation: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `shape: const StadiumBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `backgroundColor: AppColors.darkSurfaceElevated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `indicatorColor: AppColors.mutedGold.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `color: selected ? AppColors.mutedGold : onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `return IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `color: selected ? AppColors.mutedGold : onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `backgroundColor: AppColors.darkSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `indicatorColor: AppColors.mutedGold.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `selectedIconTheme: const IconThemeData(color: AppColors.mutedGold),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `unselectedIconTheme: const IconThemeData(color: onSurfaceVariant),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `selectedLabelTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `unselectedLabelTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `color: onSurfaceVariant,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `switchTheme: SwitchThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `thumbColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `if (states.contains(WidgetState.selected)) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `return AppColors.mutedGold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `return onSurfaceVariant;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `trackColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `if (states.contains(WidgetState.selected)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `return AppColors.mutedGold.withValues(alpha: 0.42);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `return AppColors.darkSurfaceHighest;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `progressIndicatorTheme: const ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `circularTrackColor: AppColors.darkSurfaceHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/app_theme.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/monochrome_workspace_colors.dart

Four visual styles × light/dark Material 3 themes. This file (monochrome workspace colors) is one concrete piece of that layer.

**L3** `class MonochromeWorkspaceColors {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `static const lightBg = Color(0xFFFAFAFA);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `static const lightCard = Color(0xFFFFFFFF);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `static const lightControl = Color(0xFFFFFFFF);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `static const lightWash = Color(0xFFF2F2F2);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static const lightInk = Color(0xFF111111);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static const lightTextSecondary = Color(0xFF6B6B6B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const lightTextTertiary = Color(0xFF8E8E8E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static const lightBorder = Color(0xFFE6E6E6);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `static const lightAccent = Color(0xFF111111);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static const lightSuccess = Color(0xFF1A1A1A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `static const lightWarning = Color(0xFF6B6B6B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `static const lightDanger = Color(0xFF2E2E2E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `static const lightGold = Color(0xFF4A4A4A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `static const darkBg = Color(0xFF111111);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `static const darkCard = Color(0xFF1A1A1A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `static const darkControl = Color(0xFF222222);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `static const darkWash = Color(0xFF181818);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `static const darkInk = Color(0xFFF5F5F5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `static const darkTextSecondary = Color(0xFFA3A3A3);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `static const darkTextTertiary = Color(0xFF7A7A7A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `static const darkBorder = Color(0xFF2E2E2E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `static const darkAccent = Color(0xFFF5F5F5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `static const darkSuccess = Color(0xFFE8E8E8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `static const darkWarning = Color(0xFF9A9A9A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `static const darkDanger = Color(0xFFD0D0D0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `static const darkGold = Color(0xFFC0C0C0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/monochrome_workspace_colors.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/monochrome_workspace_theme.dart

Four visual styles × light/dark Material 3 themes. This file (monochrome workspace theme) is one concrete piece of that layer.

**L6** `class MonochromeWorkspaceTheme {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `static const _fadeTransitions = PageTransitionsTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `builders: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `TargetPlatform.android: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `TargetPlatform.iOS: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `TargetPlatform.macOS: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `TargetPlatform.linux: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `TargetPlatform.windows: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `TargetPlatform.fuchsia: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const _radius = 4.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `static const _controlRadius = 4.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static ThemeData light() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `const ink = MonochromeWorkspaceColors.lightInk;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `const secondary = MonochromeWorkspaceColors.lightTextSecondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `const tertiary = MonochromeWorkspaceColors.lightTextTertiary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `const accent = MonochromeWorkspaceColors.lightAccent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `const bg = MonochromeWorkspaceColors.lightBg;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `const card = MonochromeWorkspaceColors.lightCard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `const border = MonochromeWorkspaceColors.lightBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `brightness: Brightness.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `colorScheme: ColorScheme.fromSeed(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `seedColor: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `primary: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `onPrimary: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `secondary: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `onSecondary: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `tertiary: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `error: MonochromeWorkspaceColors.lightDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `surface: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `onSurface: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `surfaceContainerLowest: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `surfaceContainerLow: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `surfaceContainer: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `surfaceContainerHigh: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `surfaceContainerHighest: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `outline: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final textTheme = _editorialTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `primary: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `secondary: secondary,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `tertiary: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `scaffoldBackgroundColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `canvasColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `pageTransitionsTheme: _fadeTransitions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `dividerColor: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `extensions: const [AppStyleTokens.monoLight],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `dividerTheme: const DividerThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `color: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `thickness: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `space: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `titleTextStyle: textTheme.titleLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `fontSize: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `letterSpacing: -0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `color: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `margin: const EdgeInsets.symmetric(vertical: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: secondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `modalBackgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `borderRadius: BorderRadius.vertical(top: Radius.circular(8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `backgroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: Colors.white),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `actionTextColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `behavior: SnackBarBehavior.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `fillColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `hintStyle: textTheme.bodyMedium?.copyWith(color: tertiary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `labelStyle: const TextStyle(color: secondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `borderSide: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `borderSide: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `borderSide: const BorderSide(color: accent, width: 1.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `style: _monoButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `background: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `foreground: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `filledButtonTheme: FilledButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `style: _monoButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `background: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `foreground: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `side: const BorderSide(color: border, width: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `backgroundColor: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `foregroundColor: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `focusElevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `hoverElevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `highlightElevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `backgroundColor: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `selectedColor: ink.withValues(alpha: 0.10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `switchTheme: SwitchThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `thumbColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `if (states.contains(WidgetState.selected)) return Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `return card;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `trackColor: WidgetStateProperty.resolveWith((states) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `if (states.contains(WidgetState.selected)) return accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `return MonochromeWorkspaceColors.lightWash;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `trackOutlineColor: const WidgetStatePropertyAll(border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `checkboxTheme: CheckboxThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `fillColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `if (states.contains(WidgetState.selected)) return accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `return Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `checkColor: const WidgetStatePropertyAll(Colors.white),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `side: const BorderSide(color: border, width: 1.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(3)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `progressIndicatorTheme: ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `circularTrackColor: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `linearTrackColor: MonochromeWorkspaceColors.lightWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `indicatorColor: ink.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `color: selected ? ink : tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `return IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `size: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `color: states.contains(WidgetState.selected) ? ink : tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `indicatorColor: ink.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `selectedIconTheme: const IconThemeData(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `unselectedIconTheme: const IconThemeData(color: tertiary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `listTileTheme: ListTileThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `iconColor: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `textColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `subtitleTextStyle: textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `color: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `static ThemeData dark() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `const ink = MonochromeWorkspaceColors.darkInk;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `const secondary = MonochromeWorkspaceColors.darkTextSecondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `const tertiary = MonochromeWorkspaceColors.darkTextTertiary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `const accent = MonochromeWorkspaceColors.darkAccent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `const bg = MonochromeWorkspaceColors.darkBg;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `const card = MonochromeWorkspaceColors.darkCard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `const control = MonochromeWorkspaceColors.darkControl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `const border = MonochromeWorkspaceColors.darkBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `final colorScheme = ColorScheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `primary: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `onPrimary: MonochromeWorkspaceColors.darkBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `primaryContainer: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `onPrimaryContainer: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `secondary: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `onSecondary: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `secondaryContainer: control,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `onSecondaryContainer: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `tertiary: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `onTertiary: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `error: MonochromeWorkspaceColors.darkDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `onError: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `surface: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `onSurface: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `onSurfaceVariant: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `surfaceContainerLowest: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `surfaceContainerLow: MonochromeWorkspaceColors.darkWash,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `surfaceContainer: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `surfaceContainerHigh: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `surfaceContainerHighest: const Color(0xFF2A2A2A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `outline: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `outlineVariant: const Color(0xFF3A3A3A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `shadow: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `scrim: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `inverseSurface: MonochromeWorkspaceColors.lightCard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `onInverseSurface: MonochromeWorkspaceColors.lightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `inversePrimary: MonochromeWorkspaceColors.lightAccent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `colorScheme: colorScheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `final textTheme = _editorialTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `primary: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `secondary: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `tertiary: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `scaffoldBackgroundColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `canvasColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `pageTransitionsTheme: _fadeTransitions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `dividerColor: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `disabledColor: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `iconTheme: const IconThemeData(color: secondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `primaryIconTheme: const IconThemeData(color: accent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `extensions: const [AppStyleTokens.monoDark],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `dividerTheme: const DividerThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `color: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `thickness: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `space: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `listTileTheme: ListTileThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `iconColor: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `textColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `subtitleTextStyle: textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `color: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `iconTheme: const IconThemeData(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `actionsIconTheme: const IconThemeData(color: secondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `titleTextStyle: textTheme.titleLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `fontSize: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `letterSpacing: -0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `cardTheme: CardThemeData(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `color: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `margin: const EdgeInsets.symmetric(vertical: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `backgroundColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: secondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `backgroundColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `modalBackgroundColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `borderRadius: BorderRadius.vertical(top: Radius.circular(8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `backgroundColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `actionTextColor: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `behavior: SnackBarBehavior.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `fillColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `hintStyle: textTheme.bodyMedium?.copyWith(color: tertiary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `labelStyle: const TextStyle(color: secondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `borderSide: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `borderSide: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `borderSide: const BorderSide(color: accent, width: 1.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `style: _monoButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `background: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `foreground: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `textStyle: textTheme.labelLarge,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `filledButtonTheme: FilledButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `style: _monoButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `background: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `foreground: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `side: const BorderSide(color: border, width: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `backgroundColor: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `foregroundColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `focusElevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `hoverElevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `highlightElevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `backgroundColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `selectedColor: accent.withValues(alpha: 0.16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `side: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `switchTheme: SwitchThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `thumbColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `if (states.contains(WidgetState.selected)) return bg;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `return secondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `trackColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `if (states.contains(WidgetState.selected)) return accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `return control;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `trackOutlineColor: const WidgetStatePropertyAll(border),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `checkboxTheme: CheckboxThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `fillColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `if (states.contains(WidgetState.selected)) return accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `return Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `checkColor: const WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `MonochromeWorkspaceColors.darkBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `side: const BorderSide(color: border, width: 1.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(3)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `progressIndicatorTheme: ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `circularTrackColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `linearTrackColor: control,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `indicatorColor: accent.withValues(alpha: 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `color: selected ? ink : tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `return IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `size: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `color: states.contains(WidgetState.selected) ? ink : tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `indicatorColor: accent.withValues(alpha: 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `selectedIconTheme: const IconThemeData(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `unselectedIconTheme: const IconThemeData(color: tertiary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `static TextTheme _editorialTextTheme({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `required TextTheme base,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `required Color primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `required Color secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `required Color tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `final jakarta = buildJakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `base: base,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `primary: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `secondary: secondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `tertiary: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `return jakarta.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `displayLarge: jakarta.displayLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `fontSize: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `height: 1.15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `letterSpacing: -1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `displayMedium: jakarta.displayMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `fontSize: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `height: 1.18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `letterSpacing: -0.9,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `headlineLarge: jakarta.headlineLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `fontSize: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `height: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `letterSpacing: -0.6,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `headlineMedium: jakarta.headlineMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `fontSize: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `height: 1.25,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `letterSpacing: -0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `headlineSmall: jakarta.headlineSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `fontSize: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `height: 1.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L585** `letterSpacing: -0.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `titleLarge: jakarta.titleLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `height: 1.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L591** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L592** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `labelSmall: jakarta.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `letterSpacing: 1.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `color: tertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `bodyLarge: jakarta.bodyLarge?.copyWith(height: 1.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `bodyMedium: jakarta.bodyMedium?.copyWith(height: 1.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `static ButtonStyle _monoButtonStyle({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `required Color background,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `required Color foreground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `required TextStyle? textStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `return ElevatedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `backgroundColor: background,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `foregroundColor: foreground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `shadowColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L615** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `textStyle: textStyle?.copyWith(fontWeight: FontWeight.w700, fontSize: 15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/monochrome_workspace_theme.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/nature_paper_colors.dart

Four visual styles × light/dark Material 3 themes. This file (nature paper colors) is one concrete piece of that layer.

**L3** `class NaturePaperColors {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `static const cream = Color(0xFFF8F3E8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `static const paperBeige = Color(0xFFEADFCB);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `static const sage = Color(0xFF8FA97D);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `static const forest = Color(0xFF5F7A61);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `static const softOrange = Color(0xFFD79A57);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `static const warmBrown = Color(0xFF8A6A4A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `static const paleBlue = Color(0xFFDCE9EC);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `static const sunGlow = Color(0xFFF0C387);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `static const dayInk = Color(0xFF3B2F22);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `static const dayCard = Color(0xFFFFFCF4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `static const dayCardEdge = Color(0xFFDFD2B6);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static const deepForest = Color(0xFF465D48);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `static const dayDanger = Color(0xFFAA5B4C);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `static const midnight = Color(0xFF121824);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `static const forestNight = Color(0xFF18221B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `static const moonBeige = Color(0xFFE6DCC8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `static const sageNight = Color(0xFF7E9A78);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `static const amber = Color(0xFFD7A25A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `static const dustyRose = Color(0xFFA56A6A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `static const nightInk = Color(0xFFF1EBDC);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `static const nightTextSecondary = Color(0xFFCFC7B0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `static const nightTextTertiary = Color(0xFFB3AA92);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `static const nightCard = Color(0xFF1E2924);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `static const nightCardHigh = Color(0xFF283630);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `static const nightCardHighest = Color(0xFF31403A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `static const nightBorder = Color(0xFF43514A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `static const moonGlow = Color(0xFFE8D9AE);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/theme/nature_paper_colors.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/nature_paper_theme.dart

Four visual styles × light/dark Material 3 themes. This file (nature paper theme) is one concrete piece of that layer.

**L6** `class NaturePaperTheme {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `static const _fadeTransitions = PageTransitionsTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `builders: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `TargetPlatform.android: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `TargetPlatform.iOS: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `TargetPlatform.macOS: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `TargetPlatform.linux: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `TargetPlatform.windows: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `TargetPlatform.fuchsia: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static ThemeData light() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `const ink = NaturePaperColors.dayInk;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `const forest = NaturePaperColors.forest;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `const card = NaturePaperColors.dayCard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final border = NaturePaperColors.warmBrown.withValues(alpha: 0.18);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `brightness: Brightness.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `colorScheme: ColorScheme.fromSeed(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `seedColor: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `primary: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `onPrimary: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `secondary: NaturePaperColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `onSecondary: const Color(0xFF1F2A1E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `tertiary: NaturePaperColors.softOrange,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `error: NaturePaperColors.dayDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `surface: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `onSurface: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `surfaceContainerLowest: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `surfaceContainerLow: NaturePaperColors.paperBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `surfaceContainer: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `surfaceContainerHigh: const Color(0xFFF4ECD9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `surfaceContainerHighest: NaturePaperColors.paperBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `outline: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final textTheme = buildJakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `primary: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `secondary: ink.withValues(alpha: 0.78),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `tertiary: ink.withValues(alpha: 0.56),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `scaffoldBackgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `canvasColor: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `pageTransitionsTheme: _fadeTransitions,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `dividerColor: NaturePaperColors.warmBrown.withValues(alpha: 0.14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `extensions: const [AppStyleTokens.natureLight],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `color: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `borderRadius: BorderRadius.circular(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `side: BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `contentTextStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `color: ink.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `backgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `modalBackgroundColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `borderRadius: BorderRadius.vertical(top: Radius.circular(28)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `backgroundColor: const Color(0xFF4B3D2C),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `contentTextStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `color: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `actionTextColor: NaturePaperColors.sunGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `behavior: SnackBarBehavior.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `fillColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `hintStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `color: ink.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `labelStyle: TextStyle(color: forest.withValues(alpha: 0.9)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `borderSide: BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `borderSide: BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `borderSide: const BorderSide(color: forest, width: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `style: _paperButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `background: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `foreground: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `shadow: NaturePaperColors.warmBrown.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `filledButtonTheme: FilledButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `style: _paperButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `background: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `foreground: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `shadow: NaturePaperColors.warmBrown.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `foregroundColor: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `side: BorderSide(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `color: NaturePaperColors.sage.withValues(alpha: 0.65),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `width: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `borderRadius: BorderRadius.circular(18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `foregroundColor: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `backgroundColor: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `foregroundColor: NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `elevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `focusElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `hoverElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `highlightElevation: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `backgroundColor: NaturePaperColors.sage.withValues(alpha: 0.16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `selectedColor: NaturePaperColors.sage.withValues(alpha: 0.34),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `color: NaturePaperColors.deepForest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `side: BorderSide(color: NaturePaperColors.sage.withValues(alpha: 0.4)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `backgroundColor: card.withValues(alpha: 0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `indicatorColor: NaturePaperColors.sage.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `color: selected ? forest : ink.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `return IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `color: states.contains(WidgetState.selected)`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `? forest`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `: ink.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `backgroundColor: NaturePaperColors.cream.withValues(alpha: 0.86),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `indicatorColor: NaturePaperColors.sage.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `selectedIconTheme: const IconThemeData(color: forest),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `unselectedIconTheme: IconThemeData(color: ink.withValues(alpha: 0.5)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `progressIndicatorTheme: ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `color: forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `circularTrackColor: NaturePaperColors.paperBeige.withValues(alpha: 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `static ThemeData dark() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `const ink = NaturePaperColors.nightInk;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `const inkSecondary = NaturePaperColors.nightTextSecondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `const sage = NaturePaperColors.sageNight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `const amber = NaturePaperColors.amber;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `const card = NaturePaperColors.nightCard;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `const cardHigh = NaturePaperColors.nightCardHigh;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `const border = NaturePaperColors.nightBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `final colorScheme = ColorScheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `primary: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `onPrimary: const Color(0xFF0F1A12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `primaryContainer: const Color(0xFF32463A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `onPrimaryContainer: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `secondary: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `onSecondary: const Color(0xFF1B1A12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `secondaryContainer: const Color(0xFF3B4438),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `onSecondaryContainer: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `tertiary: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `onTertiary: const Color(0xFF221708),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `error: NaturePaperColors.dustyRose,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `onError: const Color(0xFF200F0F),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `surface: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `onSurface: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `onSurfaceVariant: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `surfaceContainerLowest: NaturePaperColors.midnight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `surfaceContainerLow: NaturePaperColors.forestNight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `surfaceContainer: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `surfaceContainerHigh: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `surfaceContainerHighest: NaturePaperColors.nightCardHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `outline: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `outlineVariant: const Color(0xFF35413B),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `shadow: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `scrim: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `inverseSurface: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `onInverseSurface: const Color(0xFF23201A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `inversePrimary: NaturePaperColors.forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `colorScheme: colorScheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `final textTheme = buildJakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `primary: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `secondary: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `tertiary: NaturePaperColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `scaffoldBackgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `canvasColor: NaturePaperColors.midnight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `pageTransitionsTheme: _fadeTransitions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `dividerColor: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `disabledColor: NaturePaperColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `iconTheme: const IconThemeData(color: inkSecondary),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `primaryIconTheme: const IconThemeData(color: amber),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `extensions: const [AppStyleTokens.natureDark],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `dividerTheme: const DividerThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `color: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `thickness: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `space: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `listTileTheme: ListTileThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `iconColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `textColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `subtitleTextStyle: textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `color: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `backgroundColor: sage.withValues(alpha: 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `disabledColor: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `selectedColor: sage.withValues(alpha: 0.36),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `secondarySelectedColor: sage.withValues(alpha: 0.36),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `color: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `secondaryLabelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `color: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `side: BorderSide(color: sage.withValues(alpha: 0.45)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `tabBarTheme: TabBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `labelColor: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `unselectedLabelColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `indicatorColor: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `dividerColor: border,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `labelStyle: textTheme.labelLarge?.copyWith(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `unselectedLabelStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `iconTheme: const IconThemeData(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `actionsIconTheme: const IconThemeData(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `color: card,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `shadowColor: Colors.black.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `borderRadius: BorderRadius.circular(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `side: const BorderSide(color: border, width: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `backgroundColor: cardHigh,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(24)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `backgroundColor: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `modalBackgroundColor: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `borderRadius: BorderRadius.vertical(top: Radius.circular(28)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `backgroundColor: NaturePaperColors.nightCardHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `actionTextColor: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `behavior: SnackBarBehavior.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `popupMenuTheme: PopupMenuThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `color: NaturePaperColors.nightCardHighest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `textStyle: textTheme.bodyMedium?.copyWith(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `dropdownMenuTheme: DropdownMenuThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `textStyle: textTheme.bodyMedium?.copyWith(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `inputDecorationTheme: const InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `fillColor: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `fillColor: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `hintStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `color: NaturePaperColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `labelStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `floatingLabelStyle: textTheme.bodyMedium?.copyWith(color: amber),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `prefixIconColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `suffixIconColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `borderSide: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `borderSide: const BorderSide(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `borderSide: const BorderSide(color: amber, width: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `style: _paperButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `background: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `foreground: const Color(0xFF0F1A12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `shadow: Colors.black.withValues(alpha: 0.6),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `disabledBackground: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `disabledForeground: NaturePaperColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `filledButtonTheme: FilledButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `style: _paperButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `background: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `foreground: const Color(0xFF0F1A12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `shadow: Colors.black.withValues(alpha: 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `disabledBackground: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `disabledForeground: NaturePaperColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `foregroundColor: NaturePaperColors.moonBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `side: BorderSide(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `color: NaturePaperColors.moonBeige.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `width: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `borderRadius: BorderRadius.circular(18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `foregroundColor: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `backgroundColor: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `foregroundColor: const Color(0xFF0F1A12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `elevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `focusElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `hoverElevation: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `highlightElevation: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `backgroundColor: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `indicatorColor: amber.withValues(alpha: 0.26),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `color: selected ? amber : inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `return IconThemeData(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `color: selected ? amber : inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `backgroundColor: NaturePaperColors.forestNight.withValues(alpha: 0.88),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `indicatorColor: amber.withValues(alpha: 0.26),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `selectedIconTheme: const IconThemeData(color: amber),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `unselectedIconTheme: const IconThemeData(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `selectedLabelTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `color: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `unselectedLabelTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `color: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `switchTheme: SwitchThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `thumbColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `if (states.contains(WidgetState.selected)) return amber;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `return inkSecondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `trackColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `if (states.contains(WidgetState.selected)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `return amber.withValues(alpha: 0.42);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `return cardHigh;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `progressIndicatorTheme: const ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `color: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `circularTrackColor: cardHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `static ButtonStyle _paperButtonStyle({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `required Color background,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `required Color foreground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `required Color shadow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `TextStyle? textStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `Color? disabledBackground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `Color? disabledForeground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `return ButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `backgroundColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `if (states.contains(WidgetState.disabled)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `return disabledBackground ?? background.withValues(alpha: 0.4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `return background;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `foregroundColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `if (states.contains(WidgetState.disabled)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `return disabledForeground ?? foreground.withValues(alpha: 0.5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `return foreground;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `shadowColor: WidgetStatePropertyAll(shadow),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `elevation: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `if (states.contains(WidgetState.disabled)) return 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `if (states.contains(WidgetState.pressed)) return 7;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `if (states.contains(WidgetState.hovered)) return 5;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `return 3;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `padding: const WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `shape: WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `textStyle: WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `textStyle?.copyWith(fontWeight: FontWeight.w700, fontSize: 15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/theme/nature_paper_theme.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/neumorphic.dart

Four visual styles × light/dark Material 3 themes. This file (neumorphic) is one concrete piece of that layer.

**L8** `enum NeuElevation { flat, raised, floating, pressed }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L10** `class Neumorphic {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L11** `const Neumorphic._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `static bool isActive(BuildContext context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `AppStyleTokens.maybeOf(context)?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static Color surface(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `NeuElevation elevation = NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `return switch (elevation) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `NeuElevation.pressed =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `? SoftNeumorphismColors.nightInset`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `NeuElevation.floating =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `? SoftNeumorphismColors.nightSurfaceHigh`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `NeuElevation.flat || NeuElevation.raised =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `? SoftNeumorphismColors.nightSurface`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `: SoftNeumorphismColors.morningSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `static Color shadowColor(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `? SoftNeumorphismColors.nightShadow`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `: SoftNeumorphismColors.morningShadow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `static Color highlightColor(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `return isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `? SoftNeumorphismColors.nightHighlight`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `: SoftNeumorphismColors.morningHighlight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `static List<BoxShadow> shadows(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `NeuElevation elevation = NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `double intensity = 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final dark = shadowColor(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final light = highlightColor(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `switch (elevation) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `case NeuElevation.flat:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `return const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `case NeuElevation.pressed:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `return [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `color: dark.withValues(alpha: (isDark ? 0.85 : 0.45) * intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `offset: const Offset(4, 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `blurRadius: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `spreadRadius: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `color: light.withValues(alpha: (isDark ? 0.55 : 0.90) * intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `offset: const Offset(-3, -3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `blurRadius: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `spreadRadius: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `case NeuElevation.raised:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `return [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `color: dark.withValues(alpha: (isDark ? 0.75 : 0.38) * intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `offset: const Offset(6, 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `blurRadius: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `color: light.withValues(alpha: (isDark ? 0.50 : 0.95) * intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `offset: const Offset(-5, -5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `blurRadius: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `case NeuElevation.floating:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `return [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `color: dark.withValues(alpha: (isDark ? 0.82 : 0.42) * intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `offset: const Offset(10, 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `blurRadius: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `color: light.withValues(alpha: (isDark ? 0.55 : 1.0) * intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `offset: const Offset(-6, -6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `static BoxDecoration decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `BuildContext context, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `NeuElevation elevation = NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `BorderRadius? borderRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `Color? color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Gradient? gradient,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `double intensity = 1,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final radius =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `borderRadius ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `AppStyleTokens.maybeOf(context)?.cardRadius ?? 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `return BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `color: gradient == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `? (color ?? surface(context, elevation: elevation))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `gradient: gradient,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `boxShadow: shadows(context, elevation: elevation, intensity: intensity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `class NeuIconWell extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L132** `const NeuIconWell({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `this.size = 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `this.padding = 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `this.elevation = NeuElevation.pressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `final Color? color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final double padding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `final NeuElevation elevation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `final Widget? child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final tone = color ?? AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `final iconWidget = child ?? PaperGlyph(icon: icon, size: size, color: tone);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `if (tokens?.isNaturePaper ?? false) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final side = size + padding * 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `width: side,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `height: side,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `? NaturePaperColors.nightCardHighest`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `: NaturePaperColors.paperBeige.withValues(alpha: 0.72),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `? NaturePaperColors.moonGlow.withValues(alpha: 0.28)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `: NaturePaperColors.warmBrown.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `width: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `color: (isDark ? Colors.black : NaturePaperColors.warmBrown)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `.withValues(alpha: isDark ? 0.35 : 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `blurRadius: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `offset: const Offset(0, 3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `if (!isDark)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `const BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `color: NaturePaperColors.dayCardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `offset: Offset(0, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `child: iconWidget,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `if (tokens?.isSoftNeumorphism ?? false) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final side = size + padding * 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `width: side,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `height: side,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `decoration: Neumorphic.decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `elevation: elevation,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `borderRadius: BorderRadius.circular(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `intensity: 1.05,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `child: child ?? Icon(icon, size: size, color: tone),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `if (tokens?.isMonochromeWorkspace ?? false) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `padding: EdgeInsets.all(padding),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `borderRadius: BorderRadius.circular(4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `border: Border.all(color: tone.withValues(alpha: 0.18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `color: tone.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `child: child ?? Icon(icon, size: size, color: tone),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `padding: EdgeInsets.all(padding),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `color: tone.withValues(alpha: 0.10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `child: child ?? Icon(icon, size: size, color: tone),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `class NeuAvatar extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L233** `const NeuAvatar({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `this.radius = 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `this.elevation = NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `final double radius;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `final NeuElevation elevation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `if (!Neumorphic.isActive(context)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `return CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `radius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `backgroundColor: AppColors.accent(context).withValues(alpha: 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `final side = radius * 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `width: side,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `height: side,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `decoration: Neumorphic.decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `elevation: elevation,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `intensity: 1.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `class NeuSearchField extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L270** `const NeuSearchField({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `required this.controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `required this.hintText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `this.onChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `final TextEditingController controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `final String hintText;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `final ValueChanged<String>? onChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `final isNeu = Neumorphic.isActive(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `final field = TextField(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `controller: controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `onChanged: onChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `style: Theme.of(context).textTheme.bodyLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `hintText: hintText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `prefixIcon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `Icons.search_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `filled: !isNeu,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `fillColor: isNeu`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `? Colors.transparent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `: (isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `? scheme.surfaceContainerHighest`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `: AppColors.controlSurface(context)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `border: isNeu ? InputBorder.none : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `enabledBorder: isNeu ? InputBorder.none : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `focusedBorder: isNeu ? InputBorder.none : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `horizontal: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `vertical: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `if (!isNeu) return field;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `decoration: Neumorphic.decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `elevation: NeuElevation.pressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `intensity: 1.05,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `child: field,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `class NeuPressable extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L328** `const NeuPressable({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `this.borderRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `this.padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `this.elevation = NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `this.enabled = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `final BorderRadius? borderRadius;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `final EdgeInsetsGeometry? padding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `final Color? color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `final NeuElevation elevation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `final bool enabled;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `State<NeuPressable> createState() => _NeuPressableState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `class _NeuPressableState extends State<NeuPressable> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L352** `bool _pressed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `final neu = Neumorphic.isActive(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `final radius =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `widget.borderRadius ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `AppStyleTokens.maybeOf(context)?.cardRadius ?? 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `final elevation = (!widget.enabled || widget.onTap == null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `? NeuElevation.flat`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `: (_pressed ? NeuElevation.pressed : widget.elevation);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `final body = AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `duration: const Duration(milliseconds: 140),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `padding: widget.padding,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `transform: Matrix4.diagonal3Values(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `_pressed && neu ? 0.98 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `_pressed && neu ? 0.98 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `transformAlignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `decoration: neu`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `? Neumorphic.decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `elevation: elevation,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `color: widget.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `widget.color ?? AppStyleTokens.maybeOf(context)?.cardSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `child: widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `if (widget.onTap == null || !widget.enabled) return body;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `return GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `onTapDown: (_) => setState(() => _pressed = true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `onTapUp: (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `setState(() => _pressed = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `widget.onTap?.call();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `onTapCancel: () => setState(() => _pressed = false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `child: body,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/neumorphic.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/soft_neumorphism_colors.dart

Four visual styles × light/dark Material 3 themes. This file (soft neumorphism colors) is one concrete piece of that layer.

**L3** `class SoftNeumorphismColors {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `static const morningBg = Color(0xFFE4E2DC);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `static const morningBgCool = Color(0xFFDEE1E4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `static const morningSurface = Color(0xFFEAE8E2);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `static const morningSurfaceHigh = Color(0xFFF1EFE9);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `static const morningInset = Color(0xFFD6D4CE);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `static const sage = Color(0xFF6F9279);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `static const sageDeep = Color(0xFF54705C);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const amber = Color(0xFFC9965A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `static const morningInk = Color(0xFF242422);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `static const morningTextSecondary = Color(0xFF5E5E5A);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `static const morningTextTertiary = Color(0xFF858580);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `static const morningShadow = Color(0xFF9A978F);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `static const morningHighlight = Color(0xFFFFFFFF);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `static const morningDanger = Color(0xFFB07070);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `static const nightBg = Color(0xFF1C1F24);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `static const nightBgCool = Color(0xFF181C22);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `static const nightSurface = Color(0xFF23272D);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `static const nightSurfaceHigh = Color(0xFF2A2F36);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `static const nightInset = Color(0xFF14171B);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `static const emerald = Color(0xFF5A9A78);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `static const emeraldGlow = Color(0xFF7AB894);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `static const nightInk = Color(0xFFF2F0EB);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `static const nightTextSecondary = Color(0xFFA8A6A0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `static const nightTextTertiary = Color(0xFF8A8882);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `static const nightShadow = Color(0xFF050608);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `static const nightHighlight = Color(0xFF3E4550);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `static const nightDanger = Color(0xFFC08080);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/core/theme/soft_neumorphism_colors.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/theme/soft_neumorphism_theme.dart

Four visual styles × light/dark Material 3 themes. This file (soft neumorphism theme) is one concrete piece of that layer.

**L6** `class SoftNeumorphismTheme {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `static const _fadeTransitions = PageTransitionsTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `builders: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `TargetPlatform.android: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `TargetPlatform.iOS: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `TargetPlatform.macOS: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `TargetPlatform.linux: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `TargetPlatform.windows: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `TargetPlatform.fuchsia: FadeUpwardsPageTransitionsBuilder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `static const _radius = 18.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `static const _controlRadius = 14.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static ThemeData light() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `const ink = SoftNeumorphismColors.morningInk;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `const sage = SoftNeumorphismColors.sage;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `const surface = SoftNeumorphismColors.morningSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `const bg = SoftNeumorphismColors.morningBg;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `brightness: Brightness.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `colorScheme: ColorScheme.fromSeed(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `seedColor: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `primary: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `onPrimary: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `secondary: SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `onSecondary: const Color(0xFF2A2218),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `tertiary: SoftNeumorphismColors.sageDeep,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `error: SoftNeumorphismColors.morningDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `surface: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `onSurface: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `surfaceContainerLowest: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `surfaceContainerLow: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `surfaceContainer: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `surfaceContainerHigh: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `surfaceContainerHighest: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `outline: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final textTheme = buildJakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `primary: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `secondary: SoftNeumorphismColors.morningTextSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `tertiary: SoftNeumorphismColors.morningTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `scaffoldBackgroundColor: Colors.transparent,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `canvasColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `pageTransitionsTheme: _fadeTransitions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `dividerColor: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.22),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `extensions: const [AppStyleTokens.softNeuLight],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `color: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `backgroundColor: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `contentTextStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `color: SoftNeumorphismColors.morningTextSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `backgroundColor: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `modalBackgroundColor: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `borderRadius: BorderRadius.vertical(top: Radius.circular(28)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `backgroundColor: SoftNeumorphismColors.sageDeep,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `contentTextStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `color: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `actionTextColor: SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `behavior: SnackBarBehavior.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `fillColor: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `hintStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `color: SoftNeumorphismColors.morningTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `labelStyle: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `color: SoftNeumorphismColors.morningTextSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `borderSide: BorderSide.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `borderSide: BorderSide.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `borderSide: const BorderSide(color: sage, width: 1.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `style: _neuButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `background: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `foreground: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `shadow: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `filledButtonTheme: FilledButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `style: _neuButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `background: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `foreground: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `shadow: SoftNeumorphismColors.morningShadow.withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `style: OutlinedButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `foregroundColor: SoftNeumorphismColors.sageDeep,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `side: BorderSide(color: sage.withValues(alpha: 0.45), width: 1.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `foregroundColor: SoftNeumorphismColors.sageDeep,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `backgroundColor: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `foregroundColor: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `elevation: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `focusElevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `hoverElevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `highlightElevation: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `backgroundColor: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `selectedColor: sage.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `color: SoftNeumorphismColors.sageDeep,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `side: BorderSide.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `switchTheme: SwitchThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `thumbColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `if (states.contains(WidgetState.selected)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `return SoftNeumorphismColors.morningSurfaceHigh;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `return SoftNeumorphismColors.morningSurfaceHigh;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `trackColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `if (states.contains(WidgetState.selected)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `return sage;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `return SoftNeumorphismColors.morningInset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `trackOutlineColor: const WidgetStatePropertyAll(Colors.transparent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `progressIndicatorTheme: ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `color: sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `circularTrackColor: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `linearTrackColor: SoftNeumorphismColors.morningInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `backgroundColor: SoftNeumorphismColors.morningSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `indicatorColor: sage.withValues(alpha: 0.22),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `? SoftNeumorphismColors.sageDeep`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `: SoftNeumorphismColors.morningTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `return IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `color: states.contains(WidgetState.selected)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `? SoftNeumorphismColors.sageDeep`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `: SoftNeumorphismColors.morningTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `backgroundColor: SoftNeumorphismColors.morningSurface.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `alpha: 0.92,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `indicatorColor: sage.withValues(alpha: 0.22),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `selectedIconTheme: const IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `color: SoftNeumorphismColors.sageDeep,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `unselectedIconTheme: const IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `color: SoftNeumorphismColors.morningTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `static ThemeData dark() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `const ink = SoftNeumorphismColors.nightInk;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `const inkSecondary = SoftNeumorphismColors.nightTextSecondary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `const emerald = SoftNeumorphismColors.emerald;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `const amber = SoftNeumorphismColors.amber;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `const surface = SoftNeumorphismColors.nightSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `const surfaceHigh = SoftNeumorphismColors.nightSurfaceHigh;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `final colorScheme = ColorScheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `primary: emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `onPrimary: const Color(0xFF0E1812),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `primaryContainer: const Color(0xFF2A3F34),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `onPrimaryContainer: SoftNeumorphismColors.nightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `secondary: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `onSecondary: const Color(0xFF1A140C),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `secondaryContainer: const Color(0xFF3A3228),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `onSecondaryContainer: SoftNeumorphismColors.nightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `tertiary: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `onTertiary: const Color(0xFF0E1812),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `error: SoftNeumorphismColors.nightDanger,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `onError: const Color(0xFF1A0C0C),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `surface: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `onSurface: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `onSurfaceVariant: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `surfaceContainerLowest: SoftNeumorphismColors.nightBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `surfaceContainerLow: SoftNeumorphismColors.nightInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `surfaceContainer: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `surfaceContainerHigh: surfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `surfaceContainerHighest: const Color(0xFF323840),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `outline: SoftNeumorphismColors.nightHighlight,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `outlineVariant: const Color(0xFF2E343C),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `shadow: SoftNeumorphismColors.nightShadow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `scrim: Colors.black,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `inverseSurface: SoftNeumorphismColors.morningSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `onInverseSurface: SoftNeumorphismColors.morningInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `inversePrimary: SoftNeumorphismColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `final base = ThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `useMaterial3: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `brightness: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `colorScheme: colorScheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `final textTheme = buildJakartaTextTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `base: base.textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `primary: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `secondary: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `tertiary: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `return base.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `scaffoldBackgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `canvasColor: SoftNeumorphismColors.nightBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `pageTransitionsTheme: _fadeTransitions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `dividerColor: SoftNeumorphismColors.nightHighlight.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `alpha: 0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `disabledColor: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `textTheme: textTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `iconTheme: const IconThemeData(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `primaryIconTheme: const IconThemeData(color: emerald),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `extensions: const [AppStyleTokens.softNeuDark],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `dividerTheme: DividerThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `color: SoftNeumorphismColors.nightHighlight.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `thickness: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `space: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `listTileTheme: ListTileThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `iconColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `textColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `subtitleTextStyle: textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `color: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `appBarTheme: AppBarTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `centerTitle: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `foregroundColor: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `iconTheme: const IconThemeData(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `actionsIconTheme: const IconThemeData(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `cardTheme: CardThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `color: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `borderRadius: BorderRadius.circular(_radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `margin: const EdgeInsets.symmetric(vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `dialogTheme: DialogThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `backgroundColor: surfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `borderRadius: BorderRadius.circular(_radius),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `titleTextStyle: textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `color: ink,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `bottomSheetTheme: const BottomSheetThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `backgroundColor: SoftNeumorphismColors.nightSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `modalBackgroundColor: SoftNeumorphismColors.nightSurfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `borderRadius: BorderRadius.vertical(top: Radius.circular(28)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `snackBarTheme: SnackBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `backgroundColor: surfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `contentTextStyle: textTheme.bodyMedium?.copyWith(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `actionTextColor: amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `behavior: SnackBarBehavior.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `inputDecorationTheme: InputDecorationTheme(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `filled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `fillColor: SoftNeumorphismColors.nightInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `hintStyle: textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `color: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `labelStyle: textTheme.bodyMedium?.copyWith(color: inkSecondary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `floatingLabelStyle: textTheme.bodyMedium?.copyWith(color: emerald),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `prefixIconColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `suffixIconColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `border: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `borderSide: BorderSide.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `enabledBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `borderSide: BorderSide.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `focusedBorder: OutlineInputBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `borderSide: const BorderSide(color: emerald, width: 1.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `contentPadding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `elevatedButtonTheme: ElevatedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `style: _neuButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `background: emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `foreground: const Color(0xFF0E1812),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `shadow: SoftNeumorphismColors.nightShadow.withValues(alpha: 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `filledButtonTheme: FilledButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `style: _neuButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `background: emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `foreground: const Color(0xFF0E1812),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `shadow: SoftNeumorphismColors.nightShadow.withValues(alpha: 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `textStyle: textTheme.labelLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `outlinedButtonTheme: OutlinedButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `style: OutlinedButton.styleFrom(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `foregroundColor: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `side: BorderSide(color: emerald.withValues(alpha: 0.55), width: 1.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `textButtonTheme: TextButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `foregroundColor: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `textStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `floatingActionButtonTheme: FloatingActionButtonThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `backgroundColor: emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `foregroundColor: const Color(0xFF0E1812),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `elevation: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `focusElevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `hoverElevation: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `highlightElevation: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `extendedTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `chipTheme: ChipThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `backgroundColor: SoftNeumorphismColors.nightInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `selectedColor: emerald.withValues(alpha: 0.32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `labelStyle: textTheme.labelLarge!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `color: SoftNeumorphismColors.nightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `side: BorderSide.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `switchTheme: SwitchThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `thumbColor: const WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `SoftNeumorphismColors.nightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `trackColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `if (states.contains(WidgetState.selected)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `return emerald;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `return SoftNeumorphismColors.nightInset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `trackOutlineColor: const WidgetStatePropertyAll(Colors.transparent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `progressIndicatorTheme: ProgressIndicatorThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `color: emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `circularTrackColor: SoftNeumorphismColors.nightInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `linearTrackColor: SoftNeumorphismColors.nightInset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `navigationBarTheme: NavigationBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `backgroundColor: surfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `indicatorColor: emerald.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `elevation: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `labelTextStyle: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `return textTheme.labelSmall!.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `fontSize: 11,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `fontWeight: selected ? FontWeight.w700 : FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `? SoftNeumorphismColors.emeraldGlow`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `iconTheme: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `final selected = states.contains(WidgetState.selected);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `return IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `? SoftNeumorphismColors.emeraldGlow`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `navigationRailTheme: NavigationRailThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `backgroundColor: SoftNeumorphismColors.nightSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `indicatorColor: emerald.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `selectedIconTheme: const IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `color: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `unselectedIconTheme: const IconThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `color: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `selectedLabelTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `color: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `unselectedLabelTextStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `color: SoftNeumorphismColors.nightTextTertiary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `tabBarTheme: TabBarThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `labelColor: SoftNeumorphismColors.emeraldGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `unselectedLabelColor: inkSecondary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `indicatorColor: emerald,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `dividerColor: SoftNeumorphismColors.nightHighlight.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `alpha: 0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `labelStyle: textTheme.labelLarge?.copyWith(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `unselectedLabelStyle: textTheme.labelLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `popupMenuTheme: PopupMenuThemeData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `color: surfaceHigh,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `textStyle: textTheme.bodyMedium?.copyWith(color: ink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `static ButtonStyle _neuButtonStyle({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `required Color background,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `required Color foreground,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `required Color shadow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `TextStyle? textStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `return ButtonStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `backgroundColor: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `if (states.contains(WidgetState.disabled)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `return background.withValues(alpha: 0.40);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `if (states.contains(WidgetState.pressed)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `return Color.lerp(background, Colors.black, 0.08);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `return background;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `foregroundColor: WidgetStatePropertyAll(foreground),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `elevation: WidgetStateProperty.resolveWith((states) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `if (states.contains(WidgetState.pressed)) return 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `if (states.contains(WidgetState.hovered)) return 5;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `return 3;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `shadowColor: WidgetStatePropertyAll(shadow),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `padding: const WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `EdgeInsets.symmetric(horizontal: 24, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `shape: WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L585** `borderRadius: BorderRadius.circular(_controlRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `textStyle: WidgetStatePropertyAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `textStyle?.copyWith(fontWeight: FontWeight.w700, fontSize: 15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/theme/soft_neumorphism_theme.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/utils/responsive.dart

Responsive breakpoints and voice playback. This file (responsive) is one concrete piece of that layer.

**L4** `class Breakpoints {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L5** `static const mobile = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `static const tablet = 600;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `static const desktop = 1100;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `extension ResponsiveX on BuildContext {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `bool get isMobile => MediaQuery.sizeOf(this).width < Breakpoints.tablet;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `bool get isTablet =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `MediaQuery.sizeOf(this).width >= Breakpoints.tablet &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `MediaQuery.sizeOf(this).width < Breakpoints.desktop;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `bool get useRail => MediaQuery.sizeOf(this).width >= Breakpoints.tablet;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `double get contentMaxWidth {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final w = MediaQuery.sizeOf(this).width;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `if (w >= Breakpoints.desktop) return 960;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `if (w >= Breakpoints.tablet) return 840;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `return w;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `List<Breakpoint> responsiveBreakpoints() => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `const Breakpoint(start: 0, end: 599, name: MOBILE),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `const Breakpoint(start: 600, end: 1099, name: TABLET),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `const Breakpoint(start: 1100, end: double.infinity, name: DESKTOP),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/utils/responsive.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

## Every line — lib/core/utils/time_of_day_greeting.dart

Responsive breakpoints and voice playback. This file (time of day greeting) is one concrete piece of that layer.

**L3** `String timeOfDayGreeting(AppLocalizations l10n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `final hour = DateTime.now().hour;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `if (hour < 12) return l10n.greetingMorning;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `if (hour < 18) return l10n.greetingAfternoon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `return l10n.greetingEvening;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/utils/time_of_day_greeting.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/core/utils/voice_playback.dart

Responsive breakpoints and voice playback. This file (voice playback) is one concrete piece of that layer.

**L9** `class DetectedAudioFormat {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `const DetectedAudioFormat({required this.mimeType, required this.extension});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final String mimeType;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final String extension;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `DetectedAudioFormat detectAudioFormat(Uint8List bytes) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `if (bytes.length >= 12) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final isRiff =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `bytes[0] == 0x52 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `bytes[1] == 0x49 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `bytes[2] == 0x46 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `bytes[3] == 0x46;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final isWave =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `bytes[8] == 0x57 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `bytes[9] == 0x41 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `bytes[10] == 0x56 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `bytes[11] == 0x45;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (isRiff && isWave) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `return const DetectedAudioFormat(mimeType: 'audio/wav', extension: 'wav');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (bytes.length >= 8 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `bytes[4] == 0x66 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `bytes[5] == 0x74 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `bytes[6] == 0x79 &&`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `bytes[7] == 0x70) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `return const DetectedAudioFormat(mimeType: 'audio/mp4', extension: 'm4a');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `if (bytes.length >= 3 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `bytes[0] == 0x49 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `bytes[1] == 0x44 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `bytes[2] == 0x33) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `return const DetectedAudioFormat(mimeType: 'audio/mpeg', extension: 'mp3');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `return const DetectedAudioFormat(mimeType: 'audio/mp4', extension: 'm4a');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `String mimeTypeForExtension(String extension) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return switch (extension.toLowerCase().replaceAll('.', '')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `'wav' => 'audio/wav',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `'mp3' => 'audio/mpeg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `'aac' => 'audio/aac',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `_ => 'audio/mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `bool _audioContextReady = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `Future<void> _ensureAudioContext() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `if (_audioContextReady) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `await AudioPlayer.global.setAudioContext(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `AudioContext(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `iOS: AudioContextIOS(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `category: AVAudioSessionCategory.playback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `options: const {},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `android: const AudioContextAndroid(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `isSpeakerphoneOn: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `stayAwake: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `contentType: AndroidContentType.speech,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `usageType: AndroidUsageType.media,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `audioFocus: AndroidAudioFocus.gain,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `_audioContextReady = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Future<Uint8List> _downloadVoiceBytes(String url) async {`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L83** `final response = await http.get(Uri.parse(url));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `if (response.statusCode < 200 || response.statusCode >= 300) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `throw Exception('Could not download voice (${response.statusCode})');`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L87** `final bytes = response.bodyBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `if (bytes.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `throw Exception('Voice file is empty');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `if (bytes.length < 64) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final head = String.fromCharCodes(bytes.take(32));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `if (head.trimLeft().startsWith('{') || head.trimLeft().startsWith('<')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `throw Exception('Voice download returned an error payload');`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L97** `return Uint8List.fromList(bytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `Future<({Uint8List bytes, String mimeType, File? file})> resolveVoiceSource(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `String url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `final cache = VoiceCacheService.instance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final cachedFile = await cache.getFile(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (cachedFile != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final bytes = Uint8List.fromList(await cachedFile.readAsBytes());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `if (bytes.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `final format = detectAudioFormat(bytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final fileExt = cachedFile.path.split('.').last.toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `if (fileExt != format.extension) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final fixed = await cache.put(url, bytes, format.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `return (bytes: bytes, mimeType: format.mimeType, file: fixed);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return (bytes: bytes, mimeType: format.mimeType, file: cachedFile);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `final bytes = await _downloadVoiceBytes(url);`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L119** `final format = detectAudioFormat(bytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `final file = await cache.put(url, bytes, format.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `return (bytes: bytes, mimeType: format.mimeType, file: file);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `class VoicePlaybackController extends ChangeNotifier {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L125** `VoicePlaybackController._() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `_attachPlayerListeners(_player);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `static final VoicePlaybackController instance = VoicePlaybackController._();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `final AudioPlayer _player = AudioPlayer(playerId: 'mu_study_voice');`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `String? _activeKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `bool _playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `bool _loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `Duration _position = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `Duration _duration = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `int _generation = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `Future<void> _queue = Future.value();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `String? get activeKey => _activeKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Duration get position => _position;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `Duration get duration => _duration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `bool isPlaying(String key) => _playing && _activeKey == key;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `bool isLoading(String key) => _loading && _activeKey == key;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `void _attachPlayerListeners(AudioPlayer player) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `player.onPlayerComplete.listen((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `_playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `_position = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `player.onPlayerStateChanged.listen((state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `if (state == PlayerState.stopped || state == PlayerState.completed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `if (_playing) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `_playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `player.onPositionChanged.listen((pos) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `if (!_playing && !_loading) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `_position = pos;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `player.onDurationChanged.listen((dur) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `_duration = dur;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `Future<void> stop() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `return _enqueue(() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `_generation++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `await _safeStopAndRelease();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `_playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `_activeKey = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `_position = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `_duration = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `Future<void> toggleRemote(String url) => _toggle(url, remote: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `Future<void> toggleLocal(String path) => _toggle(path, remote: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `Future<void> _enqueue(Future<void> Function() action) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `final run = _queue.then((_) => action());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `_queue = run.catchError((_) {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `return run;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `Future<void> _safeStopAndRelease() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `await _player.stop();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `await _player.release();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `Future<void> _playResolved({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `required Uint8List bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `required String mimeType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `File? file,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `if (file != null && await file.exists()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `await _player.play(DeviceFileSource(file.path, mimeType: mimeType));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `await _player.play(BytesSource(bytes, mimeType: mimeType));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `Future<void> _toggle(String key, {required bool remote}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `return _enqueue(() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `if (_loading && _activeKey == key) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `if (_activeKey == key && _playing) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `_generation++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `await _safeStopAndRelease();`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `_playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `_activeKey = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `_position = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `_duration = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `final gen = ++_generation;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `_activeKey = key;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `_loading = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `_playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `_position = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `_duration = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `await _ensureAudioContext();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `await _safeStopAndRelease();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `if (gen != _generation) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `if (remote) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `final resolved = await resolveVoiceSource(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `if (gen != _generation) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `await _playResolved(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `bytes: resolved.bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `mimeType: resolved.mimeType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `file: resolved.file,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `final lower = key.toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `final mime = lower.endsWith('.wav')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `? 'audio/wav'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `: lower.endsWith('.mp3')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `? 'audio/mpeg'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `: 'audio/mp4';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `await _player.play(DeviceFileSource(key, mimeType: mime));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `if (gen != _generation) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `await _safeStopAndRelease();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `_playing = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `if (gen != _generation) rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `_playing = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `_activeKey = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `_position = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `_duration = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `notifyListeners();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/core/utils/voice_playback.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/l10n/domain_labels.dart

ARB translations and generated AppLocalizations. This file (domain labels) is one concrete piece of that layer.

**L5** `extension MarkCategoryL10n on MarkCategory {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `String l10nLabel(AppLocalizations l10n) => switch (this) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `MarkCategory.homework => l10n.categoryHomework,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `MarkCategory.classwork => l10n.categoryClasswork,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `MarkCategory.quiz => l10n.categoryQuiz,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `MarkCategory.test => l10n.categoryTest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `MarkCategory.exam => l10n.categoryExam,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `MarkCategory.project => l10n.categoryProject,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `MarkCategory.oral => l10n.categoryOral,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `MarkCategory.other => l10n.categoryOther,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `extension AttendanceStatusL10n on AttendanceStatus {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `String l10nLabel(AppLocalizations l10n) => switch (this) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `AttendanceStatus.present => l10n.attendancePresentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `AttendanceStatus.absent => l10n.attendanceAbsentWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `AttendanceStatus.late => l10n.attendanceLateWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `AttendanceStatus.excused => l10n.attendanceExcusedWord,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `extension SphereL10n on Sphere {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `String l10nLabel(AppLocalizations l10n) => switch (this) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `Sphere.math => l10n.sphereMath,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `Sphere.english => l10n.sphereEnglish,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `extension AppNotificationL10n on AppNotification {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `String kindLabel(AppLocalizations l10n) => switch (kind) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `'assignment' => l10n.notifKindAssignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `'announcement' => l10n.notifKindAnnouncement,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `'graded' => l10n.notifKindGraded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `'submission' => l10n.notifKindSubmission,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `'chat' => l10n.notifKindChat,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `_ => title.isNotEmpty ? title : l10n.notifKindSystem,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `IconData get kindIcon => switch (kind) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `'assignment' => Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `'announcement' => Icons.campaign_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'graded' => Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'submission' => Icons.upload_file_outlined,`
Storage write. Path folders must satisfy bucket policies.

**L49** `'chat' => Icons.chat_bubble_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `_ => Icons.notifications_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/l10n/domain_labels.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

## Every line — lib/l10n/l10n_x.dart

ARB translations and generated AppLocalizations. This file (l10n x) is one concrete piece of that layer.

**L6** `extension L10nX on BuildContext {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `AppLocalizations get l10n => AppLocalizations.of(this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/l10n/l10n_x.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/l10n/nav_labels.dart

ARB translations and generated AppLocalizations. This file (nav labels) is one concrete piece of that layer.

**L3** `String navLabel(AppLocalizations l10n, String key) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `return switch (key) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `'navDashboard' => l10n.navDashboard,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `'navAssignments' => l10n.navAssignments,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `'navMarks' => l10n.navMarks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `'navAttendance' => l10n.navAttendance,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `'navNotifications' => l10n.navNotifications,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `'navProfile' => l10n.navProfile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `'navUsers' => l10n.navUsers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `'navGroups' => l10n.navGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `'navAnalytics' => l10n.navAnalytics,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `'navGradebook' => l10n.navGradebook,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `'navChat' => l10n.navChat,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `'navFocus' => l10n.navFocus,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `_ => key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/l10n/nav_labels.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/main.dart

Process entry: load env, init Supabase, wrap ProviderScope, run MaterialApp.router.

**L18** `Future<void> main() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `WidgetsFlutterBinding.ensureInitialized();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `await Env.load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `if (!Env.isConfigured) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `runApp(const SetupRequiredScreen());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `await Supabase.initialize(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `url: Env.supabaseUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `publishableKey: Env.supabaseAnonKey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final initialThemeMode = await loadSavedThemeMode();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final initialStyle = await loadSavedAppStyle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `runApp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `ProviderScope(`
Riverpod binding. Name it after the value it exposes.

**L38** `overrides: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `appStyleProvider.overrideWith((ref) {`
Riverpod binding. Name it after the value it exposes.

**L40** `final controller = AppStyleController(ref, initial: initialStyle);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `ref.listen(currentProfileProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L42** `next.whenData((profile) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `final code = profile?.preferredStyle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `if (code != null) controller.applyFromProfile(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `return controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `}),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `themeModeProvider.overrideWith((ref) {`
Riverpod binding. Name it after the value it exposes.

**L50** `final controller = ThemeModeController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `initial: initialThemeMode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `ref.listen(currentProfileProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L55** `next.whenData((profile) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final code = profile?.preferredTheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `if (code != null) controller.applyFromProfile(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `return controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `child: const MuStudyApp(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `class MuStudyApp extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L69** `const MuStudyApp({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final router = ref.watch(routerProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L74** `final locale = ref.watch(localeProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L75** `final themeMode = ref.watch(themeModeProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L76** `final style = ref.watch(appStyleProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L78** `return MaterialApp.router(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `onGenerateTitle: (context) => AppLocalizations.of(context).appName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `debugShowCheckedModeBanner: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `theme: AppTheme.light(style),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `darkTheme: AppTheme.dark(style),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `themeMode: themeMode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `themeAnimationDuration: Duration.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `locale: locale,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `supportedLocales: AppLocalizations.supportedLocales,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `localizationsDelegates: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `AppLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `GlobalMaterialLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `GlobalWidgetsLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `GlobalCupertinoLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `builder: (context, child) => ResponsiveBreakpoints.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `child: NaturePaperBackdrop(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `child: SoftNeumorphismBackdrop(child: child!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `breakpoints: responsiveBreakpoints(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `routerConfig: router,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/main.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/assignment.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (assignment) is one concrete piece of that layer.

**L3** `class Assignment {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `const Assignment({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.description = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.dueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final AssignmentType type;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String description;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final DateTime? dueDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final DateTime? createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `factory Assignment.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return Assignment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `teacherId: json['teacher_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `groupId: json['group_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `type: assignmentTypeFromString(json['type'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `title: json['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `description: json['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `dueDate: json['due_date'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `? DateTime.parse(json['due_date'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `createdAt: json['created_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `? DateTime.parse(json['created_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `Map<String, dynamic> toInsert({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `required String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `}) => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `'teacher_id': teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'type': assignmentTypeToDb(type),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `'description': description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `'due_date': dueDate?.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `class TaskItem {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L55** `const TaskItem({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `required this.assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `this.description = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `this.attachmentUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `this.linkUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `this.sortOrder = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final String assignmentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `final String description;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `final String? attachmentUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `final String? linkUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final int sortOrder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `factory TaskItem.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return TaskItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `assignmentId: json['assignment_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `title: json['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `description: json['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `attachmentUrl: json['attachment_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `linkUrl: json['link_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `sortOrder: json['sort_order'] as int? ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `class Submission {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L87** `const Submission({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `required this.taskId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `required this.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `this.textAnswer,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `this.fileUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `this.voiceUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `this.status = SubmissionStatus.pending,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `this.score,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `this.feedback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `this.isCompleted = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `this.submittedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final String taskId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `final String studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final String? textAnswer;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final String? fileUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final String? voiceUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final SubmissionStatus status;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `final double? score;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final String? feedback;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final bool isCompleted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final DateTime? submittedAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `factory Submission.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return Submission(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `taskId: json['task_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `studentId: json['student_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `textAnswer: json['text_answer'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `fileUrl: json['file_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `voiceUrl: json['voice_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `status: SubmissionStatus.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `(e) => e.name == (json['status'] as String? ?? 'pending'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `orElse: () => SubmissionStatus.pending,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `score: json['score'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `? double.tryParse(json['score'].toString())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `feedback: json['feedback'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `isCompleted: json['is_completed'] as bool? ?? false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `submittedAt: json['submitted_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `? DateTime.parse(json['submitted_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** : `null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/models/assignment.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/attendance.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (attendance) is one concrete piece of that layer.

**L3** `class AttendanceRecord {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `const AttendanceRecord({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required this.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.status,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.sessionId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final AttendanceStatus status;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final DateTime date;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final String? sessionId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String? studentName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `factory AttendanceRecord.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final student = json['students'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final profile = student?['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return AttendanceRecord(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `groupId: json['group_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `studentId: json['student_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `status: attendanceFromString(json['status'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `date: DateTime.parse(json['date'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `sessionId: json['session_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `studentName: profile?['full_name'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `class AttendanceStats {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L38** `const AttendanceStats({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `this.present = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `this.absent = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `this.late = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `this.excused = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final int present;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final int absent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final int late;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final int excused;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `int get total => present + absent + late + excused;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `double get presentPercent => total == 0 ? 0 : (present / total) * 100;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `factory AttendanceStats.fromCounts(Map<String, int> counts) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `return AttendanceStats(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `present: counts['present'] ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `absent: counts['absent'] ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `late: counts['late'] ?? 0,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `excused: counts['excused'] ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `factory AttendanceStats.fromRecords(Iterable<AttendanceRecord> records) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `var present = 0, absent = 0, late = 0, excused = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `for (final r in records) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `switch (r.status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `case AttendanceStatus.present:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `present++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `case AttendanceStatus.absent:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `absent++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `case AttendanceStatus.late:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `late++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `case AttendanceStatus.excused:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `excused++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `return AttendanceStats(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `present: present,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `absent: absent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `late: late,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `excused: excused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/models/attendance.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/chat.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (chat) is one concrete piece of that layer.

**L3** `class ChatRoomSummary {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `const ChatRoomSummary({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required this.groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.teacherName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.memberCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.lastMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.lastMessageType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.lastMessageText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.lastMessageAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `this.lastMessageDeleted = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `this.lastMessageSenderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `this.lastMessageSenderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `this.unreadCount = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final String groupTitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final String teacherName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final int memberCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final DateTime? createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final String? lastMessageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final ChatMessageType? lastMessageType;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final String? lastMessageText;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final DateTime? lastMessageAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final bool lastMessageDeleted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final String? lastMessageSenderId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final String? lastMessageSenderName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final int unreadCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `ChatRoomSummary copyWith({int? unreadCount}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `return ChatRoomSummary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `id: id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `teacherName: teacherName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `memberCount: memberCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `createdAt: createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `lastMessageId: lastMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `lastMessageType: lastMessageType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `lastMessageText: lastMessageText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `lastMessageAt: lastMessageAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `lastMessageDeleted: lastMessageDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `lastMessageSenderId: lastMessageSenderId,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `lastMessageSenderName: lastMessageSenderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `unreadCount: unreadCount ?? this.unreadCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `factory ChatRoomSummary.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final typeRaw = json['last_message_type'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `return ChatRoomSummary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `groupId: json['group_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `groupTitle: json['group_title'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `teacherName: json['teacher_name'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `memberCount: (json['member_count'] as num?)?.toInt() ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `createdAt: json['created_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `? DateTime.parse(json['created_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `lastMessageId: json['last_message_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `lastMessageType: typeRaw == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `: chatMessageTypeFromString(typeRaw),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `lastMessageText: json['last_message_text'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `lastMessageAt: json['last_message_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `? DateTime.parse(json['last_message_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `lastMessageDeleted: json['last_message_deleted'] as bool? ?? false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `lastMessageSenderId: json['last_message_sender_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `lastMessageSenderName: json['last_message_sender_name'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `unreadCount: (json['unread_count'] as num?)?.toInt() ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `class ChatMessage {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L83** `const ChatMessage({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `required this.roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `required this.senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `required this.messageType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `this.textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `this.mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `required this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `required this.updatedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `this.isDeleted = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `this.senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `this.senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `this.senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `this.senderRole,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `this.isReadByOthers = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `this.reactions = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `this.replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `this.replyTo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final String roomId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final String senderId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final ChatMessageType messageType;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final String? textContent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `final String? mediaUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final DateTime createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final DateTime updatedAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final bool isDeleted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final String? senderName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `final String? senderUsername;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final String? senderAvatarUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final UserRole? senderRole;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `final bool isReadByOthers;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `final List<ChatReaction> reactions;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `final String? replyToMessageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `final ChatReplyPreview? replyTo;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `String get displayName {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `final name = senderName?.trim() ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `if (name.isNotEmpty) return name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `final user = senderUsername?.trim() ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `if (user.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `return user.startsWith('@') ? user.substring(1) : user;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `return '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `bool get wasEdited =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `!isDeleted && updatedAt.difference(createdAt).inSeconds.abs() > 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `List<({String emoji, int count})> get reactionSummary {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `final counts = <String, int>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final order = <String>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `for (final r in reactions) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `if (!counts.containsKey(r.emoji)) order.add(r.emoji);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `counts[r.emoji] = (counts[r.emoji] ?? 0) + 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `return [for (final emoji in order) (emoji: emoji, count: counts[emoji]!)];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `String? myReactionEmoji(String? userId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `if (userId == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `for (final r in reactions) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `if (r.userId == userId) return r.emoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `Map<String, dynamic> toCacheJson() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `return {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `'id': id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `'room_id': roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `'sender_id': senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `'message_type': messageType.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `'text_content': textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `'media_url': mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `'created_at': createdAt.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `'updated_at': updatedAt.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `'is_deleted': isDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `'reply_to_message_id': replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `if (replyTo != null) 'reply_to': replyTo!.toCacheJson(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `'profiles': {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `'full_name': senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `'username': senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `'avatar_url': senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `if (senderRole != null) 'role': senderRole!.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `'message_reads': [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `if (isReadByOthers) {'user_id': 'cached_other'},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `'message_reactions': [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `for (final r in reactions)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `'id': r.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `'message_id': r.messageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `'user_id': r.userId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `'emoji': r.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `ChatMessage copyWith({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `String? textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `String? mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `DateTime? updatedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `bool? isDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `bool? isReadByOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `String? senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `String? senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `String? senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `UserRole? senderRole,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `List<ChatReaction>? reactions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `String? replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `ChatReplyPreview? replyTo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `return ChatMessage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `id: id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `roomId: roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `senderId: senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `messageType: messageType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `textContent: textContent ?? this.textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `mediaUrl: mediaUrl ?? this.mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `createdAt: createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `updatedAt: updatedAt ?? this.updatedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `isDeleted: isDeleted ?? this.isDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `senderName: senderName ?? this.senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `senderUsername: senderUsername ?? this.senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `senderAvatarUrl: senderAvatarUrl ?? this.senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `senderRole: senderRole ?? this.senderRole,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `isReadByOthers: isReadByOthers ?? this.isReadByOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `reactions: reactions ?? this.reactions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `replyToMessageId: replyToMessageId ?? this.replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `replyTo: replyTo ?? this.replyTo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `factory ChatMessage.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `final profile = json['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `final senderId = json['sender_id'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `final reads = json['message_reads'] as List?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `final isReadByOthers =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `reads?.any((r) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `if (r is! Map) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `final readerId = r['user_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `return readerId != null && readerId != senderId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `}) ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `final reactionRows = json['message_reactions'] as List?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `final reactions =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `reactionRows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `?.whereType<Map>()`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `.map((r) => ChatReaction.fromJson(Map<String, dynamic>.from(r)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L242** `.toList() ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `const <ChatReaction>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `final roleRaw = profile?['role'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `ChatReplyPreview? replyTo;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `final replyRaw = json['reply_to'];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `if (replyRaw is Map) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `replyTo = ChatReplyPreview.fromJson(Map<String, dynamic>.from(replyRaw));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L253** `return ChatMessage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `roomId: json['room_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `senderId: senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `messageType: chatMessageTypeFromString(json['message_type'] as String?),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `textContent: json['text_content'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `mediaUrl: json['media_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `createdAt: DateTime.parse(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `json['created_at'] as String? ?? DateTime.now().toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `updatedAt: DateTime.parse(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `json['updated_at'] as String? ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `json['created_at'] as String? ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `DateTime.now().toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `isDeleted: json['is_deleted'] as bool? ?? false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `senderName: profile?['full_name'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `senderUsername: profile?['username'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `senderAvatarUrl: profile?['avatar_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `senderRole: roleRaw != null ? userRoleFromString(roleRaw) : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `isReadByOthers: isReadByOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `reactions: reactions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `replyToMessageId: json['reply_to_message_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `replyTo: replyTo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `class ChatReplyPreview {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L282** `const ChatReplyPreview({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `required this.messageType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `this.textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `this.mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `this.isDeleted = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `required this.senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `this.senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `this.senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `this.senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `final ChatMessageType messageType;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `final String? textContent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `final String? mediaUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `final bool isDeleted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `final String senderId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `final String? senderName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `final String? senderUsername;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `final String? senderAvatarUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `String get displayName {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `final name = senderName?.trim() ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `if (name.isNotEmpty) return name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `final user = senderUsername?.trim() ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `if (user.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `return user.startsWith('@') ? user.substring(1) : user;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `return '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `Map<String, dynamic> toCacheJson() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `'id': id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `'message_type': messageType.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `'text_content': textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `'media_url': mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `'is_deleted': isDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `'sender_id': senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `'profiles': {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `'full_name': senderName,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `'username': senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `'avatar_url': senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `factory ChatReplyPreview.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `final profile = json['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `return ChatReplyPreview(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `messageType: chatMessageTypeFromString(json['message_type'] as String?),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `textContent: json['text_content'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `mediaUrl: json['media_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `isDeleted: json['is_deleted'] as bool? ?? false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `senderId: json['sender_id'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `senderName: profile?['full_name'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `senderUsername: profile?['username'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `senderAvatarUrl: profile?['avatar_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `factory ChatReplyPreview.fromMessage(ChatMessage message) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `return ChatReplyPreview(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `id: message.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `messageType: message.messageType,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `textContent: message.textContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `mediaUrl: message.mediaUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `isDeleted: message.isDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `senderId: message.senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `senderName: message.senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `senderUsername: message.senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `senderAvatarUrl: message.senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `class ChatReaction {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L359** `const ChatReaction({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `required this.messageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `required this.userId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `required this.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `final String messageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `final String userId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `final String emoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `factory ChatReaction.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `return ChatReaction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `messageId: json['message_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `userId: json['user_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `emoji: json['emoji'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/models/chat.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/enums.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (enums) is one concrete piece of that layer.

**L1** `enum UserRole { student, teacher, admin }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L3** `enum Sphere { math, english }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L5** `enum AssignmentType { staticAssignment, homeworkPack }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L7** `enum AttendanceStatus { present, absent, late, excused }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L9** `enum SubmissionStatus { pending, submitted, reviewed }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L11** `enum ChatMessageType { text, voice, image, video, file }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L13** `ChatMessageType chatMessageTypeFromString(String? value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `return ChatMessageType.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `(e) => e.name == value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `orElse: () => ChatMessageType.text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `String chatMessageTypeToDb(ChatMessageType type) => type.name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `ChatMessageType chatMessageTypeForFileName(String fileName) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final lower = fileName.toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `const videos = ['.mp4', '.mov', '.webm', '.m4v', '.avi', '.mkv'];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `const images = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `'.jpg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `'.jpeg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `'.png',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `'.gif',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `'.webp',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `'.heic',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `'.heif',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `'.bmp',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `'.tif',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `'.tiff',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `if (videos.any(lower.endsWith)) return ChatMessageType.video;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (images.any(lower.endsWith)) return ChatMessageType.image;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return ChatMessageType.file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `enum MarkCategory {`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L43** `homework,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `classwork,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `quiz,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `test,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `exam,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `project,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `oral,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `other,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `extension MarkCategoryLabel on MarkCategory {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `String get label => switch (this) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `MarkCategory.homework => 'Homework',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `MarkCategory.classwork => 'Classwork',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `MarkCategory.quiz => 'Quiz',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `MarkCategory.test => 'Test',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `MarkCategory.exam => 'Exam',`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `MarkCategory.project => 'Project',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `MarkCategory.oral => 'Oral',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `MarkCategory.other => 'Other',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `MarkCategory markCategoryFromString(String? value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return MarkCategory.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `(e) => e.name == value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `orElse: () => MarkCategory.other,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `UserRole userRoleFromString(String value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return UserRole.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `(e) => e.name == value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `orElse: () => UserRole.student,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `Sphere? sphereFromString(String? value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `if (value == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `return Sphere.values.cast<Sphere?>().firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `(e) => e?.name == value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `orElse: () => null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `AssignmentType assignmentTypeFromString(String value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `if (value == 'homework_pack') return AssignmentType.homeworkPack;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `return AssignmentType.staticAssignment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `String assignmentTypeToDb(AssignmentType type) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `return type == AssignmentType.homeworkPack ? 'homework_pack' : 'static';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `AttendanceStatus attendanceFromString(String value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `return AttendanceStatus.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `(e) => e.name == value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `orElse: () => AttendanceStatus.present,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/models/enums.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/mark.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (mark) is one concrete piece of that layer.

**L3** `class Mark {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `const Mark({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required this.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.percentage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.category = MarkCategory.other,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.weight = 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.comment = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.submissionId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.markedOn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `this.studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `this.groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final String teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final double percentage;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final MarkCategory category;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final double weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final String comment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final String? submissionId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final DateTime markedOn;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final String? studentName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final String? groupTitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `bool get isFromSubmission => submissionId != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `factory Mark.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final student = json['students'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final profile = student?['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `return Mark(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `groupId: json['group_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `studentId: json['student_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `teacherId: json['teacher_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `title: json['title'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `percentage: double.tryParse('${json['percentage']}') ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `category: markCategoryFromString(json['category'] as String?),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `weight: double.tryParse('${json['weight']}') ?? 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `comment: json['comment'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `submissionId: json['submission_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `markedOn:`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `DateTime.tryParse(json['marked_on']?.toString() ?? '') ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `studentName:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `profile?['full_name'] as String? ?? profile?['username'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `groupTitle: (json['groups'] as Map?)?['title'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `Map<String, dynamic> toWrite() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `'student_id': studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `'teacher_id': teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `'category': category.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `'percentage': percentage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `'weight': weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `'comment': comment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `'marked_on': markedOn.toIso8601String().split('T').first,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `class GroupMember {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L75** `const GroupMember({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `required this.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `required this.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `required this.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `final String studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `final String fullName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `final String username;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `factory GroupMember.fromGroupStudentRow(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `final student = json['students'] as Map;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final profile = student['profiles'] as Map;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final username = profile['username'] as String? ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `return GroupMember(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `studentId: student['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `fullName: (profile['full_name'] as String?)?.isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `? profile['full_name'] as String`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `: username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `username: username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/models/mark.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

## Every line — lib/models/models.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (models) is one concrete piece of that layer.

> **CHECKPOINT — Closed lib/models/models.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/notification.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (notification) is one concrete piece of that layer.

**L1** `class AppNotification {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L2** `const AppNotification({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L3** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `required this.userId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `this.body = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `this.isRead = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `this.kind = 'system',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `this.actionPath,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.refId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String userId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String body;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final bool isRead;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String kind;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final String? actionPath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String? refId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final DateTime? createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `factory AppNotification.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `return AppNotification(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `userId: json['user_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `title: json['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `body: json['body'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `isRead: json['is_read'] as bool? ?? false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `kind: json['kind'] as String? ?? 'system',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `actionPath: json['action_path'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `refId: json['ref_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `createdAt: json['created_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `? DateTime.parse(json['created_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `class Announcement {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L43** `const Announcement({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `this.teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `this.description = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `this.imageUrls = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `this.fileUrls = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final String id;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final String? teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final String description;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final List<String> imageUrls;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final List<String> fileUrls;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final DateTime? createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `factory Announcement.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `List<String> urls(dynamic v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `(v as List<dynamic>? ?? []).map((e) => e.toString()).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return Announcement(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `teacherId: json['teacher_id'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `groupId: json['group_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `title: json['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `description: json['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `imageUrls: urls(json['image_urls']),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `fileUrls: urls(json['file_urls']),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `createdAt: json['created_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `? DateTime.parse(json['created_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/models/notification.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/models/profile.dart

JSON ↔ Dart domain types. Manual fromJson, not Freezed in this repo. This file (profile) is one concrete piece of that layer.

**L3** `class Profile {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `const Profile({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `required this.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `required this.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `this.avatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `this.isDisabled = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `this.preferredLanguage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `this.preferredTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.preferredStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String username;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final String fullName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final UserRole role;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String? avatarUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final bool isDisabled;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final String? preferredLanguage;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final String? preferredTheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final String? preferredStyle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `bool get hasChosenLanguage =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `preferredLanguage != null && preferredLanguage!.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `factory Profile.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `return Profile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `username: json['username'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `fullName: json['full_name'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `role: userRoleFromString(json['role'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `avatarUrl: json['avatar_url'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `isDisabled: json['is_disabled'] as bool? ?? false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `preferredLanguage: json['preferred_language'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `preferredTheme: json['preferred_theme'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `preferredStyle: json['preferred_style'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `Map<String, dynamic> toJson() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'full_name': fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'avatar_url': avatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `'preferred_language': preferredLanguage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `'preferred_theme': preferredTheme,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `'preferred_style': preferredStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `class TeacherProfile {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L56** `const TeacherProfile({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `required this.profileId,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `required this.sphere,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final String profileId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final Sphere sphere;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `factory TeacherProfile.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return TeacherProfile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `profileId: json['profile_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `sphere: sphereFromString(json['sphere'] as String) ?? Sphere.math,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `class StudentProfile {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L76** `const StudentProfile({required this.id, required this.profileId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `final String profileId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `factory StudentProfile.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `return StudentProfile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `profileId: json['profile_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `class StudyGroup {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L90** `const StudyGroup({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `required this.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `required this.teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `this.createdAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final String id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final String teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `final DateTime? createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `factory StudyGroup.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `return StudyGroup(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `id: json['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `teacherId: json['teacher_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `title: json['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `createdAt: json['created_at'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `? DateTime.parse(json['created_at'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/models/profile.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/admin/admin_providers.dart

Users, groups, analytics, academic years, mass generator. This file (admin providers) is one concrete piece of that layer.

**L9** `Future<int> _exactCount(String table) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final result = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `.from(table)`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L12** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L13** `.count(CountOption.exact);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `return result.count;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final adminPlatformStatsProvider = FutureProvider<Map<String, int>>((`
Riverpod binding. Name it after the value it exposes.

**L18** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final teachers = await _exactCount('teachers');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final students = await _exactCount('students');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final groups = await _exactCount('groups');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final assignments = await _exactCount('assignments');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final marks = await _exactCount('marks');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `'teachers': teachers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `'students': students,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `'groups': groups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `'assignments': assignments,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `'marks': marks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final adminGroupsProvider = FutureProvider<List<Map<String, dynamic>>>((`
Riverpod binding. Name it after the value it exposes.

**L35** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `.from('groups')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L39** `.select('id, title, teachers(sphere, profiles(full_name))')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L40** `.order('title');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `return rows.map((e) => Map<String, dynamic>.from(e)).toList();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L44** `final adminAnnouncementsProvider = FutureProvider<List<Map<String, dynamic>>>((`
Riverpod binding. Name it after the value it exposes.

**L45** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `.from('announcements')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L49** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L50** `.order('created_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `return rows.map((e) => Map<String, dynamic>.from(e)).toList();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L54** `final allProfilesProvider =`
Riverpod binding. Name it after the value it exposes.

**L55** `AsyncNotifierProvider<AllProfilesNotifier, PageResult<Profile>>(`
Riverpod binding. Name it after the value it exposes.

**L56** `AllProfilesNotifier.new,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `class AllProfilesNotifier extends AsyncNotifier<PageResult<Profile>> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L60** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `Future<PageResult<Profile>> build() => _fetch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Future<PageResult<Profile>> _fetch(int offset) async {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final limit = PageResult.fetchSize(PageResult.pageSize);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `.from('profiles')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L67** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L68** `.order('username')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `.range(offset, offset + limit - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `return PageResult.fromFetch(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `rows.map((e) => Profile.fromJson(Map<String, dynamic>.from(e))).toList(),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L75** `Future<void> loadMore() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `if (current == null || !current.hasMore || current.loadingMore) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `state = AsyncData(current.copyWith(loadingMore: true));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final next = await _fetch(current.items.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `state = AsyncData(current.append(next.items, hasMore: next.hasMore));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `state = AsyncData(current.copyWith(loadingMore: false));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/providers/admin/admin_providers.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/attendance/attendance_providers.dart

Lesson sessions + attendance upsert. This file (attendance providers) is one concrete piece of that layer.

**L7** `String dateKey(DateTime date) => DateTime(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `date.year,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `date.month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `date.day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `).toIso8601String().split('T').first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `DateTime calendarDay(DateTime date) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `DateTime(date.year, date.month, date.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `bool isAttendanceWeekend(DateTime date) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final weekday = calendarDay(date).weekday;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `return weekday == DateTime.saturday || weekday == DateTime.sunday;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `bool isAttendanceDateAllowed(DateTime date) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final day = calendarDay(date);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final today = calendarDay(DateTime.now());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `if (day.isAfter(today)) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `if (isAttendanceWeekend(day)) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `class AttendanceFutureDateException implements Exception {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L30** `const AttendanceFutureDateException();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `String toString() => 'AttendanceFutureDateException';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `class AttendanceWeekendException implements Exception {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L37** `const AttendanceWeekendException();`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `String toString() => 'AttendanceWeekendException';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `Never _rejectAttendanceDate(DateTime date) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final day = calendarDay(date);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `if (isAttendanceWeekend(day)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `throw const AttendanceWeekendException();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final today = calendarDay(DateTime.now());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `if (day.isAfter(today)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `throw const AttendanceFutureDateException();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `throw StateError('Attendance date rejected unexpectedly: $day');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `class AttendanceQuery {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L56** `const AttendanceQuery({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `this.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `required this.from,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `required this.to,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final String? studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final DateTime from;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final DateTime to;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `bool operator ==(Object other) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `other is AttendanceQuery &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `other.groupId == groupId &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `other.studentId == studentId &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `other.from == from &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `other.to == to;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `int get hashCode => Object.hash(groupId, studentId, from, to);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final attendanceRangeProvider =`
Riverpod binding. Name it after the value it exposes.

**L81** `FutureProvider.family<List<AttendanceRecord>, AttendanceQuery>((`
Riverpod binding. Name it after the value it exposes.

**L82** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `query,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `var filtered = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `.from('attendance')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L87** `.select('*, students(profiles(full_name))')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L88** `.eq('group_id', query.groupId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L89** `.gte('date', dateKey(query.from))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `.lte('date', dateKey(query.to));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `if (query.studentId != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `filtered = filtered.eq('student_id', query.studentId!);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L96** `final rows = await filtered.order('date', ascending: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `.map((e) => AttendanceRecord.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L99** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final attendanceRangeStatsProvider =`
Riverpod binding. Name it after the value it exposes.

**L103** `Provider.family<AsyncValue<AttendanceStats>, AttendanceQuery>((ref, query) {`
Riverpod binding. Name it after the value it exposes.

**L104** `return ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `.watch(attendanceRangeProvider(query))`
Riverpod binding. Name it after the value it exposes.

**L106** `.whenData(AttendanceStats.fromRecords);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final todayAttendanceProvider =`
Riverpod binding. Name it after the value it exposes.

**L110** `FutureProvider.family<Map<String, AttendanceStatus>, String>((`
Riverpod binding. Name it after the value it exposes.

**L111** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `.from('attendance')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L116** `.select('student_id, status')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L117** `.eq('group_id', groupId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L118** `.eq('date', dateKey(DateTime.now()));`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L119** `return {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `for (final row in rows)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `row['student_id'] as String: attendanceFromString(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `row['status'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `class AttendanceWeekBucket {`

Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L128** `const AttendanceWeekBucket({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `this.present = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `this.absent = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `this.late = 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `final int present;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `final int absent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `final int late;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `int get max => [present, absent, late].fold(0, (a, b) => a > b ? a : b);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `class AttendanceWeekTrend {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L142** `const AttendanceWeekTrend({required this.thisWeek, required this.lastWeek});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final AttendanceWeekBucket thisWeek;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final AttendanceWeekBucket lastWeek;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `bool get isEmpty => thisWeek.max == 0 && lastWeek.max == 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final attendanceWeekTrendProvider =`
Riverpod binding. Name it after the value it exposes.

**L151** `FutureProvider.family<AttendanceWeekTrend, String>((ref, groupId) async {`
Riverpod binding. Name it after the value it exposes.

**L152** `final today = calendarDay(DateTime.now());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `final thisFrom = today.subtract(const Duration(days: 6));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final lastTo = today.subtract(const Duration(days: 7));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `final lastFrom = today.subtract(const Duration(days: 13));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `.from('attendance')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L159** `.select('status, date')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L160** `.eq('group_id', groupId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L161** `.gte('date', dateKey(lastFrom))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `.lte('date', dateKey(today));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `var thisPresent = 0, thisAbsent = 0, thisLate = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `var lastPresent = 0, lastAbsent = 0, lastLate = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `for (final row in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `final day = calendarDay(DateTime.parse(row['date'] as String));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `final status = row['status'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `final inThisWeek = !day.isBefore(thisFrom) && !day.isAfter(today);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `final inLastWeek = !day.isBefore(lastFrom) && !day.isAfter(lastTo);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `if (inThisWeek) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `switch (status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `case 'present':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `thisPresent++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `case 'absent':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `thisAbsent++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `case 'late':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `thisLate++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `} else if (inLastWeek) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `switch (status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `case 'present':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `lastPresent++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `case 'absent':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `lastAbsent++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `case 'late':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `lastLate++;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `return AttendanceWeekTrend(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `thisWeek: AttendanceWeekBucket(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `present: thisPresent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `absent: thisAbsent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `late: thisLate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `lastWeek: AttendanceWeekBucket(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `present: lastPresent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `absent: lastAbsent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `late: lastLate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Future<void> upsertAttendance({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `required AttendanceStatus status,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `required DateTime date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `if (!isAttendanceDateAllowed(date)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `_rejectAttendanceDate(date);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `final key = dateKey(date);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L219** `'ensure_lesson_session',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `params: {'p_group_id': groupId, 'p_session_date': key},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `await supabase.from('attendance').upsert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L223** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `'student_id': studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `'status': status.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `'date': key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `}, onConflict: 'group_id,student_id,date');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `Future<void> upsertAttendanceBulk({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `required DateTime date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `required Map<String, AttendanceStatus> byStudentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `if (byStudentId.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `if (!isAttendanceDateAllowed(date)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `_rejectAttendanceDate(date);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `final key = dateKey(date);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L241** `'ensure_lesson_session',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `params: {'p_group_id': groupId, 'p_session_date': key},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `final rows = byStudentId.entries`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `(e) => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `'student_id': e.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `'status': e.value.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `'date': key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `.from('attendance')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L256** `.upsert(rows, onConflict: 'group_id,student_id,date');`
Insert-or-update. Submissions and attendance use this so retrying is safe.

> **CHECKPOINT — Closed lib/providers/attendance/attendance_providers.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/auth/auth_provider.dart

Session stream, username login RPC, profile/teacher/student loaders. This file (auth provider) is one concrete piece of that layer.

**L8** `final authStateProvider = StreamProvider<AuthState>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L9** `return supabase.auth.onAuthStateChange;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `class AuthRepository {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `AuthRepository(this._cache);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final CacheService? _cache;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `Future<String?> resolveAuthEmail(String username) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final normalized = username.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final result = await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L20** `'get_auth_email_by_username',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `params: {'p_username': normalized},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `return result as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `Future<void> signIn(String username, String password) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final email = await resolveAuthEmail(username);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (email == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `throw const AuthException('Incorrect username or password');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `await supabase.auth.signInWithPassword(email: email, password: password);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `Future<void> changePassword({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `required String username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `required String currentPassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `required String newPassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `await signIn(username, currentPassword);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `await supabase.auth.updateUser(UserAttributes(password: newPassword));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `Future<void> signOut() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `await supabase.auth.signOut();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `await clearAllAppCaches(_cache);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `User? get currentUser => supabase.auth.currentUser;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final authRepositoryProvider = Provider<AuthRepository>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L52** `return AuthRepository(ref.watch(cacheServiceProvider).valueOrNull);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L55** `class ProfileRepository {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L56** `ProfileRepository(this._cache);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final CacheService? _cache;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `Future<Profile?> fetchCurrentProfile() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final uid = supabase.auth.currentUser?.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `if (uid == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `final data = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `.from('profiles')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L67** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L68** `.eq('id', uid)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L69** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `if (data == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final profile = Profile.fromJson(Map<String, dynamic>.from(data));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L72** `await _cache?.cacheProfile(data);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `return profile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `final cached = _cache?.cachedProfile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `if (cached != null) return Profile.fromJson(cached);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `Future<void> updateProfile(String id, Map<String, dynamic> updates) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `await supabase.from('profiles').update(updates).eq('id', id);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L86** `final profileRepositoryProvider = Provider<ProfileRepository>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L87** `final cacheAsync = ref.watch(cacheServiceProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L88** `return ProfileRepository(cacheAsync.valueOrNull);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final currentProfileProvider = FutureProvider<Profile?>((ref) async {`
Riverpod binding. Name it after the value it exposes.

**L92** `ref.watch(authStateProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L93** `final repo = ref.watch(profileRepositoryProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L94** `return repo.fetchCurrentProfile();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final teacherProfileProvider = FutureProvider<TeacherProfile?>((ref) async {`
Riverpod binding. Name it after the value it exposes.

**L98** `final profile = await ref.watch(currentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L99** `if (profile == null || profile.role != UserRole.teacher) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `final data = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `.from('teachers')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L102** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L103** `.eq('profile_id', profile.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L104** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `if (data == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `return TeacherProfile.fromJson(Map<String, dynamic>.from(data));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L109** `final studentProfileProvider = FutureProvider<StudentProfile?>((ref) async {`
Riverpod binding. Name it after the value it exposes.

**L110** `final profile = await ref.watch(currentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L111** `if (profile == null || profile.role != UserRole.student) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final data = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `.from('students')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L114** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L115** `.eq('profile_id', profile.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L116** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `if (data == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `return StudentProfile.fromJson(Map<String, dynamic>.from(data));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

> **CHECKPOINT — Closed lib/providers/auth/auth_provider.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/chat/chat_providers.dart

Room summaries, unread RPC, messages, reactions. This file (chat providers) is one concrete piece of that layer.

**L15** `const chatSignedUrlTtlSeconds = 60 * 60 * 24 * 365 * 10;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `const _pageSize = 40;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `const _chatMessageSelect =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `'*, profiles(full_name, avatar_url, username, role), message_reads(user_id), message_reactions(id, message_id, user_id, emoji), reply_to:reply_to_message_id(id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `const chatReactionEmojis = ['❤■', '■', '■', '■', '■', '■'];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `String chatBucketForType(ChatMessageType type) => switch (type) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `ChatMessageType.image => 'chat-images',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `ChatMessageType.video => 'chat-videos',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `ChatMessageType.voice => 'chat-voices',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `ChatMessageType.file => 'chat-files',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `ChatMessageType.text => 'chat-images',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `String chatContentType(ChatMessageType type, String fileName) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final lower = fileName.toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `return switch (type) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `ChatMessageType.image =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `lower.endsWith('.png')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `? 'image/png'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `: lower.endsWith('.webp')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `? 'image/webp'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `: lower.endsWith('.gif')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `? 'image/gif'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `: 'image/jpeg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `ChatMessageType.video =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `lower.endsWith('.webm')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `? 'video/webm'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `: lower.endsWith('.mov')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `? 'video/quicktime'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `: 'video/mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `ChatMessageType.voice =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `lower.endsWith('.wav')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `? 'audio/wav'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `: lower.endsWith('.mp3')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `? 'audio/mpeg'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `: 'audio/mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `ChatMessageType.file => _fileContentType(lower),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `ChatMessageType.text => 'text/plain',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `String _fileContentType(String lowerName) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `if (lowerName.endsWith('.pdf')) return 'application/pdf';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `if (lowerName.endsWith('.zip')) return 'application/zip';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `if (lowerName.endsWith('.json')) return 'application/json';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `if (lowerName.endsWith('.csv')) return 'text/csv';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `if (lowerName.endsWith('.txt') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `lowerName.endsWith('.md') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `lowerName.endsWith('.py') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `lowerName.endsWith('.dart') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `lowerName.endsWith('.js') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `lowerName.endsWith('.ts') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `lowerName.endsWith('.html') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `lowerName.endsWith('.css') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `lowerName.endsWith('.sql')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `return 'text/plain';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (lowerName.endsWith('.doc') || lowerName.endsWith('.docx')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `return 'application/msword';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `if (lowerName.endsWith('.xls') || lowerName.endsWith('.xlsx')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `return 'application/vnd.ms-excel';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `if (lowerName.endsWith('.ppt') || lowerName.endsWith('.pptx')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `return 'application/vnd.ms-powerpoint';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `return 'application/octet-stream';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `String chatVoiceContentType(Uint8List bytes, String fileName) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `return detectAudioFormat(bytes).mimeType;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `String chatVoiceFileName(Uint8List bytes, String fallbackName) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final format = detectAudioFormat(bytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final stamp = DateTime.now().millisecondsSinceEpoch;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final base = fallbackName.replaceAll(RegExp(r'[^\w.\-]+'), '_');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `if (base.toLowerCase().endsWith('.${format.extension}')) return base;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final withoutExt = base.contains('.')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `? base.substring(0, base.lastIndexOf('.'))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `: base;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final safe = withoutExt.isEmpty ? 'chat_voice_$stamp' : withoutExt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `return '$safe.${format.extension}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final chatRoomsProvider = StreamProvider<List<ChatRoomSummary>>((ref) async* {`
Riverpod binding. Name it after the value it exposes.

**L103** `final profile = await ref.watch(currentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L104** `if (profile == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `yield const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `Future<List<ChatRoomSummary>> load() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `.from('chat_room_summaries')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L112** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L113** `.order('last_message_at', ascending: false, nullsFirst: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final unreadRows =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `await supabase.rpc('chat_unread_counts') as List<dynamic>? ?? [];`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L117** `final unreadMap = <String, int>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `for (final row in unreadRows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `final map = Map<String, dynamic>.from(row as Map);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L120** `final roomId = map['room_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `if (roomId == null) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `unreadMap[roomId] = (map['unread_count'] as num?)?.toInt() ?? 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `final rooms = (rows as List)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `.map((e) => ChatRoomSummary.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L127** `.map((r) => r.copyWith(unreadCount: unreadMap[r.id] ?? 0))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `rooms.sort((a, b) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `final aTime =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `a.lastMessageAt ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `a.createdAt ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `DateTime.fromMillisecondsSinceEpoch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `final bTime =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `b.lastMessageAt ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `b.createdAt ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `DateTime.fromMillisecondsSinceEpoch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `return bTime.compareTo(aTime);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `return rooms;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final changes = StreamController<void>();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final channel = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `.channel('chat-rooms-${profile.id}')`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L147** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L148** `event: PostgresChangeEvent.all,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `table: 'chat_messages',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `callback: (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `if (!changes.isClosed) changes.add(null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L156** `event: PostgresChangeEvent.all,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `table: 'message_reads',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `callback: (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `if (!changes.isClosed) changes.add(null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `.subscribe();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `ref.onDispose(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `supabase.removeChannel(channel);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `changes.close();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `yield await load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `await for (final _ in changes.stream) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `yield await load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final unreadChatCountProvider = Provider<int>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L177** `final rooms = ref.watch(chatRoomsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L178** `return rooms.maybeWhen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `data: (list) => list.fold<int>(0, (sum, r) => sum + r.unreadCount),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `orElse: () => 0,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `final chatRoomProvider = FutureProvider.family<ChatRoomSummary?, String>((`
Riverpod binding. Name it after the value it exposes.

**L185** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `final rooms = ref.watch(chatRoomsProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L189** `if (rooms != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `for (final room in rooms) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `if (room.id == roomId) return room;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final row = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `.from('chat_room_summaries')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L196** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L197** `.eq('id', roomId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L198** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `if (row == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `return ChatRoomSummary.fromJson(Map<String, dynamic>.from(row));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L203** `class ChatMessagesState {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L204** `const ChatMessagesState({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `required this.messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `this.hasMore = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `this.loadingMore = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `final List<ChatMessage> messages;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `final bool hasMore;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `final bool loadingMore;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `ChatMessagesState copyWith({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `List<ChatMessage>? messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `bool? hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `bool? loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `return ChatMessagesState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `messages: messages ?? this.messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `hasMore: hasMore ?? this.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `loadingMore: loadingMore ?? this.loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `class ChatMessagesNotifier`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L228** `extends FamilyAsyncNotifier<ChatMessagesState, String> {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `RealtimeChannel? _channel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `final _profileCache = <String, Map<String, dynamic>>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `Future<ChatMessagesState> build(String roomId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `final profile = await ref.watch(currentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L235** `if (profile == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `return const ChatMessagesState(messages: []);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `final cache = ref.read(cacheServiceProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L240** `List<ChatMessage>? cachedMessages;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `final raw = cache?.cachedChatMessages(roomId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `if (raw != null && raw.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `cachedMessages = raw.map(ChatMessage.fromJson).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `final initial = await _fetchPage(roomId, before: null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `await _persistMessages(roomId, initial.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `prefetchChatMedia(initial.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `_channel = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `.channel('chat-room-$roomId')`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L255** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L256** `event: PostgresChangeEvent.insert,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `table: 'chat_messages',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `filter: PostgresChangeFilter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `type: PostgresChangeFilterType.eq,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `column: 'room_id',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `value: roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `callback: (payload) => _onInsert(payload.newRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L267** `event: PostgresChangeEvent.update,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `table: 'chat_messages',`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `filter: PostgresChangeFilter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `type: PostgresChangeFilterType.eq,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `column: 'room_id',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `value: roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `callback: (payload) => _onUpdate(payload.newRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L278** `event: PostgresChangeEvent.delete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `table: 'chat_messages',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `filter: PostgresChangeFilter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `type: PostgresChangeFilterType.eq,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `column: 'room_id',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `value: roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `callback: (payload) => _onDelete(payload.oldRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L289** `event: PostgresChangeEvent.insert,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `table: 'message_reads',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `callback: (payload) => _onReadInsert(payload.newRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L295** `event: PostgresChangeEvent.insert,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `table: 'message_reactions',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `callback: (payload) => _onReactionUpsert(payload.newRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L301** `event: PostgresChangeEvent.update,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `table: 'message_reactions',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `callback: (payload) => _onReactionUpsert(payload.newRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `.onPostgresChanges(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L307** `event: PostgresChangeEvent.delete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `table: 'message_reactions',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `callback: (payload) => _onReactionDelete(payload.oldRecord),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `.subscribe();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `ref.onDispose(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `final ch = _channel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `if (ch != null) supabase.removeChannel(ch);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `return ChatMessagesState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `messages: initial.items,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `hasMore: initial.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `if (cachedMessages != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `prefetchChatMedia(cachedMessages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `return ChatMessagesState(messages: cachedMessages, hasMore: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `Future<void> _persistMessages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `String roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `List<ChatMessage> messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `final cache = ref.read(cacheServiceProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L337** `if (cache == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `await cache.cacheChatMessages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `messages.map((m) => m.toCacheJson()).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `Future<void> _persistCurrent() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `await _persistMessages(arg, current.messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `Future<({List<ChatMessage> items, bool hasMore})> _fetchPage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `String roomId, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `required DateTime? before,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `var filtered = supabase`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `.from('chat_messages')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L358** `.select(_chatMessageSelect)`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L359** `.eq('room_id', roomId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L361** `if (before != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `filtered = filtered.lt('created_at', before.toIso8601String());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `final rows = await filtered`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `.order('created_at', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `.limit(_pageSize + 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `final list = (rows as List)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `.map((e) => ChatMessage.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L370** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `final hasMore = list.length > _pageSize;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `final page = hasMore ? list.sublist(0, _pageSize) : list;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `return (items: page.reversed.toList(), hasMore: hasMore);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `Future<void> loadMore() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `if (current == null || !current.hasMore || current.loadingMore) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `if (current.messages.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `state = AsyncData(current.copyWith(loadingMore: true));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `final oldest = current.messages.first.createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `final page = await _fetchPage(arg, before: oldest);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `final merged = [...page.items, ...current.messages];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `ChatMessagesState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `messages: merged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `hasMore: page.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `loadingMore: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `await _persistMessages(arg, merged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `prefetchChatMedia(page.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `state = AsyncData(current.copyWith(loadingMore: false));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `Future<void> _hydrateSender(Map<String, dynamic> row) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `final senderId = row['sender_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `if (senderId == null || row['profiles'] != null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `final cached = _profileCache[senderId];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `if (cached != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `row['profiles'] = cached;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `final profileRow = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `.from('profiles')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L410** `.select('full_name, avatar_url, username, role')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L411** `.eq('id', senderId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L412** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `if (profileRow != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `final map = Map<String, dynamic>.from(profileRow);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L415** `_profileCache[senderId] = map;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `row['profiles'] = map;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `Future<void> _hydrateReply(Map<String, dynamic> row) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `final replyId = row['reply_to_message_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `if (replyId == null || row['reply_to'] != null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `final local = current?.messages.where((m) => m.id == replyId).firstOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `if (local != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `row['reply_to'] = ChatReplyPreview.fromMessage(local).toCacheJson();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `final data = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `.from('chat_messages')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L434** `.select(`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L435** `'id, message_type, text_content, media_url, is_deleted, sender_id, profiles(full_name, username, avatar_url)',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `.eq('id', replyId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L438** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `if (data != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `row['reply_to'] = Map<String, dynamic>.from(data);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L442** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `Future<void> _onInsert(Map<String, dynamic> record) async {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `final row = Map<String, dynamic>.from(record);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L447** `await _hydrateSender(row);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `await _hydrateReply(row);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `final message = ChatMessage.fromJson(row);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `if (current.messages.any((m) => m.id == message.id)) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `current.copyWith(messages: [...current.messages, message]),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `prefetchChatMedia([message]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `await _persistCurrent();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `Future<void> _onUpdate(Map<String, dynamic> record) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `final row = Map<String, dynamic>.from(record);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L462** `await _hydrateSender(row);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `final updated = ChatMessage.fromJson(row);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `final next = current.messages.map((m) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `if (m.id != updated.id) return m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `return updated.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `senderName: updated.senderName ?? m.senderName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `senderUsername: updated.senderUsername ?? m.senderUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `senderAvatarUrl: updated.senderAvatarUrl ?? m.senderAvatarUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `senderRole: updated.senderRole ?? m.senderRole,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `isReadByOthers: m.isReadByOthers || updated.isReadByOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `reactions: updated.reactions.isEmpty ? m.reactions : updated.reactions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `replyToMessageId: updated.replyToMessageId ?? m.replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `replyTo: updated.replyTo ?? m.replyTo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `}).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `state = AsyncData(current.copyWith(messages: next));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `await _persistCurrent();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `void _onDelete(Map<String, dynamic> record) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `final id = record['id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `if (id == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `current.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `messages: current.messages.where((m) => m.id != id).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `unawaited(_persistCurrent());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `void _onReadInsert(Map<String, dynamic> record) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `final messageId = record['message_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `final readerId = record['user_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `if (messageId == null || readerId == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `var changed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `final next = current.messages.map((m) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `if (m.id != messageId || m.isReadByOthers) return m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `if (readerId == m.senderId) return m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `changed = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `return m.copyWith(isReadByOthers: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `}).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `if (changed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `state = AsyncData(current.copyWith(messages: next));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `void _onReactionUpsert(Map<String, dynamic> record) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `final reaction = ChatReaction.fromJson(Map<String, dynamic>.from(record));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L518** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `if (!current.messages.any((m) => m.id == reaction.messageId)) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `final next = current.messages.map((m) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `if (m.id != reaction.messageId) return m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `final withoutUser = m.reactions`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `.where((r) => r.userId != reaction.userId)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `return m.copyWith(reactions: [...withoutUser, reaction]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `}).toList();`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `state = AsyncData(current.copyWith(messages: next));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `void _onReactionDelete(Map<String, dynamic> record) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `final id = record['id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `final messageId = record['message_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `final userId = record['user_id'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `if (current == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `final next = current.messages.map((m) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `if (messageId != null && m.id != messageId) return m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `if (messageId == null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `id != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `!m.reactions.any((r) => r.id == id)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `return m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `return m.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `reactions: m.reactions.where((r) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `if (id != null && r.id == id) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `if (userId != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `messageId != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `r.userId == userId &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `r.messageId == messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `}).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `}).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `state = AsyncData(current.copyWith(messages: next));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `Future<ChatMessage> sendText(String text, {String? replyToMessageId}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `final profile = await ref.read(currentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L564** `if (profile == null) throw StateError('Not signed in');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `final trimmed = text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `if (trimmed.isEmpty) throw ArgumentError('Empty message');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `final row = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `.from('chat_messages')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L570** `.insert({`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L571** `'room_id': arg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `'sender_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `'message_type': 'text',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `'text_content': trimmed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `'reply_to_message_id': ?replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `.select(_chatMessageSelect)`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L578** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L581** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `if (current != null && !current.messages.any((m) => m.id == message.id)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `current.copyWith(messages: [...current.messages, message]),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `await _persistCurrent();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `return message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L591** `Future<ChatMessage> sendMedia({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L592** `required ChatMessageType type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `required Uint8List bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `required String fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `String? caption,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `String? replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `void Function(double progress)? onProgress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `final profile = await ref.read(currentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L600** `if (profile == null) throw StateError('Not signed in');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `if (type == ChatMessageType.text) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `throw ArgumentError('Use sendText for text messages');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `onProgress?.call(0.05);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `final stamp = DateTime.now().millisecondsSinceEpoch;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `final safeName = type == ChatMessageType.voice`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `? chatVoiceFileName(bytes, fileName)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `: fileName.replaceAll(RegExp(r'[^\w.\-]+'), '_');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `final path = '$arg/${profile.id}/${stamp}_$safeName';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `final bucket = chatBucketForType(type);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `final contentType = type == ChatMessageType.voice`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `? chatVoiceContentType(bytes, fileName)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `: chatContentType(type, fileName);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `.from(bucket)`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L618** `.uploadBinary(`
Storage write. Path folders must satisfy bucket policies.

**L619** `path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `fileOptions: FileOptions(upsert: false, contentType: contentType),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `onProgress?.call(0.7);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `final url = await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `.from(bucket)`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L627** `.createSignedUrl(path, chatSignedUrlTtlSeconds);`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L628** `onProgress?.call(0.85);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `if (type == ChatMessageType.voice) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `final format = detectAudioFormat(bytes);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `await VoiceCacheService.instance.put(url, bytes, format.extension);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `final ext = ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `fallback: safeName.contains('.') ? safeName.split('.').last : 'bin',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `await ChatMediaCacheService.instance.put(url, bytes, ext);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `final captionText = caption?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `final row = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `.from('chat_messages')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L644** `.insert({`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L645** `'room_id': arg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `'sender_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `'message_type': chatMessageTypeToDb(type),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L648** `'text_content': (captionText == null || captionText.isEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `: captionText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `'media_url': url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `'reply_to_message_id': ?replyToMessageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L654** `.select(_chatMessageSelect)`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L655** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `onProgress?.call(1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `final message = ChatMessage.fromJson(Map<String, dynamic>.from(row));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L659** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `if (current != null && !current.messages.any((m) => m.id == message.id)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `current.copyWith(messages: [...current.messages, message]),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L664** `await _persistCurrent();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `return message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `Future<void> editMessage(String messageId, String newText) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `final trimmed = newText.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L671** `if (trimmed.isEmpty) throw ArgumentError('Empty message');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L673** `.from('chat_messages')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L674** `.update({'text_content': trimmed})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L675** `.eq('id', messageId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L678** `Future<void> deleteMessage(String messageId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L679** `await deleteMessages([messageId]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L682** `Future<void> deleteMessages(Iterable<String> messageIds) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L683** `final ids = messageIds.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L684** `if (ids.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L686** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L687** `if (current != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L688** `final idSet = ids.toSet();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L689** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L690** `current.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L691** `messages: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L692** `for (final m in current.messages)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L693** `if (idSet.contains(m.id))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L694** `m.copyWith(isDeleted: true)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L696** `m,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L700** `await _persistCurrent();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L703** `await supabase.from('chat_messages').update({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L704** `'is_deleted': true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L705** `'text_content': null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L706** `'media_url': null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `}).inFilter('id', ids);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `Future<void> toggleReaction(String messageId, String emoji) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L711** `final profile = await ref.read(currentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L712** `if (profile == null) throw StateError('Not signed in');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L714** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L715** `ChatMessage? message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L716** `if (current != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L717** `for (final m in current.messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L718** `if (m.id == messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L719** `message = m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L720** `break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L724** `final existing = message?.myReactionEmoji(profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L726** `if (current != null && message != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L727** `final nextReactions = <ChatReaction>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L728** `for (final r in message.reactions)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L729** `if (r.userId != profile.id) r,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L731** `if (existing != emoji) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L732** `nextReactions.add(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L733** `ChatReaction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L734** `id: 'local-${profile.id}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L735** `messageId: messageId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L736** `userId: profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L737** `emoji: emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L738** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L741** `state = AsyncData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L742** `current.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L743** `messages: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L744** `for (final m in current.messages)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L745** `if (m.id == messageId)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L746** `m.copyWith(reactions: nextReactions)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L747** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L748** `m,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L750** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L754** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L755** `'toggle_message_reaction',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L756** `params: {'p_message_id': messageId, 'p_emoji': emoji},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L760** `Future<void> markRoomRead() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L761** `final profile = await ref.read(currentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L762** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L764** `final unread = await supabase`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L765** `.from('chat_messages')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L766** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L767** `.eq('room_id', arg)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L768** `.eq('is_deleted', false)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L769** `.neq('sender_id', profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L771** `final ids = (unread as List)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L772** `.map((e) => (e as Map)['id'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L773** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L774** `if (ids.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L776** `final existing = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L777** `.from('message_reads')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L778** `.select('message_id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L779** `.eq('user_id', profile.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L780** `.inFilter('message_id', ids);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L781** `final already = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L782** `for (final e in existing as List) (e as Map)['message_id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L783** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L784** `final toInsert = ids`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L785** `.where((id) => !already.contains(id))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L786** `.map((id) => {'message_id': id, 'user_id': profile.id})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L787** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L788** `if (toInsert.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L790** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L791** `.from('message_reads')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L792** `.upsert(toInsert, onConflict: 'message_id,user_id');`
Insert-or-update. Submissions and attendance use this so retrying is safe.

**L793** `ref.invalidate(chatRoomsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L797** `final chatMessagesProvider =`
Riverpod binding. Name it after the value it exposes.

**L798** `AsyncNotifierProvider.family<`
Riverpod binding. Name it after the value it exposes.

**L799** `ChatMessagesNotifier,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L800** `ChatMessagesState,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L801** `String`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L802** `>(ChatMessagesNotifier.new);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/providers/chat/chat_providers.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/focus/focus_session_provider.dart

Local Pomodoro/free timer persisted in SharedPreferences. This file (focus session provider) is one concrete piece of that layer.

**L8** `const _prefsKey = 'focus_session_v1';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `enum FocusTimerMode { pomodoro, free }`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L12** `const kFocusPomodoroDuration = Duration(minutes: 25);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `class FocusSession {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L15** `const FocusSession({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `required this.assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required this.mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `required this.startedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `this.accumulated = Duration.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `this.pausedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final String assignmentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final String assignmentTitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final FocusTimerMode mode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final DateTime startedAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final Duration accumulated;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final DateTime? pausedAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `bool get isPaused => pausedAt != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `Duration elapsedAt([DateTime? now]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final n = now ?? DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (isPaused) return accumulated;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `return accumulated + n.difference(startedAt);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `Duration? remainingAt([DateTime? now]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `if (mode != FocusTimerMode.pomodoro) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `final left = kFocusPomodoroDuration - elapsedAt(now);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `return left.isNegative ? Duration.zero : left;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `bool isCompleteAt([DateTime? now]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final left = remainingAt(now);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `return left != null && left == Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `Map<String, dynamic> toJson() => {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `'assignmentId': assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `'assignmentTitle': assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `'mode': mode.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `'startedAt': startedAt.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `'accumulatedMs': accumulated.inMilliseconds,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `'pausedAt': pausedAt?.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `factory FocusSession.fromJson(Map<String, dynamic> json) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `return FocusSession(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `assignmentId: json['assignmentId'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `assignmentTitle: json['assignmentTitle'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `mode: FocusTimerMode.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `(m) => m.name == json['mode'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `orElse: () => FocusTimerMode.free,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `startedAt: DateTime.parse(json['startedAt'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `accumulated: Duration(milliseconds: json['accumulatedMs'] as int? ?? 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `pausedAt: json['pausedAt'] != null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `? DateTime.parse(json['pausedAt'] as String)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `FocusSession copyWith({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `DateTime? startedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `Duration? accumulated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `DateTime? pausedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `bool clearPausedAt = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `return FocusSession(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `assignmentId: assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `assignmentTitle: assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `mode: mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `startedAt: startedAt ?? this.startedAt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `accumulated: accumulated ?? this.accumulated,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `pausedAt: clearPausedAt ? null : (pausedAt ?? this.pausedAt),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `class FocusSummary {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L94** `const FocusSummary({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `required this.assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `required this.assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `required this.focused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `required this.mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `this.note,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `this.timedOut = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `final String assignmentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final String assignmentTitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final Duration focused;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final FocusTimerMode mode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final String? note;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final bool timedOut;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `FocusSummary copyWith({String? note}) => FocusSummary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `assignmentId: assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `assignmentTitle: assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `focused: focused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `mode: mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `note: note ?? this.note,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `timedOut: timedOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `class FocusSessionState {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L122** `const FocusSessionState({this.session, this.summary});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `final FocusSession? session;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `final FocusSummary? summary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `bool get isActive => session != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `bool get showingSummary => summary != null && session == null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `FocusSessionState copyWith({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `FocusSession? session,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `FocusSummary? summary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `bool clearSession = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `bool clearSummary = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `return FocusSessionState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `session: clearSession ? null : (session ?? this.session),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `summary: clearSummary ? null : (summary ?? this.summary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `final focusSessionProvider =`
Riverpod binding. Name it after the value it exposes.

**L144** `StateNotifierProvider<FocusSessionController, FocusSessionState>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L145** `return FocusSessionController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `final focusChatMutedProvider = Provider<bool>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L149** `return ref.watch(focusSessionProvider).isActive;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L152** `class FocusSessionController extends StateNotifier<FocusSessionState> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L153** `FocusSessionController() : super(const FocusSessionState()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `_restore();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `Future<void> _restore() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final raw = prefs.getString(_prefsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `if (raw == null || raw.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `final map = jsonDecode(raw) as Map<String, dynamic>;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `final session = FocusSession.fromJson(map);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `if (session.isCompleteAt()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `await endSession(timedOut: true, playCue: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `state = FocusSessionState(session: session);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `await prefs.remove(_prefsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `Future<void> _persist(FocusSession? session) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `if (session == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `await prefs.remove(_prefsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `await prefs.setString(_prefsKey, jsonEncode(session.toJson()));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `Future<void> start({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `required String assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `required String assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `required FocusTimerMode mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `final session = FocusSession(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `assignmentId: assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `assignmentTitle: assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `mode: mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `startedAt: DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `state = FocusSessionState(session: session);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `await _persist(session);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `Future<void> pause() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `final s = state.session;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `if (s == null || s.isPaused) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `final next = s.copyWith(accumulated: s.elapsedAt(now), pausedAt: now);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `state = state.copyWith(session: next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `await _persist(next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Future<void> resume() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `final s = state.session;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `if (s == null || !s.isPaused) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `final next = s.copyWith(startedAt: DateTime.now(), clearPausedAt: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `state = state.copyWith(session: next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `await _persist(next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `Future<void> endSession({bool timedOut = false, bool playCue = true}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `final s = state.session;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `if (s == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `final focused = timedOut && s.mode == FocusTimerMode.pomodoro`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `? kFocusPomodoroDuration`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `: s.elapsedAt();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `final summary = FocusSummary(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `assignmentId: s.assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `assignmentTitle: s.assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `focused: focused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `mode: s.mode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `timedOut: timedOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `state = FocusSessionState(summary: summary);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `await _persist(null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `if (timedOut && playCue) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `await playFocusTimeUpCue();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `void setSummaryNote(String note) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `final summary = state.summary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `if (summary == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `state = state.copyWith(summary: summary.copyWith(note: note));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `void dismissSummary() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `state = const FocusSessionState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/providers/focus/focus_session_provider.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/groups/group_providers.dart

Teacher-owned and student-membership groups. This file (group providers) is one concrete piece of that layer.

**L6** `final myGroupsProvider = FutureProvider<List<StudyGroup>>((ref) async {`
Riverpod binding. Name it after the value it exposes.

**L7** `final profile = await ref.watch(currentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L8** `if (profile == null) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `if (profile.role == UserRole.teacher) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final teacher = await ref.watch(teacherProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L12** `if (teacher == null) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `.from('groups')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L15** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L16** `.eq('teacher_id', teacher.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L17** `.order('title');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `.map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L20** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `if (profile.role == UserRole.student) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final student = await ref.watch(studentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L25** `if (student == null) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `.from('group_students')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L28** `.select('groups(*)')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L29** `.eq('student_id', student.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L30** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `(e) => StudyGroup.fromJson(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Map<String, dynamic>.from(e['groups'] as Map),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L34** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final rows = await supabase.from('groups').select().order('title');`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L40** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `.map((e) => StudyGroup.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L42** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final selectedGroupIdProvider = StateProvider<String?>((ref) => null);`
Riverpod binding. Name it after the value it exposes.

**L47** `final activeGroupIdProvider = Provider<String?>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L48** `final selected = ref.watch(selectedGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L49** `final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L50** `if (selected != null && groups.any((g) => g.id == selected)) return selected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `return groups.isEmpty ? null : groups.first.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final groupMembersProvider = FutureProvider.family<List<GroupMember>, String>((`
Riverpod binding. Name it after the value it exposes.

**L55** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `.from('group_students')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L60** `.select('students(id, profiles(full_name, username))')`

Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L61** `.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L62** `final members = rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `.map((e) => GroupMember.fromGroupStudentRow(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L64** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `members.sort(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `(a, b) => a.fullName.toLowerCase().compareTo(b.fullName.toLowerCase()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `return members;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/providers/groups/group_providers.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/homework/assignment_providers.dart

Assignments, tasks, teacher submissions queue. This file (assignment providers) is one concrete piece of that layer.

**L10** `final assignmentsProvider =`
Riverpod binding. Name it after the value it exposes.

**L11** `AsyncNotifierProvider.family<`
Riverpod binding. Name it after the value it exposes.

**L12** `AssignmentsNotifier,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `PageResult<Assignment>,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `String?`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `>(AssignmentsNotifier.new);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `class AssignmentsNotifier`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18** `extends FamilyAsyncNotifier<PageResult<Assignment>, String?> {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Future<PageResult<Assignment>> build(String? groupId) => _fetch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `Future<PageResult<Assignment>> _fetch(int offset) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final cache = ref.watch(cacheServiceProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L24** `final limit = PageResult.fetchSize(PageResult.pageSize);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `var query = supabase.from('assignments').select();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L28** `if (arg != null) query = query.eq('group_id', arg!);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L29** `final rows = await query`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `.order('created_at', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `.range(offset, offset + limit - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L33** `if (offset == 0) await cache?.cacheAssignments(list);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return PageResult.fromFetch(list.map(Assignment.fromJson).toList());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (offset == 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final cached = cache?.cachedAssignments;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (cached != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return PageResult(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `items: cached.map(Assignment.fromJson).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `hasMore: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `Future<void> loadMore() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (current == null || !current.hasMore || current.loadingMore) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `state = AsyncData(current.copyWith(loadingMore: true));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final next = await _fetch(current.items.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `state = AsyncData(current.append(next.items, hasMore: next.hasMore));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `state = AsyncData(current.copyWith(loadingMore: false));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final assignmentDetailProvider = FutureProvider.family<Assignment?, String>((`
Riverpod binding. Name it after the value it exposes.

**L64** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `) async {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final data = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L69** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L70** `.eq('id', id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L71** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `if (data == null) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `return Assignment.fromJson(Map<String, dynamic>.from(data));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L76** `final tasksProvider = FutureProvider.family<List<TaskItem>, String>((`
Riverpod binding. Name it after the value it exposes.

**L77** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L82** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L83** `.eq('assignment_id', assignmentId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L84** `.order('sort_order');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `.map((e) => TaskItem.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L87** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `final assignmentProgressProvider = FutureProvider.family<double, String>((`
Riverpod binding. Name it after the value it exposes.

**L91** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final student = await ref.watch(studentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L95** `if (student == null) return 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final tasks = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L99** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L100** `.eq('assignment_id', assignmentId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L101** `if (tasks.isEmpty) return 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `final taskIds = tasks.map((t) => t['id'] as String).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final completed = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L106** `.select('task_id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L107** `.eq('student_id', student.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L108** `.eq('is_completed', true)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L109** `.inFilter('task_id', taskIds);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `return completed.length / tasks.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/providers/homework/assignment_providers.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/homework/teacher_submissions_provider.dart

Assignments, tasks, teacher submissions queue. This file (teacher submissions provider) is one concrete piece of that layer.

**L5** `final teacherSubmissionsProvider = FutureProvider<List<Map<String, dynamic>>>((`
Riverpod binding. Name it after the value it exposes.

**L6** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `final teacher = await ref.watch(teacherProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L9** `if (teacher == null) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final assignments = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L13** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L14** `.eq('teacher_id', teacher.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L15** `final assignmentIds = assignments.map((e) => e['id'] as String).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `if (assignmentIds.isEmpty) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final tasks = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L20** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L21** `.inFilter('assignment_id', assignmentIds);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final taskIds = tasks.map((e) => e['id'] as String).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `if (taskIds.isEmpty) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L27** `.select('''`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L28** `id, status, score, feedback, text_answer, file_url, voice_url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `submitted_at,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `tasks(title, assignments(title)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `students(profiles(full_name, username))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `''')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `.inFilter('task_id', taskIds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `.order('submitted_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `return rows.map((s) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final task = s['tasks'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final assignment = task?['assignments'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final profile = (s['students'] as Map?)?['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `return {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `'id': s['id'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `'status': s['status'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `'score': s['score'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'feedback': s['feedback'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `'text_answer': s['text_answer'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `'file_url': s['file_url'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'voice_url': s['voice_url'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'task_title': task?['title'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `'assignment_title': assignment?['title'],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `'student_name':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `profile?['full_name'] ?? profile?['username'] ?? 'Student',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `}).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/providers/homework/teacher_submissions_provider.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/marks/marks_providers.dart

Gradebook CRUD and student averages. This file (marks providers) is one concrete piece of that layer.

**L11** `Future<List<Map<String, dynamic>>> _fetchMarksPage({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required int offset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `String? groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `String? studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `AcademicTerm term = AcademicTerm.all,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `String select = '*, students(profiles(full_name, username))',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final limit = PageResult.fetchSize(PageResult.pageSize);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `var query = supabase.from('marks').select(select);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L20** `if (groupId != null) query = query.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L21** `if (studentId != null) query = query.eq('student_id', studentId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L22** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `if (start != null) query = query.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `if (end != null) query = query.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final rows = await query`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `.order('marked_on', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `.order('created_at', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `.range(offset, offset + limit - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `return rows.map((e) => Map<String, dynamic>.from(e)).toList();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L34** `Map<String, double> _weightedAverages(List<Map<String, dynamic>> rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final totals = <String, double>{};`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final weights = <String, double>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `for (final row in rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final id = row['student_id'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final pct = double.tryParse('${row['percentage']}') ?? 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final w = double.tryParse('${row['weight']}') ?? 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `totals[id] = (totals[id] ?? 0) + pct * w;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `weights[id] = (weights[id] ?? 0) + w;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `return {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `for (final id in totals.keys)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `id: weights[id] == 0 ? 0 : totals[id]! / weights[id]!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final groupMarksProvider =`
Riverpod binding. Name it after the value it exposes.

**L51** `AsyncNotifierProvider.family<GroupMarksNotifier, PageResult<Mark>, String>(`
Riverpod binding. Name it after the value it exposes.

**L52** `GroupMarksNotifier.new,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `class GroupMarksNotifier extends FamilyAsyncNotifier<PageResult<Mark>, String> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L56** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `Future<PageResult<Mark>> build(String groupId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `ref.watch(selectedMarkTermProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L59** `return _fetch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `Future<PageResult<Mark>> _fetch(int offset) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final term = ref.read(selectedMarkTermProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L64** `final rows = await _fetchMarksPage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `offset: offset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `groupId: arg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `term: term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `return PageResult.fromFetch(rows.map(Mark.fromJson).toList());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `Future<void> loadMore() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (current == null || !current.hasMore || current.loadingMore) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `state = AsyncData(current.copyWith(loadingMore: true));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `final next = await _fetch(current.items.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `state = AsyncData(current.append(next.items, hasMore: next.hasMore));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `state = AsyncData(current.copyWith(loadingMore: false));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `final groupMarkAveragesProvider =`
Riverpod binding. Name it after the value it exposes.

**L87** `FutureProvider.family<Map<String, double>, String>((ref, groupId) async {`
Riverpod binding. Name it after the value it exposes.

**L88** `final term = ref.watch(selectedMarkTermProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L89** `var query = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `.from('marks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L91** `.select('student_id, percentage, weight')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L92** `.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L93** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `if (start != null) query = query.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `if (end != null) query = query.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final rows = await query;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `return _weightedAverages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `rows.map((e) => Map<String, dynamic>.from(e)).toList(),`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L103** `final groupMarkCountProvider = FutureProvider.family<int, String>((`
Riverpod binding. Name it after the value it exposes.

**L104** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final term = ref.watch(selectedMarkTermProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L108** `var query = supabase.from('marks').select('id').eq('group_id', groupId);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L109** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `if (start != null) query = query.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `if (end != null) query = query.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `final result = await query.count(CountOption.exact);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return result.count;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `final memberMarksProvider =`
Riverpod binding. Name it after the value it exposes.

**L118** `FutureProvider.family<List<Mark>, ({String groupId, String studentId})>((`
Riverpod binding. Name it after the value it exposes.

**L119** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final term = ref.watch(selectedMarkTermProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L123** `var query = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `.from('marks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L125** `.select('*, students(profiles(full_name, username))')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L126** `.eq('group_id', key.groupId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L127** `.eq('student_id', key.studentId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L128** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `if (start != null) query = query.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `if (end != null) query = query.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `final rows = await query`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `.order('marked_on', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `.order('created_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `.map((e) => Mark.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L137** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final myMarksProvider =`
Riverpod binding. Name it after the value it exposes.

**L141** `AsyncNotifierProvider<MyMarksNotifier, PageResult<Mark>>(`
Riverpod binding. Name it after the value it exposes.

**L142** `MyMarksNotifier.new,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `class MyMarksNotifier extends AsyncNotifier<PageResult<Mark>> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L146** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `Future<PageResult<Mark>> build() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `ref.watch(selectedMarkTermProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L149** `ref.watch(studentProfileProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L150** `return _fetch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `Future<PageResult<Mark>> _fetch(int offset) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L155** `if (student == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `return const PageResult(items: [], hasMore: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `final term = ref.read(selectedMarkTermProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L159** `final rows = await _fetchMarksPage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `offset: offset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `studentId: student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `term: term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `select: '*, groups(title)',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `return PageResult.fromFetch(rows.map(Mark.fromJson).toList());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `Future<void> loadMore() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `final current = state.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `if (current == null || !current.hasMore || current.loadingMore) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `state = AsyncData(current.copyWith(loadingMore: true));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `final next = await _fetch(current.items.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `state = AsyncData(current.append(next.items, hasMore: next.hasMore));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `state = AsyncData(current.copyWith(loadingMore: false));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `final myMarkStatsProvider =`
Riverpod binding. Name it after the value it exposes.

**L183** `FutureProvider<`
Riverpod binding. Name it after the value it exposes.

**L184** `List<({MarkCategory category, double percentage, double weight})>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `>((ref) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `final student = await ref.watch(studentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L187** `if (student == null) return [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `final term = ref.watch(selectedMarkTermProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L189** `var query = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `.from('marks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L191** `.select('category, percentage, weight')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L192** `.eq('student_id', student.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L193** `final start = term.startKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final end = term.endKey;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `if (start != null) query = query.gte('marked_on', start);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `if (end != null) query = query.lte('marked_on', end);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `final rows = await query;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `(e) => (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `category: markCategoryFromString(e['category'] as String?),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `percentage: double.tryParse('${e['percentage']}') ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `weight: double.tryParse('${e['weight']}') ?? 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `final myMarkAverageProvider = FutureProvider<double?>((ref) async {`
Riverpod binding. Name it after the value it exposes.

**L210** `final stats = await ref.watch(myMarkStatsProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L211** `if (stats.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `var total = 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `var weight = 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `for (final s in stats) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `total += s.percentage * s.weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `weight += s.weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `if (weight == 0) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `return total / weight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `Future<void> saveMark({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `required String teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `required String title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `required MarkCategory category,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `required double percentage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `required double weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `required String comment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `required DateTime markedOn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `String? markId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `final payload = Mark(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `id: markId ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `studentId: studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `teacherId: teacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `title: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `percentage: percentage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `category: category,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `weight: weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `comment: comment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `markedOn: markedOn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `).toWrite();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `if (markId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `await supabase.from('marks').insert(payload);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L249** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `await supabase.from('marks').update(payload).eq('id', markId);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L254** `Future<void> deleteMark(String markId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `await supabase.from('marks').delete().eq('id', markId);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**CHECKPOINT — Closed lib/providers/marks/marks_providers.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/notifications/notification_providers.dart

Realtime inbox + announcements. This file (notification providers) is one concrete piece of that layer.

**L10** `final notificationsProvider = StreamProvider<List<AppNotification>>((`
Riverpod binding. Name it after the value it exposes.

**L11** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `) async* {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final profile = await ref.watch(currentProfileProvider.future);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L14** `if (profile == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `yield [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final cache = ref.watch(cacheServiceProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L21** `Future<List<AppNotification>> load() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final rows = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `.from('notifications')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L24** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L25** `.eq('user_id', profile.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L26** `.order('created_at', ascending: false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `.limit(50);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final list = rows.map((e) => Map<String, dynamic>.from(e)).toList();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L29** `await cache?.cacheNotifications(list);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `return list.map(AppNotification.fromJson).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final changes = StreamController<void>();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final channel = supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `.channel('notifications-${profile.id}')`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L36** `.onPostgresChanges(`

Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L37** `event: PostgresChangeEvent.all,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `schema: 'public',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `table: 'notifications',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `filter: PostgresChangeFilter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `type: PostgresChangeFilterType.eq,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `column: 'user_id',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `value: profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `callback: (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `if (!changes.isClosed) changes.add(null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `.subscribe();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `ref.onDispose(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `supabase.removeChannel(channel);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `changes.close();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `yield await load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `await for (final _ in changes.stream) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `yield await load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final unreadNotificationsCountProvider = Provider<int>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L64** `final notifications = ref.watch(notificationsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L65** `return notifications.maybeWhen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `data: (list) => list.where((n) => !n.isRead).length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `orElse: () => 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final announcementsProvider =`
Riverpod binding. Name it after the value it exposes.

**L72** `FutureProvider.family<List<Announcement>, String?>((ref, groupId) async {`
Riverpod binding. Name it after the value it exposes.

**L73** `var query = supabase.from('announcements').select();`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L74** `if (groupId != null) query = query.eq('group_id', groupId);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L75** `final rows = await query.order('created_at', ascending: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `return rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `.map((e) => Announcement.fromJson(Map<String, dynamic>.from(e)))`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L78** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `Future<void> markNotificationRead(String id) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `await supabase.from('notifications').update({'is_read': true}).eq('id', id);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L85** `Future<void> markAllNotificationsRead(String userId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `.from('notifications')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L88** `.update({'is_read': true})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L89** `.eq('user_id', userId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L90** `.eq('is_read', false);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

> **CHECKPOINT — Closed lib/providers/notifications/notification_providers.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/settings/locale_provider.dart

Locale, theme mode, AppStyle. This file (locale provider) is one concrete piece of that layer.

**L8** `const _prefsKey = 'preferred_language';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final localeProvider = StateNotifierProvider<LocaleController, Locale>((ref) {`
Riverpod binding. Name it after the value it exposes.

**L11** `final controller = LocaleController(ref);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `ref.listen(currentProfileProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L13** `next.whenData((profile) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final code = profile?.preferredLanguage;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `if (code != null) controller.applyFromProfile(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `return controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `class LocaleController extends StateNotifier<Locale> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L22** `LocaleController(this._ref) : super(AppLanguage.english.locale) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `_loadLocal();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final Ref _ref;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `Future<void> _loadLocal() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final code = prefs.getString(_prefsKey);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final lang = AppLanguage.tryParse(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `if (lang != null) state = lang.locale;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `void applyFromProfile(String code) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final lang = AppLanguage.tryParse(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `if (lang == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (state.languageCode == lang.code) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `state = lang.locale;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `SharedPreferences.getInstance().then(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `(prefs) => prefs.setString(_prefsKey, lang.code),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `Future<void> setLanguage(AppLanguage language) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `await SchedulerBinding.instance.endOfFrame;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final localeChanged = state.languageCode != language.code;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (localeChanged) state = language.locale;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `await prefs.setString(_prefsKey, language.code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final profile = _ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L57** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (profile.preferredLanguage == language.code) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L61** `'preferred_language': language.code,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `_ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **CHECKPOINT — Closed lib/providers/settings/locale_provider.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/providers/settings/theme_provider.dart

Locale, theme mode, AppStyle. This file (theme provider) is one concrete piece of that layer.

**L8** `const _prefsKey = 'preferred_theme';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `const _stylePrefsKey = 'preferred_style';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `enum AppThemePreference {`
Closed set of values. Keep it aligned with the Postgres enum or the fromString helper.

**L12** `light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `system;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `String get code => name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `ThemeMode get themeMode => switch (this) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `AppThemePreference.light => ThemeMode.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `AppThemePreference.dark => ThemeMode.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `AppThemePreference.system => ThemeMode.system,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `static AppThemePreference? tryParse(String? code) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `if (code == null || code.isEmpty) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `for (final v in AppThemePreference.values) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `if (v.code == code) return v;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Future<ThemeMode> loadSavedThemeMode() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final pref = AppThemePreference.tryParse(prefs.getString(_prefsKey));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `return pref?.themeMode ?? ThemeMode.system;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `final themeModeProvider = StateNotifierProvider<ThemeModeController, ThemeMode>(`

Riverpod binding. Name it after the value it exposes.

**L40** `(ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `final controller = ThemeModeController(ref);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `ref.listen(currentProfileProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L43** `next.whenData((profile) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final code = profile?.preferredTheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `if (code != null) controller.applyFromProfile(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `return controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `class ThemeModeController extends StateNotifier<ThemeMode> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L53** `ThemeModeController(this._ref, {ThemeMode? initial})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `: super(initial ?? ThemeMode.system) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `if (initial == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `_loadLocal();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final Ref _ref;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `Future<void> _loadLocal() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final pref = AppThemePreference.tryParse(prefs.getString(_prefsKey));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `if (pref != null && mounted) state = pref.themeMode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `void applyFromProfile(String code) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `final pref = AppThemePreference.tryParse(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `if (pref == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `if (state == pref.themeMode) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `state = pref.themeMode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `SharedPreferences.getInstance().then(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `(prefs) => prefs.setString(_prefsKey, pref.code),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `Future<void> setPreference(AppThemePreference preference) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `await SchedulerBinding.instance.endOfFrame;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `if (state != preference.themeMode) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `state = preference.themeMode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `await prefs.setString(_prefsKey, preference.code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final profile = _ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L90** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `if (profile.preferredTheme == preference.code) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L94** `'preferred_theme': preference.code,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `_ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L100** `Future<AppStyle> loadSavedAppStyle() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `return AppStyle.tryParse(prefs.getString(_stylePrefsKey)) ?? AppStyle.modern;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final appStyleProvider = StateNotifierProvider<AppStyleController, AppStyle>((`
Riverpod binding. Name it after the value it exposes.

**L106** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `final controller = AppStyleController(ref);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `ref.listen(currentProfileProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L110** `next.whenData((profile) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final code = profile?.preferredStyle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `if (code != null) controller.applyFromProfile(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `return controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `class AppStyleController extends StateNotifier<AppStyle> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L119** `AppStyleController(this._ref, {AppStyle? initial})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `: super(initial ?? AppStyle.modern) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `if (initial == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `_loadLocal();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `final Ref _ref;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `Future<void> _loadLocal() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `final style = AppStyle.tryParse(prefs.getString(_stylePrefsKey));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `if (style != null && mounted) state = style;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `void applyFromProfile(String code) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `final style = AppStyle.tryParse(code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `if (style == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `if (state == style) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `state = style;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `SharedPreferences.getInstance().then(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `(prefs) => prefs.setString(_stylePrefsKey, style.code),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `Future<void> setStyle(AppStyle style) async {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `await SchedulerBinding.instance.endOfFrame;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `if (state != style) state = style;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final prefs = await SharedPreferences.getInstance();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `await prefs.setString(_stylePrefsKey, style.code);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `final profile = _ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L154** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `if (profile.preferredStyle == style.code) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `await _ref.read(profileRepositoryProvider).updateProfile(profile.id, {`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L158** `'preferred_style': style.code,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `_ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

> **CHECKPOINT — Closed lib/providers/settings/theme_provider.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/academic_years/admin_academic_years_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L12** `class AdminAcademicYearsScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `const AdminAcademicYearsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `ConsumerState<AdminAcademicYearsScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L17** `_AdminAcademicYearsScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `class _AdminAcademicYearsScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21** `extends ConsumerState<AdminAcademicYearsScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L22** `bool _busy = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Future<void> _createNext() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `setState(() => _busy = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final row = await supabase.rpc('admin_create_next_academic_year');`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L29** `final label = (row as Map)['label'] as String? ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `ref.invalidate(academicYearsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L31** `ref.invalidate(academicTermOptionsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L32** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `).showSnackBar(SnackBar(content: Text(l10n.adminYearCreated(label))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `).showSnackBar(SnackBar(content: Text(e.toString())));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `if (mounted) setState(() => _busy = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `Future<void> _setCurrent(String yearId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `setState(() => _busy = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L50** `'admin_set_current_academic_year',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `params: {'p_year_id': yearId},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `ref.invalidate(academicYearsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L54** `ref.invalidate(academicTermOptionsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L55** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `).showSnackBar(SnackBar(content: Text(e.toString())));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `if (mounted) setState(() => _busy = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `String _normalizeYearConfirm(String value) =>`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `value.trim().replaceAll(RegExp(r'[-–—■]'), '-');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `Future<void> _deleteYear(AcademicYearRecord year) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `if (year.isCurrent) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `SnackBar(content: Text(l10n.adminDeleteYearCurrentBlocked)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `final confirmController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `final expected = _normalizeYearConfirm(year.label);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `var matches = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `final confirmed = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `return StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `builder: (ctx, setLocal) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `title: Text(l10n.adminDeleteYearTitle(year.label)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `content: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `Text(l10n.adminDeleteYearBody(year.label)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `controller: confirmController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `autofocus: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `hintText: l10n.adminDeleteYearConfirmHint(year.label),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `onChanged: (value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `setLocal(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `matches = _normalizeYearConfirm(value) == expected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `onPressed: matches ? () => Navigator.pop(ctx, true) : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `foregroundColor: Theme.of(ctx).colorScheme.error,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `confirmController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `if (confirmed != true || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `setState(() => _busy = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L134** `'admin_delete_academic_year',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `params: {'p_year_id': year.id},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `ref.invalidate(academicYearsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L138** `ref.invalidate(academicTermOptionsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L139** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `SnackBar(content: Text(l10n.adminYearDeleted(year.label))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `).showSnackBar(SnackBar(content: Text(e.toString())));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `if (mounted) setState(() => _busy = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `Future<void> _editTerm(AcademicTerm term) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final termId = term.dbTermId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `final semester = term.semester;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `final start = term.startInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `final end = term.endInclusive;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `if (termId == null || semester == null || start == null || end == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `final locale = Localizations.localeOf(context).toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `final dateFmt = DateFormat.yMMMd(locale);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `var startsOn = start;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `var endsOn = end;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `final saved = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `return StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `builder: (ctx, setLocal) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `Future<void> pickStart() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `final picked = await showDatePicker(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `context: ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `initialDate: startsOn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `firstDate: DateTime(startsOn.year - 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `lastDate: DateTime(endsOn.year + 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `if (picked != null) setLocal(() => startsOn = picked);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `Future<void> pickEnd() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `final picked = await showDatePicker(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `context: ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `initialDate: endsOn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `firstDate: DateTime(startsOn.year - 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `lastDate: DateTime(endsOn.year + 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `if (picked != null) setLocal(() => endsOn = picked);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `title: Text(l10n.adminEditTermTitle(semester)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `content: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `title: Text(l10n.adminTermStartsOn),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `subtitle: Text(dateFmt.format(startsOn)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `trailing: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `Icons.calendar_today_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `onTap: pickStart,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `title: Text(l10n.adminTermEndsOn),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `subtitle: Text(dateFmt.format(endsOn)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `trailing: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `Icons.calendar_today_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `onTap: pickEnd,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `onPressed: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `if (endsOn.isBefore(startsOn)) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `SnackBar(content: Text(l10n.adminTermDatesInvalid)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `Navigator.pop(ctx, true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `child: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `if (saved != true || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `setState(() => _busy = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L250** `'admin_update_academic_term',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `params: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `'p_term_id': termId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `'p_starts_on': startsOn.toIso8601String().split('T').first,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `'p_ends_on': endsOn.toIso8601String().split('T').first,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `ref.invalidate(academicYearsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L258** `ref.invalidate(academicTermOptionsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L259** `if (!mounted) return;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `).showSnackBar(SnackBar(content: Text(l10n.adminTermUpdated)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `).showSnackBar(SnackBar(content: Text(e.toString())));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `if (mounted) setState(() => _busy = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `final years = ref.watch(academicYearsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L277** `final dateFmt = DateFormat.yMMMd(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `Localizations.localeOf(context).toString(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `appBar: AppBar(title: Text(l10n.adminAcademicYearsTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `floatingActionButton: FloatingActionButton.extended(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `onPressed: _busy ? null : _createNext,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `icon: _busy`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `? const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `: const Icon(Icons.add),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `label: Text(l10n.adminCreateNextYear),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `body: years.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `if (list.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `title: l10n.adminNoAcademicYears,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `subtitle: l10n.adminAcademicYearsHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `icon: Icons.calendar_month_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `return RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `ref.invalidate(academicYearsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L309** `await ref.read(academicYearsProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L310** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `child: ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `itemCount: list.length + 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `itemBuilder: (context, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `if (index == 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `padding: const EdgeInsets.only(bottom: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `l10n.adminAcademicYearsHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `final year = list[index - 1];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `year.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `tooltip: l10n.adminDeleteYear,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `onPressed: _busy ? null : () => _deleteYear(year),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `Icons.delete_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `color: year.isCurrent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `? AppColors.inkAlpha(context, 0.25)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `: Theme.of(context).colorScheme.error,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `if (year.isCurrent)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `horizontal: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `vertical: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `color: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `).withValues(alpha: 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `borderRadius: BorderRadius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `color: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `).withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `l10n.adminCurrentYearBadge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `onPressed: _busy`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `: () => _setCurrent(year.id),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `child: Text(l10n.adminSetCurrentYear),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `l10n.adminYearSemesters,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `for (final t in year.terms) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `padding: const EdgeInsets.only(bottom: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `child: Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `borderRadius: BorderRadius.circular(10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `onTap: _busy ? null : () => _editTerm(t),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `vertical: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `horizontal: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `Icons.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `size: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `l10n.adminSemesterRange(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `t.semester ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `dateFmt.format(t.startInclusive!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `dateFmt.format(t.endInclusive!),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `Icons.edit_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/admin/academic_years/admin_academic_years_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/analytics/admin_analytics_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L12** `class AdminAnalyticsScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `const AdminAnalyticsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final stats = ref.watch(adminPlatformStatsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L18** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `appBar: AppBar(title: Text(l10n.adminAnalytics)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `body: stats.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `data: (s) => RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L27** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `title: l10n.adminActivityDistribution,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `subtitle: l10n.adminGlobalMetrics,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `child: SfCartesianChart(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `primaryXAxis: const CategoryAxis(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `series: <CartesianSeries>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `ColumnSeries<_AdminChartData, String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `dataSource: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `l10n.adminChartAssignments,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `(s['assignments'] ?? 0).toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `l10n.adminChartMarks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `(s['marks'] ?? 0).toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `l10n.adminChartStudents,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `(s['students'] ?? 0).toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `l10n.adminChartGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `(s['groups'] ?? 0).toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `xValueMapper: (d, _) => d.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `yValueMapper: (d, _) => d.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `borderRadius: const BorderRadius.vertical(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `top: Radius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `class _AdminChartData {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L82** `_AdminChartData(this.label, this.value);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `final double value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/analytics/admin_analytics_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/announcements/admin_announcements_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `class AdminAnnouncementsScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14** `const AdminAnnouncementsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `ConsumerState<AdminAnnouncementsScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L18** `_AdminAnnouncementsScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `class _AdminAnnouncementsScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L22** `extends ConsumerState<AdminAnnouncementsScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L23** `void _refresh() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `ref.invalidate(adminAnnouncementsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L25** `ref.invalidate(announcementsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L28** `Future<void> _compose({Map<String, dynamic>? existing}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final groups = await ref.read(adminGroupsProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L30** `if (groups.isEmpty || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final title = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `text: existing?['title'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final body = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `text: existing?['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `var groupId =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `existing?['group_id'] as String? ?? groups.first['id'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `builder: (ctx, setStateLocal) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `existing == null ? l10n.announcementsNew : l10n.announcementsEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `controller: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `controller: body,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `decoration: InputDecoration(labelText: l10n.commonMessage),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `maxLines: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `if (existing == null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `initialValue: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `labelText: l10n.teacherTargetGroup,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `items: groups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `(g) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `value: g['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `child: Text(g['title'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `onChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `setStateLocal(() => groupId = v ?? groupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `ElevatedButton(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `if (title.text.trim().isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `if (existing == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `await supabase.from('announcements').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L94** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `'title': title.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `'description': body.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `.from('announcements')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L101** `.update({`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L102** `'title': title.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `'description': body.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `.eq('id', existing['id'] as String);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L107** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `_refresh();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `existing == null ? l10n.announcementsPost : l10n.commonSave,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `Future<void> _delete(Map<String, dynamic> row) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final confirmed = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `title: Text(l10n.announcementsDeleteTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `content: Text(l10n.announcementsDeleteBody),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `backgroundColor: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `if (confirmed != true) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `await supabase.from('announcements').delete().eq('id', row['id'] as String);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L144** `_refresh();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `final announcements = ref.watch(adminAnnouncementsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L150** `final groupTitles = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `for (final g in ref.watch(adminGroupsProvider).valueOrNull ?? [])`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L152** `g['id'] as String: g['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `appBar: AppBar(title: Text(l10n.announcementsAll)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `floatingActionButton: FloatingActionButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `onPressed: () => _compose(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `child: const Icon(Icons.campaign_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `body: announcements.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `data: (rows) => rows.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `? EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `title: l10n.announcementsEmpty,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `subtitle: l10n.announcementsEmptyAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `onRefresh: () async => _refresh(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `child: ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `itemCount: rows.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `itemBuilder: (_, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `final row = rows[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `final createdAt = DateTime.tryParse(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `row['created_at'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `final groupTitle = groupTitles[row['group_id'] as String?];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `final byAdmin = row['teacher_id'] == null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `row['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `PopupMenuButton<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `padding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Icons.more_horiz_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `onSelected: (v) => v == 'edit'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `? _compose(existing: row)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `: _delete(row),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `itemBuilder: (_) => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `value: 'edit',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `child: Text(l10n.commonEdit),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `value: 'delete',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `row['description'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `color: AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `height: 1.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `spacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `runSpacing: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `crossAxisAlignment: WrapCrossAlignment.center,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `if (createdAt != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `DateFormat.yMMMd().format(createdAt),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `if (groupTitle != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `'• $groupTitle',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `if (byAdmin)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `'• ${l10n.announcementsByAdmin}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/announcements/admin_announcements_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/credentials/admin_credential_print_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L14** `class AdminCredentialPrintScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L15** `const AdminCredentialPrintScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `ConsumerState<AdminCredentialPrintScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L19** `_AdminCredentialPrintScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `class _AdminCredentialPrintScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L23** `extends ConsumerState<AdminCredentialPrintScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L24** `UserRole _role = UserRole.student;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `String? _groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `bool _exporting = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `Future<void> _export() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `if (_exporting) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `setState(() => _exporting = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final slips = await fetchCredentialSlips(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `role: _role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `groupId: _role == UserRole.student ? _groupId : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `if (slips.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `throw Exception(l10n.adminCredentialNeedUsers);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final bytes = buildCredentialSlipsPdf(slips, l10n);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `const fileName = 'mu_study_login_slips.pdf';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `await SharePlus.instance.share(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `ShareParams(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `files: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `XFile.fromData(bytes, mimeType: 'application/pdf', name: fileName),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `fileNameOverrides: [fileName],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `subject: l10n.adminCredentialPrintTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `downloadFallbackEnabled: true,`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L50** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final message = e is Exception`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `? e.toString().replaceFirst(RegExp(r'^Exception: '), '')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `: '$e';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `).showSnackBar(SnackBar(content: Text(message)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `if (mounted) setState(() => _exporting = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `final groups = ref.watch(adminGroupsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L70** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `appBar: AppBar(title: Text(l10n.adminCredentialPrintTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `body: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `shellContentBottomPadding(context, extra: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `l10n.adminCredentialPrintSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `color: AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `SegmentedButton<UserRole>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `segments: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `ButtonSegment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `value: UserRole.student,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `label: Text(l10n.adminCredentialStudents),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `ButtonSegment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `value: UserRole.teacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `label: Text(l10n.adminCredentialTeachers),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `selected: {_role},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `onSelectionChanged: (next) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_role = next.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `if (_role != UserRole.student) _groupId = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `if (_role == UserRole.student) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `groups.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `loading: () => const LinearProgressIndicator(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `error: (e, _) => Text('$e'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `data: (rows) => DropdownButtonFormField<String?>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `key: ValueKey(_groupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `initialValue: _groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `labelText: l10n.adminGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `items: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `DropdownMenuItem<String?>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `value: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `child: Text(l10n.adminCredentialAllGroups),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `...rows.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `(g) => DropdownMenuItem<String?>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `value: g['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `child: Text(g['title'] as String? ?? ''),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `onChanged: (value) => setState(() => _groupId = value),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `height: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `width: double.infinity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `child: FilledButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `onPressed: _exporting ? null : _export,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `icon: _exporting`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `? const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `: const Icon(Icons.picture_as_pdf_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `_exporting`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `? l10n.commonLoading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `: l10n.adminCredentialExportPdf,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `_PreviewList(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `role: _role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `groupId: _role == UserRole.student ? _groupId : null,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `class _PreviewList extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L174** `const _PreviewList({required this.role, required this.groupId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final UserRole role;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `final String? groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `return FutureBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `key: ValueKey('${role.name}-${groupId ?? 'all'}'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `future: fetchCredentialSlipPreviews(role: role, groupId: groupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `builder: (context, snapshot) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `if (snapshot.connectionState != ConnectionState.done) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `return const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `padding: EdgeInsets.symmetric(vertical: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `child: Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `if (snapshot.hasError) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `return ErrorState(message: '${snapshot.error}');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `final slips = snapshot.data ?? const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `if (slips.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `icon: Icons.print_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `title: l10n.adminCredentialNeedUsers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `padding: const EdgeInsets.fromLTRB(8, 8, 8, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `for (final slip in slips)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `dense: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `slip.fullName.isEmpty ? slip.username : slip.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `subtitle: Text(slip.username),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/credentials/admin_credential_print_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/dashboard/admin_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `class AdminDashboardScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14** `const AdminDashboardScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final stats = ref.watch(adminPlatformStatsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L20** `return stats.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `data: (s) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `onRefresh: () async => ref.invalidate(adminPlatformStatsProvider),`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L27** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `shellContentBottomPadding(context, extra: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `l10n.adminWorkspace,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `style: Theme.of(context).textTheme.bodyLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `l10n.adminPlatformOverview,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `style: Theme.of(context).textTheme.displaySmall,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `GridView.count(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `physics: const NeverScrollableScrollPhysics(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `crossAxisCount: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `crossAxisSpacing: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `mainAxisSpacing: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `childAspectRatio: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `_AdminStatCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `title: l10n.adminTeachers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `value: s['teachers']!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `icon: Icons.school_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `_AdminStatCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `title: l10n.adminStudents,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `value: s['students']!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `icon: Icons.people_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `_AdminStatCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `title: l10n.adminGroupsLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `value: s['groups']!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `icon: Icons.groups_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `_AdminStatCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `title: l10n.adminMarksLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `value: s['marks'] ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `const SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `title: l10n.adminActivityInsights,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `subtitle: l10n.adminCurrentMetrics,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `height: 200,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `child: SfCartesianChart(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `plotAreaBorderWidth: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `primaryXAxis: const CategoryAxis(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `majorGridLines: MajorGridLines(width: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `axisLine: AxisLine(width: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `primaryYAxis: const NumericAxis(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `majorGridLines: MajorGridLines(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `width: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `color: Color(0xFFF0F0F0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `axisLine: AxisLine(width: 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `series: <CartesianSeries>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `ColumnSeries<_AdminChartData, String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `dataSource: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `l10n.adminChartTasks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `s['assignments']!.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `l10n.adminChartGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `s['groups']!.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `_AdminChartData(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `l10n.adminChartStaff,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `s['teachers']!.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `xValueMapper: (d, _) => d.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `yValueMapper: (d, _) => d.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `borderRadius: const BorderRadius.vertical(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `top: Radius.circular(6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `title: l10n.adminAcademicYears,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `subtitle: l10n.adminAcademicYearsSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `onTap: () => context.push('/admin/academic-years'),`
Push a route the user can pop (assignment detail, chat room).

**L140** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `padding: const EdgeInsets.all(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `color: AppColors.softWash(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `Icons.calendar_month_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `const SizedBox(width: 16),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `l10n.adminAcademicYearsTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `l10n.adminCreateNextYear,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `color: AppColors.inkAlpha(context, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `onTap: () => context.push('/admin/credential-print'),`
Push a route the user can pop (assignment detail, chat room).

**L186** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `padding: const EdgeInsets.all(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `color: AppColors.softWash(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `Icons.print_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `l10n.adminCredentialPrintTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `l10n.adminCredentialPrintSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `color: AppColors.inkAlpha(context, 0.35),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `onTap: () => context.push('/admin/mass-generator'),`
Push a route the user can pop (assignment detail, chat room).

**L232** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `padding: const EdgeInsets.all(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `color: AppColors.softWash(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `Icons.dataset_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `'Mass Data Generator',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `'Paste teachers, groups and students; generate in one go.',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `color: AppColors.inkAlpha(context, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `const SizedBox(height: 100),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `class _AdminStatCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L285** `const _AdminStatCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `required this.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `required this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `final int value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `final tone = AppColors.resolveAccent(context, color);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `mainAxisAlignment: MainAxisAlignment.center,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `mainAxisAlignment: MainAxisAlignment.spaceBetween,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `padding: const EdgeInsets.all(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `color: tone.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `borderRadius: BorderRadius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `child: Icon(icon, size: 18, color: tone),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `'$value',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `style: Theme.of(context).textTheme.headlineMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `letterSpacing: 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `class _AdminChartData {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L343** `_AdminChartData(this.label, this.value);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `final double value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/dashboard/admin_dashboard_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L12** `class AdminMassDataGeneratorScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `const AdminMassDataGeneratorScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `ConsumerState<AdminMassDataGeneratorScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L17** `_AdminMassDataGeneratorScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `class _AdminMassDataGeneratorScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21** `extends ConsumerState<AdminMassDataGeneratorScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L22** `final _teachersCtrl = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final _studentsCtrl = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final _groupsCtrl = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final _assignmentsCtrl = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `bool _running = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `_teachersCtrl.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `_studentsCtrl.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `_groupsCtrl.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `_assignmentsCtrl.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `static List<String> _splitLines(String raw) => raw`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `.split('\n')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `.map((e) => e.trim())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `.where((e) => e.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `List<Map<String, dynamic>> _parseTeachers() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final out = <Map<String, dynamic>>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `for (final line in _splitLines(_teachersCtrl.text)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final parts = line.split('|').map((e) => e.trim()).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `if (parts.length != 4) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `throw ArgumentError(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `'Teacher line must be: username|password|full_name|sphere. Bad: "$line"',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `final sphereRaw = parts[3].toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final sphere = Sphere.values.firstWhere(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `(s) => s.name == sphereRaw,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `orElse: () => Sphere.math,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `out.add({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `'username': parts[0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `'password': parts[1],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `'full_name': parts[2],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `'sphere': sphere.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `List<Map<String, dynamic>> _parseStudents() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final out = <Map<String, dynamic>>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `for (final line in _splitLines(_studentsCtrl.text)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `final parts = line.split('|').map((e) => e.trim()).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `if (parts.length != 3) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `throw ArgumentError(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `'Student line must be: username|password|full_name. Bad: "$line"',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `out.add({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `'username': parts[0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `'password': parts[1],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `'full_name': parts[2],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `List<Map<String, dynamic>> _parseGroups() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final out = <Map<String, dynamic>>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `for (final line in _splitLines(_groupsCtrl.text)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final parts = line.split('|').map((e) => e.trim()).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `if (parts.length != 2) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `throw ArgumentError(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `'Group line must be: teacher_username|group_title. Bad: "$line"',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `out.add({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `'teacher_username': parts[0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `'title': parts[1],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `List<Map<String, dynamic>> _parseAssignments() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final out = <Map<String, dynamic>>[];`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `for (final line in _splitLines(_assignmentsCtrl.text)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `final parts = line.split('|').map((e) => e.trim()).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `if (parts.length != 3) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `throw ArgumentError(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `'Assignment line must be: teacher_username|group_title|student_username. Bad: "$line"',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `out.add({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `'teacher_username': parts[0],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `'group_title': parts[1],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `'student_username': parts[2],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `return out;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `Future<void> _run() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `if (_running) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `setState(() => _running = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `final teachers = _parseTeachers();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `final students = _parseStudents();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `final groups = _parseGroups();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `final assignments = _parseAssignments();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L138** `'admin_mass_generate_school_data',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `params: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `'p_teachers': teachers,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `'p_students': students,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `'p_groups': groups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `'p_assignments': assignments,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `await saveLoginPasswordsByUsername([...teachers, ...students]);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `SnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `content: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `'Generated successfully.',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `ref.invalidate(adminPlatformStatsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L158** `ref.invalidate(adminGroupsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L159** `ref.invalidate(allProfilesProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L160** `} on ArgumentError catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `SnackBar(content: Text('$e')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `} on PostgrestException catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `SnackBar(content: Text(e.message)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `SnackBar(content: Text('Failed: $e')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `if (mounted) setState(() => _running = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `appBar: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `title: const Text('Mass Data Generator'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `body: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `shellContentBottomPadding(context, extra: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `const Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `'Paste data (use "|" delimiter).',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `style: TextStyle(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `'1) Teachers: username|password|full_name|sphere',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `style: TextStyle(color: AppColors.inkAlpha(context, 0.55)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `controller: _teachersCtrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `minLines: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `maxLines: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `keyboardType: TextInputType.multiline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `border: OutlineInputBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `hintText: '@teacher1|password123|Teacher One|math',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `const Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `'2) Students: username|password|full_name',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `controller: _studentsCtrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `minLines: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `maxLines: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `keyboardType: TextInputType.multiline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `border: OutlineInputBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `hintText: '@student1|password123|Student One',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `const Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `'3) Groups: teacher_username|group_title',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `controller: _groupsCtrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `minLines: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `maxLines: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `keyboardType: TextInputType.multiline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `border: OutlineInputBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `hintText: '@teacher1|English 7A',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `const Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `'4) Assignments: teacher_username|group_title|student_username',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `controller: _assignmentsCtrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `minLines: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `maxLines: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `keyboardType: TextInputType.multiline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `border: OutlineInputBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `hintText: '@teacher1|English 7A|@student1',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `const SizedBox(height: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `height: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `child: FilledButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `onPressed: _running ? null : _run,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `icon: _running`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `? const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `: const Icon(Icons.auto_fix_high_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `label: Text(_running ? 'Generating...' : 'Generate'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `'Tip: usernames should match exactly the ones used in Teachers/Students.',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `style: TextStyle(color: AppColors.inkAlpha(context, 0.55)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed**
**lib/screens/admin/data_generator/admin_mass_data_generator_screen.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/groups/admin_groups_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `class AdminGroupsScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14** `const AdminGroupsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `ConsumerState<AdminGroupsScreen> createState() => _AdminGroupsScreenState();`
Widget that can watch providers. Required for profile/theme/locale.

**L20** `class _AdminGroupsScreenState extends ConsumerState<AdminGroupsScreen> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L21** `Future<void> _createGroup() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final title = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `String? selectedTeacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `var submitting = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final teachersRes = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `.from('teachers')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L28** `.select('id, profiles(full_name, username)');`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L29** `final teachers = teachersRes as List<dynamic>;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `builder: (ctx, setLocal) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final l10n = ctx.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `title: Text(l10n.adminCreateGroup),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `content: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `controller: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `labelText: l10n.commonGroup,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `hintText: l10n.adminGroupTitleHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `initialValue: selectedTeacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `labelText: l10n.adminAssignTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `items: teachers.map((t) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final p = t['profiles'] as Map;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final name =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `p['full_name'] as String? ?? p['username'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `return DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `value: t['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `child: Text(name),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `}).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `onChanged: submitting`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `: (v) => setLocal(() => selectedTeacherId = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `onPressed: submitting ? null : () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `onPressed: submitting`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final groupTitle = title.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `if (groupTitle.isEmpty || selectedTeacherId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `setLocal(() => submitting = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `await supabase.from('groups').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L87** `'title': groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `'teacher_id': selectedTeacherId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `ref.invalidate(adminGroupsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L92** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `if (ctx.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `setLocal(() => submitting = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final message = '$e'.contains(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `'groups_teacher_title_uidx',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `? l10n.adminGroupTitleExists`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `: l10n.commonErrorWithDetails('$e');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `ScaffoldMessenger.of(ctx).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `SnackBar(content: Text(message)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: Text(l10n.adminCreateGroupButton),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `Future<void> _assignStudent(String groupId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `final students = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `.from('students')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L118** `.select('id, profiles(full_name, username)');`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L119** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `String? studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `final l10n = ctx.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `title: Text(l10n.adminAssignStudent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `content: DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `decoration: InputDecoration(labelText: l10n.adminSelectStudent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `items: students.map((s) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `final p = s['profiles'] as Map;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `return DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `value: s['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `p['full_name'] as String? ?? p['username'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `}).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `onChanged: (v) => studentId = v,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `if (studentId == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `await supabase.from('group_students').upsert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L149** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `'student_id': studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `}, onConflict: 'group_id,student_id');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `ref.invalidate(adminGroupsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L154** `ref.invalidate(groupMembersProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L155** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `child: Text(l10n.commonAssign),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `final groups = ref.watch(adminGroupsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L167** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `appBar: AppBar(title: Text(l10n.adminGroups)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `floatingActionButton: FloatingActionButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `onPressed: _createGroup,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `child: const Icon(Icons.group_add_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `body: groups.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `data: (rows) => RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `onRefresh: () async => ref.invalidate(adminGroupsProvider),`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L181** `child: ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `itemCount: rows.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `itemBuilder: (_, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `final g = rows[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `final teacher = g['teachers'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `final profile = teacher?['profiles'] as Map?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `g['title'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `'${l10n.adminTeacherLine(profile?['full_name'] as String? ?? '—')} '`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `'(${localizedSphere(l10n, teacher?['sphere'])})',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `Icons.person_add_alt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `onPressed: () => _assignStudent(g['id'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/groups/admin_groups_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/helpers/admin_labels.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L4** `String localizedRole(AppLocalizations l10n, UserRole role) => switch (role) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `UserRole.student => l10n.commonStudent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `UserRole.teacher => l10n.commonTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `UserRole.admin => l10n.commonAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `String localizedSphere(AppLocalizations l10n, Object? raw) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final name = raw?.toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `if (name == Sphere.math.name) return l10n.sphereMath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `if (name == Sphere.english.name) return l10n.sphereEnglish;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `return name ?? '—';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/helpers/admin_labels.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/notifications/admin_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class AdminNotificationsScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `const AdminNotificationsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `body: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `child: EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `title: l10n.adminActivityLog,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `subtitle: l10n.adminActivityLogSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `icon: Icons.history,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/notifications/admin_notifications_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/profile/admin_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class AdminProfileScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const AdminProfileScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `return const AppProfileScreen();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/admin/profile/admin_profile_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/admin/users/admin_users_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L17** `class AdminUsersScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18** `const AdminUsersScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `ConsumerState<AdminUsersScreen> createState() => _AdminUsersScreenState();`
Widget that can watch providers. Required for profile/theme/locale.

**L24** `class _AdminUsersScreenState extends ConsumerState<AdminUsersScreen> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `Future<void> _createUser() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final username = TextEditingController(text: '@');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final password = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final name = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `var role = UserRole.student;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `var sphere = Sphere.math;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `String? selectedGroupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final groupsRes = await supabase.from('groups').select('id, title');`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L34** `final groups = groupsRes as List<dynamic>;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `builder: (ctx, setLocal) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `final l10n = ctx.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `title: Text(l10n.adminCreateUser),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `controller: username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `labelText: l10n.loginUsername,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `hintText: '@username',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `controller: password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `decoration: InputDecoration(labelText: l10n.adminPassword),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `obscureText: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `controller: name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `decoration: InputDecoration(labelText: l10n.adminFullName),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `DropdownButtonFormField<UserRole>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `initialValue: role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `decoration: InputDecoration(labelText: l10n.adminRole),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `items: UserRole.values`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `(r) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `value: r,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `child: Text(localizedRole(l10n, r).toUpperCase()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `onChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `setLocal(() => role = v ?? UserRole.student),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `if (role == UserRole.teacher) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `DropdownButtonFormField<Sphere>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `initialValue: sphere,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `decoration: InputDecoration(labelText: l10n.adminSphere),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `items: Sphere.values`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `(s) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `value: s,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `child: Text(s.l10nLabel(l10n)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `onChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `setLocal(() => sphere = v ?? Sphere.math),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `if (role == UserRole.student) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `initialValue: selectedGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `labelText: l10n.adminInitialGroupOptional,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `items: groups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `(g) => DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `value: g['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `child: Text(g['title'] as String),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `onChanged: (v) => setLocal(() => selectedGroupId = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `final userId = await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L129** `'admin_create_user',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `params: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `'p_username': username.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `'p_password': password.text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `'p_full_name': name.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `'p_role': role.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `if (role == UserRole.teacher) 'p_sphere': sphere.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `if (role == UserRole.student && selectedGroupId != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final studentRes = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `.from('students')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L142** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L143** `.eq('profile_id', userId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L144** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final studentId = studentRes['id'] as String;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `await supabase.from('group_students').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L148** `'group_id': selectedGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `'student_id': studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `await saveLoginPassword(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `userId: '$userId',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `password: password.text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `ref.invalidate(allProfilesProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L160** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `SnackBar(content: Text(l10n.adminUserCreated)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `} on PostgrestException catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `if (ctx.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `).showSnackBar(SnackBar(content: Text(e.message)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `child: Text(l10n.adminCreateUserButton),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `Future<void> _resetPassword(Profile profile) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `final password = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `final l10n = ctx.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `title: Text(l10n.adminResetPasswordTitle(profile.username)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `content: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `controller: password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `decoration: InputDecoration(labelText: l10n.adminPassword),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `obscureText: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `await supabase.rpc(`
Remote procedure call. Authorization and writes that are too complex for a single RLS policy often live here (login email, admin create user, unread counts).

**L203** `'admin_reset_password',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `params: {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `'p_user_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `'p_password': password.text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `await saveLoginPassword(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `userId: profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `password: password.text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `child: Text(l10n.commonReset),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `Future<void> _toggleDisable(Profile profile) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `.from('profiles')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L226** `.update({'is_disabled': !profile.isDisabled})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L227** `.eq('id', profile.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L228** `ref.invalidate(allProfilesProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L231** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `final profiles = ref.watch(allProfilesProvider);`

Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L234** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `appBar: AppBar(title: Text(l10n.adminUsers)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `floatingActionButton: FloatingActionButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `onPressed: _createUser,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `child: const Icon(Icons.person_add_alt_1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `body: profiles.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `final list = page.items;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `return ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `itemCount: list.length + (page.hasMore ? 1 : 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `itemBuilder: (_, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `if (i == list.length) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `return LoadMoreButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `hasMore: page.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `loading: page.loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `onPressed: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `ref.read(allProfilesProvider.notifier).loadMore(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L263** `final p = list[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `vertical: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `child: ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `leading: CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `backgroundColor: p.isDisabled`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `? Colors.grey`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `child: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `Icons.person,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `title: Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `p.fullName.isNotEmpty ? p.fullName : p.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `subtitle: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `l10n.adminRoleLine(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `localizedRole(l10n, p.role).toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `p.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `trailing: PopupMenuButton<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `onSelected: (v) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `if (v == 'reset') _resetPassword(p);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `if (v == 'disable') _toggleDisable(p);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `itemBuilder: (_) => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `value: 'reset',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `child: Text(l10n.adminResetPassword),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `value: 'disable',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `p.isDisabled ? l10n.adminEnable : l10n.adminDisable,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/admin/users/admin_users_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/auth/language_selection_screen.dart

Splash, login, language, setup-required, disabled account. This file (language selection screen) is one concrete piece of that layer.

**L10** `class LanguageSelectionScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L11** `const LanguageSelectionScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `ConsumerState<LanguageSelectionScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L15** `_LanguageSelectionScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `class _LanguageSelectionScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19** `extends ConsumerState<LanguageSelectionScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L20** `AppLanguage? _selected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `bool _saving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `_selected = AppLanguage.fromLocale(ref.read(localeProvider));`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L34** `Future<void> _continue() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final lang = _selected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `if (lang == null || _saving) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `setState(() => _saving = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `await ref.read(localeProvider.notifier).setLanguage(lang);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L41** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `context.go('/boot');`
Replace the navigation stack location. Login uses /boot so redirect can settle.

**L43** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `if (mounted) setState(() => _saving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final profile = ref.watch(currentProfileProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L53** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `backgroundColor: AppColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `body: SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `constraints: const BoxConstraints(maxWidth: 480),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `padding: const EdgeInsets.all(28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `l10n.appName,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `style: Theme.of(context).textTheme.headlineMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `l10n.languageTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `style: Theme.of(context).textTheme.titleLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `l10n.languageSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `height: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `if (profile != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `profile.fullName.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `? profile.fullName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `: profile.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `const SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `_LanguageCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `language: AppLanguage.english,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `title: 'English',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `subtitle: l10n.languageEnglish,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `flag: 'EN',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `selected: _selected == AppLanguage.english,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `onTap: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `setState(() => _selected = AppLanguage.english),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `_LanguageCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `language: AppLanguage.russian,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `title: '[illegible]',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `subtitle: l10n.languageRussian,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `flag: 'RU',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `selected: _selected == AppLanguage.russian,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `onTap: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `setState(() => _selected = AppLanguage.russian),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `_LanguageCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `language: AppLanguage.uzbek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `title: 'O‘zbekcha',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `subtitle: l10n.languageUzbek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `flag: 'UZ',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `selected: _selected == AppLanguage.uzbek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `onTap: () => setState(() => _selected = AppLanguage.uzbek),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `onPressed: _saving || _selected == null ? null : _continue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `backgroundColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `padding: const EdgeInsets.symmetric(vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `borderRadius: BorderRadius.circular(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `child: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `? const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `width: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `height: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `: Text(l10n.languageContinue),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `class _LanguageCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L164** `const _LanguageCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `required this.language,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `required this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `required this.flag,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `required this.selected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `final AppLanguage language;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `final String subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final String flag;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `final bool selected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `child: Ink(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `color: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `: AppColors.inkAlpha(context, 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `width: selected ? 2 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `boxShadow: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `color: AppColors.accent(context).withValues(alpha: 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `offset: const Offset(0, 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `width: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `height: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `: AppColors.softWash(context, lightAlpha: 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `flag,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `color: selected ? Colors.white : AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `title,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `? Icons.radio_button_checked_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `: Icons.radio_button_off_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `color: selected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `: AppColors.inkAlpha(context, 0.25),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/auth/language_selection_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/auth/login_screen.dart

Splash, login, language, setup-required, disabled account. This file (login screen) is one concrete piece of that layer.

**L16** `class LoginScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17** `const LoginScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `ConsumerState<LoginScreen> createState() => _LoginScreenState();`
Widget that can watch providers. Required for profile/theme/locale.

**L23** `class _LoginScreenState extends ConsumerState<LoginScreen> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24** `final _usernameController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final _passwordController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `bool _loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `String? _error;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `_usernameController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `_passwordController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `Future<void> _login() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (_usernameController.text.isEmpty || _passwordController.text.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `setState(() => _error = l10n.loginMissingFields);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `_loading = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `_error = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `.read(authRepositoryProvider)`
Riverpod binding. Name it after the value it exposes.

**L50** `.signIn(_usernameController.text.trim(), _passwordController.text);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L52** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `context.go('/boot');`
Replace the navigation stack location. Login uses /boot so redirect can settle.

**L54** `} on AuthException catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `_error = l10n.loginIncorrect;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `} on PostgrestException catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `_error = _messageForPostgrest(e, l10n.loginConfigError);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `_error = kDebugMode ? 'Sign in error: $e' : l10n.loginFailed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `if (mounted) setState(() => _loading = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `String _messageForPostgrest(PostgrestException e, String configError) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final msg = e.message.toLowerCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `if (msg.contains('jwt') ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `msg.contains('api key') ||`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `msg.contains('invalid') && msg.contains('key')) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `return configError;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `return e.message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `backgroundColor: Theme.of(context).scaffoldBackgroundColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `body: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `top: -100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `right: -100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `width: 300,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `height: 300,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `color: AppColors.softWash(context, lightAlpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `bottom: -50,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `left: -50,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `width: 200,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `height: 200,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `color: AppColors.accent(context).withValues(alpha: 0.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `const _LoginNatureScenery(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `child: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `padding: const EdgeInsets.symmetric(horizontal: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `constraints: const BoxConstraints(maxWidth: 400),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `const Center(child: AppLogoImage(size: 88)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `const SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `l10n.appName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `).textTheme.displaySmall?.copyWith(letterSpacing: -1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `l10n.loginSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `style: Theme.of(context).textTheme.bodyLarge`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `const SizedBox(height: 48),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `l10n.loginUsername.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `letterSpacing: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `color: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `).withValues(alpha: 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `controller: _usernameController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `hintText: '@username',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `prefixIcon: Icon(Icons.alternate_email, size: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `textInputAction: TextInputAction.next,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `l10n.loginPassword.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `letterSpacing: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `color: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `).withValues(alpha: 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `controller: _passwordController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `hintText: '••••••••',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `prefixIcon: Icon(Icons.lock_outline, size: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `obscureText: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `onSubmitted: (_) => _login(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `if (_error != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `vertical: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `color: AppColors.errorRed.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `borderRadius: BorderRadius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `color: AppColors.errorRed.withValues(alpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `Icons.error_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `_error!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `if (!Env.isConfigured) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `context.l10n.loginConfigMissing,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `style: Theme.of(context).textTheme.bodySmall`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `const SizedBox(height: 40),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `PrimaryButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `label: l10n.loginSignIn,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `loading: _loading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `onPressed: _login,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `class _LoginNatureScenery extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L251** `const _LoginNatureScenery();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `State<_LoginNatureScenery> createState() => _LoginNatureSceneryState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `class _LoginCloud {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L258** `const _LoginCloud({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `required this.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `required this.top,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `required this.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `required this.speed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `required this.phase,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `required this.opacity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `final String asset;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `final double top;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `final double size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `final double speed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `final double phase;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `final double opacity;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `class _LoginNatureSceneryState extends State<_LoginNatureScenery>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L276** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L277** `static const _clouds = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `_LoginCloud(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `asset: PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `top: 72,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `size: 64,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `speed: 1.15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `phase: 0.05,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `opacity: 0.72,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `_LoginCloud(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `asset: PaperAssets.flatCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `top: 130,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `size: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `speed: 0.75,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `phase: 0.42,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `opacity: 0.48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `_LoginCloud(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `asset: PaperAssets.grayCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `top: 168,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `size: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `speed: 1.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `phase: 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `opacity: 0.32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `_LoginCloud(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `asset: PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `top: 210,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `size: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `speed: 0.95,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `phase: 0.22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `opacity: 0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `_LoginCloud(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `asset: PaperAssets.flatCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `top: 54,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `size: 30,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `speed: 1.55,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `phase: 0.88,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `opacity: 0.28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `AnimationController? _ctrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `void _syncController(bool nature) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `if (nature && _ctrl == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `_ctrl = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `duration: const Duration(seconds: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `)..repeat();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `} else if (!nature && _ctrl != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `_ctrl!.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `_ctrl = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `void didChangeDependencies() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `super.didChangeDependencies();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `_syncController(isNaturePaper(context));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `_ctrl?.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `_ctrl = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `final nature = isNaturePaper(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `_syncController(nature);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `final ctrl = _ctrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `if (!nature || ctrl == null) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `final width = MediaQuery.sizeOf(context).width;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `return IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `child: AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `animation: ctrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `builder: (context, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `final t = ctrl.value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `final angle = t * 2 * math.pi;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `final tealX = 28 + math.sin(angle * 0.5) * 18;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `final tealY = 110 + math.sin(angle) * 12;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `final tealTilt = -0.08 + math.sin(angle) * 0.08;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `final creamX = 40 + math.sin(angle * 0.4 + 1.8) * 14;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `final creamY = 82 + math.sin(angle + 1.2) * 10;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `final creamTilt = 0.1 + math.sin(angle + 1.2) * 0.07;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `for (final c in _clouds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `left:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `width -`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `(((t * c.speed + c.phase) % 1.0) * (width + c.size + 40)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `top: c.top,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `c.asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `width: c.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `height: c.size,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `opacity: AlwaysStoppedAnimation(c.opacity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `left: tealX,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `top: tealY,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `angle: tealTilt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `PaperAssets.tealBird,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `right: creamX,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `top: creamY,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `angle: creamTilt,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `PaperAssets.creamBird,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `width: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `height: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `bottom: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `left: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `PaperAssets.singleTree,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `height: 96,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `opacity: const AlwaysStoppedAnimation(0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `bottom: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `right: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `PaperAssets.twoTrees,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `height: 110,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `opacity: const AlwaysStoppedAnimation(0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `bottom: 96,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `right: 52,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `angle: 0.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `PaperAssets.whiteDaisy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `width: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `height: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `opacity: const AlwaysStoppedAnimation(0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/auth/login_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/auth/setup_required_screen.dart

Splash, login, language, setup-required, disabled account. This file (setup required screen) is one concrete piece of that layer.

**L7** `class SetupRequiredScreen extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const SetupRequiredScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `return MaterialApp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `onGenerateTitle: (ctx) => AppLocalizations.of(ctx).appName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `theme: AppTheme.light(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `supportedLocales: AppLocalizations.supportedLocales,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `localizationsDelegates: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `AppLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `GlobalMaterialLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `GlobalWidgetsLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `GlobalCupertinoLocalizations.delegate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `home: Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `body: SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `l10n.appName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `style: Theme.of(context).textTheme.headlineMedium`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `color: AppColors.darkChocolate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `l10n.setupRequiredTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `style: Theme.of(context).textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `l10n.setupRequiredBody,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `style: Theme.of(context).textTheme.bodyLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `const SizedBox(height: 24),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `l10n.setupOption1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `style: Theme.of(context).textTheme.titleMedium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `const SelectableText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `'cp .env.example .env\n'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `'# Edit .env and set SUPABASE_ANON_KEY from Supabase → Settings → API\n'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `'flutter run',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `l10n.setupOption2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `style: Theme.of(context).textTheme.titleMedium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `const SelectableText(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `'flutter run --dart-define-from-file=.env',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/auth/setup_required_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/auth/splash_screen.dart

Splash, login, language, setup-required, disabled account. This file (splash screen) is one concrete piece of that layer.

**L8** `class SplashScreen extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const SplashScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `State<SplashScreen> createState() => _SplashScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `class _SplashScreenState extends State<SplashScreen> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `Future<void>.delayed(const Duration(milliseconds: 1200), () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `if (mounted) context.go('/boot');`
Replace the navigation stack location. Login uses /boot so redirect can settle.

**L24** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final theme = Theme.of(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final isDark = theme.brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final bg = theme.scaffoldBackgroundColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final fg = isDark ? AppColors.darkTextPrimary : AppColors.darkChocolate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final overlay = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `? SystemUiOverlayStyle.light`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `: SystemUiOverlayStyle.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `return AnnotatedRegion<SystemUiOverlayStyle>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `value: overlay.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `statusBarColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `systemNavigationBarColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `systemNavigationBarIconBrightness: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `? Brightness.light`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `: Brightness.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `child: Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `backgroundColor: bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `body: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `const AppLogoImage(size: 88)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `.animate(onPlay: (c) => c.repeat(reverse: true))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `.scale(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `begin: const Offset(0.95, 0.95),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `end: const Offset(1.05, 1.05),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `duration: 900.ms,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `'MU Study',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `style: theme.textTheme.headlineMedium?.copyWith(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `color: fg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/auth/splash_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/list/chat_list_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L17** `class ChatListScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18** `const ChatListScreen({super.key, required this.basePath});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String basePath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `ConsumerState<ChatListScreen> createState() => _ChatListScreenState();`
Widget that can watch providers. Required for profile/theme/locale.

**L26** `class _ChatListScreenState extends ConsumerState<ChatListScreen> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27** `final _search = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `String _query = '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `_search.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `String _preview(ChatRoomSummary room, AppLocalizations l10n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `if (room.lastMessageId == null) return l10n.chatNoMessagesYet;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (room.lastMessageDeleted) return l10n.chatMessageDeleted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return switch (room.lastMessageType) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `ChatMessageType.image => l10n.chatPreviewImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `ChatMessageType.video => l10n.chatPreviewVideo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `ChatMessageType.voice => l10n.chatPreviewVoice,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `ChatMessageType.file =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `room.lastMessageText?.trim().isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `? room.lastMessageText!.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `: l10n.chatPreviewFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `ChatMessageType.text || null =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `room.lastMessageText?.trim().isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `? room.lastMessageText!.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `: l10n.chatNoMessagesYet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `String _formatTime(DateTime? dt) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `if (dt == null) return '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final local = dt.toLocal();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (local.year == now.year &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `local.month == now.month &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `local.day == now.day) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `return DateFormat.Hm().format(local);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `if (now.difference(local).inDays < 7) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `return DateFormat.E().format(local);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `return DateFormat.MMMd().format(local);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final roomsAsync = ref.watch(chatRoomsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L73** `final isNeu = Neumorphic.isActive(context);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `appBar: AppBar(title: Text(l10n.chatTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `body: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `padding: const EdgeInsets.fromLTRB(16, 8, 16, 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `child: NeuSearchField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `controller: _search,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `hintText: l10n.chatSearchHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `onChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `setState(() => _query = v.trim().toLowerCase()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `if (isNaturePaper(context))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `padding: EdgeInsets.only(left: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `PaperAssets.tealBird,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `size: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `angle: -0.06,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `child: roomsAsync.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `error: (e, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `Center(child: Text(l10n.commonErrorWithDetails('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `data: (rooms) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final filtered = _query.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `? rooms`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `: rooms`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `.where(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `(r) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `r.groupTitle.toLowerCase().contains(_query) ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `r.teacherName.toLowerCase().contains(_query),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `if (filtered.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `icon: Icons.chat_bubble_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `title: l10n.chatEmptyTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `subtitle: l10n.chatEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `return ListView.separated(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `clipBehavior: isNeu ? Clip.none : Clip.hardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `isNeu ? 16 : 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `isNeu ? 4 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `isNeu ? 16 : 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `itemCount: filtered.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `separatorBuilder: (_, _) => SizedBox(height: isNeu ? 4 : 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `itemBuilder: (context, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `final room = filtered[index];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `return _ChatRoomTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `room: room,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `preview: _preview(room, l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `timeLabel: _formatTime(room.lastMessageAt),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `onTap: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `context.push('${widget.basePath}/${room.id}'),`
Push a route the user can pop (assignment detail, chat room).

**L146** `.animate()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `.fadeIn(duration: 220.ms, delay: (20 * index).ms)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `.slideY(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `begin: 0.04,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `end: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `duration: 280.ms,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `delay: (20 * index).ms,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `class _ChatRoomTile extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L167** `const _ChatRoomTile({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `required this.room,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `required this.preview,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `required this.timeLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `final ChatRoomSummary room;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `final String preview;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final String timeLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `final me = ref.watch(currentProfileProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L183** `final unread = room.unreadCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `final isNeu = Neumorphic.isActive(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `final accent = AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `final avatarLetter = Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `room.groupTitle.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `? room.groupTitle.characters.first.toUpperCase()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `: '?',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `final content = Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `padding: EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `horizontal: isNeu ? 4 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `vertical: isNeu ? 2 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `if (isNeu)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `NeuAvatar(child: avatarLetter)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `radius: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `backgroundColor: accent.withValues(alpha: 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `child: avatarLetter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `room.groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `style: Theme.of(context).textTheme.titleMedium`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `fontWeight: unread > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `? FontWeight.w800`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `if (timeLabel.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `timeLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `style: Theme.of(context).textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `color: unread > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `? AppColors.goldAccent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `: scheme.onSurface.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `fontWeight: unread > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `? FontWeight.w700`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `l10n.chatTeacherMembers(room.teacherName, room.memberCount),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `style: Theme.of(context).textTheme.bodySmall?.copyWith(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `room.lastMessageSenderId != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `room.lastMessageSenderId == me?.id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `? '${l10n.chatYou}: $preview'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `: preview,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `style: Theme.of(context).textTheme.bodyMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `fontWeight: unread > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `? FontWeight.w600`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `: FontWeight.w400,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `color: scheme.onSurface.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `alpha: unread > 0 ? 0.9 : 0.65,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `if (unread > 0) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `horizontal: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `vertical: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `color: AppColors.goldAccent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `borderRadius: BorderRadius.circular(999),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `boxShadow: isNeu`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `color: Neumorphic.shadowColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `).withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `offset: const Offset(2, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `blurRadius: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `unread > 99 ? '99+' : '$unread',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `? const Color(0xFF1A1408)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `elevation: NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `child: content,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `final isDark = Theme.of(context).brightness == Brightness.dark;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `color: isDark ? scheme.surfaceContainerHigh : Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `child: content,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/list/chat_list_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/chat_room_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L30** `class ChatRoomScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L31** `const ChatRoomScreen({super.key, required this.roomId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final String roomId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `ConsumerState<ChatRoomScreen> createState() => _ChatRoomScreenState();`
Widget that can watch providers. Required for profile/theme/locale.

**L39** `class _ChatRoomScreenState extends ConsumerState<ChatRoomScreen>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L40** `with WidgetsBindingObserver {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `static const _nearBottomThreshold = 120.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `static const _recentActivityWindow = Duration(minutes: 3);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final _text = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final _scroll = ScrollController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final _recorder = AudioRecorder();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final _focus = FocusNode();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `bool _sending = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `bool _attachOpen = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `bool _recording = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `double _uploadProgress = 0;`
Storage write. Path folders must satisfy bucket policies.

**L54** `String? _uploadLabel;`
Storage write. Path folders must satisfy bucket policies.

**L55** `Duration _recordElapsed = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `Timer? _recordTick;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `Timer? _typingIdle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `Timer? _presenceHeartbeat;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `Timer? _presencePoll;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `RealtimeChannel? _typingChannel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `List<String> _typingNames = const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final Set<String> _presenceOnlineIds = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final Set<String> _recentlyActiveIds = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `int _onlineCount = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `bool _markedRead = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `bool _presenceSetupInFlight = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `bool _stickToBottom = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `bool _showScrollDown = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `bool _didInitialScroll = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `String? _lastTailMessageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `int _lastMessageCount = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `bool _loadMoreQueued = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Timer? _scrollFabDebounce;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `Timer? _highlightTimer;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `ChatMessage? _replyTo;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `String? _highlightedMessageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `final _messageKeys = <String, GlobalKey>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `bool _jumpingToReply = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `bool _selectionMode = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `Set<String> _selectedMessageIds = {};`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `GlobalKey _keyForMessage(String id) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `_messageKeys.putIfAbsent(id, GlobalKey.new);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `bool _canModerateMessage(ChatMessage message, String? myId, bool isAdmin) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `if (message.isDeleted) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `if (isAdmin) return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `return myId != null && message.senderId == myId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `bool _canDeleteMessage(ChatMessage message, String? myId, bool isAdmin) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `return _canModerateMessage(message, myId, isAdmin);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `Iterable<ChatMessage> _deletableMessages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `List<ChatMessage> messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `String? myId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `bool isAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `return messages.where((m) => _canDeleteMessage(m, myId, isAdmin));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `void _enterSelectionMode({String? messageId}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_selectionMode = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `_selectedMessageIds = messageId == null ? {} : {messageId};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `_attachOpen = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `_focus.unfocus();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `void _exitSelectionMode() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `_selectionMode = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `_selectedMessageIds = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `void _toggleMessageSelection(String messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `if (_selectedMessageIds.contains(messageId)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `_selectedMessageIds.remove(messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `_selectedMessageIds = {..._selectedMessageIds, messageId};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `void _selectAllMessages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `List<ChatMessage> messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `String? myId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `bool isAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `_selectedMessageIds = _deletableMessages(messages, myId, isAdmin)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `.map((m) => m.id)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `.toSet();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `Future<void> _deleteSelected() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `if (_selectedMessageIds.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final count = _selectedMessageIds.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `final ok = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `builder: (context) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `title: Text(context.l10n.chatDeleteSelectedTitle(count)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `content: Text(context.l10n.chatDeleteSelectedBody),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `onPressed: () => Navigator.pop(context, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `child: Text(context.l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `onPressed: () => Navigator.pop(context, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `child: Text(context.l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `if (ok != true) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `final me = ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L165** `final isAdmin = me?.role == UserRole.admin;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `final current = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L167** `final ids = current == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `? _selectedMessageIds.toList()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `: _deletableMessages(current.messages, me?.id, isAdmin)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `.where((m) => _selectedMessageIds.contains(m.id))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `.map((m) => m.id)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `if (ids.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L177** `.deleteMessages(ids);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `if (mounted) _exitSelectionMode();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `WidgetsBinding.instance.addObserver(this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `_text.addListener(_onTextChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `_setupTyping();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `_focus.unfocus();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `FocusManager.instance.primaryFocus?.unfocus();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `WidgetsBinding.instance.removeObserver(this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `_typingIdle?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `_recordTick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `_presenceHeartbeat?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `_presencePoll?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `_scrollFabDebounce?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `_highlightTimer?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `_scroll.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `_text.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `_focus.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `_recorder.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `final ch = _typingChannel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `_typingChannel = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `if (ch != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `await ch.untrack();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `await supabase.removeChannel(ch);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `}();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `void didChangeAppLifecycleState(AppLifecycleState state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `if (state == AppLifecycleState.resumed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `_retrackPresence();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `void _commitOnlineCount({List<String>? typingNames}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `final me = ref.read(currentProfileProvider).valueOrNull?.id;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L239** `final merged = <String>{..._presenceOnlineIds, ..._recentlyActiveIds, ?me};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `final nextCount = merged.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `final nextTyping = typingNames ?? _typingNames;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `if (nextCount == _onlineCount &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `typingNames == null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `identical(nextTyping, _typingNames)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `_onlineCount = nextCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `if (typingNames != null) _typingNames = typingNames;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `void _refreshPresence(RealtimeChannel channel, String myId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `final onlineIds = <String>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `final typingNames = <String>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `for (final entry in channel.presenceState()) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `final keyId = entry.key.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `if (keyId.isNotEmpty) onlineIds.add(keyId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `for (final presence in entry.presences) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `final payload = presence.payload;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `final id = (payload['user_id'] as String?)?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `if (id != null && id.isNotEmpty) onlineIds.add(id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `final typing = payload['typing'] == true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `final name = payload['full_name'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `final who = id ?? (keyId.isEmpty ? null : keyId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `if (typing && who != null && who != myId && name != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `typingNames.add(name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `onlineIds.add(myId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `_presenceOnlineIds`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `..clear()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `..addAll(onlineIds);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `_commitOnlineCount(typingNames: typingNames.toList()..sort());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `void _notePresenceJoin(RealtimePresenceJoinPayload payload) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `final key = payload.key.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `if (key.isNotEmpty) _presenceOnlineIds.add(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `for (final presence in payload.newPresences) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `final id = (presence.payload['user_id'] as String?)?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `if (id != null && id.isNotEmpty) _presenceOnlineIds.add(id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `_commitOnlineCount();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `void _notePresenceLeave(RealtimePresenceLeavePayload payload) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `final key = payload.key.trim();`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `if (key.isNotEmpty) _presenceOnlineIds.remove(key);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `for (final presence in payload.leftPresences) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `final id = (presence.payload['user_id'] as String?)?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `if (id != null && id.isNotEmpty) _presenceOnlineIds.remove(id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `_commitOnlineCount();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `void _updateRecentlyActive(List<ChatMessage> messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `final cutoff = DateTime.now().subtract(_recentActivityWindow);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `final active = <String>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `for (final m in messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `if (m.isDeleted) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `if (m.createdAt.isBefore(cutoff)) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `if (m.senderId.isNotEmpty) active.add(m.senderId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `if (active.length == _recentlyActiveIds.length &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `active.containsAll(_recentlyActiveIds)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `_recentlyActiveIds`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `..clear()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `..addAll(active);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `_commitOnlineCount();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `Future<void> _retrackPresence() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `final profile = ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L323** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `final channel = _typingChannel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `if (channel == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `await _setupTyping();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `await channel.track({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `'user_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `'full_name': profile.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `'typing': _text.text.trim().isNotEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `if (mounted) _refreshPresence(channel, profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `Future<void> _setupTyping() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `if (_presenceSetupInFlight || _typingChannel != null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `final profile = ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L342** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `_presenceSetupInFlight = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `_presenceHeartbeat?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `_presencePoll?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `final previous = _typingChannel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `_typingChannel = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `if (previous != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `await previous.untrack();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `await supabase.removeChannel(previous);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `final channel = supabase.channel(`
Realtime subscription. Dispose the channel on provider dispose or you leak events across logins.

**L361** `'typing-${widget.roomId}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `opts: RealtimeChannelConfig(key: profile.id),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `channel`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `..onPresenceSync((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `_refreshPresence(channel, profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `..onPresenceJoin((payload) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `_notePresenceJoin(payload);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `scheduleMicrotask(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `if (mounted) _refreshPresence(channel, profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `..onPresenceLeave((payload) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `_notePresenceLeave(payload);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `scheduleMicrotask(() {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `if (mounted) _refreshPresence(channel, profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `channel.subscribe((status, [_]) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `if (status == RealtimeSubscribeStatus.subscribed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `await channel.track({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `'user_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `'full_name': profile.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `'typing': false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `if (mounted) _refreshPresence(channel, profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `_presenceHeartbeat = Timer.periodic(const Duration(seconds: 20), (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `if (!mounted || _typingChannel == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `_retrackPresence();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `_presencePoll = Timer.periodic(const Duration(seconds: 4), (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `if (!mounted || _typingChannel == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `_refreshPresence(_typingChannel!, profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `final msgs = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L405** `if (msgs != null) _updateRecentlyActive(msgs.messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `if (mounted) _typingChannel = channel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `_presenceSetupInFlight = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `void _onTextChanged() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `if (mounted) setState(() {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `final profile = ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L417** `if (profile == null || _typingChannel == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `_typingChannel!.track({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `'user_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `'full_name': profile.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `'typing': _text.text.trim().isNotEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `_typingIdle?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `_typingIdle = Timer(const Duration(seconds: 3), () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `_typingChannel?.track({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `'user_id': profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `'full_name': profile.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `'typing': false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `bool get _isNearBottom {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `if (!_scroll.hasClients) return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `final pos = _scroll.position;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `if (pos.maxScrollExtent <= 0) return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `return pos.pixels <= _nearBottomThreshold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `void _onUserScrolled() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `if (!_scroll.hasClients) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `final nearBottom = _isNearBottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `_stickToBottom = nearBottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `final show = !nearBottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `if (show == _showScrollDown) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `_scrollFabDebounce?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `_scrollFabDebounce = Timer(const Duration(milliseconds: 60), () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `final next = !_isNearBottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `if (next != _showScrollDown) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `setState(() => _showScrollDown = next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `void _maybeLoadMoreFromScroll() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `if (!_scroll.hasClients) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `final pos = _scroll.position;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `if (pos.maxScrollExtent > pos.viewportDimension * 0.5 &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `pos.pixels >= pos.maxScrollExtent - 80) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `_requestLoadMore();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `bool _onScrollNotification(ScrollNotification notification) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `if (notification.metrics.axis != Axis.vertical) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `if (notification is ScrollUpdateNotification ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `notification is OverscrollNotification) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `_maybeLoadMoreFromScroll();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `if (_stickToBottom &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `notification is ScrollMetricsNotification &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `notification.metrics.axis == Axis.vertical) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `final m = notification.metrics;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `if (m.pixels > 1) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `_scroll.jumpTo(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `if (notification is UserScrollNotification) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `_onUserScrolled();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `void _requestLoadMore() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `if (_loadMoreQueued) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L491** `if (state == null || !state.hasMore || state.loadingMore) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `_loadMoreQueued = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `final beforeExtent = _scroll.hasClients`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `? _scroll.position.maxScrollExtent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `: 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `final beforePixels = _scroll.hasClients ? _scroll.position.pixels : 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L499** `.loadMore()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `.whenComplete(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `_loadMoreQueued = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `if (!mounted || !_scroll.hasClients) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `if (!mounted || !_scroll.hasClients) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `final afterExtent = _scroll.position.maxScrollExtent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `final delta = afterExtent - beforeExtent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `if (delta > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `_scroll.jumpTo(beforePixels + delta);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `void _maybeMarkRead(AsyncValue<ChatMessagesState> messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `if (_markedRead) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `messages.whenData((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `_markedRead = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L522** `void _onMessagesUpdated(ChatMessagesState state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `_updateRecentlyActive(state.messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `final tailId = state.messages.isEmpty ? null : state.messages.last.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `final count = state.messages.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `if (!_didInitialScroll && state.messages.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `_didInitialScroll = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `_lastTailMessageId = tailId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `_lastMessageCount = count;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `_scrollToBottom(animated: false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `final hasNewTail = tailId != null && tailId != _lastTailMessageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `final prepended = count > _lastMessageCount && !hasNewTail;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `_lastTailMessageId = tailId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `_lastMessageCount = count;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `if (prepended) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `if (!hasNewTail) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `if (_stickToBottom) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `_scrollToBottom();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L547** `} else if (!_showScrollDown && mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `setState(() => _showScrollDown = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `void _onScrollDownTap() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `_stickToBottom = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `_showScrollDown = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `_scrollToBottom();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `ref.read(chatMessagesProvider(widget.roomId).notifier).markRoomRead();`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L561** `void _beginReply(ChatMessage message) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `if (message.isDeleted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `setState(() => _replyTo = message);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `_focus.requestFocus();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `void _clearReply() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `if (_replyTo == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `setState(() => _replyTo = null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `void _flashMessage(String messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `_highlightTimer?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `setState(() => _highlightedMessageId = messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `_highlightTimer = Timer(const Duration(milliseconds: 1400), () {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `if (_highlightedMessageId == messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `setState(() => _highlightedMessageId = null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `int? _reverseIndexForMessage(String messageId, List<ChatMessage> messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `final entries = chatEntriesFor(messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `final entryIndex = entries.indexWhere((e) => e.message?.id == messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `if (entryIndex < 0) return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `return entries.length - 1 - entryIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `Future<void> _awaitLoadMoreSettled({required int previousCount}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `for (var i = 0; i < 40; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `await Future<void>.delayed(const Duration(milliseconds: 50));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `final s = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L598** `if (s == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `if (!s.loadingMore &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `(s.messages.length != previousCount || !s.hasMore)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `Future<void> _jumpNearReverseIndex(int reverseIndex) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `if (!_scroll.hasClients) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `const avgExtent = 160.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `for (var pass = 0; pass < 3; pass++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `if (!_scroll.hasClients) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `final max = _scroll.position.maxScrollExtent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `final target = (reverseIndex * avgExtent).clamp(0.0, max);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `_scroll.jumpTo(target);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `await Future<void>.delayed(const Duration(milliseconds: 32));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `Future<bool> _ensureVisibleMessage(String messageId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `final key = _keyForMessage(messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `for (var frame = 0; frame < 12; frame++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `final ctx = key.currentContext;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L622** `if (ctx != null && ctx.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `await Scrollable.ensureVisible(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `duration: const Duration(milliseconds: 280),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `alignment: 0.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L629** `return true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `await Future<void>.delayed(const Duration(milliseconds: 40));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `Future<void> _scrollToRepliedMessage(String messageId) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `if (_jumpingToReply || messageId.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `_jumpingToReply = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `_stickToBottom = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `setState(() => _showScrollDown = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `final notFoundLabel = context.l10n.chatReplyNotFound;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `final messenger = ScaffoldMessenger.of(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `for (var attempt = 0; attempt < 30; attempt++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L647** `if (state == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `ChatMessage? hit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `for (final m in state.messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `if (m.id == messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `hit = m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `if (hit != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `if (hit.isDeleted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `messenger.showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `SnackBar(content: Text(context.l10n.chatReplyDeleted)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L663** `break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `if (!state.hasMore) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `messenger.showSnackBar(SnackBar(content: Text(notFoundLabel)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L671** `if (state.loadingMore) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `await _awaitLoadMoreSettled(previousCount: state.messages.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L673** `continue;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `final before = state.messages.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L677** `await ref.read(chatMessagesProvider(widget.roomId).notifier).loadMore();`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L678** `await _awaitLoadMoreSettled(previousCount: before);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L681** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L682** `final state = ref.read(chatMessagesProvider(widget.roomId)).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L683** `ChatMessage? target;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L684** `if (state != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L685** `for (final m in state.messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L686** `if (m.id == messageId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L687** `target = m;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L688** `break;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L692** `if (state == null || target == null || target.isDeleted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L693** `messenger.showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L694** `SnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `content: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L696** `target?.isDeleted == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L697** `? context.l10n.chatReplyDeleted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `: notFoundLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L699** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L700** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L702** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L705** `final reverseIndex = _reverseIndexForMessage(messageId, state.messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L706** `if (reverseIndex != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `await _jumpNearReverseIndex(reverseIndex);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L711** `var visible = await _ensureVisibleMessage(messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L712** `if (!visible && reverseIndex != null && mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L713** `await _jumpNearReverseIndex(reverseIndex + 2);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L714** `visible = await _ensureVisibleMessage(messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L717** `if (visible && mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L718** `_flashMessage(messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L719** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L722** `if (reverseIndex != null && _scroll.hasClients) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L723** `await _jumpNearReverseIndex(reverseIndex);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L724** `if (await _ensureVisibleMessage(messageId) && mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L725** `_flashMessage(messageId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L726** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L730** `messenger.showSnackBar(SnackBar(content: Text(notFoundLabel)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L731** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L732** `_jumpingToReply = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L736** `Future<void> _sendText() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L737** `final value = _text.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L738** `if (value.isEmpty || _sending) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L739** `final replyId = _replyTo?.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L740** `setState(() => _sending = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L741** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L742** `_text.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L743** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L744** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L745** `.sendText(value, replyToMessageId: replyId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L746** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L747** `_stickToBottom = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L748** `_showScrollDown = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L749** `_replyTo = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L751** `_scrollToBottom();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L752** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L753** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L754** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L755** `SnackBar(content: Text(context.l10n.chatSendFailed('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L758** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L759** `if (mounted) setState(() => _sending = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L763** `void _scrollToBottom({bool animated = true}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L764** `void attempt([int framesLeft = 3]) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L765** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L766** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L767** `if (!_scroll.hasClients) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L768** `if (framesLeft > 0) attempt(framesLeft - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L769** `return;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L771** `final pos = _scroll.position;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L772** `if (pos.pixels <= 1) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L773** `if (framesLeft > 0) attempt(framesLeft - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L774** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L776** `if (!animated || framesLeft > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L777** `_scroll.jumpTo(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L778** `if (framesLeft > 0) attempt(framesLeft - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L779** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L781** `_scroll.animateTo(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L782** `0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L783** `duration: const Duration(milliseconds: 220),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L784** `curve: Curves.easeOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L789** `attempt(animated ? 1 : 3);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L792** `Future<void> _pickFile() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L793** `final result = await FilePicker.platform.pickFiles(withData: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L794** `if (result == null || result.files.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L795** `final file = result.files.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L796** `if (file.bytes == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L797** `final type = chatMessageTypeForFileName(file.name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L798** `await _sendMedia(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L799** `type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L800** `file.bytes!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L801** `file.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L802** `caption: type == ChatMessageType.file ? file.name : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L806** `Future<void> _sendMedia(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L807** `ChatMessageType type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L808** `Uint8List bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L809** `String fileName, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L810** `String? caption,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L811** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L812** `final replyId = _replyTo?.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L813** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L814** `_sending = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L815** `_uploadProgress = 0.02;`
Storage write. Path folders must satisfy bucket policies.

**L816** `_uploadLabel = fileName;`
Storage write. Path folders must satisfy bucket policies.

**L817** `_attachOpen = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L819** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L820** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L821** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L822** `.sendMedia(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L823** `type: type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L824** `bytes: bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L825** `fileName: fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L826** `caption: caption,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L827** `replyToMessageId: replyId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L828** `onProgress: (p) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L829** `if (mounted) setState(() => _uploadProgress = p);`
Storage write. Path folders must satisfy bucket policies.

**L830** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L832** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L833** `_stickToBottom = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L834** `_showScrollDown = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L835** `_replyTo = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L837** `_scrollToBottom();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L838** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L839** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L840** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L841** `SnackBar(content: Text(context.l10n.chatSendFailed('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L844** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L845** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L846** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L847** `_sending = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L848** `_uploadProgress = 0;`
Storage write. Path folders must satisfy bucket policies.

**L849** `_uploadLabel = null;`
Storage write. Path folders must satisfy bucket policies.

**L855** `Future<void> _pickImage() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L856** `final picker = ImagePicker();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L857** `final image = await picker.pickImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L858** `source: ImageSource.gallery,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L859** `imageQuality: 85,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L861** `if (image == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L862** `final bytes = await image.readAsBytes();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L863** `await _sendMedia(ChatMessageType.image, bytes, image.name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L866** `Future<void> _pickVideo() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L867** `final picker = ImagePicker();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L868** `final video = await picker.pickVideo(source: ImageSource.gallery);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L869** `if (video == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L870** `final bytes = await video.readAsBytes();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L871** `await _sendMedia(ChatMessageType.video, bytes, video.name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L874** `Future<String> _recordingPath() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L875** `final stamp = DateTime.now().millisecondsSinceEpoch;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L876** `final ext = kIsWeb ? 'wav' : 'm4a';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L877** `final name = 'chat_voice_$stamp.$ext';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L878** `if (kIsWeb) return name;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L879** `final dir = await getTemporaryDirectory();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L880** `return p.join(dir.path, name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L883** `Future<void> _startRecording() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L884** `if (_recording || _sending) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L885** `final allowed = await _recorder.hasPermission();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L886** `if (!allowed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L887** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L888** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L889** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L890** `).showSnackBar(SnackBar(content: Text(context.l10n.chatMicDenied)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L892** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L894** `final path = await _recordingPath();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L895** `await _recorder.start(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L896** `RecordConfig(encoder: kIsWeb ? AudioEncoder.wav : AudioEncoder.aacLc),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L897** `path: path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L899** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L900** `_recording = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L901** `_recordElapsed = Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L903** `_recordTick = Timer.periodic(const Duration(seconds: 1), (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L904** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L905** `setState(() => _recordElapsed += const Duration(seconds: 1));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L909** `Future<void> _stopRecording({required bool send}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L910** `_recordTick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L911** `final path = await _recorder.stop();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L912** `setState(() => _recording = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L913** `if (!send || path == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L915** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L916** `final bytes = await XFile(path).readAsBytes();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L917** `final name = kIsWeb`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L918** `? 'chat_voice_${DateTime.now().millisecondsSinceEpoch}.wav'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L919** `: p.basename(path);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L920** `await _sendMedia(ChatMessageType.voice, bytes, name);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L921** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L922** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L923** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L924** `SnackBar(content: Text(context.l10n.chatSendFailed('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L930** `Future<void> _editMessage(ChatMessage message) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L931** `final result = await showDialog<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L932** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L933** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L934** `builder: (context) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L935** `EditMessageDialog(initialText: message.textContent ?? ''),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L937** `if (!mounted || result == null || result.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L938** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L939** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L940** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L941** `.editMessage(message.id, result);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L942** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L943** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L944** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L945** `SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L951** `Future<void> _deleteMessage(ChatMessage message) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L952** `final ok = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L953** `context: context,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L954** `builder: (context) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L955** `title: Text(context.l10n.chatDeleteTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L956** `content: Text(context.l10n.chatDeleteBody),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L957** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L958** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L959** `onPressed: () => Navigator.pop(context, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L960** `child: Text(context.l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L961** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L962** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L963** `onPressed: () => Navigator.pop(context, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L964** `child: Text(context.l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L965** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L967** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L969** `if (ok != true) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L970** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L971** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L972** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L973** `.deleteMessage(message.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L974** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L975** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L976** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L977** `SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L983** `Future<void> _react(ChatMessage message, String emoji) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L984** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L985** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L986** `.read(chatMessagesProvider(widget.roomId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L987** `.toggleReaction(message.id, emoji);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L988** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L989** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L990** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L991** `SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L997** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L998** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L999** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1000** `final room = ref.watch(chatRoomProvider(widget.roomId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L1001** `final messages = ref.watch(chatMessagesProvider(widget.roomId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L1002** `final me = ref.watch(currentProfileProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L1003** `final isAdmin = me?.role == UserRole.admin;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1004** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1005** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1007** `ref.listen(chatMessagesProvider(widget.roomId), (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L1008** `next.whenData(_onMessagesUpdated);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1010** `ref.listen(currentProfileProvider, (prev, next) {`
Riverpod binding. Name it after the value it exposes.

**L1011** `if (next.valueOrNull != null && _typingChannel == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1012** `_setupTyping();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1016** `_maybeMarkRead(messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1018** `final roomData = room.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1019** `final title = roomData?.groupTitle ?? l10n.chatTitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1021** `final keyboardInset = MediaQuery.viewInsetsOf(context).bottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1022** `final keyboardOpen = keyboardInset > 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1024** `return PopScope(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1025** `canPop: !_selectionMode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1026** `onPopInvokedWithResult: (didPop, result) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1027** `if (!didPop && _selectionMode) _exitSelectionMode();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1028** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1029** `child: Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1032** `resizeToAvoidBottomInset: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1033** `appBar: _selectionMode`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1034** `? AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1035** `leading: IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1036** `icon: const Icon(Icons.close),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1037** `onPressed: _exitSelectionMode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1038** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1039** `title: Text(l10n.chatSelectedCount(_selectedMessageIds.length)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1040** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1041** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1042** `onPressed: messages.maybeWhen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1043** `data: (state) => state.messages.isEmpty`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1044** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1045** `: () => _selectAllMessages(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1046** `state.messages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1047** `me?.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1048** `isAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1049** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1050** `orElse: () => null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1051** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1052** `child: Text(l10n.commonSelectAll),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1053** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1054** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1055** `tooltip: l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1056** `icon: const Icon(Icons.delete_outline),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1057** `onPressed: _selectedMessageIds.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1058** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1059** `: _deleteSelected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1060** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1063** `: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1064** `titleSpacing: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1065** `title: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1066** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1067** `Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1068** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1069** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1070** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1071** `false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1072** `final accent = AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1073** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1074** `width: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1075** `height: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1076** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1077** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1078** `color: accent.withValues(alpha: 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1079** `borderRadius: BorderRadius.circular(isMono ? 4 : 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1080** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1081** `? Border.all(color: accent.withValues(alpha: 0.22))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1082** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1083** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1084** `child: Icon(Icons.groups_rounded, size: 20, color: accent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1086** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1087** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1088** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1089** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1090** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1091** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1092** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1093** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1094** `title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1095** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1096** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1097** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1098** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1099** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1102** `if (roomData != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1103** `Text.rich(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1104** `TextSpan(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1105** `style: Theme.of(context).textTheme.bodySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1106** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1107** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1109** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1110** `TextSpan(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1111** `text: l10n.chatTeacherMembers(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1112** `roomData.teacherName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1113** `roomData.memberCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1114** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1116** `if (_onlineCount > 0) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1117** `const TextSpan(text: ' · '),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1118** `TextSpan(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1119** `text: '● ${l10n.chatOnlineCount(_onlineCount)}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1120** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1121** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1122** `(AppStyleTokens.maybeOf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1123** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1124** `)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1125** `false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1126** `? AppColors.ink(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1127** `: const Color(0xFF4CAF50),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1128** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1129** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1134** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1135** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1142** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1143** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1144** `tooltip: l10n.chatSelectMessages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1145** `icon: const Icon(Icons.checklist_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1146** `onPressed: () => _enterSelectionMode(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1147** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1150** `body: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1151** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1152** `if (_uploadProgress > 0 && _uploadProgress < 1)`
Storage write. Path folders must satisfy bucket policies.

**L1153** `LinearProgressIndicator(value: _uploadProgress),`
Storage write. Path folders must satisfy bucket policies.

**L1154** `if (_uploadLabel != null && _uploadProgress > 0)`
Storage write. Path folders must satisfy bucket policies.

**L1155** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1156** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1157** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1158** `l10n.chatUploading(_uploadLabel!),`
Storage write. Path folders must satisfy bucket policies.

**L1159** `style: Theme.of(context).textTheme.bodySmall,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1160** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1162** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1163** `child: messages.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1164** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1165** `error: (e, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1166** `Center(child: Text(l10n.commonErrorWithDetails('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1167** `data: (state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1168** `if (state.messages.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1169** `return Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1170** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1171** `l10n.chatNoMessagesYet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1172** `style: Theme.of(context).textTheme.bodyLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1173** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1178** `final entries = chatEntriesFor(state.messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1179** `final loadMoreOffset = state.hasMore ? 1 : 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1180** `final galleryItems = chatMediaItemsFromMessages(state.messages);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1182** `return Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1183** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1184** `NotificationListener<ScrollNotification>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1185** `onNotification: _onScrollNotification,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1186** `child: ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1187** `reverse: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1188** `controller: _scroll,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1189** `cacheExtent: 1200,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1190** `padding: EdgeInsets.fromLTRB(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1191** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1192** `8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1193** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1194** `_selectionMode`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1195** `? shellContentBottomPadding(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1196** `: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1198** `itemCount: entries.length + loadMoreOffset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1199** `itemBuilder: (context, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1200** `if (state.hasMore && index == entries.length) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1201** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1202** `padding: const EdgeInsets.only(top: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1203** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1204** `child: state.loadingMore`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1205** `? const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1206** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1207** `height: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1208** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1209** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1210** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1212** `: TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1213** `onPressed: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1214** `.read(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1215** `chatMessagesProvider(`
Riverpod binding. Name it after the value it exposes.

**L1216** `widget.roomId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1217** `).notifier,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1219** `.loadMore(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1220** `child: Text(l10n.commonLoadMore),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1221** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1225** `final entry = entries[entries.length - 1 - index];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1226** `if (entry.date != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1227** `return ChatDateChip(date: entry.date!);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1229** `final message = entry.message!;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1230** `final mine = message.senderId == me?.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1231** `final canModerate = _canModerateMessage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1232** `message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1233** `me?.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1234** `isAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1236** `final canDelete = _canDeleteMessage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1237** `message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1238** `me?.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1239** `isAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1241** `return KeyedSubtree(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1242** `key: _keyForMessage(message.id),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1243** `child: MessageBubble(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1244** `message: message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1245** `isMine: mine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1246** `myUserId: me?.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1247** `galleryItems: galleryItems,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1248** `highlighted: _highlightedMessageId == message.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1249** `selectionMode: _selectionMode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1250** `selected: _selectedMessageIds.contains(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1251** `message.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1253** `selectable: canDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1254** `onSelectionToggle: canDelete`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1255** `? () => _toggleMessageSelection(message.id)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1256** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1257** `onSelect: canDelete`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1258** `? () => _enterSelectionMode(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1259** `messageId: message.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1261** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1262** `onJumpToReply: _selectionMode`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1263** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1264** `: (id) => _scrollToRepliedMessage(id),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1265** `onReply: _selectionMode || message.isDeleted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1266** `? null`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1267** `: () => _beginReply(message),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1268** `onReact: _selectionMode || message.isDeleted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1269** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1270** `: (emoji) => _react(message, emoji),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1271** `onEdit:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1272** `!_selectionMode &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1273** `canModerate &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1274** `message.messageType ==`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1275** `ChatMessageType.text &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1276** `!message.isDeleted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1277** `? () => _editMessage(message)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1278** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1279** `onDelete:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1280** `!_selectionMode && canDelete`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1281** `? () => _deleteMessage(message)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1282** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1283** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1285** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1286** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1287** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1288** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1289** `right: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1290** `bottom: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1291** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1292** `ignoring: !_showScrollDown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1293** `child: AnimatedScale(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1294** `scale: _showScrollDown ? 1 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1295** `duration: const Duration(milliseconds: 180),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1296** `curve: Curves.easeOutBack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1297** `child: AnimatedOpacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1298** `opacity: _showScrollDown ? 1 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1299** `duration: const Duration(milliseconds: 160),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1300** `child: Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1301** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1302** `? scheme.surfaceContainerHighest`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1303** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1304** `elevation: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1305** `shadowColor: AppColors.inkAlpha(context, 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1306** `shape: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1307** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1308** `customBorder: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1309** `onTap: _onScrollDownTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1310** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1311** `width: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1312** `height: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1313** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1314** `Icons.keyboard_arrow_down_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1315** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1316** `size: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1319** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1320** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1322** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1323** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1324** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1327** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1328** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1330** `if (!_selectionMode && _typingNames.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1331** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1332** `padding: const EdgeInsets.fromLTRB(16, 0, 16, 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1333** `child: Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1334** `alignment: Alignment.centerLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1335** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1336** `l10n.chatTyping(_typingNames.join(', ')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1337** `style: Theme.of(context).textTheme.bodySmall?.copyWith(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1338** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1339** `fontStyle: FontStyle.italic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1340** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1341** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1342** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1343** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1344** `if (!_selectionMode && _recording)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1345** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1346** `width: double.infinity,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1347** `color: AppColors.errorRed.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1348** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1349** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1350** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1351** `const Icon(Icons.mic, color: AppColors.errorRed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1352** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1353** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1354** `l10n.chatRecording(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1355** `'${_recordElapsed.inMinutes.toString().padLeft(2, '0')}:'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1356** `'${(_recordElapsed.inSeconds % 60).toString().padLeft(2, '0')}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1357** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1358** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1359** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1360** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1361** `onPressed: () => _stopRecording(send: false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1362** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1364** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1365** `onPressed: () => _stopRecording(send: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1366** `child: Text(l10n.commonSubmit),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1367** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1369** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1370** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1371** `if (!_selectionMode)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1372** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1373** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1374** `context.isMobile ? 10 : 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1375** `6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1376** `context.isMobile ? 10 : 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1377** `keyboardInset +`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1378** `(keyboardOpen`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1379** `? 8`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1380** `: (context.isMobile ? 10 : 16) +`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1381** `MediaQuery.paddingOf(context).bottom),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1382** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1383** `child: Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1384** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1385** `final dockTokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1386** `final isNeuDock = dockTokens?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1387** `final isMonoDock = dockTokens?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1388** `final dockRadius = BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1389** `isMonoDock ? 4 : (isNeuDock ? 24 : 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1391** `final dockChild = Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1392** `padding: const EdgeInsets.fromLTRB(6, 8, 8, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1393** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1394** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1395** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1396** `AnimatedSize(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1397** `duration: const Duration(milliseconds: 220),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1398** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1399** `child: _attachOpen`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1400** `? Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1401** `padding: const EdgeInsets.only(bottom: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1402** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1403** `mainAxisAlignment:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1404** `MainAxisAlignment.spaceEvenly,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1405** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1406** `AttachChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1407** `icon: Icons.image_outlined,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1408** `label: l10n.chatAttachImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1409** `onTap: _pickImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1410** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1411** `AttachChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1412** `icon: Icons.videocam_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1413** `label: l10n.chatAttachVideo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1414** `onTap: _pickVideo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1415** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1416** `AttachChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1417** `icon: Icons.attach_file_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1418** `label: l10n.chatAttachFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1419** `onTap: _pickFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1420** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1422** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1424** `: const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1425** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1426** `AnimatedSize(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1427** `duration: const Duration(milliseconds: 180),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1428** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1429** `child: _replyTo == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1430** `? const SizedBox.shrink()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1431** `: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1432** `padding: const EdgeInsets.fromLTRB(8, 0, 8, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1433** `child: ReplyComposerBanner(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1434** `message: _replyTo!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1435** `onClose: _clearReply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1436** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1437** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1438** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1439** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1440** `crossAxisAlignment: CrossAxisAlignment.end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1441** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1442** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1443** `onPressed: _sending`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1444** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1445** `: () => setState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1446** `() => _attachOpen = !_attachOpen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1447** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1448** `icon: AnimatedRotation(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1449** `turns: _attachOpen ? 0.125 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1450** `duration: const Duration(milliseconds: 200),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1451** `child: const Icon(Icons.add_circle_outline),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1453** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1454** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1455** `child: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1456** `controller: _text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1457** `focusNode: _focus,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1458** `autofocus: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1459** `minLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1460** `maxLines: 5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1461** `textInputAction: TextInputAction.send,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1462** `onSubmitted: (_) => _sendText(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1463** `style: Theme.of(context).textTheme.bodyLarge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1464** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1465** `hintText: l10n.chatMessageHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1466** `filled: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1467** `isDense: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1468** `border: InputBorder.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1469** `enabledBorder: InputBorder.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1470** `focusedBorder: InputBorder.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1471** `disabledBorder: InputBorder.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1472** `errorBorder: InputBorder.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1473** `focusedErrorBorder: InputBorder.none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1474** `contentPadding: const EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1475** `4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1476** `10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1477** `8,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1478** `10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1479** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1480** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1481** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1482** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1483** `if (_text.text.trim().isEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1484** `GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1485** `onLongPressStart: (_) => _startRecording(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1486** `onLongPressEnd: (_) => _stopRecording(send: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1487** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1488** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1489** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1490** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1491** `color: _recording`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1492** `? AppColors.danger(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1493** `: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1494** `shape: isMonoDock`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1495** `? BoxShape.rectangle`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1496** `: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1497** `borderRadius: isMonoDock`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1498** `? BorderRadius.circular(4)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1499** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1500** `boxShadow: isNeuDock`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1501** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1502** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1503** `color: Neumorphic.shadowColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1504** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1505** `).withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1506** `offset: const Offset(2, 3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1507** `blurRadius: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1508** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1510** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1511** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1512** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1513** `_recording`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1514** `? Icons.stop_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1515** `: Icons.mic_none,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1516** `color: (isDark && (isNeuDock || isMonoDock))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1517** `? const Color(0xFF0E1812)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1518** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1519** `size: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1520** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1521** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1523** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1524** `IconButton.filled(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1525** `onPressed: _sending ? null : _sendText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1526** `style: IconButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1527** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1528** `foregroundColor:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1529** `(isDark && (isNeuDock || isMonoDock))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1530** `? const Color(0xFF0E1812)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1531** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1532** `fixedSize: const Size(40, 40),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1533** `padding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1534** `shape: isMonoDock`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1535** `? RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1536** `borderRadius: BorderRadius.circular(4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1538** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1539** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1540** `icon: _sending`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1541** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1542** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1543** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1544** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1545** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1546** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1547** `(isDark &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1548** `(isNeuDock || isMonoDock))`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1549** `? const Color(0xFF0E1812)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1550** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1551** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1553** `: const Icon(Icons.send_rounded, size: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1554** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1556** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1558** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1561** `if (isNeuDock) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1562** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1563** `decoration: Neumorphic.decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1564** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1565** `elevation: NeuElevation.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1566** `borderRadius: dockRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1567** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1568** `clipBehavior: Clip.antiAlias,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1569** `child: dockChild,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1573** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1574** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1575** `? scheme.surfaceContainerHigh.withValues(alpha: 0.96)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1576** `: Colors.white.withValues(alpha: 0.94),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1577** `elevation: isMonoDock ? 0 : 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1578** `shadowColor: isMonoDock`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1579** `? Colors.transparent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1580** `: AppColors.inkAlpha(context, 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1581** `surfaceTintColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1582** `borderRadius: dockRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1583** `clipBehavior: Clip.antiAlias,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1584** `child: DecoratedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1585** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1586** `borderRadius: dockRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1587** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1588** `color: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1589** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1590** `isMonoDock ? 0.12 : 0.08,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1591** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1592** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1593** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1594** `child: dockChild,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1595** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1597** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1598** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1599** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1601** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1602** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/chat_room_screen.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/helpers/chat_list_entries.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L3** `class ChatListEntry {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L4** `const ChatListEntry._({this.date, this.message});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `factory ChatListEntry.date(DateTime date) => ChatListEntry._(date: date);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `factory ChatListEntry.message(ChatMessage message) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `ChatListEntry._(message: message);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final DateTime? date;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final ChatMessage? message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `List<ChatListEntry> chatEntriesFor(List<ChatMessage> messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final entries = <ChatListEntry>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `DateTime? lastDay;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `for (final message in messages) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `if (message.isDeleted) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final local = message.createdAt.toLocal();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final day = DateTime(local.year, local.month, local.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `if (lastDay == null || day != lastDay) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `entries.add(ChatListEntry.date(day));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `lastDay = day;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `entries.add(ChatListEntry.message(message));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `return entries;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/room/helpers/chat_list_entries.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/helpers/reply_snippets.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `String chatReplySnippet(BuildContext context, ChatReplyPreview? reply) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `if (reply == null || reply.isDeleted) return l10n.chatReplyDeleted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `return switch (reply.messageType) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `ChatMessageType.text =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `(reply.textContent ?? '').trim().isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `? l10n.chatMessageDeleted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `: reply.textContent!.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `ChatMessageType.image => l10n.chatPreviewImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `ChatMessageType.video => l10n.chatPreviewVideo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `ChatMessageType.voice => l10n.chatPreviewVoice,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `ChatMessageType.file =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `(reply.textContent?.trim().isNotEmpty == true)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `? reply.textContent!.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `: l10n.chatPreviewFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `String chatReplySnippetFromMessage(BuildContext context, ChatMessage message) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `return chatReplySnippet(context, ChatReplyPreview.fromMessage(message));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/room/helpers/reply_snippets.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/attach_chip.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class AttachChip extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const AttachChip({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final radius = isMono ? 4.0 : 14.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `return InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `width: 96,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `padding: const EdgeInsets.symmetric(vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `color: AppColors.accent(context).withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `? Border.all(color: AppColors.inkAlpha(context, 0.14))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `Icon(icon, color: AppColors.accent(context)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `).textTheme.labelSmall?.copyWith(fontWeight: FontWeight.w600),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/attach_chip.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/chat_date_chip.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L7** `class ChatDateChip extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const ChatDateChip({super.key, required this.date});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final DateTime date;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `String _label(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final today = DateTime(now.year, now.month, now.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final yesterday = today.subtract(const Duration(days: 1));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final day = DateTime(date.year, date.month, date.day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `if (day == today) return l10n.chatDateToday;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `if (day == yesterday) return l10n.chatDateYesterday;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final locale = Localizations.localeOf(context).toString();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `if (day.year == today.year) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `return DateFormat.MMMMd(locale).format(day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `return DateFormat.yMMMMd(locale).format(day);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `padding: const EdgeInsets.symmetric(vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `color: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `? scheme.surfaceContainerHighest.withValues(alpha: 0.9)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `: AppColors.sand.withValues(alpha: 0.95),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `borderRadius: BorderRadius.circular(isMono ? 4 : 999),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `? Border.all(color: AppColors.inkAlpha(context, 0.12))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `boxShadow: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `color: AppColors.inkAlpha(context, 0.06),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `blurRadius: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `offset: const Offset(0, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `child: Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `_label(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `style: Theme.of(context).textTheme.labelMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `color: AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `fontFeatures: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `? const [FontFeature.tabularFigures()]`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/room/widgets/chat_date_chip.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/chat_file.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L13** `class ChatFile extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14** `const ChatFile({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `required this.fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String fileName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `static String _mimeForName(String name) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final ext = name.contains('.') ? name.split('.').last.toLowerCase() : '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `return switch (ext) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `'csv' => 'text/csv',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `'txt' => 'text/plain',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `'pdf' => 'application/pdf',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `'json' => 'application/json',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `'doc' => 'application/msword',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `'docx' =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `'application/vnd.openxmlformats-officedocument.wordprocessingml.document',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `'xls' => 'application/vnd.ms-excel',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `'xlsx' =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `'ppt' => 'application/vnd.ms-powerpoint',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `'pptx' =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `'application/vnd.openxmlformats-officedocument.presentationml.presentation',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `'zip' => 'application/zip',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `'png' => 'image/png',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `'jpg' || 'jpeg' => 'image/jpeg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'gif' => 'image/gif',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `'webp' => 'image/webp',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `'mp4' => 'video/mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `'mov' => 'video/quicktime',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `'mp3' => 'audio/mpeg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `'m4a' => 'audio/mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `'wav' => 'audio/wav',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `'py' => 'text/x-python',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `_ => 'application/octet-stream',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `Future<void> _open(BuildContext context) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `if (kIsWeb) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final uri = Uri.tryParse(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `if (uri != null &&`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `await launchUrl(uri, mode: LaunchMode.platformDefault)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `throw StateError('launch failed');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `final ext = fileName.contains('.')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `? fileName.split('.').last`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `: ChatMediaCacheService.extensionForUrl(url, fallback: 'bin');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final cached = await ChatMediaCacheService.instance.getOrFetch(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `preferredExtension: ext,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `if (cached == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final uri = Uri.tryParse(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `if (uri != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `await launchUrl(uri, mode: LaunchMode.externalApplication)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `throw StateError('download failed');`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L84** `final safeName = fileName.replaceAll(RegExp(r'[^\w.\-]+'), '_');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `final tmp = File(p.join((await getTemporaryDirectory()).path, safeName));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `await cached.copy(tmp.path);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `await SharePlus.instance.share(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `ShareParams(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `files: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `XFile(tmp.path, mimeType: _mimeForName(safeName), name: safeName),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `fileNameOverrides: [safeName],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `downloadFallbackEnabled: true,`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `} catch (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `if (!context.mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `).showSnackBar(SnackBar(content: Text(l10n.chatOpenFileFailed)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final radius = isMono ? 4.0 : 12.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `onTap: () => _open(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `constraints: const BoxConstraints(minWidth: 180, maxWidth: 260),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `color: color.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `border: Border.all(color: color.withValues(alpha: 0.18)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `Icon(Icons.insert_drive_file_rounded, color: color, size: 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `Flexible(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `context.l10n.chatOpenFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `color: color.withValues(alpha: 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `const SizedBox(width: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `Icons.download_rounded,`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L157** `color: color.withValues(alpha: 0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/room/widgets/chat_file.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/chat_image.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L10** `class ChatImage extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L11** `const ChatImage({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.galleryItems = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `this.galleryIndex = -1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final List<ChatMediaItem> galleryItems;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final int galleryIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `State<ChatImage> createState() => _ChatImageState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `class _ChatImageState extends State<ChatImage> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L27** `File? _cachedFile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `bool _resolving = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `_resolve();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `void didUpdateWidget(covariant ChatImage oldWidget) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `super.didUpdateWidget(oldWidget);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `if (oldWidget.url != widget.url) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `_cachedFile = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `_resolving = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `_resolve();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `Future<void> _resolve() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `if (kIsWeb || !ChatMediaCacheService.instance.isSupported) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `if (mounted) setState(() => _resolving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final file = await ChatMediaCacheService.instance.getOrFetch(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `preferredExtension: ChatMediaCacheService.extensionForUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `fallback: 'jpg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `_cachedFile = file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `_resolving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `Widget _buildImage({required BoxFit fit, double? width, double? height}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final file = _cachedFile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `if (file != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `return Image.file(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `file,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `fit: fit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `width: width,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `height: height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `errorBuilder: (_, _, _) => const Icon(Icons.broken_image_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `return CachedNetworkImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `imageUrl: widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `width: width,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `height: height,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `fit: fit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `placeholder: (_, _) => SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `width: width ?? 220,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `height: height ?? 180,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `child: const Center(child: CircularProgressIndicator(strokeWidth: 2)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `errorWidget: (_, _, _) => const Icon(Icons.broken_image_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `if (_resolving && _cachedFile == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `return const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `width: 220,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `height: 180,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `child: Center(child: CircularProgressIndicator(strokeWidth: 2)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `return ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `borderRadius: BorderRadius.circular(isMono ? 4 : 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `child: GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `onTap: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final items = widget.galleryItems.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `? widget.galleryItems`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `ChatMediaItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `id: widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `url: widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `kind: ChatMediaKind.image,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final index = widget.galleryIndex >= 0 ? widget.galleryIndex : 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `showChatMediaGallery(context, items: items, initialIndex: index);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `child: _buildImage(fit: BoxFit.cover, width: 220, height: 180),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/room/widgets/chat_image.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/chat_video.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L9** `class ChatVideo extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `const ChatVideo({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.galleryItems = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.galleryIndex = -1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final List<ChatMediaItem> galleryItems;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final int galleryIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `State<ChatVideo> createState() => _ChatVideoState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `class _ChatVideoState extends State<ChatVideo> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26** `File? _thumb;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `bool _loading = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `_loadThumb();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `void didUpdateWidget(covariant ChatVideo oldWidget) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `super.didUpdateWidget(oldWidget);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (oldWidget.url != widget.url) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `_thumb = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `_loading = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `_loadThumb();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `Future<void> _loadThumb() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final file = await ChatVideoThumbnailService.instance.getThumbnail(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `_thumb = file;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `_loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `void _openFullscreen() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final items = widget.galleryItems.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `? widget.galleryItems`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `ChatMediaItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `id: widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `url: widget.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `kind: ChatMediaKind.video,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final index = widget.galleryIndex >= 0 ? widget.galleryIndex : 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `showChatMediaGallery(context, items: items, initialIndex: index);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `@override`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `borderRadius: BorderRadius.circular(isMono ? 4 : 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `width: 220,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `height: 180,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `child: Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `color: const Color(0xFF1C1C1E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `onTap: _openFullscreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `if (_thumb != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `Image.file(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `_thumb!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `fit: BoxFit.cover,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `errorBuilder: (_, _, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `const ColoredBox(color: Color(0xFF2C2C2E)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `ColoredBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `color: const Color(0xFF2C2C2E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `child: _loading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `? const Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `width: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `height: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `color: Colors.white54,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `Icons.videocam_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `size: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `color: Colors.white.withValues(alpha: 0.22),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `const ColoredBox(color: Color(0x33000000)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `Icons.play_circle_filled_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `size: 52,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `right: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `bottom: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `horizontal: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `vertical: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `color: Colors.black54,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `borderRadius: BorderRadius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `context.l10n.chatPreviewVideo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/chat_video.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/chat_voice.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class ChatVoice extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `const ChatVoice({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.url,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.timeLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.isReadByOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final String url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final String timeLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final bool isMine;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final bool isReadByOthers;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `State<ChatVoice> createState() => _ChatVoiceState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `class _ChatVoiceState extends State<ChatVoice> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `final _voice = VoicePlaybackController.instance;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `late final List<double> _bars = _waveformFor(widget.url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `static const _bg = Color(0xFF1C1C1E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `static const _playCircle = Color(0xFF2C2C2E);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `static const _played = Color(0xFFB8B0E8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `static const _unplayed = Color(0xFF8E8E93);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `static const _meta = Color(0xFFAEAEB2);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `static List<double> _waveformFor(String seed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `var hash = 2166136261;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `for (final unit in seed.codeUnits) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `hash ^= unit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `hash = (hash * 16777619) & 0x7fffffff;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `final bars = <double>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `var x = hash == 0 ? 1 : hash;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `for (var i = 0; i < 36; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `x = (1103515245 * x + 12345) & 0x7fffffff;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final n = (x % 1000) / 1000.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final envelope = 0.55 + 0.45 * (1 - (((i / 35) - 0.5).abs() * 1.4));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `bars.add((0.22 + n * 0.78) * envelope.clamp(0.35, 1.0));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `return bars;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `static String _formatDuration(Duration d) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final total = d.inSeconds.clamp(0, 99 * 60);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final m = total ~/ 60;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final s = total % 60;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `return '$m:${s.toString().padLeft(2, '0')}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `_voice.addListener(_onVoiceChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `@override`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `_voice.removeListener(_onVoiceChanged);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `void _onVoiceChanged() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `if (mounted) setState(() {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Future<void> _toggle() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `await _voice.toggleRemote(widget.url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `SnackBar(content: Text(context.l10n.voiceCouldNotPlay('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `final active = _voice.activeKey == widget.url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final playing = _voice.isPlaying(widget.url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final loading = _voice.isLoading(widget.url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `final duration = active ? _voice.duration : Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final position = active ? _voice.position : Duration.zero;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final progress = (!active || duration.inMilliseconds <= 0)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `? 0.0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `: (position.inMilliseconds / duration.inMilliseconds).clamp(0.0, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final durationLabel = duration > Duration.zero`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `? _formatDuration(playing ? duration - position : duration)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `: '0:00';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `final playRadius = BorderRadius.circular(isMono ? 4 : 20);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final playedColor = isMono ? const Color(0xFFE8E8E8) : _played;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final unplayedColor = isMono ? const Color(0xFF6E6E73) : _unplayed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `constraints: const BoxConstraints(minWidth: 220, maxWidth: 280),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `padding: const EdgeInsets.fromLTRB(10, 10, 12, 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `color: _bg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `borderRadius: BorderRadius.circular(isMono ? 6 : 999),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `? Border.all(color: Colors.white.withValues(alpha: 0.10))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `color: isMono ? Colors.white : _playCircle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `borderRadius: playRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `borderRadius: playRadius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `onTap: loading ? null : _toggle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `child: loading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `color: isMono ? Colors.black : Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `playing`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `? Icons.pause_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `: Icons.play_arrow_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `color: isMono ? Colors.black : Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `size: playing ? 22 : 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `child: SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `height: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `child: CustomPaint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `painter: VoiceWaveformPainter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `bars: _bars,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `progress: progress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `playedColor: playedColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `unplayedColor: unplayedColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `crossAxisAlignment: CrossAxisAlignment.end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `durationLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `color: _meta,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `height: 1.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `fontFeatures: [FontFeature.tabularFigures()],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `widget.timeLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `color: _meta,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `height: 1.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `fontFeatures: [FontFeature.tabularFigures()],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `if (widget.isMine) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `const SizedBox(width: 3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `widget.isReadByOthers`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `? Icons.done_all_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `: Icons.done_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `size: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `color: widget.isReadByOthers`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `? (isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `? Colors.white.withValues(alpha: 0.85)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `: const Color(0xFF9ED0FF))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `: _meta,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `class VoiceWaveformPainter extends CustomPainter {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L214** `const VoiceWaveformPainter({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `required this.bars,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `required this.progress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `required this.playedColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `required this.unplayedColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `final List<double> bars;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `final double progress;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `final Color playedColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `final Color unplayedColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `void paint(Canvas canvas, Size size) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `if (bars.isEmpty || size.width <= 0 || size.height <= 0) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `const gap = 2.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `final barWidth = ((size.width - gap * (bars.length - 1)) / bars.length)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `.clamp(1.5, 3.5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `final total = bars.length * barWidth + (bars.length - 1) * gap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `var x = (size.width - total) / 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `final playedUntil = progress * bars.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `for (var i = 0; i < bars.length; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `final h = (bars[i] * size.height).clamp(4.0, size.height);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `final top = (size.height - h) / 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `final rrect = RRect.fromRectAndRadius(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `Rect.fromLTWH(x, top, barWidth, h),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `const Radius.circular(999),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `final paint = Paint()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `..color = i + 1 <= playedUntil ? playedColor : unplayedColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `canvas.drawRRect(rrect, paint);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `x += barWidth + gap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `bool shouldRepaint(covariant VoiceWaveformPainter oldDelegate) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `oldDelegate.progress != progress ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `oldDelegate.playedColor != playedColor ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `oldDelegate.unplayedColor != unplayedColor ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `!identical(oldDelegate.bars, bars);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/chat_voice.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/edit_message_dialog.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L4** `class EditMessageDialog extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L5** `const EditMessageDialog({super.key, required this.initialText});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `final String initialText;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `State<EditMessageDialog> createState() => _EditMessageDialogState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `class _EditMessageDialogState extends State<EditMessageDialog> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L14** `late final TextEditingController _controller;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `_controller = TextEditingController(text: widget.initialText);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `_controller.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `title: Text(l10n.chatEditTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `content: TextField(controller: _controller, maxLines: 4, autofocus: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `onPressed: () => Navigator.pop(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `onPressed: () => Navigator.pop(context, _controller.text.trim()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `child: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/chat/room/widgets/edit_message_dialog.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/message_bubble.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L25** `class MessageBubble extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26** `const MessageBubble({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `required this.message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `required this.isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `this.myUserId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `this.galleryItems = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `this.highlighted = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `this.selectionMode = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `this.selected = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `this.selectable = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `this.onSelectionToggle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `this.onSelect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `this.onJumpToReply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `this.onReply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `this.onReact,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `this.onEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `this.onDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final ChatMessage message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final bool isMine;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final String? myUserId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final List<ChatMediaItem> galleryItems;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final bool highlighted;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final bool selectionMode;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final bool selected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final bool selectable;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final VoidCallback? onSelectionToggle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final VoidCallback? onSelect;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `final ValueChanged<String>? onJumpToReply;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final VoidCallback? onReply;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final ValueChanged<String>? onReact;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final VoidCallback? onEdit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final VoidCallback? onDelete;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `static const _nameColors = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `Color(0xFFE57373),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Color(0xFF5B9BD5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `Color(0xFF7CB342),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `Color(0xFFAB47BC),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `Color(0xFFFFB300),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `Color(0xFF26A69A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `Color(0xFFEC407A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `Color(0xFF42A5F5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `static const _monoNameColors = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `Color(0xFF111111),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Color(0xFF3A3A3A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `Color(0xFF5A5A5A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `Color(0xFF7A7A7A),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `Color(0xFF2E2E2E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `Color(0xFF4E4E4E),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `static const _monoNameColorsDark = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Color(0xFFF5F5F5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `Color(0xFFD0D0D0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `Color(0xFFB0B0B0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `Color(0xFF909090),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `Color(0xFFE0E0E0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `Color(0xFFC0C0C0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `Color _senderColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `String senderId, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `required bool monochrome,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `required bool isDark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `var hash = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `for (final c in senderId.codeUnits) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `hash = (hash + c) & 0x7fffffff;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final palette = monochrome`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `? (isDark ? _monoNameColorsDark : _monoNameColors)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `: _nameColors;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `return palette[hash % palette.length];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `final tokens = AppStyleTokens.maybeOf(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final isNeu = tokens?.isSoftNeumorphism ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `final isMono = tokens?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final time = DateFormat.Hm().format(message.createdAt.toLocal());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `final Color bubbleColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final Color textColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `bubbleColor = isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `? (isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `? SoftNeumorphismColors.emerald`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `: SoftNeumorphismColors.sage)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `: Neumorphic.surface(context, elevation: NeuElevation.raised);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `textColor = isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `? (isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `? const Color(0xFF0E1812)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `: SoftNeumorphismColors.morningSurfaceHigh)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `: scheme.onSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `} else if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `bubbleColor = isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `? AppColors.ink(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `: (isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `? Colors.white.withValues(alpha: 0.06)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `: Colors.black.withValues(alpha: 0.04));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `textColor = isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `? (isDark ? Colors.black : Colors.white)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `: scheme.onSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `bubbleColor = isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `? (isDark ? const Color(0xFF3E2C23) : AppColors.coffeeBrown)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `: (isDark ? AppColors.darkSurfaceElevated : Colors.white);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `textColor = isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `? (isDark ? AppColors.darkTextPrimary : Colors.white)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `: scheme.onSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `final myEmoji = message.myReactionEmoji(myUserId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final summary = message.reactionSummary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `final hasReactions = summary.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `final isAnimatedEmoji =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `!message.isDeleted &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `message.messageType == ChatMessageType.text &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `isSingleAnimatedEmojiMessage(message.textContent);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `final isVoiceMessage =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `!message.isDeleted && message.messageType == ChatMessageType.voice;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `final voiceHasCaption =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `isVoiceMessage && (message.textContent?.trim().isNotEmpty ?? false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `final voiceHasReply =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `isVoiceMessage &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `(message.replyTo != null || message.replyToMessageId != null);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `final isStandaloneVoice =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `isVoiceMessage && !voiceHasCaption && !voiceHasReply;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final metaColor = (isAnimatedEmoji || isStandaloneVoice)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `? AppColors.inkAlpha(context, 0.45)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `: textColor.withValues(alpha: 0.65);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `Widget messageColumn({required Color bodyTextColor}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `if (!isMine)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `padding: const EdgeInsets.only(bottom: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `child: SenderHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `message: message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `nameColor: _senderColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `message.senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `monochrome: isMono,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `isDark: isDark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `if (message.isDeleted)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `context.l10n.chatMessageDeleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `fontStyle: FontStyle.italic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `color: bodyTextColor.withValues(alpha: 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `else ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `if (message.replyTo != null || message.replyToMessageId != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `padding: const EdgeInsets.only(bottom: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `child: ReplyQuote(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `reply: message.replyTo,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `accent: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `? bodyTextColor.withValues(alpha: 0.72)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `: (isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `? (isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `? const Color(0xFF9ED0FF)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `: Colors.white.withValues(alpha: 0.9))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `: AppColors.accent(context)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `onSurface: bodyTextColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `sharp: isMono,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `onTap: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `final id = message.replyTo?.id ?? message.replyToMessageId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `if (id != null) onJumpToReply?.call(id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `MessageBody(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `message: message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `textColor: bodyTextColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `galleryItems: galleryItems,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `timeLabel: time,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `isMine: isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `if (message.textContent != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `message.textContent!.isNotEmpty &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `message.messageType != ChatMessageType.text &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `message.messageType != ChatMessageType.file)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `padding: const EdgeInsets.only(top: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `message.textContent!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `style: TextStyle(color: bodyTextColor, height: 1.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `if (!isVoiceMessage || voiceHasCaption) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `if (message.wasEdited)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `padding: const EdgeInsets.only(right: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `context.l10n.chatEdited,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `style: TextStyle(fontSize: 10, color: metaColor),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `Text(time, style: TextStyle(fontSize: 10, color: metaColor)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `if (isMine && !message.isDeleted) ...[`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `const SizedBox(width: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `message.isReadByOthers`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `? Icons.done_all_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `: Icons.done_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `size: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `color: message.isReadByOthers`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `? (isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `? bodyTextColor.withValues(alpha: 0.72)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `: const Color(0xFF9ED0FF))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `: metaColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `final highlightAccent = AppColors.accent(context).withValues(alpha: 0.85);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `final highlightGlow = AppColors.accent(context).withValues(alpha: 0.35);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `Widget bubbleBody;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `if (isAnimatedEmoji || isStandaloneVoice) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `final radius = BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `isMono ? 6 : (isStandaloneVoice ? 999 : 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `bubbleBody = AnimatedScale(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `scale: highlighted ? (isMono ? 1.02 : 1.06) : 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `duration: const Duration(milliseconds: 240),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `curve: isMono ? Curves.easeOutCubic : Curves.easeOutBack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `duration: const Duration(milliseconds: 240),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `curve: Curves.easeOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `color: highlighted ? highlightAccent : Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `width: 1.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `boxShadow: highlighted && !isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `color: highlightGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `offset: const Offset(0, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `child: messageColumn(bodyTextColor: textColor),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `final radius = isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `? BorderRadius.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `topLeft: const Radius.circular(6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `topRight: const Radius.circular(6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `bottomLeft: Radius.circular(isMine ? 6 : 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `bottomRight: Radius.circular(isMine ? 2 : 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `: BorderRadius.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `topLeft: const Radius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `topRight: const Radius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `bottomLeft: Radius.circular(isMine ? 20 : 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `bottomRight: Radius.circular(isMine ? 6 : 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `final List<BoxShadow> bubbleShadows;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `if (isMono) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `bubbleShadows = const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `} else if (highlighted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `bubbleShadows = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `color: highlightGlow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `blurRadius: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `offset: const Offset(0, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `} else if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `bubbleShadows = isMine`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `color: Neumorphic.shadowColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `).withValues(alpha: isDark ? 0.45 : 0.20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `offset: const Offset(3, 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `blurRadius: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `: Neumorphic.shadows(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `elevation: NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `intensity: 0.75,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `bubbleShadows = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `color: Colors.black.withValues(alpha: isDark ? 0.25 : 0.04),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `blurRadius: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `offset: const Offset(0, 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `final Color bubbleBorder;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `if (highlighted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `bubbleBorder = highlightAccent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `} else if (isMono && !isMine) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `bubbleBorder = AppColors.inkAlpha(context, 0.12);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `bubbleBorder = Colors.transparent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `bubbleBody = AnimatedScale(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `scale: highlighted ? (isMono ? 1.01 : 1.03) : 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `duration: const Duration(milliseconds: 240),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `curve: isMono ? Curves.easeOutCubic : Curves.easeOutBack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `duration: const Duration(milliseconds: 240),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `curve: Curves.easeOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `isMono ? 12 : 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `isMono ? 9 : 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `isMono ? 12 : 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `isMono ? 7 : 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `color: bubbleColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `color: bubbleBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `width: highlighted ? 1.5 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `boxShadow: bubbleShadows,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `child: messageColumn(bodyTextColor: textColor),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `final bubble = GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `onTap: selectionMode`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `? (selectable ? onSelectionToggle : null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `onLongPress: message.isDeleted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `: selectionMode`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `? (selectable ? onSelectionToggle : null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `HapticFeedback.mediumImpact();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `_showActions(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `child: bubbleBody,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `final bubbleWithReactions = Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `crossAxisAlignment: isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `? CrossAxisAlignment.end`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `bubble,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `if (hasReactions)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `Transform.translate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `offset: const Offset(0, -6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `left: isMine ? 0 : 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `right: isMine ? 4 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `child: Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `spacing: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `runSpacing: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `for (final item in summary)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `ReactionPill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `emoji: item.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `count: item.count,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `mine: myEmoji == item.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `onTap: onReact == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `: () => onReact!(item.emoji),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `return LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `final mineMax = (constraints.maxWidth * 0.78).clamp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `160.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `constraints.maxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `final incomingMax = (constraints.maxWidth - 36).clamp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `120.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `constraints.maxWidth * 0.85,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `if (isMine) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `final child = Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `alignment: Alignment.centerRight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `constraints: BoxConstraints(maxWidth: mineMax),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `padding: EdgeInsets.only(top: 4, bottom: hasReactions ? 10 : 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `child: bubbleWithReactions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `final wrapped = selectionMode && selectable`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `? _selectionRow(context, child)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `: child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `if (onReply == null || selectionMode) return wrapped;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `return SwipeToReply(onReply: onReply!, child: wrapped);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `final initials = message.displayName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `.split(RegExp(r'\s+'))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `.where((p) => p.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `.take(2)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `.map((p) => p[0].toUpperCase())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `.join();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `final senderTone = _senderColor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `message.senderId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `monochrome: isMono,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `isDark: isDark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `final incomingAvatar = isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `? Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `width: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `height: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `clipBehavior: Clip.antiAlias,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `color: senderTone.withValues(alpha: 0.16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `borderRadius: BorderRadius.circular(4),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `border: Border.all(color: senderTone.withValues(alpha: 0.28)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `image:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `message.senderAvatarUrl != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `message.senderAvatarUrl!.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `? DecorationImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `image: CachedNetworkImageProvider(`
Riverpod binding. Name it after the value it exposes.

**L489** `message.senderAvatarUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `fit: BoxFit.cover,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `child:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `message.senderAvatarUrl == null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `message.senderAvatarUrl!.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `? Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `initials.isEmpty ? '?' : initials,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `color: senderTone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `: CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `radius: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `backgroundColor: senderTone.withValues(alpha: 0.22),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `backgroundImage:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `message.senderAvatarUrl != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `message.senderAvatarUrl!.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `? CachedNetworkImageProvider(message.senderAvatarUrl!)`
Riverpod binding. Name it after the value it exposes.

**L515** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `child:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `message.senderAvatarUrl == null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `message.senderAvatarUrl!.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `? Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `initials.isEmpty ? '?' : initials,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `color: senderTone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `final incoming = Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `padding: EdgeInsets.only(top: 4, bottom: hasReactions ? 10 : 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `crossAxisAlignment: CrossAxisAlignment.end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `incomingAvatar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `Flexible(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `child: Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `alignment: Alignment.centerLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `child: ConstrainedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `constraints: BoxConstraints(maxWidth: incomingMax),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `child: bubbleWithReactions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `final wrapped = selectionMode && selectable`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `? _selectionRow(context, incoming)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `: incoming;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `if (onReply == null || selectionMode) return wrapped;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `return SwipeToReply(onReply: onReply!, child: wrapped);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `Widget _selectionRow(BuildContext context, Widget child) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `final accent = AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `final muted = AppColors.inkAlpha(context, 0.35);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `return GestureDetector(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `behavior: HitTestBehavior.opaque,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `onTap: selectable ? onSelectionToggle : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `duration: const Duration(milliseconds: 160),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `color: selected ? accent.withValues(alpha: 0.08) : Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `padding: const EdgeInsets.symmetric(horizontal: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `crossAxisAlignment: CrossAxisAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `selected ? Icons.check_circle_rounded : Icons.circle_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `size: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `color: selectable`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `? (selected ? accent : muted)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `: muted.withValues(alpha: 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `child: IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `ignoring: selectable,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `child: child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L585** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `void _showActions(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `final box = context.findRenderObject() as RenderBox?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `if (box == null || !box.hasSize) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `final overlay = Overlay.maybeOf(context, rootOverlay: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `if (overlay == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `final origin = box.localToGlobal(Offset.zero);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `final rect = origin & box.size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `final textToCopy = message.isDeleted ? null : message.textContent?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L603** `final canCopy = textToCopy != null && textToCopy.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `late OverlayEntry entry;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `void dismiss() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `if (entry.mounted) entry.remove();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `entry = OverlayEntry(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `type: MaterialType.transparency,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L615** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `Positioned.fill(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `child: GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `behavior: HitTestBehavior.opaque,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `onTap: dismiss,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `child: ColoredBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `color: Colors.black.withValues(alpha: 0.42),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L622** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `MessageActionPopover(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `anchorRect: rect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `isMine: isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L628** `emojis: onReact == null ? const [] : chatReactionEmojis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L629** `onEmoji: onReact == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `: (emoji) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `dismiss();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `onReact!(emoji);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `onReply: onReply == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `dismiss();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `onReply!();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `},`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `onCopy: !canCopy`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `await Clipboard.setData(ClipboardData(text: textToCopy));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `dismiss();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L648** `SnackBar(content: Text(context.l10n.chatCopied)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `onEdit: onEdit == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L654** `: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L655** `dismiss();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `onEdit!();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `onSelect: onSelect == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `dismiss();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `onSelect!();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L663** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L664** `onDelete: onDelete == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L665** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `dismiss();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `onDelete!();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L674** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `overlay.insert(entry);`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L680** `class MessageActionPopover extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L681** `const MessageActionPopover({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L682** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L683** `required this.anchorRect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L684** `required this.isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L685** `required this.emojis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L686** `this.onEmoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L687** `this.onReply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L688** `this.onCopy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L689** `this.onEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L690** `this.onSelect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L691** `this.onDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L694** `final Rect anchorRect;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `final bool isMine;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L696** `final List<String> emojis;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L697** `final ValueChanged<String>? onEmoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `final VoidCallback? onReply;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L699** `final VoidCallback? onCopy;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L700** `final VoidCallback? onEdit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L701** `final VoidCallback? onSelect;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L702** `final VoidCallback? onDelete;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L704** `static const _menuWidth = 220.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L705** `static const _reactionHeight = 60.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L708** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L709** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `final media = MediaQuery.of(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L711** `final screen = media.size;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L712** `final pad = media.padding;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L713** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L715** `final surface = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L716** `? const Color(0xE62A2F36)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L717** `: Colors.white.withValues(alpha: 0.96);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L718** `final dividerColor = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L719** `? Colors.white.withValues(alpha: 0.08)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L720** `: Colors.black.withValues(alpha: 0.08);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L721** `final itemColor = isDark ? Colors.white : AppColors.darkChocolate;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L723** `final reactionWidth = emojis.isEmpty ? 0.0 : (emojis.length * 48.0) + 20;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L724** `final panelWidth = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L725** `if (emojis.isNotEmpty) reactionWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L726** `_menuWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L727** `].fold<double>(0, (a, b) => a > b ? a : b);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L729** `final itemCount = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L730** `onReply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L731** `onCopy,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L732** `onEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L733** `onSelect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L734** `onDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L735** `].where((callback) => callback != null).length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L736** `final menuHeight = itemCount * 44.0 + 8;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L737** `final panelHeight =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L738** `(emojis.isEmpty ? 0.0 : _reactionHeight + 8) + menuHeight;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L739** `final bottomReserve = pad.bottom + media.viewInsets.bottom + 108;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L740** `final spaceBelow = screen.height - bottomReserve - anchorRect.bottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L741** `final spaceAbove = anchorRect.top - pad.top - 8;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L742** `final placeAbove =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L743** `spaceBelow < panelHeight + 12 && spaceAbove >= panelHeight * 0.45;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L744** `final stackTop = placeAbove`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L745** `? (anchorRect.top - panelHeight - 10).clamp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L746** `pad.top + 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L747** `math.max(pad.top + 8, screen.height - panelHeight - bottomReserve),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L749** `: (anchorRect.bottom + 10).clamp(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L750** `pad.top + 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L751** `math.max(pad.top + 8, screen.height - panelHeight - bottomReserve),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L754** `var left = isMine ? anchorRect.right - panelWidth : anchorRect.left;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L755** `left = left.clamp(12.0, screen.width - panelWidth - 12);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L757** `Widget menuDivider() =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L758** `Divider(height: 1, thickness: 1, color: dividerColor);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L760** `final menuItems = <Widget>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L761** `if (onReply != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L762** `PopoverMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L763** `icon: Icons.reply_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L764** `label: l10n.chatReply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L765** `color: itemColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L766** `onTap: onReply!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L767** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L768** `if (onReply != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L769** `(onCopy != null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L770** `onEdit != null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L771** `onSelect != null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L772** `onDelete != null))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L773** `menuDivider(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L774** `if (onCopy != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L775** `PopoverMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L776** `icon: Icons.copy_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L777** `label: l10n.chatCopyText,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L778** `color: itemColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L779** `onTap: onCopy!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L780** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L781** `if (onEdit != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L782** `PopoverMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L783** `icon: Icons.edit_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L784** `label: l10n.commonEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L785** `color: itemColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L786** `onTap: onEdit!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L787** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L788** `if (onSelect != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L789** `PopoverMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L790** `icon: Icons.check_circle_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L791** `label: l10n.chatSelect,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L792** `color: itemColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L793** `onTap: onSelect!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L794** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L795** `if (onDelete != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L796** `if (onReply != null ||`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L797** `onCopy != null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L798** `onEdit != null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L799** `onSelect != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L800** `menuDivider(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L801** `PopoverMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L802** `icon: Icons.delete_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L803** `label: l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L804** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L805** `onTap: onDelete!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L806** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L810** `return Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L811** `left: left,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L812** `top: stackTop.toDouble(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L813** `child: Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L814** `type: MaterialType.transparency,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L815** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L816** `crossAxisAlignment: isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L817** `? CrossAxisAlignment.end`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L818** `: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L819** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L820** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L821** `if (emojis.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L822** `ReactionBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L823** `emojis: emojis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L824** `surface: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L825** `onEmoji: onEmoji!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L826** `showCaretBelow: placeAbove,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L827** `isMine: isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L828** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L829** `if (emojis.isNotEmpty && menuItems.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L830** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L831** `if (menuItems.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L832** `Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L833** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L834** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L835** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L836** `false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L837** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L838** `width: _menuWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L839** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L840** `color: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L841** `borderRadius: BorderRadius.circular(isMono ? 6 : 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L842** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L843** `? Border.all(color: AppColors.inkAlpha(context, 0.12))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L844** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L845** `boxShadow: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L846** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L847** `: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L848** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L849** `color: Colors.black.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L850** `blurRadius: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L851** `offset: const Offset(0, 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L852** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L854** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L855** `clipBehavior: Clip.antiAlias,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L856** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L857** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L858** `children: menuItems,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L859** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L861** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L862** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L864** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L865** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L870** `class ReactionBar extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L871** `const ReactionBar({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L872** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L873** `required this.emojis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L874** `required this.surface,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L875** `required this.onEmoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L876** `required this.showCaretBelow,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L877** `required this.isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L880** `final List<String> emojis;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L881** `final Color surface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L882** `final ValueChanged<String> onEmoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L883** `final bool showCaretBelow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L884** `final bool isMine;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L886** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L887** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L888** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L889** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L890** `final barRadius = isMono ? 6.0 : 999.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L891** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L892** `crossAxisAlignment: isMine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L893** `? CrossAxisAlignment.end`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L894** `: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L895** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L896** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L897** `if (!showCaretBelow)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L898** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L899** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L900** `left: isMine ? 0 : 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L901** `right: isMine ? 22 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L902** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L903** `child: CustomPaint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L904** `size: const Size(14, 7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L905** `painter: CaretPainter(color: surface, pointUp: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L906** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L907** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L908** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L909** `padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L910** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L911** `color: surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L912** `borderRadius: BorderRadius.circular(barRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L913** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L914** `? Border.all(color: AppColors.inkAlpha(context, 0.12))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L915** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L916** `boxShadow: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L917** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L918** `: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L919** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L920** `color: Colors.black.withValues(alpha: 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L921** `blurRadius: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L922** `offset: const Offset(0, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L923** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L925** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L926** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L927** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L928** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L929** `for (final emoji in emojis)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L930** `InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L931** `borderRadius: BorderRadius.circular(isMono ? 4 : 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L932** `onTap: () => onEmoji(emoji),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L933** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L934** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L935** `horizontal: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L936** `vertical: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L937** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L938** `child: ChatLottieEmoji(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L939** `emoji: emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L940** `size: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L941** `playOnTap: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L942** `repeat: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L943** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L944** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L945** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L947** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L948** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L949** `if (showCaretBelow)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L950** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L951** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L952** `left: isMine ? 0 : 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L953** `right: isMine ? 22 : 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L954** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L955** `child: CustomPaint(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L956** `size: const Size(14, 7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L957** `painter: CaretPainter(color: surface, pointUp: false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L958** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L959** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L965** `class CaretPainter extends CustomPainter {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L966** `const CaretPainter({required this.color, required this.pointUp});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L968** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L969** `final bool pointUp;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L971** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L972** `void paint(Canvas canvas, Size size) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L973** `final path = Path();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L974** `if (pointUp) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L975** `path.moveTo(0, size.height);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L976** `path.lineTo(size.width / 2, 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L977** `path.lineTo(size.width, size.height);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L978** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L979** `path.moveTo(0, 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L980** `path.lineTo(size.width / 2, size.height);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L981** `path.lineTo(size.width, 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L983** `path.close();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L984** `canvas.drawPath(path, Paint()..color = color);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L987** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L988** `bool shouldRepaint(covariant CaretPainter oldDelegate) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L989** `oldDelegate.color != color || oldDelegate.pointUp != pointUp;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L992** `class PopoverMenuItem extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L993** `const PopoverMenuItem({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L994** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L995** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L996** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L997** `required this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L998** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1001** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1002** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1003** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1004** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1006** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1007** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1008** `return InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1009** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1010** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1011** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1012** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1013** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1014** `Icon(icon, size: 20, color: color),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1015** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1016** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1017** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1018** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1019** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1020** `color: color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1021** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1022** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1023** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1024** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1025** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1027** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1028** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1033** `class ReactionPill extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L1034** `const ReactionPill({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1035** `super.key,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1036** `required this.emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1037** `required this.count,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1038** `required this.mine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1039** `this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1042** `final String emoji;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1043** `final int count;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1044** `final bool mine;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1045** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1047** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1048** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1049** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1050** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1051** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1052** `final Color fill;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1053** `final Color border;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1054** `if (mine) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1055** `fill = AppColors.accent(context).withValues(alpha: isDark ? 0.32 : 0.20);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1056** `border = AppColors.accent(context).withValues(alpha: 0.85);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1057** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1058** `fill = isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1059** `? AppColors.darkSurfaceHighest`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1060** `: AppColors.sand.withValues(alpha: 0.95);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1061** `border = AppColors.inkAlpha(context, 0.12);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1063** `final chipRadius = isMono ? 4.0 : 999.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1064** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1065** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1066** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1067** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1068** `borderRadius: BorderRadius.circular(chipRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1069** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1070** `padding: const EdgeInsets.fromLTRB(6, 3, 8, 3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1071** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1072** `color: fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1073** `borderRadius: BorderRadius.circular(chipRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1074** `border: Border.all(color: border),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1075** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1076** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1077** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1078** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1079** `ChatLottieEmoji(emoji: emoji, size: 20, repeat: false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1080** `const SizedBox(width: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1081** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1082** `'$count',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1083** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1084** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1085** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1086** `color: mine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1087** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1088** `: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1089** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1090** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1092** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1093** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1094** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1099** `class SenderHeader extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L1100** `const SenderHeader({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1101** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1102** `required this.message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1103** `required this.nameColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1106** `final ChatMessage message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1107** `final Color nameColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1109** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1110** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1111** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1112** `final name = message.displayName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1113** `if (name.isEmpty) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1115** `final role = message.senderRole;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1116** `final tagLabel = switch (role) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1117** `UserRole.admin => l10n.chatRoleAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1118** `UserRole.teacher => l10n.chatRoleTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1119** `_ => null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1120** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1122** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1123** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1124** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1125** `Flexible(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1126** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1127** `name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1128** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1129** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1130** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1131** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1132** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1133** `color: nameColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1137** `if (tagLabel != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1138** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1139** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1140** `padding: const EdgeInsets.symmetric(horizontal: 7, vertical: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1141** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1142** `color: nameColor.withValues(alpha: 0.16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1143** `borderRadius: BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1144** `(AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1145** `false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1146** `? 3`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1147** `: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1150** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1151** `tagLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1152** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1153** `fontSize: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1154** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1155** `color: nameColor.withValues(alpha: 0.95),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1156** `height: 1.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1166** `class MessageBody extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L1167** `const MessageBody({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1168** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1169** `required this.message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1170** `required this.textColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1171** `this.galleryItems = const [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1172** `this.timeLabel = '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1173** `this.isMine = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1176** `final ChatMessage message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1177** `final Color textColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1178** `final List<ChatMediaItem> galleryItems;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1179** `final String timeLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1180** `final bool isMine;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1182** `int get _galleryIndex =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1183** `galleryItems.indexWhere((item) => item.id == message.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1185** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1186** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1187** `return switch (message.messageType) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1188** `ChatMessageType.text => _buildTextBody(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1189** `ChatMessageType.image => ChatImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1190** `url: message.mediaUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1191** `galleryItems: galleryItems,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1192** `galleryIndex: _galleryIndex,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1193** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1194** `ChatMessageType.video => ChatVideo(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1195** `url: message.mediaUrl!,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1196** `galleryItems: galleryItems,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1197** `galleryIndex: _galleryIndex,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1199** `ChatMessageType.voice => ChatVoice(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1200** `url: message.mediaUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1201** `timeLabel: timeLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1202** `isMine: isMine,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1203** `isReadByOthers: message.isReadByOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1205** `ChatMessageType.file => ChatFile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1206** `url: message.mediaUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1207** `fileName: message.textContent?.trim().isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1208** `? message.textContent!.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1209** `: 'file',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1210** `color: textColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1212** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1215** `Widget _buildTextBody() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1216** `final text = message.textContent ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1217** `final asset = singleAnimatedEmojiAsset(text);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1218** `final emoji = singleAnimatedEmojiGrapheme(text);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1219** `if (asset != null && emoji != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1220** `return AnimatedEmojiMessage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1221** `key: ValueKey('anim-emoji-${message.id}'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1222** `assetPath: asset,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1223** `emoji: emoji,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1224** `size: 144,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1227** `return Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1228** `text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L1229** `style: TextStyle(color: textColor, height: 1.35, fontSize: 15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/message_bubble.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/reply_composer_banner.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L9** `class ReplyComposerBanner extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `const ReplyComposerBanner({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.message,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.onClose,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `final ChatMessage message;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final VoidCallback onClose;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final accent = AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final name = message.displayName.isEmpty ? '…' : message.displayName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final snippet = chatReplySnippetFromMessage(context, message);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final showThumb =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `message.messageType == ChatMessageType.image &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `message.mediaUrl != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `message.mediaUrl!.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final radius = isMono ? 4.0 : 14.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `color: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `padding: const EdgeInsets.fromLTRB(10, 8, 4, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `width: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `borderRadius: BorderRadius.circular(isMono ? 1 : 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `if (showThumb) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `borderRadius: BorderRadius.circular(isMono ? 3 : 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `child: CachedNetworkImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `imageUrl: message.mediaUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `width: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `height: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `fit: BoxFit.cover,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `errorWidget: (_, _, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `const SizedBox(width: 36, height: 36),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `l10n.chatReplyTo(name),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `snippet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `color: AppColors.inkAlpha(context, 0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `onPressed: onClose,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `visualDensity: VisualDensity.compact,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `Icons.close_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/reply_composer_banner.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/reply_quote.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L7** `class ReplyQuote extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const ReplyQuote({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.reply,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.onSurface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `this.sharp = false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final ChatReplyPreview? reply;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final Color accent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final Color onSurface;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final bool sharp;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final name = (reply?.displayName ?? '').trim().isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `? '…'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `: reply!.displayName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final snippet = chatReplySnippet(context, reply);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final showThumb =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `reply != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `!reply!.isDeleted &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `reply!.messageType == ChatMessageType.image &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `reply!.mediaUrl != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `reply!.mediaUrl!.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final radius = sharp ? 4.0 : 10.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `color: onSurface.withValues(alpha: 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `borderRadius: BorderRadius.circular(radius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `padding: const EdgeInsets.fromLTRB(8, 6, 8, 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `width: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `height: showThumb ? 36 : 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `borderRadius: BorderRadius.circular(sharp ? 1 : 2),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (showThumb) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `borderRadius: BorderRadius.circular(sharp ? 3 : 5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `child: CachedNetworkImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `imageUrl: reply!.mediaUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `width: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `height: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `fit: BoxFit.cover,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `errorWidget: (_, _, _) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `const SizedBox(width: 32, height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `l10n.chatReplyTo(name),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `const SizedBox(height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `snippet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `color: onSurface.withValues(alpha: 0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/reply_quote.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/chat/room/widgets/swipe_to_reply.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class SwipeToReply extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const SwipeToReply({super.key, required this.child, required this.onReply});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final VoidCallback onReply;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `State<SwipeToReply> createState() => _SwipeToReplyState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `class _SwipeToReplyState extends State<SwipeToReply> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16** `static const _threshold = 56.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `double _dx = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `bool _triggered = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final progress = (_dx / _threshold).clamp(0.0, 1.0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `return GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `behavior: HitTestBehavior.translucent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `onHorizontalDragUpdate: (details) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final next = (_dx + details.delta.dx).clamp(0.0, _threshold * 1.35);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `setState(() => _dx = next);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (!_triggered && _dx >= _threshold) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `_triggered = true;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `HapticFeedback.selectionClick();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `onHorizontalDragEnd: (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final shouldReply = _dx >= _threshold;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `_dx = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `_triggered = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `if (shouldReply) widget.onReply();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `onHorizontalDragCancel: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `_dx = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `_triggered = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `alignment: Alignment.centerLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `if (progress > 0)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `Opacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `opacity: progress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `padding: const EdgeInsets.only(left: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `Icons.reply_rounded,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `size: 22 + (progress * 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `Transform.translate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `offset: Offset(_dx * 0.85, 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `child: widget.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/chat/room/widgets/swipe_to_reply.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/profile/app_profile_screen.dart

Shared profile (theme, language, password, logout). This file (app profile screen) is one concrete piece of that layer.

**L30** `class AppProfileScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L31** `const AppProfileScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final profile = ref.watch(currentProfileProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L36** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `appBar: AppBar(title: Text(l10n.profileTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `body: profile.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `data: (p) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `if (p == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `return EmptyState(title: l10n.profileNotSignedIn);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `return _ProfileBody(profile: p);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `class _ProfileBody extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L55** `const _ProfileBody({required this.profile});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final Profile profile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `ConsumerState<_ProfileBody> createState() => _ProfileBodyState();`
Widget that can watch providers. Required for profile/theme/locale.

**L63** `class _ProfileBodyState extends ConsumerState<_ProfileBody> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L64** `String _version = '—';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `bool _uploadingAvatar = false;`
Storage write. Path folders must satisfy bucket policies.

**L67** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `PackageInfo.fromPlatform()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `.then((info) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `if (mounted) setState(() => _version = info.version);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `.catchError((_) {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `String get _roleBasePath => switch (widget.profile.role) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `UserRole.student => '/student',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `UserRole.teacher => '/teacher',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `UserRole.admin => '/admin',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `List<_Shortcut> get _shortcuts {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `final base = _roleBasePath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `return switch (widget.profile.role) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `UserRole.student => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `_Shortcut(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `Icons.self_improvement_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `l10n.navFocus,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `'$base/focus',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `_Shortcut(Icons.grade_outlined, l10n.marksTitle, '$base/marks'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `l10n.assignmentsTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `'$base/assignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `Icons.event_available_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `l10n.attendanceTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `'$base/attendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `_Shortcut(Icons.chat_outlined, l10n.navChat, '$base/chat'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `UserRole.teacher => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `Icons.event_available_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `l10n.attendanceTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `'$base/attendance',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `_Shortcut(Icons.grade_outlined, l10n.teacherGradebook, '$base/marks'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `l10n.assignmentsTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `'$base/assignments',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `_Shortcut(Icons.chat_outlined, l10n.navChat, '$base/chat'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `UserRole.admin => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `_Shortcut(Icons.people_outline, l10n.navUsers, '$base/users'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `_Shortcut(Icons.groups_outlined, l10n.navGroups, '$base/groups'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `Icons.calendar_month_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `l10n.adminAcademicYears,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `'$base/academic-years',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `Icons.campaign_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `l10n.announcementsTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `'$base/announcements',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `_Shortcut(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `Icons.notifications_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `l10n.profileNotifications,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `'$base/notifications',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Future<void> _pickAvatar() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `final picker = ImagePicker();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final file = await picker.pickImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `source: ImageSource.gallery,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `maxWidth: 1024,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `maxHeight: 1024,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `imageQuality: 85,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `if (file == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `setState(() => _uploadingAvatar = true);`
Storage write. Path folders must satisfy bucket policies.

**L153** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `final bytes = await file.readAsBytes();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `final path = '${widget.profile.id}/avatar.jpg';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `.from('avatars')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L159** `.uploadBinary(`
Storage write. Path folders must satisfy bucket policies.

**L160** `path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `fileOptions: const FileOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `upsert: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `contentType: 'image/jpeg',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `final publicUrl = supabase.storage.from('avatars').getPublicUrl(path);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L168** `final stamped = '$publicUrl?t=${DateTime.now().millisecondsSinceEpoch}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `await ref.read(profileRepositoryProvider).updateProfile(`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L170** `widget.profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `{'avatar_url': stamped},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L174** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `).showSnackBar(SnackBar(content: Text(l10n.profilePhotoUpdated)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `if (mounted) setState(() => _uploadingAvatar = false);`
Storage write. Path folders must satisfy bucket policies.

**L188** `Future<void> _editName() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `final controller = TextEditingController(text: widget.profile.fullName);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `final saved = await showDialog<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `title: Text(l10n.profileEditName),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `content: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `controller: controller,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `autofocus: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `textCapitalization: TextCapitalization.words,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `decoration: InputDecoration(labelText: l10n.profileFullName),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `onPressed: () => Navigator.pop(ctx, controller.text.trim()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `child: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `if (saved == null || saved.isEmpty || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `if (saved == widget.profile.fullName) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `await ref.read(profileRepositoryProvider).updateProfile(`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L218** `widget.profile.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `{'full_name': saved},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L222** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `).showSnackBar(SnackBar(content: Text(l10n.profileNameUpdated)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `Future<void> _changePassword() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `final current = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `final next = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `final confirm = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `var saving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `builder: (ctx, setLocal) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `title: Text(l10n.profileChangePassword),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `controller: current,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `obscureText: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `labelText: l10n.profileCurrentPassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `controller: next,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `obscureText: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `labelText: l10n.profileNewPassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `TextField(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `controller: confirm,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `obscureText: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `labelText: l10n.profileConfirmPassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `l10n.profilePasswordHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `color: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `).colorScheme.onSurface.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `onPressed: saving ? null : () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `onPressed: saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `final n = next.text;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `if (n.length < 8) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `SnackBar(content: Text(l10n.profilePasswordTooShort)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `if (n != confirm.text) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `SnackBar(content: Text(l10n.profilePasswordMismatch)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `setLocal(() => saving = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `.read(authRepositoryProvider)`
Riverpod binding. Name it after the value it exposes.

**L312** `.changePassword(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `username: widget.profile.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `currentPassword: current.text,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `newPassword: n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `SnackBar(content: Text(l10n.profilePasswordChanged)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `setLocal(() => saving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `SnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `content: Text(l10n.commonCouldNotSave('$e')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `child: saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `? const SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `child: CircularProgressIndicator(strokeWidth: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `Future<void> _clearCachedData() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `final first = await showDialog<bool>(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `title: Text(l10n.profileClearCacheWarn1Title),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `content: Text(l10n.profileClearCacheWarn1Body),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `style: FilledButton.styleFrom(backgroundColor: AppColors.errorRed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `child: Text(l10n.profileClearCacheConfirm),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `if (first != true || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `final second = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `title: Text(l10n.profileClearCacheWarn2Title),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `content: Text(l10n.profileClearCacheWarn2Body),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `style: FilledButton.styleFrom(backgroundColor: AppColors.errorRed),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `child: Text(l10n.profileClearCacheConfirm),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `if (second != true || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `final cache = ref.read(cacheServiceProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L391** `await clearChatAndMediaCaches(cache);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `).showSnackBar(SnackBar(content: Text(l10n.profileClearCacheDone)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `Future<void> _shareProgress() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `final groupId = ref.read(activeGroupIdProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L407** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L408** `if (groupId == null || student == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `).showSnackBar(SnackBar(content: Text(l10n.progressReportNeedGroup)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L416** `final groupTitle =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `groups.where((g) => g.id == groupId).map((g) => g.title).firstOrNull ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `'group';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `await shareProgressReport(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `studentId: student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `studentName: widget.profile.fullName.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `? widget.profile.fullName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `: widget.profile.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `term: ref.read(selectedMarkTermProvider),`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L428** `l10n: l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `).showSnackBar(SnackBar(content: Text(l10n.progressReportFailed('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L442** `final teacher = ref.watch(teacherProfileProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L443** `final student = ref.watch(studentProfileProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L444** `final average = widget.profile.role == UserRole.student`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `? ref.watch(myMarkAverageProvider).valueOrNull`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L446** `: null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L448** `AttendanceStats? attStats;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `if (widget.profile.role == UserRole.student &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `student != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `groupId != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `final query = AttendanceQuery(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `studentId: student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `from: DateTime(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `now.year,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `now.month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `now.day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `).subtract(const Duration(days: 120)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `to: DateTime(now.year, now.month, now.day),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `attStats = ref.watch(attendanceRangeStatsProvider(query)).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L466** `final roleLabel = switch (widget.profile.role) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `UserRole.student => l10n.commonStudent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `UserRole.teacher => l10n.commonTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `UserRole.admin => l10n.commonAdmin,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `return ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `shellContentBottomPadding(context, extra: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `ProfileHeroCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `profile: widget.profile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `roleLabel: roleLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `sphereLabel: teacher == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `: '${l10n.profileSubject}: ${teacher.sphere.l10nLabel(l10n)}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `uploading: _uploadingAvatar,`
Storage write. Path folders must satisfy bucket policies.

**L487** `onEditPhoto: _pickAvatar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `onEditName: _editName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `if (widget.profile.role != UserRole.admin) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `title: l10n.profileYourGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `subtitle: groups.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `? l10n.profileNoGroups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `: l10n.profileGroupsCount(groups.length),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `decorAsset: PaperAssets.bookshelfDouble,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `child: groups.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `? Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `l10n.profileNoGroups,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `style: TextStyle(color: AppColors.inkAlpha(context, 0.5)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `for (var i = 0; i < groups.length; i++) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `if (i > 0) const Divider(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `Icons.groups_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `groups[i].title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `if (groupId == groups[i].id)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `Icons.check_circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `if (widget.profile.role == UserRole.student) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `child: _StatChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `label: l10n.profileAverageLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `value: average == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `? '—'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `: '${average.toStringAsFixed(average % 1 == 0 ? 0 : 1)}%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `child: _StatChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `label: l10n.profileAttendanceLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `value: attStats == null || attStats.total == 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `? '—'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `: '${attStats.presentPercent.toStringAsFixed(0)}%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `onTap: _shareProgress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `Icons.ios_share_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `l10n.profileShareProgress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `style: const TextStyle(fontWeight: FontWeight.w600),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `Icons.chevron_right,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `color: AppColors.inkAlpha(context, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `title: l10n.profileShortcuts,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `decorAsset: PaperAssets.puzzlePieces,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L591** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L592** `GridView.count(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `crossAxisCount: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `physics: const NeverScrollableScrollPhysics(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `mainAxisSpacing: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `crossAxisSpacing: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `childAspectRatio: 2.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `for (final s in _shortcuts)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `onTap: () => context.push(s.path),`
Push a route the user can pop (assignment detail, chat room).

**L603** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `horizontal: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `vertical: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `icon: s.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `size: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L615** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `s.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L622** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L628** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `title: l10n.profileSettings,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `decorAsset: PaperAssets.settingsGear,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `padding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `padding: const EdgeInsets.fromLTRB(16, 16, 16, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `child: LanguageSettingsTile(embedded: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `const Divider(height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `padding: const EdgeInsets.fromLTRB(16, 12, 16, 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `child: ProfileThemeSelector(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L648** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `const Divider(height: 1),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `ProfileSettingsRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `icon: Icons.notifications_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `label: l10n.profileNotifications,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `onTap: () => context.push('$_roleBasePath/notifications'),`
Push a route the user can pop (assignment detail, chat room).

**L654** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L655** `const Divider(height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `ProfileSettingsRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `icon: Icons.badge_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `label: l10n.profileEditName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `onTap: _editName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `const Divider(height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `ProfileSettingsRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L663** `icon: Icons.lock_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L664** `label: l10n.profileChangePassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L665** `onTap: _changePassword,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `const Divider(height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `ProfileSettingsRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `icon: Icons.cleaning_services_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `label: l10n.profileClearCache,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L671** `onTap: _clearCachedData,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L674** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L675** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L677** `Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L678** `child: TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L679** `onPressed: () => ref.read(authRepositoryProvider).signOut(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L680** `style: TextButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L681** `foregroundColor: AppColors.errorRed.withValues(alpha: 0.9),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L682** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L683** `icon: const Icon(Icons.logout, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L684** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L685** `widget.profile.role == UserRole.admin`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L686** `? l10n.profileSignOutAdmin`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L687** `: l10n.commonSignOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L688** `style: const TextStyle(fontWeight: FontWeight.w600),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L689** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L690** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L691** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L692** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L693** `Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L694** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `l10n.profileAppVersion(_version),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L696** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L697** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L699** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L700** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L701** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `class _Shortcut {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L708** `const _Shortcut(this.icon, this.label, this.path);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L709** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L711** `final String path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L714** `class _StatChip extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L715** `const _StatChip({required this.label, required this.value});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L716** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L717** `final String value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L719** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L720** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L721** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L722** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L723** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L724** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L725** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L726** `Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L727** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L728** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L729** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L730** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L731** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L732** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L733** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L734** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L735** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L736** `value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L737** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L738** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L739** `fontSize: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L740** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L741** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L742** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L744** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/profile/app_profile_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/profile/widgets/profile_hero_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L7** `class ProfileHeroCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L8** `const ProfileHeroCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.profile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.roleLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.onEditPhoto,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.onEditName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `this.sphereLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `this.uploading = false,`
Storage write. Path folders must satisfy bucket policies.

**L18** `final Profile profile;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final String roleLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String? sphereLabel;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final VoidCallback onEditPhoto;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final VoidCallback onEditName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final bool uploading;`
Storage write. Path folders must satisfy bucket policies.

**L25** `String get _initials {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final parts = profile.fullName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `.trim()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `.split(RegExp(r'\s+'))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `.where((p) => p.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (parts.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final u = profile.username;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `return u.isEmpty ? '?' : u.substring(0, 1).toUpperCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `if (parts.length == 1) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `return parts.first.substring(0, 1).toUpperCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `return (parts[0].substring(0, 1) + parts[1].substring(0, 1)).toUpperCase();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `final theme = Theme.of(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final isDark = theme.brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `padding: const EdgeInsets.fromLTRB(20, 24, 20, 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `borderRadius: BorderRadius.circular(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `begin: Alignment.topLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `end: Alignment.bottomRight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `colors: isDark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `? const [Color(0xFF2A2420), Color(0xFF1C1816)]`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `: const [Color(0xFFE8D9C4), Color(0xFFF5EEE4)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `color: theme.colorScheme.onSurface.withValues(alpha: 0.06),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `alignment: Alignment.bottomRight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `PaperScallopedAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `avatarSize: 88,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `child: CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `radius: 44,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `backgroundImage:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `profile.avatarUrl != null && profile.avatarUrl!.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `? CachedNetworkImageProvider(profile.avatarUrl!)`
Riverpod binding. Name it after the value it exposes.

**L74** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `child: uploading`
Storage write. Path folders must satisfy bucket policies.

**L76** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `width: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `height: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `? Colors.black`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `: (profile.avatarUrl == null ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `profile.avatarUrl!.isEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `? Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `_initials,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `? Colors.black`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `fontSize: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `color: theme.colorScheme.surface,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `shape: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `customBorder: const CircleBorder(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `onTap: uploading ? null : onEditPhoto,`
Storage write. Path folders must satisfy bucket policies.

**L109** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `padding: EdgeInsets.all(6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Icons.camera_alt_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `const SizedBox(height: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `onTap: onEditName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `borderRadius: BorderRadius.circular(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `Flexible(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `profile.fullName.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `? profile.fullName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `: profile.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `style: theme.textTheme.titleLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `Icons.edit_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `profile.username.startsWith('@')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `? profile.username`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `: '@${profile.username}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `style: TextStyle(color: AppColors.inkAlpha(context, 0.5)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `spacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `runSpacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `alignment: WrapAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `_ProfileChip(label: roleLabel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `if (sphereLabel != null) _ProfileChip(label: sphereLabel!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `class _ProfileChip extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L175** `const _ProfileChip({required this.label});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `final accent = AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `color: accent.withValues(alpha: isDark ? 0.22 : 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `borderRadius: BorderRadius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `color: accent.withValues(alpha: isDark ? 0.55 : 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/profile/widgets/profile_hero_card.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/profile/widgets/profile_settings_row.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class ProfileSettingsRow extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const ProfileSettingsRow({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `return ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `leading: PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `icon: icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `title: Text(label, style: const TextStyle(fontWeight: FontWeight.w600)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `trailing: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Icons.chevron_right,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `color: AppColors.inkAlpha(context, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/profile/widgets/profile_settings_row.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/profile/widgets/profile_theme_selector.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L12** `class ProfileThemeSelector extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `const ProfileThemeSelector({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final mode = ref.watch(themeModeProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L19** `final style = ref.watch(appStyleProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L20** `final selected = switch (mode) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `ThemeMode.light => AppThemePreference.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `ThemeMode.dark => AppThemePreference.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `ThemeMode.system => AppThemePreference.system,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `l10n.profileAppearance,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `SegmentedButton<AppThemePreference>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `segments: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `ButtonSegment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `value: AppThemePreference.light,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `label: Text(l10n.profileThemeLight),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `icon: const Icon(Icons.light_mode_outlined, size: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `ButtonSegment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `value: AppThemePreference.dark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `label: Text(l10n.profileThemeDark),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `icon: const Icon(Icons.dark_mode_outlined, size: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `ButtonSegment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `value: AppThemePreference.system,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `label: Text(l10n.profileThemeSystem),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `icon: const Icon(Icons.brightness_auto_outlined, size: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `selected: {selected},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `onSelectionChanged: (set) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final next = set.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `ref.read(themeModeProvider.notifier).setPreference(next);`

One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L60** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `l10n.profileStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final tileWidth = (constraints.maxWidth - 10) / 2;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `return Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `spacing: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `runSpacing: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `width: tileWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `child: _StyleOptionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `label: l10n.profileStyleModern,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `icon: Icons.business_center_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `previewColors: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `AppColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `AppColors.warmBeige,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `isSelected: style == AppStyle.modern,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `onTap: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `.read(appStyleProvider.notifier)`
Riverpod binding. Name it after the value it exposes.

**L90** `.setStyle(AppStyle.modern),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `width: tileWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `child: _StyleOptionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `label: l10n.profileStyleNature,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `icon: Icons.eco_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `previewColors: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `NaturePaperColors.cream,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `NaturePaperColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `NaturePaperColors.forest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `isSelected: style == AppStyle.naturePaper,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `onTap: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `.read(appStyleProvider.notifier)`
Riverpod binding. Name it after the value it exposes.

**L106** `.setStyle(AppStyle.naturePaper),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `width: tileWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `child: _StyleOptionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `label: l10n.profileStyleSoftNeu,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `icon: Icons.layers_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `previewColors: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `SoftNeumorphismColors.morningBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `SoftNeumorphismColors.sage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `SoftNeumorphismColors.amber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `isSelected: style == AppStyle.softNeumorphism,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `onTap: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `.read(appStyleProvider.notifier)`
Riverpod binding. Name it after the value it exposes.

**L122** `.setStyle(AppStyle.softNeumorphism),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `width: tileWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `child: _StyleOptionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `label: l10n.profileStyleMonochrome,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `icon: Icons.contrast_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `previewColors: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `MonochromeWorkspaceColors.lightBg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `MonochromeWorkspaceColors.lightBorder,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `MonochromeWorkspaceColors.lightInk,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `isSelected: style == AppStyle.monochromeWorkspace,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `onTap: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `.read(appStyleProvider.notifier)`
Riverpod binding. Name it after the value it exposes.

**L138** `.setStyle(AppStyle.monochromeWorkspace),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `class _StyleOptionCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L151** `const _StyleOptionCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `required this.previewColors,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `required this.isSelected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `final List<Color> previewColors;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `final bool isSelected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `final accent = AppColors.accent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `final isNeu = Neumorphic.isActive(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `final radius = BorderRadius.circular(isNeu ? 18 : 16);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `return InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `duration: const Duration(milliseconds: 200),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `decoration: isNeu`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `? Neumorphic.decoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `elevation: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `? NeuElevation.pressed`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `: NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `color: isSelected ? accent.withValues(alpha: 0.12) : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `color: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `? accent.withValues(alpha: 0.10)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `: AppColors.softWash(context, lightAlpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `borderRadius: radius,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `color: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `? accent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `: AppColors.inkAlpha(context, 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `width: isSelected ? 1.6 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `for (final c in previewColors)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `padding: const EdgeInsets.only(right: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `width: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `height: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `color: c,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `color: AppColors.inkAlpha(context, 0.12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `if (isSelected)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `Icon(Icons.check_circle, size: 16, color: accent)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `Icon(icon, size: 16, color: AppColors.inkAlpha(context, 0.4)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `color: isSelected ? accent : AppColors.inkAlpha(context, 0.75),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/profile/widgets/profile_theme_selector.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/attendance/student_attendance_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L16** `class StudentAttendanceScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17** `const StudentAttendanceScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `ConsumerState<StudentAttendanceScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L21** `_StudentAttendanceScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `class _StudentAttendanceScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `extends ConsumerState<StudentAttendanceScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L26** `late DateTime _month;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `_month = DateTime(now.year, now.month);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L39** `final student = ref.watch(studentProfileProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L41** `if (groupId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `appBar: AppBar(title: Text(l10n.attendanceTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `body: EmptyState(title: l10n.attendanceNoGroup),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final studentId = student.valueOrNull?.id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `if (studentId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `appBar: AppBar(title: Text(l10n.attendanceTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `body: student.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `data: (_) => EmptyState(title: l10n.attendanceNoGroup),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final from = DateTime(_month.year, _month.month, 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final to = DateTime(_month.year, _month.month + 1, 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final query = AttendanceQuery(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `studentId: studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `from: from,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `to: to,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `final records = ref.watch(attendanceRangeProvider(query));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L69** `final stats = ref.watch(attendanceRangeStatsProvider(query));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L71** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `appBar: AppBar(title: Text(l10n.attendanceTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `body: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `ref.invalidate(attendanceRangeProvider(query));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L76** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `shellContentBottomPadding(context, extra: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `AttendanceMonthHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `month: _month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `onPrev: () => setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `_month = DateTime(_month.year, _month.month - 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `onNext: () => setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `_month = DateTime(_month.year, _month.month + 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `stats.when(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `data: (s) => AttendanceStatsBar(stats: s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `loading: () => const SizedBox(height: 56),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `child: records.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `loading: () => const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `padding: EdgeInsets.all(32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final byDate = <DateTime, AttendanceStatus>{`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `for (final r in list)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `DateTime(r.date.year, r.date.month, r.date.day): r.status,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `l10n.attendanceHistory,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `style: Theme.of(context).textTheme.titleMedium`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `PaperAssets.calendarWithClock,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `size: 34,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `angle: 0.08,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `AttendanceCalendarGrid(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `month: _month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `statusByDate: byDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `const AttendanceLegendRow(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `if (list.isEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `l10n.attendanceCalendarEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/attendance/student_attendance_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/focus_format.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L1** `String formatFocusDuration(Duration d) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L2** `final total = d.inSeconds.clamp(0, 99 * 3600);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L3** `final h = total ~/ 3600;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L4** `final m = (total % 3600) ~/ 60;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L5** `final s = total % 60;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `if (h > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `return '$h:${m.toString().padLeft(2, '0')}:${s.toString().padLeft(2, '0')}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `return '${m.toString().padLeft(2, '0')}:${s.toString().padLeft(2, '0')}';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/focus/focus_format.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/student_focus_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L15** `class StudentFocusScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16** `const StudentFocusScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `ConsumerState<StudentFocusScreen> createState() => _StudentFocusScreenState();`
Widget that can watch providers. Required for profile/theme/locale.

**L22** `class _StudentFocusScreenState extends ConsumerState<StudentFocusScreen>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L23** `with WidgetsBindingObserver {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Timer? _tick;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final _noteController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `WidgetsBinding.instance.addObserver(this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `_tick = Timer.periodic(const Duration(seconds: 1), (_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final session = ref.read(focusSessionProvider).session;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L34** `if (session == null || session.isPaused) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `setState(() {});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `_finishIfTimedOut(session);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `WidgetsBinding.instance.removeObserver(this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `_tick?.cancel();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `_noteController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `void didChangeAppLifecycleState(AppLifecycleState state) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `if (state == AppLifecycleState.resumed) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final session = ref.read(focusSessionProvider).session;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L52** `if (session != null) _finishIfTimedOut(session);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `void _finishIfTimedOut(FocusSession session) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `if (!session.isCompleteAt()) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `ref.read(focusSessionProvider.notifier).endSession(timedOut: true);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L61** `Future<void> _pickModeAndStart(Assignment a) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final mode = await showModalBottomSheet<FocusTimerMode>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `showDragHandle: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `padding: const EdgeInsets.fromLTRB(20, 8, 20, 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `crossAxisAlignment: CrossAxisAlignment.stretch,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `l10n.focusChooseMode,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `style: Theme.of(ctx).textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `const SizedBox(height: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `color: AppColors.inkAlpha(ctx, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `borderRadius: BorderRadius.circular(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `leading: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `Icons.timer_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `color: AppColors.accent(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `title: Text(l10n.focusModePomodoro),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `subtitle: Text(l10n.focusModePomodoroHint),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `onTap: () => Navigator.pop(ctx, FocusTimerMode.pomodoro),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `borderRadius: BorderRadius.circular(14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `leading: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `Icons.all_inclusive_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `color: AppColors.accent(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `title: Text(l10n.focusModeFree),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `subtitle: Text(l10n.focusModeFreeHint),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `onTap: () => Navigator.pop(ctx, FocusTimerMode.free),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `if (mode == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `await ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `.read(focusSessionProvider.notifier)`
Riverpod binding. Name it after the value it exposes.

**L122** `.start(assignmentId: a.id, assignmentTitle: a.title, mode: mode);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `final focus = ref.watch(focusSessionProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L128** `if (focus.session != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `return ActiveSessionView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `session: focus.session!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `formatDuration: formatFocusDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `onPause: () => ref.read(focusSessionProvider.notifier).pause(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L133** `onResume: () => ref.read(focusSessionProvider.notifier).resume(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L134** `onEnd: () => ref.read(focusSessionProvider.notifier).endSession(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L135** `onOpenTask: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `context.push('/student/assignments/${focus.session!.assignmentId}'),`
Push a route the user can pop (assignment detail, chat room).

**L139** `if (focus.summary != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `final summary = focus.summary!;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `if (_noteController.text != (summary.note ?? '') &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `summary.note != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `_noteController.text = summary.note!;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `return FocusSummaryView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `summary: summary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `formatDuration: formatFocusDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `noteController: _noteController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `onNoteChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `ref.read(focusSessionProvider.notifier).setSummaryNote(v),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L151** `onOpenTask: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `ref.read(focusSessionProvider.notifier).dismissSummary();`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L153** `context.push('/student/assignments/${summary.assignmentId}');`
Push a route the user can pop (assignment detail, chat room).

**L154** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `onDone: () => ref.read(focusSessionProvider.notifier).dismissSummary(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L158** `return FocusHomeView(onStart: _pickModeAndStart);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/student/focus/student_focus_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/widgets/active_session_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L8** `class ActiveSessionView extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L9** `const ActiveSessionView({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `required this.session,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `required this.formatDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.onPause,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `required this.onResume,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.onEnd,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.onOpenTask,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final FocusSession session;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final String Function(Duration) formatDuration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final VoidCallback onPause;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final VoidCallback onResume;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final VoidCallback onEnd;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final VoidCallback onOpenTask;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final nature = isNaturePaper(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final elapsed = session.elapsedAt();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final remaining = session.remainingAt();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final display = remaining ?? elapsed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final progress = session.mode == FocusTimerMode.pomodoro`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `? (1 -`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `(remaining?.inMilliseconds ?? 0) /`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `kFocusPomodoroDuration.inMilliseconds)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `.clamp(0.0, 1.0)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `: null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `backgroundColor: nature`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `? Colors.transparent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `: Theme.of(context).scaffoldBackgroundColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `body: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `fit: StackFit.expand,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `if (nature) const FocusAmbientDecor(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `padding: const EdgeInsets.fromLTRB(24, 16, 24, 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `horizontal: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `vertical: 5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `color: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `).withValues(alpha: 0.14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `borderRadius: BorderRadius.circular(999),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `l10n.focusDoNotDisturb,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `onPressed: onEnd,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `child: Text(l10n.focusEndSession),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `const Spacer(flex: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `session.assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `style: Theme.of(context).textTheme.titleMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `color: AppColors.inkAlpha(context, 0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `session.mode == FocusTimerMode.pomodoro`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `? l10n.focusModePomodoro`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `: l10n.focusModeFree,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `const SizedBox(height: 28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `width: 220,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `height: 220,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `if (progress != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `SizedBox.expand(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `value: progress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `strokeWidth: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `backgroundColor: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `0.08,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `color: AppColors.inkAlpha(context, 0.08),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `width: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `formatDuration(display),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `style: Theme.of(context).textTheme.displayMedium`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `fontWeight: FontWeight.w300,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `letterSpacing: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `fontFeatures: const [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `FontFeature.tabularFigures(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `if (session.isPaused)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `l10n.focusPaused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `style: TextStyle(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `color: AppColors.warning(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `const Spacer(flex: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `child: OutlinedButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `onPressed: session.isPaused ? onResume : onPause,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `session.isPaused`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `? Icons.play_arrow_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `: Icons.pause_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `session.isPaused`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `? l10n.focusResume`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `: l10n.focusPause,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `child: FilledButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `onPressed: onOpenTask,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `icon: const Icon(Icons.open_in_new_rounded, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `label: Text(l10n.focusOpenAssignment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/focus/widgets/active_session_view.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/widgets/focus_ambient_decor.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class FocusAmbientDecor extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const FocusAmbientDecor({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `if (!isNaturePaper(context)) return const SizedBox.shrink();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `return IgnorePointer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `top: 80,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `left: -20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `isDark ? PaperAssets.grayCloud : PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `size: 90,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `opacity: isDark ? 0.25 : 0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `top: 140,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `right: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `PaperAssets.flatCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `size: 64,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `opacity: isDark ? 0.2 : 0.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `bottom: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `left: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `PaperAssets.singleTree,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `height: 100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `opacity: isDark ? 0.35 : 0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `bottom: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `right: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `PaperAssets.twoTrees,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `height: 120,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `opacity: isDark ? 0.4 : 0.55,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (isDark) const Positioned.fill(child: SoftFireflies()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/student/focus/widgets/focus_ambient_decor.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/widgets/focus_home_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L18** `class FocusHomeView extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19** `const FocusHomeView({super.key, required this.onStart});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final Future<void> Function(Assignment a) onStart;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L27** `final assignments = ref.watch(assignmentsProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L28** `final nature = isNaturePaper(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `body: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (groupId != null) ref.invalidate(assignmentsProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L34** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `MediaQuery.paddingOf(context).top + 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `l10n.focusTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `style: Theme.of(context).textTheme.headlineSmall`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `letterSpacing: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `l10n.focusSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `fontWeight: FontWeight.w500,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (nature)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `const PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `PaperAssets.whiteCloud,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `size: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `opacity: 0.7,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `title: l10n.focusTodaysFocus,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `subtitle: l10n.focusTodaysFocusSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `decorAsset: PaperAssets.yellowPencil,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `assignments.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `loading: () => const Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `padding: EdgeInsets.all(32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `child: CircularProgressIndicator(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final focusItems = assignmentsForFocusSession(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `page.items,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `).take(3).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `if (focusItems.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `title: l10n.focusEmptyTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `subtitle: l10n.focusEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `icon: Icons.self_improvement_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final primary = focusItems.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `padding: const EdgeInsets.all(18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `icon: Icons.task_alt_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `size: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `primary.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `fontSize: 17,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `DueDateLabel(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `assignment: primary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `compact: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `PrimaryButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `label: l10n.focusContinueHomework,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `onPressed: () => onStart(primary),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `onPressed: () => context.push(`
Push a route the user can pop (assignment detail, chat room).

**L146** `'/student/assignments/${primary.id}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `child: Text(l10n.focusOpenAssignment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `if (focusItems.length > 1) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `title: l10n.focusAlsoDue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `decorAsset: PaperAssets.stickyNotesPair,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `for (final a in focusItems.skip(1))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `onTap: () => onStart(a),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `vertical: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `icon: assignmentIsOverdue(a)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `? Icons.warning_amber_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `: Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `color: assignmentIsOverdue(a)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `? AppColors.danger(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `size: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `DueDateLabel(assignment: a, compact: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `Icons.play_circle_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `color: AppColors.inkAlpha(context, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `Icons.chat_bubble_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `l10n.focusChatMuteHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `height: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/focus/widgets/focus_home_view.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/widgets/focus_summary_view.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L10** `class FocusSummaryView extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L11** `const FocusSummaryView({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.summary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `required this.formatDuration,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.noteController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.onNoteChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `required this.onOpenTask,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `required this.onDone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final FocusSummary summary;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String Function(Duration) formatDuration;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final TextEditingController noteController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final ValueChanged<String> onNoteChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final VoidCallback onOpenTask;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final VoidCallback onDone;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final minutes = summary.focused.inMinutes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final seconds = summary.focused.inSeconds % 60;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `body: SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `if (isNaturePaper(context))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `const Align(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `alignment: Alignment.centerRight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `child: PaperDecor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `PaperAssets.beigeLeaf,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `size: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `angle: 0.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `opacity: 0.75,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `summary.timedOut ? l10n.focusTimeUp : l10n.focusSessionComplete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `).textTheme.headlineSmall?.copyWith(fontWeight: FontWeight.w800),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `summary.timedOut`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `? l10n.focusTimeUpSubtitle`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `: l10n.focusSessionCompleteSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `style: TextStyle(color: AppColors.inkAlpha(context, 0.55)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `PaperTaped(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `child: PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `padding: const EdgeInsets.all(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `summary.assignmentTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `fontSize: 17,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `l10n.focusMinutesFocused(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `minutes > 0 ? '$minutes min' : '${seconds}s',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `fontSize: 28,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `fontWeight: FontWeight.w300,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `formatDuration(summary.focused),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `fontFeatures: const [FontFeature.tabularFigures()],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `l10n.focusOptionalNote,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `controller: noteController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `onChanged: onNoteChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `maxLines: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `decoration: InputDecoration(hintText: l10n.focusOptionalNoteHint),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `PrimaryButton(label: l10n.focusSaveAndOpen, onPressed: onOpenTask),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `TextButton(onPressed: onDone, child: Text(l10n.focusDone)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/focus/widgets/focus_summary_view.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/focus/widgets/soft_fireflies.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class SoftFireflies extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const SoftFireflies({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `State<SoftFireflies> createState() => _SoftFirefliesState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `class _SoftFirefliesState extends State<SoftFireflies>`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L14** `late final AnimationController _c = AnimationController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `vsync: this,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `duration: const Duration(seconds: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `)..repeat();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `_c.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `return AnimatedBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `animation: _c,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `builder: (_, _) => CustomPaint(painter: SoftFirefliesPainter(_c.value)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `class SoftFirefliesPainter extends CustomPainter {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L35** `SoftFirefliesPainter(this.t);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `final double t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `static const _pts = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `Offset(0.2, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `Offset(0.55, 0.28),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `Offset(0.78, 0.42),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `Offset(0.35, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `void paint(Canvas canvas, Size size) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `final glow = Paint()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `..maskFilter = const MaskFilter.blur(BlurStyle.normal, 5);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final core = Paint();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `for (var i = 0; i < _pts.length; i++) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final phase = i * 1.4;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final a = t * 2 * math.pi;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final pulse = 0.3 + 0.7 * (0.5 + 0.5 * math.sin(a * 2 + phase));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `final c = Offset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `_pts[i].dx * size.width + math.sin(a + phase) * 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `_pts[i].dy * size.height + math.cos(a + phase) * 5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `glow.color = const Color(0xFFE8C57F).withValues(alpha: 0.35 * pulse);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `core.color = const Color(0xFFF6E3B4).withValues(alpha: 0.85 * pulse);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `canvas.drawCircle(c, 3.2, glow);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `canvas.drawCircle(c, 1.1, core);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `@override`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `bool shouldRepaint(covariant SoftFirefliesPainter old) => old.t != t;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/focus/widgets/soft_fireflies.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/home/student_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L27** `class StudentDashboardScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L28** `const StudentDashboardScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `ConsumerState<StudentDashboardScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L32** `_StudentDashboardScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `class _StudentDashboardScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L36** `extends ConsumerState<StudentDashboardScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L37** `late final int _dailyGreetingIndex;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `_dailyGreetingIndex = Random().nextInt(4);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `String _dailyMotivation(AppLocalizations l10n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final lines = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `l10n.dailyMotivation1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `l10n.dailyMotivation2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `l10n.dailyMotivation3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `l10n.dailyMotivation4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return lines[_dailyGreetingIndex];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final profile = ref.watch(currentProfileProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L59** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L60** `final assignments = ref.watch(assignmentsProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L61** `final notifications = ref.watch(notificationsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L62** `final announcements = ref.watch(announcementsProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L63** `final marks = ref.watch(myMarksProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L64** `final average = ref.watch(myMarkAverageProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L66** `return RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `ref.invalidate(myGroupsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L69** `ref.invalidate(myMarksProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L70** `ref.invalidate(myMarkAverageProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L71** `if (groupId != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `ref.invalidate(assignmentsProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L73** `ref.invalidate(announcementsProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L75** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `left: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `right: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `top: MediaQuery.of(context).padding.top + 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `bottom: shellContentBottomPadding(context),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `profile.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `data: (p) => Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `crossAxisAlignment: CrossAxisAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `radius: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `(p?.fullName.isNotEmpty == true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `? p!.fullName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `: p?.username ?? 'U')[0]`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `color: Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `? Colors.black`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `fontSize: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `'${l10n.greetingWithName(timeOfDayGreeting(l10n), p?.fullName.split(' ').first ?? p?.username ?? '')} ■',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `style: Theme.of(context).textTheme.headlineSmall`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `letterSpacing: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `_dailyMotivation(l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `PaperDecor(PaperAssets.whiteCloud, size: 44, opacity: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `loading: () => Skeletonizer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `enabled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `child: ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `leading: const CircleAvatar(radius: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `title: Text(l10n.greetingLoadingTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `subtitle: Text(l10n.greetingLoadingSubtitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `MarkAverageCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `average: average,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `markCount: marks.valueOrNull?.items.length ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `onTap: () => context.push('/student/marks'),`
Push a route the user can pop (assignment detail, chat room).

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `title: l10n.continueLearning,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `subtitle: l10n.continueLearningSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `decorAsset: PaperAssets.yellowPencil,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `assignments.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `final list = sortAssignmentsByDue(page.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `if (list.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `padding: const EdgeInsets.all(32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `if (isNaturePaper(context))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `const PaperEmptyIllustration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `asset: PaperAssets.envelopeSuccess,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `Icons.check_circle_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `size: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `color: AppColors.accent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `).withValues(alpha: 0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `l10n.everythingCompleted,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `fontSize: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `l10n.caughtUpToday,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `final a = list.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `final isMath = a.title.toLowerCase().contains('math');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `final accent = assignmentIsOverdue(a)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `? AppColors.danger(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `: isMath`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `? AppColors.accent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `: AppColors.goldAccent(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `final heroRadius =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `AppStyleTokens.maybeOf(context)?.cardRadius ?? 16.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `onTap: () => context.push('/student/assignments/${a.id}'),`
Push a route the user can pop (assignment detail, chat room).

**L212** `padding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `elevation: NeuElevation.floating,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `borderRadius: BorderRadius.circular(heroRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `gradient: LinearGradient(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `colors: [accent, accent.withValues(alpha: 0.82)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `begin: Alignment.topLeft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `end: Alignment.bottomRight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `child: Stack(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `Positioned(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `right: -20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `top: -20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `child: isNaturePaper(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `? Opacity(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `opacity: 0.16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `child: Transform.rotate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `angle: 0.18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `child: Image.asset(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `isMath`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `? PaperAssets.linedPaperPencil`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `: PaperAssets.openBook,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `width: 130,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `filterQuality: FilterQuality.medium,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `isMath ? Icons.calculate : Icons.menu_book,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `size: 140,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `color: Colors.white.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `child: Column(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `spacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `runSpacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `crossAxisAlignment: WrapCrossAlignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `horizontal: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `vertical: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `color: Colors.white.withValues(alpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `borderRadius: BorderRadius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `a.type == AssignmentType.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `? l10n.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `: l10n.staticTask,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `letterSpacing: 0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `DueDateLabel(assignment: a, onDark: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `fontSize: 22,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `letterSpacing: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `if (a.description.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `a.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `color: Colors.white.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `Consumer(`
Widget that can watch providers. Required for profile/theme/locale.

**L306** `builder: (context, ref, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `final progressAsync = ref.watch(`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L308** `assignmentProgressProvider(a.id),`
Riverpod binding. Name it after the value it exposes.

**L310** `final progress =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `progressAsync.valueOrNull ?? 0.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `crossAxisAlignment:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `mainAxisAlignment:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `MainAxisAlignment.spaceBetween,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `l10n.commonProgress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `color: Colors.white`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `'${(progress * 100).toInt()}%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `borderRadius: BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `child: TweenAnimationBuilder<double>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `tween: Tween<double>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `begin: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `end: progress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `duration: const Duration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `milliseconds: 1000,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `builder: (context, value, _) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `return LinearProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `value: value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `backgroundColor:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `Colors.black26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `valueColor:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `const AlwaysStoppedAnimation(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `minHeight: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `const SizedBox(width: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `padding: const EdgeInsets.all(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `decoration: const BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `Icons.play_arrow_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `color: accent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `size: 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `},`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `loading: () => const Skeletonizer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `enabled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `child: PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `child: SizedBox(height: 200, width: double.infinity),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `onTap: () => context.go('/student/focus'),`
Replace the navigation stack location. Login uses /boot so redirect can settle.

**L411** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `icon: Icons.self_improvement_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `size: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `l10n.focusTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `l10n.focusSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `title: l10n.homeworkDueThisWeek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `subtitle: l10n.homeworkDueThisWeekSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `decorAsset: PaperAssets.stickyNotesPair,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `assignments.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `final dueFocus = assignmentsDueThisWeekFocus(page.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `if (dueFocus.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `horizontal: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `vertical: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `icon: Icons.event_available_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `size: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `color: AppColors.success(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `).withValues(alpha: 0.85),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `l10n.homeworkNoDueThisWeek,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `l10n.homeworkNoDueThisWeekSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `children: dueFocus.take(5).map((a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `final isHomeworkPack = a.type == AssignmentType.homeworkPack;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L504** `final overdue = assignmentIsOverdue(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `onTap: () => context.push('/student/assignments/${a.id}'),`
Push a route the user can pop (assignment detail, chat room).

**L507** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L514** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L515** `final tone = AppColors.resolveAccent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L516** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `overdue`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `? AppColors.danger(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `: isHomeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `? AppColors.goldAccent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `final iconData = overdue`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `? Icons.warning_amber_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `: isHomeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `? Icons.inventory_2_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `: Icons.task_alt_rounded;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `if (Neumorphic.isActive(context)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `return NeuIconWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `icon: iconData,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `color: tone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `AppStyleTokens.maybeOf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `padding: const EdgeInsets.all(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `color: tone.withValues(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `alpha: isMono ? 0.08 : 0.1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `borderRadius: BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `AppStyleTokens.maybeOf(context)?.cardRadius ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `? Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `color: tone.withValues(alpha: 0.18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `child: PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `icon: iconData,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `color: tone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `DueDateLabel(assignment: a, compact: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L584** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L585** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `}).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L591** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L592** `loading: () => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L593** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `title: context.l10n.announcementsTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `subtitle: context.l10n.announcementsSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `decorAsset: PaperAssets.tanNoteGreenTab,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `announcements.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L603** `data: (list) => list.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `? EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `title: context.l10n.announcementsEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `icon: Icons.campaign_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `: DashboardAnnouncements(announcements: list),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `loading: () => Skeletonizer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `enabled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `child: PremiumCard(child: Text(l10n.loadingAnnouncements)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `notifications.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `final unread = list.where((n) => !n.isRead).length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `onTap: () => context.go('/student/notifications'),`
Replace the navigation stack location. Login uses /boot so redirect can settle.

**L622** `child: Row(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `NeuIconWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `icon: Icons.notifications_none_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L628** `child: Badge(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L629** `isLabelVisible: unread > 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `'$unread',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `style: const TextStyle(fontSize: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `backgroundColor: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `child: PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `icon: Icons.notifications_none_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L648** `l10n.notificationsCardTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L654** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L655** `unread > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `? l10n.unreadAlerts(unread)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `: l10n.upToDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L664** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L665** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L674** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L675** `loading: () => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L677** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L679** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/student/home/student_dashboard_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/home/widgets/dashboard_announcements.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L6** `class DashboardAnnouncements extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L7** `const DashboardAnnouncements({super.key, required this.announcements});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `final List<Announcement> announcements;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `static const _previewCount = 3;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `State<DashboardAnnouncements> createState() => _DashboardAnnouncementsState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `class _DashboardAnnouncementsState extends State<DashboardAnnouncements> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L18** `bool _showAll = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final sorted = [...widget.announcements]`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `..sort((a, b) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final da = a.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final db = b.createdAt ?? DateTime.fromMillisecondsSinceEpoch(0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `return db.compareTo(da);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final canExpand = sorted.length > DashboardAnnouncements._previewCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final visible = _showAll || !canExpand`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `? sorted`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `: sorted.take(DashboardAnnouncements._previewCount).toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `AnimatedSize(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `duration: const Duration(milliseconds: 280),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `curve: Curves.easeOutCubic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `alignment: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `for (final a in visible)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `ExpandableAnnouncementCard(announcement: a),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `if (canExpand)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `padding: const EdgeInsets.only(top: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `child: TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `onPressed: () => setState(() => _showAll = !_showAll),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `_showAll`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `? Icons.expand_less_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `: Icons.expand_more_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `_showAll`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `? l10n.announcementsShowLess`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `: l10n.announcementsViewAll(sorted.length),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/home/widgets/dashboard_announcements.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/home/widgets/expandable_announcement_card.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L9** `class ExpandableAnnouncementCard extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L10** `const ExpandableAnnouncementCard({super.key, required this.announcement});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `final Announcement announcement;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `State<ExpandableAnnouncementCard> createState() =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `_ExpandableAnnouncementCardState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `class _ExpandableAnnouncementCardState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20** `extends State<ExpandableAnnouncementCard> {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `static const _collapsedLines = 3;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `bool _expanded = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `TextStyle get _bodyStyle => TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `height: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `bool _exceedsCollapsedLines(double maxWidth, TextDirection direction) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final text = widget.announcement.description.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (text.isEmpty) return false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final painter = TextPainter(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `text: TextSpan(text: text, style: _bodyStyle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `maxLines: _collapsedLines,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `textDirection: direction,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `)..layout(maxWidth: maxWidth);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return painter.didExceedMaxLines;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final a = widget.announcement;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final description = a.description.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `return PaperTaped(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `alignment: -0.55,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `child: PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `radius: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `backgroundColor: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `Icons.person,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `size: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `context.l10n.commonTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `color: AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `DateFormat.MMMd().format(a.createdAt ?? DateTime.now()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `a.title,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `if (description.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `LayoutBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `builder: (context, constraints) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `final canExpand = _exceedsCollapsedLines(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `constraints.maxWidth,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `Directionality.of(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `AnimatedSize(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `duration: const Duration(milliseconds: 220),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `curve: Curves.easeInOut,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `alignment: Alignment.topCenter,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `maxLines: _expanded ? null : _collapsedLines,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `overflow: _expanded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `? TextOverflow.visible`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `style: _bodyStyle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `if (canExpand) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `SizedBox(height: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `onTap: () => setState(() => _expanded = !_expanded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `behavior: HitTestBehavior.opaque,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `_expanded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `? context.l10n.announcementsShowLess`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `: context.l10n.announcementsReadMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `SizedBox(width: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `_expanded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `? Icons.expand_less_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `: Icons.expand_more_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/home/widgets/expandable_announcement_card.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/homework/student_assignments_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L19** `class StudentAssignmentsScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20** `const StudentAssignmentsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `void _open(BuildContext context, Assignment a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `context.push('/student/assignments/${a.id}');`
Push a route the user can pop (assignment detail, chat room).

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29** `final assignments = ref.watch(assignmentsProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L31** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `extendBody: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `appBar: AppBar(title: Text(l10n.assignmentsTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `body: assignments.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `final list = sortAssignmentsByDue(page.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `if (list.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `return ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `left: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `right: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `top: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `bottom: shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `const GroupSelector(padding: EdgeInsets.only(bottom: 12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `title: l10n.assignmentsEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `subtitle: l10n.assignmentsEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `icon: Icons.task_alt_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final isNeu = Neumorphic.isActive(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `return ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `clipBehavior: isNeu ? Clip.none : Clip.hardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `left: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `right: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `top: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `bottom: shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `itemCount: list.length + 1 + (page.hasMore ? 1 : 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `itemBuilder: (_, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `if (i == 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `title: l10n.assignmentsLearningPath,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `subtitle: l10n.assignmentsActive,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `if (i == list.length + 1) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `return LoadMoreButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `hasMore: page.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `loading: page.loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `onPressed: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `.read(assignmentsProvider(groupId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L85** `.loadMore(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final a = list[i - 1];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final isPack = a.type == AssignmentType.homeworkPack;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `final isMath = a.title.toLowerCase().contains('math');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final overdue = assignmentIsOverdue(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `final accent = AppColors.resolveAccent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `overdue`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `? AppColors.danger(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `: (isMath ? AppColors.mathAccent : AppColors.englishAccent),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final iconData = overdue`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `? Icons.warning_amber_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `: isMath`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `? Icons.calculate_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `: Icons.menu_book_rounded;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `onTap: () => _open(context, a),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `NeuIconWell(icon: iconData, color: accent, size: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `if (overdue) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `l10n.homeworkLateBadge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `color: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `const SizedBox(height: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `DueDateLabel(assignment: a, compact: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `if (a.description.isNotEmpty &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `a.dueDate == null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `a.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `if (isPack) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `l10n.homeworkPack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `Icon(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `class StudentAssignmentDetailScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L190** `const StudentAssignmentDetailScreen({super.key, required this.assignmentId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `final String assignmentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `final assignment = ref.watch(assignmentDetailProvider(assignmentId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L197** `final tasks = ref.watch(tasksProvider(assignmentId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L199** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `appBar: AppBar(title: Text(l10n.assignmentDetails)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `body: assignment.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `data: (a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `if (a == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `return EmptyState(title: l10n.assignmentNotFound);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `return RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `ref.invalidate(assignmentDetailProvider(assignmentId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L210** `ref.invalidate(tasksProvider(assignmentId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L211** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `shellContentBottomPadding(context, extra: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `borderRadius: BorderRadius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `color: AppColors.accent(context).withValues(alpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `blurRadius: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `offset: const Offset(0, 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `(a.type == AssignmentType.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `? l10n.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `: l10n.staticTask)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `color: Colors.white70,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `fontSize: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `letterSpacing: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `style: Theme.of(context).textTheme.headlineSmall`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `DueDateLabel(assignment: a, onDark: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `if (a.description.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `a.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `color: Colors.white.withValues(alpha: 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `height: 1.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `const SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `title: l10n.assignmentTasks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `subtitle: l10n.assignmentTasksSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `tasks.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `data: (items) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `if (items.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `title: l10n.assignmentsEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `icon: Icons.task_alt_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `return ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `physics: const NeverScrollableScrollPhysics(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `itemCount: items.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `itemBuilder: (context, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `final t = items[index];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `onTap: () => context.push('/student/tasks/${t.id}'),`
Push a route the user can pop (assignment detail, chat room).

**L294** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `width: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `height: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `color: AppColors.softWash(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `lightAlpha: 0.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `'${index + 1}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `t.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `if (t.description.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `t.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `color: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `Icons.arrow_forward_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `color: Colors.grey,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `loading: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `const SizedBox(height: 100),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/student/homework/student_assignments_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/homework/student_task_submit_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L21** `class StudentTaskSubmitScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L22** `const StudentTaskSubmitScreen({super.key, required this.taskId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final String taskId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `ConsumerState<StudentTaskSubmitScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L28** `_StudentTaskSubmitScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `const _signedUrlTtlSeconds = 60 * 60 * 24 * 365 * 10;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `class _StudentTaskSubmitScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L34** `extends ConsumerState<StudentTaskSubmitScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L35** `final _textController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `bool _loading = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `bool _completed = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `String? _fileUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `String? _fileName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `String? _storagePath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `Uint8List? _localBytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `String? _voiceUrl;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `String? _voiceStoragePath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `Map<String, dynamic>? _task;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `_load();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `Future<void> _load() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final task = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L55** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L56** `.eq('id', widget.taskId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L57** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `if (task != null) setState(() => _task = Map<String, dynamic>.from(task));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L60** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L61** `if (student == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final sub = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L64** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L65** `.eq('task_id', widget.taskId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L66** `.eq('student_id', student.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L67** `.maybeSingle();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (sub != null && mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `final url = sub['file_url'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `final voice = sub['voice_url'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `_textController.text = sub['text_answer'] as String? ?? '';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `_fileUrl = url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `_fileName = url == null ? null : attachmentFileName(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `_storagePath = url == null ? null : storagePathFromSignedUrl(url);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `_localBytes = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `_voiceUrl = voice;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `_voiceStoragePath = voice == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `: storagePathFromSignedUrl(voice);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `_completed = sub['is_completed'] as bool? ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `Future<void> _storeAttachment({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `required String fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `required Uint8List bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `final path = '$studentId/${widget.taskId}/$fileName';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L94** `.uploadBinary(`
Storage write. Path folders must satisfy bucket policies.

**L95** `path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `fileOptions: const FileOptions(upsert: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final url = await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `.from('submissions')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L101** `.createSignedUrl(path, _signedUrlTtlSeconds);`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L102** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `_fileUrl = url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `_fileName = fileName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `_storagePath = path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `_localBytes = bytes;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `Future<void> _uploadFile() async {`
Storage write. Path folders must satisfy bucket policies.

**L112** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L113** `if (student == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final result = await FilePicker.platform.pickFiles(withData: true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `if (result == null || result.files.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `final file = result.files.first;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `if (file.bytes == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `setState(() => _loading = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `await _storeAttachment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `studentId: student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `fileName: file.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `bytes: file.bytes!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `_showError(context.l10n.taskUploadFailed('$e'));`
Storage write. Path folders must satisfy bucket policies.

**L130** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `if (mounted) setState(() => _loading = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `Future<void> _pickImage() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L137** `if (student == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `final picker = ImagePicker();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `final image = await picker.pickImage(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `source: ImageSource.gallery,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `imageQuality: 88,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `if (image == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `final bytes = await image.readAsBytes();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `setState(() => _loading = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `await _storeAttachment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `studentId: student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `fileName: image.name,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `bytes: bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `_showError(context.l10n.taskUploadFailed('$e'));`
Storage write. Path folders must satisfy bucket policies.

**L155** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `if (mounted) setState(() => _loading = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `Future<void> _removeAttachment() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `final path = _storagePath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `_fileUrl = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `_fileName = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `_storagePath = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `_localBytes = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `if (path != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `await supabase.storage.from('submissions').remove([path]);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L172** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L176** `if (student == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L180** `.update({'file_url': null})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L181** `.eq('task_id', widget.taskId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L182** `.eq('student_id', student.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L183** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `Future<void> _uploadVoice(Uint8List bytes, String fileName) async {`
Storage write. Path folders must satisfy bucket policies.

**L187** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L188** `if (student == null) return;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `setState(() => _loading = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `final path = '${student.id}/${widget.taskId}/voice/$fileName';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L195** `.uploadBinary(`
Storage write. Path folders must satisfy bucket policies.

**L196** `path,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `bytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `fileOptions: FileOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `upsert: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `contentType: fileName.endsWith('.wav')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `? 'audio/wav'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `: 'audio/mp4',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `final url = await supabase.storage`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L207** `.createSignedUrl(path, _signedUrlTtlSeconds);`
Storage read. Private buckets need a signed URL; avatars may use getPublicUrl.

**L208** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `_voiceUrl = url;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `_voiceStoragePath = path;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `if (mounted) _showError(context.l10n.taskVoiceUploadFailed('$e'));`
Storage write. Path folders must satisfy bucket policies.

**L215** `rethrow;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `if (mounted) setState(() => _loading = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `Future<void> _removeVoice() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `final path = _voiceStoragePath;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `_voiceUrl = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `_voiceStoragePath = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `if (path != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `await supabase.storage.from('submissions').remove([path]);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L231** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L235** `if (student == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L239** `.update({'voice_url': null})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L240** `.eq('task_id', widget.taskId)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L241** `.eq('student_id', student.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L242** `} catch (_) {}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `void _showError(String message) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `).showSnackBar(SnackBar(content: Text(message)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `Future<void> _save({required bool submit}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L254** `if (student == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `setState(() => _loading = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `await supabase.from('submissions').upsert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L258** `'task_id': widget.taskId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `'student_id': student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `'text_answer': _textController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `'file_url': _fileUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `'voice_url': _voiceUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `'is_completed': _completed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `if (submit) 'status': 'submitted',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `if (submit) 'submitted_at': DateTime.now().toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `}, onConflict: 'task_id,student_id');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `SnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `content: Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `submit ? context.l10n.taskSubmitted : context.l10n.taskSaved,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `ref.invalidate(assignmentProgressProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L278** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `_showError(context.l10n.taskCouldNotSave('$e'));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `if (mounted) setState(() => _loading = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `_textController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `final task = _task;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `appBar: AppBar(title: Text(task?['title'] as String? ?? l10n.taskTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `body: task == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `? const Center(child: CircularProgressIndicator())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `shellContentBottomPadding(context, extra: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `PremiumCard(child: Text(task['description'] as String? ?? '')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `if (task['attachment_url'] != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `OutlinedButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `onPressed: () => openPdfViewer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `task['attachment_url'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `title: task['title'] as String?,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `icon: const Icon(Icons.picture_as_pdf_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `label: Text(l10n.taskViewPdf),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `if (task['link_url'] != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `onPressed: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `launchUrl(Uri.parse(task['link_url'] as String)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `child: Text(l10n.taskOpenLink),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `SwitchListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `title: Text(l10n.taskMarkCompleted),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `value: _completed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `onChanged: (v) => setState(() => _completed = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `controller: _textController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `maxLines: 5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `decoration: InputDecoration(labelText: l10n.taskYourAnswer),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `l10n.taskAttachment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `_fileUrl == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `? l10n.taskAttachmentHint`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `: l10n.taskAttachmentHintReady,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `if (_fileUrl != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `AttachmentPreview(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `url: _fileUrl!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `fileName: _fileName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `bytes: _localBytes,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `enabled: !_loading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `onRemove: _loading ? null : _removeAttachment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `child: _AttachAction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `icon: Icons.image_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `label: l10n.taskImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `subtitle: l10n.taskImageSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `onPressed: _loading ? null : _pickImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `const SizedBox(width: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `child: _AttachAction(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `icon: Icons.upload_file_outlined,`
Storage write. Path folders must satisfy bucket policies.

**L381** `label: l10n.taskFile,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `subtitle: l10n.taskFileSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `onPressed: _loading ? null : _uploadFile,`
Storage write. Path folders must satisfy bucket policies.

**L384** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `if (_fileUrl != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `onPressed: _loading ? null : _pickImage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `icon: const Icon(Icons.image_outlined, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `label: Text(l10n.taskReplaceImage),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `onPressed: _loading ? null : _uploadFile,`
Storage write. Path folders must satisfy bucket policies.

**L399** `icon: const Icon(Icons.upload_file_outlined, size: 18),`
Storage write. Path folders must satisfy bucket policies.

**L400** `label: Text(l10n.taskReplaceFile),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `l10n.taskVoice,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `style: Theme.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `).textTheme.titleSmall?.copyWith(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `l10n.taskVoiceHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `const SizedBox(height: 12),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `VoiceNoteField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `voiceUrl: _voiceUrl,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `enabled: !_loading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `onUpload: _uploadVoice,`
Storage write. Path folders must satisfy bucket policies.

**L425** `onRemove: _loading ? null : _removeVoice,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `const SizedBox(height: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `PrimaryButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `label: l10n.commonSaveDraft,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `loading: _loading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `onPressed: () => _save(submit: false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `PrimaryButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `label: l10n.commonSubmit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `loading: _loading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `onPressed: () => _save(submit: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `class _AttachAction extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L446** `const _AttachAction({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `required this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `required this.onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `final String subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `final VoidCallback? onPressed;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `return Material(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `color: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `child: InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `onTap: onPressed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `child: Ink(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `color: AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `borderRadius: BorderRadius.circular(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `border: Border.all(color: AppColors.inkAlpha(context, 0.08)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `color: AppColors.accent(context).withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `child: Icon(icon, color: AppColors.accent(context), size: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L494** `subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `fontSize: 11,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/homework/student_task_submit_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/marks/student_marks_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L23** `class StudentMarksScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24** `const StudentMarksScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final marks = ref.watch(myMarksProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29** `final stats = ref.watch(myMarkStatsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L30** `final average = ref.watch(myMarkAverageProvider).valueOrNull;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L31** `final markCount = stats.valueOrNull?.length ?? 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `appBar: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `title: Text(l10n.marksTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `tooltip: l10n.progressReportShare,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `onPressed: () => _shareProgress(context, ref),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `icon: const Icon(Icons.ios_share_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `body: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `ref.invalidate(myMarksProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L48** `ref.invalidate(myMarkStatsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L49** `ref.invalidate(myMarkAverageProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L50** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `child: marks.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `data: (page) => ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `left: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `right: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `top: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `bottom: shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `const TermSelector(padding: EdgeInsets.only(bottom: 12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `MarkAverageCard(average: average, markCount: markCount),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `if (markCount > 0) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `title: context.l10n.marksByCategory,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `subtitle: context.l10n.marksWhereStrongest,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `decorAsset: PaperAssets.brainVerified,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `_CategoryBreakdown(stats: stats.valueOrNull ?? const []),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `PremiumSectionHeader(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `title: context.l10n.marksAll,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `subtitle: context.l10n.marksNewestFirst,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `decorAsset: PaperAssets.documentLayout,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (page.items.isEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `title: context.l10n.marksEmptyTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `subtitle: context.l10n.marksEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `else ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `...page.items.map((m) => _StudentMarkTile(mark: m)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `LoadMoreButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `hasMore: page.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `loading: page.loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `onPressed: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `ref.read(myMarksProvider.notifier).loadMore(),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `] else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `title: context.l10n.marksEmptyTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `subtitle: context.l10n.marksEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `Future<void> _shareProgress(BuildContext context, WidgetRef ref) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `final groupId = ref.read(activeGroupIdProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L107** `if (groupId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `).showSnackBar(SnackBar(content: Text(l10n.progressReportNeedGroup)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `final student = await ref.read(studentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L115** `final profile = await ref.read(currentProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L116** `if (student == null || profile == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `SnackBar(content: Text(l10n.progressReportFailed('No profile'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L126** `final groupTitle =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `groups.where((g) => g.id == groupId).map((g) => g.title).firstOrNull ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `'group';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `final term = ref.read(selectedMarkTermProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L131** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `).showSnackBar(SnackBar(content: Text(l10n.progressReportSharing)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `await shareProgressReport(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `studentId: student.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `studentName: profile.fullName.isNotEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `? profile.fullName`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `: profile.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `term: term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `l10n: l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `).showSnackBar(SnackBar(content: Text(l10n.progressReportDone)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `SnackBar(content: Text(l10n.progressReportFailed('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `class _CategoryBreakdown extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L164** `const _CategoryBreakdown({required this.stats});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `final List<({MarkCategory category, double percentage, double weight})> stats;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `final byCategory =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `<MarkCategory, List<({double percentage, double weight})>>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `for (final s in stats) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `byCategory.putIfAbsent(s.category, () => []).add((`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `percentage: s.percentage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `weight: s.weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `final rows = byCategory.entries.map((entry) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `final totalWeight = entry.value.fold<double>(0, (s, m) => s + m.weight);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `final weighted = entry.value.fold<double>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `(s, m) => s + m.percentage * m.weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `return (entry.key, totalWeight == 0 ? 0.0 : weighted / totalWeight);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `}).toList()..sort((a, b) => b.$2.compareTo(a.$2));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `children: rows`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `(row) => Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `padding: const EdgeInsets.symmetric(vertical: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `flex: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `row.$1.l10nLabel(l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `flex: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `child: ClipRRect(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `borderRadius: BorderRadius.circular(4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `child: LinearProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `value: row.$2 / 100,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `minHeight: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `backgroundColor: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `valueColor: AlwaysStoppedAnimation(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `percentColor(row.$2, context: context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `width: 48,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `formatPercent(row.$2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `textAlign: TextAlign.end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `class _StudentMarkTile extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L245** `const _StudentMarkTile({required this.mark});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `final Mark mark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `mark.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `fontSize: 15,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `mark.category.l10nLabel(l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `if (mark.groupTitle != null) mark.groupTitle!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `DateFormat.yMMMd().format(mark.markedOn),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `].join(' · '),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `PercentBadge(percent: mark.percentage),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/marks/student_marks_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/notifications/student_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L16** `class StudentNotificationsScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17** `const StudentNotificationsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `ConsumerState<StudentNotificationsScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L21** `_StudentNotificationsScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `class _StudentNotificationsScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `extends ConsumerState<StudentNotificationsScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `WidgetsBinding.instance.addPostFrameCallback((_) => _markAllRead());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `Future<void> _markAllRead() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `final profile = ref.read(currentProfileProvider).valueOrNull;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L35** `if (profile == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `await markAllNotificationsRead(profile.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `ref.invalidate(notificationsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L40** `void _open(AppNotification n) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `final path = n.actionPath?.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `if (path == null || path.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `context.push(path);`
Push a route the user can pop (assignment detail, chat room).

**L46** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final notifications = ref.watch(notificationsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L51** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `appBar: AppBar(title: Text(l10n.notificationsTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `body: notifications.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `if (list.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `title: l10n.notificationsEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `subtitle: l10n.notificationsEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `return ListView.separated(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `shellContentBottomPadding(context, extra: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `itemCount: list.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `separatorBuilder: (_, _) => const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `itemBuilder: (context, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `final n = list[index];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `final when = n.createdAt;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `final timeLabel = when == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `: DateFormat.MMMd().add_jm().format(when.toLocal());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `final canOpen =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `n.actionPath != null && n.actionPath!.trim().isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `onTap: canOpen ? () => _open(n) : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `child: ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `leading: PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `icon: n.kindIcon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `size: 26,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `opacity: n.isRead ? 0.45 : 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `color: n.isRead`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `? AppColors.inkAlpha(context, 0.35)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `n.kindLabel(l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `fontWeight: n.isRead ? FontWeight.w500 : FontWeight.w800,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `subtitle: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `if (n.body.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `Text(n.body),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `if (timeLabel != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `const SizedBox(height: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `timeLabel,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `trailing: canOpen`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `? Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/notifications/student_notifications_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/student/profile/student_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class StudentProfileScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const StudentProfileScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `return const AppProfileScreen();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/student/profile/student_profile_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/analytics/teacher_analytics_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L15** `class TeacherAnalyticsScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16** `const TeacherAnalyticsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L21** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `if (groupId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `appBar: AppBar(title: Text(l10n.navAnalytics)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `body: EmptyState(title: l10n.teacherNoData),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final attendance = ref.watch(attendanceWeekTrendProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L31** `final averages = ref.watch(groupMarkAveragesProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L33** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `appBar: AppBar(title: Text(l10n.navAnalytics)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `body: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `ref.invalidate(attendanceWeekTrendProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L38** `ref.invalidate(groupMarkAveragesProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L39** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `shellContentBottomPadding(context, extra: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `title: l10n.teacherMarkDistribution,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `subtitle: l10n.teacherStudentsByAverage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `decorAsset: PaperAssets.texturedTanStar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `averages.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `data: (map) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final bands = markBandPoints(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `map.values.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `if (bands.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `title: l10n.marksNoRecorded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `subtitle: l10n.teacherNoMarksTrends,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `return PaperTaped(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `alignment: 0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `child: PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `padding: const EdgeInsets.fromLTRB(16, 20, 16, 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `child: MarkDistributionDoughnut(points: bands),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `title: l10n.attendanceTrend,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `subtitle: l10n.attendanceOverall,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `decorAsset: PaperAssets.calendar,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `attendance.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `data: (trend) => PaperTaped(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `alignment: -0.45,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `child: PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `padding: const EdgeInsets.fromLTRB(12, 16, 12, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `child: AttendanceTrendBars(trend: trend),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/analytics/teacher_analytics_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/announcements/teacher_announcements_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L16** `class TeacherAnnouncementsScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17** `const TeacherAnnouncementsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `ConsumerState<TeacherAnnouncementsScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L21** `_TeacherAnnouncementsScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `class _TeacherAnnouncementsScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `extends ConsumerState<TeacherAnnouncementsScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L26** `Future<void> _create() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L28** `final groups = await ref.read(myGroupsProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L29** `if (teacher == null || groups.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final title = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final body = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `var groupId = ref.read(activeGroupIdProvider) ?? groups.first.id;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L34** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `builder: (ctx, setStateLocal) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `title: Text(l10n.announcementsNew),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `controller: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `controller: body,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `decoration: InputDecoration(labelText: l10n.commonMessage),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `maxLines: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `initialValue: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `labelText: l10n.teacherTargetGroup,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `items: groups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `(g) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `DropdownMenuItem(value: g.id, child: Text(g.title)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `onChanged: (v) => setStateLocal(() => groupId = v ?? groupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `if (title.text.trim().isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `await supabase.from('announcements').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L81** `'teacher_id': teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `'group_id': groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `'title': title.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `'description': body.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `ref.invalidate(announcementsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L88** `},`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `child: Text(l10n.announcementsPost),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `Future<void> _edit(Announcement a) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final title = TextEditingController(text: a.title);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `final body = TextEditingController(text: a.description);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `title: Text(l10n.announcementsEdit),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `controller: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `controller: body,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `decoration: InputDecoration(labelText: l10n.commonMessage),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `maxLines: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `if (title.text.trim().isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `.from('announcements')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L132** `.update({`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L133** `'title': title.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `'description': body.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `.eq('id', a.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L137** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `ref.invalidate(announcementsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L139** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `child: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `Future<void> _delete(Announcement a) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `final confirmed = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `title: Text(l10n.announcementsDeleteTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `content: Text(l10n.announcementsDeleteBody),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `backgroundColor: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `if (confirmed != true) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `await supabase.from('announcements').delete().eq('id', a.id);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L171** `ref.invalidate(announcementsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L174** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L177** `final items = ref.watch(announcementsProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L178** `final myTeacherId = ref.watch(teacherProfileProvider).valueOrNull?.id;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L179** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `appBar: AppBar(title: Text(l10n.announcementsTitle)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `floatingActionButton: FloatingActionButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `onPressed: _create,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `child: const Icon(Icons.campaign_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `body: items.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `data: (list) => ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `itemCount: list.length + 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `itemBuilder: (_, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `if (index == 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `if (list.isEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `title: l10n.announcementsEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `subtitle: l10n.announcementsEmptyTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `final i = index - 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `final announcement = list[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `final mine =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `myTeacherId != null && announcement.teacherId == myTeacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `announcement.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `if (mine)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `PopupMenuButton<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `padding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `Icons.more_horiz_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `onSelected: (v) => v == 'edit'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `? _edit(announcement)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `: _delete(announcement),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `itemBuilder: (_) => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `value: 'edit',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `child: Text(l10n.commonEdit),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `value: 'delete',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `announcement.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `color: AppColors.inkAlpha(context, 0.7),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `height: 1.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `DateFormat.yMMMd().format(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `announcement.createdAt ?? DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `if (announcement.teacherId == null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `'• ${l10n.announcementsByAdmin}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `fontSize: 11,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/announcements/teacher_announcements_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/assignments/teacher_assignments_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L23** `class TeacherAssignmentsScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24** `const TeacherAssignmentsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `ConsumerState<TeacherAssignmentsScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L28** `_TeacherAssignmentsScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `class _TeacherAssignmentsScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L32** `extends ConsumerState<TeacherAssignmentsScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L33** `void _openPack(BuildContext context, Assignment a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `context.push('/teacher/assignments/${a.id}');`
Push a route the user can pop (assignment detail, chat room).

**L37** `Future<void> _createAssignment({AssignmentTemplate? fromTemplate}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L39** `final groups = await ref.read(myGroupsProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L40** `if (teacher == null || groups.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `var targetGroupId = ref.read(activeGroupIdProvider) ?? groups.first.id;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L43** `final titleController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `text: fromTemplate?.title ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `final descController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `text: fromTemplate?.description ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `var type = fromTemplate?.type ?? AssignmentType.staticAssignment;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `DateTime? dueDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `builder: (ctx, setStateLocal) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `fromTemplate == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `? l10n.assignmentNew`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `: l10n.teacherCreateFromTemplate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `if (fromTemplate != null && fromTemplate.tasks.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `padding: const EdgeInsets.only(bottom: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `l10n.teacherTasksCopied(fromTemplate.tasks.length),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `controller: titleController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `controller: descController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `labelText: l10n.commonDescription,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `if (fromTemplate == null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `DropdownButtonFormField<AssignmentType>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `initialValue: type,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `decoration: InputDecoration(labelText: l10n.commonType),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `items: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `value: AssignmentType.staticAssignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `child: Text(l10n.staticAssignment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `DropdownMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `value: AssignmentType.homeworkPack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `child: Text(l10n.homeworkPack),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `onChanged: (v) => setStateLocal(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `() => type = v ?? AssignmentType.staticAssignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `type == AssignmentType.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `? l10n.assignmentTypeHintPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `: l10n.assignmentTypeHintStatic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `height: 1.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `] else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `InputDecorator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `decoration: InputDecoration(labelText: l10n.commonType),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `type == AssignmentType.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `? l10n.homeworkPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `: l10n.staticAssignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `initialValue: targetGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `decoration: InputDecoration(labelText: l10n.commonGroup),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `items: groups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `(g) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `DropdownMenuItem(value: g.id, child: Text(g.title)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `onChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `setStateLocal(() => targetGroupId = v ?? targetGroupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `dueDate == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `? l10n.commonNoDueDate`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `: l10n.commonDueDate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `DateFormat.yMMMd().format(dueDate!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `trailing: const Icon(Icons.calendar_today_outlined, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `onTap: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `final picked = await showDatePicker(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `context: ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `initialDate: dueDate ?? DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `firstDate: DateTime.now().subtract(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `const Duration(days: 30),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `lastDate: DateTime.now().add(const Duration(days: 365)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `if (picked != null) setStateLocal(() => dueDate = picked);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `if (titleController.text.trim().isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `Assignment? created;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `if (fromTemplate != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `created = await createAssignmentFromTemplate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `template: fromTemplate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `groupId: targetGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `dueDate: dueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `titleOverride: titleController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `descriptionOverride: descController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `final title = titleController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `final description = descController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `final row = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L193** `.insert({`
Creates a row. Failures are usually unique violations or WITH CHECK policy failures.

**L194** `'teacher_id': teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `'group_id': targetGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `'type': assignmentTypeToDb(type),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `'description': description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `'due_date': dueDate?.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `.select()`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L202** `.single();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `created = Assignment.fromJson(Map<String, dynamic>.from(row));`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L204** `if (type == AssignmentType.staticAssignment) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `await supabase.from('tasks').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L206** `'assignment_id': created.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `'description': description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `'sort_order': 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `ref.invalidate(assignmentsProvider(targetGroupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L215** `if (mounted && created.type == AssignmentType.homeworkPack) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `_openPack(context, created);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `child: Text(l10n.commonCreate),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `Future<void> _editAssignment(Assignment a) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `final titleController = TextEditingController(text: a.title);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `final descController = TextEditingController(text: a.description);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `DateTime? dueDate = a.dueDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `final saved = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `builder: (ctx) => StatefulBuilder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `builder: (ctx, setStateLocal) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `title: Text(l10n.assignmentEdit),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `controller: titleController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `const SizedBox(height: 16),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `controller: descController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `labelText: l10n.commonDescription,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `maxLines: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `dueDate == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `? l10n.commonNoDueDate`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `: l10n.commonDueDate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `DateFormat.yMMMd().format(dueDate!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `trailing: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `if (dueDate != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `tooltip: l10n.commonNoDueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `onPressed: () => setStateLocal(() => dueDate = null),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `icon: const Icon(Icons.clear, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `const Icon(Icons.calendar_today_outlined, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `onTap: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `final picked = await showDatePicker(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `context: ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `initialDate: dueDate ?? DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `firstDate: DateTime.now().subtract(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `const Duration(days: 365),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `lastDate: DateTime.now().add(const Duration(days: 365)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `if (picked != null) setStateLocal(() => dueDate = picked);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `child: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `if (saved != true || titleController.text.trim().isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `final title = titleController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `final description = descController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `.from('assignments')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L313** `.update({`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L314** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `'description': description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `'due_date': dueDate?.toIso8601String(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `.eq('id', a.id);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L319** `if (a.type == AssignmentType.staticAssignment) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `final tasks = await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `.from('tasks')`

PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L322** `.select('id')`
Read columns. Narrow the select when the row is wide; '*' is acceptable for small domain rows.

**L323** `.eq('assignment_id', a.id)`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L324** `.order('sort_order')`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `.limit(1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `if (tasks.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `await supabase.from('tasks').insert({`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L328** `'assignment_id': a.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `'title': title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `'description': description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `'sort_order': 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `.from('tasks')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L336** `.update({'title': title, 'description': description})`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L337** `.eq('id', tasks.first['id'] as String);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L340** `ref.invalidate(assignmentsProvider(a.groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L341** `ref.invalidate(assignmentDetailProvider(a.id));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L342** `ref.invalidate(tasksProvider(a.id));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L343** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `Future<void> _deleteAssignment(Assignment a) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `final confirmed = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `title: Text(l10n.assignmentDeleteTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `content: Text(l10n.assignmentDeleteBody),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `backgroundColor: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `if (confirmed != true) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `await supabase.from('assignments').delete().eq('id', a.id);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L376** `ref.invalidate(assignmentsProvider(a.groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L377** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `).showSnackBar(SnackBar(content: Text(l10n.assignmentDeleted)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `Future<void> _duplicateAssignment(Assignment source) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L391** `final groups = await ref.read(myGroupsProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L392** `if (teacher == null || groups.isEmpty || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `var targetGroupId = ref.read(activeGroupIdProvider) ?? source.groupId;`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L395** `final titleController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `text: '${source.title} (copy)',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `DateTime? dueDate = source.dueDate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `final confirmed = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `builder: (ctx) => StatefulBuilder(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `builder: (ctx, setStateLocal) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `title: Text(l10n.teacherDuplicateTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `controller: titleController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `initialValue: targetGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `decoration: InputDecoration(labelText: l10n.commonGroup),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `items: groups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `(g) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `DropdownMenuItem(value: g.id, child: Text(g.title)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `onChanged: (v) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `setStateLocal(() => targetGroupId = v ?? targetGroupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `dueDate == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `? l10n.commonNoDueDate`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `: l10n.commonDueDate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `DateFormat.yMMMd().format(dueDate!),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `trailing: const Icon(Icons.calendar_today_outlined, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `onTap: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `final picked = await showDatePicker(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `context: ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `initialDate: dueDate ?? DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `firstDate: DateTime.now().subtract(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `const Duration(days: 30),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `lastDate: DateTime.now().add(const Duration(days: 365)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `if (picked != null) setStateLocal(() => dueDate = picked);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `child: Text(l10n.teacherDuplicateAssignment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `if (confirmed != true || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `final created = await duplicateAssignment(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `source: source,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `groupId: targetGroupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `dueDate: dueDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `titleOverride: titleController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L477** `ref.invalidate(assignmentsProvider(targetGroupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L478** `if (source.groupId != targetGroupId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `ref.invalidate(assignmentsProvider(source.groupId));`

Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L481** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `).showSnackBar(SnackBar(content: Text(l10n.teacherDuplicated)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L485** `if (created.type == AssignmentType.homeworkPack) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L486** `_openPack(context, created);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `Future<void> _saveAsTemplate(Assignment assignment) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L498** `if (teacher == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `await saveAssignmentAsTemplate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L502** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L503** `assignment: assignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `).showSnackBar(SnackBar(content: Text(l10n.teacherTemplateSaved)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L512** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `Future<void> _openTemplates() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L519** `if (teacher == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `await showModalBottomSheet<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `isScrollControlled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L526** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `return _TemplatesSheet(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `onUse: (template) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `_createAssignment(fromTemplate: template);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L532** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L541** `final assignments = ref.watch(assignmentsProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L542** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `appBar: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `title: Text(l10n.assignmentsTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `tooltip: l10n.teacherTemplates,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `onPressed: _openTemplates,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `icon: const Icon(Icons.dashboard_customize_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `floatingActionButton: FloatingActionButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `onPressed: () => _createAssignment(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `child: const Icon(Icons.add),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `body: assignments.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `final list = sortAssignmentsByDue(page.items);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `final isNeu = Neumorphic.isActive(context);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `return ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `clipBehavior: isNeu ? Clip.none : Clip.hardEdge,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `itemCount: list.length + 1 + (page.hasMore ? 1 : 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `itemBuilder: (_, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `if (index == 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `if (list.isEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `title: l10n.assignmentsEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `subtitle: l10n.assignmentsEmptyTeacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `if (index == list.length + 1) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `return LoadMoreButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `hasMore: page.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L589** `loading: page.loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L590** `onPressed: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L591** `.read(assignmentsProvider(groupId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L592** `.loadMore(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L595** `final a = list[index - 1];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `final isPack = a.type == AssignmentType.homeworkPack;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `final overdue = assignmentIsOverdue(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `final tone = AppColors.resolveAccent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L600** `overdue`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L601** `? AppColors.danger(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `: isPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L603** `? AppColors.goldAccent(context)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L606** `final iconData = overdue`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `? Icons.warning_amber_rounded`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L608** `: isPack`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `? Icons.inventory_2_outlined`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `: Icons.description_outlined;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `onTap: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `isPack ? _openPack(context, a) : _editAssignment(a),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L615** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `vertical: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `NeuIconWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L622** `icon: iconData,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `color: tone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `padding: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L628** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L629** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `DueDateLabel(assignment: a, compact: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `isPack ? l10n.homeworkPack : l10n.staticTask,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `color: AppColors.inkAlpha(context, 0.45),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `PopupMenuButton<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `onSelected: (value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `switch (value) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L654** `case 'edit':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L655** `_editAssignment(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `case 'delete':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `_deleteAssignment(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `case 'duplicate':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `_duplicateAssignment(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `case 'template':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `_saveAsTemplate(a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `case 'open':`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L663** `_openPack(context, a);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L665** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `itemBuilder: (_) => [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `if (isPack)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `value: 'open',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `child: Text(l10n.teacherManagePack),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L671** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L673** `value: 'edit',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L674** `child: Text(l10n.commonEdit),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L675** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L677** `value: 'duplicate',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L678** `child: Text(l10n.teacherDuplicateAssignment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L679** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L680** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L681** `value: 'template',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L682** `child: Text(l10n.teacherSaveAsTemplate),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L683** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L684** `PopupMenuItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L685** `value: 'delete',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L686** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L687** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L689** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L691** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L693** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L696** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L697** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L703** `class _TemplatesSheet extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L704** `const _TemplatesSheet({required this.teacherId, required this.onUse});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L706** `final String teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `final ValueChanged<AssignmentTemplate> onUse;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L709** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `State<_TemplatesSheet> createState() => _TemplatesSheetState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L713** `class _TemplatesSheetState extends State<_TemplatesSheet> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L714** `late Future<List<AssignmentTemplate>> _future;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L716** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L717** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L718** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L719** `_future = fetchAssignmentTemplates(widget.teacherId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L722** `Future<void> _reload() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L723** `setState(() {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L724** `_future = fetchAssignmentTemplates(widget.teacherId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L728** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L729** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L730** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L731** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L732** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L733** `bottom: MediaQuery.of(context).viewInsets.bottom,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L734** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L735** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L736** `constraints: BoxConstraints(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L737** `maxHeight: MediaQuery.sizeOf(context).height * 0.75,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L738** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L739** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L740** `color: AppColors.sheetSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L741** `borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L742** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L743** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L744** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L745** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L746** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L747** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L748** `height: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L749** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L750** `color: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L751** `borderRadius: BorderRadius.circular(2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L752** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L753** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L754** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L755** `padding: const EdgeInsets.fromLTRB(20, 12, 20, 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L756** `child: PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L757** `title: l10n.teacherTemplates,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L758** `subtitle: l10n.teacherTemplatesEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L759** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L760** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L761** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L762** `child: FutureBuilder<List<AssignmentTemplate>>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L763** `future: _future,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L764** `builder: (context, snap) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L765** `if (snap.connectionState != ConnectionState.done) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L766** `return const Center(child: CircularProgressIndicator());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L768** `if (snap.hasError) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L769** `return ErrorState(message: '${snap.error}');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L771** `final items = snap.data ?? const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L772** `if (items.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L773** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L774** `title: l10n.teacherTemplatesEmpty,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L775** `subtitle: l10n.teacherTemplatesEmptySubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L776** `icon: Icons.dashboard_customize_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L779** `return ListView.separated(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L780** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L781** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L782** `0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L783** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L784** `MediaQuery.of(context).padding.bottom + 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L785** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L786** `itemCount: items.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L787** `separatorBuilder: (context, index) =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L788** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L789** `itemBuilder: (_, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L790** `final t = items[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L791** `final isPack = t.type == AssignmentType.homeworkPack;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L792** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L793** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L794** `horizontal: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L795** `vertical: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L796** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L797** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L798** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L799** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L800** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L801** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L802** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L803** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L804** `t.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L805** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L806** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L807** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L808** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L809** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L810** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L811** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L812** `'${isPack ? l10n.homeworkPack : l10n.staticTask}'`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L813** `'${t.tasks.isEmpty ? '' : ' · ${l10n.teacherTasksCopied(t.tasks.length)}'}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L814** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L815** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L816** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L817** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L818** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L820** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L821** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L822** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L823** `onPressed: () => widget.onUse(t),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L824** `child: Text(l10n.teacherUseTemplate),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L825** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L826** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L827** `tooltip: l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L828** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L829** `final ok = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L830** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L831** `builder: (c) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L832** `title: Text(l10n.teacherDeleteTemplate),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L833** `content: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L834** `l10n.teacherDeleteTemplateBody(t.title),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L835** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L836** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L837** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L838** `onPressed: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L839** `Navigator.pop(c, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L840** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L841** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L842** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L843** `onPressed: () => Navigator.pop(c, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L844** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L845** `l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L846** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L847** `color: Colors.red,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L848** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L849** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L850** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L852** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L854** `if (ok == true) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L855** `await deleteAssignmentTemplate(t.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L856** `await _reload();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L858** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L859** `icon: const Icon(Icons.delete_outline),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L860** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L862** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L864** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L866** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L867** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L868** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L870** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L871** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/assignments/teacher_assignments_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/attendance/teacher_attendance_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L16** `class TeacherAttendanceScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17** `const TeacherAttendanceScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `ConsumerState<TeacherAttendanceScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L21** `_TeacherAttendanceScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `class _TeacherAttendanceScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L25** `extends ConsumerState<TeacherAttendanceScreen>`
Widget that can watch providers. Required for profile/theme/locale.

**L26** `with SingleTickerProviderStateMixin {`
Riverpod binding. Name it after the value it exposes.

**L27** `final Map<String, AttendanceStatus> _edits = {};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `String? _editedGroupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `bool _saving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `late final TabController _tabs;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `late DateTime _month;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `String? _historyStudentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `_tabs = TabController(length: 2, vsync: this);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `final now = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `_month = DateTime(now.year, now.month);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `_tabs.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `AttendanceStatus _statusFor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `Map<String, AttendanceStatus> saved,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `return _edits[studentId] ?? saved[studentId] ?? AttendanceStatus.present;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `void _resetEditsIfGroupChanged(String groupId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `if (_editedGroupId != groupId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `_edits.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `_editedGroupId = groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `_historyStudentId = null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `void _markAll(List<GroupMember> members, AttendanceStatus status) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `setState(() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `for (final m in members) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `_edits[m.studentId] = status;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `Future<void> _saveAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `List<GroupMember> members,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `Map<String, AttendanceStatus> saved,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `if (members.isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `setState(() => _saving = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `final byStudent = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `for (final m in members) m.studentId: _statusFor(m.studentId, saved),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `await upsertAttendanceBulk(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `date: DateTime.now(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `byStudentId: byStudent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `ref.invalidate(todayAttendanceProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L90** `ref.invalidate(attendanceWeekTrendProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L91** `ref.invalidate(attendanceRangeProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L92** `_edits.clear();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `).showSnackBar(SnackBar(content: Text(context.l10n.attendanceSaved)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `} on AttendanceWeekendException {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `SnackBar(content: Text(context.l10n.attendanceWeekendNotAllowed)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `} on AttendanceFutureDateException {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `SnackBar(content: Text(context.l10n.attendanceFutureNotAllowed)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `SnackBar(content: Text(context.l10n.commonErrorWithDetails('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `if (mounted) setState(() => _saving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `Future<void> _cycleHistoryDay({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `required DateTime day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `required AttendanceStatus? current,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `if (!isAttendanceDateAllowed(day)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `SnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `content: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `isAttendanceWeekend(day)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `? context.l10n.attendanceWeekendNotAllowed`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `: context.l10n.attendanceFutureNotAllowed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `final next = nextAttendanceStatus(current);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `await upsertAttendance(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `studentId: studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `status: next,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `date: day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `ref.invalidate(todayAttendanceProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L151** `ref.invalidate(attendanceWeekTrendProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L152** `ref.invalidate(attendanceRangeProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L153** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `).showSnackBar(SnackBar(content: Text(context.l10n.attendanceUpdated)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `} on AttendanceWeekendException {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `SnackBar(content: Text(context.l10n.attendanceWeekendNotAllowed)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `} on AttendanceFutureDateException {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `SnackBar(content: Text(context.l10n.attendanceFutureNotAllowed)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `SnackBar(content: Text(context.l10n.commonErrorWithDetails('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L182** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `final canMarkToday = isAttendanceDateAllowed(DateTime.now());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `if (groupId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `appBar: AppBar(title: Text(l10n.attendanceMark)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `body: EmptyState(title: l10n.attendanceNoGroups),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `_resetEditsIfGroupChanged(groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final members = ref.watch(groupMembersProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L195** `final saved = ref.watch(todayAttendanceProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L196** `final memberList = members.valueOrNull ?? const <GroupMember>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `final historyStudentId =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `_historyStudentId ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `(memberList.isEmpty ? null : memberList.first.studentId);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `appBar: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `title: Text(l10n.attendanceTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `if (_tabs.index == 0)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `padding: const EdgeInsets.only(right: 16, top: 8, bottom: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `child: Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `AppStyleTokens.maybeOf(context)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `return FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `onPressed:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `!canMarkToday ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `_saving ||`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `members.valueOrNull == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `: () => _saveAll(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `members.value!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `saved.valueOrNull ?? const {},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `foregroundColor: AppColors.onAccent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `disabledBackgroundColor: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `0.12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `disabledForegroundColor: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `0.35,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `padding: const EdgeInsets.symmetric(horizontal: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `shape: RoundedRectangleBorder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `borderRadius: BorderRadius.circular(isMono ? 4 : 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `child: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `height: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `color: AppColors.onAccent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `: Text(l10n.commonSave),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `bottom: TabBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `controller: _tabs,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `onTap: (_) => setState(() {}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `tabs: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `Tab(text: l10n.attendanceMark),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `Tab(text: l10n.attendanceHistory),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `body: TabBarView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `controller: _tabs,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `members.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `final savedMap =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `saved.valueOrNull ?? const <String, AttendanceStatus>{};`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `const GroupSelector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `padding: EdgeInsets.fromLTRB(20, 16, 20, 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `padding: const EdgeInsets.fromLTRB(20, 20, 20, 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `DateFormat.yMMMMd().format(DateTime.now()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `if (!canMarkToday) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `l10n.attendanceWeekendNotAllowed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `_BulkActions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `onPresent: !canMarkToday`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `_markAll(list, AttendanceStatus.present),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `onLate: !canMarkToday`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `: () => _markAll(list, AttendanceStatus.late),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `onAbsent: !canMarkToday`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `_markAll(list, AttendanceStatus.absent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `onExcused: !canMarkToday`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `_markAll(list, AttendanceStatus.excused),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `const Divider(height: 1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `child: list.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `? EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `title: l10n.marksNoStudents,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `icon: Icons.person_off_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `padding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `children: List.generate(list.length, (index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `final member = list[index];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `horizontal: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `vertical: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `'${index + 1}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `color: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `0.3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `fontWeight: FontWeight.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `const SizedBox(width: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `crossAxisAlignment:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `member.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `fontWeight:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `FontWeight.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `color: AppColors.ink(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `member.username,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `color:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `0.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `_AttendanceToggle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `status: _statusFor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `member.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `savedMap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `onChanged: !canMarkToday`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `: (s) => setState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `() => _edits[member`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `.studentId] =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `s,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `if (index < list.length - 1)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `Divider(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `height: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `indent: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `endIndent: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L413** `color: AppColors.inkAlpha(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `0.12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `}),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L421** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `spacing: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `runSpacing: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `_LegendItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `label: l10n.attendancePresent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `_LegendItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `label: l10n.attendanceLate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `_LegendItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `label: l10n.attendanceAbsent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `_LegendItem(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `label: l10n.attendanceExcused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `color: const Color(0xFF5B8DB8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `_TeacherAttendanceHistoryTab(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `members: memberList,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `month: _month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `selectedStudentId: historyStudentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `onMonthChanged: (m) => setState(() => _month = m),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `onStudentChanged: (id) => setState(() => _historyStudentId = id),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `onDayTap: (day, current) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `if (historyStudentId == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `_cycleHistoryDay(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `studentId: historyStudentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `day: day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `current: current,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `class _TeacherAttendanceHistoryTab extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L477** `const _TeacherAttendanceHistoryTab({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L479** `required this.members,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L480** `required this.month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L481** `required this.selectedStudentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L482** `required this.onMonthChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L483** `required this.onStudentChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L484** `required this.onDayTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L487** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L488** `final List<GroupMember> members;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L489** `final DateTime month;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L490** `final String? selectedStudentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L491** `final ValueChanged<DateTime> onMonthChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L492** `final ValueChanged<String?> onStudentChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L493** `final void Function(DateTime day, AttendanceStatus? current) onDayTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L495** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L496** `Widget build(BuildContext context, WidgetRef ref) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L497** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L498** `if (members.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L499** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L500** `title: l10n.marksNoStudents,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L501** `icon: Icons.person_off_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L505** `final from = DateTime(month.year, month.month, 1);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L506** `final to = DateTime(month.year, month.month + 1, 0);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L507** `final query = AttendanceQuery(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L508** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L509** `studentId: selectedStudentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L510** `from: from,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L511** `to: to,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L513** `final records = ref.watch(attendanceRangeProvider(query));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L514** `final stats = ref.watch(attendanceRangeStatsProvider(query));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L516** `return ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L517** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L518** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L519** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L520** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L521** `shellContentBottomPadding(context, extra: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L522** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L523** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L524** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L525** `DropdownButtonFormField<String>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L527** `value: selectedStudentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L528** `decoration: InputDecoration(labelText: l10n.attendanceSelectStudent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L529** `items: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L530** `for (final m in members)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L531** `DropdownMenuItem(value: m.studentId, child: Text(m.fullName)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L533** `onChanged: onStudentChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L534** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L535** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L536** `AttendanceMonthHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L537** `month: month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L538** `onPrev: () => onMonthChanged(DateTime(month.year, month.month - 1)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L539** `onNext: () => onMonthChanged(DateTime(month.year, month.month + 1)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L540** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L541** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L542** `stats.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L543** `data: (s) => AttendanceStatsBar(stats: s),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L544** `loading: () => const SizedBox(height: 56),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L545** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L546** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L547** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L548** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L549** `l10n.attendanceTapToCycle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L550** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L551** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L552** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L553** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L554** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L555** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L556** `PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L557** `child: records.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L558** `loading: () => const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L559** `padding: EdgeInsets.all(32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L560** `child: Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L561** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L562** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L563** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L564** `final byDate = <DateTime, AttendanceStatus>{`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L565** `for (final r in list)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L566** `DateTime(r.date.year, r.date.month, r.date.day): r.status,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L567** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L568** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L569** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L570** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L571** `AttendanceCalendarGrid(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L572** `month: month,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L573** `statusByDate: byDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L574** `editable: selectedStudentId != null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L575** `onDayTap: selectedStudentId == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L576** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L577** `: (day) => onDayTap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L578** `day,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L579** `byDate[DateTime(day.year, day.month, day.day)],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L580** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L581** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L582** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L583** `const AttendanceLegendRow(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L586** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L587** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L588** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L594** `class _BulkActions extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L595** `const _BulkActions({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L596** `required this.onPresent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L597** `required this.onLate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L598** `required this.onAbsent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L599** `required this.onExcused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L602** `final VoidCallback? onPresent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L603** `final VoidCallback? onLate;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L604** `final VoidCallback? onAbsent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L605** `final VoidCallback? onExcused;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L607** `static const _excusedBlue = Color(0xFF5B8DB8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L609** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L610** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L611** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L612** `return Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L613** `spacing: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L614** `runSpacing: 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L615** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L616** `TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L617** `onPressed: onPresent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L618** `icon: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L619** `Icons.check_circle_outline,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L620** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L621** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L622** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L623** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L624** `l10n.attendanceAllPresent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L625** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L626** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L627** `fontWeight: FontWeight.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L628** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L629** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L630** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L631** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L632** `TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L633** `onPressed: onLate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L634** `icon: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L635** `Icons.schedule,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L636** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L637** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L638** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L639** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L640** `l10n.attendanceAllLate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L641** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L642** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L643** `fontWeight: FontWeight.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L644** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L645** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L646** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L647** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L648** `TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L649** `onPressed: onAbsent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L650** `icon: const Icon(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L651** `Icons.highlight_off,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L652** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L653** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L654** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L655** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L656** `l10n.attendanceAllAbsent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L657** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L658** `color: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L659** `fontWeight: FontWeight.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L660** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L661** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L662** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L663** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L664** `TextButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L665** `onPressed: onExcused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L666** `icon: const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L667** `Icons.event_available_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L668** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L669** `color: _excusedBlue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L670** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L671** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L672** `l10n.attendanceAllExcused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L673** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L674** `color: _excusedBlue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L675** `fontWeight: FontWeight.bold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L676** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L677** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L678** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L679** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L685** `class _AttendanceToggle extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L686** `const _AttendanceToggle({required this.status, required this.onChanged});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L688** `final AttendanceStatus? status;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L689** `final ValueChanged<AttendanceStatus>? onChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L691** `static const _excusedBlue = Color(0xFF5B8DB8);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L693** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L694** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L695** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L696** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L697** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L698** `_StatusButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L699** `label: context.l10n.attendancePresentShort,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L700** `isSelected: status == AttendanceStatus.present,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L701** `activeColor: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L702** `onTap: onChanged == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L703** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L704** `: () => onChanged!(AttendanceStatus.present),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L705** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L706** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L707** `_StatusButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L708** `label: context.l10n.attendanceLateShort,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L709** `isSelected: status == AttendanceStatus.late,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L710** `activeColor: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L711** `onTap: onChanged == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L712** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L713** `: () => onChanged!(AttendanceStatus.late),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L714** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L715** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L716** `_StatusButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L717** `label: context.l10n.attendanceAbsentShort,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L718** `isSelected: status == AttendanceStatus.absent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L719** `activeColor: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L720** `onTap: onChanged == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L721** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L722** `: () => onChanged!(AttendanceStatus.absent),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L723** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L724** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L725** `_StatusButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L726** `label: context.l10n.attendanceExcusedShort,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L727** `isSelected: status == AttendanceStatus.excused,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L728** `activeColor: _excusedBlue,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L729** `onTap: onChanged == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L730** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L731** `: () => onChanged!(AttendanceStatus.excused),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L732** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L738** `class _StatusButton extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L739** `const _StatusButton({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L740** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L741** `required this.isSelected,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L742** `required this.activeColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L743** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L746** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L747** `final bool isSelected;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L748** `final Color activeColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L749** `final VoidCallback? onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L751** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L752** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L753** `return GestureDetector(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L754** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L755** `child: AnimatedContainer(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L756** `duration: const Duration(milliseconds: 200),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L757** `width: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L758** `height: 36,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L759** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L760** `color: isSelected ? activeColor : AppColors.controlSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L761** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L762** `border: Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L763** `color: isSelected ? activeColor : AppColors.inkAlpha(context, 0.25),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L764** `width: 1.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L765** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L766** `boxShadow: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L767** `? [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L768** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L769** `color: activeColor.withValues(alpha: 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L770** `blurRadius: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L771** `offset: const Offset(0, 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L772** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L774** `: [],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L775** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L776** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L777** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L778** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L779** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L780** `color: isSelected`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L781** `? Colors.white`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L782** `: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L783** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L784** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L785** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L786** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L787** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L788** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L793** `class _LegendItem extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L794** `const _LegendItem({required this.label, required this.color});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L795** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L796** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L798** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L799** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L800** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L801** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L802** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L803** `width: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L804** `height: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L805** `decoration: BoxDecoration(color: color, shape: BoxShape.circle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L806** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L807** `const SizedBox(width: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L808** `Text(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L809** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L810** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L811** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L812** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L813** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L814** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L815** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/teacher/attendance/teacher_attendance_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/dashboard/teacher_dashboard_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L18** `class TeacherDashboardScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L19** `const TeacherDashboardScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final profile = ref.watch(currentProfileProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L24** `final teacher = ref.watch(teacherProfileProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L25** `final groups = ref.watch(myGroupsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L26** `final sphere = teacher.valueOrNull?.sphere;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `extendBody: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `floatingActionButton: FloatingActionButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `child: const Icon(Icons.add),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `onPressed: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `showModalBottomSheet(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `final isMono =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `AppStyleTokens.maybeOf(ctx)?.isMonochromeWorkspace ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `final sheetRadius = isMono ? 8.0 : 24.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `final iconRadius = isMono ? 4.0 : 20.0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `Widget actionIcon({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `required Color color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `required IconData icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final fill = AppColors.resolveAccent(ctx, color);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final isNeu = Neumorphic.isActive(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `if (isNeu) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `return NeuIconWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `icon: icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `color: fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `padding: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `elevation: NeuElevation.raised,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `height: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `alignment: Alignment.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `color: fill,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `borderRadius: BorderRadius.circular(iconRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `child: Icon(icon, color: AppColors.onAccent(ctx), size: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `margin: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `16 + MediaQuery.viewPaddingOf(ctx).bottom,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `color: AppColors.sheetSurface(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `borderRadius: BorderRadius.circular(sheetRadius),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `border: isMono`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `? Border.all(color: AppColors.inkAlpha(ctx, 0.12))`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `title: l10n.teacherQuickActions,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `subtitle: l10n.teacherCreateItems,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `leading: actionIcon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `color: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `icon: Icons.assignment_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `l10n.teacherNewAssignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `color: AppColors.ink(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `onTap: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `context.push('/teacher/assignments');`
Push a route the user can pop (assignment detail, chat room).

**L111** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `ListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `leading: actionIcon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `icon: Icons.campaign_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `l10n.teacherNewAnnouncement,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `color: AppColors.ink(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `onTap: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `context.push('/teacher/announcements');`
Push a route the user can pop (assignment detail, chat room).

**L128** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `body: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `ref.invalidate(currentProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L140** `ref.invalidate(teacherProfileProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L141** `ref.invalidate(myGroupsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L142** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `left: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `right: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `top: MediaQuery.of(context).padding.top + 24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `bottom: shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `profile.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `data: (p) => Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `Builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `builder: (context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `final sphereColor = AppColors.resolveAccent(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `sphere == Sphere.english`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `? AppColors.englishAccent`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `: AppColors.mathAccent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `return Container(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `padding: const EdgeInsets.all(8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `color: sphereColor.withValues(alpha: 0.1),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `borderRadius: BorderRadius.circular(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `AppStyleTokens.maybeOf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `)?.isMonochromeWorkspace ==`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `true`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `? 6`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `border:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `(AppStyleTokens.maybeOf(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `)?.isMonochromeWorkspace ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `false)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `? Border.all(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `color: sphereColor.withValues(alpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `: null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `child: PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `icon: sphere == Sphere.english`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `? Icons.menu_book`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `: Icons.calculate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `color: sphereColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `l10n.teacherWorkspace,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `style: Theme.of(context).textTheme.bodyLarge?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `l10n.greetingWithName(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `timeOfDayGreeting(l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `p?.fullName.split(' ').first ?? p?.username ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `style: Theme.of(context).textTheme.headlineSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `letterSpacing: -0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `loading: () => Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `error: (_, _) => SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `groups.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `data: (g) => Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `_StatCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `label: l10n.teacherActiveGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `value: '${g.length}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `icon: Icons.groups_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `_StatCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `label: l10n.teacherSubject,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `value: sphere?.name.toUpperCase() ?? '—',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `icon: Icons.subject_outlined,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `loading: () => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `.watch(notificationsProvider)`
Riverpod binding. Name it after the value it exposes.

**L248** `.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `final unread = list.where((n) => !n.isRead).length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `onTap: () => context.push('/teacher/notifications'),`
Push a route the user can pop (assignment detail, chat room).

**L253** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `NeuIconWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `icon: Icons.notifications_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `child: Badge(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `isLabelVisible: unread > 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `'$unread',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `style: const TextStyle(fontSize: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `backgroundColor: AppColors.danger(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `Icons.notifications_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `l10n.notificationsCardTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `unread > 0`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `? l10n.unreadAlerts(unread)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `: l10n.upToDate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `loading: () => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `error: (_, _) => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `title: l10n.teacherDailyTasks,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `subtitle: l10n.teacherNeedsAttention,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `_QuickActionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `title: l10n.attendanceMark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `subtitle: l10n.attendanceDailyTracking,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `icon: Icons.how_to_reg_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `onTap: () => context.push('/teacher/attendance'),`
Push a route the user can pop (assignment detail, chat room).

**L317** `accentColor: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `_QuickActionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `title: l10n.teacherGradebook,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `subtitle: l10n.teacherRecordMarks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `icon: Icons.grade_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `onTap: () => context.push('/teacher/marks'),`
Push a route the user can pop (assignment detail, chat room).

**L324** `accentColor: AppColors.mathAccent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `_QuickActionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `title: l10n.teacherReviewSubmissions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `subtitle: l10n.teacherCheckAssignments,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `icon: Icons.fact_check_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `onTap: () => context.push('/teacher/submissions'),`
Push a route the user can pop (assignment detail, chat room).

**L331** `accentColor: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `_QuickActionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `title: l10n.assignmentManage,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `subtitle: l10n.assignmentManageSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `icon: Icons.assignment_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `onTap: () => context.push('/teacher/assignments'),`
Push a route the user can pop (assignment detail, chat room).

**L338** `accentColor: AppColors.coffeeBrown,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `_QuickActionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `title: l10n.announcementsTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `subtitle: l10n.announcementsShareUpdates,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `icon: Icons.campaign_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `onTap: () => context.push('/teacher/announcements'),`
Push a route the user can pop (assignment detail, chat room).

**L345** `accentColor: AppColors.englishAccent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `_QuickActionCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `title: l10n.teacherPerformanceAnalytics,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `subtitle: l10n.teacherTrackProgress,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `icon: Icons.analytics_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `onTap: () => context.push('/teacher/analytics'),`
Push a route the user can pop (assignment detail, chat room).

**L352** `accentColor: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `class _StatCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L362** `const _StatCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `required this.label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `required this.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `required this.color,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `final String value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `final Color color;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `final tone = AppColors.resolveAccent(context, color);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `final isNeu = Neumorphic.isActive(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `return Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `child: PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `padding: const EdgeInsets.all(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `mainAxisAlignment: MainAxisAlignment.spaceBetween,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `letterSpacing: 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `if (isNeu)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `NeuIconWell(icon: icon, color: tone, size: 18, padding: 8)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `else`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `PaperGlyph(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `icon: icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `color: tone.withValues(alpha: 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `style: Theme.of(context).textTheme.headlineMedium?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `color: tone,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L411** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `class _QuickActionCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L421** `const _QuickActionCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L422** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `required this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `required this.icon,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L426** `required this.accentColor,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `final String subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `final IconData icon;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `final Color accentColor;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `final tone = AppColors.resolveAccent(context, accentColor);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `NeuIconWell(icon: icon, color: tone, size: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L467** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `color: AppColors.inkAlpha(context, 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/dashboard/teacher_dashboard_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/homework/teacher_homework_pack_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L15** `class TeacherHomeworkPackScreen extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L16** `const TeacherHomeworkPackScreen({super.key, required this.assignmentId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final String assignmentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `ConsumerState<TeacherHomeworkPackScreen> createState() =>`
Widget that can watch providers. Required for profile/theme/locale.

**L22** `_TeacherHomeworkPackScreenState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `class _TeacherHomeworkPackScreenState`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26** `extends ConsumerState<TeacherHomeworkPackScreen> {`
Widget that can watch providers. Required for profile/theme/locale.

**L27** `Future<void> _saveAsTemplate(Assignment? assignment) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `if (assignment == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L30** `if (teacher == null || !mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `await saveAssignmentAsTemplate(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `assignment: assignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `).showSnackBar(SnackBar(content: Text(l10n.teacherTemplateSaved)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `).showSnackBar(SnackBar(content: Text(l10n.commonCouldNotSave('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `Future<void> _showTaskDialog({TaskItem? task, int currentCount = 0}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final assignment = ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `.read(assignmentDetailProvider(widget.assignmentId))`
Riverpod binding. Name it after the value it exposes.

**L52** `.valueOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `if (assignment?.type == AssignmentType.staticAssignment) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `final isEditing = task != null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final titleController = TextEditingController(text: task?.title ?? '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `final descController = TextEditingController(text: task?.description ?? '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `final linkController = TextEditingController(text: task?.linkUrl ?? '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `final attachmentController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `text: task?.attachmentUrl ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `await showDialog<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `builder: (ctx) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `title: Text(isEditing ? l10n.teacherEditTask : l10n.teacherAddTask),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `content: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `controller: titleController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `labelText: l10n.commonTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `hintText: l10n.teacherTaskTitleHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `controller: descController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `decoration: InputDecoration(labelText: l10n.commonDescription),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `maxLines: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `controller: linkController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `labelText: l10n.teacherReferenceLinkOptional,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `hintText: l10n.teacherLinkHint,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `prefixIcon: const Icon(Icons.link),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `controller: attachmentController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `labelText: l10n.teacherAttachmentUrlOptional,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `hintText: l10n.teacherAttachmentHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `prefixIcon: const Icon(Icons.attachment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `if (isEditing)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final confirm = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `context: ctx,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `builder: (c) => AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `title: Text(l10n.teacherDeleteTask),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `content: Text(l10n.commonCannotUndo),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `onPressed: () => Navigator.pop(c, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `onPressed: () => Navigator.pop(c, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `style: const TextStyle(color: Colors.red),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `if (confirm == true) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `await supabase.from('tasks').delete().eq('id', task.id);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L134** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `ref.invalidate(tasksProvider(widget.assignmentId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L137** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `style: const TextStyle(color: Colors.red),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `onPressed: () => Navigator.pop(ctx),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `ElevatedButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `onPressed: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `if (titleController.text.trim().isEmpty) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `final Map<String, dynamic> data = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `'assignment_id': widget.assignmentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `'title': titleController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `'description': descController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `'link_url': linkController.text.trim().isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `: linkController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `'attachment_url': attachmentController.text.trim().isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `: attachmentController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `if (isEditing) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `await supabase.from('tasks').update(data).eq('id', task.id);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L165** `} else {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `data['sort_order'] = currentCount;`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `await supabase.from('tasks').insert(data);`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L170** `if (ctx.mounted) Navigator.pop(ctx);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `ref.invalidate(tasksProvider(widget.assignmentId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L172** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `child: Text(isEditing ? l10n.commonSave : l10n.teacherAddTask),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `final assignment = ref.watch(assignmentDetailProvider(widget.assignmentId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L183** `final tasks = ref.watch(tasksProvider(widget.assignmentId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L184** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `appBar: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `title: Text(l10n.teacherHomeworkPackTasks),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `tooltip: l10n.teacherSaveAsTemplate,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `onPressed: () => _saveAsTemplate(assignment.valueOrNull),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `icon: const Icon(Icons.bookmark_add_outlined),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `floatingActionButton: assignment.maybeWhen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `data: (a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `if (a == null || a.type != AssignmentType.homeworkPack) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `return null;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `return tasks.maybeWhen(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `data: (items) => FloatingActionButton.extended(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `onPressed: () => _showTaskDialog(currentCount: items.length),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `icon: const Icon(Icons.add),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `label: Text(l10n.teacherAddTask),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `orElse: () => null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `orElse: () => null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `body: assignment.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `data: (a) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `if (a == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `return EmptyState(title: l10n.assignmentNotFound);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `if (a.type != AssignmentType.homeworkPack) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `title: l10n.staticAssignment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `subtitle: l10n.assignmentTypeHintStatic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `return RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `onRefresh: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `ref.invalidate(assignmentDetailProvider(widget.assignmentId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L228** `ref.invalidate(tasksProvider(widget.assignmentId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L229** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `24,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `shellContentBottomPadding(context, hasFab: true, extra: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `color: AppColors.mutedGold,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `borderRadius: BorderRadius.circular(20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `boxShadow: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `BoxShadow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `color: AppColors.mutedGold.withValues(alpha: 0.2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `blurRadius: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `offset: const Offset(0, 10),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `l10n.homeworkPack.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `color: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `fontSize: 10,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `letterSpacing: 1.2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `a.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `style: Theme.of(context).textTheme.headlineSmall`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `if (a.description.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `a.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `color: AppColors.inkAlpha(context, 0.8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `fontSize: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `height: 1.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `const SizedBox(height: 32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `title: l10n.teacherTasksContent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `subtitle: l10n.teacherManagePack,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `tasks.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `data: (items) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `if (items.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `padding: const EdgeInsets.only(top: 24.0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `l10n.teacherNoTasksYet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `fontStyle: FontStyle.italic,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `return ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `physics: const NeverScrollableScrollPhysics(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `itemCount: items.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `itemBuilder: (context, index) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `final t = items[index];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `final hasLink =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `t.linkUrl != null && t.linkUrl!.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `final hasAttachment =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `t.attachmentUrl != null &&`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `t.attachmentUrl!.isNotEmpty;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `onTap: () => _showTaskDialog(task: t),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `padding: const EdgeInsets.all(16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `width: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `height: 32,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `color: AppColors.softWash(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `lightAlpha: 0.5,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `shape: BoxShape.circle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `child: Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `'${index + 1}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `color: AppColors.ink(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L342** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `t.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `fontSize: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `if (t.description.isNotEmpty)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `t.description,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `color: AppColors.inkAlpha(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `0.6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `if (hasLink || hasAttachment)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `padding: const EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `top: 8.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `if (hasLink)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `Icons.link,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `size: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `if (hasLink && hasAttachment)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `if (hasAttachment)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `const Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `Icons.attachment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `size: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `color: AppColors.successGreen,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `Icons.edit_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `size: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `loading: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `const SizedBox(height: 100),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L412** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L414** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L415** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L417** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/teacher/homework/teacher_homework_pack_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/homework/teacher_submissions_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L19** `class TeacherSubmissionsScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L20** `const TeacherSubmissionsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final submissions = ref.watch(teacherSubmissionsProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L25** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `appBar: AppBar(title: Text(l10n.teacherSubmissions)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `body: submissions.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `loading: () => const Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `data: (rows) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `if (rows.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `title: l10n.teacherNoSubmissions,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `subtitle: l10n.teacherNoSubmissionsSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `icon: Icons.fact_check_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `return RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `onRefresh: () async => ref.invalidate(teacherSubmissionsProvider),`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L42** `child: ListView.builder(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `shellContentBottomPadding(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `itemCount: rows.length,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `itemBuilder: (_, i) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `final row = rows[i];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `final score = row['score'] == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `: double.tryParse('${row['score']}');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `onTap: () => _review(context, ref, row),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `padding: const EdgeInsets.symmetric(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `horizontal: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `vertical: 14,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `row['task_title'] as String? ?? l10n.taskTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `const SizedBox(height: 2),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `'${row['student_name']} · ${row['status']}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `PercentBadge(percent: score, compact: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `},`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `Future<void> _review(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `WidgetRef ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `Map<String, dynamic> row,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L103** `if (!context.mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `final saved = await showModalBottomSheet<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `isScrollControlled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `showDragHandle: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `final bottom = MediaQuery.viewInsetsOf(ctx).bottom;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `padding: EdgeInsets.only(bottom: bottom),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `child: _ReviewSubmissionDialog(row: row, teacherId: teacher?.id),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `if (saved != true || !context.mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `ref.invalidate(teacherSubmissionsProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L121** `final groupId = ref.read(activeGroupIdProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L122** `if (groupId != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `ref.invalidate(groupMarksProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L124** `ref.invalidate(groupMarkAveragesProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L129** `class _ReviewSubmissionDialog extends StatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L130** `const _ReviewSubmissionDialog({required this.row, this.teacherId});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `final Map<String, dynamic> row;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `final String? teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `State<_ReviewSubmissionDialog> createState() =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `_ReviewSubmissionDialogState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `class _ReviewSubmissionDialogState extends State<_ReviewSubmissionDialog> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L141** `late final TextEditingController _scoreController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `late final TextEditingController _feedbackController;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `late final FocusNode _scoreFocus;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `bool _saving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `bool _applyToOthers = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `List<String> _snippets = const [];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `void initState() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `super.initState();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `final score = widget.row['score'];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `_scoreController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `text: score == null ? '' : '$score',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `_feedbackController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `text: widget.row['feedback'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `_scoreFocus = FocusNode();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `_loadSnippets();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `WidgetsBinding.instance.addPostFrameCallback((_) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `if (mounted) _scoreFocus.requestFocus();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `Future<void> _loadSnippets() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `final teacherId = widget.teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `if (teacherId == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `final snippets = await loadFeedbackSnippets(teacherId: teacherId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `setState(() => _snippets = snippets);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `_scoreFocus.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `_scoreController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `_feedbackController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `Future<void> _save() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `if (_saving) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `final raw = _scoreController.text.trim().replaceAll(',', '.');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `final score = raw.isEmpty ? null : double.tryParse(raw);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `if (raw.isNotEmpty && (score == null || score < 0 || score > 100)) {`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `).showSnackBar(SnackBar(content: Text(context.l10n.teacherScoreRange)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `setState(() => _saving = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `final feedback = _feedbackController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `await supabase`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `.from('submissions')`
PostgREST table access. RLS still applies. The string is the Postgres table or view name.

**L198** `.update({`
Mutates matching rows. Silent zero rows usually means SELECT policy is missing for UPDATE.

**L199** `'score': score,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `'feedback': feedback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `'status': score == null ? 'submitted' : 'reviewed',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `})`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `.eq('id', widget.row['id']);`
Equality filter. This is the WHERE clause. Wrong UUID type (profile vs student id) yields empty lists.

**L205** `final teacherId = widget.teacherId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `if (teacherId != null && feedback.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `await rememberFeedbackSnippet(teacherId: teacherId, feedback: feedback);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `var applied = 0;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `if (_applyToOthers && feedback.isNotEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `applied = await applyFeedbackToOtherPendingForSameTask(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `submissionId: widget.row['id'] as String,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `feedback: feedback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `if (applied > 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `SnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `content: Text(context.l10n.teacherFeedbackAppliedCount(applied)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `Navigator.of(context).pop(true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `setState(() => _saving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `SnackBar(content: Text(context.l10n.commonCouldNotSave('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `final fileUrl = widget.row['file_url'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `final voiceUrl = widget.row['voice_url'] as String?;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `final scheme = Theme.of(context).colorScheme;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `return SafeArea(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `child: SingleChildScrollView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `padding: const EdgeInsets.fromLTRB(20, 8, 20, 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `widget.row['student_name'] as String? ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `l10n.teacherReviewSubmission,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `widget.row['task_title'] as String? ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `color: AppColors.inkAlpha(context, 0.6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `const SizedBox(height: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `widget.row['text_answer'] as String? ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `l10n.teacherNoWrittenAnswer,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `color: AppColors.inkAlpha(context, 0.75),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `height: 1.4,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `if (voiceUrl != null && voiceUrl.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `VoiceNotePlayer(url: voiceUrl),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `if (fileUrl != null) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L278** `OutlinedButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `onPressed: () => launchUrl(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L280** `Uri.parse(fileUrl),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L281** `mode: LaunchMode.externalApplication,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `icon: const Icon(Icons.attach_file, size: 18),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `label: Text(l10n.teacherOpenAttachment),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `controller: _scoreController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L290** `focusNode: _scoreFocus,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `enabled: !_saving,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L292** `keyboardType: const TextInputType.numberWithOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L293** `decimal: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `textInputAction: TextInputAction.next,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `inputFormatters: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `FilteringTextInputFormatter.allow(RegExp(r'[0-9.,]')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `LengthLimitingTextInputFormatter(6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `labelText: l10n.commonScore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `suffixText: '%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `helperText: l10n.teacherScoreHelper,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `controller: _feedbackController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `enabled: !_saving,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `maxLines: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `decoration: InputDecoration(labelText: l10n.commonFeedback),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L313** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L314** `if (_snippets.isNotEmpty) ...[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `const SizedBox(height: 10),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L316** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L317** `l10n.teacherRecentFeedback,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `fontWeight: FontWeight.w600,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L321** `color: AppColors.inkAlpha(context, 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L322** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L323** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `const SizedBox(height: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L325** `Wrap(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `spacing: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `runSpacing: 6,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `children: _snippets.take(8).map((s) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L329** `final label = s.length > 42 ? '${s.substring(0, 42)}…' : s;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L330** `return ActionChip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `label: Text(label, style: const TextStyle(fontSize: 12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `onPressed: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L334** `: () {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `_feedbackController.text = s;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `_feedbackController.selection =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `TextSelection.collapsed(offset: s.length);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `}).toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `const SizedBox(height: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `CheckboxListTile(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `contentPadding: EdgeInsets.zero,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `dense: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `value: _applyToOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `onChanged: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `: (v) => setState(() => _applyToOthers = v ?? false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `controlAffinity: ListTileControlAffinity.leading,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `title: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `l10n.teacherApplyFeedbackToOthers,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `style: const TextStyle(fontSize: 13),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `child: TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `onPressed: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `: () => Navigator.of(context).pop(false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L368** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `child: FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `onPressed: _saving ? null : _save,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `foregroundColor:`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `? AppColors.darkChocolate`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `padding: const EdgeInsets.symmetric(vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L380** `child: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L382** `width: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `height: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L384** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L385** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L386** `color: scheme.onPrimary,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L387** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L389** `: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L390** `l10n.commonSave,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L393** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/homework/teacher_submissions_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/marks/teacher_marks_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L23** `class TeacherMarksScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L24** `const TeacherMarksScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final groupId = ref.watch(activeGroupIdProvider);`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (groupId == null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `appBar: AppBar(title: Text(l10n.teacherGradebook)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `body: EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `title: l10n.teacherNoGroups,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `subtitle: l10n.teacherNoGroupsSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `icon: Icons.groups_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `final members = ref.watch(groupMembersProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L43** `final averages = ref.watch(groupMarkAveragesProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L44** `final marks = ref.watch(groupMarksProvider(groupId));`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L45** `final markCount =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `ref.watch(groupMarkCountProvider(groupId)).valueOrNull ?? 0;`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L47** `final groups = ref.watch(myGroupsProvider).valueOrNull ?? const [];`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L48** `final groupTitle = groups`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `.where((g) => g.id == groupId)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `.map((g) => g.title)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `.firstOrNull;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `return Scaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `appBar: AppBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `title: Text(l10n.teacherGradebook),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `tooltip: l10n.marksExportCsv,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `onPressed: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `_export(context, ref, groupId, groupTitle ?? 'group'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `icon: const Icon(Icons.ios_share_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `floatingActionButtonLocation: shellFabLocation(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `floatingActionButton: FloatingActionButton.extended(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `onPressed: () => _openBulkEntry(context, ref, groupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `icon: const Icon(Icons.playlist_add_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `label: Text(l10n.teacherGradeClass),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `body: RefreshIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `onRefresh: () async => _refresh(ref, groupId),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `child: ListView(`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `left: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `right: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `top: 8,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `bottom: shellContentBottomPadding(context, hasFab: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `const GroupSelector(padding: EdgeInsets.only(bottom: 8)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `const TermSelector(padding: EdgeInsets.only(bottom: 12)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `_ClassSummaryCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `averages: averages.valueOrNull ?? const {},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `studentCount: members.valueOrNull?.length ?? 0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `markCount: markCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `title: l10n.marksStudentsHeader,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `subtitle: l10n.marksStudentsSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `members.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `data: (list) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `if (list.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `title: l10n.marksNoStudents,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `subtitle: l10n.marksNoStudentsSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `icon: Icons.person_off_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final avg = averages.valueOrNull ?? const <String, double>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `children: list`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `(m) => _StudentRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `member: m,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `average: avg[m.studentId],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `onTap: () =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `_openStudentSheet(context, ref, groupId, m),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `loading: () => const Center(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `child: Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `padding: EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `child: CircularProgressIndicator(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `PremiumSectionHeader(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `title: l10n.marksRecent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `subtitle: l10n.marksRecentSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `marks.when(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `data: (page) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `if (page.items.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `return EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `title: l10n.marksNoRecorded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `subtitle: l10n.marksNoRecordedSubtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `return Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `...page.items.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `(mark) => MarkRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `mark: mark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `onEdit: () => _openMarkEditor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `studentId: mark.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `studentName: mark.studentName ?? l10n.commonStudent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `existing: mark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `onDelete: () => _confirmDelete(context, ref, mark),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `LoadMoreButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `hasMore: page.hasMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `loading: page.loadingMore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `onPressed: () => ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `.read(groupMarksProvider(groupId).notifier)`
Riverpod binding. Name it after the value it exposes.

**L158** `.loadMore(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `loading: () => const SizedBox.shrink(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `error: (e, _) => ErrorState(message: e.toString()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `void _refresh(WidgetRef ref, String groupId) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `ref.invalidate(groupMembersProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L174** `ref.invalidate(groupMarkAveragesProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L175** `ref.invalidate(groupMarkCountProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L176** `ref.invalidate(groupMarksProvider(groupId));`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L177** `ref.invalidate(memberMarksProvider);`
Drop cached AsyncValue so the next watch refetches (after login or a mutation).

**L180** `Future<void> _export(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `WidgetRef ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `String groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `await exportGroupMarksCsv(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `term: ref.read(selectedMarkTermProvider),`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L193** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `).showSnackBar(SnackBar(content: Text(l10n.marksExportDone)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `).showSnackBar(SnackBar(content: Text(l10n.commonExportFailed('$e'))));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `Future<void> _openStudentSheet(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `WidgetRef ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `GroupMember member,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `final result = await showModalBottomSheet<Object?>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `isScrollControlled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `builder: (_) => StudentMarksSheet(groupId: groupId, member: member),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `if (context.mounted && result != null) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L224** `await _openMarkEditor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `studentId: member.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `studentName: member.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `existing: result is Mark ? result : null,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `_refresh(ref, groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `Future<void> _openBulkEntry(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L238** `WidgetRef ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `final members = await ref.read(groupMembersProvider(groupId).future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L242** `if (!context.mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `if (members.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `SnackBar(content: Text(context.l10n.marksGroupHasNoStudents)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `await showModalBottomSheet<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `isScrollControlled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L252** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L253** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `builder: (_) => BulkMarkSheet(groupId: groupId, members: members),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `_refresh(ref, groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `Future<void> _openMarkEditor(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L260** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L261** `WidgetRef ref, {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L262** `required String groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L263** `required String studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L264** `required String studentName,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L265** `Mark? existing,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L266** `}) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L267** `await showModalBottomSheet<void>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L268** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L269** `isScrollControlled: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L270** `useRootNavigator: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L271** `backgroundColor: Colors.transparent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L272** `builder: (_) => MarkEditorSheet(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L273** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L274** `studentId: studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L275** `studentName: studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L276** `existing: existing,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L277** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L279** `_refresh(ref, groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L282** `Future<void> _confirmDelete(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L283** `BuildContext context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L284** `WidgetRef ref,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L285** `Mark mark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L286** `) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L287** `if (mark.isFromSubmission) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L288** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L289** `SnackBar(content: Text(context.l10n.marksFromSubmissionDelete)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L291** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L294** `final confirmed = await showDialog<bool>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L295** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L296** `builder: (ctx) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L297** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L298** `return AlertDialog(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L299** `title: Text(l10n.marksDeleteTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L300** `content: Text(l10n.marksDeleteBody(mark.title)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L301** `actions: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L302** `TextButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L303** `onPressed: () => Navigator.pop(ctx, false),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L304** `child: Text(l10n.commonCancel),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L305** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L306** `FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L307** `style: FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L308** `backgroundColor: AppColors.errorRed,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L309** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L310** `onPressed: () => Navigator.pop(ctx, true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L311** `child: Text(l10n.commonDelete),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L312** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L315** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L318** `if (confirmed != true) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L319** `await deleteMark(mark.id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L320** `_refresh(ref, mark.groupId);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L324** `class _ClassSummaryCard extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L325** `const _ClassSummaryCard({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L326** `required this.averages,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L327** `required this.studentCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L328** `required this.markCount,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L331** `final Map<String, double> averages;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L332** `final int studentCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L333** `final int markCount;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L335** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L336** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L337** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L338** `final values = averages.values.toList();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L339** `final classAverage = values.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L340** `? null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L341** `: values.reduce((a, b) => a + b) / values.length;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L343** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L344** `padding: const EdgeInsets.all(24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L345** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L346** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L347** `Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L348** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L349** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L350** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L351** `l10n.marksOverallAverage.toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L352** `style: Theme.of(context).textTheme.labelSmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L353** `letterSpacing: 1.0,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L354** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L355** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L356** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L357** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L358** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L359** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L360** `formatPercent(classAverage),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L361** `style: Theme.of(context).textTheme.displaySmall?.copyWith(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L362** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L363** `color: classAverage == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L364** `? AppColors.inkAlpha(context, 0.3)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L365** `: percentColor(classAverage, context: context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L366** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L367** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L369** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L370** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L371** `Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L372** `crossAxisAlignment: CrossAxisAlignment.end,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L373** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L374** `_SummaryStat(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L375** `label: l10n.marksSummaryStudents,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L376** `value: '$studentCount',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L377** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L378** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L379** `_SummaryStat(label: l10n.marksSummaryMarks, value: '$markCount'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L381** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L383** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L388** `class _SummaryStat extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L389** `const _SummaryStat({required this.label, required this.value});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L391** `final String label;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L392** `final String value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L394** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L395** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L396** `return Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L397** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L398** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L399** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L400** `label,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L401** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L402** `fontSize: 13,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L403** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L404** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L405** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L406** `const SizedBox(width: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L407** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L408** `value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L409** `style: const TextStyle(fontWeight: FontWeight.w800, fontSize: 15),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L410** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L416** `class _StudentRow extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L417** `const _StudentRow({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L418** `required this.member,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L419** `required this.average,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L420** `required this.onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L423** `final GroupMember member;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L424** `final double? average;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L425** `final VoidCallback onTap;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L427** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L428** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L429** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L430** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L431** `onTap: onTap,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L432** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L433** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L434** `children: [`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L435** `CircleAvatar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L436** `radius: 18,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L437** `backgroundColor: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L438** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L439** `member.fullName.isEmpty ? '?' : member.fullName[0].toUpperCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L440** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L441** `color: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L442** `fontWeight: FontWeight.w800,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L443** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L444** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L445** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L446** `const SizedBox(width: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L447** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L448** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L449** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L450** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L451** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L452** `member.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L453** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L454** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L455** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L456** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L457** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L458** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L459** `average == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L460** `? l10n.marksNoMarksYetShort`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L461** `: percentLabel(average ?? 0, l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L462** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L463** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L464** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L465** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L466** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L468** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L469** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L470** `PercentBadge(percent: average),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L471** `const SizedBox(width: 4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L472** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L473** `Icons.chevron_right_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L474** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L475** `color: AppColors.inkAlpha(context, 0.25),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L476** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L478** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/screens/teacher/marks/teacher_marks_screen.dart**
Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L11** `class BulkMarkSheet extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L12** `const BulkMarkSheet({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.members,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final List<GroupMember> members;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `ConsumerState<BulkMarkSheet> createState() => _BulkMarkSheetState();`
Widget that can watch providers. Required for profile/theme/locale.

**L25** `class _BulkMarkSheetState extends ConsumerState<BulkMarkSheet> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L26** `final _titleController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final _weightController = TextEditingController(text: '1');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final _commentController = TextEditingController();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `late final Map<String, TextEditingController> _percentControllers = {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `for (final m in widget.members) m.studentId: TextEditingController(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `MarkCategory _category = MarkCategory.test;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `DateTime _date = DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `bool _saving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `_titleController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `_weightController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `_commentController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `for (final c in _percentControllers.values) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `c.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `Future<void> _save() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `final title = _titleController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `if (title.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `_toast(l10n.marksNeedTitle);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `final weight = double.tryParse(_weightController.text.trim()) ?? 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `if (weight <= 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `_toast(l10n.marksWeightPositive);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final entries = <String, double>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `for (final entry in _percentControllers.entries) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `final raw = entry.value.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `if (raw.isEmpty) continue;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `final value = double.tryParse(raw);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `if (value == null || value < 0 || value > 100) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `_toast(l10n.teacherScoreRange);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `entries[entry.key] = value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `if (entries.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `_toast(l10n.marksNeedAtLeastOne);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L80** `if (teacher == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `final comment = _commentController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `setState(() => _saving = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `for (final entry in entries.entries) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `await saveMark(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `groupId: widget.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `studentId: entry.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `title: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `category: _category,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `percentage: entry.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `weight: weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `comment: comment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `markedOn: _date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `if (mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `Navigator.pop(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `SnackBar(content: Text(l10n.marksSavedCount(entries.length))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `_toast(l10n.commonCouldNotSave('$e'));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `if (mounted) setState(() => _saving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `void _toast(String message) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `).showSnackBar(SnackBar(content: Text(message)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `return GradebookSheetScaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `title: l10n.marksGradeTheClass,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `subtitle: l10n.marksOnePercentEach,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `action: FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `onPressed: _saving ? null : _save,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `style: gradebookPrimaryButtonStyle(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `child: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `height: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `color: Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `? AppColors.darkChocolate`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `l10n.marksSaveMarks,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `padding: const EdgeInsets.symmetric(horizontal: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `controller: _titleController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `labelText: l10n.commonTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `hintText: l10n.marksTitleHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `controller: _commentController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `maxLines: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `labelText: l10n.teacherSharedComment,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `hintText: l10n.teacherSharedCommentHint,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `child: GradebookCategoryDropdown(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `value: _category,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `onChanged: (v) => setState(() => _category = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L172** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `width: 90,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `child: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `controller: _weightController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `keyboardType: const TextInputType.numberWithOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `decimal: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `decoration: InputDecoration(labelText: l10n.marksWeight),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `GradebookDateField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `value: _date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L190** `onChanged: (v) => setState(() => _date = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `const SizedBox(height: 24),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `l10n.marksLeaveBlankSkip,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `...widget.members.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `(m) => Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `padding: const EdgeInsets.symmetric(vertical: 6),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `m.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `style: const TextStyle(fontWeight: FontWeight.w600),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `width: 96,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `child: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `controller: _percentControllers[m.studentId],`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `textAlign: TextAlign.center,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `keyboardType: const TextInputType.numberWithOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `decimal: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `inputFormatters: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `FilteringTextInputFormatter.allow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `RegExp(r'^\d{0,3}(\.\d{0,2})?'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `decoration: const InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L226** `hintText: '—',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L227** `suffixText: '%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `isDense: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L234** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/marks/widgets/bulk_mark_sheet.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/marks/widgets/gradebook_helpers.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L9** `ButtonStyle gradebookPrimaryButtonStyle(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `final isDark = Theme.of(context).brightness == Brightness.dark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L11** `final fg = isDark ? AppColors.darkChocolate : Colors.white;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `return FilledButton.styleFrom(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `backgroundColor: AppColors.accent(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `foregroundColor: fg,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `disabledBackgroundColor: AppColors.accent(context).withValues(alpha: 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `disabledForegroundColor: fg.withValues(alpha: 0.55),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `padding: const EdgeInsets.symmetric(vertical: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `class GradebookSheetScaffold extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L22** `const GradebookSheetScaffold({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `required this.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `required this.subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `required this.child,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `required this.action,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final String title;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `final String subtitle;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final Widget child;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `final Widget action;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `return Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `padding: EdgeInsets.only(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `bottom: MediaQuery.of(context).viewInsets.bottom,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `child: Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `constraints: BoxConstraints(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `maxHeight: MediaQuery.sizeOf(context).height * 0.9,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `color: AppColors.sheetSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `borderRadius: const BorderRadius.vertical(top: Radius.circular(24)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `mainAxisSize: MainAxisSize.min,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `const SizedBox(height: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `Container(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `width: 40,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `height: 4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `decoration: BoxDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `color: AppColors.softSurface(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `borderRadius: BorderRadius.circular(2),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `padding: const EdgeInsets.fromLTRB(20, 12, 20, 0),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `child: PremiumSectionHeader(title: title, subtitle: subtitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `Flexible(child: child),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `padding: EdgeInsets.fromLTRB(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `MediaQuery.of(context).padding.bottom + 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `child: SizedBox(width: double.infinity, child: action),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `class GradebookCategoryDropdown extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L83** `const GradebookCategoryDropdown({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `required this.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `required this.onChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `final MarkCategory value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `final ValueChanged<MarkCategory> onChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `return DropdownButtonFormField<MarkCategory>(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `initialValue: value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `decoration: InputDecoration(labelText: l10n.marksCategory),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `items: MarkCategory.values`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `(c) => DropdownMenuItem(value: c, child: Text(c.l10nLabel(l10n))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L102** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `onChanged: (v) => onChanged(v ?? MarkCategory.other),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `class GradebookDateField extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L109** `const GradebookDateField({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `required this.value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `required this.onChanged,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `final DateTime value;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `final ValueChanged<DateTime> onChanged;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `return InkWell(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `onTap: () async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `final picked = await showDatePicker(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `context: context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `initialDate: value,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `firstDate: DateTime(2020),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `lastDate: DateTime.now().add(const Duration(days: 365)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `if (picked != null) onChanged(picked);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `borderRadius: BorderRadius.circular(12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `child: InputDecorator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `decoration: InputDecoration(labelText: context.l10n.marksDate),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `Text(DateFormat.yMMMd().format(value)),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `const Spacer(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `Icons.calendar_today_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `color: AppColors.inkAlpha(context, 0.4),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** ),
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/marks/widgets/gradebook_helpers.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/marks/widgets/mark_editor_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L12** `class MarkEditorSheet extends ConsumerStatefulWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L13** `const MarkEditorSheet({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `required this.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L17** `required this.studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `this.existing,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final String studentId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final String studentName;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final Mark? existing;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `ConsumerState<MarkEditorSheet> createState() => _MarkEditorSheetState();`
Widget that can watch providers. Required for profile/theme/locale.

**L30** `class _MarkEditorSheetState extends ConsumerState<MarkEditorSheet> {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L31** `late final _titleController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `text: widget.existing?.title ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `late final _percentController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `text: widget.existing == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `? ''`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `: formatPercent(widget.existing!.percentage).replaceAll('%', ''),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `late final _weightController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `text: '${widget.existing?.weight ?? 1}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `late final _commentController = TextEditingController(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `text: widget.existing?.comment ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `late MarkCategory _category = widget.existing?.category ?? MarkCategory.other;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `late DateTime _date = widget.existing?.markedOn ?? DateTime.now();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `bool _saving = false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `void dispose() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `_titleController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `_percentController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `_weightController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `_commentController.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `super.dispose();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `Future<void> _save() async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `final title = _titleController.text.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `final percent = double.tryParse(_percentController.text.trim());`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `final weight = double.tryParse(_weightController.text.trim()) ?? 1;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `if (title.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `_toast(l10n.marksNeedTitle);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `if (percent == null || percent < 0 || percent > 100) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `_toast(l10n.teacherScoreRange);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `if (weight <= 0) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `_toast(l10n.marksWeightPositive);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `final teacher = await ref.read(teacherProfileProvider.future);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L79** `if (teacher == null) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `setState(() => _saving = true);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `await saveMark(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `groupId: widget.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `studentId: widget.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `teacherId: teacher.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `title: title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `category: _category,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L89** `percentage: percent,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `weight: weight,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `comment: _commentController.text.trim(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `markedOn: _date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `markId: widget.existing?.id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `if (mounted) Navigator.pop(context);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `_toast(l10n.commonCouldNotSave('$e'));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `} finally {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `if (mounted) setState(() => _saving = false);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L103** `void _toast(String message) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `if (!mounted) return;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L106** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `).showSnackBar(SnackBar(content: Text(message)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `final linked = widget.existing?.isFromSubmission ?? false;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `return GradebookSheetScaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `title: widget.existing == null ? l10n.marksNewMark : l10n.marksEditMark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `subtitle: widget.studentName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `action: FilledButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `onPressed: _saving ? null : _save,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `style: gradebookPrimaryButtonStyle(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `child: _saving`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `? SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `width: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `height: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `child: CircularProgressIndicator(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `strokeWidth: 2,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `color: Theme.of(context).brightness == Brightness.dark`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `? AppColors.darkChocolate`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `: Colors.white,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `l10n.marksSaveMark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `child: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L138** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `padding: const EdgeInsets.symmetric(horizontal: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L140** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L141** `if (linked)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `padding: const EdgeInsets.only(bottom: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L144** `child: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `l10n.marksFromSubmissionEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `color: AppColors.warmAmber,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `height: 1.4,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L151** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L152** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `controller: _titleController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L155** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `decoration: InputDecoration(labelText: l10n.commonTitle),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L161** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `width: 110,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `child: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `controller: _percentController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `keyboardType: const TextInputType.numberWithOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `decimal: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `inputFormatters: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `FilteringTextInputFormatter.allow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `RegExp(r'^\d{0,3}(\.\d{0,2})?'),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L173** `decoration: InputDecoration(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `labelText: l10n.commonScore,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `suffixText: '%',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L179** `const SizedBox(width: 12),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `SizedBox(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `width: 90,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `child: TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `controller: _weightController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `keyboardType: const TextInputType.numberWithOptions(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `decimal: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `decoration: InputDecoration(labelText: l10n.marksWeight),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L188** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `GradebookCategoryDropdown(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `value: _category,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `onChanged: (v) => setState(() => _category = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `GradebookDateField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L199** `value: _date,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L200** `onChanged: (v) => setState(() => _date = v),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L201** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `const SizedBox(height: 16),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `TextField(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `controller: _commentController,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `maxLines: 3,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `textCapitalization: TextCapitalization.sentences,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L207** `decoration: InputDecoration(labelText: l10n.marksCommentOptional),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L208** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/marks/widgets/mark_editor_sheet.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/marks/widgets/mark_row.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L10** `class MarkRow extends StatelessWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L11** `const MarkRow({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `required this.mark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `required this.onEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `this.onDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `this.showStudent = true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final Mark mark;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `final VoidCallback onEdit;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `final VoidCallback? onDelete;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `final bool showStudent;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `Widget build(BuildContext context) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `final subtitle = [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `mark.category.l10nLabel(l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L29** `if (showStudent && mark.studentName != null) mark.studentName!,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `DateFormat.MMMd().format(mark.markedOn),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L31** `if (mark.weight != 1)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `'×${mark.weight.toStringAsFixed(mark.weight.truncateToDouble() == mark.weight ? 0 : 1)}',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `].join(' · ');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `return PremiumCard(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L36** `onTap: onEdit,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `child: Row(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `PercentBadge(percent: mark.percentage, compact: true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `const SizedBox(width: 14),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `Expanded(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `child: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `crossAxisAlignment: CrossAxisAlignment.start,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `mark.title,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `maxLines: 1,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `style: const TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `fontWeight: FontWeight.w700,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `fontSize: 15,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `subtitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `maxLines: 1,`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `overflow: TextOverflow.ellipsis,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `style: TextStyle(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `fontSize: 12,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `color: AppColors.inkAlpha(context, 0.5),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `if (mark.isFromSubmission)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `Tooltip(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `message: l10n.marksFromSubmission,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `child: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `Icons.link_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `size: 16,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `color: AppColors.inkAlpha(context, 0.3),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `if (onDelete != null)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `IconButton(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `tooltip: context.l10n.commonDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `icon: Icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `Icons.delete_outline_rounded,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `size: 20,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `color: AppColors.inkAlpha(context, 0.35),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `onPressed: onDelete,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/marks/widgets/mark_row.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/marks/widgets/student_marks_sheet.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L14** `const studentMarksSheetAdd = 'add';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `class StudentMarksSheet extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L17** `const StudentMarksSheet({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L18** `super.key,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `required this.groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `required this.member,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L23** `final String groupId;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L24** `final GroupMember member;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L28** `final marks = ref.watch(`
Rebuild this widget/provider when the dependency changes. Do not put watch in a click handler.

**L29** `memberMarksProvider((groupId: groupId, studentId: member.studentId)),`
Riverpod binding. Name it after the value it exposes.

**L31** `final mine = marks.valueOrNull ?? const <Mark>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `final average = ref`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `.watch(groupMarkAveragesProvider(groupId))`
Riverpod binding. Name it after the value it exposes.

**L34** `.valueOrNull?[member.studentId];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `return GradebookSheetScaffold(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `title: member.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `subtitle: average == null`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `? l10n.marksNoMarksYetShort`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L41** `: l10n.marksAverageLine(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `formatPercent(average),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `percentLabel(average, l10n),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `action: Column(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L46** `crossAxisAlignment: CrossAxisAlignment.stretch,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L47** `children: [`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `OutlinedButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `onPressed: () => _shareProgress(context, ref),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L50** `icon: const Icon(Icons.ios_share_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `label: Text(l10n.progressReportShare),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L52** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `const SizedBox(height: 8),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L54** `FilledButton.icon(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `style: gradebookPrimaryButtonStyle(context),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `onPressed: () => Navigator.pop(context, studentMarksSheetAdd),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `icon: const Icon(Icons.add_rounded),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `label: Text(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `l10n.marksAddMark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L60** `style: const TextStyle(fontWeight: FontWeight.w700),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L64** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `child: marks.isLoading`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `? const Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L67** `padding: EdgeInsets.all(32),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L68** `child: Center(child: CircularProgressIndicator()),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `: mine.isEmpty`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `? Padding(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `padding: const EdgeInsets.symmetric(horizontal: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L73** `child: EmptyState(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `title: l10n.marksNothingYet,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L75** `subtitle: l10n.marksAddFirst,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L76** `icon: Icons.grade_outlined,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `: ListView(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L80** `shrinkWrap: true,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L81** `padding: const EdgeInsets.symmetric(horizontal: 20),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `children: mine`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `.map(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `(mark) => MarkRow(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `mark: mark,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L86** `showStudent: false,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L87** `onEdit: () => Navigator.pop(context, mark),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `),`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `.toList(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `Future<void> _shareProgress(BuildContext context, WidgetRef ref) async {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `final l10n = context.l10n;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final groups = ref.read(myGroupsProvider).valueOrNull ?? const [];`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L98** `final groupTitle =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `groups.where((g) => g.id == groupId).map((g) => g.title).firstOrNull ??`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `'group';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `final term = ref.read(selectedMarkTermProvider);`
One-shot access, typical in onPressed. Login uses read(authRepositoryProvider).

**L103** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `).showSnackBar(SnackBar(content: Text(l10n.progressReportSharing)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `try {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `await shareProgressReport(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `studentId: member.studentId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `groupId: groupId,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `studentName: member.fullName,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `groupTitle: groupTitle,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `term: term,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `l10n: l10n,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `ScaffoldMessenger.of(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `context,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `).showSnackBar(SnackBar(content: Text(l10n.progressReportDone)));`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `} catch (e) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `if (context.mounted) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L123** `ScaffoldMessenger.of(context).showSnackBar(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `SnackBar(content: Text(l10n.progressReportFailed('$e'))),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/marks/widgets/student_marks_sheet.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/notifications/teacher_notifications_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class TeacherNotificationsScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const TeacherNotificationsScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `return const StudentNotificationsScreen();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/notifications/teacher_notifications_screen.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/screens/teacher/profile/teacher_profile_screen.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L5** `class TeacherProfileScreen extends ConsumerWidget {`
Type definition. If this is a widget, find build(). If this is a repository, find the supabase calls.

**L6** `const TeacherProfileScreen({super.key});`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `@override`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `Widget build(BuildContext context, WidgetRef ref) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `return const AppProfileScreen();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/screens/teacher/profile/teacher_profile_screen.dart**
> Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

## Every line — lib/supabase/supabase_client.dart

Thin accessor: supabase = Supabase.instance.client. This file (supabase client) is one concrete piece of that layer.

**L3** `SupabaseClient get supabase => Supabase.instance.client;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

> **CHECKPOINT — Closed lib/supabase/supabase_client.dart**
> Name the file's public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every line — lib/tools/generate_school_sql.dart

Application source used by MU Study. Read the imports first, then the public types, then the widget build or provider body.

**L4** `final teachers = <Map<String, String>>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L6** `'first_name': 'James',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L7** `'last_name': 'English',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L8** `'username': '@james',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L9** `'sphere': 'english',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L10** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L12** `'first_name': 'Malika',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L13** `'last_name': 'Math',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L14** `'username': '@malika',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L15** `'sphere': 'math',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L16** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L19** `final students = <Map<String, String>>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L20** `{'first_name': 'John', 'last_name': 'Jones', 'username': '@john_jones'},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L21** `{'first_name': 'Someone', 'last_name': 'Someone', 'username': '@someone'},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L22** `{'first_name': 'Amina', 'last_name': 'Karimova', 'username': '@amina01'},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L25** `final groups = <Map<String, String>>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L26** `{'teacher_username': '@james', 'title': 'English Group C'},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L27** `{'teacher_username': '@malika', 'title': 'Math Group A'},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L30** `final assignments = <Map<String, String>>[`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L32** `'teacher_username': '@james',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L33** `'group_title': 'English Group C',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L34** `'student_username': '@john_jones',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L35** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L37** `'teacher_username': '@james',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L38** `'group_title': 'English Group C',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L39** `'student_username': '@someone',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L40** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L42** `'teacher_username': '@malika',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L43** `'group_title': 'Math Group A',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L44** `'student_username': '@amina01',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L45** `},`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L48** `final usedPasswords = <String>{};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L49** `final rng = Random.secure();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L51** `String sqlEscape(String value) => value.replaceAll("'", "''");`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L53** `String sqlString(String value) => "'${sqlEscape(value)}'";`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L55** `String generatePassword() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L56** `const alphabet =`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L57** `'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-_';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L58** `String next() =>`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L59** `List.generate(20, (_) => alphabet[rng.nextInt(alphabet.length)]).join();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L61** `var password = next();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L62** `while (usedPasswords.contains(password)) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L63** `password = next();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L65** `usedPasswords.add(password);`

Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L66** `return password;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L69** `String normalizeUsername(String raw) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L70** `final trimmed = raw.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L71** `if (trimmed.isEmpty) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L72** `throw ArgumentError('username is required');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L74** `return trimmed.startsWith('@') ? trimmed : '@$trimmed';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L77** `String authEmail(String username) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L78** `final local = username.replaceAll('@', '').toLowerCase().trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L79** `return '$local@mustudy.internal';`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L82** `String fullName(Map<String, String> user) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L83** `final first = (user['first_name'] ?? '').trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L84** `final last = (user['last_name'] ?? '').trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L85** `return '$first $last'.trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L88** `final printedCredentials = <String>[];`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L90** `void printCreateUser({`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L91** `required Map<String, String> user,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L92** `required String role,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L93** `String? sphere,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L94** `}) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L95** `final username = normalizeUsername(user['username'] ?? '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L96** `final name = fullName(user);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L97** `final password = generatePassword();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L98** `final email = authEmail(username);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L99** `final sphereSql = sphere == null ? 'NULL' : sqlString(sphere);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L100** `printedCredentials.add(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L101** `'$username|$password|$name|$role',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L104** `stdout.writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L105** `'-- $username / $password ($role${sphere == null ? '' : ', $sphere'})',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L107** `stdout.writeln('''`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L108** `DO \$\$`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L109** `DECLARE`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L110** `v_user_id UUID := gen_random_uuid();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L111** `v_email TEXT := ${sqlString(email)};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L112** `v_username TEXT := ${sqlString(username)};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L113** `v_full_name TEXT := ${sqlString(name)};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L114** `v_password TEXT := ${sqlString(password)};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L115** `v_role public.user_role := ${sqlString(role)}::public.user_role;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L116** `v_sphere public.sphere_type := ${sphere == null ? 'NULL' : "$sphereSql::public.sphere_type"};`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L117** `v_teacher_id UUID;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L118** `BEGIN`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L119** `IF EXISTS (SELECT 1 FROM public.profiles WHERE username = v_username) THEN`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L120** `RAISE NOTICE 'skip existing user %', v_username;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L121** `RETURN;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L122** `END IF;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L124** `INSERT INTO auth.users (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L125** `instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L126** `raw_app_meta_data, raw_user_meta_data, created_at, updated_at,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L127** `confirmation_token, recovery_token, email_change_token_new, email_change,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L128** `is_sso_user, is_anonymous`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L129** `) VALUES (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L130** `'00000000-0000-0000-0000-000000000000',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L131** `v_user_id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L132** `'authenticated',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L133** `'authenticated',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L134** `v_email,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L135** `extensions.crypt(v_password, extensions.gen_salt('bf')),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L136** `NOW(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L137** `jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),`
Riverpod binding. Name it after the value it exposes.

**L138** `jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role', v_role::text),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L139** `NOW(), NOW(), '', '', '', '', false, false`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L142** `INSERT INTO auth.identities (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L143** `id, user_id, identity_data, provider, provider_id, last_sign_in_at, created_at, updated_at`
Riverpod binding. Name it after the value it exposes.

**L144** `) VALUES (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L145** `gen_random_uuid(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L146** `v_user_id,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L147** `jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L148** `'email',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L149** `v_email,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L150** `NOW(), NOW(), NOW()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L153** `INSERT INTO public.profiles (id, username, full_name, role, auth_email)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L154** `VALUES (v_user_id, v_username, v_full_name, v_role, v_email);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L156** `INSERT INTO public.login_credentials (profile_id, password, updated_at)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L157** `VALUES (v_user_id, v_password, NOW())`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L158** `ON CONFLICT (profile_id) DO UPDATE`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L159** `SET password = excluded.password,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L160** `updated_at = NOW();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L162** `IF v_role = 'teacher' THEN`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L163** `INSERT INTO public.teachers (profile_id, sphere)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L164** `VALUES (v_user_id, v_sphere)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L165** `RETURNING id INTO v_teacher_id;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L166** `ELSIF v_role = 'student' THEN`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L167** `INSERT INTO public.students (profile_id) VALUES (v_user_id);`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L168** `END IF;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L169** `END`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L170** `\$\$;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L171** `''');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L174** `void main() {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L175** `stdout.writeln('-- Generated by lib/tools/generate_school_sql.dart');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L176** `stdout.writeln('-- Copy everything below into Supabase SQL Editor and run.');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L177** `stdout.writeln(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L178** `'-- Login passwords are in the comments above each user block.',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L180** `stdout.writeln();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L181** `stdout.writeln('BEGIN;');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L182** `stdout.writeln();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L183** `stdout.writeln('''`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L184** `CREATE TABLE IF NOT EXISTS public.login_credentials (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L185** `profile_id uuid PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L186** `password text NOT NULL,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L187** `updated_at timestamptz NOT NULL DEFAULT now()`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L189** `''');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L191** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L192** `stdout.writeln('-- Teachers');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L193** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L194** `for (final teacher in teachers) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L195** `printCreateUser(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L196** `user: teacher,`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L197** `role: 'teacher',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L198** `sphere: (teacher['sphere'] ?? 'math').toLowerCase(),`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L202** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L203** `stdout.writeln('-- Students');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L204** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L205** `for (final student in students) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L206** `printCreateUser(user: student, role: 'student');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L209** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L210** `stdout.writeln('-- Groups');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L211** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L212** `for (final group in groups) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L213** `final teacherUsername = normalizeUsername(group['teacher_username'] ?? '');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L214** `final title = (group['title'] ?? '').trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L215** `stdout.writeln('''`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L216** `INSERT INTO public.groups (teacher_id, title)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L217** `SELECT t.id, ${sqlString(title)}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L218** `FROM public.teachers t`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L219** `JOIN public.profiles p ON p.id = t.profile_id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L220** `WHERE p.username = ${sqlString(teacherUsername)}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L221** `AND NOT EXISTS (`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L222** `SELECT 1 FROM public.groups g`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L223** `WHERE g.teacher_id = t.id AND g.title = ${sqlString(title)}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L225** `''');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L228** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L229** `stdout.writeln('-- Assign students to groups');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L230** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L231** `for (final assignment in assignments) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L232** `final teacherUsername = normalizeUsername(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L233** `assignment['teacher_username'] ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L235** `final groupTitle = (assignment['group_title'] ?? '').trim();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L236** `final studentUsername = normalizeUsername(`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L237** `assignment['student_username'] ?? '',`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L239** `stdout.writeln('''`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L240** `INSERT INTO public.group_students (group_id, student_id)`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L241** `SELECT g.id, s.id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L242** `FROM public.groups g`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L243** `JOIN public.teachers t ON t.id = g.teacher_id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L244** `JOIN public.profiles tp ON tp.id = t.profile_id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L245** `JOIN public.students s ON TRUE`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L246** `JOIN public.profiles sp ON sp.id = s.profile_id`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L247** `WHERE tp.username = ${sqlString(teacherUsername)}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L248** `AND g.title = ${sqlString(groupTitle)}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L249** `AND sp.username = ${sqlString(studentUsername)}`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L250** `ON CONFLICT (group_id, student_id) DO NOTHING;`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L251** `''');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L254** `stdout.writeln('COMMIT;');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L255** `stdout.writeln();`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L256** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L257** `stdout.writeln('-- ========================');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L258** `for (final line in printedCredentials) {`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**L259** `stdout.writeln('$line');`
Product logic on this line. Read the surrounding ten lines if the token is unfamiliar.

**CHECKPOINT — Closed lib/tools/generate_school_sql.dart**
Name the file’s public API in one breath. If you cannot, reread the class/provider table in the annotated chapter.

# Every localization key (EN / RU / UZ)

Each ARB key is a user-visible contract. If English exists and Uzbek does not, generation fails or falls back. Walk the keys and tick any screen you have not opened.

Key count in app_en.arb: 592.

| Key | EN | RU | UZ |
|---|---|---|---|
| appName | MU Study | MU Study | MU Study |
| shellMuStudy | MU Study | MU Study | MU Study |
| languageTitle | Choose your language | ■■■■■■■■■ ■■■■■ | Tilni tanlang |
| languageSubtitle | You can change this anytime in Profile. | ■■ ■■■■■■■ ■■■■■■■■■ ■■■ ■ ■■■■■■ ■■■■■■ ■ ■■■■■■■■. | Buni istalgan vaqtda Profil bo‘limida o‘zgartirish |
| languageEnglish | English | ■■■■■■■■■■■ | Inglizcha |
| languageRussian | Russian | ■■■■■■■■ | Ruscha |
| languageUzbek | Uzbek | ■■■■■■■■■■ | O‘zbekcha |
| languageEnglishNative | English | English | English |
| languageRussianNative | ■■■■■■■■ | ■■■■■■■■ | ■■■■■■■■ |
| languageUzbekNative | O‘zbekcha | O‘zbekcha | O‘zbekcha |
| languageContinue | Continue | ■■■■■■■■■■■ | Davom etish |
| languageSection | Language | ■■■■■ | Til |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| languageHint | App language | ■■■■■ ■■■■■■■■■■■ | Ilova tili |
| loginWelcome | Welcome back | ■ ■■■■■■■■■■■■■ | Xush kelibsiz |
| loginSubtitle | Sign in to your MU Study workspace | ■■■■■■■■ ■ ■■■■■■■■ ■■■■■■■■■■■■■ MU Study | MU Study ish maydoniga kiring |
| loginUsername | Username | ■■■■ ■■■■■■■■■■■■■ | Foydalanuvchi nomi |
| loginPassword | Password | ■■■■■■■ | Parol |
| loginSignIn | Sign in | ■■■■■■ | Kirish |
| loginMissingFields | Please enter both username and password | ■■■■■■■■ ■■■■ ■■■■■■■■■■■■■ ■ ■■■■■■■ | Foydalanuvchi nomi va parolni kiriting |
| loginIncorrect | Incorrect username or password | ■■■■■■■■■ ■■■■ ■■■■■■■■■■■■■ ■■■■ ■■■■■■■ | Foydalanuvchi nomi yoki parol noto‘g‘ri |
| loginFailed | Sign in failed. Check your connection and try again. | ■■ ■■■■■■■■ ■■■■■■. ■■■■■■■■■■ ■■■■■■■■■■■ ■ ■■■■■■■■■■ | Kirib bo‘lmadi. Ulanishni tekshirib, qayta urinib |
| loginConfigError | Invalid configuration. Contact support. | ■■■■■■■ ■■■■■■■■■■■■■■. ■■■■■■■■■■■ ■ ■■■■■■■■■■. | Noto‘g‘ri sozlama. Qo‘llab-quvvatlashga murojaat q |

| Key | EN | RU | UZ |
|---|---|---|---|
| loginConfigMissing | Configuration missing. Check your .env file. | ■■■■■■■■■■■■ ■■■■■■■■■■■■. ■■■■■■■■■■ ■■■■■ .env. | Sozlama yo‘q. .env faylini tekshiring. |
| accountDisabled | Account disabled. Contact admin. | ■■■■■■■■ ■■■■■■■■■. ■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■. | Hisob o‘chirilgan. Administratorga murojaat qiling |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| navDashboard | Dashboard | ■■■■■■■■ | Bosh sahifa |
| navAssignments | Assignments | ■■■■■■■■ | Topshiriqlar |
| navMarks | Marks | ■■■■■■■ | Baholar |
| navAttendance | Attendance | ■■■■■■■■■■■■■ | Davomat |
| navNotifications | Notifications | ■■■■■■■■■■■■ | Bildirishnomalar |
| navProfile | Profile | ■■■■■■■■ | Profil |
| navUsers | Users | ■■■■■■■■■■■■■ | Foydalanuvchilar |
| navGroups | Groups | ■■■■■■■ | Guruhlar |
| navAnalytics | Analytics | ■■■■■■■■■■ | Tahlil |
| navGradebook | Gradebook | ■■■■■■■ | Jurnal |
| navChat | Chat | ■■■■ | Chat |
| navFocus | Focus | ■■■■■■ | Fokus |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| focusTitle | Focus | ■■■■■■ | Fokus |
| focusSubtitle | Quiet study time for one assignment at a time. | ■■■■■■■■■■ ■■■■■■ — ■■■■■ ■■■■■■■■■ ■■ ■■■■. | Tinch o‘qish — bir vaqtda bitta topshiriq. |
| focusTodaysFocus | Today’s focus | ■■■■■■ ■■ ■■■■■■■■ | Bugungi fokus |
| focusTodaysFocusSubtitle | Up to three open assignments — any due date | ■■ ■■■■■ ■■■■■■■■■ ■■■■■■■■ — ■■■■■■ ■■■■■ | 3 tagacha ochiq topshiriq — muddat muhim emas |
| focusAlsoDue | More homework | ■■■■ ■■■■■■■■ | Yana topshiriqlar |
| focusContinueHomework | Continue homework | ■■■■■■■■■■■■ ■■■■■■■■■ | Uy ishini davom ettirish |
| focusOpenAssignment | Open assignment | ■■■■■■■■ ■■■■■■■■ | Topshiriqni ochish |
| focusEmptyTitle | No homework yet | ■■■■■ ■■■■ ■■■■■■■■ | Hali topshiriq yo‘q |
| focusEmptySubtitle | When your teacher posts an assignment, you can start a focus session here. | ■■■■■■ ■■■■■■■■ ■■■■■■■■■■■■ ■■■■■■■■, ■■■■■■ ■■■■■■ ■■■■■ | O‘qituvchi topshiriq e’lon qilganda, shu yerda fok |
| focusChooseMode | How do you want to focus? | ■■■■ ■■■■■■■ ■■■■■■■■■■■■■■■■■? | Qanday fokususlanasiz? |
| focusModePomodoro | Pomodoro | ■■■■■■■■■ | Pomodoro |
| focusModePomodoroHint | 25 minutes, then a break | 25 ■■■■■■, ■■■■■■ ■■■■■■■■ | 25 daqiqa, so‘ng tanaffus |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| focusModeFree | Free session | ■■■■■■■■■ ■■■■■■■ | Erkin sessiya |
| focusModeFreeHint | Count up until you stop | ■■■■ ■■ ■■■■■■■■■■ | To‘xtatgunicha hisoblash |
| focusDoNotDisturb | Chat muted | ■■■ ■■■■■■■■■■ | Chat o‘chirilgan |
| focusChatMuteHint | While a session is active, chat badges stay quiet so you can focus. Notification | ■■■■■ ■■■■■■■ ■■■■■■■■, ■■■■■■■ ■■■■■ ■■■■■■■. ■■■■■■■■■■ | Sessiya davomida chat belgilari yashirinadi. Bildi |
| focusPause | Pause | ■■■■■■ | Pauza |
| focusResume | Resume | ■■■■■■■■■■■ | Davom etish |
| focusPaused | Paused | ■■■■■■ | Pauza |
| focusEndSession | End | ■■■■■■■■■■ | Tugatish |
| focusSessionComplete | Session complete | ■■■■■■■ ■■■■■■■■■■ | Sessiya tugadi |
| focusSessionCompleteSubtitle | Nice work. Here’s a quick recap. | ■■■■■■■■■ ■■■■■■■. ■■■■■■■■ ■■■■■: | Yaxshi ish. Qisqa xulosa: |
| focusTimeUp | Time’s up! | ■■■■■■ ■■■■■■! | Vaqt tugadi! |
| focusTimeUpSubtitle | Your Pomodoro block finished. Take a short break, then keep going. | ■■■■■■■■■ ■■■■■■■■■■■. ■■■■■■■■■ ■■■■■■■■■ ■■■■■■■■ ■ ■ | Pomodoro tugadi. Qisqa tanaffus oling, so‘ng davom |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| focusMinutesFocused | {duration} focused | ■ ■■■■■■■: {duration} | Fokus: {duration} |
| focusOptionalNote | Optional note | ■■■■■■■■ (■■■■■■■■■■■■■■■) | Ixtiyoriy eslatma |
| focusOptionalNoteHint | e.g. Stuck on task #3 | ■■■■■. ■■■■■■■■ ■■ ■■■■■■■■ ■3 | masalan, 3-vazifada qiyinchilik |
| focusSaveAndOpen | Save & open assignment | ■■■■■■■■■■ ■ ■■■■■■■■ ■■■■■■■■ | Saqlash va topshiriqni ochish |
| focusDone | Done | ■■■■■■■ | Tayyor |
| chatTitle | Chat | ■■■■ | Chat |
| chatSearchHint | Search groups or teachers | ■■■■■■ ■■■■■■ ■■■■ ■■■■■■■■■ | Guruh yoki o‘qituvchi qidirish |
| chatEmptyTitle | No chat rooms yet | ■■■■■ ■■■■ ■■■■■■ | Hali chatlar yo‘q |
| chatEmptySubtitle | Join a group to start messaging your class. | ■■■■■■■■■ ■ ■■■■■■■, ■■■■■■ ■■■■■■■ ■■■■■■■. | Sinfga yozish uchun guruhga qo‘shiling. |
| chatNoMessagesYet | No messages yet | ■■■■■ ■■■■ ■■■■■■■■■■ | Hali xabarlar yo‘q |
| chatMessageDeleted | Message deleted | ■■■■■■■■■■ ■■■■■■■■ | Xabar o‘chirildi |
| chatPreviewImage | Photo | ■■■■■ | Rasm |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| chatPreviewVideo | Video | ■■■■■ | Video |
| chatVideoPlaybackSpeed | Playback speed | ■■■■■■■■ ■■■■■■■■■■■■■■■ | Ijro tezligi |
| chatVideoSpeedNormal | 1x | 1x | 1x |
| chatVideoSpeedValue | {speed}x | {speed}x | {speed}x |
| chatVideoRewind10 | -10s | -10■ | -10s |
| chatVideoForward10 | +10s | +10■ | +10s |
| chatVideoCouldNotPlay | Could not play this video | ■■ ■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■ | Videoni ijro etib bo‘lmadi |
| chatPreviewVoice | Voice message | ■■■■■■■■■ ■■■■■■■■■ | Ovozli xabar |
| chatPreviewFile | File | ■■■■ | Fayl |
| chatOpenFile | Tap to open | ■■■■■■■■ | Ochish |
| chatOpenFileFailed | Could not open this file. | ■■ ■■■■■■■■ ■■■■■■■■ ■■■■. | Faylni ochib bo‘lmadi. |
| chatYou | You | ■■ | Siz |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| chatTeacherMembers | {teacher} · {count} members | {teacher} · {count} ■■■■■■■■■■ | {teacher} · {count} a’zo |
| chatOnlineCount | {count} online | {count} ■ ■■■■ | {count} onlayn |
| chatReact | React | ■■■■■■■■ | Reaksiya |
| chatCopyText | Copy text | ■■■■■■■■■■ ■■■■■■ | Matnni nusxalash |
| chatCopied | Copied | ■■■■■■■■■■■■ | Nusxa olindi |
| chatReply | Reply | ■■■■■■■■■ | Javob berish |
| chatReplyTo | In reply to {name} | ■ ■■■■■ {name} | {name} ga javob |
| chatReplyDeleted | Deleted message | ■■■■■■■■■■ ■■■■■■■■■■ | O‘chirilgan xabar |
| chatReplyNotFound | Original message isn’t available | ■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■■ | Asl xabar topilmadi |
| chatMessageHint | Message… | ■■■■■■■■■■… | Xabar… |
| chatAttachImage | Image | ■■■■■ | Rasm |
| chatAttachVideo | Video | ■■■■■■ | Video |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| chatAttachFile | File | ■■■■■ | Fayl |
| chatTyping | {names} typing… | {names} ■■■■■■■■■… | {names} yozmoqda… |
| chatRecording | Recording {time} | ■■■■■■■ {time} | Yozilmoqda {time} |
| chatUploading | Uploading {name}… | ■■■■■■■■■ {name}… | {name} yuklanmoqda… |

| Key | EN | RU | UZ |
|---|---|---|---|
| chatSendFailed | Could not send: {error} | ■■ ■■■■■■■■ ■■■■■■■■■■: {error} | Yuborib bo‘lmadi: {error} |
| chatMicDenied | Microphone permission is required for voice messages. | ■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■ ■■■■■■■ ■ ■■■■■■■■■■. | Ovozli xabar uchun mikrofon ruxsati kerak. |
| chatEditTitle | Edit message | ■■■■■■■■■ ■■■■■■■■■■ | Xabarni tahrirlash |
| chatDeleteTitle | Delete message? | ■■■■■■■■ ■■■■■■■■■■? | Xabar o‘chirilsinmi? |
| chatDeleteBody | This removes the message for everyone in the room. | ■■■■■■■■■■ ■■■■■■ ■■■■■■■■ ■■■■ ■■■■■ ■ ■■■■■■■■. | Xabar xonadagi barcha uchun o‘chiriladi. |
| chatDeleteSelectedTitle | Delete {count} messages? | ■■■■■■■■ {count} ■■■■■■.? | {count} ta xabar o‘chirilsinmi? |
| chatDeleteSelectedBody | Selected messages will be removed for everyone in the room. | ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■ ■■■■■■■■ ■■■■ ■■■■■ ■ ■■■■■■ | Tanlangan xabarlar xonadagi barcha uchun o‘chirila |
| chatSelectedCount | {count} selected | ■■■■■■■■: {count} | {count} ta tanlandi |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| chatSelectMessages | Select messages | ■■■■■■■■ ■■■■■■■■■■ | Xabarlarni tanlash |
| chatSelect | Select | ■■■■■■■■ | Tanlash |
| commonSelectAll | Select all | ■■■■■■■■ ■■■■ | Hammasini tanlash |
| chatEdited | edited | ■■■. | tahrir |
| chatDateToday | Today | ■■■■■■■■ | Bugun |
| chatDateYesterday | Yesterday | ■■■■■■ | Kecha |
| chatRoleAdmin | admin | ■■■■■■ | admin |
| chatRoleTeacher | teacher | ■■■■■■■■ | o‘qituvchi |
| commonSave | Save | ■■■■■■■■■■ | Saqlash |
| commonCancel | Cancel | ■■■■■■■ | Bekor qilish |
| commonSubmit | Submit | ■■■■■■■■■■ | Yuborish |
| commonDelete | Delete | ■■■■■■■■ | O‘chirish |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| commonEdit | Edit | ■■■■■■■■■ | Tahrirlash |
| commonClose | Close | ■■■■■■■■ | Yopish |
| commonRetry | Retry | ■■■■■■■■■■ | Qayta urinish |
| commonTryAgain | Try again | ■■■■■■■■■■■■ ■■■■■■ | Qayta urinib ko‘ring |
| commonLoading | Loading… | ■■■■■■■■■… | Yuklanmoqda… |
| commonError | Something went wrong | ■■■-■■ ■■■■■■ ■■ ■■■■ | Nimadir noto‘g‘ri ketdi |
| commonErrorWithDetails | Error: {error} | ■■■■■■■: {error} | Xato: {error} |
| commonCouldNotSave | Could not save: {error} | ■■ ■■■■■■■■ ■■■■■■■■■■: {error} | Saqlab bo‘lmadi: {error} |

| Key | EN | RU | UZ |
|---|---|---|---|
| commonSignOut | Sign out | ■■■■■ | Chiqish |
| commonSaveDraft | Save draft | ■■■■■■■■■ ■■■■■■■■■ | Qoralamani saqlash |
| commonOpen | Open | ■■■■■■■■ | Ochish |
| commonCreate | Create | ■■■■■■■■ | Yaratish |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| commonAssign | Assign | ■■■■■■■■■■ | Biriktirish |
| commonReset | Reset | ■■■■■■■■■ | Tiklash |
| commonAdd | Add | ■■■■■■■■■ | Qo‘shish |
| commonLoadMore | Load more | ■■■■■■■■■■ ■■■■ | Yana yuklash |
| commonExport | Export | ■■■■■■■■ | Eksport |
| commonExportFailed | Could not export: {error} | ■■ ■■■■■■■■ ■■■■■■■■■■■■■■■: {error} | Eksport bo‘lmadi: {error} |
| marksTermAll | All time | ■■ ■■■■ ■■■■■■ | Butun davr |
| marksTermSemester | {yearRange} · Semester {semester} | {yearRange} · ■■■■■■■■ {semester} | {yearRange} · {semester}-semestr |
| marksExportCsv | Export CSV | ■■■■■■■■ CSV | CSV eksport |
| marksExportDone | Marks exported | ■■■■■■■ ■■■■■■■■■■■■■■■ | Baholar eksport qilindi |
| commonTeacher | Teacher | ■■■■■■■■ | O‘qituvchi |
| commonStudent | Student | ■■■■■■■ | O‘quvchi |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| commonAdmin | Administrator | ■■■■■■■■■■■■■■ | Administrator |
| commonWorkspace | Workspace | ■■■■■■■■ ■■■■■■■■■■■■■ | Ish maydoni |
| commonNoDueDate | No due date | ■■■■ ■■■■■■ | Muddat yo‘q |
| commonDueDate | Due {date} | ■■■■■ {date} | Muddat {date} |
| homeworkDueThisWeek | Due this week | ■■ ■■■■■ ■■■■■■■ | Shu hafta |
| homeworkDueThisWeekSubtitle | Overdue and upcoming deadlines | ■■■■■■■■■■■■■■ ■ ■■■■■■■■■■ ■■■■■■ | Kechikkan va yaqin muddatlar |
| homeworkNoDueThisWeek | Nothing due this week | ■■ ■■■■■ ■■■■■■■ ■■■■■■■ ■■■■ | Shu hafta muddat yo‘q |
| homeworkNoDueThisWeekSubtitle | You’re clear until next week’s deadlines. | ■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■ ■■■■■■■■■. | Keyingi hafta muddatlarigacha bo‘sh. |
| homeworkOverdue | Overdue · {date} | ■■■■■■■■■■■ · {date} | Muddati o‘tgan · {date} |
| homeworkDueToday | Due today | ■■■■■ ■■■■■■■■ | Bugun muddat |
| homeworkDueTomorrow | Due tomorrow | ■■■■■ ■■■■■■■ | Ertaga muddat |
| homeworkDueInDays | Due in {days} days | ■■■■■■ {days} ■■■. | {days} kundan keyin |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| homeworkLateBadge | Late | ■■■■■■■■■ | Kechikkan |
| commonProgress | Progress | ■■■■■■■■ | Jarayon |
| commonSubject | Subject | ■■■■■■■ | Fan |
| commonType | Type | ■■■ | Tur |
| commonTitle | Title | ■■■■■■■■ | Sarlavha |
| commonDescription | Description | ■■■■■■■■ | Tavsif |
| commonGroup | Group | ■■■■■■ | Guruh |
| commonMessage | Message | ■■■■■■■■■ | Xabar |
| commonScore | Score | ■■■■■■ | Ball |
| commonFeedback | Feedback | ■■■■■ | Izoh |
| commonStudents | Students | ■■■■■■■ | O‘quvchilar |
| commonCannotUndo | This action cannot be undone. | ■■■ ■■■■■■■■■ ■■■■■■ ■■■■■■■■■. | Bu amalni bekor qilib bo‘lmaydi. |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| profileTitle | Profile | ■■■■■■■■ | Profil |
| profileFullName | Full name | ■■■■■■■ ■■■ | To‘liq ism |
| profileNotSignedIn | Not signed in | ■■ ■■ ■■■■■■ | Tizimga kiritilmagan |
| profileAccount | Account | ■■■■■■■■ | Hisob |
| profileAccountSettings | Account Settings | ■■■■■■■■■■ ■■■■■■■■■ | Hisob sozlamalari |
| profileSignOutAdmin | Sign out of Admin Console | ■■■■■■ ■■ ■■■■■■■■ ■■■■■■■■■■■■■■■ | Admin konsolidan chiqish |
| profileSettings | Settings | ■■■■■■■■■■ | Sozlamalar |
| profileAppearance | Appearance | ■■■■■■■■■■■ | Ko‘rinish |
| profileThemeLight | Light | ■■■■■■■■ | Yorug‘ |
| profileThemeDark | Dark | ■■■■■■■ | Qorong‘u |
| profileThemeSystem | System | ■■■■■■■■■■ | Tizim |
| profileStyle | Style | ■■■■■■ | Uslub |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| profileStyleModern | Modern Professional | ■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■ | Zamonaviy professional |
| profileStyleNature | Nature Paper | ■■■■■■■■■■ ■■■■■■■ | Tabiat qog‘ozi |
| profileStyleSoftNeu | Neumorphism | ■■■■■■■■■■■ | Neumorfizm |
| profileStyleMonochrome | Monochrome Workspace | ■■■■■■■■■■■■ ■■■■■■■■■■■■ | Monoxrom ish maydoni |
| profileEditName | Edit name | ■■■■■■■■■ ■■■■ | Ismni o‘zgartirish |

| Key | EN | RU | UZ |
|---|---|---|---|
| profileNameUpdated | Name updated | ■■■ ■■■■■■■■■■ | Ism yangilandi |
| profileChangePhoto | Change photo | ■■■■■■■■ ■■■■■ | Rasmni o‘zgartirish |
| profilePhotoUpdated | Photo updated | ■■■■■ ■■■■■■■■■■ | Rasm yangilandi |
| profileChangePassword | Change password | ■■■■■■■■ ■■■■■■■ | Parolni o‘zgartirish |
| profileCurrentPassword | Current password | ■■■■■■■■ ■■■■■■■ | Joriy parol |
| profileNewPassword | New password | ■■■■■■ ■■■■■■■ | Yangi parol |
| profileConfirmPassword | Confirm new password | ■■■■■■■■■■■■ ■■■■■■ ■■■■■■■ | Yangi parolni tasdiqlang |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| profilePasswordChanged | Password updated | ■■■■■■■ ■■■■■■■■■ | Parol yangilandi |
| profilePasswordMismatch | New passwords do not match | ■■■■■■ ■■■■■■■ ■■ ■■■■■■■■■■ | Yangi parollar mos emas |
| profilePasswordTooShort | Use at least 8 characters | ■■ ■■■■■■ 8 ■■■■■■■■■ | Kamida 8 ta belgi |
| profilePasswordHint | Ask an admin if you forget your password. | ■■■■■ ■■■■■■■ ■■■■■■■ — ■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■. | Parolni unutsangiz, admindan so‘rang. |
| profileShortcuts | Shortcuts | ■■■■■■■■ ■■■■■■■ | Tezkor havolalar |
| profileYourGroups | Your groups | ■■■■■ ■■■■■■■ | Guruhlaringiz |
| profileNoGroups | No groups assigned | ■■■■■■■ ■■ ■■■■■■■■■■ | Guruh biriktirilmagan |
| profileGroupsCount | {count} groups | ■■■■■■: {count} | {count} ta guruh |
| profileAverageLabel | Term average | ■■■■■■■■ ■■ ■■■■■■■ | Davr o‘rtachasi |
| profileAttendanceLabel | Attendance | ■■■■■■■■■■■■■ | Davomat |
| profileShareProgress | Share progress report | ■■■■■■■■■■■ ■■■■■■■■ | Natija hisobotini ulashish |
| profileNotifications | Notifications | ■■■■■■■■■■■■ | Bildirishnomalar |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| profileClearCache | Clear cached data | ■■■■■■■■■ ■■■■ | Keshni tozalash |
| profileClearCacheWarn1Title | Clear cached data? | ■■■■■■■■■ ■■■■? | Kesh tozalansinmi? |
| profileClearCacheWarn1Body | This deletes offline chat messages and downloaded images, videos, voice notes, a | ■■■■■■ ■■■■■■■■ ■■■■■■■-■■■■■■■■■■ ■■■■■ ■ ■■■■■■■■■■ ■■ | Oflayn chat xabarlari va yuklab olingan rasmlar, v |
| profileClearCacheWarn2Title | Are you sure? | ■■ ■■■■■■■■? | Ishonchingiz komilmi? |
| profileClearCacheWarn2Body | This cannot be undone. Cached chat media will be removed from this device. | ■■■■ ■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■. ■■■■ ■■■■■ ■■■■■■ ■■■■■■ | Bu amalni bekor qilib bo‘lmaydi. Chat keshi shu qu |
| profileClearCacheConfirm | Clear cache | ■■■■■■■■■ ■■■■ | Keshni tozalash |
| profileClearCacheDone | Cached data cleared | ■■■■ ■■■■■■■ | Kesh tozalandi |
| profileAppVersion | MU Study · v{version} | MU Study · v{version} | MU Study · v{version} |

| Key | EN | RU | UZ |
|---|---|---|---|
| profileSubject | Subject | ■■■■■■■ | Fan |
| greetingMorning | Good Morning | ■■■■■■ ■■■■ | Xayrli tong |
| greetingAfternoon | Good Afternoon | ■■■■■■ ■■■■ | Xayrli kun |
| greetingEvening | Good Evening | ■■■■■■ ■■■■■ | Xayrli kech |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| greetingWithName | {greeting}, {name} | {greeting}, {name} | {greeting}, {name} |
| greetingLoadingTitle | Good Morning, User | ■■■■■■ ■■■■, ■■■■■■■■■■■■ | Xayrli tong, foydalanuvchi |
| greetingLoadingSubtitle | Loading your personalized greeting... | ■■■■■■■■■ ■■■■ ■■■■■■■■■■■... | Shaxsiy salomlashuv yuklanmoqda... |
| studentHomeSubtitle | Your learning space | ■■■■ ■■■■■■■■ ■■■■■■■■■■■■ | Sizning o‘quv maydoningiz |
| continueLearning | Continue Learning | ■■■■■■■■■■ ■■■■■■■■ | O‘qishni davom ettirish |
| continueLearningSubtitle | Pick up where you left off | ■■■■■■■■■■ ■ ■■■■ ■■■■■, ■■■ ■■■■■■■■■■■■ | To‘xtagan joyingizdan davom eting |
| everythingCompleted | Everything is completed. | ■■■ ■■■■■■■■■■. | Hammasi bajarilgan. |
| caughtUpToday | You're all caught up for today. | ■■ ■■■■■■■■ ■■■ ■■■■■■■. | Bugun hammasi tayyor. |
| homeworkPack | Homework Pack | ■■■■■ ■■■■■■■■■ ■■■■■■■ | Uy vazifasi to‘plami |
| staticTask | Static Task | ■■■■■■■ ■■■■■■■ | Oddiy topshiriq |
| staticAssignment | Static | ■■■■■■■ | Oddiy |
| upcomingAssignments | Upcoming Assignments | ■■■■■■■■■ ■■■■■■■ | Yaqin topshiriqlar |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| stayOnTrack | Stay on track | ■■ ■■■■■■■■■■ | Rejada qoling |
| loadingAnnouncements | Loading announcements... | ■■■■■■■■■ ■■■■■■■■■■... | E’lonlar yuklanmoqda... |
| notificationsCardTitle | Notifications | ■■■■■■■■■■■■ | Bildirishnomalar |
| unreadAlerts | You have {count} new alerts | ■ ■■■ {count} ■■■■■ ■■■■■■■■■■■ | Sizda {count} ta yangi bildirishnoma bor |
| upToDate | You are up to date | ■ ■■■ ■■■ ■■■■■■■■■■ | Hammasi yangilangan |
| announcementsTitle | Announcements | ■■■■■■■■■■ | E’lonlar |
| announcementsSubtitle | From your teachers | ■■ ■■■■■ ■■■■■■■■ | O‘qituvchilaringizdan |
| announcementsEmpty | No announcements | ■■■ ■■■■■■■■■■ | E’lonlar yo‘q |
| announcementsEmptyTeacher | Share important updates with your students. | ■■■■■■■■ ■■■■■■■ ■■■■■■■■■■■■ ■ ■■■■■■■■■. | Muhim yangiliklarni o‘quvchilar bilan ulashing. |

| Key | EN | RU | UZ |
|---|---|---|---|
| announcementsEmptyAdmin | Platform-wide announcements will appear here. | ■■■■■■■■■■ ■■ ■■■■■ ■■■■■■■■■■ ■■■■■■■■■ ■■■■■■. | Platforma bo‘ylab e’lonlar shu yerda paydo bo‘ladi |
| announcementsReadMore | Read more | ■■■■■■■ ■■■■■■ | Batafsil |
| announcementsShowLess | Show less | ■■■■■■■■■ | Yig‘ish |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| announcementsViewAll | View all ({count}) | ■■■■■■■■■ ■■■■ ({count}) | Barchasini ko‘rish ({count}) |
| announcementsAll | All Announcements | ■■■■ ■■■■■■■■■■■■ | Barcha e’lonlar |
| announcementsNew | New Announcement | ■■■■■■ ■■■■■■■■■■■■ | Yangi e’lon |
| announcementsPost | Post Announcement | ■■■■■■■■■■■■■■ ■■■■■■■■■■■ | E’lonni joylash |
| announcementsEdit | Edit Announcement | ■■■■■■■■■ ■■■■■■■■■■■■ | E’lonni tahrirlash |
| announcementsDeleteTitle | Delete announcement? | ■■■■■■■■ ■■■■■■■■■■■■? | E’lon o‘chirilsinmi? |
| announcementsDeleteBody | It will be removed for everyone in the group. | ■■■■ ■■■■■■ ■■■■■■■■ ■ ■■■■■ ■■■■■■■■■■■■ ■■■■■■■. | U guruhdagi barcha a’zolar uchun o‘chiriladi. |
| announcementsByAdmin | Administration | ■■■■■■■■■■■■■■■ | Ma’muriyat |
| announcementsShareUpdates | Share updates with your groups | ■■■■■■■■■ ■■■■■■■■■■■■■ ■ ■■■■■■■■■ | Guruhlaringiz bilan yangiliklar ulashing |
| assignmentsTitle | Assignments | ■■■■■■■■ | Topshiriqlar |
| assignmentsEmpty | No assignments | ■■■■ ■■■■■■■■ | Topshiriqlar yo‘q |
| assignmentsEmptySubtitle | You are all caught up! | ■ ■■■■ ■■■■ ■■■■■■■■■■! | Hammasi bajarilgan! |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| assignmentsEmptyTeacher | Create your first assignment to get started. | ■■■■■■■■■ ■■■■■■■ ■■■■■■■■, ■■■■■■ ■■■■■■■. | Boshlash uchun birinchi topshiriqni yarating. |
| assignmentsLearningPath | Learning Path | ■■■■■■■■ ■■■■■ | O‘quv yo‘li |
| assignmentsActive | Active assignments | ■■■■■■■■■ ■■■■■■■■ | Faol topshiriqlar |
| assignmentDetails | Assignment Details | ■■■■■■■ ■■■■■■■■ | Topshiriq tafsilotlari |
| assignmentNotFound | Assignment not found | ■■■■■■■■ ■■ ■■■■■■■■ | Topshiriq topilmadi |
| assignmentTasks | Tasks | ■■■■■■■ | Vazifalar |
| assignmentTasksSubtitle | Complete all to finish pack | ■■■■■■■■■■ ■■■■, ■■■■■■ ■■■■■■■■■■ ■■■■■■ | To‘plamni yakunlash uchun hammasini bajaring |
| assignmentNew | New Assignment | ■■■■■■ ■■■■■■■■ | Yangi topshiriq |
| assignmentManage | Manage Assignments | ■■■■■■■■■■■ ■■■■■■■■■■ | Topshiriqlarni boshqarish |
| assignmentManageSubtitle | Create static or homework packs | ■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■ | Oddiy yoki uy vazifasi to‘plamlarini yarating |

| Key | EN | RU | UZ |
|---|---|---|---|
| marksTitle | My Marks | ■■■ ■■■■■■ | Baholarim |
| marksOverallAverage | Overall average | ■■■■■ ■■■■■■■ ■■■■ | Umumiy o‘rtacha |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| marksNoneYet | No marks recorded yet. | ■■■■■■ ■■■■ ■■■. | Hali baholar yo‘q. |
| marksBasedOn | {label} · based on {count} marks. | {label} · ■■ ■■■■■■ {count} ■■■■■■. | {label} · {count} ta baho asosida. |
| marksByCategory | By category | ■■ ■■■■■■■■■■ | Turkumlar bo‘yicha |
| marksWhereStrongest | Where you are strongest | ■■■ ■■ ■■■■■■■ ■■■■■ | Eng kuchli tomonlaringiz |
| marksAll | All marks | ■■■ ■■■■■■ | Barcha baholar |
| marksNewestFirst | Newest first | ■■■■■■■ ■■■■■ | Avval yangilari |
| marksEmptyTitle | No marks yet | ■■■■■■ ■■■■ ■■■ | Hali baholar yo‘q |
| marksEmptySubtitle | Your teacher's marks will appear here. | ■■■■■■ ■■■■■■■ ■■■■■■■■ ■■■■■. | O‘qituvchi baholari shu yerda ko‘rinadi. |
| progressReportTitle | Progress report | ■■■■■ ■■ ■■■■■■■■■■■■ | Natija hisoboti |
| progressReportShare | Share progress | ■■■■■■■■■■ ■■■■■■■■■■■■■ | Natijani ulashish |
| progressReportSharing | Preparing report… | ■■■■■■■ ■■■■■… | Hisobot tayyorlanmoqda… |
| progressReportDone | Progress report ready to share | ■■■■■ ■■■■■ ■ ■■■■■■■■ | Hisobot ulashishga tayyor |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| progressReportFailed | Could not share progress: {error} | ■■ ■■■■■■■ ■■■■■■■■■■: {error} | Ulashib bo‘lmadi: {error} |
| progressReportNeedGroup | Join a group to share a progress report. | ■■■■■■■■■ ■ ■■■■■■■, ■■■■■ ■■■■■■■■■■■ ■■■■■■■■. | Hisobot ulashish uchun guruhga qo‘shiling. |
| progressReportAttendance | Attendance | ■■■■■■■■■■■■■ | Davomat |
| progressReportAttendanceRate | Attendance rate | ■■■■■■■■■■■■■ | Davomat foizi |
| progressReportNoAttendance | No attendance recorded for this term. | ■■ ■■■■■ ■■■■■■■ ■■■■■■■■■■■■■ ■■ ■■■■■■■■■. | Bu muddat uchun davomat yozilmagan. |
| progressReportSessions | sessions | ■■■■■■■■ | mashg‘ulot |
| progressReportMarksCounted | marks | ■■■■■■■ | baho |
| progressReportRecentMarks | Recent marks | ■■■■■■■■■ ■■■■■■■ | So‘nggi baholar |
| progressReportGeneratedOn | Generated {date} | ■■■■■■■■■■■■■ {date} | Yaratilgan: {date} |
| marksNoRecorded | No marks recorded | ■■■■■■■ ■■■■ | Baholar yozilmagan |

| Key | EN | RU | UZ |
|---|---|---|---|
| marksNoRecordedSubtitle | Use "Grade class" to enter marks in one pass. | ■■■■■■■■■■■ «■■■■■■■ ■■■■■■», ■■■■■ ■■■■■■■■■■ ■■■■■ | Birdaniga baho qo‘yish uchun «Sinfni baholash»dan |
| marksRecent | Recent marks | ■■■■■■■■ ■■■■■■■ | So‘nggi baholar |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| marksRecentSubtitle | Latest entries across the group | ■■■■■■■■■■ ■■■■■■■ ■■ ■■■■■■■ | Guruh bo‘yicha oxirgi yozuvlar |
| marksStudentsHeader | Students | ■■■■■■■■ | O‘quvchilar |
| marksStudentsSubtitle | Weighted average of all marks | ■■■■■■■■■■■■■■■■■■ ■■■■■ ■■■■■ ■■■■■■■ | Barcha baholarning o‘rtacha og‘irlikli bali |
| marksNoStudents | No students in this group | ■ ■■■■■ ■■■■■■■ ■■■■ ■■■■■■■■■ | Bu guruhda o‘quvchilar yo‘q |
| marksNoStudentsSubtitle | An admin can assign students to your group. | ■■■■■■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■ ■■■■■■■■■ ■ ■■■■■ ■■■■■ | Administrator guruhingizga o‘quvchilarni biriktiri |
| marksDeleteTitle | Delete mark? | ■■■■■■■■ ■■■■■■■? | Bahoni o‘chirish? |
| marksDeleteBody | "{title}" will be removed permanently. | «{title}» ■■■■■■ ■■■■■■■■ ■■■■■■■■■. | «{title}» butunlay o‘chiriladi. |
| marksGradeTheClass | Grade the class | ■■■■■■■■ ■■■■■■ | Sinfni baholash |
| marksOnePercentEach | One percent mark per student | ■■■■■ ■■■■■■■■■■■ ■■■■■■■ ■■ ■■■■■■■■ | Har bir o‘quvchiga bitta foizli baho |
| marksSaveMarks | Save marks | ■■■■■■■■■■ ■■■■■■■ | Baholarni saqlash |
| marksSaveMark | Save mark | ■■■■■■■■■■ ■■■■■■■ | Bahoni saqlash |
| marksAddMark | Add mark | ■■■■■■■■■ ■■■■■■■ | Baho qo‘shish |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| marksNothingYet | Nothing recorded yet | ■■■■■ ■■■■■■■ ■■■■ | Hali hech narsa yo‘q |
| marksAddFirst | Add the first mark for this student. | ■■■■■■■■■ ■■■■■■■ ■■■■■■■ ■■■■■■ ■■■■■■■■. | Bu o‘quvchiga birinchi bahoni qo‘shing. |
| marksSavedCount | Saved {count} marks | ■■■■■■■■■■ ■■■■■■■: {count} | {count} ta baho saqlandi |
| marksGroupHasNoStudents | This group has no students yet. | ■ ■■■■■ ■■■■■■■ ■■■■■ ■■■■ ■■■■■■■■■■. | Bu guruhda hali o‘quvchilar yo‘q. |
| marksTitleHint | e.g. Unit 3 test | ■■■■■■■■■, ■■■■■ ■■ ■■■■■ 3 | masalan, 3-mavzu testi |
| marksAverageLine | Average {percent} · {label} | ■■■■■■■■ {percent} · {label} | O‘rtacha {percent} · {label} |
| marksNoMarksYetShort | No marks yet | ■■■■■■■ ■■■■■ ■■■■ | Hali baholar yo‘q |
| marksSummaryStudents | Students | ■■■■■■■■ | O‘quvchilar |
| marksSummaryMarks | Marks | ■■■■■■■ | Baholar |
| attendanceTitle | Attendance | ■■■■■■■■■■■■■ | Davomat |

| Key | EN | RU | UZ |
|---|---|---|---|
| attendanceEmpty | No attendance records | ■■■ ■■■■■■■■ ■■■■■■■■■■■■ | Davomat yozuvlari yo‘q |
| attendanceNoGroup | No group assigned yet | ■■■■■■■ ■■■■ ■■ ■■■■■■■■■■ | Guruh hali biriktirilmagan |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| attendanceMark | Mark Attendance | ■■■■■■■■■ ■■■■■■■■■■■■■ | Davomatni belgilash |
| attendanceDailyTracking | Daily student tracking | ■■■■■■■■■■■■ ■■■■■ ■■■■■■■■■ | Kunlik o‘quvchi hisobi |
| attendanceSaved | Attendance saved successfully | ■■■■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■■ | Davomat muvaffaqiyatli saqlandi |
| attendanceNoGroups | No groups found | ■■■■■■■ ■■ ■■■■■■■■ | Guruhlar topilmadi |
| attendancePresent | P — Present | ■ — ■■■■■■■■■■■■■■■ | B — Bor |
| attendanceLate | L — Late | ■ — ■■■■■■■■ | K — Kechikdi |
| attendanceAbsent | A — Absent | ■ — ■■■■■■■■■■■■■■ | Y — Yo‘q |
| attendanceExcused | E — Excused | ■ — ■■■■■■■■■■■■■■ | S — Sababli |
| attendancePresentShort | P | ■ | B |
| attendanceLateShort | L | ■ | K |
| attendanceAbsentShort | A | ■ | Y |
| attendanceExcusedShort | E | ■ | S |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| attendanceTrend | Attendance Trend | ■■■■■■■■■ ■■■■■■■■■■■■■ | Davomat dinamikasi |
| attendanceOverall | Overall distribution | ■■■■■■ ■■■■■■■■■■■■■■ | Umumiy taqsimot |
| analyticsThisWeek | This Week | ■■■ ■■■■■■■ | Shu hafta |
| analyticsLastWeek | Last Week | ■■■■■■■■ ■■■■■■■ | O‘tgan hafta |
| analyticsTotalMarks | Total Marks: {count} | ■■■■■■ ■■■■■■■: {count} | Jami baholar: {count} |
| notificationsTitle | Notifications | ■■■■■■■■■■■■ | Bildirishnomalar |
| notificationsEmpty | All caught up! | ■■■■ ■■■■■■■■■■! | Hammasi o‘qilgan! |
| notificationsEmptySubtitle | No new notifications. | ■■■■■■ ■■■■■■■■■■■■■ ■■■■. | Yangi bildirishnomalar yo‘q. |
| notificationsNone | No notifications | ■■■■ ■■■■■■■■■■■■ | Bildirishnomalar yo‘q |
| notifKindAssignment | New homework | ■■■■■■ ■■ | Yangi uy vazifasi |
| notifKindAnnouncement | Announcement | ■■■■■■■■■■■ | E’lon |
| notifKindGraded | Work graded | ■■■■■■■ ■■■■■■■■ | Ish baholandi |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| notifKindSubmission | New submission | ■■■■■ ■■■■■ | Yangi topshiriq |
| notifKindChat | Chat message | ■■■■■■■■■ ■ ■■■■■ | Chat xabari |
| notifKindSystem | Update | ■■■■■■■■■■ | Yangilanish |
| notificationsOpen | Open | ■■■■■■■■ | Ochish |
| taskTitle | Task | ■■■■■■■■ | Vazifa |
| taskYourAnswer | Your answer | ■■■ ■■■■■■ | Javobingiz |
| taskMarkCompleted | Mark completed | ■■■■■■■■■ ■■■■■■■■■■■■ | Bajarildi deb belgilash |
| taskAttachment | Attachment | ■■■■■■■■■ | Ilova |
| taskAttachmentHint | Add a photo or document to your answer. | ■■■■■■■■■ ■■■■■ ■■■■ ■■■■■■■■■ ■ ■■■■■■■. | Javobingizga rasm yoki hujjat qo‘shing. |
| taskAttachmentHintReady | Tap to open · trash to remove. | ■■■■■■■■, ■■■■■■ ■■■■■■■■ · ■■■■■■■■ — ■■■■■■■■. | Ochish uchun bosing · o‘chirish uchun axlat qutisi |
| taskImage | Image | ■■■■■■■■■■■■ | Rasm |
| taskImageSubtitle | Gallery photo | ■■■■■ ■■ ■■■■■■■■ | Galereya fotosurati |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| taskFile | File | ■■■■■ | Fayl |
| taskFileSubtitle | PDF or document | PDF ■■■■ ■■■■■■■■■ | PDF yoki hujjat |
| taskReplaceImage | Replace with image | ■■■■■■■■■ ■■■■■■■■■■■■■ | Rasm bilan almashtirish |
| taskReplaceFile | Replace file | ■■■■■■■■■ ■■■■■ | Faylni almashtirish |
| taskVoice | Voice record | ■■■■■■■■■■ ■■■■■■■ | Ovozli yozuv |
| taskVoiceHint | Optional spoken answer — record, play back, then save. | ■■■■■■■■■■■■■■■■ ■■■■■■■ ■■■■■■ — ■■■■■■■■■, ■■■■■■■■■■■ | Ixtiyoriy og‘zaki javob — yozing, tinglang, so‘ng |
| taskViewPdf | View PDF | ■■■■■■■■ PDF | PDFni ko‘rish |
| taskOpenLink | Open link | ■■■■■■■■ ■■■■■■■ | Havolani ochish |
| taskSubmitted | Submitted | ■■■■■■■■■■■ | Yuborildi |
| taskSaved | Saved | ■■■■■■■■■■ | Saqlandi |
| taskUploadFailed | Upload failed: {error} | ■■ ■■■■■■■■ ■■■■■■■■■■: {error} | Yuklash muvaffaqiyatsiz: {error} |
| taskVoiceUploadFailed | Voice upload failed: {error} | ■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■: {error} | Ovozni yuklab bo‘lmadi: {error} |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| taskCouldNotSave | Could not save: {error} | ■■ ■■■■■■■■ ■■■■■■■■■■: {error} | Saqlab bo‘lmadi: {error} |

| Key | EN | RU | UZ |
|---|---|---|---|
| voiceTapToRecord | Tap the mic to record your answer | ████████ ██ ████████, ██████ ████████ ██████ | Javobingizni yozish uchun mikrofonni bosing |
| voiceRecording | Recording… | ███████… | Yozilmoqda… |
| voiceNote | Voice note | ██████████ ████████ | Ovozli eslatma |
| voiceTapPlay | Tap play to listen | ████████ ████████████████, ██████ ████████████ | Tinglash uchun ijro etishni bosing |
| voiceRerecord | Re-record | ██████████████ | Qayta yozish |
| voiceStudentAnswer | Student voice answer | ██████████ ██████ ████████ | O‘quvchining ovozli javobi |
| voiceMicPermission | Microphone permission is required to record. | ███ ████████ ██████ ███████ ■ ████████████. | Yozish uchun mikrofon ruxsati kerak. |
| voiceCouldNotStart | Could not start recording: {error} | ██ ████████ ███████ ███████: {error} | Yozuvni boshlab bo‘lmadi: {error} |
| voiceCouldNotSave | Could not save recording: {error} | ██ ████████ ██████████ ███████: {error} | Yozuvni saqlab bo‘lmadi: {error} |
| voiceCouldNotPlay | Could not play voice note: {error} | ██ ████████ ████████████████: {error} | Ovozni ijro etib bo‘lmadi: {error} |
| attachmentCouldNotOpen | Could not open this file. | ██ ████████ ████████ ████ ████. | Bu faylni ochib bo‘lmadi. |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| attachmentImage | Image attachment | ████████████ | Rasm ilovasi |
| attachmentFile | File attachment | ████ | Fayl ilovasi |
| attachmentPdf | PDF document | PDF-██████████ | PDF hujjat |
| attachmentTapPreview | Tap to preview | ████████ ████ ██████████ | Ko‘rish uchun bosing |
| attachmentTapOpen | Tap to open | ████████, ██████ ████████ | Ochish uchun bosing |
| pdfDefaultTitle | PDF | PDF | PDF |
| teacherWorkspace | Teacher Workspace | ████████ ████████████ ████████ | O‘qituvchi ish maydoni |
| teacherQuickActions | Quick Actions | ████████ ██████████ | Tezkor amallar |
| teacherCreateItems | Create new items | ████████ ██████ ██████████ | Yangi elementlar yaratish |
| teacherNewAssignment | New Assignment | ██████ ████████ | Yangi topshiriq |
| teacherNewAnnouncement | New Announcement | ██████ ████████████ | Yangi e’lon |
| teacherSubmissions | Submissions | ███████ | Ishlar |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherNoSubmissions | No submissions yet | ██████ ██████ ████ | Hali ishlar yo‘q |

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherNoSubmissionsSubtitle | Student work will show up here for grading. | ■■■■■■ ■■■■■■■■ ■■■■■■■■ ■■■■■ ■■■ ■■■■■■■■. | O‘quvchi ishlari baholash uchun shu yerda ko‘rinad |
| teacherGradebook | Gradebook | ■■■■■■ | Jurnal |
| teacherGradeClass | Grade class | ■■■■■■■ ■■■■■ | Sinfni baholash |
| teacherNoGroups | No groups yet | ■■■■■ ■■■■ ■■■ | Hali guruhlar yo‘q |
| teacherNoGroupsSubtitle | Ask an admin to assign you a group. | ■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■ ■■■ ■■■■■■. | Administratordan guruh biriktirishini so‘rang. |
| teacherActiveGroups | ACTIVE GROUPS | ■■■■■■■■ ■■■■■■ | FAOL GURUHLAR |
| teacherSubject | SUBJECT | ■■■■■■■ | FAN |
| teacherDailyTasks | Daily Tasks | ■■■■■■■■■■ ■■■■■■ | Kunlik vazifalar |
| teacherNeedsAttention | What needs your attention | ■■■ ■■■■■■■ ■■■■■■■■ | Diqqatingizni talab qiladiganlar |
| teacherReviewSubmissions | Review Submissions | ■■■■■■■■■ ■■■■■■ | Ishlarni tekshirish |
| teacherCheckAssignments | Check student assignments | ■■■■■■■■■ ■■■■■■■ ■■■■■■■■ | O‘quvchi topshiriqlarini tekshirish |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherRecordMarks | Record marks in percent | ■■■■■■■■■ ■■■■■■ ■ ■■■■■■■■■ | Foizli baholar qo‘yish |
| teacherPerformanceAnalytics | Performance Analytics | ■■■■■■■■■ ■■■■■■■■■■■■ | Natijalar tahlili |
| teacherTrackProgress | Track group progress | ■■■■■■■■■■■ ■■■■■■■■ ■■■■■■ | Guruh jarayonini kuzatish |
| teacherOpenAttachment | Open attachment | ■■■■■■■ ■■■■■■■■ | Ilovani ochish |
| teacherScoreHelper | Adds a homework mark to the gradebook | ■■■■■■■■■ ■■■■■■ ■■ ■■ ■ ■■■■■■ | Uy vazifasi bahosini jurnalga qo‘shadi |
| teacherScoreRange | Score must be between 0 and 100. | ■■■■■■ ■■■■■■ ■■■■ ■■ 0 ■■ 100. | Ball 0 dan 100 gacha bo‘lishi kerak. |
| teacherNoWrittenAnswer | No written answer. | ■■■■■■■■■■■ ■■■■■■ ■■■. | Yozma javob yo‘q. |
| teacherReviewSubmission | Review submission | ■■■■■■■■■ ■■■■■■ | Ishni tekshirish |
| teacherHomeworkPackTasks | Homework Pack Tasks | ■■■■■■ ■■■■■■ ■■ | Uy vazifasi to‘plami vazifalari |
| teacherAddTask | Add Task | ■■■■■■■■ ■■■■■■ | Vazifa qo‘shish |
| teacherEditTask | Edit Task | ■■■■■■■■ ■■■■■■ | Vazifani tahrirlash |
| teacherDeleteTask | Delete Task? | ■■■■■■■ ■■■■■■? | Vazifani o‘chirish? |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherTasksContent | Tasks Content | ■■■■■■■■■■ ■■■■■ | Vazifalar mazmuni |

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherManagePack | Manage items in this pack | ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■ ■■■■■■ | Bu to‘plamdagi elementlarni boshqaring |
| teacherTaskTitleHint | e.g. Reading Exercise | ■■■■■■■■■, ■■■■■■■■■■ ■■ ■■■■■■ | masalan, O‘qish mashqi |
| teacherLinkHint | https://... | https://... | https://... |
| teacherAttachmentHint | Image or file link... | ■■■■■■ ■■ ■■■■■■■■■■■ ■■■ ■■■■... | Rasm yoki fayl havolasi... |
| teacherNoData | No data available | ■■■ ■■■■■■■ | Ma’lumot yo‘q |
| teacherMarkDistribution | Mark Distribution | ■■■■■■■■■■■■■ ■■■■■■ | Baholar taqsimoti |
| teacherStudentsByAverage | Students grouped by average | ■■■■■■■■ ■■ ■■■■■■■■ ■■■■■ | O‘quvchilar o‘rtacha ball bo‘yicha |
| teacherNoMarksTrends | Add marks in the gradebook to see trends. | ■■■■■■■■■ ■■■■■■■ ■ ■■■■■■■, ■■■■■ ■■■■■■■■ ■■■■■■■■■. | Tendensiyalarni ko‘rish uchun jurnalga baho qo‘shi |
| teacherTargetGroup | Target Group | ■■■■■■■■ ■■■■■■■ | Maqsadli guruh |
| adminWorkspace | Workspace Control | ■■■■■■■■■■■ ■■■■■■■■■■■ | Platformani boshqarish |
| adminUsers | User Management | ■■■■■■■■■■■■■ | Foydalanuvchilar |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminGroups | Groups | ■■■■■■■ | Guruhlar |
| adminAnalytics | Analytics | ■■■■■■■■■■ | Tahlil |
| adminTeachers | TEACHERS | ■■■■■■■■ | O‘QITUVCHILAR |
| adminStudents | STUDENTS | ■■■■■■■■ | O‘QUVCHILAR |
| adminGroupsLabel | GROUPS | ■■■■■■■ | GURUHLAR |
| adminMarksLabel | MARKS | ■■■■■■■ | BAHOLAR |
| adminActivityInsights | Activity Insights | ■■■■■■■■■■ ■■■■■■■■■■■ | Faoliyat tahlili |
| adminCurrentMetrics | Current metrics | ■■■■■■■■ ■■■■■■■■■■■■ | Joriy ko‘rsatkichlar |
| adminCreateUser | Create New User | ■■■■■■■■ ■■■■■■■■■■■■■ | Yangi foydalanuvchi |
| adminCreateUserButton | Create User | ■■■■■■■■ ■■■■■■■■■■■■■ | Foydalanuvchi yaratish |
| adminUserCreated | User created successfully | ■■■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■ | Foydalanuvchi muvaffaqiyatli yaratildi |
| adminResetPassword | Reset password | ■■■■■■■■■ ■■■■■■■ | Parolni tiklash |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminResetPasswordTitle | Reset Password · {username} | ■■■■■ ■■■■■■■ · {username} | Parolni tiklash · {username} |

| Key | EN | RU | UZ |
|---|---|---|---|
| adminCreateGroup | Create New Group | ■■■■■■■ ■■■■■■ | Yangi guruh |
| adminCreateGroupButton | Create Group | ■■■■■■■ ■■■■■■ | Guruh yaratish |
| adminGroupTitleHint | e.g. Group 4 | ■■■■■■■■, ■■■■■■■ 4 | masalan, 4-guruh |
| adminGroupTitleExists | This teacher already has a group with that name | ■ ■■■■■■ ■■■■■■■■ ■■■■ ■■■■■ ■■■■■■■ ■ ■■■■■■ ■■■■■■■■■■ | Bu o‘qituvchida shu nomdagi guruh allaqachon bor |
| adminAssignStudent | Assign Student | ■■■■■■■■■■ ■■■■■■■■ | O‘quvchini biriktirish |
| adminActivityDistribution | Activity Distribution | ■■■■■■■■■■■■■■ ■■■■■■■■■■■ | Faoliyat taqsimoti |
| adminGlobalMetrics | Global platform metrics | ■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■ | Global platforma ko‘rsatkichlari |
| adminActivityLog | Admin Activity Log | ■■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■■■■■■■ | Admin faoliyat jurnali |
| adminActivityLogSubtitle | System-wide events and audit logs appear here. | ■■■■■■■■■■ ■■■■■■■■ ■ ■■■■■■ ■■■■■■■■■ ■■■■■. | Tizim voqealari va audit yozuvlari shu yerda paydo |
| adminTeacherLine | Teacher: {name} | ■■■■■■■■: {name} | O‘qituvchi: {name} |
| adminRoleLine | {role} · {username} | {role} · {username} | {role} · {username} |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminDisable | Disable | ■■■■■■■■■■ | O‘chirish |
| adminEnable | Enable | ■■■■■■■■■ | Yoqish |
| adminPassword | Password | ■■■■■■■ | Parol |
| adminFullName | Full name | ■■■■■■■ ■■■■ | To‘liq ism |
| adminRole | Role | ■■■■■ | Rol |
| adminSphere | Sphere | ■■■■■■ | Soha |
| percentExcellent | Excellent | ■■■■■■■■ | A’lo |
| percentGood | Good | ■■■■■■■ | Yaxshi |
| percentNeedsWork | Needs work | ■■■■■■ ■■■■■■■■■■ | Yaxshilash kerak |
| percentAtRisk | At risk | ■■■■ ■■■■■■■■ | Xavf ostida |
| emptyStateDefault | Nothing here yet | ■■■■■ ■■■■■■■ ■■■■ | Hali hech narsa yo‘q |
| setupRequiredTitle | Setup required | ■■■■■■■■■■ ■■■■■■■■■■ | Sozlash kerak |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| setupRequiredBody | Supabase is not configured. Add SUPABASE_URL and SUPABASE_ANON_KEY to your .env | Supabase ■■ ■■■■■■■■■. ■■■■■■■■■ SUPABASE_URL ■ SUPA | Supabase sozlanmagan. .env fayliga SUPABASE_URL va |
| setupOption1 | Option 1 — local .env file | ■■■■■■■■ 1 — ■■■■■■■■■■ ■■■■■ .env | 1-variant — mahalliy .env fayli |
| setupOption2 | Option 2 — dart-define | ■■■■■■■■ 2 — dart-define | 2-variant — dart-define |

| Key | EN | RU | UZ |
|---|---|---|---|
| attendanceAllPresent | All P | ■■■ ■ | Hammasi B |
| attendanceAllAbsent | All A | ■■■ ■ | Hammasi Y |
| attendanceAllLate | All L | ■■■ ■ | Hammasi K |
| attendanceAllExcused | All E | ■■■ ■ | Hammasi S |
| attendancePresentWord | Present | ■■■■■■■■■■■■■ | Bor |
| attendanceAbsentWord | Absent | ■■■■■■■■■■■■ | Yo‘q |
| attendanceLateWord | Late | ■■■■■■■ | Kechikdi |
| attendanceExcusedWord | Excused | ■■■■■■■■■■■■ | Sababli |
| attendanceWeekdayMon | Mon | ■■ | Du |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| attendanceWeekdayTue | Tue | ■■ | Se |
| attendanceWeekdayWed | Wed | ■■ | Ch |
| attendanceWeekdayThu | Thu | ■■ | Pa |
| attendanceWeekdayFri | Fri | ■■ | Ju |
| attendancePercentLabel | Rate | ■■■■ | Foiz |
| attendanceNoLesson | No lesson | ■■■ ■■■■■ | Dars yo‘q |
| attendanceHistory | Attendance history | ■■■■■■■ ■■■■■■■■■■■■ | Davomat tarixi |
| attendanceSelectStudent | Student | ■■■■■■ | O‘quvchi |
| attendanceAllStudents | All students | ■■■ ■■■■■■■ | Barcha o‘quvchilar |
| attendanceTapToCycle | Tap a day to change status | ■■■■■■■ ■■ ■■■■, ■■■■■ ■■■■■■■■ ■■■■■■ | Statusni o‘zgartirish uchun kunga bosing |
| attendanceUpdated | Attendance updated | ■■■■■■■■■■■■ ■■■■■■■■■ | Davomat yangilandi |
| attendanceCalendarEmpty | No attendance in this month | ■■■ ■■■■■■■■■■■■ ■ ■■■■ ■■■■■■■ | Bu oyda davomat yo‘q |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| attendanceFutureNotAllowed | You can’t mark attendance for a future day | ■■■■■■■ ■■■■■■■■■ ■■■■■■■■■■■■■ ■■ ■■■■■■■■ ■■■■■ | Kelajakdagi kun uchun davomat belgilab bo‘lmaydi |
| attendanceWeekendNotAllowed | Attendance can only be marked on weekdays (Mon–Fri) | ■■■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■ ■ ■■■■■■ (■■–■■) | Davomat faqat ish kunlari (du–ju) uchun belgilanad |
| chartBandExcellent | 85–100% | 85–100% | 85–100% |
| chartBandGood | 70–84% | 70–84% | 70–84% |
| chartBandNeedsWork | 50–69% | 50–69% | 50–69% |
| chartBandAtRisk | Below 50% | ■■■■ 50% | 50% dan past |
| marksWeight | Weight | ■■■■ | Og‘irlik |

| Key | EN | RU | UZ |
|---|---|---|---|
| marksCategory | Category | ■■■■■■■■■ | Turkum |
| marksDate | Date | ■■■■ | Sana |
| marksCommentOptional | Comment (optional) | ■■■■■■■■■■■ (■■■■■■■■■■■■■) | Izoh (ixtiyoriy) |
| marksNewMark | New mark | ■■■■■ ■■■■■■ | Yangi baho |
| marksEditMark | Edit mark | ■■■■■■■■ ■■■■■■ | Bahoni tahrirlash |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| marksFromSubmission | From a graded submission | ■■ ■■■■■■■■■■■■ ■■■■■■ | Tekshirilgan ishdan |
| marksNeedTitle | Give the graded item a title. | ■■■■■■■■ ■■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■■■. | Baholanadigan element uchun sarlavha kiriting. |
| marksWeightPositive | Weight must be greater than zero. | ■■■ ■■■■■■■ ■■■■■ ■■■■■■■ ■■■■■. | Og‘irlik noldan katta bo‘lishi kerak. |
| marksNeedAtLeastOne | Enter a mark for at least one student. | ■■■■■■■■ ■■■■■■■ ■■■■■ ■■ ■■■■■■■ ■■■■■■■■. | Kamida bitta o‘quvchiga baho kiriting. |
| marksLeaveBlankSkip | Leave a field blank to skip that student. | ■■■■■■■■■ ■■■■■ ■■■■■■■, ■■■■■■ ■■■■■■■■■■■ ■■■■■■■■. | O‘quvchini o‘tkazib yuborish uchun maydonni bo‘sh |
| marksFromSubmissionDelete | This mark comes from a graded submission. Clear the score in Submissions to remo | ■■■ ■■■■■■■ ■■ ■■■■■■■■■■■■■ ■■■■■■■. ■■■■■■ ■■■■■■■■ ■■ | Bu baho tekshirilgan ishdan. Uni o‘chirish uchun « |
| marksFromSubmissionEdit | This mark mirrors a graded submission. Editing it here will be overwritten the n | ■■■ ■■■■■■■ ■■■■■■■■ ■ ■■■■■■■■■■■■■ ■■■■■■■■. ■■■■■■■■■ | Bu baho tekshirilgan ishga bog‘langan. Bu yerda ta |
| categoryHomework | Homework | ■■■■■■■■■ ■■■■■■■■ | Uy vazifasi |
| categoryClasswork | Classwork | ■■■■■■■■■ ■■■■■■■ | Sinf ishi |
| categoryQuiz | Quiz | ■■■■■ | Viktorina |
| categoryTest | Test | ■■■■■ | Test |
| categoryExam | Exam | ■■■■■■■■ | Imtihon |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| categoryProject | Project | ■■■■■■■ | Loyiha |
| categoryOral | Oral | ■■■■■■ | Og‘zaki |
| categoryOther | Other | ■■■■■■■ | Boshqa |
| teacherReferenceLinkOptional | Reference Link (optional) | ■■■■■■■ (■■■■■■■■■■■■■■■) | Havola (ixtiyoriy) |
| teacherAttachmentUrlOptional | Attachment URL (optional) | URL ■■■■■■■■■ (■■■■■■■■■■■■■■■) | Ilova URL (ixtiyoriy) |
| teacherNoTasksYet | No tasks added yet. Click "Add Task" to get started. | ■■■■■■ ■■■■■ ■■■■. ■■■■■■■■ «■■■■■■■■■ ■■■■■■■», ■■■■■■ ■ | Hali vazifalar yo‘q. Boshlash uchun «Vazifa qo‘shi |

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherTemplates | Templates | ■■■■■■■ | Shablonlar |
| teacherTemplatesEmpty | No templates yet | ■■■■■■■■ ■■■■ ■■■ | Hali shablonlar yo‘q |
| teacherTemplatesEmptySubtitle | Save an assignment as a template to reuse it later. | ■■■■■■■■■ ■■■■■■■ ■■■ ■■■■■■, ■■■■■ ■■■■■■■■■■■ ■ | Qayta ishlatish uchun topshiriqni shablon sifatida |
| teacherSaveAsTemplate | Save as template | ■■■■■■■■■ ■■■ ■■■■■■ | Shablon sifatida saqlash |
| teacherTemplateSaved | Template saved | ■■■■■■ ■■■■■■■■ | Shablon saqlandi |
| teacherUseTemplate | Use template | ■■■■■■■■■■■■ ■■■■■■ | Shablondan foydalanish |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherFromTemplate | From template | ■■ ■■■■■■■ | Shablondan |
| teacherDeleteTemplate | Delete template? | ■■■■■■■ ■■■■■■? | Shablonni o‘chirish? |
| teacherDeleteTemplateBody | "{title}" will be removed. | «{title}» ■■■■■ ■■■■■■. | «{title}» o‘chiriladi. |
| teacherDuplicateAssignment | Duplicate | ■■■■■■■■■■ | Nusxa olish |
| teacherDuplicateTitle | Duplicate assignment | ■■■■■■■■■■■ ■■■■■■■ | Topshiriqdan nusxa |
| teacherDuplicated | Assignment duplicated | ■■■■■■■ ■■■■■■■■■■ | Topshiriq nusxalandi |
| assignmentEdit | Edit assignment | ■■■■■■■■ ■■■■■■■■ | Topshiriqni tahrirlash |
| assignmentDeleteTitle | Delete assignment? | ■■■■■■■ ■■■■■■■■? | Topshiriq o‘chirilsinmi? |
| assignmentDeleteBody | This permanently removes the assignment, its tasks, and student submissions. | ■■■■■■■■, ■■■■■■ ■ ■■■■■■ ■■■■■■■■ ■■■■■ ■■■■■■■ ■■ | Topshiriq, vazifalar va o‘quvchi topshiriqlari but |
| assignmentDeleted | Assignment deleted | ■■■■■■■ ■■■■■■■ | Topshiriq o‘chirildi |
| assignmentTypeHintStatic | A single piece of work. Students submit once — no extra tasks. | ■■■■ ■■■■■■■. ■■■■■■■ ■■■■■ ■■■■ ■■■ — ■■■ ■■■■■■■■■■ | Bitta ish. O‘quvchi bir marta topshiradi — qo‘shim |
| assignmentTypeHintPack | A pack of multiple tasks (workbook-style). You’ll add items next. | ■■■■■■ ■■ ■■■■■■■■■■■ ■■■■■■. ■■■■■■■ ■■■■■■■■■ ■■■■■■■. | Bir nechta vazifali to‘plam. Keyin bandlarni qo‘sh |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherCreateFromTemplate | Create from template | ■■■■■■■■ ■■ ■■■■■■■■ | Shablondan yaratish |
| teacherTasksCopied | {count} tasks included | ■■■■■■: {count} | {count} ta vazifa kiritiladi |
| teacherRecentFeedback | Reuse feedback | ■■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■ | Fikrni qayta ishlatish |
| teacherApplyFeedbackToOthers | Also apply to other pending for this task | ■■■■■■ ■■■■■■■■■■ ■ ■■■■■■■ ■■■■■■■■■■ ■■ ■■■■■ ■■■■■■■ | Shu vazifa bo‘yicha boshqa kutayotganlarga ham qo‘ |

| Key | EN | RU | UZ |
|---|---|---|---|
| teacherFeedbackAppliedCount | Feedback applied to {count} other submissions | ■■■■■ ■■■■■■■■ ■ {count} ■■■■■■ ■■■■■■■ | Fikr yana {count} ta topshiriqqa qo‘llandi |
| teacherSharedComment | Shared comment (optional) | ■■■■■ ■■■■■■■■■■■■ (■■■■■■■■■■■■■■) | Umumiy izoh (ixtiyoriy) |
| teacherSharedCommentHint | Same note on every mark you save | ■■■■ ■ ■■ ■■ ■■■■■■■■ ■■ ■■■■■ ■■■■■■■■■■■■ ■■■■■■■ | Saqlanadigan har bir bahoga bir xil izoh |
| adminPlatformOverview | Platform Overview | ■■■■■ ■■■■■■■■■■ | Platforma umumiy ko‘rinishi |
| adminAssignTeacher | Assign Teacher | ■■■■■■■■■■ ■■■■■■■■ | O‘qituvchini biriktirish |
| adminSelectStudent | Select Student | ■■■■■■■■ ■■■■■■■■ | O‘quvchini tanlash |
| adminInitialGroupOptional | Initial Group (Optional) | ■■■■■■■■■■ ■■■■■■■ (■■■■■■■■■■■■■■) | Boshlang‘ich guruh (ixtiyoriy) |
| adminChartTasks | Tasks | ■■■■■■■ | Vazifalar |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminChartGroups | Groups | ■■■■■■■ | Guruhlar |
| adminChartStaff | Staff | ■■■■■■■■■■■ | Xodimlar |
| adminChartAssignments | Assignments | ■■■■■■■■ | Topshiriqlar |
| adminChartMarks | Marks | ■■■■■■■ | Baholar |
| adminChartStudents | Students | ■■■■■■■■ | O‘quvchilar |
| adminAcademicYears | Academic years | ■■■■■■■■ ■■■■■ | O‘quv yillari |
| adminAcademicYearsSubtitle | School years and semesters for the gradebook | ■■■■■ ■ ■■■■■■■■■ ■■■■ ■■■■■■■■ ■■■■■■■ | Baholar jurnali uchun yillar va semestrlar |
| adminAcademicYearsTitle | Academic calendar | ■■■■■■■■ ■■■■■■■■■■ | O‘quv kalendari |
| adminCreateNextYear | Create next year | ■■■■■■■■ ■■■■■■■■■■ ■■■■ | Keyingi yilni yaratish |
| adminYearCreated | Created {label} | ■■■■■■■ {label} | {label} yaratildi |
| adminSetCurrentYear | Set as current | ■■■■■■■■ ■■■■■■■■ | Joriy qilish |
| adminCurrentYearBadge | Current | ■■■■■■■■ | Joriy |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminYearSemesters | Four semesters · Aug → Jun | ■■■■■■■ ■■■■■■■■■ · ■■■ → ■■■ | To‘rt semestr · avg → iyn |
| adminSemesterRange | Semester {semester} · {start} – {end} | ■■■■■■■■ {semester} · {start} – {end} | {semester}-semestr · {start} – {end} |
| adminEditTerm | Edit term | ■■■■■■■■■ ■■■■■ | Muddatni tahrirlash |
| adminEditTermTitle | Edit semester {semester} | ■■■■■■■■ {semester} | {semester}-semestr |
| adminTermStartsOn | Starts on | ■■■■■■■ | Boshlanishi |
| adminTermEndsOn | Ends on | ■■■■■■ | Tugashi |

| Key | EN | RU | UZ |
|---|---|---|---|
| adminTermDatesInvalid | End date must be on or after the start date | ■■■■ ■■■■■■■■■■ ■■■■■■■ ■■■■■ ■■ ■■■■■■■ ■■■■■ ■■■■■■■ | Tugash sanasi boshlanish sanasidan oldin bo‘lmasli |
| adminTermUpdated | Term dates updated | ■■■■ ■■■■■■■■■ ■■■■■■■■■■ | Semestr sanalari yangilandi |
| adminDeleteYear | Delete year | ■■■■■■■■ ■■■■ | Yilni o‘chirish |
| adminDeleteYearTitle | Delete {label}? | ■■■■■■■■ {label}? | {label} ni o‘chirasizmi? |
| adminDeleteYearBody | This removes the year and its four terms from the calendar. Marks are not delete | ■■■■ ■ ■■■■■■■ ■■■■■■■■■ ■■■■■■ ■■■■■■■■ ■■ ■■■■■■■■■■. | Yil va to‘rt semestr kalendardan o‘chiriladi. Baho |
| adminDeleteYearConfirmHint | Type {label} to confirm | ■■■■■■■■ {label} ■■■■ ■■■■■■■■■■■■■■■ | Tasdiqlash uchun {label} ni yozing |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminDeleteYearCurrentBlocked | Set another year as current before deleting this one. | ■■■■■■■■ ■■■■■■■■■ ■■■■■■■■ ■■■■■■■ ■■■■, ■■■■■■ ■■■■■■■■ | Avval boshqa yilni joriy qiling, keyin buni o‘chir |
| adminYearDeleted | Deleted {label} | ■■■■■■■ {label} | {label} o‘chirildi |
| adminNoAcademicYears | No academic years yet | ■■■■■■■■ ■■■■■■ ■■■■■ ■■■■ | Hali o‘quv yillari yo‘q |
| adminAcademicYearsHint | Create the next school year when you are ready. Teachers and students pick semes | ■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■ ■■■■, ■■■■■■ ■■■■■■■ ■■■■■■■ | Tayyor bo‘lganda keyingi o‘quv yilini yarating. O‘ |
| adminCredentialPrintTitle | Print login slips | ■■■■■■■ ■■■■■■■■ | Login qog‘ozlari |
| adminCredentialPrintSubtitle | Unique usernames and passwords from the database, 3 slips per page | ■■■■■■■■■■■ ■■■■■■■ ■■ ■■■■■, 3 ■■■■■■■■■ ■■ ■■■■■■■■■ | Bazadan unique loginlar, sahifada 3 ta karta |
| adminCredentialPasteLabel | Users | ■■■■■■■■■■■■■ | Foydalanuvchilar |
| adminCredentialPasteHint | username\|password\|full name @james\|Abc123xyz\|James English\|teacher @john_jones\|P | username\|password\|full name @james\|Abc123xyz\|James | username\|password\|full name @james\|Abc123xyz\|James |
| adminCredentialPerPage | Slips per page | ■■■■■■■■■ ■■ ■■■■■■■■■ | Sahifadagi kartalar |
| adminCredentialTwo | 2 | 2 | 2 |
| adminCredentialThree | 3 | 3 | 3 |
| adminCredentialExportPdf | Export PDF | ■■■■■■■■ PDF | PDF eksport |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| adminCredentialSlipTitle | Your MU Study login | ■■■ ■■■■ ■ MU Study | MU Study login |
| adminCredentialSlipHint | Keep this paper private. Change your password after first sign-in. | ■■■■■■■■ ■■■■■ ■■■■■■■ ■■■■ ■■■■. ■■■■■■■■ ■■■■■■■ ■■■■■ | Bu qog‘ozni o‘zingizda saqlang. Birinchi kirishdan |
| adminCredentialNeedUsers | No users found to print | ■■■■ ■■■■■■■■■■■■■■■ ■■■■ ■■■■■■■ | Chop etish uchun foydalanuvchi yo‘q |

| Key | EN | RU | UZ |
|---|---|---|---|
| adminCredentialNeedSql | Run sql/admin_login_slips.sql in the Supabase SQL Editor, then export again. | ■■■■■■■■■ sql/admin_login_slips.sql ■ SQL Editor S | Supabase SQL Editor’da sql/admin_login_slips.sql n |
| adminCredentialConfidential | Private and confidential. These credentials are intended only for the assigned s | ■■■■■■■■■■■■■■■■. ■■■ ■■■■■■■ ■■■■■■■ ■■■ ■■■■■■■■■■■■ | Maxfiy. Bu ma’lumotlar faqat belgilangan o‘quvchi |
| adminFirstName | First name | ■■■ | Ism |
| adminLastName | Last name | ■■■■■■■■ | Familiya |
| adminCredentialStudents | Students | ■■■■■■■■ | O‘quvchilar |
| adminCredentialTeachers | Teachers | ■■■■■■■■ | O‘qituvchilar |
| adminCredentialAllGroups | All groups | ■■■ ■■■■■■■ | Barcha guruhlar |
| sphereMath | Math | ■■■■■■■■■■■ | Matematika |
| sphereEnglish | English | ■■■■■■■■■■■ | Ingliz tili |

Practice: pick one key from this block, find it in a screen with your editor, and change the English string locally to prove the widget is wired. Then revert.

| Key | EN | RU | UZ |
|---|---|---|---|
| dailyMotivation1 | Today is a great day to make progress. | ■■■■■■■■ ■■■■■■■■■ ■■■■, ■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■. | Bugun oldinga siljish uchun ajoyib kun. |
| dailyMotivation2 | Ready to continue where you left off? | ■■■■■■■ ■■■■■■■■■■■■ ■■■■■ ■■■■■■, ■■■■ ■■■■■■■■■■■■■■? | To‘xtagan joyingizdan davom etishga tayyormisiz? |
| dailyMotivation3 | Let's conquer your goals today. | ■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■ ■■■■■■. | Bugun maqsadlaringizni zabt etaylik. |
| dailyMotivation4 | Consistency is the key to mastery. | ■■■■■■■■■■■■■ — ■■■■■ ■■■■■■■■■■■■. | Izchillik — mahorat kaliti. |

## Missing translations

RU missing: 0. UZ missing: 0.

# RLS policies in English

Each policy is a sentence you should be able to say to a school principal.

## profiles_admin_all on public.profiles

**profiles_admin_all**

```
CREATE POLICY profiles_admin_all ON public.profiles FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say profiles_admin_all aloud**
> Complete: “A row in public.profiles may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## profiles_update_self on public.profiles

**profiles_update_self**

```
CREATE POLICY profiles_update_self ON public.profiles FOR UPDATE TO authenticated
  USING ((id = auth.uid()) AND (is_disabled = false)) WITH CHECK (id = auth.uid());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say profiles_update_self aloud**
> Complete: “A row in public.profiles may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## profiles_select on public.profiles

**profiles_select**

```
CREATE POLICY profiles_select ON public.profiles FOR SELECT TO authenticated USING (
  (id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students st ON ((st.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (st.profile_id = profiles.id))
  ))
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say profiles_select aloud**
> Complete: “A row in public.profiles may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## profiles_select_groupmates on public.profiles

**profiles_select_groupmates**

```
CREATE POLICY profiles_select_groupmates ON public.profiles FOR SELECT TO authenticated USING (
  (EXISTS (
    SELECT 1 FROM (((group_students gs_self
      JOIN students s_self ON ((s_self.id = gs_self.student_id)))
      JOIN group_students gs_other ON ((gs_other.group_id = gs_self.group_id)))
      JOIN students s_other ON ((s_other.id = gs_other.student_id)))
    WHERE ((s_self.profile_id = auth.uid()) AND (s_other.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((group_students gs
      JOIN students s ON ((s.id = gs.student_id)))
      JOIN groups g ON ((g.id = gs.group_id)))
      JOIN teachers t ON ((t.id = g.teacher_id)))
    WHERE ((s.profile_id = auth.uid()) AND (t.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students s ON ((s.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (s.profile_id = profiles.id))
  ))
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say profiles_select_groupmates aloud**
> Complete: “A row in public.profiles may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## profiles_select_chat_message_senders on public.profiles

**profiles_select_chat_message_senders**

```
CREATE POLICY profiles_select_chat_message_senders ON public.profiles FOR SELECT TO
authenticated USING (
  EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE ((m.sender_id = profiles.id) AND can_access_chat_room(m.room_id))
  )
);

-- teachers / students
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say profiles_select_chat_message_senders aloud**
> Complete: “A row in public.profiles may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## teachers_admin on public.teachers

**teachers_admin**

```
CREATE POLICY teachers_admin ON public.teachers FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say teachers_admin aloud**
> Complete: “A row in public.teachers may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## teachers_select on public.teachers

**teachers_select**

```
CREATE POLICY teachers_select ON public.teachers FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'student'::user_role)
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say teachers_select aloud**
> Complete: “A row in public.teachers may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## students_admin on public.students

**students_admin**

```
CREATE POLICY students_admin ON public.students FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say students_admin aloud**
> Complete: “A row in public.students may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## students_select on public.students

**students_select**

```
CREATE POLICY students_select ON public.students FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'teacher'::user_role)
);

-- groups
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say students_select aloud**
> Complete: “A row in public.students may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## groups_select on public.groups

**groups_select**

```
CREATE POLICY groups_select ON public.groups FOR SELECT TO authenticated USING (
  is_teacher_of_group(id) OR is_student_in_group(id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say groups_select aloud**
> Complete: “A row in public.groups may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## groups_teacher_manage on public.groups

**groups_teacher_manage**

```
CREATE POLICY groups_teacher_manage ON public.groups FOR ALL TO authenticated
  USING (is_teacher_of_group(id) OR is_admin())
  WITH CHECK (is_teacher_of_group(id) OR is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say groups_teacher_manage aloud**
> Complete: “A row in public.groups may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## group_students_select on public.group_students

**group_students_select**

```
CREATE POLICY group_students_select ON public.group_students FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say group_students_select aloud**
> Complete: “A row in public.group_students may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## group_students_manage on public.group_students

**group_students_manage**

```
CREATE POLICY group_students_manage ON public.group_students FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- assignments / tasks
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say group_students_manage aloud**
> Complete: “A row in public.group_students may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## assignments_select on public.assignments

**assignments_select**

```
CREATE POLICY assignments_select ON public.assignments FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say assignments_select aloud**
> Complete: “A row in public.assignments may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## assignments_teacher on public.assignments

**assignments_teacher**

```
CREATE POLICY assignments_teacher ON public.assignments FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.profile_id
= auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say assignments_teacher aloud**
> Complete: “A row in public.assignments may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## tasks_select on public.tasks

**tasks_select**

```
CREATE POLICY tasks_select ON public.tasks FOR SELECT TO authenticated USING (
  EXISTS (
    SELECT 1 FROM assignments a
    WHERE a.id = tasks.assignment_id
      AND (is_teacher_of_group(a.group_id) OR is_student_in_group(a.group_id) OR is_admin())
  )
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say tasks_select aloud**
> Complete: “A row in public.tasks may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## tasks_teacher on public.tasks

**tasks_teacher**

```
CREATE POLICY tasks_teacher ON public.tasks FOR ALL TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin());

-- submissions
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say tasks_teacher aloud**
> Complete: “A row in public.tasks may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## submissions_select on public.submissions

**submissions_select**

```
CREATE POLICY submissions_select ON public.submissions FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id = auth.
uid())))
  OR (EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  ))
  OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say submissions_select aloud**
> Complete: “A row in public.submissions may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## submissions_student on public.submissions

**submissions_student**

```
CREATE POLICY submissions_student ON public.submissions FOR ALL TO authenticated
```

```
  USING (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id
 = auth.uid()))
  WITH CHECK (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.
profile_id = auth.uid()));
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say submissions_student aloud**
> Complete: “A row in public.submissions may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## submissions_teacher_update on public.submissions

**submissions_teacher_update**

```
CREATE POLICY submissions_teacher_update ON public.submissions FOR UPDATE TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin());

-- attendance / sessions
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say submissions_teacher_update aloud**
> Complete: “A row in public.submissions may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## attendance_select on public.attendance

**attendance_select**

```
CREATE POLICY attendance_select ON public.attendance FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say attendance_select aloud**
> Complete: “A row in public.attendance may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## attendance_teacher on public.attendance

**attendance_teacher**

```
CREATE POLICY attendance_teacher ON public.attendance FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say attendance_teacher aloud**
> Complete: “A row in public.attendance may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## lesson_sessions_select on public.lesson_sessions

**lesson_sessions_select**

```
CREATE POLICY lesson_sessions_select ON public.lesson_sessions FOR SELECT TO authenticated
USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say lesson_sessions_select aloud**
> Complete: “A row in public.lesson_sessions may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## lesson_sessions_manage on public.lesson_sessions

**lesson_sessions_manage**

```
CREATE POLICY lesson_sessions_manage ON public.lesson_sessions FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- marks
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say lesson_sessions_manage aloud**
> Complete: “A row in public.lesson_sessions may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## marks_select on public.marks

**marks_select**

```
CREATE POLICY marks_select ON public.marks FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = marks.student_id AND s.profile_id = auth.uid(
)))
  OR is_teacher_of_group(group_id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say marks_select aloud**
> Complete: “A row in public.marks may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## marks_teacher_manage on public.marks

**marks_teacher_manage**

```
CREATE POLICY marks_teacher_manage ON public.marks FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- announcements / templates / notifications
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say marks_teacher_manage aloud**
> Complete: “A row in public.marks may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## announcements_select on public.announcements

**announcements_select**

```
CREATE POLICY announcements_select ON public.announcements FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say announcements_select aloud**
> Complete: “A row in public.announcements may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## announcements_teacher on public.announcements

**announcements_teacher**

```
CREATE POLICY announcements_teacher ON public.announcements FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say announcements_teacher aloud**
> Complete: “A row in public.announcements may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## assignment_templates_teacher on public.assignment_templates

**assignment_templates_teacher**

```
CREATE POLICY assignment_templates_teacher ON public.assignment_templates FOR ALL TO
authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id
AND t.profile_id = auth.uid())) OR is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say assignment_templates_teacher aloud**
> Complete: “A row in public.assignment_templates may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## notifications_own on public.notifications

**notifications_own**

```
CREATE POLICY notifications_own ON public.notifications FOR ALL TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());

-- academic
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say notifications_own aloud**
> Complete: “A row in public.notifications may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## academic_years_select on public.academic_years

**academic_years_select**

```
CREATE POLICY academic_years_select ON public.academic_years FOR SELECT TO authenticated USING (
true);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say academic_years_select aloud**
> Complete: “A row in public.academic_years may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## academic_years_admin on public.academic_years

**academic_years_admin**

```
CREATE POLICY academic_years_admin ON public.academic_years FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say academic_years_admin aloud**
> Complete: “A row in public.academic_years may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## academic_terms_select on public.academic_terms

**academic_terms_select**

```
CREATE POLICY academic_terms_select ON public.academic_terms FOR SELECT TO authenticated USING (
true);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say academic_terms_select aloud**
> Complete: “A row in public.academic_terms may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## academic_terms_admin on public.academic_terms

**academic_terms_admin**

```
CREATE POLICY academic_terms_admin ON public.academic_terms FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- chat
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say academic_terms_admin aloud**
> Complete: “A row in public.academic_terms may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## chat_rooms_select on public.chat_rooms

**chat_rooms_select**

```
CREATE POLICY chat_rooms_select ON public.chat_rooms FOR SELECT TO authenticated
  USING (can_access_chat_room(id));
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say chat_rooms_select aloud**
> Complete: “A row in public.chat_rooms may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## chat_rooms_admin_insert on public.chat_rooms

**chat_rooms_admin_insert**

```
CREATE POLICY chat_rooms_admin_insert ON public.chat_rooms FOR INSERT TO authenticated
  WITH CHECK (is_admin() OR is_teacher_of_group(group_id));
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say chat_rooms_admin_insert aloud**
> Complete: “A row in public.chat_rooms may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## chat_messages_select on public.chat_messages

**chat_messages_select**

```
CREATE POLICY chat_messages_select ON public.chat_messages FOR SELECT TO authenticated
  USING (can_access_chat_room(room_id));
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say chat_messages_select aloud**
> Complete: “A row in public.chat_messages may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## chat_messages_insert on public.chat_messages

**chat_messages_insert**

```
CREATE POLICY chat_messages_insert ON public.chat_messages FOR INSERT TO authenticated
  WITH CHECK ((sender_id = auth.uid()) AND can_access_chat_room(room_id));
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say chat_messages_insert aloud**
> Complete: “A row in public.chat_messages may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## chat_messages_update on public.chat_messages

**chat_messages_update**

```
CREATE POLICY chat_messages_update ON public.chat_messages FOR UPDATE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)))
  WITH CHECK (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say chat_messages_update aloud**
> Complete: “A row in public.chat_messages may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## chat_messages_delete on public.chat_messages

**chat_messages_delete**

```
CREATE POLICY chat_messages_delete ON public.chat_messages FOR DELETE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
```

This is a mixed rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say chat_messages_delete aloud**
> Complete: “A row in public.chat_messages may be mixed when …”. If you need the SQL in front of you, you do not own the policy yet.

## message_reads_select on public.message_reads

**message_reads_select**

```
CREATE POLICY message_reads_select ON public.message_reads FOR SELECT TO authenticated USING (
  (user_id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  ))
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reads_select aloud**
> Complete: “A row in public.message_reads may be read when …”. If you need the SQL in front of you, you do not own the policy yet.

## message_reads_insert on public.message_reads

**message_reads_insert**

```
CREATE POLICY message_reads_insert ON public.message_reads FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  )));
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reads_insert aloud**
> Complete: “A row in public.message_reads may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## message_reads_delete on public.message_reads

**message_reads_delete**

```
CREATE POLICY message_reads_delete ON public.message_reads FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());
```

This is a mixed rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reads_delete aloud**
> Complete: "A row in public.message_reads may be mixed when …". If you need the SQL in front of you, you do not own the policy yet.

## message_reactions_select on public.message_reactions

**message_reactions_select**

```
CREATE POLICY message_reactions_select ON public.message_reactions FOR SELECT TO authenticated
USING (
  EXISTS (SELECT 1 FROM chat_messages m WHERE m.id = message_reactions.message_id AND
can_access_chat_room(m.room_id))
);
```

This is a read rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reactions_select aloud**
> Complete: "A row in public.message_reactions may be read when …". If you need the SQL in front of you, you do not own the policy yet.

## message_reactions_insert on public.message_reactions

**message_reactions_insert**

```
CREATE POLICY message_reactions_insert ON public.message_reactions FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE m.id = message_reactions.message_id AND m.is_deleted = false AND can_access_chat_room(
m.room_id)
  )));
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reactions_insert aloud**
> Complete: "A row in public.message_reactions may be write when …". If you need the SQL in front of you, you do not own the policy yet.

## message_reactions_update on public.message_reactions

**message_reactions_update**

```
CREATE POLICY message_reactions_update ON public.message_reactions FOR UPDATE TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());
```

This is a write rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reactions_update aloud**
> Complete: “A row in public.message_reactions may be write when …”. If you need the SQL in front of you, you do not own the policy yet.

## message_reactions_delete on public.message_reactions

**message_reactions_delete**

```
CREATE POLICY message_reactions_delete ON public.message_reactions FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());
```

This is a mixed rule. Ask: who is auth.uid()? Does the function is_admin / is_teacher_of_group / is_student_in_group / can_access_chat_room appear? If the policy is USING(true), it is a red flag unless the table is harmless.

> **CHECKPOINT — Say message_reactions_delete aloud**
> Complete: “A row in public.message_reactions may be mixed when …”. If you need the SQL in front of you, you do not own the policy yet.

# 30-day rebuild journal

If you work one day at a time, this journal is your calendar. Each day ends with a written proof (screenshot, SQL result, or test output). Days are not optional snacks — they are the same 30 tutorials in calendar form.

## Day 01 — Product walk

**Work:** Tutorial 1 screenshots. Write the three roles from memory.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 1 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 02 — ER diagram

**Work:** Tutorial 2. Circle profile id vs student id.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 2 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 03 — Supabase project

**Work:** Tutorial 3. URL + anon key in private notes.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 3 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 04 — .env + SetupRequired A/B

**Work:** Tutorial 4.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 4 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 05 — flutter create / tree

**Work:** Tutorial 5. pub get.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 5 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 06 — Dart drills

**Work:** Tutorial 6 tests.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 6 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 07 — Practice login UI

**Work:** Tutorial 7.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 7 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 08 — Todo State vs Riverpod note

**Work:** Tutorial 8.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 8 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 09 — Redirect trace on paper

**Work:** Tutorial 9.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 9 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 10 — Counter + FutureProvider lab

**Work:** Tutorial 10.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 10 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 11 — homework_pack mapping test

**Work:** Tutorial 11.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 11 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 12 — SQL joins in SQL Editor

**Work:** Tutorial 12.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 12 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 13 — First admin + login

**Work:** Tutorial 13.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 13 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 14 — Trace three .from() calls

**Work:** Tutorial 14.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 14 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 15 — Two-student marks RLS

**Work:** Tutorial 15.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 15 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 16 — Homework file upload

**Work:** Tutorial 16.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 16 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 17 — Two-device chat

**Work:** Tutorial 17.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 17 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 18 — RU + UZ switch

**Work:** Tutorial 18.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 18 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 19 — Four styles × dark

**Work:** Tutorial 19.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 19 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 20 — Assignment notify inbox

**Work:** Tutorial 20.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 20 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 21 — Full auth loop freeze-test

**Work:** Tutorial 21.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 21 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 22 — Pack create → submit → grade

**Work:** Tutorial 22.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 22 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 23 — Reactions + unread

**Work:** Tutorial 23.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 23 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 24 — Attendance unique session

**Work:** Tutorial 24.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 24 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 25 — Announcement + out-of-group deny

**Work:** Tutorial 25.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 25 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 26 — Component inventory

**Work:** Tutorial 26.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 26 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 27 — flutter test + one new unit test

**Work:** Tutorial 27.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 27 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 28 — Network tab on gradebook

**Work:** Tutorial 28.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 28 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

## Day 29 — Airplane mode marks

**Work:** Tutorial 29.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 29 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

# Day 30 — Release APK/web + password rotation plan

**Work:** Tutorial 30.

Morning: read the tutorial without touching the keyboard. Write three questions. Afternoon: do the TASK. Evening: run the CHECKPOINT and paste the proof into your private notes file. If the checkpoint fails, do not start the next day — use Appendix D.

Reflection prompts: What object did you create today (table, widget, provider)? What would a student see if you shipped only through today? What secret did you almost put in git?

> **CHECKPOINT — Day 30 done**
> You have an artifact (file, query, screenshot) dated today. Tomorrow-you can reproduce the step from that artifact without this paragraph.

# Oral exam — 50 questions with answers

Cover the answer. Speak for thirty seconds. Then read the book’s answer and grade yourself.

## Question 1

Why is profiles.id the Auth user id?

> **Model answer**
> RLS uses auth.uid(). If profile ids were random, every policy would need a join. Equality is simpler and harder to get wrong.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 2

Why synthetic emails?

> **Model answer**
> Supabase Auth signInWithPassword requires an email. The school wants usernames. auth_email is the adapter. The RPC hides the adapter from attackers probing disabled accounts.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 3

How do you prevent students from reading all marks?

> **Model answer**
> ENABLE ROW LEVEL SECURITY + SELECT policies that compare student_id to the caller’s students row (or teacher group membership, or is_admin). The Flutter list is not the control.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 4

What is the difference between students.id and profiles.id?

> **Model answer**
> profiles.id is the login identity. students.id is the roster identity used on attendance, marks, submissions, group_students. Mixing them yields empty joins.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 5

Why a junction table for group membership?

> **Model answer**
> A student can belong to more than one group (math and English). group_id on students cannot model that.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 6

Why does creating a group create a chat room?

> **Model answer**
> Trigger ensure_chat_room_for_group keeps a 1:1 invariant. Flutter must not insert chat_rooms ad hoc.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 7

How are unread counts computed?

> **Model answer**
> chat_unread_counts RPC, not a column on chat_rooms. Reads live in message_reads.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 8

Why REPLICA IDENTITY FULL on reactions?

> **Model answer**
> Realtime UPDATE/DELETE payloads need old keys. Default replica identity may omit them.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 9

Where is focus data?

> **Model answer**
> SharedPreferences. There is no focus table. Offline still works.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 10

How do four visual styles work?

> **Model answer**
> AppStyle enum + AppTheme.light/dark(style) + optional backdrops. Persisted as preferred_style.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 11

What happens if you skip first_admin_bootstrap?

> **Model answer**
> admin_create_user requires is_admin(). Chicken and egg. Bootstrap inserts the first admin like the RPC does.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 12

Why revoke seed_demo_data from authenticated?

> **Model answer**
> Otherwise any logged-in student could flood the school with fake users.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 13

How does language gating work?

> **Model answer**
> profiles.preferred_language null → router /language. After save, home opens.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 14

Why Duration.zero on theme animation?

> **Model answer**
> Styles do not interpolate cleanly. Instant switch is a product choice.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 15

What is a homework_pack?

> **Model answer**
> Assignment type with ordered tasks students submit. static is display-only.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 16

How are voice notes stored?

> **Model answer**
> record package → file → Storage chat-voices or submissions voice_url.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 17

How do credential slips work?

> **Model answer**
> Admin screen builds a PDF of usernames/passwords. Optional login_credentials table stores plaintext for reprint — treat as sensitive.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 18

What is academic_years_one_current?

> **Model answer**
> Partial unique index ensuring only one is_current year.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 19

Why maybeSingle on profile fetch?

> **Model answer**
> A signed-in Auth user might lack a profile (partial bootstrap). single() would throw.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 20

How do you test RLS?

> **Model answer**
> Two real JWTs (two app logins), never the SQL Editor postgres role.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 21

What does ShellRoute buy you?

> **Model answer**
> Persistent nav chrome while child routes change. One AppShell, three destination lists.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 22

Why invalidate profile after login?

> **Model answer**
> The FutureProvider may have cached null from the guest session.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 23

What is action_path on notifications?

> **Model answer**
> Deep link for go_router when the student taps the inbox row.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 24

Why jsonb for announcement files?

> **Model answer**
> Variable-length list of URLs without a child table. Trade-off: harder to query individual files.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 25

How is member_count defined?

> **Model answer**
> count(group_students)+1 for the teacher.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

### Question 26

What breaks if you use user_metadata.role in RLS?

> **Model answer**
> Users can edit user_metadata. They would escalate to admin.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 27

Why dart-define-from-file?

> **Model answer**
> Inject secrets at build/run without relying only on an asset .env.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 28

What is _sanitizeKey?

> **Model answer**
> Rejects placeholders and service-role-shaped keys so the app fails closed.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 29

How do you page assignments?

> **Model answer**
> PageResult + limit/offset or cursor in assignment providers + LoadMoreButton.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 30

What is the blast radius of deleting a teacher?

> **Model answer**
> ON DELETE CASCADE on groups.teacher_id removes groups and, cascading, assignments, chat, etc. Know this before delete UI.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 31

Why shared ChatRoomScreen?

> **Model answer**
> Behavior is identical; only basePath changes. Duplicating 1k lines three times is how bugs fork.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 32

How do replies stay in-room?

> **Model answer**
> chat_messages_reply_same_room trigger/function rejects cross-room reply_to.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 33

What is mark weight?

> **Model answer**
> Weighted average: sum(percentage*weight)/sum(weight). Views implement it.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 34

Why four academic terms?

> **Model answer**
> CHECK semester 1–4. Bounds come from academic_semester_bounds.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 35

How does mass generate work?

> **Model answer**
> admin_mass_generate_school_data SECURITY DEFINER after is_admin(). Dev/demo only on real schools.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 36

What is Nature Paper?

> **Model answer**
> An AppStyle with backdrop PNGs and paper décor widgets.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 37

How do you localize a new string?

> **Model answer**
> Add key to app_en.arb, app_ru.arb, app_uz.arb, flutter gen-l10n, use context.l10n.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 38

Why no Forgot password?

> **Model answer**
> Internal emails are not mailboxes. Admin resets via RPC.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 39

What is the web layout rule?

> **Model answer**
> Bottom bar on phones, rail on tablet/web widths. No permanent desktop-only sidebar.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 40

How do you know Realtime is live?

> **Model answer**
> Second device updates without refresh; publication includes the table.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 41

What is CacheService?

> **Model answer**
> SharedPreferences JSON for profile, assignments, notifications, dashboard when the network fails.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 42

Why publishable key vs service role?

> **Model answer**
> Publishable is designed for clients with RLS. Service role bypasses RLS and is a breach if shipped.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 43

How is a disabled user blocked?

> **Model answer**
> RPC filters is_disabled; router also sends /account-disabled if a profile loads as disabled.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 44

What file is the router?

> **Model answer**
> lib/core/router/app_router.dart

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 45

Name three SECURITY DEFINER functions

> **Model answer**
> get_auth_email_by_username, admin_create_user, is_admin (and many admin_* / notify helpers).

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 46

What unique key is on submissions?

> **Model answer**
> (task_id, student_id)

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 47

What unique key is on attendance?

> **Model answer**
> (group_id, student_id, date) and (session_id, student_id)

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 48

How do charts get data?

> **Model answer**
> Aggregates from marks/attendance views or provider-side grouping into Syncfusion series.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 49

What is go_router refreshListenable?

> **Model answer**
> _RouterNotifier that notifyListeners when auth or profile changes so redirect re-runs.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

## Question 50

Why include .env in assets?

> **Model answer**
> dotenv load on devices that are not using dart-define. Still gitignore the file contents in source control.

Follow-up you should also be able to answer: which file or SQL object proves this? If you cannot name one, you memorized a slogan, not the system.

# Generated localization Dart (reference)

These files are produced by flutter gen-l10n. Do not hand-edit. They are printed so you can see how an ARB key becomes a getter. If they are missing, run the generator.

## app_localizations.dart

**lib/l10n/app_localizations.dart**

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:flutter/widgets.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:intl/intl.dart' as intl;

import 'app_localizations_en.dart';
import 'app_localizations_ru.dart';
import 'app_localizations_uz.dart';

// ignore_for_file: type=lint

/// Callers can lookup localized strings with an instance of AppLocalizations
/// returned by `AppLocalizations.of(context)`.
///
/// Applications need to include `AppLocalizations.delegate()` in their app's
/// `localizationDelegates` list, and the locales they support in the app's
/// `supportedLocales` list. For example:
///
/// ```dart
/// import 'l10n/app_localizations.dart';
///
/// return MaterialApp(
///   localizationsDelegates: AppLocalizations.localizationsDelegates,
///   supportedLocales: AppLocalizations.supportedLocales,
///   home: MyApplicationHome(),
/// );
/// ```
///
/// ## Update pubspec.yaml
///
/// Please make sure to update your pubspec.yaml to include the following
/// packages:
///
/// ```yaml
/// dependencies:
///   # Internationalization support.
///   flutter_localizations:
///     sdk: flutter
///   intl: any # Use the pinned version from flutter_localizations
///
///   # Rest of dependencies
/// ```
///
/// ## iOS Applications
///
/// iOS applications define key application metadata, including supported
/// locales, in an Info.plist file that is built into the application bundle.
/// To configure the locales supported by your app, you'll need to edit this
/// file.
///
/// First, open your project's ios/Runner.xcworkspace Xcode workspace file.
/// Then, in the Project Navigator, open the Info.plist file under the Runner
/// project's Runner folder.
///
/// Next, select the Information Property List item, select Add Item from the
/// Editor menu, then select Localizations from the pop-up menu.
///
/// Select and expand the newly-created Localizations item then, for each
/// locale your application supports, add a new item and select the locale
```

```
/// you wish to add from the pop-up menu in the Value field. This list should
/// be consistent with the languages listed in the AppLocalizations.supportedLocales
/// property.
abstract class AppLocalizations {
  AppLocalizations(String locale)
    : localeName = intl.Intl.canonicalizedLocale(locale.toString());

  final String localeName;

  static AppLocalizations of(BuildContext context) {
    return Localizations.of<AppLocalizations>(context, AppLocalizations)!;
  }

  static const LocalizationsDelegate<AppLocalizations> delegate =
      _AppLocalizationsDelegate();

  /// A list of this localizations delegate along with the default localizations
  /// delegates.
  ///
  /// Returns a list of localizations delegates containing this delegate along with
  /// GlobalMaterialLocalizations.delegate, GlobalCupertinoLocalizations.delegate,
  /// and GlobalWidgetsLocalizations.delegate.
  ///
  /// Additional delegates can be added by appending to this list in
  /// MaterialApp. This list does not have to be used at all if a custom list
  /// of delegates is preferred or required.
  static const List<LocalizationsDelegate<dynamic>> localizationsDelegates =
      <LocalizationsDelegate<dynamic>>[
        delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
      ];

  /// A list of this localizations delegate's supported locales.
  static const List<Locale> supportedLocales = <Locale>[
    Locale('en'),
    Locale('ru'),
    Locale('uz'),
  ];

  /// No description provided for @appName.
  ///
  /// In en, this message translates to:
  /// **'MU Study'**
  String get appName;

  /// No description provided for @shellMuStudy.
  ///
  /// In en, this message translates to:
  /// **'MU Study'**
  String get shellMuStudy;

  /// No description provided for @languageTitle.
  ///
  /// In en, this message translates to:
  /// **'Choose your language'**
  String get languageTitle;

  /// No description provided for @languageSubtitle.
  ///
  /// In en, this message translates to:
  /// **'You can change this anytime in Profile.'**
  String get languageSubtitle;

  /// No description provided for @languageEnglish.
  ///
  /// In en, this message translates to:
  /// **'English'**
  String get languageEnglish;

  /// No description provided for @languageRussian.
  ///
```

```
  /// In en, this message translates to:
  /// **'Russian'**
  String get languageRussian;

  /// No description provided for @languageUzbek.
  ///
  /// In en, this message translates to:
  /// **'Uzbek'**
  String get languageUzbek;

  /// No description provided for @languageEnglishNative.
  ///
  /// In en, this message translates to:
  /// **'English'**
  String get languageEnglishNative;

  /// No description provided for @languageRussianNative.
  ///
  /// In en, this message translates to:
  /// **'[redacted]'**
  String get languageRussianNative;

  /// No description provided for @languageUzbekNative.
  ///
  /// In en, this message translates to:
  /// **'O‘zbekcha'**
  String get languageUzbekNative;

  /// No description provided for @languageContinue.
  ///
  /// In en, this message translates to:
  /// **'Continue'**
  String get languageContinue;

  /// No description provided for @languageSection.
  ///
  /// In en, this message translates to:
  /// **'Language'**
  String get languageSection;

  /// No description provided for @languageHint.
  ///
  /// In en, this message translates to:
  /// **'App language'**
  String get languageHint;

  /// No description provided for @loginWelcome.
  ///
  /// In en, this message translates to:
  /// **'Welcome back'**
  String get loginWelcome;

  /// No description provided for @loginSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Sign in to your MU Study workspace'**
  String get loginSubtitle;

  /// No description provided for @loginUsername.
  ///
  /// In en, this message translates to:
  /// **'Username'**
  String get loginUsername;

  /// No description provided for @loginPassword.
  ///
  /// In en, this message translates to:
  /// **'Password'**
  String get loginPassword;

  /// No description provided for @loginSignIn.
  ///
  /// In en, this message translates to:
```

```
  /// **'Sign in'**
  String get loginSignIn;

  /// No description provided for @loginMissingFields.
  ///
  /// In en, this message translates to:
  /// **'Please enter both username and password'**
  String get loginMissingFields;

  /// No description provided for @loginIncorrect.
  ///
  /// In en, this message translates to:
  /// **'Incorrect username or password'**
  String get loginIncorrect;

  /// No description provided for @loginFailed.
  ///
  /// In en, this message translates to:
  /// **'Sign in failed. Check your connection and try again.'**
  String get loginFailed;

  /// No description provided for @loginConfigError.
  ///
  /// In en, this message translates to:
  /// **'Invalid configuration. Contact support.'**
  String get loginConfigError;

  /// No description provided for @loginConfigMissing.
  ///
  /// In en, this message translates to:
  /// **'Configuration missing. Check your .env file.'**
  String get loginConfigMissing;

  /// No description provided for @accountDisabled.
  ///
  /// In en, this message translates to:
  /// **'Account disabled. Contact admin.'**
  String get accountDisabled;

  /// No description provided for @navDashboard.
  ///
  /// In en, this message translates to:
  /// **'Dashboard'**
  String get navDashboard;

  /// No description provided for @navAssignments.
  ///
  /// In en, this message translates to:
  /// **'Assignments'**
  String get navAssignments;

  /// No description provided for @navMarks.
  ///
  /// In en, this message translates to:
  /// **'Marks'**
  String get navMarks;

  /// No description provided for @navAttendance.
  ///
  /// In en, this message translates to:
  /// **'Attendance'**
  String get navAttendance;

  /// No description provided for @navNotifications.
  ///
  /// In en, this message translates to:
  /// **'Notifications'**
  String get navNotifications;

  /// No description provided for @navProfile.
  ///
  /// In en, this message translates to:
  /// **'Profile'**
```

```
String get navProfile;

/// No description provided for @navUsers.
///
/// In en, this message translates to:
/// **'Users'**
String get navUsers;

/// No description provided for @navGroups.
///
/// In en, this message translates to:
/// **'Groups'**
String get navGroups;

/// No description provided for @navAnalytics.
///
/// In en, this message translates to:
/// **'Analytics'**
String get navAnalytics;

/// No description provided for @navGradebook.
///
/// In en, this message translates to:
/// **'Gradebook'**
String get navGradebook;

/// No description provided for @navChat.
///
/// In en, this message translates to:
/// **'Chat'**
String get navChat;

/// No description provided for @navFocus.
///
/// In en, this message translates to:
/// **'Focus'**
String get navFocus;

/// No description provided for @focusTitle.
///
/// In en, this message translates to:
/// **'Focus'**
String get focusTitle;

/// No description provided for @focusSubtitle.
///
/// In en, this message translates to:
/// **'Quiet study time for one assignment at a time.'**
String get focusSubtitle;

/// No description provided for @focusTodaysFocus.
///
/// In en, this message translates to:
/// **'Today’s focus'**
String get focusTodaysFocus;

/// No description provided for @focusTodaysFocusSubtitle.
///
/// In en, this message translates to:
/// **'Up to three open assignments — any due date'**
String get focusTodaysFocusSubtitle;

/// No description provided for @focusAlsoDue.
///
/// In en, this message translates to:
/// **'More homework'**
String get focusAlsoDue;

/// No description provided for @focusContinueHomework.
///
/// In en, this message translates to:
/// **'Continue homework'**
String get focusContinueHomework;
```

```
  /// No description provided for @focusOpenAssignment.
  ///
  /// In en, this message translates to:
  /// **'Open assignment'**
  String get focusOpenAssignment;

  /// No description provided for @focusEmptyTitle.
  ///
  /// In en, this message translates to:
  /// **'No homework yet'**
  String get focusEmptyTitle;

  /// No description provided for @focusEmptySubtitle.
  ///
  /// In en, this message translates to:
  /// **'When your teacher posts an assignment, you can start a focus session here.'**
  String get focusEmptySubtitle;

  /// No description provided for @focusChooseMode.
  ///
  /// In en, this message translates to:
  /// **'How do you want to focus?'**
  String get focusChooseMode;

  /// No description provided for @focusModePomodoro.
  ///
  /// In en, this message translates to:
  /// **'Pomodoro'**
  String get focusModePomodoro;

  /// No description provided for @focusModePomodoroHint.
  ///
  /// In en, this message translates to:
  /// **'25 minutes, then a break'**
  String get focusModePomodoroHint;

  /// No description provided for @focusModeFree.
  ///
  /// In en, this message translates to:
  /// **'Free session'**
  String get focusModeFree;

  /// No description provided for @focusModeFreeHint.
  ///
  /// In en, this message translates to:
  /// **'Count up until you stop'**
  String get focusModeFreeHint;

  /// No description provided for @focusDoNotDisturb.
  ///
  /// In en, this message translates to:
  /// **'Chat muted'**
  String get focusDoNotDisturb;

  /// No description provided for @focusChatMuteHint.
  ///
  /// In en, this message translates to:
  /// **'While a session is active, chat badges stay quiet so you can focus. Notifications
still arrive in the background.'**
  String get focusChatMuteHint;

  /// No description provided for @focusPause.
  ///
  /// In en, this message translates to:
  /// **'Pause'**
  String get focusPause;

  /// No description provided for @focusResume.
  ///
  /// In en, this message translates to:
  /// **'Resume'**
  String get focusResume;
```

```
  /// No description provided for @focusPaused.
  ///
  /// In en, this message translates to:
  /// **'Paused'**
  String get focusPaused;

  /// No description provided for @focusEndSession.
  ///
  /// In en, this message translates to:
  /// **'End'**
  String get focusEndSession;

  /// No description provided for @focusSessionComplete.
  ///
  /// In en, this message translates to:
  /// **'Session complete'**
  String get focusSessionComplete;

  /// No description provided for @focusSessionCompleteSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Nice work. Here’s a quick recap.'**
  String get focusSessionCompleteSubtitle;

  /// No description provided for @focusTimeUp.
  ///
  /// In en, this message translates to:
  /// **'Time’s up!'**
  String get focusTimeUp;

  /// No description provided for @focusTimeUpSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Your Pomodoro block finished. Take a short break, then keep going.'**
  String get focusTimeUpSubtitle;

  /// No description provided for @focusMinutesFocused.
  ///
  /// In en, this message translates to:
  /// **'{duration} focused'**
  String focusMinutesFocused(String duration);

  /// No description provided for @focusOptionalNote.
  ///
  /// In en, this message translates to:
  /// **'Optional note'**
  String get focusOptionalNote;

  /// No description provided for @focusOptionalNoteHint.
  ///
  /// In en, this message translates to:
  /// **'e.g. Stuck on task #3'**
  String get focusOptionalNoteHint;

  /// No description provided for @focusSaveAndOpen.
  ///
  /// In en, this message translates to:
  /// **'Save & open assignment'**
  String get focusSaveAndOpen;
```

**lib/l10n/app_localizations.dart — continued (2/8)**

```
  /// No description provided for @focusDone.
  ///
  /// In en, this message translates to:
  /// **'Done'**
  String get focusDone;

  /// No description provided for @chatTitle.
  ///
  /// In en, this message translates to:
  /// **'Chat'**
  String get chatTitle;
```

```
  /// No description provided for @chatSearchHint.
  ///
  /// In en, this message translates to:
  /// **'Search groups or teachers'**
  String get chatSearchHint;

  /// No description provided for @chatEmptyTitle.
  ///
  /// In en, this message translates to:
  /// **'No chat rooms yet'**
  String get chatEmptyTitle;

  /// No description provided for @chatEmptySubtitle.
  ///
  /// In en, this message translates to:
  /// **'Join a group to start messaging your class.'**
  String get chatEmptySubtitle;

  /// No description provided for @chatNoMessagesYet.
  ///
  /// In en, this message translates to:
  /// **'No messages yet'**
  String get chatNoMessagesYet;

  /// No description provided for @chatMessageDeleted.
  ///
  /// In en, this message translates to:
  /// **'Message deleted'**
  String get chatMessageDeleted;

  /// No description provided for @chatPreviewImage.
  ///
  /// In en, this message translates to:
  /// **'Photo'**
  String get chatPreviewImage;

  /// No description provided for @chatPreviewVideo.
  ///
  /// In en, this message translates to:
  /// **'Video'**
  String get chatPreviewVideo;

  /// No description provided for @chatVideoPlaybackSpeed.
  ///
  /// In en, this message translates to:
  /// **'Playback speed'**
  String get chatVideoPlaybackSpeed;

  /// No description provided for @chatVideoSpeedNormal.
  ///
  /// In en, this message translates to:
  /// **'1x'**
  String get chatVideoSpeedNormal;

  /// No description provided for @chatVideoSpeedValue.
  ///
  /// In en, this message translates to:
  /// **'{speed}x'**
  String chatVideoSpeedValue(double speed);

  /// No description provided for @chatVideoRewind10.
  ///
  /// In en, this message translates to:
  /// **'-10s'**
  String get chatVideoRewind10;

  /// No description provided for @chatVideoForward10.
  ///
  /// In en, this message translates to:
  /// **'+10s'**
  String get chatVideoForward10;

  /// No description provided for @chatVideoCouldNotPlay.
```

```
  ///
  /// In en, this message translates to:
  /// **'Could not play this video'**
  String get chatVideoCouldNotPlay;

  /// No description provided for @chatPreviewVoice.
  ///
  /// In en, this message translates to:
  /// **'Voice message'**
  String get chatPreviewVoice;

  /// No description provided for @chatPreviewFile.
  ///
  /// In en, this message translates to:
  /// **'File'**
  String get chatPreviewFile;

  /// No description provided for @chatOpenFile.
  ///
  /// In en, this message translates to:
  /// **'Tap to open'**
  String get chatOpenFile;

  /// No description provided for @chatOpenFileFailed.
  ///
  /// In en, this message translates to:
  /// **'Could not open this file.'**
  String get chatOpenFileFailed;

  /// No description provided for @chatYou.
  ///
  /// In en, this message translates to:
  /// **'You'**
  String get chatYou;

  /// No description provided for @chatTeacherMembers.
  ///
  /// In en, this message translates to:
  /// **'{teacher} · {count} members'**
  String chatTeacherMembers(String teacher, int count);

  /// No description provided for @chatOnlineCount.
  ///
  /// In en, this message translates to:
  /// **'{count} online'**
  String chatOnlineCount(int count);

  /// No description provided for @chatReact.
  ///
  /// In en, this message translates to:
  /// **'React'**
  String get chatReact;

  /// No description provided for @chatCopyText.
  ///
  /// In en, this message translates to:
  /// **'Copy text'**
  String get chatCopyText;

  /// No description provided for @chatCopied.
  ///
  /// In en, this message translates to:
  /// **'Copied'**
  String get chatCopied;

  /// No description provided for @chatReply.
  ///
  /// In en, this message translates to:
  /// **'Reply'**
  String get chatReply;

  /// No description provided for @chatReplyTo.
  ///
```

```
  /// In en, this message translates to:
  /// **'In reply to {name}'**
  String chatReplyTo(String name);

  /// No description provided for @chatReplyDeleted.
  ///
  /// In en, this message translates to:
  /// **'Deleted message'**
  String get chatReplyDeleted;

  /// No description provided for @chatReplyNotFound.
  ///
  /// In en, this message translates to:
  /// **'Original message isn’t available'**
  String get chatReplyNotFound;

  /// No description provided for @chatMessageHint.
  ///
  /// In en, this message translates to:
  /// **'Message…'**
  String get chatMessageHint;

  /// No description provided for @chatAttachImage.
  ///
  /// In en, this message translates to:
  /// **'Image'**
  String get chatAttachImage;

  /// No description provided for @chatAttachVideo.
  ///
  /// In en, this message translates to:
  /// **'Video'**
  String get chatAttachVideo;

  /// No description provided for @chatAttachFile.
  ///
  /// In en, this message translates to:
  /// **'File'**
  String get chatAttachFile;

  /// No description provided for @chatTyping.
  ///
  /// In en, this message translates to:
  /// **'{names} typing…'**
  String chatTyping(String names);

  /// No description provided for @chatRecording.
  ///
  /// In en, this message translates to:
  /// **'Recording {time}'**
  String chatRecording(String time);

  /// No description provided for @chatUploading.
  ///
  /// In en, this message translates to:
  /// **'Uploading {name}…'**
  String chatUploading(String name);

  /// No description provided for @chatSendFailed.
  ///
  /// In en, this message translates to:
  /// **'Could not send: {error}'**
  String chatSendFailed(String error);

  /// No description provided for @chatMicDenied.
  ///
  /// In en, this message translates to:
  /// **'Microphone permission is required for voice messages.'**
  String get chatMicDenied;

  /// No description provided for @chatEditTitle.
  ///
  /// In en, this message translates to:
```

```
  /// **'Edit message'**
  String get chatEditTitle;

  /// No description provided for @chatDeleteTitle.
  ///
  /// In en, this message translates to:
  /// **'Delete message?'**
  String get chatDeleteTitle;

  /// No description provided for @chatDeleteBody.
  ///
  /// In en, this message translates to:
  /// **'This removes the message for everyone in the room.'**
  String get chatDeleteBody;

  /// No description provided for @chatDeleteSelectedTitle.
  ///
  /// In en, this message translates to:
  /// **'Delete {count} messages?'**
  String chatDeleteSelectedTitle(int count);

  /// No description provided for @chatDeleteSelectedBody.
  ///
  /// In en, this message translates to:
  /// **'Selected messages will be removed for everyone in the room.'**
  String get chatDeleteSelectedBody;

  /// No description provided for @chatSelectedCount.
  ///
  /// In en, this message translates to:
  /// **'{count} selected'**
  String chatSelectedCount(int count);

  /// No description provided for @chatSelectMessages.
  ///
  /// In en, this message translates to:
  /// **'Select messages'**
  String get chatSelectMessages;

  /// No description provided for @chatSelect.
  ///
  /// In en, this message translates to:
  /// **'Select'**
  String get chatSelect;

  /// No description provided for @commonSelectAll.
  ///
  /// In en, this message translates to:
  /// **'Select all'**
  String get commonSelectAll;

  /// No description provided for @chatEdited.
  ///
  /// In en, this message translates to:
  /// **'edited'**
  String get chatEdited;

  /// No description provided for @chatDateToday.
  ///
  /// In en, this message translates to:
  /// **'Today'**
  String get chatDateToday;

  /// No description provided for @chatDateYesterday.
  ///
  /// In en, this message translates to:
  /// **'Yesterday'**
  String get chatDateYesterday;

  /// No description provided for @chatRoleAdmin.
  ///
  /// In en, this message translates to:
  /// **'admin'**
```

```
String get chatRoleAdmin;

/// No description provided for @chatRoleTeacher.
///
/// In en, this message translates to:
/// **'teacher'**
String get chatRoleTeacher;

/// No description provided for @commonSave.
///
/// In en, this message translates to:
/// **'Save'**
String get commonSave;

/// No description provided for @commonCancel.
///
/// In en, this message translates to:
/// **'Cancel'**
String get commonCancel;

/// No description provided for @commonSubmit.
///
/// In en, this message translates to:
/// **'Submit'**
String get commonSubmit;

/// No description provided for @commonDelete.
///
/// In en, this message translates to:
/// **'Delete'**
String get commonDelete;

/// No description provided for @commonEdit.
///
/// In en, this message translates to:
/// **'Edit'**
String get commonEdit;

/// No description provided for @commonClose.
///
/// In en, this message translates to:
/// **'Close'**
String get commonClose;

/// No description provided for @commonRetry.
///
/// In en, this message translates to:
/// **'Retry'**
String get commonRetry;

/// No description provided for @commonTryAgain.
///
/// In en, this message translates to:
/// **'Try again'**
String get commonTryAgain;

/// No description provided for @commonLoading.
///
/// In en, this message translates to:
/// **'Loading…'**
String get commonLoading;

/// No description provided for @commonError.
///
/// In en, this message translates to:
/// **'Something went wrong'**
String get commonError;

/// No description provided for @commonErrorWithDetails.
///
/// In en, this message translates to:
/// **'Error: {error}'**
String commonErrorWithDetails(String error);
```

```
  /// No description provided for @commonCouldNotSave.
  ///
  /// In en, this message translates to:
  /// **'Could not save: {error}'**
  String commonCouldNotSave(String error);

  /// No description provided for @commonSignOut.
  ///
  /// In en, this message translates to:
  /// **'Sign out'**
  String get commonSignOut;

  /// No description provided for @commonSaveDraft.
  ///
  /// In en, this message translates to:
  /// **'Save draft'**
  String get commonSaveDraft;

  /// No description provided for @commonOpen.
  ///
  /// In en, this message translates to:
  /// **'Open'**
  String get commonOpen;

  /// No description provided for @commonCreate.
  ///
  /// In en, this message translates to:
  /// **'Create'**
  String get commonCreate;

  /// No description provided for @commonAssign.
  ///
  /// In en, this message translates to:
  /// **'Assign'**
  String get commonAssign;

  /// No description provided for @commonReset.
  ///
  /// In en, this message translates to:
  /// **'Reset'**
  String get commonReset;

  /// No description provided for @commonAdd.
  ///
  /// In en, this message translates to:
  /// **'Add'**
  String get commonAdd;

  /// No description provided for @commonLoadMore.
  ///
  /// In en, this message translates to:
  /// **'Load more'**
  String get commonLoadMore;

  /// No description provided for @commonExport.
  ///
  /// In en, this message translates to:
  /// **'Export'**
  String get commonExport;

  /// No description provided for @commonExportFailed.
  ///
  /// In en, this message translates to:
  /// **'Could not export: {error}'**
  String commonExportFailed(String error);

  /// No description provided for @marksTermAll.
  ///
  /// In en, this message translates to:
  /// **'All time'**
  String get marksTermAll;

  /// No description provided for @marksTermSemester.
```

```
  ///
  /// In en, this message translates to:
  /// **'{yearRange} · Semester {semester}'**
  String marksTermSemester(String yearRange, int semester);

  /// No description provided for @marksExportCsv.
  ///
  /// In en, this message translates to:
  /// **'Export CSV'**
  String get marksExportCsv;

  /// No description provided for @marksExportDone.
  ///
  /// In en, this message translates to:
  /// **'Marks exported'**
  String get marksExportDone;

  /// No description provided for @commonTeacher.
  ///
  /// In en, this message translates to:
  /// **'Teacher'**
  String get commonTeacher;

  /// No description provided for @commonStudent.
  ///
  /// In en, this message translates to:
  /// **'Student'**
  String get commonStudent;

  /// No description provided for @commonAdmin.
  ///
  /// In en, this message translates to:
  /// **'Administrator'**
  String get commonAdmin;

  /// No description provided for @commonWorkspace.
  ///
  /// In en, this message translates to:
  /// **'Workspace'**
  String get commonWorkspace;

  /// No description provided for @commonNoDueDate.
  ///
  /// In en, this message translates to:
  /// **'No due date'**
  String get commonNoDueDate;

  /// No description provided for @commonDueDate.
  ///
  /// In en, this message translates to:
  /// **'Due {date}'**
  String commonDueDate(String date);

  /// No description provided for @homeworkDueThisWeek.
  ///
  /// In en, this message translates to:
  /// **'Due this week'**
  String get homeworkDueThisWeek;

  /// No description provided for @homeworkDueThisWeekSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Overdue and upcoming deadlines'**
  String get homeworkDueThisWeekSubtitle;

  /// No description provided for @homeworkNoDueThisWeek.
  ///
  /// In en, this message translates to:
  /// **'Nothing due this week'**
  String get homeworkNoDueThisWeek;

  /// No description provided for @homeworkNoDueThisWeekSubtitle.
  ///
```

```
  /// In en, this message translates to:
  /// **'You’re clear until next week’s deadlines.'**
  String get homeworkNoDueThisWeekSubtitle;

  /// No description provided for @homeworkOverdue.
  ///
  /// In en, this message translates to:
  /// **'Overdue · {date}'**
```

**lib/l10n/app_localizations.dart — continued (3/8)**

```
  String homeworkOverdue(String date);

  /// No description provided for @homeworkDueToday.
  ///
  /// In en, this message translates to:
  /// **'Due today'**
  String get homeworkDueToday;

  /// No description provided for @homeworkDueTomorrow.
  ///
  /// In en, this message translates to:
  /// **'Due tomorrow'**
  String get homeworkDueTomorrow;

  /// No description provided for @homeworkDueInDays.
  ///
  /// In en, this message translates to:
  /// **'Due in {days} days'**
  String homeworkDueInDays(int days);

  /// No description provided for @homeworkLateBadge.
  ///
  /// In en, this message translates to:
  /// **'Late'**
  String get homeworkLateBadge;

  /// No description provided for @commonProgress.
  ///
  /// In en, this message translates to:
  /// **'Progress'**
  String get commonProgress;

  /// No description provided for @commonSubject.
  ///
  /// In en, this message translates to:
  /// **'Subject'**
  String get commonSubject;

  /// No description provided for @commonType.
  ///
  /// In en, this message translates to:
  /// **'Type'**
  String get commonType;

  /// No description provided for @commonTitle.
  ///
  /// In en, this message translates to:
  /// **'Title'**
  String get commonTitle;

  /// No description provided for @commonDescription.
  ///
  /// In en, this message translates to:
  /// **'Description'**
  String get commonDescription;

  /// No description provided for @commonGroup.
  ///
  /// In en, this message translates to:
  /// **'Group'**
  String get commonGroup;

  /// No description provided for @commonMessage.
```

```
  ///
  /// In en, this message translates to:
  /// **'Message'**
  String get commonMessage;

  /// No description provided for @commonScore.
  ///
  /// In en, this message translates to:
  /// **'Score'**
  String get commonScore;

  /// No description provided for @commonFeedback.
  ///
  /// In en, this message translates to:
  /// **'Feedback'**
  String get commonFeedback;

  /// No description provided for @commonStudents.
  ///
  /// In en, this message translates to:
  /// **'Students'**
  String get commonStudents;

  /// No description provided for @commonCannotUndo.
  ///
  /// In en, this message translates to:
  /// **'This action cannot be undone.'**
  String get commonCannotUndo;

  /// No description provided for @profileTitle.
  ///
  /// In en, this message translates to:
  /// **'Profile'**
  String get profileTitle;

  /// No description provided for @profileFullName.
  ///
  /// In en, this message translates to:
  /// **'Full name'**
  String get profileFullName;

  /// No description provided for @profileNotSignedIn.
  ///
  /// In en, this message translates to:
  /// **'Not signed in'**
  String get profileNotSignedIn;

  /// No description provided for @profileAccount.
  ///
  /// In en, this message translates to:
  /// **'Account'**
  String get profileAccount;

  /// No description provided for @profileAccountSettings.
  ///
  /// In en, this message translates to:
  /// **'Account Settings'**
  String get profileAccountSettings;

  /// No description provided for @profileSignOutAdmin.
  ///
  /// In en, this message translates to:
  /// **'Sign out of Admin Console'**
  String get profileSignOutAdmin;

  /// No description provided for @profileSettings.
  ///
  /// In en, this message translates to:
  /// **'Settings'**
  String get profileSettings;

  /// No description provided for @profileAppearance.
  ///
```

```
  /// In en, this message translates to:
  /// **'Appearance'**
  String get profileAppearance;

  /// No description provided for @profileThemeLight.
  ///
  /// In en, this message translates to:
  /// **'Light'**
  String get profileThemeLight;

  /// No description provided for @profileThemeDark.
  ///
  /// In en, this message translates to:
  /// **'Dark'**
  String get profileThemeDark;

  /// No description provided for @profileThemeSystem.
  ///
  /// In en, this message translates to:
  /// **'System'**
  String get profileThemeSystem;

  /// No description provided for @profileStyle.
  ///
  /// In en, this message translates to:
  /// **'Style'**
  String get profileStyle;

  /// No description provided for @profileStyleModern.
  ///
  /// In en, this message translates to:
  /// **'Modern Professional'**
  String get profileStyleModern;

  /// No description provided for @profileStyleNature.
  ///
  /// In en, this message translates to:
  /// **'Nature Paper'**
  String get profileStyleNature;

  /// No description provided for @profileStyleSoftNeu.
  ///
  /// In en, this message translates to:
  /// **'Neumorphism'**
  String get profileStyleSoftNeu;

  /// No description provided for @profileStyleMonochrome.
  ///
  /// In en, this message translates to:
  /// **'Monochrome Workspace'**
  String get profileStyleMonochrome;

  /// No description provided for @profileEditName.
  ///
  /// In en, this message translates to:
  /// **'Edit name'**
  String get profileEditName;

  /// No description provided for @profileNameUpdated.
  ///
  /// In en, this message translates to:
  /// **'Name updated'**
  String get profileNameUpdated;

  /// No description provided for @profileChangePhoto.
  ///
  /// In en, this message translates to:
  /// **'Change photo'**
  String get profileChangePhoto;

  /// No description provided for @profilePhotoUpdated.
  ///
  /// In en, this message translates to:
```

```
  /// **'Photo updated'**
  String get profilePhotoUpdated;

  /// No description provided for @profileChangePassword.
  ///
  /// In en, this message translates to:
  /// **'Change password'**
  String get profileChangePassword;

  /// No description provided for @profileCurrentPassword.
  ///
  /// In en, this message translates to:
  /// **'Current password'**
  String get profileCurrentPassword;

  /// No description provided for @profileNewPassword.
  ///
  /// In en, this message translates to:
  /// **'New password'**
  String get profileNewPassword;

  /// No description provided for @profileConfirmPassword.
  ///
  /// In en, this message translates to:
  /// **'Confirm new password'**
  String get profileConfirmPassword;

  /// No description provided for @profilePasswordChanged.
  ///
  /// In en, this message translates to:
  /// **'Password updated'**
  String get profilePasswordChanged;

  /// No description provided for @profilePasswordMismatch.
  ///
  /// In en, this message translates to:
  /// **'New passwords do not match'**
  String get profilePasswordMismatch;

  /// No description provided for @profilePasswordTooShort.
  ///
  /// In en, this message translates to:
  /// **'Use at least 8 characters'**
  String get profilePasswordTooShort;

  /// No description provided for @profilePasswordHint.
  ///
  /// In en, this message translates to:
  /// **'Ask an admin if you forget your password.'**
  String get profilePasswordHint;

  /// No description provided for @profileShortcuts.
  ///
  /// In en, this message translates to:
  /// **'Shortcuts'**
  String get profileShortcuts;

  /// No description provided for @profileYourGroups.
  ///
  /// In en, this message translates to:
  /// **'Your groups'**
  String get profileYourGroups;

  /// No description provided for @profileNoGroups.
  ///
  /// In en, this message translates to:
  /// **'No groups assigned'**
  String get profileNoGroups;

  /// No description provided for @profileGroupsCount.
  ///
  /// In en, this message translates to:
  /// **'{count} groups'**
```

```
  String profileGroupsCount(int count);

  /// No description provided for @profileAverageLabel.
  ///
  /// In en, this message translates to:
  /// **'Term average'**
  String get profileAverageLabel;

  /// No description provided for @profileAttendanceLabel.
  ///
  /// In en, this message translates to:
  /// **'Attendance'**
  String get profileAttendanceLabel;

  /// No description provided for @profileShareProgress.
  ///
  /// In en, this message translates to:
  /// **'Share progress report'**
  String get profileShareProgress;

  /// No description provided for @profileNotifications.
  ///
  /// In en, this message translates to:
  /// **'Notifications'**
  String get profileNotifications;

  /// No description provided for @profileClearCache.
  ///
  /// In en, this message translates to:
  /// **'Clear cached data'**
  String get profileClearCache;

  /// No description provided for @profileClearCacheWarn1Title.
  ///
  /// In en, this message translates to:
  /// **'Clear cached data?'**
  String get profileClearCacheWarn1Title;

  /// No description provided for @profileClearCacheWarn1Body.
  ///
  /// In en, this message translates to:
  /// **'This deletes offline chat messages and downloaded images, videos, voice notes, and
files. You will need an internet connection to load them again.'**
  String get profileClearCacheWarn1Body;

  /// No description provided for @profileClearCacheWarn2Title.
  ///
  /// In en, this message translates to:
  /// **'Are you sure?'**
  String get profileClearCacheWarn2Title;

  /// No description provided for @profileClearCacheWarn2Body.
  ///
  /// In en, this message translates to:
  /// **'This cannot be undone. Cached chat media will be removed from this device.'**
  String get profileClearCacheWarn2Body;

  /// No description provided for @profileClearCacheConfirm.
  ///
  /// In en, this message translates to:
  /// **'Clear cache'**
  String get profileClearCacheConfirm;

  /// No description provided for @profileClearCacheDone.
  ///
  /// In en, this message translates to:
  /// **'Cached data cleared'**
  String get profileClearCacheDone;

  /// No description provided for @profileAppVersion.
  ///
  /// In en, this message translates to:
  /// **'MU Study · v{version}'**
```

```
String profileAppVersion(String version);

/// No description provided for @profileSubject.
///
/// In en, this message translates to:
/// **'Subject'**
String get profileSubject;

/// No description provided for @greetingMorning.
///
/// In en, this message translates to:
/// **'Good Morning'**
String get greetingMorning;

/// No description provided for @greetingAfternoon.
///
/// In en, this message translates to:
/// **'Good Afternoon'**
String get greetingAfternoon;

/// No description provided for @greetingEvening.
///
/// In en, this message translates to:
/// **'Good Evening'**
String get greetingEvening;

/// No description provided for @greetingWithName.
///
/// In en, this message translates to:
/// **'{greeting}, {name}'**
String greetingWithName(String greeting, String name);

/// No description provided for @greetingLoadingTitle.
///
/// In en, this message translates to:
/// **'Good Morning, User'**
String get greetingLoadingTitle;

/// No description provided for @greetingLoadingSubtitle.
///
/// In en, this message translates to:
/// **'Loading your personalized greeting...'**
String get greetingLoadingSubtitle;

/// No description provided for @studentHomeSubtitle.
///
/// In en, this message translates to:
/// **'Your learning space'**
String get studentHomeSubtitle;

/// No description provided for @continueLearning.
///
/// In en, this message translates to:
/// **'Continue Learning'**
String get continueLearning;

/// No description provided for @continueLearningSubtitle.
///
/// In en, this message translates to:
/// **'Pick up where you left off'**
String get continueLearningSubtitle;

/// No description provided for @everythingCompleted.
///
/// In en, this message translates to:
/// **'Everything is completed.'**
String get everythingCompleted;

/// No description provided for @caughtUpToday.
///
/// In en, this message translates to:
/// **'You\'re all caught up for today.'**
String get caughtUpToday;
```

```
  /// No description provided for @homeworkPack.
  ///
  /// In en, this message translates to:
  /// **'Homework Pack'**
  String get homeworkPack;

  /// No description provided for @staticTask.
  ///
  /// In en, this message translates to:
  /// **'Static Task'**
  String get staticTask;

  /// No description provided for @staticAssignment.
  ///
  /// In en, this message translates to:
  /// **'Static'**
  String get staticAssignment;

  /// No description provided for @upcomingAssignments.
  ///
  /// In en, this message translates to:
  /// **'Upcoming Assignments'**
  String get upcomingAssignments;

  /// No description provided for @stayOnTrack.
  ///
  /// In en, this message translates to:
  /// **'Stay on track'**
  String get stayOnTrack;

  /// No description provided for @loadingAnnouncements.
  ///
  /// In en, this message translates to:
  /// **'Loading announcements...'**
  String get loadingAnnouncements;

  /// No description provided for @notificationsCardTitle.
  ///
  /// In en, this message translates to:
  /// **'Notifications'**
  String get notificationsCardTitle;

  /// No description provided for @unreadAlerts.
  ///
  /// In en, this message translates to:
  /// **'You have {count} new alerts'**
  String unreadAlerts(int count);

  /// No description provided for @upToDate.
  ///
  /// In en, this message translates to:
  /// **'You are up to date'**
  String get upToDate;

  /// No description provided for @announcementsTitle.
  ///
  /// In en, this message translates to:
  /// **'Announcements'**
  String get announcementsTitle;

  /// No description provided for @announcementsSubtitle.
  ///
  /// In en, this message translates to:
  /// **'From your teachers'**
  String get announcementsSubtitle;

  /// No description provided for @announcementsEmpty.
  ///
  /// In en, this message translates to:
  /// **'No announcements'**
  String get announcementsEmpty;

  /// No description provided for @announcementsEmptyTeacher.
```

```
  ///
  /// In en, this message translates to:
  /// **'Share important updates with your students.'**
  String get announcementsEmptyTeacher;
```

**lib/l10n/app_localizations.dart — continued (4/8)**

```
  /// No description provided for @announcementsEmptyAdmin.
  ///
  /// In en, this message translates to:
  /// **'Platform-wide announcements will appear here.'**
  String get announcementsEmptyAdmin;

  /// No description provided for @announcementsReadMore.
  ///
  /// In en, this message translates to:
  /// **'Read more'**
  String get announcementsReadMore;

  /// No description provided for @announcementsShowLess.
  ///
  /// In en, this message translates to:
  /// **'Show less'**
  String get announcementsShowLess;

  /// No description provided for @announcementsViewAll.
  ///
  /// In en, this message translates to:
  /// **'View all ({count})'**
  String announcementsViewAll(int count);

  /// No description provided for @announcementsAll.
  ///
  /// In en, this message translates to:
  /// **'All Announcements'**
  String get announcementsAll;

  /// No description provided for @announcementsNew.
  ///
  /// In en, this message translates to:
  /// **'New Announcement'**
  String get announcementsNew;

  /// No description provided for @announcementsPost.
  ///
  /// In en, this message translates to:
  /// **'Post Announcement'**
  String get announcementsPost;

  /// No description provided for @announcementsEdit.
  ///
  /// In en, this message translates to:
  /// **'Edit Announcement'**
  String get announcementsEdit;

  /// No description provided for @announcementsDeleteTitle.
  ///
  /// In en, this message translates to:
  /// **'Delete announcement?'**
  String get announcementsDeleteTitle;

  /// No description provided for @announcementsDeleteBody.
  ///
  /// In en, this message translates to:
  /// **'It will be removed for everyone in the group.'**
  String get announcementsDeleteBody;

  /// No description provided for @announcementsByAdmin.
  ///
  /// In en, this message translates to:
  /// **'Administration'**
  String get announcementsByAdmin;

  /// No description provided for @announcementsShareUpdates.
```

```
  ///
  /// In en, this message translates to:
  /// **'Share updates with your groups'**
  String get announcementsShareUpdates;

  /// No description provided for @assignmentsTitle.
  ///
  /// In en, this message translates to:
  /// **'Assignments'**
  String get assignmentsTitle;

  /// No description provided for @assignmentsEmpty.
  ///
  /// In en, this message translates to:
  /// **'No assignments'**
  String get assignmentsEmpty;

  /// No description provided for @assignmentsEmptySubtitle.
  ///
  /// In en, this message translates to:
  /// **'You are all caught up!'**
  String get assignmentsEmptySubtitle;

  /// No description provided for @assignmentsEmptyTeacher.
  ///
  /// In en, this message translates to:
  /// **'Create your first assignment to get started.'**
  String get assignmentsEmptyTeacher;

  /// No description provided for @assignmentsLearningPath.
  ///
  /// In en, this message translates to:
  /// **'Learning Path'**
  String get assignmentsLearningPath;

  /// No description provided for @assignmentsActive.
  ///
  /// In en, this message translates to:
  /// **'Active assignments'**
  String get assignmentsActive;

  /// No description provided for @assignmentDetails.
  ///
  /// In en, this message translates to:
  /// **'Assignment Details'**
  String get assignmentDetails;

  /// No description provided for @assignmentNotFound.
  ///
  /// In en, this message translates to:
  /// **'Assignment not found'**
  String get assignmentNotFound;

  /// No description provided for @assignmentTasks.
  ///
  /// In en, this message translates to:
  /// **'Tasks'**
  String get assignmentTasks;

  /// No description provided for @assignmentTasksSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Complete all to finish pack'**
  String get assignmentTasksSubtitle;

  /// No description provided for @assignmentNew.
  ///
  /// In en, this message translates to:
  /// **'New Assignment'**
  String get assignmentNew;

  /// No description provided for @assignmentManage.
  ///
```

```
  /// In en, this message translates to:
  /// **'Manage Assignments'**
  String get assignmentManage;

  /// No description provided for @assignmentManageSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Create static or homework packs'**
  String get assignmentManageSubtitle;

  /// No description provided for @marksTitle.
  ///
  /// In en, this message translates to:
  /// **'My Marks'**
  String get marksTitle;

  /// No description provided for @marksOverallAverage.
  ///
  /// In en, this message translates to:
  /// **'Overall average'**
  String get marksOverallAverage;

  /// No description provided for @marksNoneYet.
  ///
  /// In en, this message translates to:
  /// **'No marks recorded yet.'**
  String get marksNoneYet;

  /// No description provided for @marksBasedOn.
  ///
  /// In en, this message translates to:
  /// **'{label} · based on {count} marks.'**
  String marksBasedOn(String label, int count);

  /// No description provided for @marksByCategory.
  ///
  /// In en, this message translates to:
  /// **'By category'**
  String get marksByCategory;

  /// No description provided for @marksWhereStrongest.
  ///
  /// In en, this message translates to:
  /// **'Where you are strongest'**
  String get marksWhereStrongest;

  /// No description provided for @marksAll.
  ///
  /// In en, this message translates to:
  /// **'All marks'**
  String get marksAll;

  /// No description provided for @marksNewestFirst.
  ///
  /// In en, this message translates to:
  /// **'Newest first'**
  String get marksNewestFirst;

  /// No description provided for @marksEmptyTitle.
  ///
  /// In en, this message translates to:
  /// **'No marks yet'**
  String get marksEmptyTitle;

  /// No description provided for @marksEmptySubtitle.
  ///
  /// In en, this message translates to:
  /// **'Your teacher\'s marks will appear here.'**
  String get marksEmptySubtitle;

  /// No description provided for @progressReportTitle.
  ///
  /// In en, this message translates to:
```

```
  /// **'Progress report'**
  String get progressReportTitle;

  /// No description provided for @progressReportShare.
  ///
  /// In en, this message translates to:
  /// **'Share progress'**
  String get progressReportShare;

  /// No description provided for @progressReportSharing.
  ///
  /// In en, this message translates to:
  /// **'Preparing report…'**
  String get progressReportSharing;

  /// No description provided for @progressReportDone.
  ///
  /// In en, this message translates to:
  /// **'Progress report ready to share'**
  String get progressReportDone;

  /// No description provided for @progressReportFailed.
  ///
  /// In en, this message translates to:
  /// **'Could not share progress: {error}'**
  String progressReportFailed(String error);

  /// No description provided for @progressReportNeedGroup.
  ///
  /// In en, this message translates to:
  /// **'Join a group to share a progress report.'**
  String get progressReportNeedGroup;

  /// No description provided for @progressReportAttendance.
  ///
  /// In en, this message translates to:
  /// **'Attendance'**
  String get progressReportAttendance;

  /// No description provided for @progressReportAttendanceRate.
  ///
  /// In en, this message translates to:
  /// **'Attendance rate'**
  String get progressReportAttendanceRate;

  /// No description provided for @progressReportNoAttendance.
  ///
  /// In en, this message translates to:
  /// **'No attendance recorded for this term.'**
  String get progressReportNoAttendance;

  /// No description provided for @progressReportSessions.
  ///
  /// In en, this message translates to:
  /// **'sessions'**
  String get progressReportSessions;

  /// No description provided for @progressReportMarksCounted.
  ///
  /// In en, this message translates to:
  /// **'marks'**
  String get progressReportMarksCounted;

  /// No description provided for @progressReportRecentMarks.
  ///
  /// In en, this message translates to:
  /// **'Recent marks'**
  String get progressReportRecentMarks;

  /// No description provided for @progressReportGeneratedOn.
  ///
  /// In en, this message translates to:
  /// **'Generated {date}'**
```

```
  String progressReportGeneratedOn(String date);

  /// No description provided for @marksNoRecorded.
  ///
  /// In en, this message translates to:
  /// **'No marks recorded'**
  String get marksNoRecorded;

  /// No description provided for @marksNoRecordedSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Use \"Grade class\" to enter marks in one pass.'**
  String get marksNoRecordedSubtitle;

  /// No description provided for @marksRecent.
  ///
  /// In en, this message translates to:
  /// **'Recent marks'**
  String get marksRecent;

  /// No description provided for @marksRecentSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Latest entries across the group'**
  String get marksRecentSubtitle;

  /// No description provided for @marksStudentsHeader.
  ///
  /// In en, this message translates to:
  /// **'Students'**
  String get marksStudentsHeader;

  /// No description provided for @marksStudentsSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Weighted average of all marks'**
  String get marksStudentsSubtitle;

  /// No description provided for @marksNoStudents.
  ///
  /// In en, this message translates to:
  /// **'No students in this group'**
  String get marksNoStudents;

  /// No description provided for @marksNoStudentsSubtitle.
  ///
  /// In en, this message translates to:
  /// **'An admin can assign students to your group.'**
  String get marksNoStudentsSubtitle;

  /// No description provided for @marksDeleteTitle.
  ///
  /// In en, this message translates to:
  /// **'Delete mark?'**
  String get marksDeleteTitle;

  /// No description provided for @marksDeleteBody.
  ///
  /// In en, this message translates to:
  /// **'\"{title}\" will be removed permanently.'**
  String marksDeleteBody(String title);

  /// No description provided for @marksGradeTheClass.
  ///
  /// In en, this message translates to:
  /// **'Grade the class'**
  String get marksGradeTheClass;

  /// No description provided for @marksOnePercentEach.
  ///
  /// In en, this message translates to:
  /// **'One percent mark per student'**
  String get marksOnePercentEach;
```

```
  /// No description provided for @marksSaveMarks.
  ///
  /// In en, this message translates to:
  /// **'Save marks'**
  String get marksSaveMarks;

  /// No description provided for @marksSaveMark.
  ///
  /// In en, this message translates to:
  /// **'Save mark'**
  String get marksSaveMark;

  /// No description provided for @marksAddMark.
  ///
  /// In en, this message translates to:
  /// **'Add mark'**
  String get marksAddMark;

  /// No description provided for @marksNothingYet.
  ///
  /// In en, this message translates to:
  /// **'Nothing recorded yet'**
  String get marksNothingYet;

  /// No description provided for @marksAddFirst.
  ///
  /// In en, this message translates to:
  /// **'Add the first mark for this student.'**
  String get marksAddFirst;

  /// No description provided for @marksSavedCount.
  ///
  /// In en, this message translates to:
  /// **'Saved {count} marks'**
  String marksSavedCount(int count);

  /// No description provided for @marksGroupHasNoStudents.
  ///
  /// In en, this message translates to:
  /// **'This group has no students yet.'**
  String get marksGroupHasNoStudents;

  /// No description provided for @marksTitleHint.
  ///
  /// In en, this message translates to:
  /// **'e.g. Unit 3 test'**
  String get marksTitleHint;

  /// No description provided for @marksAverageLine.
  ///
  /// In en, this message translates to:
  /// **'Average {percent} · {label}'**
  String marksAverageLine(String percent, String label);

  /// No description provided for @marksNoMarksYetShort.
  ///
  /// In en, this message translates to:
  /// **'No marks yet'**
  String get marksNoMarksYetShort;

  /// No description provided for @marksSummaryStudents.
  ///
  /// In en, this message translates to:
  /// **'Students'**
  String get marksSummaryStudents;

  /// No description provided for @marksSummaryMarks.
  ///
  /// In en, this message translates to:
  /// **'Marks'**
  String get marksSummaryMarks;

  /// No description provided for @attendanceTitle.
```

```
  ///
  /// In en, this message translates to:
  /// **'Attendance'**
  String get attendanceTitle;

  /// No description provided for @attendanceEmpty.
  ///
  /// In en, this message translates to:
  /// **'No attendance records'**
  String get attendanceEmpty;

  /// No description provided for @attendanceNoGroup.
  ///
  /// In en, this message translates to:
  /// **'No group assigned yet'**
  String get attendanceNoGroup;

  /// No description provided for @attendanceMark.
  ///
  /// In en, this message translates to:
  /// **'Mark Attendance'**
  String get attendanceMark;

  /// No description provided for @attendanceDailyTracking.
  ///
  /// In en, this message translates to:
  /// **'Daily student tracking'**
  String get attendanceDailyTracking;

  /// No description provided for @attendanceSaved.
  ///
  /// In en, this message translates to:
  /// **'Attendance saved successfully'**
  String get attendanceSaved;

  /// No description provided for @attendanceNoGroups.
  ///
  /// In en, this message translates to:
  /// **'No groups found'**
  String get attendanceNoGroups;

  /// No description provided for @attendancePresent.
  ///
  /// In en, this message translates to:
  /// **'P — Present'**
  String get attendancePresent;

  /// No description provided for @attendanceLate.
  ///
  /// In en, this message translates to:
  /// **'L — Late'**
  String get attendanceLate;

  /// No description provided for @attendanceAbsent.
  ///
  /// In en, this message translates to:
  /// **'A — Absent'**
  String get attendanceAbsent;
```

**lib/l10n/app_localizations.dart — continued (5/8)**

```
  /// No description provided for @attendanceExcused.
  ///
  /// In en, this message translates to:
  /// **'E — Excused'**
  String get attendanceExcused;

  /// No description provided for @attendancePresentShort.
  ///
  /// In en, this message translates to:
  /// **'P'**
  String get attendancePresentShort;

  /// No description provided for @attendanceLateShort.
```

```
  ///
  /// In en, this message translates to:
  /// **'L'**
  String get attendanceLateShort;

  /// No description provided for @attendanceAbsentShort.
  ///
  /// In en, this message translates to:
  /// **'A'**
  String get attendanceAbsentShort;

  /// No description provided for @attendanceExcusedShort.
  ///
  /// In en, this message translates to:
  /// **'E'**
  String get attendanceExcusedShort;

  /// No description provided for @attendanceTrend.
  ///
  /// In en, this message translates to:
  /// **'Attendance Trend'**
  String get attendanceTrend;

  /// No description provided for @attendanceOverall.
  ///
  /// In en, this message translates to:
  /// **'Overall distribution'**
  String get attendanceOverall;

  /// No description provided for @analyticsThisWeek.
  ///
  /// In en, this message translates to:
  /// **'This Week'**
  String get analyticsThisWeek;

  /// No description provided for @analyticsLastWeek.
  ///
  /// In en, this message translates to:
  /// **'Last Week'**
  String get analyticsLastWeek;

  /// No description provided for @analyticsTotalMarks.
  ///
  /// In en, this message translates to:
  /// **'Total Marks: {count}'**
  String analyticsTotalMarks(int count);

  /// No description provided for @notificationsTitle.
  ///
  /// In en, this message translates to:
  /// **'Notifications'**
  String get notificationsTitle;

  /// No description provided for @notificationsEmpty.
  ///
  /// In en, this message translates to:
  /// **'All caught up!'**
  String get notificationsEmpty;

  /// No description provided for @notificationsEmptySubtitle.
  ///
  /// In en, this message translates to:
  /// **'No new notifications.'**
  String get notificationsEmptySubtitle;

  /// No description provided for @notificationsNone.
  ///
  /// In en, this message translates to:
  /// **'No notifications'**
  String get notificationsNone;

  /// No description provided for @notifKindAssignment.
  ///
```

```
  /// In en, this message translates to:
  /// **'New homework'**
  String get notifKindAssignment;

  /// No description provided for @notifKindAnnouncement.
  ///
  /// In en, this message translates to:
  /// **'Announcement'**
  String get notifKindAnnouncement;

  /// No description provided for @notifKindGraded.
  ///
  /// In en, this message translates to:
  /// **'Work graded'**
  String get notifKindGraded;

  /// No description provided for @notifKindSubmission.
  ///
  /// In en, this message translates to:
  /// **'New submission'**
  String get notifKindSubmission;

  /// No description provided for @notifKindChat.
  ///
  /// In en, this message translates to:
  /// **'Chat message'**
  String get notifKindChat;

  /// No description provided for @notifKindSystem.
  ///
  /// In en, this message translates to:
  /// **'Update'**
  String get notifKindSystem;

  /// No description provided for @notificationsOpen.
  ///
  /// In en, this message translates to:
  /// **'Open'**
  String get notificationsOpen;

  /// No description provided for @taskTitle.
  ///
  /// In en, this message translates to:
  /// **'Task'**
  String get taskTitle;

  /// No description provided for @taskYourAnswer.
  ///
  /// In en, this message translates to:
  /// **'Your answer'**
  String get taskYourAnswer;

  /// No description provided for @taskMarkCompleted.
  ///
  /// In en, this message translates to:
  /// **'Mark completed'**
  String get taskMarkCompleted;

  /// No description provided for @taskAttachment.
  ///
  /// In en, this message translates to:
  /// **'Attachment'**
  String get taskAttachment;

  /// No description provided for @taskAttachmentHint.
  ///
  /// In en, this message translates to:
  /// **'Add a photo or document to your answer.'**
  String get taskAttachmentHint;

  /// No description provided for @taskAttachmentHintReady.
  ///
  /// In en, this message translates to:
```

```
  /// **'Tap to open · trash to remove.'**
  String get taskAttachmentHintReady;

  /// No description provided for @taskImage.
  ///
  /// In en, this message translates to:
  /// **'Image'**
  String get taskImage;

  /// No description provided for @taskImageSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Gallery photo'**
  String get taskImageSubtitle;

  /// No description provided for @taskFile.
  ///
  /// In en, this message translates to:
  /// **'File'**
  String get taskFile;

  /// No description provided for @taskFileSubtitle.
  ///
  /// In en, this message translates to:
  /// **'PDF or document'**
  String get taskFileSubtitle;

  /// No description provided for @taskReplaceImage.
  ///
  /// In en, this message translates to:
  /// **'Replace with image'**
  String get taskReplaceImage;

  /// No description provided for @taskReplaceFile.
  ///
  /// In en, this message translates to:
  /// **'Replace file'**
  String get taskReplaceFile;

  /// No description provided for @taskVoice.
  ///
  /// In en, this message translates to:
  /// **'Voice record'**
  String get taskVoice;

  /// No description provided for @taskVoiceHint.
  ///
  /// In en, this message translates to:
  /// **'Optional spoken answer — record, play back, then save.'**
  String get taskVoiceHint;

  /// No description provided for @taskViewPdf.
  ///
  /// In en, this message translates to:
  /// **'View PDF'**
  String get taskViewPdf;

  /// No description provided for @taskOpenLink.
  ///
  /// In en, this message translates to:
  /// **'Open link'**
  String get taskOpenLink;

  /// No description provided for @taskSubmitted.
  ///
  /// In en, this message translates to:
  /// **'Submitted'**
  String get taskSubmitted;

  /// No description provided for @taskSaved.
  ///
  /// In en, this message translates to:
  /// **'Saved'**
```

```
String get taskSaved;

/// No description provided for @taskUploadFailed.
///
/// In en, this message translates to:
/// **'Upload failed: {error}'**
String taskUploadFailed(String error);

/// No description provided for @taskVoiceUploadFailed.
///
/// In en, this message translates to:
/// **'Voice upload failed: {error}'**
String taskVoiceUploadFailed(String error);

/// No description provided for @taskCouldNotSave.
///
/// In en, this message translates to:
/// **'Could not save: {error}'**
String taskCouldNotSave(String error);

/// No description provided for @voiceTapToRecord.
///
/// In en, this message translates to:
/// **'Tap the mic to record your answer'**
String get voiceTapToRecord;

/// No description provided for @voiceRecording.
///
/// In en, this message translates to:
/// **'Recording…'**
String get voiceRecording;

/// No description provided for @voiceNote.
///
/// In en, this message translates to:
/// **'Voice note'**
String get voiceNote;

/// No description provided for @voiceTapPlay.
///
/// In en, this message translates to:
/// **'Tap play to listen'**
String get voiceTapPlay;

/// No description provided for @voiceRerecord.
///
/// In en, this message translates to:
/// **'Re-record'**
String get voiceRerecord;

/// No description provided for @voiceStudentAnswer.
///
/// In en, this message translates to:
/// **'Student voice answer'**
String get voiceStudentAnswer;

/// No description provided for @voiceMicPermission.
///
/// In en, this message translates to:
/// **'Microphone permission is required to record.'**
String get voiceMicPermission;

/// No description provided for @voiceCouldNotStart.
///
/// In en, this message translates to:
/// **'Could not start recording: {error}'**
String voiceCouldNotStart(String error);

/// No description provided for @voiceCouldNotSave.
///
/// In en, this message translates to:
/// **'Could not save recording: {error}'**
String voiceCouldNotSave(String error);
```

```
  /// No description provided for @voiceCouldNotPlay.
  ///
  /// In en, this message translates to:
  /// **'Could not play voice note: {error}'**
  String voiceCouldNotPlay(String error);

  /// No description provided for @attachmentCouldNotOpen.
  ///
  /// In en, this message translates to:
  /// **'Could not open this file.'**
  String get attachmentCouldNotOpen;

  /// No description provided for @attachmentImage.
  ///
  /// In en, this message translates to:
  /// **'Image attachment'**
  String get attachmentImage;

  /// No description provided for @attachmentFile.
  ///
  /// In en, this message translates to:
  /// **'File attachment'**
  String get attachmentFile;

  /// No description provided for @attachmentPdf.
  ///
  /// In en, this message translates to:
  /// **'PDF document'**
  String get attachmentPdf;

  /// No description provided for @attachmentTapPreview.
  ///
  /// In en, this message translates to:
  /// **'Tap to preview'**
  String get attachmentTapPreview;

  /// No description provided for @attachmentTapOpen.
  ///
  /// In en, this message translates to:
  /// **'Tap to open'**
  String get attachmentTapOpen;

  /// No description provided for @pdfDefaultTitle.
  ///
  /// In en, this message translates to:
  /// **'PDF'**
  String get pdfDefaultTitle;

  /// No description provided for @teacherWorkspace.
  ///
  /// In en, this message translates to:
  /// **'Teacher Workspace'**
  String get teacherWorkspace;

  /// No description provided for @teacherQuickActions.
  ///
  /// In en, this message translates to:
  /// **'Quick Actions'**
  String get teacherQuickActions;

  /// No description provided for @teacherCreateItems.
  ///
  /// In en, this message translates to:
  /// **'Create new items'**
  String get teacherCreateItems;

  /// No description provided for @teacherNewAssignment.
  ///
  /// In en, this message translates to:
  /// **'New Assignment'**
  String get teacherNewAssignment;

  /// No description provided for @teacherNewAnnouncement.
```

```
  ///
  /// In en, this message translates to:
  /// **'New Announcement'**
  String get teacherNewAnnouncement;

  /// No description provided for @teacherSubmissions.
  ///
  /// In en, this message translates to:
  /// **'Submissions'**
  String get teacherSubmissions;

  /// No description provided for @teacherNoSubmissions.
  ///
  /// In en, this message translates to:
  /// **'No submissions yet'**
  String get teacherNoSubmissions;

  /// No description provided for @teacherNoSubmissionsSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Student work will show up here for grading.'**
  String get teacherNoSubmissionsSubtitle;

  /// No description provided for @teacherGradebook.
  ///
  /// In en, this message translates to:
  /// **'Gradebook'**
  String get teacherGradebook;

  /// No description provided for @teacherGradeClass.
  ///
  /// In en, this message translates to:
  /// **'Grade class'**
  String get teacherGradeClass;

  /// No description provided for @teacherNoGroups.
  ///
  /// In en, this message translates to:
  /// **'No groups yet'**
  String get teacherNoGroups;

  /// No description provided for @teacherNoGroupsSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Ask an admin to assign you a group.'**
  String get teacherNoGroupsSubtitle;

  /// No description provided for @teacherActiveGroups.
  ///
  /// In en, this message translates to:
  /// **'ACTIVE GROUPS'**
  String get teacherActiveGroups;

  /// No description provided for @teacherSubject.
  ///
  /// In en, this message translates to:
  /// **'SUBJECT'**
  String get teacherSubject;

  /// No description provided for @teacherDailyTasks.
  ///
  /// In en, this message translates to:
  /// **'Daily Tasks'**
  String get teacherDailyTasks;

  /// No description provided for @teacherNeedsAttention.
  ///
  /// In en, this message translates to:
  /// **'What needs your attention'**
  String get teacherNeedsAttention;

  /// No description provided for @teacherReviewSubmissions.
  ///
```

```
  /// In en, this message translates to:
  /// **'Review Submissions'**
  String get teacherReviewSubmissions;

  /// No description provided for @teacherCheckAssignments.
  ///
  /// In en, this message translates to:
  /// **'Check student assignments'**
  String get teacherCheckAssignments;

  /// No description provided for @teacherRecordMarks.
  ///
  /// In en, this message translates to:
  /// **'Record marks in percent'**
  String get teacherRecordMarks;

  /// No description provided for @teacherPerformanceAnalytics.
  ///
  /// In en, this message translates to:
  /// **'Performance Analytics'**
  String get teacherPerformanceAnalytics;

  /// No description provided for @teacherTrackProgress.
  ///
  /// In en, this message translates to:
  /// **'Track group progress'**
  String get teacherTrackProgress;

  /// No description provided for @teacherOpenAttachment.
  ///
  /// In en, this message translates to:
  /// **'Open attachment'**
  String get teacherOpenAttachment;

  /// No description provided for @teacherScoreHelper.
  ///
  /// In en, this message translates to:
  /// **'Adds a homework mark to the gradebook'**
  String get teacherScoreHelper;

  /// No description provided for @teacherScoreRange.
  ///
  /// In en, this message translates to:
  /// **'Score must be between 0 and 100.'**
  String get teacherScoreRange;

  /// No description provided for @teacherNoWrittenAnswer.
  ///
  /// In en, this message translates to:
  /// **'No written answer.'**
  String get teacherNoWrittenAnswer;

  /// No description provided for @teacherReviewSubmission.
  ///
  /// In en, this message translates to:
  /// **'Review submission'**
```

**lib/l10n/app_localizations.dart — continued (6/8)**

```
  String get teacherReviewSubmission;

  /// No description provided for @teacherHomeworkPackTasks.
  ///
  /// In en, this message translates to:
  /// **'Homework Pack Tasks'**
  String get teacherHomeworkPackTasks;

  /// No description provided for @teacherAddTask.
  ///
  /// In en, this message translates to:
  /// **'Add Task'**
  String get teacherAddTask;

  /// No description provided for @teacherEditTask.
```

```
  ///
  /// In en, this message translates to:
  /// **'Edit Task'**
  String get teacherEditTask;

  /// No description provided for @teacherDeleteTask.
  ///
  /// In en, this message translates to:
  /// **'Delete Task?'**
  String get teacherDeleteTask;

  /// No description provided for @teacherTasksContent.
  ///
  /// In en, this message translates to:
  /// **'Tasks Content'**
  String get teacherTasksContent;

  /// No description provided for @teacherManagePack.
  ///
  /// In en, this message translates to:
  /// **'Manage items in this pack'**
  String get teacherManagePack;

  /// No description provided for @teacherTaskTitleHint.
  ///
  /// In en, this message translates to:
  /// **'e.g. Reading Exercise'**
  String get teacherTaskTitleHint;

  /// No description provided for @teacherLinkHint.
  ///
  /// In en, this message translates to:
  /// **'https://...'**
  String get teacherLinkHint;

  /// No description provided for @teacherAttachmentHint.
  ///
  /// In en, this message translates to:
  /// **'Image or file link...'**
  String get teacherAttachmentHint;

  /// No description provided for @teacherNoData.
  ///
  /// In en, this message translates to:
  /// **'No data available'**
  String get teacherNoData;

  /// No description provided for @teacherMarkDistribution.
  ///
  /// In en, this message translates to:
  /// **'Mark Distribution'**
  String get teacherMarkDistribution;

  /// No description provided for @teacherStudentsByAverage.
  ///
  /// In en, this message translates to:
  /// **'Students grouped by average'**
  String get teacherStudentsByAverage;

  /// No description provided for @teacherNoMarksTrends.
  ///
  /// In en, this message translates to:
  /// **'Add marks in the gradebook to see trends.'**
  String get teacherNoMarksTrends;

  /// No description provided for @teacherTargetGroup.
  ///
  /// In en, this message translates to:
  /// **'Target Group'**
  String get teacherTargetGroup;

  /// No description provided for @adminWorkspace.
  ///
```

```
  /// In en, this message translates to:
  /// **'Workspace Control'**
  String get adminWorkspace;

  /// No description provided for @adminUsers.
  ///
  /// In en, this message translates to:
  /// **'User Management'**
  String get adminUsers;

  /// No description provided for @adminGroups.
  ///
  /// In en, this message translates to:
  /// **'Groups'**
  String get adminGroups;

  /// No description provided for @adminAnalytics.
  ///
  /// In en, this message translates to:
  /// **'Analytics'**
  String get adminAnalytics;

  /// No description provided for @adminTeachers.
  ///
  /// In en, this message translates to:
  /// **'TEACHERS'**
  String get adminTeachers;

  /// No description provided for @adminStudents.
  ///
  /// In en, this message translates to:
  /// **'STUDENTS'**
  String get adminStudents;

  /// No description provided for @adminGroupsLabel.
  ///
  /// In en, this message translates to:
  /// **'GROUPS'**
  String get adminGroupsLabel;

  /// No description provided for @adminMarksLabel.
  ///
  /// In en, this message translates to:
  /// **'MARKS'**
  String get adminMarksLabel;

  /// No description provided for @adminActivityInsights.
  ///
  /// In en, this message translates to:
  /// **'Activity Insights'**
  String get adminActivityInsights;

  /// No description provided for @adminCurrentMetrics.
  ///
  /// In en, this message translates to:
  /// **'Current metrics'**
  String get adminCurrentMetrics;

  /// No description provided for @adminCreateUser.
  ///
  /// In en, this message translates to:
  /// **'Create New User'**
  String get adminCreateUser;

  /// No description provided for @adminCreateUserButton.
  ///
  /// In en, this message translates to:
  /// **'Create User'**
  String get adminCreateUserButton;

  /// No description provided for @adminUserCreated.
  ///
  /// In en, this message translates to:
```

```
  /// **'User created successfully'**
  String get adminUserCreated;

  /// No description provided for @adminResetPassword.
  ///
  /// In en, this message translates to:
  /// **'Reset password'**
  String get adminResetPassword;

  /// No description provided for @adminResetPasswordTitle.
  ///
  /// In en, this message translates to:
  /// **'Reset Password · {username}'**
  String adminResetPasswordTitle(String username);

  /// No description provided for @adminCreateGroup.
  ///
  /// In en, this message translates to:
  /// **'Create New Group'**
  String get adminCreateGroup;

  /// No description provided for @adminCreateGroupButton.
  ///
  /// In en, this message translates to:
  /// **'Create Group'**
  String get adminCreateGroupButton;

  /// No description provided for @adminGroupTitleHint.
  ///
  /// In en, this message translates to:
  /// **'e.g. Group 4'**
  String get adminGroupTitleHint;

  /// No description provided for @adminGroupTitleExists.
  ///
  /// In en, this message translates to:
  /// **'This teacher already has a group with that name'**
  String get adminGroupTitleExists;

  /// No description provided for @adminAssignStudent.
  ///
  /// In en, this message translates to:
  /// **'Assign Student'**
  String get adminAssignStudent;

  /// No description provided for @adminActivityDistribution.
  ///
  /// In en, this message translates to:
  /// **'Activity Distribution'**
  String get adminActivityDistribution;

  /// No description provided for @adminGlobalMetrics.
  ///
  /// In en, this message translates to:
  /// **'Global platform metrics'**
  String get adminGlobalMetrics;

  /// No description provided for @adminActivityLog.
  ///
  /// In en, this message translates to:
  /// **'Admin Activity Log'**
  String get adminActivityLog;

  /// No description provided for @adminActivityLogSubtitle.
  ///
  /// In en, this message translates to:
  /// **'System-wide events and audit logs appear here.'**
  String get adminActivityLogSubtitle;

  /// No description provided for @adminTeacherLine.
  ///
  /// In en, this message translates to:
  /// **'Teacher: {name}'**
```

```
  String adminTeacherLine(String name);

  /// No description provided for @adminRoleLine.
  ///
  /// In en, this message translates to:
  /// **'{role} · {username}'**
  String adminRoleLine(String role, String username);

  /// No description provided for @adminDisable.
  ///
  /// In en, this message translates to:
  /// **'Disable'**
  String get adminDisable;

  /// No description provided for @adminEnable.
  ///
  /// In en, this message translates to:
  /// **'Enable'**
  String get adminEnable;

  /// No description provided for @adminPassword.
  ///
  /// In en, this message translates to:
  /// **'Password'**
  String get adminPassword;

  /// No description provided for @adminFullName.
  ///
  /// In en, this message translates to:
  /// **'Full name'**
  String get adminFullName;

  /// No description provided for @adminRole.
  ///
  /// In en, this message translates to:
  /// **'Role'**
  String get adminRole;

  /// No description provided for @adminSphere.
  ///
  /// In en, this message translates to:
  /// **'Sphere'**
  String get adminSphere;

  /// No description provided for @percentExcellent.
  ///
  /// In en, this message translates to:
  /// **'Excellent'**
  String get percentExcellent;

  /// No description provided for @percentGood.
  ///
  /// In en, this message translates to:
  /// **'Good'**
  String get percentGood;

  /// No description provided for @percentNeedsWork.
  ///
  /// In en, this message translates to:
  /// **'Needs work'**
  String get percentNeedsWork;

  /// No description provided for @percentAtRisk.
  ///
  /// In en, this message translates to:
  /// **'At risk'**
  String get percentAtRisk;

  /// No description provided for @emptyStateDefault.
  ///
  /// In en, this message translates to:
  /// **'Nothing here yet'**
  String get emptyStateDefault;
```

```
  /// No description provided for @setupRequiredTitle.
  ///
  /// In en, this message translates to:
  /// **'Setup required'**
  String get setupRequiredTitle;

  /// No description provided for @setupRequiredBody.
  ///
  /// In en, this message translates to:
  /// **'Supabase is not configured. Add SUPABASE_URL and SUPABASE_ANON_KEY to your .env file.
'**
  String get setupRequiredBody;

  /// No description provided for @setupOption1.
  ///
  /// In en, this message translates to:
  /// **'Option 1 — local .env file'**
  String get setupOption1;

  /// No description provided for @setupOption2.
  ///
  /// In en, this message translates to:
  /// **'Option 2 — dart-define'**
  String get setupOption2;

  /// No description provided for @attendanceAllPresent.
  ///
  /// In en, this message translates to:
  /// **'All P'**
  String get attendanceAllPresent;

  /// No description provided for @attendanceAllAbsent.
  ///
  /// In en, this message translates to:
  /// **'All A'**
  String get attendanceAllAbsent;

  /// No description provided for @attendanceAllLate.
  ///
  /// In en, this message translates to:
  /// **'All L'**
  String get attendanceAllLate;

  /// No description provided for @attendanceAllExcused.
  ///
  /// In en, this message translates to:
  /// **'All E'**
  String get attendanceAllExcused;

  /// No description provided for @attendancePresentWord.
  ///
  /// In en, this message translates to:
  /// **'Present'**
  String get attendancePresentWord;

  /// No description provided for @attendanceAbsentWord.
  ///
  /// In en, this message translates to:
  /// **'Absent'**
  String get attendanceAbsentWord;

  /// No description provided for @attendanceLateWord.
  ///
  /// In en, this message translates to:
  /// **'Late'**
  String get attendanceLateWord;

  /// No description provided for @attendanceExcusedWord.
  ///
  /// In en, this message translates to:
  /// **'Excused'**
  String get attendanceExcusedWord;
```

```
  /// No description provided for @attendanceWeekdayMon.
  ///
  /// In en, this message translates to:
  /// **'Mon'**
  String get attendanceWeekdayMon;

  /// No description provided for @attendanceWeekdayTue.
  ///
  /// In en, this message translates to:
  /// **'Tue'**
  String get attendanceWeekdayTue;

  /// No description provided for @attendanceWeekdayWed.
  ///
  /// In en, this message translates to:
  /// **'Wed'**
  String get attendanceWeekdayWed;

  /// No description provided for @attendanceWeekdayThu.
  ///
  /// In en, this message translates to:
  /// **'Thu'**
  String get attendanceWeekdayThu;

  /// No description provided for @attendanceWeekdayFri.
  ///
  /// In en, this message translates to:
  /// **'Fri'**
  String get attendanceWeekdayFri;

  /// No description provided for @attendancePercentLabel.
  ///
  /// In en, this message translates to:
  /// **'Rate'**
  String get attendancePercentLabel;

  /// No description provided for @attendanceNoLesson.
  ///
  /// In en, this message translates to:
  /// **'No lesson'**
  String get attendanceNoLesson;

  /// No description provided for @attendanceHistory.
  ///
  /// In en, this message translates to:
  /// **'Attendance history'**
  String get attendanceHistory;

  /// No description provided for @attendanceSelectStudent.
  ///
  /// In en, this message translates to:
  /// **'Student'**
  String get attendanceSelectStudent;

  /// No description provided for @attendanceAllStudents.
  ///
  /// In en, this message translates to:
  /// **'All students'**
  String get attendanceAllStudents;

  /// No description provided for @attendanceTapToCycle.
  ///
  /// In en, this message translates to:
  /// **'Tap a day to change status'**
  String get attendanceTapToCycle;

  /// No description provided for @attendanceUpdated.
  ///
  /// In en, this message translates to:
  /// **'Attendance updated'**
  String get attendanceUpdated;

  /// No description provided for @attendanceCalendarEmpty.
```

```
  ///
  /// In en, this message translates to:
  /// **'No attendance in this month'**
  String get attendanceCalendarEmpty;

  /// No description provided for @attendanceFutureNotAllowed.
  ///
  /// In en, this message translates to:
  /// **'You can’t mark attendance for a future day'**
  String get attendanceFutureNotAllowed;

  /// No description provided for @attendanceWeekendNotAllowed.
  ///
  /// In en, this message translates to:
  /// **'Attendance can only be marked on weekdays (Mon–Fri)'**
  String get attendanceWeekendNotAllowed;

  /// No description provided for @chartBandExcellent.
  ///
  /// In en, this message translates to:
  /// **'85–100%'**
  String get chartBandExcellent;

  /// No description provided for @chartBandGood.
  ///
  /// In en, this message translates to:
  /// **'70–84%'**
  String get chartBandGood;

  /// No description provided for @chartBandNeedsWork.
  ///
  /// In en, this message translates to:
  /// **'50–69%'**
  String get chartBandNeedsWork;

  /// No description provided for @chartBandAtRisk.
  ///
  /// In en, this message translates to:
  /// **'Below 50%'**
  String get chartBandAtRisk;

  /// No description provided for @marksWeight.
  ///
  /// In en, this message translates to:
  /// **'Weight'**
  String get marksWeight;

  /// No description provided for @marksCategory.
  ///
  /// In en, this message translates to:
  /// **'Category'**
  String get marksCategory;

  /// No description provided for @marksDate.
  ///
  /// In en, this message translates to:
  /// **'Date'**
  String get marksDate;

  /// No description provided for @marksCommentOptional.
  ///
  /// In en, this message translates to:
```

**lib/l10n/app_localizations.dart — continued (7/8)**

```
  /// **'Comment (optional)'**
  String get marksCommentOptional;

  /// No description provided for @marksNewMark.
  ///
  /// In en, this message translates to:
  /// **'New mark'**
  String get marksNewMark;
```

```
  /// No description provided for @marksEditMark.
  ///
  /// In en, this message translates to:
  /// **'Edit mark'**
  String get marksEditMark;

  /// No description provided for @marksFromSubmission.
  ///
  /// In en, this message translates to:
  /// **'From a graded submission'**
  String get marksFromSubmission;

  /// No description provided for @marksNeedTitle.
  ///
  /// In en, this message translates to:
  /// **'Give the graded item a title.'**
  String get marksNeedTitle;

  /// No description provided for @marksWeightPositive.
  ///
  /// In en, this message translates to:
  /// **'Weight must be greater than zero.'**
  String get marksWeightPositive;

  /// No description provided for @marksNeedAtLeastOne.
  ///
  /// In en, this message translates to:
  /// **'Enter a mark for at least one student.'**
  String get marksNeedAtLeastOne;

  /// No description provided for @marksLeaveBlankSkip.
  ///
  /// In en, this message translates to:
  /// **'Leave a field blank to skip that student.'**
  String get marksLeaveBlankSkip;

  /// No description provided for @marksFromSubmissionDelete.
  ///
  /// In en, this message translates to:
  /// **'This mark comes from a graded submission. Clear the score in Submissions to remove it.
'**
  String get marksFromSubmissionDelete;

  /// No description provided for @marksFromSubmissionEdit.
  ///
  /// In en, this message translates to:
  /// **'This mark mirrors a graded submission. Editing it here will be overwritten the next
time the submission is re-graded.'**
  String get marksFromSubmissionEdit;

  /// No description provided for @categoryHomework.
  ///
  /// In en, this message translates to:
  /// **'Homework'**
  String get categoryHomework;

  /// No description provided for @categoryClasswork.
  ///
  /// In en, this message translates to:
  /// **'Classwork'**
  String get categoryClasswork;

  /// No description provided for @categoryQuiz.
  ///
  /// In en, this message translates to:
  /// **'Quiz'**
  String get categoryQuiz;

  /// No description provided for @categoryTest.
  ///
  /// In en, this message translates to:
  /// **'Test'**
  String get categoryTest;
```

```
  /// No description provided for @categoryExam.
  ///
  /// In en, this message translates to:
  /// **'Exam'**
  String get categoryExam;

  /// No description provided for @categoryProject.
  ///
  /// In en, this message translates to:
  /// **'Project'**
  String get categoryProject;

  /// No description provided for @categoryOral.
  ///
  /// In en, this message translates to:
  /// **'Oral'**
  String get categoryOral;

  /// No description provided for @categoryOther.
  ///
  /// In en, this message translates to:
  /// **'Other'**
  String get categoryOther;

  /// No description provided for @teacherReferenceLinkOptional.
  ///
  /// In en, this message translates to:
  /// **'Reference Link (optional)'**
  String get teacherReferenceLinkOptional;

  /// No description provided for @teacherAttachmentUrlOptional.
  ///
  /// In en, this message translates to:
  /// **'Attachment URL (optional)'**
  String get teacherAttachmentUrlOptional;

  /// No description provided for @teacherNoTasksYet.
  ///
  /// In en, this message translates to:
  /// **'No tasks added yet. Click \"Add Task\" to get started.'**
  String get teacherNoTasksYet;

  /// No description provided for @teacherTemplates.
  ///
  /// In en, this message translates to:
  /// **'Templates'**
  String get teacherTemplates;

  /// No description provided for @teacherTemplatesEmpty.
  ///
  /// In en, this message translates to:
  /// **'No templates yet'**
  String get teacherTemplatesEmpty;

  /// No description provided for @teacherTemplatesEmptySubtitle.
  ///
  /// In en, this message translates to:
  /// **'Save an assignment as a template to reuse it later.'**
  String get teacherTemplatesEmptySubtitle;

  /// No description provided for @teacherSaveAsTemplate.
  ///
  /// In en, this message translates to:
  /// **'Save as template'**
  String get teacherSaveAsTemplate;

  /// No description provided for @teacherTemplateSaved.
  ///
  /// In en, this message translates to:
  /// **'Template saved'**
  String get teacherTemplateSaved;

  /// No description provided for @teacherUseTemplate.
```

```
  ///
  /// In en, this message translates to:
  /// **'Use template'**
  String get teacherUseTemplate;

  /// No description provided for @teacherFromTemplate.
  ///
  /// In en, this message translates to:
  /// **'From template'**
  String get teacherFromTemplate;

  /// No description provided for @teacherDeleteTemplate.
  ///
  /// In en, this message translates to:
  /// **'Delete template?'**
  String get teacherDeleteTemplate;

  /// No description provided for @teacherDeleteTemplateBody.
  ///
  /// In en, this message translates to:
  /// **'\"{title}\" will be removed.'**
  String teacherDeleteTemplateBody(String title);

  /// No description provided for @teacherDuplicateAssignment.
  ///
  /// In en, this message translates to:
  /// **'Duplicate'**
  String get teacherDuplicateAssignment;

  /// No description provided for @teacherDuplicateTitle.
  ///
  /// In en, this message translates to:
  /// **'Duplicate assignment'**
  String get teacherDuplicateTitle;

  /// No description provided for @teacherDuplicated.
  ///
  /// In en, this message translates to:
  /// **'Assignment duplicated'**
  String get teacherDuplicated;

  /// No description provided for @assignmentEdit.
  ///
  /// In en, this message translates to:
  /// **'Edit assignment'**
  String get assignmentEdit;

  /// No description provided for @assignmentDeleteTitle.
  ///
  /// In en, this message translates to:
  /// **'Delete assignment?'**
  String get assignmentDeleteTitle;

  /// No description provided for @assignmentDeleteBody.
  ///
  /// In en, this message translates to:
  /// **'This permanently removes the assignment, its tasks, and student submissions.'**
  String get assignmentDeleteBody;

  /// No description provided for @assignmentDeleted.
  ///
  /// In en, this message translates to:
  /// **'Assignment deleted'**
  String get assignmentDeleted;

  /// No description provided for @assignmentTypeHintStatic.
  ///
  /// In en, this message translates to:
  /// **'A single piece of work. Students submit once — no extra tasks.'**
  String get assignmentTypeHintStatic;

  /// No description provided for @assignmentTypeHintPack.
  ///
```

```
  /// In en, this message translates to:
  /// **'A pack of multiple tasks (workbook-style). You'll add items next.'**
  String get assignmentTypeHintPack;

  /// No description provided for @teacherCreateFromTemplate.
  ///
  /// In en, this message translates to:
  /// **'Create from template'**
  String get teacherCreateFromTemplate;

  /// No description provided for @teacherTasksCopied.
  ///
  /// In en, this message translates to:
  /// **'{count} tasks included'**
  String teacherTasksCopied(int count);

  /// No description provided for @teacherRecentFeedback.
  ///
  /// In en, this message translates to:
  /// **'Reuse feedback'**
  String get teacherRecentFeedback;

  /// No description provided for @teacherApplyFeedbackToOthers.
  ///
  /// In en, this message translates to:
  /// **'Also apply to other pending for this task'**
  String get teacherApplyFeedbackToOthers;

  /// No description provided for @teacherFeedbackAppliedCount.
  ///
  /// In en, this message translates to:
  /// **'Feedback applied to {count} other submissions'**
  String teacherFeedbackAppliedCount(int count);

  /// No description provided for @teacherSharedComment.
  ///
  /// In en, this message translates to:
  /// **'Shared comment (optional)'**
  String get teacherSharedComment;

  /// No description provided for @teacherSharedCommentHint.
  ///
  /// In en, this message translates to:
  /// **'Same note on every mark you save'**
  String get teacherSharedCommentHint;

  /// No description provided for @adminPlatformOverview.
  ///
  /// In en, this message translates to:
  /// **'Platform Overview'**
  String get adminPlatformOverview;

  /// No description provided for @adminAssignTeacher.
  ///
  /// In en, this message translates to:
  /// **'Assign Teacher'**
  String get adminAssignTeacher;

  /// No description provided for @adminSelectStudent.
  ///
  /// In en, this message translates to:
  /// **'Select Student'**
  String get adminSelectStudent;

  /// No description provided for @adminInitialGroupOptional.
  ///
  /// In en, this message translates to:
  /// **'Initial Group (Optional)'**
  String get adminInitialGroupOptional;

  /// No description provided for @adminChartTasks.
  ///
  /// In en, this message translates to:
```

```
  /// **'Tasks'**
  String get adminChartTasks;

  /// No description provided for @adminChartGroups.
  ///
  /// In en, this message translates to:
  /// **'Groups'**
  String get adminChartGroups;

  /// No description provided for @adminChartStaff.
  ///
  /// In en, this message translates to:
  /// **'Staff'**
  String get adminChartStaff;

  /// No description provided for @adminChartAssignments.
  ///
  /// In en, this message translates to:
  /// **'Assignments'**
  String get adminChartAssignments;

  /// No description provided for @adminChartMarks.
  ///
  /// In en, this message translates to:
  /// **'Marks'**
  String get adminChartMarks;

  /// No description provided for @adminChartStudents.
  ///
  /// In en, this message translates to:
  /// **'Students'**
  String get adminChartStudents;

  /// No description provided for @adminAcademicYears.
  ///
  /// In en, this message translates to:
  /// **'Academic years'**
  String get adminAcademicYears;

  /// No description provided for @adminAcademicYearsSubtitle.
  ///
  /// In en, this message translates to:
  /// **'School years and semesters for the gradebook'**
  String get adminAcademicYearsSubtitle;

  /// No description provided for @adminAcademicYearsTitle.
  ///
  /// In en, this message translates to:
  /// **'Academic calendar'**
  String get adminAcademicYearsTitle;

  /// No description provided for @adminCreateNextYear.
  ///
  /// In en, this message translates to:
  /// **'Create next year'**
  String get adminCreateNextYear;

  /// No description provided for @adminYearCreated.
  ///
  /// In en, this message translates to:
  /// **'Created {label}'**
  String adminYearCreated(String label);

  /// No description provided for @adminSetCurrentYear.
  ///
  /// In en, this message translates to:
  /// **'Set as current'**
  String get adminSetCurrentYear;

  /// No description provided for @adminCurrentYearBadge.
  ///
  /// In en, this message translates to:
  /// **'Current'**
```

```
  String get adminCurrentYearBadge;

  /// No description provided for @adminYearSemesters.
  ///
  /// In en, this message translates to:
  /// **'Four semesters · Aug → Jun'**
  String get adminYearSemesters;

  /// No description provided for @adminSemesterRange.
  ///
  /// In en, this message translates to:
  /// **'Semester {semester} · {start} – {end}'**
  String adminSemesterRange(int semester, String start, String end);

  /// No description provided for @adminEditTerm.
  ///
  /// In en, this message translates to:
  /// **'Edit term'**
  String get adminEditTerm;

  /// No description provided for @adminEditTermTitle.
  ///
  /// In en, this message translates to:
  /// **'Edit semester {semester}'**
  String adminEditTermTitle(int semester);

  /// No description provided for @adminTermStartsOn.
  ///
  /// In en, this message translates to:
  /// **'Starts on'**
  String get adminTermStartsOn;

  /// No description provided for @adminTermEndsOn.
  ///
  /// In en, this message translates to:
  /// **'Ends on'**
  String get adminTermEndsOn;

  /// No description provided for @adminTermDatesInvalid.
  ///
  /// In en, this message translates to:
  /// **'End date must be on or after the start date'**
  String get adminTermDatesInvalid;

  /// No description provided for @adminTermUpdated.
  ///
  /// In en, this message translates to:
  /// **'Term dates updated'**
  String get adminTermUpdated;

  /// No description provided for @adminDeleteYear.
  ///
  /// In en, this message translates to:
  /// **'Delete year'**
  String get adminDeleteYear;

  /// No description provided for @adminDeleteYearTitle.
  ///
  /// In en, this message translates to:
  /// **'Delete {label}?'**
  String adminDeleteYearTitle(String label);

  /// No description provided for @adminDeleteYearBody.
  ///
  /// In en, this message translates to:
  /// **'This removes the year and its four terms from the calendar. Marks are not deleted. To
confirm, type {label} below.'**
  String adminDeleteYearBody(String label);

  /// No description provided for @adminDeleteYearConfirmHint.
  ///
  /// In en, this message translates to:
  /// **'Type {label} to confirm'**
```

```
  String adminDeleteYearConfirmHint(String label);

  /// No description provided for @adminDeleteYearCurrentBlocked.
  ///
  /// In en, this message translates to:
  /// **'Set another year as current before deleting this one.'**
  String get adminDeleteYearCurrentBlocked;

  /// No description provided for @adminYearDeleted.
  ///
  /// In en, this message translates to:
  /// **'Deleted {label}'**
  String adminYearDeleted(String label);

  /// No description provided for @adminNoAcademicYears.
  ///
  /// In en, this message translates to:
  /// **'No academic years yet'**
  String get adminNoAcademicYears;

  /// No description provided for @adminAcademicYearsHint.
  ///
  /// In en, this message translates to:
  /// **'Create the next school year when you are ready. Teachers and students pick semesters
in the gradebook. Tap a term to edit its dates.'**
```

**lib/l10n/app_localizations.dart — continued (8/8)**

```
  String get adminAcademicYearsHint;

  /// No description provided for @adminCredentialPrintTitle.
  ///
  /// In en, this message translates to:
  /// **'Print login slips'**
  String get adminCredentialPrintTitle;

  /// No description provided for @adminCredentialPrintSubtitle.
  ///
  /// In en, this message translates to:
  /// **'Unique usernames and passwords from the database, 3 slips per page'**
  String get adminCredentialPrintSubtitle;

  /// No description provided for @adminCredentialPasteLabel.
  ///
  /// In en, this message translates to:
  /// **'Users'**
  String get adminCredentialPasteLabel;

  /// No description provided for @adminCredentialPasteHint.
  ///
  /// In en, this message translates to:
  /// **'username|password|full name\n@james|Abc123xyz|James English|teacher\
n@john_jones|Pass456|John Jones'**
  String get adminCredentialPasteHint;

  /// No description provided for @adminCredentialPerPage.
  ///
  /// In en, this message translates to:
  /// **'Slips per page'**
  String get adminCredentialPerPage;

  /// No description provided for @adminCredentialTwo.
  ///
  /// In en, this message translates to:
  /// **'2'**
  String get adminCredentialTwo;

  /// No description provided for @adminCredentialThree.
  ///
  /// In en, this message translates to:
  /// **'3'**
  String get adminCredentialThree;

  /// No description provided for @adminCredentialExportPdf.
```

```
  ///
  /// In en, this message translates to:
  /// **'Export PDF'**
  String get adminCredentialExportPdf;

  /// No description provided for @adminCredentialSlipTitle.
  ///
  /// In en, this message translates to:
  /// **'Your MU Study login'**
  String get adminCredentialSlipTitle;

  /// No description provided for @adminCredentialSlipHint.
  ///
  /// In en, this message translates to:
  /// **'Keep this paper private. Change your password after first sign-in.'**
  String get adminCredentialSlipHint;

  /// No description provided for @adminCredentialNeedUsers.
  ///
  /// In en, this message translates to:
  /// **'No users found to print'**
  String get adminCredentialNeedUsers;

  /// No description provided for @adminCredentialNeedSql.
  ///
  /// In en, this message translates to:
  /// **'Run sql/admin_login_slips.sql in the Supabase SQL Editor, then export again.'**
  String get adminCredentialNeedSql;

  /// No description provided for @adminCredentialConfidential.
  ///
  /// In en, this message translates to:
  /// **'Private and confidential. These credentials are intended only for the assigned student.
'**
  String get adminCredentialConfidential;

  /// No description provided for @adminFirstName.
  ///
  /// In en, this message translates to:
  /// **'First name'**
  String get adminFirstName;

  /// No description provided for @adminLastName.
  ///
  /// In en, this message translates to:
  /// **'Last name'**
  String get adminLastName;

  /// No description provided for @adminCredentialStudents.
  ///
  /// In en, this message translates to:
  /// **'Students'**
  String get adminCredentialStudents;

  /// No description provided for @adminCredentialTeachers.
  ///
  /// In en, this message translates to:
  /// **'Teachers'**
  String get adminCredentialTeachers;

  /// No description provided for @adminCredentialAllGroups.
  ///
  /// In en, this message translates to:
  /// **'All groups'**
  String get adminCredentialAllGroups;

  /// No description provided for @sphereMath.
  ///
  /// In en, this message translates to:
  /// **'Math'**
  String get sphereMath;

  /// No description provided for @sphereEnglish.
```

```
  ///
  /// In en, this message translates to:
  /// **'English'**
  String get sphereEnglish;

  /// No description provided for @dailyMotivation1.
  ///
  /// In en, this message translates to:
  /// **'Today is a great day to make progress.'**
  String get dailyMotivation1;

  /// No description provided for @dailyMotivation2.
  ///
  /// In en, this message translates to:
  /// **'Ready to continue where you left off?'**
  String get dailyMotivation2;

  /// No description provided for @dailyMotivation3.
  ///
  /// In en, this message translates to:
  /// **'Let\'s conquer your goals today.'**
  String get dailyMotivation3;

  /// No description provided for @dailyMotivation4.
  ///
  /// In en, this message translates to:
  /// **'Consistency is the key to mastery.'**
  String get dailyMotivation4;
}

class _AppLocalizationsDelegate
    extends LocalizationsDelegate<AppLocalizations> {
  const _AppLocalizationsDelegate();

  @override
  Future<AppLocalizations> load(Locale locale) {
    return SynchronousFuture<AppLocalizations>(lookupAppLocalizations(locale));
  }

  @override
  bool isSupported(Locale locale) =>
      <String>['en', 'ru', 'uz'].contains(locale.languageCode);

  @override
  bool shouldReload(_AppLocalizationsDelegate old) => false;
}

AppLocalizations lookupAppLocalizations(Locale locale) {
  // Lookup logic when only language code is specified.
  switch (locale.languageCode) {
    case 'en':
      return AppLocalizationsEn();
    case 'ru':
      return AppLocalizationsRu();
    case 'uz':
      return AppLocalizationsUz();
  }

  throw FlutterError(
    'AppLocalizations.delegate failed to load unsupported locale "$locale". This is likely '
    'an issue with the localizations generation tool. Please file an issue '
    'on GitHub with a reproducible sample app and the gen-l10n configuration '
    'that was used.',
  );
}
```

# app_localizations_en.dart

**lib/l10n/app_localizations_en.dart**

```
// ignore: unused_import
import 'package:intl/intl.dart' as intl;
import 'app_localizations.dart';

// ignore_for_file: type=lint

/// The translations for English (`en`).
class AppLocalizationsEn extends AppLocalizations {
  AppLocalizationsEn([String locale = 'en']) : super(locale);

  @override
  String get appName => 'MU Study';

  @override
  String get shellMuStudy => 'MU Study';

  @override
  String get languageTitle => 'Choose your language';

  @override
  String get languageSubtitle => 'You can change this anytime in Profile.';

  @override
  String get languageEnglish => 'English';

  @override
  String get languageRussian => 'Russian';

  @override
  String get languageUzbek => 'Uzbek';

  @override
  String get languageEnglishNative => 'English';

  @override
  String get languageRussianNative => '[redacted]';

  @override
  String get languageUzbekNative => 'O‘zbekcha';

  @override
  String get languageContinue => 'Continue';

  @override
  String get languageSection => 'Language';

  @override
  String get languageHint => 'App language';

  @override
  String get loginWelcome => 'Welcome back';

  @override
  String get loginSubtitle => 'Sign in to your MU Study workspace';

  @override
  String get loginUsername => 'Username';

  @override
  String get loginPassword => 'Password';

  @override
  String get loginSignIn => 'Sign in';

  @override
  String get loginMissingFields => 'Please enter both username and password';

  @override
  String get loginIncorrect => 'Incorrect username or password';
```

```
  @override
  String get loginFailed =>
      'Sign in failed. Check your connection and try again.';

  @override
  String get loginConfigError => 'Invalid configuration. Contact support.';

  @override
  String get loginConfigMissing =>
      'Configuration missing. Check your .env file.';

  @override
  String get accountDisabled => 'Account disabled. Contact admin.';

  @override
  String get navDashboard => 'Dashboard';

  @override
  String get navAssignments => 'Assignments';

  @override
  String get navMarks => 'Marks';

  @override
  String get navAttendance => 'Attendance';

  @override
  String get navNotifications => 'Notifications';

  @override
  String get navProfile => 'Profile';

  @override
  String get navUsers => 'Users';

  @override
  String get navGroups => 'Groups';

  @override
  String get navAnalytics => 'Analytics';

  @override
  String get navGradebook => 'Gradebook';

  @override
  String get navChat => 'Chat';

  @override
  String get navFocus => 'Focus';

  @override
  String get focusTitle => 'Focus';

  @override
  String get focusSubtitle => 'Quiet study time for one assignment at a time.';

  @override
  String get focusTodaysFocus => 'Today’s focus';

  @override
  String get focusTodaysFocusSubtitle =>
      'Up to three open assignments — any due date';

  @override
  String get focusAlsoDue => 'More homework';

  @override
  String get focusContinueHomework => 'Continue homework';

  @override
  String get focusOpenAssignment => 'Open assignment';

  @override
```

```
  String get focusEmptyTitle => 'No homework yet';

  @override
  String get focusEmptySubtitle =>
      'When your teacher posts an assignment, you can start a focus session here.';

  @override
  String get focusChooseMode => 'How do you want to focus?';

  @override
  String get focusModePomodoro => 'Pomodoro';

  @override
  String get focusModePomodoroHint => '25 minutes, then a break';

  @override
  String get focusModeFree => 'Free session';

  @override
  String get focusModeFreeHint => 'Count up until you stop';

  @override
  String get focusDoNotDisturb => 'Chat muted';

  @override
  String get focusChatMuteHint =>
      'While a session is active, chat badges stay quiet so you can focus. Notifications still
arrive in the background.';

  @override
  String get focusPause => 'Pause';

  @override
  String get focusResume => 'Resume';

  @override
  String get focusPaused => 'Paused';

  @override
  String get focusEndSession => 'End';

  @override
  String get focusSessionComplete => 'Session complete';

  @override
  String get focusSessionCompleteSubtitle => 'Nice work. Here’s a quick recap.';

  @override
  String get focusTimeUp => 'Time’s up!';

  @override
  String get focusTimeUpSubtitle =>
      'Your Pomodoro block finished. Take a short break, then keep going.';

  @override
  String focusMinutesFocused(String duration) {
    return '$duration focused';
  }

  @override
  String get focusOptionalNote => 'Optional note';

  @override
  String get focusOptionalNoteHint => 'e.g. Stuck on task #3';

  @override
  String get focusSaveAndOpen => 'Save & open assignment';

  @override
  String get focusDone => 'Done';

  @override
  String get chatTitle => 'Chat';
```

```
@override
String get chatSearchHint => 'Search groups or teachers';

@override
String get chatEmptyTitle => 'No chat rooms yet';

@override
String get chatEmptySubtitle => 'Join a group to start messaging your class.';

@override
String get chatNoMessagesYet => 'No messages yet';

@override
String get chatMessageDeleted => 'Message deleted';

@override
String get chatPreviewImage => 'Photo';

@override
String get chatPreviewVideo => 'Video';

@override
String get chatVideoPlaybackSpeed => 'Playback speed';

@override
String get chatVideoSpeedNormal => '1x';

@override
String chatVideoSpeedValue(double speed) {
  final intl.NumberFormat speedNumberFormat =
      intl.NumberFormat.decimalPattern(localeName);
  final String speedString = speedNumberFormat.format(speed);

  return '${speedString}x';
}

@override
String get chatVideoRewind10 => '-10s';

@override
String get chatVideoForward10 => '+10s';

@override
String get chatVideoCouldNotPlay => 'Could not play this video';

@override
String get chatPreviewVoice => 'Voice message';

@override
String get chatPreviewFile => 'File';

@override
String get chatOpenFile => 'Tap to open';

@override
String get chatOpenFileFailed => 'Could not open this file.';

@override
String get chatYou => 'You';

@override
String chatTeacherMembers(String teacher, int count) {
  return '$teacher · $count members';
}

@override
String chatOnlineCount(int count) {
  return '$count online';
}

@override
String get chatReact => 'React';
```

```
@override
String get chatCopyText => 'Copy text';

@override
String get chatCopied => 'Copied';

@override
String get chatReply => 'Reply';

@override
String chatReplyTo(String name) {
  return 'In reply to $name';
}

@override
String get chatReplyDeleted => 'Deleted message';

@override
String get chatReplyNotFound => 'Original message isn’t available';

@override
String get chatMessageHint => 'Message…';

@override
String get chatAttachImage => 'Image';

@override
String get chatAttachVideo => 'Video';

@override
String get chatAttachFile => 'File';

@override
String chatTyping(String names) {
  return '$names typing…';
}

@override
String chatRecording(String time) {
  return 'Recording $time';
}

@override
String chatUploading(String name) {
  return 'Uploading $name…';
}

@override
String chatSendFailed(String error) {
  return 'Could not send: $error';
}

@override
String get chatMicDenied =>
    'Microphone permission is required for voice messages.';

@override
String get chatEditTitle => 'Edit message';

@override
String get chatDeleteTitle => 'Delete message?';

@override
String get chatDeleteBody =>
    'This removes the message for everyone in the room.';

@override
String chatDeleteSelectedTitle(int count) {
  return 'Delete $count messages?';
}

@override
String get chatDeleteSelectedBody =>
```

```
    'Selected messages will be removed for everyone in the room.';

@override
String chatSelectedCount(int count) {
  return '$count selected';
}

@override
String get chatSelectMessages => 'Select messages';

@override
String get chatSelect => 'Select';

@override
String get commonSelectAll => 'Select all';

@override
String get chatEdited => 'edited';

@override
String get chatDateToday => 'Today';

@override
String get chatDateYesterday => 'Yesterday';

@override
String get chatRoleAdmin => 'admin';

@override
String get chatRoleTeacher => 'teacher';

@override
String get commonSave => 'Save';

@override
String get commonCancel => 'Cancel';

@override
String get commonSubmit => 'Submit';

@override
String get commonDelete => 'Delete';

@override
String get commonEdit => 'Edit';

@override
String get commonClose => 'Close';

@override
String get commonRetry => 'Retry';

@override
String get commonTryAgain => 'Try again';

@override
String get commonLoading => 'Loading…';

@override
String get commonError => 'Something went wrong';

@override
String commonErrorWithDetails(String error) {
  return 'Error: $error';
}

@override
String commonCouldNotSave(String error) {
  return 'Could not save: $error';
}

@override
String get commonSignOut => 'Sign out';
```

```
  @override
  String get commonSaveDraft => 'Save draft';

  @override
  String get commonOpen => 'Open';

  @override
  String get commonCreate => 'Create';

  @override
  String get commonAssign => 'Assign';

  @override
  String get commonReset => 'Reset';

  @override
  String get commonAdd => 'Add';

  @override
  String get commonLoadMore => 'Load more';

  @override
  String get commonExport => 'Export';

  @override
  String commonExportFailed(String error) {
    return 'Could not export: $error';
  }

  @override
  String get marksTermAll => 'All time';

  @override
  String marksTermSemester(String yearRange, int semester) {
    return '$yearRange · Semester $semester';
  }

  @override
  String get marksExportCsv => 'Export CSV';

  @override
  String get marksExportDone => 'Marks exported';

  @override
  String get commonTeacher => 'Teacher';

  @override
  String get commonStudent => 'Student';

  @override
  String get commonAdmin => 'Administrator';

  @override
  String get commonWorkspace => 'Workspace';

  @override
  String get commonNoDueDate => 'No due date';

  @override
  String commonDueDate(String date) {
    return 'Due $date';
  }

  @override
  String get homeworkDueThisWeek => 'Due this week';

  @override
  String get homeworkDueThisWeekSubtitle => 'Overdue and upcoming deadlines';

  @override
  String get homeworkNoDueThisWeek => 'Nothing due this week';

  @override
```

```
  String get homeworkNoDueThisWeekSubtitle =>
      'You’re clear until next week’s deadlines.';

  @override
  String homeworkOverdue(String date) {
    return 'Overdue · $date';
  }

  @override
  String get homeworkDueToday => 'Due today';

  @override
  String get homeworkDueTomorrow => 'Due tomorrow';

  @override
  String homeworkDueInDays(int days) {
    return 'Due in $days days';
  }

  @override
  String get homeworkLateBadge => 'Late';

  @override
  String get commonProgress => 'Progress';

  @override
  String get commonSubject => 'Subject';

  @override
  String get commonType => 'Type';

  @override
  String get commonTitle => 'Title';

  @override
  String get commonDescription => 'Description';

  @override
  String get commonGroup => 'Group';

  @override
  String get commonMessage => 'Message';

  @override
  String get commonScore => 'Score';

  @override
  String get commonFeedback => 'Feedback';

  @override
  String get commonStudents => 'Students';

  @override
  String get commonCannotUndo => 'This action cannot be undone.';

  @override
  String get profileTitle => 'Profile';

  @override
  String get profileFullName => 'Full name';

  @override
  String get profileNotSignedIn => 'Not signed in';

  @override
  String get profileAccount => 'Account';

  @override
  String get profileAccountSettings => 'Account Settings';

  @override
  String get profileSignOutAdmin => 'Sign out of Admin Console';
```

```
  @override
  String get profileSettings => 'Settings';

  @override
  String get profileAppearance => 'Appearance';

  @override
  String get profileThemeLight => 'Light';

  @override
  String get profileThemeDark => 'Dark';

  @override
  String get profileThemeSystem => 'System';

  @override
  String get profileStyle => 'Style';

  @override
  String get profileStyleModern => 'Modern Professional';

  @override
  String get profileStyleNature => 'Nature Paper';

  @override
  String get profileStyleSoftNeu => 'Neumorphism';

  @override
  String get profileStyleMonochrome => 'Monochrome Workspace';

  @override
  String get profileEditName => 'Edit name';

  @override
  String get profileNameUpdated => 'Name updated';

  @override
  String get profileChangePhoto => 'Change photo';

  @override
  String get profilePhotoUpdated => 'Photo updated';

  @override
  String get profileChangePassword => 'Change password';

  @override
  String get profileCurrentPassword => 'Current password';

  @override
  String get profileNewPassword => 'New password';

  @override
  String get profileConfirmPassword => 'Confirm new password';

  @override
  String get profilePasswordChanged => 'Password updated';

  @override
  String get profilePasswordMismatch => 'New passwords do not match';

  @override
  String get profilePasswordTooShort => 'Use at least 8 characters';

  @override
  String get profilePasswordHint => 'Ask an admin if you forget your password.';

  @override
  String get profileShortcuts => 'Shortcuts';

  @override
  String get profileYourGroups => 'Your groups';

  @override
```

```
  String get profileNoGroups => 'No groups assigned';

  @override
  String profileGroupsCount(int count) {
    return '$count groups';
  }

  @override
  String get profileAverageLabel => 'Term average';

  @override
  String get profileAttendanceLabel => 'Attendance';

  @override
```

**lib/l10n/app_localizations_en.dart — continued (2/4)**

```
  String get profileShareProgress => 'Share progress report';

  @override
  String get profileNotifications => 'Notifications';

  @override
  String get profileClearCache => 'Clear cached data';

  @override
  String get profileClearCacheWarn1Title => 'Clear cached data?';

  @override
  String get profileClearCacheWarn1Body =>
      'This deletes offline chat messages and downloaded images, videos, voice notes, and files.
 You will need an internet connection to load them again.';

  @override
  String get profileClearCacheWarn2Title => 'Are you sure?';

  @override
  String get profileClearCacheWarn2Body =>
      'This cannot be undone. Cached chat media will be removed from this device.';

  @override
  String get profileClearCacheConfirm => 'Clear cache';

  @override
  String get profileClearCacheDone => 'Cached data cleared';

  @override
  String profileAppVersion(String version) {
    return 'MU Study · v$version';
  }

  @override
  String get profileSubject => 'Subject';

  @override
  String get greetingMorning => 'Good Morning';

  @override
  String get greetingAfternoon => 'Good Afternoon';

  @override
  String get greetingEvening => 'Good Evening';

  @override
  String greetingWithName(String greeting, String name) {
    return '$greeting, $name';
  }

  @override
  String get greetingLoadingTitle => 'Good Morning, User';

  @override
  String get greetingLoadingSubtitle => 'Loading your personalized greeting...';
```

```
@override
String get studentHomeSubtitle => 'Your learning space';

@override
String get continueLearning => 'Continue Learning';

@override
String get continueLearningSubtitle => 'Pick up where you left off';

@override
String get everythingCompleted => 'Everything is completed.';

@override
String get caughtUpToday => 'You\'re all caught up for today.';

@override
String get homeworkPack => 'Homework Pack';

@override
String get staticTask => 'Static Task';

@override
String get staticAssignment => 'Static';

@override
String get upcomingAssignments => 'Upcoming Assignments';

@override
String get stayOnTrack => 'Stay on track';

@override
String get loadingAnnouncements => 'Loading announcements...';

@override
String get notificationsCardTitle => 'Notifications';

@override
String unreadAlerts(int count) {
  return 'You have $count new alerts';
}

@override
String get upToDate => 'You are up to date';

@override
String get announcementsTitle => 'Announcements';

@override
String get announcementsSubtitle => 'From your teachers';

@override
String get announcementsEmpty => 'No announcements';

@override
String get announcementsEmptyTeacher =>
    'Share important updates with your students.';

@override
String get announcementsEmptyAdmin =>
    'Platform-wide announcements will appear here.';

@override
String get announcementsReadMore => 'Read more';

@override
String get announcementsShowLess => 'Show less';

@override
String announcementsViewAll(int count) {
  return 'View all ($count)';
}

@override
```

```
  String get announcementsAll => 'All Announcements';

  @override
  String get announcementsNew => 'New Announcement';

  @override
  String get announcementsPost => 'Post Announcement';

  @override
  String get announcementsEdit => 'Edit Announcement';

  @override
  String get announcementsDeleteTitle => 'Delete announcement?';

  @override
  String get announcementsDeleteBody =>
      'It will be removed for everyone in the group.';

  @override
  String get announcementsByAdmin => 'Administration';

  @override
  String get announcementsShareUpdates => 'Share updates with your groups';

  @override
  String get assignmentsTitle => 'Assignments';

  @override
  String get assignmentsEmpty => 'No assignments';

  @override
  String get assignmentsEmptySubtitle => 'You are all caught up!';

  @override
  String get assignmentsEmptyTeacher =>
      'Create your first assignment to get started.';

  @override
  String get assignmentsLearningPath => 'Learning Path';

  @override
  String get assignmentsActive => 'Active assignments';

  @override
  String get assignmentDetails => 'Assignment Details';

  @override
  String get assignmentNotFound => 'Assignment not found';

  @override
  String get assignmentTasks => 'Tasks';

  @override
  String get assignmentTasksSubtitle => 'Complete all to finish pack';

  @override
  String get assignmentNew => 'New Assignment';

  @override
  String get assignmentManage => 'Manage Assignments';

  @override
  String get assignmentManageSubtitle => 'Create static or homework packs';

  @override
  String get marksTitle => 'My Marks';

  @override
  String get marksOverallAverage => 'Overall average';

  @override
  String get marksNoneYet => 'No marks recorded yet.';
```

```
  @override
  String marksBasedOn(String label, int count) {
    return '$label · based on $count marks.';
  }

  @override
  String get marksByCategory => 'By category';

  @override
  String get marksWhereStrongest => 'Where you are strongest';

  @override
  String get marksAll => 'All marks';

  @override
  String get marksNewestFirst => 'Newest first';

  @override
  String get marksEmptyTitle => 'No marks yet';

  @override
  String get marksEmptySubtitle => 'Your teacher\'s marks will appear here.';

  @override
  String get progressReportTitle => 'Progress report';

  @override
  String get progressReportShare => 'Share progress';

  @override
  String get progressReportSharing => 'Preparing report…';

  @override
  String get progressReportDone => 'Progress report ready to share';

  @override
  String progressReportFailed(String error) {
    return 'Could not share progress: $error';
  }

  @override
  String get progressReportNeedGroup =>
      'Join a group to share a progress report.';

  @override
  String get progressReportAttendance => 'Attendance';

  @override
  String get progressReportAttendanceRate => 'Attendance rate';

  @override
  String get progressReportNoAttendance =>
      'No attendance recorded for this term.';

  @override
  String get progressReportSessions => 'sessions';

  @override
  String get progressReportMarksCounted => 'marks';

  @override
  String get progressReportRecentMarks => 'Recent marks';

  @override
  String progressReportGeneratedOn(String date) {
    return 'Generated $date';
  }

  @override
  String get marksNoRecorded => 'No marks recorded';

  @override
  String get marksNoRecordedSubtitle =>
```

```
    'Use \"Grade class\" to enter marks in one pass.';

@override
String get marksRecent => 'Recent marks';

@override
String get marksRecentSubtitle => 'Latest entries across the group';

@override
String get marksStudentsHeader => 'Students';

@override
String get marksStudentsSubtitle => 'Weighted average of all marks';

@override
String get marksNoStudents => 'No students in this group';

@override
String get marksNoStudentsSubtitle =>
    'An admin can assign students to your group.';

@override
String get marksDeleteTitle => 'Delete mark?';

@override
String marksDeleteBody(String title) {
  return '\"$title\" will be removed permanently.';
}

@override
String get marksGradeTheClass => 'Grade the class';

@override
String get marksOnePercentEach => 'One percent mark per student';

@override
String get marksSaveMarks => 'Save marks';

@override
String get marksSaveMark => 'Save mark';

@override
String get marksAddMark => 'Add mark';

@override
String get marksNothingYet => 'Nothing recorded yet';

@override
String get marksAddFirst => 'Add the first mark for this student.';

@override
String marksSavedCount(int count) {
  return 'Saved $count marks';
}

@override
String get marksGroupHasNoStudents => 'This group has no students yet.';

@override
String get marksTitleHint => 'e.g. Unit 3 test';

@override
String marksAverageLine(String percent, String label) {
  return 'Average $percent · $label';
}

@override
String get marksNoMarksYetShort => 'No marks yet';

@override
String get marksSummaryStudents => 'Students';

@override
```

```
  String get marksSummaryMarks => 'Marks';

  @override
  String get attendanceTitle => 'Attendance';

  @override
  String get attendanceEmpty => 'No attendance records';

  @override
  String get attendanceNoGroup => 'No group assigned yet';

  @override
  String get attendanceMark => 'Mark Attendance';

  @override
  String get attendanceDailyTracking => 'Daily student tracking';

  @override
  String get attendanceSaved => 'Attendance saved successfully';

  @override
  String get attendanceNoGroups => 'No groups found';

  @override
  String get attendancePresent => 'P — Present';

  @override
  String get attendanceLate => 'L — Late';

  @override
  String get attendanceAbsent => 'A — Absent';

  @override
  String get attendanceExcused => 'E — Excused';

  @override
  String get attendancePresentShort => 'P';

  @override
  String get attendanceLateShort => 'L';

  @override
  String get attendanceAbsentShort => 'A';

  @override
  String get attendanceExcusedShort => 'E';

  @override
  String get attendanceTrend => 'Attendance Trend';

  @override
  String get attendanceOverall => 'Overall distribution';

  @override
  String get analyticsThisWeek => 'This Week';

  @override
  String get analyticsLastWeek => 'Last Week';

  @override
  String analyticsTotalMarks(int count) {
    return 'Total Marks: $count';
  }

  @override
  String get notificationsTitle => 'Notifications';

  @override
  String get notificationsEmpty => 'All caught up!';

  @override
  String get notificationsEmptySubtitle => 'No new notifications.';
```

```
  @override
  String get notificationsNone => 'No notifications';

  @override
  String get notifKindAssignment => 'New homework';

  @override
  String get notifKindAnnouncement => 'Announcement';

  @override
  String get notifKindGraded => 'Work graded';

  @override
  String get notifKindSubmission => 'New submission';

  @override
  String get notifKindChat => 'Chat message';

  @override
  String get notifKindSystem => 'Update';

  @override
  String get notificationsOpen => 'Open';

  @override
  String get taskTitle => 'Task';

  @override
  String get taskYourAnswer => 'Your answer';

  @override
  String get taskMarkCompleted => 'Mark completed';

  @override
  String get taskAttachment => 'Attachment';

  @override
  String get taskAttachmentHint => 'Add a photo or document to your answer.';

  @override
  String get taskAttachmentHintReady => 'Tap to open · trash to remove.';

  @override
  String get taskImage => 'Image';

  @override
  String get taskImageSubtitle => 'Gallery photo';

  @override
  String get taskFile => 'File';

  @override
  String get taskFileSubtitle => 'PDF or document';

  @override
  String get taskReplaceImage => 'Replace with image';

  @override
  String get taskReplaceFile => 'Replace file';

  @override
  String get taskVoice => 'Voice record';

  @override
  String get taskVoiceHint =>
      'Optional spoken answer — record, play back, then save.';

  @override
  String get taskViewPdf => 'View PDF';

  @override
  String get taskOpenLink => 'Open link';
```

```
  @override
  String get taskSubmitted => 'Submitted';

  @override
  String get taskSaved => 'Saved';

  @override
  String taskUploadFailed(String error) {
    return 'Upload failed: $error';
  }

  @override
  String taskVoiceUploadFailed(String error) {
    return 'Voice upload failed: $error';
  }

  @override
  String taskCouldNotSave(String error) {
    return 'Could not save: $error';
  }

  @override
  String get voiceTapToRecord => 'Tap the mic to record your answer';

  @override
  String get voiceRecording => 'Recording…';

  @override
  String get voiceNote => 'Voice note';

  @override
  String get voiceTapPlay => 'Tap play to listen';

  @override
  String get voiceRerecord => 'Re-record';

  @override
  String get voiceStudentAnswer => 'Student voice answer';

  @override
  String get voiceMicPermission =>
      'Microphone permission is required to record.';

  @override
  String voiceCouldNotStart(String error) {
    return 'Could not start recording: $error';
  }

  @override
  String voiceCouldNotSave(String error) {
    return 'Could not save recording: $error';
  }

  @override
  String voiceCouldNotPlay(String error) {
    return 'Could not play voice note: $error';
  }

  @override
  String get attachmentCouldNotOpen => 'Could not open this file.';

  @override
  String get attachmentImage => 'Image attachment';

  @override
  String get attachmentFile => 'File attachment';

  @override
  String get attachmentPdf => 'PDF document';

  @override
  String get attachmentTapPreview => 'Tap to preview';
```

```
  @override
  String get attachmentTapOpen => 'Tap to open';

  @override
  String get pdfDefaultTitle => 'PDF';

  @override
  String get teacherWorkspace => 'Teacher Workspace';

  @override
  String get teacherQuickActions => 'Quick Actions';

  @override
  String get teacherCreateItems => 'Create new items';

  @override
  String get teacherNewAssignment => 'New Assignment';

  @override
  String get teacherNewAnnouncement => 'New Announcement';

  @override
  String get teacherSubmissions => 'Submissions';

  @override
  String get teacherNoSubmissions => 'No submissions yet';

  @override
  String get teacherNoSubmissionsSubtitle =>
      'Student work will show up here for grading.';

  @override
  String get teacherGradebook => 'Gradebook';

  @override
  String get teacherGradeClass => 'Grade class';

  @override
  String get teacherNoGroups => 'No groups yet';

  @override
  String get teacherNoGroupsSubtitle => 'Ask an admin to assign you a group.';

  @override
  String get teacherActiveGroups => 'ACTIVE GROUPS';

  @override
  String get teacherSubject => 'SUBJECT';

  @override
  String get teacherDailyTasks => 'Daily Tasks';

  @override
```

**lib/l10n/app_localizations_en.dart — continued (3/4)**

```
  String get teacherNeedsAttention => 'What needs your attention';

  @override
  String get teacherReviewSubmissions => 'Review Submissions';

  @override
  String get teacherCheckAssignments => 'Check student assignments';

  @override
  String get teacherRecordMarks => 'Record marks in percent';

  @override
  String get teacherPerformanceAnalytics => 'Performance Analytics';

  @override
  String get teacherTrackProgress => 'Track group progress';

  @override
```

```
  String get teacherOpenAttachment => 'Open attachment';

  @override
  String get teacherScoreHelper => 'Adds a homework mark to the gradebook';

  @override
  String get teacherScoreRange => 'Score must be between 0 and 100.';

  @override
  String get teacherNoWrittenAnswer => 'No written answer.';

  @override
  String get teacherReviewSubmission => 'Review submission';

  @override
  String get teacherHomeworkPackTasks => 'Homework Pack Tasks';

  @override
  String get teacherAddTask => 'Add Task';

  @override
  String get teacherEditTask => 'Edit Task';

  @override
  String get teacherDeleteTask => 'Delete Task?';

  @override
  String get teacherTasksContent => 'Tasks Content';

  @override
  String get teacherManagePack => 'Manage items in this pack';

  @override
  String get teacherTaskTitleHint => 'e.g. Reading Exercise';

  @override
  String get teacherLinkHint => 'https://...';

  @override
  String get teacherAttachmentHint => 'Image or file link...';

  @override
  String get teacherNoData => 'No data available';

  @override
  String get teacherMarkDistribution => 'Mark Distribution';

  @override
  String get teacherStudentsByAverage => 'Students grouped by average';

  @override
  String get teacherNoMarksTrends =>
      'Add marks in the gradebook to see trends.';

  @override
  String get teacherTargetGroup => 'Target Group';

  @override
  String get adminWorkspace => 'Workspace Control';

  @override
  String get adminUsers => 'User Management';

  @override
  String get adminGroups => 'Groups';

  @override
  String get adminAnalytics => 'Analytics';

  @override
  String get adminTeachers => 'TEACHERS';

  @override
```

```
  String get adminStudents => 'STUDENTS';

  @override
  String get adminGroupsLabel => 'GROUPS';

  @override
  String get adminMarksLabel => 'MARKS';

  @override
  String get adminActivityInsights => 'Activity Insights';

  @override
  String get adminCurrentMetrics => 'Current metrics';

  @override
  String get adminCreateUser => 'Create New User';

  @override
  String get adminCreateUserButton => 'Create User';

  @override
  String get adminUserCreated => 'User created successfully';

  @override
  String get adminResetPassword => 'Reset password';

  @override
  String adminResetPasswordTitle(String username) {
    return 'Reset Password · $username';
  }

  @override
  String get adminCreateGroup => 'Create New Group';

  @override
  String get adminCreateGroupButton => 'Create Group';

  @override
  String get adminGroupTitleHint => 'e.g. Group 4';

  @override
  String get adminGroupTitleExists =>
      'This teacher already has a group with that name';

  @override
  String get adminAssignStudent => 'Assign Student';

  @override
  String get adminActivityDistribution => 'Activity Distribution';

  @override
  String get adminGlobalMetrics => 'Global platform metrics';

  @override
  String get adminActivityLog => 'Admin Activity Log';

  @override
  String get adminActivityLogSubtitle =>
      'System-wide events and audit logs appear here.';

  @override
  String adminTeacherLine(String name) {
    return 'Teacher: $name';
  }

  @override
  String adminRoleLine(String role, String username) {
    return '$role · $username';
  }

  @override
  String get adminDisable => 'Disable';
```

```
  @override
  String get adminEnable => 'Enable';

  @override
  String get adminPassword => 'Password';

  @override
  String get adminFullName => 'Full name';

  @override
  String get adminRole => 'Role';

  @override
  String get adminSphere => 'Sphere';

  @override
  String get percentExcellent => 'Excellent';

  @override
  String get percentGood => 'Good';

  @override
  String get percentNeedsWork => 'Needs work';

  @override
  String get percentAtRisk => 'At risk';

  @override
  String get emptyStateDefault => 'Nothing here yet';

  @override
  String get setupRequiredTitle => 'Setup required';

  @override
  String get setupRequiredBody =>
      'Supabase is not configured. Add SUPABASE_URL and SUPABASE_ANON_KEY to your .env file.';

  @override
  String get setupOption1 => 'Option 1 — local .env file';

  @override
  String get setupOption2 => 'Option 2 — dart-define';

  @override
  String get attendanceAllPresent => 'All P';

  @override
  String get attendanceAllAbsent => 'All A';

  @override
  String get attendanceAllLate => 'All L';

  @override
  String get attendanceAllExcused => 'All E';

  @override
  String get attendancePresentWord => 'Present';

  @override
  String get attendanceAbsentWord => 'Absent';

  @override
  String get attendanceLateWord => 'Late';

  @override
  String get attendanceExcusedWord => 'Excused';

  @override
  String get attendanceWeekdayMon => 'Mon';

  @override
  String get attendanceWeekdayTue => 'Tue';
```

```
  @override
  String get attendanceWeekdayWed => 'Wed';

  @override
  String get attendanceWeekdayThu => 'Thu';

  @override
  String get attendanceWeekdayFri => 'Fri';

  @override
  String get attendancePercentLabel => 'Rate';

  @override
  String get attendanceNoLesson => 'No lesson';

  @override
  String get attendanceHistory => 'Attendance history';

  @override
  String get attendanceSelectStudent => 'Student';

  @override
  String get attendanceAllStudents => 'All students';

  @override
  String get attendanceTapToCycle => 'Tap a day to change status';

  @override
  String get attendanceUpdated => 'Attendance updated';

  @override
  String get attendanceCalendarEmpty => 'No attendance in this month';

  @override
  String get attendanceFutureNotAllowed =>
      'You can’t mark attendance for a future day';

  @override
  String get attendanceWeekendNotAllowed =>
      'Attendance can only be marked on weekdays (Mon–Fri)';

  @override
  String get chartBandExcellent => '85–100%';

  @override
  String get chartBandGood => '70–84%';

  @override
  String get chartBandNeedsWork => '50–69%';

  @override
  String get chartBandAtRisk => 'Below 50%';

  @override
  String get marksWeight => 'Weight';

  @override
  String get marksCategory => 'Category';

  @override
  String get marksDate => 'Date';

  @override
  String get marksCommentOptional => 'Comment (optional)';

  @override
  String get marksNewMark => 'New mark';

  @override
  String get marksEditMark => 'Edit mark';

  @override
  String get marksFromSubmission => 'From a graded submission';
```

```
  @override
  String get marksNeedTitle => 'Give the graded item a title.';

  @override
  String get marksWeightPositive => 'Weight must be greater than zero.';

  @override
  String get marksNeedAtLeastOne => 'Enter a mark for at least one student.';

  @override
  String get marksLeaveBlankSkip => 'Leave a field blank to skip that student.';

  @override
  String get marksFromSubmissionDelete =>
      'This mark comes from a graded submission. Clear the score in Submissions to remove it.';

  @override
  String get marksFromSubmissionEdit =>
      'This mark mirrors a graded submission. Editing it here will be overwritten the next time
the submission is re-graded.';

  @override
  String get categoryHomework => 'Homework';

  @override
  String get categoryClasswork => 'Classwork';

  @override
  String get categoryQuiz => 'Quiz';

  @override
  String get categoryTest => 'Test';

  @override
  String get categoryExam => 'Exam';

  @override
  String get categoryProject => 'Project';

  @override
  String get categoryOral => 'Oral';

  @override
  String get categoryOther => 'Other';

  @override
  String get teacherReferenceLinkOptional => 'Reference Link (optional)';

  @override
  String get teacherAttachmentUrlOptional => 'Attachment URL (optional)';

  @override
  String get teacherNoTasksYet =>
      'No tasks added yet. Click \"Add Task\" to get started.';

  @override
  String get teacherTemplates => 'Templates';

  @override
  String get teacherTemplatesEmpty => 'No templates yet';

  @override
  String get teacherTemplatesEmptySubtitle =>
      'Save an assignment as a template to reuse it later.';

  @override
  String get teacherSaveAsTemplate => 'Save as template';

  @override
  String get teacherTemplateSaved => 'Template saved';

  @override
  String get teacherUseTemplate => 'Use template';
```

```
  @override
  String get teacherFromTemplate => 'From template';

  @override
  String get teacherDeleteTemplate => 'Delete template?';

  @override
  String teacherDeleteTemplateBody(String title) {
    return '\"$title\" will be removed.';
  }

  @override
  String get teacherDuplicateAssignment => 'Duplicate';

  @override
  String get teacherDuplicateTitle => 'Duplicate assignment';

  @override
  String get teacherDuplicated => 'Assignment duplicated';

  @override
  String get assignmentEdit => 'Edit assignment';

  @override
  String get assignmentDeleteTitle => 'Delete assignment?';

  @override
  String get assignmentDeleteBody =>
      'This permanently removes the assignment, its tasks, and student submissions.';

  @override
  String get assignmentDeleted => 'Assignment deleted';

  @override
  String get assignmentTypeHintStatic =>
      'A single piece of work. Students submit once — no extra tasks.';

  @override
  String get assignmentTypeHintPack =>
      'A pack of multiple tasks (workbook-style). You'll add items next.';

  @override
  String get teacherCreateFromTemplate => 'Create from template';

  @override
  String teacherTasksCopied(int count) {
    return '$count tasks included';
  }

  @override
  String get teacherRecentFeedback => 'Reuse feedback';

  @override
  String get teacherApplyFeedbackToOthers =>
      'Also apply to other pending for this task';

  @override
  String teacherFeedbackAppliedCount(int count) {
    return 'Feedback applied to $count other submissions';
  }

  @override
  String get teacherSharedComment => 'Shared comment (optional)';

  @override
  String get teacherSharedCommentHint => 'Same note on every mark you save';

  @override
  String get adminPlatformOverview => 'Platform Overview';

  @override
  String get adminAssignTeacher => 'Assign Teacher';
```

```
  @override
  String get adminSelectStudent => 'Select Student';

  @override
  String get adminInitialGroupOptional => 'Initial Group (Optional)';

  @override
  String get adminChartTasks => 'Tasks';

  @override
  String get adminChartGroups => 'Groups';

  @override
  String get adminChartStaff => 'Staff';

  @override
  String get adminChartAssignments => 'Assignments';

  @override
  String get adminChartMarks => 'Marks';

  @override
  String get adminChartStudents => 'Students';

  @override
  String get adminAcademicYears => 'Academic years';

  @override
  String get adminAcademicYearsSubtitle =>
      'School years and semesters for the gradebook';

  @override
  String get adminAcademicYearsTitle => 'Academic calendar';

  @override
  String get adminCreateNextYear => 'Create next year';

  @override
  String adminYearCreated(String label) {
    return 'Created $label';
  }

  @override
  String get adminSetCurrentYear => 'Set as current';

  @override
  String get adminCurrentYearBadge => 'Current';

  @override
  String get adminYearSemesters => 'Four semesters · Aug → Jun';

  @override
  String adminSemesterRange(int semester, String start, String end) {
    return 'Semester $semester · $start – $end';
  }

  @override
  String get adminEditTerm => 'Edit term';

  @override
  String adminEditTermTitle(int semester) {
    return 'Edit semester $semester';
  }

  @override
  String get adminTermStartsOn => 'Starts on';

  @override
  String get adminTermEndsOn => 'Ends on';

  @override
  String get adminTermDatesInvalid =>
      'End date must be on or after the start date';
```

```
  @override
  String get adminTermUpdated => 'Term dates updated';

  @override
  String get adminDeleteYear => 'Delete year';

  @override
  String adminDeleteYearTitle(String label) {
    return 'Delete $label?';
  }

  @override
  String adminDeleteYearBody(String label) {
    return 'This removes the year and its four terms from the calendar. Marks are not deleted.
To confirm, type $label below.';
  }

  @override
  String adminDeleteYearConfirmHint(String label) {
    return 'Type $label to confirm';
  }

  @override
  String get adminDeleteYearCurrentBlocked =>
      'Set another year as current before deleting this one.';

  @override
  String adminYearDeleted(String label) {
    return 'Deleted $label';
  }

  @override
  String get adminNoAcademicYears => 'No academic years yet';

  @override
  String get adminAcademicYearsHint =>
      'Create the next school year when you are ready. Teachers and students pick semesters in
the gradebook. Tap a term to edit its dates.';

  @override
  String get adminCredentialPrintTitle => 'Print login slips';

  @override
  String get adminCredentialPrintSubtitle =>
      'Unique usernames and passwords from the database, 3 slips per page';

  @override
  String get adminCredentialPasteLabel => 'Users';

  @override
  String get adminCredentialPasteHint =>
      'username|password|full name\n@james|Abc123xyz|James English|teacher\
n@john_jones|Pass456|John Jones';

  @override
  String get adminCredentialPerPage => 'Slips per page';

  @override
  String get adminCredentialTwo => '2';

  @override
  String get adminCredentialThree => '3';

  @override
  String get adminCredentialExportPdf => 'Export PDF';

  @override
  String get adminCredentialSlipTitle => 'Your MU Study login';

  @override
  String get adminCredentialSlipHint =>
```

**lib/l10n/app_localizations_en.dart — continued (4/4)**

```
        'Keep this paper private. Change your password after first sign-in.';

  @override
  String get adminCredentialNeedUsers => 'No users found to print';

  @override
  String get adminCredentialNeedSql =>
      'Run sql/admin_login_slips.sql in the Supabase SQL Editor, then export again.';

  @override
  String get adminCredentialConfidential =>
      'Private and confidential. These credentials are intended only for the assigned student.';

  @override
  String get adminFirstName => 'First name';

  @override
  String get adminLastName => 'Last name';

  @override
  String get adminCredentialStudents => 'Students';

  @override
  String get adminCredentialTeachers => 'Teachers';

  @override
  String get adminCredentialAllGroups => 'All groups';

  @override
  String get sphereMath => 'Math';

  @override
  String get sphereEnglish => 'English';

  @override
  String get dailyMotivation1 => 'Today is a great day to make progress.';

  @override
  String get dailyMotivation2 => 'Ready to continue where you left off?';

  @override
  String get dailyMotivation3 => 'Let\'s conquer your goals today.';

  @override
  String get dailyMotivation4 => 'Consistency is the key to mastery.';
}
```

## app_localizations_ru.dart

**lib/l10n/app_localizations_ru.dart**

```
// ignore: unused_import
import 'package:intl/intl.dart' as intl;
import 'app_localizations.dart';

// ignore_for_file: type=lint

/// The translations for Russian (`ru`).
class AppLocalizationsRu extends AppLocalizations {
  AppLocalizationsRu([String locale = 'ru']) : super(locale);

  @override
  String get appName => 'MU Study';

  @override
  String get shellMuStudy => 'MU Study';

  @override
  String get languageTitle => '[illegible]';

  @override
  String get languageSubtitle =>
      '[illegible].';

  @override
  String get languageEnglish => '[illegible]';

  @override
  String get languageRussian => '[illegible]';

  @override
  String get languageUzbek => '[illegible]';

  @override
  String get languageEnglishNative => 'English';

  @override
  String get languageRussianNative => '[illegible]';

  @override
  String get languageUzbekNative => 'O‘zbekcha';

  @override
  String get languageContinue => '[illegible]';

  @override
  String get languageSection => '[illegible]';

  @override
  String get languageHint => '[illegible]';

  @override
  String get loginWelcome => '[illegible]';

  @override
  String get loginSubtitle => '[illegible] MU Study';

  @override
  String get loginUsername => '[illegible]';

  @override
  String get loginPassword => '[illegible]';

  @override
  String get loginSignIn => '[illegible]';

  @override
  String get loginMissingFields => '[illegible]';

  @override
```

```
  String get loginIncorrect => '████████ ███ ████████████ ███ ███████';

  @override
  String get loginFailed =>
      '██ ████████ ██████. ██████████ ██████████ ■ ██████████ ██████.';

  @override
  String get loginConfigError => '███████ ██████████████. ██████████ ■ ██████████.';

  @override
  String get loginConfigMissing =>
      '██████████████ ████████████. ██████████ █████ .env.';

  @override
  String get accountDisabled =>
      '████████ █████████. ██████████ ■ ████████████████.';

  @override
  String get navDashboard => '████████';

  @override
  String get navAssignments => '████████';

  @override
  String get navMarks => '███████';

  @override
  String get navAttendance => '███████████████';

  @override
  String get navNotifications => '████████████';

  @override
  String get navProfile => '████████';

  @override
  String get navUsers => '██████████████';

  @override
  String get navGroups => '███████';

  @override
  String get navAnalytics => '██████████';

  @override
  String get navGradebook => '███████';

  @override
  String get navChat => '████';

  @override
  String get navFocus => '██████';

  @override
  String get focusTitle => '██████';

  @override
  String get focusSubtitle => '██████████ ██████ — █████ ████████ ██ ████.';

  @override
  String get focusTodaysFocus => '██████ ██ ████████';

  @override
  String get focusTodaysFocusSubtitle =>
      '██ █████ █████████ ████████ — ██████ █████';

  @override
  String get focusAlsoDue => '████ ████████';

  @override
  String get focusContinueHomework => '███████████ █████████';

  @override
```

```
  String get focusOpenAssignment => '[redacted]';

  @override
  String get focusEmptyTitle => '[redacted]';

  @override
  String get focusEmptySubtitle =>
      '[redacted]';

  @override
  String get focusChooseMode => '[redacted]?';

  @override
  String get focusModePomodoro => '[redacted]';

  @override
  String get focusModePomodoroHint => '25 [redacted]';

  @override
  String get focusModeFree => '[redacted]';

  @override
  String get focusModeFreeHint => '[redacted]';

  @override
  String get focusDoNotDisturb => '[redacted]';

  @override
  String get focusChatMuteHint =>
      '[redacted]';

  @override
  String get focusPause => '[redacted]';

  @override
  String get focusResume => '[redacted]';

  @override
  String get focusPaused => '[redacted]';

  @override
  String get focusEndSession => '[redacted]';

  @override
  String get focusSessionComplete => '[redacted]';

  @override
  String get focusSessionCompleteSubtitle => '[redacted]';

  @override
  String get focusTimeUp => '[redacted]';

  @override
  String get focusTimeUpSubtitle =>
      '[redacted]';

  @override
  String focusMinutesFocused(String duration) {
    return '[redacted]: $duration';
  }

  @override
  String get focusOptionalNote => '[redacted]';

  @override
  String get focusOptionalNoteHint => '[redacted]';

  @override
  String get focusSaveAndOpen => '[redacted]';

  @override
  String get focusDone => '[redacted]';
```

```
@override
String get chatTitle => '[illegible]';

@override
String get chatSearchHint => '[illegible]';

@override
String get chatEmptyTitle => '[illegible]';

@override
String get chatEmptySubtitle => '[illegible]';

@override
String get chatNoMessagesYet => '[illegible]';

@override
String get chatMessageDeleted => '[illegible]';

@override
String get chatPreviewImage => '[illegible]';

@override
String get chatPreviewVideo => '[illegible]';

@override
String get chatVideoPlaybackSpeed => '[illegible]';

@override
String get chatVideoSpeedNormal => '1x';

@override
String chatVideoSpeedValue(double speed) {
  final intl.NumberFormat speedNumberFormat =
      intl.NumberFormat.decimalPattern(localeName);
  final String speedString = speedNumberFormat.format(speed);

  return '${speedString}x';
}

@override
String get chatVideoRewind10 => '-10[illegible]';

@override
String get chatVideoForward10 => '+10[illegible]';

@override
String get chatVideoCouldNotPlay => '[illegible]';

@override
String get chatPreviewVoice => '[illegible]';

@override
String get chatPreviewFile => '[illegible]';

@override
String get chatOpenFile => '[illegible]';

@override
String get chatOpenFileFailed => '[illegible]';

@override
String get chatYou => '[illegible]';

@override
String chatTeacherMembers(String teacher, int count) {
  return '$teacher · $count [illegible]';
}

@override
String chatOnlineCount(int count) {
  return '$count [illegible]';
}
```

```
  @override
  String get chatReact => '████████';

  @override
  String get chatCopyText => '████████████ ██████';

  @override
  String get chatCopied => '██████████████';

  @override
  String get chatReply => '██████████';

  @override
  String chatReplyTo(String name) {
    return '■ ██████ $name';
  }

  @override
  String get chatReplyDeleted => '███████████ ███████████';

  @override
  String get chatReplyNotFound => '██████████ ███████████ ████████████';

  @override
  String get chatMessageHint => '████████████…';

  @override
  String get chatAttachImage => '█████';

  @override
  String get chatAttachVideo => '██████';

  @override
  String get chatAttachFile => '█████';

  @override
  String chatTyping(String names) {
    return '$names ██████████…';
  }

  @override
  String chatRecording(String time) {
    return '███████ $time';
  }

  @override
  String chatUploading(String name) {
    return '██████████ $name…';
  }

  @override
  String chatSendFailed(String error) {
    return '██ ████████ ████████████: $error';
  }

  @override
  String get chatMicDenied =>
      '████ ████████████ ████████████ ██████ ███████ ■ ████████████.';

  @override
  String get chatEditTitle => '██████████ ████████████';

  @override
  String get chatDeleteTitle => '████████ ████████████?';

  @override
  String get chatDeleteBody => '████████████ ██████ ████████ ████ █████ ■ █████████.';

  @override
  String chatDeleteSelectedTitle(int count) {
    return '████████ $count ██████.?';
  }
```

```
  @override
  String get chatDeleteSelectedBody =>
      '██████████ ██████████ ██████ ████████ ███ ████ ■ ████████.';

  @override
  String chatSelectedCount(int count) {
    return '████████: $count';
  }

  @override
  String get chatSelectMessages => '████████ ██████████';

  @override
  String get chatSelect => '████████';

  @override
  String get commonSelectAll => '████████ ███';

  @override
  String get chatEdited => '███.';

  @override
  String get chatDateToday => '████████';

  @override
  String get chatDateYesterday => '██████';

  @override
  String get chatRoleAdmin => '██████';

  @override
  String get chatRoleTeacher => '████████';

  @override
  String get commonSave => '██████████';

  @override
  String get commonCancel => '███████';

  @override
  String get commonSubmit => '██████████';

  @override
  String get commonDelete => '████████';

  @override
  String get commonEdit => '█████████';

  @override
  String get commonClose => '████████';

  @override
  String get commonRetry => '██████████';

  @override
  String get commonTryAgain => '████████████ ██████';

  @override
  String get commonLoading => '█████████…';

  @override
  String get commonError => '███-██ ██████ ██ ███';

  @override
  String commonErrorWithDetails(String error) {
    return '███████: $error';
  }

  @override
  String commonCouldNotSave(String error) {
    return '██ ████████ ██████████: $error';
  }
```

```
  @override
  String get commonSignOut => '██████';

  @override
  String get commonSaveDraft => '██████████ █████████';

  @override
  String get commonOpen => '████████';

  @override
  String get commonCreate => '████████';

  @override
  String get commonAssign => '██████████';

  @override
  String get commonReset => '█████████';

  @override
  String get commonAdd => '█████████';

  @override
  String get commonLoadMore => '██████████ ████';

  @override
  String get commonExport => '████████';

  @override
  String commonExportFailed(String error) {
    return '██ ████████ ████████████████: $error';
  }

  @override
  String get marksTermAll => '██ ███ ██████';

  @override
  String marksTermSemester(String yearRange, int semester) {
    return '$yearRange · ████████ $semester';
  }

  @override
  String get marksExportCsv => '████████ CSV';

  @override
  String get marksExportDone => '███████ ████████████████';

  @override
  String get commonTeacher => '████████';

  @override
  String get commonStudent => '███████';

  @override
  String get commonAdmin => '███████████████';

  @override
  String get commonWorkspace => '████████ ████████████';

  @override
  String get commonNoDueDate => '████ ██████';

  @override
  String commonDueDate(String date) {
    return '█████ $date';
  }

  @override
  String get homeworkDueThisWeek => '██ █████ ███████';

  @override
  String get homeworkDueThisWeekSubtitle => '██████████████ ■ ██████████ ██████';

  @override
```

```
  String get homeworkNoDueThisWeek => '██ ████ ██████ ██████ ███';

  @override
  String get homeworkNoDueThisWeekSubtitle =>
      '██ ██████████ ██████████ ██████ █████████.';

  @override
  String homeworkOverdue(String date) {
    return '██████████ · $date';
  }

  @override
  String get homeworkDueToday => '████ ███████';

  @override
  String get homeworkDueTomorrow => '████ ██████';

  @override
  String homeworkDueInDays(int days) {
    return '█████ $days ██.';
  }

  @override
  String get homeworkLateBadge => '██████████';

  @override
  String get commonProgress => '█████████';

  @override
  String get commonSubject => '████████';

  @override
  String get commonType => '███';

  @override
  String get commonTitle => '█████████';

  @override
  String get commonDescription => '█████████';

  @override
  String get commonGroup => '██████';

  @override
  String get commonMessage => '██████████';

  @override
  String get commonScore => '███████';

  @override
  String get commonFeedback => '██████';

  @override
  String get commonStudents => '████████';

  @override
  String get commonCannotUndo => '███ █████████ ██████ █████████.';

  @override
  String get profileTitle => '████████';

  @override
  String get profileFullName => '███████ ███';

  @override
  String get profileNotSignedIn => '██ ██ ██████';

  @override
  String get profileAccount => '████████';

  @override
  String get profileAccountSettings => '██████████ █████████';
```

```
  @override
  String get profileSignOutAdmin => '[illegible]';

  @override
  String get profileSettings => '[illegible]';

  @override
  String get profileAppearance => '[illegible]';

  @override
  String get profileThemeLight => '[illegible]';

  @override
  String get profileThemeDark => '[illegible]';

  @override
  String get profileThemeSystem => '[illegible]';

  @override
  String get profileStyle => '[illegible]';

  @override
  String get profileStyleModern => '[illegible]';

  @override
  String get profileStyleNature => '[illegible]';

  @override
  String get profileStyleSoftNeu => '[illegible]';

  @override
  String get profileStyleMonochrome => '[illegible]';

  @override
  String get profileEditName => '[illegible]';

  @override
  String get profileNameUpdated => '[illegible]';

  @override
  String get profileChangePhoto => '[illegible]';

  @override
  String get profilePhotoUpdated => '[illegible]';

  @override
  String get profileChangePassword => '[illegible]';

  @override
  String get profileCurrentPassword => '[illegible]';

  @override
  String get profileNewPassword => '[illegible]';

  @override
  String get profileConfirmPassword => '[illegible]';

  @override
  String get profilePasswordChanged => '[illegible]';

  @override
  String get profilePasswordMismatch => '[illegible]';

  @override
  String get profilePasswordTooShort => '[illegible] 8 [illegible]';

  @override
  String get profilePasswordHint =>
      '[illegible].';

  @override
  String get profileShortcuts => '[illegible]';
```

```
  @override
  String get profileYourGroups => '████ ██████';

  @override
  String get profileNoGroups => '██████ ██ ██████████';

  @override
  String profileGroupsCount(int count) {
    return '█████: $count';
```

**lib/l10n/app_localizations_ru.dart — continued (2/4)**

```
  }

  @override
  String get profileAverageLabel => '███████ ██ ██████';

  @override
  String get profileAttendanceLabel => '█████████████';

  @override
  String get profileShareProgress => '██████████ ███████';

  @override
  String get profileNotifications => '████████████';

  @override
  String get profileClearCache => '█████████ ███';

  @override
  String get profileClearCacheWarn1Title => '█████████ ███?';

  @override
  String get profileClearCacheWarn1Body =>
      '██████ ████████ ███████-██████████ ████ ▪ ██████████ ████████████, ██████, ██████████ ▪ ██████.
███ ██████████ █████████ ████████████ █████████.';

  @override
  String get profileClearCacheWarn2Title => '██ ████████?';

  @override
  String get profileClearCacheWarn2Body =>
      '███ █████████ ███████ █████████. ███ ████ ██████ ███████ ▪ ██████ ███████████.';

  @override
  String get profileClearCacheConfirm => '█████████ ███';

  @override
  String get profileClearCacheDone => '███ ███████';

  @override
  String profileAppVersion(String version) {
    return 'MU Study · v$version';
  }

  @override
  String get profileSubject => '████████';

  @override
  String get greetingMorning => '███████ ████';

  @override
  String get greetingAfternoon => '███████ ████';

  @override
  String get greetingEvening => '███████ ██████';

  @override
  String greetingWithName(String greeting, String name) {
    return '$greeting, $name';
  }

  @override
```

```
  String get greetingLoadingTitle => '[redacted]';

  @override
  String get greetingLoadingSubtitle => '[redacted]...';

  @override
  String get studentHomeSubtitle => '[redacted]';

  @override
  String get continueLearning => '[redacted]';

  @override
  String get continueLearningSubtitle =>
      '[redacted]';

  @override
  String get everythingCompleted => '[redacted].';

  @override
  String get caughtUpToday => '[redacted].';

  @override
  String get homeworkPack => '[redacted]';

  @override
  String get staticTask => '[redacted]';

  @override
  String get staticAssignment => '[redacted]';

  @override
  String get upcomingAssignments => '[redacted]';

  @override
  String get stayOnTrack => '[redacted]';

  @override
  String get loadingAnnouncements => '[redacted]...';

  @override
  String get notificationsCardTitle => '[redacted]';

  @override
  String unreadAlerts(int count) {
    return '[redacted] $count [redacted]';
  }

  @override
  String get upToDate => '[redacted]';

  @override
  String get announcementsTitle => '[redacted]';

  @override
  String get announcementsSubtitle => '[redacted]';

  @override
  String get announcementsEmpty => '[redacted]';

  @override
  String get announcementsEmptyTeacher =>
      '[redacted].';

  @override
  String get announcementsEmptyAdmin =>
      '[redacted].';

  @override
  String get announcementsReadMore => '[redacted]';

  @override
  String get announcementsShowLess => '[redacted]';
```

```
@override
String announcementsViewAll(int count) {
  return '[redacted] [redacted] ($count)';
}

@override
String get announcementsAll => '[redacted] [redacted]';

@override
String get announcementsNew => '[redacted] [redacted]';

@override
String get announcementsPost => '[redacted] [redacted]';

@override
String get announcementsEdit => '[redacted] [redacted]';

@override
String get announcementsDeleteTitle => '[redacted] [redacted]?';

@override
String get announcementsDeleteBody =>
    '[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted].';

@override
String get announcementsByAdmin => '[redacted]';

@override
String get announcementsShareUpdates => '[redacted] [redacted] [redacted] [redacted]';

@override
String get assignmentsTitle => '[redacted]';

@override
String get assignmentsEmpty => '[redacted] [redacted]';

@override
String get assignmentsEmptySubtitle => '[redacted] [redacted] [redacted] [redacted]!';

@override
String get assignmentsEmptyTeacher =>
    '[redacted] [redacted] [redacted], [redacted] [redacted].';

@override
String get assignmentsLearningPath => '[redacted] [redacted]';

@override
String get assignmentsActive => '[redacted] [redacted]';

@override
String get assignmentDetails => '[redacted] [redacted]';

@override
String get assignmentNotFound => '[redacted] [redacted] [redacted]';

@override
String get assignmentTasks => '[redacted]';

@override
String get assignmentTasksSubtitle => '[redacted] [redacted], [redacted] [redacted] [redacted]';

@override
String get assignmentNew => '[redacted] [redacted]';

@override
String get assignmentManage => '[redacted] [redacted]';

@override
String get assignmentManageSubtitle =>
    '[redacted] [redacted] [redacted] [redacted] [redacted]';

@override
String get marksTitle => '[redacted] [redacted]';
```

```
@override
String get marksOverallAverage => '██████ ████████ █████';

@override
String get marksNoneYet => '███████ █████ ████.';

@override
String marksBasedOn(String label, int count) {
  return '$label · ██ ███████ $count ███████.';
}

@override
String get marksByCategory => '██ ████████████';

@override
String get marksWhereStrongest => '████ ██ ████████ ██████';

@override
String get marksAll => '████ ███████';

@override
String get marksNewestFirst => '████████ ██████';

@override
String get marksEmptyTitle => '███████ █████ ████';

@override
String get marksEmptySubtitle => '███████ ████████ ██████████ ██████.';

@override
String get progressReportTitle => '██████ ██ ██████████████';

@override
String get progressReportShare => '████████████ ████████████████';

@override
String get progressReportSharing => '████████ ██████…';

@override
String get progressReportDone => '██████ ██████ ■ ██████████';

@override
String progressReportFailed(String error) {
  return '██ ████████ ████████████: $error';
}

@override
String get progressReportNeedGroup =>
    '██████████ ■ ███████, ██████ ████████████ ████████.';

@override
String get progressReportAttendance => '████████████████';

@override
String get progressReportAttendanceRate => '████████████████';

@override
String get progressReportNoAttendance =>
    '██ ██████ ███████ ████████████████ ██ ██████████.';

@override
String get progressReportSessions => '████████';

@override
String get progressReportMarksCounted => '███████';

@override
String get progressReportRecentMarks => '██████████ ███████';

@override
String progressReportGeneratedOn(String date) {
  return '████████████████ $date';
}
```

```
  @override
  String get marksNoRecorded => '███████ ███';

  @override
  String get marksNoRecordedSubtitle =>
      '████████████ «████████ ██████», ██████ ██████████ ███████ ██████.';

  @override
  String get marksRecent => '█████████ ███████';

  @override
  String get marksRecentSubtitle => '██████████ ███████ ██ ███████';

  @override
  String get marksStudentsHeader => '████████';

  @override
  String get marksStudentsSubtitle => '██████████████████ █████ ████ ███████';

  @override
  String get marksNoStudents => '█ ████ ███████ ███ █████████';

  @override
  String get marksNoStudentsSubtitle =>
      '███████████████ ██████ ██████████ ████████ ■ ████ ███████.';

  @override
  String get marksDeleteTitle => '████████ ███████?';

  @override
  String marksDeleteBody(String title) {
    return '«$title» ██████ ████████ █████████.';
  }

  @override
  String get marksGradeTheClass => '████████ ██████';

  @override
  String get marksOnePercentEach => '████ ███████████ ███████ ██ ████████';

  @override
  String get marksSaveMarks => '██████████ ███████';

  @override
  String get marksSaveMark => '██████████ ███████';

  @override
  String get marksAddMark => '█████████ ███████';

  @override
  String get marksNothingYet => '█████ ███████ ███';

  @override
  String get marksAddFirst => '█████████ ███████ ███████ ██████ ████████.';

  @override
  String marksSavedCount(int count) {
    return '██████████ ███████: $count';
  }

  @override
  String get marksGroupHasNoStudents => '■ ████ ███████ ████ ███ █████████.';

  @override
  String get marksTitleHint => '█████████, ████ ██ ████ 3';

  @override
  String marksAverageLine(String percent, String label) {
    return '████████ $percent · $label';
  }

  @override
  String get marksNoMarksYetShort => '███████ ████ ███';
```

```
  @override
  String get marksSummaryStudents => '████████';

  @override
  String get marksSummaryMarks => '███████';

  @override
  String get attendanceTitle => '██████████████';

  @override
  String get attendanceEmpty => '████ ████████ ██████████████';

  @override
  String get attendanceNoGroup => '███████ ████ ██ ██████████';

  @override
  String get attendanceMark => '█████████ ██████████████';

  @override
  String get attendanceDailyTracking => '████████████ █████ █████████';

  @override
  String get attendanceSaved => '██████████████ ████████ ██████████';

  @override
  String get attendanceNoGroups => '███████ ██ ████████';

  @override
  String get attendancePresent => '█ — ████████████████';

  @override
  String get attendanceLate => '█ — ████████';

  @override
  String get attendanceAbsent => '█ — ██████████████';

  @override
  String get attendanceExcused => '█ — ██████████████';

  @override
  String get attendancePresentShort => '█';

  @override
  String get attendanceLateShort => '█';

  @override
  String get attendanceAbsentShort => '█';

  @override
  String get attendanceExcusedShort => '█';

  @override
  String get attendanceTrend => '█████████ ██████████████';

  @override
  String get attendanceOverall => '██████ ████████████████';

  @override
  String get analyticsThisWeek => '████ ███████';

  @override
  String get analyticsLastWeek => '████████ ███████';

  @override
  String analyticsTotalMarks(int count) {
    return '██████ ███████: $count';
  }

  @override
  String get notificationsTitle => '█████████████';

  @override
  String get notificationsEmpty => '████ ██████████!';
```

```
  @override
  String get notificationsEmptySubtitle => '██████ ████████████ ████.';

  @override
  String get notificationsNone => '████ ████████████';

  @override
  String get notifKindAssignment => '██████ ██';

  @override
  String get notifKindAnnouncement => '████████████';

  @override
  String get notifKindGraded => '███████ ████████';

  @override
  String get notifKindSubmission => '██████ ██████';

  @override
  String get notifKindChat => '██████████ ■ █████';

  @override
  String get notifKindSystem => '████████████';

  @override
  String get notificationsOpen => '████████';

  @override
  String get taskTitle => '████████';

  @override
  String get taskYourAnswer => '████ ██████';

  @override
  String get taskMarkCompleted => '█████████ ████████████';

  @override
  String get taskAttachment => '█████████';

  @override
  String get taskAttachmentHint => '█████████ █████ ████ █████████ ■ ███████.';

  @override
  String get taskAttachmentHintReady =>
      '████████, ██████ ████████ · ████████ — ████████.';

  @override
  String get taskImage => '████████████';

  @override
  String get taskImageSubtitle => '█████ ██ ████████';

  @override
  String get taskFile => '█████';

  @override
  String get taskFileSubtitle => 'PDF ████ █████████';

  @override
  String get taskReplaceImage => '█████████ ██████████████';

  @override
  String get taskReplaceFile => '█████████ █████';

  @override
  String get taskVoice => '██████████ ███████';

  @override
  String get taskVoiceHint =>
      '████████████████ ███████ ██████ — █████████, ████████████ ■ ██████████.';

  @override
  String get taskViewPdf => '████████ PDF';
```

```
  @override
  String get taskOpenLink => '████████ ███████';

  @override
  String get taskSubmitted => '████████████';

  @override
  String get taskSaved => '██████████';

  @override
  String taskUploadFailed(String error) {
    return '██ ████████ ██████████: $error';
  }

  @override
  String taskVoiceUploadFailed(String error) {
    return '██ ████████ ██████████ ██████: $error';
  }

  @override
  String taskCouldNotSave(String error) {
    return '██ ████████ ██████████: $error';
  }

  @override
  String get voiceTapToRecord => '████████ ██ █████████, ██████ █████████ ██████';

  @override
  String get voiceRecording => '███████…';

  @override
  String get voiceNote => '██████████ ████████';

  @override
  String get voiceTapPlay => '████████ ████████████████, ██████ ████████████';

  @override
  String get voiceRerecord => '███████████████';

  @override
  String get voiceStudentAnswer => '██████████ ██████ ████████';

  @override
  String get voiceMicPermission => '███ ███████ ██████ ███████ ■ ██████████.';

  @override
  String voiceCouldNotStart(String error) {
    return '██ ████████ ███████ ███████: $error';
  }

  @override
  String voiceCouldNotSave(String error) {
    return '██ ████████ ██████████ ███████: $error';
  }

  @override
  String voiceCouldNotPlay(String error) {
    return '██ ████████ ████████████████: $error';
  }

  @override
  String get attachmentCouldNotOpen => '██ ████████ ████████ █████ █████.';

  @override
  String get attachmentImage => '████████████';

  @override
  String get attachmentFile => '█████';

  @override
  String get attachmentPdf => 'PDF-█████████';

  @override
```

```
  String get attachmentTapPreview => '████████ ███ ██████████';

  @override
  String get attachmentTapOpen => '████████, ██████ ████████';

  @override
  String get pdfDefaultTitle => 'PDF';

  @override
  String get teacherWorkspace => '████████ ████████████ ████████';

  @override
  String get teacherQuickActions => '████████ █████████';

  @override
  String get teacherCreateItems => '████████ ██████ █████████';

  @override
  String get teacherNewAssignment => '██████ ████████';

  @override
  String get teacherNewAnnouncement => '██████ ███████████';

  @override
  String get teacherSubmissions => '███████';

  @override
  String get teacherNoSubmissions => '██████ █████ ███';

  @override
  String get teacherNoSubmissionsSubtitle =>
      '███████ █████████ █████████ ██████ ███ ██████████.';

  @override
  String get teacherGradebook => '███████';

  @override
  String get teacherGradeClass => '████████ ██████';

  @override
```

**lib/l10n/app_localizations_ru.dart — continued (3/4)**

```
  String get teacherNoGroups => '██████ █████ ███';

  @override
  String get teacherNoGroupsSubtitle =>
      '██████████ ████████████████ ██████████ ███ ███████.';

  @override
  String get teacherActiveGroups => '█████████ ███████';

  @override
  String get teacherSubject => '████████';

  @override
  String get teacherDailyTasks => '████████████ ███████';

  @override
  String get teacherNeedsAttention => '███ ████████ ██████████';

  @override
  String get teacherReviewSubmissions => '██████████ ███████';

  @override
  String get teacherCheckAssignments => '██████████ ████████ ██████████';

  @override
  String get teacherRecordMarks => '██████████ ███████ ■ ███████████';

  @override
  String get teacherPerformanceAnalytics => '██████████ ██████████████';

  @override
```

```
  String get teacherTrackProgress => '[redacted]';

  @override
  String get teacherOpenAttachment => '[redacted]';

  @override
  String get teacherScoreHelper => '[redacted]';

  @override
  String get teacherScoreRange => '[redacted] 0 [redacted] 100.';

  @override
  String get teacherNoWrittenAnswer => '[redacted].';

  @override
  String get teacherReviewSubmission => '[redacted]';

  @override
  String get teacherHomeworkPackTasks => '[redacted]';

  @override
  String get teacherAddTask => '[redacted]';

  @override
  String get teacherEditTask => '[redacted]';

  @override
  String get teacherDeleteTask => '[redacted]?';

  @override
  String get teacherTasksContent => '[redacted]';

  @override
  String get teacherManagePack => '[redacted]';

  @override
  String get teacherTaskTitleHint => '[redacted]';

  @override
  String get teacherLinkHint => 'https://...';

  @override
  String get teacherAttachmentHint => '[redacted]...';

  @override
  String get teacherNoData => '[redacted]';

  @override
  String get teacherMarkDistribution => '[redacted]';

  @override
  String get teacherStudentsByAverage => '[redacted]';

  @override
  String get teacherNoMarksTrends =>
      '[redacted].';

  @override
  String get teacherTargetGroup => '[redacted]';

  @override
  String get adminWorkspace => '[redacted]';

  @override
  String get adminUsers => '[redacted]';

  @override
  String get adminGroups => '[redacted]';

  @override
  String get adminAnalytics => '[redacted]';

  @override
```

```
  String get adminTeachers => '████████';

  @override
  String get adminStudents => '████████';

  @override
  String get adminGroupsLabel => '███████';

  @override
  String get adminMarksLabel => '███████';

  @override
  String get adminActivityInsights => '██████████ ███████████';

  @override
  String get adminCurrentMetrics => '████████ ███████████';

  @override
  String get adminCreateUser => '████████ ██████████████';

  @override
  String get adminCreateUserButton => '████████ ██████████████';

  @override
  String get adminUserCreated => '██████████████ ████████ ███████';

  @override
  String get adminResetPassword => '█████████ ███████';

  @override
  String adminResetPasswordTitle(String username) {
    return '██████ ███████ · $username';
  }

  @override
  String get adminCreateGroup => '████████ ███████';

  @override
  String get adminCreateGroupButton => '████████ ███████';

  @override
  String get adminGroupTitleHint => '█████████, ███████ 4';

  @override
  String get adminGroupTitleExists =>
      '█ ██████ ████████ ███ ████ ███████ █ ██████ ██████████';

  @override
  String get adminAssignStudent => '██████████ ████████';

  @override
  String get adminActivityDistribution => '███████████████ ███████████';

  @override
  String get adminGlobalMetrics => '███████████ ███████████ ██████████';

  @override
  String get adminActivityLog => '███████ ███████████ ████████████████';

  @override
  String get adminActivityLogSubtitle =>
      '██████████ ████████ █ ██████ █████████ ██████.';

  @override
  String adminTeacherLine(String name) {
    return '████████: $name';
  }

  @override
  String adminRoleLine(String role, String username) {
    return '$role · $username';
  }
```

```
  @override
  String get adminDisable => '[redacted]';

  @override
  String get adminEnable => '[redacted]';

  @override
  String get adminPassword => '[redacted]';

  @override
  String get adminFullName => '[redacted]';

  @override
  String get adminRole => '[redacted]';

  @override
  String get adminSphere => '[redacted]';

  @override
  String get percentExcellent => '[redacted]';

  @override
  String get percentGood => '[redacted]';

  @override
  String get percentNeedsWork => '[redacted]';

  @override
  String get percentAtRisk => '[redacted]';

  @override
  String get emptyStateDefault => '[redacted]';

  @override
  String get setupRequiredTitle => '[redacted]';

  @override
  String get setupRequiredBody =>
      'Supabase [redacted]. [redacted] SUPABASE_URL [redacted] SUPABASE_ANON_KEY [redacted] [redacted] .env.';

  @override
  String get setupOption1 => '[redacted] 1 — [redacted] [redacted] .env';

  @override
  String get setupOption2 => '[redacted] 2 — dart-define';

  @override
  String get attendanceAllPresent => '[redacted]';

  @override
  String get attendanceAllAbsent => '[redacted]';

  @override
  String get attendanceAllLate => '[redacted]';

  @override
  String get attendanceAllExcused => '[redacted]';

  @override
  String get attendancePresentWord => '[redacted]';

  @override
  String get attendanceAbsentWord => '[redacted]';

  @override
  String get attendanceLateWord => '[redacted]';

  @override
  String get attendanceExcusedWord => '[redacted]';

  @override
  String get attendanceWeekdayMon => '[redacted]';
```

```
  @override
  String get attendanceWeekdayTue => '■■';

  @override
  String get attendanceWeekdayWed => '■■';

  @override
  String get attendanceWeekdayThu => '■■';

  @override
  String get attendanceWeekdayFri => '■■';

  @override
  String get attendancePercentLabel => '■■■■';

  @override
  String get attendanceNoLesson => '■■■ ■■■■■';

  @override
  String get attendanceHistory => '■■■■■■■ ■■■■■■■■■■■■';

  @override
  String get attendanceSelectStudent => '■■■■■■■';

  @override
  String get attendanceAllStudents => '■■■ ■■■■■■■';

  @override
  String get attendanceTapToCycle => '■■■■■■■ ■■ ■■■■, ■■■■■ ■■■■■■■■ ■■■■■■■';

  @override
  String get attendanceUpdated => '■■■■■■■■■■■■ ■■■■■■■■■■';

  @override
  String get attendanceCalendarEmpty => '■■■■ ■■■■■■■■■■■■ ■ ■■■■■ ■■■■■■■';

  @override
  String get attendanceFutureNotAllowed =>
      '■■■■■■■ ■■■■■■■■■ ■■■■■■■■■■■■■ ■■ ■■■■■■■■ ■■■■■';

  @override
  String get attendanceWeekendNotAllowed =>
      '■■■■■■■■■■■■■ ■■■■■■■■■■■ ■■■■■■■ ■ ■■■■■■ (■■–■■)';

  @override
  String get chartBandExcellent => '85–100%';

  @override
  String get chartBandGood => '70–84%';

  @override
  String get chartBandNeedsWork => '50–69%';

  @override
  String get chartBandAtRisk => '■■■■■ 50%';

  @override
  String get marksWeight => '■■■■';

  @override
  String get marksCategory => '■■■■■■■■■■';

  @override
  String get marksDate => '■■■■■';

  @override
  String get marksCommentOptional => '■■■■■■■■■■■■■ (■■■■■■■■■■■■■■■)';

  @override
  String get marksNewMark => '■■■■■■ ■■■■■■■';

  @override
  String get marksEditMark => '■■■■■■■■■ ■■■■■■■';
```

```
  @override
  String get marksFromSubmission => '■■ ■■■■■■■■■■■■ ■■■■■■■';

  @override
  String get marksNeedTitle => '■■■■■■■■ ■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■.';

  @override
  String get marksWeightPositive => '■■■■ ■■■■■■■ ■■■■■ ■■■■■■■ ■■■■■.';

  @override
  String get marksNeedAtLeastOne => '■■■■■■■■ ■■■■■■■ ■■■■■ ■■ ■■■■■■■ ■■■■■■■■.';

  @override
  String get marksLeaveBlankSkip =>
      '■■■■■■■■■ ■■■■■ ■■■■■■■, ■■■■■■ ■■■■■■■■■■■ ■■■■■■■■.';

  @override
  String get marksFromSubmissionDelete =>
      '■■■■ ■■■■■■■ ■■ ■■■■■■■■■■■■■ ■■■■■■■. ■■■■■■ ■■■■■■■■ ■■, ■■■■■■■■■ ■■■■■ ■ ■■■■■■■■ «■■■■■■■».';

  @override
  String get marksFromSubmissionEdit =>
      '■■■■ ■■■■■■■ ■■■■■■■■ ■ ■■■■■■■■■■■■■ ■■■■■■■■. ■■■■■■■■■■ ■■■■■■ ■■■■■■ ■■■■■■■■■■■■■■ ■■■■
■■■■■■■■■■ ■■■■■■■■■.';

  @override
  String get categoryHomework => '■■■■■■■■■ ■■■■■■■■';

  @override
  String get categoryClasswork => '■■■■■■■■■ ■■■■■■■';

  @override
  String get categoryQuiz => '■■■■■';

  @override
  String get categoryTest => '■■■■■';

  @override
  String get categoryExam => '■■■■■■■■';

  @override
  String get categoryProject => '■■■■■■■';

  @override
  String get categoryOral => '■■■■■■';

  @override
  String get categoryOther => '■■■■■■■';

  @override
  String get teacherReferenceLinkOptional => '■■■■■■■ (■■■■■■■■■■■■■■■)';

  @override
  String get teacherAttachmentUrlOptional => 'URL ■■■■■■■■■ (■■■■■■■■■■■■■■■)';

  @override
  String get teacherNoTasksYet =>
      '■■■■■■ ■■■■■ ■■■■. ■■■■■■■■ «■■■■■■■■■ ■■■■■■■», ■■■■■■ ■■■■■■■.';

  @override
  String get teacherTemplates => '■■■■■■■■';

  @override
  String get teacherTemplatesEmpty => '■■■■■■■■■ ■■■■■ ■■■■';

  @override
  String get teacherTemplatesEmptySubtitle =>
      '■■■■■■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■, ■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■.';

  @override
  String get teacherSaveAsTemplate => '■■■■■■■■■■ ■■■■ ■■■■■■■';

  @override
```

```
  String get teacherTemplateSaved => '██████ ████████';

  @override
  String get teacherUseTemplate => '████████████ ██████';

  @override
  String get teacherFromTemplate => '██ ███████';

  @override
  String get teacherDeleteTemplate => '███████ ██████?';

  @override
  String teacherDeleteTemplateBody(String title) {
    return '«$title» ██████ ██████.';
  }

  @override
  String get teacherDuplicateAssignment => '████████████';

  @override
  String get teacherDuplicateTitle => '██████████ ███████';

  @override
  String get teacherDuplicated => '███████ ████████████';

  @override
  String get assignmentEdit => '████████ ███████';

  @override
  String get assignmentDeleteTitle => '███████ ███████?';

  @override
  String get assignmentDeleteBody =>
      '███████, ██████ ■ ██████ ████████ ██████ ███████ ██████████████.';

  @override
  String get assignmentDeleted => '███████ ███████';

  @override
  String get assignmentTypeHintStatic =>
      '████ ███████. ██████ ██████ ████ ███ — ███ ████████████████ ██████.';

  @override
  String get assignmentTypeHintPack =>
      '██████ ██ ██████████ ██████. ██████ ████████ ███████.';

  @override
  String get teacherCreateFromTemplate => '███████ ██ ███████';

  @override
  String teacherTasksCopied(int count) {
    return '██████: $count';
  }

  @override
  String get teacherRecentFeedback => '████████ ████████████ ██████';

  @override
  String get teacherApplyFeedbackToOthers =>
      '██████ ██████████ ■ ██████ ██████████ ██ ████ ███████';

  @override
  String teacherFeedbackAppliedCount(int count) {
    return '██████ ████████ ■ $count ██████ ███████';
  }

  @override
  String get teacherSharedComment => '██████ ████████████ (███████████████)';

  @override
  String get teacherSharedCommentHint =>
      '████ ■ ██ ██ ███████ ██ ████ ████████████ ███████';
```

```
  @override
  String get adminPlatformOverview => '██████ ██████████';

  @override
  String get adminAssignTeacher => '██████████ ████████';

  @override
  String get adminSelectStudent => '████████ ████████';

  @override
  String get adminInitialGroupOptional => '██████████ ███████ (███████████████)';

  @override
  String get adminChartTasks => '███████';

  @override
  String get adminChartGroups => '███████';

  @override
  String get adminChartStaff => '███████████';

  @override
  String get adminChartAssignments => '████████';

  @override
  String get adminChartMarks => '███████';

  @override
  String get adminChartStudents => '████████';

  @override
  String get adminAcademicYears => '████████ █████';

  @override
  String get adminAcademicYearsSubtitle => '█████ ■ ██████████ ████ ████████ ███████';

  @override
  String get adminAcademicYearsTitle => '████████ ██████████';

  @override
  String get adminCreateNextYear => '████████ ██████████ ████';

  @override
  String adminYearCreated(String label) {
    return '███████ $label';
  }

  @override
  String get adminSetCurrentYear => '████████ ████████';

  @override
  String get adminCurrentYearBadge => '████████';

  @override
  String get adminYearSemesters => '███████ ██████████ · ████ → ████';

  @override
  String adminSemesterRange(int semester, String start, String end) {
    return '████████ $semester · $start – $end';
  }

  @override
  String get adminEditTerm => '██████████ ██████';

  @override
  String adminEditTermTitle(int semester) {
    return '████████ $semester';
  }

  @override
  String get adminTermStartsOn => '███████';

  @override
```

```
  String get adminTermEndsOn => '██████';

  @override
  String get adminTermDatesInvalid =>
      '█████ ██████████ ███████ █████ ██ ███████ █████ ███████';

  @override
  String get adminTermUpdated => '█████ █████████ ██████████';

  @override
  String get adminDeleteYear => '████████ ███';

  @override
  String adminDeleteYearTitle(String label) {
    return '████████ $label?';
  }

  @override
  String adminDeleteYearBody(String label) {
    return '███ ■ ███████ █████████ ██████ ████████ ██ ██████████. ███████ ██ ██████████. ███
██████████████ ████████ $label █████.';
  }

  @override
  String adminDeleteYearConfirmHint(String label) {
    return '████████ $label ███ ███████████████';
  }

  @override
  String get adminDeleteYearCurrentBlocked =>
      '████████ █████████ ████████ ███████ ███, ██████ ████████ █████.';

  @override
  String adminYearDeleted(String label) {
    return '███████ $label';
  }

  @override
  String get adminNoAcademicYears => '████████ ██████ █████ ███';

  @override
  String get adminAcademicYearsHint =>
      '█████████ ██████████ ████████ ███, ██████ ███████ ███████. ████████ ■ ████████ █████████ █████████
■ ████████. ████████ ██ ████████, ██████ █████████ █████.';

  @override
  String get adminCredentialPrintTitle => '███████ ████████';

  @override
  String get adminCredentialPrintSubtitle =>
      '████████████ ███████ ██ █████, 3 █████████ ██ █████████';

  @override
  String get adminCredentialPasteLabel => '███████████████';

  @override
```

**lib/l10n/app_localizations_ru.dart — continued (4/4)**

```
  String get adminCredentialPasteHint =>
      'username|password|full name\n@james|Abc123xyz|James English|teacher\
n@john_jones|Pass456|John Jones';

  @override
  String get adminCredentialPerPage => '█████████ ██ █████████';

  @override
  String get adminCredentialTwo => '2';

  @override
  String get adminCredentialThree => '3';

  @override
```

```
  String get adminCredentialExportPdf => '████████ PDF';

  @override
  String get adminCredentialSlipTitle => '███ ████ ■ MU Study';

  @override
  String get adminCredentialSlipHint =>
      '████████ ████ ██████ ███ ████. ████████ ██████ ██████ ████████ ██████.';

  @override
  String get adminCredentialNeedUsers => '███ ████████████ ███ ███████';

  @override
  String get adminCredentialNeedSql =>
      '██████████ sql/admin_login_slips.sql ■ SQL Editor Supabase, ██████ ████████████ ██████.';

  @override
  String get adminCredentialConfidential =>
      '████████████████. ███ ███████ ███████ ███ ████████████ ████████.';

  @override
  String get adminFirstName => '███';

  @override
  String get adminLastName => '████████';

  @override
  String get adminCredentialStudents => '████████';

  @override
  String get adminCredentialTeachers => '████████';

  @override
  String get adminCredentialAllGroups => '███ ███████';

  @override
  String get sphereMath => '███████████';

  @override
  String get sphereEnglish => '███████████';

  @override
  String get dailyMotivation1 =>
      '████████ █████████ ████, ██████ ████████████ ███████.';

  @override
  String get dailyMotivation2 =>
      '███████ ██████████ ■ ████ ██████, ███ ██████████████?';

  @override
  String get dailyMotivation3 => '████████ ████████ ██████████ ██████ ██████.';

  @override
  String get dailyMotivation4 => '████████████ — ████ ■ ███████████.';
}
```

# app_localizations_uz.dart

**lib/l10n/app_localizations_uz.dart**

```
// ignore: unused_import
import 'package:intl/intl.dart' as intl;
import 'app_localizations.dart';

// ignore_for_file: type=lint

/// The translations for Uzbek (`uz`).
class AppLocalizationsUz extends AppLocalizations {
  AppLocalizationsUz([String locale = 'uz']) : super(locale);

  @override
  String get appName => 'MU Study';

  @override
  String get shellMuStudy => 'MU Study';

  @override
  String get languageTitle => 'Tilni tanlang';

  @override
  String get languageSubtitle =>
      'Buni istalgan vaqtda Profil bo‘limida o‘zgartirishingiz mumkin.';

  @override
  String get languageEnglish => 'Inglizcha';

  @override
  String get languageRussian => 'Ruscha';

  @override
  String get languageUzbek => 'O‘zbekcha';

  @override
  String get languageEnglishNative => 'English';

  @override
  String get languageRussianNative => '[redacted]';

  @override
  String get languageUzbekNative => 'O‘zbekcha';

  @override
  String get languageContinue => 'Davom etish';

  @override
  String get languageSection => 'Til';

  @override
  String get languageHint => 'Ilova tili';

  @override
  String get loginWelcome => 'Xush kelibsiz';

  @override
  String get loginSubtitle => 'MU Study ish maydoniga kiring';

  @override
  String get loginUsername => 'Foydalanuvchi nomi';

  @override
  String get loginPassword => 'Parol';

  @override
  String get loginSignIn => 'Kirish';

  @override
  String get loginMissingFields => 'Foydalanuvchi nomi va parolni kiriting';

  @override
```

```
  String get loginIncorrect => 'Foydalanuvchi nomi yoki parol noto‘g‘ri';

  @override
  String get loginFailed =>
      'Kirib bo‘lmadi. Ulanishni tekshirib, qayta urinib ko‘ring.';

  @override
  String get loginConfigError =>
      'Noto‘g‘ri sozlama. Qo‘llab-quvvatlashga murojaat qiling.';

  @override
  String get loginConfigMissing => 'Sozlama yo‘q. .env faylini tekshiring.';

  @override
  String get accountDisabled =>
      'Hisob o‘chirilgan. Administratorga murojaat qiling.';

  @override
  String get navDashboard => 'Bosh sahifa';

  @override
  String get navAssignments => 'Topshiriqlar';

  @override
  String get navMarks => 'Baholar';

  @override
  String get navAttendance => 'Davomat';

  @override
  String get navNotifications => 'Bildirishnomalar';

  @override
  String get navProfile => 'Profil';

  @override
  String get navUsers => 'Foydalanuvchilar';

  @override
  String get navGroups => 'Guruhlar';

  @override
  String get navAnalytics => 'Tahlil';

  @override
  String get navGradebook => 'Jurnal';

  @override
  String get navChat => 'Chat';

  @override
  String get navFocus => 'Fokus';

  @override
  String get focusTitle => 'Fokus';

  @override
  String get focusSubtitle => 'Tinch o‘qish — bir vaqtda bitta topshiriq.';

  @override
  String get focusTodaysFocus => 'Bugungi fokus';

  @override
  String get focusTodaysFocusSubtitle =>
      '3 tagacha ochiq topshiriq — muddat muhim emas';

  @override
  String get focusAlsoDue => 'Yana topshiriqlar';

  @override
  String get focusContinueHomework => 'Uy ishini davom ettirish';

  @override
```

```
  String get focusOpenAssignment => 'Topshiriqni ochish';

  @override
  String get focusEmptyTitle => 'Hali topshiriq yo‘q';

  @override
  String get focusEmptySubtitle =>
      'O‘qituvchi topshiriq e’lon qilganda, shu yerda fokus sessiyasini boshlashingiz mumkin.';

  @override
  String get focusChooseMode => 'Qanday fokususlanasiz?';

  @override
  String get focusModePomodoro => 'Pomodoro';

  @override
  String get focusModePomodoroHint => '25 daqiqa, so‘ng tanaffus';

  @override
  String get focusModeFree => 'Erkin sessiya';

  @override
  String get focusModeFreeHint => 'To‘xtatgunicha hisoblash';

  @override
  String get focusDoNotDisturb => 'Chat o‘chirilgan';

  @override
  String get focusChatMuteHint =>
      'Sessiya davomida chat belgilari yashirinadi. Bildirishnomalar fonda keladi.';

  @override
  String get focusPause => 'Pauza';

  @override
  String get focusResume => 'Davom etish';

  @override
  String get focusPaused => 'Pauza';

  @override
  String get focusEndSession => 'Tugatish';

  @override
  String get focusSessionComplete => 'Sessiya tugadi';

  @override
  String get focusSessionCompleteSubtitle => 'Yaxshi ish. Qisqa xulosa:';

  @override
  String get focusTimeUp => 'Vaqt tugadi!';

  @override
  String get focusTimeUpSubtitle =>
      'Pomodoro tugadi. Qisqa tanaffus oling, so‘ng davom eting.';

  @override
  String focusMinutesFocused(String duration) {
    return 'Fokus: $duration';
  }

  @override
  String get focusOptionalNote => 'Ixtiyoriy eslatma';

  @override
  String get focusOptionalNoteHint => 'masalan, 3-vazifada qiyinchilik';

  @override
  String get focusSaveAndOpen => 'Saqlash va topshiriqni ochish';

  @override
  String get focusDone => 'Tayyor';
```

```
  @override
  String get chatTitle => 'Chat';

  @override
  String get chatSearchHint => 'Guruh yoki o‘qituvchi qidirish';

  @override
  String get chatEmptyTitle => 'Hali chatlar yo‘q';

  @override
  String get chatEmptySubtitle => 'Sinfga yozish uchun guruhga qo‘shiling.';

  @override
  String get chatNoMessagesYet => 'Hali xabarlar yo‘q';

  @override
  String get chatMessageDeleted => 'Xabar o‘chirildi';

  @override
  String get chatPreviewImage => 'Rasm';

  @override
  String get chatPreviewVideo => 'Video';

  @override
  String get chatVideoPlaybackSpeed => 'Ijro tezligi';

  @override
  String get chatVideoSpeedNormal => '1x';

  @override
  String chatVideoSpeedValue(double speed) {
    final intl.NumberFormat speedNumberFormat =
        intl.NumberFormat.decimalPattern(localeName);
    final String speedString = speedNumberFormat.format(speed);

    return '${speedString}x';
  }

  @override
  String get chatVideoRewind10 => '-10s';

  @override
  String get chatVideoForward10 => '+10s';

  @override
  String get chatVideoCouldNotPlay => 'Videoni ijro etib bo‘lmadi';

  @override
  String get chatPreviewVoice => 'Ovozli xabar';

  @override
  String get chatPreviewFile => 'Fayl';

  @override
  String get chatOpenFile => 'Ochish';

  @override
  String get chatOpenFileFailed => 'Faylni ochib bo‘lmadi.';

  @override
  String get chatYou => 'Siz';

  @override
  String chatTeacherMembers(String teacher, int count) {
    return '$teacher · $count a’zo';
  }

  @override
  String chatOnlineCount(int count) {
    return '$count onlayn';
  }
```

```
@override
String get chatReact => 'Reaksiya';

@override
String get chatCopyText => 'Matnni nusxalash';

@override
String get chatCopied => 'Nusxa olindi';

@override
String get chatReply => 'Javob berish';

@override
String chatReplyTo(String name) {
  return '$name ga javob';
}

@override
String get chatReplyDeleted => 'O‘chirilgan xabar';

@override
String get chatReplyNotFound => 'Asl xabar topilmadi';

@override
String get chatMessageHint => 'Xabar…';

@override
String get chatAttachImage => 'Rasm';

@override
String get chatAttachVideo => 'Video';

@override
String get chatAttachFile => 'Fayl';

@override
String chatTyping(String names) {
  return '$names yozmoqda…';
}

@override
String chatRecording(String time) {
  return 'Yozilmoqda $time';
}

@override
String chatUploading(String name) {
  return '$name yuklanmoqda…';
}

@override
String chatSendFailed(String error) {
  return 'Yuborib bo‘lmadi: $error';
}

@override
String get chatMicDenied => 'Ovozli xabar uchun mikrofon ruxsati kerak.';

@override
String get chatEditTitle => 'Xabarni tahrirlash';

@override
String get chatDeleteTitle => 'Xabar o‘chirilsinmi?';

@override
String get chatDeleteBody => 'Xabar xonadagi barcha uchun o‘chiriladi.';

@override
String chatDeleteSelectedTitle(int count) {
  return '$count ta xabar o‘chirilsinmi?';
}

@override
```

```
  String get chatDeleteSelectedBody =>
      'Tanlangan xabarlar xonadagi barcha uchun o‘chiriladi.';

  @override
  String chatSelectedCount(int count) {
    return '$count ta tanlandi';
  }

  @override
  String get chatSelectMessages => 'Xabarlarni tanlash';

  @override
  String get chatSelect => 'Tanlash';

  @override
  String get commonSelectAll => 'Hammasini tanlash';

  @override
  String get chatEdited => 'tahrir';

  @override
  String get chatDateToday => 'Bugun';

  @override
  String get chatDateYesterday => 'Kecha';

  @override
  String get chatRoleAdmin => 'admin';

  @override
  String get chatRoleTeacher => 'o‘qituvchi';

  @override
  String get commonSave => 'Saqlash';

  @override
  String get commonCancel => 'Bekor qilish';

  @override
  String get commonSubmit => 'Yuborish';

  @override
  String get commonDelete => 'O‘chirish';

  @override
  String get commonEdit => 'Tahrirlash';

  @override
  String get commonClose => 'Yopish';

  @override
  String get commonRetry => 'Qayta urinish';

  @override
  String get commonTryAgain => 'Qayta urinib ko‘ring';

  @override
  String get commonLoading => 'Yuklanmoqda…';

  @override
  String get commonError => 'Nimadir noto‘g‘ri ketdi';

  @override
  String commonErrorWithDetails(String error) {
    return 'Xato: $error';
  }

  @override
  String commonCouldNotSave(String error) {
    return 'Saqlab bo‘lmadi: $error';
  }

  @override
```

```
  String get commonSignOut => 'Chiqish';

  @override
  String get commonSaveDraft => 'Qoralamani saqlash';

  @override
  String get commonOpen => 'Ochish';

  @override
  String get commonCreate => 'Yaratish';

  @override
  String get commonAssign => 'Biriktirish';

  @override
  String get commonReset => 'Tiklash';

  @override
  String get commonAdd => 'Qo‘shish';

  @override
  String get commonLoadMore => 'Yana yuklash';

  @override
  String get commonExport => 'Eksport';

  @override
  String commonExportFailed(String error) {
    return 'Eksport bo‘lmadi: $error';
  }

  @override
  String get marksTermAll => 'Butun davr';

  @override
  String marksTermSemester(String yearRange, int semester) {
    return '$yearRange · $semester-semestr';
  }

  @override
  String get marksExportCsv => 'CSV eksport';

  @override
  String get marksExportDone => 'Baholar eksport qilindi';

  @override
  String get commonTeacher => 'O‘qituvchi';

  @override
  String get commonStudent => 'O‘quvchi';

  @override
  String get commonAdmin => 'Administrator';

  @override
  String get commonWorkspace => 'Ish maydoni';

  @override
  String get commonNoDueDate => 'Muddat yo‘q';

  @override
  String commonDueDate(String date) {
    return 'Muddat $date';
  }

  @override
  String get homeworkDueThisWeek => 'Shu hafta';

  @override
  String get homeworkDueThisWeekSubtitle => 'Kechikkan va yaqin muddatlar';

  @override
  String get homeworkNoDueThisWeek => 'Shu hafta muddat yo‘q';
```

```
@override
String get homeworkNoDueThisWeekSubtitle =>
    'Keyingi hafta muddatlarigacha bo‘sh.';

@override
String homeworkOverdue(String date) {
  return 'Muddati o‘tgan · $date';
}

@override
String get homeworkDueToday => 'Bugun muddat';

@override
String get homeworkDueTomorrow => 'Ertaga muddat';

@override
String homeworkDueInDays(int days) {
  return '$days kundan keyin';
}

@override
String get homeworkLateBadge => 'Kechikkan';

@override
String get commonProgress => 'Jarayon';

@override
String get commonSubject => 'Fan';

@override
String get commonType => 'Tur';

@override
String get commonTitle => 'Sarlavha';

@override
String get commonDescription => 'Tavsif';

@override
String get commonGroup => 'Guruh';

@override
String get commonMessage => 'Xabar';

@override
String get commonScore => 'Ball';

@override
String get commonFeedback => 'Izoh';

@override
String get commonStudents => 'O‘quvchilar';

@override
String get commonCannotUndo => 'Bu amalni bekor qilib bo‘lmaydi.';

@override
String get profileTitle => 'Profil';

@override
String get profileFullName => 'To‘liq ism';

@override
String get profileNotSignedIn => 'Tizimga kiritilmagan';

@override
String get profileAccount => 'Hisob';

@override
String get profileAccountSettings => 'Hisob sozlamalari';

@override
String get profileSignOutAdmin => 'Admin konsolidan chiqish';
```

```
  @override
  String get profileSettings => 'Sozlamalar';

  @override
  String get profileAppearance => 'Ko‘rinish';

  @override
  String get profileThemeLight => 'Yorug‘';

  @override
  String get profileThemeDark => 'Qorong‘u';

  @override
  String get profileThemeSystem => 'Tizim';

  @override
  String get profileStyle => 'Uslub';

  @override
  String get profileStyleModern => 'Zamonaviy professional';

  @override
  String get profileStyleNature => 'Tabiat qog‘ozi';

  @override
  String get profileStyleSoftNeu => 'Neumorfizm';

  @override
  String get profileStyleMonochrome => 'Monoxrom ish maydoni';

  @override
  String get profileEditName => 'Ismni o‘zgartirish';

  @override
  String get profileNameUpdated => 'Ism yangilandi';

  @override
  String get profileChangePhoto => 'Rasmni o‘zgartirish';

  @override
  String get profilePhotoUpdated => 'Rasm yangilandi';

  @override
  String get profileChangePassword => 'Parolni o‘zgartirish';

  @override
  String get profileCurrentPassword => 'Joriy parol';

  @override
  String get profileNewPassword => 'Yangi parol';

  @override
  String get profileConfirmPassword => 'Yangi parolni tasdiqlang';

  @override
  String get profilePasswordChanged => 'Parol yangilandi';

  @override
  String get profilePasswordMismatch => 'Yangi parollar mos emas';

  @override
  String get profilePasswordTooShort => 'Kamida 8 ta belgi';

  @override
  String get profilePasswordHint => 'Parolni unutsangiz, admindan so‘rang.';

  @override
  String get profileShortcuts => 'Tezkor havolalar';

  @override
  String get profileYourGroups => 'Guruhlaringiz';

  @override
```

```
  String get profileNoGroups => 'Guruh biriktirilmagan';

  @override
  String profileGroupsCount(int count) {
```

**lib/l10n/app_localizations_uz.dart — continued (2/4)**

```
    return '$count ta guruh';
  }

  @override
  String get profileAverageLabel => 'Davr o‘rtachasi';

  @override
  String get profileAttendanceLabel => 'Davomat';

  @override
  String get profileShareProgress => 'Natija hisobotini ulashish';

  @override
  String get profileNotifications => 'Bildirishnomalar';

  @override
  String get profileClearCache => 'Keshni tozalash';

  @override
  String get profileClearCacheWarn1Title => 'Kesh tozalansinmi?';

  @override
  String get profileClearCacheWarn1Body =>
      'Oflayn chat xabarlari va yuklab olingan rasmlar, videolar, ovozli xabarlar va fayllar
o‘chiriladi. Ularni qayta yuklash uchun internet kerak bo‘ladi.';

  @override
  String get profileClearCacheWarn2Title => 'Ishonchingiz komilmi?';

  @override
  String get profileClearCacheWarn2Body =>
      'Bu amalni bekor qilib bo‘lmaydi. Chat keshi shu qurilmadan o‘chiriladi.';

  @override
  String get profileClearCacheConfirm => 'Keshni tozalash';

  @override
  String get profileClearCacheDone => 'Kesh tozalandi';

  @override
  String profileAppVersion(String version) {
    return 'MU Study · v$version';
  }

  @override
  String get profileSubject => 'Fan';

  @override
  String get greetingMorning => 'Xayrli tong';

  @override
  String get greetingAfternoon => 'Xayrli kun';

  @override
  String get greetingEvening => 'Xayrli kech';

  @override
  String greetingWithName(String greeting, String name) {
    return '$greeting, $name';
  }

  @override
  String get greetingLoadingTitle => 'Xayrli tong, foydalanuvchi';

  @override
  String get greetingLoadingSubtitle => 'Shaxsiy salomlashuv yuklanmoqda...';
```

```
@override
String get studentHomeSubtitle => 'Sizning o‘quv maydoningiz';

@override
String get continueLearning => 'O‘qishni davom ettirish';

@override
String get continueLearningSubtitle => 'To‘xtagan joyingizdan davom eting';

@override
String get everythingCompleted => 'Hammasi bajarilgan.';

@override
String get caughtUpToday => 'Bugun hammasi tayyor.';

@override
String get homeworkPack => 'Uy vazifasi to‘plami';

@override
String get staticTask => 'Oddiy topshiriq';

@override
String get staticAssignment => 'Oddiy';

@override
String get upcomingAssignments => 'Yaqin topshiriqlar';

@override
String get stayOnTrack => 'Rejada qoling';

@override
String get loadingAnnouncements => 'E’lonlar yuklanmoqda...';

@override
String get notificationsCardTitle => 'Bildirishnomalar';

@override
String unreadAlerts(int count) {
  return 'Sizda $count ta yangi bildirishnoma bor';
}

@override
String get upToDate => 'Hammasi yangilangan';

@override
String get announcementsTitle => 'E’lonlar';

@override
String get announcementsSubtitle => 'O‘qituvchilaringizdan';

@override
String get announcementsEmpty => 'E’lonlar yo‘q';

@override
String get announcementsEmptyTeacher =>
    'Muhim yangiliklarni o‘quvchilar bilan ulashing.';

@override
String get announcementsEmptyAdmin =>
    'Platforma bo‘ylab e’lonlar shu yerda paydo bo‘ladi.';

@override
String get announcementsReadMore => 'Batafsil';

@override
String get announcementsShowLess => 'Yig‘ish';

@override
String announcementsViewAll(int count) {
  return 'Barchasini ko‘rish ($count)';
}

@override
```

```
  String get announcementsAll => 'Barcha e’lonlar';

  @override
  String get announcementsNew => 'Yangi e’lon';

  @override
  String get announcementsPost => 'E’lonni joylash';

  @override
  String get announcementsEdit => 'E’lonni tahrirlash';

  @override
  String get announcementsDeleteTitle => 'E’lon o‘chirilsinmi?';

  @override
  String get announcementsDeleteBody =>
      'U guruhdagi barcha a’zolar uchun o‘chiriladi.';

  @override
  String get announcementsByAdmin => 'Ma’muriyat';

  @override
  String get announcementsShareUpdates =>
      'Guruhlaringiz bilan yangiliklar ulashing';

  @override
  String get assignmentsTitle => 'Topshiriqlar';

  @override
  String get assignmentsEmpty => 'Topshiriqlar yo‘q';

  @override
  String get assignmentsEmptySubtitle => 'Hammasi bajarilgan!';

  @override
  String get assignmentsEmptyTeacher =>
      'Boshlash uchun birinchi topshiriqni yarating.';

  @override
  String get assignmentsLearningPath => 'O‘quv yo‘li';

  @override
  String get assignmentsActive => 'Faol topshiriqlar';

  @override
  String get assignmentDetails => 'Topshiriq tafsilotlari';

  @override
  String get assignmentNotFound => 'Topshiriq topilmadi';

  @override
  String get assignmentTasks => 'Vazifalar';

  @override
  String get assignmentTasksSubtitle =>
      'To‘plamni yakunlash uchun hammasini bajaring';

  @override
  String get assignmentNew => 'Yangi topshiriq';

  @override
  String get assignmentManage => 'Topshiriqlarni boshqarish';

  @override
  String get assignmentManageSubtitle =>
      'Oddiy yoki uy vazifasi to‘plamlarini yarating';

  @override
  String get marksTitle => 'Baholarim';

  @override
  String get marksOverallAverage => 'Umumiy o‘rtacha';
```

```
@override
String get marksNoneYet => 'Hali baholar yo‘q.';

@override
String marksBasedOn(String label, int count) {
  return '$label · $count ta baho asosida.';
}

@override
String get marksByCategory => 'Turkumlar bo‘yicha';

@override
String get marksWhereStrongest => 'Eng kuchli tomonlaringiz';

@override
String get marksAll => 'Barcha baholar';

@override
String get marksNewestFirst => 'Avval yangilari';

@override
String get marksEmptyTitle => 'Hali baholar yo‘q';

@override
String get marksEmptySubtitle => 'O‘qituvchi baholari shu yerda ko‘rinadi.';

@override
String get progressReportTitle => 'Natija hisoboti';

@override
String get progressReportShare => 'Natijani ulashish';

@override
String get progressReportSharing => 'Hisobot tayyorlanmoqda…';

@override
String get progressReportDone => 'Hisobot ulashishga tayyor';

@override
String progressReportFailed(String error) {
  return 'Ulashib bo‘lmadi: $error';
}

@override
String get progressReportNeedGroup =>
    'Hisobot ulashish uchun guruhga qo‘shiling.';

@override
String get progressReportAttendance => 'Davomat';

@override
String get progressReportAttendanceRate => 'Davomat foizi';

@override
String get progressReportNoAttendance =>
    'Bu muddat uchun davomat yozilmagan.';

@override
String get progressReportSessions => 'mashg‘ulot';

@override
String get progressReportMarksCounted => 'baho';

@override
String get progressReportRecentMarks => 'So‘nggi baholar';

@override
String progressReportGeneratedOn(String date) {
  return 'Yaratilgan: $date';
}

@override
String get marksNoRecorded => 'Baholar yozilmagan';
```

```
  @override
  String get marksNoRecordedSubtitle =>
      'Birdaniga baho qo‘yish uchun «Sinfni baholash»dan foydalaning.';

  @override
  String get marksRecent => 'So‘nggi baholar';

  @override
  String get marksRecentSubtitle => 'Guruh bo‘yicha oxirgi yozuvlar';

  @override
  String get marksStudentsHeader => 'O‘quvchilar';

  @override
  String get marksStudentsSubtitle =>
      'Barcha baholarning o‘rtacha og‘irlikli bali';

  @override
  String get marksNoStudents => 'Bu guruhda o‘quvchilar yo‘q';

  @override
  String get marksNoStudentsSubtitle =>
      'Administrator guruhingizga o‘quvchilarni biriktirishi mumkin.';

  @override
  String get marksDeleteTitle => 'Bahoni o‘chirish?';

  @override
  String marksDeleteBody(String title) {
    return '«$title» butunlay o‘chiriladi.';
  }

  @override
  String get marksGradeTheClass => 'Sinfni baholash';

  @override
  String get marksOnePercentEach => 'Har bir o‘quvchiga bitta foizli baho';

  @override
  String get marksSaveMarks => 'Baholarni saqlash';

  @override
  String get marksSaveMark => 'Bahoni saqlash';

  @override
  String get marksAddMark => 'Baho qo‘shish';

  @override
  String get marksNothingYet => 'Hali hech narsa yo‘q';

  @override
  String get marksAddFirst => 'Bu o‘quvchiga birinchi bahoni qo‘shing.';

  @override
  String marksSavedCount(int count) {
    return '$count ta baho saqlandi';
  }

  @override
  String get marksGroupHasNoStudents => 'Bu guruhda hali o‘quvchilar yo‘q.';

  @override
  String get marksTitleHint => 'masalan, 3-mavzu testi';

  @override
  String marksAverageLine(String percent, String label) {
    return 'O‘rtacha $percent · $label';
  }

  @override
  String get marksNoMarksYetShort => 'Hali baholar yo‘q';

  @override
```

```
  String get marksSummaryStudents => 'O‘quvchilar';

  @override
  String get marksSummaryMarks => 'Baholar';

  @override
  String get attendanceTitle => 'Davomat';

  @override
  String get attendanceEmpty => 'Davomat yozuvlari yo‘q';

  @override
  String get attendanceNoGroup => 'Guruh hali biriktirilmagan';

  @override
  String get attendanceMark => 'Davomatni belgilash';

  @override
  String get attendanceDailyTracking => 'Kunlik o‘quvchi hisobi';

  @override
  String get attendanceSaved => 'Davomat muvaffaqiyatli saqlandi';

  @override
  String get attendanceNoGroups => 'Guruhlar topilmadi';

  @override
  String get attendancePresent => 'B — Bor';

  @override
  String get attendanceLate => 'K — Kechikdi';

  @override
  String get attendanceAbsent => 'Y — Yo‘q';

  @override
  String get attendanceExcused => 'S — Sababli';

  @override
  String get attendancePresentShort => 'B';

  @override
  String get attendanceLateShort => 'K';

  @override
  String get attendanceAbsentShort => 'Y';

  @override
  String get attendanceExcusedShort => 'S';

  @override
  String get attendanceTrend => 'Davomat dinamikasi';

  @override
  String get attendanceOverall => 'Umumiy taqsimot';

  @override
  String get analyticsThisWeek => 'Shu hafta';

  @override
  String get analyticsLastWeek => 'O‘tgan hafta';

  @override
  String analyticsTotalMarks(int count) {
    return 'Jami baholar: $count';
  }

  @override
  String get notificationsTitle => 'Bildirishnomalar';

  @override
  String get notificationsEmpty => 'Hammasi o‘qilgan!';
```

```
  @override
  String get notificationsEmptySubtitle => 'Yangi bildirishnomalar yo‘q.';

  @override
  String get notificationsNone => 'Bildirishnomalar yo‘q';

  @override
  String get notifKindAssignment => 'Yangi uy vazifasi';

  @override
  String get notifKindAnnouncement => 'E’lon';

  @override
  String get notifKindGraded => 'Ish baholandi';

  @override
  String get notifKindSubmission => 'Yangi topshiriq';

  @override
  String get notifKindChat => 'Chat xabari';

  @override
  String get notifKindSystem => 'Yangilanish';

  @override
  String get notificationsOpen => 'Ochish';

  @override
  String get taskTitle => 'Vazifa';

  @override
  String get taskYourAnswer => 'Javobingiz';

  @override
  String get taskMarkCompleted => 'Bajarildi deb belgilash';

  @override
  String get taskAttachment => 'Ilova';

  @override
  String get taskAttachmentHint => 'Javobingizga rasm yoki hujjat qo‘shing.';

  @override
  String get taskAttachmentHintReady =>
      'Ochish uchun bosing · o‘chirish uchun axlat qutisi.';

  @override
  String get taskImage => 'Rasm';

  @override
  String get taskImageSubtitle => 'Galereya fotosurati';

  @override
  String get taskFile => 'Fayl';

  @override
  String get taskFileSubtitle => 'PDF yoki hujjat';

  @override
  String get taskReplaceImage => 'Rasm bilan almashtirish';

  @override
  String get taskReplaceFile => 'Faylni almashtirish';

  @override
  String get taskVoice => 'Ovozli yozuv';

  @override
  String get taskVoiceHint =>
      'Ixtiyoriy og‘zaki javob — yozing, tinglang, so‘ng saqlang.';

  @override
  String get taskViewPdf => 'PDFni ko‘rish';
```

```
  @override
  String get taskOpenLink => 'Havolani ochish';

  @override
  String get taskSubmitted => 'Yuborildi';

  @override
  String get taskSaved => 'Saqlandi';

  @override
  String taskUploadFailed(String error) {
    return 'Yuklash muvaffaqiyatsiz: $error';
  }

  @override
  String taskVoiceUploadFailed(String error) {
    return 'Ovozni yuklab bo‘lmadi: $error';
  }

  @override
  String taskCouldNotSave(String error) {
    return 'Saqlab bo‘lmadi: $error';
  }

  @override
  String get voiceTapToRecord => 'Javobingizni yozish uchun mikrofonni bosing';

  @override
  String get voiceRecording => 'Yozilmoqda…';

  @override
  String get voiceNote => 'Ovozli eslatma';

  @override
  String get voiceTapPlay => 'Tinglash uchun ijro etishni bosing';

  @override
  String get voiceRerecord => 'Qayta yozish';

  @override
  String get voiceStudentAnswer => 'O‘quvchining ovozli javobi';

  @override
  String get voiceMicPermission => 'Yozish uchun mikrofon ruxsati kerak.';

  @override
  String voiceCouldNotStart(String error) {
    return 'Yozuvni boshlab bo‘lmadi: $error';
  }

  @override
  String voiceCouldNotSave(String error) {
    return 'Yozuvni saqlab bo‘lmadi: $error';
  }

  @override
  String voiceCouldNotPlay(String error) {
    return 'Ovozni ijro etib bo‘lmadi: $error';
  }

  @override
  String get attachmentCouldNotOpen => 'Bu faylni ochib bo‘lmadi.';

  @override
  String get attachmentImage => 'Rasm ilovasi';

  @override
  String get attachmentFile => 'Fayl ilovasi';

  @override
  String get attachmentPdf => 'PDF hujjat';

  @override
```

```
  String get attachmentTapPreview => 'Ko‘rish uchun bosing';

  @override
  String get attachmentTapOpen => 'Ochish uchun bosing';

  @override
  String get pdfDefaultTitle => 'PDF';

  @override
  String get teacherWorkspace => 'O‘qituvchi ish maydoni';

  @override
  String get teacherQuickActions => 'Tezkor amallar';

  @override
  String get teacherCreateItems => 'Yangi elementlar yaratish';

  @override
  String get teacherNewAssignment => 'Yangi topshiriq';

  @override
  String get teacherNewAnnouncement => 'Yangi e‘lon';

  @override
  String get teacherSubmissions => 'Ishlar';

  @override
  String get teacherNoSubmissions => 'Hali ishlar yo‘q';

  @override
```

**lib/l10n/app_localizations_uz.dart — continued (3/4)**

```
  String get teacherNoSubmissionsSubtitle =>
      'O‘quvchi ishlari baholash uchun shu yerda ko‘rinadi.';

  @override
  String get teacherGradebook => 'Jurnal';

  @override
  String get teacherGradeClass => 'Sinfni baholash';

  @override
  String get teacherNoGroups => 'Hali guruhlar yo‘q';

  @override
  String get teacherNoGroupsSubtitle =>
      'Administratordan guruh biriktirishini so‘rang.';

  @override
  String get teacherActiveGroups => 'FAOL GURUHLAR';

  @override
  String get teacherSubject => 'FAN';

  @override
  String get teacherDailyTasks => 'Kunlik vazifalar';

  @override
  String get teacherNeedsAttention => 'Diqqatingizni talab qiladiganlar';

  @override
  String get teacherReviewSubmissions => 'Ishlarni tekshirish';

  @override
  String get teacherCheckAssignments => 'O‘quvchi topshiriqlarini tekshirish';

  @override
  String get teacherRecordMarks => 'Foizli baholar qo‘yish';

  @override
  String get teacherPerformanceAnalytics => 'Natijalar tahlili';

  @override
```

```
  String get teacherTrackProgress => 'Guruh jarayonini kuzatish';

  @override
  String get teacherOpenAttachment => 'Ilovani ochish';

  @override
  String get teacherScoreHelper => 'Uy vazifasi bahosini jurnalga qo‘shadi';

  @override
  String get teacherScoreRange => 'Ball 0 dan 100 gacha bo‘lishi kerak.';

  @override
  String get teacherNoWrittenAnswer => 'Yozma javob yo‘q.';

  @override
  String get teacherReviewSubmission => 'Ishni tekshirish';

  @override
  String get teacherHomeworkPackTasks => 'Uy vazifasi to‘plami vazifalari';

  @override
  String get teacherAddTask => 'Vazifa qo‘shish';

  @override
  String get teacherEditTask => 'Vazifani tahrirlash';

  @override
  String get teacherDeleteTask => 'Vazifani o‘chirish?';

  @override
  String get teacherTasksContent => 'Vazifalar mazmuni';

  @override
  String get teacherManagePack => 'Bu to‘plamdagi elementlarni boshqaring';

  @override
  String get teacherTaskTitleHint => 'masalan, O‘qish mashqi';

  @override
  String get teacherLinkHint => 'https://...';

  @override
  String get teacherAttachmentHint => 'Rasm yoki fayl havolasi...';

  @override
  String get teacherNoData => 'Ma’lumot yo‘q';

  @override
  String get teacherMarkDistribution => 'Baholar taqsimoti';

  @override
  String get teacherStudentsByAverage => 'O‘quvchilar o‘rtacha ball bo‘yicha';

  @override
  String get teacherNoMarksTrends =>
      'Tendensiyalarni ko‘rish uchun jurnalga baho qo‘shing.';

  @override
  String get teacherTargetGroup => 'Maqsadli guruh';

  @override
  String get adminWorkspace => 'Platformani boshqarish';

  @override
  String get adminUsers => 'Foydalanuvchilar';

  @override
  String get adminGroups => 'Guruhlar';

  @override
  String get adminAnalytics => 'Tahlil';

  @override
```

```
  String get adminTeachers => 'O‘QITUVCHILAR';

  @override
  String get adminStudents => 'O‘QUVCHILAR';

  @override
  String get adminGroupsLabel => 'GURUHLAR';

  @override
  String get adminMarksLabel => 'BAHOLAR';

  @override
  String get adminActivityInsights => 'Faoliyat tahlili';

  @override
  String get adminCurrentMetrics => 'Joriy ko‘rsatkichlar';

  @override
  String get adminCreateUser => 'Yangi foydalanuvchi';

  @override
  String get adminCreateUserButton => 'Foydalanuvchi yaratish';

  @override
  String get adminUserCreated => 'Foydalanuvchi muvaffaqiyatli yaratildi';

  @override
  String get adminResetPassword => 'Parolni tiklash';

  @override
  String adminResetPasswordTitle(String username) {
    return 'Parolni tiklash · $username';
  }

  @override
  String get adminCreateGroup => 'Yangi guruh';

  @override
  String get adminCreateGroupButton => 'Guruh yaratish';

  @override
  String get adminGroupTitleHint => 'masalan, 4-guruh';

  @override
  String get adminGroupTitleExists =>
      'Bu o‘qituvchida shu nomdagi guruh allaqachon bor';

  @override
  String get adminAssignStudent => 'O‘quvchini biriktirish';

  @override
  String get adminActivityDistribution => 'Faoliyat taqsimoti';

  @override
  String get adminGlobalMetrics => 'Global platforma ko‘rsatkichlari';

  @override
  String get adminActivityLog => 'Admin faoliyat jurnali';

  @override
  String get adminActivityLogSubtitle =>
      'Tizim voqealari va audit yozuvlari shu yerda paydo bo‘ladi.';

  @override
  String adminTeacherLine(String name) {
    return 'O‘qituvchi: $name';
  }

  @override
  String adminRoleLine(String role, String username) {
    return '$role · $username';
  }
```

```
  @override
  String get adminDisable => 'O‘chirish';

  @override
  String get adminEnable => 'Yoqish';

  @override
  String get adminPassword => 'Parol';

  @override
  String get adminFullName => 'To‘liq ism';

  @override
  String get adminRole => 'Rol';

  @override
  String get adminSphere => 'Soha';

  @override
  String get percentExcellent => 'A’lo';

  @override
  String get percentGood => 'Yaxshi';

  @override
  String get percentNeedsWork => 'Yaxshilash kerak';

  @override
  String get percentAtRisk => 'Xavf ostida';

  @override
  String get emptyStateDefault => 'Hali hech narsa yo‘q';

  @override
  String get setupRequiredTitle => 'Sozlash kerak';

  @override
  String get setupRequiredBody =>
      'Supabase sozlanmagan. .env fayliga SUPABASE_URL va SUPABASE_ANON_KEY qo‘shing.';

  @override
  String get setupOption1 => '1-variant — mahalliy .env fayli';

  @override
  String get setupOption2 => '2-variant — dart-define';

  @override
  String get attendanceAllPresent => 'Hammasi B';

  @override
  String get attendanceAllAbsent => 'Hammasi Y';

  @override
  String get attendanceAllLate => 'Hammasi K';

  @override
  String get attendanceAllExcused => 'Hammasi S';

  @override
  String get attendancePresentWord => 'Bor';

  @override
  String get attendanceAbsentWord => 'Yo‘q';

  @override
  String get attendanceLateWord => 'Kechikdi';

  @override
  String get attendanceExcusedWord => 'Sababli';

  @override
  String get attendanceWeekdayMon => 'Du';
```

```
  @override
  String get attendanceWeekdayTue => 'Se';

  @override
  String get attendanceWeekdayWed => 'Ch';

  @override
  String get attendanceWeekdayThu => 'Pa';

  @override
  String get attendanceWeekdayFri => 'Ju';

  @override
  String get attendancePercentLabel => 'Foiz';

  @override
  String get attendanceNoLesson => 'Dars yo‘q';

  @override
  String get attendanceHistory => 'Davomat tarixi';

  @override
  String get attendanceSelectStudent => 'O‘quvchi';

  @override
  String get attendanceAllStudents => 'Barcha o‘quvchilar';

  @override
  String get attendanceTapToCycle => 'Statusni o‘zgartirish uchun kunga bosing';

  @override
  String get attendanceUpdated => 'Davomat yangilandi';

  @override
  String get attendanceCalendarEmpty => 'Bu oyda davomat yo‘q';

  @override
  String get attendanceFutureNotAllowed =>
      'Kelajakdagi kun uchun davomat belgilab bo‘lmaydi';

  @override
  String get attendanceWeekendNotAllowed =>
      'Davomat faqat ish kunlari (du–ju) uchun belgilanadi';

  @override
  String get chartBandExcellent => '85–100%';

  @override
  String get chartBandGood => '70–84%';

  @override
  String get chartBandNeedsWork => '50–69%';

  @override
  String get chartBandAtRisk => '50% dan past';

  @override
  String get marksWeight => 'Og‘irlik';

  @override
  String get marksCategory => 'Turkum';

  @override
  String get marksDate => 'Sana';

  @override
  String get marksCommentOptional => 'Izoh (ixtiyoriy)';

  @override
  String get marksNewMark => 'Yangi baho';

  @override
  String get marksEditMark => 'Bahoni tahrirlash';
```

```
  @override
  String get marksFromSubmission => 'Tekshirilgan ishdan';

  @override
  String get marksNeedTitle => 'Baholanadigan element uchun sarlavha kiriting.';

  @override
  String get marksWeightPositive => 'Og‘irlik noldan katta bo‘lishi kerak.';

  @override
  String get marksNeedAtLeastOne => 'Kamida bitta o‘quvchiga baho kiriting.';

  @override
  String get marksLeaveBlankSkip =>
      'O‘quvchini o‘tkazib yuborish uchun maydonni bo‘sh qoldiring.';

  @override
  String get marksFromSubmissionDelete =>
      'Bu baho tekshirilgan ishdan. Uni o‘chirish uchun «Ishlar» bo‘limida ballni tozalang.';

  @override
  String get marksFromSubmissionEdit =>
      'Bu baho tekshirilgan ishga bog‘langan. Bu yerda tahrirlash keyingi tekshiruvda qayta
yoziladi.';

  @override
  String get categoryHomework => 'Uy vazifasi';

  @override
  String get categoryClasswork => 'Sinf ishi';

  @override
  String get categoryQuiz => 'Viktorina';

  @override
  String get categoryTest => 'Test';

  @override
  String get categoryExam => 'Imtihon';

  @override
  String get categoryProject => 'Loyiha';

  @override
  String get categoryOral => 'Og‘zaki';

  @override
  String get categoryOther => 'Boshqa';

  @override
  String get teacherReferenceLinkOptional => 'Havola (ixtiyoriy)';

  @override
  String get teacherAttachmentUrlOptional => 'Ilova URL (ixtiyoriy)';

  @override
  String get teacherNoTasksYet =>
      'Hali vazifalar yo‘q. Boshlash uchun «Vazifa qo‘shish»ni bosing.';

  @override
  String get teacherTemplates => 'Shablonlar';

  @override
  String get teacherTemplatesEmpty => 'Hali shablonlar yo‘q';

  @override
  String get teacherTemplatesEmptySubtitle =>
      'Qayta ishlatish uchun topshiriqni shablon sifatida saqlang.';

  @override
  String get teacherSaveAsTemplate => 'Shablon sifatida saqlash';

  @override
```

```
  String get teacherTemplateSaved => 'Shablon saqlandi';

  @override
  String get teacherUseTemplate => 'Shablondan foydalanish';

  @override
  String get teacherFromTemplate => 'Shablondan';

  @override
  String get teacherDeleteTemplate => 'Shablonni o‘chirish?';

  @override
  String teacherDeleteTemplateBody(String title) {
    return '«$title» o‘chiriladi.';
  }

  @override
  String get teacherDuplicateAssignment => 'Nusxa olish';

  @override
  String get teacherDuplicateTitle => 'Topshiriqdan nusxa';

  @override
  String get teacherDuplicated => 'Topshiriq nusxalandi';

  @override
  String get assignmentEdit => 'Topshiriqni tahrirlash';

  @override
  String get assignmentDeleteTitle => 'Topshiriq o‘chirilsinmi?';

  @override
  String get assignmentDeleteBody =>
      'Topshiriq, vazifalar va o‘quvchi topshiriqlari butunlay o‘chiriladi.';

  @override
  String get assignmentDeleted => 'Topshiriq o‘chirildi';

  @override
  String get assignmentTypeHintStatic =>
      'Bitta ish. O‘quvchi bir marta topshiradi — qo‘shimcha vazifalar yo‘q.';

  @override
  String get assignmentTypeHintPack =>
      'Bir nechta vazifali to‘plam. Keyin bandlarni qo‘shasiz.';

  @override
  String get teacherCreateFromTemplate => 'Shablondan yaratish';

  @override
  String teacherTasksCopied(int count) {
    return '$count ta vazifa kiritiladi';
  }

  @override
  String get teacherRecentFeedback => 'Fikrni qayta ishlatish';

  @override
  String get teacherApplyFeedbackToOthers =>
      'Shu vazifa bo‘yicha boshqa kutayotganlarga ham qo‘llash';

  @override
  String teacherFeedbackAppliedCount(int count) {
    return 'Fikr yana $count ta topshiriqqa qo‘llandi';
  }

  @override
  String get teacherSharedComment => 'Umumiy izoh (ixtiyoriy)';

  @override
  String get teacherSharedCommentHint =>
      'Saqlanadigan har bir bahoga bir xil izoh';
```

```
  @override
  String get adminPlatformOverview => 'Platforma umumiy ko‘rinishi';

  @override
  String get adminAssignTeacher => 'O‘qituvchini biriktirish';

  @override
  String get adminSelectStudent => 'O‘quvchini tanlash';

  @override
  String get adminInitialGroupOptional => 'Boshlang‘ich guruh (ixtiyoriy)';

  @override
  String get adminChartTasks => 'Vazifalar';

  @override
  String get adminChartGroups => 'Guruhlar';

  @override
  String get adminChartStaff => 'Xodimlar';

  @override
  String get adminChartAssignments => 'Topshiriqlar';

  @override
  String get adminChartMarks => 'Baholar';

  @override
  String get adminChartStudents => 'O‘quvchilar';

  @override
  String get adminAcademicYears => 'O‘quv yillari';

  @override
  String get adminAcademicYearsSubtitle =>
      'Baholar jurnali uchun yillar va semestrlar';

  @override
  String get adminAcademicYearsTitle => 'O‘quv kalendari';

  @override
  String get adminCreateNextYear => 'Keyingi yilni yaratish';

  @override
  String adminYearCreated(String label) {
    return '$label yaratildi';
  }

  @override
  String get adminSetCurrentYear => 'Joriy qilish';

  @override
  String get adminCurrentYearBadge => 'Joriy';

  @override
  String get adminYearSemesters => 'To‘rt semestr · avg → iyn';

  @override
  String adminSemesterRange(int semester, String start, String end) {
    return '$semester-semestr · $start – $end';
  }

  @override
  String get adminEditTerm => 'Muddatni tahrirlash';

  @override
  String adminEditTermTitle(int semester) {
    return '$semester-semestr';
  }

  @override
  String get adminTermStartsOn => 'Boshlanishi';
```

```
  @override
  String get adminTermEndsOn => 'Tugashi';

  @override
  String get adminTermDatesInvalid =>
      'Tugash sanasi boshlanish sanasidan oldin bo‘lmasligi kerak';

  @override
  String get adminTermUpdated => 'Semestr sanalari yangilandi';

  @override
  String get adminDeleteYear => 'Yilni o‘chirish';

  @override
  String adminDeleteYearTitle(String label) {
    return '$label ni o‘chirasizmi?';
  }

  @override
  String adminDeleteYearBody(String label) {
    return 'Yil va to‘rt semestr kalendardan o‘chiriladi. Baholar o‘chirilmaydi. Tasdiqlash
uchun pastda $label ni yozing.';
  }

  @override
  String adminDeleteYearConfirmHint(String label) {
    return 'Tasdiqlash uchun $label ni yozing';
  }

  @override
  String get adminDeleteYearCurrentBlocked =>
      'Avval boshqa yilni joriy qiling, keyin buni o‘chiring.';

  @override
  String adminYearDeleted(String label) {
    return '$label o‘chirildi';
  }

  @override
  String get adminNoAcademicYears => 'Hali o‘quv yillari yo‘q';

  @override
```

**lib/l10n/app_localizations_uz.dart — continued (4/4)**

```
  String get adminAcademicYearsHint =>
      'Tayyor bo‘lganda keyingi o‘quv yilini yarating. O‘qituvchilar va o‘quvchilar jurnalda
semestrni tanlaydi. Sanalarni o‘zgartirish uchun semestrni bosing.';

  @override
  String get adminCredentialPrintTitle => 'Login qog‘ozlari';

  @override
  String get adminCredentialPrintSubtitle =>
      'Bazadan unique loginlar, sahifada 3 ta karta';

  @override
  String get adminCredentialPasteLabel => 'Foydalanuvchilar';

  @override
  String get adminCredentialPasteHint =>
      'username|password|full name\n@james|Abc123xyz|James English|teacher\
n@john_jones|Pass456|John Jones';

  @override
  String get adminCredentialPerPage => 'Sahifadagi kartalar';

  @override
  String get adminCredentialTwo => '2';

  @override
  String get adminCredentialThree => '3';

  @override
```

```
  String get adminCredentialExportPdf => 'PDF eksport';

  @override
  String get adminCredentialSlipTitle => 'MU Study login';

  @override
  String get adminCredentialSlipHint =>
      'Bu qog‘ozni o‘zingizda saqlang. Birinchi kirishdan keyin parolni o‘zgartiring.';

  @override
  String get adminCredentialNeedUsers => 'Chop etish uchun foydalanuvchi yo‘q';

  @override
  String get adminCredentialNeedSql =>
      'Supabase SQL Editor’da sql/admin_login_slips.sql ni ishga tushiring, keyin qayta eksport
qiling.';

  @override
  String get adminCredentialConfidential =>
      'Maxfiy. Bu ma’lumotlar faqat belgilangan o‘quvchi uchun.';

  @override
  String get adminFirstName => 'Ism';

  @override
  String get adminLastName => 'Familiya';

  @override
  String get adminCredentialStudents => 'O‘quvchilar';

  @override
  String get adminCredentialTeachers => 'O‘qituvchilar';

  @override
  String get adminCredentialAllGroups => 'Barcha guruhlar';

  @override
  String get sphereMath => 'Matematika';

  @override
  String get sphereEnglish => 'Ingliz tili';

  @override
  String get dailyMotivation1 => 'Bugun oldinga siljish uchun ajoyib kun.';

  @override
  String get dailyMotivation2 =>
      'To‘xtagan joyingizdan davom etishga tayyormisiz?';

  @override
  String get dailyMotivation3 => 'Bugun maqsadlaringizni zabt etaylik.';

  @override
  String get dailyMotivation4 => 'Izchillik — mahorat kaliti.';
}
```

# Appendix A — Complete backend SQL

Run this script in the SQL Editor on a brand-new Supabase project, top to bottom. It is the recovered live schema (enums, tables, views, functions, triggers, grants). After it succeeds, run Appendix I to create the first admin. Then apply RLS from export/rls_policies.sql if the complete file's policy section is split.

**COMPLETE_BACKEND.sql**

```
-- ============================================================
-- MU Study — Complete Backend Rebuild Script
-- Source: live Supabase project nmmdvdprczukaskxxdrs (EXACT)
-- Run in SQL Editor on a brand-new project, top to bottom.
-- ============================================================

CREATE EXTENSION IF NOT EXISTS pgcrypto WITH SCHEMA extensions;

-- ---------- ENUMS ----------
CREATE TYPE public.user_role AS ENUM ('student', 'teacher', 'admin');
CREATE TYPE public.sphere_type AS ENUM ('math', 'english');
CREATE TYPE public.assignment_type AS ENUM ('static', 'homework_pack');
CREATE TYPE public.attendance_status AS ENUM ('present', 'absent', 'late', 'excused');
CREATE TYPE public.submission_status AS ENUM ('pending', 'submitted', 'reviewed');
CREATE TYPE public.chat_message_type AS ENUM ('text', 'voice', 'image', 'video', 'file');
CREATE TYPE public.mark_category AS ENUM (
  'homework', 'classwork', 'quiz', 'test', 'exam', 'project', 'oral', 'other'
);

-- ---------- TABLES ----------
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  username TEXT NOT NULL UNIQUE,
  full_name TEXT NOT NULL DEFAULT '',
  role public.user_role NOT NULL,
  auth_email TEXT NOT NULL UNIQUE,
  avatar_url TEXT,
  is_disabled BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  preferred_language TEXT,
  preferred_theme TEXT,
  preferred_style TEXT,
  CONSTRAINT profiles_preferred_language_check CHECK (
    preferred_language IS NULL OR preferred_language = ANY (ARRAY['en'::text, 'ru'::text, 'uz'::
text])
  ),
  CONSTRAINT profiles_preferred_theme_check CHECK (
    preferred_theme IS NULL OR preferred_theme = ANY (ARRAY['light'::text, 'dark'::text,
'system'::text])
  ),
  CONSTRAINT profiles_preferred_style_check CHECK (
    preferred_style IS NULL OR preferred_style = ANY (ARRAY['modern'::text, 'naturePaper'::text,
 'softNeumorphism'::text, 'monochromeWorkspace'::text])
  )
);
CREATE INDEX idx_profiles_username ON public.profiles USING btree (username);

CREATE TABLE public.teachers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  profile_id UUID NOT NULL UNIQUE REFERENCES public.profiles(id) ON DELETE CASCADE,
  sphere public.sphere_type NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public.students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  profile_id UUID NOT NULL UNIQUE REFERENCES public.profiles(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public.groups (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
```

```
  title TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX groups_teacher_title_uidx ON public.groups USING btree (teacher_id, title);
CREATE INDEX idx_groups_teacher ON public.groups USING btree (teacher_id);

CREATE TABLE public.group_students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (group_id, student_id)
);

CREATE TABLE public.assignments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  type public.assignment_type NOT NULL DEFAULT 'static',
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  due_date TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_assignments_group ON public.assignments USING btree (group_id);

CREATE TABLE public.tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  assignment_id UUID NOT NULL REFERENCES public.assignments(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  attachment_url TEXT,
  link_url TEXT,
  sort_order INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public.submissions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID NOT NULL REFERENCES public.tasks(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  text_answer TEXT,
  file_url TEXT,
  status public.submission_status NOT NULL DEFAULT 'pending',
  score NUMERIC,
  feedback TEXT,
  is_completed BOOLEAN NOT NULL DEFAULT false,
  submitted_at TIMESTAMPTZ,
  voice_url TEXT,
  UNIQUE (task_id, student_id),
  CONSTRAINT submissions_score_percent_check CHECK (
    score IS NULL OR (score >= 0 AND score <= 100)
  )
);

CREATE TABLE public.lesson_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  session_date DATE NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (group_id, session_date)
);
CREATE INDEX idx_lesson_sessions_group_date ON public.lesson_sessions USING btree (group_id,
session_date DESC);

CREATE TABLE public.attendance (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  status public.attendance_status NOT NULL,
  date DATE NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  session_id UUID NOT NULL REFERENCES public.lesson_sessions(id) ON DELETE CASCADE,
```

```
  UNIQUE (group_id, student_id, date),
  UNIQUE (session_id, student_id)
);

CREATE TABLE public.marks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES public.students(id) ON DELETE CASCADE,
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  submission_id UUID UNIQUE REFERENCES public.submissions(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  category public.mark_category NOT NULL DEFAULT 'other',
  percentage NUMERIC NOT NULL,
  weight NUMERIC NOT NULL DEFAULT 1,
  comment TEXT NOT NULL DEFAULT '',
  marked_on DATE NOT NULL DEFAULT CURRENT_DATE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT marks_percentage_check CHECK (percentage >= 0 AND percentage <= 100),
  CONSTRAINT marks_weight_check CHECK (weight > 0)
);
CREATE INDEX idx_marks_group ON public.marks USING btree (group_id, marked_on DESC);
CREATE INDEX idx_marks_student ON public.marks USING btree (student_id, marked_on DESC);

CREATE TABLE public.announcements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID REFERENCES public.teachers(id) ON DELETE CASCADE,
  group_id UUID NOT NULL REFERENCES public.groups(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  image_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
  file_urls JSONB NOT NULL DEFAULT '[]'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE public.notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  body TEXT NOT NULL DEFAULT '',
  is_read BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  kind TEXT NOT NULL DEFAULT 'system',
  action_path TEXT,
  ref_id UUID
);
CREATE INDEX idx_notifications_user ON public.notifications USING btree (user_id, is_read,
created_at DESC);
CREATE INDEX idx_notifications_user_kind_unread ON public.notifications USING btree (user_id,
kind, is_read) WHERE (is_read = false);

CREATE TABLE public.assignment_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  teacher_id UUID NOT NULL REFERENCES public.teachers(id) ON DELETE CASCADE,
  type public.assignment_type NOT NULL DEFAULT 'static',
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  tasks JSONB NOT NULL DEFAULT '[]'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX assignment_templates_teacher_idx ON public.assignment_templates USING btree (
teacher_id, updated_at DESC);

CREATE TABLE public.academic_years (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  start_year INTEGER NOT NULL UNIQUE,
  label TEXT NOT NULL,
  is_current BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by UUID REFERENCES auth.users(id) ON DELETE SET NULL,
  CONSTRAINT academic_years_start_year_check CHECK (start_year >= 2000 AND start_year <= 2100)
);
```

```
CREATE UNIQUE INDEX academic_years_one_current ON public.academic_years USING btree ((true))
WHERE is_current;

CREATE TABLE public.academic_terms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  academic_year_id UUID NOT NULL REFERENCES public.academic_years(id) ON DELETE CASCADE,
  semester INTEGER NOT NULL,
  starts_on DATE NOT NULL,
  ends_on DATE NOT NULL,
  UNIQUE (academic_year_id, semester),
  CONSTRAINT academic_terms_semester_check CHECK (semester >= 1 AND semester <= 4),
  CONSTRAINT academic_terms_range_check CHECK (ends_on >= starts_on)
);
CREATE INDEX idx_academic_terms_dates ON public.academic_terms USING btree (starts_on, ends_on);
CREATE INDEX idx_academic_terms_year ON public.academic_terms USING btree (academic_year_id);

CREATE TABLE public.chat_rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  group_id UUID NOT NULL UNIQUE REFERENCES public.groups(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_chat_rooms_group ON public.chat_rooms USING btree (group_id);

CREATE TABLE public.chat_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES public.chat_rooms(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  message_type public.chat_message_type NOT NULL DEFAULT 'text',
  text_content TEXT,
  media_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  is_deleted BOOLEAN NOT NULL DEFAULT false,
  reply_to_message_id UUID REFERENCES public.chat_messages(id) ON DELETE SET NULL,
  CONSTRAINT chat_messages_content_check CHECK (
    is_deleted OR (
      ((message_type = 'text'::chat_message_type) AND (text_content IS NOT NULL) AND (length(
TRIM(BOTH FROM text_content)) > 0))
      OR ((message_type <> 'text'::chat_message_type) AND (media_url IS NOT NULL) AND (length(
TRIM(BOTH FROM media_url)) > 0))
    )
  )
);
CREATE INDEX idx_chat_messages_room_created ON public.chat_messages USING btree (room_id,
created_at DESC);
CREATE INDEX idx_chat_messages_sender ON public.chat_messages USING btree (sender_id);
CREATE INDEX idx_chat_messages_reply_to ON public.chat_messages USING btree (
reply_to_message_id) WHERE (reply_to_message_id IS NOT NULL);

CREATE TABLE public.message_reads (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES public.chat_messages(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  read_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (message_id, user_id)
);
CREATE INDEX idx_message_reads_user ON public.message_reads USING btree (user_id, message_id);

CREATE TABLE public.message_reactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID NOT NULL REFERENCES public.chat_messages(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  emoji TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (message_id, user_id),
  CONSTRAINT message_reactions_emoji_len CHECK (char_length(emoji) >= 1 AND char_length(emoji)
<= 16)
);
CREATE INDEX idx_message_reactions_message ON public.message_reactions USING btree (message_id);
ALTER TABLE public.message_reactions REPLICA IDENTITY FULL;


-- ---------- VIEWS ----------
```

```
CREATE OR REPLACE VIEW public.chat_room_summaries AS
 SELECT cr.id,
    cr.group_id,
    cr.created_at,
    g.title AS group_title,
    tp.full_name AS teacher_name,
    (( SELECT (count(*))::integer AS count
           FROM group_students gs
          WHERE (gs.group_id = g.id)) + 1) AS member_count,
    lm.id AS last_message_id,
    lm.message_type AS last_message_type,
    lm.text_content AS last_message_text,
    lm.media_url AS last_message_media_url,
    lm.created_at AS last_message_at,
    lm.is_deleted AS last_message_deleted,
    lm.sender_id AS last_message_sender_id,
    sp.full_name AS last_message_sender_name
   FROM (((((chat_rooms cr
     JOIN groups g ON ((g.id = cr.group_id)))
     JOIN teachers t ON ((t.id = g.teacher_id)))
     JOIN profiles tp ON ((tp.id = t.profile_id)))
     LEFT JOIN LATERAL ( SELECT m.id,
            m.room_id,
            m.sender_id,
            m.message_type,
            m.text_content,
            m.media_url,
            m.created_at,
            m.updated_at,
            m.is_deleted
           FROM chat_messages m
          WHERE (m.room_id = cr.id)
          ORDER BY m.created_at DESC
         LIMIT 1) lm ON (true))
     LEFT JOIN profiles sp ON ((sp.id = lm.sender_id)));

CREATE OR REPLACE VIEW public.student_mark_averages AS
 SELECT student_id,
    round((sum((percentage * weight)) / NULLIF(sum(weight), (0)::numeric)), 2) AS
average_percent,
    (count(*))::integer AS marks_count,
    max(marked_on) AS last_marked_on
   FROM marks
  GROUP BY student_id;

CREATE OR REPLACE VIEW public.student_group_mark_averages AS
 SELECT student_id,
    group_id,
    round((sum((percentage * weight)) / NULLIF(sum(weight), (0)::numeric)), 2) AS
average_percent,
    (count(*))::integer AS marks_count,
    max(marked_on) AS last_marked_on
   FROM marks
  GROUP BY student_id, group_id;

-- FILE: fn__demo_insert_user.sql
```

**COMPLETE_BACKEND.sql — continued (2/6)**

```
CREATE OR REPLACE FUNCTION public._demo_insert_user(p_username text, p_password text,
p_full_name text, p_role user_role)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE v_user_id UUID := gen_random_uuid(); v_email TEXT;
BEGIN
  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';
  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
```

```
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000', v_user_id, 'authenticated', 'authenticated',
    v_email, extensions.crypt(p_password, extensions.gen_salt('bf')), NOW(),
    jsonb_build_object('provider','email','providers',ARRAY['email']),
    jsonb_build_object('username', p_username, 'full_name', p_full_name, 'role', p_role::text),
    NOW(), NOW(), '', '', '', '', false, false
  );
  INSERT INTO auth.identities (id, user_id, identity_data, provider, provider_id,
last_sign_in_at, created_at, updated_at)
  VALUES (
    gen_random_uuid(), v_user_id,
    jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
    'email', v_email, NOW(), NOW(), NOW()
  );
  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);
  IF p_role = 'student' THEN INSERT INTO public.students (profile_id) VALUES (v_user_id); END
IF;
  RETURN v_user_id;
END; $function$

-- FILE: fn_academic_semester_bounds.sql
CREATE OR REPLACE FUNCTION public.academic_semester_bounds(p_start_year integer, p_semester
integer)
 RETURNS TABLE(starts_on date, ends_on date)
 LANGUAGE plpgsql
 IMMUTABLE
AS $function$
BEGIN
  IF p_semester < 1 OR p_semester > 4 THEN
    RAISE EXCEPTION 'semester must be 1–4';
  END IF;

  RETURN QUERY
  SELECT
    CASE p_semester
      WHEN 1 THEN make_date(p_start_year, 8, 1)
      WHEN 2 THEN make_date(p_start_year, 11, 11)
      WHEN 3 THEN make_date(p_start_year + 1, 1, 11)
      ELSE make_date(p_start_year + 1, 3, 28)
    END,
    CASE p_semester
      WHEN 1 THEN make_date(p_start_year, 11, 2)
      WHEN 2 THEN make_date(p_start_year, 12, 28)
      WHEN 3 THEN make_date(p_start_year + 1, 3, 20)
      ELSE make_date(p_start_year + 1, 6, 19)
    END;
END;
$function$

-- FILE: fn_academic_year_label.sql
CREATE OR REPLACE FUNCTION public.academic_year_label(p_start_year integer)
 RETURNS text
 LANGUAGE sql
 IMMUTABLE
AS $function$
  SELECT p_start_year::TEXT || '–' || lpad(((p_start_year + 1) % 100)::TEXT, 2, '0');
$function$

-- FILE: fn_admin_create_next_academic_year.sql
CREATE OR REPLACE FUNCTION public.admin_create_next_academic_year()
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_next INT;
  v_now DATE := CURRENT_DATE;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can create academic years';
```

```
  END IF;

  SELECT COALESCE(MAX(start_year), CASE
    WHEN EXTRACT(MONTH FROM v_now) >= 8 THEN EXTRACT(YEAR FROM v_now)::INT - 1
    ELSE EXTRACT(YEAR FROM v_now)::INT - 2
  END) + 1
  INTO v_next
  FROM public.academic_years;

  RETURN public.create_academic_year_with_terms(
    v_next,
    FALSE,
    auth.uid()
  );
END;
$function$

-- FILE: fn_admin_create_user.sql
CREATE OR REPLACE FUNCTION public.admin_create_user(p_username text, p_password text,
p_full_name text, p_role user_role, p_sphere sphere_type DEFAULT NULL::sphere_type)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';

  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000',
    v_user_id,
    'authenticated',
    'authenticated',
    v_email,
    extensions.crypt(p_password, extensions.gen_salt('bf')),
    NOW(),
    jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),
    jsonb_build_object(
      'username', p_username,
      'full_name', p_full_name,
      'role', p_role::text
    ),
    NOW(),
    NOW(),
    '',
    '',
    '',
    '',
    false,
    false
  );

  INSERT INTO auth.identities (
    id, user_id, identity_data, provider, provider_id,
    last_sign_in_at, created_at, updated_at
  ) VALUES (
    gen_random_uuid(),
    v_user_id,
    jsonb_build_object(
      'sub', v_user_id::text,
      'email', v_email,
```

```
      'email_verified', true
    ),
    'email',
    v_email,
    NOW(),
    NOW(),
    NOW()
  );

  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);

  IF EXISTS (
    SELECT 1
    FROM information_schema.tables
    WHERE table_schema = 'public' AND table_name = 'login_credentials'
  ) THEN
    INSERT INTO public.login_credentials (profile_id, password, updated_at)
    VALUES (v_user_id, p_password, NOW())
    ON CONFLICT (profile_id) DO UPDATE
      SET password = excluded.password,
          updated_at = NOW();
  END IF;

  IF p_role = 'teacher' THEN
    IF p_sphere IS NULL THEN
      RAISE EXCEPTION 'Sphere required for teacher';
    END IF;
    -- Do NOT auto-create Group 1/2/3. Admins add groups manually.
    INSERT INTO public.teachers (profile_id, sphere)
    VALUES (v_user_id, p_sphere);
  ELSIF p_role = 'student' THEN
    INSERT INTO public.students (profile_id) VALUES (v_user_id);
  END IF;

  RETURN v_user_id;
END;
$function$

-- FILE: fn_admin_delete_academic_year.sql
CREATE OR REPLACE FUNCTION public.admin_delete_academic_year(p_year_id uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
  v_remaining INT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can delete academic years';
  END IF;

  SELECT * INTO v_year
  FROM public.academic_years
  WHERE id = p_year_id;

  IF v_year.id IS NULL THEN
    RAISE EXCEPTION 'Academic year not found';
  END IF;

  IF v_year.is_current THEN
    RAISE EXCEPTION 'Cannot delete the current academic year. Set another year as current first.
';
  END IF;

  SELECT COUNT(*) INTO v_remaining FROM public.academic_years;
  IF v_remaining <= 1 THEN
    RAISE EXCEPTION 'Cannot delete the last academic year';
  END IF;

  DELETE FROM public.academic_years
```

```
  WHERE id = p_year_id
  RETURNING * INTO v_year;

  RETURN v_year;
END;
$function$

-- FILE: fn_admin_mass_generate_school_data.sql
CREATE OR REPLACE FUNCTION public.admin_mass_generate_school_data(p_teachers jsonb, p_students
jsonb, p_groups jsonb, p_assignments jsonb)
 RETURNS jsonb
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth', 'extensions'
AS $function$
DECLARE
  v_errors TEXT[] := ARRAY[]::TEXT[];

  v_teacher_upserted INT := 0;
  v_student_upserted INT := 0;
  v_groups_created INT := 0;
  v_assignments_inserted INT := 0;

  -- Auth/profile ids
  v_profile_id UUID;
  v_user_id UUID;
  v_teacher_id UUID;
  v_student_id UUID;
  v_group_id UUID;

  -- temp strings
  v_username TEXT;
  v_password TEXT;
  v_full_name TEXT;
  v_sphere TEXT;
  v_email TEXT;

  v_teacher_username TEXT;
  v_group_title TEXT;
  v_student_username TEXT;

  v_row JSONB;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  -- ============================
  -- Teachers + teacher rows
  -- ============================
  IF p_teachers IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_teachers)
    LOOP
      v_username := trim(both from v_row->>'username');
      v_password := v_row->>'password';
      v_full_name := trim(both from v_row->>'full_name');
      v_sphere := lower(trim(both from v_row->>'sphere'));

      IF v_username IS NULL OR v_username = '' OR v_password IS NULL OR v_password = '' THEN
        v_errors := v_errors || format('Teacher missing username/password: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      IF v_sphere IS NULL OR (v_sphere <> 'math' AND v_sphere <> 'english') THEN
        v_errors := v_errors || format('Teacher invalid sphere for %s: %s', v_username,
COALESCE(v_sphere,'<null>'));
        CONTINUE;
      END IF;

      v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

      -- Existing profile?
      SELECT id INTO v_profile_id
```

```
      FROM public.profiles
      WHERE username = v_username
      LIMIT 1;

      IF v_profile_id IS NULL THEN
        v_user_id := gen_random_uuid();

        INSERT INTO auth.users (
          instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
          raw_app_meta_data, raw_user_meta_data,
          created_at, updated_at,
          confirmation_token, recovery_token, email_change_token_new, email_change,
          is_sso_user, is_anonymous
        ) VALUES (
          '00000000-0000-0000-0000-000000000000',
          v_user_id,
          'authenticated',
          'authenticated',
          v_email,
          extensions.crypt(v_password, extensions.gen_salt('bf')),
          NOW(),
          jsonb_build_object('provider','email','providers',ARRAY['email']),
          jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role',
'teacher'),
          NOW(),
          NOW(),
          '', '', '', '',
          false, false
        );

        INSERT INTO auth.identities (
          id, user_id, identity_data, provider, provider_id,
          last_sign_in_at, created_at, updated_at
        ) VALUES (
          gen_random_uuid(),
          v_user_id,
          jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
          'email',
          v_user_id::text,
          NOW(), NOW(), NOW()
        );

        INSERT INTO public.profiles (id, username, full_name, role, auth_email)
        VALUES (v_user_id, v_username, COALESCE(v_full_name,'') , 'teacher', v_email);

        v_profile_id := v_user_id;
      ELSE
        -- Update password + basic profile fields
        v_user_id := v_profile_id;
        UPDATE auth.users
        SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
            updated_at = NOW()
        WHERE id = v_user_id;

        UPDATE public.profiles
        SET full_name = COALESCE(v_full_name,''),
            role = 'teacher',
            auth_email = v_email
        WHERE id = v_user_id;
      END IF;

      -- Upsert teachers row
      INSERT INTO public.teachers (profile_id, sphere)
      VALUES (
        v_profile_id,
        CASE WHEN v_sphere = 'math' THEN 'math'::public.sphere_type ELSE 'english'::public.
sphere_type END
      )
      ON CONFLICT (profile_id) DO UPDATE
      SET sphere = EXCLUDED.sphere;

      v_teacher_upserted := v_teacher_upserted + 1;
    END LOOP;
```

```
  END IF;

  -- ==========================
  -- Students + student rows
  -- ==========================
  IF p_students IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_students)
    LOOP
      v_username := trim(both from v_row->>'username');
      v_password := v_row->>'password';
      v_full_name := trim(both from v_row->>'full_name');

      IF v_username IS NULL OR v_username = '' OR v_password IS NULL OR v_password = '' THEN
        v_errors := v_errors || format('Student missing username/password: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

      SELECT id INTO v_profile_id
      FROM public.profiles
      WHERE username = v_username
      LIMIT 1;

      IF v_profile_id IS NULL THEN
        v_user_id := gen_random_uuid();

        INSERT INTO auth.users (
          instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
          raw_app_meta_data, raw_user_meta_data,
          created_at, updated_at,
          confirmation_token, recovery_token, email_change_token_new, email_change,
          is_sso_user, is_anonymous
        ) VALUES (
          '00000000-0000-0000-0000-000000000000',
          v_user_id,
          'authenticated',
          'authenticated',
          v_email,
          extensions.crypt(v_password, extensions.gen_salt('bf')),
          NOW(),
          jsonb_build_object('provider','email','providers',ARRAY['email']),
          jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role',
'student'),
          NOW(),
          NOW(),
          '', '', '', '',
          false, false
        );

        INSERT INTO auth.identities (
          id, user_id, identity_data, provider, provider_id,
          last_sign_in_at, created_at, updated_at
        ) VALUES (
          gen_random_uuid(),
          v_user_id,
          jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
          'email',
          v_user_id::text,
          NOW(), NOW(), NOW()
        );

        INSERT INTO public.profiles (id, username, full_name, role, auth_email)
        VALUES (v_user_id, v_username, COALESCE(v_full_name,'') , 'student', v_email);

        v_profile_id := v_user_id;
      ELSE
        v_user_id := v_profile_id;
        UPDATE auth.users
        SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
            updated_at = NOW()
        WHERE id = v_user_id;
```

```
        UPDATE public.profiles
        SET full_name = COALESCE(v_full_name,''),
```

**COMPLETE_BACKEND.sql — continued (3/6)**

```
            role = 'student',
            auth_email = v_email
        WHERE id = v_user_id;
      END IF;

      INSERT INTO public.students (profile_id)
      VALUES (v_profile_id)
      ON CONFLICT (profile_id) DO NOTHING;

      v_student_upserted := v_student_upserted + 1;
    END LOOP;
  END IF;

  -- ==========================
  -- Groups (exact titles)
  -- ==========================
  IF p_groups IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_groups)
    LOOP
      v_teacher_username := trim(both from v_row->>'teacher_username');
      v_group_title := trim(both from v_row->>'title');

      IF v_teacher_username IS NULL OR v_teacher_username = '' OR v_group_title IS NULL OR
v_group_title = '' THEN
        v_errors := v_errors || format('Group missing teacher_username/title: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      SELECT t.id INTO v_teacher_id
      FROM public.teachers t
      JOIN public.profiles p ON p.id = t.profile_id
      WHERE p.username = v_teacher_username
        AND p.role = 'teacher'
      LIMIT 1;

      IF v_teacher_id IS NULL THEN
        v_errors := v_errors || format('Unknown teacher for group %s (teacher_username=%s)',
v_group_title, v_teacher_username);
        CONTINUE;
      END IF;

      SELECT g.id INTO v_group_id
      FROM public.groups g
      WHERE g.teacher_id = v_teacher_id
        AND g.title = v_group_title
      LIMIT 1;

      IF v_group_id IS NULL THEN
        INSERT INTO public.groups (teacher_id, title)
        VALUES (v_teacher_id, v_group_title);
        v_groups_created := v_groups_created + 1;
      END IF;
    END LOOP;
  END IF;

  -- ==========================
  -- Assign students -> groups
  -- ==========================
  IF p_assignments IS NOT NULL THEN
    FOR v_row IN SELECT * FROM jsonb_array_elements(p_assignments)
    LOOP
      v_teacher_username := trim(both from v_row->>'teacher_username');
      v_group_title := trim(both from v_row->>'group_title');
      v_student_username := trim(both from v_row->>'student_username');

      IF v_teacher_username IS NULL OR v_teacher_username = '' OR
         v_group_title IS NULL OR v_group_title = '' OR
         v_student_username IS NULL OR v_student_username = '' THEN
```

```
        v_errors := v_errors || format('Assignment missing teacher_username/group_title/
student_username: %s', v_row::TEXT);
        CONTINUE;
      END IF;

      -- Resolve group_id by teacher_username + group_title
      SELECT g.id INTO v_group_id
      FROM public.groups g
      JOIN public.teachers t ON t.id = g.teacher_id
      JOIN public.profiles p ON p.id = t.profile_id
      WHERE p.username = v_teacher_username
        AND p.role = 'teacher'
        AND g.title = v_group_title
      LIMIT 1;

      IF v_group_id IS NULL THEN
        v_errors := v_errors || format('Unknown group for assignment %s -> (teacher=%s,
group_title=%s)', v_student_username, v_teacher_username, v_group_title);
        CONTINUE;
      END IF;

      -- Resolve student_id
      SELECT s.id INTO v_student_id
      FROM public.students s
      JOIN public.profiles p ON p.id = s.profile_id
      WHERE p.username = v_student_username
        AND p.role = 'student'
      LIMIT 1;

      IF v_student_id IS NULL THEN
        v_errors := v_errors || format('Unknown student username: %s', v_student_username);
        CONTINUE;
      END IF;

      INSERT INTO public.group_students (group_id, student_id)
      VALUES (v_group_id, v_student_id)
      ON CONFLICT (group_id, student_id) DO NOTHING;

      -- We don't have a portable "rowcount" here without extra GET DIAGNOSTICS.
      -- Still count by checking existence after insert.
      IF EXISTS (
        SELECT 1 FROM public.group_students gs
        WHERE gs.group_id = v_group_id AND gs.student_id = v_student_id
      ) THEN
        v_assignments_inserted := v_assignments_inserted + 1;
      END IF;
    END LOOP;
  END IF;

  RETURN jsonb_build_object(
    'teachers_upserted', v_teacher_upserted,
    'students_upserted', v_student_upserted,
    'groups_created', v_groups_created,
    'assignments_inserted', v_assignments_inserted,
    'errors', to_jsonb(v_errors)
  );
END;
$function$

-- FILE: fn_admin_reset_password.sql
CREATE OR REPLACE FUNCTION public.admin_reset_password(p_user_id uuid, p_password text)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'auth', 'public', 'extensions'
AS $function$
BEGIN
  IF NOT public.is_admin() THEN RAISE EXCEPTION 'Not authorized'; END IF;
  UPDATE auth.users SET encrypted_password = extensions.crypt(p_password, extensions.gen_salt(
'bf')), updated_at = NOW() WHERE id = p_user_id;
END; $function$

-- FILE: fn_admin_set_current_academic_year.sql
```

```
CREATE OR REPLACE FUNCTION public.admin_set_current_academic_year(p_year_id uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can set the current academic year';
  END IF;

  UPDATE public.academic_years SET is_current = FALSE WHERE is_current;

  UPDATE public.academic_years
  SET is_current = TRUE
  WHERE id = p_year_id
  RETURNING * INTO v_year;

  IF v_year.id IS NULL THEN
    RAISE EXCEPTION 'Academic year not found';
  END IF;

  RETURN v_year;
END;
$function$

-- FILE: fn_admin_update_academic_term.sql
CREATE OR REPLACE FUNCTION public.admin_update_academic_term(p_term_id uuid, p_starts_on date,
p_ends_on date)
 RETURNS academic_terms
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_term public.academic_terms;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Only admins can edit term dates';
  END IF;

  IF p_ends_on < p_starts_on THEN
    RAISE EXCEPTION 'End date must be on or after start date';
  END IF;

  UPDATE public.academic_terms
  SET starts_on = p_starts_on, ends_on = p_ends_on
  WHERE id = p_term_id
  RETURNING * INTO v_term;

  IF v_term.id IS NULL THEN
    RAISE EXCEPTION 'Academic term not found';
  END IF;

  RETURN v_term;
END;
$function$

-- FILE: fn_attendance_sync_session.sql
CREATE OR REPLACE FUNCTION public.attendance_sync_session()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  IF NEW.date IS NOT NULL AND NEW.date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot mark attendance for a future date'
      USING ERRCODE = 'check_violation';
  END IF;
```

```
  IF NEW.session_id IS NULL AND NEW.group_id IS NOT NULL AND NEW.date IS NOT NULL THEN
    NEW.session_id := public.ensure_lesson_session(NEW.group_id, NEW.date::date);
  ELSIF NEW.session_id IS NOT NULL THEN
    SELECT ls.group_id, ls.session_date
      INTO NEW.group_id, NEW.date
    FROM public.lesson_sessions ls
    WHERE ls.id = NEW.session_id;
  END IF;

  IF NEW.date IS NOT NULL AND NEW.date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot mark attendance for a future date'
      USING ERRCODE = 'check_violation';
  END IF;

  RETURN NEW;
END;
$function$

-- FILE: fn_can_access_chat_room.sql
CREATE OR REPLACE FUNCTION public.can_access_chat_room(rid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT public.is_admin()
    OR EXISTS (
      SELECT 1
      FROM public.chat_rooms cr
      WHERE cr.id = rid
        AND (
          public.is_teacher_of_group(cr.group_id)
          OR public.is_student_in_group(cr.group_id)
        )
    );
$function$

-- FILE: fn_chat_messages_reply_same_room.sql
CREATE OR REPLACE FUNCTION public.chat_messages_reply_same_room()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE
  v_room UUID;
BEGIN
  IF NEW.reply_to_message_id IS NULL THEN
    RETURN NEW;
  END IF;
  IF NEW.reply_to_message_id = NEW.id THEN
    RAISE EXCEPTION 'message cannot reply to itself';
  END IF;
  SELECT room_id INTO v_room
  FROM public.chat_messages
  WHERE id = NEW.reply_to_message_id;
  IF v_room IS NULL THEN
    RAISE EXCEPTION 'reply target not found';
  END IF;
  IF v_room <> NEW.room_id THEN
    RAISE EXCEPTION 'reply must be in the same chat room';
  END IF;
  RETURN NEW;
END;
$function$

-- FILE: fn_chat_storage_room_id.sql
CREATE OR REPLACE FUNCTION public.chat_storage_room_id(object_name text)
 RETURNS uuid
 LANGUAGE plpgsql
 IMMUTABLE
 SET search_path TO 'public'
AS $function$
DECLARE
  part TEXT := split_part(object_name, '/', 1);
```

```
BEGIN
  IF part ~* '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$' THEN
    RETURN part::UUID;
  END IF;
  RETURN NULL;
END;
$function$

-- FILE: fn_chat_unread_counts.sql
CREATE OR REPLACE FUNCTION public.chat_unread_counts()
 RETURNS TABLE(room_id uuid, unread_count bigint)
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT
    m.room_id,
    COUNT(*)::BIGINT AS unread_count
  FROM public.chat_messages m
  WHERE m.is_deleted = FALSE
    AND m.sender_id <> auth.uid()
    AND public.can_access_chat_room(m.room_id)
    AND NOT EXISTS (
      SELECT 1
      FROM public.message_reads r
      WHERE r.message_id = m.id
        AND r.user_id = auth.uid()
    )
  GROUP BY m.room_id;
$function$

-- FILE: fn_create_academic_year_with_terms.sql
CREATE OR REPLACE FUNCTION public.create_academic_year_with_terms(p_start_year integer,
p_is_current boolean DEFAULT false, p_created_by uuid DEFAULT NULL::uuid)
 RETURNS academic_years
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_year public.academic_years;
  v_sem INT;
  v_bounds RECORD;
BEGIN
  IF p_is_current THEN
    UPDATE public.academic_years SET is_current = FALSE WHERE is_current;
  END IF;

  INSERT INTO public.academic_years (start_year, label, is_current, created_by)
  VALUES (
    p_start_year,
    public.academic_year_label(p_start_year),
    p_is_current,
    p_created_by
  )
  RETURNING * INTO v_year;

  FOR v_sem IN 1..4 LOOP
    SELECT * INTO v_bounds
    FROM public.academic_semester_bounds(p_start_year, v_sem);

    INSERT INTO public.academic_terms (
      academic_year_id, semester, starts_on, ends_on
    ) VALUES (
      v_year.id, v_sem, v_bounds.starts_on, v_bounds.ends_on
    );
  END LOOP;

  RETURN v_year;
END;
$function$

-- FILE: fn_current_profile_role.sql
```

```
CREATE OR REPLACE FUNCTION public.current_profile_role()
 RETURNS user_role
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$ SELECT role FROM public.profiles WHERE id = auth.uid(); $function$

-- FILE: fn_ensure_chat_room_for_group.sql
CREATE OR REPLACE FUNCTION public.ensure_chat_room_for_group()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  INSERT INTO public.chat_rooms (group_id)
  VALUES (NEW.id)
  ON CONFLICT (group_id) DO NOTHING;
  RETURN NEW;
END;
$function$

-- FILE: fn_ensure_lesson_session.sql
CREATE OR REPLACE FUNCTION public.ensure_lesson_session(p_group_id uuid, p_session_date date)
 RETURNS uuid
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  sid UUID;
BEGIN
  IF p_session_date > CURRENT_DATE THEN
    RAISE EXCEPTION 'Cannot create a lesson session for a future date'
      USING ERRCODE = 'check_violation';
  END IF;

  INSERT INTO public.lesson_sessions (group_id, session_date)
  VALUES (p_group_id, p_session_date)
  ON CONFLICT (group_id, session_date)
  DO UPDATE SET group_id = EXCLUDED.group_id
  RETURNING id INTO sid;
  RETURN sid;
END;
$function$

-- FILE: fn_get_auth_email_by_username.sql
CREATE OR REPLACE FUNCTION public.get_auth_email_by_username(p_username text)
 RETURNS text
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT auth_email FROM public.profiles
  WHERE lower(username) = lower(trim(p_username)) AND is_disabled = FALSE LIMIT 1;
$function$

-- FILE: fn_is_admin.sql
CREATE OR REPLACE FUNCTION public.is_admin()
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin' AND is_disabled =
FALSE
  );
$function$

-- FILE: fn_is_student_in_group.sql
CREATE OR REPLACE FUNCTION public.is_student_in_group(gid uuid)
 RETURNS boolean
```

```
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = gid AND s.profile_id = auth.uid()
  );
$function$

-- FILE: fn_is_teacher_of_group.sql
CREATE OR REPLACE FUNCTION public.is_teacher_of_group(gid uuid)
 RETURNS boolean
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT EXISTS (
    SELECT 1 FROM public.groups g
    JOIN public.teachers t ON t.id = g.teacher_id
    WHERE g.id = gid AND t.profile_id = auth.uid()
  );
$function$

-- FILE: fn_notify_chat_message.sql
CREATE OR REPLACE FUNCTION public.notify_chat_message(p_user_id uuid, p_title text, p_body text,
 p_action_path text, p_room_id uuid)
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_id UUID;
BEGIN
  SELECT id INTO v_id
  FROM public.notifications
  WHERE user_id = p_user_id
    AND kind = 'chat'
    AND ref_id = p_room_id
    AND is_read = FALSE
  ORDER BY created_at DESC
  LIMIT 1;

  IF v_id IS NOT NULL THEN
    UPDATE public.notifications
    SET title = p_title,
        body = COALESCE(p_body, ''),
        action_path = p_action_path,
        created_at = NOW()
    WHERE id = v_id;
  ELSE
    PERFORM public.notify_user(
      p_user_id, p_title, p_body, 'chat', p_action_path, p_room_id
    );
  END IF;
END;
$function$

-- FILE: fn_notify_user.sql
CREATE OR REPLACE FUNCTION public.notify_user(p_user_id uuid, p_title text, p_body text, p_kind
text DEFAULT 'system'::text, p_action_path text DEFAULT NULL::text, p_ref_id uuid DEFAULT NULL::
uuid)
```

**COMPLETE_BACKEND.sql — continued (4/6)**

```
 RETURNS void
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
```

```
  INSERT INTO public.notifications (user_id, title, body, kind, action_path, ref_id)
  VALUES (
    p_user_id,
    p_title,
    COALESCE(p_body, ''),
    COALESCE(NULLIF(trim(p_kind), ''), 'system'),
    p_action_path,
    p_ref_id
  );
END;
$function$

-- FILE: fn_profile_route_prefix.sql
CREATE OR REPLACE FUNCTION public.profile_route_prefix(p_user_id uuid)
 RETURNS text
 LANGUAGE sql
 STABLE SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
  SELECT CASE role
    WHEN 'teacher' THEN '/teacher'
    WHEN 'admin' THEN '/admin'
    ELSE '/student'
  END
  FROM public.profiles
  WHERE id = p_user_id;
$function$

-- FILE: fn_seed_demo_data.sql
CREATE OR REPLACE FUNCTION public.seed_demo_data()
 RETURNS text
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public', 'auth'
AS $function$
DECLARE
  v_admin UUID; v_math UUID; v_eng UUID; v_student UUID;
  v_math_tid UUID; v_eng_tid UUID; v_student_sid UUID;
  v_group UUID; v_pack UUID;
BEGIN
  IF EXISTS (SELECT 1 FROM public.profiles LIMIT 1) THEN RETURN 'already_seeded'; END IF;

  v_admin := public._demo_insert_user('@admin', 'password123', 'Platform Admin', 'admin');
  v_math := public._demo_insert_user('@math_teacher', 'password123', 'Amina Math', 'teacher');
  v_eng := public._demo_insert_user('@english_teacher', 'password123', 'James English',
'teacher');
  v_student := public._demo_insert_user('@test_user', 'password123', 'Demo Student', 'student');

  INSERT INTO public.teachers (profile_id, sphere) VALUES (v_math, 'math') RETURNING id INTO
v_math_tid;
  INSERT INTO public.teachers (profile_id, sphere) VALUES (v_eng, 'english') RETURNING id INTO
v_eng_tid;

  INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group A') RETURNING
id INTO v_group;
  INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group B');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_math_tid, 'Math Group C');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group A');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group B');
  INSERT INTO public.groups (teacher_id, title) VALUES (v_eng_tid, 'English Group C');

  SELECT id INTO v_student_sid FROM public.students WHERE profile_id = v_student;
  INSERT INTO public.group_students (group_id, student_id) VALUES (v_group, v_student_sid);

  INSERT INTO public.assignments (teacher_id, group_id, type, title, description, due_date)
  VALUES (v_math_tid, v_group, 'static', 'Read page 12', 'Review chapter on linear equations.',
NOW() + INTERVAL '2 days');

  INSERT INTO public.assignments (teacher_id, group_id, type, title, description, due_date)
  VALUES (v_math_tid, v_group, 'homework_pack', 'Algebra Week 1 Pack', 'Complete all tasks
below.', NOW() + INTERVAL '5 days')
  RETURNING id INTO v_pack;
```

```
  INSERT INTO public.tasks (assignment_id, title, description, sort_order, link_url)
  VALUES (v_pack, 'Practice set A', 'Solve exercises 1-10.', 1, 'https://www.khanacademy.org/
math/algebra');
  INSERT INTO public.tasks (assignment_id, title, description, sort_order)
  VALUES (v_pack, 'Written reflection', 'Explain how you checked your answers.', 2);

  INSERT INTO public.announcements (teacher_id, group_id, title, description)
  VALUES (v_math_tid, v_group, 'Welcome to Math Group A', 'Office hours every Wednesday.');

  INSERT INTO public.attendance (group_id, student_id, status, date)
  VALUES (v_group, v_student_sid, 'present', CURRENT_DATE);

  INSERT INTO public.notifications (user_id, title, body)
  VALUES (v_student, 'Welcome to MU Study', 'Your demo account is ready.');

  RETURN 'seeded';
END; $function$

-- FILE: fn_sync_submission_mark.sql
CREATE OR REPLACE FUNCTION public.sync_submission_mark()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE v_group_id UUID; v_teacher_id UUID; v_title TEXT;
BEGIN
  IF NEW.score IS NULL THEN
    DELETE FROM public.marks WHERE submission_id = NEW.id;
    RETURN NEW;
  END IF;

  SELECT a.group_id, a.teacher_id, tk.title
    INTO v_group_id, v_teacher_id, v_title
  FROM public.tasks tk
  JOIN public.assignments a ON a.id = tk.assignment_id
  WHERE tk.id = NEW.task_id;

  IF v_group_id IS NULL THEN RETURN NEW; END IF;

  INSERT INTO public.marks (
    group_id, student_id, teacher_id, submission_id, title, category, percentage, comment
  ) VALUES (
    v_group_id, NEW.student_id, v_teacher_id, NEW.id,
    COALESCE(v_title, 'Homework'), 'homework', NEW.score, COALESCE(NEW.feedback, '')
  )
  ON CONFLICT (submission_id) DO UPDATE SET
    percentage = EXCLUDED.percentage,
    title = EXCLUDED.title,
    comment = EXCLUDED.comment,
    updated_at = NOW();

  RETURN NEW;
END; $function$

-- FILE: fn_toggle_message_reaction.sql
CREATE OR REPLACE FUNCTION public.toggle_message_reaction(p_message_id uuid, p_emoji text)
 RETURNS message_reactions
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_uid UUID := auth.uid();
  v_existing public.message_reactions;
  v_result public.message_reactions;
BEGIN
  IF v_uid IS NULL THEN
    RAISE EXCEPTION 'Not authenticated';
  END IF;

  IF p_emoji IS NULL OR char_length(trim(p_emoji)) < 1 THEN
    RAISE EXCEPTION 'Emoji required';
```

```
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM public.chat_messages m
    WHERE m.id = p_message_id
      AND m.is_deleted = FALSE
      AND public.can_access_chat_room(m.room_id)
  ) THEN
    RAISE EXCEPTION 'Message not found or inaccessible';
  END IF;

  SELECT * INTO v_existing
  FROM public.message_reactions
  WHERE message_id = p_message_id AND user_id = v_uid;

  IF v_existing.id IS NOT NULL AND v_existing.emoji = p_emoji THEN
    DELETE FROM public.message_reactions WHERE id = v_existing.id;
    RETURN v_existing;
  END IF;

  INSERT INTO public.message_reactions (message_id, user_id, emoji)
  VALUES (p_message_id, v_uid, p_emoji)
  ON CONFLICT (message_id, user_id)
  DO UPDATE SET emoji = EXCLUDED.emoji, created_at = NOW()
  RETURNING * INTO v_result;

  RETURN v_result;
END;
$function$

-- FILE: fn_touch_chat_message_updated_at.sql
CREATE OR REPLACE FUNCTION public.touch_chat_message_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
BEGIN
  NEW.updated_at := NOW();
  RETURN NEW;
END;
$function$

-- FILE: fn_touch_updated_at.sql
CREATE OR REPLACE FUNCTION public.touch_updated_at()
 RETURNS trigger
 LANGUAGE plpgsql
 SET search_path TO 'public'
AS $function$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END; $function$

-- FILE: fn_trg_announcement_notify.sql
CREATE OR REPLACE FUNCTION public.trg_announcement_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_prefix TEXT;
BEGIN
  FOR r IN
    SELECT s.profile_id AS uid
    FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = NEW.group_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    PERFORM public.notify_user(
      r.uid,
      'Announcement',
      NEW.title,
```

```
      'announcement',
      v_prefix || '/dashboard',
      NEW.id
    );
  END LOOP;
  RETURN NEW;
END;
$function$

-- FILE: fn_trg_assignment_notify.sql
CREATE OR REPLACE FUNCTION public.trg_assignment_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_prefix TEXT;
BEGIN
  FOR r IN
    SELECT s.profile_id AS uid
    FROM public.group_students gs
    JOIN public.students s ON s.id = gs.student_id
    WHERE gs.group_id = NEW.group_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    PERFORM public.notify_user(
      r.uid,
      'New homework',
      NEW.title,
      'assignment',
      v_prefix || '/assignments/' || NEW.id::text,
      NEW.id
    );
  END LOOP;
  RETURN NEW;
END;
$function$

-- FILE: fn_trg_chat_message_notify.sql
CREATE OR REPLACE FUNCTION public.trg_chat_message_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  r RECORD;
  v_group_id UUID;
  v_group_title TEXT;
  v_sender_name TEXT;
  v_preview TEXT;
  v_prefix TEXT;
  v_path TEXT;
BEGIN
  IF NEW.is_deleted THEN
    RETURN NEW;
  END IF;

  SELECT cr.group_id, g.title
    INTO v_group_id, v_group_title
  FROM public.chat_rooms cr
  JOIN public.groups g ON g.id = cr.group_id
  WHERE cr.id = NEW.room_id;

  IF v_group_id IS NULL THEN RETURN NEW; END IF;

  SELECT COALESCE(full_name, username, 'Someone')
    INTO v_sender_name
  FROM public.profiles
  WHERE id = NEW.sender_id;
```

```
  v_preview := CASE NEW.message_type
    WHEN 'text' THEN left(COALESCE(NEW.text_content, ''), 120)
    WHEN 'image' THEN 'Photo'
    WHEN 'video' THEN 'Video'
    WHEN 'voice' THEN 'Voice message'
    ELSE 'New message'
  END;

  FOR r IN
    SELECT DISTINCT uid FROM (
      SELECT s.profile_id AS uid
      FROM public.group_students gs
      JOIN public.students s ON s.id = gs.student_id
      WHERE gs.group_id = v_group_id
      UNION
      SELECT t.profile_id AS uid
      FROM public.groups g
      JOIN public.teachers t ON t.id = g.teacher_id
      WHERE g.id = v_group_id
    ) members
    WHERE uid IS DISTINCT FROM NEW.sender_id
  LOOP
    v_prefix := COALESCE(public.profile_route_prefix(r.uid), '/student');
    v_path := v_prefix || '/chat/' || NEW.room_id::text;
    PERFORM public.notify_chat_message(
      r.uid,
      COALESCE(v_group_title, 'Chat'),
      COALESCE(v_sender_name, 'Someone') || ': ' || v_preview,
      v_path,
      NEW.room_id
    );
  END LOOP;

  RETURN NEW;
END;
$function$

-- FILE: fn_trg_mark_notify.sql
CREATE OR REPLACE FUNCTION public.trg_mark_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_uid UUID;
  v_prefix TEXT;
  v_body TEXT;
BEGIN
  IF TG_OP = 'UPDATE'
     AND NEW.percentage IS NOT DISTINCT FROM OLD.percentage
     AND NEW.comment IS NOT DISTINCT FROM OLD.comment THEN
    RETURN NEW;
  END IF;

  SELECT profile_id INTO v_uid FROM public.students WHERE id = NEW.student_id;
  IF v_uid IS NULL THEN RETURN NEW; END IF;

  v_prefix := COALESCE(public.profile_route_prefix(v_uid), '/student');
  v_body := NEW.title || ' — ' || ROUND(NEW.percentage)::TEXT || '%';
  IF NEW.comment IS NOT NULL AND length(trim(NEW.comment)) > 0 THEN
    v_body := v_body || E'\n' || left(trim(NEW.comment), 160);
  END IF;

  PERFORM public.notify_user(
    v_uid,
    'Work graded',
    v_body,
    'graded',
    v_prefix || '/marks',
    NEW.id
  );
  RETURN NEW;
```

```
END;
$function$

-- FILE: fn_trg_submission_notify.sql
CREATE OR REPLACE FUNCTION public.trg_submission_notify()
 RETURNS trigger
 LANGUAGE plpgsql
 SECURITY DEFINER
 SET search_path TO 'public'
AS $function$
DECLARE
  v_teacher_profile UUID;
  v_student_name TEXT;
  v_task_title TEXT;
  v_assignment_id UUID;
  v_prefix TEXT;
  v_is_submit BOOLEAN;
BEGIN
  v_is_submit :=
    (TG_OP = 'INSERT' AND (NEW.file_url IS NOT NULL OR NEW.text_answer IS NOT NULL OR NEW.
submitted_at IS NOT NULL))
    OR (
      TG_OP = 'UPDATE'
      AND (
        (OLD.file_url IS DISTINCT FROM NEW.file_url AND NEW.file_url IS NOT NULL)
        OR (OLD.text_answer IS DISTINCT FROM NEW.text_answer AND NEW.text_answer IS NOT NULL)
        OR (OLD.submitted_at IS DISTINCT FROM NEW.submitted_at AND NEW.submitted_at IS NOT NULL)
        OR (OLD.is_completed IS DISTINCT FROM NEW.is_completed AND NEW.is_completed IS TRUE)
      )
    );

  IF NOT v_is_submit THEN
    RETURN NEW;
  END IF;

  IF TG_OP = 'UPDATE'
     AND OLD.score IS DISTINCT FROM NEW.score
     AND OLD.file_url IS NOT DISTINCT FROM NEW.file_url
     AND OLD.text_answer IS NOT DISTINCT FROM NEW.text_answer
     AND OLD.submitted_at IS NOT DISTINCT FROM NEW.submitted_at
     AND OLD.is_completed IS NOT DISTINCT FROM NEW.is_completed THEN
    RETURN NEW;
  END IF;

  SELECT t.profile_id, tk.title, a.id
    INTO v_teacher_profile, v_task_title, v_assignment_id
  FROM public.tasks tk
  JOIN public.assignments a ON a.id = tk.assignment_id
  JOIN public.teachers t ON t.id = a.teacher_id
  WHERE tk.id = NEW.task_id;

  IF v_teacher_profile IS NULL THEN RETURN NEW; END IF;

  SELECT COALESCE(p.full_name, p.username, 'Student')
    INTO v_student_name
  FROM public.students s
  JOIN public.profiles p ON p.id = s.profile_id
  WHERE s.id = NEW.student_id;

  v_prefix := COALESCE(public.profile_route_prefix(v_teacher_profile), '/teacher');

  PERFORM public.notify_user(
    v_teacher_profile,
    'New submission',
    COALESCE(v_student_name, 'Student') || ' · ' || COALESCE(v_task_title, 'Task'),
    'submission',
    v_prefix || '/assignments/' || v_assignment_id::text,
    NEW.id
  );
  RETURN NEW;
END;
$function$
```

```
-- ---------- TRIGGERS ----------
CREATE TRIGGER announcement_created_notify AFTER INSERT ON announcements FOR EACH ROW EXECUTE
FUNCTION trg_announcement_notify();
CREATE TRIGGER assignment_created_notify AFTER INSERT ON assignments FOR EACH ROW EXECUTE
FUNCTION trg_assignment_notify();
CREATE TRIGGER tr_attendance_sync_session BEFORE INSERT OR UPDATE ON attendance FOR EACH ROW
EXECUTE FUNCTION attendance_sync_session();
CREATE TRIGGER tr_chat_message_notify AFTER INSERT ON chat_messages FOR EACH ROW EXECUTE
FUNCTION trg_chat_message_notify();
CREATE TRIGGER tr_chat_messages_reply_same_room BEFORE INSERT OR UPDATE OF reply_to_message_id,
room_id ON chat_messages FOR EACH ROW EXECUTE FUNCTION chat_messages_reply_same_room();
CREATE TRIGGER tr_chat_messages_updated_at BEFORE UPDATE ON chat_messages FOR EACH ROW EXECUTE
FUNCTION touch_chat_message_updated_at();
CREATE TRIGGER tr_groups_create_chat_room AFTER INSERT ON groups FOR EACH ROW EXECUTE FUNCTION
ensure_chat_room_for_group();
```

**COMPLETE_BACKEND.sql — continued (5/6)**

```
CREATE TRIGGER tr_mark_notify AFTER INSERT OR UPDATE ON marks FOR EACH ROW EXECUTE FUNCTION
trg_mark_notify();
CREATE TRIGGER tr_marks_touch BEFORE UPDATE ON marks FOR EACH ROW EXECUTE FUNCTION
touch_updated_at();
CREATE TRIGGER tr_submission_mark AFTER INSERT OR UPDATE OF score, feedback ON submissions FOR
EACH ROW EXECUTE FUNCTION sync_submission_mark();
CREATE TRIGGER tr_submission_notify AFTER INSERT OR UPDATE ON submissions FOR EACH ROW EXECUTE
FUNCTION trg_submission_notify();
```

```
-- MU Study — Complete RLS Policies (EXACT from live project)
-- Run AFTER helper functions (is_admin, is_teacher_of_group, is_student_in_group,
-- can_access_chat_room, current_profile_role) exist.

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.teachers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.groups ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.group_students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.submissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.attendance ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.lesson_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.marks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.announcements ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignment_templates ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.academic_years ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.academic_terms ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_rooms ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reads ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reactions ENABLE ROW LEVEL SECURITY;

-- profiles
CREATE POLICY profiles_admin_all ON public.profiles FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY profiles_update_self ON public.profiles FOR UPDATE TO authenticated
  USING ((id = auth.uid()) AND (is_disabled = false)) WITH CHECK (id = auth.uid());
CREATE POLICY profiles_select ON public.profiles FOR SELECT TO authenticated USING (
  (id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students st ON ((st.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (st.profile_id = profiles.id))
  ))
);
CREATE POLICY profiles_select_groupmates ON public.profiles FOR SELECT TO authenticated USING (
  (EXISTS (
    SELECT 1 FROM (((group_students gs_self
      JOIN students s_self ON ((s_self.id = gs_self.student_id)))
```

```
      JOIN group_students gs_other ON ((gs_other.group_id = gs_self.group_id)))
      JOIN students s_other ON ((s_other.id = gs_other.student_id)))
    WHERE ((s_self.profile_id = auth.uid()) AND (s_other.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((group_students gs
      JOIN students s ON ((s.id = gs.student_id)))
      JOIN groups g ON ((g.id = gs.group_id)))
      JOIN teachers t ON ((t.id = g.teacher_id)))
    WHERE ((s.profile_id = auth.uid()) AND (t.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students s ON ((s.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (s.profile_id = profiles.id))
  ))
);
CREATE POLICY profiles_select_chat_message_senders ON public.profiles FOR SELECT TO
authenticated USING (
  EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE ((m.sender_id = profiles.id) AND can_access_chat_room(m.room_id))
  )
);

-- teachers / students
CREATE POLICY teachers_admin ON public.teachers FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY teachers_select ON public.teachers FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'student'::user_role)
);
CREATE POLICY students_admin ON public.students FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY students_select ON public.students FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'teacher'::user_role)
);

-- groups
CREATE POLICY groups_select ON public.groups FOR SELECT TO authenticated USING (
  is_teacher_of_group(id) OR is_student_in_group(id) OR is_admin()
);
CREATE POLICY groups_teacher_manage ON public.groups FOR ALL TO authenticated
  USING (is_teacher_of_group(id) OR is_admin())
  WITH CHECK (is_teacher_of_group(id) OR is_admin());

CREATE POLICY group_students_select ON public.group_students FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY group_students_manage ON public.group_students FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- assignments / tasks
CREATE POLICY assignments_select ON public.assignments FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY assignments_teacher ON public.assignments FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.profile_id
= auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());

CREATE POLICY tasks_select ON public.tasks FOR SELECT TO authenticated USING (
  EXISTS (
    SELECT 1 FROM assignments a
    WHERE a.id = tasks.assignment_id
      AND (is_teacher_of_group(a.group_id) OR is_student_in_group(a.group_id) OR is_admin())
  )
);
CREATE POLICY tasks_teacher ON public.tasks FOR ALL TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
```

```
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin());

-- submissions
CREATE POLICY submissions_select ON public.submissions FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id = auth.
uid()))
  OR (EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  ))
  OR is_admin()
);
CREATE POLICY submissions_student ON public.submissions FOR ALL TO authenticated
  USING (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id
= auth.uid()))
  WITH CHECK (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.
profile_id = auth.uid()));
CREATE POLICY submissions_teacher_update ON public.submissions FOR UPDATE TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin());

-- attendance / sessions
CREATE POLICY attendance_select ON public.attendance FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY attendance_teacher ON public.attendance FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());
CREATE POLICY lesson_sessions_select ON public.lesson_sessions FOR SELECT TO authenticated
USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY lesson_sessions_manage ON public.lesson_sessions FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- marks
CREATE POLICY marks_select ON public.marks FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = marks.student_id AND s.profile_id = auth.uid(
)))
  OR is_teacher_of_group(group_id) OR is_admin()
);
CREATE POLICY marks_teacher_manage ON public.marks FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- announcements / templates / notifications
CREATE POLICY announcements_select ON public.announcements FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY announcements_teacher ON public.announcements FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());
CREATE POLICY assignment_templates_teacher ON public.assignment_templates FOR ALL TO
authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id
AND t.profile_id = auth.uid())) OR is_admin());
CREATE POLICY notifications_own ON public.notifications FOR ALL TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());
```

```
-- academic
CREATE POLICY academic_years_select ON public.academic_years FOR SELECT TO authenticated USING (
true);
CREATE POLICY academic_years_admin ON public.academic_years FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY academic_terms_select ON public.academic_terms FOR SELECT TO authenticated USING (
true);
CREATE POLICY academic_terms_admin ON public.academic_terms FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- chat
CREATE POLICY chat_rooms_select ON public.chat_rooms FOR SELECT TO authenticated
  USING (can_access_chat_room(id));
CREATE POLICY chat_rooms_admin_insert ON public.chat_rooms FOR INSERT TO authenticated
  WITH CHECK (is_admin() OR is_teacher_of_group(group_id));

CREATE POLICY chat_messages_select ON public.chat_messages FOR SELECT TO authenticated
  USING (can_access_chat_room(room_id));
CREATE POLICY chat_messages_insert ON public.chat_messages FOR INSERT TO authenticated
  WITH CHECK ((sender_id = auth.uid()) AND can_access_chat_room(room_id));
CREATE POLICY chat_messages_update ON public.chat_messages FOR UPDATE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)))
  WITH CHECK (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
CREATE POLICY chat_messages_delete ON public.chat_messages FOR DELETE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));

CREATE POLICY message_reads_select ON public.message_reads FOR SELECT TO authenticated USING (
  (user_id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  ))
);
CREATE POLICY message_reads_insert ON public.message_reads FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  )));
CREATE POLICY message_reads_delete ON public.message_reads FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());

CREATE POLICY message_reactions_select ON public.message_reactions FOR SELECT TO authenticated
USING (
  EXISTS (SELECT 1 FROM chat_messages m WHERE m.id = message_reactions.message_id AND
can_access_chat_room(m.room_id))
);
CREATE POLICY message_reactions_insert ON public.message_reactions FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE m.id = message_reactions.message_id AND m.is_deleted = false AND can_access_chat_room(
m.room_id)
  )));
CREATE POLICY message_reactions_update ON public.message_reactions FOR UPDATE TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());
CREATE POLICY message_reactions_delete ON public.message_reactions FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());


-- ---------- STORAGE BUCKETS ----------
INSERT INTO storage.buckets (id, name, public) VALUES
  ('avatars', 'avatars', true),
  ('submissions', 'submissions', false),
  ('assignments', 'assignments', false),
  ('announcements', 'announcements', false),
  ('chat-images', 'chat-images', false),
  ('chat-videos', 'chat-videos', false),
  ('chat-voices', 'chat-voices', false),
  ('chat-files', 'chat-files', false)
ON CONFLICT (id) DO NOTHING;

-- Storage policies (EXACT from live)
CREATE POLICY storage_avatars_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'avatars') WITH CHECK (bucket_id = 'avatars');
CREATE POLICY storage_avatars_read ON storage.objects FOR SELECT TO public
```

```
  USING (bucket_id = 'avatars');
CREATE POLICY storage_submissions_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'submissions') WITH CHECK (bucket_id = 'submissions');
CREATE POLICY storage_assignments_auth ON storage.objects FOR ALL TO authenticated
  USING (bucket_id = 'assignments') WITH CHECK (bucket_id = 'assignments');
CREATE POLICY storage_announcements_auth ON storage.objects FOR ALL TO authenticated
```

**COMPLETE_BACKEND.sql — continued (6/6)**

```
  USING (bucket_id = 'announcements') WITH CHECK (bucket_id = 'announcements');

CREATE POLICY storage_chat_images_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-images' AND can_access_chat_room(chat_storage_room_id(name)));
CREATE POLICY storage_chat_images_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-images' AND can_access_chat_room(chat_storage_room_id(name))
AND (storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_images_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-images' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-images' AND (is_admin() OR (storage.foldername(name))[2] = (
auth.uid())::text));
CREATE POLICY storage_chat_images_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-images' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));

CREATE POLICY storage_chat_videos_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-videos' AND can_access_chat_room(chat_storage_room_id(name)));
CREATE POLICY storage_chat_videos_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-videos' AND can_access_chat_room(chat_storage_room_id(name))
AND (storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_videos_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-videos' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-videos' AND (is_admin() OR (storage.foldername(name))[2] = (
auth.uid())::text));
CREATE POLICY storage_chat_videos_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-videos' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));

CREATE POLICY storage_chat_voices_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-voices' AND can_access_chat_room(chat_storage_room_id(name)));
CREATE POLICY storage_chat_voices_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-voices' AND can_access_chat_room(chat_storage_room_id(name))
AND (storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_voices_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-voices' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-voices' AND (is_admin() OR (storage.foldername(name))[2] = (
auth.uid())::text));
CREATE POLICY storage_chat_voices_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-voices' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));

CREATE POLICY storage_chat_files_select ON storage.objects FOR SELECT TO authenticated
  USING (bucket_id = 'chat-files' AND can_access_chat_room(chat_storage_room_id(name)));
CREATE POLICY storage_chat_files_insert ON storage.objects FOR INSERT TO authenticated
  WITH CHECK (bucket_id = 'chat-files' AND can_access_chat_room(chat_storage_room_id(name)) AND
(storage.foldername(name))[2] = (auth.uid())::text);
CREATE POLICY storage_chat_files_update ON storage.objects FOR UPDATE TO authenticated
  USING (bucket_id = 'chat-files' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text))
  WITH CHECK (bucket_id = 'chat-files' AND (is_admin() OR (storage.foldername(name))[2] = (auth.
uid())::text));
CREATE POLICY storage_chat_files_delete ON storage.objects FOR DELETE TO authenticated
  USING (bucket_id = 'chat-files' AND (is_admin() OR (storage.foldername(name))[2] = (auth.uid(
))::text));


-- ---------- REALTIME ----------
-- Add tables to supabase_realtime publication (Dashboard or):
ALTER PUBLICATION supabase_realtime ADD TABLE public.notifications;
ALTER PUBLICATION supabase_realtime ADD TABLE public.chat_messages;
ALTER PUBLICATION supabase_realtime ADD TABLE public.message_reads;
```

```
ALTER PUBLICATION supabase_realtime ADD TABLE public.message_reactions;


-- ---------- GRANTS (recommended hardening vs live) ----------
-- EXACT live: many RPCs executable by anon; harden for production:
REVOKE EXECUTE ON FUNCTION public.seed_demo_data() FROM anon, authenticated;
REVOKE EXECUTE ON FUNCTION public._demo_insert_user(text, text, text, user_role) FROM anon,
authenticated;
GRANT EXECUTE ON FUNCTION public.get_auth_email_by_username(text) TO anon, authenticated;
GRANT EXECUTE ON FUNCTION public.admin_create_user(text, text, text, user_role, sphere_type) TO
authenticated;
GRANT EXECUTE ON FUNCTION public.admin_reset_password(uuid, text) TO authenticated;
GRANT EXECUTE ON FUNCTION public.chat_unread_counts() TO authenticated;
GRANT EXECUTE ON FUNCTION public.toggle_message_reaction(uuid, text) TO authenticated;
GRANT EXECUTE ON FUNCTION public.ensure_lesson_session(uuid, date) TO authenticated;
GRANT EXECUTE ON FUNCTION public.can_access_chat_room(uuid) TO authenticated;
```

# RLS policies (exact export)

**rls_policies.sql**

```
-- MU Study — Complete RLS Policies (EXACT from live project)
-- Run AFTER helper functions (is_admin, is_teacher_of_group, is_student_in_group,
-- can_access_chat_room, current_profile_role) exist.

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.teachers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.groups ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.group_students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.submissions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.attendance ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.lesson_sessions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.marks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.announcements ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assignment_templates ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.academic_years ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.academic_terms ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_rooms ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.chat_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reads ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reactions ENABLE ROW LEVEL SECURITY;

-- profiles
CREATE POLICY profiles_admin_all ON public.profiles FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY profiles_update_self ON public.profiles FOR UPDATE TO authenticated
  USING ((id = auth.uid()) AND (is_disabled = false)) WITH CHECK (id = auth.uid());
CREATE POLICY profiles_select ON public.profiles FOR SELECT TO authenticated USING (
  (id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students st ON ((st.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (st.profile_id = profiles.id))
  ))
);
CREATE POLICY profiles_select_groupmates ON public.profiles FOR SELECT TO authenticated USING (
  (EXISTS (
    SELECT 1 FROM (((group_students gs_self
      JOIN students s_self ON ((s_self.id = gs_self.student_id)))
      JOIN group_students gs_other ON ((gs_other.group_id = gs_self.group_id)))
      JOIN students s_other ON ((s_other.id = gs_other.student_id)))
    WHERE ((s_self.profile_id = auth.uid()) AND (s_other.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((group_students gs
      JOIN students s ON ((s.id = gs.student_id)))
      JOIN groups g ON ((g.id = gs.group_id)))
      JOIN teachers t ON ((t.id = g.teacher_id)))
    WHERE ((s.profile_id = auth.uid()) AND (t.profile_id = profiles.id))
  )) OR (EXISTS (
    SELECT 1 FROM (((teachers t
      JOIN groups g ON ((g.teacher_id = t.id)))
      JOIN group_students gs ON ((gs.group_id = g.id)))
      JOIN students s ON ((s.id = gs.student_id)))
    WHERE ((t.profile_id = auth.uid()) AND (s.profile_id = profiles.id))
  ))
);
CREATE POLICY profiles_select_chat_message_senders ON public.profiles FOR SELECT TO
authenticated USING (
  EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE ((m.sender_id = profiles.id) AND can_access_chat_room(m.room_id))
  )
);

-- teachers / students
```

```
CREATE POLICY teachers_admin ON public.teachers FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY teachers_select ON public.teachers FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'student'::user_role)
);
CREATE POLICY students_admin ON public.students FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY students_select ON public.students FOR SELECT TO authenticated USING (
  (profile_id = auth.uid()) OR is_admin() OR (current_profile_role() = 'teacher'::user_role)
);

-- groups
CREATE POLICY groups_select ON public.groups FOR SELECT TO authenticated USING (
  is_teacher_of_group(id) OR is_student_in_group(id) OR is_admin()
);
CREATE POLICY groups_teacher_manage ON public.groups FOR ALL TO authenticated
  USING (is_teacher_of_group(id) OR is_admin())
  WITH CHECK (is_teacher_of_group(id) OR is_admin());

CREATE POLICY group_students_select ON public.group_students FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY group_students_manage ON public.group_students FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- assignments / tasks
CREATE POLICY assignments_select ON public.assignments FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY assignments_teacher ON public.assignments FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.profile_id
= auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignments.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());

CREATE POLICY tasks_select ON public.tasks FOR SELECT TO authenticated USING (
  EXISTS (
    SELECT 1 FROM assignments a
    WHERE a.id = tasks.assignment_id
      AND (is_teacher_of_group(a.group_id) OR is_student_in_group(a.group_id) OR is_admin())
  )
);
CREATE POLICY tasks_teacher ON public.tasks FOR ALL TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin())
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (assignments a JOIN teachers t ON t.id = a.teacher_id)
    WHERE a.id = tasks.assignment_id AND t.profile_id = auth.uid()
  )) OR is_admin());

-- submissions
CREATE POLICY submissions_select ON public.submissions FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id = auth.
uid()))
  OR (EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  ))
  OR is_admin()
);
CREATE POLICY submissions_student ON public.submissions FOR ALL TO authenticated
  USING (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.profile_id
= auth.uid()))
  WITH CHECK (EXISTS (SELECT 1 FROM students s WHERE s.id = submissions.student_id AND s.
profile_id = auth.uid()));
CREATE POLICY submissions_teacher_update ON public.submissions FOR UPDATE TO authenticated
  USING ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin())
```

```
  WITH CHECK ((EXISTS (
    SELECT 1 FROM (tasks tk JOIN assignments a ON a.id = tk.assignment_id)
    WHERE tk.id = submissions.task_id AND is_teacher_of_group(a.group_id)
  )) OR is_admin());

-- attendance / sessions
CREATE POLICY attendance_select ON public.attendance FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY attendance_teacher ON public.attendance FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());
CREATE POLICY lesson_sessions_select ON public.lesson_sessions FOR SELECT TO authenticated
USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY lesson_sessions_manage ON public.lesson_sessions FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- marks
CREATE POLICY marks_select ON public.marks FOR SELECT TO authenticated USING (
  (EXISTS (SELECT 1 FROM students s WHERE s.id = marks.student_id AND s.profile_id = auth.uid(
)))
  OR is_teacher_of_group(group_id) OR is_admin()
);
CREATE POLICY marks_teacher_manage ON public.marks FOR ALL TO authenticated
  USING (is_teacher_of_group(group_id) OR is_admin())
  WITH CHECK (is_teacher_of_group(group_id) OR is_admin());

-- announcements / templates / notifications
CREATE POLICY announcements_select ON public.announcements FOR SELECT TO authenticated USING (
  is_teacher_of_group(group_id) OR is_student_in_group(group_id) OR is_admin()
);
CREATE POLICY announcements_teacher ON public.announcements FOR ALL TO authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = announcements.teacher_id AND t.
profile_id = auth.uid())) OR is_admin());
CREATE POLICY assignment_templates_teacher ON public.assignment_templates FOR ALL TO
authenticated
  USING ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id AND t.
profile_id = auth.uid())) OR is_admin())
  WITH CHECK ((EXISTS (SELECT 1 FROM teachers t WHERE t.id = assignment_templates.teacher_id
AND t.profile_id = auth.uid())) OR is_admin());
CREATE POLICY notifications_own ON public.notifications FOR ALL TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());

-- academic
CREATE POLICY academic_years_select ON public.academic_years FOR SELECT TO authenticated USING (
true);
CREATE POLICY academic_years_admin ON public.academic_years FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());
CREATE POLICY academic_terms_select ON public.academic_terms FOR SELECT TO authenticated USING (
true);
CREATE POLICY academic_terms_admin ON public.academic_terms FOR ALL TO authenticated
  USING (is_admin()) WITH CHECK (is_admin());

-- chat
CREATE POLICY chat_rooms_select ON public.chat_rooms FOR SELECT TO authenticated
  USING (can_access_chat_room(id));
CREATE POLICY chat_rooms_admin_insert ON public.chat_rooms FOR INSERT TO authenticated
  WITH CHECK (is_admin() OR is_teacher_of_group(group_id));

CREATE POLICY chat_messages_select ON public.chat_messages FOR SELECT TO authenticated
  USING (can_access_chat_room(room_id));
CREATE POLICY chat_messages_insert ON public.chat_messages FOR INSERT TO authenticated
  WITH CHECK ((sender_id = auth.uid()) AND can_access_chat_room(room_id));
CREATE POLICY chat_messages_update ON public.chat_messages FOR UPDATE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)))
  WITH CHECK (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
CREATE POLICY chat_messages_delete ON public.chat_messages FOR DELETE TO authenticated
  USING (is_admin() OR ((sender_id = auth.uid()) AND can_access_chat_room(room_id)));
```

```
CREATE POLICY message_reads_select ON public.message_reads FOR SELECT TO authenticated USING (
  (user_id = auth.uid()) OR is_admin() OR (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  ))
);
CREATE POLICY message_reads_insert ON public.message_reads FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m WHERE m.id = message_reads.message_id AND
can_access_chat_room(m.room_id)
  )));
CREATE POLICY message_reads_delete ON public.message_reads FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());

CREATE POLICY message_reactions_select ON public.message_reactions FOR SELECT TO authenticated
USING (
  EXISTS (SELECT 1 FROM chat_messages m WHERE m.id = message_reactions.message_id AND
can_access_chat_room(m.room_id))
);
CREATE POLICY message_reactions_insert ON public.message_reactions FOR INSERT TO authenticated
  WITH CHECK ((user_id = auth.uid()) AND (EXISTS (
    SELECT 1 FROM chat_messages m
    WHERE m.id = message_reactions.message_id AND m.is_deleted = false AND can_access_chat_room(
m.room_id)
  )));
CREATE POLICY message_reactions_update ON public.message_reactions FOR UPDATE TO authenticated
  USING (user_id = auth.uid()) WITH CHECK (user_id = auth.uid());
CREATE POLICY message_reactions_delete ON public.message_reactions FOR DELETE TO authenticated
  USING ((user_id = auth.uid()) OR is_admin());
```

# Appendix B — All dependencies explained

Versions are from pubspec.yaml at generation time. Run flutter pub get after edits. audioplayers_darwin is overridden to 6.3.0 for a platform resolution pin.

| Package | Version | Why MU Study needs it |
|---|---|---|
| supabase_flutter | ^2.9.1 | Auth, PostgREST, Storage, Realtime client |
| flutter_riverpod | ^2.6.1 | State: providers, notifiers, streams |
| go_router | ^14.8.1 | Path routing + redirects |
| freezed_annotation | ^2.4.4 | Available; models are hand-written today |
| json_annotation | ^4.9.0 | Optional codegen annotations |
| shared_preferences | ^2.5.3 | Theme, locale, focus, caches |
| responsive_framework | ^1.5.1 | Breakpoints for phone vs rail |
| flutter_animate | ^4.5.2 | Small motion on cards/emoji |
| cached_network_image | ^3.4.1 | Avatar and media decode cache |
| shimmer / skeletonizer | ^3 / ^2 | Loading placeholders |
| file_picker / image_picker | ^8 / ^1 | Homework and chat attachments |
| syncfusion_flutter_pdfviewer | ^30 | In-app PDF sheets |
| syncfusion_flutter_charts | ^30 | Teacher/admin analytics |
| syncfusion_flutter_pdf | ^30 | Credential slips + progress PDF |
| intl | ^0.20.2 | Dates; used with l10n |
| url_launcher | ^6.3.1 | Open assignment link_url |
| google_fonts | ^6.2.1 | Typography for styles |
| flutter_dotenv | ^5.2.1 | Load .env when dart-define absent |
| record / audioplayers | ^6 | Voice notes and playback |
| path_provider / path / cross_file | Local temp files for recordings | |
| share_plus | ^11 | Share progress/credential PDFs |
| video_player / get_thumbnail_video | Chat video | |
| http / crypto | Downloads and hashing helpers | |
| package_info_plus | ^9 | About / version on profile |
| lottie / characters | Animated emoji + grapheme safety | |
| flutter_cache_manager | ^3.4.2 | Custom cache stores for chat |
| flutter_localizations / cupertino_icons | SDK i18n + icons | |
| build_runner / freezed / json_serializable | dev — optional codegen | |
| flutter_lints / flutter_test | dev — analyze and tests | |

## What you do not depend on

- firebase_core / firebase_messaging — not used (Tutorial 20).
- provider / bloc — Riverpod only.

- auto_route — go_router only.

# Appendix C — Video resources by topic

| Resource | URL |
|---|---|
| What is Flutter? — Flutter Official | https://www.youtube.com/watch?v=fDkQWZDu6UU |
| What is Supabase? — Supabase Channel | https://www.youtube.com/watch?v=WiwfiP2jtjY |
| Database Basics — Crash Course | https://www.youtube.com/watch?v=oCgc3fumTW0 |
| SQL for Beginners — freeCodeCamp | https://www.youtube.com/watch?v=HXV3zeQKqGY |
| PostgreSQL Tutorial — Amigoscode | https://www.youtube.com/watch?v=qw--VYLpxG4 |
| SQL for Beginners — Programming with Mosh | https://www.youtube.com/watch?v=7S_tz1z_5bA |
| Supabase Setup & Tutorial — Official | https://www.youtube.com/watch?v=I6nnp6kS3F8 |
| Supabase Full Course — Codementor | https://www.youtube.com/watch?v=rypsG5tqz8Q |
| Environment Variables — Fireship | https://www.youtube.com/watch?v=HXZpY4sUmqQ |
| flutter_dotenv package walkthrough | https://pub.dev/packages/flutter_dotenv |
| Flutter Project Structure — The Net Ninja | https://www.youtube.com/watch?v=u3R2cCP_ynI |
| Flutter for Beginners — Udacity | https://www.youtube.com/watch?v=7nAMS3cAr3c |
| Dart for Beginners — Fireship | https://www.youtube.com/watch?v=Ej_Pcr4uC2Q |
| Dart Null Safety — Dart Official | https://www.youtube.com/watch?v=IqD_wjsDMKU |
| Flutter Widgets — The Net Ninja | https://www.youtube.com/watch?v=u_rbIqnRDms |
| Flutter Layout Tutorial — Flutter Official | https://www.youtube.com/watch?v=_58-kTsPq9c |
| Material Design in Flutter — Google | https://www.youtube.com/watch?v=DL0Ix1lnK4k |
| Stateful vs Stateless — Flutter Official | https://www.youtube.com/watch?v=wE7khGHVkYY |
| Flutter State Management — Fireship | https://www.youtube.com/watch?v=lTnXr8XO0Ys |
| GoRouter — Flutter Official | https://www.youtube.com/watch?v=Y9DLpnBckso |
| Navigation in Flutter — Fireship | https://www.youtube.com/watch?v=xYPCrZ5iHzQ |
| Riverpod Tutorial — Fireship | https://www.youtube.com/watch?v=9EDFCZ0SqEA |
| Riverpod for State Management — Codepur | https://www.youtube.com/watch?v=GxXSiSkJUvY |
| Riverpod Documentation | https://riverpod.dev/ |
| Riverpod Complete Course — Tomic Riedel | https://www.youtube.com/watch?v=W_nqHaYVHjE |
| Freezed Package — Codepur | https://www.youtube.com/watch?v=PkJCU-Qm3QM |
| Dart Models & Serialization — Fireship | https://www.youtube.com/watch?v=xXCG15nVP_8 |
| Supabase Auth — Official | https://www.youtube.com/watch?v=6ow_jW7iS24 |
| Flutter Authentication — Fireship | https://www.youtube.com/watch?v=yH3NQFr3k0s |
| Supabase Database Queries — Official | https://www.youtube.com/watch?v=zVlc_O_oJgA |
| Flutter + Supabase — Fireship | https://www.youtube.com/watch?v=eKvj2P3sJVU |
| Supabase RLS — Official | https://www.youtube.com/watch?v=Ow_Utn9OIMQ |
| PostgreSQL Row-Level Security — Amigoscode | https://www.youtube.com/watch?v=2IXAHGQJ0as |
| Supabase Storage — Official | https://www.youtube.com/watch?v=s7EwCRwxo5s |
| File Upload in Flutter — Fireship | https://www.youtube.com/watch?v=KLY_2vtuWsQ |
| Supabase Real-time — Official | https://www.youtube.com/watch?v=lsKDFTL1YbQ |
| Real-time Chat Flutter — Fireship | https://www.youtube.com/watch?v=F7OX-pxjvPQ |

| Resource | URL |
|---|---|
| Flutter Localization — Official | https://www.youtube.com/watch?v=Zw3-fHMjVLw |
| i18n in Flutter — Fireship | https://www.youtube.com/watch?v=QDmKQkeSLfI |
| Flutter Theming — Official | https://www.youtube.com/watch?v=oP3asNgo_Tw |
| Dark Mode in Flutter — Fireship | https://www.youtube.com/watch?v=h3bkKqFtaO4 |
| Firebase Cloud Messaging — Flutter | https://www.youtube.com/watch?v=aIRVqk9oE94 |
| Push Notifications in Flutter — Fireship | https://www.youtube.com/watch?v=7AtVo2H3g1s |
| Flutter Testing — Official | https://www.youtube.com/watch?v=X8uCMbpnb5s |
| Flutter Performance — Official | https://www.youtube.com/watch?v=pk05Vmtfl5g |
| Flutter Release Builds — Official | https://www.youtube.com/watch?v=Yx1kSisJo4k |
| HTTP Crash Course — Traversy | https://www.youtube.com/watch?v=iYM2zFP3Zn0 |
| JSON Crash Course — Traversy | https://www.youtube.com/watch?v=iiADhChRriM |
| JWT explained — Fireship | https://www.youtube.com/watch?v=7Q17ubqLfaM |
| Git & GitHub — freeCodeCamp | https://www.youtube.com/watch?v=RGOj5yH7evk |

Videos teach the ecosystem. This book teaches MU Study. If a video shows email signup or Firebase, translate the idea; do not copy the architecture.

# Appendix D — Troubleshooting guide

| Symptom | Likely why | Diagnose | Fixed when |
|---|---|---|---|
| SetupRequiredScreen | Key empty/sanitized | Print Env after load; check eyJ/sb_publishable_ | Splash or login appears |
| Login always incorrect | RPC null, bad password, unconfirmed email | SELECT auth_email, is_disabled; Auth settings confirm email | Session exists |
| Bounces to login after sign-in | No profiles row | SELECT * FROM profiles WHERE id = auth user id | Profile loads |
| Stuck on /boot | Profile Future hanging | Network + RLS on profiles_select | Redirect to language or home |
| Stuck on /language | preferred_language null forever | Update failed RLS profiles_update_self | Column set; home opens |
| Empty assignment list | Wrong group or RLS | group_students membership; teacher_id | Rows appear |
| Save mark does nothing | UPDATE without SELECT policy | Add marks SELECT for teacher | Row percentage changes |
| Storage 403 | Path folders / policy | roomId and uid segments | Upload 200 |
| Realtime silent | Not in publication | supabase_realtime tables | Live insert shows |
| Chat under nav bar | extendBody / padding | shellContentBottomPadding | Timestamp visible |
| gen-l10n fails | Key missing in ru/uz | Diff ARB keys | flutter gen-l10n exits 0 |
| Nature Paper looks empty | Assets not copied | pubspec assets/images/nature-paper | Backdrop paints |
| Mic permission crash | Manifest/plist | RECORD_AUDIO + usage string | Record starts |
| Duplicate group title | unique (teacher_id, title) | Pick another title | Insert succeeds |
| Teacher create fails | sphere null | Pass math or english | teachers row exists |
| Student sees all marks | RLS off or service_role in app | Remove god key; enable RLS | Only own rows |
| Demo RPC callable | EXECUTE granted to anon | REVOKE from anon, authenticated | Permission denied |

## FAQ

**Can students register?** No. Admin creates accounts.

**Can I use Google login?** Not in this product. Username + password only.

**Where is Arabic?** Not shipped. Add a new ARB only as a new product decision.

**Where is FCM?** Not shipped. Inbox + Realtime is the notification system.

**Why @ in usernames?** Cosmetic. Email strips @ before @mustudy.internal.

# Appendix E — Architecture diagrams

## Data flow

**ascii**

```
 Widget  →  Riverpod  →  supabase.from / rpc / storage / channel
                                ↓
                        JWT + anon key
                                ↓
                  PostgREST / Auth / Storage / Realtime
                                ↓
                  PostgreSQL RLS + triggers + views
```

## Widget hierarchy (role session)

**ascii**

```
 MaterialApp.router
   ResponsiveBreakpoints
     NaturePaperBackdrop
       SoftNeumorphismBackdrop
         ShellRoute (AppShell + nav)
           Feature screen (dashboard, chat, …)
             Components (cards, lists)
```

## Provider dependencies (core)

**ascii**

```
 authStateProvider --> currentProfileProvider --> teacherProfileProvider
                   --> currentProfileProvider --> studentProfileProvider
                   --> routerProvider (refreshListenable)
 currentProfileProvider --> locale/theme applyFromProfile
 myGroupsProvider --> activeGroupId --> assignments / attendance / announcements
 chatRoomsProvider --> unreadChatCountProvider (shell badge)
 notificationsProvider --> unreadNotificationsCountProvider
```

## Database relationship sketch

**ascii**

```
 auth.users 1■■1 profiles 1■■0..1 teachers 1■■* groups 1■■1 chat_rooms
                 ■■0..1 students *■■* groups (group_students)
 groups 1■■* assignments 1■■* tasks 1■■* submissions
 groups 1■■* lesson_sessions 1■■* attendance
 groups 1■■* marks  (optional submissions.submission_id)
 groups 1■■* announcements     profiles 1■■* notifications
 chat_rooms 1■■* chat_messages 1■■* reads/reactions
 academic_years 1■■* academic_terms
```

# Appendix F — Code style guide

## Naming

- Files: snake_case.dart matching the main type (login_screen.dart).
- Widgets: PascalCase, private State with leading underscore.
- Providers: camelCase + Provider suffix (currentProfileProvider).
- JSON keys: snake_case matching Postgres.
- RPCs: snake_case matching SQL (get_auth_email_by_username).

## File organization

- Screens do not embed raw SQL strings; they call providers.
- Providers do not build widgets.
- Models do not import Flutter.
- Shared chat/profile live outside role folders.

## Comments

Comment why, not what. The router's note about /boot and semantics.parentDataDirty is the right kind of comment. Do not comment // increment counter.

## Best practices in this repo

1. maybeSingle when zero rows are valid.
2. if (!mounted) return after await in State.
3. l10n for every user-visible string.
4. Dispose channels and controllers.
5. Never log passwords or full JWTs.

# Appendix M — Full Column Catalog (Every Table)

Inspected from information_schema on the live MU Study Supabase database. Recreate every column with these nullability and default rules.

## Table public.academic_terms

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| academic_year_id | uuid | NO | — | FK → academic_years.id CASCADE |
| semester | int4 | NO | — | 1–4 |
| starts_on | date | NO | — | |
| ends_on | date | NO | — | |

## Table public.academic_years

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| start_year | int4 | NO | — | UNIQUE |
| label | text | NO | — | |
| is_current | bool | NO | false | partial unique one current |
| created_at | timestamptz | NO | now() | |
| created_by | uuid | YES | — | |

## Table public.announcements

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| teacher_id | uuid | YES | — | FK teachers CASCADE; null for admin |
| group_id | uuid | NO | — | FK groups CASCADE |
| title | text | NO | — | |
| description | text | NO | '' | |
| image_urls | jsonb | NO | [] | |
| file_urls | jsonb | NO | [] | |
| created_at | timestamptz | NO | now() | |

## Table public.assignment_templates

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| teacher_id | uuid | NO | — | FK teachers CASCADE |
| type | assignment_type | NO | static | |
| title | text | NO | — | |
| description | text | NO | '' | |
| tasks | jsonb | NO | [] | task drafts |
| created_at | timestamptz | NO | now() | |
| updated_at | timestamptz | NO | now() | |

## Table public.assignments

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| teacher_id | uuid | NO | — | FK teachers CASCADE |
| group_id | uuid | NO | — | FK groups CASCADE |
| type | assignment_type | NO | static | static\|homework_pack |
| title | text | NO | — | |
| description | text | NO | '' | |
| due_date | timestamptz | YES | — | |
| created_at | timestamptz | NO | now() | |

## Table public.attendance

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| group_id | uuid | NO | — | FK groups CASCADE |
| student_id | uuid | NO | — | FK students CASCADE |
| status | attendance_status | NO | — | |
| date | date | NO | — | UNIQUE with group+student |
| created_at | timestamptz | NO | now() | |
| session_id | uuid | NO | — | FK lesson_sessions CASCADE |

## Table public.chat_messages

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| room_id | uuid | NO | — | FK chat_rooms CASCADE |
| sender_id | uuid | NO | — | FK profiles CASCADE |
| message_type | chat_message_type | NO | text | |
| text_content | text | YES | — | |
| media_url | text | YES | — | signed URL |
| created_at | timestamptz | NO | now() | |
| updated_at | timestamptz | NO | now() | |
| is_deleted | bool | NO | false | soft delete |
| reply_to_message_id | uuid | YES | — | FK self SET NULL |

## Table public.chat_rooms

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| group_id | uuid | NO | — | UNIQUE FK groups CASCADE |
| created_at | timestamptz | NO | now() | |

## Table public.group_students

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| group_id | uuid | NO | — | FK groups CASCADE |
| student_id | uuid | NO | — | FK students CASCADE |
| created_at | timestamptz | NO | now() | |

## Table public.groups

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| teacher_id | uuid | NO | — | FK teachers CASCADE |
| title | text | NO | — | UNIQUE with teacher_id |
| created_at | timestamptz | NO | now() | |

## Table public.lesson_sessions

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| group_id | uuid | NO | — | FK groups CASCADE |
| session_date | date | NO | — | UNIQUE with group_id |
| created_at | timestamptz | NO | now() | |

## Table public.marks

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| group_id | uuid | NO | — | FK groups CASCADE |
| student_id | uuid | NO | — | FK students CASCADE |
| teacher_id | uuid | NO | — | FK teachers CASCADE |
| submission_id | uuid | YES | — | UNIQUE FK submissions CASCADE |
| title | text | NO | — | |
| category | mark_category | NO | other | |
| percentage | numeric | NO | — | |
| weight | numeric | NO | 1 | |
| comment | text | NO | '' | |
| marked_on | date | NO | CURRENT_DATE | |
| created_at | timestamptz | NO | now() | |
| updated_at | timestamptz | NO | now() | |

## Table public.message_reactions

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| message_id | uuid | NO | — | FK chat_messages CASCADE |
| user_id | uuid | NO | — | FK profiles CASCADE |
| emoji | text | NO | — | |
| created_at | timestamptz | NO | now() | |

## Table public.message_reads

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| message_id | uuid | NO | — | FK chat_messages CASCADE |
| user_id | uuid | NO | — | FK profiles CASCADE |
| read_at | timestamptz | NO | now() | |

## Table public.notifications

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| user_id | uuid | NO | — | FK profiles CASCADE |
| title | text | NO | — | |
| body | text | NO | '' | |
| is_read | bool | NO | false | |
| created_at | timestamptz | NO | now() | |
| kind | text | NO | system | |
| action_path | text | YES | — | deep link |
| ref_id | uuid | YES | — | |

## Table public.profiles

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | — | PK = auth.users.id |
| username | text | NO | — | UNIQUE |
| full_name | text | NO | '' | |
| role | user_role | NO | — | |
| auth_email | text | NO | — | UNIQUE synthetic |
| avatar_url | text | YES | — | |
| is_disabled | bool | NO | false | |
| created_at | timestamptz | NO | now() | |
| preferred_language | text | YES | — | en\|ru\|uz |
| preferred_theme | text | YES | — | light\|dark\|system |
| preferred_style | text | YES | — | AppStyle name |

## Table public.students

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| profile_id | uuid | NO | — | UNIQUE FK profiles CASCADE |
| created_at | timestamptz | NO | now() | |

## Table public.submissions

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| task_id | uuid | NO | — | FK tasks CASCADE |
| student_id | uuid | NO | — | FK students CASCADE |
| text_answer | text | YES | — | |
| file_url | text | YES | — | |
| status | submission_status | NO | pending | |
| score | numeric | YES | — | |
| feedback | text | YES | — | |
| is_completed | bool | NO | false | |
| submitted_at | timestamptz | YES | — | |
| voice_url | text | YES | — | |

## Table public.tasks

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| assignment_id | uuid | NO | — | FK assignments CASCADE |
| title | text | NO | — | |
| description | text | NO | '' | |
| attachment_url | text | YES | — | |
| link_url | text | YES | — | |
| sort_order | int4 | NO | 0 | |
| created_at | timestamptz | NO | now() | |

## Table public.teachers

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| id | uuid | NO | gen_random_uuid() | PK |
| profile_id | uuid | NO | — | UNIQUE FK profiles CASCADE |
| sphere | sphere_type | NO | — | math\|english |
| created_at | timestamptz | NO | now() | |

## Views (not base tables)

- chat_room_summaries — see Appendix C
- student_mark_averages — student_id, average_percent, marks_count, last_marked_on
- student_group_mark_averages — + group_id

> **TASK — Column audit**
> After creating tables, run the Chapter 34 column query and diff against this appendix. Any missing column is a defect.

# Appendix H — ARB localization catalogs

## app_en.arb

**lib/l10n/app_en.arb**

```
{
  "@@locale": "en",
  "appName": "MU Study",
  "shellMuStudy": "MU Study",

  "languageTitle": "Choose your language",
  "languageSubtitle": "You can change this anytime in Profile.",
  "languageEnglish": "English",
  "languageRussian": "Russian",
  "languageUzbek": "Uzbek",
  "languageEnglishNative": "English",
  "languageRussianNative": "[illegible]",
  "languageUzbekNative": "O‘zbekcha",
  "languageContinue": "Continue",
  "languageSection": "Language",
  "languageHint": "App language",

  "loginWelcome": "Welcome back",
  "loginSubtitle": "Sign in to your MU Study workspace",
  "loginUsername": "Username",
  "loginPassword": "Password",
  "loginSignIn": "Sign in",
  "loginMissingFields": "Please enter both username and password",
  "loginIncorrect": "Incorrect username or password",
  "loginFailed": "Sign in failed. Check your connection and try again.",
  "loginConfigError": "Invalid configuration. Contact support.",
  "loginConfigMissing": "Configuration missing. Check your .env file.",
  "accountDisabled": "Account disabled. Contact admin.",

  "navDashboard": "Dashboard",
  "navAssignments": "Assignments",
  "navMarks": "Marks",
  "navAttendance": "Attendance",
  "navNotifications": "Notifications",
  "navProfile": "Profile",
  "navUsers": "Users",
  "navGroups": "Groups",
  "navAnalytics": "Analytics",
  "navGradebook": "Gradebook",
  "navChat": "Chat",
  "navFocus": "Focus",

  "focusTitle": "Focus",
  "focusSubtitle": "Quiet study time for one assignment at a time.",
  "focusTodaysFocus": "Today’s focus",
  "focusTodaysFocusSubtitle": "Up to three open assignments — any due date",
  "focusAlsoDue": "More homework",
  "focusContinueHomework": "Continue homework",
  "focusOpenAssignment": "Open assignment",
  "focusEmptyTitle": "No homework yet",
  "focusEmptySubtitle": "When your teacher posts an assignment, you can start a focus session
here.",
  "focusChooseMode": "How do you want to focus?",
  "focusModePomodoro": "Pomodoro",
  "focusModePomodoroHint": "25 minutes, then a break",
  "focusModeFree": "Free session",
  "focusModeFreeHint": "Count up until you stop",
  "focusDoNotDisturb": "Chat muted",
  "focusChatMuteHint": "While a session is active, chat badges stay quiet so you can focus.
Notifications still arrive in the background.",
  "focusPause": "Pause",
  "focusResume": "Resume",
  "focusPaused": "Paused",
  "focusEndSession": "End",
  "focusSessionComplete": "Session complete",
```

```
"focusSessionCompleteSubtitle": "Nice work. Here’s a quick recap.",
"focusTimeUp": "Time’s up!",
"focusTimeUpSubtitle": "Your Pomodoro block finished. Take a short break, then keep going.",
"focusMinutesFocused": "{duration} focused",
"@focusMinutesFocused": {
  "placeholders": {
    "duration": { "type": "String" }
  }
},
"focusOptionalNote": "Optional note",
"focusOptionalNoteHint": "e.g. Stuck on task #3",
"focusSaveAndOpen": "Save & open assignment",
"focusDone": "Done",

"chatTitle": "Chat",
"chatSearchHint": "Search groups or teachers",
"chatEmptyTitle": "No chat rooms yet",
"chatEmptySubtitle": "Join a group to start messaging your class.",
"chatNoMessagesYet": "No messages yet",
"chatMessageDeleted": "Message deleted",
"chatPreviewImage": "Photo",
"chatPreviewVideo": "Video",
"chatVideoPlaybackSpeed": "Playback speed",
"chatVideoSpeedNormal": "1x",
"chatVideoSpeedValue": "{speed}x",
"@chatVideoSpeedValue": {
  "placeholders": {
    "speed": { "type": "double", "format": "decimalPattern" }
  }
},
"chatVideoRewind10": "-10s",
"chatVideoForward10": "+10s",
"chatVideoCouldNotPlay": "Could not play this video",
"chatPreviewVoice": "Voice message",
"chatPreviewFile": "File",
"chatOpenFile": "Tap to open",
"chatOpenFileFailed": "Could not open this file.",
"chatYou": "You",
"chatTeacherMembers": "{teacher} · {count} members",
"@chatTeacherMembers": {
  "placeholders": {
    "teacher": { "type": "String" },
    "count": { "type": "int" }
  }
},
"chatOnlineCount": "{count} online",
"@chatOnlineCount": {
  "placeholders": {
    "count": { "type": "int" }
  }
},
"chatReact": "React",
"chatCopyText": "Copy text",
"chatCopied": "Copied",
"chatReply": "Reply",
"chatReplyTo": "In reply to {name}",
"@chatReplyTo": {
  "placeholders": {
    "name": { "type": "String" }
  }
},
"chatReplyDeleted": "Deleted message",
"chatReplyNotFound": "Original message isn’t available",
"chatMessageHint": "Message…",
"chatAttachImage": "Image",
"chatAttachVideo": "Video",
"chatAttachFile": "File",
"chatTyping": "{names} typing…",
"@chatTyping": { "placeholders": { "names": { "type": "String" } } },
"chatRecording": "Recording {time}",
"@chatRecording": { "placeholders": { "time": { "type": "String" } } },
"chatUploading": "Uploading {name}…",
"@chatUploading": { "placeholders": { "name": { "type": "String" } } },
```

```
"chatSendFailed": "Could not send: {error}",
"@chatSendFailed": { "placeholders": { "error": { "type": "String" } } },
"chatMicDenied": "Microphone permission is required for voice messages.",
"chatEditTitle": "Edit message",
"chatDeleteTitle": "Delete message?",
"chatDeleteBody": "This removes the message for everyone in the room.",
"chatDeleteSelectedTitle": "Delete {count} messages?",
"@chatDeleteSelectedTitle": { "placeholders": { "count": { "type": "int" } } },
"chatDeleteSelectedBody": "Selected messages will be removed for everyone in the room.",
"chatSelectedCount": "{count} selected",
"@chatSelectedCount": { "placeholders": { "count": { "type": "int" } } },
"chatSelectMessages": "Select messages",
"chatSelect": "Select",
"commonSelectAll": "Select all",
"chatEdited": "edited",
"chatDateToday": "Today",
"chatDateYesterday": "Yesterday",
"chatRoleAdmin": "admin",
"chatRoleTeacher": "teacher",

"commonSave": "Save",
"commonCancel": "Cancel",
"commonSubmit": "Submit",
"commonDelete": "Delete",
"commonEdit": "Edit",
"commonClose": "Close",
"commonRetry": "Retry",
"commonTryAgain": "Try again",
"commonLoading": "Loading…",
"commonError": "Something went wrong",
"commonErrorWithDetails": "Error: {error}",
"@commonErrorWithDetails": { "placeholders": { "error": { "type": "String" } } },
"commonCouldNotSave": "Could not save: {error}",
"@commonCouldNotSave": { "placeholders": { "error": { "type": "String" } } },
"commonSignOut": "Sign out",
"commonSaveDraft": "Save draft",
"commonOpen": "Open",
"commonCreate": "Create",
"commonAssign": "Assign",
"commonReset": "Reset",
"commonAdd": "Add",
"commonLoadMore": "Load more",
"commonExport": "Export",
"commonExportFailed": "Could not export: {error}",
"@commonExportFailed": { "placeholders": { "error": { "type": "String" } } },
"marksTermAll": "All time",
"marksTermSemester": "{yearRange} · Semester {semester}",
"@marksTermSemester": {
  "placeholders": {
    "yearRange": { "type": "String" },
    "semester": { "type": "int" }
  }
},
"marksExportCsv": "Export CSV",
"marksExportDone": "Marks exported",
"commonTeacher": "Teacher",
"commonStudent": "Student",
"commonAdmin": "Administrator",
"commonWorkspace": "Workspace",
"commonNoDueDate": "No due date",
"commonDueDate": "Due {date}",
"@commonDueDate": { "placeholders": { "date": { "type": "String" } } },
"homeworkDueThisWeek": "Due this week",
"homeworkDueThisWeekSubtitle": "Overdue and upcoming deadlines",
"homeworkNoDueThisWeek": "Nothing due this week",
"homeworkNoDueThisWeekSubtitle": "You’re clear until next week’s deadlines.",
"homeworkOverdue": "Overdue · {date}",
"@homeworkOverdue": { "placeholders": { "date": { "type": "String" } } },
"homeworkDueToday": "Due today",
"homeworkDueTomorrow": "Due tomorrow",
"homeworkDueInDays": "Due in {days} days",
"@homeworkDueInDays": { "placeholders": { "days": { "type": "int" } } },
"homeworkLateBadge": "Late",
```

```
  "commonProgress": "Progress",
  "commonSubject": "Subject",
  "commonType": "Type",
  "commonTitle": "Title",
  "commonDescription": "Description",
  "commonGroup": "Group",
  "commonMessage": "Message",
  "commonScore": "Score",
  "commonFeedback": "Feedback",
  "commonStudents": "Students",
  "commonCannotUndo": "This action cannot be undone.",

  "profileTitle": "Profile",
  "profileFullName": "Full name",
  "profileNotSignedIn": "Not signed in",
  "profileAccount": "Account",
  "profileAccountSettings": "Account Settings",
  "profileSignOutAdmin": "Sign out of Admin Console",
  "profileSettings": "Settings",
  "profileAppearance": "Appearance",
  "profileThemeLight": "Light",
  "profileThemeDark": "Dark",
  "profileThemeSystem": "System",
  "profileStyle": "Style",
  "profileStyleModern": "Modern Professional",
  "profileStyleNature": "Nature Paper",
  "profileStyleSoftNeu": "Neumorphism",
  "profileStyleMonochrome": "Monochrome Workspace",
  "profileEditName": "Edit name",
  "profileNameUpdated": "Name updated",
  "profileChangePhoto": "Change photo",
  "profilePhotoUpdated": "Photo updated",
  "profileChangePassword": "Change password",
  "profileCurrentPassword": "Current password",
  "profileNewPassword": "New password",
  "profileConfirmPassword": "Confirm new password",
  "profilePasswordChanged": "Password updated",
  "profilePasswordMismatch": "New passwords do not match",
  "profilePasswordTooShort": "Use at least 8 characters",
  "profilePasswordHint": "Ask an admin if you forget your password.",
  "profileShortcuts": "Shortcuts",
  "profileYourGroups": "Your groups",
  "profileNoGroups": "No groups assigned",
  "profileGroupsCount": "{count} groups",
  "@profileGroupsCount": { "placeholders": { "count": { "type": "int" } } },
  "profileAverageLabel": "Term average",
  "profileAttendanceLabel": "Attendance",
  "profileShareProgress": "Share progress report",
  "profileNotifications": "Notifications",
  "profileClearCache": "Clear cached data",
  "profileClearCacheWarn1Title": "Clear cached data?",
  "profileClearCacheWarn1Body": "This deletes offline chat messages and downloaded images,
videos, voice notes, and files. You will need an internet connection to load them again.",
  "profileClearCacheWarn2Title": "Are you sure?",
  "profileClearCacheWarn2Body": "This cannot be undone. Cached chat media will be removed from
this device.",
  "profileClearCacheConfirm": "Clear cache",
  "profileClearCacheDone": "Cached data cleared",
  "profileAppVersion": "MU Study · v{version}",
  "@profileAppVersion": { "placeholders": { "version": { "type": "String" } } },
  "profileSubject": "Subject",

  "greetingMorning": "Good Morning",
  "greetingAfternoon": "Good Afternoon",
  "greetingEvening": "Good Evening",
  "greetingWithName": "{greeting}, {name}",
  "@greetingWithName": {
    "placeholders": {
      "greeting": { "type": "String" },
      "name": { "type": "String" }
    }
  },
  "greetingLoadingTitle": "Good Morning, User",
```

```
"greetingLoadingSubtitle": "Loading your personalized greeting...",
"studentHomeSubtitle": "Your learning space",
"continueLearning": "Continue Learning",
"continueLearningSubtitle": "Pick up where you left off",
"everythingCompleted": "Everything is completed.",
"caughtUpToday": "You're all caught up for today.",
"homeworkPack": "Homework Pack",
"staticTask": "Static Task",
"staticAssignment": "Static",
"upcomingAssignments": "Upcoming Assignments",
"stayOnTrack": "Stay on track",
"loadingAnnouncements": "Loading announcements...",
"notificationsCardTitle": "Notifications",
"unreadAlerts": "You have {count} new alerts",
"@unreadAlerts": { "placeholders": { "count": { "type": "int" } } },
"upToDate": "You are up to date",

"announcementsTitle": "Announcements",
"announcementsSubtitle": "From your teachers",
"announcementsEmpty": "No announcements",
"announcementsEmptyTeacher": "Share important updates with your students.",
"announcementsEmptyAdmin": "Platform-wide announcements will appear here.",
"announcementsReadMore": "Read more",
"announcementsShowLess": "Show less",
"announcementsViewAll": "View all ({count})",
"@announcementsViewAll": {
  "placeholders": {
    "count": { "type": "int" }
  }
},
"announcementsAll": "All Announcements",
"announcementsNew": "New Announcement",
"announcementsPost": "Post Announcement",
"announcementsEdit": "Edit Announcement",
"announcementsDeleteTitle": "Delete announcement?",
"announcementsDeleteBody": "It will be removed for everyone in the group.",
"announcementsByAdmin": "Administration",
"announcementsShareUpdates": "Share updates with your groups",

"assignmentsTitle": "Assignments",
"assignmentsEmpty": "No assignments",
"assignmentsEmptySubtitle": "You are all caught up!",
"assignmentsEmptyTeacher": "Create your first assignment to get started.",
"assignmentsLearningPath": "Learning Path",
"assignmentsActive": "Active assignments",
"assignmentDetails": "Assignment Details",
"assignmentNotFound": "Assignment not found",
"assignmentTasks": "Tasks",
"assignmentTasksSubtitle": "Complete all to finish pack",
"assignmentNew": "New Assignment",
"assignmentManage": "Manage Assignments",
"assignmentManageSubtitle": "Create static or homework packs",

"marksTitle": "My Marks",
"marksOverallAverage": "Overall average",
"marksNoneYet": "No marks recorded yet.",
"marksBasedOn": "{label} · based on {count} marks.",
"@marksBasedOn": {
  "placeholders": {
    "label": { "type": "String" },
    "count": { "type": "int" }
  }
},
"marksByCategory": "By category",
"marksWhereStrongest": "Where you are strongest",
"marksAll": "All marks",
"marksNewestFirst": "Newest first",
"marksEmptyTitle": "No marks yet",
"marksEmptySubtitle": "Your teacher's marks will appear here.",
"progressReportTitle": "Progress report",
"progressReportShare": "Share progress",
"progressReportSharing": "Preparing report…",
```

**lib/l10n/app_en.arb — continued (2/3)**

```
  "progressReportDone": "Progress report ready to share",
  "progressReportFailed": "Could not share progress: {error}",
  "@progressReportFailed": {
    "placeholders": {
      "error": { "type": "String" }
    }
  },
  "progressReportNeedGroup": "Join a group to share a progress report.",
  "progressReportAttendance": "Attendance",
  "progressReportAttendanceRate": "Attendance rate",
  "progressReportNoAttendance": "No attendance recorded for this term.",
  "progressReportSessions": "sessions",
  "progressReportMarksCounted": "marks",
  "progressReportRecentMarks": "Recent marks",
  "progressReportGeneratedOn": "Generated {date}",
  "@progressReportGeneratedOn": {
    "placeholders": {
      "date": { "type": "String" }
    }
  },
  "marksNoRecorded": "No marks recorded",
  "marksNoRecordedSubtitle": "Use \"Grade class\" to enter marks in one pass.",
  "marksRecent": "Recent marks",
  "marksRecentSubtitle": "Latest entries across the group",
  "marksStudentsHeader": "Students",
  "marksStudentsSubtitle": "Weighted average of all marks",
  "marksNoStudents": "No students in this group",
  "marksNoStudentsSubtitle": "An admin can assign students to your group.",
  "marksDeleteTitle": "Delete mark?",
  "marksDeleteBody": "\"{title}\" will be removed permanently.",
  "@marksDeleteBody": { "placeholders": { "title": { "type": "String" } } },
  "marksGradeTheClass": "Grade the class",
  "marksOnePercentEach": "One percent mark per student",
  "marksSaveMarks": "Save marks",
  "marksSaveMark": "Save mark",
  "marksAddMark": "Add mark",
  "marksNothingYet": "Nothing recorded yet",
  "marksAddFirst": "Add the first mark for this student.",
  "marksSavedCount": "Saved {count} marks",
  "@marksSavedCount": { "placeholders": { "count": { "type": "int" } } },
  "marksGroupHasNoStudents": "This group has no students yet.",
  "marksTitleHint": "e.g. Unit 3 test",
  "marksAverageLine": "Average {percent} · {label}",
  "@marksAverageLine": {
    "placeholders": {
      "percent": { "type": "String" },
      "label": { "type": "String" }
    }
  },
  "marksNoMarksYetShort": "No marks yet",
  "marksSummaryStudents": "Students",
  "marksSummaryMarks": "Marks",

  "attendanceTitle": "Attendance",
  "attendanceEmpty": "No attendance records",
  "attendanceNoGroup": "No group assigned yet",
  "attendanceMark": "Mark Attendance",
  "attendanceDailyTracking": "Daily student tracking",
  "attendanceSaved": "Attendance saved successfully",
  "attendanceNoGroups": "No groups found",
  "attendancePresent": "P — Present",
  "attendanceLate": "L — Late",
  "attendanceAbsent": "A — Absent",
  "attendanceExcused": "E — Excused",
  "attendancePresentShort": "P",
  "attendanceLateShort": "L",
  "attendanceAbsentShort": "A",
  "attendanceExcusedShort": "E",
  "attendanceTrend": "Attendance Trend",
  "attendanceOverall": "Overall distribution",
  "analyticsThisWeek": "This Week",
```

```
"analyticsLastWeek": "Last Week",
"analyticsTotalMarks": "Total Marks: {count}",
"@analyticsTotalMarks": {
  "placeholders": {
    "count": { "type": "int" }
  }
},

"notificationsTitle": "Notifications",
"notificationsEmpty": "All caught up!",
"notificationsEmptySubtitle": "No new notifications.",
"notificationsNone": "No notifications",
"notifKindAssignment": "New homework",
"notifKindAnnouncement": "Announcement",
"notifKindGraded": "Work graded",
"notifKindSubmission": "New submission",
"notifKindChat": "Chat message",
"notifKindSystem": "Update",
"notificationsOpen": "Open",

"taskTitle": "Task",
"taskYourAnswer": "Your answer",
"taskMarkCompleted": "Mark completed",
"taskAttachment": "Attachment",
"taskAttachmentHint": "Add a photo or document to your answer.",
"taskAttachmentHintReady": "Tap to open · trash to remove.",
"taskImage": "Image",
"taskImageSubtitle": "Gallery photo",
"taskFile": "File",
"taskFileSubtitle": "PDF or document",
"taskReplaceImage": "Replace with image",
"taskReplaceFile": "Replace file",
"taskVoice": "Voice record",
"taskVoiceHint": "Optional spoken answer — record, play back, then save.",
"taskViewPdf": "View PDF",
"taskOpenLink": "Open link",
"taskSubmitted": "Submitted",
"taskSaved": "Saved",
"taskUploadFailed": "Upload failed: {error}",
"@taskUploadFailed": { "placeholders": { "error": { "type": "String" } } },
"taskVoiceUploadFailed": "Voice upload failed: {error}",
"@taskVoiceUploadFailed": { "placeholders": { "error": { "type": "String" } } },
"taskCouldNotSave": "Could not save: {error}",
"@taskCouldNotSave": { "placeholders": { "error": { "type": "String" } } },

"voiceTapToRecord": "Tap the mic to record your answer",
"voiceRecording": "Recording…",
"voiceNote": "Voice note",
"voiceTapPlay": "Tap play to listen",
"voiceRerecord": "Re-record",
"voiceStudentAnswer": "Student voice answer",
"voiceMicPermission": "Microphone permission is required to record.",
"voiceCouldNotStart": "Could not start recording: {error}",
"@voiceCouldNotStart": { "placeholders": { "error": { "type": "String" } } },
"voiceCouldNotSave": "Could not save recording: {error}",
"@voiceCouldNotSave": { "placeholders": { "error": { "type": "String" } } },
"voiceCouldNotPlay": "Could not play voice note: {error}",
"@voiceCouldNotPlay": { "placeholders": { "error": { "type": "String" } } },
"attachmentCouldNotOpen": "Could not open this file.",
"attachmentImage": "Image attachment",
"attachmentFile": "File attachment",
"attachmentPdf": "PDF document",
"attachmentTapPreview": "Tap to preview",
"attachmentTapOpen": "Tap to open",
"pdfDefaultTitle": "PDF",

"teacherWorkspace": "Teacher Workspace",
"teacherQuickActions": "Quick Actions",
"teacherCreateItems": "Create new items",
"teacherNewAssignment": "New Assignment",
"teacherNewAnnouncement": "New Announcement",
"teacherSubmissions": "Submissions",
"teacherNoSubmissions": "No submissions yet",
```

```
"teacherNoSubmissionsSubtitle": "Student work will show up here for grading.",
"teacherGradebook": "Gradebook",
"teacherGradeClass": "Grade class",
"teacherNoGroups": "No groups yet",
"teacherNoGroupsSubtitle": "Ask an admin to assign you a group.",
"teacherActiveGroups": "ACTIVE GROUPS",
"teacherSubject": "SUBJECT",
"teacherDailyTasks": "Daily Tasks",
"teacherNeedsAttention": "What needs your attention",
"teacherReviewSubmissions": "Review Submissions",
"teacherCheckAssignments": "Check student assignments",
"teacherRecordMarks": "Record marks in percent",
"teacherPerformanceAnalytics": "Performance Analytics",
"teacherTrackProgress": "Track group progress",
"teacherOpenAttachment": "Open attachment",
"teacherScoreHelper": "Adds a homework mark to the gradebook",
"teacherScoreRange": "Score must be between 0 and 100.",
"teacherNoWrittenAnswer": "No written answer.",
"teacherReviewSubmission": "Review submission",
"teacherHomeworkPackTasks": "Homework Pack Tasks",
"teacherAddTask": "Add Task",
"teacherEditTask": "Edit Task",
"teacherDeleteTask": "Delete Task?",
"teacherTasksContent": "Tasks Content",
"teacherManagePack": "Manage items in this pack",
"teacherTaskTitleHint": "e.g. Reading Exercise",
"teacherLinkHint": "https://...",
"teacherAttachmentHint": "Image or file link...",
"teacherNoData": "No data available",
"teacherMarkDistribution": "Mark Distribution",
"teacherStudentsByAverage": "Students grouped by average",
"teacherNoMarksTrends": "Add marks in the gradebook to see trends.",
"teacherTargetGroup": "Target Group",

"adminWorkspace": "Workspace Control",
"adminUsers": "User Management",
"adminGroups": "Groups",
"adminAnalytics": "Analytics",
"adminTeachers": "TEACHERS",
"adminStudents": "STUDENTS",
"adminGroupsLabel": "GROUPS",
"adminMarksLabel": "MARKS",
"adminActivityInsights": "Activity Insights",
"adminCurrentMetrics": "Current metrics",
"adminCreateUser": "Create New User",
"adminCreateUserButton": "Create User",
"adminUserCreated": "User created successfully",
"adminResetPassword": "Reset password",
"adminResetPasswordTitle": "Reset Password · {username}",
"@adminResetPasswordTitle": { "placeholders": { "username": { "type": "String" } } },
"adminCreateGroup": "Create New Group",
"adminCreateGroupButton": "Create Group",
"adminGroupTitleHint": "e.g. Group 4",
"adminGroupTitleExists": "This teacher already has a group with that name",
"adminAssignStudent": "Assign Student",
"adminActivityDistribution": "Activity Distribution",
"adminGlobalMetrics": "Global platform metrics",
"adminActivityLog": "Admin Activity Log",
"adminActivityLogSubtitle": "System-wide events and audit logs appear here.",
"adminTeacherLine": "Teacher: {name}",
"@adminTeacherLine": { "placeholders": { "name": { "type": "String" } } },
"adminRoleLine": "{role} · {username}",
"@adminRoleLine": {
  "placeholders": {
    "role": { "type": "String" },
    "username": { "type": "String" }
  }
},
"adminDisable": "Disable",
"adminEnable": "Enable",
"adminPassword": "Password",
"adminFullName": "Full name",
"adminRole": "Role",
```

```
  "adminSphere": "Sphere",

  "percentExcellent": "Excellent",
  "percentGood": "Good",
  "percentNeedsWork": "Needs work",
  "percentAtRisk": "At risk",
  "emptyStateDefault": "Nothing here yet",
  "setupRequiredTitle": "Setup required",
  "setupRequiredBody": "Supabase is not configured. Add SUPABASE_URL and SUPABASE_ANON_KEY to
your .env file.",
  "setupOption1": "Option 1 — local .env file",
  "setupOption2": "Option 2 — dart-define",

  "attendanceAllPresent": "All P",
  "attendanceAllAbsent": "All A",
  "attendanceAllLate": "All L",
  "attendanceAllExcused": "All E",
  "attendancePresentWord": "Present",
  "attendanceAbsentWord": "Absent",
  "attendanceLateWord": "Late",
  "attendanceExcusedWord": "Excused",
  "attendanceWeekdayMon": "Mon",
  "attendanceWeekdayTue": "Tue",
  "attendanceWeekdayWed": "Wed",
  "attendanceWeekdayThu": "Thu",
  "attendanceWeekdayFri": "Fri",
  "attendancePercentLabel": "Rate",
  "attendanceNoLesson": "No lesson",
  "attendanceHistory": "Attendance history",
  "attendanceSelectStudent": "Student",
  "attendanceAllStudents": "All students",
  "attendanceTapToCycle": "Tap a day to change status",
  "attendanceUpdated": "Attendance updated",
  "attendanceCalendarEmpty": "No attendance in this month",
  "attendanceFutureNotAllowed": "You can’t mark attendance for a future day",
  "attendanceWeekendNotAllowed": "Attendance can only be marked on weekdays (Mon–Fri)",

  "chartBandExcellent": "85–100%",
  "chartBandGood": "70–84%",
  "chartBandNeedsWork": "50–69%",
  "chartBandAtRisk": "Below 50%",

  "marksWeight": "Weight",
  "marksCategory": "Category",
  "marksDate": "Date",
  "marksCommentOptional": "Comment (optional)",
  "marksNewMark": "New mark",
  "marksEditMark": "Edit mark",
  "marksFromSubmission": "From a graded submission",
  "marksNeedTitle": "Give the graded item a title.",
  "marksWeightPositive": "Weight must be greater than zero.",
  "marksNeedAtLeastOne": "Enter a mark for at least one student.",
  "marksLeaveBlankSkip": "Leave a field blank to skip that student.",
  "marksFromSubmissionDelete": "This mark comes from a graded submission. Clear the score in
Submissions to remove it.",
  "marksFromSubmissionEdit": "This mark mirrors a graded submission. Editing it here will be
overwritten the next time the submission is re-graded.",

  "categoryHomework": "Homework",
  "categoryClasswork": "Classwork",
  "categoryQuiz": "Quiz",
  "categoryTest": "Test",
  "categoryExam": "Exam",
  "categoryProject": "Project",
  "categoryOral": "Oral",
  "categoryOther": "Other",

  "teacherReferenceLinkOptional": "Reference Link (optional)",
  "teacherAttachmentUrlOptional": "Attachment URL (optional)",
  "teacherNoTasksYet": "No tasks added yet. Click \"Add Task\" to get started.",

  "teacherTemplates": "Templates",
  "teacherTemplatesEmpty": "No templates yet",
```

```
  "teacherTemplatesEmptySubtitle": "Save an assignment as a template to reuse it later.",
  "teacherSaveAsTemplate": "Save as template",
  "teacherTemplateSaved": "Template saved",
  "teacherUseTemplate": "Use template",
  "teacherFromTemplate": "From template",
  "teacherDeleteTemplate": "Delete template?",
  "teacherDeleteTemplateBody": "\"{title}\" will be removed.",
  "@teacherDeleteTemplateBody": {
    "placeholders": {
      "title": { "type": "String" }
    }
  },
  "teacherDuplicateAssignment": "Duplicate",
  "teacherDuplicateTitle": "Duplicate assignment",
  "teacherDuplicated": "Assignment duplicated",
  "assignmentEdit": "Edit assignment",
  "assignmentDeleteTitle": "Delete assignment?",
  "assignmentDeleteBody": "This permanently removes the assignment, its tasks, and student
submissions.",
  "assignmentDeleted": "Assignment deleted",
  "assignmentTypeHintStatic": "A single piece of work. Students submit once — no extra tasks.",
  "assignmentTypeHintPack": "A pack of multiple tasks (workbook-style). You'll add items next.",
  "teacherCreateFromTemplate": "Create from template",
  "teacherTasksCopied": "{count} tasks included",
  "@teacherTasksCopied": {
    "placeholders": {
      "count": { "type": "int" }
    }
  },
  "teacherRecentFeedback": "Reuse feedback",
  "teacherApplyFeedbackToOthers": "Also apply to other pending for this task",
  "teacherFeedbackAppliedCount": "Feedback applied to {count} other submissions",
  "@teacherFeedbackAppliedCount": {
    "placeholders": {
      "count": { "type": "int" }
    }
  },
  "teacherSharedComment": "Shared comment (optional)",
  "teacherSharedCommentHint": "Same note on every mark you save",

  "adminPlatformOverview": "Platform Overview",
  "adminAssignTeacher": "Assign Teacher",
  "adminSelectStudent": "Select Student",
  "adminInitialGroupOptional": "Initial Group (Optional)",
  "adminChartTasks": "Tasks",
  "adminChartGroups": "Groups",
  "adminChartStaff": "Staff",
  "adminChartAssignments": "Assignments",
```

**lib/l10n/app_en.arb — continued (3/3)**

```
  "adminChartMarks": "Marks",
  "adminChartStudents": "Students",

  "adminAcademicYears": "Academic years",
  "adminAcademicYearsSubtitle": "School years and semesters for the gradebook",
  "adminAcademicYearsTitle": "Academic calendar",
  "adminCreateNextYear": "Create next year",
  "adminYearCreated": "Created {label}",
  "@adminYearCreated": { "placeholders": { "label": { "type": "String" } } },
  "adminSetCurrentYear": "Set as current",
  "adminCurrentYearBadge": "Current",
  "adminYearSemesters": "Four semesters · Aug → Jun",
  "adminSemesterRange": "Semester {semester} · {start} – {end}",
  "@adminSemesterRange": {
    "placeholders": {
      "semester": { "type": "int" },
      "start": { "type": "String" },
      "end": { "type": "String" }
    }
  },
  "adminEditTerm": "Edit term",
  "adminEditTermTitle": "Edit semester {semester}",
```

```
  "@adminEditTermTitle": {
    "placeholders": {
      "semester": { "type": "int" }
    }
  },
  "adminTermStartsOn": "Starts on",
  "adminTermEndsOn": "Ends on",
  "adminTermDatesInvalid": "End date must be on or after the start date",
  "adminTermUpdated": "Term dates updated",
  "adminDeleteYear": "Delete year",
  "adminDeleteYearTitle": "Delete {label}?",
  "@adminDeleteYearTitle": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearBody": "This removes the year and its four terms from the calendar. Marks are
not deleted. To confirm, type {label} below.",
  "@adminDeleteYearBody": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearConfirmHint": "Type {label} to confirm",
  "@adminDeleteYearConfirmHint": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearCurrentBlocked": "Set another year as current before deleting this one.",
  "adminYearDeleted": "Deleted {label}",
  "@adminYearDeleted": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminNoAcademicYears": "No academic years yet",
  "adminAcademicYearsHint": "Create the next school year when you are ready. Teachers and
students pick semesters in the gradebook. Tap a term to edit its dates.",

  "adminCredentialPrintTitle": "Print login slips",
  "adminCredentialPrintSubtitle": "Unique usernames and passwords from the database, 3 slips
per page",
  "adminCredentialPasteLabel": "Users",
  "adminCredentialPasteHint": "username|password|full name\n@james|Abc123xyz|James
English|teacher\n@john_jones|Pass456|John Jones",
  "adminCredentialPerPage": "Slips per page",
  "adminCredentialTwo": "2",
  "adminCredentialThree": "3",
  "adminCredentialExportPdf": "Export PDF",
  "adminCredentialSlipTitle": "Your MU Study login",
  "adminCredentialSlipHint": "Keep this paper private. Change your password after first sign-in.
",
  "adminCredentialNeedUsers": "No users found to print",
  "adminCredentialNeedSql": "Run sql/admin_login_slips.sql in the Supabase SQL Editor, then
export again.",
  "adminCredentialConfidential": "Private and confidential. These credentials are intended only
for the assigned student.",
  "adminFirstName": "First name",
  "adminLastName": "Last name",
  "adminCredentialStudents": "Students",
  "adminCredentialTeachers": "Teachers",
  "adminCredentialAllGroups": "All groups",

  "sphereMath": "Math",
  "sphereEnglish": "English",

  "dailyMotivation1": "Today is a great day to make progress.",
  "dailyMotivation2": "Ready to continue where you left off?",
  "dailyMotivation3": "Let's conquer your goals today.",
  "dailyMotivation4": "Consistency is the key to mastery."
}
```

## app_ru.arb

**lib/l10n/app_ru.arb**

```
{
  "@@locale": "ru",
  "appName": "MU Study",
  "shellMuStudy": "MU Study",

  "languageTitle": "[redacted]",
  "languageSubtitle": "[redacted].",
  "languageEnglish": "[redacted]",
  "languageRussian": "[redacted]",
  "languageUzbek": "[redacted]",
  "languageEnglishNative": "English",
  "languageRussianNative": "[redacted]",
  "languageUzbekNative": "O‘zbekcha",
  "languageContinue": "[redacted]",
  "languageSection": "[redacted]",
  "languageHint": "[redacted]",

  "loginWelcome": "[redacted]",
  "loginSubtitle": "[redacted] MU Study",
  "loginUsername": "[redacted]",
  "loginPassword": "[redacted]",
  "loginSignIn": "[redacted]",
  "loginMissingFields": "[redacted]",
  "loginIncorrect": "[redacted]",
  "loginFailed": "[redacted].",
  "loginConfigError": "[redacted].",
  "loginConfigMissing": "[redacted] .env.",
  "accountDisabled": "[redacted].",

  "navDashboard": "[redacted]",
  "navAssignments": "[redacted]",
  "navMarks": "[redacted]",
  "navAttendance": "[redacted]",
  "navNotifications": "[redacted]",
  "navProfile": "[redacted]",
  "navUsers": "[redacted]",
  "navGroups": "[redacted]",
  "navAnalytics": "[redacted]",
  "navGradebook": "[redacted]",
  "navChat": "[redacted]",
  "navFocus": "[redacted]",

  "focusTitle": "[redacted]",
  "focusSubtitle": "[redacted] — [redacted].",
  "focusTodaysFocus": "[redacted]",
  "focusTodaysFocusSubtitle": "[redacted] — [redacted]",
  "focusAlsoDue": "[redacted]",
  "focusContinueHomework": "[redacted]",
  "focusOpenAssignment": "[redacted]",
  "focusEmptyTitle": "[redacted]",
  "focusEmptySubtitle": "[redacted], [redacted].",
  "focusChooseMode": "[redacted]?",
  "focusModePomodoro": "[redacted]",
  "focusModePomodoroHint": "25 [redacted], [redacted]",
  "focusModeFree": "[redacted]",
  "focusModeFreeHint": "[redacted]",
  "focusDoNotDisturb": "[redacted]",
  "focusChatMuteHint": "[redacted].",
  "focusPause": "[redacted]",
  "focusResume": "[redacted]",
  "focusPaused": "[redacted]",
  "focusEndSession": "[redacted]",
  "focusSessionComplete": "[redacted]",
  "focusSessionCompleteSubtitle": "[redacted]:",
  "focusTimeUp": "[redacted]!",
  "focusTimeUpSubtitle": "[redacted].",
  "focusMinutesFocused": "[redacted]: {duration}",
  "focusOptionalNote": "[redacted] ([redacted])",
```

```
  "focusOptionalNoteHint": "[redacted]. [redacted] [redacted] [redacted] [redacted]3",
  "focusSaveAndOpen": "[redacted] [redacted] [redacted] [redacted]",
  "focusDone": "[redacted]",

  "chatTitle": "[redacted]",
  "chatSearchHint": "[redacted] [redacted] [redacted] [redacted]",
  "chatEmptyTitle": "[redacted] [redacted] [redacted]",
  "chatEmptySubtitle": "[redacted] [redacted] [redacted], [redacted] [redacted] [redacted].",
  "chatNoMessagesYet": "[redacted] [redacted] [redacted]",
  "chatMessageDeleted": "[redacted] [redacted]",
  "chatPreviewImage": "[redacted]",
  "chatPreviewVideo": "[redacted]",
  "chatVideoPlaybackSpeed": "[redacted] [redacted]",
  "chatVideoSpeedNormal": "1x",
  "chatVideoSpeedValue": "{speed}x",
  "@chatVideoSpeedValue": {
    "placeholders": {
      "speed": { "type": "double", "format": "decimalPattern" }
    }
  },
  "chatVideoRewind10": "-10[redacted]",
  "chatVideoForward10": "+10[redacted]",
  "chatVideoCouldNotPlay": "[redacted] [redacted] [redacted] [redacted]",
  "chatPreviewVoice": "[redacted] [redacted]",
  "chatPreviewFile": "[redacted]",
  "chatOpenFile": "[redacted]",
  "chatOpenFileFailed": "[redacted] [redacted] [redacted] [redacted].",
  "chatYou": "[redacted]",
  "chatTeacherMembers": "{teacher} · {count} [redacted]",
  "@chatTeacherMembers": {
    "placeholders": {
      "teacher": { "type": "String" },
      "count": { "type": "int" }
    }
  },
  "chatOnlineCount": "{count} [redacted] [redacted]",
  "@chatOnlineCount": {
    "placeholders": {
      "count": { "type": "int" }
    }
  },
  "chatReact": "[redacted]",
  "chatCopyText": "[redacted] [redacted]",
  "chatCopied": "[redacted]",
  "chatReply": "[redacted]",
  "chatReplyTo": "[redacted] [redacted] {name}",
  "@chatReplyTo": {
    "placeholders": {
      "name": { "type": "String" }
    }
  },
  "chatReplyDeleted": "[redacted] [redacted]",
  "chatReplyNotFound": "[redacted] [redacted] [redacted]",
  "chatMessageHint": "[redacted]…",
  "chatAttachImage": "[redacted]",
  "chatAttachVideo": "[redacted]",
  "chatAttachFile": "[redacted]",
  "chatTyping": "{names} [redacted]…",
  "@chatTyping": { "placeholders": { "names": { "type": "String" } } },
  "chatRecording": "[redacted] {time}",
  "@chatRecording": { "placeholders": { "time": { "type": "String" } } },
  "chatUploading": "[redacted] {name}…",
  "@chatUploading": { "placeholders": { "name": { "type": "String" } } },
  "chatSendFailed": "[redacted] [redacted] [redacted]: {error}",
  "@chatSendFailed": { "placeholders": { "error": { "type": "String" } } },
  "chatMicDenied": "[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted].",
  "chatEditTitle": "[redacted] [redacted]",
  "chatDeleteTitle": "[redacted] [redacted]?",
  "chatDeleteBody": "[redacted] [redacted] [redacted] [redacted] [redacted] [redacted].",
  "chatDeleteSelectedTitle": "[redacted] {count} [redacted].?",
  "chatDeleteSelectedBody": "[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted].",
  "chatSelectedCount": "[redacted]: {count}",
  "chatSelectMessages": "[redacted] [redacted]",
```

```
  "chatSelect": "████████",
  "commonSelectAll": "████████ ███",
  "chatEdited": "███.",
  "chatDateToday": "████████",
  "chatDateYesterday": "██████",
  "chatRoleAdmin": "██████",
  "chatRoleTeacher": "████████",

  "commonSave": "██████████",
  "commonCancel": "███████",
  "commonSubmit": "██████████",
  "commonDelete": "████████",
  "commonEdit": "█████████",
  "commonClose": "████████",
  "commonRetry": "██████████",
  "commonTryAgain": "████████████ ██████",
  "commonLoading": "██████████…",
  "commonError": "████-██ ██████ ██ ████",
  "commonErrorWithDetails": "███████: {error}",
  "commonCouldNotSave": "██ ████████ ██████████: {error}",
  "commonSignOut": "██████",
  "commonSaveDraft": "██████████ █████████",
  "commonOpen": "████████",
  "commonCreate": "█████████",
  "commonAssign": "██████████",
  "commonReset": "█████████",
  "commonAdd": "█████████",
  "commonLoadMore": "██████████ ████",
  "commonExport": "█████████",
  "commonExportFailed": "██ ████████ ████████████████: {error}",
  "@commonExportFailed": { "placeholders": { "error": { "type": "String" } } },
  "marksTermAll": "██ ████ ██████",
  "marksTermSemester": "{yearRange} · ████████ {semester}",
  "@marksTermSemester": {
    "placeholders": {
      "yearRange": { "type": "String" },
      "semester": { "type": "int" }
    }
  },
  "marksExportCsv": "████████ CSV",
  "marksExportDone": "███████ ████████████████",
  "commonTeacher": "████████",
  "commonStudent": "███████",
  "commonAdmin": "████████████████",
  "commonWorkspace": "█████████ ██████████████",
  "commonNoDueDate": "████ ██████",
  "commonDueDate": "█████ {date}",
  "homeworkDueThisWeek": "██ █████ ███████",
  "homeworkDueThisWeekSubtitle": "██████████████ ■ ██████████ ██████",
  "homeworkNoDueThisWeek": "██ █████ ███████ ███████ ████",
  "homeworkNoDueThisWeekSubtitle": "██ ██████████ ██████████ ███████ █████████.",
  "homeworkOverdue": "████████████ · {date}",
  "@homeworkOverdue": { "placeholders": { "date": { "type": "String" } } },
  "homeworkDueToday": "█████ ████████",
  "homeworkDueTomorrow": "█████ ███████",
  "homeworkDueInDays": "██████ {days} ██.",
  "@homeworkDueInDays": { "placeholders": { "days": { "type": "int" } } },
  "homeworkLateBadge": "███████████",
  "commonProgress": "██████████",
  "commonSubject": "█████████",
  "commonType": "████",
  "commonTitle": "██████████",
  "commonDescription": "██████████",
  "commonGroup": "███████",
  "commonMessage": "██████████",
  "commonScore": "███████",
  "commonFeedback": "██████",
  "commonStudents": "█████████",
  "commonCannotUndo": "████ ██████████ ███████ ██████████.",

  "profileTitle": "█████████",
  "profileFullName": "███████ ████",
  "profileNotSignedIn": "██ ██ ██████",
```

```
  "profileAccount": "████████",
  "profileAccountSettings": "██████████ █████████",
  "profileSignOutAdmin": "██████ ██ ████████ ████████████████",
  "profileSettings": "██████████",
  "profileAppearance": "████████████",
  "profileThemeLight": "████████",
  "profileThemeDark": "███████",
  "profileThemeSystem": "██████████",
  "profileStyle": "██████",
  "profileStyleModern": "████████████ ██████████████████",
  "profileStyleNature": "██████████ ███████",
  "profileStyleSoftNeu": "████████████",
  "profileStyleMonochrome": "████████████ ██████████████",
  "profileEditName": "██████████ ████",
  "profileNameUpdated": "████ ██████████",
  "profileChangePhoto": "████████ █████",
  "profilePhotoUpdated": "█████ ██████████",
  "profileChangePassword": "████████ ███████",
  "profileCurrentPassword": "████████ ███████",
  "profileNewPassword": "██████ ███████",
  "profileConfirmPassword": "████████████ ██████ ███████",
  "profilePasswordChanged": "███████ █████████",
  "profilePasswordMismatch": "██████ ███████ ██ ██████████",
  "profilePasswordTooShort": "██ ██████ 8 █████████",
  "profilePasswordHint": "█████ ███████ ███████ — ████████████ ■ ████████████████.",
  "profileShortcuts": "████████ ███████",
  "profileYourGroups": "█████ ███████",
  "profileNoGroups": "███████ ██ ██████████",
  "profileGroupsCount": "██████: {count}",
  "@profileGroupsCount": { "placeholders": { "count": { "type": "int" } } },
  "profileAverageLabel": "████████ ██ ███████",
  "profileAttendanceLabel": "███████████████",
  "profileShareProgress": "████████████ ████████",
  "profileNotifications": "██████████████",
  "profileClearCache": "██████████ ████",
  "profileClearCacheWarn1Title": "██████████ ████?",
  "profileClearCacheWarn1Body": "██████ ████████ ███████-██████████ █████ ■ ██████████ ██████████████,
██████, ██████████ ■ ██████. ████ ██████████ ██████████ ██████████████ ██████████.",
  "profileClearCacheWarn2Title": "██ ████████?",
  "profileClearCacheWarn2Body": "████ ██████████ ███████ ██████████. ████ █████ ██████ ███████ ■ ██████
████████████.",
  "profileClearCacheConfirm": "██████████ ████",
  "profileClearCacheDone": "████ ███████",
  "profileAppVersion": "MU Study · v{version}",
  "@profileAppVersion": { "placeholders": { "version": { "type": "String" } } },
  "profileSubject": "████████",

  "greetingMorning": "███████ █████",
  "greetingAfternoon": "███████ █████",
  "greetingEvening": "███████ ██████",
  "greetingWithName": "{greeting}, {name}",
  "greetingLoadingTitle": "███████ █████, ███████████████",
  "greetingLoadingSubtitle": "██████████ █████ ██████████████...",
  "studentHomeSubtitle": "█████ ████████ ████████████████",
  "continueLearning": "████████████ ██████████",
  "continueLearningSubtitle": "████████████ ■ ██████ ███████, ████ ████████████████",
  "everythingCompleted": "████ ████████████.",
  "caughtUpToday": "██ ████████ ████ ████████.",
  "homeworkPack": "███████ ██████████ ████████",
  "staticTask": "████████ ████████",
  "staticAssignment": "█████████",
  "upcomingAssignments": "██████████ ████████",
  "stayOnTrack": "██ ████████████",
  "loadingAnnouncements": "██████████ ████████████...",
  "notificationsCardTitle": "██████████████",
  "unreadAlerts": "■ ████ {count} ███████ ██████████████",
  "upToDate": "■ ████ ████ ████████████",

  "announcementsTitle": "████████████",
  "announcementsSubtitle": "██ ███████ ██████████",
  "announcementsEmpty": "████ ████████████",
  "announcementsEmptyTeacher": "██████████ ████████ ████████████████ ■ ██████████.",
  "announcementsEmptyAdmin": "████████████ ██ ██████ ██████████ ██████████ ███████.",
```

```
  "announcementsReadMore": "[redacted]",
  "announcementsShowLess": "[redacted]",
  "announcementsViewAll": "[redacted] ([redacted] ({count})",
  "announcementsAll": "[redacted]",
  "announcementsNew": "[redacted]",
  "announcementsPost": "[redacted]",
  "announcementsEdit": "[redacted]",
  "announcementsDeleteTitle": "[redacted]?",
  "announcementsDeleteBody": "[redacted] [redacted].",
  "announcementsByAdmin": "[redacted]",
  "announcementsShareUpdates": "[redacted]",

  "assignmentsTitle": "[redacted]",
  "assignmentsEmpty": "[redacted]",
  "assignmentsEmptySubtitle": "[redacted]!",
  "assignmentsEmptyTeacher": "[redacted], [redacted].",
  "assignmentsLearningPath": "[redacted]",
  "assignmentsActive": "[redacted]",
  "assignmentDetails": "[redacted]",
  "assignmentNotFound": "[redacted]",
  "assignmentTasks": "[redacted]",
  "assignmentTasksSubtitle": "[redacted], [redacted]",
  "assignmentNew": "[redacted]",
  "assignmentManage": "[redacted]",
  "assignmentManageSubtitle": "[redacted]",

  "marksTitle": "[redacted]",
  "marksOverallAverage": "[redacted]",
  "marksNoneYet": "[redacted].",
  "marksBasedOn": "{label} · [redacted] {count} [redacted].",
  "marksByCategory": "[redacted]",
  "marksWhereStrongest": "[redacted]",
  "marksAll": "[redacted]",
  "marksNewestFirst": "[redacted]",
  "marksEmptyTitle": "[redacted]",
  "marksEmptySubtitle": "[redacted].",
  "progressReportTitle": "[redacted]",
  "progressReportShare": "[redacted]",
  "progressReportSharing": "[redacted]…",
  "progressReportDone": "[redacted]",
  "progressReportFailed": "[redacted]: {error}",
  "@progressReportFailed": {
    "placeholders": {
      "error": { "type": "String" }
    }
  },
  "progressReportNeedGroup": "[redacted], [redacted].",
  "progressReportAttendance": "[redacted]",
  "progressReportAttendanceRate": "[redacted]",
  "progressReportNoAttendance": "[redacted].",
  "progressReportSessions": "[redacted]",
  "progressReportMarksCounted": "[redacted]",
  "progressReportRecentMarks": "[redacted]",
  "progressReportGeneratedOn": "[redacted] {date}",
  "@progressReportGeneratedOn": {
    "placeholders": {
      "date": { "type": "String" }
    }
  },
  "marksNoRecorded": "[redacted]",
```

**lib/l10n/app_ru.arb — continued (2/3)**

```
  "marksNoRecordedSubtitle": "[redacted] «[redacted]», [redacted].",
  "marksRecent": "[redacted]",
  "marksRecentSubtitle": "[redacted]",
  "marksStudentsHeader": "[redacted]",
  "marksStudentsSubtitle": "[redacted]",
  "marksNoStudents": "[redacted]",
  "marksNoStudentsSubtitle": "[redacted].",
  "marksDeleteTitle": "[redacted]?",
  "marksDeleteBody": "«{title}» [redacted].",
  "marksGradeTheClass": "[redacted]",
```

```
  "marksOnePercentEach": "■■■■■ ■■■■■■■■■■■ ■■■■■■■ ■■ ■■■■■■■■",
  "marksSaveMarks": "■■■■■■■■■■ ■■■■■■■",
  "marksSaveMark": "■■■■■■■■■■ ■■■■■■■",
  "marksAddMark": "■■■■■■■■■ ■■■■■■■",
  "marksNothingYet": "■■■■■ ■■■■■■■ ■■■■",
  "marksAddFirst": "■■■■■■■■■ ■■■■■■■ ■■■■■■■ ■■■■■■ ■■■■■■■■.",
  "marksSavedCount": "■■■■■■■■■■■ ■■■■■■■: {count}",
  "marksGroupHasNoStudents": "■ ■■■■■ ■■■■■■■ ■■■■■ ■■■■ ■■■■■■■■■.",
  "marksTitleHint": "■■■■■■■■■, ■■■■■ ■■ ■■■■■ 3",
  "marksAverageLine": "■■■■■■■■ {percent} · {label}",
  "marksNoMarksYetShort": "■■■■■■■ ■■■■■ ■■■■",
  "marksSummaryStudents": "■■■■■■■■■",
  "marksSummaryMarks": "■■■■■■■■",

  "attendanceTitle": "■■■■■■■■■■■■■■",
  "attendanceEmpty": "■■■■ ■■■■■■■■ ■■■■■■■■■■■■■■■",
  "attendanceNoGroup": "■■■■■■■ ■■■■ ■■ ■■■■■■■■■■■",
  "attendanceMark": "■■■■■■■■■ ■■■■■■■■■■■■■■",
  "attendanceDailyTracking": "■■■■■■■■■■■■ ■■■■■ ■■■■■■■■■",
  "attendanceSaved": "■■■■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■■■■",
  "attendanceNoGroups": "■■■■■■■ ■■ ■■■■■■■■",
  "attendancePresent": "■ — ■■■■■■■■■■■■■■■",
  "attendanceLate": "■ — ■■■■■■■■",
  "attendanceAbsent": "■ — ■■■■■■■■■■■■■■",
  "attendanceExcused": "■ — ■■■■■■■■■■■■■■",
  "attendancePresentShort": "■",
  "attendanceLateShort": "■",
  "attendanceAbsentShort": "■",
  "attendanceExcusedShort": "■",
  "attendanceTrend": "■■■■■■■■■ ■■■■■■■■■■■■■■",
  "attendanceOverall": "■■■■■■ ■■■■■■■■■■■■■■■",
  "analyticsThisWeek": "■■■■ ■■■■■■■",
  "analyticsLastWeek": "■■■■■■■■ ■■■■■■■",
  "analyticsTotalMarks": "■■■■■■ ■■■■■■■: {count}",
  "@analyticsTotalMarks": {
    "placeholders": {
      "count": { "type": "int" }
    }
  },

  "notificationsTitle": "■■■■■■■■■■■■■",
  "notificationsEmpty": "■■■■ ■■■■■■■■■■!",
  "notificationsEmptySubtitle": "■■■■■■ ■■■■■■■■■■■■ ■■■■.",
  "notificationsNone": "■■■■ ■■■■■■■■■■■■■",
  "notifKindAssignment": "■■■■■■ ■■■",
  "notifKindAnnouncement": "■■■■■■■■■■■■■",
  "notifKindGraded": "■■■■■■■ ■■■■■■■■■",
  "notifKindSubmission": "■■■■■■ ■■■■■■",
  "notifKindChat": "■■■■■■■■■■ ■ ■■■■■",
  "notifKindSystem": "■■■■■■■■■■■■",
  "notificationsOpen": "■■■■■■■■",

  "taskTitle": "■■■■■■■■",
  "taskYourAnswer": "■■■■ ■■■■■■",
  "taskMarkCompleted": "■■■■■■■■■ ■■■■■■■■■■■■■",
  "taskAttachment": "■■■■■■■■■■",
  "taskAttachmentHint": "■■■■■■■■■ ■■■■■ ■■■■ ■■■■■■■■■ ■ ■■■■■■■.",
  "taskAttachmentHintReady": "■■■■■■■■, ■■■■■■ ■■■■■■■■ · ■■■■■■■■ — ■■■■■■■■.",
  "taskImage": "■■■■■■■■■■■■■",
  "taskImageSubtitle": "■■■■■ ■■ ■■■■■■■■",
  "taskFile": "■■■■■",
  "taskFileSubtitle": "PDF ■■■■ ■■■■■■■■■",
  "taskReplaceImage": "■■■■■■■■■ ■■■■■■■■■■■■■■",
  "taskReplaceFile": "■■■■■■■■■ ■■■■■",
  "taskVoice": "■■■■■■■■■■ ■■■■■■■",
  "taskVoiceHint": "■■■■■■■■■■■■■■■■■ ■■■■■■■ ■■■■■■ — ■■■■■■■■■, ■■■■■■■■■■■■■ ■ ■■■■■■■■■■.",
  "taskViewPdf": "■■■■■■■■ PDF",
  "taskOpenLink": "■■■■■■■■ ■■■■■■■",
  "taskSubmitted": "■■■■■■■■■■■■",
  "taskSaved": "■■■■■■■■■■■",
  "taskUploadFailed": "■■ ■■■■■■■■ ■■■■■■■■■■: {error}",
  "taskVoiceUploadFailed": "■■ ■■■■■■■■ ■■■■■■■■■■ ■■■■■■: {error}",
  "taskCouldNotSave": "■■ ■■■■■■■■ ■■■■■■■■■■: {error}",
```

```
  "voiceTapToRecord": "[redacted], [redacted]",
  "voiceRecording": "[redacted]…",
  "voiceNote": "[redacted]",
  "voiceTapPlay": "[redacted], [redacted]",
  "voiceRerecord": "[redacted]",
  "voiceStudentAnswer": "[redacted]",
  "voiceMicPermission": "[redacted].",
  "voiceCouldNotStart": "[redacted]: {error}",
  "voiceCouldNotSave": "[redacted]: {error}",
  "voiceCouldNotPlay": "[redacted]: {error}",
  "attachmentCouldNotOpen": "[redacted].",
  "attachmentImage": "[redacted]",
  "attachmentFile": "[redacted]",
  "attachmentPdf": "PDF-[redacted]",
  "attachmentTapPreview": "[redacted]",
  "attachmentTapOpen": "[redacted], [redacted]",
  "pdfDefaultTitle": "PDF",

  "teacherWorkspace": "[redacted]",
  "teacherQuickActions": "[redacted]",
  "teacherCreateItems": "[redacted]",
  "teacherNewAssignment": "[redacted]",
  "teacherNewAnnouncement": "[redacted]",
  "teacherSubmissions": "[redacted]",
  "teacherNoSubmissions": "[redacted]",
  "teacherNoSubmissionsSubtitle": "[redacted].",
  "teacherGradebook": "[redacted]",
  "teacherGradeClass": "[redacted]",
  "teacherNoGroups": "[redacted]",
  "teacherNoGroupsSubtitle": "[redacted].",
  "teacherActiveGroups": "[redacted]",
  "teacherSubject": "[redacted]",
  "teacherDailyTasks": "[redacted]",
  "teacherNeedsAttention": "[redacted]",
  "teacherReviewSubmissions": "[redacted]",
  "teacherCheckAssignments": "[redacted]",
  "teacherRecordMarks": "[redacted]",
  "teacherPerformanceAnalytics": "[redacted]",
  "teacherTrackProgress": "[redacted]",
  "teacherOpenAttachment": "[redacted]",
  "teacherScoreHelper": "[redacted]",
  "teacherScoreRange": "[redacted] 0 [redacted] 100.",
  "teacherNoWrittenAnswer": "[redacted].",
  "teacherReviewSubmission": "[redacted]",
  "teacherHomeworkPackTasks": "[redacted]",
  "teacherAddTask": "[redacted]",
  "teacherEditTask": "[redacted]",
  "teacherDeleteTask": "[redacted]?",
  "teacherTasksContent": "[redacted]",
  "teacherManagePack": "[redacted]",
  "teacherTaskTitleHint": "[redacted]",
  "teacherLinkHint": "https://...",
  "teacherAttachmentHint": "[redacted]...",
  "teacherNoData": "[redacted]",
  "teacherMarkDistribution": "[redacted]",
  "teacherStudentsByAverage": "[redacted]",
  "teacherNoMarksTrends": "[redacted], [redacted].",
  "teacherTargetGroup": "[redacted]",

  "adminWorkspace": "[redacted]",
  "adminUsers": "[redacted]",
  "adminGroups": "[redacted]",
  "adminAnalytics": "[redacted]",
  "adminTeachers": "[redacted]",
  "adminStudents": "[redacted]",
  "adminGroupsLabel": "[redacted]",
  "adminMarksLabel": "[redacted]",
  "adminActivityInsights": "[redacted]",
  "adminCurrentMetrics": "[redacted]",
  "adminCreateUser": "[redacted]",
  "adminCreateUserButton": "[redacted]",
  "adminUserCreated": "[redacted]",
  "adminResetPassword": "[redacted]",
```

```
  "adminResetPasswordTitle": "██████ ███████ · {username}",
  "adminCreateGroup": "████████ ███████",
  "adminCreateGroupButton": "████████ ███████",
  "adminGroupTitleHint": "█████████, ███████ 4",
  "adminGroupTitleExists": "█ ██████ ████████ ███ ████ ███████ █ ██████ ██████████",
  "adminAssignStudent": "██████████ ████████",
  "adminActivityDistribution": "████████████████ ████████████",
  "adminGlobalMetrics": "████████████ ████████████ ██████████",
  "adminActivityLog": "███████ ████████████ ██████████████████",
  "adminActivityLogSubtitle": "██████████ ████████ █ ██████ ████████ ██████.",
  "adminTeacherLine": "████████: {name}",
  "adminRoleLine": "{role} · {username}",
  "adminDisable": "████████████",
  "adminEnable": "██████████",
  "adminPassword": "████████",
  "adminFullName": "███████ ████",
  "adminRole": "█████",
  "adminSphere": "██████",

  "percentExcellent": "████████",
  "percentGood": "███████",
  "percentNeedsWork": "██████ ██████████",
  "percentAtRisk": "████ ████████",
  "emptyStateDefault": "█████ ███████ ████",
  "setupRequiredTitle": "██████████ ████████████",
  "setupRequiredBody": "Supabase ██ ██████████. ██████████ SUPABASE_URL ■ SUPABASE_ANON_KEY ■ ██████ .
env.",
  "setupOption1": "████████ 1 — ██████████ ██████ .env",
  "setupOption2": "████████ 2 — dart-define",

  "attendanceAllPresent": "████ ■",
  "attendanceAllAbsent": "████ ■",
  "attendanceAllLate": "████ ■",
  "attendanceAllExcused": "████ ■",
  "attendancePresentWord": "████████████████",
  "attendanceAbsentWord": "███████████████",
  "attendanceLateWord": "█████████",
  "attendanceExcusedWord": "████████████████",
  "attendanceWeekdayMon": "███",
  "attendanceWeekdayTue": "███",
  "attendanceWeekdayWed": "███",
  "attendanceWeekdayThu": "███",
  "attendanceWeekdayFri": "███",
  "attendancePercentLabel": "██████",
  "attendanceNoLesson": "████ ███████",
  "attendanceHistory": "████████ ████████████████",
  "attendanceSelectStudent": "████████",
  "attendanceAllStudents": "████ █████████",
  "attendanceTapToCycle": "████████ ██ ██████, ██████ ██████████ ███████",
  "attendanceUpdated": "████████████████ ████████████",
  "attendanceCalendarEmpty": "████ ████████████████ ■ ██████ ███████",
  "attendanceFutureNotAllowed": "███████ ██████████ ████████████████ ██ ████████ ██████",
  "attendanceWeekendNotAllowed": "████████████████ ████████████ ███████ ■ ██████ (██–██)",

  "chartBandExcellent": "85–100%",
  "chartBandGood": "70–84%",
  "chartBandNeedsWork": "50–69%",
  "chartBandAtRisk": "██████ 50%",

  "marksWeight": "████",
  "marksCategory": "████████████",
  "marksDate": "██████",
  "marksCommentOptional": "██████████████ (████████████████)",
  "marksNewMark": "███████ ███████",
  "marksEditMark": "██████████ ███████",
  "marksFromSubmission": "██ ██████████████ ████████",
  "marksNeedTitle": "█████████ ██████████ ████████████████ ██████████.",
  "marksWeightPositive": "████ ███████ ██████ ███████ ██████.",
  "marksNeedAtLeastOne": "█████████ ███████ ██████ ██ ███████ █████████.",
  "marksLeaveBlankSkip": "██████████ ██████ ████████, ███████ ██████████████ █████████.",
  "marksFromSubmissionDelete": "████ ███████ ██ ██████████████ ███████. ███████ ████████ ██, ██████████
██████ ■ ████████ «███████».",
  "marksFromSubmissionEdit": "████ ███████ ████████ ■ ██████████████ ████████. ██████████ ███████ ██████
```

```
  ████████████ ███ ██████████ █████████.",

    "categoryHomework": "█████████ ████████",
    "categoryClasswork": "█████████ ███████",
    "categoryQuiz": "█████",
    "categoryTest": "█████",
    "categoryExam": "█████████",
    "categoryProject": "███████",
    "categoryOral": "██████",
    "categoryOther": "███████",

    "teacherReferenceLinkOptional": "███████ (████████████████)",
    "teacherAttachmentUrlOptional": "URL ██████████ (████████████████)",
    "teacherNoTasksYet": "██████ ██████ ████. ████████ «██████████ ███████», ██████ ███████.",

    "teacherTemplates": "████████",
    "teacherTemplatesEmpty": "██████████ ██████ ████",
    "teacherTemplatesEmptySubtitle": "███████████ ████████ ████ ███████, ██████ ██████████████ ██████.",
    "teacherSaveAsTemplate": "███████████ ████ ███████",
    "teacherTemplateSaved": "███████ ██████████",
    "teacherUseTemplate": "██████████████ ███████",
    "teacherFromTemplate": "███ ████████",
    "teacherDeleteTemplate": "█████████ ████████?",
    "teacherDeleteTemplateBody": "«{title}» ██████ ███████.",
    "@teacherDeleteTemplateBody": {
      "placeholders": {
        "title": { "type": "String" }
      }
    },
    "teacherDuplicateAssignment": "████████████",
    "teacherDuplicateTitle": "████████████ ████████",
    "teacherDuplicated": "████████ ████████████",
    "assignmentEdit": "██████████ ████████",
    "assignmentDeleteTitle": "█████████ ████████?",
    "assignmentDeleteBody": "█████████, ███████ ■ ███████ ██████████ ██████ ████████ ██████████████.",
    "assignmentDeleted": "█████████ ██████████",
    "assignmentTypeHintStatic": "█████ ████████. ███████ ██████ ████ ███ — ███ ████████████████ ██████.",
    "assignmentTypeHintPack": "██████ ██ ████████████ ██████. ███████ ██████████ ███████.",
    "teacherCreateFromTemplate": "████████ ██ ████████",
    "teacherTasksCopied": "██████: {count}",
    "@teacherTasksCopied": {
      "placeholders": {
        "count": { "type": "int" }
      }
    },
    "teacherRecentFeedback": "██████████ ██████████████ ██████",
    "teacherApplyFeedbackToOthers": "██████ ███████████ ■ ███████ ███████████ ██ █████ ███████",
    "teacherFeedbackAppliedCount": "██████ ██████████ ■ {count} ███████ ████████",
    "@teacherFeedbackAppliedCount": {
      "placeholders": {
        "count": { "type": "int" }
      }
    },
    "teacherSharedComment": "██████ ████████████ (████████████████)",
    "teacherSharedCommentHint": "█████ ■ ██ ██ ████████ ██ ██████ ████████████ ████████",

    "adminPlatformOverview": "██████ ███████████",
    "adminAssignTeacher": "███████████ ████████",
    "adminSelectStudent": "████████ ████████",
    "adminInitialGroupOptional": "██████████ ███████ (████████████████)",
    "adminChartTasks": "███████",
    "adminChartGroups": "███████",
    "adminChartStaff": "████████████",
    "adminChartAssignments": "████████",
    "adminChartMarks": "███████",
    "adminChartStudents": "████████",

    "adminAcademicYears": "████████ ██████",
    "adminAcademicYearsSubtitle": "█████ ■ ██████████ ████ ████████ ███████",
    "adminAcademicYearsTitle": "████████ ███████████",
    "adminCreateNextYear": "████████ ██████████ ████",
    "adminYearCreated": "███████ {label}",
    "@adminYearCreated": { "placeholders": { "label": { "type": "String" } } },
```

```
  "adminSetCurrentYear": "[redacted]",
  "adminCurrentYearBadge": "[redacted]",
  "adminYearSemesters": "[redacted] · [redacted] → [redacted]",
  "adminSemesterRange": "[redacted] {semester} · {start} – {end}",
  "@adminSemesterRange": {
    "placeholders": {
      "semester": { "type": "int" },
      "start": { "type": "String" },
      "end": { "type": "String" }
    }
  },
  "adminEditTerm": "[redacted]",
  "adminEditTermTitle": "[redacted] {semester}",
  "@adminEditTermTitle": {
    "placeholders": {
      "semester": { "type": "int" }
    }
  },
  "adminTermStartsOn": "[redacted]",
  "adminTermEndsOn": "[redacted]",
  "adminTermDatesInvalid": "[redacted]",
  "adminTermUpdated": "[redacted]",
  "adminDeleteYear": "[redacted]",
  "adminDeleteYearTitle": "[redacted] {label}?",
  "@adminDeleteYearTitle": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
```

**lib/l10n/app_ru.arb — continued (3/3)**

```
  "adminDeleteYearBody": "[redacted]",
  "@adminDeleteYearBody": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearConfirmHint": "[redacted] {label} [redacted]",
  "@adminDeleteYearConfirmHint": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearCurrentBlocked": "[redacted]",
  "adminYearDeleted": "[redacted] {label}",
  "@adminYearDeleted": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminNoAcademicYears": "[redacted]",
  "adminAcademicYearsHint": "[redacted]",

  "adminCredentialPrintTitle": "[redacted]",
  "adminCredentialPrintSubtitle": "[redacted]",
  "adminCredentialPasteLabel": "[redacted]",
  "adminCredentialPasteHint": "username|password|full name\n@james|Abc123xyz|James
English|teacher\n@john_jones|Pass456|John Jones",
  "adminCredentialPerPage": "[redacted]",
  "adminCredentialTwo": "2",
  "adminCredentialThree": "3",
  "adminCredentialExportPdf": "[redacted] PDF",
  "adminCredentialSlipTitle": "[redacted] MU Study",
  "adminCredentialSlipHint": "[redacted]",
  "adminCredentialNeedUsers": "[redacted]",
  "adminCredentialNeedSql": "[redacted] sql/admin_login_slips.sql [redacted] SQL Editor Supabase, [redacted]",
  "adminCredentialConfidential": "[redacted]",
  "adminFirstName": "[redacted]",
```

```
  "adminLastName": "[redacted]",
  "adminCredentialStudents": "[redacted]",
  "adminCredentialTeachers": "[redacted]",
  "adminCredentialAllGroups": "[redacted]",

  "sphereMath": "[redacted]",
  "sphereEnglish": "[redacted]",

  "dailyMotivation1": "[redacted]",
  "dailyMotivation2": "[redacted]",
  "dailyMotivation3": "[redacted]",
  "dailyMotivation4": "[redacted]"
}
```

# app_uz.arb

**lib/l10n/app_uz.arb**

```
{
  "@@locale": "uz",
  "appName": "MU Study",
  "shellMuStudy": "MU Study",

  "languageTitle": "Tilni tanlang",
  "languageSubtitle": "Buni istalgan vaqtda Profil bo‘limida o‘zgartirishingiz mumkin.",
  "languageEnglish": "Inglizcha",
  "languageRussian": "Ruscha",
  "languageUzbek": "O‘zbekcha",
  "languageEnglishNative": "English",
  "languageRussianNative": "[redacted]",
  "languageUzbekNative": "O‘zbekcha",
  "languageContinue": "Davom etish",
  "languageSection": "Til",
  "languageHint": "Ilova tili",

  "loginWelcome": "Xush kelibsiz",
  "loginSubtitle": "MU Study ish maydoniga kiring",
  "loginUsername": "Foydalanuvchi nomi",
  "loginPassword": "Parol",
  "loginSignIn": "Kirish",
  "loginMissingFields": "Foydalanuvchi nomi va parolni kiriting",
  "loginIncorrect": "Foydalanuvchi nomi yoki parol noto‘g‘ri",
  "loginFailed": "Kirib bo‘lmadi. Ulanishni tekshirib, qayta urinib ko‘ring.",
  "loginConfigError": "Noto‘g‘ri sozlama. Qo‘llab-quvvatlashga murojaat qiling.",
  "loginConfigMissing": "Sozlama yo‘q. .env faylini tekshiring.",
  "accountDisabled": "Hisob o‘chirilgan. Administratorga murojaat qiling.",

  "navDashboard": "Bosh sahifa",
  "navAssignments": "Topshiriqlar",
  "navMarks": "Baholar",
  "navAttendance": "Davomat",
  "navNotifications": "Bildirishnomalar",
  "navProfile": "Profil",
  "navUsers": "Foydalanuvchilar",
  "navGroups": "Guruhlar",
  "navAnalytics": "Tahlil",
  "navGradebook": "Jurnal",
  "navChat": "Chat",
  "navFocus": "Fokus",

  "focusTitle": "Fokus",
  "focusSubtitle": "Tinch o‘qish — bir vaqtda bitta topshiriq.",
  "focusTodaysFocus": "Bugungi fokus",
  "focusTodaysFocusSubtitle": "3 tagacha ochiq topshiriq — muddat muhim emas",
  "focusAlsoDue": "Yana topshiriqlar",
  "focusContinueHomework": "Uy ishini davom ettirish",
  "focusOpenAssignment": "Topshiriqni ochish",
  "focusEmptyTitle": "Hali topshiriq yo‘q",
  "focusEmptySubtitle": "O‘qituvchi topshiriq e’lon qilganda, shu yerda fokus sessiyasini
boshlashingiz mumkin.",
  "focusChooseMode": "Qanday fokususlanasiz?",
  "focusModePomodoro": "Pomodoro",
  "focusModePomodoroHint": "25 daqiqa, so‘ng tanaffus",
  "focusModeFree": "Erkin sessiya",
  "focusModeFreeHint": "To‘xtatgunicha hisoblash",
  "focusDoNotDisturb": "Chat o‘chirilgan",
  "focusChatMuteHint": "Sessiya davomida chat belgilari yashirinadi. Bildirishnomalar fonda
keladi.",
  "focusPause": "Pauza",
  "focusResume": "Davom etish",
  "focusPaused": "Pauza",
  "focusEndSession": "Tugatish",
  "focusSessionComplete": "Sessiya tugadi",
  "focusSessionCompleteSubtitle": "Yaxshi ish. Qisqa xulosa:",
  "focusTimeUp": "Vaqt tugadi!",
  "focusTimeUpSubtitle": "Pomodoro tugadi. Qisqa tanaffus oling, so‘ng davom eting.",
  "focusMinutesFocused": "Fokus: {duration}",
```

```
"focusOptionalNote": "Ixtiyoriy eslatma",
"focusOptionalNoteHint": "masalan, 3-vazifada qiyinchilik",
"focusSaveAndOpen": "Saqlash va topshiriqni ochish",
"focusDone": "Tayyor",

"chatTitle": "Chat",
"chatSearchHint": "Guruh yoki o‘qituvchi qidirish",
"chatEmptyTitle": "Hali chatlar yo‘q",
"chatEmptySubtitle": "Sinfga yozish uchun guruhga qo‘shiling.",
"chatNoMessagesYet": "Hali xabarlar yo‘q",
"chatMessageDeleted": "Xabar o‘chirildi",
"chatPreviewImage": "Rasm",
"chatPreviewVideo": "Video",
"chatVideoPlaybackSpeed": "Ijro tezligi",
"chatVideoSpeedNormal": "1x",
"chatVideoSpeedValue": "{speed}x",
"@chatVideoSpeedValue": {
  "placeholders": {
    "speed": { "type": "double", "format": "decimalPattern" }
  }
},
"chatVideoRewind10": "-10s",
"chatVideoForward10": "+10s",
"chatVideoCouldNotPlay": "Videoni ijro etib bo‘lmadi",
"chatPreviewVoice": "Ovozli xabar",
"chatPreviewFile": "Fayl",
"chatOpenFile": "Ochish",
"chatOpenFileFailed": "Faylni ochib bo‘lmadi.",
"chatYou": "Siz",
"chatTeacherMembers": "{teacher} · {count} a’zo",
"@chatTeacherMembers": {
  "placeholders": {
    "teacher": { "type": "String" },
    "count": { "type": "int" }
  }
},
"chatOnlineCount": "{count} onlayn",
"@chatOnlineCount": {
  "placeholders": {
    "count": { "type": "int" }
  }
},
"chatReact": "Reaksiya",
"chatCopyText": "Matnni nusxalash",
"chatCopied": "Nusxa olindi",
"chatReply": "Javob berish",
"chatReplyTo": "{name} ga javob",
"@chatReplyTo": {
  "placeholders": {
    "name": { "type": "String" }
  }
},
"chatReplyDeleted": "O‘chirilgan xabar",
"chatReplyNotFound": "Asl xabar topilmadi",
"chatMessageHint": "Xabar…",
"chatAttachImage": "Rasm",
"chatAttachVideo": "Video",
"chatAttachFile": "Fayl",
"chatTyping": "{names} yozmoqda…",
"@chatTyping": { "placeholders": { "names": { "type": "String" } } },
"chatRecording": "Yozilmoqda {time}",
"@chatRecording": { "placeholders": { "time": { "type": "String" } } },
"chatUploading": "{name} yuklanmoqda…",
"@chatUploading": { "placeholders": { "name": { "type": "String" } } },
"chatSendFailed": "Yuborib bo‘lmadi: {error}",
"@chatSendFailed": { "placeholders": { "error": { "type": "String" } } },
"chatMicDenied": "Ovozli xabar uchun mikrofon ruxsati kerak.",
"chatEditTitle": "Xabarni tahrirlash",
"chatDeleteTitle": "Xabar o‘chirilsinmi?",
"chatDeleteBody": "Xabar xonadagi barcha uchun o‘chiriladi.",
"chatDeleteSelectedTitle": "{count} ta xabar o‘chirilsinmi?",
"chatDeleteSelectedBody": "Tanlangan xabarlar xonadagi barcha uchun o‘chiriladi.",
"chatSelectedCount": "{count} ta tanlandi",
```

```
"chatSelectMessages": "Xabarlarni tanlash",
"chatSelect": "Tanlash",
"commonSelectAll": "Hammasini tanlash",
"chatEdited": "tahrir",
"chatDateToday": "Bugun",
"chatDateYesterday": "Kecha",
"chatRoleAdmin": "admin",
"chatRoleTeacher": "o‘qituvchi",

"commonSave": "Saqlash",
"commonCancel": "Bekor qilish",
"commonSubmit": "Yuborish",
"commonDelete": "O‘chirish",
"commonEdit": "Tahrirlash",
"commonClose": "Yopish",
"commonRetry": "Qayta urinish",
"commonTryAgain": "Qayta urinib ko‘ring",
"commonLoading": "Yuklanmoqda…",
"commonError": "Nimadir noto‘g‘ri ketdi",
"commonErrorWithDetails": "Xato: {error}",
"commonCouldNotSave": "Saqlab bo‘lmadi: {error}",
"commonSignOut": "Chiqish",
"commonSaveDraft": "Qoralamani saqlash",
"commonOpen": "Ochish",
"commonCreate": "Yaratish",
"commonAssign": "Biriktirish",
"commonReset": "Tiklash",
"commonAdd": "Qo‘shish",
"commonLoadMore": "Yana yuklash",
"commonExport": "Eksport",
"commonExportFailed": "Eksport bo‘lmadi: {error}",
"@commonExportFailed": { "placeholders": { "error": { "type": "String" } } },
"marksTermAll": "Butun davr",
"marksTermSemester": "{yearRange} · {semester}-semestr",
"@marksTermSemester": {
  "placeholders": {
    "yearRange": { "type": "String" },
    "semester": { "type": "int" }
  }
},
"marksExportCsv": "CSV eksport",
"marksExportDone": "Baholar eksport qilindi",
"commonTeacher": "O‘qituvchi",
"commonStudent": "O‘quvchi",
"commonAdmin": "Administrator",
"commonWorkspace": "Ish maydoni",
"commonNoDueDate": "Muddat yo‘q",
"commonDueDate": "Muddat {date}",
"homeworkDueThisWeek": "Shu hafta",
"homeworkDueThisWeekSubtitle": "Kechikkan va yaqin muddatlar",
"homeworkNoDueThisWeek": "Shu hafta muddat yo‘q",
"homeworkNoDueThisWeekSubtitle": "Keyingi hafta muddatlarigacha bo‘sh.",
"homeworkOverdue": "Muddati o‘tgan · {date}",
"@homeworkOverdue": { "placeholders": { "date": { "type": "String" } } },
"homeworkDueToday": "Bugun muddat",
"homeworkDueTomorrow": "Ertaga muddat",
"homeworkDueInDays": "{days} kundan keyin",
"@homeworkDueInDays": { "placeholders": { "days": { "type": "int" } } },
"homeworkLateBadge": "Kechikkan",
"commonProgress": "Jarayon",
"commonSubject": "Fan",
"commonType": "Tur",
"commonTitle": "Sarlavha",
"commonDescription": "Tavsif",
"commonGroup": "Guruh",
"commonMessage": "Xabar",
"commonScore": "Ball",
"commonFeedback": "Izoh",
"commonStudents": "O‘quvchilar",
"commonCannotUndo": "Bu amalni bekor qilib bo‘lmaydi.",

"profileTitle": "Profil",
"profileFullName": "To‘liq ism",
```

```
  "profileNotSignedIn": "Tizimga kiritilmagan",
  "profileAccount": "Hisob",
  "profileAccountSettings": "Hisob sozlamalari",
  "profileSignOutAdmin": "Admin konsolidan chiqish",
  "profileSettings": "Sozlamalar",
  "profileAppearance": "Ko‘rinish",
  "profileThemeLight": "Yorug‘",
  "profileThemeDark": "Qorong‘u",
  "profileThemeSystem": "Tizim",
  "profileStyle": "Uslub",
  "profileStyleModern": "Zamonaviy professional",
  "profileStyleNature": "Tabiat qog‘ozi",
  "profileStyleSoftNeu": "Neumorfizm",
  "profileStyleMonochrome": "Monoxrom ish maydoni",
  "profileEditName": "Ismni o‘zgartirish",
  "profileNameUpdated": "Ism yangilandi",
  "profileChangePhoto": "Rasmni o‘zgartirish",
  "profilePhotoUpdated": "Rasm yangilandi",
  "profileChangePassword": "Parolni o‘zgartirish",
  "profileCurrentPassword": "Joriy parol",
  "profileNewPassword": "Yangi parol",
  "profileConfirmPassword": "Yangi parolni tasdiqlang",
  "profilePasswordChanged": "Parol yangilandi",
  "profilePasswordMismatch": "Yangi parollar mos emas",
  "profilePasswordTooShort": "Kamida 8 ta belgi",
  "profilePasswordHint": "Parolni unutsangiz, admindan so‘rang.",
  "profileShortcuts": "Tezkor havolalar",
  "profileYourGroups": "Guruhlaringiz",
  "profileNoGroups": "Guruh biriktirilmagan",
  "profileGroupsCount": "{count} ta guruh",
  "@profileGroupsCount": { "placeholders": { "count": { "type": "int" } } },
  "profileAverageLabel": "Davr o‘rtachasi",
  "profileAttendanceLabel": "Davomat",
  "profileShareProgress": "Natija hisobotini ulashish",
  "profileNotifications": "Bildirishnomalar",
  "profileClearCache": "Keshni tozalash",
  "profileClearCacheWarn1Title": "Kesh tozalansinmi?",
  "profileClearCacheWarn1Body": "Oflayn chat xabarlari va yuklab olingan rasmlar, videolar,
ovozli xabarlar va fayllar o‘chiriladi. Ularni qayta yuklash uchun internet kerak bo‘ladi.",
  "profileClearCacheWarn2Title": "Ishonchingiz komilmi?",
  "profileClearCacheWarn2Body": "Bu amalni bekor qilib bo‘lmaydi. Chat keshi shu qurilmadan
o‘chiriladi.",
  "profileClearCacheConfirm": "Keshni tozalash",
  "profileClearCacheDone": "Kesh tozalandi",
  "profileAppVersion": "MU Study · v{version}",
  "@profileAppVersion": { "placeholders": { "version": { "type": "String" } } },
  "profileSubject": "Fan",

  "greetingMorning": "Xayrli tong",
  "greetingAfternoon": "Xayrli kun",
  "greetingEvening": "Xayrli kech",
  "greetingWithName": "{greeting}, {name}",
  "greetingLoadingTitle": "Xayrli tong, foydalanuvchi",
  "greetingLoadingSubtitle": "Shaxsiy salomlashuv yuklanmoqda...",
  "studentHomeSubtitle": "Sizning o‘quv maydoningiz",
  "continueLearning": "O‘qishni davom ettirish",
  "continueLearningSubtitle": "To‘xtagan joyingizdan davom eting",
  "everythingCompleted": "Hammasi bajarilgan.",
  "caughtUpToday": "Bugun hammasi tayyor.",
  "homeworkPack": "Uy vazifasi to‘plami",
  "staticTask": "Oddiy topshiriq",
  "staticAssignment": "Oddiy",
  "upcomingAssignments": "Yaqin topshiriqlar",
  "stayOnTrack": "Rejada qoling",
  "loadingAnnouncements": "E’lonlar yuklanmoqda...",
  "notificationsCardTitle": "Bildirishnomalar",
  "unreadAlerts": "Sizda {count} ta yangi bildirishnoma bor",
  "upToDate": "Hammasi yangilangan",

  "announcementsTitle": "E’lonlar",
  "announcementsSubtitle": "O‘qituvchilaringizdan",
  "announcementsEmpty": "E’lonlar yo‘q",
  "announcementsEmptyTeacher": "Muhim yangiliklarni o‘quvchilar bilan ulashing.",
```

```
  "announcementsEmptyAdmin": "Platforma bo‘ylab e’lonlar shu yerda paydo bo‘ladi.",
  "announcementsReadMore": "Batafsil",
  "announcementsShowLess": "Yig‘ish",
  "announcementsViewAll": "Barchasini ko‘rish ({count})",
  "announcementsAll": "Barcha e’lonlar",
  "announcementsNew": "Yangi e’lon",
  "announcementsPost": "E’lonni joylash",
  "announcementsEdit": "E’lonni tahrirlash",
  "announcementsDeleteTitle": "E’lon o‘chirilsinmi?",
  "announcementsDeleteBody": "U guruhdagi barcha a’zolar uchun o‘chiriladi.",
  "announcementsByAdmin": "Ma’muriyat",
  "announcementsShareUpdates": "Guruhlaringiz bilan yangiliklar ulashing",

  "assignmentsTitle": "Topshiriqlar",
  "assignmentsEmpty": "Topshiriqlar yo‘q",
  "assignmentsEmptySubtitle": "Hammasi bajarilgan!",
  "assignmentsEmptyTeacher": "Boshlash uchun birinchi topshiriqni yarating.",
  "assignmentsLearningPath": "O‘quv yo‘li",
  "assignmentsActive": "Faol topshiriqlar",
  "assignmentDetails": "Topshiriq tafsilotlari",
  "assignmentNotFound": "Topshiriq topilmadi",
  "assignmentTasks": "Vazifalar",
  "assignmentTasksSubtitle": "To‘plamni yakunlash uchun hammasini bajaring",
  "assignmentNew": "Yangi topshiriq",
  "assignmentManage": "Topshiriqlarni boshqarish",
  "assignmentManageSubtitle": "Oddiy yoki uy vazifasi to‘plamlarini yarating",

  "marksTitle": "Baholarim",
  "marksOverallAverage": "Umumiy o‘rtacha",
  "marksNoneYet": "Hali baholar yo‘q.",
  "marksBasedOn": "{label} · {count} ta baho asosida.",
  "marksByCategory": "Turkumlar bo‘yicha",
  "marksWhereStrongest": "Eng kuchli tomonlaringiz",
  "marksAll": "Barcha baholar",
  "marksNewestFirst": "Avval yangilari",
  "marksEmptyTitle": "Hali baholar yo‘q",
  "marksEmptySubtitle": "O‘qituvchi baholari shu yerda ko‘rinadi.",
  "progressReportTitle": "Natija hisoboti",
  "progressReportShare": "Natijani ulashish",
  "progressReportSharing": "Hisobot tayyorlanmoqda…",
  "progressReportDone": "Hisobot ulashishga tayyor",
  "progressReportFailed": "Ulashib bo‘lmadi: {error}",
  "@progressReportFailed": {
    "placeholders": {
      "error": { "type": "String" }
    }
  },
  "progressReportNeedGroup": "Hisobot ulashish uchun guruhga qo‘shiling.",
  "progressReportAttendance": "Davomat",
  "progressReportAttendanceRate": "Davomat foizi",
  "progressReportNoAttendance": "Bu muddat uchun davomat yozilmagan.",
  "progressReportSessions": "mashg‘ulot",
  "progressReportMarksCounted": "baho",
  "progressReportRecentMarks": "So‘nggi baholar",
```

**lib/l10n/app_uz.arb — continued (2/3)**

```
  "progressReportGeneratedOn": "Yaratilgan: {date}",
  "@progressReportGeneratedOn": {
    "placeholders": {
      "date": { "type": "String" }
    }
  },
  "marksNoRecorded": "Baholar yozilmagan",
  "marksNoRecordedSubtitle": "Birdaniga baho qo‘yish uchun «Sinfni baholash»dan foydalaning.",
  "marksRecent": "So‘nggi baholar",
  "marksRecentSubtitle": "Guruh bo‘yicha oxirgi yozuvlar",
  "marksStudentsHeader": "O‘quvchilar",
  "marksStudentsSubtitle": "Barcha baholarning o‘rtacha og‘irlikli bali",
  "marksNoStudents": "Bu guruhda o‘quvchilar yo‘q",
  "marksNoStudentsSubtitle": "Administrator guruhingizga o‘quvchilarni biriktirishi mumkin.",
  "marksDeleteTitle": "Bahoni o‘chirish?",
  "marksDeleteBody": "«{title}» butunlay o‘chiriladi.",
```

```
"marksGradeTheClass": "Sinfni baholash",
"marksOnePercentEach": "Har bir o‘quvchiga bitta foizli baho",
"marksSaveMarks": "Baholarni saqlash",
"marksSaveMark": "Bahoni saqlash",
"marksAddMark": "Baho qo‘shish",
"marksNothingYet": "Hali hech narsa yo‘q",
"marksAddFirst": "Bu o‘quvchiga birinchi bahoni qo‘shing.",
"marksSavedCount": "{count} ta baho saqlandi",
"marksGroupHasNoStudents": "Bu guruhda hali o‘quvchilar yo‘q.",
"marksTitleHint": "masalan, 3-mavzu testi",
"marksAverageLine": "O‘rtacha {percent} · {label}",
"marksNoMarksYetShort": "Hali baholar yo‘q",
"marksSummaryStudents": "O‘quvchilar",
"marksSummaryMarks": "Baholar",

"attendanceTitle": "Davomat",
"attendanceEmpty": "Davomat yozuvlari yo‘q",
"attendanceNoGroup": "Guruh hali biriktirilmagan",
"attendanceMark": "Davomatni belgilash",
"attendanceDailyTracking": "Kunlik o‘quvchi hisobi",
"attendanceSaved": "Davomat muvaffaqiyatli saqlandi",
"attendanceNoGroups": "Guruhlar topilmadi",
"attendancePresent": "B — Bor",
"attendanceLate": "K — Kechikdi",
"attendanceAbsent": "Y — Yo‘q",
"attendanceExcused": "S — Sababli",
"attendancePresentShort": "B",
"attendanceLateShort": "K",
"attendanceAbsentShort": "Y",
"attendanceExcusedShort": "S",
"attendanceTrend": "Davomat dinamikasi",
"attendanceOverall": "Umumiy taqsimot",
"analyticsThisWeek": "Shu hafta",
"analyticsLastWeek": "O‘tgan hafta",
"analyticsTotalMarks": "Jami baholar: {count}",
"@analyticsTotalMarks": {
  "placeholders": {
    "count": { "type": "int" }
  }
},

"notificationsTitle": "Bildirishnomalar",
"notificationsEmpty": "Hammasi o‘qilgan!",
"notificationsEmptySubtitle": "Yangi bildirishnomalar yo‘q.",
"notificationsNone": "Bildirishnomalar yo‘q",
"notifKindAssignment": "Yangi uy vazifasi",
"notifKindAnnouncement": "E’lon",
"notifKindGraded": "Ish baholandi",
"notifKindSubmission": "Yangi topshiriq",
"notifKindChat": "Chat xabari",
"notifKindSystem": "Yangilanish",
"notificationsOpen": "Ochish",

"taskTitle": "Vazifa",
"taskYourAnswer": "Javobingiz",
"taskMarkCompleted": "Bajarildi deb belgilash",
"taskAttachment": "Ilova",
"taskAttachmentHint": "Javobingizga rasm yoki hujjat qo‘shing.",
"taskAttachmentHintReady": "Ochish uchun bosing · o‘chirish uchun axlat qutisi.",
"taskImage": "Rasm",
"taskImageSubtitle": "Galereya fotosurati",
"taskFile": "Fayl",
"taskFileSubtitle": "PDF yoki hujjat",
"taskReplaceImage": "Rasm bilan almashtirish",
"taskReplaceFile": "Faylni almashtirish",
"taskVoice": "Ovozli yozuv",
"taskVoiceHint": "Ixtiyoriy og‘zaki javob — yozing, tinglang, so‘ng saqlang.",
"taskViewPdf": "PDFni ko‘rish",
"taskOpenLink": "Havolani ochish",
"taskSubmitted": "Yuborildi",
"taskSaved": "Saqlandi",
"taskUploadFailed": "Yuklash muvaffaqiyatsiz: {error}",
"taskVoiceUploadFailed": "Ovozni yuklab bo‘lmadi: {error}",
```

```
"taskCouldNotSave": "Saqlab bo‘lmadi: {error}",

"voiceTapToRecord": "Javobingizni yozish uchun mikrofonni bosing",
"voiceRecording": "Yozilmoqda…",
"voiceNote": "Ovozli eslatma",
"voiceTapPlay": "Tinglash uchun ijro etishni bosing",
"voiceRerecord": "Qayta yozish",
"voiceStudentAnswer": "O‘quvchining ovozli javobi",
"voiceMicPermission": "Yozish uchun mikrofon ruxsati kerak.",
"voiceCouldNotStart": "Yozuvni boshlab bo‘lmadi: {error}",
"voiceCouldNotSave": "Yozuvni saqlab bo‘lmadi: {error}",
"voiceCouldNotPlay": "Ovozni ijro etib bo‘lmadi: {error}",
"attachmentCouldNotOpen": "Bu faylni ochib bo‘lmadi.",
"attachmentImage": "Rasm ilovasi",
"attachmentFile": "Fayl ilovasi",
"attachmentPdf": "PDF hujjat",
"attachmentTapPreview": "Ko‘rish uchun bosing",
"attachmentTapOpen": "Ochish uchun bosing",
"pdfDefaultTitle": "PDF",

"teacherWorkspace": "O‘qituvchi ish maydoni",
"teacherQuickActions": "Tezkor amallar",
"teacherCreateItems": "Yangi elementlar yaratish",
"teacherNewAssignment": "Yangi topshiriq",
"teacherNewAnnouncement": "Yangi e’lon",
"teacherSubmissions": "Ishlar",
"teacherNoSubmissions": "Hali ishlar yo‘q",
"teacherNoSubmissionsSubtitle": "O‘quvchi ishlari baholash uchun shu yerda ko‘rinadi.",
"teacherGradebook": "Jurnal",
"teacherGradeClass": "Sinfni baholash",
"teacherNoGroups": "Hali guruhlar yo‘q",
"teacherNoGroupsSubtitle": "Administratordan guruh biriktirishini so‘rang.",
"teacherActiveGroups": "FAOL GURUHLAR",
"teacherSubject": "FAN",
"teacherDailyTasks": "Kunlik vazifalar",
"teacherNeedsAttention": "Diqqatingizni talab qiladiganlar",
"teacherReviewSubmissions": "Ishlarni tekshirish",
"teacherCheckAssignments": "O‘quvchi topshiriqlarini tekshirish",
"teacherRecordMarks": "Foizli baholar qo‘yish",
"teacherPerformanceAnalytics": "Natijalar tahlili",
"teacherTrackProgress": "Guruh jarayonini kuzatish",
"teacherOpenAttachment": "Ilovani ochish",
"teacherScoreHelper": "Uy vazifasi bahosini jurnalga qo‘shadi",
"teacherScoreRange": "Ball 0 dan 100 gacha bo‘lishi kerak.",
"teacherNoWrittenAnswer": "Yozma javob yo‘q.",
"teacherReviewSubmission": "Ishni tekshirish",
"teacherHomeworkPackTasks": "Uy vazifasi to‘plami vazifalari",
"teacherAddTask": "Vazifa qo‘shish",
"teacherEditTask": "Vazifani tahrirlash",
"teacherDeleteTask": "Vazifani o‘chirish?",
"teacherTasksContent": "Vazifalar mazmuni",
"teacherManagePack": "Bu to‘plamdagi elementlarni boshqaring",
"teacherTaskTitleHint": "masalan, O‘qish mashqi",
"teacherLinkHint": "https://...",
"teacherAttachmentHint": "Rasm yoki fayl havolasi...",
"teacherNoData": "Ma’lumot yo‘q",
"teacherMarkDistribution": "Baholar taqsimoti",
"teacherStudentsByAverage": "O‘quvchilar o‘rtacha ball bo‘yicha",
"teacherNoMarksTrends": "Tendensiyalarni ko‘rish uchun jurnalga baho qo‘shing.",
"teacherTargetGroup": "Maqsadli guruh",

"adminWorkspace": "Platformani boshqarish",
"adminUsers": "Foydalanuvchilar",
"adminGroups": "Guruhlar",
"adminAnalytics": "Tahlil",
"adminTeachers": "O‘QITUVCHILAR",
"adminStudents": "O‘QUVCHILAR",
"adminGroupsLabel": "GURUHLAR",
"adminMarksLabel": "BAHOLAR",
"adminActivityInsights": "Faoliyat tahlili",
"adminCurrentMetrics": "Joriy ko‘rsatkichlar",
"adminCreateUser": "Yangi foydalanuvchi",
"adminCreateUserButton": "Foydalanuvchi yaratish",
```

```
  "adminUserCreated": "Foydalanuvchi muvaffaqiyatli yaratildi",
  "adminResetPassword": "Parolni tiklash",
  "adminResetPasswordTitle": "Parolni tiklash · {username}",
  "adminCreateGroup": "Yangi guruh",
  "adminCreateGroupButton": "Guruh yaratish",
  "adminGroupTitleHint": "masalan, 4-guruh",
  "adminGroupTitleExists": "Bu o‘qituvchida shu nomdagi guruh allaqachon bor",
  "adminAssignStudent": "O‘quvchini biriktirish",
  "adminActivityDistribution": "Faoliyat taqsimoti",
  "adminGlobalMetrics": "Global platforma ko‘rsatkichlari",
  "adminActivityLog": "Admin faoliyat jurnali",
  "adminActivityLogSubtitle": "Tizim voqealari va audit yozuvlari shu yerda paydo bo‘ladi.",
  "adminTeacherLine": "O‘qituvchi: {name}",
  "adminRoleLine": "{role} · {username}",
  "adminDisable": "O‘chirish",
  "adminEnable": "Yoqish",
  "adminPassword": "Parol",
  "adminFullName": "To‘liq ism",
  "adminRole": "Rol",
  "adminSphere": "Soha",

  "percentExcellent": "A’lo",
  "percentGood": "Yaxshi",
  "percentNeedsWork": "Yaxshilash kerak",
  "percentAtRisk": "Xavf ostida",
  "emptyStateDefault": "Hali hech narsa yo‘q",
  "setupRequiredTitle": "Sozlash kerak",
  "setupRequiredBody": "Supabase sozlanmagan. .env fayliga SUPABASE_URL va SUPABASE_ANON_KEY
qo‘shing.",
  "setupOption1": "1-variant — mahalliy .env fayli",
  "setupOption2": "2-variant — dart-define",

  "attendanceAllPresent": "Hammasi B",
  "attendanceAllAbsent": "Hammasi Y",
  "attendanceAllLate": "Hammasi K",
  "attendanceAllExcused": "Hammasi S",
  "attendancePresentWord": "Bor",
  "attendanceAbsentWord": "Yo‘q",
  "attendanceLateWord": "Kechikdi",
  "attendanceExcusedWord": "Sababli",
  "attendanceWeekdayMon": "Du",
  "attendanceWeekdayTue": "Se",
  "attendanceWeekdayWed": "Ch",
  "attendanceWeekdayThu": "Pa",
  "attendanceWeekdayFri": "Ju",
  "attendancePercentLabel": "Foiz",
  "attendanceNoLesson": "Dars yo‘q",
  "attendanceHistory": "Davomat tarixi",
  "attendanceSelectStudent": "O‘quvchi",
  "attendanceAllStudents": "Barcha o‘quvchilar",
  "attendanceTapToCycle": "Statusni o‘zgartirish uchun kunga bosing",
  "attendanceUpdated": "Davomat yangilandi",
  "attendanceCalendarEmpty": "Bu oyda davomat yo‘q",
  "attendanceFutureNotAllowed": "Kelajakdagi kun uchun davomat belgilab bo‘lmaydi",
  "attendanceWeekendNotAllowed": "Davomat faqat ish kunlari (du–ju) uchun belgilanadi",

  "chartBandExcellent": "85–100%",
  "chartBandGood": "70–84%",
  "chartBandNeedsWork": "50–69%",
  "chartBandAtRisk": "50% dan past",

  "marksWeight": "Og‘irlik",
  "marksCategory": "Turkum",
  "marksDate": "Sana",
  "marksCommentOptional": "Izoh (ixtiyoriy)",
  "marksNewMark": "Yangi baho",
  "marksEditMark": "Bahoni tahrirlash",
  "marksFromSubmission": "Tekshirilgan ishdan",
  "marksNeedTitle": "Baholanadigan element uchun sarlavha kiriting.",
  "marksWeightPositive": "Og‘irlik noldan katta bo‘lishi kerak.",
  "marksNeedAtLeastOne": "Kamida bitta o‘quvchiga baho kiriting.",
  "marksLeaveBlankSkip": "O‘quvchini o‘tkazib yuborish uchun maydonni bo‘sh qoldiring.",
  "marksFromSubmissionDelete": "Bu baho tekshirilgan ishdan. Uni o‘chirish uchun «Ishlar»
```

```
bo‘limida ballni tozalang.",
  "marksFromSubmissionEdit": "Bu baho tekshirilgan ishga bog‘langan. Bu yerda tahrirlash
keyingi tekshiruvda qayta yoziladi.",

  "categoryHomework": "Uy vazifasi",
  "categoryClasswork": "Sinf ishi",
  "categoryQuiz": "Viktorina",
  "categoryTest": "Test",
  "categoryExam": "Imtihon",
  "categoryProject": "Loyiha",
  "categoryOral": "Og‘zaki",
  "categoryOther": "Boshqa",

  "teacherReferenceLinkOptional": "Havola (ixtiyoriy)",
  "teacherAttachmentUrlOptional": "Ilova URL (ixtiyoriy)",
  "teacherNoTasksYet": "Hali vazifalar yo‘q. Boshlash uchun «Vazifa qo‘shish»ni bosing.",

  "teacherTemplates": "Shablonlar",
  "teacherTemplatesEmpty": "Hali shablonlar yo‘q",
  "teacherTemplatesEmptySubtitle": "Qayta ishlatish uchun topshiriqni shablon sifatida saqlang.
",
  "teacherSaveAsTemplate": "Shablon sifatida saqlash",
  "teacherTemplateSaved": "Shablon saqlandi",
  "teacherUseTemplate": "Shablondan foydalanish",
  "teacherFromTemplate": "Shablondan",
  "teacherDeleteTemplate": "Shablonni o‘chirish?",
  "teacherDeleteTemplateBody": "«{title}» o‘chiriladi.",
  "@teacherDeleteTemplateBody": {
    "placeholders": {
      "title": { "type": "String" }
    }
  },
  "teacherDuplicateAssignment": "Nusxa olish",
  "teacherDuplicateTitle": "Topshiriqdan nusxa",
  "teacherDuplicated": "Topshiriq nusxalandi",
  "assignmentEdit": "Topshiriqni tahrirlash",
  "assignmentDeleteTitle": "Topshiriq o‘chirilsinmi?",
  "assignmentDeleteBody": "Topshiriq, vazifalar va o‘quvchi topshiriqlari butunlay o‘chiriladi.
",
  "assignmentDeleted": "Topshiriq o‘chirildi",
  "assignmentTypeHintStatic": "Bitta ish. O‘quvchi bir marta topshiradi — qo‘shimcha vazifalar
yo‘q.",
  "assignmentTypeHintPack": "Bir nechta vazifali to‘plam. Keyin bandlarni qo‘shasiz.",
  "teacherCreateFromTemplate": "Shablondan yaratish",
  "teacherTasksCopied": "{count} ta vazifa kiritiladi",
  "@teacherTasksCopied": {
    "placeholders": {
      "count": { "type": "int" }
    }
  },
  "teacherRecentFeedback": "Fikrni qayta ishlatish",
  "teacherApplyFeedbackToOthers": "Shu vazifa bo‘yicha boshqa kutayotganlarga ham qo‘llash",
  "teacherFeedbackAppliedCount": "Fikr yana {count} ta topshiriqqa qo‘llandi",
  "@teacherFeedbackAppliedCount": {
    "placeholders": {
      "count": { "type": "int" }
    }
  },
  "teacherSharedComment": "Umumiy izoh (ixtiyoriy)",
  "teacherSharedCommentHint": "Saqlanadigan har bir bahoga bir xil izoh",

  "adminPlatformOverview": "Platforma umumiy ko‘rinishi",
  "adminAssignTeacher": "O‘qituvchini biriktirish",
  "adminSelectStudent": "O‘quvchini tanlash",
  "adminInitialGroupOptional": "Boshlang‘ich guruh (ixtiyoriy)",
  "adminChartTasks": "Vazifalar",
  "adminChartGroups": "Guruhlar",
  "adminChartStaff": "Xodimlar",
  "adminChartAssignments": "Topshiriqlar",
  "adminChartMarks": "Baholar",
  "adminChartStudents": "O‘quvchilar",

  "adminAcademicYears": "O‘quv yillari",
```

```
  "adminAcademicYearsSubtitle": "Baholar jurnali uchun yillar va semestrlar",
  "adminAcademicYearsTitle": "O‘quv kalendari",
  "adminCreateNextYear": "Keyingi yilni yaratish",
  "adminYearCreated": "{label} yaratildi",
  "@adminYearCreated": { "placeholders": { "label": { "type": "String" } } },
  "adminSetCurrentYear": "Joriy qilish",
  "adminCurrentYearBadge": "Joriy",
  "adminYearSemesters": "To‘rt semestr · avg → iyn",
  "adminSemesterRange": "{semester}-semestr · {start} – {end}",
  "@adminSemesterRange": {
    "placeholders": {
      "semester": { "type": "int" },
      "start": { "type": "String" },
      "end": { "type": "String" }
    }
  },
  "adminEditTerm": "Muddatni tahrirlash",
  "adminEditTermTitle": "{semester}-semestr",
  "@adminEditTermTitle": {
    "placeholders": {
      "semester": { "type": "int" }
    }
  },
  "adminTermStartsOn": "Boshlanishi",
```

**lib/l10n/app_uz.arb — continued (3/3)**

```
  "adminTermEndsOn": "Tugashi",
  "adminTermDatesInvalid": "Tugash sanasi boshlanish sanasidan oldin bo‘lmasligi kerak",
  "adminTermUpdated": "Semestr sanalari yangilandi",
  "adminDeleteYear": "Yilni o‘chirish",
  "adminDeleteYearTitle": "{label} ni o‘chirasizmi?",
  "@adminDeleteYearTitle": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearBody": "Yil va to‘rt semestr kalendardan o‘chiriladi. Baholar o‘chirilmaydi.
Tasdiqlash uchun pastda {label} ni yozing.",
  "@adminDeleteYearBody": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearConfirmHint": "Tasdiqlash uchun {label} ni yozing",
  "@adminDeleteYearConfirmHint": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminDeleteYearCurrentBlocked": "Avval boshqa yilni joriy qiling, keyin buni o‘chiring.",
  "adminYearDeleted": "{label} o‘chirildi",
  "@adminYearDeleted": {
    "placeholders": {
      "label": { "type": "String" }
    }
  },
  "adminNoAcademicYears": "Hali o‘quv yillari yo‘q",
  "adminAcademicYearsHint": "Tayyor bo‘lganda keyingi o‘quv yilini yarating. O‘qituvchilar va
o‘quvchilar jurnalda semestrni tanlaydi. Sanalarni o‘zgartirish uchun semestrni bosing.",

  "adminCredentialPrintTitle": "Login qog‘ozlari",
  "adminCredentialPrintSubtitle": "Bazadan unique loginlar, sahifada 3 ta karta",
  "adminCredentialPasteLabel": "Foydalanuvchilar",
  "adminCredentialPasteHint": "username|password|full name\n@james|Abc123xyz|James
English|teacher\n@john_jones|Pass456|John Jones",
  "adminCredentialPerPage": "Sahifadagi kartalar",
  "adminCredentialTwo": "2",
  "adminCredentialThree": "3",
  "adminCredentialExportPdf": "PDF eksport",
  "adminCredentialSlipTitle": "MU Study login",
  "adminCredentialSlipHint": "Bu qog‘ozni o‘zingizda saqlang. Birinchi kirishdan keyin parolni
o‘zgartiring.",
```

```
  "adminCredentialNeedUsers": "Chop etish uchun foydalanuvchi yo‘q",
  "adminCredentialNeedSql": "Supabase SQL Editor’da sql/admin_login_slips.sql ni ishga
tushiring, keyin qayta eksport qiling.",
  "adminCredentialConfidential": "Maxfiy. Bu ma’lumotlar faqat belgilangan o‘quvchi uchun.",
  "adminFirstName": "Ism",
  "adminLastName": "Familiya",
  "adminCredentialStudents": "O‘quvchilar",
  "adminCredentialTeachers": "O‘qituvchilar",
  "adminCredentialAllGroups": "Barcha guruhlar",

  "sphereMath": "Matematika",
  "sphereEnglish": "Ingliz tili",

  "dailyMotivation1": "Bugun oldinga siljish uchun ajoyib kun.",
  "dailyMotivation2": "To‘xtagan joyingizdan davom etishga tayyormisiz?",
  "dailyMotivation3": "Bugun maqsadlaringizni zabt etaylik.",
  "dailyMotivation4": "Izchillik — mahorat kaliti."
}
```

# Appendix I — Bootstrapping the first admin

> **WARNING — Replace the password**
> The script uses YOUR_STRONG_PASSWORD as a placeholder. Never leave password123 on a reachable project.

**first_admin_bootstrap.sql**

```
-- First admin bootstrap (EXACT pattern mirroring admin_create_user).
-- Run in Supabase SQL Editor AFTER any app user exists.
-- Replace YOUR_STRONG_PASSWORD. Username example: @admin

DO $$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_username TEXT := '@admin';
  v_password TEXT := 'YOUR_STRONG_PASSWORD';
  v_full_name TEXT := 'System Admin';
  v_email TEXT;
BEGIN
  v_email := lower(trim(replace(v_username, '@', ''))) || '@mustudy.internal';

  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000',
    v_user_id,
    'authenticated',
    'authenticated',
    v_email,
    extensions.crypt(v_password, extensions.gen_salt('bf')),
    NOW(),
    jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),
    jsonb_build_object('username', v_username, 'full_name', v_full_name, 'role', 'admin'),
    NOW(), NOW(), '', '', '', '', false, false
  );

  INSERT INTO auth.identities (
    id, user_id, identity_data, provider, provider_id,
    last_sign_in_at, created_at, updated_at
  ) VALUES (
    gen_random_uuid(),
    v_user_id,
    jsonb_build_object('sub', v_user_id::text, 'email', v_email, 'email_verified', true),
    'email',
    v_email,
    NOW(), NOW(), NOW()
  );

  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(v_username), v_full_name, 'admin', v_email);

  RAISE NOTICE 'Created admin id=% email=% username=%', v_user_id, v_email, v_username;
END $$;

-- VERIFY:
-- SELECT id, username, role, auth_email, is_disabled FROM profiles WHERE role = 'admin';
-- Then login in the app with username @admin and YOUR_STRONG_PASSWORD.
```

## optional_login_credentials.sql

**optional_login_credentials.sql**

```
-- Admin printable login slips.
-- Stores the password that was set at create/reset time (Auth only keeps a hash).
```

```
CREATE TABLE IF NOT EXISTS public.login_credentials (
  profile_id uuid PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
  password text NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now()
);

ALTER TABLE public.login_credentials ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON TABLE public.login_credentials FROM PUBLIC, anon, authenticated;

CREATE OR REPLACE FUNCTION public.admin_save_login_password(
  p_user_id uuid,
  p_password text
) RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'not allowed';
  END IF;

  INSERT INTO public.login_credentials (profile_id, password, updated_at)
  VALUES (p_user_id, p_password, now())
  ON CONFLICT (profile_id) DO UPDATE
    SET password = excluded.password,
        updated_at = now();
END;
$$;

CREATE OR REPLACE FUNCTION public.admin_list_login_slips(
  p_role public.user_role DEFAULT 'student',
  p_group_id uuid DEFAULT NULL
)
RETURNS TABLE (
  profile_id uuid,
  username text,
  full_name text,
  password text
)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth, extensions
AS $$
DECLARE
  r record;
  v_password text;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'not allowed';
  END IF;

  FOR r IN
    SELECT * FROM (
      SELECT DISTINCT ON (p.id)
        p.id,
        p.username,
        coalesce(nullif(trim(p.full_name), ''), p.username) AS full_name,
        c.password AS saved_password
      FROM public.profiles p
      LEFT JOIN public.login_credentials c ON c.profile_id = p.id
      LEFT JOIN public.students s ON s.profile_id = p.id
      LEFT JOIN public.group_students gs ON gs.student_id = s.id
      WHERE p.role = p_role
        AND coalesce(p.is_disabled, false) = false
        AND (
          p_group_id IS NULL
          OR gs.group_id = p_group_id
        )
      ORDER BY p.id
    ) unique_profiles
    ORDER BY unique_profiles.full_name, unique_profiles.username
  LOOP
```

```
    v_password := r.saved_password;
    IF v_password IS NULL OR v_password = '' THEN
      LOOP
        v_password := substr(replace(gen_random_uuid()::text, '-', ''), 1, 16);
        EXIT WHEN NOT EXISTS (
          SELECT 1 FROM public.login_credentials WHERE password = v_password
        );
      END LOOP;

      UPDATE auth.users
      SET encrypted_password = extensions.crypt(v_password, extensions.gen_salt('bf')),
          updated_at = now()
      WHERE id = r.id;

      INSERT INTO public.login_credentials (profile_id, password, updated_at)
      VALUES (r.id, v_password, now())
      ON CONFLICT (profile_id) DO UPDATE
        SET password = excluded.password,
            updated_at = now();
    END IF;

    profile_id := r.id;
    username := r.username;
    full_name := r.full_name;
    password := v_password;
    RETURN NEXT;
  END LOOP;
END;
$$;

REVOKE ALL ON FUNCTION public.admin_save_login_password(uuid, text) FROM PUBLIC;
REVOKE ALL ON FUNCTION public.admin_list_login_slips(public.user_role, uuid) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION public.admin_save_login_password(uuid, text) TO authenticated;
GRANT EXECUTE ON FUNCTION public.admin_list_login_slips(public.user_role, uuid) TO
authenticated;
```

## optional_fix_admin_create_user.sql

**optional_fix_admin_create_user.sql**

```
-- Stop auto-creating Group 1/2/3 when a teacher is created.
-- Admins create groups with exact titles from the Groups screen.
-- Also prevent duplicate (teacher_id, title) rows.

CREATE OR REPLACE FUNCTION public.admin_create_user(
  p_username TEXT,
  p_password TEXT,
  p_full_name TEXT,
  p_role public.user_role,
  p_sphere public.sphere_type DEFAULT NULL
) RETURNS UUID
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public, auth, extensions
AS $$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Not authorized';
  END IF;

  v_email := lower(trim(replace(p_username, '@', ''))) || '@mustudy.internal';

  INSERT INTO auth.users (
    instance_id, id, aud, role, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    confirmation_token, recovery_token, email_change_token_new, email_change,
    is_sso_user, is_anonymous
  ) VALUES (
    '00000000-0000-0000-0000-000000000000',
```

```
    v_user_id,
    'authenticated',
    'authenticated',
    v_email,
    extensions.crypt(p_password, extensions.gen_salt('bf')),
    NOW(),
    jsonb_build_object('provider', 'email', 'providers', ARRAY['email']),
    jsonb_build_object(
      'username', p_username,
      'full_name', p_full_name,
      'role', p_role::text
    ),
    NOW(),
    NOW(),
    '',
    '',
    '',
    '',
    false,
    false
  );

  INSERT INTO auth.identities (
    id, user_id, identity_data, provider, provider_id,
    last_sign_in_at, created_at, updated_at
  ) VALUES (
    gen_random_uuid(),
    v_user_id,
    jsonb_build_object(
      'sub', v_user_id::text,
      'email', v_email,
      'email_verified', true
    ),
    'email',
    v_email,
    NOW(),
    NOW(),
    NOW()
  );

  INSERT INTO public.profiles (id, username, full_name, role, auth_email)
  VALUES (v_user_id, trim(p_username), p_full_name, p_role, v_email);

  IF EXISTS (
    SELECT 1
    FROM information_schema.tables
    WHERE table_schema = 'public' AND table_name = 'login_credentials'
  ) THEN
    INSERT INTO public.login_credentials (profile_id, password, updated_at)
    VALUES (v_user_id, p_password, NOW())
    ON CONFLICT (profile_id) DO UPDATE
      SET password = excluded.password,
          updated_at = NOW();
  END IF;

  IF p_role = 'teacher' THEN
    IF p_sphere IS NULL THEN
      RAISE EXCEPTION 'Sphere required for teacher';
    END IF;
    -- Do NOT auto-create Group 1/2/3. Admins add groups manually.
    INSERT INTO public.teachers (profile_id, sphere)
    VALUES (v_user_id, p_sphere);
  ELSIF p_role = 'student' THEN
    INSERT INTO public.students (profile_id) VALUES (v_user_id);
  END IF;

  RETURN v_user_id;
END;
$$;

GRANT EXECUTE ON FUNCTION public.admin_create_user(
  TEXT, TEXT, TEXT, public.user_role, public.sphere_type
) TO authenticated;
```

```
-- Deduplicate same title for the same teacher (keep oldest).
DELETE FROM public.groups g
USING public.groups newer
WHERE g.teacher_id = newer.teacher_id
  AND g.title = newer.title
  AND g.created_at > newer.created_at;

-- One title per teacher.
CREATE UNIQUE INDEX IF NOT EXISTS groups_teacher_title_uidx
  ON public.groups (teacher_id, title);

-- Remove empty auto-groups named Group 1/2/3 with no students.
DELETE FROM public.groups g
WHERE g.title ~ '^Group [123]$'
  AND NOT EXISTS (
    SELECT 1 FROM public.group_students gs WHERE gs.group_id = g.id
  );
```

**CHECKPOINT — Admin exists**
SELECT username, role, auth_email FROM profiles WHERE role = 'admin'; Then log in from the app.

# Appendix J — Config files

## pubspec.yaml

**pubspec.yaml**

```
name: mu_study
description: MU Study — School Management & Study Platform
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: ^3.11.5

dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter
  cupertino_icons: ^1.0.8
  supabase_flutter: ^2.9.1
  flutter_riverpod: ^2.6.1
  go_router: ^14.8.1
  freezed_annotation: ^2.4.4
  json_annotation: ^4.9.0
  shared_preferences: ^2.5.3
  responsive_framework: ^1.5.1
  flutter_animate: ^4.5.2
  cached_network_image: ^3.4.1
  shimmer: ^3.0.0
  skeletonizer: ^2.0.1
  file_picker: ^8.3.7
  image_picker: ^1.1.2
  syncfusion_flutter_pdfviewer: ^30.1.41
  syncfusion_flutter_charts: ^30.1.41
  intl: ^0.20.2
  url_launcher: ^6.3.1
  google_fonts: ^6.2.1
  flutter_dotenv: ^5.2.1
  record: ^6.2.1
  audioplayers: ^6.7.1
  path_provider: ^2.1.6
  path: ^1.9.1
  cross_file: ^0.3.5+4
  share_plus: ^11.0.0
  video_player: ^2.11.1
  http: ^1.6.0
  crypto: ^3.0.7
  syncfusion_flutter_pdf: ^30.1.41
  package_info_plus: ^9.0.1
  lottie: ^3.3.3
  characters: ^1.4.1
  flutter_cache_manager: ^3.4.2
  get_thumbnail_video: ^0.7.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0
  build_runner: ^2.4.15
  freezed: ^2.5.8
  json_serializable: ^6.9.5

dependency_overrides:
  audioplayers_darwin: 6.3.0

flutter:
  generate: true
  uses-material-design: true
  assets:
    - assets/images/
```

```
    - assets/images/logo/
    - assets/images/nature-paper/
    - assets/lottie/
    - .env
```

## l10n.yaml

**l10n.yaml**

```
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
output-class: AppLocalizations
nullable-getter: false
```

## Makefile

**Makefile**

```
run:
  flutter run
clean:
  flutter clean
get:
  flutter pub get
ch:
  flutter run -d Chrome

apk:
  flutter build apk --release
  cp build/app/outputs/flutter-apk/app-release.apk build/app/outputs/flutter-apk/mu_study.apk
  @echo "Built: build/app/outputs/flutter-apk/mu_study.apk"
```

## analysis_options.yaml

**analysis_options.yaml**

```
# This file configures the analyzer, which statically analyzes Dart code to
# check for errors, warnings, and lints.
#
# The issues identified by the analyzer are surfaced in the UI of Dart-enabled
# IDEs (https://dart.dev/tools#ides-and-editors). The analyzer can also be
# invoked from the command line by running `flutter analyze`.

# The following line activates a set of recommended lints for Flutter apps,
# packages, and plugins designed to encourage good coding practices.
include: package:flutter_lints/flutter.yaml

linter:
  # The lint rules applied to this project can be customized in the
  # section below to disable rules from the `package:flutter_lints/flutter.yaml`
  # included above or to enable additional rules. A list of all available lints
  # and their documentation is published at https://dart.dev/lints.
  #
  # Instead of disabling a lint rule for the entire project in the
  # section below, it can also be suppressed for a single line of code
  # or a specific dart file by using the `// ignore: name_of_lint` and
  # `// ignore_for_file: name_of_lint` syntax on the line or in the file
  # producing the lint.
  rules:
    # avoid_print: false  # Uncomment to disable the `avoid_print` rule
    # prefer_single_quotes: true  # Uncomment to enable the `prefer_single_quotes` rule

# Additional information about this file can be found at
# https://dart.dev/guides/language/analysis-options
```

## android/app/src/main/AndroidManifest.xml

**android/app/src/main/AndroidManifest.xml**

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.RECORD_AUDIO"/>
    <uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS"/>
    <application
        android:label="MU Study"
        android:name="${applicationName}"
        android:icon="@mipmap/ic_launcher">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:launchMode="singleTop"
            android:taskAffinity=""
            android:theme="@style/LaunchTheme"
            android:
configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layoutDi
rection|fontScale|screenLayout|density|uiMode"
            android:hardwareAccelerated="true"
            android:windowSoftInputMode="adjustResize">
            <!-- Specifies an Android theme to apply to this Activity as soon as
                 the Android process has started. This theme is visible to the user
                 while the Flutter UI initializes. After that, this theme continues
                 to determine the Window background behind the Flutter UI. -->
            <meta-data
              android:name="io.flutter.embedding.android.NormalTheme"
              android:resource="@style/NormalTheme"
              />
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <!-- Don't delete the meta-data below.
             This is used by the Flutter tool to generate GeneratedPluginRegistrant.java -->
        <meta-data
            android:name="flutterEmbedding"
            android:value="2" />
    </application>
    <!-- Required to query activities that can process text, see:
         https://developer.android.com/training/package-visibility and
         https://developer.android.com/reference/android/content/Intent#ACTION_PROCESS_TEXT.

         In particular, this is used by the Flutter engine in io.flutter.plugin.text.
ProcessTextPlugin. -->
    <queries>
        <intent>
            <action android:name="android.intent.action.PROCESS_TEXT"/>
            <data android:mimeType="text/plain"/>
        </intent>
    </queries>
</manifest>
```

## ios/Runner/Info.plist

**ios/Runner/Info.plist**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.
0.dtd">
<plist version="1.0">
<dict>
  <key>CADisableMinimumFrameDurationOnPhone</key>
  <true/>
  <key>CFBundleDevelopmentRegion</key>
  <string>$(DEVELOPMENT_LANGUAGE)</string>
  <key>CFBundleDisplayName</key>
  <string>MU Study</string>
  <key>CFBundleExecutable</key>
  <string>$(EXECUTABLE_NAME)</string>
  <key>CFBundleIdentifier</key>
  <string>$(PRODUCT_BUNDLE_IDENTIFIER)</string>
  <key>CFBundleInfoDictionaryVersion</key>
```

```
  <string>6.0</string>
  <key>CFBundleName</key>
  <string>MU Study</string>
  <key>CFBundlePackageType</key>
  <string>APPL</string>
  <key>CFBundleShortVersionString</key>
  <string>$(FLUTTER_BUILD_NAME)</string>
  <key>CFBundleSignature</key>
  <string>????</string>
  <key>CFBundleVersion</key>
  <string>$(FLUTTER_BUILD_NUMBER)</string>
  <key>LSRequiresIPhoneOS</key>
  <true/>
  <key>NSMicrophoneUsageDescription</key>
  <string>MU Study needs the microphone so you can record voice answers for homework.</string>
  <key>NSPhotoLibraryUsageDescription</key>
  <string>MU Study needs photo access so you can attach images to homework.</string>
  <key>UIApplicationSceneManifest</key>
  <dict>
    <key>UIApplicationSupportsMultipleScenes</key>
    <false/>
    <key>UISceneConfigurations</key>
    <dict>
      <key>UIWindowSceneSessionRoleApplication</key>
      <array>
        <dict>
          <key>UISceneClassName</key>
          <string>UIWindowScene</string>
          <key>UISceneConfigurationName</key>
          <string>flutter</string>
          <key>UISceneDelegateClassName</key>
          <string>$(PRODUCT_MODULE_NAME).SceneDelegate</string>
          <key>UISceneStoryboardFile</key>
          <string>Main</string>
        </dict>
      </array>
    </dict>
  </dict>
  <key>UIApplicationSupportsIndirectInputEvents</key>
  <true/>
  <key>UILaunchStoryboardName</key>
  <string>LaunchScreen</string>
  <key>UIMainStoryboardFile</key>
  <string>Main</string>
  <key>UISupportedInterfaceOrientations</key>
  <array>
    <string>UIInterfaceOrientationPortrait</string>
    <string>UIInterfaceOrientationLandscapeLeft</string>
    <string>UIInterfaceOrientationLandscapeRight</string>
  </array>
  <key>UISupportedInterfaceOrientations~ipad</key>
  <array>
    <string>UIInterfaceOrientationPortrait</string>
    <string>UIInterfaceOrientationPortraitUpsideDown</string>
    <string>UIInterfaceOrientationLandscapeLeft</string>
    <string>UIInterfaceOrientationLandscapeRight</string>
  </array>
</dict>
</plist>
```

## Repository tests

**test/emoji_message_test.dart**

```
import 'dart:io';

import 'package:flutter_test/flutter_test.dart';
import 'package:mu_study/core/chat/animated_emoji_registry.dart';
import 'package:mu_study/core/chat/emoji_message.dart';

void main() {
  group('singleAnimatedEmojiAsset', () {
    test('animates exact single supported emoji', () {
      expect(singleAnimatedEmojiAsset('❤■'), contains('Heart.json'));
      expect(singleAnimatedEmojiAsset('❤'), contains('Heart.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('LMAO-2 side tears.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('LMAO.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Thumb.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Fire.json'));
      // Party.json is party-face; party-popper lives in Confetti emoji.json.
      expect(singleAnimatedEmojiAsset('■'), contains('Confetti emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Party.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Cool emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Sleepy.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Crying emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Sad emoji to Crying emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('check-mark.json'));
      expect(singleAnimatedEmojiAsset('✔■'), contains('check-mark.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('smiley emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Mad emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Angry.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Star Strike Emoji.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Emoji Walking.json'));
      expect(singleAnimatedEmojiAsset('■'), contains('Middle finger.json'));
    });

    test('trims outer whitespace and normalizes VS16', () {
      expect(singleAnimatedEmojiAsset(' ■ '), contains('LMAO-2 side tears.json'));
      expect(singleAnimatedEmojiAsset('■\uFE0F'), contains('LMAO-2 side tears.json'));
      expect(singleAnimatedEmojiAsset('■\uFE0F'), contains('Thumb.json'));
    });

    test('rejects multiples, text mixes, spaces, unsupported', () {
      expect(singleAnimatedEmojiAsset('❤■❤■'), isNull);
      expect(singleAnimatedEmojiAsset('■❤■'), isNull);
      expect(singleAnimatedEmojiAsset('Hello ❤■'), isNull);
      expect(singleAnimatedEmojiAsset('❤■ hello'), isNull);
      expect(singleAnimatedEmojiAsset('■■■'), isNull);
      expect(singleAnimatedEmojiAsset('■ ■'), isNull);
      expect(singleAnimatedEmojiAsset('A'), isNull);
      expect(singleAnimatedEmojiAsset('1'), isNull);
      expect(singleAnimatedEmojiAsset('■'), isNull);
      expect(singleAnimatedEmojiAsset(''), isNull);
      expect(singleAnimatedEmojiAsset(null), isNull);
    });
  });

  group('lottie asset coverage', () {
    test('every asset file is referenced by the registry', () {
      final dir = Directory('assets/lottie');
      final onDisk = dir
          .listSync()
          .whereType<File>()
          .map((f) => f.uri.pathSegments.last)
          .where((n) => n.endsWith('.json'))
          .toSet();
      final mapped = registeredAnimatedEmojiFilenames
          .union(kAnimatedEmojiRedundantFilenames);
      final unused = onDisk.difference(mapped);
      expect(
        unused,
        isEmpty,
        reason: 'Unused Lottie files: ${unused.join(', ')}',
```

```
      );
    });

    test('every registry filename exists on disk', () {
      final dir = Directory('assets/lottie');
      final onDisk = dir
          .listSync()
          .whereType<File>()
          .map((f) => f.uri.pathSegments.last)
          .where((n) => n.endsWith('.json'))
          .toSet();
      final missing = registeredAnimatedEmojiFilenames.difference(onDisk);
      expect(
        missing,
        isEmpty,
        reason: 'Missing Lottie files: ${missing.join(', ')}',
      );
    });
  });
}
```

**test/shell_layout_test.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:go_router/go_router.dart';
import 'package:mu_study/components/layouts/app_shell.dart';
import 'package:mu_study/l10n/app_localizations.dart';
import 'package:mu_study/models/models.dart';
import 'package:mu_study/providers/notifications/notification_providers.dart';

const List<ShellDestination> _destinations = [
  ShellDestination(
    labelKey: 'navDashboard',
    icon: Icons.dashboard_outlined,
    path: '/dashboard',
  ),
  ShellDestination(
    labelKey: 'navMarks',
    icon: Icons.grade_outlined,
    path: '/marks',
  ),
  ShellDestination(
    labelKey: 'navProfile',
    icon: Icons.person_outline,
    path: '/profile',
  ),
];

/// Stands in for the screens inside the shell: its own [Scaffold] with a
/// floating action button, exactly as the teacher and admin screens are built.
class _ShellScreen extends StatelessWidget {
  const _ShellScreen();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Screen')),
      floatingActionButtonLocation: shellFabLocation(context),
      floatingActionButton: FloatingActionButton(
        onPressed: () {},
        child: const Icon(Icons.add),
      ),
      body: ListView(
        padding: EdgeInsets.only(
          bottom: shellContentBottomPadding(context, hasFab: true),
        ),
        children: [
          for (var i = 0; i < 30; i++)
            ListTile(key: ValueKey('row-$i'), title: Text('Row $i')),
        ],
```

```
      ),
    );
  }
}

Widget _app() {
  final GoRouter router = GoRouter(
    initialLocation: '/dashboard',
    routes: [
      ShellRoute(
        builder: (_, _, child) => AppShell(
          role: UserRole.student,
          destinations: _destinations,
          child: child,
        ),
        routes: [
          for (final ShellDestination d in _destinations)
            GoRoute(path: d.path, builder: (_, _) => const _ShellScreen()),
        ],
      ),
    ],
  );

  return ProviderScope(
    overrides: [unreadNotificationsCountProvider.overrideWithValue(0)],
    child: MaterialApp.router(
      locale: const Locale('en'),
      supportedLocales: AppLocalizations.supportedLocales,
      localizationsDelegates: const [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      routerConfig: router,
    ),
  );
}

/// Configures the test view as a device of [size] logical pixels with the
/// given safe area and keyboard insets.
void _useDevice(
  WidgetTester tester, {
  required Size size,
  double bottom = 0,
  double left = 0,
  double right = 0,
  double keyboard = 0,
}) {
  tester.view
    ..devicePixelRatio = 1
    ..physicalSize = size
    ..viewPadding = FakeViewPadding(bottom: bottom, left: left, right: right)
    ..padding = FakeViewPadding(
      bottom: keyboard > 0 ? 0 : bottom,
      left: left,
      right: right,
    )
    ..viewInsets = FakeViewPadding(bottom: keyboard);
  addTearDown(tester.view.reset);
}

Rect _navBar(WidgetTester tester) =>
    tester.getRect(find.byKey(ShellNavBar.pillKey));

Rect _fab(WidgetTester tester) =>
    tester.getRect(find.byType(FloatingActionButton));

void main() {
  group('mobile shell', () {
    testWidgets('FAB floats clear of the nav bar on a home indicator device', (
      tester,
    ) async {
```

```
  const Size size = Size(393, 852); // iPhone 16 Pro
  _useDevice(tester, size: size, bottom: 34);
  await tester.pumpWidget(_app());
  await tester.pumpAndSettle();

  final Rect navBar = _navBar(tester);
  final Rect fab = _fab(tester);

  expect(fab.bottom, moreOrLessEquals(navBar.top - kShellFabGap));
  expect(fab.right, lessThanOrEqualTo(size.width - 16));
  // The pill stops at the home indicator instead of sitting inside it.
  expect(navBar.bottom, moreOrLessEquals(size.height - 34));
  expect(navBar.left, moreOrLessEquals(kShellNavBarSideMargin));
});

testWidgets('reserved inset matches the space the nav bar really takes', (
  tester,
) async {
  const Size size = Size(393, 852);
  _useDevice(tester, size: size, bottom: 34);
  await tester.pumpWidget(_app());
  await tester.pumpAndSettle();

  final BuildContext context = tester.element(find.byType(_ShellScreen));
  expect(
    shellBottomInset(context),
    moreOrLessEquals(size.height - _navBar(tester).top),
  );
});

testWidgets('nav bar keeps a gap from the edge without a home indicator', (
  tester,
) async {
  const Size size = Size(360, 640); // small Android phone, button nav
  _useDevice(tester, size: size);
  await tester.pumpWidget(_app());
  await tester.pumpAndSettle();

  final Rect navBar = _navBar(tester);
  expect(navBar.bottom, moreOrLessEquals(size.height - 16));
  expect(_fab(tester).bottom, moreOrLessEquals(navBar.top - kShellFabGap));
});

testWidgets('nav bar grows with an Android gesture inset', (tester) async {
  const Size size = Size(412, 915);
  _useDevice(tester, size: size, bottom: 24);
  await tester.pumpWidget(_app());
  await tester.pumpAndSettle();

  final Rect navBar = _navBar(tester);
  expect(navBar.height, moreOrLessEquals(kShellNavBarHeight));
  expect(navBar.bottom, moreOrLessEquals(size.height - 24));
  expect(_fab(tester).bottom, moreOrLessEquals(navBar.top - kShellFabGap));
});

testWidgets('icons stay vertically centred in the pill', (tester) async {
  _useDevice(tester, size: const Size(393, 852), bottom: 34);
  await tester.pumpWidget(_app());
  await tester.pumpAndSettle();

  final Rect navBar = _navBar(tester);
  final Finder icons = find.descendant(
    of: find.byKey(ShellNavBar.pillKey),
    matching: find.byType(Icon),
  );
  expect(icons, findsNWidgets(_destinations.length));

  for (final Element icon in icons.evaluate()) {
    expect(
      tester.getRect(find.byWidget(icon.widget)).center.dy,
      moreOrLessEquals(navBar.center.dy, epsilon: 0.5),
    );
  }
```

```
    });

    testWidgets('scrolled content stops above the FAB', (tester) async {
      _useDevice(tester, size: const Size(393, 852), bottom: 34);
      await tester.pumpWidget(_app());
      await tester.pumpAndSettle();

      await tester.drag(find.byType(ListView), const Offset(0, -2000));
      await tester.pumpAndSettle();

      expect(
        tester.getRect(find.byKey(const ValueKey('row-29'))).bottom,
        lessThanOrEqualTo(_fab(tester).top),
      );
    });

    testWidgets('FAB rides above the keyboard and the bar stays put', (
      tester,
    ) async {
      const Size size = Size(393, 852);
      _useDevice(tester, size: size, bottom: 34);
      await tester.pumpWidget(_app());
      await tester.pumpAndSettle();

      final Rect restingBar = _navBar(tester);
      final Rect restingFab = _fab(tester);

      // A keyboard shorter than the space under the button leaves it alone.
      _useDevice(tester, size: size, bottom: 34, keyboard: 60);
      await tester.pumpAndSettle();
      expect(_fab(tester), restingFab);

      _useDevice(tester, size: size, bottom: 34, keyboard: 336);
      await tester.pumpAndSettle();
      expect(
        _fab(tester).bottom,
        moreOrLessEquals(size.height - 336 - kFloatingActionButtonMargin),
      );
      // The bar is covered by the keyboard, not pushed around by it.
      expect(_navBar(tester), restingBar);
    });
  });

  group('rail shell', () {
    testWidgets('tablets drop the nav bar but keep the FAB in the safe area', (
      tester,
    ) async {
      const Size size = Size(834, 1194); // iPad Air
      _useDevice(tester, size: size, bottom: 20);
      await tester.pumpWidget(_app());
      await tester.pumpAndSettle();

      expect(find.byKey(ShellNavBar.pillKey), findsNothing);
      expect(
        _fab(tester).bottom,
        moreOrLessEquals(size.height - 20 - kShellFabGap),
      );
    });

    testWidgets('FAB stays on screen in landscape', (tester) async {
      const Size size = Size(852, 393); // iPhone 16 Pro, landscape
      _useDevice(tester, size: size, bottom: 21, left: 47, right: 47);
      await tester.pumpWidget(_app());
      await tester.pumpAndSettle();

      final Rect fab = _fab(tester);
      expect(fab.bottom, lessThanOrEqualTo(size.height - 21));
      expect(fab.right, lessThanOrEqualTo(size.width - 47));
      expect(fab.top, greaterThan(0));
    });
  });
}
```

**test/widget_test.dart**

```
import 'package:flutter_test/flutter_test.dart';
import 'package:mu_study/screens/auth/setup_required_screen.dart';

void main() {
  testWidgets('Setup screen shows configuration hint', (tester) async {
    await tester.pumpWidget(const SetupRequiredScreen());
    expect(find.textContaining('Supabase is not configured'), findsOneWidget);
  });
}
```

# Asset inventory

Binary images and Lottie files cannot be reconstructed from text. Copy assets/ from the original tree for visual fidelity. Logic runs without them (Material fallbacks).

| Path | Kind |
|---|---|
| assets/images/logo/logo.jpeg | Image |
| assets/images/nature-paper/avatar-silhouette.png | Image |
| assets/images/nature-paper/beige-leaf.png | Image |
| assets/images/nature-paper/bookshelf-single-shelf.png | Image |
| assets/images/nature-paper/bookshelf-two-shelves.png | Image |
| assets/images/nature-paper/brain-verified.png | Image |
| assets/images/nature-paper/browser-dashboard.png | Image |
| assets/images/nature-paper/calendar-with-clock.png | Image |
| assets/images/nature-paper/calendar.png | Image |
| assets/images/nature-paper/cardboard-star.png | Image |
| assets/images/nature-paper/chat-bubbles.png | Image |
| assets/images/nature-paper/clipboard-checklist.png | Image |
| assets/images/nature-paper/clipboard-profile.png | Image |
| assets/images/nature-paper/clipboard-with-pencil.png | Image |
| assets/images/nature-paper/close-button.png | Image |
| assets/images/nature-paper/cream-bird.png | Image |
| assets/images/nature-paper/cream-paper-green-tape.png | Image |
| assets/images/nature-paper/division-button.png | Image |
| assets/images/nature-paper/document-layout.png | Image |
| assets/images/nature-paper/download-inbox.png | Image |
| assets/images/nature-paper/dusty-rose-flower.png | Image |
| assets/images/nature-paper/envelope-success.png | Image |
| assets/images/nature-paper/flat-cloud.png | Image |
| assets/images/nature-paper/gray-cloud.png | Image |
| assets/images/nature-paper/group-chat.png | Image |
| assets/images/nature-paper/hanging-garland.png | Image |
| assets/images/nature-paper/home-house.png | Image |
| assets/images/nature-paper/kraft-paper-strip.png | Image |
| assets/images/nature-paper/landscape-hills-footer.png | Image |
| assets/images/nature-paper/lined-paper-pencil.png | Image |
| assets/images/nature-paper/magnifying-glass.png | Image |
| assets/images/nature-paper/manila-folder.png | Image |
| assets/images/nature-paper/market-stall.png | Image |
| assets/images/nature-paper/minus-button.png | Image |
| assets/images/nature-paper/notification-bell-alert.png | Image |

| Path | Kind |
|---|---|
| assets/images/nature-paper/open-book.png | Image |
| assets/images/nature-paper/paper-funnel.png | Image |
| assets/images/nature-paper/puzzle-pieces.png | Image |
| assets/images/nature-paper/rounded-tan-star.png | Image |
| assets/images/nature-paper/sage-scalloped-frame.png | Image |
| **Path** | **Kind** |
| assets/images/nature-paper/salmon-flower.png | Image |
| assets/images/nature-paper/settings-gear.png | Image |
| assets/images/nature-paper/single-tree.png | Image |
| assets/images/nature-paper/spiral-notebook.png | Image |
| assets/images/nature-paper/sticky-notes-pair.png | Image |
| assets/images/nature-paper/striped-washi-tape.png | Image |
| assets/images/nature-paper/tan-note-green-tab.png | Image |
| assets/images/nature-paper/tan-seed.png | Image |
| assets/images/nature-paper/teacher-chalkboard.png | Image |
| assets/images/nature-paper/teal-bird.png | Image |
| assets/images/nature-paper/terracotta-scalloped-frame.png | Image |
| assets/images/nature-paper/textured-tan-star.png | Image |
| assets/images/nature-paper/torn-paper-scrap.png | Image |
| assets/images/nature-paper/two-trees.png | Image |
| assets/images/nature-paper/wall-clock.png | Image |
| assets/images/nature-paper/white-cloud.png | Image |
| assets/images/nature-paper/white-daisy.png | Image |
| assets/images/nature-paper/yellow-pencil.png | Image |
| assets/lottie/100% Emoji.json | Lottie |
| assets/lottie/Alien Emoji.json | Lottie |
| assets/lottie/Angry Face with Horns.json | Lottie |
| assets/lottie/Angry Face with Symbols on Mouth Emoji.json | Lottie |
| assets/lottie/Angry Face.json | Lottie |
| assets/lottie/Angry.json | Lottie |
| assets/lottie/Animated Clown Face.json | Lottie |
| assets/lottie/Anxious Face with Sweat.json | Lottie |
| assets/lottie/Astonished Face.json | Lottie |
| assets/lottie/Backhand Index Pointing Down.json | Lottie |
| assets/lottie/Backhand Index Pointing Right.json | Lottie |
| assets/lottie/Backhand Index Pointing Up.json | Lottie |
| assets/lottie/Bomb Emoji.json | Lottie |
| assets/lottie/Call Me Hand.json | Lottie |
| assets/lottie/Calm.json | Lottie |

| Path | Kind |
|---|---|
| assets/lottie/Clapping Hands.json | Lottie |
| assets/lottie/Cold Face.json | Lottie |
| assets/lottie/Confetti emoji.json | Lottie |
| assets/lottie/Confounded Face.json | Lottie |
| assets/lottie/Cool emoji.json | Lottie |
| assets/lottie/Cowboy Hat Face.json | Lottie |
| assets/lottie/Crossed Fingers.json | Lottie |
| **Path** | **Kind** |
| assets/lottie/Crying emoji.json | Lottie |
| assets/lottie/Dizzy Face.json | Lottie |
| assets/lottie/Drooling Face.json | Lottie |
| assets/lottie/Emoji 30 - Lovely Kiss.json | Lottie |
| assets/lottie/Emoji Walking.json | Lottie |
| assets/lottie/Eyes.json | Lottie |
| assets/lottie/Fire.json | Lottie |
| assets/lottie/FlyingMoney.json | Lottie |
| assets/lottie/Halo.json | Lottie |
| assets/lottie/Hand.json | Lottie |
| assets/lottie/Heart eyes.json | Lottie |
| assets/lottie/Heart.json | Lottie |
| assets/lottie/High Temperature.json | Lottie |
| assets/lottie/Holding back tears.json | Lottie |
| assets/lottie/Hourglass.json | Lottie |
| assets/lottie/Index finger.json | Lottie |
| assets/lottie/LMAO-2 side tears.json | Lottie |
| assets/lottie/LMAO.json | Lottie |
| assets/lottie/Laptop.json | Lottie |
| assets/lottie/Mad emoji.json | Lottie |
| assets/lottie/Medal Animations.json | Lottie |
| assets/lottie/Middle finger.json | Lottie |
| assets/lottie/Money.json | Lottie |
| assets/lottie/Muscle.json | Lottie |
| assets/lottie/Neutral face.json | Lottie |
| assets/lottie/Party.json | Lottie |
| assets/lottie/Pensive.json | Lottie |
| assets/lottie/Please.json | Lottie |
| assets/lottie/Puke Emoji.json | Lottie |
| assets/lottie/Rocket.json | Lottie |
| assets/lottie/Rolling Eyes Emoji.json | Lottie |

| Path | Kind |
|---|---|
| assets/lottie/Sad Emoji.json | Lottie |
| assets/lottie/Sad emoji to Crying emoji.json | Lottie |
| assets/lottie/Sick face with thermometer.json | Lottie |
| assets/lottie/Sleepy.json | Lottie |
| assets/lottie/Sparkles Emoji.json | Lottie |
| assets/lottie/Star Strike Emoji.json | Lottie |
| assets/lottie/Starstruck.json | Lottie |
| assets/lottie/Taxi.json | Lottie |
| assets/lottie/Thinking.json | Lottie |
| **Path** | **Kind** |
| assets/lottie/Thumb.json | Lottie |
| assets/lottie/Unicorn Emoji.json | Lottie |
| assets/lottie/Winking Emoji.json | Lottie |
| assets/lottie/X.json | Lottie |
| assets/lottie/Yawn Emoji.json | Lottie |
| assets/lottie/Yummy.json | Lottie |
| assets/lottie/Zany tongue.json | Lottie |
| assets/lottie/Zipper mouth Emoji.json | Lottie |
| assets/lottie/check-mark.json | Lottie |
| assets/lottie/emoji head explosion.json | Lottie |
| assets/lottie/injury head face emoji.json | Lottie |
| assets/lottie/smiley emoji.json | Lottie |

# Appendix K — Glossary

| Term | Meaning in MU Study |
|---|---|
| anon / publishable key | Public client key. Safe to ship in the app if RLS is correct. Still do not post it in a public gist with your project URL if you can avoid it. |
| service_role key | Bypasses RLS. Server only. Never in Flutter. |
| auth.uid() | UUID of the signed-in user inside Postgres. |
| auth_email | Synthetic email on profiles, usually username@mustudy.internal. |
| RLS | Row Level Security — per-row SQL predicates Postgres applies automatically. |
| SECURITY DEFINER | Function runs as owner; must self-authorize. |
| PostgREST | HTTP API generated from Postgres tables and RPCs. |
| RPC | Remote Procedure Call — supabase.rpc('name', params). |
| JWT | Signed token proving a session. |
| ProviderScope | Riverpod root widget in main.dart. |
| ConsumerWidget | Widget that can watch Riverpod providers. |
| AsyncValue | Riverpod loading / error / data triad. |
| ShellRoute | go_router nest that keeps a navigation chrome while child routes change. |
| homework_pack | Assignment type that contains tasks students submit. |
| static assignment | Assignment type that is informational / display-only. |
| sphere | Teacher subject: math or english. |
| lesson_session | One group + one date; attendance rows hang off it. |
| chat_room_summaries | View joining rooms, group title, last message, member_count. |
| ARB | Application Resource Bundle — Flutter translation file. |
| AppStyle | modern \| naturePaper \| softNeumorphism \| monochromeWorkspace. |
| signed URL | Time-limited Storage download link for a private object. |
| replica identity FULL | Needed so Realtime UPDATE/DELETE payloads include old rows (reactions). |
| is_disabled | Soft-ban flag; login RPC hides the email. |
| preferred_language | en, ru, or uz stored on profiles. |
| Focus | Local study timer; not a database feature. |

# Extra practice quizzes

## Self-check quiz (cover the answers)

**Q1.** Why does login call an RPC instead of concatenating username + '@mustudy.internal' in Dart?

> **Answer 1**
> The RPC is the single source of truth, hides disabled users, and can change the email scheme without shipping a new app.

**Q2.** Why is profiles.id equal to auth.users.id?

> **Answer 2**
> So auth.uid() in RLS matches the profile primary key with no extra join.

**Q3.** What unique constraint stops two groups named Algebra A for the same teacher?

> **Answer 3**
> UNIQUE (teacher_id, title) / groups_teacher_title_uidx.

**Q4.** Why does member_count in chat_room_summaries add 1?

> **Answer 4**
> It counts group_students plus the teacher.

**Q5.** Which bucket is public?

> **Answer 5**
> avatars. Submissions, assignments, announcements, and chat-* are private.

**Q6.** Where is focus session state stored?

> **Answer 6**
> SharedPreferences via focusSessionProvider — no SQL table.

**Q7.** Name three SECURITY DEFINER functions an admin uses.

> **Answer 7**
> admin_create_user, admin_reset_password, admin_create_next_academic_year (and others in the atlas).

**Q8.** What happens if a student queries marks for the whole table?

> **Answer 8**
> RLS returns only rows they are allowed to see — typically their own.

**Q9.** Why themeAnimationDuration is Duration.zero?

**Answer 9**
Instant theme/style switches without interpolating incompatible palettes.

**Q10.** How do chat uploads prove the uploader?

**Answer 10**
Object path folder 2 must equal auth.uid(); storage policies enforce it.

## Table drill — academic_terms

Primary key column to remember: **id**. Count of documented columns: 5.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — academic_terms independence**
In SQL Editor, run SELECT COUNT(*) FROM academic_terms; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — academic_years

Primary key column to remember: **id**. Count of documented columns: 6.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — academic_years independence**
In SQL Editor, run SELECT COUNT(*) FROM academic_years; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — announcements

Primary key column to remember: **id**. Count of documented columns: 8.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — announcements independence**
In SQL Editor, run SELECT COUNT(*) FROM announcements; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — assignment_templates

Primary key column to remember: **id**. Count of documented columns: 8.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — assignment_templates independence**
> In SQL Editor, run SELECT COUNT(*) FROM assignment_templates; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

### Table drill — assignments

Primary key column to remember: **id**. Count of documented columns: 8.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — assignments independence**
> In SQL Editor, run SELECT COUNT(*) FROM assignments; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

### Table drill — attendance

Primary key column to remember: **id**. Count of documented columns: 7.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — attendance independence**
> In SQL Editor, run SELECT COUNT(*) FROM attendance; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

### Table drill — chat_messages

Primary key column to remember: **id**. Count of documented columns: 10.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — chat_messages independence**
> In SQL Editor, run SELECT COUNT(*) FROM chat_messages; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

### Table drill — chat_rooms

Primary key column to remember: **id**. Count of documented columns: 3.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — chat_rooms independence**
> In SQL Editor, run SELECT COUNT(*) FROM chat_rooms; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — group_students

Primary key column to remember: **id**. Count of documented columns: 4.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — group_students independence**
> In SQL Editor, run SELECT COUNT(*) FROM group_students; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — groups

Primary key column to remember: **id**. Count of documented columns: 4.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — groups independence**
> In SQL Editor, run SELECT COUNT(*) FROM groups; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — lesson_sessions

Primary key column to remember: **id**. Count of documented columns: 4.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — lesson_sessions independence**
> In SQL Editor, run SELECT COUNT(*) FROM lesson_sessions; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — marks

Primary key column to remember: **id**. Count of documented columns: 13.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — marks independence**
> In SQL Editor, run SELECT COUNT(*) FROM marks; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — message_reactions

Primary key column to remember: **id**. Count of documented columns: 5.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — message_reactions independence**
In SQL Editor, run SELECT COUNT(*) FROM message_reactions; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — message_reads

Primary key column to remember: **id**. Count of documented columns: 4.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — message_reads independence**
In SQL Editor, run SELECT COUNT(*) FROM message_reads; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — notifications

Primary key column to remember: **id**. Count of documented columns: 9.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — notifications independence**
In SQL Editor, run SELECT COUNT(*) FROM notifications; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — profiles

Primary key column to remember: **id**. Count of documented columns: 11.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — profiles independence**
In SQL Editor, run SELECT COUNT(*) FROM profiles; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — students

Primary key column to remember: **id**. Count of documented columns: 3.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

**CHECKPOINT — students independence**
In SQL Editor, run SELECT COUNT(*) FROM students; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — submissions

Primary key column to remember: **id**. Count of documented columns: 11.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — submissions independence**
> In SQL Editor, run SELECT COUNT(*) FROM submissions; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — tasks

Primary key column to remember: **id**. Count of documented columns: 8.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — tasks independence**
> In SQL Editor, run SELECT COUNT(*) FROM tasks; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

## Table drill — teachers

Primary key column to remember: **id**. Count of documented columns: 4.

Write a SELECT that returns at most five rows from this table with the primary key and created_at if it exists. Then write the Flutter equivalent using .from().select().limit(5).

> **CHECKPOINT — teachers independence**
> In SQL Editor, run SELECT COUNT(*) FROM teachers; as a signed-in role you understand. If you are the anon role with no JWT, expect RLS to hide rows. That emptiness is success, not a broken table.

# Appendix L — Foundation essays

These essays are slow explanations of ideas the tutorials assume. Read them when a tutorial feels like it skipped a year of computer science. They are written against MU Study, not a generic blog.

## How a tap becomes a database row

A student taps Submit on a homework task. Flutter builds a gesture, the screen calls a Riverpod notifier, the notifier calls supabase.from('submissions').upsert(...). That HTTP request hits PostgREST on your Supabase project with the user's JWT. Postgres checks RLS. If the student owns the student_id and belongs to the group, the row is written. A trigger may insert notifications for the teacher. Realtime notifies subscribed clients. The teacher's submissions list rebuilds.

If any layer lies — a forged student_id, a missing JWT, a policy that uses user_metadata — the system is unsafe. MU Study keeps authorization in Postgres helpers (is_admin, is_teacher_of_group, is_student_in_group) so the Flutter client is not the security boundary.

Practice: pick one button in the running app and write this chain on paper: widget → provider → table/RPC → policy → trigger → UI refresh.

## HTTP, JSON, and PostgREST

Supabase’s Data API is PostgREST: your table names become URL paths. GET /rest/v1/assignments?group_id=eq. is what .from('assignments').select().eq('group_id', id) becomes. The Flutter SDK hides the HTTP, but you should still think in requests: method, headers (apikey, Authorization: Bearer ), and JSON body.

JSON objects use snake_case column names from Postgres. Dart models convert to camelCase (full_name → fullName). If you forget the mapping, fromJson throws or shows empty names. That is a model bug, not a database bug.

Errors arrive as PostgrestException with a message and often a Postgres code (42501 insufficient privilege for RLS denials that surface as empty results, 23505 unique violation, 23503 foreign key).

## JWT sessions without mysticism

After signInWithPassword, Supabase returns an access token (JWT) and a refresh token. The access token is a signed JSON object: subject (user id), expiry, role authenticated. Every .from() call sends that token. RLS reads auth.uid() from it.

MU Study does not invent emails in the client. It asks get_auth_email_by_username for the synthetic address stored on profiles.auth_email. Disabled users get NULL, so login fails the same way as a wrong password — no account-enumeration toast.

Deleting a user in the dashboard does not instantly kill an already-issued JWT. For school software, keep sessions reasonably short and disable profiles (is_disabled) so the RPC and policies stop the account even if a token still exists.

## Postgres types used in MU Study

uuid — primary keys, generated with gen_random_uuid(). Never use serial integers for public IDs in this project; the live schema is UUID throughout.

text — usernames, titles, bodies. Prefer text over varchar unless you have a length rule.

timestamptz — created_at, due_date, submitted_at. Always timezone-aware.

date — attendance.date, lesson_sessions.session_date, marks.marked_on.

boolean — is_disabled, is_read, is_deleted, is_current, is_completed.

numeric — marks.percentage (0–100), marks.weight (>0), submissions.score.

jsonb — announcement image_urls/file_urls, assignment_templates.tasks.

enums — user_role, sphere_type, assignment_type, attendance_status, submission_status, chat_message_type, mark_category. Changing an enum in production requires ALTER TYPE; plan names carefully.

## Git hygiene for a school app

.env is gitignored. If you accidentally commit a key, rotate it in Supabase (Settings → API) and treat the old key as public.

Do not commit service_role. Do not commit printed password PDFs from Admin → Credentials.

Demo passwords such as password123 are for throwaway local projects only. The README says every account including @admin must get a real password before anyone outside the team can reach the instance.

Prefer small commits that match a tutorial checkpoint: “add login RPC wiring”, not “stuff”.

## Widget trees and rebuilds

Flutter rebuilds widgets when an ancestor setState or a watched provider changes. A ConsumerWidget that watches currentProfileProvider rebuilds when the profile Future completes. A StatelessWidget that only receives a Profile as a constructor argument rebuilds only when its parent rebuilds.

MU Study uses themeAnimationDuration: Duration.zero so style switches do not animate through a muddy blend. That is a product choice, not a Flutter default.

Heavy lists (chat, gradebook) should not rebuild the entire shell. Providers are scoped so a new message does not re-fetch academic years.

## Security principles encoded in this repo

Authorization data lives in profiles.role and teachers/students tables, not in editable user_metadata. RLS and RPCs read those tables (or auth.uid()).

Views that join sensitive rows must be treated carefully. chat_room_summaries is only safe if the underlying tables are RLS-protected and the view is not a security-definer leak. Prefer security_invoker views on modern Postgres.

Storage paths encode roomId and profileId. Policies parse the object name. A client that uploads to someone else's folder must 403.

seed_demo_data and _demo_insert_user are development scaffolding. EXECUTE is revoked from anon and authenticated on a hardened project.

## How teachers and groups actually work

Creating a teacher does not create Group 1/2/3. Admins type exact group titles under Admin → Groups. A unique index (teacher_id, title) prevents duplicate names for the same teacher.

Inserting a group fires ensure_chat_room_for_group so a 1:1 chat_rooms row exists. Students appear in chat only after group_students membership.

Teachers have a required sphere: math or english. The admin_create_user RPC raises 'Sphere required for teacher' if it is missing.

## Homework packs versus static assignments

assignment_type is static or homework_pack. Static assignments are display-only in the student app (announcement-like work). Homework packs contain ordered tasks. Students open a pack, then a task, then submit text, file, and/or voice.

One submission per (task_id, student_id). Teachers review in the submissions queue and may write score + feedback. A trigger can sync a mark row.

Teachers can save assignment_templates as JSON drafts and reuse them.

## Focus mode is local

Student Focus is a Pomodoro or free-count timer stored in SharedPreferences. There is no focus_sessions table. Chat badges can be muted during a session while notifications still arrive in the inbox table.

This is a good example of not putting every UX idea in Postgres. If the phone is offline, focus still works.